

iTAC.MES.Suite

IMSApi

IMSApi Functions in Release 9.10

Copyright

The dissemination and reproduction of this publication or parts or excerpts thereof, for whatever reason and in whichever form, are not permitted without the explicit written approval by iTAC Software AG. Photographs, diagrams and texts are subject to copyright and may only be used for other purposes with the approval of iTAC Software AG.

The iTAC products and services mentioned in the text and the respective logos and brand names are protected or registered brands of iTAC Software AG in Germany and other countries worldwide. The software products offered by iTAC Software AG may include software components of other software manufacturers.

All other logos, trademarks, brand names and names of products and services are registered and protected brands and trademarks of the relevant companies and their further use is subject to approval by these companies. We draw to your attention that it is not permitted to change or modify trademarks or brand names and that this may be legally prosecuted by the owner.

We accept no liability for references to third party suppliers, Internet links, and other source references, in any shape or form. The information and text included in this publication are intended only for informational purposes. Products may exhibit country-specific differences.

All statements are subject to change and changes do not require notification.

© **Copyright 01/2019 iTAC Software AG**, Montabaur

Table of Contents

1	General information on the IMSApi	37
1.1	Introduction	37
1.2	IMSApi service	40
1.3	Overview of IMSApi connection	45
1.3.1	Communication model	45
1.3.2	IMSApi Java Library	46
1.3.3	IMSApi Shared Lib.	47
1.3.4	IMSApi DotNet	48
1.3.5	IMSApi COM	49
1.3.6	IMSApi socket adapter	50
1.3.7	IMSApi PLC Connector	51
1.3.8	IMSApi iTAC.OPC-UA.Service	53
1.3.9	IMSApi REST	54
1.4	Session handling	55
2	IMSApi functions	57
3	Group "ADVICE" (advice notices)	59
3.1	adviceConfirmAdvice	59
3.1.1	Function description	59
3.1.2	Input and output parameters	59
3.1.3	Output values	60
3.2	adviceGetAdvice	61
3.2.1	Function description	61
3.2.2	Input and output parameters	62
3.2.3	Output values	64
4	Group "APS" (work order planning)	65
4.1	apsGetEquipments	65
4.1.1	Function description	65
4.1.2	Input and output parameters	70
4.1.3	Output values	71
4.2	apsGetStations	72
4.2.1	Function description	72
4.2.2	Input and output parameters	75
4.2.3	Output values	76
4.3	apsGetWorkergroups	77

4.3.1	Function description	77
4.3.2	Input and output parameters	80
4.3.3	Output values	81
4.4	apsGetWorkers	82
4.4.1	Function description	82
4.4.2	Input and output parameters	85
4.4.3	Output values	86
4.5	apsGetWorkorderDetails	87
4.5.1	Function description	87
4.5.2	Input and output parameters	98
4.5.3	Output values	99
4.6	apsGetWorkorderDetailsOptGroup	100
4.6.1	Function description	100
4.6.2	Input and output parameters	111
4.6.3	Output values	113
4.7	apsGetWorkorders	114
4.7.1	Function description	114
4.7.2	Input and output parameters	120
4.7.3	Output values	121
5	Group "ATTRIB" (attributes)	123
5.1	attribAppendAttributeValues	123
5.1.1	Function description	123
5.1.2	Input and output parameters	149
5.1.3	Output values	151
5.2	attribCheckAttributeValues	154
5.2.1	Function description	154
5.2.2	Input and output parameters	180
5.2.3	Output values	182
5.3	attribCreateAttribute	184
5.3.1	Function description	184
5.3.2	Input and output parameters	185
5.3.3	Output values	186
5.4	attribCreateAttributeClass	187
5.4.1	Function description	187
5.4.2	Input and output parameters	190
5.4.3	Output values	191
5.5	attribCreateAttributeCode	192
5.5.1	Function description	192
5.5.2	Input and output parameters	195
5.5.3	Output values	196
5.6	attribCreateAttributeCodeByCatalog	197

5.6.1	Function description	197
5.6.2	Input and output parameters	199
5.6.3	Output values	200
5.7	attribCreateAttributeCondition	201
5.7.1	Function description	201
5.7.2	Input and output parameters	203
5.7.3	Output values	204
5.8	attribEditAttributeClassState	205
5.8.1	Function description	205
5.8.2	Input and output parameters	206
5.8.3	Output values	207
5.9	attribGetAllAttributeValue	208
5.9.1	Function description	208
5.9.2	Input and output parameters	216
5.9.3	Output values	217
5.10	attribGetAttributeClass	218
5.10.1	Function description	218
5.10.2	Input and output parameters	225
5.10.3	Output values	227
5.11	attribGetAttributeCode	228
5.11.1	Function description	228
5.11.2	Input and output parameters	232
5.11.3	Output values	233
5.12	attribGetAttributeCondition	234
5.12.1	Function description	234
5.12.2	Input and output parameters	237
5.12.3	Output values	238
5.13	attribGetAttributeHistory	239
5.13.1	Function description	239
5.13.2	Input and output parameters	255
5.13.3	Output values	257
5.14	attribGetAttributeValues	259
5.14.1	Function description	259
5.14.2	Input and output parameters	285
5.14.3	Output values	287
5.15	attribGetAttributeValuesForObjectType	289
5.15.1	Function description	289
5.15.2	Input and output parameters	295
5.15.3	Output values	296
5.16	attribGetObjectsForAttributeValues	298
5.16.1	Function description	298

5.16.2	Input and output parameters	335
5.16.3	Output values	337
5.17	attribManageAttributeClassAttribute	338
5.17.1	Function description	338
5.17.2	Input and output parameters	341
5.17.3	Output values	342
5.18	attribManageAttributeClassCondition	343
5.18.1	Function description	343
5.18.2	Input and output parameters	347
5.18.3	Output values	348
5.19	attribManageAttributeCondition	349
5.19.1	Function description	349
5.19.2	Input and output parameters	353
5.19.3	Output values	354
5.20	attribRemoveAttributeClassValue	355
5.20.1	Function description	355
5.20.2	Input and output parameters	359
5.20.3	Output values	360
5.21	attribRemoveAttributeValue	362
5.21.1	Function description	362
5.21.2	Input and output parameters	364
5.21.3	Output values	366
5.22	attribUpdateAttributeCode	368
5.22.1	Function description	368
5.22.2	Input and output parameters	371
5.22.3	Output values	372
6	Group "BATCH" (bin).....	373
6.1	batchAssignBatchNumberToWorkOrder	373
6.1.1	Function description	373
6.1.2	Input and output parameters	374
6.1.3	Output values	376
6.2	batchGetBatchInfo	378
6.2.1	Function description	378
6.2.2	Input and output parameters	381
6.2.3	Output values	382
6.3	batchGetRegisteredBatch	383
6.3.1	Function description	383
6.3.2	Input and output parameters	384
6.3.3	Output values	385
6.4	batchMergeBatch	386
6.4.1	Function description	386

6.4.2	Input and output parameters	387
6.4.3	Output values	388
6.5	batchRegisterBatch	389
6.5.1	Function description	389
6.5.2	Input and output parameters	390
6.5.3	Output values	391
6.6	batchSplitBatchNumberToSerialNumber	392
6.6.1	Function description	392
6.6.2	Input and output parameters	392
6.6.3	Output values	394
6.7	batchUnregBatch	395
6.7.1	Function description	395
6.7.2	Input and output parameters	397
6.7.3	Output values	398
6.8	batchUnregisterBatch	399
6.8.1	Function description	399
6.8.2	Input and output parameters	400
6.8.3	Output values	401
7	Group "CONFIG" (configuration)	403
7.1	configCreateParameters	403
7.1.1	Function description	403
7.1.2	Input and output parameters	410
7.1.3	Output values	411
7.2	configDeleteParameters	412
7.2.1	Function description	412
7.2.2	Input and output parameters	416
7.2.3	Output values	417
7.3	configDeleteValues	418
7.3.1	Function description	418
7.3.2	Input and output parameters	421
7.3.3	Output values	422
7.4	configGetDimensionValues	423
7.4.1	Function description	423
7.4.2	Input and output parameters	424
7.4.3	Output values	425
7.5	configGetParameters	426
7.5.1	Function description	426
7.5.2	Input and output parameters	432
7.5.3	Output values	433
7.6	configGetValues	434
7.6.1	Function description	434

7.6.2	Input and output parameters	440
7.6.3	Output values	442
7.7	configUpdateValues.....	443
7.7.1	Function description	443
7.7.2	Input and output parameters	446
7.7.3	Output values	447
7.8	configUpdateParameters	448
7.8.1	Function description	448
7.8.2	Input and output parameters	455
7.8.3	Output values	456
8	Group "CRP" (continuous replenishment).....	457
8.1	crpChangeTransportOrder.....	457
8.1.1	Function description	457
8.1.2	Input and output parameters	463
8.1.3	Output values	465
8.2	crpCreateTransportOrder.....	467
8.2.1	Function description	467
8.2.2	Input and output parameters	471
8.2.3	Output values	473
8.3	crpGetRoute	475
8.3.1	Function description	475
8.3.2	Input and output parameters	479
8.3.3	Output values	481
8.4	crpGetTransportOrder.....	482
8.4.1	Function description	482
8.4.2	Input and output parameters	488
8.4.3	Output values	490
8.5	crpManageRoute	491
8.5.1	Function description	491
8.5.2	Input and output parameters	494
8.5.3	Output values	496
9	Group "CUSTOM" (customer function)	499
9.1	customFunction	499
9.1.1	Function description	499
9.1.2	Input and output parameters	500
9.1.3	Output values	501
9.2	customGetDbObjectData.....	502
9.2.1	Function description	502
9.2.2	Input and output parameters	505
9.2.3	Output values	506
9.3	customGetDbObjectMetaData.....	508

9.3.1	Function description	508
9.3.2	Input and output parameters	510
9.3.3	Output values	511
9.4	customGetFunctionInfo	512
9.4.1	Function description	512
9.4.2	Input and output parameters	517
9.4.3	Output values	519
9.5	customGetFunctions	520
9.5.1	Function description	520
9.5.2	Input and output parameters	522
9.5.3	Output values	523
9.6	customModifyDbObjectData	524
9.6.1	Function description	524
9.6.2	Input and output parameters	529
9.6.3	Output values	531
10	Group "EDA" (external data)	533
10.1	edaAssignExternalData	533
10.1.1	Function description	533
10.1.2	Input and output parameters	534
10.1.3	Output values	535
10.2	edaGetExternalDataForObject	537
10.2.1	Function description	537
10.2.2	Input and output parameters	540
10.2.3	Output values	541
10.3	edaGetNextFreeExternalData	542
10.3.1	Function description	542
10.3.2	Input and output parameters	544
10.3.3	Output values	545
10.4	edaGetObjectsForExternalData	547
10.4.1	Function description	547
10.4.2	Input and output parameters	549
10.4.3	Output values	550
10.5	edaManageExternalData	551
10.5.1	Function description	551
10.5.2	Input and output parameters	553
10.5.3	Output values	554
11	Group "EQU" (production means)	557
11.1	equAssignCarrierToCarrier	557
11.1.1	Function description	557
11.1.2	Input and output parameters	559
11.1.3	Output values	560

11.2	equAssignCavityToEquipment.....	561
11.2.1	Function description	561
11.2.2	Input and output parameters	564
11.2.3	Output values	565
11.3	equAssignSerialNumberToCarrier	567
11.3.1	Function description	567
11.3.2	Input and output parameters	569
11.3.3	Output values	570
11.4	equCheckEquipmentData	571
11.4.1	Function description	571
11.4.2	Input and output parameters	576
11.4.3	Output values	577
11.5	equCreateEquipment.....	578
11.5.1	Function description	578
11.5.2	Input and output parameters	582
11.5.3	Output values	583
11.6	equGetAssetEquipmentSetup	584
11.6.1	Function description	584
11.6.2	Input and output parameters	587
11.6.3	Output values	588
11.7	equGetAssignedCavity	590
11.7.1	Function description	590
11.7.2	Input and output parameters	594
11.7.3	Output values	595
11.8	equGetCarrierAssigned	596
11.8.1	Function description	596
11.8.2	Input and output parameters	600
11.8.3	Output values	601
11.9	equGetCarrierAssignmentHistoryData	603
11.9.1	Function description	603
11.9.2	Input and output parameters	607
11.9.3	Output values	608
11.10	equGetCarrierAssignRule.....	610
11.10.1	Function description	610
11.10.2	Input and output parameters	614
11.10.3	Output values	616
11.11	equGetCarrierLocation	618
11.11.1	Function description	618
11.11.2	Input and output parameters	620
11.11.3	Output values	621
11.12	equGetCarrierToCarrier	623

11.12.1 Function description	623
11.12.2 Input and output parameters	625
11.12.3 Output values	626
11.13 equGetCarrierToSerialNumber	628
11.13.1 Function description	628
11.13.2 Input and output parameters	630
11.13.3 Output values	631
11.14 equGetCavityEquipmentAssignRule	632
11.14.1 Function description	632
11.14.2 Input and output parameters	635
11.14.3 Output values	636
11.15 equGetEquipment	637
11.15.1 Function description	637
11.15.2 Input and output parameters	643
11.15.3 Output values	644
11.16 equGetEquipmentLocationHist	645
11.16.1 Function description	645
11.16.2 Input and output parameters	648
11.16.3 Output values	649
11.17 equGetPartCavityAssignRule	650
11.17.1 Function description	650
11.17.2 Input and output parameters	653
11.17.3 Output values	654
11.18 equGetRequiredEquipmentData	655
11.18.1 Function description	655
11.18.2 Input and output parameters	658
11.18.3 Output values	660
11.19 equGetRequiredEquipmentDetail	661
11.19.1 Function description	661
11.19.2 Input and output parameters	666
11.19.3 Output values	667
11.20 equGetSetupEquipmentData	668
11.20.1 Function description	668
11.20.2 Input and output parameters	671
11.20.3 Output values	672
11.21 equManageCavityEquipmentRule	673
11.21.1 Function description	673
11.21.2 Input and output parameters	675
11.21.3 Output values	676
11.22 equManagePartCavityRule	677
11.22.1 Function description	677

11.22.2 Input and output parameters	679
11.22.3 Output values	680
11.23 equRemoveCarrierFromCarrier	682
11.23.1 Function description	682
11.23.2 Input and output parameters	684
11.23.3 Output values	685
11.24 equRemoveCavityFromEquipment	686
11.24.1 Function description	686
11.24.2 Input and output parameters	688
11.24.3 Output values	689
11.25 equRemoveEquipment	690
11.25.1 Function description	690
11.25.2 Input and output parameters	690
11.25.3 Output values	691
11.26 equRemoveSerialNumberFromCarrier	692
11.26.1 Function description	692
11.26.2 Input and output parameters	694
11.26.3 Output values	695
11.27 equReportQuantity	696
11.27.1 Function description	696
11.27.2 Input and output parameters	700
11.27.3 Output values	701
11.28 equSetEquipmentLocation	703
11.28.1 Function description	703
11.28.2 Input and output parameters	706
11.28.3 Output values	707
11.29 equSetLocation	708
11.29.1 Function description	708
11.29.2 Input and output parameters	709
11.29.3 Output values	710
11.30 equSetupAssetEquipment	711
11.30.1 Function description	711
11.30.2 Input and output parameters	715
11.30.3 Output values	716
11.31 equUpdateAssignedCavity	718
11.31.1 Function description	718
11.31.2 Input and output parameters	721
11.31.3 Output values	722
11.32 equUpdateEquipment	724
11.32.1 Function description	724
11.32.2 Input and output parameters	727

11.32.3 Output values	728
11.33 equUpdateEquipmentData	729
11.33.1 Function description	729
11.33.2 Input and output parameters	731
11.33.3 Output values	732
12 Group "EXEC" (ruleset).....	735
12.1 execFormula	735
12.1.1 Function description	735
12.1.2 Input and output parameters	738
12.1.3 Output values	740
12.2 execFormulaCatalog.....	741
12.2.1 Function description	741
12.2.2 Input and output parameters	743
12.2.3 Output values	744
12.3 execRuleSet	745
12.3.1 Function description	745
12.3.2 Input and output parameters	745
12.3.3 Output values	747
12.4 execTimerProcess	748
12.4.1 Function description	748
12.4.2 Input and output parameters	750
12.4.3 Output values	751
13 Group "IMSAPI" (metadata)	753
13.1 imsapiGetErrorList	753
13.1.1 Function description	753
13.1.2 Input and output parameters	754
13.1.3 Output values	756
13.2 imsapiGetErrorText	757
13.2.1 Function description	757
13.2.2 Input and output parameters	757
13.2.3 Output values	758
13.3 imsapiGetFunctionInfo.....	759
13.3.1 Function description	759
13.3.2 Input and output parameters	759
13.3.3 Output values	760
13.4 imsapiGetFunctions.....	761
13.4.1 Function description	761
13.4.2 Input and output parameters	761
13.4.3 Output values	762
13.5 imsapiGetGroups.....	763
13.5.1 Function description	763

13.5.2	Input and output parameters	763
13.5.3	Output values	764
13.6	imsapiGetParameterConditions	765
13.6.1	Function description	765
13.6.2	Input and output parameters	765
13.6.3	Output values	766
13.7	imsapiGetParameterInfo	767
13.7.1	Function description	767
13.7.2	Input and output parameters	767
13.7.3	Output values	769
13.8	imsapiGetSystemInfos	770
13.8.1	Function description	770
13.8.2	Input and output parameters	771
13.8.3	Output values	772
14	Group "LOCK" (lock)	773
14.1	lockGetLockedObjects	773
14.1.1	Function description	773
14.1.2	Input and output parameters	777
14.1.3	Output values	779
14.2	lockGetLockGroups	780
14.2.1	Function description	780
14.2.2	Input and output parameters	783
14.2.3	Output values	785
14.3	lockObjects	786
14.3.1	Function description	786
14.3.2	Input and output parameters	789
14.3.3	Output values	790
14.4	lockUnlockObjects	792
14.4.1	Function description	792
14.4.2	Input and output parameters	795
14.4.3	Output values	797
15	Group "MDA" (document management)	799
15.1	mdaActivateDocument	799
15.1.1	Function description	799
15.1.2	Input and output parameters	801
15.1.3	Output values	802
15.2	mdaActivateRecipe	803
15.2.1	Function description	803
15.2.2	Input and output parameters	803
15.2.3	Output values	804
15.3	mdaCreateRecipe	805

15.3.1	Function description	805
15.3.2	Input and output parameters	812
15.3.3	Output values	814
15.4	mdaGetDocumentContent	816
15.4.1	Function description	816
15.4.2	Input and output parameters	816
15.4.3	Output values	817
15.5	mdaGetDocuments	818
15.5.1	Function description	818
15.5.2	Input and output parameters	824
15.5.3	Output values	825
15.6	mdaGetRecipeData	827
15.6.1	Function description	827
15.6.2	Input and output parameters	833
15.6.3	Output values	835
15.7	mdaGetRecipeHeaderAndVersion	836
15.7.1	Function description	836
15.7.2	Input and output parameters	846
15.7.3	Output values	847
15.8	mdaUploadCustomFunction	848
15.8.1	Function description	848
15.8.2	Input and output parameters	850
15.8.3	Output values	851
15.9	mdaUploadDocument	852
15.9.1	Function description	852
15.9.2	Input and output parameters	861
15.9.3	Output values	863
16	Group "MDATA" (master data)	865
16.1	mdataBomVerify	865
16.1.1	Function description	865
16.1.2	Input and output parameters	869
16.1.3	Output values	871
16.2	mdataChangeWorkorderBomData	872
16.2.1	Function description	872
16.2.2	Input and output parameters	876
16.2.3	Output values	877
16.3	mdataChangeWorkorderData	879
16.3.1	Function description	879
16.3.2	Input and output parameters	883
16.3.3	Output values	884
16.4	mdataChangeWorkOrderWorkStepState	885

16.4.1	Function description	885
16.4.2	Input and output parameters	889
16.4.3	Output values	890
16.5	mdataCreateDetectionPoint.....	891
16.5.1	Function description	891
16.5.2	Input and output parameters	894
16.5.3	Output values	895
16.6	mdataCreateOrderCluster	897
16.6.1	Function description	897
16.6.2	Input and output parameters	900
16.6.3	Output values	901
16.7	mdataCreateSpecialTime	903
16.7.1	Function description	903
16.7.2	Input and output parameters	904
16.7.3	Output values	905
16.8	mdataCreateWorkorder	906
16.8.1	Function description	906
16.8.2	Input and output parameters	912
16.8.3	Output values	913
16.9	mdataDeleteDetectionPoint.....	915
16.9.1	Function description	915
16.9.2	Input and output parameters	915
16.9.3	Output values	916
16.10	mdataDeleteOrderCluster	917
16.10.1	Function description	917
16.10.2	Input and output parameters	918
16.10.3	Output values	919
16.11	mdataGetAssignedShiftModel	920
16.11.1	Function description	920
16.11.2	Input and output parameters	923
16.11.3	Output values	924
16.12	mdataGetAssignedSpecialTime.....	926
16.12.1	Function description	926
16.12.2	Input and output parameters	930
16.12.3	Output values	931
16.13	mdataGetBomData	933
16.13.1	Function description	933
16.13.2	Input and output parameters	938
16.13.3	Output values	939
16.14	mdataGetCalendarData	941
16.14.1	Function description	941

16.14.2	Input and output parameters	943
16.14.3	Output values	944
16.15	mdataGetContactData	945
16.15.1	Function description	945
16.15.2	Input and output parameters	948
16.15.3	Output values	949
16.16	mdataGetCostCenter	950
16.16.1	Function description	950
16.16.2	Input and output parameters	953
16.16.3	Output values	954
16.17	mdataGetDetectionPoint	955
16.17.1	Function description	955
16.17.2	Input and output parameters	961
16.17.3	Output values	962
16.18	mdataGetFailureDataForPart	963
16.18.1	Function description	963
16.18.2	Input and output parameters	966
16.18.3	Output values	967
16.19	mdataGetFailureDataForStation	968
16.19.1	Function description	968
16.19.2	Input and output parameters	972
16.19.3	Output values	973
16.20	mdataGetFailureDefinition	974
16.20.1	Function description	974
16.20.2	Input and output parameters	979
16.20.3	Output values	980
16.21	mdataGetFormulaCatalog	981
16.21.1	Function description	981
16.21.2	Input and output parameters	983
16.21.3	Output values	984
16.22	mdataGetMachineAssetStructure	985
16.22.1	Function description	985
16.22.2	Input and output parameters	992
16.22.3	Output values	993
16.23	mdataGetMessageCause	995
16.23.1	Function description	995
16.23.2	Input and output parameters	997
16.23.3	Output values	998
16.24	mdataGetMessageClass	999
16.24.1	Function description	999
16.24.2	Input and output parameters	1001

16.24.3	Output values	1002
16.25	mdataGetMessageEffect	1003
16.25.1	Function description	1003
16.25.2	Input and output parameters	1005
16.25.3	Output values	1006
16.26	mdataGetOrderCluster	1007
16.26.1	Function description	1007
16.26.2	Input and output parameters	1011
16.26.3	Output values	1012
16.27	mdataGetPartData	1013
16.27.1	Function description	1013
16.27.2	Input and output parameters	1017
16.27.3	Output values	1018
16.28	mdataGetProductInfo	1019
16.28.1	Function description	1019
16.28.2	Input and output parameters	1021
16.28.3	Output values	1023
16.29	mdataGetProductsForLineOrStation	1024
16.29.1	Function description	1024
16.29.2	Input and output parameters	1028
16.29.3	Output values	1029
16.30	mdataGetShiftData	1031
16.30.1	Function description	1031
16.30.2	Input and output parameters	1035
16.30.3	Output values	1036
16.31	mdataGetShiftModel	1038
16.31.1	Function description	1038
16.31.2	Input and output parameters	1041
16.31.3	Output values	1042
16.32	mdataGetStationConfiguration	1043
16.32.1	Function description	1043
16.32.2	Input and output parameters	1047
16.32.3	Output values	1048
16.33	mdataGetTimeEvent	1049
16.33.1	Function description	1049
16.33.2	Input and output parameters	1051
16.33.3	Output values	1052
16.34	mdataGetUserData	1053
16.34.1	Function description	1053
16.34.2	Input and output parameters	1056
16.34.3	Output values	1057

16.35mdataGetUserGroupData	1058
16.35.1 Function description	1058
16.35.2 Input and output parameters	1061
16.35.3 Output values	1062
16.36mdataGetUserSkill	1063
16.36.1 Function description	1063
16.36.2 Input and output parameters	1066
16.36.3 Output values	1067
16.37mdataGetWageType	1068
16.37.1 Function description	1068
16.37.2 Input and output parameters	1072
16.37.3 Output values	1073
16.38mdataGetWeekSchedule	1074
16.38.1 Function description	1074
16.38.2 Input and output parameters	1076
16.38.3 Output values	1077
16.39mdataGetWorkorders	1078
16.39.1 Function description	1078
16.39.2 Input and output parameters	1084
16.39.3 Output values	1085
16.40mdataGetWorkplanActivities	1086
16.40.1 Function description	1086
16.40.2 Input and output parameters	1095
16.40.3 Output values	1097
16.41mdataGetWorkplanData	1098
16.41.1 Function description	1098
16.41.2 Input and output parameters	1105
16.41.3 Output values	1106
16.42mdataGetWorkstepSkill	1108
16.42.1 Function description	1108
16.42.2 Input and output parameters	1111
16.42.3 Output values	1112
16.43mdataRequestWorkorder	1114
16.43.1 Function description	1114
16.43.2 Input and output parameters	1116
16.43.3 Output values	1117
16.44mdataSetProductionCycleTime	1118
16.44.1 Function description	1118
16.44.2 Input and output parameters	1118
16.44.3 Output values	1120
16.45mdataUpdateDetectionPoint	1121

16.45.1 Function description	1121
16.45.2 Input and output parameters	1124
16.45.3 Output values	1125
16.46 mdataUpdateOrderCluster	1127
16.46.1 Function description	1127
16.46.2 Input and output parameters	1131
16.46.3 Output values	1133
17 Group "MDC" (machine data collection)	1135
17.1 mdcAcknowledgeStationConditions	1135
17.1.1 Function description	1135
17.1.2 Input and output parameters	1140
17.1.3 Output values	1141
17.2 mdcCloseStationMessage	1143
17.2.1 Function description	1143
17.2.2 Input and output parameters	1145
17.2.3 Output values	1146
17.3 mdcCreateConditionCode	1147
17.3.1 Function description	1147
17.3.2 Input and output parameters	1147
17.3.3 Output values	1149
17.4 mdcCreateLog	1150
17.4.1 Function description	1150
17.4.2 Input and output parameters	1150
17.4.3 Output values	1151
17.5 mdcCreateLogData	1152
17.5.1 Function description	1152
17.5.2 Input and output parameters	1154
17.5.3 Output values	1155
17.6 mdcGetConditionCodes	1157
17.6.1 Function description	1157
17.6.2 Input and output parameters	1160
17.6.3 Output values	1161
17.7 mdcGetKeyPerformanceIndicators	1162
17.7.1 Function description	1162
17.7.2 Input and output parameters	1167
17.7.3 Output values	1169
17.8 mdcGetLog	1170
17.8.1 Function description	1170
17.8.2 Input and output parameters	1170
17.8.3 Output values	1171
17.9 mdcGetLogData	1172

17.9.1	Function description	1172
17.9.2	Input and output parameters	1175
17.9.3	Output values	1176
17.10	mdcGetMessageType	1177
17.10.1	Function description	1177
17.10.2	Input and output parameters	1178
17.10.3	Output values	1179
17.11	mdcGetRegisteredUser	1180
17.11.1	Function description	1180
17.11.2	Input and output parameters	1182
17.11.3	Output values	1183
17.12	mdcGetStationConditions	1184
17.12.1	Function description	1184
17.12.2	Input and output parameters	1190
17.12.3	Output values	1191
17.13	mdcGetStationConditionsByFilter	1192
17.13.1	Function description	1192
17.13.2	Input and output parameters	1198
17.13.3	Output values	1200
17.14	mdcManageConditionCode	1201
17.14.1	Function description	1201
17.14.2	Input and output parameters	1204
17.14.3	Output values	1205
17.15	mdcRegisterUserAtLine	1207
17.15.1	Function description	1207
17.15.2	Input and output parameters	1208
17.15.3	Output values	1209
17.16	mdcUpdateStationCondition	1210
17.16.1	Function description	1210
17.16.2	Input and output parameters	1214
17.16.3	Output values	1215
17.17	mdcUploadDetectionPoint	1217
17.17.1	Function description	1217
17.17.2	Input and output parameters	1220
17.17.3	Output values	1221
17.18	mdcUploadStationCondition	1223
17.18.1	Function description	1223
17.18.2	Input and output parameters	1226
17.18.3	Output values	1227
18	Group "ML" (material & logistics)	1229
18.1	mlChangeMaterialBinData	1229

18.1.1	Function description	1229
18.1.2	Input and output parameters	1231
18.1.3	Output values	1232
18.2	mlCreateNewMaterialBin	1234
18.2.1	Function description	1234
18.2.2	Input and output parameters	1239
18.2.3	Output values	1240
18.3	mlGetIncomingGoods	1242
18.3.1	Function description	1242
18.3.2	Input and output parameters	1246
18.3.3	Output values	1247
18.4	mlGetMaterialBinData	1248
18.4.1	Function description	1248
18.4.2	Input and output parameters	1253
18.4.3	Output values	1254
18.5	mlGetMaterialTransactions	1255
18.5.1	Function description	1255
18.5.2	Input and output parameters	1258
18.5.3	Output values	1259
18.6	mlGetNextMaterialBinNumber	1260
18.6.1	Function description	1260
18.6.2	Input and output parameters	1260
18.6.3	Output values	1261
18.7	mlGetPurchaseOrderData	1262
18.7.1	Function description	1262
18.7.2	Input and output parameters	1266
18.7.3	Output values	1267
18.8	mlGetStorage	1268
18.8.1	Function description	1268
18.8.2	Input and output parameters	1270
18.8.3	Output values	1271
18.9	mlGetStorageData	1272
18.9.1	Function description	1272
18.9.2	Input and output parameters	1278
18.9.3	Output values	1280
18.10	mlSetMaterialBinLocation	1281
18.10.1	Function description	1281
18.10.2	Input and output parameters	1281
18.10.3	Output values	1282
18.11	mlSplitMaterialBin	1283
18.11.1	Function description	1283

18.11.2 Input and output parameters	1286
18.11.3 Output values	1287
18.12 mlUpdateStorage	1289
18.12.1 Function description	1289
18.12.2 Input and output parameters	1292
18.12.3 Output values	1294
18.13 mlUpdateStorageCell	1295
18.13.1 Function description	1295
18.13.2 Input and output parameters	1297
18.13.3 Output values	1298
18.14 mlUpdateStorageGroup	1299
18.14.1 Function description	1299
18.14.2 Input and output parameters	1301
18.14.3 Output values	1302
18.15 mlUploadMaterialBinBooking	1303
18.15.1 Function description	1303
18.15.2 Input and output parameters	1306
18.15.3 Output values	1307
19 Group "MSG" (iTAC.Messaging)	1309
19.1 msgPublish	1309
19.1.1 Function description	1309
19.1.2 Input and output parameters	1310
19.1.3 Output values	1311
19.2 msgSubscribe	1312
19.2.1 Function description	1312
19.2.2 Input and output parameters	1312
19.2.3 Output values	1313
19.3 msgUnsubscribe	1314
19.3.1 Function description	1314
19.3.2 Input and output parameters	1314
19.3.3 Output values	1315
20 Group "MSL" (Moisture Sensitivity Level)	1317
20.1 mslBeginDryObject	1317
20.1.1 Function description	1317
20.1.2 Input and output parameters	1320
20.1.3 Output values	1321
20.2 mslEndDryObject	1323
20.2.1 Function description	1323
20.2.2 Input and output parameters	1325
20.2.3 Output values	1326
20.3 mslGetMinDryTimeForSerialnumber	1328

20.3.1	Function description	1328
20.3.2	Input and output parameters	1331
20.3.3	Output values	1332
20.4	mslGetObjectInfo	1333
20.4.1	Function description	1333
20.4.2	Input and output parameters	1339
20.4.3	Output values	1340
20.5	mslStartObjectExpiration	1341
20.5.1	Function description	1341
20.5.2	Input and output parameters	1343
20.5.3	Output values	1344
20.6	mslStopObjectExpiration	1346
20.6.1	Function description	1346
20.6.2	Input and output parameters	1348
20.6.3	Output values	1349
21	Group "PM" (production management)	1351
21.1	pmGetBooking	1351
21.1.1	Function description	1351
21.1.2	Input and output parameters	1358
21.1.3	Output values	1359
21.2	pmGetFailureData	1360
21.2.1	Function description	1360
21.2.2	Input and output parameters	1364
21.2.3	Output values	1365
21.3	pmGetFailureTallyCard	1366
21.3.1	Function description	1366
21.3.2	Input and output parameters	1370
21.3.3	Output values	1371
21.4	pmGetProductQuantity	1372
21.4.1	Function description	1372
21.4.2	Input and output parameters	1377
21.4.3	Output values	1379
21.5	pmUpdateBooking	1380
21.5.1	Function description	1380
21.5.2	Input and output parameters	1385
21.5.3	Output values	1386
21.6	pmUploadBooking	1388
21.6.1	Function description	1388
21.6.2	Input and output parameters	1395
21.6.3	Output values	1396
21.7	pmUploadQuantityAndFailureData	1398

21.7.1	Function description	1398
21.7.2	Input and output parameters	1401
21.7.3	Output values	1403
21.8	pmUploadQuantityFailureTallyCardAndFailureData	1405
21.8.1	Function description	1405
21.8.2	Input and output parameters	1409
21.8.3	Output values	1410
22	Group "REG" (registration)	1413
22.1	regCheckLicense	1413
22.1.1	Function description	1413
22.1.2	Input and output parameters	1417
22.1.3	Output values	1418
22.2	regGetRegisteredUser	1419
22.2.1	Function description	1419
22.2.2	Input and output parameters	1419
22.2.3	Output values	1420
22.3	regLogin	1421
22.3.1	Function description	1421
22.3.2	Input and output parameters	1423
22.3.3	Output values	1424
22.4	regLogout	1425
22.4.1	Function description	1425
22.4.2	Input and output parameters	1425
22.4.3	Output values	1426
22.5	regRegisterUser	1427
22.5.1	Function description	1427
22.5.2	Input and output parameters	1427
22.5.3	Output values	1428
22.6	regUnregisterUser	1429
22.6.1	Function description	1429
22.6.2	Input and output parameters	1429
22.6.3	Output values	1430
23	Group "SETUP" (setup)	1431
23.1	setupAttachPreSetup	1431
23.1.1	Function description	1431
23.1.2	Input and output parameters	1435
23.1.3	Output values	1436
23.2	setupChangeSetupForSerialNumber	1438
23.2.1	Function description	1438
23.2.2	Input and output parameters	1442
23.2.3	Output values	1443

23.3	setupCheck.....	1444
23.3.1	Function description	1444
23.3.2	Input and output parameters	1445
23.3.3	Output values	1448
23.4	setupCheckPreSetup.....	1450
23.4.1	Function description	1450
23.4.2	Input and output parameters	1456
23.4.3	Output values	1458
23.5	setupCreatePreSetup	1460
23.5.1	Function description	1460
23.5.2	Input and output parameters	1466
23.5.3	Output values	1468
23.6	setupGetMaterialSetup	1470
23.6.1	Function description	1470
23.6.2	Input and output parameters	1473
23.6.3	Output values	1475
23.7	setupGetPreSetupDetails	1476
23.7.1	Function description	1476
23.7.2	Input and output parameters	1482
23.7.3	Output values	1484
23.8	setupGetRequiredMaterialForPreSetup	1486
23.8.1	Function description	1486
23.8.2	Input and output parameters	1490
23.8.3	Output values	1491
23.9	setupGetSetupDataBySerialNumber	1492
23.9.1	Function description	1492
23.9.2	Input and output parameters	1496
23.9.3	Output values	1497
23.10	setupGetSetupDataBySerialNumberRef	1498
23.10.1	Function description	1498
23.10.2	Input and output parameters	1503
23.10.3	Output values	1504
23.11	setupGetSetupName	1506
23.11.1	Function description	1506
23.11.2	Input and output parameters	1506
23.11.3	Output values	1507
23.12	setupGetStationSetupDataForMaterialBin.....	1508
23.12.1	Function description	1508
23.12.2	Input and output parameters	1509
23.12.3	Output values	1510
23.13	setupSearchPreSetup.....	1511

23.13.1	Function description	1511
23.13.2	Input and output parameters	1515
23.13.3	Output values	1517
23.14	setupStateChange	1518
23.14.1	Function description	1518
23.14.2	Input and output parameters	1519
23.14.3	Output values	1520
23.15	setupUpdateMaterialSetup	1522
23.15.1	Function description	1522
23.15.2	Input and output parameters	1525
23.15.3	Output values	1527
23.16	setupUpdatePreSetup	1529
23.16.1	Function description	1529
23.16.2	Input and output parameters	1535
23.16.3	Output values	1537
23.17	setupUpdatePreSetupPosition.....	1539
23.17.1	Function description	1539
23.17.2	Input and output parameters	1543
23.17.3	Output values	1545
24	Group "SHIP" (packing)	1547
24.1	shipActivateShippingLotAtKap.....	1547
24.1.1	Function description	1547
24.1.2	Input and output parameters	1547
24.1.3	Output values	1548
24.2	shipAddChildLotToParentLot	1549
24.2.1	Function description	1549
24.2.2	Input and output parameters	1550
24.2.3	Output values	1551
24.3	shipAddSerialNumberToShippingLot.....	1553
24.3.1	Function description	1553
24.3.2	Input and output parameters	1554
24.3.3	Output values	1555
24.4	shipCheckSerialNumberAddToShippingLot.....	1557
24.4.1	Function description	1557
24.4.2	Input and output parameters	1558
24.4.3	Output values	1559
24.5	shipCheckSerialNumberFromShippingLot.....	1561
24.5.1	Function description	1561
24.5.2	Input and output parameters	1561
24.5.3	Output values	1562
24.6	shipCompleteLot.....	1563

24.6.1	Function description	1563
24.6.2	Input and output parameters	1564
24.6.3	Output values	1565
24.7	shipDeactivateShippingLotAtKap	1566
24.7.1	Function description	1566
24.7.2	Input and output parameters	1566
24.7.3	Output values	1567
24.8	shipGetChildLotsForParentLot	1568
24.8.1	Function description	1568
24.8.2	Input and output parameters	1571
24.8.3	Output values	1572
24.9	shipGetLotFromSerialNumber	1573
24.9.1	Function description	1573
24.9.2	Input and output parameters	1576
24.9.3	Output values	1577
24.10	shipGetSerialNumberDataForShippingLot	1578
24.10.1	Function description	1578
24.10.2	Input and output parameters	1580
24.10.3	Output values	1581
24.11	shipGetShippingLotInfo	1582
24.11.1	Function description	1582
24.11.2	Input and output parameters	1587
24.11.3	Output values	1588
24.12	shipGetShippingPrefs	1589
24.12.1	Function description	1589
24.12.2	Input and output parameters	1593
24.12.3	Output values	1594
24.13	shipMoveChildLots	1595
24.13.1	Function description	1595
24.13.2	Input and output parameters	1596
24.13.3	Output values	1597
24.14	shipRemoveChildLots	1598
24.14.1	Function description	1598
24.14.2	Input and output parameters	1599
24.14.3	Output values	1600
24.15	shipRemoveSerialNumberFromShippingLot	1601
24.15.1	Function description	1601
24.15.2	Input and output parameters	1602
24.15.3	Output values	1603
24.16	shipReuseCompletedShippingLot	1604
24.16.1	Function description	1604

24.16.2	Input and output parameters	1607
24.16.3	Output values	1608
24.17	shipSendLot.....	1609
24.17.1	Function description	1609
24.17.2	Input and output parameters	1610
24.17.3	Output values	1611
25	Group "SMT" (placement data).....	1613
25.1	smtCheckMaterialBin.....	1613
25.1.1	Function description	1613
25.1.2	Input and output parameters	1618
25.1.3	Output values	1619
25.2	smtSerialNumberCheckIn.....	1620
25.2.1	Function description	1620
25.2.2	Input and output parameters	1624
25.2.3	Output values	1626
25.3	smtSetupData	1628
25.3.1	Function description	1628
25.3.2	Input and output parameters	1634
25.3.3	Output values	1636
26	Group "TR" (traceability).....	1639
26.1	trActivateOrderCluster	1639
26.1.1	Function description	1639
26.1.2	Input and output parameters	1641
26.1.3	Output values	1642
26.2	trActivateWorkorder.....	1644
26.2.1	Function description	1644
26.2.2	Input and output parameters	1644
26.2.3	Output values	1646
26.3	trActivateWorkstep.....	1647
26.3.1	Function description	1647
26.3.2	Input and output parameters	1649
26.3.3	Output values	1650
26.4	trAddExternalSerialNumber	1652
26.4.1	Function description	1652
26.4.2	Input and output parameters	1657
26.4.3	Output values	1659
26.5	trAddSerialNumberToPool	1661
26.5.1	Function description	1661
26.5.2	Input and output parameters	1663
26.5.3	Output values	1664
26.6	trAddSerialNumberToSerialNumberPool	1666

26.6.1	Function description	1666
26.6.2	Input and output parameters	1667
26.6.3	Output values	1668
26.7	trAssignSerialNumberForProductOrWorkOrder.....	1669
26.7.1	Function description	1669
26.7.2	Input and output parameters	1670
26.7.3	Output values	1672
26.8	trAssignSerialNumberMergeAndUploadState	1674
26.8.1	Function description	1674
26.8.2	Input and output parameters	1675
26.8.3	Output values	1677
26.9	trAssignSerialnumberToOrderCluster	1679
26.9.1	Function description	1679
26.9.2	Input and output parameters	1682
26.9.3	Output values	1683
26.10	trChangePoolSerialNumber	1685
26.10.1	Function description	1685
26.10.2	Input and output parameters	1687
26.10.3	Output values	1688
26.11	trChangeSerialNumberPool	1689
26.11.1	Function description	1689
26.11.2	Input and output parameters	1690
26.11.3	Output values	1691
26.12	trChangeWorkOrder	1692
26.12.1	Function description	1692
26.12.2	Input and output parameters	1694
26.12.3	Output values	1695
26.13	trCheckMergedPartsForSerialNumber	1696
26.13.1	Function description	1696
26.13.2	Input and output parameters	1698
26.13.3	Output values	1699
26.14	trCheckMergedPartsForSerialNumberComplete	1700
26.14.1	Function description	1700
26.14.2	Input and output parameters	1701
26.14.3	Output values	1702
26.15	trCheckSerialNumberState	1703
26.15.1	Function description	1703
26.15.2	Input and output parameters	1707
26.15.3	Output values	1708
26.16	trCheckUserSkill	1711
26.16.1	Function description	1711

26.16.2	Input and output parameters	1713
26.16.3	Output values	1714
26.17	trCheckUserSkills	1715
26.17.1	Function description	1715
26.17.2	Input and output parameters	1717
26.17.3	Output values	1718
26.18	trCreateSerialNumberPool.....	1720
26.18.1	Function description	1720
26.18.2	Input and output parameters	1721
26.18.3	Output values	1722
26.19	trDeleteSerialNumberPool	1723
26.19.1	Function description	1723
26.19.2	Input and output parameters	1723
26.19.3	Output values	1724
26.20	trGetCapturedRepairHistoryData.....	1725
26.20.1	Function description	1725
26.20.2	Input and output parameters	1729
26.20.3	Output values	1730
26.21	trGetMergeHistory	1731
26.21.1	Function description	1731
26.21.2	Input and output parameters	1735
26.21.3	Output values	1736
26.22	trGetMergeParts	1737
26.22.1	Function description	1737
26.22.2	Input and output parameters	1740
26.22.3	Output values	1741
26.23	trGetMultiStationSettings	1742
26.23.1	Function description	1742
26.23.2	Input and output parameters	1746
26.23.3	Output values	1747
26.24	trGetNextProductionStep.....	1748
26.24.1	Function description	1748
26.24.2	Input and output parameters	1751
26.24.3	Output values	1752
26.25	trGetNextSerialNumber	1753
26.25.1	Function description	1753
26.25.2	Input and output parameters	1754
26.25.3	Output values	1755
26.26	trGetPools.....	1756
26.26.1	Function description	1756
26.26.2	Input and output parameters	1758

26.26.3	Output values	1759
26.27	trGetPoolSerialNumber	1760
26.27.1	Function description	1760
26.27.2	Input and output parameters	1762
26.27.3	Output values	1764
26.28	trGetProductQuantity	1765
26.28.1	Function description	1765
26.28.2	Input and output parameters	1770
26.28.3	Output values	1772
26.29	trGetResultDataForSerialNumber	1773
26.29.1	Function description	1773
26.29.2	Input and output parameters	1776
26.29.3	Output values	1778
26.30	trGetSerialNumberActivityUploadInfo	1779
26.30.1	Function description	1779
26.30.2	Input and output parameters	1782
26.30.3	Output values	1783
26.31	trGetSerialNumberBySerialNumberRef	1784
26.31.1	Function description	1784
26.31.2	Input and output parameters	1784
26.31.3	Output values	1785
26.32	trGetSerialNumberForWorkorderAndWorkstep	1786
26.32.1	Function description	1786
26.32.2	Input and output parameters	1789
26.32.3	Output values	1791
26.33	trGetSerialNumberHistoryData	1793
26.33.1	Function description	1793
26.33.2	Input and output parameters	1801
26.33.3	Output values	1803
26.34	trGetSerialNumberInfo	1804
26.34.1	Function description	1804
26.34.2	Input and output parameters	1808
26.34.3	Output values	1809
26.35	trGetSerialNumberPoolInfo	1810
26.35.1	Function description	1810
26.35.2	Input and output parameters	1811
26.35.3	Output values	1813
26.36	trGetSerialNumberUploadInfo	1814
26.36.1	Function description	1814
26.36.2	Input and output parameters	1818
26.36.3	Output values	1819

26.37trGetStationResult	1820
26.37.1 Function description	1820
26.37.2 Input and output parameters	1824
26.37.3 Output values	1826
26.38trGetStationSetting	1827
26.38.1 Function description	1827
26.38.2 Input and output parameters	1831
26.38.3 Output values	1832
26.39trGetTestplanData	1833
26.39.1 Function description	1833
26.39.2 Input and output parameters	1845
26.39.3 Output values	1847
26.40trGetTopFailures	1848
26.40.1 Function description	1848
26.40.2 Input and output parameters	1852
26.40.3 Output values	1853
26.41trGetWorkOrderForStation	1854
26.41.1 Function description	1854
26.41.2 Input and output parameters	1861
26.41.3 Output values	1862
26.42trIdentifyPoolSerialNumber	1863
26.42.1 Function description	1863
26.42.2 Input and output parameters	1863
26.42.3 Output values	1864
26.43trMergeParts	1865
26.43.1 Function description	1865
26.43.2 Input and output parameters	1866
26.43.3 Output values	1868
26.44trRemoveMergeParts	1870
26.44.1 Function description	1870
26.44.2 Input and output parameters	1871
26.44.3 Output values	1872
26.45trRemoveSerialNumberFromOrder	1873
26.45.1 Function description	1873
26.45.2 Input and output parameters	1874
26.45.3 Output values	1875
26.46trRemoveSerialNumberFromSerialNumberPool	1877
26.46.1 Function description	1877
26.46.2 Input and output parameters	1877
26.46.3 Output values	1878
26.47trSerialNumberPassing	1879

26.47.1 Function description	1879
26.47.2 Input and output parameters	1881
26.47.3 Output values	1882
26.48trSimulateMerge	1885
26.48.1 Function description	1885
26.48.2 Input and output parameters	1888
26.48.3 Output values	1889
26.49trSplitPanel	1891
26.49.1 Function description	1891
26.49.2 Input and output parameters	1893
26.49.3 Output values	1894
26.50trSwitchSerialNumber	1896
26.50.1 Function description	1896
26.50.2 Input and output parameters	1897
26.50.3 Output values	1899
26.51trTestPaa	1900
26.51.1 Function description	1900
26.51.2 Input and output parameters	1901
26.51.3 Output values	1902
26.52trTestSpa	1903
26.52.1 Function description	1903
26.52.2 Input and output parameters	1904
26.52.3 Output values	1905
26.53trUploadActivity	1906
26.53.1 Function description	1906
26.53.2 Input and output parameters	1909
26.53.3 Output values	1911
26.54trUploadFailureAndResultData	1913
26.54.1 Function description	1913
26.54.2 Input and output parameters	1919
26.54.3 Output values	1922
26.55trUploadFailuresForMergedParts	1926
26.55.1 Function description	1926
26.55.2 Input and output parameters	1931
26.55.3 Output values	1933
26.56trUploadPmQuantity	1936
26.56.1 Function description	1936
26.56.2 Input and output parameters	1937
26.56.3 Output values	1938
26.57trUploadResultDataAndRecipe	1939
26.57.1 Function description	1939

26.57.2	Input and output parameters	1943
26.57.3	Output values	1945
26.58	trUploadState	1948
26.58.1	Function description	1948
26.58.2	Input and output parameters	1950
26.58.3	Output values	1952
26.59	trUploadStationResult	1954
26.59.1	Function description	1954
26.59.2	Input and output parameters	1958
26.59.3	Output values	1960
26.60	trUploadTestplanResult	1962
26.60.1	Function description	1962
26.60.2	Input and output parameters	1968
26.60.3	Output values	1971
26.61	trVerifyMerge	1974
26.61.1	Function description	1974
26.61.2	Input and output parameters	1975
26.61.3	Output values	1977
26.62	trVerifyMergeProduct	1979
26.62.1	Function description	1979
26.62.2	Input and output parameters	1980
26.62.3	Output values	1981
27	References	1983
27.1	IMSApi - error codes	1983
27.1.1	Technological error codes	1984
27.1.2	Group "COMMON" (error codes)	1986
27.1.3	Group "TR" (error codes)	1996
27.1.4	Group "SHIP" (error codes)	2003
27.1.5	Group "ML" (error codes)	2006
27.1.6	Group "SETUP" (error codes)	2009
27.1.7	Group "MDA" (error codes)	2011
27.1.8	Group "ATTRIB" (error codes)	2013
27.1.9	Group "ADVICE" (error codes)	2015
27.1.10	Group "MDATA" (error codes)	2016
27.1.11	Group "MDC" (error codes)	2018
27.1.12	Group "EQU" (error codes)	2020
27.1.13	Group "IMSAPI" (error codes)	2023
27.1.14	Group "CUSTOM" (error codes)	2024
27.1.15	Group "SMT" (error codes)	2025
27.1.16	Group "MSL" (error codes)	2026
27.1.17	Group "EDA" (error codes)	2027

27.1.18	Group "CONFIG" (error codes)	2028
27.1.19	Group "PM" (error codes)	2029
27.1.20	Group "CRP" (error codes)	2030
27.1.21	Group "APS" (error codes)	2032
27.1.22	Group "MSG" (error codes)	2033
27.2	Replaced functions	2034
27.2.1	setupReplaceSetupForSerialNumber	2034
27.2.1.1	Function description	2034
27.2.1.2	Input and output parameters	2034
27.2.1.3	Output values	2036
27.2.2	smtConsumption	2037
27.2.2.1	Function description	2037
27.2.2.2	Input and output parameters	2038
27.2.2.3	Output values	2038
27.2.3	smtEventSetup	2040
27.2.3.1	Function description	2040
27.2.3.2	Input and output parameters	2041
27.2.3.3	Output values	2041
27.2.4	smtSerialNumberSetup	2043
27.2.4.1	Function description	2043
27.2.4.2	Input and output parameters	2044
27.2.4.3	Output values	2044
27.3	Configuration parameter	2046
27.3.1	Station parameter (IMSApi)	2046
27.3.2	Server parameter (IMSApi)	2085
27.3.3	Site parameter (IMSApi)	2088
27.4	Change history	2100
Index		2121

1

General information on the IMSApi



Advice

In the reference part of this documentation, you will find a list of all changes to the predecessor versions (see section »Change history« [▶▶▶])!

1.1

Introduction

Various functionalities of the iTAC system are available via the IMSApi (iTAC.MES.Suite **A**pplication **P**rogramming **I**nterface) for separate applications and integration in other system environments.

The production equipment is connected via the respective IMSApi functions. Together with the possibilities of the IMSApi functions, the basic functions permit the mapping of the specific processes within a production machine/ production line.

Thus, the IMSApi is characterized by its documentation and openness as well as the fact that it remains constant over very long periods of time. This means that external programs which were developed for a specific version of the IMSApi will also function with subsequent versions. At the same time, however, each new version of the IMSApi will bring with it new functions, without having a negative effect on the existing functionality.

Test option

The API module in the myiTAC workplace offers the option of testing all IMSApi functions. Individual functions as well as entire scenarios can be tested in this way. Enter the appropriate input parameters for individual IMSApi functions directly or, for complex test scenarios, in a previously prepared XML file. Afterwards, you can analyze the results in the output.



Advice

When using the API module, note that the `sessionContext` function parameter (first parameter in each IMSApi function) cannot be edited. The information passed there is automatically taken over from the current myiTAC workplace settings: `locale` according to interface language, `persId` according to logged-in user (the `sessionId` cannot be influenced)!

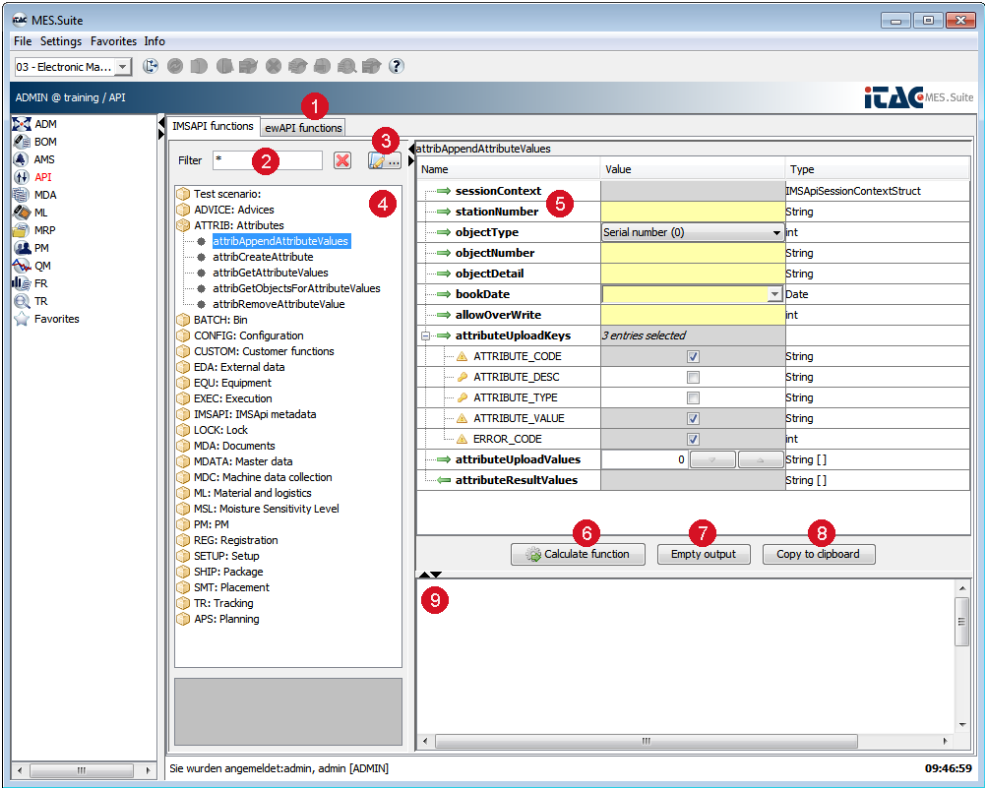


Fig. 1-1 API module (IMSApi functions) in the myiTAC workplace

1	Changes to the test area for the ewAPI functions; these are provided by the ewAPI service (predecessor of the IMSApi service). Advice: The ewAPI functions are described in a separate document (see ewAPI-Description-[X.XX-VX.X-XX].pdf)! The IMSApi functions should always be used for new installations!
2	Field for filtering according to specific entries; entries must be confirmed with <Enter>. To delete a filter, click the adjacent button with the red X. Advice: The wildcard characters "*" and "?" can be used when entering the filter: "*" = Placeholder for an arbitrary number of characters "?" = Placeholder for exactly one character Entries are NOT case sensitive!

3	INTERNAL function: Enables the selection of an XML file in which input parameters of multiple IMSApi functions are specified for individual test scenarios. In this way the system can calculate any IMSApi functions in a specific order using the desired values. Advice: All imported XML files can be selected under (4) – "Test scenario" entry!
4	Selection of the desired IMSApi function; the respective input parameters are displayed under (5).
5	Depending on the selected IMSApi function, the respective input parameters are listed here. The "Value" column can be edited directly. Confirm the input of each value with <Enter>. Advice: The input parameters of the function (top level) are always marked as mandatory fields (yellow background). Due to technical reasons, this also applies to the special cases where input is not required, i.e. where an empty string is valid. Here, an input field with yellow background can remain empty and the function still be successfully executed. The IMSApi documentation contains special case descriptions under the input parameters concerned.
6	The function is calculated and the results are displayed in the output (9).
7	The output is cleared.
8	The entire output is copied to the clipboard.
9	Display of the function results.

1.2

IMSApi service

The IMSApi functions are all implemented in a single service named `/mes/imsapi/service`. The implementation of the IMSApi service is developed 100% in Java and can be accessed via IHAP (iTAC High Availability Protocol). The following information is provided for each function:

- Return data type in the function (Java notation)
- Name of the function
- Parameter list, each with the following attributes
 - direction of the parameter (in, out, inout)
 - data type of the parameter (Java notation)
 - name of the parameter

Example of a function declaration

```
int attribGetObjectsForAttributeValues
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int objectType
  in String attributeCode
  in String attributeValue
  in int maxRows
  in KeyValue[] filterKeys
    String key
    String value
  in String[] resultKeys
  out String[] resultValues
```

Data types can also represent self-defined data types here. These may be structures or arrays; arrays can be recognized by the square brackets []. In the above example, `IMSApiSessionContextStruct` and `KeyValue` are self-defined data types.

**Advice**

With `out` and `inout` parameters, note that the "holder" data types of the respective parameter data type are used.

When passing floating point values within strings, a "." must always be used as the decimal separator; the use of "," causes data to be misinterpreted and may cause the software to behave incorrectly!

*All character strings (data type = `string`) are always to be used as **Unicode** at the IMSApi interface.*

IMSApi data types

The following IMSApi data types are possible here:

IMSApi data type	Meaning
<code>int</code>	32-bit signed integer value
<code>long</code>	64-bit signed integer value
<code>double</code>	Floating point number (64 Bit)
<code>boolean</code>	TRUE or FALSE
<code>String</code>	UNICODE string of arbitrary length
<code>String[]</code>	Array with UNICODE strings of arbitrary length
<code>DateMs</code>	64-bit signed integer value (corresponds to <code>long</code>) Advice: iTAC-specific data type used exclusively for processing time stamps!

Tab. 1-1 Description of the IMSApi data types

There are here absolutely no inherent dependencies on specific operating systems, hardware details (big/little endian of the CPU), programming languages etc.

All systems for which a communication component is offered can use the IMSApi. All advantages of the ARTES client library are available here ("LoadBalancing", "FailOver", etc.).

A range of libraries are available to application programmers who would like to use the IMSApi (see section »Overview of IMSApi connection« [▶▶▶]).

IMSApi mandatory fields

For all parameters/keys with mandatory field designation, the interface expects a concrete value; the passing of an empty string usually results in an interface error.

Logging "IMSApi service"

The iTAC system creates a log file for each "technical" service; this also applies for the IMSApi service. The scope of these log files with regard to size, the number of generations and the log level (information depth) can be configured in the myiTAC workplace using the dialog shown below (ADM module -> Configuration -> Client configuration" - Application type "logging").



Advice

At this point, we would like to **expressly** point out that we reserve the right to make changes to the format and content of the log output without prior notice. This is particularly important to note if you plan to read out certain contents "automatically" from the log file!

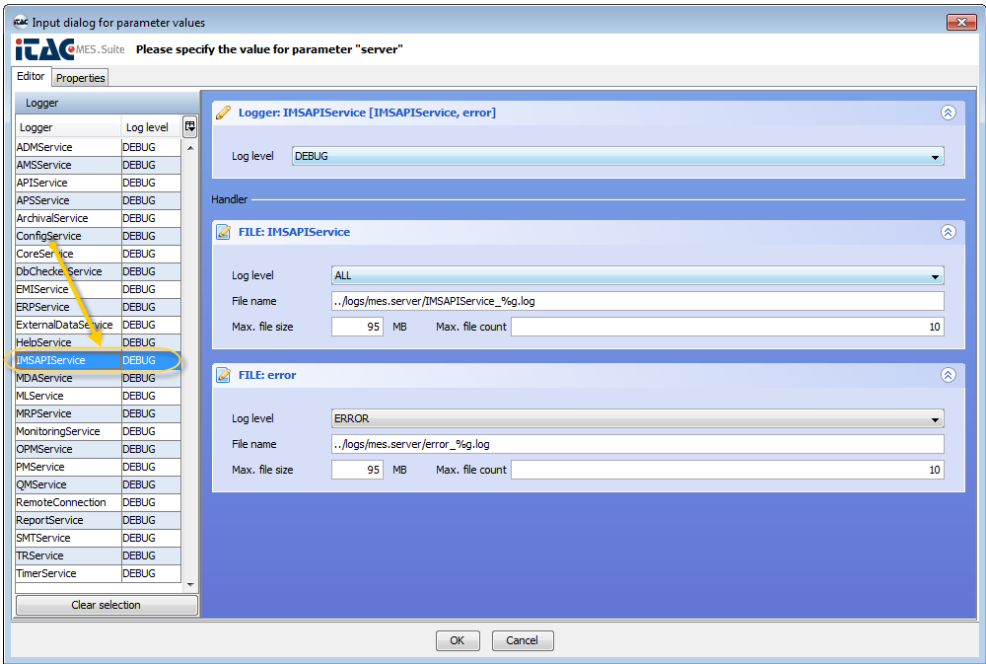


Fig. 1-2 Logging "IMSApi service"

"Key Value Pair"
structures

In many cases, IMSApi functions that access complex data objects (recipes, work plan structures, order data, etc.) use so-called "Key Value Pair" structures. These functions can be used flexibly, since which of the possible parameters (keys) are to be used and which are not relevant to the given call is defined in the function call itself.

Technical background

All parameters from the "Key Value Pair" structures exist as data type "string" and are, thus, untyped. This results in a corresponding amount of additional work in the creation of client applications. All high-level programming languages do, however, provide appropriate methods for type conversion.

The caller must specify the parameters ("keys") required within the input array. The sequence specified there by the developer must be adhered to by the server when filling the return array. This eliminates the need to repeatedly specify the parameters in the return array.



Example

Of the possible parameters ("name", "LIL", "LSL", "min", "max", "tol"), the developer needs only the following three parameters ("min", "max", "tol"). For this purpose, he fills the input array as follows:

```
* input[0] = "min"
* input[1] = "max"
* input[2] = "tol"
```

If the server returns a total of two measured values, the array that is returned has the following form (example):

```
* output[0] = "minValue1"
* output[1] = "maxValue1"
* output[2] = "toleranceValue1"
* output[3] = "minValue2"
* output[4] = "maxValue2"
* output[5] = "toleranceValue2"
```

If the caller were to send the parameters in a different order, the server would likewise need to respond using this (different) sequence.

Application types

When using "Key Value Pair" structures, a distinction is made between two application types:

- 1. "Key Value Pair" as **filter** for searching for specific records
- 2. "Key Value Pair" as **IN/OUT array** for the processing sequence

re: 1. With this application type, the "Key Value Pair" structure is used to pass filter criteria to the server; these are typically "get" functions with which specific search criteria can be selected from a list.



Example

Possible keys: partNumber, partDesc, workorderNumber, workorderDesc

Key	Value
partNumber	A0123456*
workorderDesc	LP*

Whether or not wildcard characters (e.g. *) can be used for a given parameter is specified in the description of the function.

re 2. In this application type, a "Key Value Pair" structure is passed with the requested/to-be-uploaded parameters. This list of parameters shows what information is to be processed in what sequence. In addition, a "Key Value Pair" structure is passed with the requested/to-be-uploaded values. Included in this – in the same sequence as the parameters – are the corresponding values.



Example

Possible keys: partNumber, partDesc, partGrpNumber, partGrpDesc, compName, compType

resultKeys: *partNumber, partDesc, compName*

resultValues: *A123456, Part1, X12
B34556, Part2, Y23*

For this application type, there are two "Key Value Pair" structures to fill, whereby the client always specifies which parameters are used.

1.3

Overview of IMSApi connection

With the provision of various libraries, work for the developer is simplified for a number of operating systems and programming languages. With one exception (IMSApi REST), all of the connection possibilities named below automatically contain the ARTES Client Library, which enables LoadBalancing and FailOver.

1.3.1

Communication model

IMSApi communication model

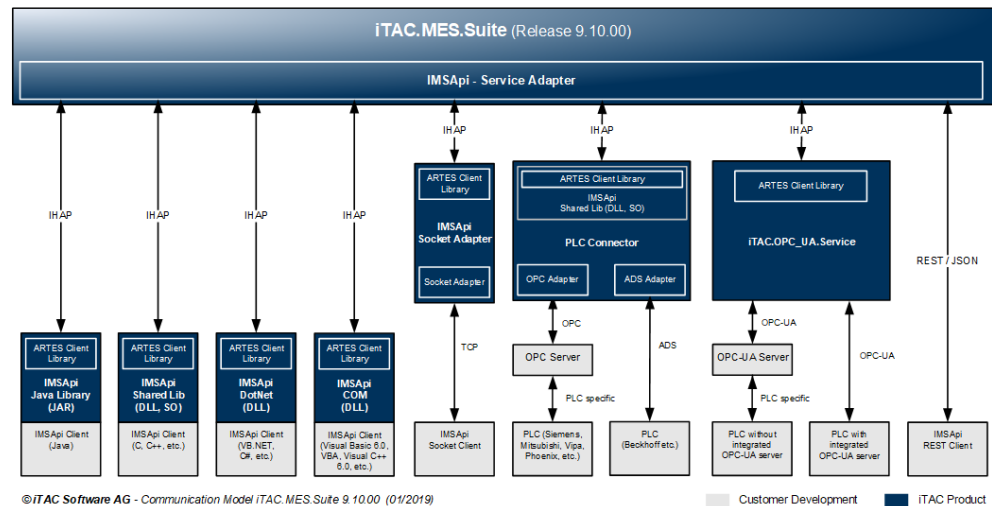


Fig. 1-3 IMSApi communication model

Connection possibilities

	Languages	Active comp.	Linkage (files)	3rd Party
IMSApi Java Library	Java	no	jar files	no
IMSApi Shared Lib ^{a)}	C / C++	no	dll, lib, h / so, h	no
IMSApi DotNet ^{b)}	all .NET languages	no	dll	no
IMSApi COM ^{c), d)}	all Microsoft COM languages	no	dll	no
IMSApi socket adapter	various	yes	-	no
IMSApi PLC Connector	PLC, config	yes	config	yes ^{e)}
IMSApi OPC_UA.Service	PLC, config	yes	config	yes ^{f)}
IMSApi REST	various	no	-	no

Tab. 1-2 Overview of the connection possibilities

- Re ^{a)} 32-bit and 64-bit libraries
 Re ^{b)} .NET Framework as of version 2.0 required
 Re ^{c)} 32-bit library (can be used in 32-bit applications only)
 Re ^{d)} .NET Framework as of version 2.0 required
 Re ^{e)} OPC server of the PLC
 Re ^{f)} OPC-UA server of the PLC

Advice: The released operating systems are given in the official system requirements (for further information, see the documentation »System requirements«)!

1.3.2

IMSApi Java Library

Accessing the IMSApi by means of a Java application

For Java client applications, we make the IMSApi Java Library available; this can be used directly by Java applications. All features of the ARTES Client Library are immediately available to the application (e.g. FailOver, LoadBalancing). The appropriate JAR files and an example Java client are provided to the IMSApi client developer for this purpose.

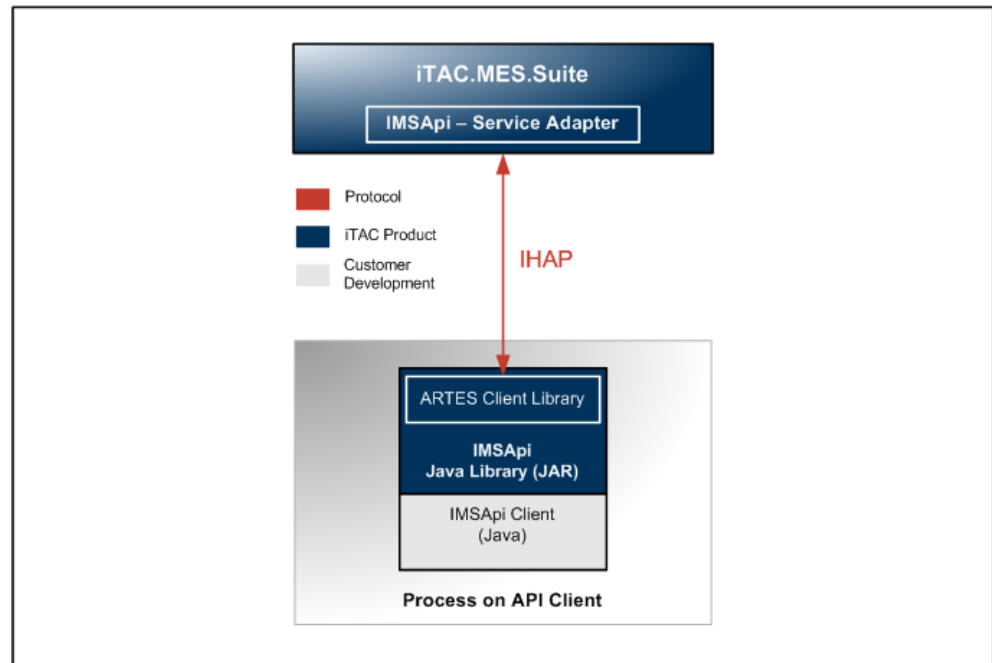


Fig. 1-4 *IMSApi Java Library*

1.3.3

IMSApi Shared Lib

Accessing the IMSApi by means of shared libraries (DLL, SO)

Shared libraries are available for Windows and Linux. These are supplied in the form of "dll" (dynamic link library for Windows) or "so" (shared object for Linux) files. In addition, the required C header files, import library files etc. are also supplied. These contain all mapping of the functions and data types from C/C++ to CORBA-IDL; this information is absolutely necessary for developing an IMSApi-Client.

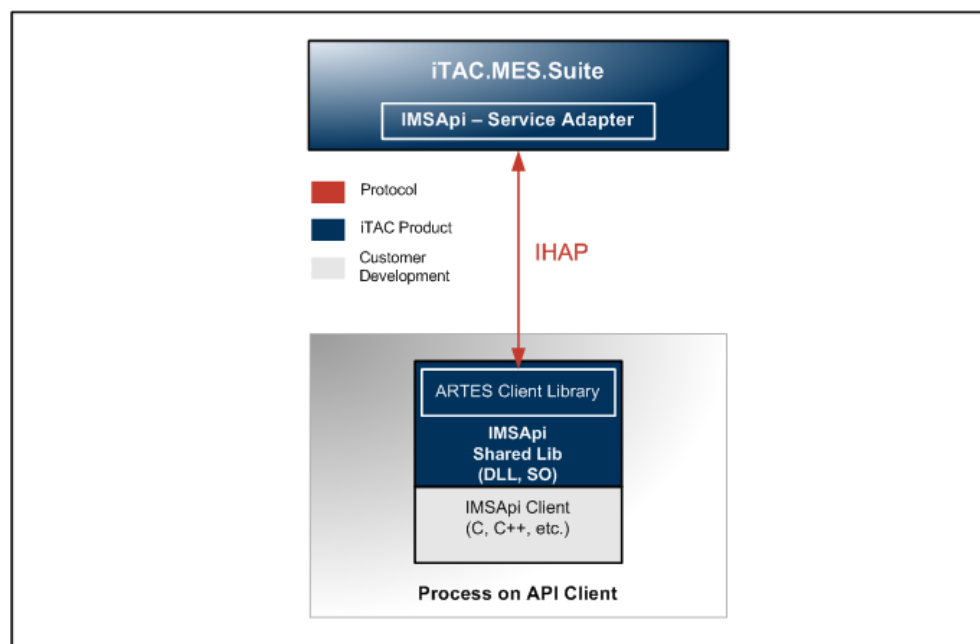


Fig. 1-5 IMSApi Shared Lib

The shared libraries can be used by an application developer in the accustomed manner for the respective operating system or programming language. Development is usually performed in C or C++. Shared libraries can, in principle, be used by nearly any programming language or programming environment. iTAC only provides support for C/C++, however.

Example programs are supplied with the shared libraries to illustrate their use with the C programming language.

The connection between the shared library and the ARTES Client Library is performed internally by means of JNI.



Advice

Further information can be found in the »IMSApi-SharedLibrary« documentation.

1.3.4

IMSApi DotNet

Accessing the IMSApi by means of .NET (DLL)

A .NET library is available. This so-called assembly in the form of a "dll" can be used by all .NET languages on Windows platforms.

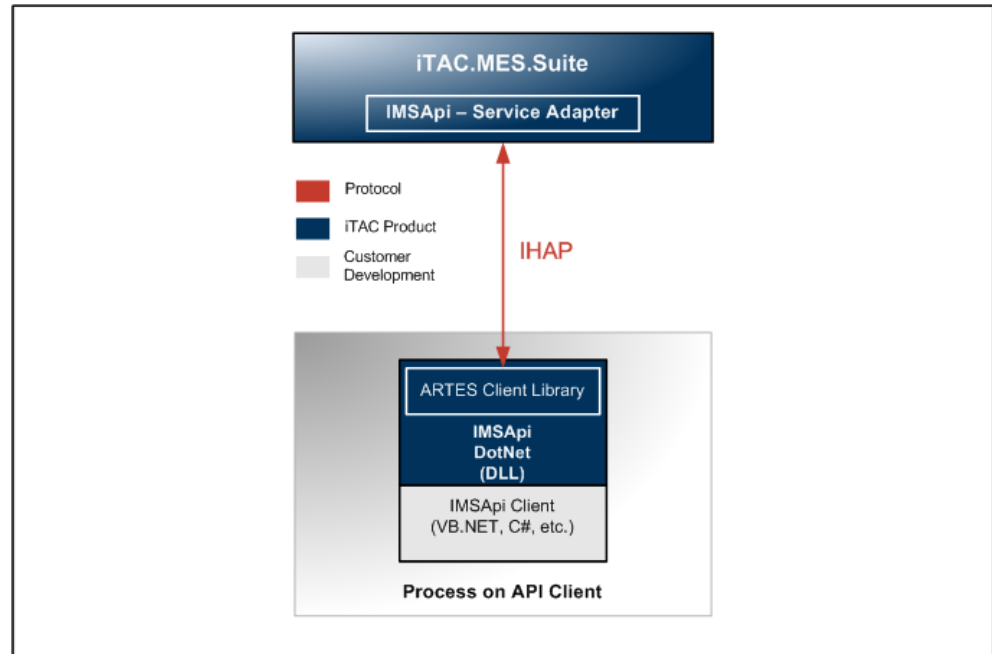


Fig. 1-6 *IMSApi DotNet*

The "IMSApi DotNet" library can be integrated by an application developer in the method typical for the given .NET programming language.

Supplied together with the library is an example program that illustrates use with the C# programming language. The library contains a version of the ARTES Client Library completely ported to C#. With this, the clients of IMSApi DotNet likewise have access to high-availability mechanisms such as "FailOver" and "LoadBalancing".

**Advice**

Further information can be found in the »IMSApi-DotNet« documentation.

1.3.5

IMSApi COM

Accessing the IMSApi by means of Microsoft COM (DLL)

A COM library is available. This DLL can be used by all languages on Windows platforms that support the "Microsoft Component Object Model" (COM).

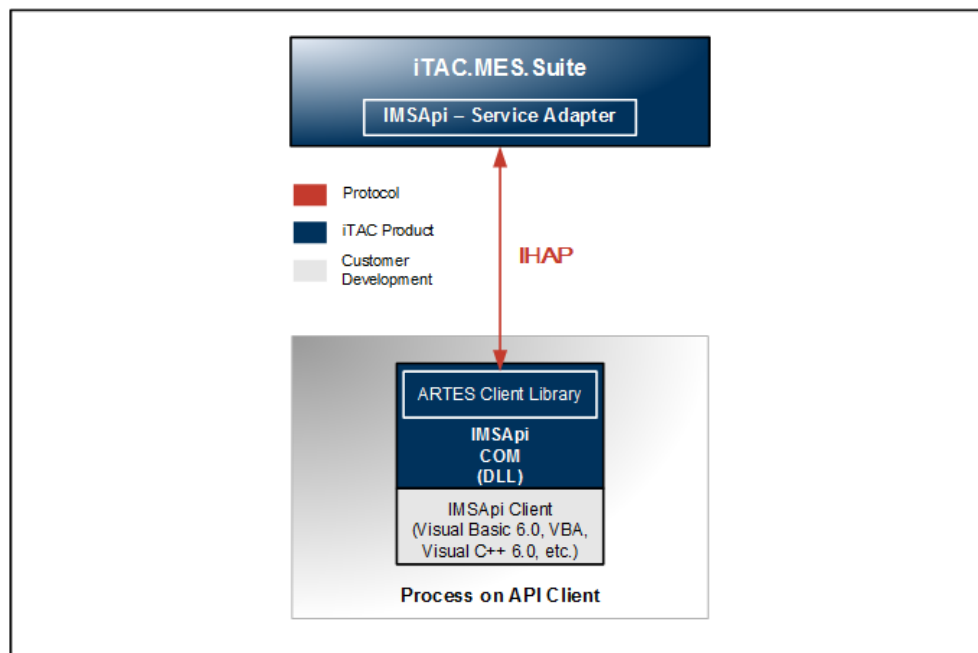


Fig. 1-7 IMSApi COM

The "IMSApi COM" library can be integrated by an application developer in the method typical for the given programming language.

Supplied together with the library are two example projects which demonstrate use of the library in Visual Basic 6.0 and Visual Basic for Applications (as a Microsoft Excel 2003 template).



Advice

*The library can be used **only** in 32-bit applications. Further information can be found in the »IMSApi-COM« documentation.*

1.3.6

IMSAPI socket adapter

Accessing the IMSAPI without special client libraries

With the IMSAPI socket adapter, it is possible to call up IMSAPI functions via sockets and serial interfaces (RS232) without special client libraries. This is performed using a simple, string-based protocol. The socket adapter should be used whenever no other IMSAPI library technology (e.g. IMSAPI Shared Lib, IMSAPI DotNet etc.) can be used; e.g. with "exotic" operating systems (e.g. OS-9, SCO-UNIX, etc.) and/or with programming languages that cannot handle the standard libraries.

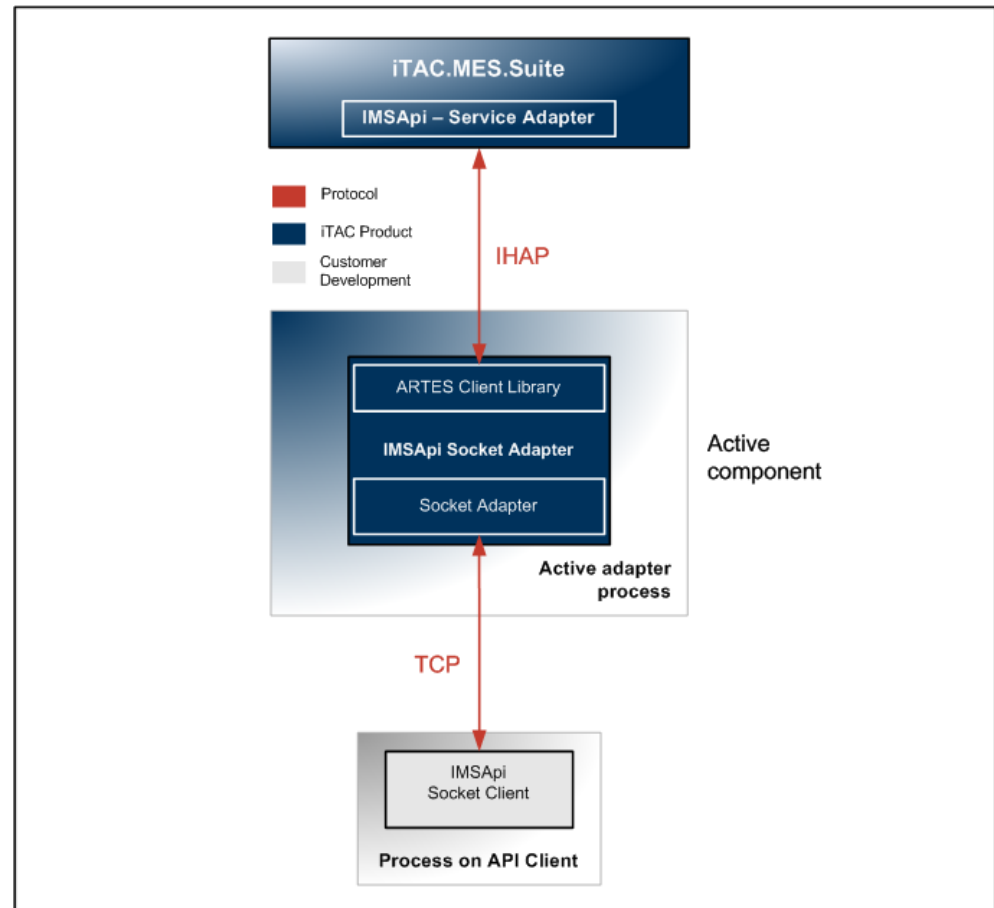


Fig. 1-8 IMSAPI socket adapter

**Advice**

Further information can be found in the »IMSAPI-SocketAdapter« documentation.

1.3.7

IMSapi PLC Connector

Accessing the IMSapi within PLC systems

The PLC Connector is available for using the IMSapi directly from PLC systems. In this case, the corresponding OPC server is used on the PLC side. The OPC server is generally available from the PLC manufacturer.

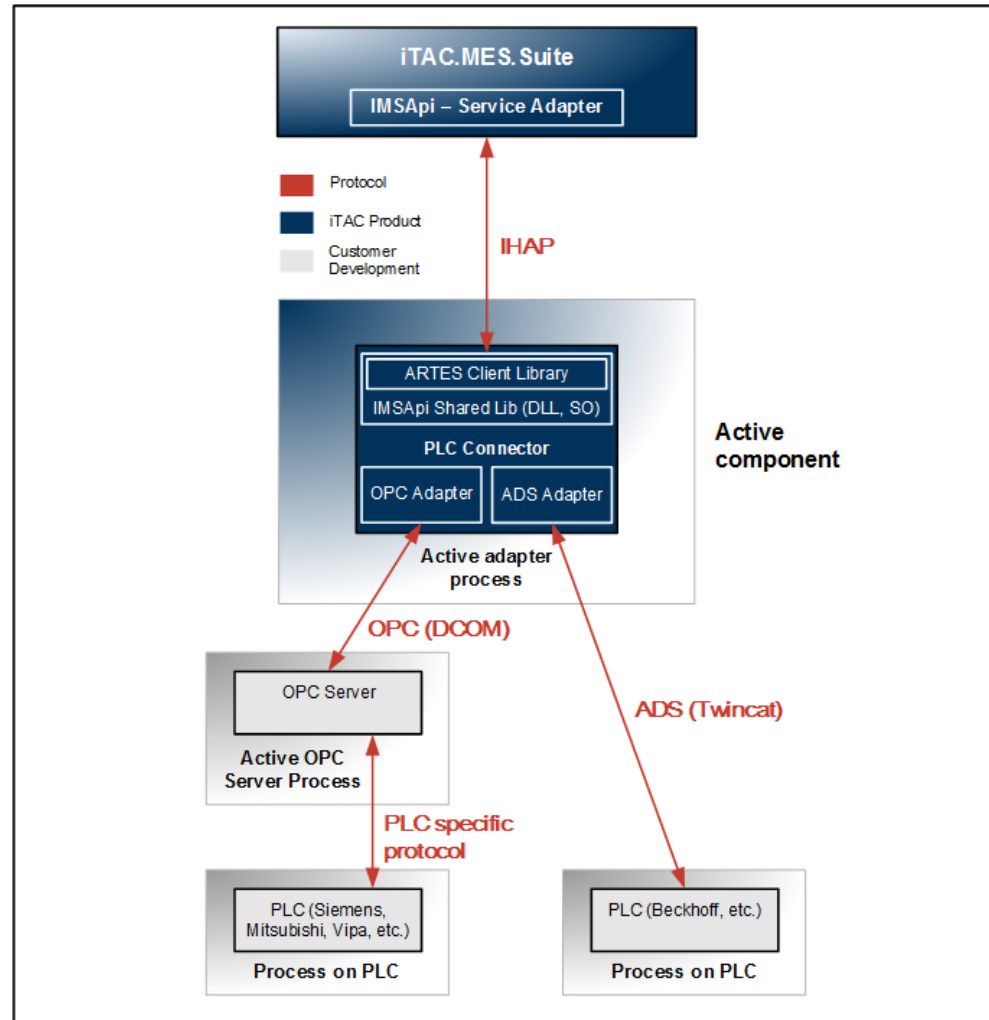


Fig. 1-9 IMSapi PLC Connector

In this case, the PLC adapter communicates as an OPC client with the OPC server of the PLC and then transforms the calls into ARTES/IHAP; these are then communicated to the IMSapi server by the ARTES Client Library. Because the PLC adapter is an active process which communicates by means of OPC and because OPC is based on Microsoft COM/DCOM, this process must be installed on a Windows system. Microsoft does not supply a COM/DCOM implementation for other systems (such as e.g. Linux). This will, however, change with future OPC versions. OPC will then be based on a platform independent protocol (OPC UA by means of SOAP).

The development of a PLC program which communicates with the IMSapi server by means of the PLC Connector occurs by means of three methods:

- OPC variable definition in the OPC server of the PLC
- Configuration of the PLC Connector
- Optional program development on the PLC

In addition to communicating by means of OPC, with whose assistance the most common PLCs can be coupled, a connection to Beckhoff PLCs can be performed by means of TwinCAT ADS in the PLC adapter.

A concrete implementation of a PLC IMSApi-Client is generally performed in cooperation with PLC developers and iTAC.Consulting.



Example

Siemens OPC Server
Kepware OPC Server

1.3.8

IMSApi iTAC.OPC-UA.Service

Accessing the IMSApi within OPC-UA-based PLC systems

The iTAC.OPC-UA.Service is available for using the IMSApi directly from OPC-UA-based PLC systems. Here, either the associated OPC-UA server or a PLC with integrated OPC-UA server is used on the PLC side. In the case of a separate OPC-UA server, this is usually available from the PLC manufacturer.

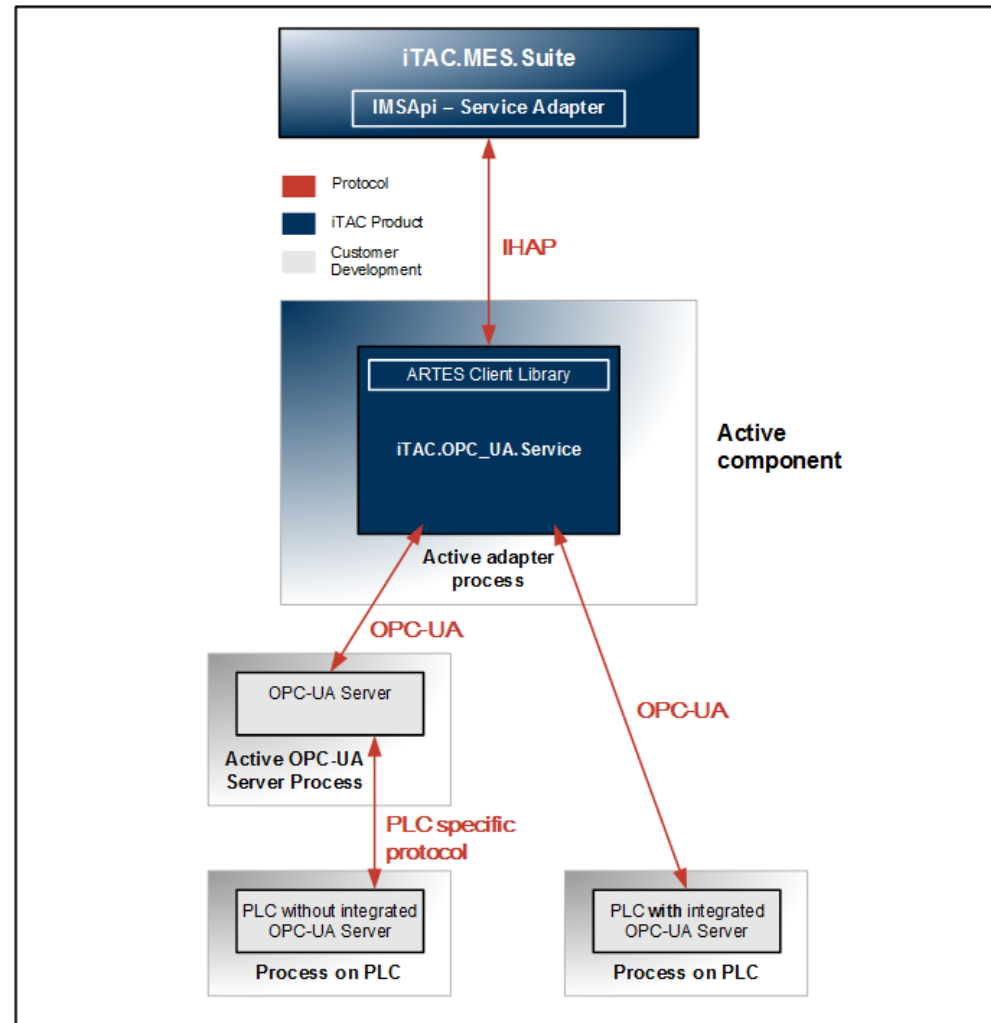


Fig. 1-10 IMSApi iTAC.OPC-UA.Service

In this case, the iTAC.OPC-UA.Service communicates as an OPC-UA client with the OPC-UA server of the PLC and then transforms the calls into ARTES/IHAP; these are then communicated to the IMSApi server by the ARTES Client Library. The iTAC.OPC-UA.Service is designed as a separate process and can be installed as a 100% pure Java process on all released server operating systems. The development of a PLC program which communicates with the IMSApi server by means of the iTAC.OPC-UA.Services occurs by means of three methods:

- OPC-UA variable definition in the OPC-UA server of the PLC
- Configuration of the iTAC.OPC-UA.Service
- Optional program development on the PLC

A concrete implementation of a PLC IMSApi-Client is generally performed in cooperation with PLC developers and iTAC.Consulting.

1.3.9

IMSApi REST

Accessing the IMSApi by means of REST/JSON

The IMSApi is also available in the application server as a REST service.

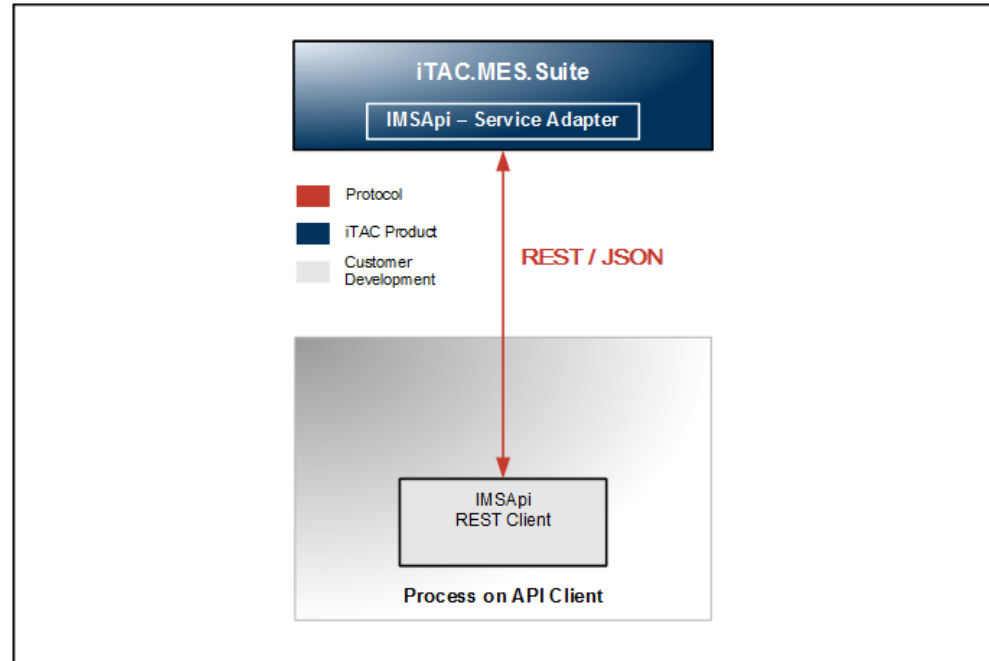


Fig. 1-11 IMSApi REST

As a result, IMSApi calls are also possible from HTML5 or JavaScript, for example. Access is provided in particular for the development of applications on mobile devices.

Only calls in JSON format are supported; a library is not offered. Connection via REST should only take place if no other suitable library is available.

**Advice**

It is important to note that NO FailOver/LoadBalancing is available if the REST service is used! The ARTES layer, which provides the high-availability mechanisms, is not used here! Further information can be found in the »IMSApi-REST« documentation.

1.4

Session handling

When using the »IMSApi Shared Lib« [▶▶▶], »IMSApi DotNet« [▶▶▶] and »IMSApi COM« [▶▶▶] API libraries, note the following:

- When calling the technical functions (see section »IMSApi functions« [▶▶▶]), the first parameter is ALWAYS the `sessionContext` structure. This structure contains three items: The `sessionId`, the `persID` and the `locale`. The `sessionId` is the unique designation of the active session. The `persID` is used for the user-related booking of data. With the `locale` element, it is possible to explicitly execute each API call for a specific language. The `locale` must be specified as a unicode string in the form [language_designator]_ [country_designator] (e.g. "de_DE" or "en_US").
- The `sessionContext` structure must first be ascertained by means of the »regLogin« [▶▶▶] function; the language currently set on the server is returned as `locale`; if necessary, this language setting can be changed by the developer.
- Multiple sessions may be active simultaneously (multiple logins). The passed `sessionContext` specifies the session for which the respective call is being executed.



Advice

The »regLogin« [▶▶▶] function should only be called exactly **ONCE** once for a given user/station! It is only necessary to call the function again if the current session is no longer valid. Each technical IMSAPI function has the value "-3" (session invalid) as one of the possible output values. **ONLY** in this situation is it necessary to obtain a new session by calling the »regLogin« function again. After being called, each technical function must **ALWAYS** be checked for the output value "-3"; if necessary, a new »regLogin« is then to be performed!

Cyclical calling of the »regLogin« and »regLogout« [▶▶▶] functions with the same user/station is unnecessary, problematic with respect to performance and should therefore not be used in any case!

2

IMSApi functions

In the following sections, all functions – divided into application groups – are shown with their variables and their input and output values.



Advice

On each call of the functions listed here, the first parameter is ALWAYS the `sessionContext` structure; this structure contains the `sessionId`, the `persId` and the `locale`. The `sessionContext` must first be defined using the »regLogin« [▶▶▶] function!

3 Group "ADVICE" (advice notices)

3.1 adviceConfirmAdvice

3.1.1 Function description

This function is used to collect the data necessary for confirming a customer-specific advice dialog. Here, the data for user authentication and advice confirmation is passed to the server.



Advice

Advice notices managed in the iTAC system can be displayed in external applications as dialog boxes. The appearance of the advice notices is event-driven and can be configured for various objects. This function is always used in combination with the »adviceGetAdvice« [▶▶▶] function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int adviceConfirmAdvice
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String userName
in String password
in int adviceId
in int confirmationStatus
in String[] adviceEditTextArray
```

3.1.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
userName	Login name of the user (user identification)
password	Password of the user
adviceId	Internal identification number of the advice
confirmationStatus	Confirmation state [0; 1]: 0 = advice not confirmed 1 = advice confirmed

Parameter name	Explanation
<code>adviceEditTextArray</code>	Supplemental text for the advice (optional)

Tab. 3-1 Parameter (in) »adviceConfirmAdvice«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

3.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-5	Failure	Password is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-1001	Failure	Advice is unknown
-1002	Failure	Advice confirmation has failed
-2105	Failure	No TR or PM license present for the station

3.2

adviceGetAdvice

3.2.1

Function description

This function is used to query customer-specific advice data. On the basis of the entered reference data, the function returns an array of non-confirmed advice notices.



Advice

Advice notices managed in the iTAC system can be displayed in external applications as dialog boxes. The appearance of the advice notices is event-driven and can be configured for various objects. This function is always used in combination with the »adviceConfirmAdvice« [▶▶▶] function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int adviceGetAdvice
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in boolean ignoreStationNumber
in boolean checkStationGroup
in boolean checkPartGroup
in KeyValue[] adviceFilters
    String key
    String value
out Advice[] adviceArray
    int id
    String text
    String name
    int mdaRootKey
    int adviceType
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `adviceArray`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ERP_GROUP_NUMBER	ERP group number in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String
USER_NAME	Login name of the user (user identification) Advice: This can be used to specifically search for advice that must still be confirmed by a given user!	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 3-2 "Key Value Pair": *adviceFilters*

3.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
ignoreStationNumber	Controls whether the station number is taken into account when ascertaining the advice data [0; 1] 1 = stationNumber is not evaluated (TRUE) 0 = stationNumber is evaluated (FALSE)
checkStationGroup	Controls whether the machine group of the station is also taken into account when ascertaining the advice data (stationNumber) [0; 1] 1 = machine group is taken into account (TRUE) 0 = machine group is not taken into account (FALSE)
checkPartGroup	Controls whether the part group of the part is also taken into account when ascertaining the advice data (PART_NUMBER) [0; 1] 1 = part group is taken into account (TRUE) 0 = part group is not taken into account (FALSE)
adviceFilters	"Key Value Pair" structure with the following variables:
key	List of the advice parameters on the basis of which the output is to be filtered; the corresponding values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": <i>adviceFilters</i> table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": <i>adviceFilters</i> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 3-3 Parameter (in) »*adviceGetAdvice*«

Parameter (out)

Parameter name	Explanation
adviceArray	Array of the structure with the following variables
id	Unique designator (ID) for the advice
text	Supplementary text for the advice; multiple lines must be separated using a semicolon
name	Advice name
mdaRootKey	Root directory to the referenced MDA items
adviceType	Designator for advice type [0; 1; 2]: 0 = Once during the period: Within the validity period, the advice appears only once for respective user (at the first occurrence of the event). 1 = Always during the period: Within the validity period, the advice appears repeatedly for the respective user (at each occurrence of the event). 2 = Always: Regardless of the validity period, the advice appears repeatedly for the respective user (at each occurrence of the event).

Tab. 3-4 Parameter (out) »adviceGetAdvice«

3.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-125	Failure	ERP group not found
-130	Failure	Work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

4 Group "APS" (work order planning)

4.1 apsGetEquipments

4.1.1 Function description

This function is used to query production-means-related planning results of the APS service. Depending on the `EQUIPMENT_TYPE` key, the production means are ascertained separately according to the PM classes "Equipment (work step)", "Equipment (asset)" and "Carrier".



Advice

Currently, only production means of the "Equipment (work step)" PM class are planned and output here by the APS service.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetEquipments
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] apsGetEquipmentsFilters
    String key
    String value
in String[] apsGetEquipmentsResultKeys
out String[] apsGetEquipmentsResultValues
out boolean[] hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `apsGetEquipmentsFilters` and `apsGetEquipmentsResultKeys` as well as (out) `apsGetEquipmentsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐ DateMs
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavities Advice: If this key is not passed along, the [E] value is used by default. Note that the PM class must match the selected PM type (see key EQUIPMENT_TYPE)!	☐ String
EQUIPMENT_GROUP	Number of the equipment group in the iTAC system Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Use of "Equipment (work step)": In addition to the classic grouping, equipment groups can also be used when assigning equipment to work steps. Use of "Equipment (asset)": Here, equipment groups are used only for grouping without any other control function. Background: The use of equipment groups is optional. If they are to be used, they must be stored in the master data (ADM module).	☐ String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number and only needs to be specified if the EQUIPMENT_NUMBER is not unique! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Key	Explanation	IMSApi data type
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
WORKORDER_NUMBER	Number of the production order for which the production means were planned Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	String

Tab. 4-1 "Key Value Pair": *apsGetEquipmentsFilters*

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
APS_PRODUCTION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
APS_PRODUCTION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
EQUIPMENT_CLASS	PM class; classifies the production means into equipment and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity	☐ String
EQUIPMENT_GROUP	Number of the equipment group in the iTAC system Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Use of "Equipment (work step)": In addition to the classic grouping, equipment groups can also be used when assigning equipment to work steps. Use of "Equipment (asset)": Here, equipment groups are used only for grouping without any other control function. Background: The use of equipment groups is optional. If they are to be used, they must be stored in the master data (ADM module).	☐ String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number!	☐ String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number!	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E)	☐ String
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐ int

Key	Explanation	IMSApi data type
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	int
WORKORDER_NUMBER	Number of the production order for which the production means were planned	String
WORKSTEP_DESC	Description of the work step for which the production means were planned	String
WORKSTEP_NUMBER	Number of the work step for which the production means were planned	int

Tab. 4-2 "Key Value Pair": *apsGetEquipmentsResultKeys*

4.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetEquipments- Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetEquipmentsFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetEquipmentsFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
apsGetEquipments- ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetEquipmentsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) apsGetEquipmentsResultValues !

Tab. 4-3 Parameter (in) »apsGetEquipments«

Parameter (out)

Parameter name	Explanation
apsGetEquipments- ResultValues	Depending on parameter (in) apsGetEquipmentsResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) apsGetEquipmentsResultKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-4 Parameter (out) »apsGetEquipments«

4.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

4.2 **apsGetStations**

4.2.1 Function description

This function makes it possible to query station-related planning results of the APS service.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetStations
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] apsGetStationsFilters
    String key
    String value
in String[] apsGetStationsResultKeys
out String[] apsGetStationsResultValues
out boolean[] hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `apsGetStationsFilters` and `apsGetStationsResultKeys` as well as (out) `apsGetStationsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐ DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐ int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 4-5 "Key Value Pair": *apsGetStationsFilters*

Definition of the output parameters for the found data

Key	Explanation		IMSApi data type
APS_PRODUCTION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_PRODUCTION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int

Tab. 4-6 "Key Value Pair": *apsGetStationsResultKeys*

4.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
KeyValuePair	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetStationsFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": apsGetStationsFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
apsGetStationsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetStationsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) apsGetStationsResultValues !

Tab. 4-7 Parameter (in) »apsGetStations«

Parameter (out)

Parameter name	Explanation
apsGetStationsResultValues	Depending on parameter (in) apsGetStationsResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) apsGetStationsResultKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the START_POSITION and MAX_ROW_COUNT keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-8 Parameter (out) »apsGetStations«

4.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

4.3 **apsGetWorkergroups**

4.3.1 Function description

This function is used to query person-group-related planning results of the APS service. Only the user groups provided in the iTAC system that are of type **APS user group** are interpreted.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetWorkergroups
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] apsGetWorkergroupsFilters
    String key
    String value
  in String[] apsGetWorkergroupsResultKeys
  out String[] apsGetWorkergroupsResultValues
  out boolean[] hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `apsGetWorkergroupsFilters` and `apsGetWorkergroupsResultKeys` as well as (out) `apsGetWorkergroupsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐ DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐ int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
USER_GROUP_NAME	Name of the APS user group in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 4-9 "Key Value Pair": *apsGetWorkergroupsFilters*

Definition of the output parameters for the found data

Key	Explanation		IMSApi data type
APS_PRODUCTION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_PRODUCTION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
USER_GROUP_DESCRIPTION	Description of the APS user group	☐	String
USER_GROUP_NAME	Name of the APS user group in the iTAC system	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int

Tab. 4-10 "Key Value Pair": *apsGetWorkergroupsResultKeys*

4.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetWorker-groupsFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkergroupsFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetWorkergroupsFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
apsGetWorker-groupsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkergroupsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>apsGetWorkergroupsResultValues</code> !

Tab. 4-11 Parameter (in) »apsGetWorkergroups«

Parameter (out)

Parameter name	Explanation
apsGetWorker-groupsResultValues	Depending on parameter (in) <code>apsGetWorkergroupsResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>apsGetWorkergroupsResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-12 Parameter (out) »apsGetWorkergroups«

4.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

4.4 **apsGetWorkers**

4.4.1 Function description

This function makes it possible to query person-related planning results of the APS service.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetWorkers
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] apsGetWorkersFilters
    String key
    String value
  in String[] apsGetWorkersResultKeys
  out String[] apsGetWorkersResultValues
  out boolean[] hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `apsGetWorkersFilters` and `apsGetWorkersResultKeys` as well as (out) `apsGetWorkersResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐ DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐ int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
USER_NAME	Login name of the user (user identification)	☐ String
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 4-13 "Key Value Pair": *apsGetWorkersFilters*

Definition of the output parameters for the found data

Key	Explanation		IMSApi data type
APS_PRODUC-TION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_PRODUC-TION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
USER_FIRST_NAME	First name of the user	☐	String
USER_LAST_NAME	Last name of the user	☐	String
USER_NAME	Login name of the user (user identification)	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int

Tab. 4-14 "Key Value Pair": *apsGetWorkersResultKeys*

4.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetWorkersFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkersFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetWorkersFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
apsGetWorkersResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkersResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>apsGetWorkersResultValues</code> !

Tab. 4-15 Parameter (in) »apsGetWorkers«

Parameter (out)

Parameter name	Explanation
apsGetWorkersResultValues	Depending on parameter (in) <code>apsGetWorkersResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>apsGetWorkersResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-16 Parameter (out) »apsGetWorkers«

4.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

4.5

apsGetWorkorderDetails

4.5.1

Function description

This function makes it possible to query order-based planning results of the APS service. Unlike the »apsGetWorkorders« [▶▶▶] IMSApi function, planning results can be queried down to the work step and machine level.



Advice

This API function is affected by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 4572 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

This function should NO LONGER be used for new installations because the edit options have been extended with release 9.00.00 (for further information, see the new function »apsGetWorkorderDetailsOptGroup« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetWorkorderDetails
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] apsGetWorkorderDetailsFilters
    String key
    String value
in String[] apsGetWorkorderDetailResultKeys
out String[] apsGetWorkorderDetailResultValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) apsGetWorkorderDetailsFilters and apsGetWorkorderDetailResultKeys as well as (out) apsGetWorkorderDetailResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation		IMSApi data type
APS_JOB_ID	Unique job designator within the APS service for work orders/work steps that are to be processed in parallel . This ID is particularly relevant if different work orders are manufactured in a group (key word "work order clusters"), whereby work steps of different work orders can then be planned/manufactured simultaneously at one station. Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
APS_JOB_ID_SEQ	Unique job designator within the APS service for work orders/work steps that are to be processed sequentially Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐	DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
OPERATION_MODE	"Data ascertainment" function mode [1; 2]: 1 = only setup times are ascertained 2 = only production times are ascertained Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐	int
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String

Key	Explanation		IMSApi data type
PLANNING_WORKORDER_NUMBER	Number of the plan order; these orders are exclusively work orders that have the order state planned. Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Background: For as long as the work order is not opened (released), it exists in the system as a so-called "plan order". After order release, the plan order becomes the actual production order (cf. the key WORKORDER_NUMBER). Normally, plan order and production order numbers are identical; however, the system provides the possibility of assigning different numbers.	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐	int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
SORT_FLAG	Sorting mode for the output [0; 1; 2; 3]: 0 = ascending according to APS_PRODUCTION_START_DATE 1 = ascending according to WORKORDER_NUMBER 2 = ascending according to PLANNING_WORKORDER_NUMBER 3 = ascending according to CUSTOMER_WORKORDER_NUMBER Advice: If this key is not passed along, the [0] value is used by default. If a different value is entered, the work orders are output unsorted!	☐	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: Multiple state values may be passed in the input (separated by ";", " or ",")! Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐	String

Key	Explanation	IMSApi data type
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific) Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)! Advice: Multiple state values may be passed in the input (separated by ";", " or ";")! Background: With work order activation (work order changes to the "started" state), the state of the individual work steps is converted by the system to "started". In this context, the activation time (key ACTIVATE_DATE) is also set by the system.	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice "work step": If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration! Advice: Multiple state values may be passed in the input (separated by ";", " or ";")!	☐ String

Tab. 4-17 "Key Value Pair": `apsGetWorkorderDetailsFilters`

Definition of the output parameters for the planned work orders

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
APS_END_DATE	The initial production end time (delivery date) planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The point in time is set when the work order is started (cf. the PLANNED_SHIP_DATE key)!	☐	DateMs
APS_JOB_ID	Unique job designator within the APS service for work orders/work steps that are to be processed in parallel. This ID is particularly relevant if different work orders are manufactured in a group (key word "work order clusters"), whereby work steps of different work orders can then be planned/manufactured simultaneously at one station. Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
APS_JOB_ID_SEQ	Unique job designator within the APS service for work orders/work steps that are to be processed sequentially Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	☐	int
APS_PRODUCTION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_PRODUCTION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_RESULT	APS planning result [1; 4; 8; 16]: 1 = not planned 4 = late 8 = critical 16 = OK	☐	int
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs

Key	Explanation		IMSApi data type
APS_START_DATE	The initial production start time planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>The point in time is set when the work order is started (cf. the PLANNED_START_DATE key)!</i>	☐	DateMs
APS_STATE	APS planning state [1; 2; 4; 8; 32]: 1 = unplanned 2 = partially planned 4 = planned 8 = started 32 = end	☐	int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems.	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: <i>The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!</i>	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CLUSTER_DESC	Description of the work order cluster	☐	String
CLUSTER_ID	Internal identification number of the work order cluster	☐	long
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐	String
EARLIEST_START_LAY	Earliest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Background: Time-related process specifications can be stored for each work step of an order-based work plan – this data generally comes from a connected ERP system and is used as the basis for planning. By default, it is possible to store the earliest and latest start times for the processes "Production", "Setup", "Stripdown", "Lay", "Transport" and "Wait".	☐	DateMs
EARLIEST_START_PRODUCTION	Earliest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs

Key	Explanation		IMSApi data type
EARLIEST_START_SETUP	Earliest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_STRIPDOWN	Earliest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_TRANSPORT	Earliest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_WAIT	Earliest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumerical additional information 1	☐	String
INFO_TXT_2	Alphanumerical additional information 2	☐	String
INFO_TXT_3	Alphanumerical additional information 3	☐	String
INFO_TXT_4	Alphanumerical additional information 4	☐	String
INFO_TXT_5	Alphanumerical additional information 5	☐	String
INFO_TXT_6	Alphanumerical additional information 6	☐	String
LATEST_START_LAY	Latest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_PRODUCTION	Latest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_SETUP	Latest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_STRIPDOWN	Latest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_TRANSPORT	Latest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_WAIT	Latest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs

Key	Explanation	IMSApi data type
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐ int
OPTIMIZATION_GROUP_CODE	APS service - unique designator for the optimization group	☐ String
OPTIMIZATION_GROUP_ID	APS service - internal identification number of the optimization group	☐ long
OPTIMIZATION_GROUP_TYPE	APS service - optimization group type [1; 2; 4]: 1 = StaticSetup 2 = DynamicSetup 4 = ParallelUsage	☐ int
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PLANNED_SHIP_DATE	Planned delivery date for the work order according to the last planning run (cf. the APS_END_DATE key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production) according to the last planning run (cf. the APS_START_DATE key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PLANNING_WORKORDER_NUMBER	Number of the plan order (only order state planned)	☐ String
PRIORITY_CODE	APS service - unique designator for the work order priority	☐ String
PRIORITY_NAME	APS service - description for the work order priority	☐ String
PRIORITY_VALUE	Value for the work order priority; the greater the value, the higher the prioritization	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master	☐ String
PRODUCTION_CONTROLLER_DESC	Description of the production controller	☐ String
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double
QUANTITY_FAIL	Number of "fail" bookings	☐ int
QUANTITY_PASS	Number of "pass" bookings	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings	☐ int
QUANTITY_SPLIT	Split quantity from the detailed planning tool (Ganttplan)	☐ int

Key	Explanation		IMSApi data type
QUANTITY_WORKORDER_PASS	Number of serial numbers with the "PASS" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WORKORDER_SCRAP	Number of serial numbers with the "SCRAP" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WORKORDER_TOTAL	Order quantity (total)	☐	int
QUANTITY_WORKSTEP_FAIL	Number of serial numbers booked on the work step with the "FAIL" state	☐	int
QUANTITY_WORKSTEP_PASS	Number of serial numbers booked on the work step with the "PASS" state	☐	int
QUANTITY_WORKSTEP_SCRAP	Number of serial numbers booked on the work step with the "SCRAP" state	☐	int
REQUIRED_WORKER_CNT_PRODUCTION	Number of employees who are to be planned for execution of the work step (production).	☐	int
REQUIRED_WORKER_CNT_SETUP	Number of employees who are to be planned for the necessary work step preparations (e.g. setup).	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
SPLIT_MAX	Maximum partial quantity of a work step for the "Automatic splitting" application type Background: The automated splitting of work steps can be activated in the detailed planning tool (Ganttplan). This requires corresponding specifications to be entered for minimum and maximum partial quantities. Advice: If the value [0] is output, automatic splitting is switched off; in this case, the value for SPLIT_MIN is always [0]!	☐	int
SPLIT_MIN	Minimum partial quantity of a work step for the "Automatic splitting" application type Advice: See "Background" and "Advice" under SPLIT_MAX!	☐	int
WORKER_UTILIZATION_PRODUCTION	Utilization rate (in percent) of the employees required for execution of the work step (production).	☐	double
WORKER_UTILIZATION_SETUP	Utilization rate of the required employees (in percent) for preparation of the work step (e.g. setup).	☐	double
WORKORDER_DESC	Order description	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Key	Explanation		IMSApi data type
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐	long
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String
WORKPLAN_VERS	Work plan version from the iTAC system	☐	int
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐	String

Key	Explanation	IMSApi data type
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 4-18 "Key Value Pair": *apsGetWorkorderDetailResultKeys*

4.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetWorkorder-DetailsFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkorderDetailsFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetWorkorderDetailsFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
apsGetWorkorder-DetailResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkorderDetailResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>apsGetWorkorderDetailResult-Values</code> !

Tab. 4-19 Parameter (in) »apsGetWorkorderDetails«

Parameter (out)

Parameter name	Explanation
apsGetWorkorder-DetailResultValues	Depending on parameter (in) <code>apsGetWorkorderDetailResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>apsGetWorkorderDetailResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-20 Parameter (out) »apsGetWorkorderDetails«

4.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station
-3600	Failure	The result gave multiple optimization groups -> Use the IMSApi "apsGetWorkorderDetailsOptGroup"
		Advice: The value -3600 is output only if the keys OPTIMIZATION_GROUP_CODE, OPTIMIZATION_GROUP_ID and OPTIMIZATION_GROUP_TYPE are to be output, and more than one optimization group is stored!

4.6

apsGetWorkorderDetailsOptGroup

4.6.1

Function description

This function makes it possible to query planning results of the APS service with the optimization groups assigned to a work step (static, dynamic, parallel). Unlike the »apsGetWorkorders« [▶▶▶] IMSApi function, planning results can be queried down to the work step and machine level.

**Advice**

This API function is affected by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 4572 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »apsGetWorkorderDetails« [▶▶▶] function; the output of multiple optimization groups has been added!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetWorkorderDetailsOptGroup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] apsGetWorkorderDetailsOptGroupFilters
    String key
    String value
in String[] apsGetWorkorderDetailsOptGroupResultKeys
out String[] apsGetWorkorderDetailsOptGroupResultValues
in KeyValue[] apsWorkorderOptimizationFilters
    String key
    String value
in String[] apsWorkorderOptimizationResultDataKeys
out String[] apsWorkorderOptimizationResultDataValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) apsGetWorkorderDetailsOptGroupFilters, apsGetWorkorderDetailsOptGroupResultKeys, apsWorkorderOptimizationFilters and apsWorkorderOptimizationResultDataKeys as well as (out) apsGetWorkorderDetailsOptGroupResultValues and apsWorkorderOptimizationResultDataValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values (planning results)

Key	Explanation		IMSApi data type
APS_JOB_ID	Unique job designator within the APS service for work orders/work steps that are to be processed in parallel . This ID is particularly relevant if different work orders are manufactured in a group (key word "work order clusters"), whereby work steps of different work orders can then be planned/manufactured simultaneously at one station. Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
APS_JOB_ID_SEQ	Unique job designator within the APS service for work orders/work steps that are to be processed sequentially Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐	DateMs

Key	Explanation		IMSApi data type
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with <code>START_POSITION</code> , it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
OPERATION_MODE	"Data ascertainment" function mode [1; 2]: 1 = only setup times are ascertained 2 = only production times are ascertained Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐	int
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
PLANNING_WORKORDER_NUMBER	Number of the plan order; these orders are exclusively work orders that have the order state planned . Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Background: For as long as the work order is not opened (released), it exists in the system as a so-called "plan order". After order release, the plan order becomes the actual production order (cf. the key <code>WORKORDER_NUMBER</code>). Normally, plan order and production order numbers are identical; however, the system provides the possibility of assigning different numbers.	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐	int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
SORT_FLAG	Sorting mode for the output [0; 1; 2; 3]: 0 = ascending according to <code>APS_PRODUCTION_START_DATE</code> 1 = ascending according to <code>WORKORDER_NUMBER</code> 2 = ascending according to <code>PLANNING_WORKORDER_NUMBER</code> 3 = ascending according to <code>CUSTOMER_WORKORDER_NUMBER</code> Advice: If this key is not passed along, the [0] value is used by default. If a different value is entered, the work orders are output unsorted!	☐	int

Key	Explanation	IMSApi data type
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: Multiple state values may be passed in the input (separated by "," or ";")! Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific) Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)! Advice: Multiple state values may be passed in the input (separated by "," or ";")! Background: With work order activation (work order changes to the "started" state), the state of the individual work steps is converted by the system to "started". In this context, the activation time (key ACTIVATE_DATE) is also set by the system.	☐ String

Key	Explanation	IMSApi data type
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice work step: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key <code>WORKSTEP_STATE</code> (cross-station total state) must also be taken into consideration! Advice: Multiple state values may be passed in the input (separated by ";", " or ";")!	String

Tab. 4-21 "Key Value Pair": `apsGetWorkorderDetailsOptGroupFilters`

Definition of the output parameters (planning results)

Key	Explanation	IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key <code>WORKSTEP_STATE</code>) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	DateMs
APS_END_DATE	The initial production end time (delivery date) planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The point in time is set when the work order is started (cf. the <code>PLANNED_SHIP_DATE</code> key)!	DateMs
APS_JOB_ID	Unique job designator within the APS service for work orders/work steps that are to be processed in parallel. This ID is particularly relevant if different work orders are manufactured in a group (key word "work order clusters"), whereby work steps of different work orders can then be planned/manufactured simultaneously at one station. Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	String
APS_JOB_ID_SEQ	Unique job designator within the APS service for work orders/work steps that are to be processed sequentially Background: APS jobs are created in the detailed planning tool (Ganttplan) or in the ERP system in order to, for example, group several work orders/work steps together under one task (key word "work order groups").	String
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	int

Key	Explanation		IMSApi data type
APS_PRODUC-TION_END_DATE	The most recent production end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_PRODUC-TION_START_DATE	The most recent production start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_RESULT	APS planning result [1; 4; 8; 16]: 1 = not planned 4 = late 8 = critical 16 = OK	☐	int
APS_SETUP_END_DATE	Setup end time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_SETUP_START_DATE	Setup start time planned by the APS service for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
APS_START_DATE	The initial production start time planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>The point in time is set when the work order is started (cf. the PLANNED_START_DATE key)!</i>	☐	DateMs
APS_STATE	APS planning state [1; 2; 4; 8; 32]: 1 = unplanned 2 = partially planned 4 = planned 8 = started 32 = end	☐	int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems.	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: <i>The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!</i>	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CLUSTER_DESC	Description of the work order cluster	☐	String
CLUSTER_ID	Internal identification number of the work order cluster	☐	long
CLUSTER_NAME	Unique name for the work order cluster	☐	String

Key	Explanation	IMSApi data type
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐ String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐ String
EARLIEST_START_LAY	Earliest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Background: Time-related process specifications can be stored for each work step of an order-based work plan – this data generally comes from a connected ERP system and is used as the basis for planning. By default, it is possible to store the earliest and latest start times for the processes "Production", "Setup", "Stripdown", "Lay", "Transport" and "Wait".	☐ DateMs
EARLIEST_START_PRODUCTION	Earliest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
EARLIEST_START_SETUP	Earliest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
EARLIEST_START_STRIPDOWN	Earliest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
EARLIEST_START_TRANSPORT	Earliest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
EARLIEST_START_WAIT	Earliest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
INFO_NUMBER_1	Numerical additional information 1	☐ double
INFO_NUMBER_2	Numerical additional information 2	☐ double
INFO_NUMBER_3	Numerical additional information 3	☐ double
INFO_NUMBER_4	Numerical additional information 4	☐ double
INFO_TXT_1	Alphanumerical additional information 1	☐ String
INFO_TXT_2	Alphanumerical additional information 2	☐ String
INFO_TXT_3	Alphanumerical additional information 3	☐ String
INFO_TXT_4	Alphanumerical additional information 4	☐ String
INFO_TXT_5	Alphanumerical additional information 5	☐ String
INFO_TXT_6	Alphanumerical additional information 6	☐ String
LATEST_START_LAY	Latest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs

Key	Explanation		IMSApi data type
LATEST_START_PRODUCTION	Latest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_SETUP	Latest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_STRIPDOWN	Latest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_TRANSPORT	Latest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_WAIT	Latest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
OPERATION_MODE	Function mode "data ascertainment" [1; 2]: 1 = Setup time 2 = Production time	☐	int
OPTIMIZATION_GROUP_ID	APS service - internal identification number of the optimization group	☐	long
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PLANNED_SHIP_DATE	Planned delivery date for the work order according to the last planning run (cf. the APS_END_DATE key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production) according to the last planning run (cf. the APS_START_DATE key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
PLANNING_WORKORDER_NUMBER	Number of the plan order (only order state planned)	☐	String
PRIORITY_CODE	APS service - unique designator for the work order priority	☐	String
PRIORITY_NAME	APS service - description for the work order priority	☐	String
PRIORITY_VALUE	Value for the work order priority; the greater the value, the higher the prioritization	☐	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
PROCESS_VERSION	Process version from the iTAC system	☐	int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master	☐	String
PRODUCTION_CONTROLLER_DESC	Description of the production controller	☐	String

Key	Explanation	IMSApi data type
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double
QUANTITY_FAIL	Number of "fail" bookings	☐ int
QUANTITY_PASS	Number of "pass" bookings	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings	☐ int
QUANTITY_SPLIT	Split quantity from the detailed planning tool (Ganttplan)	☐ int
QUANTITY_WORK- KORDER_PASS	Number of serial numbers with the "PASS" state booked on the last work step with obligatory confirmation	☐ int
QUANTITY_WORK- KORDER_SCRAP	Number of serial numbers with the "SCRAP" state booked on the last work step with obligatory confirmation	☐ int
QUANTITY_WORK- KORDER_TOTAL	Order quantity (total)	☐ int
QUANTITY_WORKSTEP_ -FAIL	Number of serial numbers booked on the work step with the "FAIL" state	☐ int
QUANTITY_WORKSTEP_ -PASS	Number of serial numbers booked on the work step with the "PASS" state	☐ int
QUANTITY_WORK- STEP_SCRAP	Number of serial numbers booked on the work step with the "SCRAP" state	☐ int
REQUIRED_WORK- ER_CNT_PRODUCTION	Number of employees who are to be planned for execution of the work step (production).	☐ int
REQUIRED_WORK- ER_CNT_SETUP	Number of employees who are to be planned for the necessary work step preparations (e.g. setup).	☐ int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
SPLIT_MAX	Maximum partial quantity of a work step for the "Automatic splitting" application type Background: The automated splitting of work steps can be activated in the detailed planning tool (Ganttplan). This requires corresponding specifications to be entered for minimum and maximum partial quantities. Advice: If the value [0] is output, automatic splitting is switched off; in this case, the value for SPLIT_MIN is always [0]!	☐ int
SPLIT_MIN	Minimum partial quantity of a work step for the "Automatic splitting" application type Advice: See "Background" and "Advice" under SPLIT_MAX!	☐ int
WORKER_UTILIZA- TION_PRODUCTION	Utilization rate (in percent) of the employees required for execution of the work step (production).	☐ double

Key	Explanation		IMSApi data type
WORKER_UTILIZATION_SETUP	Utilization rate of the required employees (in percent) for preparation of the work step (e.g. setup).	☐	double
WORKORDER_DESC	Order description	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐	long
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String
WORKPLAN_VERS	Work plan version from the iTAC system	☐	int
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐	String

Key	Explanation	IMSApi data type
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	String

Tab. 4-22 "Key Value Pair": *apsGetWorkorderDetailsOptGroupResultKeys*

Searching for specific parameter values (optimization groups)

Key	Explanation	IMSApi data type
OPTIMIZATION_GROUP_CODE	APS service - unique designator for the optimization group	String
OPTIMIZATION_GROUP_TYPE	APS service - optimization group type [1; 2; 4]: 1 = StaticSetup 2 = DynamicSetup 4 = ParallelUsage	int

Tab. 4-23 "Key Value Pair": *apsWorkorderOptimizationFilters*

Definition of the output parameters (optimization groups)

Key	Explanation	IMSApi data type
OPTIMIZATION_GROUP_CODE	APS service - unique designator for the optimization group	String
OPTIMIZATION_GROUP_ID	APS service - internal identification number of the optimization group	long
OPTIMIZATION_GROUP_TYPE	APS service - optimization group type [1; 2; 4]: 1 = StaticSetup 2 = DynamicSetup 4 = ParallelUsage	int
WORKORDER_NUMBER	Production order number	String
WORKSTEP_NUMBER	Number of the work step	int

Tab. 4-24 "Key Value Pair": *apsWorkorderOptimizationResultDataKeys*

4.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetWorkorder-DetailsOptGroup-Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkorderDetailsOptGroupFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetWorkorderDetailsOptGroupFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
apsGetWorkorder-DetailsOpt-GroupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkorderDetailsOptGroupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>apsGetWorkorderDetailsOptGroupResultValues</code> !
apsWorkorderOpti-mizationFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsWorkorderOptimizationFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsWorkorderOptimizationFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
apsWorkorderOpti-mizationResult-DataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsWorkorderOptimizationResultDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>apsWorkorderOptimizationResultDataValues</code> !

Tab. 4-25 Parameter (in) »apsGetWorkorderDetails«

Parameter (out)

Parameter name	Explanation
apsGetWorkorder-DetailsOpt-GroupResultValues	Depending on parameter (in) apsGetWorkorderDetailsOpt-GroupResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) apsGetWorkorderDetailsOpt-GroupResultKeys!
apsWorkorderOptimizationResult-DataValues	Depending on parameter (in) apsWorkorderOptimizationResultDataKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) apsWorkorderOptimizationResultDataKeys!
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-26 Parameter (out) »apsGetWorkorderDetails«

4.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

4.7

apsGetWorkorders

4.7.1

Function description

This function makes it possible to query order-based planning results of the APS service. Unlike the »apsGetWorkorderDetails« [▶▶▶] IMSApi function, NO planning results can be queried down to the work step and machine level.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	APS license

Function declaration

```
int apsGetWorkorders
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] apsGetWorkordersFilters
    String key
    String value
in String[] apsGetWorkordersResultKeys
out String[] apsGetWorkordersResultValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) apsGetWorkordersFilters and apsGetWorkordersResultKeys as well as (out) apsGetWorkordersResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐ String
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [0] passes along the current date and time; if [-1] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐ DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PLANNING_WORKORDER_NUMBER	Number of the plan order; these orders are exclusively work orders that have the order state planned . Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Background: For as long as the work order is not opened (released), it exists in the system as a so-called "plan order". After order release, the plan order becomes the actual production order (cf. the key WORKORDER_NUMBER). Normally, plan order and production order numbers are identical; however, the system provides the possibility of assigning different numbers.	☐ String
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
SORT_FLAG	Sorting mode for the output [0; 1; 2; 3]: 0 = ascending according to APS_PRODUCTION_START_DATE 1 = ascending according to WORKORDER_NUMBER 2 = ascending according to PLANNING_WORKORDER_NUMBER 3 = ascending according to CUSTOMER_WORKORDER_NUMBER Advice: If this key is not passed along, the [0] value is used by default. If a different value is entered, the work orders are output unsorted!	☐ int

Key	Explanation	IMSApi data type
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: Multiple state values may be passed in the input (separated by ",")! Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific) Special case: If [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 4-27 "Key Value Pair": *apsGetWorkordersFilters*

Definition of the output parameters for the planned work orders

Key	Explanation		IMSApi data type
APS_END_DATE	The initial production end time (delivery date) planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: The point in time is set when the work order is started (cf. the PLANNED_SHIP_DATE key)!	☐	DateMs
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	☐	int
APS_RESULT	APS planning result [1; 4; 8; 16]: 1 = not planned 4 = late 8 = critical 16 = OK	☐	int
APS_START_DATE	The initial production start time planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The point in time is set when the work order is started (cf. the PLANNED_START_DATE key)!	☐	DateMs
APS_STATE	APS planning state [1; 2; 4; 8; 32]: 1 = unplanned 2 = partially planned 4 = planned 8 = started 32 = end	☐	String
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CLUSTER_DESC	Description of the work order cluster	☐	String
CLUSTER_ID	Internal identification number of the work order cluster	☐	long
CLUSTER_NAME	Unique name for the work order cluster	☐	String

Key	Explanation		IMSApi data type
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐	String
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumerical additional information 1	☐	String
INFO_TXT_2	Alphanumerical additional information 2	☐	String
INFO_TXT_3	Alphanumerical additional information 3	☐	String
INFO_TXT_4	Alphanumerical additional information 4	☐	String
INFO_TXT_5	Alphanumerical additional information 5	☐	String
INFO_TXT_6	Alphanumerical additional information 6	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PLANNED_SHIP_DATE	Planned delivery date for the work order according to the last planning run (cf. the <code>APS_END_DATE</code> key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production) according to the last planning run (cf. the <code>APS_START_DATE</code> key); the total time is specified in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
PLANNING_WOR-KORDER_NUMBER	Number of the plan order (only order state planned)	☐	String
PRIORITY_CODE	APS service - unique designator for the work order priority	☐	String
PRIORITY_NAME	APS service - description for the work order priority	☐	String
PRIORITY_VALUE	Value for the work order priority; the greater the value, the higher the prioritization	☐	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
PROCESS_VERSION	Process version from the iTAC system	☐	int
PRODUCTION_CONTROL-LER	Production controller; this is a customer-specific grouping that is set up in the part master	☐	String
PRODUCTION_CONTROL-LER_DESC	Description of the production controller	☐	String

Key	Explanation		IMSApi data type
QUANTITY_WOR-KORDER_PASS	Number of serial numbers with the "PASS" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WOR-KORDER_SCRAP	Number of serial numbers with the "SCRAP" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WOR-KORDER_TOTAL	Order quantity (total)	☐	int
WORKORDER_DESC	Order description	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐	long
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String
WORKPLAN_VERS	Work plan version from the iTAC system	☐	int
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐	String

Tab. 4-28 "Key Value Pair": *apsGetWorkordersResultKeys*

4.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
apsGetWorkorders-Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkordersFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": apsGetWorkordersFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
apsGetWorkordersResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": apsGetWorkordersResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) apsGetWorkordersResultValues !

Tab. 4-29 Parameter (in) »apsGetWorkorders«

Parameter (out)

Parameter name	Explanation
apsGetWorkordersResultValues	Depending on parameter (in) apsGetWorkordersResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) apsGetWorkordersResultKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 4-30 Parameter (out) »apsGetWorkorders«

4.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2108	Failure	No APS license present for the station

5

Group "ATTRIB" (attributes)

5.1

attribAppendAttributeValues

5.1.1

Function description

This function enables the assignment of attributes to a wide range of data objects. This allows individual information, such as a customer-specific material number and specific manufacturing characteristic, to be assigned to the data objects. Multiple, different attribute codes with the corresponding values can be assigned to a data object (successively); an attribute code may only be used once per data object, however. If an already existing attribute code is again assigned with this function, the original value can be overwritten if necessary (parameter `allowOverWrite`). Each attribute assignment is processed as a unique record in an array.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Note that only the actual value for the (`ATTRIBUTE_VALUE`) attribute is assigned with this function. The (`ATTRIBUTE_CODE`) attribute must already exist in the system; the attribute is created in the ADM module below the "Setup -> Attributes" function or the »`attribCreateAttribute`« [▶▶▶] IMSApi functions or the expanded version »`attribCreateAttributeCode`« [▶▶▶].

Prior checking of the attribute value to establish whether it meets the attribute value conditions, can be performed using the IMSApi function »`attribCheckAttributeValues`« [▶▶▶].

Refer to the following table for which data objects can be assigned attributes on the basis of which entries.

Function matrix *objectType*

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the "I" (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-1 Function matrix *objectType*

Special characteristics for work order attributes (1):

- An attribute code may only be used multiple times for a work order if the ATTRIBUTE_OBJECT_KEY key is unique.
- If, in addition to the work order, a part number is passed – this part number must be contained in the order BOM – a reference is established between the work order attribute and this component. This can be used e.g. to assign a concrete IC from the BOM to the "software version" work order attribute with characteristic "V1.2" as attribute value. This IC can also be queried again at a later time.

Special characteristics for BOM component attributes (14):

- If there are multiple alternative BOM components in a BOM at a particular mounting place, the attribute is always assigned to the regular components (according to the BOM module, the "Alt. level" is 0).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribAppendAttributeValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in DateMs bookDate
in int allowOverWrite
in String[] attributeUploadKeys
in String[] attributeUploadValues
out String[] attributeResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) attributeUploadKeys, attributeUploadValues and (out) attributeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Serial number attributes (objectType = 0)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String

Key	Explanation	IMSApi data type
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-2 "Key Value Pair" (serial number attributes): attributeUploadKeys

Work order attributes (objectType = 1)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String

Key	Explanation		IMSApi data type
ATTRIBUTE_OBJECT_KEY	Key term for the attribute Advice: If this parameter is used, multiple attribute values (ATTRIBUTE_VALUE) can be assigned to an attribute (ATTRIBUTE_CODE); for this purpose, the ATTRIBUTE_OBJECT_KEY must be unique! Special case: If an empty string is passed here and the PART_NUMBER parameter used, the passed part number is automatically used as ATTRIBUTE_OBJECT_KEY!	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒	String
ATTRIBUTE_VALUE_DESC	Description of the attribute value (optional) Advice: This parameter is optional and enables the amendment of the attribute value with a description!	☐	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
PART_NUMBER	Part number in the iTAC system Advice: This parameter is optional and enables the assignment of work order attributes with a part reference; the part number must be contained in the order BOM. If no part reference is to be established, an empty string or the value [-1] must be passed here!	☐	String

Tab. 5-3 "Key Value Pair" (work order attributes): attributeUploadKeys

Container attributes (objectType = 2)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-4 "Key Value Pair" (container attributes): attributeUploadKeys

Site attributes (objectType = 3)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-5 "Key Value Pair" (site attributes): attributeUploadKeys

Line attributes (objectType = 4)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-6 "Key Value Pair" (line attributes): attributeUploadKeys

Cell attributes (objectType = 5)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-7 "Key Value Pair" (cell attributes): attributeUploadKeys

Machine group attributes (objectType = 6)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-8 "Key Value Pair" (machine group attributes): attributeUploadKeys

Station attributes (objectType = 7)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-9 "Key Value Pair" (station attributes): attributeUploadKeys

ERP group attributes (objectType = 8)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-10 "Key Value Pair" (ERP group attributes): attributeUploadKeys

Part group attributes (objectType = 9)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-11 "Key Value Pair" (part group attributes): attributeUploadKeys

Part attributes (objectType = 10)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-12 "Key Value Pair" (part attributes): attributeUploadKeys

Work plan attributes (objectType = 11)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-13 "Key Value Pair" (work plan attributes): attributeUploadKeys

Work step attributes (objectType = 12)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-14 "Key Value Pair" (work step attributes): attributeUploadKeys

BOM attributes (objectType = 13)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-15 "Key Value Pair" (BOM attributes): attributeUploadKeys

BOM component attributes (objectType = 14)

Advice: If there are multiple alternative BOM components in a BOM at a particular mounting place, the attribute is always assigned to the regular components (according to the BOM module, the "Alt. level" is 0)!

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-16 "Key Value Pair" (BOM component attributes): attributeUploadKeys

Production means attribute (objectType = 15)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-17 "Key Value Pair" (production means attributes): attributeUploadKeys

User attributes (objectType = 16)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-18 "Key Value Pair" (user attributes): attributeUploadKeys

Order-based BOM component attributes (objectType = 17)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-19 "Key Value Pair" (order-based BOM component attributes): attributeUploadKeys

Order attributes (objectType = 18)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int

Tab. 5-20 "Key Value Pair" (order attributes): attributeUploadKeys

Receiving position attribute (objectType = 19)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-21 "Key Value Pair" (receiving position attribute): attributeUploadKeys

Storage place (objectType = 20)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-22 "Key Value Pair" (storage place attributes): attributeUploadKeys

Resource (objectType = 21)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: A concrete value must be passed for each ATTRIBUTE_CODE; the passing of an empty string results in an interface error!	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-23 "Key Value Pair" (resource attributes): attributeUploadKeys

5.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
objectNumber	Depending on the attribute category (see <code>objectType</code>), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the <code>objectDetail</code> parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
objectDetail	Depending on the attribute category (see <code>objectType</code>) and the specified data object (see <code>objectNumber</code>), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "objectType" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
bookDate	Booking time for the attribute value assignment (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Parameter name	Explanation
allowOverWrite	Determines whether an existing attribute may be overwritten [0; 1; 2]: 0 = Attribute data CANNOT be overwritten (for further information, see the error codes -910, -920 and -930) 1 = Attribute data can be overwritten (a history entry is created) 2 = Attribute data can be overwritten (NO history entry is created)
attributeUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>attributeUploadValues</code> parameter. A description of the possible parameters (dependent on <code>objectType</code>) can be found above in the corresponding table for attribute category "Key Value Pair" (xxx): attributeUploadKeys . Advice: Only the parameters passed here are processed by the function!
attributeUploadValues	Values for the parameters selected by means of <code>attributeUploadKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>attributeUploadKeys</code> ! Refer to the corresponding table above for attribute category "Key Value Pair" (xxx): attributeUploadKeys to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-24 Parameter (in) »attribAppendAttributeValues«

Parameter (out)

Parameter name	Explanation
attributeResultValues	The corresponding data is output here only in the event of an error – depending on the <code>attributeUploadKeys</code> and <code>attributeUploadValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>attributeUploadKeys</code> !

Tab. 5-25 Parameter (out) »attribAppendAttributeValues«

5.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
903	Warning	Attribute validation -> Attribute class condition not fulfilled
902	Warning	Attribute validation -> Attribute value condition not fulfilled
901	Warning	Attribute validation -> Type is invalid
7	Warning	At least one record of the array has been processed with a warning
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found

-121	Failure	Part not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-503	Failure	Container not found
-574	Failure	Storage location not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-905	Failure	Invalid value for "objectType"
-906	Failure	The attribute value in combination with "attributeValueKey" is not unique
-910	Failure	Attribute code is already assigned to the serial number
-920	Failure	Attribute code is already assigned to the work order
-930	Failure	Attribute code is already assigned to the container
-935	Failure	Attribute code is already assigned
-950	Failure	Attribute validation -> Attribute class not found
-951	Failure	Attribute validation -> Attribute class not active
-952	Failure	Attribute validation -> Attribute not part of attribute class
-954	Failure	Attribute value cannot be modified
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2136	Failure	Resource not found

5.2

attribCheckAttributeValues

5.2.1

Function description

This function is used to check whether the passed attribute value fulfills the attribute value conditions. Attribute value conditions refer to test criteria which are to be taken into consideration when an attribute value is entered. It is only checked whether adoption of the attribute value is possible. If the attribute value is indeed to be assigned, the IMSApi function »attribAppendAttributeValues« [▶▶▶] must be executed.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Note that only the actual value for the (ATTRIBUTE_VALUE) attribute is checked with this function. The (ATTRIBUTE_CODE) attribute must already exist in the system; the attribute is created in the ADM module below the "Setup -> Attributes" function or the »attribCreateAttribute« [▶▶▶] IMSApi functions or the expanded version »attribCreateAttributeCode« [▶▶▶].

Refer to the following table for which data objects attributes can be checked on the basis of which entries.

Function matrix ob-
jectType

In the following function matrix, the dependencies between the parameters objectType or the keys OBJECT_TYPE (attribute category), objectNumber or OBJECT_NUMBER (data object number) and objectDetail or OBJECT_DETAIL (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the objectType parameter must be passed with "0". With the objectNumber parameter, the desired serial number is passed; with the objectDetail parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Orders	18	Order designator Advice: <i>The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!</i>	Order position
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-26 Function matrix *objectType***Special characteristics for work order attributes (1):**

- An attribute code may only be used multiple times for a work order if the ATTRIBUTE_OBJECT_KEY key is unique.

Special characteristics for BOM component attributes (14):

- If there are multiple alternative BOM components in a BOM at a particular mounting place, the attribute assignment is always checked against the regular component (according to the BOM module, the "Alt. level" is 0).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribCheckAttributeValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in DateMs bookDate
in int allowOverWrite
in String[] attributeUploadKeys
in String[] attributeUploadValues
out String[] attributeResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) attributeUploadKeys, attributeUploadValues and (out) attributeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Serial number attributes (objectType = 0)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	int

Tab. 5-27 "Key Value Pair" (serial number attributes): *attributeUploadKeys*

Work order attributes (objectType = 1)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_OBJECT_KEY	Key term for the attribute Advice: If this parameter is used, multiple attribute values (ATTRIBUTE_VALUE) can be assigned to an attribute (ATTRIBUTE_CODE); for this purpose, the ATTRIBUTE_OBJECT_KEY must be unique! Special case: If an empty string is passed here and the PART_NUMBER parameter used, the passed part number is automatically used as ATTRIBUTE_OBJECT_KEY!	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_DESC	Description of the attribute value (optional) Advice: This parameter is optional and enables the amendment of the attribute value with a description!	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	int

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PART_NUMBER	Part number in the iTAC system Advice: This parameter is optional and enables the assignment of work order attributes with a part reference; the part number must be contained in the order BOM. If no part reference is to be established, an empty string or the value [-1] must be passed here!	☐ String

Tab. 5-28 "Key Value Pair" (work order attributes): `attributeUploadKeys`

Container attributes (objectType = 2)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-29 "Key Value Pair" (container attributes): attributeUploadKeys

Site attributes (objectType = 3)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-30 "Key Value Pair" (site attributes): attributeUploadKeys

Line attributes (objectType = 4)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-31 "Key Value Pair" (line attributes): attributeUploadKeys

Cell attributes (objectType = 5)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int

Tab. 5-32 "Key Value Pair" (cell attributes): attributeUploadKeys

Machine group attributes (objectType = 6)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-33 "Key Value Pair" (machine group attributes): attributeUploadKeys

Station attributes (objectType = 7)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-34 "Key Value Pair" (station attributes): attributeUploadKeys

ERP group attributes (objectType = 8)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-35 "Key Value Pair" (ERP group attributes): attributeUploadKeys

Part group attributes (objectType = 9)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-36 "Key Value Pair" (part group attributes): attributeUploadKeys

Part attributes (objectType = 10)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-37 "Key Value Pair" (part attributes): attributeUploadKeys

Work plan attributes (objectType = 11)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int

Tab. 5-38 "Key Value Pair" (work plan attributes): attributeUploadKeys

Work step attributes (objectType = 12)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-39 "Key Value Pair" (work step attributes): attributeUploadKeys

BOM attributes (objectType = 13)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-40 "Key Value Pair" (BOM attributes): attributeUploadKeys

BOM component attributes (objectType = 14)

Advice: If there are multiple alternative BOM components in a BOM at a particular mounting place, the attribute is always assigned to the regular components (according to the BOM module, the "Alt. level" is 0)!

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-41 "Key Value Pair" (BOM component attributes): *attributeUploadKeys*

Production means attribute (objectType = 15)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-42 "Key Value Pair" (production means attributes): attributeUploadKeys

User attributes (objectType = 16)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-43 "Key Value Pair" (user attributes): attributeUploadKeys

Order-based BOM component attributes (objectType = 17)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-44 "Key Value Pair" (order-based BOM component attributes): attributeUploadKeys

Order attributes (objectType = 18)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-45 "Key Value Pair" (order attributes): attributeUploadKeys

Receiving position attribute (objectType = 19)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int

Tab. 5-46 "Key Value Pair" (receiving position attribute): attributeUploadKeys

Storage place (objectType = 20)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-47 "Key Value Pair" (storage place attributes): attributeUploadKeys

Resource (objectType = 21)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code in the case of an unsuccessful check; at present, this can be the warnings 901, 902 or 903. Irrespective of the error code, the processing procedure is NOT terminated; the record is processed – possibly with restrictions. Advice: As soon as the test routine for a record (array element) encounters a problem, one of the corresponding technical error codes previously mentioned is output here; in this case, the function itself outputs a general warning (see error code 7)!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-48 "Key Value Pair" (resource attributes): attributeUploadKeys

5.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
objectNumber	Depending on the attribute category (see <code>objectType</code>), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the <code>objectDetail</code> parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
objectDetail	Depending on the attribute category (see <code>objectType</code>) and the specified data object (see <code>objectNumber</code>), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "objectType" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
bookDate	Planned booking time; required for the attribute value check; specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 This allows time stamps from the past to be specified if necessary. Special case: [-1] passes the current date and the current time!

Parameter name	Explanation
allowOverWrite	Determines whether an existing attribute may be overwritten [0; 1; 2]: 0 = Attribute data CANNOT be overwritten (for further information, see the error codes -910, -920 and -930) 1 = Attribute data can be overwritten (a history entry is created) 2 = Attribute data can be overwritten (NO history entry is created)
attributeUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>attributeUploadValues</code> parameter. A description of the possible parameters (dependent on <code>objectType</code>) can be found above in the corresponding table for attribute category "Key Value Pair" (xxx): attributeUploadKeys . Advice: Only the parameters passed here are processed by the function!
attributeUploadValues	Values for the parameters selected by means of <code>attributeUploadKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>attributeUploadKeys</code> ! Refer to the corresponding table above for attribute category "Key Value Pair" (xxx): attributeUploadKeys to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-49 Parameter (in) »attribCheckAttributeValues«

Parameter (out)

Parameter name	Explanation
attributeResultValues	The corresponding data is output here only in the event of an error – depending on the <code>attributeUploadKeys</code> and <code>attributeUploadValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>attributeUploadKeys</code> !

Tab. 5-50 Parameter (out) »attribCheckAttributeValues«

5.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
903	Warning	Attribute validation -> Attribute class condition not fulfilled
902	Warning	Attribute validation -> Attribute value condition not fulfilled
901	Warning	Attribute validation -> Type is invalid
7	Warning	At least one record of the array has been processed with a warning
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found
-122	Failure	Part group not found

-125	Failure	ERP group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-503	Failure	Container not found
-574	Failure	Storage location not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-905	Failure	Invalid value for "objectType"
-906	Failure	The attribute value in combination with "attributeValueKey" is not unique
-910	Failure	Attribute code is already assigned to the serial number
-920	Failure	Attribute code is already assigned to the work order
-930	Failure	Attribute code is already assigned to the container
-950	Failure	Attribute validation -> Attribute class not found
-951	Failure	Attribute validation -> Attribute class not active
-952	Failure	Attribute validation -> Attribute not part of attribute class
-954	Failure	Attribute value cannot be modified
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2136	Failure	Resource not found

5.3

attribCreateAttribute

5.3.1

Function description

This function enables the creation of attributes for a wide range of data objects (see list below).



Advice

Alternatively, attributes can also be created in the ADM module under the "Setup -> Attributes" function. Attribute values are assigned using the IMSApi function »[attribAppendAttributeValues](#)« [▶▶▶]!

This function should NO LONGER be used for new installations because the processing possibilities were extended with release 8.50.01 and are provided by another API function (see »[attribCreateAttributeCode](#)« [▶▶▶])!

New attributes can be created in the system for the following data objects:

- Serial numbers (objectType = 0)
- Work orders (objectType = 1)
- Containers / Lots (objectType = 2)
- Sites (objectType = 3)
- Lines (objectType = 4)
- Cells (objectType = 5)
- Machine groups (objectType = 6)
- Stations (objectType = 7)
- ERP groups (objectType = 8)
- Part groups (objectType = 9)
- Parts (objectType = 10)
- Work plans (objectType = 11)
- Work steps (objectType = 12)
- BOMs (objectType = 13)
- BOM components (objectType = 14)
- Production means (objectType = 15)
- Users (objectType = 16)
- Order-based BOM components (objectType = 17)
- Orders (objectType = 18)
- Receiving position (objectType = 19)
- Storage places (objectType = 20)
- Resources (objectType = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribCreateAttribute
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String attributeCode
in String attributeDesc
in String uniquenessType
    
```

5.3.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
attributeCode	Attribute code
attributeDesc	Attribute designator
uniquenessType	Determines whether the attribute values must be unique for this attribute [N; A]: N = Attribute values do not need to be unique ("non-unique") A = Attribute values must be "unique" If a different value is passed, key type "unknown value" is stored for the attribute. Advice: The check for value uniqueness is performed within a site!

Tab. 5-51 Parameter (in) »attribCreateAttribute«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

5.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-196	Failure	A parameter is invalid
-905	Failure	Invalid value for "objectType"
-907	Failure	Attribute already exists
-2105	Failure	No TR or PM license present for the station

5.4 attribCreateAttributeClass

5.4.1 Function description

This function creates the header structure for a new attribute class in the iTAC system; the key `OBJECT_TYPE` is used to determine the attribute categories for which the new class is to apply. If the passed attribute class name already exists in the iTAC system, a new version of the attribute class is created automatically. Adaptation of the attribute class state, assignment of the desired attributes as well as definition of class-specific attribute value conditions are performed using separate API functions (see »[attribEditAttributeClassState«](#) [▶▶▶], »[attribManageAttributeClassAttribute«](#) [▶▶▶] and »[attribManageAttributeClassCondition«](#) [▶▶▶]).

Attribute classes can be created for the following data objects:

- Serial numbers (`OBJECT_TYPE = 0`)
- Work orders (`OBJECT_TYPE = 1`)
- Containers / Lots (`OBJECT_TYPE = 2`)
- Sites (`OBJECT_TYPE = 3`)
- Lines (`OBJECT_TYPE = 4`)
- Cells (`OBJECT_TYPE = 5`)
- Machine groups (`OBJECT_TYPE = 6`)
- Stations (`OBJECT_TYPE = 7`)
- ERP groups (`OBJECT_TYPE = 8`)
- Part groups (`OBJECT_TYPE = 9`)
- Parts (`OBJECT_TYPE = 10`)
- Work plans (`OBJECT_TYPE = 11`)
- Work steps (`OBJECT_TYPE = 12`)
- BOMs (`OBJECT_TYPE = 13`)
- BOM components (`OBJECT_TYPE = 14`)
- Production means (`OBJECT_TYPE = 15`)
- Users (`OBJECT_TYPE = 16`)
- Order-based BOM components (`OBJECT_TYPE = 17`)
- Orders (`OBJECT_TYPE = 18`)
- Receiving position (`OBJECT_TYPE = 19`)
- Storage places (`OBJECT_TYPE = 20`)
- Resources (`OBJECT_TYPE = 21`)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribCreateAttributeClass
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeClassCreateFilter
    String key
    String value
in String[] classCreateKeys
in String[] classCreateValues
in String[] classCreateResultKeys
out String[] classCreateResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeClassCreateFilter`, `classCreateKeys`, `classCreateValues` **attributed** `classCreateResultKeys` as well as (out) `classCreateResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for attribute class to be created (header data)

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_-DESC	Description of the attribute class	☐	String
ATTRIBUTE_-CLASS_NAME	Name of the attribute class Advice: If the name already exists in the iTAC system, a new version is created automatically!	☒	String
ATTRIBUTE_-CLASS_VERSION_ERP	Version of the attribute class in the ERP system	☐	String

Tab. 5-52 "Key Value Pair": `attributeClassCreateFilter`

Definition of attribute categories applicable for attribute class

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category for which the attribute class applies [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	int

Tab. 5-53 "Key Value Pair": *classCreateKeys*

Output values for created attribute class

Key	Explanation	IMSApi data type
ATTRIBUTE - CLASS_VERSION	Version of the attribute class in the iTAC system Advice: The version is determined automatically by the iTAC system!	int

Tab. 5-54 "Key Value Pair": *classCreateResultKeys*

5.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeClass-CreateFilter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeClassCreateFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeClassCreateFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
classCreateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>classCreateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": classCreateKeys table. Advice: Only the parameters passed here are processed by the function!
classCreateValues	Values for the parameters selected by means of <code>classCreateKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>classCreateKeys</code> ! Refer to the "Key Value Pair": classCreateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
classCreateResultKeys	List of the parameters that are to be output by the function. A description of the possible parameters can be found above in the "Key Value Pair": classCreateResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>classCreateResultValues</code> !

Tab. 5-55 Parameter (in) »attribCreateAttributeClass«

Parameter (out)

Parameter name	Explanation
classCreateResultValues	Depending on parameter (in) <code>classCreateResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>classCreateResultKeys</code> !

Tab. 5-56 Parameter (out) »attribCreateAttributeClass«

5.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-905	Failure	Invalid value for "objectType"
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.5 attribCreateAttributeCode

5.5.1 Function description

This function creates new attributes in the iTAC system; the key `OBJECT_TYPE` is used to determine the data object for which the new attribute is to apply. The passed attribute code must be unique and must not yet exist in the iTAC system.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! For new installations, it is **essential** to use this API function because the functionality has been extended compared to the »attribCreateAttribute« [▶▶▶] function! Alternatively, attributes can also be created in the ADM module under the "Setup -> Attributes" function. The assignment of attribute values is performed by means of the »attribAppendAttributeValues« [▶▶▶] IMSApi function!*

New attributes can be created for the following data objects:

- Serial numbers (`OBJECT_TYPE = 0`)
- Work orders (`OBJECT_TYPE = 1`)
- Containers / Lots (`OBJECT_TYPE = 2`)
- Sites (`OBJECT_TYPE = 3`)
- Lines (`OBJECT_TYPE = 4`)
- Cells (`OBJECT_TYPE = 5`)
- Machine groups (`OBJECT_TYPE = 6`)
- Stations (`OBJECT_TYPE = 7`)
- ERP groups (`OBJECT_TYPE = 8`)
- Part groups (`OBJECT_TYPE = 9`)
- Parts (`OBJECT_TYPE = 10`)
- Work plans (`OBJECT_TYPE = 11`)
- Work steps (`OBJECT_TYPE = 12`)
- BOMs (`OBJECT_TYPE = 13`)
- BOM components (`OBJECT_TYPE = 14`)
- Production means (`OBJECT_TYPE = 15`)
- Users (`OBJECT_TYPE = 16`)
- Order-based BOM components (`OBJECT_TYPE = 17`)
- Orders (`OBJECT_TYPE = 18`)
- Receiving position (`OBJECT_TYPE = 19`)
- Storage places (`OBJECT_TYPE = 20`)
- Resources (`OBJECT_TYPE = 21`)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribCreateAttributeCode
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] codeKeys
in String[] codeValues
out String[] codeResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `codeKeys` and `codeValues` as well as (out) `codeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code (unique designator)	☒ String
AT-TRIB-UTE_CODE_APS_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE) Advice: If this key is not passed along, the [1] value is used by default!	☐ boolean
ATTRIBUTE_CODE_ERP_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process with the connected ERP system [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE) Advice: If this key is not passed along, the [0] value is used by default!	☐ boolean

Key	Explanation		IMSApi data type
AT-TRIB-UTE_CODE_SCT_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the SCT-Service (Supply Chain Traceability) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE) Advice: If this key is not passed along, the [0] value is used by default!	☐	boolean
AT-TRIBUTE_CODE_UNIQUE	Indicates whether the attribute values must be unique for this attribute [0; 1]: 0 = Attribute values do NOT need to be unique (FALSE) 1 = Attribute values must be unique (TRUE) Advice: The check for value uniqueness is performed within a site!	☐	boolean
ATTRIBUTE_DESC	Attribute designator	☐	String
COPY_TO_CATALOG	Determines whether the attribute is additionally created in the attribute catalog [0; 1]: 0 = Attribute is NOT created in the catalog (FALSE) 1 = Attribute is created in the catalog (TRUE) Advice: If this key is not passed along, the [0] value is used by default!	☐	boolean
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! The data type UNDEF was used only for data records prior to release 8.50.01; it CANNOT be specified here!	☒	String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service Advice: If this key is not passed along, the [ANY] value is used by default!	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐	int

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	int

Tab. 5-57 "Key Value Pair": *codeKeys*

5.5.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
codeKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>codeValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": codeKeys table. Advice: Only the parameters passed here are processed by the function!
codeValues	Values for the parameters selected by means of <code>codeKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>codeKeys</code>! Refer to the "Key Value Pair": codeKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-58 Parameter (in) »attribCreateAttributeCode«

Parameter (out)

Parameter name	Explanation
<code>codeResultValues</code>	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>codeKeys</code> and <code>codeValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>codeKeys</code> !

Tab. 5-59 Parameter (out) »attribCreateAttributeCode«

5.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-905	Failure	Invalid value for "objectType"
-907	Failure	Attribute already exists
-953	Failure	Attribute -> "Edited by" - Designator is not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.6 attribCreateAttributeCodeByCatalog

5.6.1 Function description

This function enables a regular attribute to be created from an attribute catalog entry; the key `OBJECT_TYPE` is used to determine the data object for which the "copied" attribute is then to apply.

Attributes can be created in the system for the following data objects:

- Serial numbers (`OBJECT_TYPE = 0`)
- Work orders (`OBJECT_TYPE = 1`)
- Containers / Lots (`OBJECT_TYPE = 2`)
- Sites (`OBJECT_TYPE = 3`)
- Lines (`OBJECT_TYPE = 4`)
- Cells (`OBJECT_TYPE = 5`)
- Machine groups (`OBJECT_TYPE = 6`)
- Stations (`OBJECT_TYPE = 7`)
- ERP groups (`OBJECT_TYPE = 8`)
- Part groups (`OBJECT_TYPE = 9`)
- Parts (`OBJECT_TYPE = 10`)
- Work plans (`OBJECT_TYPE = 11`)
- Work steps (`OBJECT_TYPE = 12`)
- BOMs (`OBJECT_TYPE = 13`)
- BOM components (`OBJECT_TYPE = 14`)
- Production means (`OBJECT_TYPE = 15`)
- Users (`OBJECT_TYPE = 16`)
- Order-based BOM components (`OBJECT_TYPE = 17`)
- Orders (`OBJECT_TYPE = 18`)
- Receiving position (`OBJECT_TYPE = 19`)
- Storage places (`OBJECT_TYPE = 20`)
- Resources (`OBJECT_TYPE = 21`)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribCreateAttributeCodeByCatalog
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeCodeFilter
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `attributeCodeFilter`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code (unique designator)	☒ String
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☒ int

Tab. 5-60 "Key Value Pair": `attributeCodeFilter`

5.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeCodeFilter	Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeCodeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeCodeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-61 Parameter (in) »attribCreateAttributeCodeByCatalog«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

5.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-905	Failure	Invalid value for "objectType"
-907	Failure	Attribute already exists
-942	Failure	Attribute class not found
-946	Failure	Attribute code catalog not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.7 attribCreateAttributeCondition

5.7.1 Function description

This function creates new attribute value conditions in the iTAC system. Specifications regarding the data type, MIN/MAX limits and/or fixed values (permitted values) can be assigned here. If the name of the attribute value condition already exists, a new version is created automatically. Attribute value conditions are assigned to attributes using the API function »attribManageAttributeCondition« [▶▶▶].



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR CODE`). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribCreateAttributeCondition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] conditionKeys
in String[] conditionValues
out String[] conditionResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `conditionKeys` and `conditionValues` as well as (out) `conditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ATTRIBUTE_CONDITION_DESC	Description of the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition Advice: If the name already exists in the iTAC system, a new version is created automatically (+1)!	☐ String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition Advice: If more values are to be passed, they must be separated by commas (e.g. 22,33,44)!	☐ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition Advice: No entry is required for this key; the key is used to output the version number of the newly created attribute value condition; it is NOT possible to specify a version number!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input!	☒ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ String

Tab. 5-62 "Key Value Pair": conditionKeys

5.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
conditionKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>conditionValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": conditionKeys table. Advice: Only the parameters passed here are processed by the function!
conditionValues	Values for the parameters selected using <code>codeKeys</code>; no entry is necessary for the parameters <code>ATTRIBUTE_CONDITION_VER-SION</code> and <code>ERROR_CODE</code> because they are filled automatically by the system. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>conditionKeys</code>! Refer to the "Key Value Pair": conditionKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-63 Parameter (in) »attribCreateAttributeCondition«

Parameter (out)

Parameter name	Explanation
conditionResult-Values	Depending on parameter (in) <code>conditionKeys</code> and <code>conditionValues</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>conditionKeys</code>!

Tab. 5-64 Parameter (out) »attribCreateAttributeCondition«

5.7.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-940	Failure	Attribute value condition -> Type is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.8 attribEditAttributeClassState

5.8.1 Function description

This function is used to change the state of an attribute class existing in the system. This allows, for example, newly created attribute classes (which at first are in the "initialized" state) to be released and activated.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribEditAttributeClassState
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeClassData
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) attributeClassData).



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation	IMSApi data type
ACTIVE	"Attribute class" activation state [0; 1]: 0 = NOT active 1 = Active Advice: If the key is not passed, the current state remains unchanged!	☐ int
ATTRIBUTE_ - CLASS_NAME	Attribute class name Advice: This value is evaluated together with the passed version number (see key ATTRIBUTE_CLASS_VERSION); if there is not a match, the function terminates with a failure!	☒ String

Key	Explanation	IMSApi data type
ATTRIBUTE_- CLASS_STATE	"Attribute class" state [D; I; L; R]: D = deleted I = initial L = invalid R = released The following state transitions are allowed: - from I -> D or R - from R -> D or L - from L -> R or D or I Advice: The state [D] (deleted) cannot be changed. If the key is not passed, the current state remains unchanged!	☐ String
ATTRIBUTE_- CLASS_VERSION	Version of the attribute class in the iTAC system Advice: This value is evaluated together with the passed attribute class name (see key ATTRIBUTE_CLASS_NAME); if there is not a match, the function terminates with a failure!	☒ int

Tab. 5-65 "Key Value Pair": attributeClassData

5.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeClassData	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeClassData table.
value	Value list for the parameters passed by means of key.
	Advice: Refer to the "Key Value Pair": attributeClassData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-66 Parameter (in) »attribEditAttributeClassState«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

5.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-942	Failure	Attribute class not found
-943	Failure	Attribute class not released
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.9

attribGetAllAttributeValue

5.9.1

Function description

This function enables all attribute values and the associated attribute information to be queried; a filter can be set here so that only certain attributes (ATTRIBUTE_CODE), attribute categories (OBJECT_TYPE) or attribute values (ATTRIBUTE_VALUE) are queried. In combination with the keys MAX_BOOK_DATE and MIN_BOOK_DATE, the result can be restricted to a certain period under review. All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

The table below shows which attribute categories are output with which data object number and, where applicable, with which data object details.

Function matrix objectType

In the following function matrix, the dependencies between the parameters objectType or the keys OBJECT_TYPE (attribute category), objectNumber or OBJECT_NUMBER (data object number) and objectDetail or OBJECT_DETAIL (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the objectType parameter must be passed with "0". With the objectNumber parameter, the desired serial number is passed; with the objectDetail parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-67 Function matrix objectType

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAllAttributeValue
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeValueSearchFilter
    String key
    String value
in String[] attributeValueSearchKeys
in String[] attributeValueSearchValues
in String[] attributeValueResultKeys
out String[] attributeValueSearchResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) attributeValueSearchFilter, attributeValueSearchKeys, attributeValueSearchValues and attributeValueResultKeys as well as (out) attributeValueSearchResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Restriction of result list

Key	Explanation	IMSApi data type
MAX_BOOK_DATE	Time stamp for the end of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, there is no restriction!	☐ DateMs
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Key	Explanation	IMSApi data type
MIN_BOOK_DATE	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>If the key is not passed, there is no restriction!</i>	☐ DateMs

Tab. 5-68 "Key Value Pair": *attributeValueSearchFilter*

Searching for specific attribute values

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
ATTRIBUTE_VALUE	Value of the attribute Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
ATTRIBUTE_VAL- UE_VALIDATION_FAIL- URE_LIST	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the following validation errors: 901 = Data type is invalid 902 = Attribute value condition not fulfilled 903 = Attribute class condition not fulfilled 904 = Attribute class not found 905 = Attribute class not active 906 = Attribute not part of attribute class Advice: <i>Using this key it is possible to ascertain the attribute data for which one of the above-listed validation errors is stored!</i>	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Advice: If the key is not passed, there is no restriction; multiple entries are NOT possible!	int

Tab. 5-69 "Key Value Pair": attributeValueSearchKeys

Definition of the output parameters for the found attribute values

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String

Key	Explanation	IMSApi data type
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« (▶▶▶)) by the check routine and stored in the record; at present, this can be the following validation errors: 901 = Data type is invalid 902 = Attribute value condition not fulfilled 903 = Attribute class condition not fulfilled 904 = Attribute class not found 905 = Attribute class not active 906 = Attribute not part of attribute class	☐ int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ String
BOM_VERSION	BOM version from the iTAC system Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ int
BOM_VERSION_ERP	BOM version in the ERP system Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Key	Explanation	IMSApi data type
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
OBJECT_DETAIL	Depending on the attribute category (see OBJECT_TYPE) and the data object (see OBJECT_NUMBER), the data-object detail specification is – where relevant – output here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing of the attribute category requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!	☐ String
OBJECT_NUMBER	Depending on the attribute category (see OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is output here. Advice: Note that, for certain attribute categories, the OBJECT_DETAIL parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!	☐ String
OBJECT_TYPE	Attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☐ int
PART_DESC	Part description in the iTAC system Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ String
PART_NUMBER	Part number in the iTAC system Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 17 = Order-based BOM component attribute	☐ String

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 12 = Work step attribute	int
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project! This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 11 = Work plan attribute 12 = Work step attribute	String
WORKPLAN_VERS	Work plan version from the iTAC system Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 11 = Work plan attribute 12 = Work step attribute	int
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: This key can only be output if the found attribute belongs to one of the attribute categories named here (OBJECT_TYPE): 11 = Work plan attribute 12 = Work step attribute	String

Tab. 5-70 "Key Value Pair": attributeValueResultKeys

5.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeValue-SearchFilter	Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the result list is to be restricted; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeValueSearchFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeValueSearchFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attributeValue-SearchKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>attributeValueSearchValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeValueSearchKeys table.
attributeValue-SearchValues	Value list for the parameters passed by means of <code>attributeValueSearchKeys</code> . Advice: Refer to the "Key Value Pair": attributeValueSearchKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attributeValueResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": attributeValueResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>attributeValueSearchResultValues</code> !

Tab. 5-71 Parameter (in) »attribGetAllAttributeValue«

Parameter (out)

Parameter name	Explanation
attributeValue-SearchResultValues	Depending on parameter (in) <code>attributeValueResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>attributeValueResultKeys</code> !

Tab. 5-72 Parameter (out) »attribGetAllAttributeValue«

5.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-905	Failure	Invalid value for "objectType"
-948	Failure	Attribute -> Unknown validation error
-953	Failure	Attribute -> "Edited by" - Designator is not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

5.10

attribGetAttributeClass

5.10.1

Function description

This function enables attribute class information to be queried; not only the header structures are output here but, if required, also information regarding the assigned attributes and the attribute value conditions. The query can be filtered according to a wide range of different criteria (e.g. attribute class name, the attribute class state and also the attribute class for a specific attribute). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

**Advice**

*The result tables described below contain the key `ATTRIBUTE_CLASS_ID`. This key **MUST** be activated even if it is not marked as a mandatory field; for technical reasons, result output is only possible after the key is activated!*

A query can be performed for the following data objects:

- Serial numbers (OBJECT_TYPE = 0)
- Work orders (OBJECT_TYPE = 1)
- Containers / Lots (OBJECT_TYPE = 2)
- Sites (OBJECT_TYPE = 3)
- Lines (OBJECT_TYPE = 4)
- Cells (OBJECT_TYPE = 5)
- Machine groups (OBJECT_TYPE = 6)
- Stations (OBJECT_TYPE = 7)
- ERP groups (OBJECT_TYPE = 8)
- Part groups (OBJECT_TYPE = 9)
- Parts (OBJECT_TYPE = 10)
- Work plans (OBJECT_TYPE = 11)
- Work steps (OBJECT_TYPE = 12)
- BOMs (OBJECT_TYPE = 13)
- BOM components (OBJECT_TYPE = 14)
- Production means (OBJECT_TYPE = 15)
- Users (OBJECT_TYPE = 16)
- Order-based BOM components (OBJECT_TYPE = 17)
- Orders (OBJECT_TYPE = 18)
- Receiving position (OBJECT_TYPE = 19)
- Storage places (OBJECT_TYPE = 20)
- Resources (OBJECT_TYPE = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeClass
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeClassFilter
    String key
    String value
in String[] classResultKeys
out String[] classResultValues
in String[] classTypeResultKeys
out String[] classTypeResultValues
in String[] classItemResultKeys
out String[] classItemResultValues
in String[] classItemConditionResultKeys
out String[] classItemConditionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see the parameters (in) attributeClassFilter, classResultKeys, classTypeResultKeys, classItemResultKeys and classItemConditionResultKeys as well as (out) classResultValues, classTypeResultValues, classItemResultValues and classItemConditionResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific attribute class values

Key	Explanation	IMSApi data type
ACTIVE	"Attribute class" activation state [0; 1]: 0 = NOT active 1 = Active Advice: If the key is not passed, there is no restriction!	☐ int
ATTRIBUTE_ - CLASS_NAME	Attribute class name Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
ATTRIBUTE_ - CLASS_STATE	"Attribute class" state [D; I; L; R]: D = deleted I = initial L = invalid R = released Advice: If the key is not passed, the value [R] is used by default; note the interaction with the key ATTRIBUTE_CLASS_VERSION! Special case: If the value [-1] is passed, this filter function is deactivated; there is no restriction!	☐ String
ATTRIBUTE_ - CLASS_VERSION	Version of the attribute class in the iTAC system Advice: If the key is not passed, only released and active records with the highest version number are taken into consideration; note the interaction with the key ATTRIBUTE_CLASS_STATE! Special case: If the value [-1] is passed, this filter function is deactivated; there is no restriction!	☐ int
ATTRIBUTE_CODE	Attribute code (unique designator) Advice: If the key is not passed or passed with [-1], there is no restriction!	☐ String
FUNC_MODE	Function mode "attribute value conditions" [1;2]: 1 = All attribute value conditions 2 = ONLY applied attribute value conditions Background: In mode [1], both the conditions of the contained attribute codes – these also include the original conditions – and the conditions of the attribute class are output. In mode [2], only those conditions that are actually applied are output. In this case, attribute value conditions of the attribute class override attribute value conditions of the contained attributes. Advice: If this key is not passed along, the [1] value is used by default!	☐ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Advice: If the key is not passed, there is no restriction; multiple entries are NOT possible!	int

Tab. 5-73 "Key Value Pair": *attributeClassFilter*

Definition of the output parameters for attribute classes -> **header data**

Key	Explanation	IMSApi data type
ACTIVE	"Attribute class" activation state [0; 1]: 0 = NOT active 1 = Active	int
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_ID	Internal identification number of the attribute class Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)! This key MUST be activated even if it is not marked as a mandatory field; for technical reasons, result output is only possible after the key is activated.	long
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_STATE	"Attribute class" state [D; I; L; R]: D = deleted I = initial L = invalid R = released	String
ATTRIBUTE_CLASS_VERSION	Version of the attribute class in the iTAC system	int

Tab. 5-74 "Key Value Pair": *classResultKeys*

Definition of the output parameters for attribute classes -> **attribute categories**

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_ID	Internal identification number of the attribute class Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)! This key MUST be activated even if it is not marked as a mandatory field; for technical reasons, result output is only possible after the key is activated.	☐ long
OBJECT_TYPE	Attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☐ int

Tab. 5-75 "Key Value Pair": classTypeResultKeys

Definition of the output parameters for attribute classes -> **attribute codes**

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_ID	Internal identification number of the attribute class Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)! This key MUST be activated even if it is not marked as a mandatory field; for technical reasons, result output is only possible after the key is activated.	☐ long
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIB-UTE_CODE_APS_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean

Key	Explanation		IMSApi data type
ATTRIBUTE_CODE_ERP_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process with the connected ERP system [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐	boolean
ATTRIBUTE_CODE_SCT_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the SCT-Service (Supply Chain Traceability) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐	boolean
ATTRIBUTE_CODE_UNIQUE	Indicates whether the attribute values must be unique for this attribute [0; 1]: 0 = Attribute values do NOT need to be unique (FALSE) 1 = Attribute values must be unique (TRUE) Advice: The check for value uniqueness is performed within a site!	☐	boolean
ATTRIBUTE_DESC	Attribute designator	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐	String
OBJECT_TYPE	Attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☐	int

Tab. 5-76 "Key Value Pair": *classItemResultKeys*

Definition of the output parameters for attribute classes -> **attribute value conditions**

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_ID	Internal identification number of the attribute class Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)! This key MUST be activated even if it is not marked as a mandatory field; for technical reasons, result output is only possible after the key is activated.	☐ long
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_CONDITION_AT_CLASS_ENABLED	Indicates whether the attribute value condition is active in the attribute class [0; 1]: 0 = Condition is NOT active (FALSE) 1 = Condition is active (TRUE) Advice: All active attribute value conditions within an attribute class are "OR" linked; i.e. the fulfillment of one condition is sufficient for validation!	☐ boolean
ATTRIBUTE_CONDITION_AT_CLASS_LEVEL	Indicates whether the attribute value condition is class-specific [0; 1]: 0 = Condition is NOT class-specific (FALSE) 1 = Condition is class-specific (TRUE) Advice: The value [0] is only output for conditions for which referencing from the attribute code to the attribute class is unchanged (key word "original condition")!	☐ boolean
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition	☐ String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition	☐ int

Key	Explanation	IMSApi data type
OBJECT_TYPE	Attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☐ int

Tab. 5-77 "Key Value Pair": *classItemConditionResultKeys*

5.10.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeClass-Filter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeClassFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeClassFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
classResultKeys	List of the parameters (header data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": classResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>classResultValues</code> !

Parameter name	Explanation
classTypeResultKeys	List of the parameters (attribute categories) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": classTypeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) classTypeResultValues!
classItemResultKeys	List of the parameters (attribute codes) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": classItemResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) classItemResultValues!
classItemConditionResultKeys	List of the parameters (attribute value conditions) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": classItemConditionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) classItemConditionResultValues!

Tab. 5-78 Parameter (in) »attribGetAttributeClass«

Parameter (out)

Parameter name	Explanation
classResultValues	Depending on parameter (in) classResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) classResultKeys!
classTypeResultValues	Depending on parameter (in) classTypeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) classTypeResultKeys!
classItemResultValues	Depending on parameter (in) classItemResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) classItemResultKeys!
classItemConditionResultValues	Depending on parameter (in) classItemConditionResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) classItemConditionResultKeys!

Tab. 5-79 Parameter (out) »attribGetAttributeClass«

5.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-168	Failure	Invalid value for the function mode (control flag)
-174	Failure	Number of records to be output has fallen below the minimum value
-905	Failure	Invalid value for "objectType"
-937	Failure	Attribute class -> State flag is not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

5.11

attribGetAttributeCode

5.11.1

Function description

This function enables the attribute data in the iTAC system to be queried; in this case the attribute codes with the respective properties. In addition to the attribute master data, information relating to the assigned attribute value conditions can also be queried using this function. The query can be filtered according to certain attribute codes and the attribute type. All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

If the attribute values are to be queried for attributes, the API function »[attribGetAttributeValue](#)« [▶▶▶] or »[attribGetAttributeValues](#)« [▶▶▶] must be used depending on the application type.

**Advice**

Special case: If the value [-1] is entered for the attribute category (OBJECT_TYPE), (only) the attributes are read out of the attribute catalog!

A query can be performed for the following data objects:

- Serial numbers (OBJECT_TYPE = 0)
- Work orders (OBJECT_TYPE = 1)
- Containers / Lots (OBJECT_TYPE = 2)
- Sites (OBJECT_TYPE = 3)
- Lines (OBJECT_TYPE = 4)
- Cells (OBJECT_TYPE = 5)
- Machine groups (OBJECT_TYPE = 6)
- Stations (OBJECT_TYPE = 7)
- ERP groups (OBJECT_TYPE = 8)
- Part groups (OBJECT_TYPE = 9)
- Parts (OBJECT_TYPE = 10)
- Work plans (OBJECT_TYPE = 11)
- Work steps (OBJECT_TYPE = 12)
- BOMs (OBJECT_TYPE = 13)
- BOM components (OBJECT_TYPE = 14)
- Production means (OBJECT_TYPE = 15)
- Users (OBJECT_TYPE = 16)
- Order-based BOM components (OBJECT_TYPE = 17)
- Orders (OBJECT_TYPE = 18)
- Receiving position (OBJECT_TYPE = 19)
- Storage places (OBJECT_TYPE = 20)
- Resources (OBJECT_TYPE = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeCode
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeCodeFilter
    String key
    String value
in String[] codeResultKeys
out String[] codeResultValues
in String[] conditionResultKeys
out String[] conditionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) attributeCodeFilter, codeResultKeys and conditionResultKeys as well as (out) codeResultValues and conditionResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific attribute data

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☒ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Special case: If the value [-1] is passed, (only) the attributes are read out from the attribute catalog!	☒ int

Tab. 5-80 "Key Value Pair": *attributeCodeFilter*Definition of the output parameters for attribute data -> **attribute codes**

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☐ String
AT-TRIB-UTE_CODE_APS_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
ATTRIBUTE_CODE_ERP_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process with the connected ERP system [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
AT-TRIB-UTE_CODE_SCT_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the SCT-Service (Supply Chain Traceability) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
AT-TRIB-UTE_CODE_UNIQUE	Indicates whether the attribute values must be unique for this attribute [0; 1]: 0 = Attribute values do NOT need to be unique (FALSE) 1 = Attribute values must be unique (TRUE) Advice: The check for value uniqueness is performed within a site!	☐ boolean
ATTRIBUTE_DESC	Attribute designator	☐ String

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	String

Tab. 5-81 "Key Value Pair": *codeResultKeys*

Definition of the output parameters for attribute data -> **attribute value conditions**

Key	Explanation	IMSApi data type
ATTRIBUTE_CONDITION_DESC	Description of the attribute value condition	String
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition	String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition	String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition	String
ATTRIBUTE_CONDITION_STATE	"Attribute value condition" state [D; I; L; R]: D = deleted I = initial L = invalid R = released	String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition	String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN	String

Tab. 5-82 "Key Value Pair": *conditionResultKeys*

5.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeCodeFilter	Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeCodeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeCodeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
codeResultKeys	List of the parameters (attribute codes) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": codeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>codeResultValues</code> !
conditionResultKeys	List of the parameters (attribute value conditions) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": conditionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>conditionResultValues</code> !

Tab. 5-83 Parameter (in) »attribGetAttributeCode«

Parameter (out)

Parameter name	Explanation
codeResultValues	Depending on parameter (in) <code>codeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>codeResultKeys</code> !
conditionResultValues	Depending on parameter (in) <code>conditionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>conditionResultKeys</code> !

Tab. 5-84 Parameter (out) »attribGetAttributeCode«

5.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-905	Failure	Invalid value for "objectType"
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

5.12 attribGetAttributeCondition

5.12.1 Function description

This function enables the attribute value conditions in the iTAC system to be queried; the query can be filtered according to a wide range of different criteria (e.g. condition name, condition version, state, MIN/MAX limits, etc.). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeCondition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeConditionFilter
    String key
    String value
in String[] conditionResultKeys
out String[] conditionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeConditionFilter` and `conditionResultKeys` as well as (out) `conditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific attribute value conditions

Key	Explanation	IMSApi data type
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input!	☐ String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input!	☐ String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
ATTRIBUTE_CONDITION_STATE	"Attribute value condition" state [D; I; L; R]: D = deleted I = initial L = invalid R = released Advice: If this key is not passed along, the [R] value is used by default! Special case: If the value [-1] is passed, this filter function is deactivated; there is no restriction!	☐ String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters! Note that the entire value list is always interpreted here; if the filter is to be used for ascertaining individual values, the appropriate wildcards must be used in the entry. Example: Attribute value conditions which contain the default value 80 are to be found. Assumption: An attribute value condition with a value list 70, 80, and 90 (represented internally as the string "70,80,90") exists in the iTAC system. To find this attribute value condition, e.g. [*80*] must be entered as the input value; there would be no hits with the value [80].	☐ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 5-85 "Key Value Pair": attributeConditionFilter

Definition of output parameters for attribute value conditions

Key	Explanation	IMSApi data type
ATTRIBUTE_CONDITION_DESC	Description of the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition	☐ String
ATTRIBUTE_CONDITION_STATE	"Attribute value condition" state [D; I; L; R]: D = deleted I = initial L = invalid R = released	☐ String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN	☐ String

Tab. 5-86 "Key Value Pair": *conditionResultKeys*

5.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeConditionFilter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeConditionFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeConditionFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
conditionResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": conditionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>conditionResultValues</code> !

Tab. 5-87 Parameter (in) »attribGetAttributeCondition«

Parameter (out)

Parameter name	Explanation
conditionResultValues	Depending on parameter (in) <code>conditionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>conditionResultKeys</code> !

Tab. 5-88 Parameter (out) »attribGetAttributeCondition«

5.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

5.13

attribGetAttributeHistory

5.13.1

Function description

This function is used to query the process history of the assigned attributes – more precisely their values – for a particular data object (e.g. a work order or a serial number) within the period under review. The desired object number must be known in order to execute the function.

Results (attributes) are found for the specified data object **ONLY** if an existing attribute value was changed in the period under review. It therefore also follows that attributes whose value has never been changed are not taken into consideration by this function.



Advice

This API function is influenced by station parameter code no. 18500 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

Function matrix objectType

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is **ALWAYS** required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-89 Function matrix *objectType*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeHistory
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in String[] attributeCodeArray
in KeyValue[] attributeHistoryFilter
    String key
    String value
in String[] attributeHistoryResultKeys
out String[] attributeHistoryResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) *attributeHistoryFilter* and *attributeHistoryResultKeys* as well as (out) *attributeHistoryResultValues*). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, 01-01-1970, 0:00:00,000 is used as the end time!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the key is not passed, 12-31-3000, 23:59:59.999 is used as the end time!	☐ DateMs
MAX_ROWS	Maximum number of records in the array returned per function call; the entry is NOT limited by the system, but should not be greater than necessary (excessive data quantities reduce system performance). Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 5-90 "Key Value Pair": *attributeHistoryFilter*

Serial number attributes (objectType = 0)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-91 "Key Value Pair" (serial number attributes): *attributeHistoryResultKeys*

Work order attributes (objectType = 1)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_OBJECT_KEY	Key term for the attribute Advice: A value is output here only if a key term was defined for the work order attribute during creation!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_DESC	Description of the attribute value Advice: A value is output here only if a description for the attribute value was passed for the work order attribute during creation!	☐ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: As long as a part reference was defined for the work order attribute during creation, the corresponding part number of the component appears here; otherwise, an empty string is returned here!	☐ String

Tab. 5-92 "Key Value Pair" (work order attributes): attributeHistoryResultKeys

Container attributes (objectType = 2)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-93 "Key Value Pair" (container attributes): attributeHistoryResultKeys**Site attributes** (objectType = 3)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-94 "Key Value Pair" (site attributes): attributeHistoryResultKeys

Line attributes (objectType = 4)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01! ☐	String

Tab. 5-95 "Key Value Pair" (line attributes): attributeHistoryResultKeys

Cell attributes (objectType = 5)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01! ☐	String

Tab. 5-96 "Key Value Pair" (cell attributes): attributeHistoryResultKeys

Machine group attributes (objectType = 6)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-97 "Key Value Pair" (machine group attributes): attributeHistoryResultKeys**Station attributes** (objectType = 7)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-98 "Key Value Pair" (station attributes): attributeHistoryResultKeys

ERP group attributes (objectType = 8)

Key	Explanation		IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
CHANGE_COMMENT	Individual change comment	☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐	long
CHANGE_USER_NAME	User name of the "changing" user	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String

Tab. 5-99 "Key Value Pair" (ERP group attributes): attributeHistoryResultKeys

Part group attributes (objectType = 9)

Key	Explanation		IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
CHANGE_COMMENT	Individual change comment	☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐	long
CHANGE_USER_NAME	User name of the "changing" user	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String

Tab. 5-100 "Key Value Pair" (part group attributes): attributeHistoryResultKeys

Part attributes (objectType = 10)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-101 "Key Value Pair" (part attributes): attributeHistoryResultKeys**Work plan attributes** (objectType = 11)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-102 "Key Value Pair" (work plan attributes): attributeHistoryResultKeys

Work step attributes (objectType = 12)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01! ☐	String

Tab. 5-103 "Key Value Pair" (work step attributes): attributeHistoryResultKeys

BOM attributes (objectType = 13)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01! ☐	String

Tab. 5-104 "Key Value Pair" (BOM attributes): attributeHistoryResultKeys

BOM component attributes (objectType = 14)

Key	Explanation		IMSApi data type
ALTERNATIVE	Designator for a regular and alternative component in the BOM [0; >0]: 0 = regular component 1 = first alternative component 2 = second alternative component ... = etc. Advice: The output value corresponds to the "Alt. level" field in the BOM editing dialog of the BOM module!	☐	int
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
CHANGE_COMMENT	Individual change comment	☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐	long
CHANGE_USER_NAME	User name of the "changing" user	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String

Tab. 5-105 "Key Value Pair" (BOM component attributes): attributeHistoryResultKeys

Production means attribute (objectType = 15)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-106 "Key Value Pair" (production means attributes): attributeHistoryResultKeys

User attributes (objectType = 16)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-107 "Key Value Pair" (user attributes): attributeHistoryResultKeys

Order-based BOM component attributes (objectType = 17)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
BOM_POSITION_INDEX	Position number of the BOM component	☐ long
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ STRING
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-108 "Key Value Pair" (order-based BOM component attributes): attributeHistoryResultKeys

Order attributes (objectType = 18)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-109 "Key Value Pair" (order attributes): attributeHistoryResultKeys

Receiving position attribute (objectType = 19)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
CHANGE_COMMENT	Individual change comment	☐ String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CHANGE_USER_ID	User identification of the "changing" user	☐ long
CHANGE_USER_NAME	User name of the "changing" user	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Tab. 5-110 "Key Value Pair" (receiving position attributes): attributeHistoryResultKeys

Storage place attributes (objectType = 20)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String

Tab. 5-111 "Key Value Pair" (storage place attributes): attributeHistoryResultKeys

Resource attributes (objectType = 21)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code ☒	String
ATTRIBUTE_VALUE	Value of the attribute ☒	String
CHANGE_COMMENT	Individual change comment ☐	String
CHANGE_DATE	Change date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. ☐	DateMs
CHANGE_USER_ID	User identification of the "changing" user ☐	long
CHANGE_USER_NAME	User name of the "changing" user ☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String

Tab. 5-112 "Key Value Pair" (resource attributes): attributeHistoryResultKeys

5.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
objectNumber	Depending on the attribute category (see <code>objectType</code>), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the <code>objectDetail</code> parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
objectDetail	Depending on the attribute category (see <code>objectType</code>) and the specified data object (see <code>objectNumber</code>), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "objectType" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!
attributeCodeArray	List of the attribute codes that are to be taken into consideration by the function Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Special case: If an empty array is passed (array length= 0), all attributes are taken into account!
attributeHistory-Filter	"Key Value Pair" structure with the following variables:

Parameter name	Explanation
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>attributeHistoryFilter</code> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <code>attributeHistoryFilter</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
<code>attributeHistoryResultKeys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair" (xxx): <code>attributeHistoryResultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>attributeHistoryResultValues</code> !

Tab. 5-113 Parameter (in) »`attribGetAttributeHistory`«

Parameter (out)

Parameter name	Explanation
<code>attributeHistoryResultValues</code>	Depending on parameter (in) <code>attributeHistoryResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>attributeHistoryResultKeys</code> !

Tab. 5-114 Parameter (out) »`attribGetAttributeHistory`«

5.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found

-130	Failure	Work order not found
-174	Failure	Number of records to be output has fallen below the minimum value
-574	Failure	Storage location not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-936	Failure	Attribute not found
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value
-2136	Failure	Resource not found

5.14

attribGetAttributeValues

5.14.1

Function description

This function returns the available attributes for a specified attribute category (e.g. for the data object "Serial numbers"). If a call is to be used to ascertain data for different attribute categories, the IMSApi function »`attribGetAttributeValuesForObjectType`« [▶▶▶] must be used.



Advice

This API function is influenced by station parameter code no. 13600 [▶▶▶] as well as by site parameter code no. 18501 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*The array returned contains **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Refer to the following table for which data objects attributes can be ascertained on the basis of which entries.

Function matrix objectType

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position
Receiving position	19	Receiving number	Receiving position

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-115 Function matrix *objectType*

Special characteristics for work order attributes (1):

- If the order number is not known, a serial number can be used instead to ascertain the work order.
- If, in addition to the work order, a part number is passed – this part number must be contained in the order BOM – the work order attributes with a certain part reference can be specifically queried.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in String[] attributeCodeArray
in int allMergeLevel
in String[] attributeResultKeys
out String[] attributeResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `attributeResultKeys` and (out) `attributeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Serial number attributes (objectType = 0)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-116 "Key Value Pair" (serial number attributes): attributeResultKeys

Work order attributes (objectType = 1)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String

Key	Explanation	IMSApi data type
ATTRIBUTE_- CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_- CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_- CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_OB- JECT_KEY	Key term for the attribute Advice: A value is output here only if a key term was defined for the work order attribute during creation!	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_- DESC	Description of the attribute value Advice: A value is output here only if a description for the attribute value was passed for the work order attribute during creation!	☐ String
ATTRIBUTE_VAL- UE_VALIDATION_FAIL- URE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PART_NUMBER	Part number in the iTAC system Advice: As long as a part reference was defined for the work order attribute during creation, the corresponding part number of the component appears here; otherwise, an empty string is returned here!	☐ String

Tab. 5-117 "Key Value Pair" (work order attributes): *attributeResultKeys*

Container attributes (objectType = 2)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-118 "Key Value Pair" (container attributes): attributeResultKeys

Site attributes (objectType = 3)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-119 "Key Value Pair" (site attributes): attributeResultKeys

Line attributes (objectType = 4)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-120 "Key Value Pair" (line attributes): attributeResultKeys

Cell attributes (objectType = 5)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-121 "Key Value Pair" (cell attributes): attributeResultKeys

Machine group attributes (objectType = 6)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-122 "Key Value Pair" (machine group attributes): attributeResultKeys

Station attributes (objectType = 7)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-123 "Key Value Pair" (station attributes): attributeResultKeys

ERP group attributes (objectType = 8)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-124 "Key Value Pair" (ERP group attributes): attributeResultKeys

Part group attributes (objectType = 9)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-125 "Key Value Pair" (part group attributes): attributeResultKeys

Part attributes (objectType = 10)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-126 "Key Value Pair" (part attributes): attributeResultKeys

Work plan attributes (objectType = 11)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-127 "Key Value Pair" (work plan attributes): attributeResultKeys

Work step attributes (objectType = 12)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-128 "Key Value Pair" (work step attributes): attributeResultKeys

BOM attributes (objectType = 13)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-129 "Key Value Pair" (BOM attributes): attributeResultKeys

BOM component attributes (objectType = 14)

Key	Explanation		IMSApi data type
ALTERNATIVE	Designator for a regular and alternative component in the BOM [0; >0]: 0 = regular component 1 = first alternative component 2 = second alternative component ... = etc. Advice: The output value corresponds to the "Alt. level" field in the BOM editing dialog of the BOM module!	☐	int
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☒	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☒	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐	int
COMP_PART_DESC	Part description in the iTAC system (component)	☐	String
COMP_PART_NUMBER	Part number in the iTAC system (component) Advice: This field is of particular interest if there are multiple BOM alternatives for the queried mounting place!	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String

Key	Explanation	IMSApi data type
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-130 "Key Value Pair" (BOM component attributes): *attributeResultKeys*

Production means attribute (objectType = 15)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-131 "Key Value Pair" (production means attributes): attributeResultKeys

User attributes (objectType = 16)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-132 "Key Value Pair" (user attributes): attributeResultKeys

Order-based BOM component attributes (objectType = 17)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
BOM_POSITION_INDEX	Position number of the BOM component	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
IS_DELTA	Indicates whether the BOM component attribute deviates between the original and order-based BOM [0; 1]: 0 = Attribute is identical (FALSE) 1 = Attribute is NOT identical (TRUE)	☐ boolean

Tab. 5-133 "Key Value Pair" (order-based BOM component attributes): attributeResultKeys

Order attributes (objectType = 18)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-134 "Key Value Pair" (order attributes): attributeResultKeys

Receiving position attribute (objectType = 19)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-135 "Key Value Pair" (receiving position attributes): attributeResultKeys

Storage place attributes (objectType = 20)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-136 "Key Value Pair" (storage place attributes): attributeResultKeys

Resource attributes (objectType = 21)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☒ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: This parameter is optional and enables the classification of attributes into individual types. If no organization by attribute type is to be performed, an empty string or the value [-1] must be passed here!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☒ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 5-137 "Key Value Pair" (resource attributes): attributeResultKeys

5.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
objectNumber	Depending on the attribute category (see <code>objectType</code>), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the <code>objectDetail</code> parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the <code>objectType</code> = 1 case, a serial number can be passed here instead of the order number; the work order is then ascertained on the basis of the serial number.
objectDetail	Depending on the attribute category (see <code>objectType</code>) and the specified data object (see <code>objectNumber</code>), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "objectType" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the case <code>objectType</code> = 1, the input of a serial number position may also be necessary if work order attributes are ascertained on the basis of a serial number (see special case <code>objectNumber</code>)!

Parameter name	Explanation
attributeCodeArray	List of the attribute codes that are to be ascertained by the function Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Special case: If an empty array is passed (array length= 0), all attributes are returned!
allMergeLevel	Integration level mode [0; 1]: 0 = Only the current integration level 1 = All integration levels Advice: Field is only evaluated if serial number attributes are to be ascertained (objectType = 0)!
attributeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): attributeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) attributeResultValues!

Tab. 5-138 Parameter (in) »attribGetAttributeValues«

Parameter (out)

Parameter name	Explanation
attributeResultValues	Depending on parameter (in) attributeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) attributeResultKeys!

Tab. 5-139 Parameter (out) »attribGetAttributeValues«

5.14.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-503	Failure	Container not found

-574	Failure	Storage location not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-911	Failure	Attribute not found for the serial number
-913	Failure	No attributes found for the serial number
-923	Failure	No attributes found for the work order
-931	Failure	No attributes found for the container
-936	Failure	Attribute not found
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2105	Failure	No TR or PM license present for the station
-2136	Failure	Resource not found

5.15

attribGetAttributeValuesForObjectType

5.15.1

Function description

This function returns the available attributes for different attribute categories (e.g. for the data objects "Serial numbers", "Work order" and "Container"). Cf. the IMSApi function »[attribGetAttributeValues](#)« [▶▶▶]; this is used to query data for a specific attribute category.



Advice

*This API function is influenced by site parameter code no. 18501 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! The array returned contains **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns the initially ascertained error code; in this case, correctly processed records are NOT output either!*

Refer to the following table for which data objects attributes can be ascertained on the basis of which entries.

Function matrix objectType

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-140 Function matrix *objectType*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetAttributeValuesForObjectType
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeFilterKeys
    String key
    String value
in String[] attributeFilterArrayKeys
in String[] attributeFilterArrayValues
in String[] attributeResultKeys
out String[] attributeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) attributeFilterKeys, attributeFilterArrayKeys, attributeFilterArrayValues and attributeResultKeys as well as (out) attributeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for function execution (attribute category)

Key	Explanation	IMSApi data type
ALL_MERGE_LEVEL	Integration level mode [0; 1]: 0 = Only the current integration level (default setting) 1 = All integration levels Advice: This key is only evaluated if serial number attributes are to be ascertained (cf. the OBJECT_TYPE = 0 key). If the key is not passed, the value [0] is used by default!	☐ int
MAX_ROWS	Maximum number of records in the array returned per function call; the entry is NOT limited by the system, but should not be greater than necessary (excessive data quantities reduce system performance). Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☒ int

Tab. 5-141 "Key Value Pair": attributeFilterKeys

Searching for specific parameter values (data object)

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☒ String
OBJECT_DETAIL	Depending on the attribute category (see OBJECT_TYPE) and the specified data object (see OBJECT_NUMBER), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "OBJECT_TYPE" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the case OBJECT_TYPE = 1, the input of a serial number position may also be necessary if work order attributes are ascertained on the basis of a serial number (see special case OBJECT_NUMBER)!	☐ String
OBJECT_NUMBER	Depending on the attribute category (see OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the OBJECT_DETAIL parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the OBJECT_TYPE = 1 case, a serial number can be passed here instead of the order number; the work order is then ascertained on the basis of the serial number.	☒ String

Tab. 5-142 "Key Value Pair": attributeFilterArrayKeys

Definition of the output parameters for the found data (attribute data)

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: A value is output here only if an attribute type was defined for the attribute during assignment!	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String

Key	Explanation		IMSApi data type
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the following validation errors: 901 = Data type is invalid 902 = Attribute value condition not fulfilled 903 = Attribute class condition not fulfilled 904 = Attribute class not found 905 = Attribute class not active 906 = Attribute not part of attribute class	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here. In the case of one or more incorrectly processed records, the function itself returns the initially ascertained error code; in this case, correctly processed records are NOT output either!	☒	int
OBJECT_DETAIL	Depending on the attribute category (see OBJECT_TYPE) and the data object (see OBJECT_NUMBER), the data-object detail specification is – where relevant – output here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing of the attribute category requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!	☐	String
OBJECT_NUMBER	Depending on the attribute category (see OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is output here. Advice: Note that, for certain attribute categories, the OBJECT_DETAIL parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!	☐	String

Tab. 5-143 "Key Value Pair": attributeResultKeys

5.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
attributeFilter-Keys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the result list is to be restricted; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeFilterKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attributeFilter-ArrayKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>attributeFilterArrayValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeFilterArrayKeys table.
attributeFilter-ArrayValues	Value list for the parameters passed by means of <code>attributeFilterArrayKeys</code> . Advice: Refer to the "Key Value Pair": attributeFilterArrayKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attributeResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": attributeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>attributeResultValues</code> !

Tab. 5-144 Parameter (in) »attribGetAttributeValuesForObjectType«

Parameter (out)

Parameter name	Explanation
attributeResult-Values	Depending on parameter (in) <code>attributeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>attributeResultKeys</code> !

Tab. 5-145 Parameter (out) »attribGetAttributeValuesForObjectType«

5.15.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found

-130	Failure	Work order not found
-174	Failure	Number of records to be output has fallen below the minimum value
-503	Failure	Container not found
-574	Failure	Storage location not found
-576	Failure	Receiving not found
-577	Failure	Receiving position not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-905	Failure	Invalid value for "objectType"
-913	Failure	No attributes found for the serial number
-923	Failure	No attributes found for the work order
-931	Failure	No attributes found for the container
-936	Failure	Attribute not found
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value
-2136	Failure	Resource not found

5.16

attribGetObjectsForAttributeValues

5.16.1

Function description

This function returns a wide range of data objects for one or more attributes/attribute values that are to be specified. The following data objects can be ascertained:

- Serial numbers (objectType = 0)
- Work orders (objectType = 1)
- Containers / Lots (objectType = 2)
- Sites (objectType = 3)
- Lines (objectType = 4)
- Cells (objectType = 5)
- Machine groups (objectType = 6)
- Stations (objectType = 7)
- ERP groups (objectType = 8)
- Part groups (objectType = 9)
- Parts (objectType = 10)
- Work plans (objectType = 11)
- Work steps (objectType = 12)
- BOMs (objectType = 13)
- BOM components (objectType = 14)
- Production means (objectType = 15)
- Users (objectType = 16)
- Order-based BOM components (objectType = 17)
- Orders (objectType = 18)
- Receiving position (objectType = 19)
- Storage places (objectType = 20)
- Resources (objectType = 21)

**Advice**

This API function is influenced by station parameter code nos. 18500 [▶▶▶] and 18510 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribGetObjectsForAttributeValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String attributeCode
in String attributeValue
in int maxRows
in KeyValue[] attributeFilters
    String key
    String value
in String[] objectResultKeys
out String[] objectResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) attributeFilters and objectResultKeys as well as parameter (out) objectResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐). The tables are described in the following sequence:

"Key Value Pair" (serial number attributes): attributeFilters

"Key Value Pair" (serial number attributes): objectResultKeys

"Key Value Pair" (work order attributes): attributeFilters

"Key Value Pair" (work order attributes): objectResultKeys

"Key Value Pair" (container attributes): attributeFilters

"Key Value Pair" (container attributes): objectResultKeys

"Key Value Pair" (site attributes): attributeFilters

"Key Value Pair" (site attributes): objectResultKeys

"Key Value Pair" (line attributes): attributeFilters

"Key Value Pair" (line attributes): objectResultKeys

"Key Value Pair" (cell attributes): attributeFilters

"Key Value Pair" (cell attributes): objectResultKeys

"Key Value Pair" (machine group attributes): attributeFilters

"Key Value Pair" (machine group attributes): objectResultKeys

"Key Value Pair" (station attributes): attributeFilters

"Key Value Pair" (station attributes): objectResultKeys

"Key Value Pair" (ERP group attributes): attributeFilters

"Key Value Pair" (ERP group attributes): objectResultKeys

"Key Value Pair" (part group attributes): attributeFilters

"Key Value Pair" (part group attributes): objectResultKeys

"Key Value Pair" (part attributes): attributeFilters
"Key Value Pair" (part attributes): objectResultKeys
"Key Value Pair" (work plan attributes): attributeFilters
"Key Value Pair" (work plan attributes): objectResultKeys
"Key Value Pair" (work step attributes): attributeFilters
"Key Value Pair" (work step attributes): objectResultKeys
"Key Value Pair" (BOM attributes): attributeFilters
"Key Value Pair" (BOM attributes): objectResultKeys
"Key Value Pair" (BOM component attributes): attributeFilters
"Key Value Pair" (BOM component attributes): objectResultKeys
"Key Value Pair" (production means attributes): attributeFilters
"Key Value Pair" (production means attributes): objectResultKeys
"Key Value Pair" (user attributes): attributeFilters
"Key Value Pair" (user attributes): objectResultKeys
"Key Value Pair" (order-based BOM component attributes): attributeFilters
"Key Value Pair" (order-based BOM component attributes): objectResultKeys
"Key Value Pair" (order attributes): attributeFilters
"Key Value Pair" (order attributes): objectResultKeys
"Key Value Pair" (receiving position attributes): attributeFilters
"Key Value Pair" (receiving position attributes): objectResultKeys
"Key Value Pair" (storage place attribute): attributeFilters
"Key Value Pair" (storage place attributes): objectResultKeys
"Key Value Pair" (resource attributes): attributeFilters
"Key Value Pair" (resource attributes): objectResultKeys

Serial number attributes (objectType = 0)

Key	Explanation	IMSApi data type
ALL_MERGE_LEVEL	Integration level mode [0; 1]: 0 = Output of the serial number data for the current integration level (default setting); this mode is used for all entries "not equal to" [1]. 1 = Output of the serial number data for the current and the highest integration level. Advice: If the value 1 is passed, this function additionally returns data of the highest integration level. Here, the integration level (i.e. the serial number) at which the attribute was found is irrelevant. Accordingly, the following keys are filled with values in this mode only: - PART_MASTER_DESC - PART_MASTER_NUMBER - SERIAL_MASTER_NUMBER - SERIAL_MASTER_NUMBER_POS - WORKORDER_MASTER_NUMBER	int
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs

Tab. 5-146 "Key Value Pair" (serial number attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	int

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
PART_DESC	Part description in the iTAC system	String
PART_MASTER_DESC	Part description in the iTAC system (highest integration level) Advice: This parameter is only output if 1 was passed for the ALL_MERGE_LEVEL filter!	String
PART_MASTER_NUMBER	Part number in the iTAC system (highest integration level) Advice: This parameter is only output if 1 was passed for the ALL_MERGE_LEVEL filter!	String
PART_NUMBER	Part number in the iTAC system	String
SERIAL_MASTER_NUMBER	Serial number of the final product (highest integration level) Advice: This parameter is only output if 1 was passed for the ALL_MERGE_LEVEL filter!	String
SERIAL_MASTER_NUMBER_POS	Position of the single panel in the unit-array (highest integration level) Advice: This parameter is only output if 1 was passed for the ALL_MERGE_LEVEL filter!	String
SERIAL_NUMBER	Serial number of the panel	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	String
WORKORDER_MASTER_NUMBER	Number of the production order (highest integration level) Advice: This parameter is only output if 1 was passed for the ALL_MERGE_LEVEL filter!	String
WORKORDER_NUMBER	Production order number	String

Tab. 5-147 "Key Value Pair" (serial number attributes): *objectResultKeys*

Work order attributes (objectType = 1)

Key	Explanation	IMSApi data type
ALL_MERGE_LEVEL	- not currently used -	☐ int
ATTRIBUTE_OBJECT_KEY	Key term for the attribute Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String
ATTRIBUTE_TYPE	Attribute type Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 5-148 "Key Value Pair" (work order attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_OBJECT_KEY	Key term for the attribute	☐ String
ATTRIBUTE_PART_NUMBER	Part number in the iTAC system Advice: This parameter is only output if a part reference was established during the assignment of work order attributes!	☐ String
ATTRIBUTE_TYPE	Attribute type	☐ String
ATTRIBUTE_VALUE	Attribute value	☐ String
ATTRIBUTE_VALUE_DESC	Description of the attribute value	☐ String

Key	Explanation		IMSApi data type
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐	int
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
QUANTITY	Order quantity	☐	double
WORKORDER_ID	Internal identification number of the production order	☐	long
WORKORDER_NUMBER	Production order number	☐	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String

Tab. 5-149 "Key Value Pair" (work order attributes): *objectResultKeys*

Container attributes (objectType = 2)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-150 "Key Value Pair" (container attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
CLASSIFICATION	Classification of the container	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs

Key	Explanation		IMSApi data type
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number of the container material in the iTAC system	☐	String
QUANTITY_ACTUAL	Current quantity in the material bin	☐	double
QUANTITY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double

Tab. 5-151 "Key Value Pair" (container attributes): *objectResultKeys*

Site attributes (objectType = 3)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs

Tab. 5-152 "Key Value Pair" (site attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
SITE_DESC	Description of the site	String
SITE_NUMBER	Site number	String
SITE_NUMBER_EXT	Site number from an external system	String

Tab. 5-153 "Key Value Pair" (site attributes): objectResultKeys

Line attributes (objectType = 4)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs

Tab. 5-154 "Key Value Pair" (line attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
LINE_DESC	Line description	String
LINE_NUMBER	Line number	String

Tab. 5-155 "Key Value Pair" (line attributes): objectResultKeys

Cell attributes (objectType = 5)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs

Tab. 5-156 "Key Value Pair" (cell attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	int
CELL_DESC	Cell description	String
CELL_NUMBER	Cell number	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String

Tab. 5-157 "Key Value Pair" (cell attributes): objectResultKeys

Machine group attributes (objectType = 6)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-158 "Key Value Pair" (machine group attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
MACHINE_GROUP_DESC	Description of the machine group	☐ String
MACHINE_GROUP_NUMBER	Machine group number	☐ String

Tab. 5-159 "Key Value Pair" (machine group attributes): objectResultKeys

Station attributes (objectType = 7)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-160 "Key Value Pair" (station attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 5-161 "Key Value Pair" (station attributes): objectResultKeys

ERP group attributes (objectType = 8)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs

Tab. 5-162 "Key Value Pair" (ERP group attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String
ATTRIBUTE_DESC	Attribute designator	String
ATTRIBUTE_VALUE	Value of the attribute	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
ERP_GROUP_DESC	Description of the ERP group	String
ERP_GROUP_NUMBER	ERP group number in the iTAC system	String

Tab. 5-163 "Key Value Pair" (ERP group attributes): objectResultKeys

Part group attributes (objectType = 9)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_GROUP	Part group in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PART_GROUP_TYPE	Part group type [B; E; F; K; L; M; W; S]: B = Unit E = Equipment F = Tool feeder K = Component L = Printed circuit board M = Material work plan W = Master work plan S = Lot Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 5-164 "Key Value Pair" (part group attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
PART_GROUP	Part group in the iTAC system	String
PART_GROUP_DESC	Part group description in the iTAC system	String
PART_GROUP_TYPE	Part group type [B; E; F; K; L; M; W; S]: B = Unit E = Equipment F = Tool feeder K = Component L = Printed circuit board M = Material work plan W = Master work plan S = Lot	String

Tab. 5-165 "Key Value Pair" (part group attributes): *objectResultKeys*

Part attributes (objectType = 10)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-166 "Key Value Pair" (part attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String

Tab. 5-167 "Key Value Pair" (part attributes): objectResultKeys

Work plan attributes (objectType = 11)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 5-168 "Key Value Pair" (work plan attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String

Key	Explanation	IMSApi data type
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: <i>If no problem is found during attribute assignment, the error code 0 is output here!</i>	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: <i>The data type UNDEF was used only for data records prior to release 8.50.01!</i>	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	☐ String
WORKPLAN_ID	Internal identification number of the work plan	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 5-169 "Key Value Pair" (work plan attributes): *objectResultKeys*

Work step attributes (objectType = 12)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 5-170 "Key Value Pair" (work step attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String

Key	Explanation		IMSApi data type
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« (▶▶▶)) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	☐	String
WORKPLAN_ID	Internal identification number of the work plan	☐	String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	String

Tab. 5-171 "Key Value Pair" (work step attributes): *objectResultKeys*

BOM attributes (objectType = 13)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 5-172 "Key Value Pair" (BOM attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
BOM_ID	Internal identification number of the BOM	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ String
BOM_VERSION_ERP	BOM version in the ERP system	☐ String

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
PART_DESC	Part description in the iTAC system	String
PART_NUMBER	Part number in the iTAC system	String

Tab. 5-173 "Key Value Pair" (BOM attributes): *objectResultKeys*

BOM component attributes (objectType = 14)

Key	Explanation	IMSApi data type
COMP_PART_NUMBER	Part number in the iTAC system (component) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	String
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	DateMs
PART_NUMBER	Part number in the iTAC system (product) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	String

Tab. 5-174 "Key Value Pair" (BOM component attributes): *attributeFilters*

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	String
ATTRIBUTE_CLASS_NAME	Attribute class name	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	String
ATTRIBUTE_CODE	Attribute code	String

Key	Explanation	IMSApi data type
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
BOM_ID	Internal identification number of the BOM	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ String
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_PART_DESC	Part description in the iTAC system (component)	☐ String
COMP_PART_NUMBER	Part number in the iTAC system (component)	☐ String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
PART_DESC	Part description in the iTAC system (product)	☐ String
PART_NUMBER	Part number in the iTAC system (product)	☐ String

Tab. 5-175 "Key Value Pair" (BOM component attributes): *objectResultKeys*

Production means attribute (objectType = 15)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number (device class, carrier class, mold/holder class) on the basis of which the production means was created Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 5-176 "Key Value Pair" (production means attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String

Key	Explanation		IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means into equipment and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity	☐	String
EQUIPMENT_DESCRIPTION	Description of the production means Advice: Here, depending on the EQUIPMENT_TYPE, this is the respective equipment, cavity or carrier description!	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number (cf. EQUIPMENT_NUMBER)!	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number!	☐	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐	String
PART_DESC	Part description (device class, carrier class, mold/holder class) on the basis of which the production means was created	☐	String
PART_NUMBER	Part number (device class, carrier class, mold/holder class) on the basis of which the production means was created	☐	String

Tab. 5-177 "Key Value Pair" (production means attributes): *objectResultKeys*

User attributes (objectType = 16)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-178 "Key Value Pair" (user attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
USER_FIRST_NAME	First name of the user	☐ String
USER_LAST_NAME	Last name of the user	☐ String
USER_NAME	Login name of the user (user identification)	☐ String

Tab. 5-179 "Key Value Pair" (user attributes): objectResultKeys

Order-based BOM component attributes (objectType = 17)

Key	Explanation	IMSApi data type
COMP_PART_NUMBER	Part number in the iTAC system (component) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
PART_NUMBER	Part number in the iTAC system (product) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String

Tab. 5-180 "Key Value Pair" (order-based BOM component attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
BOM_ID	Internal identification number of the BOM	☐ String

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	String
BOM_VERSION	BOM version from the iTAC system	String
BOM_VERSION_ERP	BOM version in the ERP system	String
COMP_NAME	Mounting place conformant with the iTAC BOM	String
COMP_PART_DESC	Part description in the iTAC system (component)	String
COMP_PART_NUMBER	Part number in the iTAC system (component)	String
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
PART_DESC	Part description in the iTAC system (product)	String
PART_NUMBER	Part number in the iTAC system (product)	String
WORKORDER_NUMBER	Production order number	String
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	String

Tab. 5-181 "Key Value Pair" (order-based BOM component attributes): *objectResultKeys*

Order attributes (objectType = 18)

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒	DateMs
PURCHASE_ORDER_ITEM	Order position	☐	String
PURCHASE_ORDER_NUMBER	Order number	☐	String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usually used in an ERP system	☐	String

Tab. 5-182 "Key Value Pair" (order attributes): attributeFilters

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☐	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☐	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐	int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐	String
PART_DESC	Part description in the iTAC system	☐	String

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system	☐ String
PURCHASE_DELIVERY- _STATE	Delivery state [0; 1; 19]: 0 = Open 1 = Closed 19 = Deleted	☐ int
PURCHASE_ORDER_ITEM	Order position	☐ String
PURCHASE_ORDER_NUM- BER	Order number	☐ String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usu- ally used in an ERP system	☐ String
SUPPLIER_CODE	Supplier designator (code)	☐ String
SUPPLIER_NAME	Supplier name	☐ String

Tab. 5-183 "Key Value Pair" (order attributes): *objectResultKeys*

Receiving position attribute (objectType = 19)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
DATE_TO	Time stamp for the end of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the key is not passed, 12-31-3000, 23:59:59.999 is used as the end time!	☐ DateMs
DELIVERY_NOTE	Delivery slip number	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String

Tab. 5-184 "Key Value Pair" (receiving position attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
DELIVERY_DATE	Delivery time according to delivery slip date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	DateMs
DELIVERY_NOTE	Delivery slip number	String
DELIVERY_QUANTITY	Display of the delivery quantity for the order position	String
DELIVERY_STATE	State of the receiving position [T; K]: T = open K = complete	String
INCOMING_GOODS_NUMBER	Number of the receiving	String
INCOMING_GOODS_POSITION	Position number in the receiving	String
MANUFACTURER_CODE	Manufacturer's designator (code)	String
MANUFACTURER_NAME	Manufacturer's name	String
PART_DESC	Part description in the iTAC system	String
PART_NUMBER	Part number in the iTAC system	String
PURCHASE_ORDER_ITEM	Order position	String
PURCHASE_ORDER_NUMBER	Order number	String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usually used in an ERP system	String
RECEIVING_DATE	Point in time of the receiving booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	DateMs

Tab. 5-185 "Key Value Pair" (receiving position attributes): *objectResultKeys*

Storage place attributes (objectType = 20)

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒	DateMs
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐	String
STORAGE_STATE	Designator (code) for the storage place state Advice: Storage states are customer-specific and are created during the course of the initial commissioning!	☐	String
STORAGE_TYPE	Designator (code) for the storage place type Advice: Storage types are customer-specific and are created during the course of the initial commissioning!	☐	String

Tab. 5-186 "Key Value Pair" (storage place attributes): attributeFilters

Key	Explanation		IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐	String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐	String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐	int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐	String
ATTRIBUTE_CODE	Attribute code	☐	String
ATTRIBUTE_DESC	Attribute designator	☐	String
ATTRIBUTE_VALUE	Value of the attribute	☐	String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐	int

Key	Explanation	IMSApi data type
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	String
STORAGE_BARCODE	Bar code for the storage place; it does not have to be the same as the storage place number (STORAGE_NUMBER)	String
STORAGE_DESC	Storage description in the iTAC system	String
STORAGE_ID	Internal identification number of the storage place	String
STORAGE_NUMBER	Storage place number in the iTAC system	String
STORAGE_STATE	Designator (code) for the storage place state Advice: Storage states are customer-specific and are created during the course of the initial commissioning!	String
STORAGE_TYPE	Designator (code) for the storage place type Advice: Storage types are customer-specific and are created during the course of the initial commissioning!	String

Tab. 5-187 "Key Value Pair" (storage place attributes): *objectResultKeys*

Resource attributes (objectType = 21)

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the attribute value assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs

Tab. 5-188 "Key Value Pair" (resource attributes): attributeFilters

Key	Explanation	IMSApi data type
ATTRIBUTE_CLASS_DESC	Attribute class description	☐ String
ATTRIBUTE_CLASS_NAME	Attribute class name	☐ String
ATTRIBUTE_CLASS_VERSION	Attribute class version in the iTAC system	☐ int
ATTRIBUTE_CLASS_VERSION_ERP	Attribute class version in the ERP system	☐ String
ATTRIBUTE_CODE	Attribute code	☐ String
ATTRIBUTE_DESC	Attribute designator	☐ String
ATTRIBUTE_VALUE	Value of the attribute	☐ String
ATTRIBUTE_VALUE_VALIDATION_FAILURE	Error code which was ascertained in the context of attribute assignment (see the IMSApi function »attribAppendAttributeValues« [▶▶▶]) by the check routine and stored in the record; at present, this can be the warning 901, 902, 903, 904, 905 or 906. Advice: If no problem is found during attribute assignment, the error code 0 is output here!	☐ int
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN - UNDEF (only for data records prior to release 8.50.01) Advice: The data type UNDEF was used only for data records prior to release 8.50.01!	☐ String
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String

Tab. 5-189 "Key Value Pair" (resource attributes): objectResultKeys

5.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute
attributeCode	Attribute code Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!
attributeValue	Value of the attribute Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!
maxRows	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!
attributeFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters (dependent on the <code>objectType</code>) can be found above in the corresponding table for attribute category "Key Value Pair" (xxx): attributeFilters .
value	Value list for the parameters passed by means of <code>key</code> Advice: Refer to the corresponding table above for attribute category "Key Value Pair" (xxx): attributeFilters to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
<code>objectResultKeys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters (dependent on <code>object-Type</code>) can be found above in the corresponding table for attribute category "Key Value Pair" (xxx): <code>objectResultKeys</code> . Advice: Only the parameters passed here are output by the function; see parameter (out) <code>objectResultValues</code> !

Tab. 5-190 Parameter (in) »`attribGetObjectsForAttributeValues`«

Parameter (out)

Parameter name	Explanation
<code>objectResultValues</code>	Depending on parameter (in) <code>objectResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>objectResultKeys</code> !

Tab. 5-191 Parameter (out) »`attribGetObjectsForAttributeValues`«

5.16.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-912	Failure	No serial numbers found for the attribute
-922	Failure	No work orders found for the attribute
-932	Failure	No containers found for the attribute
-954	Failure	Attribute value cannot be modified
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.17

attribManageAttributeClassAttribute

5.17.1

Function description

This function is used to manage the attributes belonging to an attribute class. Attributes existing in the system can be assigned to a certain attribute class, or an already assigned attribute can be removed. The attribute (`ATTRIBUTE_CODE`), the associated attribute category (`OBJECT_TYPE`) and the function mode to be used (`FUNC_MODE` -> Add/Remove) must always be specified.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Editing is possible for the following data objects:

- Serial numbers (`OBJECT_TYPE = 0`)
- Work orders (`OBJECT_TYPE = 1`)
- Containers / Lots (`OBJECT_TYPE = 2`)
- Sites (`OBJECT_TYPE = 3`)
- Lines (`OBJECT_TYPE = 4`)
- Cells (`OBJECT_TYPE = 5`)
- Machine groups (`OBJECT_TYPE = 6`)
- Stations (`OBJECT_TYPE = 7`)
- ERP groups (`OBJECT_TYPE = 8`)
- Part groups (`OBJECT_TYPE = 9`)
- Parts (`OBJECT_TYPE = 10`)
- Work plans (`OBJECT_TYPE = 11`)
- Work steps (`OBJECT_TYPE = 12`)
- BOMs (`OBJECT_TYPE = 13`)
- BOM components (`OBJECT_TYPE = 14`)
- Production means (`OBJECT_TYPE = 15`)
- Users (`OBJECT_TYPE = 16`)
- Order-based BOM components (`OBJECT_TYPE = 17`)
- Orders (`OBJECT_TYPE = 18`)
- Receiving position (`OBJECT_TYPE = 19`)
- Storage places (`OBJECT_TYPE = 20`)
- Resources (`OBJECT_TYPE = 21`)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribManageAttributeClassAttribute
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeClassManageAttributeFilter
    String key
    String value
in String[] classAttributeKeys
in String[] classAttributeValues
out String[] classAttributeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) attributeClassManageAttributeFilter, classAttributeKeys ad classAttributeValues as well as (out) classAttributeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining desired attribute class

Key	Explanation	IMSApi data type
ATTRIBUTE_ - CLASS_NAME	Name of the attribute class	☒ String
ATTRIBUTE_ - CLASS_VERSION	Version of the attribute class in the iTAC system	☒ int

Tab. 5-192 "Key Value Pair": attributeClassManageAttributeFilter

Input values for attribute code and for function execution

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: The specified attribute code must match the attribute category (see key OBJECT_TYPE)!	☒ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☐ int
FUNC_MODE	Function mode "Add/Remove" [1; 2]: 1 = Attribute code is added 2 = Attribute code is removed Advice: The function is results-oriented; in this context, this means that the function is executed successfully (see error code 0) even if the attribute code to be deleted was not found in the attribute class or an attribute code to be added already exists!	☒ int
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Advice: The specified attribute category must match the attribute code (see key ATTRIBUTE_CODE)!	☒ int

Tab. 5-193 "Key Value Pair": classAttributeKeys

5.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeClass- ManageAttribute- Filter	Key Value Pair" structure with the following variables:
key	List of the parameters (attribute class) on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeClassManageAttributeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeClassManageAttributeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
classAttribute- Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>classAttributeValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": classAttributeKeys table. Advice: Only the parameters passed here are processed by the function!
classAt- tributeValues	Values for the parameters selected by means of <code>classAttributeKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>classAttributeKeys</code> ! Refer to the "Key Value Pair": classAttributeKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-194 Parameter (in) »attribManageAttributeClassAttribute«

Parameter (out)

Parameter name	Explanation
classAttributeRe- sultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>classAttributeKeys</code> and <code>classAttributeValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>classAttributeKeys</code> !

Tab. 5-195 Parameter (out) »attribManageAttributeClassAttribute«

5.17.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-905	Failure	Invalid value for "objectType"
-941	Failure	Attribute value condition not found
-942	Failure	Attribute class not found
-944	Failure	Attribute class -> Invalid type
-945	Failure	Attribute value condition not released
-947	Failure	Attribute class has an invalid state
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

5.18

attribManageAttributeClassCondition

5.18.1

Function description

This function is used to manage class-specific attribute value conditions. Specific attribute value conditions which are valid only in combination with the attribute class can be assigned, edited or removed using this function (see `FUNC_MODE`). This provides the possibility of overriding or deactivating an attribute value condition (key word "original condition") assigned to the attribute.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Editing is possible for the following data objects:

- Serial numbers (OBJECT_TYPE = 0)
- Work orders (OBJECT_TYPE = 1)
- Containers / Lots (OBJECT_TYPE = 2)
- Sites (OBJECT_TYPE = 3)
- Lines (OBJECT_TYPE = 4)
- Cells (OBJECT_TYPE = 5)
- Machine groups (OBJECT_TYPE = 6)
- Stations (OBJECT_TYPE = 7)
- ERP groups (OBJECT_TYPE = 8)
- Part groups (OBJECT_TYPE = 9)
- Parts (OBJECT_TYPE = 10)
- Work plans (OBJECT_TYPE = 11)
- Work steps (OBJECT_TYPE = 12)
- BOMs (OBJECT_TYPE = 13)
- BOM components (OBJECT_TYPE = 14)
- Production means (OBJECT_TYPE = 15)
- Users (OBJECT_TYPE = 16)
- Order-based BOM components (OBJECT_TYPE = 17)
- Orders (OBJECT_TYPE = 18)
- Receiving position (OBJECT_TYPE = 19)
- Storage places (OBJECT_TYPE = 20)
- Resources (OBJECT_TYPE = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribManageAttributeClassCondition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeClassManageAttributeFilter
    String key
    String value
in String[] classConditionKeys
in String[] classConditionValues
out String[] classConditionResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeClassManageAttributeFilter`, `classConditionKeys` and `classConditionValues` as well as (out) `classConditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining desired attribute class

Key	Explanation	IMSApi data type
ATTRIBUTE_ - CLASS_NAME	Name of the attribute class	☒ String
ATTRIBUTE_ - CLASS_VERSION	Version of the attribute class in the iTAC system	☒ int

Tab. 5-196 "Key Value Pair": `attributeClassManageAttributeFilter`

Input values for attribute value condition and for function execution

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: The specified attribute code must match the attribute category (see key OBJECT_TYPE)!	☒ String
ATTRIBUTE_CONDITION_AT_CLASS_ENABLED	Indicates whether the attribute value condition is active in the attribute class [0; 1]: 0 = Condition is NOT active (FALSE) 1 = Condition is active (TRUE) Advice: All active attribute value conditions within an attribute class are "OR" linked; i.e. the fulfillment of one condition is sufficient for validation!	☐ boolean
ATTRIBUTE_CONDITION_DESC	Description of the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MAX	Upper limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_MIN	Lower limit for the attribute value condition	☐ String
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition	☒ String
ATTRIBUTE_CONDITION_VALUES	List of values allowed for the attribute value condition Advice: If more values are to be passed, they must be separated by commas (e.g. 22,33,44)!	☐ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition	☒ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☐ int
FUNC_MODE	Function mode "Add/Remove" [1; 2]: 1 = Attribute value condition is added 2 = Attribute value condition is removed Advice: The function is results-oriented; in this context, this means that the function is executed successfully (see error code 0) even if the attribute value condition to be deleted was not found in the attribute class/attribute code or an attribute value condition to be added already exists! Special case "Editing of attribute value condition": The behavior described in the advice notice means that it is possible to edit most data fields of an already assigned attribute value condition (e.g. MIN/MAX limit values, the version, etc.).	☒ int

Key	Explanation	IMSApi data type
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Advice: The specified attribute category must match the attribute code (see key ATTRIBUTE_CODE)!	☒ int

Tab. 5-197 "Key Value Pair": *classConditionKeys*

5.18.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeClass- ManageAttribute- Filter	Key Value Pair" structure with the following variables:
key	List of the parameters (attribute class) on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeClassManageAttributeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeClassManageAttributeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
classCondition- Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>classConditionValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": classConditionKeys table. Advice: Only the parameters passed here are processed by the function!
classCondition- Values	Values for the parameters selected by means of <code>classConditionKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>classConditionKeys</code> ! Refer to the "Key Value Pair": classConditionKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-198 Parameter (in) »attribManageAttributeClassCondition«

Parameter (out)

Parameter name	Explanation
classConditionRe- sultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>classConditionKeys</code> and <code>classConditionValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>classConditionKeys</code> !

Tab. 5-199 Parameter (out) »attribManageAttributeClassCondition«

5.18.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-905	Failure	Invalid value for "objectType"
-942	Failure	Attribute class not found
-944	Failure	Attribute class -> Invalid type
-947	Failure	Attribute class has an invalid state
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

5.19

attribManageAttributeCondition

5.19.1

Function description

This function is used to manage attribute-specific attribute value conditions. An attribute value condition existing in the system can be assigned a certain attribute, or an already assigned condition can be removed again. In addition to the attribute value conditions (ATTRIBUTE_CONDITION_NAME), the attribute (ATTRIBUTE_CODE), the attribute category (OBJECT_TYPE) and the function mode to be used (FUNC_MODE -> Add/Remove) must also always be specified.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Editing is possible for the following data objects:

- Serial numbers (OBJECT_TYPE = 0)
- Work orders (OBJECT_TYPE = 1)
- Containers / Lots (OBJECT_TYPE = 2)
- Sites (OBJECT_TYPE = 3)
- Lines (OBJECT_TYPE = 4)
- Cells (OBJECT_TYPE = 5)
- Machine groups (OBJECT_TYPE = 6)
- Stations (OBJECT_TYPE = 7)
- ERP groups (OBJECT_TYPE = 8)
- Part groups (OBJECT_TYPE = 9)
- Parts (OBJECT_TYPE = 10)
- Work plans (OBJECT_TYPE = 11)
- Work steps (OBJECT_TYPE = 12)
- BOMs (OBJECT_TYPE = 13)
- BOM components (OBJECT_TYPE = 14)
- Production means (OBJECT_TYPE = 15)
- Users (OBJECT_TYPE = 16)
- Order-based BOM components (OBJECT_TYPE = 17)
- Orders (OBJECT_TYPE = 18)
- Receiving position (OBJECT_TYPE = 19)
- Storage places (OBJECT_TYPE = 20)
- Resources (OBJECT_TYPE = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int attribManageAttributeCondition
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] attributeCodeFilter
    String key
    String value
  in String[] conditionKeys
  in String[] conditionValues
  out String[] conditionResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeCodeFilter`, `conditionKeys` and `conditionValues` as well as (out) `conditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining desired attribute code

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code Advice: <i>The specified attribute code must match the attribute category (see key OBJECT_TYPE)!</i>	☒ String
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute Advice: <i>The specified attribute category must match the attribute code (see key ATTRIBUTE_CODE)!</i>	☒ int

Tab. 5-200 "Key Value Pair": `attributeCodeFilter`

Input values for attribute value condition and for function execution

Key	Explanation	IMSApi data type
ATTRIBUTE_CONDITION_NAME	Name of the attribute value condition	☒ String
ATTRIBUTE_CONDITION_VERSION	Version of the attribute value condition Advice: <i>If the key is not passed, the highest valid version number in the system is always considered!</i> The behavior for the "Remove" function mode, described in the advice notice, means that it may not be possible to remove an assigned attribute value condition if a higher version exists in the system. Recommendation for removal: Always specify the concrete version number.	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!</i>	☐ int
FUNC_MODE	Function mode "Add/Remove" [1; 2]: 1 = Attribute value condition is added 2 = Attribute value condition is removed Advice: <i>The function is results-oriented; in this context, this means that the function is executed successfully (see error code 0) even if the attribute value condition to be deleted was not found in the attribute code or an attribute value condition to be added is already provided!</i>	☒ int

Tab. 5-201 "Key Value Pair": conditionKeys

5.19.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeCodeFilter	"Key Value Pair" structure with the following variables
key	List of the parameters (attribute code) on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeCodeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeCodeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
conditionKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>conditionValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": conditionKeys table. Advice: Only the parameters passed here are processed by the function!
conditionValues	Values for the parameters selected by means of <code>conditionKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>conditionKeys</code> ! Refer to the "Key Value Pair": conditionKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-202 Parameter (in) »attribManageAttributeCondition«

Parameter (out)

Parameter name	Explanation
conditionResult-Values	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>conditionKeys</code> and <code>conditionValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>classConditionKeys</code> !

Tab. 5-203 Parameter (out) »attribManageAttributeCondition«

5.19.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-168	Failure	Invalid value for the function mode (control flag)
-901	Failure	Attribute code not found
-940	Failure	Attribute value condition -> Type is invalid
-941	Failure	Attribute value condition not found
-946	Failure	Attribute code catalog not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

5.20

attribRemoveAttributeClassValue

5.20.1

Function description

This function removes one or all attribute values for a certain data object, provided that the associated attributes are NOT referenced to the data object separately but by means of an attribute class. If values for attributes without class reference are to be deleted, the API function »attribRemoveAttributeValue« [▶▶▶] must be used.



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = -939) in the event of one or more faulty records!

Refer to the following table for which data objects attribute values can be deleted on the basis of which entries.

Function matrix ob-
jectType

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4
Stations	7	Station number	Station type (T, M) -> see advice for 4

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the "I" (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position
Receiving position	19	Receiving number	Receiving position

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-204 Function matrix *objectType*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribRemoveAttributeClassValue
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] attributeRemoveKeys
in String[] attributeRemoveValues
out String[] attributeRemoveResult
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) *attributeRemoveKeys* and *attributeRemoveValues* as well as (out) *attributeRemoveResult*). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation	IMSApi data type
ATTRIBUTE_ - CLASS_NAME	Name of the attribute class Special case: If the key is not passed or is passed with the value [-1], all attribute classes are taken into consideration! Advice: Depending on the setting for ATTRIBUTE_CODE, all values can if required be deleted for all attributes of all used attribute classes!	☐ String
ATTRIBUTE_CODE	Attribute code Special case: If the key is not passed or is passed with the value [-1], all attribute values for all assigned attributes are deleted!	☐ String

Key	Explanation	IMSApi data type
ATTRIBUTE_OBJECT_KEY	Key term for the attribute Advice: <i>Field is only evaluated if work order attributes are to be processed (OBJECT_TYPE = 1)!</i> Special case: If the value [-1] is passed, all entries are deleted independent of the key term!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -939)!</i>	☐ int
OBJECT_DETAIL	Depending on the attribute category (see OBJECT_TYPE) and the specified data object (see OBJECT_NUMBER), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: <i>The field is only evaluated if the processing for the listed "OBJECT_TYPE" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!</i> Special case: For the case OBJECT_TYPE = 1, the input of a serial number position may also be necessary if work order attributes are ascertained on the basis of a serial number (see special case OBJECT_NUMBER)!	☐ String
OBJECT_NUMBER	Depending on the attribute category (see OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is passed here. Advice: <i>Note that, for certain attribute categories, the OBJECT_DETAIL parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix!</i> Special case: For the OBJECT_TYPE = 1 case, a serial number can be passed here instead of the order number; the work order is then ascertained on the basis of the serial number.	☒ String
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☒ int

Tab. 5-205 "Key Value Pair": *attributeRemoveKeys*

5.20.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
attributeRemoveKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <i>attributeRemoveValues</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attributeRemoveKeys table. Advice: Only the parameters passed here are processed by the function!
attributeRemoveValues	Values for the parameters selected by means of <i>attributeRemoveKeys</i> ; no input is required for the <i>ERROR_CODE</i> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <i>attributeRemoveKeys</i> ! Refer to the "Key Value Pair": attributeRemoveKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-206 Parameter (in) »*attribRemoveAttributeClassValue*«

Parameter (out)

Parameter name	Explanation
attributeRemoveResult	Depending on parameter (in) <i>attributeRemoveKeys</i> and <i>attributeRemoveValues</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>attributeRemoveKeys</i> !

Tab. 5-207 Parameter (out) »*attribRemoveAttributeClassValue*«

5.20.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-503	Failure	Container not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character

-911	Failure	Attribute not found for the serial number
-921	Failure	Attribute not found for the work order
-933	Failure	No attribute found for the container
-936	Failure	Attribute not found
-939	Failure	Attribute -> Removal failed
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

5.21 attribRemoveAttributeValue

5.21.1

Function description

This function removes one or all attribute values for a certain data object, provided that the associated attributes are referenced to the data object separately and NOT by means of an attribute class. If values for attributes with class reference are to be deleted, the API function »attribRemoveAttributeClassValue« [▶▶▶] must be used.

Refer to the following table for which data objects attribute values can be deleted on the basis of which entries.

*Function matrix ob-
jectType*

In the following function matrix, the dependencies between the parameters `objectType` or the keys `OBJECT_TYPE` (attribute category), `objectNumber` or `OBJECT_NUMBER` (data object number) and `objectDetail` or `OBJECT_DETAIL` (data object details) are mapped. Depending on the desired attribute category, a data object number that is appropriate for the attribute category is ALWAYS required. For some data objects, it is also necessary to specify an appropriate detail specification so that it can be uniquely defined.



Example

If the "serial number" data object is to be processed, the `objectType` parameter must be passed with "0". With the `objectNumber` parameter, the desired serial number is passed; with the `objectDetail` parameter, the serial number position is passed or output.

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Serial numbers	0	Serial number	Serial number position
Work orders	1	Order number (iTAC)	- not relevant -
Container	2	Container number	- not relevant -
Sites	3	Site number	- not relevant -
Lines	4	Line number	Line type (T, M) Advice: Entry is optional; if this parameter is not passed, only type [T] is taken into consideration by default!
Cells	5	Cell number	Cell type (T, M) -> see advice for 4
Machine groups	6	Machine group number	Machine group type (T, M) -> see advice for 4
Stations	7	Station number	Station type (T, M) -> see advice for 4
ERP groups	8	ERP group number	ERP group type (M, P) Advice: Entry is optional; if this parameter is not passed, types [M] and [P] are taken into consideration by default!
Part groups	9	Part group name	- not relevant -
Parts	10	Part number	- not relevant -

Data objects /	objectType OBJECT_TYPE	objectNumber OBJECT_NUMBER	objectDetail OBJECT_DETAIL
Work plans	11	WORKPLAN_ID Advice: The ID can be ascertained with »mdataGetWorkplanData« [▶▶▶]!	- not relevant -
Work steps	12	WORKPLAN_ID -> see advice for 11	Work step number
BOMs	13	BOM_ID Advice: The ID can be ascertained with »mdataGetBomData« [▶▶▶] or !	- not relevant -
BOM components	14	BOM_ID -> see advice for 13	Mounting place
Production means	15	Equipment number Cavity number Carrier number Advice: Whether the production means is equipment, a cavity, or a carrier depends on the PM type/PM class of the used part!	Equipment index PM class Carrier index PM class Example: 2 E The PM class appended with the " " (pipe) character divides the production means into equipment, cavities and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: If the number of the production means is not unique and no PM class was passed, only entries with the PM class [E] are taken into consideration!
User	16	User identification	- not relevant -
Order-based BOM components	17	Order number (iTAC)	Position number of the BOM component Advice: The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)!
Orders	18	Order designator Advice: The order designator consists of the order type and the order number (separated by ",", e.g., "BGM,1120012345")!	Order position
Receiving position	19	Receiving number	Receiving position
Storage place	20	Bar code of the storage place or – if no bar code is used – the storage place number	- not relevant -
Resource	21	Name of the resource	- not relevant -

Tab. 5-208 Function matrix objectType

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribRemoveAttributeValue
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in String attributeCode
in String attributeValueKey
```

5.21.2**Input and output parameters****Parameter (in)**

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute

Parameter name	Explanation
objectNumber	Depending on the attribute category (see <code>objectType</code>), the desired data object (e.g. the concrete serial number) is passed here. Advice: Note that, for certain attribute categories, the <code>objectDetail</code> parameter must be observed for unique determination; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the <code>objectType = 1</code> case, a serial number can be passed here instead of the order number; the work order is then ascertained on the basis of the serial number.
objectDetail	Depending on the attribute category (see <code>objectType</code>) and the specified data object (see <code>objectNumber</code>), the data-object detail specification is – where required – passed here (e.g., the serial number position if the data object is a serial number). Advice: The field is only evaluated if the processing for the listed "objectType" requires a detail specification; refer back to section »Function description« [▶▶▶] for a corresponding function matrix! Special case: For the case <code>objectType = 1</code>, the input of a serial number position may also be necessary if work order attributes are ascertained on the basis of a serial number (see special case <code>objectNumber</code>)!
attributeCode	Attribute code Special case: If the value [-1] is passed, all attribute values for all assigned attributes are deleted!
attributeValueKey	Key term for the attribute Advice: Field is only evaluated if work order attributes are to be processed (<code>objectType = 1</code>)! Special case: If the value [-1] is passed, all entries are deleted independent of the key term!

Tab. 5-209 Parameter (in) »attribRemoveAttributeValue«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

5.21.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-109	Failure	Line not found
-110	Failure	Serial no. not found
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-125	Failure	ERP group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-131	Failure	No work order found for serial number
-503	Failure	Container not found
-574	Failure	Storage location not found
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character

-911	Failure	Attribute not found for the serial number
-921	Failure	Attribute not found for the work order
-933	Failure	No attribute found for the container
-936	Failure	Attribute not found
-954	Failure	Attribute value cannot be modified
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2061	Failure	Machine group is not unique
-2105	Failure	No TR or PM license present for the station

5.22

attribUpdateAttributeCode

5.22.1

Function description

This function enables the attribute master data in the iTAC system to be processed; in this way, it is possible to change the properties of a certain attribute code (e.g. the data type), the attribute description or the relevance for certain data exchange processes. If required, this function can also be used to copy an attribute to the attribute catalog. Note, however, that only stock attributes with an assigned attribute category (OBJECT_TYPE) can be changed. Entries that exist ONLY in the attribute catalog cannot be changed using this function.



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

The attribute data can be adapted for the following data objects:

- Serial numbers (OBJECT_TYPE = 0)
- Work orders (OBJECT_TYPE = 1)
- Containers / Lots (OBJECT_TYPE = 2)
- Sites (OBJECT_TYPE = 3)
- Lines (OBJECT_TYPE = 4)
- Cells (OBJECT_TYPE = 5)
- Machine groups (OBJECT_TYPE = 6)
- Stations (OBJECT_TYPE = 7)
- ERP groups (OBJECT_TYPE = 8)
- Part groups (OBJECT_TYPE = 9)
- Parts (OBJECT_TYPE = 10)
- Work plans (OBJECT_TYPE = 11)
- Work steps (OBJECT_TYPE = 12)
- BOMs (OBJECT_TYPE = 13)
- BOM components (OBJECT_TYPE = 14)
- Production means (OBJECT_TYPE = 15)
- Users (OBJECT_TYPE = 16)
- Order-based BOM components (OBJECT_TYPE = 17)
- Orders (OBJECT_TYPE = 18)
- Receiving position (OBJECT_TYPE = 19)
- Storage places (OBJECT_TYPE = 20)
- Resources (OBJECT_TYPE = 21)

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int attribUpdateAttributeCode
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] codeKeys
in String[] codeValues
out String[] codeResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `codeKeys` and `codeValues` as well as (out) `codeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ATTRIBUTE_CODE	Attribute code (unique designator)	☒ String
AT-TRIB-UTE_CODE_APS_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
ATTRIBUTE_CODE_ERP_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process with the connected ERP system [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
AT-TRIB-UTE_CODE_SCT_TRANSFER	Indicates whether the attribute is to be taken into consideration by the data exchange process of the SCT-Service (Supply Chain Traceability) [0; 1]: 0 = Attribute is NOT taken into consideration (FALSE) 1 = Attribute is taken into consideration (TRUE)	☐ boolean
AT-TRIB-UTE_CODE_UNIQUE	Indicates whether the attribute values must be unique for this attribute [0; 1]: 0 = Attribute values do NOT need to be unique (FALSE) 1 = Attribute values must be unique (TRUE) Advice: The check for value uniqueness is performed within a site!	☐ boolean

Key	Explanation	IMSApi data type
ATTRIBUTE_DESC	Attribute designator	☐ String
CHANGE_COMMENT	Individual change comment	☐ String
COPY_TO_CATALOG	Determines whether the attribute is additionally created in the attribute catalog [0; 1]: 0 = Attribute is NOT created in the catalog (FALSE) 1 = Attribute is created in the catalog (TRUE) Advice: If this key is not passed along, the [0] value is used by default!	☐ boolean
DATA_TYPE	Data type for the attribute value: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Pay attention to upper- and lower-case letters during input! Special case: If an invalid data type is passed, the entry UNDEF is assigned; UNDEF is only valid for data records prior to release 8.50.01 and should be set to a valid value using this function!	☐ String
EDITABLE_BY	Editing release for the attribute [ANY; ERP]: ANY = Attribute can be changed via any access ERP = Attribute can be changed ONLY via the ERP service	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
OBJECT_TYPE	Determines the attribute category [0; 1; 2; 3; 4; 5; 6; 7; 8; 9; 10; 11; 12; 13; 14; 15; 16; 17; 18; 19; 20; 21]: 0 = Serial number attribute 1 = Work order attribute 2 = Container attribute 3 = Site attribute 4 = Line attribute 5 = Cell attribute 6 = Machine group attribute 7 = Station attribute 8 = ERP group attribute 9 = Part group attribute 10 = Part attribute 11 = Work plan attribute 12 = Work step attribute 13 = BOM attribute 14 = BOM component attribute 15 = Production means attribute 16 = User attribute 17 = Order-based BOM component attribute 18 = Order attribute 19 = Receiving position attribute 20 = Storage place attribute 21 = Resource attribute	☒ int

Tab. 5-210 "Key Value Pair": codeKeys

5.22.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
codeKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>codeValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": codeKeys table. Advice: Only the parameters passed here are processed by the function!
codeValues	Values for the parameters selected by means of <code>codeKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>codeKeys</code>! Refer to the "Key Value Pair": codeKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 5-211 Parameter (in) »attribUpdateAttributeCode«

Parameter (out)

Parameter name	Explanation
codeResultValues	Depending on parameter (in) <code>codeKeys</code> and <code>codeValues</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>codeKeys</code>!

Tab. 5-212 Parameter (out) »attribUpdateAttributeCode«

5.22.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-901	Failure	Attribute code not found
-905	Failure	Invalid value for "objectType"
-946	Failure	Attribute code catalog not found
-949	Failure	Attribute code -> Change not allowed
-953	Failure	Attribute -> "Edited by" - Designator is not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

6 Group "BATCH" (bin)

6.1 batchAssignBatchNumberToWorkOrder

6.1.1 Function description

This function is used to create a material bin for "bin-based traceability" and assign it to a work order. It is also possible for material bins that are already known in the system to be used again; in this case, it is important that the bin is empty and not registered at any station (see the function »batchUnregisterBatch« [▶▶▶]).

The part number of the unit which is to be manufactured with the work order is automatically assigned to the material bin. Here, the work order does not need to be active on the station. To determine the work order the bin should be assigned to, and to determine which tests are to be carried out for this, the input values `partNumber` (part), `bomVersion` (BOM version), `bomIndex` (BOM version index), `bomVersionErp` (BOM version in the ERP system) and `workOrderNumber` (work order) may be combined.

If no work order is passed by means of `workOrderNumber`, a search for a suitable work order is carried out based on the passed product data (`partNumber` and `bomVersion` or `bomIndex` and `bomVersionErp`). If no work order is found, a new work order is created. It is a condition for automatic work order creation that the work plan uses work order type "01".



Advice

This API function is influenced by station parameter code nos. 13301 [▶▶▶], 13302 [▶▶▶], 13311 [▶▶▶] and 13312 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

In the context of bin-based traceability, the »batchGetBatchInfo« [▶▶▶], »batchGetRegisteredBatch« [▶▶▶], »batchMergeBatch« [▶▶▶], »batchRegisterBatch« [▶▶▶], »batchSplitBatchNoToSerialNumber« [▶▶▶] and »batchUnregisterBatch« [▶▶▶] functions must also be taken into account!

Combinations if the order number is known:

- `partNumber`, `bomVersion`, `bomIndex`, `bomVersionErp` and `workOrderNumber`
all dependencies are checked for plausibility, the work order passed must match the part and BOM version
- `partNumber` and `workOrderNumber`
the work order passed must match the part, it is not checked whether the BOM version is correct
- `workOrderNumber`
of the bins is assigned to the work order passed without further checking

Combinations if the order number is unknown:

- `partNumber` and `bomVersion` (or `bomIndex` and `bomVersionErp`) searches for a suitable WO for the bin based on the passed part and on the BOM version; if several WOs are found, the bin is assigned to the most recent WO; if no WO is found, a new WO is created
- `partNumber` searches for a suitable WO for the bin based on the passed part; if several WOs are found, the bin is assigned to the most recent WO; if no WO is found, a new WO is created

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchAssignBatchNumberToWorkOrder
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String partNumber
in String bomVersion
in String bomIndex
in String bomVersionErp
in int processLayer
in String batchNumber
in double quantity
in int activateWorkOrder
```

6.1.2

Input and output parameters

Parameter (in)



Advice

For this function, a unique product version must be passed; this can be defined in various ways by means of the `partNumber`, `bomVersion`, `bomIndex`, `bomVersionErp` and `workOrderNumber` parameters. For example, if a product version is uniquely defined on the basis of the `workOrderNumber`, all of the other parameters mentioned above can then be passed with [-1]. A product version can also be specified by means of a `partNumber` in combination with `bomVersion` or `bomVersionErp`. Note that all passed parameters are always checked against one another; if the values do not match, the function aborts with a failure!

Parameter name	Explanation
<code>sessionContext</code>	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)

Parameter name	Explanation
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order Advice: If an empty string is passed, a suitable work order is searched for on the basis of the product data (partNumber, bomVersion, bomIndex and/or bomVersionErp); observe the advice above the table here!
partNumber	Part number in the iTAC system Advice: If this field is not to be evaluated (workOrderNumber is used), the value [-1] must be passed; for this purpose, observe the advice above the table!
bomVersion	BOM version from the iTAC system Advice: If this field is not to be evaluated (workOrderNumber and/or partNumber with bomVersionErp is used), the value [-1] must be passed; for this purpose, observe the advice above the table!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomVersionErp	BOM version in the ERP system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
processLayer	Orientation of the PCB during the work step [-1; 0; 1; 2]: -1 = if activateWorkOrder = 0 0 = component side 1 = soldering side 2 = position independent Advice: The orientation of the PCB only needs to be passed if work order activation is used (activateWorkOrder = 1)!
batchNumber	Number of the material bin (container no.) Advice: The part number of the unit which is to be manufactured with the work order is automatically assigned to the material bin!
quantity	Quantity in the material bin Advice: The value passed here determines the maximum possible number of units contained in the bin!
activateWorkOrder	Mode "Work order activation" [0; 1]: 1 = Activate work order 0 = Do not activate work order Advice: If work order activation is used (activateWorkOrder = 1), the orientation of the PCB of the current work step must be passed (see processLayer)!

Tab. 6-1 Parameter (in) »batchAssignBatchNumberToWorkOrder«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

6.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-121	Failure	Part not found
-139	Failure	BOM version does not match the version from the work order
-169	Failure	Quantity is invalid
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-219	Failure	Serial number is not finalized
-242	Failure	BOM version is invalid
-355	Failure	Production order is part of a work order cluster
-502	Failure	Failure creating the material bin
-503	Failure	Container not found

-504	Failure	An existing bin number can only be created if the old bin is empty
-505	Failure	Failure deactivating the material bin
-507	Failure	Material bin is available but empty
-508	Failure	Material bin is already stored
-531	Failure	Supplier not found
-532	Failure	Material lot is locked
-534	Failure	Material bin is already registered
-2105	Failure	No TR or PM license present for the station

6.2 batchGetBatchInfo

6.2.1 Function description

This function returns detailed information on several material bins. In addition to the general bin data, product-related data is also output (order number, part, BOM version etc.). If the bin is currently registered on a station, this information can be called up as well in combination with the time of registration.



Advice

In the context of bin-based traceability, the »batchAssignBatchNumberToWorkOrder« [▶▶▶], »batchGetRegisteredBatch« [▶▶▶], »batchMergeBatch« [▶▶▶], »batchRegisterBatch« [▶▶▶], »batchSplitBatchNoToSerialNumber« [▶▶▶] and »batchUnregisterBatch« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchGetBatchInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] batchInfoFilter
    String key
    String value
in String[] batchInfoResultKeys
out String[] batchInfoResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) batchInfoResultKeys and batchInfoFilter as well as (out) batchInfoResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
BATCH_NUMBER	Number of the material bin (container no.) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
BATCH_STATE	Bin state [0; 1]: 0 = inactive 1 = active	☐ int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
PART_NUMBER	Part number of the assigned unit	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 6-2 "Key Value Pair": *batchInfoFilter*

Definition of the output parameters for the found bins

Key	Explanation	IMSApi data type
BATCH_NUMBER	Number of the material bin (container no.)	☐ String
BATCH_QUANTITY_CURRENT	Current quantity in the material bin	☐ double
BATCH_QUANTITY_TOTAL	Original quantity in the material bin (original container quantity)	☐ double
BATCH_STATE	Bin state [0; 1]: 0 = inactive 1 = active	☐ int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
PART_NUMBER	Part number of the assigned unit	☐ String
REGISTER_DATE	Point in time at which the bin was registered at the station (registration time); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: This parameter is only output if the bin is currently registered at the station!	☐ DateMs
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This parameter is only output if the bin is currently registered at the station!	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 6-3 "Key Value Pair": *batchInfoResultKeys*

6.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
batchInfoFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": batchInfoFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": batchInfoFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
batchInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": batchInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>batchInfoResultValues</code> !

Tab. 6-4 Parameter (in) »batchGetBatchInfo«

Parameter (out)

Parameter name	Explanation
batchInfoResultValues	Depending on parameter (in) <code>batchInfoResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>batchInfoResultKeys</code> !

Tab. 6-5 Parameter (out) »batchGetBatchInfo«

6.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2116	Failure	BOM -> Type is invalid

6.3 batchGetRegisteredBatch

6.3.1 Function description

This function returns the material bins currently registered on the passed station.



Advice

In the context of bin-based traceability, the »batchAssignBatchNumberToWorkOrder« [▶▶▶], »batchGetBatchInfo« [▶▶▶], »batchMergeBatch« [▶▶▶], »batchRegisterBatch« [▶▶▶], »batchSplitBatchNoToSerialNumber« [▶▶▶] and »batchUnregisterBatch« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchGetRegisteredBatch
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String[] batchResultKeys
out String[] batchResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) batchResultKeys and (out) batchResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BATCH_NUMBER	Number of the material bin (container no.)	☐	String
BATCH_QUANTITY_CURRENT	Current quantity in the material bin	☐	double
BATCH_QUANTITY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double

Key	Explanation	IMSApi data type
BATCH_STATE	Bin state [0; 1]: 0 = inactive 1 = active	☐ int
PART_DESC	Part description of the assigned unit	☐ String
PART_NUMBER	Part number of the assigned unit	☐ String
REGISTER_DATE	Point in time at which the bin was registered at the station (registration time); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
WORKORDER_NUMBER	Production order number	☐ String

Tab. 6-6 "Key Value Pair": *batchResultKeys*

6.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
batchResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": batchResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>batchResultValues</i> !

Tab. 6-7 Parameter (in) »batchGetRegisteredBatch«

Parameter (out)

Parameter name	Explanation
batchResultValues	Depending on parameter (in) <i>batchResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>batchResultKeys</i> !

Tab. 6-8 Parameter (out) »batchGetRegisteredBatch«

6.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-240	Failure	No work step found for work order, orientation of the PCB and station
-503	Failure	Container not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

6.4 batchMergeBatch

6.4.1 Function description

This function installs the units which are contained in a material bin and which are not yet serialized either in a parent serial number (application type "from bin in serial number") or in a different bin of a parent work order (application type "from bin in bin"). Independent of the application type, a change in integration level (work order change) must always be performed when executing this function.

Application type "from bin in serial number": If the `duplicateSerialNumber` parameter has the value [0], only the reference serial numbers of two products are merged. If the value of the `duplicateSerialNumber` parameter is [1], every sub-number (single panel) is merged with a master according to a specified order. Take special notice here that scrapped serial numbers of a multiple panel can only be merged with another product if server parameter code no. 2032 [▶▶▶] is deactivated. Reason: If the parameter is active, all scrapped serial numbers are automatically deactivated, making product merging impossible.



Advice

This API function is influenced by station parameter code nos. 7206 [▶▶▶] and 7208 [▶▶▶]. In addition, the "Work order generation" station setting ("Setup" tab) must be activated. Further information on this topic can be found in the reference section of this documentation! In the context of bin-based traceability, the »batchAssignBatch-NumberToWorkOrder« [▶▶▶], »batchGetBatchInfo« [▶▶▶], »batchGetRegistered-Batch« [▶▶▶], »batchRegisterBatch« [▶▶▶], »batchSplitBatchNoToSerialNumber« [▶▶▶] and »batchUnregisterBatch« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchMergeBatch
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberRef
in String serialNumberPos
in String batchNumber
in int processLayer
in double usedBatchQuantity
in int duplicateSerialNumber
in int ignoreBatchComplete
```

6.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	"from bin in serial number": Serial number in which a non-serialized unit is to be installed; serial numbers from multiple panels can also be used here in connection with the <code>serialNumberPos</code> and <code>duplicateSerialNumber</code> parameters. "From bin in bin": Number of the material bin into which the unit is to be inserted (number of units corresponds to the <code>usedBatchQuantity</code> parameter).
serialNumberPos	Position of the single panel in the unit-array Special case: If the value [-1] is passed, the position is ascertained automatically! Advice: In the "from bin in bin" application type, a "1" must be passed here!
batchNumber	Number of the material bin (container no.) Advice: The non-serialized units are removed from this bin!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
usedBatchQuantity	"from bin in serial number": Quantity by which the bin quantity is reduced on each function call (normally "1"). Special case: If the value [-1] is passed here, the quantity is ascertained from the BOM and, if applicable, calculated using the number of panels (<code>duplicateSerialNumber = 1</code>)! "From bin in bin": Number of units which are to be taken over into the new bin
duplicateSerial-Number	Referencing to a single panel [0; 1]: 0 = Action only applies for the specified serial no. (a position of the multiple panel) 1 = Action is executed on all stored single panels on the basis of the reference serial no. to be specified Advice: In the "from bin in bin" application type, this parameter is not evaluated!
ignoreBatchComplete	Bin check [0; 1]: 0 = Incomplete material bins cannot be used 1 = Incomplete material bins can be used

Tab. 6-9 Parameter (in) »batchMergeBatch«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

6.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-155	Failure	Invalid value for "duplicateSerialNumber" [0; 1]
-223	Failure	There are multiple valid serial numbers for the merge
-503	Failure	Container not found
-506	Failure	Material bin is not yet processed at another station
-511	Failure	Material bin is not registered at the station
-512	Failure	Material bin is already registered at another station
-520	Failure	Material bin and serial number belong to another work order
-521	Failure	Material bin and serial number belong to the same work order
-2062	Failure	Bin is empty
-2105	Failure	No TR or PM license present for the station

6.5

batchRegisterBatch

6.5.1

Function description

This function registers one or several material bins at a station (see station parameter code no. 7228 [▶▶▶]). From this point in time until unregistering occurs (»batchUnregisterBatch« [▶▶▶]), the setup data at the station are assigned to the registered material bins. Unlike continuously serialized manufacturing, this condition is responsible for the larger gray area during tracking. Here, different containers for a given raw material can be set up during the bin registration and unregistration periods. It is then no longer possible to uniquely assign the units contained in the bins. During registration, a check is performed to determine whether the material bin(s) was (were) completely processed by all previous stations. Use the `ignoreBatchComplete` parameter to individually decide whether incompletely processed material bins may also be used.



Advice

This API function is influenced by station parameter code no. 7228 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

In the context of bin-based traceability, the »batchAssignBatchNumberToWorkOrder« [▶▶▶], »batchGetBatchInfo« [▶▶▶], »batchGetRegisteredBatch« [▶▶▶], »batchMergeBatch« [▶▶▶], »batchSplitBatchNoToSerialNumber« [▶▶▶] and »batchUnregisterBatch« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchRegisterBatch
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String batchNumber
in int processLayer
in int ignoreBatchComplete
in DateMs registerDate
```

6.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
batchNumber	Number of the material bin (container no.)
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
ignoreBatchComplete	Bin check [0; 1]: 0 = Incomplete material bins cannot be used 1 = Incomplete material bins can be used
registerDate	Point in time at which the bin is registered at the station (registration time); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!

Tab. 6-10 Parameter (in) »batchRegisterBatch«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

6.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-133	Failure	No active work order found for the station
		Advice: The value -133 is also output if the work order active on the station does not match the work order to which the batch belongs!
-240	Failure	No work step found for work order, orientation of the PCB and station
-503	Failure	Container not found
-506	Failure	Material bin is not yet processed at another station
-512	Failure	Material bin is already registered at another station
-513	Failure	A material bin is already registered at this station
-533	Failure	State of the material bin is invalid
-534	Failure	Material bin is already registered
-579	Failure	Max. number of registered containers exceeded
-580	Failure	Container belongs to another work order or work step
-2105	Failure	No TR or PM license present for the station

6.6 batchSplitBatchNumberToSerialNumber

6.6.1 Function description

This function copies the units which are contained in a material bin and which have not yet been serialized in serial numbers (1:1). Before doing this, the necessary serial numbers must first be created; these must match the product version of the bin (see »trAssignSerialNumberForProductOrWorkOrder« [▶▶▶]). Thus, it is possible to create "n" individual items from one bin. In doing so, all relevant information for the bin is transferred to the serial number.

Requirements for this are that the work order assigned to the bin (»batchAssignBatchNoToWorkorder« [▶▶▶]) must correspond to the work order of the serialized part and that the used serial number must already be known.



Advice

The material bin may not be registered on **any** station at the time the function call is made (unregister with »batchUnregisterBatch« [▶▶▶] if necessary). In the context of bin-based traceability, the »batchAssignBatchNumberToWorkOrder« [▶▶▶], »batchGetBatchInfo« [▶▶▶], »batchGetRegisteredBatch« [▶▶▶], »batchMergeBatch« [▶▶▶] and »batchRegisterBatch« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchSplitBatchNumberToSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
inout String serialNumberRef
inout String serialNumberPos
in String batchNumber
in int processLayer
in double usedBatchQuantity
in int duplicateSerialNumber
in int ignoreBatchComplete
```

6.6.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system

Parameter name	Explanation
batchNumber	Number of material bin (container no.); the non-serial-numbered units are removed from this bin. Advice: <i>The material bin may not be registered on ANY station at the time the function call is made (unregister with »batchUnregisterBatch« [▶▶] if necessary)!</i>
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
usedBatchQuantity	Quantity by which the bin quantity is reduced on each function call (normally "1"). Special case: If the value [-1] is passed here, the quantity is ascertained from the BOM and, if applicable, calculated using the number of panels (<code>duplicateSerialNumber = 1</code>)!
duplicateSerial- Number	Referencing to a single panel [0; 1]: 0 = Action only applies for the specified serial no. (a position of the multiple panel) 1 = Action is executed on all stored single panels on the basis of the reference serial no. to be specified
ignoreBatchCom- plete	Bin check [0; 1]: 0 = Incomplete material bins cannot be used 1 = Incomplete material bins can be used

Tab. 6-11 Parameter (in) »batchSplitBatchNumberToSerialNumber«

Parameter (inout)

Parameter name	Explanation
serialNumberRef	in: Serial number into which a non-serialized unit is to be copied; in connection with the <code>serialNumberPos</code> and <code>duplicateSerialNumber</code> parameters, serial numbers of multiple panels can also be used here. out: Serial number into which a non-serialized unit was copied; i.e. "in" value = "out" value
serialNumberPos	in: Position of the single panel in the unit-array Advice: <i>If the value [-1] is passed, the position is ascertained automatically!</i> out: Position of the single panel on the unit-array; i.e. "in" value = "out" value Advice: <i>If [-1] is passed for the "in" value, the ascertained position is output as the "out" value!</i>

Tab. 6-12 Parameter (inout) »batchSplitBatchNumberToSerialNumber«

6.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-503	Failure	Container not found
-506	Failure	Material bin is not yet processed at another station
-512	Failure	Material bin is already registered at another station
-520	Failure	Material bin and serial number belong to another work order
-521	Failure	Material bin and serial number belong to the same work order
-2105	Failure	No TR or PM license present for the station

6.7

batchUnregBatch

6.7.1

Function description

This function unregisters a material bin from a station. The `BATCH_COMPLETE` key is used to determine whether the work step was interrupted due to the material bin being logged off or whether the work step was processed fully. A state booking is performed for the material bin on each unregistration. If required, the processing time (`CYCLE_TIME`) can also be passed.

Here, it is important to note the `ignoreBatchComplete` parameter in the `»batchRegisterBatch«` [▶▶▶] function; it is used to decide in each individual case whether material bins which have not been fully processed can be logged on (used) again.



Advice

This API function is influenced by station parameter code nos. 4016 [▶▶▶] and 8500 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! In the context of bin-based traceability, the `»batchAssignBatchNumberToWorkOrder«` [▶▶▶], `»batchGetBatchInfo«` [▶▶▶], `»batchGetRegisteredBatch«` [▶▶▶], `»batchMergeBatch«` [▶▶▶], `»batchRegisterBatch«` [▶▶▶] and `»batchSplitBatchNoToSerialNumber«` [▶▶▶] functions must also be taken into account!

*For new installations, it is **essential** to use this API function because the functionality has been extended compared to the `»batchUnregisterBatch«` [▶▶▶] function!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchUnregBatch
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in KeyValue[] batchUploadKeyValues
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `batchUploadKeyValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Definition of input values for function execution

Key	Explanation		IMSApi data type
BATCH_COMPLETE	Complete processing [0; 1]: 1 = Processing fully finished 0 = Processing only interrupted	☐	int
BATCH_NUMBER	Number of the material bin (container no.)	☒	String
BATCH_QUANTITY_TO-TAL	Current quantity in the material bin Advice: <i>This quantity is captured with each function call in the iTAC system (PDC booking). In order for the quantity bookings to be evaluated in the QM module, station parameter code no. 4016 [▶▶▶] must be active!</i>	☒	double
CYCLE_TIME	Processing time at the station in seconds	☐	double
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☒	int
UNREGISTER_DATE	Point in time at which the bin is logged off at the station; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed, the current date and time are used by default!	☐	DateMs

Tab. 6-13 "Key Value Pair": *batchUploadKeyValues*

6.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
KeyValuePair	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": batchUploadKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the " Key Value Pair ": batchUploadKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 6-14 Parameter (in) »batchUnregBatch«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

6.7.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-169	Failure	Quantity is invalid
-240	Failure	No work step found for work order, orientation of the PCB and station
-503	Failure	Container not found
-511	Failure	Material bin is not registered at the station
-2012	Failure	Invalid value for "batchComplete" [0; 1]
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

6.8 batchUnregisterBatch

6.8.1 Function description

This function unregisters a material bin from a station. The `batchComplete` parameter is used to determine whether the work step was interrupted due to the material bin being logged off or whether the work step was processed fully. A state booking is performed for the material bin on each unregistration.

Here, it is important to note the `ignoreBatchComplete` parameter in the »batchRegisterBatch« [▶▶▶] function; it is used to decide in each individual case whether material bins which have not been fully processed can be logged on (used) again.



Advice

This API function is influenced by station parameter code nos. 4016 [▶▶▶] and 8500 [▶▶▶] as well as server parameter code no. 19200 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! In the context of bin-based traceability, the functions »batchAssignBatchNumberToWorkOrder« [▶▶▶], »batchGetBatchInfo« [▶▶▶], »batchGetRegisteredBatch« [▶▶▶], »batch-MergeBatch« [▶▶▶], »batchRegisterBatch« [▶▶▶] and »batchSplitBatchNoToSerial-Number« [▶▶▶] must also be taken into account!

For new installations, this function should NO LONGER be used because the edit options have been extended with release 8.52.00 (for further information, see the new function »batchUnregBatch« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int batchUnregisterBatch
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String batchNumber
  in int processLayer
  in double batchTotalQuantity
  in int batchComplete
  in DateMs unregisterDate
```

6.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
batchNumber	Number of the material bin (container no.)
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
batchTotalQuantity	Current quantity in the material bin Advice: This quantity is captured with each function call in the iTAC system (PDC booking). In order for the quantity bookings to be evaluated in the QM module, station parameter code no. 4016 [▶▶▶] must be active!
batchComplete	Complete processing [0; 1]: 1 = Processing fully finished 0 = Processing only interrupted
unregisterDate	Point in time at which the bin is logged off at the station; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!

Tab. 6-15 Parameter (in) »batchUnregisterBatch«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

6.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-130	Failure	Work order not found
-169	Failure	Quantity is invalid
-240	Failure	No work step found for work order, orientation of the PCB and station
-503	Failure	Container not found
-511	Failure	Material bin is not registered at the station
-2012	Failure	Invalid value for "batchComplete" [0; 1]
-2105	Failure	No TR or PM license present for the station

7 Group "CONFIG" (configuration)

7.1 configCreateParameters

7.1.1 Function description

This function makes it possible to create configuration parameters in the Config-Framework.



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function declaration

```
int configCreateParameters
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in KeyValue[] options
    String key
    String value
in String[] parameterUploadKeys
in String[] parameterUploadValues
in String[] parameterResultKeys
out String[] parameterResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) options, parameterUploadKeys, parameterUploadValues and parameterResultKeys as well as (out) parameterResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Definition of the input parameters for the parameter creation

Key	Explanation	IMSApi data type
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String
PARAMETER_HELP	Help text for the parameter	☐ String
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_NAME key can also be used for passing along the metatype. The following values are possible: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (the decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (the decimal character "." is prefixed) Advice: This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP])!	☐ long

Key	Explanation		IMSApi data type
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_ID key can also be used for passing along the metatype. The following values are possible: - STRING - INTEGER - LONG - DOUBLE (the decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (the decimal character "." is prefixed) Advice: This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP])!	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height") Advice: The parameter names passed here are automatically prefixed by the hierarchy level "Customer."; this method is necessary for explicit data storage!	☐	String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set) Advice: If more values are to be passed, they must be separated by commas!	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting Advice: If no sequence has been specified, sorting is performed automatically according to the PARAMETER_NAME key!	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level (for generating directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_NAME key can also be used.	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level (for generating directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_ID key can also be used. Advice: The parameter names passed here are automatically prefixed by the hierarchy level "Customer."; this method is necessary for explicit data storage!	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String

Key	Explanation		IMSApi data type
PARAMETER_TYPE_ID	Internal identification number of the parameter type; alternatively, the <code>PARAMETER_TYPE_NAME</code> key can also be used for passing along the parameter type. The following values are possible: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed) Advice: If the value [-1] or [-2] is passed along here, the metatype must be determined for the further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>)!	☐	long
PARAMETER_TYPE_NAME	Name of the parameter type; alternatively, the <code>PARAMETER_TYPE_ID</code> key can also be used for passing along the parameter type. The following values are possible: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed) Advice: If the value [LIST] or [MAP] is passed along here, the metatype must be determined for further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>)!	☐	String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐	String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐	String

Tab. 7-1 "Key Value Pair": `parameterUploadKeys`

Definition of the output parameters for the created configuration parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String
PARAMETER_HELP	Help text for the parameter	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐ long

Key	Explanation		IMSApi data type
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter: - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐	String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set)	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String
PARAMETER_TYPE_ID	Internal identification number of the parameter type: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐	long

Key	Explanation	IMSApi data type
PARAMETER_TYPE_NAME	Name of the parameter type: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐ String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐ String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐ String

Tab. 7-2 "Key Value Pair": *parameterResultKeys*

7.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
parameterUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>parameterUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": parameterUploadKeys table.
parameterUploadValues	Value list for the parameters passed by means of <code>parameterUploadKeys</code> Advice: Refer to the "Key Value Pair": parameterUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterResultKeys	List of the parameters that are to be output as a result by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterResultKeys table. Advice: Only the values of the parameters passed here are output by the function; see parameter (out) <code>parameterResultValues</code> !

Tab. 7-3 Parameter (in) »configCreateParameters«

Parameter (out)

Parameter name	Explanation
parameterResultValues	Depending on parameter (in) <code>parameterResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>parameterResultKeys</code> !

Tab. 7-4 Parameter (out) »configCreateParameters«

7.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3301	Failure	Parameter already exists
-3302	Failure	Parameter not found
-3304	Failure	Configuration type not found

7.2 configDeleteParameters

7.2.1 Function description

With this function you can delete configuration parameters from the ConfigFramework; if a parameter is deleted, its stored values are also deleted.



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function declaration

```

int configDeleteParameters
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in KeyValue[] options
    String key
    String value
in KeyValue[] parameterFilter
    String key
    String value
in String[] parameterResultKeys
out String[] parameterResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `options`, `parameterFilter` and `parameterResultKeys` as well as (out) `parameterResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Filter for the parameters to be deleted

Key	Explanation	IMSApi data type
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height") Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level (for deleting complete directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_NAME key can also be used.	☐ long
PARAMETER_PARENT_NAME	Name of the parent parameter level (for deleting complete directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_ID key can also be used.	☐ String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐ String

Tab. 7-5 "Key Value Pair": `parameterFilter`

Definition of the output parameters for deleted configuration parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String
PARAMETER_HELP	Help text for the parameter	☐ String

Key	Explanation		IMSApi data type
PARAMETER_ID	Internal identification number of the parameter	☐	long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐	long
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter: - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐	String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set)	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String

Key	Explanation	IMSApi data type
PARAMETER_TYPE_ID	Internal identification number of the parameter type: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐ long
PARAMETER_TYPE_NAME	Name of the parameter type: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐ String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐ String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐ String

Tab. 7-6 "Key Value Pair": *parameterResultKeys*

7.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
parameterFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": parameterFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": parameterFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) parameterResultValues!

Tab. 7-7 Parameter (in) »configDeleteParameters«

Parameter (out)

Parameter name	Explanation
parameterResultValues	Depending on parameter (in) parameterResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) parameterResultKeys!

Tab. 7-8 Parameter (out) »configDeleteParameters«

7.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3303	Failure	Validity range not found
-3304	Failure	Configuration type not found
-3305	Failure	Parameter filter not unique
-3313	Failure	Parameter could not be deleted
-3314	Failure	Authorization for parameter is missing

7.3

configDeleteValues

7.3.1

Function description

This function makes it possible to delete the values for selected configuration parameters. Per function call, only the parameter values of a specific context can be deleted (see `configContext`).

**Advice**

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function declaration

```
int configDeleteValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
KeyValue[] options
    String key
    String value
KeyValue[] configContext
    String key
    String value
in String[] uploadKeys
in String[] uploadValues
in String[] resultKeys
out String[] resultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `options`, `configContext`, `uploadKeys` and `resultKeys` as well as (out) `resultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Specifications for the context (storage location) of the values to be deleted

Key	Explanation	IMSApi data type
CONFIG_APPID	Dimension module: application ID	☐ String
CONFIG_APPTYPE	Dimension module: application type	☐ String
CONFIG_CLUSTER	Dimension module: server-node network	☐ String
CONFIG_GROUP	Dimension module: user group	☐ String
CONFIG_HOST	Dimension module: computer name	☐ String
CONFIG_IDENTITY	Dimension module: identifier	☐ String
CONFIG_ROLE	Dimension module: User role	☐ String
CONFIG_SITE	Dimension module: site	☐ String
CONFIG_STATION	Dimension module: station	☐ String
CONFIG_USER	Dimension module: user	☐ String

Tab. 7-9 "Key Value Pair": *configContext*

Definition of the configuration parameters whose values are to be deleted

Key	Explanation	IMSApi data type
CONFIG_CONTEXT_ID	Internal identification number for a dimension path to a parameter value Advice: This key can be used as an alternative to the key CONFIG_DIMSET!	☐ long
CONFIG_DIMSET	Dimension path to the parameter value; the definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: CLUSTER,APPTYPE; the values for the modules must be passed along by means of the <i>configContext</i> structure.	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String

Tab. 7-10 "Key Value Pair": *uploadKeys*

Definition of the output parameters for the deleted parameter values

Key	Explanation	IMSApi data type
CONFIG_DIMSET	Dimension path for the parameter value	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_INDEX	Unique reference number (sequential number) for each record in the array (beginning with "0")	☐ int
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String

Tab. 7-11 "Key Value Pair": *resultKeys*

7.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
configContext	"Key Value Pair" structure with the following variables:
key	List of the parameters for the context; the corresponding values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": configContext table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": configContext table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
uploadKeys	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the uploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": uploadKeys table.
uploadValues	Value list for the parameters passed by means of uploadKeys. Advice: Refer to the "Key Value Pair": uploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
resultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": resultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) resultValues!

Tab. 7-12 Parameter (in) »configDeleteValues«

Parameter (out)

Parameter name	Explanation
resultValues	Depending on parameter (in) resultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) resultKeys!

Tab. 7-13 Parameter (out) »configDeleteValues«

7.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3305	Failure	Parameter filter not unique
-3307	Failure	Parameter value not found
-3308	Failure	Parameter value not unique
-3309	Failure	Parameter key is missing

7.4 configGetDimensionValues

7.4.1 Function description

This function makes it possible to ascertain values for a specific dimension module – e.g. the application ID (APPID).

Function declaration

```
int configGetDimensionValues
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in KeyValue[] options
    String key
    String value
  in KeyValue[] dimensionFilter
    String key
    String value
  out String[] resultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `options` and `dimensionFilter`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
CONFIG_APPID	Dimension module: application ID Advice: If this key is used, the other keys may not be passed along! During the input, the wildcard "*" can be used; NO distinction is made between upper case and lower case	☐ String
CONFIG_APPTYPE	Dimension module: application type Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_CLUSTER	Dimension module: server-node network Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_GROUP	Dimension module: user group Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_HOST	Dimension module: computer name Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_IDENTITY	Dimension module: identifier Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String

Key	Explanation	IMSApi data type
CONFIG_ROLE	Dimension module: User role Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_SITE	Dimension module: site Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_STATION	Dimension module: station Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String
CONFIG_USER	Dimension module: user Advice: The advice for the CONFIG_APPID key is also valid here!	☐ String

Tab. 7-14 "Key Value Pair": *dimensionFilter*

7.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
dimensionFilter	"Key Value Pair" structure with the following variables:
key	A parameter (dimension module), on whose basis the function is to be executed; the associated value is passed along with the parameter value. A description of the possible parameters can be found above in the "Key Value Pair": dimensionFilter table.
value	Value for the parameter passed by means of key. Advice: Refer to the "Key Value Pair": dimensionFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 7-15 Parameter (in) »configGetDimensionValues«

Parameter (out)

Parameter name	Explanation
resultValues	Depending on parameter (in) dimensionFilter , the found data is output here. Advice: The data are output in the sequence in which they were ascertained by the function!

Tab. 7-16 Parameter (out) »configGetDimensionValues«

7.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3311	Failure	Dimension filter is missing
-3312	Failure	Only one dimension allowed as filter criterion

7.5 configGetParameters

7.5.1 Function description

With this function, specific configuration parameters from the ConfigFramework can be ascertained in a focused manner.

Function declaration

```

int configGetParameters
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
    KeyValue[] options
    String key
    String value
    KeyValue[] parameterFilter
    String key
    String value
  in String[] parameterResultKeys
  out String[] parameterResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) options, parameterFilter and parameterResultKeys as well as (out) parameterResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameters

Key	Explanation	IMSApi data type
PARAMETER_ID	Internal identification number of the parameter Advice: <i>If this key is used, the keys PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE may NOT be passed!</i>	☐ long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_NAME key can also be used for passing along the metatype. The following values are possible: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (the decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (the decimal character "." is prefixed) This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP]). Advice: <i>With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!</i>	☐ long
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_ID key can also be used for passing along the metatype. The following values are possible: - STRING - INTEGER - LONG - DOUBLE (the decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (the decimal character "." is prefixed) This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP]). Advice: <i>With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!</i>	☐ String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height") Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String

Key	Explanation	IMSApi data type
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level (for ascertaining complete directories in the ConfigFramework). Advice: <i>If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_NAME and PARAMETER_SCOPE may NOT be passed along!</i>	☐ long
PARAMETER_PARENT_NAME	Name of the parent parameter level (for ascertaining complete directories in the ConfigFramework). Advice: <i>If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID and PARAMETER_SCOPE may NOT be passed along!</i>	☐ String
PARAMETER_SCOPE	Validity range (APPTYPE) Advice: <i>If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID and PARAMETER_PARENT_NAME may NOT be passed along!</i>	☐ String
PARAMETER_TYPE_ID	Internal identification number of the parameter type; alternatively, the PARAMETER_TYPE_NAME key can also be used for passing along the parameter type. The following values are possible: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed) If the value [-1] or [-2] is passed along, the metatype must be determined for the further specification of the parameter (for further information, see PARAMETER_METATYPE_ID and PARAMETER_METATYPE_NAME). Advice: <i>With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!</i>	☐ long

Key	Explanation	IMSApi data type
PARAMETER_TYPE_NAME	Name of the parameter type; alternatively, the PARAMETER_TYPE_ID key can also be used for passing along the parameter type. The following values are possible: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed) If the value [LIST] or [MAP] is passed along, the metatype must be determined for the further specification of the parameter (for further information, see PARAMETER_METATYPE_ID and PARAMETER_METATYPE_NAME). Advice: With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!	☐ String

Tab. 7-17 "Key Value Pair": parameterFilter

Definition of the output parameters for the found parameter

Key	Explanation	IMSApi data type
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String

Key	Explanation		IMSApi data type
PARAMETER_HELP	Help text for the parameter	☐	String
PARAMETER_ID	Internal identification number of the parameter	☐	long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐	long
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter: - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐	String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set)	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String

Key	Explanation	IMSApi data type
PARAMETER_TYPE_ID	Internal identification number of the parameter type: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐ long
PARAMETER_TYPE_NAME	Name of the parameter type: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐ String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐ String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐ String

Tab. 7-18 "Key Value Pair": *parameterResultKeys*

7.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
parameterFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": parameterFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": parameterFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterResultKeys table. Advice: Only the parameters passed here are output by the function; see <i>parameter (out)</i> parameterResultValues !

Tab. 7-19 Parameter (in) »configGetParameters«

Parameter (out)

Parameter name	Explanation
parameterResultValues	Depending on parameter (in) parameterResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of <i>parameter (in)</i> parameterResultKeys !

Tab. 7-20 Parameter (out) »configGetParameters«

7.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3303	Failure	Validity range not found
-3304	Failure	Configuration type not found
-3305	Failure	Parameter filter not unique
-3315	Failure	Parameter filter is missing

7.6 configGetValues

7.6.1 Function description

This function makes it possible to output the values for selected configuration parameters. Per function call, only the parameter values of a specific context can be ascertained (see `configContext`).

Function declaration

```
int configGetValues
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in KeyValue[] options
    String key
    String value
in KeyValue[] configContext
    String key
    String value
in KeyValue[] parameterFilter
    String key
    String value
in String[] parameterResultKeys
out String[] parameterResultValues
in String[] resultKeys
out String[] resultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `options`, `configContext`, `parameterFilter`, `parameterResultKeys` and `resultKeys`, as well as (out) `parameterResultValues` and `resultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Specifications for the context (storage location) of the values to be ascertained

Key	Explanation		IMSApi data type
CONFIG_APPID	Dimension module: application ID	☐	String
CONFIG_APPTYPE	Dimension module: application type	☐	String
CONFIG_CLUSTER	Dimension module: server-node network	☐	String
CONFIG_GROUP	Dimension module: user group	☐	String

Key	Explanation	IMSApi data type
CONFIG_HOST	Dimension module: computer name	☐ String
CONFIG_IDENTITY	Dimension module: identifier	☐ String
CONFIG_ROLE	Dimension module: User role	☐ String
CONFIG_SITE	Dimension module: site	☐ String
CONFIG_STATION	Dimension module: station	☐ String
CONFIG_USER	Dimension module: user	☐ String

Tab. 7-21 "Key Value Pair": *configContext*

Search for specific parameters to ascertain values

Key	Explanation	IMSApi data type
PARAMETER_ID	Internal identification number of the parameter Advice: <i>If this key is used, the keys PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE may NOT be passed!</i>	☐ long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_NAME key can also be used for passing along the metatype. The following values are possible: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (the decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (the decimal character "." is prefixed) This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP]). Advice: <i>With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!</i>	☐ long

Key	Explanation		IMSApi data type
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_ID key can also be used for passing along the metatype. The following values are possible: - STRING - INTEGER - LONG - DOUBLE (the decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (the decimal character "." is prefixed) This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP]). Advice: With this, the result set ascertained based on the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID, PARAMETER_PARENT_NAME and PARAMETER_SCOPE will be further limited ("AND" linked)!	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height") Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level (for ascertaining complete directories in the ConfigFramework). Advice: If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_NAME and PARAMETER_SCOPE may NOT be passed along!	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level (for ascertaining complete directories in the ConfigFramework). Advice: If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID and PARAMETER_SCOPE may NOT be passed along!	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE) Advice: If this key is used, the keys PARAMETER_ID, PARAMETER_NAME, PARAMETER_PARENT_ID and PARAMETER_PARENT_NAME may NOT be passed along!	☐	String

Key	Explanation	IMSApi data type
PARAMETER_TYPE_ID	Internal identification number of the parameter type; alternatively, the <code>PARAMETER_TYPE_NAME</code> key can also be used for passing along the parameter type. The following values are possible: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed) If the value [-1] or [-2] is passed along, the metatype must be determined for the further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>). Advice: <i>With this, the result set ascertained based on the keys <code>PARAMETER_ID</code>, <code>PARAMETER_NAME</code>, <code>PARAMETER_PARENT_ID</code>, <code>PARAMETER_PARENT_NAME</code> and <code>PARAMETER_SCOPE</code> will be further limited ("AND" linked)!</i>	☐ long
PARAMETER_TYPE_NAME	Name of the parameter type; alternatively, the <code>PARAMETER_TYPE_ID</code> key can also be used for passing along the parameter type. The following values are possible: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed) If the value [LIST] or [MAP] is passed along, the metatype must be determined for the further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>). Advice: <i>With this, the result set ascertained based on the keys <code>PARAMETER_ID</code>, <code>PARAMETER_NAME</code>, <code>PARAMETER_PARENT_ID</code>, <code>PARAMETER_PARENT_NAME</code> and <code>PARAMETER_SCOPE</code> will be further limited ("AND" linked)!</i>	☐ String

Tab. 7-22 "Key Value Pair": `parameterFilter`

Definition of the output parameters for the found parameter

Key	Explanation		IMSApi data type
PARAMETER_COMMENT	Comments on the parameter	☐	String
PARAMETER_DESCRIPTION	Description of the parameter	☐	String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐	String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐	String
PARAMETER_HELP	Help text for the parameter	☐	String
PARAMETER_ID	Internal identification number of the parameter	☐	long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐	long
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter: - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐	String

Key	Explanation		IMSApi data type
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set)	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String
PARAMETER_TYPE_ID	Internal identification number of the parameter type: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐	long
PARAMETER_TYPE_NAME	Name of the parameter type: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐	String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐	String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐	String

Tab. 7-23 "Key Value Pair": *parameterResultKeys*

Definition of the output parameters for the found parameter values

Key	Explanation	IMSApi data type
CONFIG_DIMSET	Dimension path for the parameter value	☐ String
CONFIG_KEY	Parameter key Advice: Parameter keys are used in cases where the parameter can contain a list of values; the parameter key identifies the respective list item (in the configuration framework, this is then the entry in the "#" columnn)!	☐ String
CONFIG_VALUE	Parameter value	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_INDEX	Unique reference number (sequential number) for each record in the array (beginning with "0")	☐ int
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String

Tab. 7-24 "Key Value Pair": *resultKeys*

7.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
configContext	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": configContext table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": configContext table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": parameterFilter table.

Parameter name	Explanation
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": parameterFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) parameterResultValues!
resultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": resultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) resultValues!

Tab. 7-25 Parameter (in) »configGetValues«

Parameter (out)

Parameter name	Explanation
parameterResult-Values	Depending on parameter (in) parameterResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) parameterResultKeys!
resultValues	Depending on parameter (in) resultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) resultKeys!

Tab. 7-26 Parameter (out) »configGetValues«

7.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3303	Failure	Validity range not found
-3304	Failure	Configuration type not found
-3305	Failure	Parameter filter not unique
-3315	Failure	Parameter filter is missing

7.7

configUpdateValues

7.7.1

Function description

This function makes it possible to create or adjust values for selected configuration parameters. Per function call, only the parameter values of a specific context can be updated (see `configContext`).



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function declaration

```
int configUpdateValues
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in KeyValue[] options
    String key
    String value
  in KeyValue[] configContext
    String key
    String value
  in String[] uploadKeys
  in String[] uploadValues
  in String[] resultKeys
  out String[] resultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `options`, `configContext`, `uploadKeys` and `resultKeys` as well as (out) `resultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Specifications for the context (storage location) of the values to be updated

Key	Explanation	IMSApi data type
CONFIG_APPID	Dimension module: application ID	☐ String
CONFIG_APPTYPE	Dimension module: application type	☐ String
CONFIG_CLUSTER	Dimension module: server-node network	☐ String
CONFIG_GROUP	Dimension module: user group	☐ String
CONFIG_HOST	Dimension module: computer name	☐ String
CONFIG_IDENTITY	Dimension module: identifier	☐ String
CONFIG_ROLE	Dimension module: User role	☐ String
CONFIG_SITE	Dimension module: site	☐ String
CONFIG_STATION	Dimension module: station	☐ String
CONFIG_USER	Dimension module: user	☐ String

Tab. 7-27 "Key Value Pair": *configContext*

Definition of the configuration parameters whose values are to be updated

Key	Explanation	IMSApi data type
CONFIG_DIMSET	Dimension path to the parameter value; the definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: <i>CLUSTER,APPTYPE</i> ; the values for the modules must be passed along by means of the <i>configContext</i> structure.	☐ String
CONFIG_KEY	Parameter key Advice: <i>Parameter keys are used in cases where the parameter can contain a list of values; the parameter key identifies the respective list item (in the configuration framework, this is then the entry in the "#" column)!</i>	☐ String
CONFIG_VALUE	Parameter value	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String

Tab. 7-28 "Key Value Pair": *uploadKeys*

Definition of the output parameters for the updated parameter values

Key	Explanation	IMSApi data type
CONFIG_DIMSET	Dimension path for the parameter value	☐ String
CONFIG_KEY	Parameter key Advice: <i>Parameter keys are used in cases where the parameter can contain a list of values; the parameter key identifies the respective list item (in the configuration framework, this is then the entry in the "#" column)!</i>	☐ String
CONFIG_VALUE	Parameter value	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒ int
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_INDEX	Unique reference number (sequential number) for each record in the array (beginning with "0")	☐ int
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String

Tab. 7-29 "Key Value Pair": *resultKeys*

7.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
configContext	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": configContext table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": configContext table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
uploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the uploadValues parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": uploadKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!
uploadValues	Values for the parameters selected by means of uploadKeys Advice: The data is assigned in the order in which it was entered by means of parameter (in) uploadKeys! Refer to the corresponding table for attribute category "Key Value Pair": "uploadKeys" to see whether special rules apply during the input of values (e.g. use of wildcards)!
resultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": resultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) resultValues!

Tab. 7-30 Parameter (in) »configUpdateValues«

Parameter (out)

Parameter name	Explanation
resultValues	Depending on parameter (in) resultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) resultKeys!

Tab. 7-31 Parameter (out) »configUpdateValues«

7.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3305	Failure	Parameter filter not unique
-3307	Failure	Parameter value not found
-3308	Failure	Parameter value not unique
-3309	Failure	Parameter key is missing
-3316	Failure	Parameter value has invalid format

7.8 configUpdateParameters

7.8.1 Function description

This function makes it possible to update configuration parameters in the Config-Framework.



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function declaration

```

int configUpdateParameters
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in KeyValue[] options
    String key
    String value
in String[] parameterUploadKeys
in String[] parameterUploadValues
in String[] parameterResultKeys
out String[] parameterResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) options, parameterUploadKeys and parameterResultKeys as well as (out) parameterResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Definition of the input parameters for the parameter update

Key	Explanation	IMSApi data type
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String
PARAMETER_HELP	Help text for the parameter	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter; alternatively, the <code>PARAMETER_METATYPE_NAME</code> key can also be used for passing along the metatype. The following values are possible: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (the decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (the decimal character "." is prefixed) Advice: This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for <code>PARAMETER_TYPE_ID</code> the values [-1] or [-2] or for <code>PARAMETER_TYPE_NAME</code> the values [LIST] or [MAP])!	☐ long

Key	Explanation		IMSApi data type
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter; alternatively, the PARAMETER_METATYPE_ID key can also be used for passing along the metatype. The following values are possible: - STRING - INTEGER - LONG - DOUBLE (the decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (the decimal character "." is prefixed) Advice: This parameter will only be interpreted if "List" or "Mapping" have been selected as the parameter type (for further information, see for PARAMETER_TYPE_ID the values [-1] or [-2] or for PARAMETER_TYPE_NAME the values [LIST] or [MAP])!	☐	String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height") Advice: The parameter names passed here are automatically prefixed by the hierarchy level "Customer."; this method is necessary for explicit data storage!	☐	String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set) Advice: If more values are to be passed, they must be separated by commas!	☐	String
PARAMETER_ORDERING	Parameter sequence for the individual sorting Advice: If no sequence has been specified, sorting is performed automatically according to the PARAMETER_NAME key!	☐	String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level (for generating directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_NAME key can also be used.	☐	long
PARAMETER_PARENT_NAME	Name of the parent parameter level (for generating directories in the ConfigFramework); alternatively, the PARAMETER_PARENT_ID key can also be used. Advice: The parameter names passed here are automatically prefixed by the hierarchy level "Customer."; this method is necessary for explicit data storage!	☐	String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐	String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐	String

Key	Explanation		IMSApi data type
PARAMETER_TYPE_ID	Internal identification number of the parameter type; alternatively, the <code>PARAMETER_TYPE_NAME</code> key can also be used for passing along the parameter type. The following values are possible: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed) Advice: If the value [-1] or [-2] is passed along here, the metatype must be determined for the further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>)!	☐	long
PARAMETER_TYPE_NAME	Name of the parameter type; alternatively, the <code>PARAMETER_TYPE_ID</code> key can also be used for passing along the parameter type. The following values are possible: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed) Advice: If the value [LIST] or [MAP] is passed along here, the metatype must be determined for further specification of the parameter (for further information, see <code>PARAMETER_METATYPE_ID</code> and <code>PARAMETER_METATYPE_NAME</code>)!	☐	String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐	String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐	String

Tab. 7-32 "Key Value Pair": `parameterUploadKeys`

Definition of the output parameters for the updated configuration parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒ int
PARAMETER_COMMENT	Comments on the parameter	☐ String
PARAMETER_DESCRIPTION	Description of the parameter	☐ String
PARAMETER_DIMPATH	Dimension path(s) to the parameter value; the individual path specifications (storage locations) are separated by commas. The definition of a dimension path is always based on the following dimension modules: - CLUSTER - APPTYPE - APPID - HOST - USER - GROUP - ROLE - SITE - STATION - IDENTITY Example: {CLUSTER,APPTYPE},{CLUSTER,APPTYPE,APPID},{CLUSTER,APPTYPE,APPID,STATION} – a parameter created in this manner may include values at the APPTYPE, APPID or STATION dimension levels.	☐ String
PARAMETER_DISPLAYNAME	Display name of the parameter in the configuration	☐ String
PARAMETER_HELP	Help text for the parameter	☐ String
PARAMETER_ID	Internal identification number of the parameter	☐ long
PARAMETER_META- TYPE_ID	Internal identification number of the metatype for the parameter: 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐ long

Key	Explanation	IMSApi data type
PARAMETER_META- TYPE_NAME	Name of the metatype for the parameter: - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐ String
PARAMETER_NAME	Parameter name (internal); hierarchy levels are separated by "." (e.g. "CustomClient.height")	☐ String
PARAMETER_OPTIONS	Optional settings for the parameter; the following options are available: NO_HISTORY = parameter values will NOT be historicized IMSAPI = parameter was created by means of IMSApi function (is automatically set)	☐ String
PARAMETER_ORDERING	Parameter sequence for the individual sorting	☐ String
PARAMETER_PARENT_ID	Internal identification number of the parent parameter level	☐ long
PARAMETER_PARENT_NAME	Name of the parent parameter level	☐ String
PARAMETER_PROPERTIES	Additional characteristics of the parameter	☐ String
PARAMETER_SCOPE	Validity range (APPTYPE)	☐ String
PARAMETER_TYPE_ID	Internal identification number of the parameter type: -1 = LIST (list of types) -2 = MAP (mapping of STRING to types) 0 = VOID (for the directory) 1 = STRING 2 = INTEGER 3 = LONG 4 = DOUBLE (decimal character "." is prefixed) 5 = BOOLEAN 6 = TEXT (multiple-line text editor) 12 = PROPERTIES 15 = BYTE 16 = SHORT 17 = CHARACTER 18 = FLOAT (decimal character "." is prefixed)	☐ long

Key	Explanation	IMSApi data type
PARAMETER_TYPE_NAME	Name of the parameter type: - LIST (list of types) - MAP (mapping of STRING to types) - VOID (for the directory) - STRING - INTEGER - LONG - DOUBLE (decimal character "." is prefixed) - BOOLEAN - TEXT (multiple-line text editor) - PROPERTIES - BYTE - SHORT - CHARACTER - FLOAT (decimal character "." is prefixed)	☐ String
PARAMETER_VISIBILITY_CHECKER	Verification of the visibility setting (for internal use only)	☐ String
PARAMETER_VISIBILITY_PROPERTIES	Characteristics of the visibility setting (for internal use only)	☐ String

Tab. 7-33 "Key Value Pair": *parameterResultKeys*

7.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
options	Advice: This structure is intended for future function extensions; currently, its contents are not evaluated but must be passed along as an EMPTY structure!
key	- not currently used -
value	- not currently used -
parameterUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the parameterUploadValues parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": parameterUploadKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!
parameterUploadValues	Values for the parameters selected by means of parameterUploadKeys. Advice: The data is assigned in the order in which it was entered by means of parameter (in) parameterUploadKeys! Refer to the corresponding table for attribute category "Key Value Pair": " parameterUploadKeys " to see whether special rules apply during the input of values (e.g. use of wildcards)!
parameterResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) parameterResultValues!

Tab. 7-34 Parameter (in) »configUpdateParameters«

Parameter (out)

Parameter name	Explanation
parameterResultValues	Depending on parameter (in) parameterResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) parameterResultKeys!

Tab. 7-35 Parameter (out) »configUpdateParameters«

7.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-3300	Failure	Dimension is unknown
-3302	Failure	Parameter not found
-3304	Failure	Configuration type not found
-3305	Failure	Parameter filter not unique
-3314	Failure	Authorization for parameter is missing
-3315	Failure	Parameter filter is missing

8 Group "CRP" (continuous replenishment)

8.1 crpChangeTransportOrder

8.1.1 Function description

This function allows the adaptation of transport orders which already exist in the system. Here, not only the header data of a transport order can be changed, but also the picking and commissioning lists as well as the assignments of production orders. Depending on the respective function mode for the data to be transferred (see key `UPDATE_FLAG`), production orders can be assigned or removed and picking and commissioning lists can be added, updated or deleted.

If new transport orders are to be created, the IMSApi function »`crpCreateTransportOrder`« [▶▶▶] must be used.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

When processing transport orders, observe the following basic principles:

- The assignment of production orders, picking lists and commissioning lists is optional.
- A transport order can be assigned one or more production orders.
- A transport order can be assigned one or more picking lists.
- A picking list can be assigned one or more commissioning lists; without a picking list, it is NOT possible to assign a commissioning list to a transport order.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	CRP-Service (basic): Site license for a certain number of stop points
--	--

Function declaration

```
int crpChangeTransportOrder
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] transportOrderUploadKeyValues
    String key
    String value
in String[] productionOrderUploadKeys
in String[] productionOrderUploadValues
out String[] productionOrderUploadResult
in String[] pickingItemUploadKeys
in String[] pickingItemUploadValues
out String[] pickingItemUploadResult
in String[] commissionItemUploadKeys
in String[] commissionItemUploadValues
out String[] commissionItemUploadResult
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) transportOrderUploadKeyValues, productionOrderUploadKeys, pickingItemUploadKeys and commissionItemUploadKeys as well as (out) productionOrderUploadResult, pickingItemUploadResult and commissionItemUploadResult). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for the transport order to be changed (header data)

Key	Explanation		IMSApi data type
COMMENT	Individual comment on transport order	☐	String
DESCRIPTION	Description of the transport order	☐	String
EARLIEST_DELIVERY_DATE	Earliest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not used, no time stamp is passed; depending on the function mode (see key UPDATE_FLAG), the corresponding field remains empty or unchanged!	☐	DateMs
EARLIEST_START_DATE	Earliest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐	DateMs
LATEST_DELIVERY_DATE	Latest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐	DateMs
LATEST_START_DATE	Latest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐	DateMs
PRIORITY_CODE	Unique designator for the work order priority	☐	String
STATE_COMMENT	Individual comment regarding the current transport order state (see key TRANSPORT_ORDER_STATE)	☐	String

Key	Explanation	IMSApi data type
STOP_POINT_ACTUAL	Name of the stop point which defines the current location of the transport order (see the advice notice under the key STOP_POINT_DESTINATION). Advice: If the key is not passed, the following state applies for the transport order: "not yet arrived at a stop point". If a "current location" has already been assigned, it is reset!	☐ String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the transport order. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐ String
TRANSPORT_ORDER_NUMBER	Transport order number Advice: The transport order specified here must exist in the system; the number cannot be changed using this function!	☒ String
TRANSPORT_ORDER_STATE	"Transport order" state [I; RO; O; C; S; RT; T; R; RU; D; C]: IC = IN_CREATION -> Being created RO = READY_TO_ORDER -> Ready for ordering O = ORDERED -> Requested I = IN_COMMISSION -> Being commissioned S = STOPPED -> Withheld RT = READY_TO_TRANSFER -> Ready for collection IT = IN_TRANSFER -> Being delivered R = RECEIVED -> At receiving RU = READY_TO_UNLOAD -> Ready for unloading D = DELIVERED -> Delivered C = CANCELED -> Canceled Advice: With state changes, observe the following list with the permitted state transitions! IC -> State change to RO possible RO -> State change to IC and O possible O -> State change to I and C possible I -> State change to S and RT possible S -> State change to I and RT possible RT -> State change to S and IT possible IT -> State change to R , RU and D possible R -> State change to RU and D possible RU -> State change to D possible D -> "Positive" end state: change no longer possible C -> "Negative" end state: change no longer possible	☐ String

Tab. 8-1 "Key Value Pair": transportOrderUploadKeyValues

Input values for the referenced production orders (optional)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☒ int
UPDATE_FLAG	"Update" mode [0; 2]: 0 = Production order is added 2 = Production order is removed	☒ int
WORKORDER_NUMBER	Production order number	☒ String

Tab. 8-2 "Key Value Pair": *productionOrderUploadKeys*

Input values for the picking list (optional)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on picking list	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM (no plausibility check) Advice: If multiple mounting places are to be passed for the current position (component), they must be entered separated by commas!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☒ int
PART_NUMBER	Part number in the iTAC system (component)	☐ String
PICKING_POSITION	Commissioning position Advice: The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": commissionItemUploadKeys)!	☒ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
QUANTITY	Material quantity of the component (see key PART_NUMBER) at the mounting place (if necessary, also as decimal value)	☐ double
STATION_NUMBER	Station number of the work station in the iTAC system; here, this is the target station for the picking list	☐ String

Key	Explanation		IMSApi data type
STORAGE_BARCODE	Bar code for the storage place of the component (PART_NUMBER) Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Advice: The keys STORAGE_BARCODE, STORAGE_ID or STORAGE_NUMBER can be used for specifying the storage place. If several of these keys are passed, the values must be compatible with each other!	☐	String
STORAGE_ID	Internal identification number of the storage place Advice: See the advice notices under STORAGE_BARCODE!	☐	long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notices under STORAGE_BARCODE!	☐	String
UPDATE_FLAG	"Update" mode [0; 1; 3]: 0 = Picking list is added 1 = Picking list is updated 3 = Picking list is removed	☒	int
WORKORDER_NUMBER	Production order number Advice: If the picking list is assigned a production order, then this production order must also be assigned to the transport order!	☐	String

Tab. 8-3 "Key Value Pair": *pickingItemUploadKeys*

Input value for the commissioning list (optional)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on commissioning list	☐ String
COMMISSION_NUMBER	Unique commissioning list number	☒ int
COMMISSION_STATE	"Commissioning list" state [N; C; I; R; P]: N = NEW -> New C = CANCELED -> Canceled I = IN_COMMISSION -> Being commissioned R = COMMISSIONED -> Commissioned P = PARTIALLY -> Partially collected	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM (no plausibility check) Advice: If multiple mounting places are to be passed for the current position (component), they must be entered separated by commas!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☒ int
OBJECT_NUMBER	Depending on the selected data object type (see key OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is passed here.	☐ String
OBJECT_TYPE	Determines the data object type [0; 1; 2; 3]: 0 = Container 1 = Serial number 2 = Bin (key word "bin-based traceability") 3 = Lot	☐ int
PART_NUMBER	Part number in the iTAC system (component)	☐ String
PICKING_POSITION	Commissioning position Advice: The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": pickingItemUploadKeys)!	☒ int
QUANTITY	Material quantity of the component (see key PART_NUMBER) at the mounting place (if necessary, also as decimal value)	☐ double
STATE_COMMENT	Individual comment regarding the current commissioning list (see key COMMISSION_STATE)	☐ String
UPDATE_FLAG	"Update" mode [0; 1; 3]: 0 = Commissioning list is added 1 = Commissioning list is updated 3 = Commissioning list is removed	☒ int

Tab. 8-4 "Key Value Pair": commissionItemUploadKeys

8.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
transportOrderUploadKeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": transportOrderUploadKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": transportOrderUploadKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
productionOrderUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>productionOrderUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": productionOrderUploadKeys table. Advice: Only the parameters passed here are processed by the function!
productionOrderUploadValues	Values for the parameters selected by means of <code>productionOrderUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>productionOrderUploadKeys</code> ! Refer to the "Key Value Pair": productionOrderUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
pickingItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>pickingItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pickingItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!
pickingItemUploadValues	Values for the parameters selected by means of <code>pickingItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>pickingItemUploadKeys</code> ! Refer to the "Key Value Pair": pickingItemUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
commissionItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>commissionItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": commissionItemUploadKeys table. Advice: <i>Only the parameters passed here are processed by the function!</i>
commissionItemUploadValues	Values for the parameters selected by means of <code>commissionItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: <i>The data is assigned in the order in which it was entered by means of parameter (in) <code>pickingItemUploadKeys</code>! Refer to the "Key Value Pair": commissionItemUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!</i>

Tab. 8-5 Parameter (in) »crpChangeTransportOrder«

Parameter (out)

Parameter name	Explanation
productionOrderUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>productionOrderUploadKeys</code> and <code>productionOrderUploadValues</code> . Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>productionOrderUploadKeys</code>!</i>
pickingItemUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>pickingItemUploadKeys</code> and <code>pickingItemUploadValues</code> . Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>pickingItemUploadKeys</code>!</i>
commissionItemUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>commissionItemUploadKeys</code> and <code>commissionItemUploadValues</code> . Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>commissionItemUploadKeys</code>!</i>

Tab. 8-6 Parameter (out) »crpChangeTransportOrder«

8.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-2070	Failure	IMSApi key was passed multiple times to the function
-2124	Failure	A value was passed multiple times to the function
-3500	Failure	CRP license does not exist for station (stop point)
-3510	Failure	Transport order not found
-3513	Failure	Invalid value for PICKING_POSITION
-3514	Failure	Invalid value for OBJECT_TYPE

-3515	Failure	Stop point not found
-3516	Failure	Object number is empty; it is required if an object type is passed
-3517	Failure	Object number not found
-3518	Failure	Part number is required
-3519	Failure	Part no. not found
-3520	Failure	Production order is required
-3521	Failure	Invalid value for UPDATE_FLAG (transport order - production order)
-3522	Failure	Invalid value for UPDATE_FLAG (transport order - picking list)
-3523	Failure	Invalid value for UPDATE_FLAG (transport order - commissioning list)
-3524	Failure	Production order for this transport order not found
-3525	Failure	Production order for this transport order already exists
-3526	Failure	Picking list for this transport order not found
-3527	Failure	Picking list for this transport order already exists
-3528	Failure	Commissioning list for this transport order not found
-3529	Failure	Commissioning list for this transport order already exists
-3530	Failure	State of the transport order is invalid
-3531	Failure	State of the commissioning list for this transport order is invalid
-3533	Failure	Production order for picking list is not a valid production order for this transport order
-3534	Failure	Production order for this transport order is still referenced in the picking list

8.2

crpCreateTransportOrder

8.2.1

Function description

This function makes it possible to create transport orders in the iTAC system. The corresponding entries are divided into four structures; the header data of the new transport order, the assigned production orders as well as the picking and commissioning lists.

When processing transport orders, observe the following basic principles:

- The assignment of production orders, picking lists and commissioning lists is optional.
- A transport order can be assigned one or more production orders.
- A transport order can be assigned one or more picking lists.
- A picking list can be assigned one or more commissioning lists; without a picking list, it is NOT possible to assign commissioning lists to a transport order.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	CRP-Service (basic): Site license for a certain number of stop points
--	--

Function declaration

```
int crpCreateTransportOrder
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] transportOrderUploadKeyValues
    String key
    String value
in String[] productionOrderUploadKeys
in String[] productionOrderUploadValues
in String[] pickingItemUploadKeys
in String[] pickingItemUploadValues
in String[] commissionItemUploadKeys
in String[] commissionItemUploadValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) transportOrderUploadKeyValues, productionOrderUploadKeys, pickingItemUploadKeys and commissionItemUploadKeys). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for the new transport order (header data)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on transport order	☐ String
DESCRIPTION	Description of the transport order	☐ String
EARLIEST_DELIVERY_DATE	Earliest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the key is not used, no time stamp is passed; depending on the function mode (see key UPDATE_FLAG), the corresponding field remains empty or unchanged!	☐ DateMs
EARLIEST_START_DATE	Earliest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐ DateMs
LATEST_DELIVERY_DATE	Latest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐ DateMs
LATEST_START_DATE	Latest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See the advice notice under the key EARLIEST_DELIVERY_DATE!	☐ DateMs
PRIORITY_CODE	Unique designator for the work order priority	☐ String
STATE_COMMENT	Individual comment regarding the current transport order state (see key TRANSPORT_ORDER_STATE)	☐ String
STOP_POINT_ACTUAL	Name of the stop point which defines the current location of the transport order (see the advice notice under the key STOP_POINT_DESTINATION). Advice: If the key is not passed, the following state applies for the transport order: "not yet arrived at a stop point"!	☐ String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the transport order. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐ String
TRANSPORT_ORDER_NUMBER	Transport order number Advice: The number specified here must NOT already exist in the system and must NOT exceed a length of 40 characters!	☒ String

Key	Explanation	IMSApi data type
TRANS- PORT_ORDER_STATE	"Transport order" state [I; RO; O; C; S; RT; T; R; RU; D; C]: IC = IN_CREATION -> Being created RO = READY_TO_ORDER -> Ready for ordering O = ORDERED -> Requested I = IN_COMMISSION -> Being commissioned S = STOPPED -> Withheld RT = READY_TO_TRANSFER -> Ready for collection IT = IN_TRANSFER -> Being delivered R = RECEIVED -> At receiving RU = READY_TO_UNLOAD -> Ready for unloading D = DELIVERED -> Delivered C = CANCELED -> Canceled Advice: The state IC (IN_CREATION) should be used for transport orders which are to be newly created!	String

Tab. 8-7 "Key Value Pair": *transportOrderUploadKeyValues*

Input values for the referenced production orders (optional)

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number Advice: The passing of an order number is only obligatory if the "Key Value Pair" structure <i>productionOrderUploadKeys</i> is passed!	String

Tab. 8-8 "Key Value Pair": *productionOrderUploadKeys*

Input values for the picking list (optional)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on picking list	String
COMP_NAME	Mounting place conformant with the iTAC BOM (no plausibility check) Advice: If multiple mounting places are to be passed for the current position (component), they must be entered separated by commas!	String
PART_NUMBER	Part number in the iTAC system (component)	String
PICKING_POSITION	Commissioning position Advice: The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": commissionItemUploadKeys)!	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	int
QUANTITY	Material quantity of the component (see key <i>PART_NUMBER</i>) at the mounting place (if necessary, also as decimal value)	double
STATION_NUMBER	Station number of the work station in the iTAC system; here, this is the target station for the picking list	String

Key	Explanation	IMSApi data type
STORAGE_BARCODE	Bar code for the storage place of the component (PART_NUMBER) Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Advice: The keys STORAGE_BARCODE, STORAGE_ID or STORAGE_NUMBER can be used for specifying the storage place. If several of these keys are passed, the values must be compatible with each other!	☐ String
STORAGE_ID	Internal identification number of the storage place Advice: See the advice notices under STORAGE_BARCODE!	☐ long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notices under STORAGE_BARCODE!	☐ String
WORKORDER_NUMBER	Production order number Advice: If the picking list is assigned a production order, then this production order must also be assigned to the transport order!	☐ String

Tab. 8-9 "Key Value Pair": *pickingItemUploadKeys*

Input value for the commissioning list (optional)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on commissioning list	☐ String
COMMISSION_NUMBER	Unique commissioning list number	☒ int
COMMISSION_STATE	"Commissioning list" state [N; C; I; R; P]: N = NEW -> New C = CANCELED -> Canceled I = IN_COMMISSION -> Being commissioned R = COMMISSIONED -> Commissioned P = PARTIALLY -> Partially collected	☐ String
COMP_NAME	Mounting place conformant with the iTAC BOM (no plausibility check) Advice: If multiple mounting places are to be passed for the current position (component), they must be entered separated by commas!	☐ String
OBJECT_NUMBER	Depending on the selected data object type (see key OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is passed here.	☐ String
OBJECT_TYPE	Determines the data object type [0; 1; 2; 3]: 0 = Container 1 = Serial number 2 = Bin (key word "bin-based traceability") 3 = Lot	☐ int
PART_NUMBER	Part number in the iTAC system (component)	☒ String

Key	Explanation	IMSApi data type
PICKING_POSITION	Commissioning position Advice: The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": pickingItemUploadKeys)!	☒ int
QUANTITY	Material quantity of the component (see key PART_NUMBER) at the mounting place (if necessary, also as decimal value)	☒ double
STATE_COMMENT	Individual comment regarding the current commissioning list (see key COMMISSION_STATE)	☐ String

Tab. 8-10 "Key Value Pair": commissionItemUploadKeys

8.2.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
transportOrderUploadKeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": transportOrderUploadKeyValues table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": transportOrderUploadKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
productionOrderUploadKeys	List of the parameters (production orders) that are to be passed by the function; the corresponding values are passed with the productionOrderUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": productionOrderUploadKeys table. Advice: Only the parameters passed here are processed by the function; the structure can remain empty, if no production orders are to be passed!
productionOrderUploadValues	Values for the parameters selected by means of productionOrderUploadKeys. Advice: The data is assigned in the order in which it was entered by means of parameter (in) productionOrderUploadKeys! Refer to the "Key Value Pair": productionOrderUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
pickingItemUploadKeys	List of the parameters (picking lists) that are to be passed by the function; the corresponding values are passed with the <code>pickingItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pickingItemUploadKeys table. Advice: Only the parameters passed here are processed by the function; the structure can remain empty, if no picking lists are to be passed!
pickingItemUploadValues	Values for the parameters selected by means of <code>pickingItemUploadKeys</code>. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>pickingItemUploadKeys</code>! Refer to the "Key Value Pair": pickingItemUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
commissionItemUploadKeys	List of the parameters (commissioning lists) that are to be passed by the function; the corresponding values are passed with the <code>commissionItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": commissionItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!; the structure can remain empty, if no commissioning lists are to be passed!
commissionItemUploadValues	Values for the parameters selected by means of <code>commissionItemUploadKeys</code>. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>pickingItemUploadKeys</code>! Refer to the "Key Value Pair": commissionItemUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 8-11 Parameter (in) »crpCreateTransportOrder«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

8.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-2070	Failure	IMSApi key was passed multiple times to the function
-3500	Failure	CRP license does not exist for station (stop point)
-3513	Failure	Invalid value for PICKING_POSITION
-3514	Failure	Invalid value for OBJECT_TYPE
-3515	Failure	Stop point not found
-3516	Failure	Object number is empty; it is required if an object type is passed
-3517	Failure	Object number not found

-3518	Failure	Part number is required
-3519	Failure	Part no. not found
-3520	Failure	Production order is required
-3525	Failure	Production order for this transport order already exists
-3527	Failure	Picking list for this transport order already exists
-3529	Failure	Commissioning list for this transport order already exists
-3530	Failure	State of the transport order is invalid
-3531	Failure	State of the commissioning list for this transport order is invalid
-3532	Failure	Transport order already exists
-3533	Failure	Production order for picking list is not a valid production order for this transport order
-3535	Failure	Commissioning list number for this transport order is too long

8.3

crpGetRoute

8.3.1

Function description

This function enables route information to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., start and lead times, route states or comments). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.



Advice

Here, the term "route" stands for a planned material delivery journey via certain route segments on which different transport orders are processed!

The results are output in three parts: the header data of the route, the assigned route segments and the transport orders assigned there. The output structures can be put into context using the key `ROUTE_NUMBER`.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

CRP-Service (basic): Site license for a certain number of stop points
--

Function declaration

```
int crpGetRoute
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] routeFilter
    String key
    String value
in String[] routeResultKeys
out String[] routeResultValues
in String[] routeSegmentResultKeys
out String[] routeSegmentResultValues
in String[] routeSegmentTransportOrderResultKeys
out String[] routeSegmentTransportOrderResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `routeFilter`, `routeResultKeys`, `routeSegmentResultKeys` and `routeSegmentTransportOrderResultKeys` and (out) `routeResultValues`, `routeSegmentResultValues` and `routeSegmentTransportOrderResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
COMMENT	Individual comment on route Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String
EARLIEST_DELIVERY_ - DATE_FROM	Time stamp for the start of the period under review (with respect to the earliest point in time for completely delivering the entire route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
EARLIEST_DELIVERY_ - DATE_TO	Time stamp for the end of the period under review (with respect to the earliest point in time for completely delivering the entire route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
EARLIEST_START_ - DATE_FROM	Time stamp for the start of the period under review (with respect to the earliest start time for processing the route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
EARLIEST_START_ - DATE_TO	Time stamp for the end of the period under review (with respect to the earliest start time for processing the route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_DELIVERY_ - DATE_FROM	Time stamp for the start of the period under review (with respect to the latest point in time for completely delivering the entire route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_DELIVERY_ - DATE_TO	Time stamp for the end of the period under review (with respect to the latest point in time for completely delivering the entire route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_START_DATE_ - FROM	Time stamp for the start of the period under review (with respect to the latest start time for processing the route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_START_DATE_ - TO	Time stamp for the end of the period under review (with respect to latest start time for processing the route); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int
ROUTE_NUMBER	Unique name of the route Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String

Key	Explanation	IMSApi data type
ROUTE_STATE	State of the route (arbitrary text) Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters! Special case "Multiple filter": To filter by multiple entries (OR link), the desired state values must be passed separated by a comma; NO wildcards are used in this case.	☐ String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the route. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the route (see the advice notice under the key STOP_POINT_DESTINATION).	☐ String

Tab. 8-12 "Key Value Pair": *routeFilter*

Definition of the output parameters for the found routes (header data)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on route	☐ String
DESCRIPTION	Route description	☐ String
EARLIEST_DELIVERY_DATE	Earliest point in time for completely delivering the entire route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
EARLIEST_START_DATE	Earliest start time for processing the route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
LATEST_DELIVERY_DATE	Latest point in time for completely delivering the entire route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
LATEST_START_DATE	Latest start time for processing the route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
ROUTE_NUMBER	Unique name of the route Advice: Based on this entry, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
ROUTE_STATE	State of the route	☐ String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the route.	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the route.	☐ String

Tab. 8-13 "Key Value Pair": *routeResultKeys*

Definition of output parameters for the associated route segments

Key	Explanation	IMSApi data type
COMMENT	Individual comment on route segment	☐ String
EQUIPMENT_INDEX	Individual designator for the carrier	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system	☐ String
ROUTE_NUMBER	Unique name of the route Advice: Based on this entry, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
ROUTE_SEGMENT_NUMBER	Unique name of the route segment Advice: The data sets "transport order" and "route segment" are put into context on the basis of this number (see the parameter of the same name in the table "Key Value Pair": routeSegmentTransportOrderResultKeys)!	☐ String
ROUTE_SEGMENT_SEQUENCE	Sequence number which is used as the basis for determining the sequence in which the individual route segments are worked through.	☐ int
ROUTE_SEGMENT_STATE	State of the route segment	☐ String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the route segment.	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the route segment.	☐ String

Tab. 8-14 "Key Value Pair": routeSegmentResultKeys

Definition of output parameters for the associated transport orders

Key	Explanation	IMSApi data type
ROUTE_NUMBER	Unique name of the route Advice: Based on this entry, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
ROUTE_SEGMENT_NUMBER	Unique name of the route segment Advice: The data sets "transport order" and "route segment" are put into context on the basis of this number (see the parameter of the same name in the table "Key Value Pair": routeSegmentResultKeys)!	☐ String
TRANSPORT_ORDER_NUMBER	Transport order number	☐ String

Tab. 8-15 "Key Value Pair": routeSegmentTransportOrderResultKeys

8.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
routeFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": routeFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": routeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
routeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": routeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) routeResultValues !
routeSegmentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": routeSegmentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) routeSegmentResultValues !
routeSegmentTransportOrderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": routeSegmentTransportOrderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) routeSegmentTransportOrderResultValues !

Tab. 8-16 Parameter (in) »crpGetRoute«

Parameter (out)

Parameter name	Explanation
routeResultValues	Depending on parameter (in) routeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) routeResultKeys !
routeSegmentResultValues	Depending on parameter (in) routeSegmentResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) routeSegmentResultKeys !

Parameter name	Explanation
routeSegment-TransportOrderResultValues	Depending on parameter (in) <code>routeSegmentTransportOrderResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>routeSegmentTransportOrderResultKeys</code>!</i>

Tab. 8-17 *Parameter (out) »crpGetRoute«*

8.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2119	Failure	Number of records to be output has exceeded the maximum value
-3500	Failure	CRP license does not exist for station (stop point)

8.4

crpGetTransportOrder

8.4.1

Function description

This function enables transport order information to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., start and lead times, transport order states or priority identification). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

The results are output in four parts: the header data of the transport order, the assigned production orders, the assigned picking lists and – only if picking lists exist – the commissioning lists assigned there. The output structures can be put into context using the keys `TRANSPORT_ORDER_NUMBER` and `PICKING_POSITION`.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

CRP-Service (basic): Site license for a certain number of stop points
--

Function declaration

```
int crpGetTransportOrder
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] transportOrderFilterKeyValues
    String key
    String value
in String[] transportOrderResultKeys
out String[] transportOrderResultValues
in String[] productionOrderResultKeys
out String[] productionOrderResultValues
in String[] pickingItemResultKeys
out String[] pickingItemResultValues
in String[] commissionItemResultKeys
out String[] commissionItemResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see the parameters (in) `transportOrderFilterKeyValues`, `transportOrderResultKeys`, `productionOrderResultKeys`, `pickingItemResultKeys` and `commissionItemResultKeys` as well as (out) `transportOrderResultValues`, `productionOrderResultValues`, `pickingItemResultValues` and `commissionItemResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation		IMSApi data type
EARLIEST_DELIVERY_- DATE_FROM	Time stamp for the start of the period under review (with respect to the earliest point in time for completely delivering the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
EARLIEST_DELIVERY_- DATE_TO	Time stamp for the end of the period under review (with respect to the earliest point in time for completely delivering the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
EARLIEST_START_- DATE_FROM	Time stamp for the start of the period under review (with respect to the earliest start time for processing the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
EARLIEST_START_- DATE_TO	Time stamp for the end of the period under review (with respect to the earliest start time for processing the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_DELIVERY_- DATE_FROM	Time stamp for the start of the period under review (with respect to the latest point in time for completely delivering the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_DELIVERY_- DATE_TO	Time stamp for the end of the period under review (with respect to the latest point in time for completely delivering the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_START_DATE_- FROM	Time stamp for the start of the period under review (with respect to the latest start time for processing the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
LATEST_START_DATE_- TO	Time stamp for the end of the period under review (with respect to the latest start time for processing the transport order); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int
PRIORITY_CODE	Unique designator for the work order priority	☐	String
STOP_POINT_ACTUAL	Name of the stop point which defines the current location of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐	String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the transport order. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	☐	String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐	String

Key	Explanation	IMSApi data type
TRANS- PORT_ORDER_NUMBER	Transport order number Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
TRANS- PORT_ORDER_STATE	"Transport order" state [I; RO; O; C; S; RT; T; R; RU; D; C]: IC = IN_CREATION -> Being created RO = READY_TO_ORDER -> Ready for ordering O = ORDERED -> Requested I = IN_COMMISSION -> Being commissioned S = STOPPED -> Withheld RT = READY_TO_TRANSFER -> Ready for collection IT = IN_TRANSFER -> Being delivered R = RECEIVED -> At receiving RU = READY_TO_UNLOAD -> Ready for unloading D = DELIVERED -> Delivered C = CANCELED -> Canceled Advice: To filter by multiple entries (OR link), the desired values must be passed separated by a comma (e.g.: "IC,O,S")!	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 8-18 "Key Value Pair": transportOrderFilterKeyValues

Definition of the output parameters for the found transport orders (header data)

Key	Explanation	IMSApi data type
COMMENT	Individual comment on transport order	☐ String
DESCRIPTION	Description of the transport order	☐ String
EARLIEST_DELIVERY_DATE	Earliest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
EARLIEST_START_DATE	Earliest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
LATEST_DELIVERY_DATE	Latest point in time for completely delivering the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
LATEST_START_DATE	Latest start time for processing the transport order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PRIORITY_CODE	Unique designator for the work order priority	☐ String
STATE_COMMENT	Individual comment regarding the current transport order state (see key TRANSPORT_ORDER_STATE)	☐ String
STOP_POINT_ACTUAL	Name of the stop point which defines the current location of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐ String

Key	Explanation	IMSApi data type
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the transport order. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	☐ String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the transport order (see the advice notice under the key STOP_POINT_DESTINATION).	☐ String
TRANSPORT_ORDER_NUMBER	Transport order number Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
TRANSPORT_ORDER_STATE	"Transport order" state [I; RO; O; C; S; RT; T; R; RU; D; C]: IC = IN_CREATION -> Being created RO = READY_TO_ORDER -> Ready for ordering O = ORDERED -> Requested I = IN_COMMISSION -> Being commissioned S = STOPPED -> Withheld RT = READY_TO_TRANSFER -> Ready for collection IT = IN_TRANSFER -> Being delivered R = RECEIVED -> At receiving RU = READY_TO_UNLOAD -> Ready for unloading D = DELIVERED -> Delivered C = CANCELED -> Canceled	☐ String

Tab. 8-19 "Key Value Pair": transportOrderResultKeys

Definition of output parameters for the associated production orders

Key	Explanation	IMSApi data type
TRANSPORT_ORDER_NUMBER	Transport order number Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 8-20 "Key Value Pair": productionOrderResultKeys

Definition of output parameters for the associated picking lists

Key	Explanation	IMSApi data type
COMMENT	Individual comment on picking list	☐ String
COMP_NAME	Mounting place Advice: If there are multiple mounting places for the current position (component), the individual entries are output separated by commas!	☐ String
PART_NUMBER	Part number in the iTAC system (component)	☐ String
PICKING_POSITION	Commissioning position Advice: The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": commissionItemResultKeys)!	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
QUANTITY	Material quantity of the component (see key PART_NUMBER) at the mounting place (if necessary, also as decimal value)	☐ double
STATION_NUMBER	Station number of the work station in the iTAC system; here, this is the target station for the picking list	☐ String
STORAGE_BARCODE	Bar code for the storage place of the component (PART_NUMBER) Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_ID	Internal identification number of the storage place	☐ long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐ String
TRANSPORT_ORDER_NUMBER	Transport order number Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ String
WORKORDER_NUMBER	Production order number Advice: If the picking list is assigned a production order, then this production order must also be assigned to the transport order!	☐ String

Tab. 8-21 "Key Value Pair": pickingItemResultKeys

Definition of output parameters for the associated commissioning lists

Key	Explanation	IMSApi data type
COMMENT	Individual comment on commissioning list	☐ String
COMMISSION_NUMBER	Unique commissioning list number	☐ int
COMMISSION_STATE	"Commissioning list" state [N; C; I; R; P]: N = NEW -> New C = CANCELED -> Canceled I = IN_COMMISSION -> Being commissioned R = COMMISSIONED -> Commissioned P = PARTIALLY -> Partially collected	☐ String
COMP_NAME	Mounting place Advice: <i>If there are multiple mounting places for the current position (component), the individual entries are output separated by commas!</i>	☐ String
OBJECT_NUMBER	Depending on the selected data object type (see key OBJECT_TYPE), the desired data object (e.g. the concrete serial number) is output here.	☐ String
OBJECT_TYPE	Determines the data object type [0; 1; 2; 3]: 0 = Container 1 = Serial number 2 = Bin (key word "bin-based traceability") 3 = Lot	☐ int
PART_NUMBER	Part number in the iTAC system (component)	☐ String
PICKING_POSITION	Commissioning position Advice: <i>The data sets "picking list" and "commissioning list" can be put into context on the basis of this entry (see the key of the same name in the table "Key Value Pair": pickingItemResultKeys)!</i>	☐ int
QUANTITY	Material quantity of the component (see key PART_NUMBER) at the mounting place (if necessary, also as decimal value)	☐ double
STATE_COMMENT	Individual comment regarding the current commissioning list (see key COMMISSION_STATE)	☐ String
TRANS- PORT_ORDER_NUMBER	Transport order number Advice: <i>Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!</i>	☐ String

Tab. 8-22 "Key Value Pair": *commissionItemResultKeys*

8.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
transportOrder-FilterKeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": transportOrderFilterKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": transportOrderFilterKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
transportOrderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": transportOrderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>transportOrderResultValues</code> !
productionOrderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": productionOrderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>productionOrderResultValues</code> !
pickingItemResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": pickingItemResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>pickingItemResultValues</code> !
commissionItemResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": commissionItemResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>commissionItemResultValues</code> !

Tab. 8-23 Parameter (in) »crpGetTransportOrder«

Parameter (out)

Parameter name	Explanation
transportOrderResultValues	Depending on parameter (in) <code>transportOrderResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>transportOrderResultKeys</code>!</i>
productionOrderResultValues	Depending on parameter (in) <code>productionOrderResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>productionOrderResultKeys</code>!</i>
pickingItemResultValues	Depending on parameter (in) <code>pickingItemResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>pickingItemResultKeys</code>!</i>
commissionItemResultValues	Depending on parameter (in) <code>commissionItemResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>commissionItemResultKeys</code>!</i>

Tab. 8-24 Parameter (out) »`crpGetTransportOrder`«

8.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2119	Failure	Number of records to be output has exceeded the maximum value
-3500	Failure	CRP license does not exist for station (stop point)
-3515	Failure	Stop point not found
-3530	Failure	State of the transport order is invalid

8.5

crpManageRoute

8.5.1

Function description

This function is used for managing routes; here, the term "route" stands for a planned material delivery journey via certain route segments on which different transport orders are processed. Depending on the function mode (see key `UPDATE_FLAG`), complete routes or individual route segments can be created, adapted or deleted. If required, this function can also be used only to assign transport orders to the desired route segments or to undo an existing assignment.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

When processing routes, observe the following basic principles:

- A route can be assigned one or more route segments.
- A route segment can be assigned one or more transport orders.
- When a route is deleted, all route segments are also deleted and with them also the references to the transport orders.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	CRP-Service (basic): Site license for a certain number of stop points
--	--

Function declaration

```
int crpManageRoute
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] routeUploadKeyValues
    String key
    String value
in String[] routeSegmentUploadKeys
in String[] routeSegmentUploadValues
out String[] routeSegmentUploadResult
in String[] routeSegmentTransportOrderUploadKeys
in String[] routeSegmentTransportOrderUploadValues
out String[] routeSegmentTransportOrderUploadResult
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `routeUploadKeyValues`, `routeSegmentUploadKeys` and `routeSegmentTransportOrderUploadKeys` as well as (out) `routeSegmentUploadResult` and `routeSegmentTransportOrderUploadResult`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for the new route / the route to be processed (header data)

Key	Explanation		IMSApi data type
COMMENT	Individual comment on route	☐	String
DESCRIPTION	Route description	☐	String
EARLIEST_DELIVERY_DATE	Earliest point in time for completely delivering the entire route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>If the key is not used, no time stamp is passed; depending on the function mode (see key UPDATE_FLAG), the corresponding field remains empty or unchanged!</i>	☐	DateMs
EARLIEST_START_DATE	Earliest start time for processing the route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See the advice notice under the key EARLIEST_DELIVERY_DATE!</i>	☐	DateMs
LATEST_DELIVERY_DATE	Latest point in time for completely delivering the entire route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See the advice notice under the key EARLIEST_DELIVERY_DATE!</i>	☐	DateMs
LATEST_START_DATE	Latest start time for processing the route; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See the advice notice under the key EARLIEST_DELIVERY_DATE!</i>	☐	DateMs
ROUTE_NUMBER	Unique name of the route	☒	String
ROUTE_STATE	State of the route (arbitrary text)	☐	String
STOP_POINT_DESTINATION	Name of the stop point which defines the end point of the route. Advice: <i>Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!</i>	☐	String
STOP_POINT_SOURCE	Name of the stop point which defines the start point of the route (see the advice notice under the key STOP_POINT_DESTINATION).	☐	String

Key	Explanation	IMSApi data type
UPDATE_FLAG	"Update" mode [0; 1; 3]: 0 = Route is added 1 = Route is updated 3 = Route is removed Advice regarding [1]: This key relates only to the updating of the header data; if only route segments are to be changed, this key must NOT be passed!	int

Tab. 8-25 "Key Value Pair": routeUploadKeyValues

Input values for the assigned route segments

Key	Explanation	IMSApi data type
COMMENT	Individual comment on route segment	String
EQUIPMENT_INDEX	Individual designator for the carrier Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: If the carrier number is non-unique, the EQUIPMENT_INDEX key must also be passed!	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	int
ROUTE_SEGMENT_NUMBER	Unique name of the route segment Advice: The sequence in which the individual route segments are processed/worked through is determined using the key ROUTE_SEGMENT_SEQUENCE!	String
ROUTE_SEGMENT_SEQUENCE	Sequence number which is used as the basis for determining the sequence in which the individual route segments are worked through. Advice: The numbers are worked through in ascending order; consecutive numbering (+1) is not forced!	int
ROUTE_SEGMENT_STATE	State of the route segment (arbitrary text)	String
ROUTE_SEGMENT_DESTINATION	Name of the stop point which defines the end point of the route segment. Advice: Stop points are created in combination with storage places; detailed information on this topic can be found in the user manual under ADM module → Storage management!	String
ROUTE_SEGMENT_SOURCE	Name of the stop point which defines the start point of the route segment (see the advice notice under the key ROUTE_SEGMENT_DESTINATION).	String

Key	Explanation	IMSApi data type
UPDATE_FLAG	"Update" mode [0; 1; 3]: 0 = Route segment is added 1 = Route segment is updated 3 = Route segment is removed Advice regarding [1]: This key relates only to the updating of the route segments; if only transport orders are to be changed, this key must NOT be passed!	int

Tab. 8-26 "Key Value Pair": routeSegmentUploadKeys

Input values for the assigned transport orders

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	int
ROUTE_SEGMENT_NUMBER	Unique name of the route segment Advice: The data sets "transport order" and "route segment" are put into context on the basis of this number (see the parameter of the same name in the table "Key Value Pair": routeSegmentUploadKeys)!	String
TRANSPORT_ORDER_NUMBER	Transport order number	String
UPDATE_FLAG	"Update" mode [0; 3]: 0 = Transport order is assigned 3 = Transport order is removed Advice regarding [1]: This key relates only to the updating of the route segments; if only transport orders are to be changed, this key must NOT be passed!	int

Tab. 8-27 "Key Value Pair": routeSegmentTransportOrderUploadKeys

8.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
routeUploadKey-Values	"Key Value Pair" structure with the following variables:

Parameter name	Explanation
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>routeUploadKeyValues</code> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <code>routeUploadKeyValues</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
<code>routeSegmentUploadKeys</code>	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>routeSegmentUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>routeSegmentUploadKeys</code> table. Advice: Only the parameters passed here are processed by the function!
<code>routeSegmentUploadValues</code>	Values for the parameters selected by means of <code>routeSegmentUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>routeSegmentUploadKeys</code> ! Refer to the "Key Value Pair": <code>routeSegmentUploadKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
<code>routeSegmentTransportOrderUploadKeys</code>	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>routeSegmentTransportOrderUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>routeSegmentTransportOrderUploadKeys</code> table. Advice: Only the parameters passed here are processed by the function!
<code>routeSegmentTransportOrderUploadValues</code>	Values for the parameters selected by means of <code>routeSegmentTransportOrderUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>routeSegmentTransportOrderUploadKeys</code> ! Refer to the "Key Value Pair": <code>routeSegmentTransportOrderUploadKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 8-28 Parameter (in) »`crpManageRoute`«

Parameter (out)

Parameter name	Explanation
routeSegmentUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>routeSegmentUploadKeys</code> and <code>routeSegmentUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>routeSegmentUploadKeys</code> !
routeSegmentTransportOrderUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>routeSegmentTransportOrderUploadKeys</code> and <code>routeSegmentTransportOrderUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>routeSegmentTransportOrderUploadKeys</code> !

Tab. 8-29 Parameter (out) »crpManageRoute«

8.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key

-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2120	Failure	At least one record in the array could not be processed
-3500	Failure	CRP license does not exist for station (stop point)
-3501	Failure	Route not found
-3502	Failure	Route already exists
-3503	Failure	Route segment not found
-3504	Failure	Route segment already exists
-3505	Failure	Route segment -> Sequence already exists
-3506	Failure	Invalid value for UPDATE_FLAG (route)
-3507	Failure	Invalid value for UPDATE_FLAG (route segment)
-3508	Failure	ROUTE_SEGMENT_SEQUENCE missing in route segment
-3509	Failure	Invalid value for UPDATE_FLAG (transport order)
-3510	Failure	Transport order not found
-3511	Failure	Transport order not assigned
-3512	Failure	Transport order already assigned
-3515	Failure	Stop point not found

9 Group "CUSTOM" (customer function)

9.1 customFunction

9.1.1 Function description

This function enables the grouping together of various IMSApi functions into a "customer method"; in this way, complex, customer-specific IMSApi workflows can be represented and called centrally via one method. The workflows can also include branches and conditions and are implemented using programming methods on the server side.



Advice

A customer method is programmed in JavaScript. To make them known in the iTAC system, the respective JavaScript files must be stored as MDA items. A customer method is called on the basis of the MDA item name in connection with the API function names contained therein. Older customer methods (prior to release 7) can be used without any modification to the call, provided the MDA item of the customer method was stored under the name "customFunction"!

The call can be made from any client. From the perspective of a client, implementation of workflows can thereby be considerably simplified, since just one function needs to be called instead of several.

From the perspective of a client, an IMSApi workflow thus generally consists of two calls: one call prior to the operation to check all necessary conditions, and one call afterwards to send the results. As a side effect, this reduces the transaction time, which has a positive impact above all on logs or connections with high delay times. The concept enables subsequent changes to the workflow without intervening in the program logic of the client.



Advice

Which IMSApi workflows can be meaningfully grouped into a method in this way depends on the individual project requirements. These methods must be explicitly developed for each customer-specific application type. Therefore, only the functional framework is described here!

Function declaration

```
int customFunction
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String methodName
  in String[] inArgs
  out String[] outArgs
  out String customErrorString
```

9.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
methodName	Name of the customer-specific method
inArgs	Array with the variables that are required depending on the customer method and the IMSApi functions that are grouped together there

Tab. 9-1 *Parameter (in) »customFunction«**Parameter (out)*

Parameter name	Explanation
outArgs	Array with the variables that are output depending on the customer method and the IMSApi functions that are grouped together there
customErrorString	Customer-specific output text

Tab. 9-2 *Parameter (out) »customFunction«*

9.1.3

Output values

Error codes

In addition to the standard error codes listed below, this function also returns corresponding "technical" error codes as the result; these arise from the IMSApi functions used in the `customFunction`.

In addition to these standardized IMSApi error messages, a fixed block of numbers is defined for customer-specific error messages:

-30,000 to -32,000 = Reserved range for customer-specific failures

30,000 to 32,000 = Reserved range for customer-specific warnings



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-1500	Failure	Script not found
-1501	Failure	Script is not active
-1502	Failure	Script was not executed

9.2 customGetDbObjectData

9.2.1 Function description

This function is used to query **data** from customer-specific database objects (tables and views in the CUST schema); here, the output can be influenced using the SQL filter conditions (e.g. EQUAL, LIKE or IN). Unlike the "normal" SQL-based SELECT statements, here the used filter conditions are always AND-linked so that only the data to which all used filter conditions apply is returned.



Advice

Note that customer-specific tables and views must be created manually using corresponding SQL scripts. The required SQL scripts must be specified during the course of commissioning, taking the iTAC specifications into consideration (for "tables", see the item »Specification "DbObject" (table)« [▶▶▶] under the function »customModifyDbObjectData« [▶▶▶]), and implemented together with the iTAC employee responsible for the project!

In the background, a SELECT statement compliant with the SQL standard is formed on the basis of this function and sent to the database; the result of the query is then output by the function by means of the "Key Value Pair" structure dbColumnResultValues.

TIP: Detailed information on the customer-specific database objects (e.g. table column names, data field lengths and types) can be queried by means of the IMSApi function »customGetDbObjectMetaData« [▶▶▶]!

Example "SELECT"

A function call with the parameters given in the example would result in the SELECT statement shown below.

```
KeyValue[] aDbObjectSearchKeyValues = new KeyValue[] {
    new KeyValue("DB_OBJECT_NAME", "CT_TEST_TABLE"),
    new KeyValue("MAX_ROWS", "123")
};
String[] aDbColumnFilterKeys = {
    "DB_COLUMN_NAME",
    "VALUE",
    "DB_CONDITION"
};
String[] aDbColumnFilterValues = {
    "COL1", "A,B,C", "IN",
    "COL2", "1", "EQUAL"
};
String[] aDbColumnOrderKeys = {
    "DB_COLUMN_NAME",
    "DB_COLUMN_ORDER_BY_DIRECTION"
};
String[] aDbColumnOrderValues = {
    "COL1", "ASC"
};
String[] aDbColumnResultKeys = {
    "COL1", "COL2"
};
```

```
SELECT COL1, COL2 FROM CUST.CT_TEST_TABLE WHERE COL1 ON (?, ?, ?) AND COL2 = ? ORDER BY COL1 ASC;
/* Parameter = 'A', 'B', 'C', '1' */
```

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

User tables (count): Site license for a certain number of tables

Function declaration

```
int customGetDBObjectData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue dbObjectSearchKeyValues
    String key
    String value
  in String dbColumnFilterKeys
  in String dbColumnFilterValues
  in String dbColumnOrderKeys
  in String dbColumnOrderValues
  in String dbColumnResultKeys
  out String dbColumnResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) dbObjectSearchKeyValues, dbColumnFilterKeys, dbColumnOrderKeys and dbColumnResultKeys as well as (out) dbColumnResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Definition of the database object to be queried (table/view)

Key	Explanation	IMSApi data type
DB_OBJECT_NAME	Name of the database object; this is the table name or the name of the view Advice: Customer-specific tables always begin with the prefix "CT_" and customer-specific views with "CV_"!	☒ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 9-3 "Key Value Pair": dbObjectSearchKeyValues

Definition of the filter criteria for the output

Key	Explanation	IMSApi data type
DB_COLUMN_NAME	Name of the table column to which the filter condition is applied.	☒ String
DB_CONDITION	Filter condition according to the SQL standard; the following items are supported: - EQUAL - LIKE - GREATER - GREATER_EQUALS - LESS - LESS_EQUALS - IN Advice: Pay attention to upper- and lower-case letters during input!	☒ String
VALUE	Value for the filter condition according to the key DB_CONDITION Advice: For the filter condition IN, the limit values must be entered separated by commas!	☒ String

Tab. 9-4 "Key Value Pair": dbColumnFilterKeys

Definition of the sort order for the output

Key	Explanation	IMSApi data type
DB_COLUMN_NAME	Name of the table column used as the basis for sorting the output.	☒ String
DB - COLUMN_ORDER_BY_DIRECTION	Sort order "ascending/descending" [ASC; DESC]: ASC = Ascending sorting (alphanumeric) DESC = Descending sorting (alphanumeric)	☐ String

Tab. 9-5 "Key Value Pair": dbColumnOrderKeys

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
[0] - [n]	Placeholder for the names of the table columns to be output.	☒ String

Tab. 9-6 "Key Value Pair": dbColumnResultKeys

9.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
dbObjectSearch- KeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbObjectSearchKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": dbObjectSearchKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnFilter- Keys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>dbColumnFilterValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnFilterKeys table. Advice: If this structure is passed multiple times, the individual filter conditions are AND-linked!
dbColumnFilter- Values	Value list for the parameters passed by means of <code>dbColumnFilterKeys</code> . Advice: Refer to the "Key Value Pair": dbColumnFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnOrderKeys	List of the parameters on the basis of which the output is to be sorted; the respective values are passed with the <code>dbColumnOrderValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnOrderKeys table.
dbColumnOrderVal- ues	Value list for the parameters passed by means of <code>dbColumnOrderKeys</code> . Advice: Refer to the "Key Value Pair": dbColumnOrderKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnResult- Keys	List of the "free" parameters (here "column names") based on which the SELECT function is to be executed; the respective values are passed with the parameter <code>dbColumnInsertValues</code> (for further information, see also the table "Key Value Pair": dbColumnInsertKeys below). TIP: Information on the table column names in a customer-specific database object can be queried using the IMSApi function »customGetDbObjectMetadata« [▶▶▶]!

Tab. 9-7 Parameter (in) »customGetDbObjectData«

Parameter (out)

Parameter name	Explanation
dbColumnResult-Values	Depending on parameter (in) dbColumnResultKeys, the corresponding data is output here.

Tab. 9-8 Parameter (out) »customGetDbObjectData«

9.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-168	Failure	Invalid value for the function mode (control flag)
-174	Failure	Number of records to be output has fallen below the minimum value
-1504	Failure	DbObject not found
-1506	Failure	DbObject not licensed
-1507	Failure	DbObject -> Maximum number of columns exceeded

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

9.3 customGetDbObjectMetaData

9.3.1 Function description

This function enables **metadata** from customer-specific database objects (tables and views in the CUST schema) to be queried; here it is possible to filter according to a wide range of different criteria (e.g. name/type of the database object). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.



Advice

Note that customer-specific tables and views must be created manually using corresponding SQL scripts. The required SQL scripts must be specified during the course of commissioning, taking the iTAC specifications into consideration (for "tables", see the item »Specification "DbObject" (table)« [▶▶▶] under the function »customModifyDbObjectData« [▶▶▶]), and implemented together with the iTAC employee responsible for the project!

Licensing NO special licensing is required for this API function.

Function declaration

```
int customGetDbObjectMetaData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] dbObjectFilterKeyValues
    String key
    String value
in String[] dbColumnMetaDataResultKeys
out String[] dbColumnMetaDataResultValues
```

"Key Value Pair" structures This function contains "Key Value Pair" structures (see parameters (in) dbObjectFilterKeyValues and dbColumnMetaDataResultKeys as well as (out) dbColumnMetaDataResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DB_FORCE_REFRESH_METADATACACHE	Querying/update of "metadata cache" [0; 1]: 0 = Metadata is queried from the metadata cache (FALSE) 1 = Metadata is queried directly from the database (TRUE); here the metadata cache is regenerated automatically Advice: If the key is not passed, the value [FALSE] is used by default; the value [TRUE] should only be used temporarily (not in the productive context) because large data quantities cause a reduction in system performance during regeneration!	boolean
DB_OBJECT_NAME	Name of the database object; this is the table name or the name of the view (please note here the key DB_OBJECT_TYP). Advice: The wildcard "*" can be used during entry; if the key is not passed, there is no restriction! Customer-specific tables always begin with the prefix "CT_" and customer-specific views with "CV_"!	String
DB_OBJECT_TYP	Type of database object [TABLE; VIEW]: TABLE = Database table VIEW = Database view Advice: If this key is not passed along, the [TABLE] value is used by default!	String

Tab. 9-9 "Key Value Pair": dbObjectFilterKeyValues

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
DB_COLUMN_NAME	Name of the table column	String
DB_DATA_LENGTH	Total number of digits = p of "NUMBER (p, s)" Advice: See advice for key DB_DATA_TYPE!	int
DB_DATA_NULLABLE	"Zero allowed" designator [0; 1]: 0 = Table is not allowed to be [empty] (FALSE) 1 = Table is allowed to be [empty] (TRUE) Advice: See advice for key DB_DATA_TYPE!	boolean
DB_DATA_PRECISION	Digits before and after decimal point = s of "NUMBER (p, s)" Advice: See advice for key DB_DATA_TYPE!	int
DB_DATA_TYPE	Data type for the table column: - STRING - INTEGER - LONG - FLOAT - DOUBLE - DATE - BOOLEAN Advice: Note that this key can be interpreted differently depending on the used database!	String
DB_OBJECT_NAME	Name of the database object; this is the table name or the name of the view	String

Tab. 9-10 "Key Value Pair": dbColumnMetaDataResultKeys

9.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
dbObjectFilter- KeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbObjectFilterKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": dbObjectFilterKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnMeta- DataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnMetaDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>dbColumnMetaDataResultValues</code> !

Tab. 9-11 Parameter (in) »customGetDbObjectMetaData«

Parameter (out)

Parameter name	Explanation
dbColumnMeta- DataResultValues	Depending on parameter (in) <code>dbColumnMetaDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>dbColumnMetaDataResultKeys</code> !

Tab. 9-12 Parameter (out) »customGetDbObjectMetaData«

9.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-168	Failure	Invalid value for the function mode (control flag)
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

9.4

customGetFunctionInfo

9.4.1

Function description

This function returns detailed information (metadata) on **one** particular customer method. This includes the function header information, information on the respective input and output parameters as well as the possible output values (error codes). An overview of the possible metadata and the associated variables can be found below under »Metadata« [▶▶▶].



Advice

For the information to be output, it must be stored as metadata in the script of the customer method. The script of the customer method must be available as an MDA item in the MDA module under the MDA type "Script (custom function)"!

*As a rule, **multiple** functions can be contained in **one** customer method (i.e. in one script)!*

Background: A customer method can be used to map complex, customer-specific IMSApi workflows. With smart.FactoryAppStudio, iTAC offers two tools with which customer methods including the desired metadata can be created and directly integrated as a script in the MDA module. The system uses the metadata item `FUNCTION_TYPE` to determine whether the customer method is a general customer method (created using the "IMSApi FlowDesigner" tool) or a customer method specially developed for the OPCUA service ("OPCUA Studio" tool).

Definition of "metadata"

The metadata of a customer method includes information from the MDA module, more precisely from the MDA item and the script stored there. All variables together with the data source are listed below. The name of the data module corresponds to the key given in the tables below.

Information from MDA module

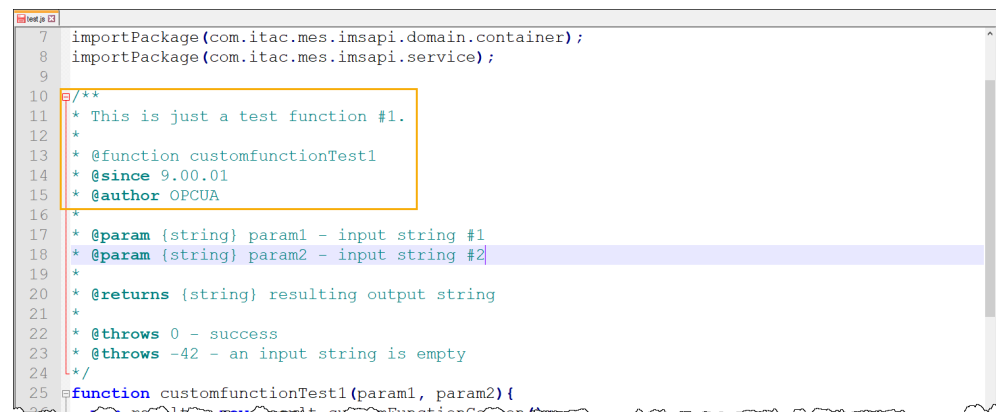
- Data module `MDA_NAME`: see the "Name" field in the figure
- Data module `MDA_STATUS`: see the "Status" field in the figure
- Data module `MDA_VERSION`: see the "Version" field in the figure

State	Name	Version	Active	Description	Type	Info	Valid from	Valid to
OK	CF	1.1	☒	test customGetFunctionInfo	Script (custom function)	test.js	2018-05-03 08:12:08	3000-12-31

Fig. 9-1 CF metadata of "MDA module"

Information from script

- Function header information:
 - Data module FUNCTION_DESC: string that is initiated WITHOUT the key term; in the example shown here: *This is just a test function #1.*
 - Data module FUNCTION_NAME: string that is initiated with the key term **@function**; in the example shown here: *customfunctionTest1*
 - Data module SINCE_VERSION: string that is initiated with the key term **@since**; in the example shown here: *9.00.01*
 - Data module FUNCTION_TYPE: string that is initiated with the key term **@author**; in the example shown here: *OPCUA* - accordingly, the IMSApi returns the output **1** for the function type "OPCUA"; all other entries for **@author** would result in **0** in the output.



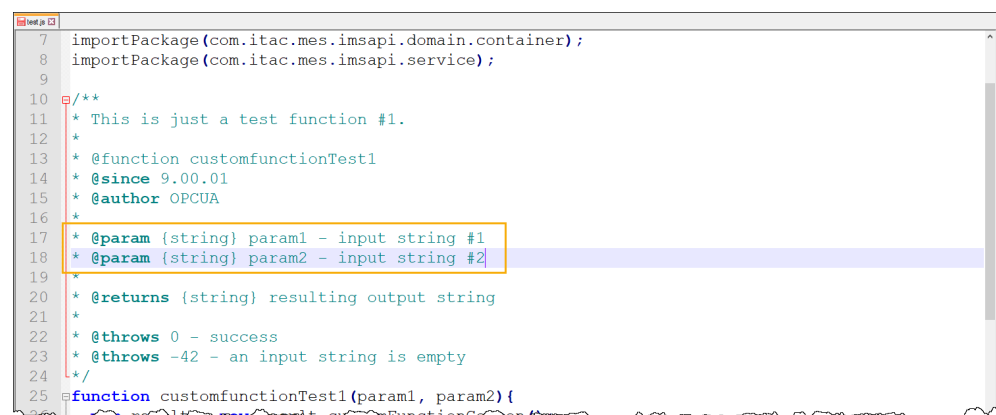
```

7 importPackage(com.itac.mes.imsapi.domain.container);
8 importPackage(com.itac.mes.imsapi.service);
9
10 /**
11  * This is just a test function #1.
12  *
13  * @function customfunctionTest1
14  * @since 9.00.01
15  * @author OPCUA
16  *
17  * @param {string} param1 - input string #1
18  * @param {string} param2 - input string #2
19  *
20  * @returns {string} resulting output string
21  *
22  * @throws 0 - success
23  * @throws -42 - an input string is empty
24  */
25 function customfunctionTest1(param1, param2){

```

Fig. 9-2 CF metadata of "function header"

- INPUT parameters:
 - Data module CF_DATA_TYPE: string that is initiated with the key term **@param** and is in curly brackets; in the example shown here: *string*
 - Data module CF_IN_PARAM_NAME: string that is initiated with the key term **@param** and is between the characters "}" and "-"; in the example shown here: *param1* and *param2*
 - Data module CF_IN_PARAM_DESC: string that is initiated with the key term **@param** and comes after the "-" character; in the example shown here: *input string #1* and *input string #2*



```

7 importPackage(com.itac.mes.imsapi.domain.container);
8 importPackage(com.itac.mes.imsapi.service);
9
10 /**
11  * This is just a test function #1.
12  *
13  * @function customfunctionTest1
14  * @since 9.00.01
15  * @author OPCUA
16  *
17  * @param {string} param1 - input string #1
18  * @param {string} param2 - input string #2
19  *
20  * @returns {string} resulting output string
21  *
22  * @throws 0 - success
23  * @throws -42 - an input string is empty
24  */
25 function customfunctionTest1(param1, param2){

```

Fig. 9-3 CF metadata of "INPUT parameters"

- OUTPUT parameters:
 - Data module CF_DATA_TYPE: string that is initiated with the key term **@returns** and is in curly brackets; in the example shown here: *string*
 - Data module CF_OUT_PARAM_DESC: string that is initiated with the key term **@returns** and comes after the bracket term; in the example shown here: *resulting output string*

```

7 importPackage(com.itac.mes.imsapi.domain.container);
8 importPackage(com.itac.mes.imsapi.service);
9
10 /**
11  * This is just a test function #1.
12  *
13  * @function customfunctionTest1
14  * @since 9.00.01
15  * @author OPCUA
16  *
17  * @param {string} param1 - input string #1
18  * @param {string} param2 - input string #2
19  *
20  * @returns {string} resulting output string
21  *
22  * @throws 0 - success
23  * @throws -42 - an input string is empty
24  */
25 function customfunctionTest1(param1, param2){

```

Fig. 9-4 CF metadata of "OUTPUT parameters"

- Output values (error codes): initiated by **@throws**
 - Data module CF_ERROR_CODE: numerical value that is initiated with the key term **@throws**; in the example shown here: *0* and *-42*
 - Data module CF_ERROR_TEXT: string that is initiated with the key term **@throws** and comes after the "-" character; in the example shown here: *success* and *an input string is empty*

```

7 importPackage(com.itac.mes.imsapi.domain.container);
8 importPackage(com.itac.mes.imsapi.service);
9
10 /**
11  * This is just a test function #1.
12  *
13  * @function customfunctionTest1
14  * @since 9.00.01
15  * @author OPCUA
16  *
17  * @param {string} param1 - input string #1
18  * @param {string} param2 - input string #2
19  *
20  * @returns {string} resulting output string
21  *
22  * @throws 0 - success
23  * @throws -42 - an input string is empty
24  */
25 function customfunctionTest1(param1, param2){

```

Fig. 9-5 CF metadata of "output values"

Licensing

NO special licensing is required for this API function.

Function declaration

```
int customGetFunctionInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in KeyValue[] functionInfoFilterKeyValues
    String key
    String value
in String[] functionInfoResultKeys
in String[] parameterInfoResultKeys
in String[] returnInfoResultKeys
in String[] errorCodeResultKeys
out String[] functionInfoResultValues
out String[] parameterInfoResultValues
out String[] returnInfoResultValues
out String[] errorCodeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) functionInfoFilterKeyValues, functionInfoResultKeys, parameterInfoResultKeys, returnInfoResultKeys and errorCodeResultKeys as well as (out), functionInfoResultValues, parameterInfoResultValues, returnInfoResultValues, errorCodeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNCTION_NAME	Name of the function Advice: A customer method, i.e. a script, can contain multiple independent functions! The keys MDA_NAME and FUNCTION_NAME together form the "methodName", which is used to identify the desired function in the customer method!	☒ String
MDA_NAME	Unique name of the MDA item Advice: The keys MDA_NAME and FUNCTION_NAME together form the "methodName", which is used to identify the function in the customer method!	☒ String

Tab. 9-13 "Key Value Pair": functionInfoFilterKeyValues

Output data for ascertained function header information

Key	Explanation	IMSApi data type
FUNCTION_DESC	Short description of the function	☐ String
FUNCTION_NAME	Name of the function	☐ String
FUNCTION_TYPE	Function type in the customer method [0; 1]: 0 = NOT relevant for the OPCUA service 1 = Relevant for the OPCUA service	☐ int
MDA_NAME	Unique name of the MDA item	☐ String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	☐ int
MDA_VERSION	Version number of the MDA item	☐ String
SINCE_VERSION	Version of the function	☐ String

Tab. 9-14 "Key Value Pair": *functionInfoResultKeys*

Output data for ascertained INPUT parameters

Key	Explanation	IMSApi data type
CF_DATA_TYPE	Data type of the INPUT parameters; the following JAVA data types are supported: - String - Number - Boolean - Object - Symbol	☐ String
CF_IN_PARAM_DESC	Description of the INPUT parameter	☐ String
CF_IN_PARAM_NAME	Name of the INPUT parameter	☐ String

Tab. 9-15 "Key Value Pair": *parameterInfoResultKeys*

Output data for ascertained OUTPUT parameters

Key	Explanation	IMSApi data type
CF_DATA_TYPE	Data type of the OUTPUT parameters; the following JAVA data types are supported: - String - Number - Boolean - Object - Symbol	☐ String
CF_OUT_PARAM_DESC	Description of the OUTPUT parameter	☐ String

Tab. 9-16 "Key Value Pair": *returnInfoResultKeys*

Output data for ascertained output values (error codes)

Key	Explanation	IMSApi data type
CF_ERROR_CODE	Output value (error code) of the function Advice: The output value [0] means that the function was executed without failure/warning!	☐ int
CF_ERROR_TEXT	Output text for output value (CF_ERROR_CODE)	☐ String

Tab. 9-17 "Key Value Pair": *errorCodeResultKeys*

9.4.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
functionInfoFilterKeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": functionInfoFilterKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": functionInfoFilterKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
functionInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": functionInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>parameterInfoResultValues</code> !
parameterInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": parameterInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>parameterInfoResultValues</code> !
returnInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": returnInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>returnInfoResultValues</code> !
errorCodeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": errorCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>errorCodeResultValues</code> !

Tab. 9-18 Parameter (in) »*customGetFunctionInfo*«

Parameter (out)

Parameter name	Explanation
parameterInfoResultValues	Depending on parameter (in) <code>functionInfoResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>functionInfoResultKeys</code>!</i>
parameterInfoResultValues	Depending on parameter (in) <code>parameterInfoResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>parameterInfoResultKeys</code>!</i>
returnInfoResultValues	Depending on parameter (in) <code>returnInfoResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>returnInfoResultKeys</code>!</i>
errorCodeResultValues	Depending on parameter (in) <code>errorCodeResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>errorCodeResultKeys</code>!</i>

Tab. 9-19 Parameter (out) »customGetFunctionInfo«

9.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1509	Warning	Function metadata in script is incorrect
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1500	Failure	Script not found
-1501	Failure	Script is not active
-2070	Failure	IMSApi key was passed multiple times to the function

9.5

customGetFunctions

9.5.1

Function description

This function outputs the customer method stored in the MDA module, together with the associated function header information. Only MDA items that are **released** and **active** are taken into consideration. Different filter criteria can be applied for the query (at present the name of the MDA item and the function type). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

**Advice**

*For customer methods to be output, they must be available as an MDA item in the MDA module under the MDA type "Script (custom function)"! As a rule, **multiple** functions can be contained in **one** customer method (i.e. in one script). Details on the output information can be found under the function »[customGetFunctionInfo](#)« [▶▶▶] (see there the description of the metadata).*

Background: A customer method can be used to map complex, customer-specific IMSApi workflows. With smart.FactoryAppStudio, iTAC offers two tools with which customer methods including the desired metadata can be created and directly integrated as a script in the MDA module. The system uses the metadata item `FUNCTION_TYPE` to determine whether the customer method is a general customer method (created using the "IMSApi FlowDesigner" tool) or a customer method specially developed for the OPCUA service ("OPCUA Studio" tool).

Licensing

NO special licensing is required for this API function.

Function declaration

```
int customGetFunctions
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in KeyValue[] functionFilterKeyValues
    String key
    String value
  in String[] functionInfoResultKeys
  out String[] functionInfoResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `functionFilterKeyValues` and `functionInfoResultKeys` as well as (out) `functionInfoResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNCTION_TYPE	Function type in the customer method [0; 1]: 0 = NOT relevant for the OPCUA service 1 = Relevant only for the OPCUA service Advice: <i>If this key is not passed, there is no restriction!</i>	☐ int
MDA_NAME	Unique name of the MDA item Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String

Tab. 9-20 "Key Value Pair": *functionFilterKeyValues*

Output parameters for ascertained customer methods

Key	Explanation	IMSApi data type
FUNCTION_DESC	Short description of the function Advice: <i>A customer method, i.e. a script, can contain multiple independent functions!</i>	☐ String
FUNCTION_NAME	Name of the function Advice: <i>A customer method, i.e. a script, can contain multiple independent functions!</i>	☐ String
FUNCTION_TYPE	Function type in the customer method [0; 1]: 0 = NOT relevant for the OPCUA service 1 = Relevant for the OPCUA service Advice: <i>A customer method, i.e. a script, can contain multiple independent functions!</i>	☐ int
MDA_NAME	Unique name of the MDA item	☐ String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	☐ int
MDA_VERSION	Version number of the MDA item	☐ String
SINCE_VERSION	Version of the function Advice: <i>A customer method, i.e. a script, can contain multiple independent functions!</i>	☐ String

Tab. 9-21 "Key Value Pair": *functionInfoResultKeys*

9.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
functionFilter- KeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": functionFilterKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": functionFilterKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
functionInfoRe- sultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": functionInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>functionInfoResultValues</code> !

Tab. 9-22 Parameter (in) »customGetFunctions«

Parameter (out)

Parameter name	Explanation
functionInfoRe- sultValues	Depending on parameter (in) <code>functionInfoResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>functionInfoResultKeys</code> !

Tab. 9-23 Parameter (out) »customGetFunctions«

9.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-1510	Failure	Function type is invalid
-2070	Failure	IMSApi key was passed multiple times to the function

9.6 customModifyDbObjectData

9.6.1 Function description

This function enables the editing of data in customer-specific database objects in the CUST schema; this applies **only** for tables – views cannot be edited using this function. Table rows can be inserted (**INSERT**), updated (**UPDATE**) and deleted (**DELETE**). The new creation or structural changes of database objects – this also includes the creation of new table columns – must be performed manually via SQL scripts (see following advice).



Advice

*Note that customer-specific database objects must be created manually using corresponding **SQL scripts**. The required SQL scripts must be specified during the course of commissioning, taking the iTAC specifications into consideration, and implemented together with the iTAC employee responsible for the project!*

*In the array returned, all processed records are always returned – in the event of an error, with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = **-2120**)!*

Specification "DbObject" (table)

The table below lists the specifications for generating a customer-specific table in the CUST schema; the described fields must be present in every new table.

Name	DB type / zero allowed	Description
ID	NUMBER(19,0) ☐	Primary key of the record; this is filled during INSERT and must be used as reference during UPDATE / DELETE.
PLANT_ID	NUMBER(19,0) ☐	Internal identification number of the site; when a new record is created, this identification number is automatically ascertained and entered on the basis of the station number of the API.
CREATED	TIMESTAMP ☐	Date of creation of the server
STAMP	TIMESTAMP ☐	Date on which the server was last changed
KAP_ID	NUMBER(19,0) ☐	Internal identification number of the station which has booked the record (key word "API caller").
USER_ID	NUMBER(19,0) ☒	Internal identification number of the user who has booked the record (key word "API caller").

Tab. 9-24 Base table "DbObject"

Customer-specific columns must be added to the described "base table" in advance; the following applies here:

- A separate sequence must be created for each table (an associated sequence table under MsSQL); sequence name: **SEQ_CT_[TABLENAME]**.
- Tables must begin with the prefix "CT_" (views with "CV_").
- Table names and column names must not exceed 20 characters in length.
- A table must NOT contain more than 45 columns.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	User tables (count): Site license for a certain number of tables
--	---



Advice

Important information for support and project implementation:

- Each user table **MUST** be licensed!
- The maximum usable number of columns per table is 45!

Function declaration

```
int customModifyDBObjectData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] dbObjectModifyKeyValues
    String key
    String value
in String[] dbColumnInsertKeys
in String[] dbColumnInsertValues
in String[] dbColumnInsertResultKeys
in String[] dbColumnUpdateKeys
in String[] dbColumnUpdateValues
in String[] dbColumnUpdateResultKeys
in String[] dbRowDeleteFilterKeys
in String[] dbRowDeleteFilterValues
out String[] dbColumnInsertResultValues
out String[] dbColumnUpdateResultValues
out String[] dbColumnDeleteResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) dbObjectModifyKeyValues, dbColumnInsertKeys, dbColumnInsertResultKeys, dbColumnUpdateKeys, dbColumnUpdateValues, dbColumnUpdateResultKeys, dbRowDeleteFilterKeys and dbRowDeleteFilterValues as well as (out) dbColumnInsertResultValues, dbColumnUpdateResultValues and dbColumnDeleteResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Definition of the database object to be adapted (DB table/DB view)

Key	Explanation	IMSApi data type
DB_OBJECT_NAME	Name of the database object; this is the table name Advice: <i>Customer-specific tables always begin with the prefix "CT_"!</i>	☒ String

Tab. 9-25 "Key Value Pair": *dbObjectModifyKeyValues***INSERT: Definition of the input parameters for "new record" (table row)**

Key	Explanation	IMSApi data type
[0] - [n]	Placeholder of the names of new table rows in the above-named database object (table); the associated values are passed using the structure <code>dbColumnInsertValues</code> . Advice: <i>Structural changes – e.g., the insertion of new table columns – are not possible with this function!</i>	☒ String

Tab. 9-26 "Key Value Pair": *dbColumnInsertKeys***INSERT: Definition of the output parameters for the new record**

Key	Explanation	IMSApi data type
DB_DATA_ID	Primary key of the record	☐ long
DB_ERROR_CODE	Error code: Database-specific return in the event of a failure	☐ int
DB_ERROR_MESSAGE	Error message: Database-specific return in the event of a failure	☐ String
DB_SQL_STATE	SQL state: Database-specific return in the event of a failure	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!</i>	☒ int

Tab. 9-27 "Key Value Pair": *dbColumnInsertResultKeys***UPDATE: Definition of the input parameters for the record to be changed**

Key	Explanation	IMSApi data type
DB_COLUMN_NAME	Name of the table column	☒ String
DB_DATA_ID	Primary key of the record Advice: <i>The primary key value is defined by the system during INSERT; if necessary, the key can be ascertained using the IMSApi function »customGetDbObjectData« [▶▶▶]!</i>	☒ long
VALUE	Value of the record	☒ String

Tab. 9-28 "Key Value Pair": *dbColumnUpdateKeys*

UPDATE: Definition of the output parameters for the changed record

Key	Explanation	IMSApi data type
DB_DATA_ID	Primary key of the record	☐ long
DB_ERROR_CODE	Error code: Database-specific return in the event of a failure	☐ int
DB_ERROR_MESSAGE	Error message: Database-specific return in the event of a failure	☐ String
DB_SQL_STATE	SQL state: Database-specific return in the event of a failure	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒ int

Tab. 9-29 "Key Value Pair": *dbColumnUpdateResultKeys*

DELETE: Definition of the record to be deleted with output parameter (IN/OUT)

Key	Explanation	IMSApi data type
DB_DATA_ID	Primary key of the record Advice: The primary key value is defined by the system during INSERT; if necessary, the key can be ascertained using the IMSApi function »customGetDbObjectData« [▶▶▶]!	☐ long
DB_ERROR_CODE	Error code: Database-specific return in the event of a failure	☐ int
DB_ERROR_MESSAGE	Error message: Database-specific return in the event of a failure	☐ String
DB_SQL_STATE	SQL state: Database-specific return in the event of a failure	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒ int

Tab. 9-30 "Key Value Pair": *dbRowDeleteFilterKeys*

Example "INSERT"

A function call with the parameters given in the example would result in the INSERT statement shown below.

```
KeyValue[] adbObjectModifyKeyValues = new KeyValue[]{
    new KeyValue("DB_OBJECT_NAME", "CT_TEST_TABLE")
};
String[] adbColumnInsertKeys = {
    "COL1", "COL2"
};
//two rows
String[] adbColumnInsertValues = {
    "A", "1", // first row: COL1=A, COL2=1
    "B", "2" //second row: COL1=B, COL2=2
};
String[] adbColumnInsertResultKeys = {
    "DB_DATA_ID",
    "ERROR_CODE",
    "DB_ERROR_CODE",
    "DB_ERROR_MESSAGE",
    "DB_SQL_STATE"
};
```

```
INSERT INTO cust.ct_test_table (coll, col2, id, user_id, kap_id, plant_id, created, stamp) VALUES (?, ?, ?, ?, ?, ?, ?, ?);
/** Parameters: {A,1,41,-11,5,3000000,2016-10-18 08:31:34.906,2016-10-18 08:31:34.906} */
INSERT INTO cust.ct_test_table (coll, col2, id, user_id, kap_id, plant_id, created, stamp) VALUES (?, ?, ?, ?, ?, ?, ?, ?);
/** Parameters: {B,2,42,-11,5,3000000,2016-10-18 08:31:34.906,2016-10-18 08:31:34.906} */
```

Example "UPDATE"

A function call with the parameters given in the example would result in the UPDATE statement shown below.

```
String[] adbColumnUpdateKeys = {
    "DB_DATA_ID",
    "DB_COLUMN_NAME",
    "VALUE"
};
String[] adbColumnUpdateValues = {
    "1", "COL1", "B",
    "1", "COL2", "2"
};
String[] adbColumnUpdateResultKeys = {
    "DB_DATA_ID",
    "ERROR_CODE2",
    "DB_ERROR_CODE",
    "DB_ERROR_MESSAGE",
    "DB_SQL_STATE"
};
```

```
UPDATE cust.ct_test_table SET coll = ?, col2 = ?, stamp = ? WHERE id = ?;
/** Parameters: {B,2,2016-10-18 08:37:14.794,-99} */
```

Example "DELETE"

A function call with the parameters given in the example would result in the DELETE statement shown below.

```
String[] adbRowDeleteFilterKeys = {
    "DB_DATA_ID",
    "ERROR_CODE",
    "DB_ERROR_CODE",
    "DB_ERROR_MESSAGE",
    "DB_SQL_STATE"
};
String[] adbRowDeleteFilterValues = {
    "1", "", "", "", ""
};
```

```
DELETE FROM cust.ct_test_table WHERE id = ?;
/** Parameters: {1} */
```

9.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
dbObjectModifyKeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbObjectModifyKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": dbObjectModifyKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnInsertKeys	List of the "free" parameters based on which the INSERT function is to be executed; the respective values are passed with the parameter <code>dbColumnInsertValues</code> (for further information, see also the table "Key Value Pair": dbColumnInsertKeys below).
dbColumnInsertValues	INSERT value list for the parameters passed by means of <code>dbColumnInsertKeys</code> .
dbColumnInsertResultKeys	List of the parameters that are to be ascertained by the INSERT function. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnInsertResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>dbColumnInsertResultValues</code> !
dbColumnUpdateKeys	List of the parameters on the basis of which the UPDATE function is to be executed; the respective values are passed with the <code>dbColumnUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
dbColumnUpdateValues	UPDATE values for the parameters passed using <code>dbColumnUpdateKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>dbColumnUpdateKeys</code> ! Refer to the "Key Value Pair": dbColumnUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dbColumnUpdateResultKeys	List of the parameters that are to be ascertained by the UPDATE function. A description of the possible parameters can be found above in the "Key Value Pair": dbColumnUpdateResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>dbColumnUpdateResultValues</code> !

Parameter name	Explanation
dbRowDeleteFilterKeys	List of the parameters on the basis of which the DELETE function is to be executed; the respective values are passed with the dbRowDeleteFilterValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": dbRowDeleteFilterKeys table. Advice: Only the parameters passed here are processed by the function! Note that this is an IN/OUT structure!
dbRowDeleteFilterValues	Values for the parameters selected using dbRowDeleteFilterKeys ; no entry is necessary for the parameters DB_ERROR_CODE , DB_ERROR_MESSAGE , DB_SQL_STATE and ERROR_CODE because they are filled automatically by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) dbRowDeleteFilterKeys ! Refer to the "Key Value Pair": dbRowDeleteFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 9-31 Parameter (in) »customModifyDBObjectData«

Parameter (out)

Parameter name	Explanation
dbColumnInsertResultValues	INSERT: Depending on the parameter (in) dbColumnInsertResultKeys , information on the inserted records is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) dbColumnInsertResultKeys !
dbColumnUpdateResultValues	UPDATE: Depending on the parameter (in) dbColumnUpdateResultKeys , information on the updated records is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) dbColumnUpdateResultKeys !
dbColumnDeleteResultValues	DELETE: Depending on the parameter (in) dbRowDeleteFilterKeys , information on the deleted records is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) dbRowDeleteFilterKeys !

Tab. 9-32 Parameter (out) »customModifyDBObjectData«

9.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-1503	Failure	DBObject invalid
-1504	Failure	DBObject not found
-1505	Failure	DBObject -> Change failed
-1506	Failure	DBObject not licensed
-1507	Failure	DBObject -> Maximum number of columns exceeded
-2070	Failure	IMSApi key was passed multiple times to the function
-2120	Failure	At least one record in the array could not be processed

10 Group "EDA" (external data)

10.1 edaAssignExternalData

10.1.1 Function description

This function makes it possible to assign one or more numerical values from one or more number lists (key word "external data") to a serial number or remove this assignment (key HANDLING_MODE). Assigned numerical values are given the "used" state and can, thus, not be assigned elsewhere.



Advice

Currently, only serial numbers can be processed in the context of "external data". The function is designed in such a way that additional object types can be added if necessary!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int edaAssignExternalData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in String[] assignExternalDataUploadKeys
in String[] assignExternalDataUploadValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) assignExternalDataUploadKeys and (out) assignExternalDataUploadValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
EDA_LIST_NAME	Name of the number list Advice: This key is optional since a number (EDA_VALUE) must be unique for a number list type (EDA_TYPE); for this reason, the name of the number list is not required in order to ascertain the number list!	☐ String
EDA_TYPE	Number list type	☒ String
EDA_VALUE	Numerical value from the number list	☒ String
HANDLING_MODE	Function mode [0; 1]: 0 = numerical value is added 1 = numerical value is removed	☒ int

Tab. 10-1 "Key Value Pair": *assignExternalDataUploadKeys*

10.1.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type [0]: 0 = Serial number Advice: Currently, external data can only be assigned to serial numbers (objectType = 0)!
objectNumber	Serial number Advice: This value must match the selected object type (cf. parameter objectType); currently, external data can only be assigned to serial numbers!
objectDetail	Serial number position Advice: Field is only evaluated if serial numbers are to be processed (objectType = 0)!
assignExternalDataUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the assignExternalDataUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignExternalDataUploadKeys table. Advice: Only the parameters passed here are processed by the function!
assignExternalDataUploadValues	Values for the parameters selected by means of assignExternalDataUploadKeys. Advice: The data is assigned in the order in which it was entered by means of parameter (in) assignExternalDataUploadKeys! Refer to the "Key Value Pair" (xxx): assignExternalDataUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 10-2 Parameter (in) »edaAssignExternalData«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

10.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-130	Failure	Work order not found

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3202	Failure	External data not assigned
-3204	Failure	External data -> Too few data
-3207	Failure	Invalid value for the control parameter
-3208	Failure	Invalid state
-3210	Failure	Type not found

10.2 edaGetExternalDataForObject

10.2.1 Function description

By means of this function, it is possible to ascertain the number values assigned to a serial number (key word "external data").



Advice

The array returned contains records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Currently, only serial numbers can be processed in the context of "external data". The function is designed in such a way that additional object types can be added if necessary!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int edaGetExternalDataForObject
in IMSApiSessionContextStruct sessionContext
  long sessionId
  long persId
  String locale
in String stationNumber
in String[] externalDataFilterKeys
in String[] externalDataFilterValues
in String[] externalDataResultKeys
out String[] externalDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) externalDataFilterKeys, externalDataFilterValues and externalDataResultKeys as well as (out) externalDataResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DISSOLVING_LEVEL	Resolve the product structure [0; 1]: 0 = Product structure is NOT resolved 1 = Product structure is resolved in both directions; here, the numbers are ascertained and output for all serial numbers from the product structure	☐ int
EDA_OBJECT_TYPE	Determines the object type [0]: 0 = Serial number Advice: <i>Currently, external data can only be assigned to serial numbers (objectType = 0)!</i>	☒ int
OBJECT_DETAIL	Serial number position Advice: <i>Field is only evaluated if serial numbers are to be processed (EDA_OBJECT_TYPE = 0)!</i>	☐ String
OBJECT_NUMBER	Serial number Advice: <i>This value must match the selected object type (cf. parameter EDA_OBJECT_TYPE); currently external data can only be assigned to serial numbers!</i>	☒ String

Tab. 10-3 "Key Value Pair": `externalDataFilterKeys`

Definition of the output parameters for the found "external data"

Key	Explanation	IMSApi data type
EDA_FIRST_VALUE	First numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LAST_VALUE	Last numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LIST_DESCRIPTION	Description of the number list	☐ String
EDA_LIST_NAME	Name of the number list	☐ String
EDA_OBJECT_TYPE	Object type [0]: 0 = Serial number	☐ int
EDA_TYPE	Number list type	☐ String
EDA_VALUE	Assigned numerical value from the number list	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☐ int

Key	Explanation	IMSApi data type
LEVEL	Display of the integration level relative to the passed serial number [> 0; 0; < 0]: > 0 = Number of integration levels downward 0 = Passed serial number < 0 = Number of integration levels upward Thus, subordinate integration levels are returned with values > 0; i.e., the value increases the deeper the merge structure is resolved. For parent integration levels, a level < 0 is returned; i.e., the highest integration level has the lowest value. Example: <i>The integration levels should be resolved in all directions and over all integration levels; query with SN3:</i> <pre> PROD_A (SN1) LEVEL = -2 -> PROD_B (SN2) LEVEL = -1 -> PROD_C (SN3) LEVEL = 0 -> PROD_D (SN4) LEVEL = 1 -> PROD_E (SN5) LEVEL = 2 </pre> Advice: <i>This key may only be selected if the DISSOLVING_LEVEL key is passed with 1 in the "Key Value Pair": externalDataFilter-Keys table!</i>	☐ int
OBJECT_DETAIL	Serial number position	☐ String
OBJECT_NUMBER	Serial number	☐ String

Tab. 10-4 "Key Value Pair": *externalDataResultKeys*

10.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
externalDataFilterKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>externalDataFilterValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": externalDataFilterKeys table.
externalDataFilterValues	Value list for the parameters passed by means of <code>externalDataFilterKeys</code> . Advice: Refer to the "Key Value Pair": externalDataFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
externalDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": externalDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>externalDataResultValues</code> !

Tab. 10-5 Parameter (in) »edaGetExternalDataForObject«

Parameter (out)

Parameter name	Explanation
externalDataResultValues	Depending on parameter (in) <code>externalDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>externalDataResultKeys</code> !

Tab. 10-6 Parameter (out) »edaGetExternalDataForObject«

10.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3207	Failure	Invalid value for the control parameter

10.3

edaGetNextFreeExternalData

10.3.1

Function description

By means of this function, it is possible to ascertain one or more free number values from a number list (key word "external data"); under certain conditions, it is possible to directly assign the ascertained number values (see parameter `functionMode`).

**Advice**

Before numbers can be ascertained with this function, the number list must be assigned to a part or part group in the setup dialog of the ADM module!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int edaGetNextFreeExternalData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int objectType
  in String objectNumber
  in String objectDetail
  in int functionMode
  in String[] nextFreeExternalDataFilterKeys
  in String[] nextFreeExternalDataFilterValues
  in String[] nextFreeExternalDataResultKeys
  out String[] nextFreeExternalDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `nextFreeExternalDataFilterKeys`, `nextFreeExternalDataFilterValues` and `nextFreeExternalDataResultKeys` as well as (out) `nextFreeExternalDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
EDA_LIST_NAME	Name of the number list Advice: This key is optional since a number (EDA_VALUE) must be unique for a number list type (EDA_TYPE); for this reason, the name of the number list is not required in order to ascertain the number list!	☐ String
EDA_TYPE	Number list type	☒ String
RESULT_QUANTITY	Number of record to be output; here, this is the number of required numerical values Advice: If the requested number of numerical values cannot be ascertained, the function outputs a failure (see error code -3204)!	☒ int

Tab. 10-7 "Key Value Pair": *nextFreeExternalDataFilterKeys*

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
EDA_FIRST_VALUE	First numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LAST_VALUE	Last numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LIST_DESCRIPTION	Description of the number list	☐ String
EDA_LIST_NAME	Name of the number list	☐ String
EDA_TYPE	Number list type	☐ String
EDA_VALUE	Ascertained numerical value from the number list	☐ String

Tab. 10-8 "Key Value Pair": *nextFreeExternalDataResultKeys*

10.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type on the basis of which the number list is to be ascertained [0; 1; 2; 3]: 0 = serial number (part or part group is ascertained on the basis of the serial number); here, the number list is first searched for on the part and, if not successful, on the part group 1 = part number; here, the number list is searched for only on the passed part 2 = part group; here, the number list is searched for only on the passed part group 3 = order number (part or part group is ascertained on the basis of the work order); here, the number list is first searched for on the part and, if not successful, on the part group Advice: Before numbers can be ascertained with this function, the number list must first be assigned to a part or part group in the set-up dialog of the ADM module!
objectNumber	Serial number, part number, part group or order number Advice: This value must match the selected object type (cf. parameter objectType)!
objectDetail	Serial number position Advice: Field is only evaluated if the query is based on a serial number (EDA_OBJECT_TYPE = 0)! Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
functionMode	"State change" function mode [0; 1; 2]: 0 = ascertained number(s) retain the "free" state 1 = ascertained number(s) are given the "reserved" state 2 = ascertained number(s) are given the "assigned" state Advice: Automatic assignment (mode [2]) is only possible if the passed object objectNumber is unique; this is currently only possible for objectType = 0!
nextFreeExternalDataFilterKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the nextFreeExternalDataFilterValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": nextFreeExternalDataFilterKeys table.
nextFreeExternalDataFilterValues	Value list for the parameters passed by means of nextFreeExternalDataFilterKeys. Advice: Refer to the "Key Value Pair": nextFreeExternalDataFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
nextFreeExternalDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": nextFreeExternalDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) nextFreeExternalDataResultValues !

Tab. 10-9 Parameter (in) »edaGetNextFreeExternalData«

Parameter (out)

Parameter name	Explanation
nextFreeExternalDataResultValues	Depending on parameter (in) nextFreeExternalDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) nextFreeExternalDataResultKeys !

Tab. 10-10 Parameter (out) »edaGetNextFreeExternalData«

10.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found

-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-130	Failure	Work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3202	Failure	External data not assigned
-3204	Failure	External data -> Too few data
-3208	Failure	Invalid state
-3210	Failure	Type not found
-3211	Failure	Type is invalid

10.4 edaGetObjectsForExternalData

10.4.1 Function description

By means of this function, it is possible to ascertain the serial numbers assigned to a numerical value (key word "external data").



Advice

*The array returned contains records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Currently, only serial numbers can be processed in the context of "external data". The function is designed in such a way that additional object types can be added if necessary!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int edaGetObjectsForExternalData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] objectExternalDataFilterKeys
  in String[] objectExternalDataFilterValues
  in String[] objectExternalDataResultKeys
  out String[] objectExternalDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) objectExternalDataFilterKeys, objectExternalDataFilterValues and objectExternalDataResultKeys as well as (out) objectExternalDataResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
DISSOLVING_LEVEL	"Serial number ascertaining" mode [0; 1]: 0 = product structure is NOT resolved; the original serial number is always returned – even if it was already mounted, i.e., is inactive. 1 = product structure is resolved; based on the entered serial number, the total product structure is searched.	☐ String
EDA_LIST_NAME	Name of the number list	☐ String
EDA_TYPE	Number list type	☐ String
EDA_VALUE	Assigned numerical value from the number list	☐ String

Tab. 10-11 "Key Value Pair": *objectExternalDataFilterKeys*

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
EDA_FIRST_VALUE	First numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LAST_VALUE	Last numerical value from the number list according to EDA_LIST_NAME	☐ String
EDA_LIST_DESCRIPTION	Description of the number list	☐ String
EDA_LIST_NAME	Name of the number list	☐ String
EDA_OBJECT_TYPE	Determines the object type [0]: 0 = Serial number Advice: Currently, external data can only be assigned to serial numbers (EDA_OBJECT_TYPE = 0)!	☐ int
EDA_TYPE	Number list type	☐ String
EDA_VALUE	Assigned numerical value from the number list	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
LEVEL	Level in the product structure [-1; 0; >0]: > 0 = integration level of the serial number; the value specifies the integration level (level) in which the ascertained serial number is located for the queried serial number 0 = serial number is a master serial number -1 = serial number is an inactive slave serial number Advice: The value [-1] is only output if the DISSOLVING_LEVEL key was passed with [0] and the ascertained serial number is inactive (mounted)!	☐ int
OBJECT_DETAIL	Serial number position	☐ String
OBJECT_NUMBER	Serial number	☐ String

Tab. 10-12 "Key Value Pair": *objectExternalDataResultKeys*

10.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectExternal-DataFilterKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>objectExternalDataFilterValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": objectExternalDataFilterKeys table.
objectExternal-DataFilterValues	Value list for the parameters passed by means of <code>objectExternalDataFilterKeys</code> . Advice: Refer to the "Key Value Pair": objectExternalDataFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
objectExternal-DataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": objectExternalDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>objectExternalDataResultValues</code> !

Tab. 10-13 Parameter (in) »edaGetObjectsForExternalData «

Parameter (out)

Parameter name	Explanation
objectExternal-DataResultValues	Depending on parameter (in) <code>objectExternalDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>objectExternalDataResultKeys</code> !

Tab. 10-14 Parameter (out) »edaGetObjectsForExternalData «

10.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3207	Failure	Invalid value for the control parameter

10.5 edaManageExternalData

10.5.1 Function description

This function can be used to manage number lists; depending on the function mode, (see parameter `functionMode`), number lists can be created, adapted as well as deleted, provided they do not yet contain any used numerical values.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int edaManageExternalData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int functionMode
in String[] partGroupArray
in String[] partNumberArray
in KeyValue[] manageExternalData
    String key
    String value
in String[] externalDataValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `manageExternalData`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
DATE_CREATED	Date of creation/date of change for the number list; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs
EDA_FIRST_VALUE	First numerical value from the number list according to EDA_LIST_NAME Special case: If the value [-1] is passed, the EDA_FIRST_VALUE from the first value of the externalDataValues parameter is ascertained and assigned. If an empty string is passed, no "first numerical value" is created. Advice: This key is only evaluated if creating number lists (see functionMode0!)	☐ String
EDA_LAST_VALUE	Last numerical value from the number list according to EDA_LIST_NAME Special case: If the value [-1] is passed, the EDA_FIRST_VALUE from the first value of the externalDataValues parameter is ascertained and assigned. If an empty string is passed, no "first numerical value" is created. Advice: This key is only evaluated if creating number lists (see functionMode0!)	☐ String
EDA_LIST_DESCRIPTION	Description of the number list	☐ String
EDA_LIST_NAME	Name of the number list	☐ String
EDA_LIST_QUANTITY	Maximum number of numerical values in the list Advice: This key is only evaluated if creating number lists (see functionMode0!)	☐ int
EDA_TYPE	Number list type	☐ String

Tab. 10-15 "Key Value Pair": manageExternalData

10.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
functionMode	Function mode [0; 1; 2; 3; 4; 5]: 0 = create new number list; assign to part or part group by means of <code>partGroupArray</code> or <code>partNumberArray</code> 1 = add numerical values to number list; if the specified number list is not found, it is created in the same way as in mode [0]. 2 = delete number list (including part/part group assignment); deletion is only possible if no numerical values have yet been used. 3 = delete numerical values from number list; deletion is only possible if the numerical value has not yet been used. 4 = assign number list to part or part group (see <code>partGroupArray</code> or <code>partNumberArray</code>) 5 = remove number list assignment for part or part group (see <code>partGroupArray</code> or <code>partNumberArray</code>)
partGroupArray	Part group(s) in the iTAC system (as array) Advice: This parameter is only evaluated if changing list assignments (see <code>functionMode0, 4 and 5</code>)!
partNumberArray	Part number(s) in the iTAC system (as array) Advice: This parameter is only evaluated if changing list assignments (see <code>functionMode0, 4 and 5</code>)!
manageExternalData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>manageExternalData</code> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <code>manageExternalData</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
externalDataValues	Numerical value(s) to be processed (as array) Advice: This parameter is only evaluated if processing numerical values (see <code>functionMode1 and 3</code>)!

Tab. 10-16 Parameter (in) »edaManageExternalData«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

10.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-15	Failure	At least one passed value is too large for the respective database field
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3207	Failure	Invalid value for the control parameter
-3209	Failure	Maximum size exceeded

-3210	Failure	Type not found
-3211	Failure	Type is invalid
-3212	Failure	External data already exist
-3213	Failure	External data - > No name specified

11 Group "EQU" (production means)

11.1 equAssignCarrierToCarrier

11.1.1 Function description

This function is used to assign carrier to carrier; a parent carrier (carrier unit) can be assigned one or more directly subordinate carrier components according to the assignment rule. In addition to the assignment rule, a check is also performed to determine whether the carrier component to be assigned is used elsewhere.

Background to "assignment rule for carriers": Generally speaking, a carrier consists of several different components. These include a fundamental bearer (with/without wheels), possibly different rack elements (bays, mounting rods) and the product holders (receptacles, hook systems). The structure of a carrier is stored as an assignment rule in the master data of the respective part (carrier class); this makes the structure verifiable.



Advice

*The carrier is a subordinate data class of the **production means**.
The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equAssignCarrierToCarrier
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String equipmentNumber
  in String equipmentIndex
  in String[] assignCarrierToCarrierKeys
  in String[] assignCarrierToCarrierValues
  out String[] assignCarrierToCarrierResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) assignCarrierToCarrierKeys and assignCarrierToCarrierValues as well as (out) assignCarrierToCarrierResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier (here the subordinate component, cf. input parameter <code>equipmentIndex</code>) Advice: The index only needs to be passed if the <code>EQUIPMENT_NUMBER</code> itself is NON-unique!	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system (here the subordinate component, cf. input parameter <code>equipmentNumber</code>) Advice: If the carrier number is non-unique, the <code>EQUIPMENT_INDEX</code> key must also be passed!	☒	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒	int

Tab. 11-1 "Key Value Pair": `assignCarrierToCarrierKeys`

11.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentNumber	Number of the carrier in the iTAC system (here the parent carrier unit, cf. key EQUIPMENT_NUMBER)
equipmentIndex	Individual designator for the carrier (here the parent carrier unit, cf. key EQUIPMENT_INDEX) Default: 0 Advice: The index is required in combination with the carrier number for unique designation; i.e. the carrier number itself does not necessarily need to be unique!
assignCarrierTo-CarrierKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the assignCarrierToCarrierValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignCarrierToCarrierKeys table. Advice: Only the parameters passed here are processed by the function!
assignCarrierTo-CarrierValues	Values for the parameters selected by means of assignCarrierToCarrierKeys; no entry is necessary for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) assignCarrierToCarrierKeys! Refer to the "Key Value Pair": assignCarrierToCarrierKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-2 Parameter (in) »equAssignCarrierToCarrier«

Parameter (out)

Parameter name	Explanation
assignCarrierTo-CarrierResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) assignCarrierToCarrierKeys and assignCarrierToCarrierValues. Advice: The data is output in the order in which it was entered by means of parameter (in) assignCarrierToCarrierKeys!

Tab. 11-3 Parameter (out) »equAssignCarrierToCarrier«

11.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1322	Failure	Carrier is already assigned to a carrier
-1323	Failure	Carrier must not be assigned to the carrier
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

11.2

equAssignCavityToEquipment

11.2.1

Function description

This function is used to assign (setup) cavities (injection molds) to work-step-related equipment (here in the context of an "injection mold holder"). It is possible to assign one item of equipment one or more cavities, also different mold classes. In the background, a check is performed for each cavity to determine whether it complies with the defined assignment rule and whether the cavity to be assigned is already being used elsewhere (see the advice notice for the key word "unique setup").

If the assignment of a cavity to equipment is to be removed, the IMSApi function »equRemoveCavityFromEquipment« [▶▶▶] must be used. If the data of an assigned cavity is to be changed (e.g. the activation designator or the work order assignment), the IMSApi function »equUpdateAssignedCavity« [▶▶▶] must be used.



Advice

Key word "unique setup": The "Unique setup" characteristic can be set in the part master data of the cavity (mold class); a unique setup is required in the system by default. This means that a cavity can be assigned to only one mold holder at any one time. For special cases associated with data preparation, the possibility of multiple setups may be necessary, at least for data-technical reasons. If the "Unique setup" characteristic is NOT set, the uniqueness test during setup does not return the otherwise usual error code -1343.

Background to "assignment rules for cavities": The term "cavity" comes from injection molding; a cavity stands for a certain injection mold and can therefore be used for manufacturing a certain product shape. In this context, the equipment is used as an injection mold holder. An injection mold holder can hold multiple, even different cavities. This allows different products to be manufactured simultaneously (key word "work order cluster"). Which injection molds (cavities) can be used in an injection mold holder (equipment) is stored in the master data of the respective part (holder class) as an assignment rule. Assignment rules can be managed using the IMSApi function »equManageCavityEquipmentRule« [▶▶▶].

In addition to the "Cavity to equipment" assignment rule, there is a second, independent rule; it is called the "Product to cavity" rule and is managed using the IMSApi function »equManagePartCavityRule« [▶▶▶]. This rule determines which cavity (mold class) can be used to manufacture which products (unit parts). Note that these products (part numbers/work orders) may differ despite their uniform shape; e.g. owing to a different material, different colors or different further processing. The product actually to be manufactured is assigned to the cavity based on the work order number (for further information, see the IMSApi function »equUpdateAssignedCavity« [▶▶▶]). Only when the work order number has been assigned is the stored assignment rule checked automatically.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*).



Advice

A cavity (PM class = **M**) is a subordinate data object of equipment of PM class = **E**; equipment and therefore also the cavities belong to the "production means" grouping.

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equAssignCavityToEquipment
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] assignEquipmentKeys
    String key
    String value
in String[] assignCavityKeys
in String[] assignCavityValues
out String[] assignCavityResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) assignEquipmentKeys, assignCavityKeys and assignCavityValues as well as (out) assignCavityResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input parameters for the desired equipment (header data)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed!	☒ String

Tab. 11-4 "Key Value Pair": *assignEquipmentKeys*

Input parameters for the cavities to be assigned

Key	Explanation	IMSApi data type
CAVITY_ACTIVE	Activation specification for "cavity" [0; 1]: 0 = Cavity is NOT activated 1 = Cavity is activated Advice: A cavity is only taken into account from a processing viewpoint if it is active. If the key is not passed, the value [1] is used by default!	☐ int
CAVITY_INDEX	Individual designator for the cavity Advice: The index only needs to be passed if the CAVITY_NUMBER itself is NON-unique!	☐ String
CAVITY_NUMBER	Number of the cavity in the iTAC system Advice: If the cavity number is non-unique, the CAVITY_INDEX key must also be passed!	☒ String
CAVITY_POSITION	Position number of the cavity on the equipment Advice: The specified position number must not yet be assigned for the equipment!	☒ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐ int

Tab. 11-5 "Key Value Pair": *assignCavityKeys*

11.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
assignEquipment-Keys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignEquipmentKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": assignEquipmentKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
assignCavityKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>assignCavityValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignCavityKeys table. Advice: Only the parameters passed here are processed by the function!
assignCavityValues	Values for the parameters selected by means of <code>assignCavityKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>assignCavityKeys</code> ! Refer to the "Key Value Pair": assignCavityKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-6 Parameter (in) »equAssignCavityToEquipment«

Parameter (out)

Parameter name	Explanation
assignCavityResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>assignCavityKeys</code> and <code>assignCavityValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>assignCavityKeys</code> !

Tab. 11-7 Parameter (out) »equAssignCavityToEquipment«

11.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1308	Warning	Cavity is invalid
1307	Warning	Cavity test returned warnings
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1332	Failure	Cavity not found
-1333	Failure	Cavity is not unique
-1334	Failure	Cavity part is not allowed for assignment
-1338	Failure	Part number of cavity is required

-1342	Failure	More than one assigned cavity found
-1343	Failure	Not a unique cavity setup
-1344	Failure	Cavity position is already occupied for equipment
-1345	Failure	Equipment cavity has an invalid state
-1346	Failure	Equipment cavity has expired
-1347	Failure	Cavity is invalid
-1348	Failure	Equipment cavity not available
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.3

equAssignSerialNumberToCarrier

11.3.1

Function description

This function is used to assign products (in the form of serial numbers) to carriers; in this case, a carrier can be assigned one or more serial numbers – also different products. A check is performed in the background to determine whether the serial number complies with the defined assignment rule and whether the serial number to be assigned is already used elsewhere.

Background to "assignment rule for products": Due to their physical nature, carriers can be used only for certain products in a certain quantity. Using the "product assignment" function, exactly this assignment to a carrier is defined in the master data of the respective part (carrier class); this makes the assignment verifiable. The products (parts) that can be assigned to the current carrier, and in what quantity – this is then the number of serial numbers – are defined there. The quantity indicates the maximum storage capacity of the carrier for the respective product.



Advice

*The carrier is a subordinate data class of the **production means**.
The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equAssignSerialNumberToCarrier
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String equipmentNumber
in String equipmentIndex
in int setState
in String[] assignSerialNumberToCarrierKeys
in String[] assignSerialNumberToCarrierValues
out String[] assignSerialNumberToCarrierResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) assignSerialNumberToCarrierKeys and assignSerialNumberToCarrierValues as well as (out) assignSerialNumberToCarrierResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the SERIAL_NUMBER must be unique!	☐	String

Tab. 11-8 "Key Value Pair": *assignSerialNumberToCarrierKeys*

11.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentNumber	Number of the carrier in the iTAC system
equipmentIndex	Individual designator for the carrier Default: 0 Advice: The index is required in combination with the carrier number for unique designation; i.e. the carrier number itself does not necessarily need to be unique!
setState	Adaptation of "Carrier state" [0; 1]: 0 = State remains unchanged 1 = State is changed to "in use"
assignSerialNumberToCarrierKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>assignSerialNumberToCarrierValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignSerialNumberToCarrierKeys table. Advice: Only the parameters passed here are processed by the function!
assignSerialNumberToCarrierValues	Values for the parameters selected by means of <code>assignSerialNumberToCarrierKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>assignSerialNumberToCarrierKeys</code> ! Refer to the "Key Value Pair": assignSerialNumberToCarrierKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-9 Parameter (in) »equAssignSerialNumberToCarrier«

Parameter (out)

Parameter name	Explanation
assignSerialNumberToCarrierResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>assignSerialNumberToCarrierKeys</code> and <code>assignSerialNumberToCarrierValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>assignSerialNumberToCarrierKeys</code> !

Tab. 11-10 Parameter (out) »equAssignSerialNumberToCarrier«

11.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1326	Failure	Serial number must not be assigned to the carrier
-1327	Failure	Serial number is already assigned to a carrier
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

11.4

equCheckEquipmentData

11.4.1

Function description

This function checks whether the equipment required for the passed work order is valid and set up at the station. All equipment set up at the station is checked (irrespective of the work order). As a result, it is possible that invalid/expired equipment which is not required to finish the work order is also output.



Advice

*If the equipment to be checked is used in the context of an "injection mold holder", the **cavities** (injection molds) assigned there are also checked!*

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).



Advice

This API function is influenced by site parameter code nos. 4570 [▶▶▶] and 13030 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation.

The various `upload` functions (e.g. »trUploadState« [▶▶▶]) that increment the usage counter for the maintenance interval do not check the equipment for validity; to check, this API function must be used explicitly!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR PM license
--	-----------------------------------

Function declaration

```
int equCheckEquipmentData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String serialNumber
in String partNumber
in int processLayer
in int returnFailuresOnly
in String[] equipmentResultKeys
out String[] equipmentResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equipmentResultKeys` and (out) `equipmentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data-type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
CAVITY_ASSIGNED	Indicates whether the equipment is currently assigned cavities [0; 1]: 0 = NO cavity assigned 1 = Cavity assigned Advice: The value [1] can only be output if the equipment has the "Requires cavity" characteristic (see the key CAVITY_REQUIRED)!	☐	int
CAVITY_CHECKSTATE	Check state for the cavity [0; 1; 2; 3; 5; 6]: 0 = OK: Cavity is valid and set up 1 = Error: Cavity must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [1] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [2] is output! 2 = Warning: Cavity must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [2] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [1] is output! 3 = Error: Cavity state is invalid; the cavity state must have the "available" state 5 = Error: Cavity is invalid; period between "valid from" and "valid to" has been exceeded Advice: [5] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [6] is output! 6 = Warning: Cavity is invalid; period between "valid from" and "valid to" has been exceeded Advice: [6] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [5] is output!	☐	int
CAVITY_INDEX	Individual designator for the cavity	☐	String
CAVITY_NUMBER	Number of the cavity in the iTAC system	☐	String
CAVITY_POSITION	Position number of the cavity on the equipment	☐	int

Key	Explanation	IMSApi data type
CAVITY_REQUIRED	Designator for the "Requires cavity" characteristic [0; 1]: 0 = Characteristic is NOT set 1 = Characteristic is set Advice: <i>If the value [1] is output, the equipment in question can be used as a mold holder in the context of injection molding. This designator is defined in the master data of the part on the basis of which the equipment was created (see there the "Tool management" tab)!</i>	☐ int
CAVITY_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Cavity" are relevant: by default, this is then only the entry "M": M = Cavity (PM class = M)	☐ String
EQUIPMENT_CHECK-STATE	Check state for the equipment [0; 1; 2; 3; 4; 5; 6]: 0 = OK: Equipment is valid and set up 1 = Error: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: <i>[1] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [2] is output!</i> 2 = Warning: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: <i>[2] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [1] is output!</i> 3 = Error: Equipment state is invalid; the equipment state must have the "available" state 4 = Error: Equipment is not set up Advice: <i>The affected internal equipment grouping is output with parameter EQUIPMENT_GROUP!</i> 5 = Error: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: <i>[5] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [6] is output!</i> 6 = Warning: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: <i>[6] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [5] is output!</i>	☐ int
EQUIPMENT_DESCRIPTION	Description of the equipment	☐ String
EQUIPMENT_FACTOR	Factor for the equipment Background: The equipment factor indicates how the usage counter for equipment utilization is calculated. As a result, it is also possible to map application types from the field of mechanics. For example, n strokes (i.e. n equipment operations) are required for punching a part (factor > 1). Or, in the context of multiple panel processing, if multiple parts are manufactured simultaneously in one equipment operation (factor < 1); here, the value is used as a multiplier. If e.g. a quadruple panel is being processed, the factor 0.25 must be entered.	☐ double
EQUIPMENT_GROUP	Name of the internal equipment grouping in the iTAC system; internal groups are created automatically when equipment is assigned to work steps. Advice: <i>This parameter is only output if the value [4] is output for EQUIPMENT_CHECKSTATE!</i>	☐ String

Key	Explanation	IMSApi data type
EQUIPMENT_GROUP_MASTER_DATA	Number of the equipment group in the iTAC system Advice: <i>The use of equipment groups is optional; if they are to be used, they must be stored in the master data (ADM module)!</i>	☐ String
EQUIPMENT_GROUP_MASTER_DATA_DESC	Description of the equipment group in the iTAC system (see the advice notice under EQUIPMENT_GROUP_MASTER_DATA)	☐ String
EQUIPMENT_GRP_ITEM_TYP	Assignment type for the equipment [0; 1]: 0 = Equipment was assigned directly 1 = Equipment was assigned indirectly via a part (device class) (see PART_NUMBER)	☐ int
EQUIPMENT_INDEX	Individual designator for the equipment	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☐ String
EQUIPMENT_NUMBER_EXT	External equipment number	☐ String
EQUIPMENT_STATE	State of the equipment [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: <i>The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!</i>	☐ String
INFO_TXT_1	Individual additional information 1	☐ String
INFO_TXT_2	Individual additional information 2	☐ String
INFO_TXT_3	Individual additional information 3	☐ String
PART_GROUP	Part group for the part (device class)	☐ String
PART_NUMBER	Part number (device class) on the basis of which the equipment was created	☐ String
SETUP_POSITION	Machine position on which the equipment is set up.	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Key	Explanation	IMSApi data type
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	String
WORKPLAN_NUMBER	Part number for the work plan	String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	String
WORKPLAN_VERS	Work plan version in the iTAC system	int
WORKPLAN_VERS_ERP	ERP change number for the work plan	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_NUMBER	Work step number in the iTAC system	int

Tab. 11-11 "Key Value Pair": *equipmentResultKeys*

11.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order Advice: If an empty string or [-1] is passed, the work order active on the station is used or ascertained on the basis of the serial number (serialNumber). If order and serial number are being passed, the serial number must be part of the WO!
serialNumber	Serial number of the panel Advice: If [-1] is passed here, the field is not evaluated; in this case, the work order is ascertained on the basis of the workOrderNumber parameter. If order and serial number are being passed, the serial number must be part of the WO!
partNumber	Part number in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
returnFailuresOnly	Determines which equipment is output [0; 1]: 0 = All equipment is output 1 = Only missing or invalid equipment is output
equipmentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": equipmentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) equipmentResultValues!

Tab. 11-12 Parameter (in) »equCheckEquipmentData«

Parameter (out)

Parameter name	Explanation
equipmentResultValues	Depending on parameter (in) equipmentResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) equipmentResultKeys!

Tab. 11-13 Parameter (out) »equCheckEquipmentData«

11.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1307	Warning	Cavity test returned warnings
1300	Warning	Equipment test returned warnings
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-128	Failure	Work step not found
-130	Failure	Work order not found
-132	Failure	Work order is not unique
-133	Failure	No active work order found for the station
-355	Failure	Production order is part of a work order cluster
-1309	Failure	Equipment test returned failure
-1349	Failure	Setup check for cavity and equipment has failed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.5

equCreateEquipment

11.5.1

Function description

This function is used to create production means; a distinction between the production means classes "Equipment (work step)", "Equipment (asset)", "Cavity" and "Carrier" is currently made in the iTAC system. For which production means class (abbreviated: PM class) a new entry is created depends on the selected production means type (abbreviated: PM type, key: `EQUIPMENT_TYPE`). The PM type indirectly determines the PM class to be used.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equCreateEquipment
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String equipmentUploadKeys
  in String[] equipmentUploadValues
  out String[] equipmentUploadResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equipmentUploadKeys` and `equipmentUploadValues` as well as (out) `equipmentUploadResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
CALIBRATION_INTER- VAL	Default value for the period between two maintenance dates (in days); by default, this value is added to the "Last maintenance" date and used as the default value for the "Next maintenance" following maintenance of a piece of production means.	☐	int
EQUIPMENT_DEFAULT_ FACTOR	Default factor for the equipment master Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"!	☐	double
EQUIPMENT_DESCRIP- TION	Description of the production means Advice: Here, depending on the EQUIPMENT_TYPE, this is the respective equipment, cavity or carrier description!	☐	String
EQUIPMENT_FACTOR	Factor for the equipment Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Background: The equipment factor indicates how the usage counter for equipment utilization is calculated. As a result, it is also possible to map application types from the field of mechanics. For example, n strokes (i.e. n equipment operations) are required for punching a part (factor > 1). Or, in the context of multiple panel processing, if multiple parts are manufactured simultaneously in one equipment operation (factor < 1); here, the value is used as a multiplier. If e.g. a quadruple panel is being processed, the factor 0.25 must be entered.	☐	double
EQUIPMENT_GROUP_ MASTER_DATA	Number of the equipment group in the iTAC system Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Use of "Equipment (work step)": In addition to the classic grouping, equipment groups can also be used when assigning equipment to work steps. Use of "Equipment (asset)": Here, equipment groups are used only for grouping without any other control function. Background: The use of equipment groups is optional. If they are to be used, they must be stored in the master data (ADM module).	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number and only needs to be specified if the EQUIPMENT_NUMBER is not unique!	☐	String

Key	Explanation		IMSApi data type
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number!	☒	String
EQUIPMENT_NUMBER_EXT	External number of the production means Advice: Depending on the EQUIPMENT_TYPE, this is the external equipment number, cavity number, or carrier number!	☒	String
EQUIPMENT_STATE	State of the production means [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used (for carriers only) 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted Advice: Depending on the EQUIPMENT_TYPE, this is the equipment or carrier state; the state [10] is possible for carriers ONLY! If the key is not passed, the value [0] is used by default!	☐	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☒	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
EXPIRATION_DATE	Default value for the point in time at which the production means are next due for maintenance; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
EXPIRATION_DATE_FINAL	Default value for the point in time at which the production means may be used for the last time (expiry date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
EXPIRE_AFTER_CNT_FAIL	Default value for the maximum permissible faulty usages for these production means within a maintenance interval	☐	long
EXPIRE_AFTER_CNT_FAIL_FINAL	Default value for the maximum permissible faulty usages for these production means	☐	long
EXPIRE_AFTER_CNT_TOTAL	Default value for the maximum permissible usages for these production means within a maintenance interval	☐	long

Key	Explanation		IMSApi data type
EXPIRE_AFTER_CNT_- TOTAL_FINAL	Default value for the total maximum permissible usages for these production means	☐	long
EXPIRE_AFTER_TIME_- FINAL	Default value for the total maximum period (in hours) that the equipment/cavity may be actively set up Advice: <i>This key is NOT relevant for production means of the PM class "Carrier"!</i>	☐	long
EXPIRE_AFTER_- TIME_USAGE	Default value for the maximum period (in hours) that the equipment/cavity may be actively set up between two maintenance intervals. Advice: <i>This key is NOT relevant for production means of the PM class "Carrier"!</i>	☐	long
EXPIRE_CNT_THRESH- OLD_PERCENT	Limit value as a percentage which is used to define for the "Max. usages" counter the number by which the default value can be exceeded before the production means are locked for a pending order.	☐	double
EXPIRE_TIME_THRESH- OLD_HOUR	Limit value in hours which specifies the number of hours by which the specified maintenance date may be exceeded before the production means are locked for a pending order.	☐	int
PART_NUMBER	Part number (device class, carrier class, mold/holder class) on the basis of which the production means is created.	☒	String
STORAGE_BARCODE	Bar code for the storage place Advice: <i>The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!</i>	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: <i>See the advice notice under STORAGE_BARCODE!</i>	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String
VALID_FROM	Start of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
VALID_TO	End of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs

Tab. 11-14 "Key Value Pair": *equipmentUploadKeys*

11.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentUpload-Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>equipmentUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentUploadKeys table. Advice: Only the parameters passed here are processed by the function!
equipmentUpload-Values	Values for the parameters selected by means of <code>equipmentUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>equipmentUploadKeys</code>! Refer to the "Key Value Pair": equipmentUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-15 Parameter (in) »equCreateEquipment«

Parameter (out)

Parameter name	Explanation
equipmentUpload-ResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>equipmentUploadKeys</code> and <code>equipmentUploadValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equipmentUploadKeys</code>!

Tab. 11-16 Parameter (out) »equCreateEquipment«

11.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-531	Failure	Supplier not found
-574	Failure	Storage location not found
-581	Failure	Manufacturer number is not unique
-1304	Failure	Production means have an invalid state
-1312	Failure	Production means type is invalid
-1316	Failure	Production means part not found
-1320	Failure	Production means already exist
-1321	Failure	Production means class is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2120	Failure	At least one record in the array could not be processed

11.6

equGetAssetEquipmentSetup

11.6.1

Function description

This function is used to query information on set-up, asset-related equipment. The query can be made about any item in the machine assets – at present these are: site, cell, line, machine group, station and resource (see keys `ENTITY_TYPE` and `ENTITY_KEY DISSOLVING_LEVEL`). If required, the set-up asset equipment of subordinate machine asset structures can also be determined (see key `DISSOLVING_LEVEL`).

The function also supports **searching in the backward direction**; i.e. if the query is performed on the basis of a known equipment number (without any other restrictions), the entire machine asset structure is searched and the entry on which the equipment is currently set up is output. If necessary, this search can be restricted to certain levels/objects of the machine assets.

**Advice**

The IMSApi »equSetupAssetEquipment« [▶▶▶] must be used for the setup and stripdown process of asset-related equipment!

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for asset-related equipment** (PM class: **A**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

Function matrix `ENTITY_TYPE (asset)`

The dependencies between the keys `ENTITY_TYPE` (data object) and `ENTITY_KEY` (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Resources	11	Resource number

Tab. 11-17 Function matrix `ENTITY_TYPE`

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetAssetEquipmentSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] getAssetEquipmentSetupFilters
    String key
    String value
in String[] getAssetEquipmentSetupResultKeys
out String[] getAssetEquipmentSetupResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `getAssetEquipmentSetupFilters` and `getAssetEquipmentSetupResultKeys` as well as (out) `getAssetEquipmentSetupResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DISSOLVING_LEVEL	Resolution of "machine asset structure" [0; 1]: 0 = Only the data on the current level 1 = All data on the current and subordinate levels Advice: If this key is not passed along, the [0] value is used by default!	☐ int
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [▶▶▶]!	☐ String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 11]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 11 = Resources Advice: With the standard search, the ENTITY_TYPE together with the ENTITY_KEY must normally be passed. If, however, it is certain that all data object details used in the site are unique, the key does not have to be passed.	☐ int

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment Advice: <i>The index only needs to be passed if the EQUIPMENT_NUMBER is used and is itself NON-unique!</i>	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: <i>If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed!</i> Application type "backward search": This key activates the backward search and is only relevant if a machine asset entry on which known equipment is set up is searched for. If necessary, this search can be restricted with the ENTITY_TYPE, ENTITY_KEY and DIS-SOLVING_LEVEL keys. If the EQUIPMENT_NUMBER is not passed, a valid ENTITY_KEY MUST be passed for the standard search.	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int

Tab. 11-18 "Key Value Pair": *getAssetEquipmentSetupFilters*

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is output here.	☐ String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 11]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 11 = Resources	☐ int
EQUIPMENT_INDEX	Individual designator for the equipment	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☐ String

Tab. 11-19 "Key Value Pair": *getAssetEquipmentSetupResultKeys*

11.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
getAssetEquipmentSetupFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": getAssetEquipmentSetupFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": getAssetEquipmentSetupFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
getAssetEquipmentSetupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": getAssetEquipmentSetupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>getAssetEquipmentSetupResultValues</code> !

Tab. 11-20 Parameter (in) »equGetAssetEquipmentSetup«

Parameter (out)

Parameter name	Explanation
getAssetEquipmentSetupResultValues	Depending on parameter (in) <code>getAssetEquipmentSetupResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>getAssetEquipmentSetupResultKeys</code> !

Tab. 11-21 Parameter (out) »equGetAssetEquipmentSetup«

11.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1303	Warning	The equipment has expired
1301	Warning	The equipment is already set up
1300	Warning	Equipment test returned warnings
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-574	Failure	Storage location not found
-1301	Failure	Production means not found

-1302	Failure	Production means are invalid
-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1305	Failure	Production means are expired
-1306	Failure	Production means are unavailable
-1307	Failure	Production means data is not consistent
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2072	Failure	Invalid value for "desolvingLevel"
-2105	Failure	No TR or PM license present for the station
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value
-2134	Failure	Entity type is invalid

11.7

equGetAssignedCavity

11.7.1

Function description

This function is used to query cavities (injection molds) that are currently assigned to equipment (here in the context of an "injection mold holder"). Search criteria such as equipment number and state, production means type, part number of the cavity (mold class) or cavity position are available. All search criteria are "AND" coupled such that only the production means data that meet all used filter conditions are returned.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

**Advice**

*A cavity (PM class = **M**) is a subordinate data object of equipment of PM class = **E**; equipment and therefore also the cavities belong to the "production means" grouping!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetAssignedCavity
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] cavityGetAssignedCavityFilters
    String key
    String value
in String[] cavityGetAssignedCavityResultKeys
out String[] cavityGetAssignedCavityResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `cavityGetAssignedCavityFilters` and `cavityGetAssignedCavityResultKeys` as well as (out) `cavityGetAssignedCavityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
CAVITY_ACTIVE	"Cavity" activation designator [0; 1]: 0 = Not active 1 = Active Advice: A cavity is only taken into account from a processing view-point if it is active!	☐ int
CAVITY_PART_NUMBER	Part number (mold class) on the basis of which the cavity was created Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
CAVITY_POSITION	Position number of the cavity on the equipment	☐ int
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☐ String
EQUIPMENT_NUMBER_EXT	External equipment number	☐ String
EQUIPMENT_STATE	State of the equipment [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	☐ String

Key	Explanation	IMSApi data type
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int
WORKORDER_NUMBER	Production order number Advice: Filtering by work order number only makes sense if assignment has already been performed using the IMSApi function »equUpdateAssignedCavity« [▶▶▶]!	☐ String

Tab. 11-22 "Key Value Pair": cavityGetAssignedCavityFilters

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
CAVITY_ACTIVE	"Cavity" activation designator [0; 1]: 0 = Not active 1 = Active	☐ int
CAVITY_INDEX	Individual designator for the cavity	☐ String
CAVITY_NUMBER	Number of the cavity in the iTAC system	☐ String
CAVITY_PART_NUMBER	Part number (mold class) on the basis of which the cavity was created	☐ String
CAVITY_POSITION	Position number of the cavity on the equipment	☐ int
CAVITY_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Cavity" are relevant: by default, this is then only the entry "M": M = Cavity (PM class = M)	☐ String

Key	Explanation	IMSApi data type
CLUSTER_NAME	Unique name for the work order cluster Advice: <i>Work order clusters are used to merge different work orders and to manufacture them in a group (in parallel)!</i> Background to "work order cluster": An injection mold holder can hold multiple, even different cavities. This allows different products/work orders to be manufactured at the same time; for this purpose, the "work order cluster" concept is applied.	☐ String
CLUSTER_POSITION	Unique position number of the work order (WORKORDER_NUMBER) in the cluster	☐ int
EQUIPMENT_INDEX	Individual designator for the equipment	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☐ String
EQUIPMENT_NUMBER_EXT	External equipment number	☐ String
EQUIPMENT_STATE	State of the equipment [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E)	☐ String
WORKORDER_NUMBER	Production order number Advice: <i>A work order number can only be ascertained if assignment is performed beforehand using the IMSApi function »equUpdateAssignedCavity« [▶▶▶]!</i>	☐ String

Tab. 11-23 "Key Value Pair": *cavityGetAssignedCavityResultKeys*

11.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
cavityGetAs-signedCavityFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": cavityGetAssignedCavityFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": cavityGetAssignedCavityFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
cavityGetAs-signedCavityResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": cavityGetAssignedCavityResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>cavityGetAssignedCavityResultValues</code> !

Tab. 11-24 Parameter (in) »equGetAssignedCavity«

Parameter (out)

Parameter name	Explanation
cavityGetAs-signedCavityResultValues	Depending on parameter (in) <code>cavityGetAssignedCavityResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>cavityGetAssignedCavityResultKeys</code> !

Tab. 11-25 Parameter (out) »equGetAssignedCavity«

11.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1304	Failure	Production means have an invalid state
-1312	Failure	Production means type is invalid
-1336	Failure	Production means type not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.8

equGetCarrierAssigned

11.8.1

Function description

This function is used to query carriers to which products (in the form of serial numbers) and/or other carriers are assigned; here, the assignment information can be ascertained based on a wide variety of criteria (e.g. resolution depth and search direction in the carrier structures, part group/part number of the carrier class, carrier number, carrier state, etc.).

Apart from the ascertained carriers themselves, this function can if required also output the serial numbers and carriers assigned there (in separate structures). The result is **ONLY** the current assignment; if assignments from the past are also to be ascertained, the »equGetCarrierAssignmentHistoryData« [▶▶▶] function must be used.

**Advice**

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierAssigned
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] carrierGetFilters
    String key
    String value
  in String[] carrierMainResultKeys
  in String[] carrierSerialNoAssignedResultKeys
  in String[] carrierCarrierAssignedResultKeys
  out String[] carrierMainResultValues
  out String[] carrierSerialNoAssignedResultValues
  out String[] carrierCarrierAssignedResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) carrierGetFilters, carrierMainResultKeys, carrierSerialNoAssignedResultKeys and carrierCarrierAssignedResultKeys as well as (out) carrierMainResultValues, carrierSerialNoAssignedResultValues and carrierCarrierAssignedResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_NUMBER_EXT	External number of the carrier Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String
LEVEL	Determines the resolution depth and search direction in the carrier structures – always based on the reference object [-1; 0; 1]: -1 = All structure levels above and below 0 = One structure level higher and lower 1 = All structure levels below Advice: If this key is not passed along, the [0] value is used by default!	☐ int

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int
PART_GROUP	Part group for the part (carrier class) Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String

Tab. 11-26 "Key Value Pair": *carrierGetFilters*

Output parameters for ascertained carriers (reference objects)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system	☐ String
EQUIPMENT_NUMBER_EXT	External number of the carrier	☐ String
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C)	☐ String
INDEX	Unique reference number (sequential number) for each record in the array Advice: <i>The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see parameter of the same name in the "Key Value Pair": carrierSerialNoAssignedResultKeys and "Key Value Pair": carrierCarrierAssignedResultKeys tables)!</i>	☒ long
PART_GROUP	Part group for the part (carrier class)	☐ String
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created	☐ String

Tab. 11-27 "Key Value Pair": *carrierMainResultKeys*

Output parameters for ascertained product assignments

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier Advice: See the advice notice under the key EQUIPMENT_NUMBER!	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: Here, this is the carrier to which the serial numbers have been assigned – may under certain circumstances differ from the referenced entry as defined in the INDEX key!	☐ String
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": carrierMainResultKeys table)!	☐ long
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Tab. 11-28 "Key Value Pair": carrierSerialNoAssignedResultKeys

Output parameters for ascertained carrier assignments

Key	Explanation	IMSApi data type
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": carrierMainResultKeys table)!	☐ long
PARENT_CARRIER	Number of the parent carrier in the iTAC system Advice: Depending on the selected resolution depth and search direction (see key LEVEL), the reference object itself may also be output here. If the referenced carrier is a carrier base (see key INDEX), no value is output here!	☐ String
PARENT_CARRIER_CNT_USAGE	Number of usages for PARENT_CARRIER (since the last maintenance)	☐ long
PARENT_CARRIER_INDEX	Individual designator for the parent carrier	☐ String
SLAVE_CARRIER	Number of the subordinate carrier in the iTAC system Advice: Depending on the selected resolution depth and search direction (see key LEVEL), the reference object itself may also be output here. If the referenced carrier is an entry from the lowest structure level (see key INDEX), no value is output here!	☐ String

Key	Explanation	IMSApi data type
SLAVE_CARRIER_CNT_USAGE	Number of usages for SLAVE_CARRIER (since the last maintenance)	☐ long
SLAVE_CARRIER_INDEX	Individual designator for the subordinate carrier	☐ String

Tab. 11-29 "Key Value Pair": *carrierCarrierAssignedResultKeys*

11.8.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierGetFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": carrierGetFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierMainResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierMainResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) carrierMainResultValues !
carrierSerialNoAssignedResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierSerialNoAssignedResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) carrierSerialNoAssignedResultValues !
carrierCarrierAssignedResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierCarrierAssignedResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) carrierCarrierAssignedResultValues !

Tab. 11-30 Parameter (in) »*equGetCarrierAssigned*«

Parameter (out)

Parameter name	Explanation
carrierMainResultValues	Depending on parameter (in) <code>carrierMainResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierMainResultKeys</code> !
carrierSerialNoAssignedResultValues	Depending on parameter (in) <code>carrierSerialNoAssignedResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierSerialNoAssignedResultKeys</code> !
carrierCarrierAssignedResultValues	Depending on parameter (in) <code>carrierCarrierAssignedResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierCarrierAssignedResultKeys</code> !

Tab. 11-31 Parameter (out) »equGetCarrierAssigned«

11.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found

-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2117	Failure	Value has exceeded the maximum size
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.9

equGetCarrierAssignmentHistoryData

11.9.1

Function description

This function is used to query assignment information on **one** carrier over a selectable period under review. As a result, it is possible to ascertain the history of assigned products (in the form of serial numbers) and/or the history of assigned carriers. The function is performed based on one serial number or one carrier. If ONLY current assignment information is to be ascertained – here, if required, also for multiple carriers in a single query – the »equGetCarrierAssigned« [▶▶▶] function can be used.



Advice

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierAssignmentHistoryData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] carrierGetFilters
    String key
    String value
in String[] carrierSerialNumberAssignedHistoryResultKeys
in String[] carrierCarrierAssignedHistoryResultKeys
out String[] carrierSerialNumberAssignedHistoryResultValues
out String[] carrierCarrierAssignedHistoryResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) carrierGetFilters, carrierSerialNumberAssignedHistoryResultKeys and carrierCarrierAssignedHistoryResultKeys as well as (out) carrierSerialNumberAssignedHistoryResultValues and carrierCarrierAssignedHistoryResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the product or carrier assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review! Advice: If this key is not passed along, the [-1] value is used by default!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; the booking time for cancelation of the product and carrier assignment applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes the current date and the current time as the end of the period under review! Advice: If this key is not passed along, the [-1] value is used by default!	☐	DateMs
EQUIPMENT_INDEX	Individual designator for the carrier Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER, the EQUIPMENT_NUMBER or the EQUIPMENT_NUMBER_EXT must be passed!	☐	String
EQUIPMENT_NUMBER_EXT	External number of the carrier Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER, the EQUIPMENT_NUMBER or the EQUIPMENT_NUMBER_EXT must be passed!	☐	String
LEVEL	Determines the resolution depth and search direction in the carrier structures – always based on the reference object [-1; 0; 1]: -1 = All structure levels above and below 0 = One structure level higher and lower 1 = All structure levels below Advice: If this key is not passed along, the [0] value is used by default!	☐	int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER, the EQUIPMENT_NUMBER or the EQUIPMENT_NUMBER_EXT must be passed!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the SERIAL_NUMBER must be unique!	☐ String

Tab. 11-32 "Key Value Pair": *carrierGetFilters*

Output parameters for ascertained product assignments

Key	Explanation	IMSApi data type
DATE_FROM	Booking time at which the serial number was assigned (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_TO	Booking time at which the assignment to the serial number was canceled again (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the serial number is currently still assigned, the value "32535215999999" is output as the end time (corresponds to 31-12-3000, 23:59:59.999)!	☐ DateMs
EQUIPMENT_INDEX	Individual designator for the carrier Advice: See the advice notice under the key EQUIPMENT_NUMBER!	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: Here, this is the carrier to which the serial numbers have been assigned. Depending on the selected resolution depth and search direction (see key LEVEL), this may deviate from the queried data object!	☐ String
INDEX	Unique reference number (current number) for the related records in this array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another!	☐ long
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Tab. 11-33 "Key Value Pair": *carrierSerialNumberAssignedHistoryResultKeys*

Output parameters for ascertained carrier assignments

Key	Explanation		IMSApi data type
DATE_FROM	Booking time at which the carrier was assigned (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_TO	Booking time at which the assignment to the carrier was canceled again (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the carrier is currently still assigned, 31-12-3000, 23:59:59.999 is output as the end time!	☐	DateMs
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐	String
INDEX	Unique reference number (current number) for the related records in this array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another!	☐	long
PARENT_CARRIER	Number of the parent carrier in the iTAC system Advice: Depending on the selected resolution depth and search direction (see key LEVEL), the reference object itself may also be output here!	☐	String
PARENT_CARRIER_CNT_USAGE	Number of usages for PARENT_CARRIER (since the last maintenance)	☐	long
PARENT_CARRIER_INDEX	Individual designator for the parent carrier	☐	String
SLAVE_CARRIER	Number of the subordinate carrier in the iTAC system Advice: Depending on the selected resolution depth and search direction (see key LEVEL), the reference object itself may also be output here!	☐	String
SLAVE_CARRIER_CNT_USAGE	Number of usages for SLAVE_CARRIER (since the last maintenance)	☐	long
SLAVE_CARRIER_INDEX	Individual designator for the subordinate carrier	☐	String

Tab. 11-34 "Key Value Pair": carrierCarrierAssignedHistoryResultKeys

11.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierGetFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": carrierGetFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierSerialNumberAssignedHistoryResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierSerialNumberAssignedHistoryResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) carrierSerialNumberAssignedHistoryResultValues !
carrierCarrierAssignedHistoryResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierCarrierAssignedHistoryResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) carrierCarrierAssignedHistoryResultValues !

Tab. 11-35 Parameter (in) »equGetCarrierAssignmentHistoryData«

Parameter (out)

Parameter name	Explanation
carrierSerialNumberAssignedHistoryResultValues	Depending on parameter (in) carrierSerialNumberAssignedHistoryResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) carrierSerialNumberAssignedHistoryResultKeys !
carrierCarrierAssignedHistoryResultValues	Depending on parameter (in) carrierCarrierAssignedHistoryResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) carrierCarrierAssignedHistoryResultKeys !

Tab. 11-36 Parameter (out) »equGetCarrierAssignmentHistoryData«

11.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
1303	Warning	The equipment has expired
-2011	Failure	End date precedes start date
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2117	Failure	Value has exceeded the maximum size
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.10

equGetCarrierAssignRule

11.10.1

Function description

This function is used to query carriers and the assignment rules that are stored there and that apply to products and carrier components; it is possible to ascertain the assignment rules using a wide variety of criteria (e.g. part group/part number of the carrier class, carrier number, carrier state, etc.). All search criteria are "AND" coupled such that only the data sets that meet all used filter conditions are returned.

**Advice**

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

Background to "assignment rule for products": Due to their physical nature, carriers can be used only for certain products in a certain quantity. Using the "product assignment" function, exactly this assignment to a carrier is defined in the master data of the respective part (carrier class); this makes the assignment verifiable. The products (parts) that can be assigned to the current carrier, and in what quantity – this is then the number of serial numbers – are defined there. The quantity indicates the maximum storage capacity of the carrier for the respective product.

Background to "assignment rule for carriers": Generally speaking, a carrier consists of several different components. These include a fundamental bearer (with/without wheels), possibly different rack elements (bays, mounting rods) and the product holders (receptacles, hook systems). The structure of a carrier is stored as an assignment rule in the master data of the respective part (carrier class); this makes the structure verifiable.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierAssignRule
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] carrierGetFilters
    String key
    String value
in String[] carrierGetMainResultKeys
in String[] carrierGetSerialNoAssignRuleResultKeys
in String[] carrierGetCarrierAssignRuleResultKeys
out String[] carrierGetMainResultValues
out String[] carrierGetSerialNoAssignRuleResultValues
out String[] carrierGetCarrierAssignRuleResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `carrier-GetFilters`, `carrierGetMainResultKeys`, `carrierGetSerialNoAssignRuleResultKeys` and `carrierGetCarrierAssignRuleResultKeys` as well as (out) `carrierGetMainResultValues`, `carrierGetSerialNoAssignRuleResultValues` and `carrierGetCarrierAssignRuleResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_NUMBER_EXT	External number of the carrier Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String

Key	Explanation	IMSApi data type
LEVEL	Resolution of the part group by part [0; 1]: 0 = Part group is NOT resolved 1 = Part group is resolved Advice: If the value [1] is passed, a separate result data record is output for each part of the part group. If the key is not passed, the value [0] is used by default!	☐ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int
PART_GROUP	Part group for the part (carrier class)	☐ String
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created	☐ String

Tab. 11-37 "Key Value Pair": *carrierGetFilters*

Output parameters for ascertained carriers (reference objects)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system	☐ String
EQUIPMENT_NUMBER_EXT	External number of the carrier	☐ String
EQUIPMENT_STATE	State of the carrier [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C)	☐ String
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": <i>carrierGetSerialNoAssignRuleResultKeys</i> and "Key Value Pair": <i>carrierGetCarrierAssignRuleResultKeys</i> tables)!	☒ long
PART_GROUP	Part group for the part (carrier class)	☐ String
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created	☐ String

Tab. 11-38 "Key Value Pair": *carrierGetMainResultKeys*

Output parameters for ascertained assignment rules (product)

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": carrierGetMainResultKeys table)!	☐ long
PART_GROUP	Part group in the iTAC system Advice: If part groups are used in the assignment rule, the assignment applies to all parts contained there with correspondingly uniform quantity specification (see also the advice notice under PART_NUMBER)!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: Only products (serial numbers) with this part number may be assigned to the referenced carrier (see INDEX)!	☐ String
QUANTITY	Maximum number of products on the referenced carrier (see INDEX) Advice: The quantity indicates the maximum storage capacity of the carrier for the respective product!	☐ double
VALID_FROM	Start of the validity period for the product assignment; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
VALID_TO	End of the validity period for the product assignment; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Tab. 11-39 "Key Value Pair": carrierGetSerialNoAssignRuleResultKeys

Output parameters for ascertained assignment rules (carrier)

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": carrierGetMainResultKeys table)!	☐ long
PART_GROUP	Part group in the iTAC system Advice: If part groups are used in the assignment rule, the assignment applies to all parts contained there (carrier class) with correspondingly uniform quantity specification!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: Only carrier components with this part number may be assigned to the referenced carrier (see INDEX)!	☐ String
QUANTITY	Maximum number of carrier components on the referenced carrier (see INDEX) Advice: The quantity indicates the maximum storage capacity of the carrier for the respective carrier component!	☐ double

Key	Explanation	IMSApi data type
VALID_FROM	Start of the validity period for the carrier assignment; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
VALID_TO	End of the validity period for the carrier assignment; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Tab. 11-40 "Key Value Pair": *carrierGetCarrierAssignRuleResultKeys*

11.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierGetFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": carrierGetFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierGetMainResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetMainResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>carrierGetMainResultValues</code> !
carrierGetSerialNoAssignRuleResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetSerialNoAssignRuleResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>carrierGetSerialNoAssignRuleResultValues</code> !
carrierGetCarrierAssignRuleResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierGetCarrierAssignRuleResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>carrierGetCarrierAssignRuleResultValues</code> !

Tab. 11-41 Parameter (in) »*equGetCarrierAssignRule*«

Parameter (out)

Parameter name	Explanation
carrierGetMainResultValues	Depending on parameter (in) <code>carrierGetMainResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>carrierGetMainResultKeys</code>!</i>
carrierGetSerialNoAssignRuleResultValues	Depending on parameter (in) <code>carrierGetSerialNoAssignRuleResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>carrierGetSerialNoAssignRuleResultKeys</code>!</i>
carrierGetCarrierAssignRuleResultValues	Depending on parameter (in) <code>carrierGetCarrierAssignRuleResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>carrierGetCarrierAssignRuleResultKeys</code>!</i>

Tab. 11-42 Parameter (out) »equGetCarrierAssignRule«

11.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2117	Failure	Value has exceeded the maximum size

-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.11

equGetCarrierLocation

11.11.1

Function description

This function is used to query storage place data of one or more carriers. The function is performed based on one serial number or one carrier. ONLY the current storage place data is output as the result; if storage place data from the past is also to be ascertained, the »equGetCarrierLocationHist« [▶▶▶] function must be used. If the search filter "carrier" is used – multi selection using wildcards is possible here – multiple storage places may be output. If the search filter "serial number" is used, the function will of course return only one storage place as the result.

**Advice**

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

*The array returned contains records that were **incorrectly** processed along with a corresponding error code (reason for error); i.e., if, for example, no storage place was found for the searched carrier. In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierLocation
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] carrierLocationFilters
    String key
    String value
  in String[] carrierLocationResultKeys
  out String[] carrierLocationResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `carrierLocationFilters` and `carrierLocationResultKeys` as well as (out) `carrierLocationResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER or the EQUIPMENT_NUMBER must be passed!	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int
SERIAL_NUMBER	Serial number of the panel Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER or the EQUIPMENT_NUMBER must be passed!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the SERIAL_NUMBER must be unique!	☐ String

Tab. 11-43 "Key Value Pair": carrierLocationFilters

Output parameters for ascertained storage places

Key	Explanation	IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means as equipment, cavities, and carriers [A; C; E; M]; in this context, this is only the "Carrier" PM class: C = Carriers	☐ String
EQUIPMENT_INDEX	Individual designator for the carrier	☐ String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system	☒ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C)	☐ String

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record; i.e., if, for example, no storage place was found for the searched carrier. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐ String

Tab. 11-44 "Key Value Pair": *carrierLocationResultKeys*

11.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierLocation- Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierLocationFilters table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": carrierLocationFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierLocation- ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierLocationResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>carrierLocationResultValues</i> !

Tab. 11-45 Parameter (in) »equGetCarrierLocation«

Parameter (out)

Parameter name	Explanation
carrierLocation-ResultValues	Depending on parameter (in) <code>carrierLocationResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierLocationResultKeys</code> !

Tab. 11-46 Parameter (out) »equGetCarrierLocation«

11.11.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key

-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1329	Failure	Production means not stored
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

11.12

equGetCarrierToCarrier

11.12.1

Function description

This function is used to query possible carriers to which a certain carrier component could be assigned. Starting with the carrier component to be specified, only the carrier structures immediately above are considered. Only carriers that have the "available" state are output.

Background to "assignment rule for carriers": Generally speaking, a carrier consists of several different components. These include a fundamental bearer (with/without wheels), possibly different rack elements (bays, mounting rods) and the product holders (receptacles, hook systems). The structure of a carrier is stored as an assignment rule in the master data of the respective part (carrier class); this makes the structure verifiable.



Advice

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierToCarrier
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] carrierToCarrierFilters
    String key
    String value
in String[] carrierToCarrierResultKeys
out String[] carrierToCarrierResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `carrierToCarrierFilters` and `carrierToCarrierResultKeys` as well as (out) `carrierToCarrierResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier Advice: <i>The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!</i>	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system	☒	String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒	int

Tab. 11-47 "Key Value Pair": *carrierToCarrierFilters*

Output parameters for ascertained carriers

Key	Explanation		IMSApi data type
CNT_USAGE	Number of usages (since the last maintenance)	☐	long
EQUIPMENT_INDEX	Individual designator for the carrier	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: <i>Only carriers that have the "available" state are output!</i>	☒	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C)	☐	String
EXPIRE_AFTER_CNT_TOTAL	Default value for the maximum permissible usages for these production means within a maintenance interval	☐	long
EXPIRE_AFTER_CNT_TOTAL_FINAL	Default value for the total maximum permissible usages for these production means	☐	long
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created	☐	String
QUANTITY	Maximum number of carrier components on the carrier Advice: <i>The quantity indicates the maximum storage capacity of the carrier for the respective carrier component!</i>	☐	double
STORAGE_BARCODE	Bar code for the storage place	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system	☐	String

Tab. 11-48 "Key Value Pair": *carrierToCarrierResultKeys*

11.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierToCarrier- Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierToCarrierFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": carrierToCarrierFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierToCarrier- ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierToCarrierResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>carrierToCarrierResultValues</code> !

Tab. 11-49 Parameter (in) »equGetCarrierToCarrier«

Parameter (out)

Parameter name	Explanation
carrierToCarrier- ResultValues	Depending on parameter (in) <code>carrierToCarrierResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierToCarrierResultKeys</code> !

Tab. 11-50 Parameter (out) »equGetCarrierToCarrier«

11.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

11.13 equGetCarrierToSerialNumber

11.13.1 Function description

This function is used to query possible carriers to which a certain serial number could be assigned. Starting with the serial number to be specified, the part number of the product is ascertained and then searched for in the assignment rules over all assignment structures. Only carriers that have the "available" state are output.

Background to "assignment rule for products": Due to their physical nature, carriers can be used only for certain products in a certain quantity. Using the "product assignment" function, exactly this assignment to a carrier is defined in the master data of the respective part (carrier class); this makes the assignment verifiable. The products (parts) that can be assigned to the current carrier, and in what quantity – this is then the number of serial numbers – are defined there. The quantity indicates the maximum storage capacity of the carrier for the respective product.



Advice

*The carrier is a subordinate data class of the **production means**. The carrier, which assumes the role of the fundamental bearer – from a structural viewpoint it is on the top level – is also referred to as the carrier base!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCarrierToSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] carrierToSerialNumberFilters
    String key
    String value
in String[] carrierToSerialNumberResultKeys
out String[] carrierToSerialNumberResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `carrierToSerialNumberFilters` and `carrierToSerialNumberResultKeys` as well as (out) `carrierToSerialNumberResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: <i>If this parameter is not passed, the SERIAL_NUMBER must be unique!</i>	☐	String

Tab. 11-51 "Key Value Pair": *carrierToSerialNumberFilters*

Output parameters for ascertained carriers

Key	Explanation		IMSApi data type
CNT_USAGE	Number of usages (since the last maintenance)	☐	long
EQUIPMENT_INDEX	Individual designator for the carrier	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system Advice: <i>Only carriers that have the "available" state are output!</i>	☒	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Carrier" are relevant: by default, this is then only the entry "C": C = Standard carrier (PM class = C)	☐	String
EXPIRE_AFTER_CNT_TOTAL	Default value for the maximum permissible usages for these production means within a maintenance interval	☐	long
EXPIRE_AFTER_CNT_TOTAL_FINAL	Default value for the total maximum permissible usages for these production means	☐	long
PART_NUMBER	Part number (carrier class) on the basis of which the carrier was created	☐	String
QUANTITY	Maximum number of products on the carrier Advice: <i>The quantity indicates the maximum storage capacity of the carrier for the respective product!</i>	☐	double
STORAGE_BARCODE	Bar code for the storage place	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system	☐	String

Tab. 11-52 "Key Value Pair": *carrierToSerialNumberResultKeys*

11.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
carrierToSerial- NumberFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": carrierToSerialNumberFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": carrierToSerialNumberFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
carrierToSerial- NumberResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": carrierToSerialNumberResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>carrierToSerialNumberResultValues</code> !

Tab. 11-53 Parameter (in) »equGetCarrierToSerialNumber«

Parameter (out)

Parameter name	Explanation
carrierToSerial- NumberResultVal- ues	Depending on parameter (in) <code>carrierToSerialNumberResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>carrierToSerialNumberResultKeys</code> !

Tab. 11-54 Parameter (out) »equGetCarrierToSerialNumber«

11.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

11.14

equGetCavityEquipmentAssignRule

11.14.1

Function description

This function is used to query equipment (here in the context of an "injection mold holder") and the assignment rules that are stored there and that apply to cavities (injection molds). The assignment rules can be searched for using different criteria (e.g. equipment number, part number of the holder class, production means type, etc.). All search criteria are "AND" coupled such that only the data sets that meet all used filter conditions are returned.

Background to "Cavity to equipment" assignment rule: In addition to the "Product to cavity" assignment rule [▶▶▶], there is a second, independent rule; it is called the "Cavity to equipment" rule. The term "cavity" comes from injection molding; a cavity stands for a certain injection mold and can therefore be used for manufacturing a certain product shape. In this context, the equipment is used as an injection mold holder. An injection mold holder can hold multiple, even different cavities. This allows different products to be manufactured simultaneously (key word "work order cluster"). Which injection molds (cavities) can be used in an injection mold holder (equipment) is stored in the master data of the respective part (holder class) as an assignment rule.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*).

**Advice**

A cavity (PM class = M) is a subordinate data object of equipment of PM class = E; equipment and therefore also the cavities belong to the "production means" grouping!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetCavityEquipmentAssignRule
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] cavityEquipmentAssignRuleFilters
    String key
    String value
in String[] cavityEquipmentAssignRuleResultKeys
out String[] cavityEquipmentAssignRuleResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) cavityEquipmentAssignRuleFilters and cavityEquipmentAssignRuleResultKeys as well as (out) cavityEquipmentAssignRuleResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: If this parameter is passed, the key EQUIPMENT_PART_NUMBER is not interpreted!	☐ String
EQUIPMENT_PART_NUMBER	Part number (holder class) on the basis of which the equipment was created Advice: This parameter is interpreted only if the keys EQUIPMENT_INDEX and EQUIPMENT_NUMBER are not used. The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int

Tab. 11-55 "Key Value Pair": *cavityEquipmentAssignRuleFilters*

Definition of the output parameters for the found assignment rules

Key	Explanation	IMSApi data type
CAVITY_PART_DESC	Part description (mold class) on the basis of which the cavity was created	☐ String
CAVITY_PART_NUMBER	Part number (mold class) on the basis of which the cavity was created	☐ String
CAVITY_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Cavity" are relevant: by default, this is then only the entry "M": M = Cavity (PM class = M)	☐ String
CAVITY_UNIQUE	"Unique setup" designator of the mold class [0; 1]: 0 = Setup does NOT have to be unique (special case) 1 = Setup must be unique (default) Background: The "Unique setup" characteristic can be set in the part master data of the cavity (mold class); a unique setup is required in the system by default. This means that a cavity can be assigned to only one mold holder at any one time. For special cases associated with data preparation, the possibility of multiple setups may be necessary, at least for data-technical reasons. If the "Unique setup" characteristic is NOT set, the uniqueness test during setup does not return the otherwise usual errors (cf. IMSApi function » equAssign-CavityToEquipment « [▶▶▶], error code -1343).	☐ int
EQUIPMENT_PART_DESC	Part description (holder class) on the basis of which the equipment was created	☐ String
EQUIPMENT_PART_NUMBER	Part number (holder class) on the basis of which the equipment was created	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E)	☐ String

Tab. 11-56 "Key Value Pair": *cavityEquipmentAssignRuleResultKeys*

11.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
cavityEquipmentAssignRuleFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": cavityEquipmentAssignRuleFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": cavityEquipmentAssignRuleFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
cavityEquipmentAssignRuleResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": cavityEquipmentAssignRuleResultKeys table. Advice: Only the parameters passed here are output by the function; see <i>parameter (out)</i> <code>cavityEquipmentAssignRuleResultValues</code> !

Tab. 11-57 Parameter (in) »equGetCavityEquipmentAssignRule«

Parameter (out)

Parameter name	Explanation
cavityEquipmentAssignRuleResultValues	Depending on parameter (in) <code>cavityEquipmentAssignRuleResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of <i>parameter (in)</i> <code>cavityEquipmentAssignRuleResultKeys</code> !

Tab. 11-58 Parameter (out) »equGetCavityEquipmentAssignRule«

11.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1312	Failure	Production means type is invalid
-1336	Failure	Production means type not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.15

equGetEquipment

11.15.1

Function description

On the basis of special search criteria, this function ascertains matching production means; a distinction between the production means classes "Equipment (work step)", "Equipment (asset)", "Cavity", and "Carrier" is currently made in the iTAC system. For which production means class (abbreviated: PM class) the query is made depends on the selected production means type (abbreviated: PM type, key: EQUIPMENT_TYPE). The PM type indirectly determines the PM class to be used.

Search criteria such as production means state, production means type, part number (device class, carrier class, mold/holder class), internal/external production means numbers or production means index are available. All search criteria are "AND" coupled such that only the production means data that meet all used filter conditions are returned.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.



Advice

This API function is influenced by site parameter code no. 13030 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetEquipment
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] equipmentGetFilters
    String key
    String value
in String[] equipmentGetResultKeys
out String[] equipmentGetResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `equipmentGetFilters` and `equipmentGetResultKeys` as well as (out) `equipmentGetResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavities Advice: Note that the PM class must match the selected PM type (see key EQUIPMENT_TYPE)!	☐	String
EQUIPMENT_GROUP	Number of the equipment group in the iTAC system Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Use of "Equipment (work step)": In addition to the classic grouping, equipment groups can also be used when assigning equipment to work steps. Use of "Equipment (asset)": Here, equipment groups are used only for grouping without any other control function. Background: The use of equipment groups is optional. If they are to be used, they must be stored in the master data (ADM module).	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number and only needs to be specified if the EQUIPMENT_NUMBER is not unique! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☒	String
EQUIPMENT_NUMBER_EXT	External production means number (e.g., the designation of the manufacturer) Advice: Depending on the EQUIPMENT_TYPE, this is the external equipment number, cavity number, or carrier number! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String

Key	Explanation	IMSApi data type
EQUIPMENT_STATE	State of the production means [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used (for carriers only) 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted Advice: Multiple state values may be passed in the input (separated by ",")!	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	String
PART_GROUP	Part group for the part (device class, carrier class, mold/holder class)	String
PART_NUMBER	Part number (device class, carrier class, mold/holder class) on the basis of which the production means was created. Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	String

Tab. 11-59 "Key Value Pair": *equipmentGetFilters*

Definition of the output parameters for the ascertained production means

Key	Explanation	IMSApi data type
CALIBRATION_COMMENT	Individual maintenance comment	String
CALIBRATION_DATE	Point in time at which the production means were maintained the last time, i.e., the state was changed from "in maintenance" to "available"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
CALIBRATION_INTERVAL	Default value for the period between two maintenance dates (in days); by default, this value is added to the "Last maintenance" date and used as the default value for the "Next maintenance" following maintenance of a piece of production means.	int

Key	Explanation		IMSApi data type
CAVITY_ASSIGNED	Indicates whether the equipment is currently assigned cavities [0; 1]: 0 = NO cavity assigned 1 = Cavity assigned Advice: The value [1] can only be output if the equipment has the "Requires cavity" characteristic (see the key CAVITY_REQUIRED)! Advice: This key is only relevant for production means of the PM class "Equipment (work step)"!	☐	int
CAVITY_REQUIRED	Designator for the "Requires cavity" characteristic [0; 1]: 0 = Characteristic is NOT set 1 = Characteristic is set Advice: If the value [1] is output, the equipment in question can be used as a mold holder in the context of injection molding. This designator is defined in the master data of the part on the basis of which the equipment was created (see there the "Tool management" tab)! Advice: This key is only relevant for production means of the PM class "Equipment (work step)"!	☐	int
CNT_USAGE	Number of usages (since the last maintenance)	☐	long
CNT_USAGE_FAIL	Number of faulty usages (since the last maintenance)	☐	long
CNT_USAGE_FAIL_TOTAL	Total number of faulty usages (since the first use)	☐	long
CNT_USAGE_TOTAL	Total number of usages (since the first use)	☐	long
EQUIPMENT_CHECK-STATE	Check state for the production means [0; 1; 2; 3; 5; 6]: 0 = OK: Production means are valid 1 = Error: Production means must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [1] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [2] is output! 2 = Warning: Production means must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [2] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [1] is output! 3 = Error: Production means state is invalid; the production means state must have the "available" state 5 = Error: Production means are invalid; period between "valid from" and "valid to" has been exceeded Advice: [5] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [6] is output! 6 = Warning: Production means are invalid; period between "valid from" and "valid to" has been exceeded Advice: [6] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [5] is output!	☐	int
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavity	☐	String
EQUIPMENT_DESCRIPTION	Description of the production means Advice: Here, depending on the EQUIPMENT_TYPE, this is the respective equipment, cavity or carrier description!	☐	String

Key	Explanation	IMSApi data type
EQUIPMENT_FACTOR	Factor for the equipment Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Background: The equipment factor indicates how the usage counter for equipment utilization is calculated. As a result, it is also possible to map application types from the field of mechanics. For example, n strokes (i.e. n equipment operations) are required for punching a part (factor > 1). Or, in the context of multiple panel processing, if multiple parts are manufactured simultaneously in one equipment operation (factor < 1); here, the value is used as a multiplier. If e.g. a quadruple panel is being processed, the factor 0.25 must be entered.	☐ double
EQUIPMENT_GROUP	Number of the equipment group in the iTAC system Advice: This key is NOT relevant for production means of the PM classes "Carrier" and "Cavity"! Background: The use of equipment groups is optional. If they are to be used, they must be stored in the master data (ADM module); if the found equipment does not belong to a grouping, an empty string is output.	☐ String
EQUIPMENT_INDEX	Individual designator for the production means	☐ String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number!	☐ String
EQUIPMENT_NUMBER_EXT	External number of the production means Advice: Depending on the EQUIPMENT_TYPE, this is the external equipment number, cavity number, or carrier number!	☐ String
EQUIPMENT_REMAINING_USAGE	Open usage time in days; the time remaining before maintenance of the production means is used	☐ int
EQUIPMENT_STATE	State of the production means [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used (for carriers only) 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E)	☐ String
ERROR_CODE	- parameter without technical function - Advice: For technical reasons, this key must be passed; it always returns the value [0]!	☒ int

Key	Explanation		IMSApi data type
EXPIRATION_DATE	Default value for the point in time at which the production means are next due for maintenance; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
EXPIRATION_DATE_FINAL	Default value for the point in time at which the production means may be used for the last time (expiry date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
EXPIRE_AFTER_CNT_FAIL	Default value for the maximum permissible faulty usages for these production means within a maintenance interval	☐	long
EXPIRE_AFTER_CNT_FAIL_FINAL	Default value for the maximum permissible faulty usages for these production means	☐	long
EXPIRE_AFTER_CNT_TOTAL	Default value for the maximum permissible usages for these production means within a maintenance interval	☐	long
EXPIRE_AFTER_CNT_TOTAL_FINAL	Default value for the total maximum permissible usages for these production means	☐	long
EXPIRE_AFTER_TIME_FINAL	Default value for the total maximum period (in hours) that the equipment/cavity may be actively set up Advice: This key is NOT relevant for production means of the PM class "Carrier"!	☐	long
EXPIRE_AFTER_TIME_USAGE	Default value for the maximum period (in hours) that the equipment/cavity may be actively set up between two maintenance intervals. Advice: This key is NOT relevant for production means of the PM class "Carrier"!	☐	long
EXPIRE_CNT_THRESH_OLD_PERCENT	Limit value as a percentage which is used to define for the "Max. usages" counter the number by which the default value can be exceeded before the production means are locked for a pending order.	☐	double
EXPIRE_TIME_THRESH_OLD_HOUR	Limit value in hours which specifies the number of hours by which the specified maintenance date may be exceeded before the production means are locked for a pending order.	☐	int
IN_MAINT_SINCE	Point in time since the production means have had "In maintenance" state.	☐	DateMs
LAST_USAGE_DATE	Point in time at which the equipment was booked to a work step for the last time. Advice: This key is NOT relevant for production means of the PM class "Carrier"!	☐	DateMs
PART_DESC	Display of the part description (device class, carrier class, mold/holder class)	☐	String
PART_GROUP	Part group for the part (device class, carrier class, mold/holder class)	☐	String
PART_NUMBER	Part number (device class, carrier class, mold/holder class) on the basis of which the production means was created	☐	String
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐	String

Key	Explanation	IMSApi data type
TIME_USAGE_SEC	Length of time in seconds that the equipment/cavity was actively set up since the last maintenance. Advice: This key is NOT relevant for production means of the PM class "Carrier"!	☐ long
TIME_USAGE_SEC_TOTAL	Length of time in seconds that the equipment/cavity was actively set up since the first use. Advice: This key is NOT relevant for production means of the PM class "Carrier"!	☐ long
VALID_FROM	Start of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
VALID_TO	End of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Tab. 11-60 "Key Value Pair": *equipmentGetResultKeys*

11.15.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentGetFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentGetFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": equipmentGetFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
equipmentGetResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": equipmentGetResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>equipmentGetResultValues</i> !

Tab. 11-61 Parameter (in) »*equGetEquipment*«

Parameter (out)

Parameter name	Explanation
equipmentGetResultValues	Depending on parameter (in) <i>equipmentGetResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>equipmentGetResultKeys</i> !

Tab. 11-62 Parameter (out) »*equGetEquipment*«

11.15.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1301	Failure	Production means not found
-1312	Failure	Production means type is invalid
-1321	Failure	Production means class is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.16

equGetEquipmentLocationHist

11.16.1

Function description

This function is used to query storage place data on **one** production means over a selectable period under review. As a result it is possible to ascertain the storage place history – i.e. when and where a production means was stored. A distinction between the production means classes "Equipment (work step)", "Equipment (asset)", "Cavity", and "Carrier" is currently made in the iTAC system.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetEquipmentLocationHist
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] equipmentLocationFilters
    String key
    String value
in String[] equipmentLocationResultKeys
out String[] equipmentLocationResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `equipmentLocationFilters` and `equipmentLocationResultKeys` as well as (out) `equipmentLocationResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the booking time for the storage applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☒ DateMs
DATE_TO	Time stamp for the end of the period under review; the booking time for the removal from storage / restorage applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes 31.12.3000, 23:59:59.999 as the end of the period under review!	☒ DateMs
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavity Advice: If this key is not passed along, the [E] value is used by default!	☐ String
EQUIPMENT_INDEX	Individual designator for the production means Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system	☒ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 11-63 "Key Value Pair": `equipmentLocationFilters`

Output parameters for ascertained storage places

Key	Explanation	IMSApi data type
DATE_FROM	Booking time at which the carrier was stored (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_TO	Booking time at which the production means were removed from storage or restored (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If the production means are currently still stored, the value "32535215999999" is output as the end time (corresponds to 31-12-3000, 23:59:59.999)!	☐ DateMs

Key	Explanation		IMSApi data type
EQUIPMENT_INDEX	Individual designator for the production means	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system	☒	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐	String
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐	String

Tab. 11-64 "Key Value Pair": *equipmentLocationResultKeys*

11.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentLocationFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentLocationFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": equipmentLocationFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
equipmentLocationResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": equipmentLocationResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>equipmentLocationResultValues</code> !

Tab. 11-65 Parameter (in) »equGetEquipmentLocationHist«

Parameter (out)

Parameter name	Explanation
equipmentLocationResultValues	Depending on parameter (in) <code>equipmentLocationResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equipmentLocationResultKeys</code> !

Tab. 11-66 Parameter (out) »equGetEquipmentLocationHist«

11.16.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2011	Failure	End date precedes start date
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

11.17

equGetPartCavityAssignRule

11.17.1

Function description

This function is used to query cavities (here in the context of an "injection mold") and the assignment rules stored there for products that are allowed to be manufactured using these cavities. The assignment rules can be searched for using different criteria (e.g. cavity number, part number of the mold class, production means type, etc.). All search criteria are "AND" coupled such that only the data sets that meet all used filter conditions are returned.

Background to "Product to cavity" assignment rule: In addition to the "Cavity to equipment" assignment rule [▶▶▶], there is a second, independent rule; it is called the "Product to cavity" rule. This rule determines which cavity (mold class) can be used to manufacture which products (unit parts). Note that these products (part numbers/work orders) may differ despite their uniform shape; e.g. owing to a different material, different colors or different further processing. The product actually to be manufactured is assigned to the cavity based on the work order number (for further information, see the IMSApi function »equUpdateAssignedCavity« [▶▶▶]). Only when the work order number has been assigned is the stored assignment rule checked automatically.

**Advice**

A cavity (PM class = M) is a subordinate data object of equipment of PM class = E; equipment and therefore also the cavities belong to the "production means" grouping!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetPartCavityAssignRule
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] partCavityAssignRuleFilters
    String key
    String value
  in String[] partCavityAssignRuleResultKeys
  out String[] partCavityAssignRuleResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `partCavityAssignRuleFilters` and `partCavityAssignRuleResultKeys` as well as (out) `partCavityAssignRuleResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
CAVITY_INDEX	Individual designator for the cavity Advice: The index only needs to be passed if the CAVITY_NUMBER itself is NON-unique!	☐ String
CAVITY_NUMBER	Number of the cavity in the iTAC system Advice: If this parameter is passed, the key CAVITY_PART_NUMBER is not interpreted!	☐ String
CAVITY_PART_NUMBER	Part number (mold class) on the basis of which the cavity was created Advice: This parameter is interpreted only if the keys CAVITY_INDEX and CAVITY_NUMBER are not used. The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
CAVITY_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Cavity" are relevant: by default, this is then only the entry "M": M = Cavity (PM class = M) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 11-67 "Key Value Pair": `partCavityAssignRuleFilters`

Definition of the output parameters for the found assignment rules

Key	Explanation	IMSApi data type
CAVITY_PART_DESC	Part description (mold class) on the basis of which the cavity was created	☐ String
CAVITY_PART_NUMBER	Part number (mold class) on the basis of which the cavity was created	☐ String
CAVITY_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Cavity" are relevant: by default, this is then only the entry "M": M = Cavity (PM class = M)	☐ String
CAVITY_UNIQUE	"Unique setup" designator of the mold class [0; 1]: 0 = Setup does NOT have to be unique (special case) 1 = Setup must be unique (default) Background: The "Unique setup" characteristic can be set in the part master data of the cavity (mold class); a unique setup is required in the system by default. This means that a cavity can be assigned to only one mold holder at any one time. For special cases associated with data preparation, the possibility of multiple setups may be necessary, at least for data-technical reasons. If the "Unique setup" characteristic is NOT set, the uniqueness test during setup does not return the otherwise usual errors (cf. IMSApi function »equAssign-CavityToEquipment« [▶▶▶], error code -1343).	☐ int
PART_DESC	Part description of the product	☐ String
PART_NUMBER	Part number of the product	☐ String

Tab. 11-68 "Key Value Pair": *partCavityAssignRuleResultKeys*

11.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partCavityAssign- RuleFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": partCavityAssignRuleFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": partCavityAssignRuleFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
partCavityAssign- RuleResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": partCavityAssignRuleResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>partCavityAssignRuleResultValues</code> !

Tab. 11-69 Parameter (in) »equGetPartCavityAssignRule«

Parameter (out)

Parameter name	Explanation
partCavityAssign- RuleResultValues	Depending on parameter (in) <code>partCavityAssignRuleResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>partCavityAssignRuleResultKeys</code> !

Tab. 11-70 Parameter (out) »equGetPartCavityAssignRule«

11.17.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-1312	Failure	Production means type is invalid
-1336	Failure	Production means type not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

11.18

equGetRequiredEquipmentData

11.18.1

Function description

This function outputs information regarding the equipment that is required for processing a specific work step, i.e. has been deposited in the work plan. The respective work step is ascertained on the basis of an order number or serial number to be specified, the PCB orientation and the querying station.



Advice

*If the equipment is used in the context of an "injection mold holder", information on the **cavities** (injection molds) assigned there is also output!*

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).



Advice

This API function is affected by station parameter code no. 8310 [▶▶▶] as well as by site parameter code nos. 4570 [▶▶▶] and 13030 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

This function should NO LONGER be used for new installations because the edit options have been extended with release 9.00.00 (for further information, see the new function »equGetRequiredEquipmentDetail« [▶▶▶]). The most significant difference is that the new function can read out the required equipment from a work plan, and is able to do this without knowledge of a concrete work order number!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetRequiredEquipmentData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String serialNumber
in String partNumber
in int processLayer
in String pmGroup
in String[] equipmentResultKeys
out String[] equipmentResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equipmentResultKeys` and (out) `equipmentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
CAVITY_ASSIGNED	Indicates whether the equipment is currently assigned cavities [0; 1]: 0 = NO cavity assigned 1 = Cavity assigned Advice: The value [1] can only be output if the equipment has the "Requires cavity" characteristic (see the key <code>CAVITY_REQUIRED</code>)!	☐	int
CAVITY_REQUIRED	Designator for the "Requires cavity" characteristic [0; 1]: 0 = Characteristic is NOT set 1 = Characteristic is set Advice: If the value [1] is output, the equipment in question can be used as a mold holder in the context of injection molding. This designator is defined in the master data of the part on the basis of which the equipment was created (see there the "Tool management" tab)!	☐	int
EQUIPMENT_CHECK-STATE	Check state for the equipment [0; 1; 2; 3; 5; 6]: 0 = OK: Equipment is valid 1 = Error: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [1] is only output if site parameter code no. 13030 is set to <code>FALSE</code> ; otherwise the value [2] is output! 2 = Warning: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [2] is only output if site parameter code no. 13030 is set to <code>TRUE</code> ; otherwise the value [1] is output! 3 = Error: Equipment state is invalid; the equipment state must have the "available" state 5 = Error: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: [5] is only output if site parameter code no. 13030 is set to <code>FALSE</code> ; otherwise the value [6] is output! 6 = Warning: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: [6] is only output if site parameter code no. 13030 is set to <code>TRUE</code> ; otherwise the value [5] is output!	☐	int
EQUIPMENT_DESCRIPTION	Description of the equipment	☐	String

Key	Explanation		IMSApi data type
EQUIPMENT_FACTOR	Factor for the equipment Background: The equipment factor indicates how the usage counter for equipment utilization is calculated. As a result, it is also possible to map application types from the field of mechanics. For example, <i>n</i> strokes (i.e. <i>n</i> equipment operations) are required for punching a part (factor > 1). Or, in the context of multiple panel processing, if multiple parts are manufactured simultaneously in one equipment operation (factor < 1); here, the value is used as a multiplier. If e.g. a quadruple panel is being processed, the factor 0.25 must be entered.	☐	double
EQUIPMENT_GROUP	Name of the internal equipment grouping in the iTAC system; internal groups are created automatically when equipment is assigned to work steps.	☐	String
EQUIPMENT_GROUP_MASTER_DATA	Number of the equipment group in the iTAC system Advice: <i>The use of equipment groups is optional; if they are to be used, they must be stored in the master data (ADM module)!</i>	☐	String
EQUIPMENT_GROUP_MASTER_DATA_DESC	Description of the equipment group in the iTAC system (see the advice notice under EQUIPMENT_GROUP_MASTER_DATA)	☐	String
EQUIPMENT_GRP_ITEM_TYP	Assignment type for the equipment [0; 1]: 0 = Equipment was assigned directly 1 = Equipment was assigned indirectly via a part (device class) (see PART_NUMBER)	☐	int
EQUIPMENT_INDEX	Individual designator for the equipment	☐	String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☐	String
EQUIPMENT_NUMBER_EXT	External equipment number	☐	String
EQUIPMENT_STATE	State of the equipment [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	☐	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: <i>The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!</i>	☐	String
INFO_TXT_1	Individual additional information 1	☐	String
INFO_TXT_2	Individual additional information 2	☐	String

Key	Explanation	IMSApi data type
INFO_TXT_3	Individual additional information 3	☐ String
PART_GROUP	Part group for the part (device class)	☐ String
PART_NUMBER	Part number (device class) on the basis of which the equipment was created	☐ String
SETUP_POSITION	Machine position on which the equipment is set up.	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☐ String
WORKPLAN_NUMBER	Part number for the work plan	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKPLAN_VERS	Work plan version in the iTAC system	☐ int
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int

Tab. 11-71 "Key Value Pair": *equipmentResultKeys*

11.18.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system

Parameter name	Explanation
<code>workOrderNumber</code>	Number of a production order Advice: If an empty string or [-1] is passed, the work order active on the station is used or ascertained on the basis of the serial number (<code>serialNumber</code>). If order and serial number are being passed, the serial number must be part of the WO! If neither a work order nor a serial number is passed, the system tries to ascertain a valid work plan/work step on the basis of the part number (<code>partNumber</code>) and station (<code>stationNumber</code>)!
<code>serialNumber</code>	Serial number of the panel Advice: If [-1] is passed here, the field is not evaluated; in this case, the work order is ascertained on the basis of the <code>workOrderNumber</code> parameter. If order and serial number are being passed, the serial number must be part of the WO!
<code>partNumber</code>	Part number in the iTAC system Advice: If no work order is found (see <code>workOrderNumber</code>), this part number is used to try to ascertain a valid work plan and – in combination with the station and the layer – a unique work step!
<code>processLayer</code>	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: This parameter is evaluated only if a work order (<code>workOrderNumber</code>) is found or a part number (<code>partNumber</code>) is passed!
<code>pmGroup</code>	Name of the internal equipment grouping in the iTAC system; internal groups are created automatically when equipment is assigned to work steps. Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Special case: If the value [-1] is passed, all found equipment is returned independent of the internal equipment group!
<code>equipmentResultKeys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>equipmentResultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>equipmentResultValues</code>!

Tab. 11-72 Parameter (in) »equGetRequiredEquipmentData«

Parameter (out)

Parameter name	Explanation
<code>equipmentResultValues</code>	Depending on parameter (in) <code>equipmentResultKeys</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equipmentResultKeys</code>!

Tab. 11-73 Parameter (out) »equGetRequiredEquipmentData«

11.18.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-128	Failure	Work step not found
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-240	Failure	No work step found for work order, orientation of the PCB and station
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.19

equGetRequiredEquipmentDetail

11.19.1

Function description

This function outputs information regarding the equipment that is required for processing a specific work step, i.e. has been deposited in the work plan. The respective work step is determined on the basis of special search criteria. Here the specification of the PCB orientation and the querying station is obligatory; the part number, work order number and/or serial number are available as optional search criteria.



Advice

*If the equipment is used in the context of an "injection mold holder", information on the **cavities** (injection molds) assigned there is also output!*

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).



Advice

This API function is affected by station parameter code no. 8310 [▶▶▶] as well as by site parameter code nos. 4570 [▶▶▶] and 13030 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!
*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »equGetRequiredEquipmentData« [▶▶▶] function. The most significant difference is that the new function can read out the required equipment from a work plan, and is able to do this without knowledge of a concrete work order number!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equGetRequiredEquipmentDetail
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] requiredEquipmentFilterKeys
    String key
    String value
in String[] requiredEquipmentResultKeys
out String[] requiredEquipmentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `requiredEquipmentFilterKeys` and `requiredEquipmentResultKeys` as well as (out) `requiredEquipmentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system Advice: <i>It is important to note the interaction between the keys PART_NUMBER, SERIAL_NUMBER and WORKORDER_NUMBER. If neither a part, a work order nor a serial number is passed, the system attempts to determine a valid work plan/work step on the basis of the active work order on the station (cf. stationNumber). If only the part is passed, the master work plan is interpreted. If order and serial number are being passed together, the serial number must be part of the work order!</i>	☐ String
PM_GROUP	Name of the internal equipment grouping in the iTAC system; internal groups are created automatically when equipment is assigned to work steps. Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! In the output, the key for this data field is instead called EQUIPMENT_GROUP.</i>	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☒ int
SERIAL_NUMBER	Serial number of the panel Advice: <i>It is important to note the interaction between the keys PART_NUMBER, SERIAL_NUMBER and WORKORDER_NUMBER. If neither a part, a work order nor a serial number is passed, the system attempts to determine a valid work plan/work step on the basis of the active work order on the station (cf. stationNumber). If only the part is passed, the master work plan is interpreted. If order and serial number are being passed together, the serial number must be part of the work order!</i>	☐ String

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Number of a production order Advice: It is important to note the interaction between the keys PART_NUMBER, SERIAL_NUMBER and WORKORDER_NUMBER. If neither a part, a work order nor a serial number is passed, the system attempts to determine a valid work plan/work step on the basis of the active work order on the station (cf. stationNumber). If only the part is passed, the master work plan is interpreted. If order and serial number are being passed together, the serial number must be part of the work order!	String
WORKPLAN_VERS	Work plan version Advice: If the key is not passed, the highest released and valid work plan version is automatically used provided no ERP change number is passed (see WORKPLAN_VERS_ERP)!	int
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: If the key is not passed, the highest released and valid work plan version is automatically used, provided that no work plan version is passed (see WORKPLAN_VERS)!	String

Tab. 11-74 "Key Value Pair": requiredEquipmentFilterKeys

Definition of the output parameters for the ascertained equipment

Key	Explanation	IMSApi data type
CAVITY_ASSIGNED	Indicates whether the equipment is currently assigned cavities [0; 1]: 0 = NO cavity assigned 1 = Cavity assigned Advice: The value [1] can only be output if the equipment has the "Requires cavity" characteristic (see the key CAVITY_REQUIRED)!	int
CAVITY_REQUIRED	Designator for the "Requires cavity" characteristic [0; 1]: 0 = Characteristic is NOT set 1 = Characteristic is set Advice: If the value [1] is output, the equipment in question can be used as a mold holder in the context of injection molding. This designator is defined in the master data of the part on the basis of which the equipment was created (see there the "Tool management" tab)!	int

Key	Explanation	IMSApi data type
EQUIPMENT_CHECK-STATE	Check state for the equipment [0; 1; 2; 3; 5; 6]: 0 = OK: Equipment is valid 1 = Error: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [1] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [2] is output! 2 = Warning: Equipment must either be maintained or the point in time of the last permitted use has been exceeded. Advice: [2] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [1] is output! 3 = Error: Equipment state is invalid; the equipment state must have the "available" state 5 = Error: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: [5] is only output if site parameter code no. 13030 is set to FALSE; otherwise the value [6] is output! 6 = Warning: Equipment is invalid; period between "valid from" and "valid to" has been exceeded Advice: [6] is only output if site parameter code no. 13030 is set to TRUE; otherwise the value [5] is output!	int
EQUIPMENT_DESCRIPTION	Description of the equipment	String
EQUIPMENT_FACTOR	Factor for the equipment Background: The equipment factor indicates how the usage counter for equipment utilization is calculated. As a result, it is also possible to map application types from the field of mechanics. For example, n strokes (i.e. n equipment operations) are required for punching a part (factor > 1). Or, in the context of multiple panel processing, if multiple parts are manufactured simultaneously in one equipment operation (factor < 1); here, the value is used as a multiplier. If e.g. a quadruple panel is being processed, the factor 0.25 must be entered.	double
EQUIPMENT_GROUP	Name of the internal equipment grouping in the iTAC system; internal groups are created automatically when equipment is assigned to work steps.	String
EQUIPMENT_GROUP_MASTER_DATA	Number of the equipment group in the iTAC system Advice: The use of equipment groups is optional; if they are to be used, they must be stored in the master data (ADM module)!	String
EQUIPMENT_GROUP_MASTER_DATA_DESC	Description of the equipment group in the iTAC system (see the advice notice under EQUIPMENT_GROUP_MASTER_DATA)	String
EQUIPMENT_GRP_ITEM_TYP	Assignment type for the equipment [0; 1]: 0 = Equipment was assigned directly 1 = Equipment was assigned indirectly via a part (device class) (see PART_NUMBER)	int
EQUIPMENT_INDEX	Individual designator for the equipment	String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	String
EQUIPMENT_NUMBER_EXT	External equipment number	String

Key	Explanation	IMSApi data type
EQUIPMENT_STATE	State of the equipment [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	☐ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String
INFO_TXT_1	Individual additional information 1	☐ String
INFO_TXT_2	Individual additional information 2	☐ String
INFO_TXT_3	Individual additional information 3	☐ String
PART_GROUP	Part group for the part (device class)	☐ String
PART_NUMBER	Part number (device class) on the basis of which the equipment was created	☐ String
SETUP_POSITION	Machine position on which the equipment is set up.	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	☐ String
WORKPLAN_NUMBER	Part number for the work plan	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int

Key	Explanation	IMSApi data type
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 11-75 "Key Value Pair": *requiredEquipmentResultKeys*

11.19.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
requiredEquipmentFilterKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": requiredEquipmentFilterKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": requiredEquipmentFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
requiredEquipmentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": requiredEquipmentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>requiredEquipmentResultValues</i> !

Tab. 11-76 Parameter (in) »*equGetRequiredEquipmentDetail*«

Parameter (out)

Parameter name	Explanation
requiredEquipmentResultValues	Depending on parameter (in) <i>requiredEquipmentResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>requiredEquipmentResultKeys</i> !

Tab. 11-77 Parameter (out) »*equGetRequiredEquipmentDetail*«

11.19.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-128	Failure	Work step not found
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-240	Failure	No work step found for work order, orientation of the PCB and station
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.20 equGetSetupEquipmentData

11.20.1 Function description

This function outputs all equipment set up on the passed station independent of the active work order.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int equGetSetupEquipmentData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] equipmentSetupFilters
    String key
    String value
  in String[] equipmentSetupResultKeys
  out String[] equipmentSetupResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `equipmentSetupFilters` and `equipmentSetupResultKeys` as well as (out) `equipmentSetupResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	String

Tab. 11-78 "Key Value Pair": *equipmentSetupFilters*

Definition of the output parameters for the found equipment

Key	Explanation	IMSApi data type
CAVITY_ASSIGNED	Indicates whether the equipment is currently assigned cavities [0; 1]: 0 = NO cavity assigned 1 = Cavity assigned Advice: The value [1] can only be output if the equipment has the "Requires cavity" characteristic (see the key CAVITY_REQUIRED)!	int
CAVITY_REQUIRED	Designator for the "Requires cavity" characteristic [0; 1]: 0 = Characteristic is NOT set 1 = Characteristic is set Advice: If the value [1] is output, the equipment in question can be used as a mold holder in the context of injection molding. This designator is defined in the master data of the part on the basis of which the equipment was created (see there the "Tool management" tab)!	int
EQUIPMENT_DESCRIPTION	Description of the equipment	String
EQUIPMENT_INDEX	Individual designator for the equipment	String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	String
EQUIPMENT_NUMBER_EXT	External equipment number	String
EQUIPMENT_STATE	Equipment state [0; 20; 30; 40; 50; 90]: 0 = available 20 = being repaired 30 = being maintained 40 = defective 50 = locked 90 = deleted	String

Key	Explanation		IMSApi data type
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]; in this context, only entries from the PM class "Equipment (work step)" are relevant: by default, these are the following entries: E = Standard equipment (PM class = E) F = Feeder (PM class = E) N = Nozzle (PM class = E) H = Head (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E)	☐	String
FAILURES_BEFORE_EXPIRATION	Number of possible remaining faulty usages before the equipment must be maintained	☐	int
INFO_TXT_1	Individual additional information 1	☐	String
INFO_TXT_2	Individual additional information 2	☐	String
INFO_TXT_3	Individual additional information 3	☐	String
SECONDS_BEFORE_EXPIRATION	Next maintenance date in seconds or point in time of the last valid use	☐	int
SETUP_POSITION	Machine position on which the equipment is set up.	☐	String
USAGES_BEFORE_EXPIRATION	Number of possible remaining usages before the equipment must be maintained	☐	int

Tab. 11-79 "Key Value Pair": *equipmentSetupResultKeys*

11.20.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentSetupFilters	"Key Value Pair" structure with the following variables:
key	List of the equipment parameters on the basis of which the output is to be filtered; the corresponding values are passed with the value parameter. A description of the possible equipment parameters can be found above in the "Key Value Pair": equipmentSetupFilters table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": equipmentSetupFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
equipmentSetupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": equipmentSetupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) equipmentSetupResultValues!

Tab. 11-80 Parameter (in) »equGetSetupEquipmentData«

Parameter (out)

Parameter name	Explanation
equipmentSetupResultValues	Depending on parameter (in) equipmentSetupResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) equipmentSetupResultKeys!

Tab. 11-81 Parameter (out) »equGetSetupEquipmentData«

11.20.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.21

equManageCavityEquipmentRule

11.21.1

Function description

This function is used to manage rules for the assignment of cavities (PM class: M) to equipment (PM class: E). New rules can be created and existing rules can be edited. The assignment rule is based on the master data parts for cavities and equipment and defines which mold classes (cavity parts) can be used in which holder class (equipment part). Here it is therefore not a question of the physical assignment but rather of the theoretical structure.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Background to "Cavity to equipment" assignment rule: In addition to the "Product to cavity" assignment rule [▶▶▶], there is a second, independent rule; it is called the "Cavity to equipment" rule. The term "cavity" comes from injection molding; a cavity stands for a certain injection mold and can therefore be used for manufacturing a certain product shape. In this context, the equipment is used as an injection mold holder. An injection mold holder can hold multiple, even different cavities. This allows different products to be manufactured simultaneously (key word "work order cluster"). Which injection molds (cavities) can be used in an injection mold holder (equipment) is stored in the master data of the respective part (holder class) as an assignment rule.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: E). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equManageCavityEquipmentRule
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String equipmentPartNumber
in String[] manageCavityEquipmentRuleKeys
in String[] manageCavityEquipmentRuleValues
out String[] manageCavityEquipmentRuleResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `manageCavityEquipmentRuleKeys` and `manageCavityEquipmentRuleValues` as well as (out) `manageCavityEquipmentRuleResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Definition of the input parameters for the function execution

Key	Explanation	IMSApi data type
CAVITY_PART_NUMBER	Part number of the cavity (mold class)	☒ String
DELETE_MODE	Change mode [0; 1]: 0 = Add cavity to rule 1 = Remove cavity from rule Advice: If this key is not passed along, the [0] value is used by default!	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐ int

Tab. 11-82 "Key Value Pair": `manageCavityEquipmentRuleKeys`

11.21.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentPartNumber	Part number of the equipment (holder class)
manageCavityEquipmentRuleKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>manageCavityEquipmentRuleValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": manageCavityEquipmentRuleKeys table. Advice: Only the parameters passed here are processed by the function!
manageConditionCodeUploadValues	Values for the parameters selected by means of <code>manageCavityEquipmentRuleKeys</code>; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>manageCavityEquipmentRuleKeys</code>! Refer to the "Key Value Pair": manageCavityEquipmentRuleKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-83 Parameter (in) »equManageCavityEquipmentRule«

Parameter (out)

Parameter name	Explanation
manageCavityEquipmentRuleResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>manageCavityEquipmentRuleKeys</code> and <code>manageCavityEquipmentRuleValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>manageCavityEquipmentRuleKeys</code>!

Tab. 11-84 Parameter (out) »equManageCavityEquipmentRule«

11.21.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1316	Failure	Production means part not found
-1321	Failure	Production means class is invalid
-1338	Failure	Part number of cavity is required
-1339	Failure	Cavity part cannot be managed
-1340	Failure	Cavity part has already been removed from equipment part
-1341	Failure	Cavity part is already assigned to an equipment part
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.22

equManagePartCavityRule

11.22.1

Function description

This function is used to manage rules for the assignment of products to cavities (PM class: M). New rules can be created and existing rules can be edited.

The assignment rule is based on the master data parts for products and cavities and defines which products can be manufactured with which cavity (mold class). Here it is not a question of the physical assignment but rather of the theoretical structure.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Background to "Product to cavity" assignment rule: In addition to the "Cavity to equipment" assignment rule [▶▶▶], there is a second, independent rule; it is called the "Product to cavity" rule. This rule determines which cavity (mold class) can be used to manufacture which products (unit parts). Note that these products (part numbers/work orders) may differ despite their uniform shape; e.g. owing to a different material, different colors or different further processing. The product actually to be manufactured is assigned to the cavity based on the work order number (for further information, see the IMSApi function »equUpdateAssignedCavity« [▶▶▶]). Only when the work order number has been assigned is the stored assignment rule checked automatically.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equManagePartCavityRule
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String cavityPartNumber
in String[] managePartCavityRuleKeys
in String[] managePartCavityRuleValues
out String[] managePartCavityRuleResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) managePartCavityRuleKeys and managePartCavityRuleValues as well as (out) managePartCavityRuleResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Definition of the input parameters for the function execution

Key	Explanation	IMSApi data type
DELETE_MODE	Change mode [0; 1]: 0 = Add product to rule 1 = Remove product from rule Advice: If this key is not passed along, the [0] value is used by default!	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐ int
PART_NUMBER	Part number of the product	☒ String

Tab. 11-85 "Key Value Pair": *managePartCavityRuleKeys*

11.22.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
cavityPartNumber	Part number of the cavity (mold class)
managePartCavityRuleKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>managePartCavityRuleValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": managePartCavityRuleKeys table. Advice: Only the parameters passed here are processed by the function!
managePartCavityRuleValues	Values for the parameters selected by means of <code>managePartCavityRuleKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>managePartCavityRuleKeys</code> ! Refer to the "Key Value Pair": managePartCavityRuleKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-86 Parameter (in) »equManagePartCavityRule«

Parameter (out)

Parameter name	Explanation
managePartCavityRuleResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>managePartCavityRuleKeys</code> and <code>managePartCavityRuleValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>managePartCavityRuleKeys</code> !

Tab. 11-87 Parameter (out) »equManagePartCavityRule«

11.22.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-123	Failure	Part group type not found
-196	Failure	A parameter is invalid
-1316	Failure	Production means part not found
-1321	Failure	Production means class is invalid
-1330	Failure	Unit part has already been removed from cavity part
-1331	Failure	Unit part is already assigned to a cavity part
-1337	Failure	Unit part must not be managed

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.23

equRemoveCarrierFromCarrier

11.23.1

Function description

This function is used to cancel a carrier assignment; subordinate carrier components can be removed from a carrier (carrier unit). It is not possible to remove carrier components over multiple carrier structure levels. Furthermore, carriers can only be removed from the structure if they themselves do not have carrier or product assignments.

If a multilevel carrier structure is to be resolved, it must be processed successively from the bottom to the top structure level.

**Advice**

*The carrier is a subordinate data class of the **production means**. The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Background to "assignment rule for carriers": Generally speaking, a carrier consists of several different components. These include a fundamental bearer (with/without wheels), possibly different rack elements (bays, mounting rods) and the product holders (receptacles, hook systems). The structure of a carrier is stored as an assignment rule in the master data of the respective part (carrier class); this makes the structure verifiable.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equRemoveCarrierFromCarrier
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String equipmentNumber
  in String equipmentIndex
  in String[] removeCarrierFromCarrierKeys
  in String[] removeCarrierFromCarrierValues
  out String[] removeCarrierFromCarrierResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `removeCarrierFromCarrierKeys` and `removeCarrierFromCarrierValues` as well as (out) `removeCarrierFromCarrierResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
EQUIPMENT_INDEX	Individual designator for the carrier (here the subordinate component, cf. input parameter <code>equipmentIndex</code>) Advice: <i>The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!</i>	☐	String
EQUIPMENT_NUMBER	Number of the carrier in the iTAC system (here the subordinate component, cf. input parameter <code>equipmentNumber</code>) Advice: <i>If the carrier number is non-unique, the EQUIPMENT_INDEX key must also be passed!</i>	☒	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!</i>	☒	int

Tab. 11-88 "Key Value Pair": `removeCarrierFromCarrierKeys`

11.23.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentNumber	Number of the carrier in the iTAC system (here the parent carrier unit, cf. key EQUIPMENT_NUMBER)
equipmentIndex	Individual designator for the carrier (here the parent carrier unit, cf. key EQUIPMENT_INDEX) Default: 0 Advice: The index is required in combination with the carrier number for unique designation; i.e. the carrier number itself does not necessarily need to be unique!
removeCarrierFromCarrierKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>removeCarrierFromCarrierValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": removeCarrierFromCarrierKeys table. Advice: Only the parameters passed here are processed by the function!
removeCarrierFromCarrierValues	Values for the parameters selected by means of <code>removeCarrierFromCarrierKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>removeCarrierFromCarrierKeys</code> ! Refer to the "Key Value Pair": removeCarrierFromCarrierKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-89 Parameter (in) »equRemoveCarrierFromCarrier«

Parameter (out)

Parameter name	Explanation
removeCarrierFromCarrierResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>removeCarrierFromCarrierKeys</code> and <code>removeCarrierFromCarrierValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>removeCarrierFromCarrierKeys</code> !

Tab. 11-90 Parameter (out) »equRemoveCarrierFromCarrier«

11.23.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1324	Failure	Carrier is not assigned to a carrier
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.24

equRemoveCavityFromEquipment

11.24.1

Function description

This function is used to remove an existing assignment of cavity (injection mold) to work-step-related equipment (here in the context of an "injection mold holder").

**Advice**

The assignment of a cavity to equipment is performed using the IMSApi function »equAssignCavityToEquipment« [▶▶▶]!

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

**Advice**

*A cavity (PM class = **M**) is a subordinate data object of equipment of PM class = **E**; equipment and therefore also the cavities belong to the "production means" grouping.*

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equRemoveCavityFromEquipment
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] removeEquipmentKeys
    String key
    String value
in String[] removeCavityKeys
in String[] removeCavityValues
out String[] removeCavityResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `removeEquipmentKeys`, `removeCavityKeys` and `removeCavityValues` as well as (out) `removeCavityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input parameters for the desired equipment (header data)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed!	☒ String

Tab. 11-91 "Key Value Pair": *removeEquipmentKeys*

Input parameters for the cavities to be removed

Key	Explanation	IMSApi data type
CAVITY_INDEX	Individual designator for the cavity Advice: The index only needs to be passed if the CAVITY_NUMBER itself is NON-unique!	☐ String
CAVITY_NUMBER	Number of the cavity in the iTAC system Advice: If the cavity number is non-unique, the CAVITY_INDEX key must also be passed!	☒ String
CAVITY_POSITION	Position number of the cavity on the equipment	☒ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐ int

Tab. 11-92 "Key Value Pair": *removeCavityKeys*

11.24.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
removeEquipmentKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": removeEquipmentKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": removeEquipmentKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
removeCavityKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>removeCavityValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": removeCavityKeys table. Advice: Only the parameters passed here are processed by the function!
removeCavityValues	Values for the parameters selected by means of <code>removeCavityKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>removeCavityKeys</code> ! Refer to the "Key Value Pair": removeCavityKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-93 Parameter (in) »equRemoveCavityFromEquipment«

Parameter (out)

Parameter name	Explanation
removeCavityResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>removeCavityKeys</code> and <code>removeCavityValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>removeCavityKeys</code> !

Tab. 11-94 Parameter (out) »equRemoveCavityFromEquipment«

11.24.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1332	Failure	Cavity not found
-1333	Failure	Cavity is not unique
-1335	Failure	Cavity has already been removed from equipment
-1338	Failure	Part number of cavity is required
-1342	Failure	More than one assigned cavity found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.25 equRemoveEquipment

11.25.1 Function description

This function removes all equipment set up on a station for a work order; equipment for other work orders remain set up.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equRemoveEquipment
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String serialNumber
in int processLayer
```

11.25.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order Advice: If an empty string or [-1] is passed, the work order active on the station is used or ascertained on the basis of the serial number (serialNumber). If order and serial number are being passed, the serial number must be part of the WO!
serialNumber	Serial number of the panel Advice: If [-1] is passed here, the field is not evaluated; in this case, the work order is ascertained on the basis of the workOrderNumber parameter. If order and serial number are being passed, the serial number must be part of the WO!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent

Tab. 11-95 Parameter (in) »equRemoveEquipment«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

11.25.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-130	Failure	Work order not found
-230	Failure	Station is invalid for this work order
-1308	Failure	Station is not assigned to the work order
-2105	Failure	No TR or PM license present for the station

11.26

equRemoveSerialNumberFromCarrier

11.26.1

Function description

This function is used to cancel a product assignment to the carrier; it is possible to remove one or more assigned products (in the form of serial numbers) from a carrier. It is not possible to remove serial numbers over multiple carrier structure levels.

If the last serial number is removed from an empty carrier – i.e. without any other subordinate carrier components – the carrier is automatically switched to the "available" state.

**Advice**

*The carrier is a subordinate data class of the **production means**. The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Background to "assignment rule for products": Due to their physical nature, carriers can be used only for certain products in a certain quantity. Using the "product assignment" function, exactly this assignment to a carrier is defined in the master data of the respective part (carrier class); this makes the assignment verifiable. The products (parts) that can be assigned to the current carrier, and in what quantity – this is then the number of serial numbers – are defined there. The quantity indicates the maximum storage capacity of the carrier for the respective product.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equRemoveSerialNumberFromCarrier
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String equipmentNumber
  in String equipmentIndex
  in String[] removeSerialNumberFromCarrierKeys
  in String[] removeSerialNumberFromCarrierValues
  out String[] removeSerialNumberFromCarrierResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `removeSerialNumberFromCarrierKeys` and `removeSerialNumberFromCarrierValues` as well as (out) `removeSerialNumberFromCarrierResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☒ int
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the SERIAL_NUMBER must be unique!	☐ String

Tab. 11-96 "Key Value Pair": *removeSerialNumberFromCarrierKeys*

11.26.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentNumber	Number of the carrier in the iTAC system
equipmentIndex	Individual designator for the carrier Default: 0 Advice: The index is required in combination with the carrier number for unique designation; i.e. the carrier number itself does not necessarily need to be unique!
removeSerialNumberFromCarrierKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>removeSerialNumberFromCarrierValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": removeSerialNumberFromCarrierKeys table. Advice: Only the parameters passed here are processed by the function!
removeSerialNumberFromCarrierValues	Values for the parameters selected by means of <code>removeSerialNumberFromCarrierKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>removeSerialNumberFromCarrierKeys</code> ! Refer to the "Key Value Pair": attributeUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-97 Parameter (in) »equRemoveSerialNumberFromCarrier«

Parameter (out)

Parameter name	Explanation
removeSerialNumberFromCarrierResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>removeSerialNumberFromCarrierKeys</code> and <code>removeSerialNumberFromCarrierValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>removeSerialNumberFromCarrierKeys</code> !

Tab. 11-98 Parameter (out) »equRemoveSerialNumberFromCarrier«

11.26.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1325	Failure	Serial number is not assigned to a carrier
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted

11.27

equReportQuantity

11.27.1

Function description

This function – based on the production means management concept of iTAC.MES.Suite – can be used to capture usage figures for production means; the production means do NOT necessarily have to be set up on a station to allow this.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.

Interpretation of the usage counter enables, for example, the initiation of maintenance processes when certain default values are exceeded, and the locking of the production means for production. With incorrect production means operation, a failure code can additionally be passed with a failure description; failure codes that do not yet exist are created automatically.

**Advice**

This API function is influenced by station parameter code no. 8402 [▶▶▶] (usage counter "Cavity") and site parameter 13060 [▶▶▶] (automatic maintenance order). Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Special case "placement procedure" -> This special case applies only to "Equipment (work step)" and there ONLY to the equipment types **F** (Feeder), **N** (Nozzle) and **H** (Head): Apart from the pure booking of usage counts, this function can also be used to book detailed data of the placement procedure in the system. Failure-free and faulty placement procedures are passed separated according to the "feeder", "nozzle" and "head" equipment types. With faulty placement procedures (so-called "mispicks"), a failure code can additionally be passed with a failure description. Equipment or failure codes that have been passed using the function but are not yet available in the system are created automatically. The booked placement procedure data is displayed in the TR module (function *MIS machine data* → *Equipment*, "SMT overview" tab) and in the Dashboard-Client.

**Advice**

Special case "placement procedure" for equipment (work step): *This API function is influenced by site parameter code no. 13040 [▶▶▶], 18300 [▶▶▶], 18301 [▶▶▶] and 18306 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equReportQuantity
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] equReportQuantityKeys
in String[] equReportQuantityValues
out String[] equReportQuantityResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equReportQuantityKeys` and `equReportQuantityValues` as well as (out) `equReportQuantityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed or passed with [-1], the current date and time are used!	☐ DateMs
DATE_FROM	Time stamp for the start of the period in which the specified usage quantities occurred; specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Special case: If the key is passed with [-1], the current date and time are used!	☒ DateMs
DATE_TO	Time stamp for the end of the period in which the specified usage quantities occurred; specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Special case: If the key is passed with [-1], the current date and time are used!	☒ DateMs

Key	Explanation		IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means into equipment and carriers [A; C; E]: A = Equipment (asset) C = Carrier E = Equipment (work step) Restriction regarding "cavity": The PM class "M" (Cavity) is not supported here; the usage counts from the parent equipment are applied for cavities, provided that configuration parameter code no. 8402 [▶▶▶] is active. Advice: Note that the PM class must match the selected PM type (see key EQUIPMENT_TYPE)!	☐	String
EQUIPMENT_DESCRIPTION	Description of the production means Advice: Here, depending on the EQUIPMENT_TYPE, this is the respective equipment or carrier description!	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number and carrier number and only needs to be specified if the EQUIPMENT_NUMBER is not unique!	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number or carrier number! Special case: For the special case "placement procedure" [▶▶▶] described above, a new production means is created in the system when an unknown number is passed. Make sure that the correct production means type is always selected for the passed number (see EQUIPMENT_TYPE – only F, H and N are possible for new creation).	☒	String
EQUIPMENT_TRACK	Position specification for equipment (e.g. feeder lane) Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above!	☐	String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Restriction regarding "cavity": The production means types based on the PM class "M" (Cavity) are not supported here because the usage counts from the parent equipment are applied for cavities, provided that configuration parameter code no. 8402 [▶▶▶] is active. Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☒	String

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FAILURE_TYPE_CODE	Failure code for the placement procedure data; if the passed failure code is unknown, a corresponding entry is created in the system. Failure codes are booked without further plausibility check; do not confuse them with the station-specific failure types of the iTAC system. Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above! Advice: This key is only evaluated if the QUANTITY_FAIL key is passed!	☐ String
FAILURE_TYPE_DESC	Description of the failure code (for new creation) Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above!	☐ String
PLACEMENT_RECIPE	Name of the placement program Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above!	☐ String
QUANTITY_FAIL	Number of incorrect equipment usages – for the above-mentioned special case "Placement procedure", these are e.g. the mispicks – which are to be booked with the API call. Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above! Advice: If a value is passed here, the FAILURE_TYPE_CODE key must also be passed!	☐ int
QUANTITY_PASS	Number of successful equipment usages that are to be booked with the API call. Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above!	☒ int
SETUP_POSITION	Machine position on which the equipment is set up. Special case: This key is only relevant for the Special case "placement procedure" [▶▶▶] described above!	☐ String
WORKORDER_NUMBER	Production order number (order-based feedback)	☐ String

Tab. 11-99 "Key Value Pair": equReportQuantityKeys

11.27.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equReportQuantityKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>equReportQuantityValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equReportQuantityKeys table. Advice: Only the parameters passed here are processed by the function!
equReportQuantityValues	Values for the parameters selected by means of <code>equReportQuantityKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>equReportQuantityKeys</code> ! Refer to the "Key Value Pair": equReportQuantityKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-100 Parameter (in) »equReportQuantity«

Parameter (out)

Parameter name	Explanation
equReportQuantityResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>equReportQuantityKeys</code> and <code>equReportQuantityValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equReportQuantityKeys</code> !

Tab. 11-101 Parameter (out) »equReportQuantity«

11.27.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1312	Failure	Production means type is invalid
-1313	Failure	Number of equipment items is invalid
-1314	Failure	Equipment -> Default part for feeder/nozzle/head not found
-1315	Failure	Equipment -> Default part for feeder/nozzle/head not configured
-1316	Failure	Production means part not found
-1317	Failure	Equipment -> Default part for feeder/nozzle/head type not found
-1321	Failure	Production means class is invalid

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.28

equSetEquipmentLocation

11.28.1

Function description

This function is used to store or restore one or more production means, or to remove one or more production means from storage. A distinction between the production means classes "Equipment (work step)", "Equipment (asset)", "Cavity", and "Carrier" is currently made in the iTAC system.

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.

When storage, restorage or removal from storage takes place depends on the following conditions:

- **Storage:** A valid storage place is passed and the passed production means currently has **no** storage place.
- **Restorage:** A valid storage place is passed and the passed production means currently has a **different** storage place.
- **Removal from storage:** **No** storage place is passed.

For the production means class **Carrier**, the entire carrier structure is always taken into consideration; i.e. when a carrier number is passed, all parent and subordinate carriers are also stored, restored or removed from storage.

For the production means class **Cavity**, the cavities currently assigned to equipment can only be restored together with the equipment; i.e. when an equipment number is passed, all subordinate cavities are also stored, restored or removed from storage.



Advice

*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »equSetLocation« [▶▶▶] function!
In the array returned, only **incorrectly** processed records with a corresponding error code (reason for error) are returned. In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int equSetEquipmentLocation
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] equipmentLocationKeys
in String[] equipmentLocationValues
out String[] equipmentLocationResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equipmentLocationKeys` and `equipmentLocationValues` as well as (out) `equipmentLocationResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavity Advice: If this key is not passed along, the [E] value is used by default. Note that the PM class must match the selected PM type (see key EQUIPMENT_TYPE)!	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system	☒	String

Key	Explanation	IMSApi data type
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☒ String
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Special cases: If the bar code and storage place number are passed, it is always the bar code that is used for storage and restorage. If neither the bar code nor the storage place number is passed, the production means is removed from storage!	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice and special cases under STORAGE_BARCODE!	☐ String

Tab. 11-102 "Key Value Pair": equipmentLocationKeys

11.28.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentLocationKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>equipmentLocationValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentLocationKeys table. Advice: Only the parameters passed here are processed by the function!
equipmentLocationValues	Values for the parameters selected by means of <code>equipmentLocationKeys</code> ; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>equipmentLocationKeys</code> ! Refer to the "Key Value Pair": attributeUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-103 Parameter (in) »equSetEquipmentLocation«

Parameter (out)

Parameter name	Explanation
equipmentLocationResultValues	The corresponding data is output here only in the event of an error – depending on parameter (in) <code>equipmentLocationKeys</code> and <code>equipmentLocationValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equipmentLocationKeys</code> !

Tab. 11-104 Parameter (out) »equSetEquipmentLocation«

11.28.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.29 equSetLocation

11.29.1 Function description

This function places the passed equipment at a specified storage location; if the equipment is already at different storage location, it is moved. Already stored equipment can be removed from the storage by passing a "0" for the new storage location.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*).



Advice

This function should NO LONGER be used for new installations because the edit options have been extended with release 9.01.00 (for further information, see the new function »equSetEquipmentLocation« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equSetLocation
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String equipmentNo
  in String equipmentIndex
  in String location
```


11.29.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentNo	Number of the equipment in the iTAC system
equipmentIndex	Individual designator for the equipment Default: 0 Advice: The index is required in combination with the equipment number for unique designation; i.e. the equipment number itself does not necessarily need to be unique!
location	Storage place number in the iTAC system Special case: If the value [0] is passed here, the equipment is removed from the storage!

Tab. 11-105 Parameter (in) »equSetLocation«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

11.29.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1302	Warning	Equipment is already assigned to the storage place
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-528	Failure	Storage place was not found
-535	Failure	Storage place is not unique
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1311	Failure	The equipment is set up
-2105	Failure	No TR or PM license present for the station

11.30

equSetupAssetEquipment

11.30.1

Function description

This function is used to set up and strip down asset-related equipment on any entries from the machine assets – at present, these are: site, cell, line, machine group, station and resource. The condition for the setup process is that the device class, specifically the part number on the basis of which the asset equipment was created, has been assigned at the entry concerned in the machine asset structure. Furthermore, the equipment to be set up must currently not be set up on any other object. For the strippdown process, all asset equipment subordinate to the specified machine asset entry can optionally be stripped down.



Tip

*If equipment is to be stripped down for a certain machine asset entry and, in this context, different equipment is to be set up, this is possible with **one** function call. Here it is important that the record with the strippdown information is passed first.*

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for asset-related equipment** (PM class: **A**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

Function matrix *ENTITY_TYPE* (asset)

The dependencies between the keys *ENTITY_TYPE* (data object) and *ENTITY_KEY* (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Resources	11	Resource number

Tab. 11-106 Function matrix *ENTITY_TYPE*



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equSetupAssetEquipment
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] setupAssetEquipmentUploadKeys
in String[] setupAssetEquipmentUploadValues
in String[] setupAssetEquipmentResultKeys
out String[] setupAssetEquipmentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) setupAssetEquipmentUploadKeys, setupAssetEquipmentUploadValues and setupAssetEquipmentResultKeys as well as (out) setupAssetEquipmentResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input parameters for setup and stripdwn process to be performed

Key	Explanation	IMSApi data type
DISSOLVING_LEVEL	Resolution of "machine asset structure" [0; 1]: 0 = Stripdown only on the current machine asset object 1 = Stripdown on the current and all subordinate machine asset objects Advice: If this key is not passed along, the [0] value is used by default! The key is only evaluated if [1] has been selected as the mode for SETUP_FLAG!	☐ int
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [▶▶▶]!	☒ String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 11]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 11 = Resources	☒ int
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER is used and is itself NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed! If [0] is selected as the mode for SETUP_FLAG, the EQUIPMENT_NUMBER must be passed! Special case "stripdwn": If the value [1] is passed for SETUP_FLAG, specification of the EQUIPMENT_NUMBER is optional; if it is passed, it must match the currently set-up equipment; otherwise the stripdwn process will terminate with an error message.	☐ String
INFO	Individual additional information about the setup and stripdwn process (max. 4000 characters)	☐ String
SETUP_FLAG	"Setup" change mode [0; 1]: 0 = rig equipment 1 = strip down equipment Advice: If the value [0] is passed here, the key EQUIPMENT_NUMBER must also be passed!	☒ int

Tab. 11-107 "Key Value Pair": setupAssetEquipmentUploadKeys

Output parameters for performed setup and strippdown process

Key	Explanation	IMSApi data type
DISSOLVING_LEVEL	Resolution of "machine asset structure" [0; 1]: 0 = Stripdown only on the current machine asset object 1 = Stripdown on the current and all subordinate machine asset objects	☐ int
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is output here.	☒ String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 11]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 11 = Resources	☒ int
EQUIPMENT_INDEX	Individual designator for the equipment Advice: Depending on SETUP_FLAG, this is the designator of the stripped-down or set-up equipment!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: Depending on SETUP_FLAG, this is the number of the stripped-down or set-up equipment!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐ int
SETUP_FLAG	"Setup" change mode [0; 1]: 0 = rig equipment 1 = strip down equipment	☐ int

Tab. 11-108 "Key Value Pair": setupAssetEquipmentResultKeys

11.30.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
setupAssetEquipmentUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>setupAssetEquipmentUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": setupAssetEquipmentUploadKeys table. Advice: Only the parameters passed here are processed by the function!
setupAssetEquipmentUploadValues	Values for the parameters selected by means of <code>setupAssetEquipmentUploadKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>setupAssetEquipmentUploadKeys</code> ! Refer to the "Key Value Pair": setupAssetEquipmentUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
setupAssetEquipmentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupAssetEquipmentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>setupAssetEquipmentResultValues</code> !

Tab. 11-109 Parameter (in) »equSetupAssetEquipment«

Parameter (out)

Parameter name	Explanation
setupAssetEquipmentResultValues	Depending on parameter (in) <code>setupAssetEquipmentResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>setupAssetEquipmentResultKeys</code> !

Tab. 11-110 Parameter (out) »equSetupAssetEquipment«

11.30.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1306	Warning	Entity already stripped down
1305	Warning	Entity already set up
1303	Warning	The equipment has expired
1301	Warning	The equipment is already set up
1300	Warning	Equipment test returned warnings
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-574	Failure	Storage location not found
-1301	Failure	Production means not found
-1302	Failure	Production means are invalid

-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1305	Failure	Production means are expired
-1306	Failure	Production means are unavailable
-1309	Failure	Equipment test returned failure
-1310	Failure	Invalid value for "setupFlag"
-1328	Failure	Asset equipment must not be set up
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2072	Failure	Invalid value for "desolvingLevel"
-2105	Failure	No TR or PM license present for the station
-2134	Failure	Entity type is invalid
-2135	Failure	Entity key is invalid
-2136	Failure	Resource not found

11.31

equUpdateAssignedCavity

11.31.1

Function description

This function is used to adapt an existing assignment of cavity (injection molds) to work-step-related equipment (here in the context of an "injection mold holder"). In addition to changing the activation designator and/or position number, this function also assigns the product which is to actually be manufactured. Product assignment is performed based on the work order number in combination with the corresponding work order cluster information.

**Advice**

The assignment of a cavity to equipment is performed using the IMSApi function »equAssignCavityToEquipment« [▶▶▶]. When a work order is assigned, the "Product to cavity" assignment rule stored in the system is checked (for further information, see also the IMSApi function »equManagePartCavityRule« [▶▶▶])!

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).

**Advice**

*A cavity (PM class = **M**) is a subordinate data object of equipment of PM class = **E**; equipment and therefore also the cavities belong to the "production means" grouping.*

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equUpdateAssignedCavity
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] updateEquipmentKeys
    String key
    String value
  in String[] updateAssignedCavityKeys
  in String[] updateAssignedCavityValues
  out String[] updateAssignedCavityResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `updateEquipmentKeys`, `updateAssignedCavityKeys` and `updateAssignedCavityValues` as well as (out) `updateAssignedCavityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input parameters for the desired equipment (header data)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system Advice: If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed!	☒ String

Tab. 11-111 "Key Value Pair": `updateEquipmentKeys`

Input parameters for the cavities to be adapted

Key	Explanation	IMSApi data type
CAVITY_ACTIVE	Activation specification for "cavity" [0; 1]: 0 = Cavity is NOT activated 1 = Cavity is activated Advice: If this key is not passed along, the [1] value is used by default! A cavity is only taken into account from a processing viewpoint if it is active.	☐ int
CAVITY_INDEX	Individual designator for the cavity Advice: The index only needs to be passed if the CAVITY_NUMBER itself is NON-unique!	☐ String
CAVITY_NUMBER	Number of the cavity in the iTAC system Advice: If the cavity number is non-unique, the CAVITY_INDEX key must also be passed!	☒ String
CAVITY_POSITION	Position number of the cavity on the equipment Advice: If the cavity is to be moved to a different position, the used number must not yet be occupied! If the cavity is not to be moved – e.g. if only the activation designator is to be changed – the current position number must be passed here!	☒ int

Key	Explanation	IMSApi data type
CLUSTER_NAME	Unique name for the work order cluster Advice: <i>Work order clusters are used to merge different work orders and to manufacture them in a group (in parallel)!</i> Background to "work order cluster": An injection mold holder can hold multiple, even different cavities. This allows different products/ work orders to be manufactured at the same time; for this purpose, the "work order cluster" concept is applied.	☐ int
CLUSTER_POSITION	Unique position number of the work order (WORKORDER_NUMBER) in the cluster	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record. Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (error code = 5)!</i>	☐ int
WORKORDER_NUMBER	Production order number Advice: <i>The actual product and the associated part number are ascertained using the work order number!</i>	☐ String

Tab. 11-112 "Key Value Pair": `updateAssignedCavityKeys`

11.31.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
updateEquipmentKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": updateEquipmentKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": updateEquipmentKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
updateAssignedCavityKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>updateAssignedCavityValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": updateAssignedCavityKeys table. Advice: Only the parameters passed here are processed by the function!
updateAssignedCavityValues	Values for the parameters selected by means of <code>updateAssignedCavityKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>updateAssignedCavityKeys</code> ! Refer to the "Key Value Pair": updateAssignedCavityKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-113 Parameter (in) »equUpdateAssignedCavity«

Parameter (out)

Parameter name	Explanation
updateAssignedCavityResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>updateAssignedCavityKeys</code> and <code>updateAssignedCavityValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>updateAssignedCavityKeys</code> !

Tab. 11-114 Parameter (out) »equUpdateAssignedCavity«

11.31.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1308	Warning	Cavity is invalid
1307	Warning	Cavity test returned warnings
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-361	Failure	Work order cluster -> Position is invalid for work order
-370	Failure	Work order cluster -> Position not unique for work order
-371	Failure	Work order cluster -> Position not found for work order
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once

-1332	Failure	Cavity not found
-1333	Failure	Cavity is not unique
-1334	Failure	Cavity part is not allowed for assignment
-1338	Failure	Part number of cavity is required
-1342	Failure	More than one assigned cavity found
-1343	Failure	Not a unique cavity setup
-1344	Failure	Cavity position is already occupied for equipment
-1345	Failure	Equipment cavity has an invalid state
-1346	Failure	Equipment cavity has expired
-1347	Failure	Cavity is invalid
-1348	Failure	Equipment cavity not available
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.32 equUpdateEquipment

11.32.1 Function description

This function can be used to update certain characteristics of existing production means. It is thereby possible, e.g., to change the state for production means (EQUIPMENT_STATE). A distinction between the production means classes "Equipment (work step)", "Equipment (asset)", "Cavity", and "Carrier" is currently made in the iTAC system (EQUIPMENT_CLASS).

Differences between "production means classes": To distinguish more clearly between the production means classes, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module* → *Master data* → *Production means*). The different characteristics of "Equipment (work step)", "Equipment (asset)", "Carriers" and "Cavities" mean that some of the available keys are not equally relevant; note the corresponding advice notices. If only the general term "Production means" is used, the key – and therefore the underlying characteristic – is independent of the PM class and applies equally to all equipment, carriers and cavities.



Advice

*This API function is influenced by site parameter code no. 18309 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equUpdateEquipment
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] equipmentUpdateKeys
  in String[] equipmentUpdateValues
  out String[] equipmentUpdateResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) equipmentUpdateKeys and equipmentUpdateValues as well as (out) equipmentUpdateResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
EQUIPMENT_CHANGE_COMMENT	Individual change comment; the text entered here is saved in the change history of the respective production means; it can be viewed there at any time.	☐	String
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: <i>If this key is not passed, it must be ensured that the specified production means (EQUIPMENT_NUMBER) is not used multiple times for different classes!</i>	☐	String
EQUIPMENT_INDEX	Individual designator for the production means Advice: <i>The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!</i>	☐	String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: <i>If the equipment number is non-unique, the EQUIPMENT_INDEX key and, if applicable, the EQUIPMENT_CLASS key must also be passed!</i>	☒	String
EQUIPMENT_STATE	State of the production means [0; 10; 20; 30; 40; 50; 90]: 0 = Available 10 = Being used (for carriers only) 20 = Being repaired 30 = Being maintained 40 = Defective 50 = Locked 90 = Deleted Advice: <i>Depending on the EQUIPMENT_CLASS key, this is the equipment or carrier state; the state [10] is possible for carriers ONLY! If the key is not passed, the value [0] is used by default!</i> Advice: <i>Changing the equipment state does not automatically affect the end of the validity period; i.e. if the end of the validity period is to be changed, the new value must be passed using VALID_TO!</i>	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒	int

Key	Explanation	IMSApi data type
SETUP_POSITION	Machine position on which the equipment is set up. Advice: <i>This key applies ONLY to production means of the PM class "Equipment (work step)"!</i>	☐ String
VALID_FROM	Start of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes along the current date and time; if [-2] is passed along, the date and time of 01-01-1970, 0:00:00.000 is used as start time!	☐ DateMs
VALID_TO	End of the validity period for the production means; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes along the current date and time; if [-2] is passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time! Advice: <i>If the production means state is changed (cf. the EQUIPMENT_STATE key), this does NOT automatically affect the end of the validity period!</i>	☐ DateMs

Tab. 11-115 "Key Value Pair": *equipmentUpdateKeys*

11.32.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
equipmentUpdateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>equipmentUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
equipmentUpdateValues	Values for the parameters selected by means of <code>equipmentUpdateKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>equipmentUpdateKeys</code>! Refer to the "Key Value Pair": equipmentUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-116 Parameter (in) »equUpdateEquipment«

Parameter (out)

Parameter name	Explanation
equipmentUpdateResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>equipmentUpdateKeys</code> and <code>equipmentUpdateValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>equipmentUpdateKeys</code>!

Tab. 11-117 Parameter (out) »equUpdateEquipment«

11.32.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1304	Warning	Equipment was stripped down
7	Warning	At least one record of the array has been processed with a warning
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1301	Failure	Production means not found
-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1312	Failure	Production means type is invalid
-1318	Failure	Production means -> Change not allowed
-1321	Failure	Production means class is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

11.33 equUpdateEquipmentData

11.33.1 Function description

This function defines which equipment is assigned to the station. The `setupFlag` parameter can be used to define whether the passed equipment is to be assigned to (set up) or removed from (stripped down) the station.

Differences between "Equipment (work step)" and "Equipment (asset)": The different characteristics of work-step-related and asset-related equipment mean that a number of functions cannot be used to the same extent. This IMSAPI function is relevant **ONLY for work-step-related equipment** (PM class: **E**). To distinguish more clearly between the equipment, or more precisely the underlying production means class, it is essential to read the corresponding chapter in the user's guide (see there under *ADM module → Master data → Production means*).



Advice

This API function is affected by station parameter code no. 8410 [▶▶▶] as well as by site parameter code nos. 13030 [▶▶▶] and 13050 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Background to "Equipment setup with cavities": If equipment is to be used in the context of an "injection mold holder", cavities (injection molds) must be assigned there beforehand (for further information, see the IMSAPI function [»equAssignCavityToEquipment«](#) [▶▶▶]). For equipment with cavities, not only the usage counter for the equipment but also the usage counters of the subordinate (active) cavities are started at the time of equipment setup. The same applies for the strippdown process; if the equipment is stripped down from the station, all usage counters are stopped.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int equUpdateEquipmentData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int setupFlag
in String serialNumber
in String workOrderNumber
in int processLayer
in String[] equipmentUploadKeys
in String[] equipmentUploadValues
out String[] equipmentResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `equipmentUploadKeys` and `equipmentUploadValues` as well as (out) `equipmentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means as equipment, cavities, and carriers [A; C; E; M]; in this context, this is only the "Equipment (work step)" PM class: E = Equipment (work step)	☐	String
EQUIPMENT_INDEX	Individual designator for the equipment Special case: If the equipment number is unique without the addition of the index, an empty string or the value [-1] can be passed here!	☐	String
EQUIPMENT_NUMBER	Number of the equipment in the iTAC system	☒	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
SETUP_POSITION	Machine position on which the equipment is set up.	☐	String

Tab. 11-118 "Key Value Pair": *equipmentUploadKeys*

11.33.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
setupFlag	"Setup" change mode [0; 1]: 0 = rig equipment 1 = strip down equipment
serialNumber	Serial number of the panel Advice: Parameter not currently used!
workOrderNumber	Number of a production order Advice: Parameter not currently used!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: Parameter not currently used!
equipmentUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>equipmentUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": equipmentUploadKeys table. Advice: Only the parameters passed here are processed by the function!
equipmentUploadValues	Values for the parameters selected by means of <code>equipmentUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) equipmentUploadKeys! Refer to the "Key Value Pair": equipmentUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 11-119 Parameter (in) »updateEquipmentData«

Parameter (out)

Parameter name	Explanation
equipmentResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>equipmentUploadKeys</code> and <code>equipmentUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) equipmentUploadKeys!

Tab. 11-120 Parameter (out) »updateEquipmentData«

11.33.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1308	Warning	Cavity is invalid
1307	Warning	Cavity test returned warnings
1303	Warning	The equipment has expired
1301	Warning	The equipment is already set up
1300	Warning	Equipment test returned warnings
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-230	Failure	Station is invalid for this work order
-1301	Failure	Production means not found
-1302	Failure	Production means are invalid
-1303	Failure	Production means number found more than once

-1304	Failure	Production means have an invalid state
-1305	Failure	Production means are expired
-1306	Failure	Production means are unavailable
-1310	Failure	Invalid value for "setupFlag"
-1345	Failure	Equipment cavity has an invalid state
-1346	Failure	Equipment cavity has expired
-1347	Failure	Cavity is invalid
-1348	Failure	Equipment cavity not available
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

12 Group "EXEC" (ruleset)

12.1 execFormula

12.1.1 Function description

This function executes one or more formulas and makes the respective results available for retrieval.



Advice

The formula to be executed must already be stored in the system; formulas are identified via a unique designator (see below `FORMULA_ID`). If, for example, a formula from a test plan is applied, the `FORMULA_ID` must be determined beforehand using the API function »trGetTestplanData« [▶▶▶]!

*The array returned (`resultValues`) contains only **incorrectly** processed data records with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int execFormula
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] formulaReferences
    String key
    String value
in String[] formulaKeys
in String[] formulaValues
in String[] parameterKeys
in String[] parameterValues
in String[] resultKeys
out String[] resultValues
in String[] missingParameterKeys
out String[] missingParameterValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `formulaReferences`, `formulaKeys`, `parameterKeys`, `resultKeys` and (out) `resultValues`, `missingParameterKeys` and `missingParameterValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Reference object

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel Advice: It is mandatory that a serial number be passed: this is used to ascertain whether formula results were already stored in the data-base; in this case, only the formula results are updated. Furthermore, concrete attribute values are ascertained on the basis of the serial number (serial number attributes or work order attributes) provided they are used in the formula as parameters!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String

Tab. 12-1 "Key Value Pair": *formulaReferences*

Identification number of the formula

Key	Explanation	IMSApi data type
FORMULA_ID	Internal identification number of the formula used for the calculation Advice: The formula is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long

Tab. 12-2 "Key Value Pair": *formulaKeys*

Formula value for the test characteristic

**Advice**

This "Key Value Pair" structure is only relevant for the **TESTSTEP** parameter type (normal formula parameter); the respective values for the **SNR_ATTRIB** (serial number attribute) and **WO_ATTRIB** (work order attribute) parameter types are automatically ascertained and passed by the system!

Key	Explanation	IMSApi data type
MEASURE_VALUE	Formula value; the following format definitions are valid: Decimal EN (example: 1366.665) [0-9]*.[0-9]* Advice: This is used to pass the formula value of the test characteristic currently ascertained on the client (see TEST_STEP_ID) and process it using the specified formula. If there is already a measured value for the test characteristic in the database, the value passed here is still used for the calculation!	☒ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long

Tab. 12-3 "Key Value Pair": *parameterKeys*

Result of the formula calculation

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FORMULA	Formula string (e.g. "a + b")	☐ String
FORMULA_DESCRIPTION	Description of the formula	☐ String
FORMULA_ID	Internal identification number of the formula used for the calculation	☒ long
FORMULA_NAME	Formula name	☐ String
FORMULA_RESULT	Result of the calculation	☒ double
MISSING_PARAMETER	Number of missing formula parameters: 0 = No missing parameters 0 = Number of missing parameters Advice: The value corresponds to the number of faulty records in the missingParameterKeys "Key Value Pair" structure!	☒ int

Tab. 12-4 "Key Value Pair": *resultKeys*

Formula parameter missing

Key	Explanation	IMSApi data type
FORMULA_ID	Internal identification number of the formula with the missing formula parameters	☒ long
PARAMETER_DESCRIPTION	Description of the missing formula parameter	☐ String
PARAMETER_KEY	Parameter name in the formula (e.g. "a")	☐ String
PARAMETER_REFERENCE	Reference objects for the formula parameter; output is dependent on PARAMETER_TYPE and is made up of the following components: - TESTSTEP = "[test step] - [measurement step name]" - SNR_ATTRIB = "[attribute code] - [attribute designator]" - WO_ATTRIB = "[attribute code] - [attribute designator]"	☐ String
PARAMETER_TYPE	Type of formula parameter: TESTSTEP = (normal) formula parameter SNR_ATTRIB = serial number attribute WO_ATTRIB = work order attribute	☐ String

Tab. 12-5 "Key Value Pair": *missingParameterKeys*

12.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
formulaReferences	"Key Value Pair" structure with the following variables:
key	List of the reference object parameters for which a formula is to be calculated; the corresponding values are passed with the value parameter. A description of the possible object parameters can be found above in the "Key Value Pair": formulaReferences table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": formulaReferences table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
formulaKeys	List of the parameters that are to be passed by the function (here only FORMULA_ID); the corresponding values are passed with the formulaValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": formulaKeys table. Advice: Only the parameters passed here are processed by the function!

Parameter name	Explanation
formulaValues	Values for the parameters selected by means of <code>formulaKeys</code> (here only <code>FORMULA_ID</code>). Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>formulaKeys</code>! Refer to the "Key Value Pair": <code>formulaKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
parameterKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>parameterValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>parameterKeys</code> table. Advice: Only the parameters passed here are processed by the function!
parameterValues	Values for the parameters selected by means of <code>parameterKeys</code>. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>parameterKeys</code>! Refer to the "Key Value Pair": <code>parameterKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
resultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>resultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>resultValues</code>!
missingParameterKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>missingParameterKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>missingParameterValues</code>!

Tab. 12-6 Parameter (in) »execFormula«

Parameter (out)

Parameter name	Explanation
resultValues	Depending on parameter (in) <code>resultKeys</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>resultKeys</code>!
missingParameterValues	Depending on parameter (in) <code>missingParameterKeys</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>missingParameterKeys</code>!

Tab. 12-7 Parameter (out) »execFormula«

12.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-199	Failure	Formula not found
-2000	Failure	Formula parameter missing
-2001	Failure	Formula parameter not convertible
-2002	Failure	Formula is invalid
-2003	Failure	Formula calculation failed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

12.2

execFormulaCatalog

12.2.1

Function description

This function executes one or more formulas from the formula catalog and makes the respective results available for retrieval. The index can be used to execute one and the same formula multiple times in a single call. The index unambiguously identifies the formula to be executed and the parameters to be used. Unlike the IMSApi function »execFormula« [▶▶▶], this function has no reference to a serial number and/or to a test plan.



Advice

*The formula to be executed must already be stored in the formula catalog of the iTAC system; formulas are identified by a unique name (see `FORMULA_NAME` below). The formula name as well as other parameters of the formula can be queried in advance by means of the API function »mdataGetFormulaCatalog« [▶▶▶]! The array returned contains only records that were processed **incorrectly**, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int execFormulaCatalog
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] formulaKeys
  in String[] formulaValues
  in String[] parameterKeys
  in String[] parameterValues
  in String[] formularResultKeys
  out String[] formularResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `formulaKeys`, `formulaValues`, `parameterKeys`, `parameterValues` and `formularResultKeys` as well as (out) `formularResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Passing of formulas to be executed

Key	Explanation	IMSApi data type
FORMULA_NAME	Formula name	☒ String
INDEX	Unique reference number (sequential number) for each record in the array; Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": parameterKeys and "Key Value Pair": formularResultKeys tables)! Background: On the basis of this reference number, it is possible to execute the same formula multiple times with just one call.	☒ long

Tab. 12-8 "Key Value Pair": formulaKeys

Passing of parameters required for respective formula

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array; Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": formulaKeys and "Key Value Pair": formularResultKeys tables)!	☒ long
PARAMETER_NAME	Name of the parameter	☒ String
PARAMETER_VALUE	Value of the parameter	☒ double

Tab. 12-9 "Key Value Pair": parameterKeys

Output of calculation results

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
FORMULA_RESULT	Result of the calculation	☐ double
INDEX	Unique reference number (sequential number) for each record in the array; Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see key of the same name in the "Key Value Pair": formulaKeys and "Key Value Pair": parameterKeys tables)!	☐ long

Tab. 12-10 "Key Value Pair": formularResultKeys

12.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
formulaKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>formulaValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": formulaKeys table. Advice: Only the parameters passed here are processed by the function!
formulaValues	Values for the parameters selected by means of <code>formulaKeys</code>. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>formulaKeys</code>! Refer to the "Key Value Pair": formulaKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
parameterKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>parameterValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": parameterKeys table. Advice: Only the parameters passed here are processed by the function!
parameterValues	Values for the parameters selected by means of <code>parameterKeys</code>. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>parameterKeys</code>! Refer to the "Key Value Pair": parameterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
formularResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": formularResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>formularResultValues</code>!

Tab. 12-11 Parameter (in) »execFormulaCatalog«

Parameter (out)

Parameter name	Explanation
formularResult-Values	Depending on parameter (in) <code>formularResultKeys</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>formularResultKeys</code>!

Tab. 12-12 Parameter (out) »execFormulaCatalog«

12.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-199	Failure	Formula not found
-2001	Failure	Formula parameter not convertible
-2002	Failure	Formula is invalid
-2003	Failure	Formula calculation failed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

12.3

execRuleSet

12.3.1

Function description

This function loads a decision table (MDA item), injects a so-called "RuleObject", evaluates the rules and returns the result.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	RULE Engine (site license)

Function declaration

```
int execRuleSet
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String ruleType
in String ruleSet
in KeyValue[] ruleProperties
    String key
    String value
out RuleResult[] ruleResults
    String ruleResult
int prio
```

12.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
ruleType	Rule types [S; C; SP]: S = Rules for serial numbers C = Rules for containers SP = Rules for storage places
ruleSet	Name of the decision table (MDA item)
ruleProperties	"Key Value Pair" structure with the following variables:

Parameter name	Explanation
key	List of the rule parameters for the selected rule type; the corresponding values are passed with the <code>value</code> parameter. The following keys exist for rule type S: <code>SERIAL_NUMBER</code> = Serial number of the panel <code>WORKORDER_NUMBER</code> = Number of the production order Advice: Note that both keys must always be passed for rule type [S]; pay attention to upper- and lower-case letters! The following keys exist for rule type C: <code>CONTAINER_NUMBER</code> = Container number The following keys exist for rule type SP: <code>STORAGE_BARCODE</code> = Bar code for the storage place <code>STORAGE_NUMBER</code> = Storage place number Advice: If both keys are specified for rule type [SP], they must refer to the same storage place!
value	Value list for the rule parameters passed by means of <code>key</code> .

Tab. 12-13 Parameter (in) »execRuleSet«

Parameter (out)

Parameter name	Explanation
ruleResults	Array of the structure with the following variables:
ruleResult	Result of the rule evaluation
prio	Priority ID

Tab. 12-14 Parameter (out) »execRuleSet«

12.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-144	Failure	No decision table found for this rule
-145	Failure	Failure while accessing the rule
-146	Failure	Failure in the rule
-147	Failure	Rule type is not correct
-148	Failure	Required rule parameters are missing
-503	Failure	Container not found
-574	Failure	Storage location not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2106	Failure	No iRule license present for the site

12.4

execTimerProcess

12.4.1

Function description

This function is used to manually execute the timers available in the iTAC system. However, this only applies to timers which have been activated. This allows a timer to be executed irrespective of its next planned execution time.



Advice

The array returned is only filled if the selected timer CANNOT be started because the associated process is already being executed at the current point in time. In this case, information about the ongoing process is then output!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int execTimerProcess
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] timerProcessFilter
    String key
    String value
  in String[] timerProcessResultKeys
  out String[] timerProcessResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `timerProcessFilter` and `timerProcessResultKeys` as well as (out) `timerProcessResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Passing of the timer to be started

Key	Explanation	IMSApi data type
TIMER_ID	Name of the timer available in the iTAC system (e.g. "timer.erp.ido-creceive").	☒ String

Tab. 12-15 "Key Value Pair": *timerProcessFilter*

Definition of the output parameters in the event of an error

Key	Explanation	IMSApi data type
HOSTNAME	Name of the computer on which the timer is currently being executed (unique designation of a computer in a network).	☐ String
START_TIME	Start time of the timer in milliseconds Example: 1510845103096 milliseconds correspond to 16.11.2017, 16:11:43.000	☐ DateMS

Tab. 12-16 "Key Value Pair": *timerProcessResultKeys*

12.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
timerProcessFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": timerProcessFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": timerProcessFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
timerProcessResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": timerProcessResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>timerProcessResultValues</code> !

Tab. 12-17 Parameter (in) »execTimerProcess«

Parameter (out)

Parameter name	Explanation
timerProcessResultValues	The corresponding data is output here only in the event of an error – depending on the <code>timerProcessResultKeys</code> parameter (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>timerProcessResultKeys</code> !

Tab. 12-18 Parameter (out) »execTimerProcess«

12.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

13 Group "IMSAPI" (metadata)

13.1 imsapiGetErrorList

13.1.1 Function description

With this function, the error codes and associated output texts (usually they are the short error descriptions) that exist in the system can be queried. You can thereby search for individual **error codes**, for specific **function names** or for specific **function groups** (e.g., LOCK or MDA).

These three search filters are executed according to a specified priority; they are not linked by "OR". If the "error codes" filter is used (`errorCodes` parameter), the "function name" filter (`functionNameArray` parameter) and "function group" (`groupNameArray` parameter) are ignored. If the "function name" filter is used, the "function group" filter is ignored; in this case, the "error codes" filter may not be set. If the "function group" filter is to be used, the "function name" and "error codes" filters may not be set.



Advice

*The array returned contains **incorrectly** processed records with a corresponding error code (see key `RETURN_CODE`). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Function declaration

```
int imsapiGetErrorList
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String[] localeArray
  in String[] groupNameArray
  in String[] functionNameArray
  in int[] errorCodes
  in String[] errorCodeResultKeys
  out String errorCodeResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `errorCodeResultKeys` and (out) `errorCodeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation	IMSApi data type
ERROR_CODE	Output value from the function	☐ String
ERROR_TEXT	Output text for output value (ERROR_CODE)	☐ String
LOCALE	Language of the output text (ERROR_TEXT); by default, the output texts are stored in the following languages: de = German en = English fr = French	☐ String
RETURN_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int

Tab. 13-1 "Key Value Pair": *errorCodeResultKeys*

13.1.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
localeArray	Language of the output text; by default, the output texts are stored in the following languages: de = German en = English fr = French At least one language must be passed; if the passed language code is not found, the current system language is used by default. Advice: If an empty array is passed along (array length = 0), then it terminates the function with a failure (error code = -105)!

Parameter name	Explanation
groupNameArray	Filter for function group(s) whose error codes – and, if necessary, the associated output texts too – are to be ascertained; in the background, at first all functions of this group/these groups will be ascertained and then the error codes used there. Currently, there are the following function groups: - ADVICE (advice) - APS (work order planning) - ATTRIB (attributes) - BATCH (bin) - CONFIG (configuration) - CUSTOM (customer function) - EDA (external data) - EQU (production means) - EXEC (ruleset) - IMSAPI (metadata) - LOCK (lock) - MDA (document management) - MDATA (master data) - MDC (machine data collection) - ML (material & logistics) - MSL (Moisture Sensitivity Level) - PM (production management) - REG (registration) - SHIP (packing) - SETUP (setup) - SMT (placement data) - TR (traceability) Special case: If an array element is passed along with the value [-1], the error codes of all function groups are output! Advice: Pay attention to the proper use of upper case and lower case; for instance, entering "Tr" will not produce a result! If an empty array is passed along (array length = 0), then this parameter will not be evaluated. This is also true if the filters "function name" and "error code" are used!
functionNameArray	Filter for function name(s) whose error codes – and, if necessary, the associated output texts too – are to be ascertained. Special case: If an array element is passed along with the value [-1], the error codes of all functions are output! Advice: If an empty array is passed along (array length = 0), then this parameter is not evaluated; this also applies if the "error code" filter is used!
errorCodes	Filter for error code(s) that are to be ascertained; with this, you can make a focused search to ascertain the output texts of already recognized error codes. Advice: If an empty array is passed along (array length = 0), then this parameter is not evaluated; in this case, the "function name" or "function group" filter must be used!
errorCodeResultKeys	List of the error code parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": errorCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) errorCodeResultValues!

Tab. 13-2 Parameter (in) »imsapiGetErrorList«

Parameter (out)

Parameter name	Explanation
errorCodeResult-Values	Depending on parameter (in) <code>errorCodeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>errorCodeResultKeys</code> !

Tab. 13-3 Parameter (out) »imsapiGetErrorList«

13.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-13	Failure	Error code not found
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1400	Failure	Function group not found
-1401	Failure	Function names not found
-2070	Failure	IMSApi key was passed multiple times to the function

13.2

imsapiGetErrorText

13.2.1

Function description

This function outputs a brief, explanatory output text for a specific error code.

Function declaration

```
int imsapiGetErrorText
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in int errorCode
out String errorString
```

13.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	- Parameter is not evaluated -
persId	- Parameter is not evaluated -
locale	Language settings (e.g. "de_DE") Advice: The output text is output in the language specified here (currently supported are "de_DE", "en_US" and "fr_FR")!
errorCode	Output value from the function

Tab. 13-4 Parameter (in) »imsapiGetErrorText«

Parameter (out)

Parameter name	Explanation
errorString	Output text for output value (errorCode)

Tab. 13-5 Parameter (out) »imsapiGetErrorText«

13.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of this function if necessary!

Error code	Category	Output text
0	OK	[EMPTY]
-10	Failure	Server error
-13	Failure	Error code not found
-105	Failure	A required parameter is missing

13.3 imsapiGetFunctionInfo

13.3.1 Function description

This function returns information on a specific IMSApi function; this includes the function header information as well as details on the respective input and output parameters.

Function declaration

```
int imsapiGetFunctionInfo
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String functionName
  out IMSApiFunctionStruct functionInfo
    String functionName
    String functionDescr
    String groupName
    String sinceVersion
  out IMSApiParameterStruct[] parameterArray
    String paramName
    String paramDescr
    String dataType
    String conditionParamName
    boolean hasKeys
    String keyParamName
  int direction
```

13.3.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
functionName	Name of the IMSApi function for which information is output

Tab. 13-6 Parameter (in) »imsapiGetFunctionInfo«

Parameter (out)

Parameter name	Explanation
functionInfo	Structure with the following variables:
functionName	Name of the IMSApi function
functionDescr	Short description of the IMSApi function
groupName	Name of the function group
sinceVersion	Release version upon introduction of the IMSApi function
parameterArray	Array of the structure with the following variables:
paramName	Parameter name
paramDescr	Short description of the parameter
dataType	IMSApi data type

Parameter name	Explanation
conditionParam- Name	Parameter that influences the parameter output by means of paramName Background: A parameter dependency exists if a variable "Key Value Pair" structure that can take on different features depending on the parameter (conditionParamName) ascertained here is stored for the passed parameter (paramName) (see, e.g., the »attribAppendAttributeValues« [▶▶▶] functions and there the attributeUploadKeys and objectType parameters). There, objectType controls which attribute category is to be created, which in turn affects the required input parameters (attributeUploadKeys).
hasKeys	"Key Value Pair" designator [true; false]: true = Parameter contains a "Key Value Pair" structure false = Parameter without "Key Value Pair" structure
keyParamName	Parameter name of the "dependent" parameter Background: A parameter dependency exists if the passed parameter (parameterName) is part of a "Key Value Pair" structure (see, e.g., the »attribAppendAttributeValues« [▶▶▶] functions). If the attributeUploadKeys parameter were to be passed, the attributeUploadValues parameter would be output here.
direction	Designation of input or output parameter [1; 2]: 1 = Parameter (in) 2 = Parameter (out)

Tab. 13-7 Parameter (out) »imsapiGetFunctionInfo«

13.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-1401	Failure	Function names not found

13.4 imsapiGetFunctions

13.4.1 Function description

This function returns all IMSApi functions included in a function group.

Function declaration

```
int imsapiGetFunctions
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String groupName
out ImsApiFunctionStruct[] functionArray
    String functionName
    String functionDescr
    String groupName
    String sinceVersion
```

13.4.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
groupName	Name of the function group for which the IMSApi functions are output

Tab. 13-8 Parameter (in) »imsapiGetFunctions«

Parameter (out)

Parameter name	Explanation
functionArray	Array of the structure with the following variables:
functionName	Name of the IMSApi function
functionDescr	Short description of the IMSApi function
groupName	Name of the function group
sinceVersion	Release version upon introduction of the IMSApi function

Tab. 13-9 Parameter (out) »imsapiGetFunctions«

13.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-1400	Failure	Function group not found

13.5 imsapiGetGroups

13.5.1 Function description

This function returns all function groups that exist in the system.

Function declaration

```
int imsapiGetGroups
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  out ImsApiGroupStruct[] groupArray
    String groupName
    String groupDescr
```

13.5.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)

Tab. 13-10 Parameter (in) »imsapiGetGroups«

Parameter (out)

Parameter name	Explanation
ImsApiGroupStruct	Array of the structure with the following variables:
groupName	Name of the function group
groupDescr	Short description of the function group

Tab. 13-11 Parameter (out) »imsapiGetGroups«

13.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error

13.6 imsapiGetParameterConditions

13.6.1 Function description

If there is a dependency between two parameters of an IMSApi function, this function can be used to ascertain the "influencing" parameter; all stored parameter values are output in an array.



Advice

A parameter dependency exists if a variable "Key Value Pair" structure that can take on different features depending on the parameter (*conditionParamName*) ascertained here is stored for the passed parameter (*parameterName*) (see, e.g., the »*attribAppendAttributeValues*« [▶▶▶] functions and there the *attributeUploadKeys* and *objectType* parameters). There, *objectType* controls which attribute category is to be created, which in turn affects the required input parameters (*attributeUploadKeys*).

Function declaration

```
int imsapiGetParameterConditions
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String functionName
in String parameterName
out String conditionParamName
out ImsApiParamConditionStruct[] parameterConditionArray
    String conditionValue
    String conditionDescr
    String sinceVersion
```

13.6.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
functionName	Name of the IMSApi function
parameterName	Parameter name for which the "influencing" parameter is ascertained

Tab. 13-12 Parameter (in) »imsapiGetParameterConditions«

Parameter (out)

Parameter name	Explanation
conditionParamName	Parameter that influences the parameter passed by means of <i>parameterName</i>
parameterConditionArray	Array of the structure with the following variables:
conditionValue	Parameter value

Parameter name	Explanation
conditionDescr	Short description of the parameter value
sinceVersion	Release version upon introduction of the parameter value

Tab. 13-13 Parameter (out) »imsapiGetParameterConditions«

13.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-1401	Failure	Function names not found
-1402	Failure	Function parameter not found

13.7 imsapiGetParameterInfo

13.7.1 Function description

This function returns information on a specific parameter of an IMSApi function. If a "Key Value Pair" structure is stored for the queried parameter, this is output – if applicable, in dependence on a controlling parameter value; each key is listed separately in the output array.

Function declaration

```
int imsapiGetParameterInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String functionName
in String parameterName
in String conditionValue
out ImsApiParameterStruct parameter
    String paramName
    String paramDescr
    String dataType
    String conditionParamName
    boolean hasKeys
    String keyParamName
    int direction
out ImsApiParamConditionStruct condition
    String conditionValue
    String conditionDescr
    String sinceVersion
out ImsApiKeyStruct[] keys
    String keyName
    String keyDescr
    String dataType
    String sinceVersion
    boolean isRequired
```

13.7.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
functionName	Name of the IMSApi function
parameterName	Parameter name for which the information is to be output
conditionValue	Parameter value of the "influencing" parameter Advice: This parameter is only relevant if the queried parameter (parameterName) has a dependency on another parameter (see below conditionParamName)! A parameter dependency exists if, for a passed parameter, a "Key Value Pair" structure is stored which takes on different features on the basis of this parameter value.

Tab. 13-14 Parameter (in) »imsapiGetParameterInfo«

Parameter (out)

Parameter name	Explanation
parameter	Structure with the following variables:
paramName	Parameter name (corresponds to the <code>parameterName</code> input parameter)
paramDescr	Short description of the parameter
dataType	IMSApi data type
conditionParamName	Parameter that influences the parameter output by means of <code>paramName</code> Background: A parameter dependency exists if a variable "Key Value Pair" structure that can take on different features depending on the parameter (<code>conditionParamName</code>) ascertained here is stored for the passed parameter (<code>paramName</code>) (see, e.g., the » attribAppendAttributeValues « [▶▶▶] functions and there the <code>attributeUploadKeys</code> and <code>objectType</code> parameters). There, <code>objectType</code> controls which attribute category is to be created, which in turn affects the required input parameters (<code>attributeUploadKeys</code>).
hasKeys	"Key Value Pair" designator [true; false]: true = Parameter contains a "Key Value Pair" structure false = Parameter without "Key Value Pair" structure
keyParamName	Parameter name of the "dependent" parameter Background: A parameter dependency exists if the passed parameter (<code>parameterName</code>) is part of a "Key Value Pair" structure (see, e.g., the » attribAppendAttributeValues « [▶▶▶] functions). If the <code>attributeUploadKeys</code> parameter were to be passed, the <code>attributeUploadValues</code> parameter would be output here.
direction	Designation of input or output parameter [1; 2]: 1 = Parameter (in) 2 = Parameter (out)
condition	Array of the structure with the following variables: Advice: This structure is only filled with values if there is an "influencing" parameter (see <code>conditionParamName</code>)!
conditionValue	Parameter value
conditionDescr	Short description of the parameter value
sinceVersion	Release version upon introduction of the parameter value
keys	Array of the structure with the following variables: Advice: This structure is only filled with values if a "Key Value Pair" structure is stored for the passed parameter (see <code>parameterName</code>)!
keyName	Name of the key
keyDescr	Short description of the key
dataType	IMSApi data type
sinceVersion	Release version upon introduction of the key
isRequired	"Mandatory parameter" designation

Tab. 13-15 Parameter (out) »imsapiGetParameterInfo«

13.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-1401	Failure	Function names not found
-1402	Failure	Function parameter not found

13.8 imsapiGetSystemInfos

13.8.1 Function description

This function returns selected system information.

Function declaration

```

int imsapiGetSystemInfos
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String[] systemInfoResultKeys
out String[] systemInfoResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `systemInfoResultKeys` and (out) `systemInfoResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
DB_ADMIN_VERSION	Version of the internal database administration tool	☐	double
DB_INSTALL_STAMP	Point in time of the database installation; specification of the total time in seconds since 01-01-1970, 0:00:00. Example: 1262443605 seconds correspond to 01-02-2010, 14:46:45	☐	DateMs
DB_JDBC_DRIVER	Version of the used JDBC driver Example: 12.1.0.1.0	☐	String
DB_JDBC_URL	URL (connection string) for "MES system" database connection Example: jdbc:oracle:thin:@server99:8888:mesdb	☐	String
DB_READONLY_JDBC_URL	URL (connection string) for "backup system" database connection (active data guard) Example: jdbc:oracle:thin:@server98:8888:backupdb	☐	String
DB_REPLICA_JDBC_URL	URL (connection string) for "BigData system" database connection (replica of the MES database) Example: jdbc:oracle:thin:@server97:8888:bigdatadb	☐	String

Key	Explanation	IMSApi data type
DB_VERSION	Current database version of the installed iTAC system Example: <i>release 8.00</i>	☐ String
DB_VERSION_DATE	Time of creation for the current database version (see key DB_VERSION); specification of the total time in seconds since 01-01-1970, 0:00:00. Example: <i>1262443605 seconds correspond to 01-02-2010, 14:46:45</i>	☐ DateMs
MES_BUILD_TIME	Time of creation (build date) of the installed program version (see key MES_VERSION) Example: <i>20110314.172002 -> corresponds to 03-14-2011 at 17:20:02</i>	☐ String
MES_VERSION	Programm version of the installed iTAC system Example: <i>8.00.00-7</i>	☐ String

Tab. 13-16 "Key Value Pair": *systemInfoResultKeys*

13.8.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
systemInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": systemInfoResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) systemInfoResultValues!</i>

Tab. 13-17 Parameter (in) »imsapiGetSystemInfos«

Parameter (out)

Parameter name	Explanation
systemInfoResultValues	Depending on parameter (in) systemInfoResultKeys, the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) systemInfoResultKeys!</i>

Tab. 13-18 Parameter (out) »imsapiGetSystemInfos«

13.8.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function

14 Group "LOCK" (lock)

14.1 lockGetLockedObjects

14.1.1 Function description

This function is used for querying locked serial numbers, containers or material lots. Objects locked in the past that were already unlocked again at the time the query was made (historical lock entries) can also be queried using this function. All search criteria for ascertaining the desired data are "AND" linked. As a result, only those objects that satisfy all used filter conditions are returned.



Advice

When interpreting the output, note the following sort order:

1. Name of the affected object (locks and unlocks)
2. Name of the lock group

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)	OR	TR license (passive)	OR	PM license
--	---------------------	----	----------------------	----	------------

Function declaration

```
int lockGetLockedObjects
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in String lockGroupName
in DateMs lockDateFrom
in DateMs lockDateTo
in int lockState
in int maxRows
in String[] objectResultKeys
out String[] objectResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) objectResultKeys and (out) objectResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Serial number (objectType = 0)

Key	Explanation		IMSApi data type
LOCK_COUNT	Number of currently active locks for the object	☐	int
LOCK_DATE	Point in time the lock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
LOCK_INFORMATION	Lock information (reason for locking)	☐	String
LOCK_STATION	Station number of the work station in the iTAC system that performed the lock	☐	String
LOCK_USER	Login name of the user in the iTAC system who performed the lock	☐	String
LOCKGROUP_NAME	Lock group	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the serial number!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the serial number!	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String
UNLOCK_DATE	Point in time the unlock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐	DateMs
UNLOCK_INFORMATION	Lock information (reason for unlocking) Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐	String
UNLOCK_STATION	Station number of the work station in the iTAC system that performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐	String
UNLOCK_USER	Login name of the user in the iTAC system who performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 14-1 "Key Value Pair" (serial number): objectResultKeys

Container (objectType = 1)

Key	Explanation	IMSApi data type
LOCK_COUNT	Number of currently active locks for the object	☐ int
LOCK_DATE	Point in time the lock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
LOCK_INFORMATION	Lock information (reason for locking)	☐ String
LOCK_STATION	Station number of the work station in the iTAC system that performed the lock	☐ String
LOCK_USER	Login name of the user in the iTAC system who performed the lock	☐ String
LOCKGROUP_NAME	Lock group	☐ String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐ String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the container material!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the container material!	☐ String
UNLOCK_DATE	Point in time the unlock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ DateMs
UNLOCK_INFORMATION	Lock information (reason for unlocking) Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String
UNLOCK_STATION	Station number of the work station in the iTAC system that performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String
UNLOCK_USER	Login name of the user in the iTAC system who performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String

Tab. 14-2 "Key Value Pair" (container): objectResultKeys

Material lot (objectType = 2)

Key	Explanation	IMSApi data type
LOCK_COUNT	Number of currently active locks for the object	☐ int
LOCK_DATE	Point in time the lock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
LOCK_INFORMATION	Lock information (reason for locking)	☐ String
LOCK_STATION	Station number of the work station in the iTAC system that performed the lock	☐ String
LOCK_USER	Login name of the user in the iTAC system who performed the lock	☐ String
LOCKGROUP_NAME	Lock group	☐ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_DATE_CODE	DateCode for the material lot	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
UNLOCK_DATE	Point in time the unlock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ DateMs
UNLOCK_INFORMATION	Lock information (reason for unlocking) Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String
UNLOCK_STATION	Station number of the work station in the iTAC system that performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String
UNLOCK_USER	Login name of the user in the iTAC system who performed the unlock Advice: In order for unlock information to be output, mode [1] or [2] must be passed for the lockState input parameter!	☐ String

Tab. 14-3 "Key Value Pair" (material lot): objectResultKeys

14.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type being searched for [0; 1; 2]: 0 = Serial number 1 = Container 2 = Material lot
objectNumber	Serial number, container number or material lot number Advice: This value is dependent on the selected object type (cf. parameter objectType)! If an empty string or [-1] is passed, the parameter is not evaluated; no restriction occurs!
objectDetail	Serial number position Advice: Field is only evaluated if serial number locks are to be taken into account (objectType = 0)!
lockGroupName	Lock group Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If an empty string or [-1] is passed, the parameter is not evaluated; no restriction occurs with respect to the lock group!
lockDateFrom	Start of the review period; specification of the total time in sec. since 01-01-1970, 0:00:00 Example: 1100000000 sec. (1.1 bn) correspond to 11-09-2004, 12:33:20 Special case: If the value [-1] is passed, the start of the period under review is not restricted! If the values for lockDateFrom and lockDateTo are identical, the review period is not evaluated and there are, thus, no time restrictions!
lockDateTo	End of the period under review; specification of the total time in sec. since 01-01-1970, 0:00:00 Example: 1100000000 sec. (1.1 bn) correspond to 11-09-2004, 12:33:20 Special case: If the value [-1] is passed, the end of the period under review is not restricted! If the values for lockDateFrom and lockDateTo are identical, the review period is not evaluated and there are, thus, no time restrictions!
lockState	Lock state [0; 1; 2] 0 = Only currently locked objects 1 = Only previously locked objects 2 = Both currently as well as previously locked objects Advice: Only if the value [1] or [2] is passed can unlock information be ascertained via the "Key Value Pair" structure (see the corresponding "UNLOCK" parameter there)!

Parameter name	Explanation
<code>maxRows</code>	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>
<code>objectResultKeys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the <code>object-Type</code>) can be found above in the corresponding "Key Value Pair" (xxx): <code>objectResultKeys</code> table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) <code>objectResultValues</code>!</i>

Tab. 14-4 Parameter (in) »lockGetLockedObjects«

Parameter (out)

Parameter name	Explanation
<code>objectResultValues</code>	Depending on parameter (in) <code>objectResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>objectResultKeys</code>!</i>

Tab. 14-5 Parameter (out) »lockGetLockedObjects«

14.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-198	Failure	Invalid value for "objectType"
-503	Failure	Container not found
-509	Failure	Material lot not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

14.2 lockGetLockGroups

14.2.1 Function description

This function enables lock group information to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., lock group state, lock group name or lock group class). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

The results are output in three parts: the lock groups, the lock classes and – only if the entry "Station" was selected as the ejection mode – the associated ejection stations. The three output structures can be put into context using the key `INDEX`.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int lockGetLockGroups
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] lockGroupFilter
    String key
    String value
  in String[] lockGroupResultKeys
  in String[] lockClassResultKeys
  in String[] lockOutResultKeys
  out String[] lockGroupResultValues
  out String[] lockClassResultValues
  out String[] lockOutResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `lockGroupFilter`, `lockGroupResultKeys`, `lockClassResultKeys` and `lockOutResultKeys` as well as (out) `lockGroupResultValues`, `lockClassResultValues` and `lockOutResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
LOCK_CLASS	Name of the lock class Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
LOCKGROUP_NAME	Name of the lock group Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
LOCKGROUP_STATE	State of the lock group [I; R; D; L]: I = initial R = released D = deleted L = invalid Advice: Multiple state values may be passed in the input (separated by ";")! Special case: If no value is passed, only released lock groups are considered by default (state = R)!	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 14-6 "Key Value Pair": lockGroupFilter

Definition of the output parameters for the found lock groups

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the lock groups can be put into context with the lock classes and – if existent – the assigned ejection stations (see the parameter by the same name in the other tables)!	☐ long
LOCK_OUT_MODE	Ejection mode [I; N; S]: I = immediately N = never S = station Advice: Lock groups that have the entry "Unknown value" as the ejection mode in the setup dialog (old data records) are output here with N!	☐ String
LOCKGROUP_NAME	Name of the lock class	☐ String
LOCKGROUP_STATE	State of the lock group [I; R; D; L]: I = initial R = released D = deleted L = invalid	☐ String

Tab. 14-7 "Key Value Pair": lockGroupResultKeys

Definition of the output parameters for the found **lock classes**

Key	Explanation	IMSApi data type
BOOKING_ALLOWED	"Booking allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be booked 1 = Serial number can continue to be booked	☐ int
CHANGE_ORDER_ALLOWED	"Modification order allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be processed in a modification order 1 = Serial number can continue to be processed in a modification order	☐ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: <i>The records of an array must be uniquely numbered; based on this number, the lock groups can be put into context with the lock classes and – if existent – the assigned ejection stations (see the parameter by the same name in the other tables)!</i>	☐ long
LOCK_CLASS	Name of the lock class	☐ String
LOCK_CLASS_DESC	Description of the lock class	☐ String
LOCK_LOT	"Automatically lock lot" mode [0; 1]: 0 = Lot is also locked automatically when a serial number is locked 1 = Lot is NOT locked when a serial serial number is locked	☐ int
MATERIAL_SETUP_ALLOWED	"Setup allowed" for locked containers [0; 1]: 0 = Container is NO LONGER allowed to be set up 1 = Container can be set up	☐ int
MERGE_ALLOWED	"Merging allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be merged 1 = Serial number can continue to be merged	☐ int
PACKAGING_ALLOWED	"Packing allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be packed 1 = Serial number can continue to be packed	☐ int
REPAIR_ORDER_ALLOWED	"Repair order allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be processed in a repair order 1 = Serial number can be processed in a repair order	☐ int
SCT_ALLOWED	"SCT allowed" for locked serial numbers [0; 1]: 0 = Serial number is NO LONGER allowed to be shipped using the SCT-Service 1 = Serial number can continue to be shipped using the SCT-Service	☐ int

Tab. 14-8 "Key Value Pair": *lockClassResultKeys*

Definition of output parameters for **ejection stations**

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: <i>The records of an array must be uniquely numbered; based on this number, the lock groups can be put into context with the lock classes and – if existent – the assigned ejection stations (see the parameter by the same name in the other tables)!</i>	☐ long
STATION_NUMBER	Station number of the work station in the iTAC system which was assigned as ejection station	☐ String

Tab. 14-9 "Key Value Pair": *lockOutResultKeys*

14.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables: sessionId Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
lockGroupFilter	"Key Value Pair" structure with the following variables: key List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": lockGroupFilter table. value Value list for the parameters passed by means of <code>key</code>. Advice: <i>Refer to the "Key Value Pair": lockGroupFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!</i>

Tab. 14-10 Parameter (in) »lockGetLockGroups«

Parameter (out)

Parameter name	Explanation
lockGroupResult-Values	Depending on parameter (in) <code>lockGroupResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>lockGroupResultKeys</code>!</i>
lockClassResult-Values	Depending on parameter (in) <code>lockClassResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>lockClassResultKeys</code>!</i>
lockOutResultValues	Depending on parameter (in) <code>lockOutResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>lockOutResultKeys</code>!</i>

Tab. 14-11 *Parameter (out) »lockGetLockGroups«*

14.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-281	Failure	Lock group not found
-1126	Failure	Lock group has an invalid state
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

14.3

lockObjects

14.3.1

Function description

This function is used for locking serial numbers, containers or material lots. If necessary, affected subobjects can be locked as well. If, e.g., a material lot is locked, you can select whether all thereby affected containers and serial numbers are to be locked as well.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int lockObjects
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int objectType
  in String lockGroupName
  in String lockInformation
  in DateMs lockDate
  in int lockDependencies
  in String[] objectUploadKeys
  in String[] objectUploadValues
  out String[] objectResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) objectUploadKeys and objectUploadValues as well as (out) objectResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Serial number (objectType = 0)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the locking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Container" and "Material lot")!	☐ int
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_DUPLICATE	Locking of the entire multiple panel [0; 1]: 0 = Multiple panel lock not active (default setting); this value is used for all "not equal" input [1]. 1 = Multiple panel lock active Advice: If the value [1] is passed, all serial numbers of the multiple panel are locked; for this purpose, the passed serial number (SERIAL_NUMBER) must be a valid reference serial number!	☐ int
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String
WORKORDER_NUMBER	Production order number Advice: If a serial number was taken over into the next integration level during mounting, the corresponding production order must be specified for identifying the desired integration level!	☐ String

Tab. 14-12 "Key Value Pair" (serial number): objectUploadKeys

Container (objectType = 1)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒ String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release) Advice: This can be used to influence the container state!	☐ String
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the locking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Serial number" and "Material lot")!	☐ int

Tab. 14-13 "Key Value Pair" (container): objectUploadKeys**Material lot** (objectType = 2)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the material lot!	☐ String
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the locking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Serial number" and "Container")!	☐ int
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☒ String
SUPPLIER_DATE_CODE	DateCode for the material lot	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String

Tab. 14-14 "Key Value Pair" (material lot): objectUploadKeys

14.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type to be locked [0; 1; 2]: 0 = Serial number 1 = Container 2 = Material lot
lockGroupName	Lock group
lockInformation	Lock information (reason for locking)
lockDate	Point in time the lock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
lockDependencies	Depending on objectType, defines which subordinate objects are also to be locked objectType = 0, locking of a serial number [0; 1]: 0 = Only the serial number 1 = Serial number and affected lot objectType = 1, locking of a container [0; 1; 2]: 0 = Only the container 1 = Container and all affected serial numbers 2 = Container, affected serial numbers and lot objectType = 2, locking of a material lot [0; 1; 2; 3]: 0 = Only the material lot 1 = Material lot and all affected containers 2 = Material lot, affected containers and serial numbers 3 = Material lot, affected containers, serial numbers and lot
objectUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the objectUploadValues parameter. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): objectUploadKeys table. Advice: Only the parameters passed here are processed by the function!
objectUploadValues	Value list for the parameters passed by means of objectUploadKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) objectUploadKeys! Refer to the corresponding "Key Value Pair" (xxx): objectUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 14-15 Parameter (in) »lockObjects«

Parameter (out)

Parameter name	Explanation
objectResultValues	The corresponding data is output here only in the event of an error – depending on the <code>objectUploadKeys</code> and <code>objectUploadValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>objectUploadKeys</code>!

Tab. 14-16 Parameter (out) »lockObjects«

14.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-164	Failure	Wrong parameter
-198	Failure	Invalid value for "objectType"

-271	Failure	Serial number is packed
-280	Failure	Serial number has already been locked with this lock group
-282	Failure	Serial number is not locked
-355	Failure	Production order is part of a work order cluster
-400	Failure	Lot not found
-503	Failure	Container not found
-509	Failure	Material lot not found
-518	Failure	Container state is invalid
-531	Failure	Supplier not found
-537	Failure	Container is already locked with the lock group
-538	Failure	Material lot is already locked with the lock group
-546	Failure	The material lot no. is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted

14.4

lockUnlockObjects

14.4.1

Function description

This function is used for unlocking serial numbers, containers or material lots. If necessary, affected subobjects can be unlocked as well. If, e.g., a material lot is unlocked, you can select whether all thereby affected containers and serial numbers are to be unlocked as well.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int lockUnlockObjects
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int objectType
  in String lockGroupName
  in String unlockInformation
  in int unlockCompleteGroup
  in DateMs unlockDate
  in int lockDependencies
  in String[] objectUploadKeys
  in String[] objectUploadValues
  out String[] objectResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) objectUploadKeys and objectUploadValues as well as (out) objectResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Serial number (objectType = 0)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the unlocking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Container" and "Material lot")!	☐ int
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_DUPLICATE	Unlocking of the entire multiple panel [0; 1]: 0 = Multiple panel unlocking not active (default setting); this value is used for all "not equal" input [1]. 1 = Multiple panel unlocking active Advice: If the value [1] is passed, all serial numbers of the multiple panel are unlocked; for this purpose, the passed serial number (SERIAL_NUMBER) must be a valid reference serial number!	☐ int
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String
WORKORDER_NUMBER	Production order number Advice: If a serial number was taken over into the next integration level during mounting, the corresponding production order must be specified for identifying the desired integration level!	☐ String

Tab. 14-17 "Key Value Pair" (serial number): objectUploadKeys

Container (objectType = 1)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒ String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release) Advice: This can be used to influence the container state!	☐ String
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the unlocking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Serial number" and "Material lot")!	☐ int

Tab. 14-18 "Key Value Pair" (container): objectUploadKeys**Material lot** (objectType = 2)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the material lot!	☐ String
REFERENCE	Reference number of the lock Advice: On the basis of this number, the data objects also affected by the locking can be put into context (see parameter of the same name in the "Key Value Pair" tables "Serial number" and "Container")!	☐ int
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☒ String
SUPPLIER_DATE_CODE	DateCode for the material lot	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String

Tab. 14-19 "Key Value Pair" (material lot): objectUploadKeys

14.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type [0; 1; 2] to be unlocked: 0 = Serial number 1 = Container 2 = Material lot
lockGroupName	Lock group Advice: Only if the value [1] is passed for the unlockComplete-Group parameter must a unique lock group absolutely be specified here! If the value [0] is passed there, an empty string or [-1] must be passed here!
unlockInformation	Lock information (reason for unlocking)
unlockComplete-Group	Determines whether all objects in the lock group (lockGroupName) are to be unlocked. If the group unlock is active, lots in addition to the objects "material", "container" and "serial number" are also taken into account and, if necessary, unlocked [0; 1]: 0 = No group unlocking 1 = All objects are unlocked Advice: If the value [1] is passed, all objects of the passed lock group (see lockGroupName) are unlocked. Only if the value [0] is passed must the parameters of the "Key Value Pair" structures be filled (see parameters (in) objectUploadKeys and objectUploadValues)!
unlockDate	Point in time the unlock operation was performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
lockDependencies	Depending on objectType, defines which subordinate objects are also to be unlocked objectType = 1, unlocking of a container [0; 1]: 0 = Only the container 1 = Container and all affected serial numbers objectType = 2, unlocking of a material lot [0; 1; 2]: 0 = Only the material lot 1 = Material lot and all affected containers 2 = Material lot, container and all serial numbers

Parameter name	Explanation
objectUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>objectUploadValues</code> parameter. A description of the possible parameters (depending on the <code>objectType</code>) can be found above in the corresponding "Key Value Pair" (xxx): objectUploadKeys table. Advice: Only the parameters passed here are processed by the function! Special case: If the value [1] is passed for <code>unlockCompleteGroup</code>, no input is necessary; all objects of the passed/found groups are unlocked!
objectUploadValues	Value list for the parameters passed by means of <code>objectUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: Refer to the corresponding "Key Value Pair" (xxx): objectUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)! Special case: If the value [1] is passed for <code>unlockCompleteGroup</code>, no input is necessary; all objects of the passed/found groups are unlocked!

Tab. 14-20 Parameter (in) »lockUnlockObjects«

Parameter (out)

Parameter name	Explanation
objectResultValues	The corresponding data is output here only in the event of an error – depending on the <code>objectUploadKeys</code> and <code>objectUploadValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>objectUploadKeys</code>!

Tab. 14-21 Parameter (out) »lockUnlockObjects«

14.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-164	Failure	Wrong parameter
-198	Failure	Invalid value for "objectType"
-271	Failure	Serial number is packed
-282	Failure	Serial number is not locked
-400	Failure	Lot not found
-503	Failure	Container not found
-509	Failure	Material lot not found
-518	Failure	Container state is invalid

-531	Failure	Supplier not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted

15 Group "MDA" (document management)

15.1 mdaActivateDocument

15.1.1 Function description

This function activates or deactivates one or more MDA items. If activated, the currently active version of the MDA item is deactivated automatically. If deactivated, a major version will then no longer have an active minor version. For unambiguous identification of an MDA item, either the name (MDA_NAME) together with the desired version number (MDA_VERSION) or the internal identification number (MDA_DOCUMENT_ID) must be specified.

The function can be used for almost all MDA types, e.g. "Document", "Link", "Work instruction", "Script", "Report". The only exception here is "Recipe"; the IMSApi function »mdaActivateRecipe« [▶▶▶] must be used in this case.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaActivateDocument
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] mdaActivateDocumentKeys
in String[] mdaActivateDocumentValues
out String[] mdaActivateDocumentResult
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mdaActivateDocumentKeys and mdaActivateDocumentValues as well as (out) mdaActivateDocumentResult). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Input values for documents to be activated/deactivated

Key	Explanation		IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
MDA_ACTIVE	"Activate/deactivate" the function mode [0; 1]: 0 = MDA item is deactivated 1 = MDA item is activated Advice: In mode [0], there is no active version in the system following execution of the function (default setting) (however, see also the MDA_VERSION key). In mode [1], the currently active version of the MDA item (if present) is deactivated automatically!	☒	int
MDA_DOCUMENT_ID	Internal identification number for the MDA item; the identification number can be ascertained using the IMSApi function »mdaGetDocuments« [▶▶▶]. Advice: Even though this key is not identified as a mandatory field, either the MDA_DOCUMENT_ID or the MDA_NAME in combination with the MDA_VERSION must be passed!	☐	long
MDA_NAME	Unique name of an MDA item Advice: Even though this key is not identified as a mandatory field, either the MDA_DOCUMENT_ID or the MDA_NAME in combination with the MDA_VERSION must be passed!	☐	String
MDA_VERSION	Version number of the MDA item together with major and minor version number -> e.g.: 1.3) Special case: If the key is not passed or is passed with [-1], ALL active MDA items are deactivated; this then applies generally to all major versions! Advice: Even though this key is not identified as a mandatory field, either the MDA_DOCUMENT_ID or the MDA_NAME in combination with the MDA_VERSION must be passed! Background: With MDA items, a distinction is made between major and minor versions; only one minor version of a major version can be active at any one time. If multiple major versions exist, then multiple minor versions of an MDA item can be active.	☐	String

Tab. 15-1 "Key Value Pair": mdaActivateDocumentKeys

15.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdaActivateDocumentKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>mdaActivateDocumentValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdaActivateDocumentKeys table. Advice: Only the parameters passed here are processed by the function!
mdaActivateDocumentValues	Values for the parameters selected by means of <code>mdaActivateDocumentKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>mdaActivateDocumentKeys</code>! Refer to the "Key Value Pair": mdaActivateDocumentKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 15-2 Parameter (in) »mdaActivateDocument«

Parameter (out)

Parameter name	Explanation
mdaActivateDocumentResult	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>mdaActivateDocumentKeys</code> and <code>mdaActivateDocumentValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mdaActivateDocumentKeys</code>!

Tab. 15-3 Parameter (out) »mdaActivateDocument«

15.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-717	Failure	Invalid value for "functionMode" [0; 1]
-732	Failure	Document not found
-733	Failure	Document is already activated
-734	Failure	Document is already deactivated
-735	Failure	Document is not unique
-736	Failure	Invalid value for MDA_ACTIVE [0;1]
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

15.2 mdaActivateRecipe

15.2.1 Function description

This function activates or deactivates a specified recipe version. In the case of activation, all other recipe versions are automatically deactivated. To identify a recipe uniquely, the recipe header structure and the recipe version to be activated must be passed.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaActivateRecipe
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int functionMode
in int recipeId
in int recipeVersionId
```

15.2.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
functionMode	Defines whether the recipe is activated or deactivated [0; 1]: 0 = Recipe is deactivated 1 = Recipe is activated
recipeId	Identification number of the recipe header structure Advice: This number must be ascertained directly from the database; details available upon request!
recipeVersionId	Identification number of the recipe version Advice: This number must be ascertained directly from the database; details available upon request!

Tab. 15-4 Parameter (in) »mdaActivateRecipe«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

15.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-704	Failure	Recipe not activated
-705	Failure	Recipe not deactivated
-717	Failure	Invalid value for "functionMode" [0; 1]
-2105	Failure	No TR or PM license present for the station

15.3

mdaCreateRecipe

15.3.1

Function description

This function creates a new recipe. If a recipe for the passed object already exists, only a new recipe version is created based on the header structure defined in the existing recipe.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*In addition to the successfully processed records, the arrays returned also contain **incorrectly** processed records with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

With this function it is possible to, among other things, directly assign the new recipe to the part; in this case the recipe can be used for all product versions of the part. If the assignment is to be more detailed, the recipe can also be assigned directly to a specific BOM version, a specific process version if applicable or a specific change index of a part. The possible combination of input parameters can be found in the following table:

Recipe for:	Part	BOM version	Process version	Change index	Work step	Order related work step
PART_NUMBER	X	X	X	X	X	-
BOM_VERSION	-	X	X	-	-	-
PROCESS_VERSION	-	-	X	-	-	-
REVISION_INDEX	-	-	-	X	-	-
STATION_NUMBER	[-1]	[-1]	[-1]	[-1]	-	-
MACHINE_GROUP_NUMBER	[-1]	[-1]	[-1]	[-1]	-	-
WORKPLAN_ALT	-	-	-	-	X	-
WORKPLAN_VERS	-	-	-	-	X	-
WORKSTEP_NUMBER	-	-	-	-	X	X
WORKORDER_NUMBER	-	[-1]	(X)	-	[-1]	X

Tab. 15-5 Possible combinations of the parameters of the `recipeFilters` array

An "X" identifies the input parameters that must absolutely be passed; optional parameters are identified with [-1]. "(X)" indicates input parameters which can be used as an alternative to "X" entries (for details, see the tables below). If an optional parameter is passed, it is checked against the mandatory parameters. If the data do not match, the function aborts with an error message.



Example

If a recipe is to be created for a specific process version, the parameters `PART_NUMBER`, `BOM_VERSION` and `PROCESS_VERSION` must be filled with concrete product data; alternatively, (`WORKORDER_NUMBER`) can be used in place of the three parameters because all three mandatory parameters can be derived from the work order. The parameters `STATION_NUMBER` or `MACHINE_GROUP_NUMBER` are optional.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaCreateRecipe
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String versionName
in String versionDesc
in int majorVersionChange
in boolean activate
in DateMs creationDate
in int objectType
in KeyValue[] recipeFilters
    String key
    String value
in String[] recipeUploadKeys
in String[] recipeUploadValues
out String[] recipeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `recipeFilters`, `recipeUploadKeys` and `recipeUploadValues` as well as parameter (out) `recipeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Recipe for **default object** (objectType = 0)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: See the advice under BOM_VERSION!	☐ String
PROCESS_VERSION	Process version from the iTAC system Advice: See the advice under BOM_VERSION!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐ String

Tab. 15-6 "Key Value Pair" (default object): recipeFilters

Recipe for **part number** (objectType = 1)

Key	Explanation	IMSApi data type
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
PART_NUMBER	Part number in the iTAC system	☒ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String

Tab. 15-7 "Key Value Pair" (part number): recipeFilters

Recipe for **BOM version** (objectType = 2)

Key	Explanation		IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒	int
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system	☒	String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
WORKORDER_NUMBER	Production order number Advice: If this key is used, BOM_VERSION and PART_NUMBER must have the appropriate entries!	☐	String

Tab. 15-8 "Key Value Pair" (BOM version): recipeFilters

Recipe for **process version** (objectType = 3)

Key	Explanation		IMSApi data type
BOM_VERSION	BOM version from the iTAC system Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐	int
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: See the advice under BOM_VERSION!	☐	String
PROCESS_VERSION	Process version from the iTAC system Advice: See the advice under BOM_VERSION!	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐	String

Tab. 15-9 "Key Value Pair" (process version): recipeFilters

Recipe for **change index** (objectType = 4)

Key	Explanation		IMSApi data type
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system	☒	String
REVISION_INDEX	Change index of the BOM from an external ERP system (SAP)	☒	String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String

Tab. 15-10 "Key Value Pair" (change index): recipeFilters

Recipe for **work step** (objectType = 5)

Key	Explanation		IMSApi data type
PART_NUMBER	Part number in the iTAC system	☒	String
WORKORDER_NUMBER	Production order number Advice: If this key is used, PART_NUMBER, WORKPLAN_ALT and WORKPLAN_VERS must have the appropriate entries!	☐	String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☒	String
WORKPLAN_VERS	Work plan version from the iTAC system	☒	int
WORKSTEP_NUMBER	Work step number in the iTAC system Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system can be passed directly instead of the work step number!	☒	int

Tab. 15-11 "Key Value Pair" (work step): recipeFilters

Recipe for **order-based work step** (objectType = 6)

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number	☒ String
WORKSTEP_NUMBER	Work step number in the iTAC system Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system can be passed directly instead of the work step number!	☒ int

Tab. 15-12 "Key Value Pair" (order-based work step): recipeFilters

Recipe parameters

Key	Explanation	IMSApi data type
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	☐ int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green). Advice: When entering, note that the passed value is permitted to have exactly one predecimal place and a maximum of two postdecimal places (valid entries are e.g.: "1", "0.3" or "0.65")!	☐ double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red). Advice: see also Advice for the key CP_GREEN!	☐ double
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green). Advice: see also Advice for the key CP_GREEN!	☐ double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red). Advice: see also Advice for the key CP_GREEN!	☐ double
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	☐ int
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!	☐ int

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
FUNCTION	Function name from the recipe	☐ String
FUNCTION_DESC	Explanation of the function	☐ String
FUNCTION_STEP	Function step from the recipe	☐ String
FUNCTION_STEP_DESC	Explanation of the function step	☐ String
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_LIMIT	Lower limit for the measurement step Advice: Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!	☐ String
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	☒ String
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ". " D = Decimal DE ", " Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐ String
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis) Advice: When entering, note that the passed value is permitted to have exactly one predecimal place and a maximum of two postdecimal places; additionally, the value must not be greater than "0.50" (values entries are e.g.: "0", "0.3" or "0.45")!	☐ double
NOMINAL	Nominal value Advice: Alternatively, the limit value can also be defined with the LOWER_LIMIT and UPPER_LIMIT parameters; the LOWER_LIMIT and UPPER_LIMIT parameters override this setting!	☐ String
RECIPE_VERSION_ID	Identification number of the recipe version	☐ long
REMARK	Supplementary comment	☐ String
TEST_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐ double
TOLERANCE	Tolerance value for measurement step Advice: Alternatively, the limit value can also be defined with the LOWER_LIMIT and UPPER_LIMIT parameters; the LOWER_LIMIT and UPPER_LIMIT parameters override this setting!	☐ String

Key	Explanation	IMSApi data type
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐ String
UNIT	Unit for the value passed	☐ String
UPPER_ACTION_LIMIT	Upper intervention limit	☐ String
UPPER_LIMIT	Upper limit for the measurement step Advice: <i>Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!</i>	☐ String
UPPER_SCRAP_LIMIT	Upper scrap limit	☐ String
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: <i>This parameter is an internal designator and does not currently have an official use!</i>	☐ String
WEIGHTING	Weighting for the PAA [0 - 9]	☐ int

Tab. 15-13 "Key Value Pair": *recipeUploadKeys*

15.3.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
versionName	Version name (NOT version number) Advice: <i>Versioning is automatically performed by the system – the official version number is displayed in the MDA module in the "Version" field; the <code>majorVersionChange</code> parameter can, however, also be used to influence the major version number!</i> Special case: If an empty string or [-1] is passed, the field is automatically filled by the system (e.g. "0.02")!
versionDesc	Version description Special case: If an empty string or [-1] is passed, the field is automatically filled by the system (e.g. "Created by Upload-Interface")!
majorVersion-Change	Definition of the major version number Advice: <i>Although versioning is automatically performed by the system, it is only possible here to influence the version number preceding the comma for NEW recipes.</i> Special case: If an empty string or the value [-1] is passed, the new recipe is used for the major version number "1"; if a recipe already exists, only the minor version is incremented by "1".

Parameter name	Explanation
activate	Automatically activates the recipe after it has been created [0; 1]: 1 Recipe is activated (TRUE) 0 = Recipe is NOT activated (FALSE) Advice: If the value [1] is passed, all other active versions are automatically deactivated!
creationDate	Time of creation for the recipe (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
objectType	Determines the object to which the recipe is to be assigned [0; 1; 2; 3; 4; 5; 6] 0 = Default object 1 = Part number 2 = BOM version 3 = Process version 4 = Change index 5 = Work step 6 = Order-based work step
recipeFilters	"Key Value Pair" structure with the following variables:
key	List of the object parameters to which the new recipe is to be assigned; the corresponding values are passed with the value parameter. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): recipeFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the corresponding "Key Value Pair" (xxx): recipeFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
recipeUploadKeys	List of the parameters that are to be transmitted by the function. A description of the possible parameters can be found above in the "Key Value Pair": recipeUploadKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) recipeUploadValues !
recipeUploadValues	Value list for the parameters passed by means of recipeUploadKeys ; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: Refer to the "Key Value Pair": recipeUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 15-14 Parameter (in) »mdaCreateRecipe«

Parameter (out)

Parameter name	Explanation
recipeResultValues	The corresponding data is output here – depending on the parameters (in) recipeUploadKeys and recipeUploadValues – both for successful processing and for incorrect processing. Advice: The data is output in the order in which it was entered by means of parameter (in) recipeUploadKeys !

Tab. 15-15 Parameter (out) »mdaCreateRecipe«

15.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-242	Failure	BOM version is invalid
-704	Failure	Recipe not activated
-709	Failure	Measurement step names are not unique
-716	Failure	Recipe not created
-718	Failure	Object type invalid

-719	Failure	Station or machine group not set
-2013	Failure	Value is outside of the validity range
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

15.4 mdaGetDocumentContent

15.4.1 Function description

This function allows the content of a document (MDA item) to be called up in the form of a byte array.



Advice

The required designator for the document (*documentId*) can be queried by means of the »mdaGetDocuments« [▶▶▶] API function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaGetDocumentContent
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in long documentId
out byte[] content
```

15.4.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
documentId	Unique designator (ID) for the document

Tab. 15-16 Parameter (in) »mdaGetDocumentContent«

Parameter (out)

Parameter name	Explanation
content	Content of the document as byte-array

Tab. 15-17 Parameter (out) »mdaGetDocumentContent«

15.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-2105	Failure	No TR or PM license present for the station

15.5

mdaGetDocuments

15.5.1

Function description

This function returns the assigned MDA items for the different data objects; these are e.g. part number, part group, work plan, work step, BOM, production means (equipment or carrier), container, work order, serial number, advice notices, and so on. The function allows the result set to be limited to certain file extensions or element states. Note that for the objects "work order" and "serial number", it is possible to specify here whether and which (sub-)objects which are referenced to MDA items should be output as well.

**Advice**

This API function is influenced by `documentLevel`, `documentShare` and `documentURL` server parameters; all details necessary for outputting the MDA items are defined there (output path, URL etc.). Further information on this topic can be found in the documentation for the ADM module (Server configuration -> View "Config-Framework")!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaGetDocuments
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] attributeFilters
    String key
    String value
in KeyValue[] dataTypeFilters
    String key
    String value
in String[] mdaResultKeys
out String[] mdaResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeFilters`, `dataTypeFilters` and `mdaResultKeys` as well as parameter (out) `mdaResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific data objects

Key	Explanation	IMSApi data type
ADVICE_ID	Unique designator (ID) for the advice	☐ long
BOM_VERSION	BOM version from the iTAC system Advice: This parameter can only be used in combination with PART_NUMBER, SERIAL_NUMBER or WORKORDER_NUMBER!	☐ int
BOM_VERSION_ERP	BOM version in the ERP system Advice: This parameter can only be used in combination with PART_NUMBER, SERIAL_NUMBER or WORKORDER_NUMBER!	☐ String
CONTAINER_NUMBER	Container number	☐ String
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carrier E = Equipment (work step) M = Cavity Advice: Note that the PM class must match the selected PM type (see key EQUIPMENT_TYPE)! Special case "downward compatibility": If the number of the production means is not unique and neither EQUIPMENT_CLASS nor EQUIPMENT_TYPE is passed, only entries from PM class E are taken into consideration!	☐ String
EQUIPMENT_INDEX	Individual designator for the production means Advice: Depending on the EQUIPMENT_TYPE, this designator then serves as an addition to the equipment number, cavity number and carrier number and only needs to be specified if the EQUIPMENT_NUMBER is not unique! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_NUMBER	Production means number in the iTAC system; alternatively, EQUIPMENT_NUMBER_EXT can also be used Advice: Depending on the EQUIPMENT_TYPE, this is the equipment number, cavity number, or carrier number! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
EQUIPMENT_NUMBER_EXT	External production means number (e.g., the designation of the manufacturer); alternatively, EQUIPMENT_NUMBER can also be used Advice: Depending on the EQUIPMENT_TYPE, this is the external equipment number, cavity number, or carrier number! Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Key	Explanation		IMSApi data type
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module! Special case "downward compatibility": If the number of the production means is not unique and neither EQUIPMENT_CLASS nor EQUIPMENT_TYPE is passed, only entries from PM class E are taken into consideration!	☐	String
FAILURE_TYPE_CODE	Failure type code	☐	String
INCOMING_GOODS_NUMBER	Number of the receiving Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
INCOMING_GOODS_POSITION	Position number in the receiving	☐	int
MDA_ROOT_KEY	Root directory to the referenced MDA items	☐	long
MDA_TARGET_TYPE	Referenced data objects for serial number or work order. Here, it is possible to specify whether and which MDA items of referenced (sub)objects are also to be output; the following values are possible: 100 = MDA items of the part 101 = MDA items of the work plan 102 = MDA items of the work step 104 = MDA items of the BOM version 107 = MDA items of the production order 111 = MDA items of the part group 122 = MDA items of the serial number Advice: This parameter is only evaluated for the SERIAL_NUMBER and work order WORKORDER_NUMBER data objects!	☐	int
PART_GROUP	Part group in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: This parameter can only be used in combination with STATION_NUMBER!	☐	int

Key	Explanation	IMSApi data type
PROCESS_VERSION	Process version from the iTAC system	☐ int
RESOURCE_NAME	Name of the resource Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ int
SERIAL_NUMBER	Serial number of the panel Advice: This parameter must be combined with the MDA_TARGET_TYPE parameter!	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☐ long
WORKORDER_ID	Internal identification number of the production order	☐ long
WORKORDER_NUMBER	Production order number Advice: If necessary, this parameter can be combined with the MDA_TARGET_TYPE parameter; MDA_TARGET_TYPE 122 must not be used here!	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project! This parameter can only be used in combination with WORKPLAN_NUMBER and WORKPLAN_VERS!	☐ String
WORKPLAN_NUMBER	Part number for the work plan Advice: This parameter can only be used in combination with WORKPLAN_ALT and WORKPLAN_VERS!	☐ String
WORKPLAN_VERS	Work plan version from the iTAC system Advice: This parameter can only be used in combination with WORKPLAN_ALT and WORKPLAN_NUMBER!	☐ int
WORKSTEP_AG	Number of the processing thread Advice: This parameter can only be used in combination with WORKSTEP_NUMBER (as a restriction)!	☐ int
WORKSTEP_AVO	Work step number in the ERP system (AVO) Advice: This parameter can only be used in combination with WORKPLAN_NUMBER and WORKSTEP_NUMBER (as a restriction)!	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system Advice: This parameter can only be used in combination with WORKPLAN_ALT, WORKPLAN_NUMBER and WORKPLAN_VERS!	☐ int

Tab. 15-18 "Key Value Pair": attributeFilters

Searching for specific data parameters

Key	Explanation	IMSApi data type
MDA_ACTIVE	Code for the active (current) version of an MDA item [0; 1]: 0 = inactive 1 = active	☐ int
MDA_DATA_TYPE	MDA type [2; 3; 4; 5; 6; 7; 8; 10; 12; 13; 14]: 2 = Recipe 3 = Document 4 = SAP link 5 = Link 6 = Text component 7 = Work instruction 8 = Script (document) 10 = Rule (document) 12 = Report (document) 13 = Script (custom function) 14 = DNC (document)	☐ int
MDA_FILE_EXTENSION	File extension of the MDA item (e.g., pdf , doc , etc.) Advice: When entering the search value, note that the MDA items are found for which the file name checked-in for this purpose ends with the character string specified here; entries are NOT case sensitive!	☐ String
MDA_FILE_PATTERN	File name with placeholders (e.g., ?_IT3322.*) Advice: The wildcard characters "**" and "?" can be used when entering the file-name pattern: "**" = Placeholder for an arbitrary number of characters "?" = Placeholder for exactly one character Entries are NOT case sensitive!	☐ String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	☐ int

Tab. 15-19 "Key Value Pair": *dataTypeFilters*

Definition of the output parameters for the found MDA items

Key	Explanation	IMSApi data type
DATE_CHANGED	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_CREATED	Date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
INFO	Individual additional information on the MDA item	☐ String
MDA_ACTIVE	Code for the active (current) version of an MDA item [0; 1]: 0 = inactive 1 = active Advice: Only one minor version of an MDA item can be active at any given time!	☐ int

Key	Explanation	IMSApi data type
MDA_DATA_TYPE	MDA type [2; 3; 4; 5; 6; 7; 8; 10;12; 13; 14]: 2 = Recipe 3 = Document 4 = SAP link 5 = Link 6 = Text component 7 = Work instruction 8 = Script (document) 10 = Rule (document) 12 = Report (document) 13 = Script (custom function) 14 = DNC (document)	int
MDA_DESC	Description of the MDA item	String
MDA_DOC_TYPE	- currently has no use -	String
MDA_DOCUMENT_ID	Unique designator (ID) for the attached document Advice: <i>The content of a document can be called up on the basis of this ID (see API function »mdaGetDocumentContent« [▶▶▶])!</i>	long
MDA_FILE_ID	Unique designator (ID) for the attached file	String
MDA_FILE_NAME	File name of the attached file	String
MDA_FILE_PATH	Path of the returned file on the server pool	String
MDA_NAME	Unique name of an MDA item	String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	int
MDA_URL_NAME	URL of the link	String
MDA_VERSION	Version number of the MDA item	String
MDA_VERSION_DESC	Supplementary description of the current version	String
MDA_VERSION_NAME	Alternative version name Advice: <i>Versioning is automatically performed by the system – the official version number is passed with the MDA_VERSION parameter!</i>	String
PROTECTED	Indicates whether the MDA item is protected against unauthorized changes [0; 1]: 0 = MDA item is NOT protected (FALSE) 1 = MDA item is protected (TRUE)	boolean

Tab. 15-20 "Key Value Pair": *mdaResultKeys*

15.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
attributeFilters	"Key Value Pair" structure with the following variables:
key	List of the data objects for which MDA items are to be output; the corresponding values are passed with the <code>value</code> parameter. A description of the possible data objects can be found above in the "Key Value Pair": attributeFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": attributeFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
dataTypeFilters	"Key Value Pair" structure with the following variables:
key	List of the data parameters for which MDA items are to be output; the corresponding values are passed with the <code>value</code> parameter. A description of the possible data objects can be found above in the "Key Value Pair": dataTypeFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": dataTypeFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mdaResultKeys	List of the MDA item parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdaResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mdaResultValues</code> !

Tab. 15-21 Parameter (in) »mdaGetDocuments«

Parameter (out)

Parameter name	Explanation
mdaResultValues	Depending on parameter (in) <code>mdaResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mdaResultKeys</code> !

Tab. 15-22 Parameter (out) »mdaGetDocuments«

15.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-122	Failure	Part group not found
-130	Failure	Work order not found
-195	Failure	Mount point not found
-337	Failure	Failure type not found
-718	Failure	Object type invalid
-730	Failure	Creation of the file failed
-1312	Failure	Production means type is invalid
-1321	Failure	Production means class is invalid
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2130	Failure	Test plan data not found

15.6

mdaGetRecipeData

15.6.1

Function description

This function returns the data of a specific recipe. The recipe can be ascertained either on the basis of the linked object (see "Key Value Pair" structure `recipeFilters` depending on the `objectType`) or directly via the unique identification number of the recipe version (`recipeVersionId`).

Furthermore, it is possible to perform an object-independent recipe search; recipes can be ascertained even if it is not known on which object type a recipe is dependent (for further information, see `objectType = 0`).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaGetRecipeData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int recipeVersionId
in String function
in String functionStep
in String measurementName
in double sequentialNumber
in String type
in String measurementType
in int objectType
in KeyValue[] recipeFilters
    String key
    String value
in String[] recipeResultKeys
out String[] recipeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `recipeFilters` and `recipeResultKeys` as well as parameter (out) `recipeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Recipe for **variable object** (objectType = 0)

Key	Explanation		IMSApi data type
BOM_VERSION	BOM version from the iTAC system Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed! Special case: If the value [-1] is passed, the newest valid version is automatically ascertained!	☐	int
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: See the advice under BOM_VERSION!	☐	String
PROCESS_VERSION	Process version from the iTAC system Advice: See the advice under BOM_VERSION! Special case: If the value [-1] is passed, the newest valid version is automatically ascertained!	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐	String

Tab. 15-23 "Key Value Pair" (variable object): recipeFilters

Recipe for **part number** (objectType = 1)

Key	Explanation		IMSApi data type
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system	☒	String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐	String

Tab. 15-24 "Key Value Pair" (part number): recipeFilters

Recipe for **BOM version** (objectType = 2)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
PART_NUMBER	Part number in the iTAC system	☒ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number Advice: If this key is used, BOM_VERSION and PART_NUMBER must have the appropriate entries!	☐ String

Tab. 15-25 "Key Value Pair" (BOM version): *recipeFilters*Recipe for **process version** (objectType = 3)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed! Special case: If the value [-1] is passed, the newest valid version is automatically ascertained!	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: See the advice under BOM_VERSION!	☐ String
PROCESS_VERSION	Process version from the iTAC system Advice: See the advice under BOM_VERSION! Special case: If the value [-1] is passed, the newest valid version is automatically ascertained!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the BOM_VERSION, PART_NUMBER and PROCESS_VERSION keys or the WORKORDER_NUMBER key must be passed!	☐ String

Tab. 15-26 "Key Value Pair" (process version): *recipeFilters*

Recipe for **change index** (objectType = 4)

Key	Explanation	IMSApi data type
MACHINE_GROUP_NUMBER	Number of the machine group Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String
PART_NUMBER	Part number in the iTAC system	☒ String
REVISION_INDEX	Change index of the BOM from an external ERP system (SAP) Advice: This variable identifies a product version and can be used as an alternative to bomVersion and processVersion!	☒ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the MACHINE_GROUP_NUMBER or the STATION_NUMBER must be passed!	☐ String

Tab. 15-27 "Key Value Pair" (change index): recipeFilters

Recipe for **work step** (objectType = 5)

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system	☒ String
WORKORDER_NUMBER	Production order number Advice: If this key is used, PART_NUMBER, WORKPLAN_ALT and WORKPLAN_VERS must have the appropriate entries!	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☒ String
WORKPLAN_VERS	Work plan version from the iTAC system	☒ int
WORKSTEP_NUMBER	Work step number in the iTAC system	☒ int

Tab. 15-28 "Key Value Pair" (work step): recipeFilters

Recipe for **order-based work step** (objectType = 6)

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number	☒ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☒ int

Tab. 15-29 "Key Value Pair" (order-based work step): recipeFilters

Definition of the output parameters for the found recipes

Key	Explanation		IMSApi data type
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	☐	int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green).	☐	double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green).	☐	double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	☐	int
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!	☐	int
FUNCTION	Function name from the recipe	☐	String
FUNCTION_DESC	Explanation of the function	☐	String
FUNCTION_STEP	Function step from the recipe	☐	String
FUNCTION_STEP_DESC	Explanation of the function step	☐	String
MAX_VALUE	Maximum limit value	☐	String
MAX_VALUE_EG_AUS	Upper scrap limit	☐	String
MAX_VALUE_EG_OK	Upper intervention limit	☐	String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	☐	String

Key	Explanation		IMSApi data type
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN "." D = Decimal DE "," Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐	String
MIN_VALUE	Minimum limit value	☐	String
MIN_VALUE_EG_AUS	Lower scrap limit	☐	String
MIN_VALUE_EG_OK	Lower intervention limit	☐	String
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis)	☐	double
NOMINAL	Nominal value	☐	String
REMARK	Supplementary comment	☐	String
TEST_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐	double
TOLERANCE	Tolerance value for measurement step	☐	String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐	String
UNIT	Unit for the value passed	☐	String
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: This parameter is an internal designator and does not currently have an official use!	☐	String
WEIGHTING	Weighting for the PAA [0 - 9]	☐	int

Tab. 15-30 "Key Value Pair": *recipeResultKeys*

15.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
recipeVersionId	Identification number of the recipe version Special case: If [-1] is passed, the recipe is ascertained on the basis of the <code>stationNumber</code> and <code>PART_NUMBER</code> parameters (see the "Key Value Pair" (xxx): recipeFilters table!
function	Name of the function group in the recipe Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)!
functionStep	Name of the function group step in the recipe Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)!
measurementName	Unique identifier of the recipe parameter (measurement step name) Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)!
sequentialNumber	Test step no. in the recipe (measurement step number) Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)!
type	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)!
measurementType	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ". " D = Decimal DE ", " Special case: If an empty string or [-1] is passed, the field is not evaluated (no restriction)! Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!

Parameter name	Explanation
objectType	Determines the object for which the recipe information is to be queried [0; 1; 2; 3; 4; 5; 6] 0 = Variable object 1 = Part number 2 = BOM version 3 = Process version 4 = Change index 5 = Work step 6 = Order-based work step Advice: An object-independent search is enabled in mode "0"; i.e., even if it is not known on which object type a recipe is dependent, recipes can be ascertained on the basis of the input in the "Key Value Pair" (variable object): recipeFilters table. If nothing is found on the lowest level (PROCESS_VERSION), the search automatically continues on the next levels (BOM_VERSION, PART_NUMBER)!
recipeFilters	"Key Value Pair" structure with the following variables:
key	List of the recipe parameters on the basis of which the output is to be filtered; the corresponding values are passed with the value parameter. A description of the possible container parameters can be found above in the "Key Value Pair" (xxx): recipeFilters table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair" (xxx): recipeFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
recipeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": recipeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) recipeResultValues!

Tab. 15-31 Parameter (in) »mdaGetRecipeData«

Parameter (out)

Parameter name	Explanation
recipeResultValues	Depending on parameter (in) recipeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) recipeResultKeys!

Tab. 15-32 Parameter (out) »mdaGetRecipeData«

15.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-164	Failure	Wrong parameter
-701	Failure	Recipe data not found
-702	Failure	Recipe header data not found
-706	Failure	Recipe not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

15.7

mdaGetRecipeHeaderAndVersion

15.7.1

Function description

This function returns the header structure and version information of a recipe. The recipe header information can be ascertained on the basis of the linked object (see `recipeHeaderFilter` "Key Value Pair" structure depending on the `objectType`). Furthermore, it is possible to perform an object-independent recipe search; recipes can be ascertained even if it is not known on which object type a recipe is dependent (for further information, see `objectType = 0`).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaGetRecipeHeaderAndVersion
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in KeyValue[] recipeHeaderFilter
    String key
    String value
in String[] recipeHeaderResultKeys
out String[] recipeHeaderResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `recipeHeaderFilter` and `recipeHeaderResultKeys` as well as (out) `recipeHeaderResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Recipe for **variable object** (objectType = 0)

Key	Explanation	IMSApi data type
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing Advice: If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!	int
MACHINE_GROUP_NUMBER	Number of the machine group	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	int
PART_GROUP	Part group in the iTAC system	String
PART_NUMBER	Part number in the iTAC system Advice: Alternatively, the part number can also be passed using the order number (see the WORKORDER_NUMBER key). If both the part number and the order number are passed, they are checked for plausibility with respect to each other!	String
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!	int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!	long
RECIPE_TITLE	Title of the recipe header structure	String
STATION_NUMBER	Station number of the work station in the iTAC system	String
WORKORDER_NUMBER	Production order number Advice: This key can be used as an alternative for determining the part number (PART_NUMBER). If both the part number and the order number are passed, they are checked for plausibility with respect to each other!	String

Tab. 15-33 "Key Value Pair" (variable object): *recipeHeaderFilter*

Recipe for **part number** (objectType = 1)

Key	Explanation	IMSApi data type
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐ int
PART_GROUP	Part group in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER or the PART_GROUP must be passed!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER or the PART_GROUP must be passed!</i>	☐ String
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: <i>If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!</i>	☐ int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: <i>If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!</i>	☐ long
RECIPE_TITLE	Title of the recipe header structure	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 15-34 "Key Value Pair" (part number): *recipeHeaderFilter*

Recipe for **BOM version** (objectType = 2)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒ int
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐ int
PART_GROUP	Part group in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER, the PART_GROUP or the WORKORDER_NUMBER must be passed!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER, the PART_GROUP or the WORKORDER_NUMBER must be passed!</i>	☐ String
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: <i>If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!</i>	☐ int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: <i>If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!</i>	☐ long
RECIPE_TITLE	Title of the recipe header structure	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number Advice: <i>This key can be used as an alternative for determining the part number (PART_NUMBER). If both the part number and the order number are passed, they are checked for plausibility with respect to each other!</i>	☐ String

Tab. 15-35 "Key Value Pair" (BOM version): *recipeHeaderFilter*

Recipe for **process version** (objectType = 3)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system Advice: <i>If the key is not passed, the work order number MUST be specified (see WORKORDER_NUMBER)!</i>	☐ int
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐ int
PART_GROUP	Part group in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER, the PART_GROUP or the WORKORDER_NUMBER must be passed!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER, the PART_GROUP or the WORKORDER_NUMBER must be passed!</i>	☐ String
PROCESS_VERSION	Process version from the iTAC system Advice: <i>If the key is not passed, the work order number MUST be specified (see WORKORDER_NUMBER)!</i>	☐ int
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: <i>If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!</i>	☐ int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: <i>If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!</i>	☐ long
RECIPE_TITLE	Title of the recipe header structure	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number Advice: <i>This key can be used as an alternative for determining the part number (PART_NUMBER), the BOM version (BOM_VERSION) and the process version (PROCESS_VERSION). If the work order number is passed in combination with one or more of the three named keys, a plausibility check is performed on the data!</i>	☐ String

Tab. 15-36 "Key Value Pair" (process version): *recipeHeaderFilter*

Recipe for **change index** (objectType = 4)

Key	Explanation	IMSApi data type
BOM_VERSION_ERP	BOM version in the ERP system ☒	String
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing ☐	int
MACHINE_GROUP_NUMBER	Number of the machine group ☐	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i> ☐	int
PART_GROUP	Part group in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER or the PART_GROUP must be passed!</i> ☐	String
PART_NUMBER	Part number in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the PART_NUMBER or the PART_GROUP must be passed!</i> ☐	String
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: <i>If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!</i> ☐	int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: <i>If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!</i> ☐	long
RECIPE_TITLE	Title of the recipe header structure ☐	String
STATION_NUMBER	Station number of the work station in the iTAC system ☐	String

Tab. 15-37 "Key Value Pair" (change index): *recipeHeaderFilter*

Recipe for **work step** (objectType = 5)

Key	Explanation	IMSApi data type
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing ☐	int
MACHINE_GROUP_NUMBER	Number of the machine group ☐	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i> ☐	int

Key	Explanation		IMSApi data type
PART_GROUP	Part group in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the PART_NUMBER, the PART_GROUP or the WORKORDER_NUMBER must be passed!	☐	String
PART_NUMBER	Part number in the iTAC system	☒	String
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!	☐	int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!	☐	long
RECIPE_TITLE	Title of the recipe header structure	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
WORKORDER_NUMBER	Production order number Advice: This key can be used as an alternative for determining the part number (PART_NUMBER). If both the part number and the order number are passed, they are checked for plausibility with respect to each other!	☐	String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☐	String
WORKPLAN_VERS	Work plan version in the iTAC system	☒	int
WORKSTEP_NUMBER	Work step number in the iTAC system	☒	int

Tab. 15-38 "Key Value Pair" (work step): recipeHeaderFilter

Recipe for **order-based work step** (objectType = 6)

Key	Explanation	IMSApi data type
DEPARTMENT_CODE	Department designator [31000; 31001; 31002] 31000 = Engineering 31001 = Pre-production 31002 = Manufacturing	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☐ int
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Only inactive recipes 1 = Only active recipes Advice: If the key is not passed or passed with [-1], there is no restriction; active and inactive recipes are then reviewed!	☐ int
RECIPE_STATE	Recipe state [30000; 30001; 30003; 30005; 30006] 30000 = Not initialized 30001 = Initialized 30003 = Released 30005 = In the release process 30006 = In process Advice: If the key is not passed or passed with [-1], there is no restriction; all recipes are then reviewed!	☐ long
RECIPE_TITLE	Title of the recipe header structure	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number	☒ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☒ int

Tab. 15-39 "Key Value Pair" (order-based work step): *recipeHeaderFilter*

Definition of the output parameters for the found recipes

Key	Explanation	IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String

Key	Explanation		IMSApi data type
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
MACHINE_GROUP_NUMBER	Number of the machine group	☐	String
MDA_CHECKED_OUT	MDA editing state (recipe) [0; 1]: 0 = Recipe is NOT checked out for editing 1 = Recipe is checked out for editing	☐	int
MDA_VERSION	Version number of the MDA item	☐	String
MDA_VERSION_DESC	Supplementary description of the current version	☐	String
MDA_VERSION_NAME	Alternative version name Advice: Versioning is automatically performed by the system – the official version number is passed with the <code>MDA_VERSION</code> parameter!	☐	String
OBJECT_TYPE	Display of the object type [0; 1; 2; 3; 4; 5; 6] 0 = Variable object 1 = Part number 2 = BOM 3 = Process version 4 = Change index 5 = Work step 6 = Order-based work step	☐	int
PART_DESC	Part description in the iTAC system	☐	String
PART_GROUP	Part group in the iTAC system	☐	String
PART_GROUP_DESC	Part group description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PROCESS_VERSION	Process version from the iTAC system	☐	int
RECIPE_ACTIVE	Recipe activation designator [0; 1]: 0 = Recipe is NOT active 1 = Recipe is active	☐	int
RECIPE_ID	Identification number of the recipe header structure	☐	long
RECIPE_TITLE	Title of the recipe header structure	☐	String
RECIPE_VERSION_CHANGE_DESC	Change description for recipe version	☐	String
RECIPE_VERSION_ID	Identification number of the recipe version	☐	long
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Key	Explanation	IMSApi data type
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	String
WORKPLAN_NUMBER	Part number for the work plan	String
WORKPLAN_VERS	Work plan version in the iTAC system	int
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_DESC	Description of the work step	String
WORKSTEP_NUMBER	Work step number in the iTAC system	int

Tab. 15-40 "Key Value Pair": *recipeHeaderResultKeys*

15.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object for which the recipe information is to be queried [0; 1; 2; 3; 4; 5; 6] 0 = Variable object 1 = Part number 2 = BOM version 3 = Process version 4 = Change index 5 = Work step 6 = Order-based work step Advice: An object-independent search is enabled in mode "0"; i.e., even if it is not known on which object type a recipe is dependent, recipe header information can be ascertained on the basis of the input in the "Key Value Pair" (variable object): recipeHeaderFilter table!
recipeHeaderFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): recipeHeaderFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the corresponding "Key Value Pair" (xxx): recipeHeaderFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
recipeHeaderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": recipeHeaderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) recipeHeaderResultValues!

Tab. 15-41 Parameter (in) »mdaGetRecipeHeaderAndVersion«

Parameter (out)

Parameter name	Explanation
recipeHeaderResultValues	Depending on parameter (in) recipeHeaderResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) recipeHeaderResultKeys!

Tab. 15-42 Parameter (out) »mdaGetRecipeHeaderAndVersion«

15.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-702	Failure	Recipe header data not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

15.8 mdaUploadCustomFunction

15.8.1 Function description

This function is used to upload the script of a customer method (CustomFunction) to the MDA module. The script is transferred as a byte array and stored as an MDA item under the MDA type "Script (Custom Function)".



Advice

During transfer, the script is checked to determine whether the function metadata is used correctly. If failures are found, the function outputs corresponding warnings – however, the script is still transferred!

Details on the metadata can be found in the function »customGetFunctionInfo« [▶▶▶] under »Definition of "metadata"« [▶▶▶]!

Background: A customer method can be used to map complex, customer-specific IMSApi workflows. With smart.FactoryAppStudio, iTAC offers two tools with which customer methods including the desired metadata can be created and directly integrated as a script in the MDA module. The system uses the metadata item `FUNCTION_TYPE` to determine whether the customer method is a general customer method (created using the "IMSApi FlowDesigner" tool) or a customer method specially developed for the OPCUA service ("OPCUA Studio" tool).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdaUploadCustomFunction
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] mdaValues
    String key
    String value
  in byte[] content
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `mdaValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Definition of the MDA item parameters

Key	Explanation		IMSApi data type
INFO	Individual additional information on the MDA item	☐	String
MDA_ACTIVE	Code for the active (current) version of an MDA item [0; 1]: 0 = inactive 1 = active	☐	int
MDA_DESC	Description of the MDA item	☒	String
MDA_FILE_NAME	Name of the attached file	☒	String
MDA_GROUP_PATH	Name and path of the MDA group	☐	String
MDA_MAJOR_VERSION	Major version number (X.x) Advice: If no or an invalid major version number is passed, a new number is created automatically (highest existing number + 1)!	☐	int
MDA_MINOR_VERSION	Minor version number (x.X) Advice: If no minor version number or an invalid minor version number is passed and the values for MDA_VERSION_DESC and MDA_VERSION_NAME have changed, a new number is created automatically (highest existing number + 1)!	☐	int
MDA_NAME	Unique name of the MDA item	☒	String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	☒	int
MDA_VERSION_DESC	Supplementary description of the current version	☒	String
MDA_VERSION_NAME	Alternative version name	☒	String

Tab. 15-43 "Key Value Pair": *mdaValues*

15.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdaValues	"Key Value Pair" structure with the following variables:
key	List of the MDA item parameters that are to be passed by the function; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": mdaValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the " Key Value Pair ": mdaValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
content	Content of the script as byte array

Tab. 15-44 Parameter (in) »mdaUploadCustomFunction«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

15.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1509	Warning	Function metadata in script is incorrect
1508	Warning	No valid function metadata found in script
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-724	Failure	MDA header non-unique
-728	Failure	Description not found for the MDA object
-729	Failure	Invalid value for "MDA state"
-731	Failure	MDA item -> Version is already in use
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

15.9 mdaUploadDocument

15.9.1 Function description

This function is used to create an MDA item by uploading a file (in the form of a byte array) and assigning it to an object. Objects for which an MDA item can be created in this way are:

- Part number
- Work plan
- Work step
- BOM
- Process version
- Station
- Work order
- Change index
- Part group
- Container
- Production means (equipment number, cavity number, or carrier number without index)
- Production means + index (equipment number, cavity number, or carrier number with index)
- Site (all)
- Serial number
- Process version / station
- Attributive test step
- Variable test step
- Resource

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	--

Function declaration

```
int mdaUploadDocument
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in int targetType
    in KeyValue[] attributeValues
        String key
        String value
    in KeyValue[] mdaValues
        String key
        String value
    in byte[] content
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `attributeValues` and `mdaValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐). The tables are described in the following sequence:

- Data object parameter for **Part** (targetType = 100)
- Data object parameter for **Work plan** (targetType = 101)
- Data object parameter for **Work step** (targetType = 102)
- Data object parameter for **BOM** (targetType = 104)
- Data object parameter for **Process version** (targetType = 105)
- Data object parameter for **Station** (targetType = 106)
- Data object parameter for **Work order** (targetType = 107)
- Data object parameter for **Change index** (targetType = 108)
- Data object parameter for **Part group** (targetType = 111)
- Data object parameter for **Container** (targetType = 115)
- Data object parameter for **Production means** (targetType = 116)
- Data object parameter for **Production means + index** (targetType = 117)
- Data object parameter for **Site (all)** (targetType = 118)
- Data object parameter for **Serial number** (targetType = 122)
- Data object parameter for **Process version / station** (targetType = 123)
- Data object parameter for **Attributive test step** (targetType = 124)
- Data object parameter for **Variable test step** (targetType = 125)
- Data object parameter for **Resource** (targetType = 126)
- Definition of the **MDA item parameters**

Part (targetType = 100)



Advice

To specify the part, one of the `PART_NUMBER`, `SERIAL_NUMBER` or `WORKORDER_NUMBER` parameters must be passed; if multiple parameters are passed, the first found parameter is used (sequence: Part - serial number - work order)!

Key	Explanation		IMSApi data type
PART_NUMBER	Part number in the iTAC system	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 15-45 "Key Value Pair" (part): *attributeValues*

Work plan (targetType = 101)

Key	Explanation	IMSApi data type
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	String
WORKPLAN_NUMBER	Part number for the work plan	String
WORKPLAN_VERS	Work plan version from the iTAC system	int

Tab. 15-46 "Key Value Pair" (work plan): attributeValues**Work step** (targetType = 102)**Advice**

To specify the work step, either the `WORKSTEP_AVO` or `WORKSTEP_NUMBER` parameter must be passed in addition to the mandatory fields!

Key	Explanation	IMSApi data type
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	String
WORKPLAN_NUMBER	Part number for the work plan	String
WORKPLAN_VERS	Work plan version from the iTAC system	int
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_NUMBER	Work step number in the iTAC system	int

Tab. 15-47 "Key Value Pair" (work step): attributeValues

BOM (targetType = 104)



Advice

To specify the BOM, one of the *PART_NUMBER*, *SERIAL_NUMBER* or *WORKORDER_NUMBER* parameters must be passed in addition to *BOM_VERSION*; if multiple parameters are passed, the first found parameter is used (sequence: Part group - part - serial number - work order)!

Key	Explanation		IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒	int
PART_NUMBER	Part number in the iTAC system	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 15-48 "Key Value Pair" (parts list): *attributeValues*

Process version (targetType = 105)



Advice

To specify the process version, one of the *PART_NUMBER*, *SERIAL_NUMBER* or *WORKORDER_NUMBER* parameters must be passed in addition to *BOM_VERSION*; if multiple parameters are passed, the first found parameter is used (sequence: serial number - order - part)!

If *SERIAL_NUMBER* and *WORKORDER_NUMBER* are not used, *PROCESS_VERSION* must be passed in addition to *PART_NUMBER* and *BOM_VERSION*.

Key	Explanation		IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒	int
PART_NUMBER	Part number in the iTAC system	☐	String
PROCESS_VERSION	Process version from the iTAC system	☐	int
SERIAL_NUMBER	Serial number of the panel	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 15-49 "Key Value Pair" (process version): *attributeValues*

Station (targetType = 106)

Key	Explanation		IMSApi data type
STATION_NUMBER	Station number of the work station in the iTAC system	☒	String

Tab. 15-50 "Key Value Pair" (station): *attributeValues*

Work order (targetType = 107)

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number	☒ String

Tab. 15-51 "Key Value Pair" (station): *attributeValues***Change index (targetType = 108)****Advice**

To specify the BOM on the basis of a change number from the ERP system, one of the *PART_NUMBER*, *SERIAL_NUMBER* or *WORKORDER_NUMBER* parameters must be passed in addition to the *BOM_VERSION_ERP*; if multiple parameters are passed, the first found parameter is used (sequence: Part - serial number - WO)!

Key	Explanation	IMSApi data type
BOM_VERSION_ERP	BOM version in the ERP system	☒ String
PART_NUMBER	Part number in the iTAC system	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 15-52 "Key Value Pair" (change index): *attributeValues***Part group (targetType = 111)****Advice**

To specify the part group, one of the *PART_GROUP*, *PART_NUMBER*, *SERIAL_NUMBER* or *WORKORDER_NUMBER* parameters must be passed; if multiple parameters are passed, the first found parameter is used (sequence: Part group - part - serial number - work order)!

Key	Explanation	IMSApi data type
PART_GROUP	Part group in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 15-53 "Key Value Pair" (part group): *attributeValues*

Container (targetType = 115)

Key	Explanation	IMSApi data type
CONTAINER_NUMBER	Container number	☒ String

Tab. 15-54 "Key Value Pair" (container): attributeValues

Production means (targetType = 116)



Advice

Special case "downward compatibility": If the number of the production means is not unique and neither `EQUIPMENT_CLASS` nor `EQUIPMENT_TYPE` is passed, only entries from PM class **E** are taken into consideration!

Key	Explanation	IMSApi data type
EQUIPMENT_CLASS	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavities Advice: Note that the PM class must match the selected PM type (see key <code>EQUIPMENT_TYPE</code>)!	☐ String
EQUIPMENT_NUMBER	Number of the production means in the iTAC system Advice: Depending on the <code>EQUIPMENT_TYPE</code> , this is the equipment number, cavity number, or carrier number!	☒ String
EQUIPMENT_TYPE	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String

Tab. 15-55 "Key Value Pair" (production means): attributeValues

Production means + index (targetType = 117)

**Advice**

Special case "downward compatibility": If the number of the production means is not unique and neither `EQUIPMENT_CLASS` nor `EQUIPMENT_TYPE` is passed, only entries from PM class **E** are taken into consideration!

Key	Explanation	IMSApi data type
<code>EQUIPMENT_CLASS</code>	PM class; classifies the production means into equipment, cavities, and carriers [A; C; E; M]: A = Equipment (asset) C = Carriers E = Equipment (work step) M = Cavities Advice: Note that the PM class must match the selected PM type (see key <code>EQUIPMENT_TYPE</code>)!	☐ String
<code>EQUIPMENT_INDEX</code>	Individual designator for the production means Advice: Depending on the <code>EQUIPMENT_TYPE</code> , this designator then serves as an addition to the equipment number, cavity number and carrier number specified (cf. <code>EQUIPMENT_NUMBER</code>)! Background: The index is used in combination with the equipment, cavity or carrier number for unique designation; i.e. the numbers themselves do not necessarily need to be unique here!	☒ String
<code>EQUIPMENT_NUMBER</code>	Number of the production means in the iTAC system Advice: Depending on the <code>EQUIPMENT_TYPE</code> , this is the equipment number, cavity number, or carrier number!	☒ String
<code>EQUIPMENT_TYPE</code>	Production means type [A; C; E; F; H; M; N; PU; PS; TE; TU]: A = Asset equipment (PM class = A) C = Standard carrier (PM class = C) E = Standard equipment (PM class = E) F = Feeder (PM class = E) H = Head (PM class = E) M = Cavity (PM class = M) N = Nozzle (PM class = E) PU = Production utilities (PM class = E) PS = Pre-setup (PM class = E) TE = Test equipment (PM class = E) TU = Test utilities (PM class = E) Advice: The production means type (also referred to as "PM type") specifies production means with respect to purpose and defines the PM class! The above-named entries are included by default; if necessary, further customer-specific types can be added in the ADM module!	☐ String

Tab. 15-56 "Key Value Pair" (production means + index): `attributeValues`

Site (all) (targetType = 118)

Key	Explanation	IMSApi data type
- No entries -		

Tab. 15-57 "Key Value Pair" (site (all)): *attributeValues*

Serial number (targetType = 122)

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel	☒ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 15-58 "Key Value Pair" (serial number): *attributeValues*

Process version / station (targetType = 123)

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☒ int
PART_NUMBER	Part number in the iTAC system	☐ String
PROCESS_VERSION	Process version from the iTAC system	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☒ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 15-59 "Key Value Pair" (process version / station): *attributeValues*

Attributive test step (targetType = 124)

Key	Explanation	IMSApi data type
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long

Tab. 15-60 "Key Value Pair" (attributive test step): *attributeValues*

Variable test step (targetType = 125)

Key	Explanation	IMSApi data type
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long

Tab. 15-61 "Key Value Pair" (variable test step): attributeValues**Resource** (targetType = 126)

Key	Explanation	IMSApi data type
RESOURCE_NAME	Name of the resource	☒ String

Tab. 15-62 "Key Value Pair" (resource): attributeValues**Definition of the MDA item parameters**

Key	Explanation	IMSApi data type
INFO	Individual additional information on the MDA item	☐ String
MDA_ACTIVE	Code for the active (current) version of an MDA item [0; 1]: 0 = inactive 1 = active	☐ int
MDA_DATA_TYPE	MDA type [3; 4; 5; 8; 14]: 3 = Document 4 = SAP link 5 = Link 8 = Script (document) 14 = DNC (document) Advice: MDA type 8 can only be used in combination with data object 118 "Site (all)"! Advice: The MDA type 14 can be used only in combination with the data objects 100 "Part", 106 "Station", 116 "Production means", 117 "Production means + index" as well as 118 "Site (all)"!	☒ int
MDA_DESC	Description of the MDA item	☒ String
MDA_FILE_NAME	Name of the attached file Advice: In the case of the MDA types 3 and 8, the file name must NOT be longer than 255 characters!	☐ String
MDA_GROUP_PATH	Name and path of the MDA group	☐ String
MDA_MAJOR_VERSION	Major version number (X.x) Advice: If no or an invalid major version number is passed, a new number is created automatically (highest existing number + 1)!	☐ int
MDA_MINOR_VERSION	Minor version number (x.X) Advice: If no minor version number or an invalid minor version number is passed and the values for MDA_VERSION_DESC and MDA_VERSION_NAME have changed, a new number is created automatically (highest existing number + 1)!	☐ int

Key	Explanation	IMSApi data type
MDA_NAME	Unique name of the MDA item	☒ String
MDA_STATUS	State of the MDA item [0; 1; 2; 3]: 0 = UNUSED 1 = IN_PROCESS 2 = RELEASED 3 = CANCELED	☒ int
MDA_URL_NAME	URL of the link Advice: In the case of the MDA type 5, the URL must NOT have more than 512 characters!	☐ String
MDA_VERSION_DESC	Supplementary description of the current version	☒ String
MDA_VERSION_NAME	Alternative version name	☒ String

Tab. 15-63 "Key Value Pair": *mdaValues*

15.9.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
targetType	Data object to which the MDA element is to be assigned: 100 = Part 101 = Work plan 102 = Work step 104 = BOM 105 = Process version 106 = Station 107 = Work order 108 = Change index 111 = Part group 115 = Container 116 = Production means 117 = Production means + index 118 = Site (all) 122 = Serial number 123 = Process version / station 124 = Attributive test step 125 = Variable test step 126 = Resource Advice: A description of the possible data object parameters can be found above in the corresponding "Key Value Pair" (xxx): attributeValues table!
attributeValues	"Key Value Pair" structure with the following variables:

Parameter name	Explanation
key	List of the data object parameters for which an MDA item is to be created; the corresponding values are passed with the <code>value</code> parameter. A description of the possible data objects can be found above in the corresponding "Key Value Pair" (xxx): attributeValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the corresponding "Key Value Pair" (xxx): attributeValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mdaValues	"Key Value Pair" structure with the following variables:
key	List of the MDA item parameters that are to be passed by the function; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdaValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdaValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
content	Content of the document as byte-array

Tab. 15-64 Parameter (in) »mdaUploadDocument«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

15.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-186	Failure	Product version is not valid
-187	Failure	Product version not found
-503	Failure	Container not found

-718	Failure	Object type invalid
-722	Failure	Invalid value for "targetType"
-724	Failure	MDA header non-unique
-726	Failure	File name not found for the MDA object
-727	Failure	URL not found for the MDA object
-728	Failure	Description not found for the MDA object
-729	Failure	Invalid value for "MDA state"
-731	Failure	MDA item -> Version is already in use
-1301	Failure	Production means not found
-1312	Failure	Production means type is invalid
-1321	Failure	Production means class is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16 Group "MDATA" (master data)

16.1 mdataBomVerify

16.1.1 Function description

This function can be used to compare workplace-related BOM information (e.g., from a placement program) with a specific BOM from the iTAC system (also an order BOM). The function is generally used at a pre-setup station of an SMD line. If there are discrepancies between placement program and order BOM, a list with all deviating positions is returned as the result.

When working with this function, it is necessary to differentiate between search and comparison criteria that are passed with the `bomUploadKeys` "Key Value Pair" structure. Search criteria (Key `PART_NUMBER` and/or `COMP_NAME`) are mandatory and are used to assign placement program data to BOM positions for the comparison. Comparison criteria (all other parameters in the structure with the exception of `COMMENT`) are optional and enable individual "restriction" of the comparison, since all of the used comparison criteria must be exactly defined in the parts list.

In addition to the successful comparison – the placement program matches the order BOM (output value (Errorcode) "0" – the program distinguishes between the four following error cases:

1. **Search criteria not found:** If the search criteria passed in the `bomUploadKeys` "Key Value Pair" structure (Key `PART_NUMBER` and/or `COMP_NAME`) are not found for the position, the entered data is returned with an appropriate comment (cf. Key `COMMENT`) in the structure. In this case, the function returns "-1102" as the output value (error code).
2. **Component values are different:** If the values of a component differ between the placement program and BOM position, the corrected data is returned with an appropriate comment (cf. Key `COMMENT`) in the structure. In this case, the function returns "-1102" as the output value (error code).
3. **Component not found in the BOM:** If individual components from the placement program data are not found in the order BOM, the entered data are returned with an appropriate comment (cf. Key `COMMENT`) in the structure. In this case, the function returns "-1102" as the output value (error code).
4. **Component not found in the placement program:** If there are components (BOM positions) in the order BOM that are not contained in the placement program, these missing positions are returned with an appropriate comment (cf. Key `COMMENT`) in structure. In this case, the function returns "-1102" as the output value (error code).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR PM license
--	-----------------------------------

Function declaration

```

int mdataBomVerify
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String workOrderNumber
  in String productNumber
  in String bareBoardNumber
  in String bomIndex
  in String bomInfo
  in int bomValidFrom
  in String bomVersionERP
  in int verifyCompNameBased
  in int createNewBom
  in String[] bomUploadKeys
  in String[] bomUploadValues
  out String[] bomResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `bomUploadKeys` and `bomUploadValues` as well as (out) `bomResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ALTERNATIVE	Designator for regular or alternative component in the parts list [0; 1]: 0 = regular component 1 = alternative component	☐	int
BOM_INFO	Individual BOM information (free text) Advice: <i>In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!</i>	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: <i>This key is only relevant for the comparison of specific semi-finished products (key word "product in product")!</i> Background: Normally, BOM_VARIANT is used in addition to BOM_VERSION_ERP in order to find an appropriate sub-BOM version. Merging a semifinished product with a BOM type deviating from the parent BOM is perfectly possible. For example, a semifinished product with BOM type U can be contained in a parent work order with BOM type F . BOMs without BOM type can be used universally. This means that BOMs without a BOM type are also returned for e.g. a queried semifinished product with BOM type F .	☐	String
BOM_VERSION_ERP	BOM version in the ERP system (semifinished product) Advice: <i>This key is only relevant for the comparison of specific semi-finished products (key word "product in product")!</i>	☐	String
COMMENT	Error comment automatically generated by the system (no input required here). Depending on the error case (see the four error case descriptions above), the following comments can be returned: Error case 1: "a required parameter is missing" Error case 2: "data for component are wrong" "data for mounting place are wrong" Error case 3: "mounting place not in the BOM" "component not in the BOM" Error case 4: "mounting place missing" (also for "component missing") Advice: <i>Which of the two comments is returned for error cases 2 and 3 is dependent on whether or not the mounting place check (see parameter verifyCompNameBased) is activated!</i>	☐	String
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: <i>If, a value of [1] is passed for verifyCompNameBased, the mounting location must absolutely be passed, as it is interpreted in combination with the part number (PART_NUMBER) as a search criterion (OR link)!</i>	☒	String
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.)	☐	String
ERP_POS_NUMBER	Work step number of the processing process in the ERP system for the component	☐	String
MACHINE_GROUP_NUMBER	Number of the machine group on which the placement is carried out	☐	String

Key	Explanation		IMSApi data type
PART_DESC	Part description of the component in the iTAC system	☐	String
PART_GROUP	Part group in the iTAC system	☐	String
PART_NUMBER	Part number of the component in the iTAC system Advice: Unlike the other parameters, input of the part no. as a search criterion is absolutely required! If the <code>verifyCompName-Based</code> input parameter is passed with [1], the mounting place (COMP_NAME) is also interpreted as a search criterion (OR link).	☒	String
POSITION_TYPE	Definition of the component type [11500 - 11502]: 11500 = Raw material 11501 = Semi-finished 11502 = Finished product	☐	int
PROCESS_INFO_TXT	Process information	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
PROCESS_TYPE	Selection of a use type [10300 - 10304]: 10300 = placement 10301 = visual inspection 10302 = component preparation 10303 = provision by customer	☐	int
PRODUCT_FLAG	Designator for raw material / product[0; 1]: 0 = Raw material (component) 1 = Product (unit) Advice: This value corresponds to the product designator from the order BOM (cf. "Product" field for the BOM components in the BOM module). Note here that this value does not necessarily need to match the product designator from the part master!	☐	int
QUANTITY	Quantity of the part	☐	double
SETUP_FLAG	Designator for the setup requirement [0; 1]: 0 = No setup required 1 = Setup required	☐	int
UNIT	Unit for the value passed	☐	String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐	int

Tab. 16-1 "Key Value Pair": `bomUploadKeys`

16.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: If the value [-1] is passed, this parameter is not evaluated!
workOrderNumber	Number of a production order Special case: If the value [-1] is passed, this parameter is not evaluated; in this case, the part (productNumber) must absolutely be passed!
productNumber	Part number of the finished product in the iTAC system Special case: If the value [-1] is passed, this parameter is not evaluated; in this case, the work order (workOrderNumber) must absolutely be passed!
bareBoardNumber	Part no. of the fundamental bearer, e.g., number of the "bare" printed circuit board Special case: If the value [-1] is passed, this parameter is not evaluated!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Special case: If the value [-1] is passed, this parameter is not evaluated! Advice: In the BOM module this parameter is displayed in the BOM header table in the "Change No." field!
bomInfo	Individual BOM information (free text) Special case: If the value [-1] is passed, this parameter is not evaluated! Advice: In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!
bomValidFrom	Point in time at which the BOM becomes valid Special case: If the value [-1] is passed, this parameter is not evaluated!
bomVersionERP	BOM version in the ERP system Special case: If the value [-1] is passed, this parameter is not evaluated! Advice: In the BOM module this parameter is displayed in the BOM header table in the "Change index" field!

Parameter name	Explanation
verifyCompName-Based	Check of the mounting places/components requiring setup [0; 1; 2; 3]: 0 = No check 1 = Only the mounting places are checked 2 = Only the components requiring setup are checked 3 = Only those components requiring setup are checked according to their mounting place Advice: If the value [1] is passed, the mounting place (COMP_NAME) is interpreted as a search criterion in the bomUploadKeys "Key Value Pair" structure in addition to the part no. (PART_NUMBER)!
createNewBom	BOM creation [0; 1]: 0 = BOM is checked but not created; all parameters passed in the bomUploadKeys "Key Value Pair" structure are interpreted as search or comparison criterion 1 = BOM is created on the basis of the parameters passed in the bomUploadKeys "Key Value Pair" structure Advice: This parameter is intended for future use; currently, only mode 0 is used!
bomUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the bomUploadValues parameter. A description of the possible parameters (dependent on objectType) can be found above in the corresponding table for attribute category "Key Value Pair": bomUploadKeys. Advice: Only the parameters passed here are processed by the function!
bomUploadValues	Values for the parameters selected by means of bomUploadKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) bomUploadKeys! Refer to the corresponding table for attribute category "Key Value Pair": "bomUploadKeys" to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 16-2 Parameter (in) »mdataBomVerify«

Parameter (out)

Parameter name	Explanation
bomResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) bomUploadKeys and bomUploadValues. Advice: The data is output in the order in which it was entered by means of parameter (in) bomUploadKeys!

Tab. 16-3 Parameter (out) »mdataBomVerify«

16.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-159	Failure	Invalid value for "verifyCompNameBased" [0; 1]
-160	Failure	Invalid value for "createNewBom" [0; 1]
-242	Failure	BOM version is invalid
-1102	Failure	BOM comparison failed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2116	Failure	BOM -> Type is invalid

16.2

mdataChangeWorkorderBomData

16.2.1

Function description

This function makes it possible to subsequently adapt bills-of-material that are stored for a work order; a function call can be used to change one or more BOM positions (e.g., the quantity specification). Furthermore, entirely new or alternative BOM positions (components) can also be added. The following function modes are currently available via the `FUNC_MODE` key:

- Update a BOM position (`FUNC_MODE = 0`)
- Add a BOM position (`FUNC_MODE = 1`)
- Add an alternative component (`FUNC_MODE = 2`)
- Delete a BOM position (`FUNC_MODE = 3`)

On the basis of this function, bills-of-material in the current production process can be changed, which considerably increases the flexibility in the implementation of customer-specific processes.

For understanding: If the bill-of-material originally assigned during work order creation is adapted for the work order, one speaks of an order-based bill-of-material; the original bill-of-material always remains unchanged in this case.

**Advice**

*The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = **-1108**) in the event of one or more faulty records!*

Application types

For all application types in which an existing BOM position must be specified – this affects `FUNC_MODE`: **0**, **2** and **3** – the following procedure applies for the **ascertainment of the desired BOM position**:

- Using the `BOM_POSITION_INDEX` key, referencing to the existing BOM position is performed. If there is not a match, the process is canceled with an error message (error code = **-1111**).

Advice: *The position number can be ascertained with »mdataGetBomData« [▶▶▶] (there, see key `BOM_POSITION_INDEX` in structure `bomDataResultKeys`)!*

- If the `BOM_POSITION_INDEX` key is not specified, the `COMP_NAME` and `COMP_TYPE` keys are used to determine the BOM position; this is then, however, always the record with the main alternative "0" ("Alt level" in the BOM module). If there is not a match, the process is canceled with an error message (error code = **-1111**).
- If the `BOM_POSITION_INDEX`, `COMP_NAME` and `COMP_TYPE` keys are passed together, they are checked against one another. If there is not a match, the process is canceled with a corresponding error message (error code = **-1109**, **-1110** or **-1111**).

The following applies for the **Update a BOM position** application type
(FUNC_MODE = 0):

- This application type involves the adaptation of a specific BOM position for the work order. An already-existing, order-based variant can be updated (again) here. If there is not yet an order-based variant, it is created here for the first time.
- To identify the desired BOM position, see the standard process **Ascertaining the desired BOM position** described above.
- The QUANTITY, ALTERNATIVE, COMP_PART_NUMBER and – optionally with semifinished products – COMP_BOM_VERSION and COMP_BOM_VERSION_ERP keys can be changed. Process data can NOT be adapted.

The following applies for the **Add a BOM position** application type
(FUNC_MODE = 1):

- A BOM position added in this way applies only for the order-based variant; the original BOM list is - for technological reasons - expanded with this new item as well. But because it is created with the quantity "0", it has virtually no function.
- To create a new BOM position, the COMP_NAME and COMP_TYPE keys must be passed; the mounting place specified according to COMP_NAME must not yet exist.
- The BOM_POSITION_INDEX key must not be used; it is generated automatically by the system during creation.
- In addition to the QUANTITY and COMP_PART_NUMBER keys, process data must ALSO be passed in this application type; i.e., the MACHINE_GROUP_NUMBER, PROCESS_LAYER, PROCESS_TYPE and SETUP_FLAG keys must absolutely be used.

The following applies for the **Add an alternative component** application type
(FUNC_MODE = 2):

- To identify the desired BOM position, see the standard process **Ascertaining the desired BOM position** described above.
- The part number of the new, alternative component is passed using the COMP_PART_NUMBER key; this must exist in the iTAC master data.
- The sequential number for alternative components is passed using the ALTERNATIVE key ("Alt level" field in the BOM module); it determines the probability of use; if the key is not passed, the highest existing value "+1" is used by default.

The following applies for the **Delete a BOM position** application type
(FUNC_MODE = 3):

- The only BOM positions that can be deleted are those that either exist in an order-based variant (the original BOM position is then valid) or were inserted as new BOM position (see above FUNC_MODE = 1).
Advice: A BOM position contained in the original BOM list can NOT be deleted; it can, however, be rendered virtually functionless by creating an order-based variant with the quantity "0" (see above FUNC_MODE = 0)!
- To identify the desired BOM position, see the standard process **Ascertaining the desired BOM position** described above.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataChangeWorkorderBomData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workorderNumber
in String[] changeWorkorderBomUploadKeys
in String[] changeWorkorderBomUploadValues
out String[] changeWorkorderBomResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `changeWorkorderBomUploadKeys` and `changeWorkorderBomUploadValues` as well as (out) `changeWorkorderBomResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ALTERNATIVE	Sequential number for alternative components ("Alt level" field in the BOM module); if there are multiple alternatives for a BOM component, this can be used to define the order of use (prioritization). Advice: BOM components with the value [0] have the highest probability of use; if the key is not passed, the highest existing value "+1" is used by default!	☐ int
BOM_POSITION_INDEX	Position number of the BOM component Advice: The position number can be ascertained with »mdataGet-BomData« [▶▶▶] (there, see key BOM_POSITION_INDEX in structure bomDataResultKeys)! Depending on the function mode, this key has various dependencies to the COMP_NAME and COMP_TYPE keys (refer back to item "Application types" for further information).	☐ long
BOM_VERSION	BOM version of the component that is to be adapted Advice: This field is only relevant if the BOM position itself is a unit with its own bill-of-material (key word "product in product")!	☐ int

Key	Explanation		IMSApi data type
CHANGE_COMMENT	Individual change comment	☐	String
COMP_NAME	Mounting place conformant with the iTAC BOM Depending on the function mode, this key has various dependencies to the BOM_POSITION_INDEX and COMP_TYPE keys (refer back to item "Application types" for further information).	☐	String
COMP_PART_NUMBER	Part number in the iTAC system (component) Advice: Depending on the application type, this may be the part number of a component that is to be changed – this must then be known in the bill-of-material – or it is a new part number; here, it must exist in at least the iTAC master data!	☐	String
COMP_TYPE	Component type: 10200 = Electrical component 10201 = Mechanical component 10202 = Printed circuit board (component side) 10206 = Printed circuit board (soldering side) 10207 = Documents/drawings Background: Component types are process-related groups that are permanently specified in the system. Depending on the function mode, this key has various dependencies to the BOM_POSITION_INDEX and COMP_NAME keys (refer back to item "Application types" for further information).	☐	long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -1108)!	☐	int
FUNC_MODE	Function mode "BOM change" [0; 1; 2; 3]: 0 = Update a BOM position 1 = Add a BOM position 2 = Add an alternative component 3 = Delete a BOM position Advice on [3]: With the deletion of an order-based BOM position – assuming it was not an additionally added BOM position – the BOM position from the original BOM list is valid again!	☒	int
MACHINE_GROUP_NUMBER	Number of the machine group on which the production process is to be performed according to PROCESS_TYPE Advice: This key is only relevant if a new BOM position is to be added (FUNC_MODE = 1)!	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: This key is only relevant if a new BOM position is to be added (FUNC_MODE = 1)!	☐	int

Key	Explanation	IMSApi data type
PROCESS_TYPE	Selection of a use type [10300 - 10304]: 10300 = placement 10301 = visual inspection 10302 = component preparation 10303 = provision by customer 10304 = provision to external Advice: This key is only relevant if a new BOM position is to be added (FUNC_MODE = 1)!	☐ int
QUANTITY	Quantity of the part (if necessary, also as decimal value)	☐ double
SETUP_FLAG	Designator for the setup requirement [0; 1]: 0 = No setup required 1 = Setup required Advice: This key is only relevant if a new BOM position is to be added (FUNC_MODE = 1)!	☐ int

Tab. 16-4 "Key Value Pair": *changeWorkorderBomUploadKeys*

16.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workorderNumber	Number of the production order (iTAC order number) to which the BOM is to be adapted
changeWorkorder-BomUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <i>changeWorkorder-BomUploadValues</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": change-WorkorderBomUploadKeys table. Advice: Only the parameters passed here are processed by the function!
changeWorkorder-BomUploadValues	Values for the parameters selected by means of <i>changeWorkorderBomUploadKeys</i> ; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <i>changeWorkorderBomUploadKeys</i> ! Refer to the "Key Value Pair": changeWorkorderBomUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 16-5 Parameter (in) »mdataChangeWorkorderBomData«

Parameter (out)

Parameter name	Explanation
changeWorkorder-BomResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>changeWorkorderBomUploadKeys</code> and <code>changeWorkorderBomUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>changeWorkorderBomUploadKeys</code> !

Tab. 16-6 Parameter (out) »mdataChangeWorkorderBomData«

16.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found

-168	Failure	Invalid value for the function mode (control flag)
-1101	Failure	BOM not found
-1107	Failure	BOM position not processed
-1108	Failure	Not all BOM positions processed
-1109	Failure	BOM position -> Component name (mounting place) invalid
-1110	Failure	BOM position -> Component type invalid
-1111	Failure	BOM position not found
-1112	Failure	BOM position not created
-1113	Failure	BOM position could not be updated
-1127	Failure	Alternative BOM position already exists
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.3

mdataChangeWorkorderData

16.3.1

Function description

Depending on the current order state, this function can be used to make certain changes to the header structures of a production order.



Advice

This API function is influenced by site parameter code no. 3011 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! Depending on the current state of the work order that is to be changed, only certain order data can be changed (see function matrix)!

If a parameter is marked with an **x**, it can – depending on the current order state – be changed. The order state is defined by a fixed letter code; the following states apply: **R** = planned, **N** = not released, **F** = opened (released), **S** = started, **P** = paused, **E** = finished and **X** = canceled (deleted)

Key	current order state						
	R	N	F	S	P	E	X
APS_ORDER_FIXATION	x	x	x				
CUSTOMER_CODE	x						
CUSTOMER_WORKORDER_NUMBER	x						
INDEX_*	x	x	x	x	x	x	
INFO_NUMBER_*	x	x	x	x	x	x	
INFO_TXT_*	x	x	x	x	x	x	
PRIORITY_CODE	x	x	x				
PRODUCTION_CONTROLLER	x	x	x	x	x	x	
RETROGRAD	x	x	x				
WORKORDER_COMMENT	x	x	x	x	x	x	
WORKORDER_DELIVER_DATE_PLANNED	x						
WORKORDER_DESC	x	x	x				
WORKORDER_EARLIEST_DELIVER_DATE	x						
WORKORDER_EARLIEST_START_DATE	x						
WORKORDER_MAX_QUANTITY	x	x	x	x	x		
WORKORDER_QUANTITY	x	x	x	x	x		
WORKORDER_START_DATE_PLANNED	x						
WORKORDER_STATE	x ¹	x ²	x ³	x ⁴	x ⁵		

Tab. 16-7 Function matrix *mdataChangeWorkorderData*

- 1 - State change to **F**, **N** and **X** possible
- 2 - State change to **F** possible
- 3 - State change to **R**, **N**, **S** and **X** possible
- 4 - State change to **P** and **E** possible
- 5 - State change to **S** and **E** possible

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataChangeWorkorderData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue workorderChangeData
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `workorderChangeData`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	☐	int
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐	String
CUSTOMER_WORKORDER_NUMBER	Customer-specific number for the production order Advice: Even though this key is not identified as a mandatory field, either the CUSTOMER_WORKORDER_NUMBER or the WORKORDER_NUMBER must be passed!	☐	String
INDEX_1	Index field 1 (alphanumeric) for individual information	☐	String
INDEX_2	Index field 2 (alphanumeric) for individual information	☐	String

Key	Explanation	IMSApi data type
INDEX_3	Index field 3 (alphanumeric) for individual information	☐ String
INDEX_4	Index field 4 (alphanumeric) for individual information	☐ String
INDEX_5	Index field 5 (alphanumeric) for individual information	☐ String
INFO_NUMBER_1	Numerical additional information 1	☐ double
INFO_NUMBER_2	Numerical additional information 2	☐ double
INFO_NUMBER_3	Numerical additional information 3	☐ double
INFO_NUMBER_4	Numerical additional information 4	☐ double
INFO_TXT_1	Alphanumeric additional information 1	☐ String
INFO_TXT_2	Alphanumeric additional information 2	☐ String
INFO_TXT_3	Alphanumeric additional information 3	☐ String
INFO_TXT_4	Alphanumeric additional information 4	☐ String
INFO_TXT_5	Alphanumeric additional information 5	☐ String
INFO_TXT_6	Alphanumeric additional information 6	☐ String
PRIORITY_CODE	APS service - unique designator for the work order priority	☐ String
PRODUCTION_CONTROLLER	Production controller code Advice: <i>This is a customer-specific grouping that is assigned to the part master!</i>	☐ String
RETROGRAD	Material removal mode for the work order. The modes that are available for selection are dependent on site parameter code no. 3011 [▶▶▶]; there, the degrees of freedom for manual processing are defined (see also advice below). A distinction is made between the following modes [0; 1; 2; 3]: 0 = No retrograde booking: the material for the production order is considered to be available for allocation during the entire manufacturing process depending on the actual removal from storage 1 = Complete quantity on first WS: the complete order quantity is booked on the first work step which is obligated to supply confirmation. Thus, the availability for allocation is not dependent on the time of actual removal from storage 2 = RM quantity on the first WS: only the actual quantity resulting from the feedback from the first work step is completely booked. Thus, the availability for allocation is not dependent on the time of actual removal from storage 3 = RM quantity at assigned WS: only the actual quantity resulting from the feedback from the individual work steps is booked. Thus, the availability for allocation is not dependent on the time of actual removal from storage	☐ int
WORKORDER_DELIVER_DATE_PLANNED	Planned delivery date for the work order	☐ DateMs
WORKORDER_COMMENT	WO note	☐ String
WORKORDER_DESC	Order description	☐ String

Key	Explanation		IMSApi data type
WORKORDER_EARLIEST_DELIVER_DATE	Earliest delivery date for the work order	☐	DateMs
WORKORDER_EARLIEST_START_DATE	Earliest start date for the work order (start of production)	☐	DateMs
WORKORDER_MAX_QUANTITY	Maximum order quantity	☐	int
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the CUSTOMER_WORKORDER_NUMBER or the WORKORDER_NUMBER must be passed!	☐	String
WORKORDER_QUANTITY	Order quantity Advice: The order quantity can be both increased and reduced; a reduction is, however, possible only down to the quantity of the already assigned serial numbers!	☐	int
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐	long
WORKORDER_START_DATE_PLANNED	Planned start date for the work order (start of production)	☐	DateMs
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐	String

Tab. 16-8 "Key Value Pair": workorderChangeData

16.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workorderChange-Data	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": workorderChangeData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workorderChangeData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 16-9 Parameter (in) »mdataChangeWorkorderData«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

16.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-137	Failure	No work order could be created
		Advice: The value -137 is output if a work order cannot be changed from state R (planned) to state F (opened)!
-138	Failure	Order state is invalid
-2014	Failure	Customer not found
-2060	Failure	Changing the order quantity is not allowed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.4

mdataChangeWorkOrderWorkStepState

16.4.1

Function description

This function is used to change the state of a work step. The requirement for this is that the associated work order is no longer in the "planned" state, but is at least released ("opened" work order state) or has progressed even further along the production process.



Advice

*This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module! The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Use of "Work step state"

The matrix below shows the possible transitions from work step state to work step state. An **x** indicates the possible state transitions.

Current work step state	Possible new work step state					
	P	R	S	B	L	F
P = planned	---	x			x	
R = released	x	---	x		x	x
S = started			---	x		x
B = interrupted			x	---	x	x
L = locked		x			---	
F = finished						---

Tab. 16-10 Function matrix "State change"

Apart from the possibility of changing the work step state manually using an IMSApi function, there are also a number of conditions which change the state automatically (depending on the system configuration); this particularly concerns the interaction with the associated work order. Always note the following **basic rules** with regard to work step state:

- The work step state can only be changed when the work order has been released. If the work order has not yet been released (i.e. only planned), then the work step will not have a state.
- If the work order is activated (and therefore changed to the "started" work order state), the work step state is also automatically changed to "started". If the work order is activated on only one station, this applies only to the work step concerned; if the work order is activated on the line, this applies to all stations/work steps concerned.
- As soon as a work step is set to "started", the work order is also set automatically to "started", if this has not already happened.
- If the work order is finished, the work steps are NOT changed to the "finished" state. Work steps must be "finished" manually – there is no order-dependent automatism that does this. Unlike the work order, finished work steps can no longer be started.

Furthermore, it is important to note that the system makes a distinction between the **cross-station** and **station-specific** work step state. Cross-station means the total state of all stations assigned to the work step via the ERP group. The following points show the basis on which the total state is formed from the individual state:

- If all individual states have the same value, the total state will also adopt this value.
- As soon as one of the stations has one of the following states, then this is the total state; the following sequence applies (override rule):
 - 1 = started**
 - 2 = interrupted** (no 1 present)
 - 3 = locked** (no 1 or 2 present)
 - 4 = released** (no 1, 2 or 3 present)
 - 5 = planned** (no 1, 2, 3 or 4 present)
 - 6 = finished** (no 1, 2, 3, 4 or 5 present)



Advice

*Apart from the "normal" process-controlling application types, the work step state is also used for work order detailed planning by means of **APS service**. As soon as a work step has been started, this is taken into consideration by the APS service during work order planning!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataChangeWorkOrderWorkStepState
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] workstepChangeKeys
in String[] workstepChangeValues
in String[] workstepResultKeys
out String[] workstepResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workstepChangeKeys` and `workstepResultKeys` as well as (out) `workstepResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Information on the work step

Key	Explanation	IMSApi data type
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order Advice: Even though this key is not identified as a mandatory field, either the CUSTOMER_WORKORDER_NUMBER or the WORKORDER_NUMBER must be passed!	☐ String
FUNC_MODE	Function mode "work step status" [0; 1]: 0 = Change applies only to the current station 1 = Change applies to all stations Advice: If the value [1] is passed, the performed state change applies across all stations to all entries of the ERP group assigned at the work step (total state)!	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐ int
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORKSTEP_STATE)!	☐ double
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Number of a production order Advice: Even though this key is not identified as a mandatory field, either the CUSTOMER_WORKORDER_NUMBER or the WORKORDER_NUMBER must be passed!	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: Whether this work step state applies as the total state (of all stations assigned to the work step via the ERP group) or applies only to the passed station is controlled by the FUNC_MODE key!	☒ String

Tab. 16-11 "Key Value Pair": workstepChangeKeys

Definition of output parameters for error feedback

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☐ int
PREVIOUS_WORK-STEP_STATE	Previous work step state [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished	☐ String
PROGRESS_PERCENT	Progress of the work step in percent Advice: <i>This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!</i>	☐ double
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Number of a production order	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 16-12 "Key Value Pair": *workstepResultKeys*

16.4.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workstep-ChangeKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>workstepChangeValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": workstepChangeKeys table. Advice: Only the parameters passed here are processed by the function!
workstepChangeValues	Values for the parameters selected by means of <code>workstepChangeKeys</code> Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>workstepChangeKeys</code>! Refer to the "Key Value Pair": workstepChangeKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workstepResult-Keys	List of the parameters that are to be ascertained by the function in the event of an error. A description of the possible parameters can be found above in the "Key Value Pair": workstepResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workstepResultValues</code>!

Tab. 16-13 Parameter (in) »mdataChangeWorkOrderWorkStepState«

Parameter (out)

Parameter name	Explanation
workstepResult-Values	The corresponding data is output here only in the event of an error – depending on the <code>workstepResultKeys</code> parameter (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>workstepResultKeys</code>!

Tab. 16-14 Parameter (out) »mdataChangeWorkOrderWorkStepState«

16.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-168	Failure	Invalid value for the function mode (control flag)
-1129	Failure	State change for work step not allowed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.5 mdataCreateDetectionPoint

16.5.1 Function description

This function makes it possible to create data capture points.



Advice

Only data capture points of the "Message" and "Measurement data" types are supported at present; other types will be added as required!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataCreateDetectionPoint
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String detectionPointName
in int detectionPointType
in KeyValue[] detectionPointKeyValues
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `detectionPointKeyValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Data capture point of the "message" type (detectionPointType = 0)

Key	Explanation		IMSApi data type
ACTION_PLAN	Description of the action to be performed	☐	String
ACTIVE	"Data capture point" activation state [0; 1]: 0 = Data capture point is NOT activated 1 = Data capture point is activated immediately Advice: If this key is not passed along, the [1] value is used by default!	☐	int
CONDITION_CODE	Unique designator of the machine condition/machine message in the iTAC system; this entry must have been assigned to the station passed using STATION_NUMBER. Advice: The newly created data capture point is linked to this entry!	☒	String
CONTROL	Name of the control unit	☐	String
DATA_BLOCK	Address range inside the control unit	☐	String
DETECTION_POINT_DESC	Description of the data capture point	☐	String
INFO_TXT_1	Individual additional information 1 on the data capture point	☐	String
INFO_TXT_2	Individual additional information 2 on the data capture point	☐	String
INFO_TXT_3	Individual additional information 3 on the data capture point	☐	String
INFO_TXT_4	Individual additional information 4 on the data capture point	☐	String
INFO_TXT_5	Individual additional information 5 on the data capture point	☐	String
LOCATION	Position of the capture point in the production plant (message location)	☐	String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: If the data capture point is to be assigned a message cause, the message cause must already exist in the iTAC system!	☐	String
MESSAGE_CLASS_NAME	Name of the message class Advice: If the data capture point is to be assigned a message class, the message class must already exist in the iTAC system!	☐	String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: If the data capture point is to be assigned a message effect, the message effect must already exist in the iTAC system!	☐	String
MESSAGE_INFO_TXT_1	Individual additional information 1 on the message	☐	String
MESSAGE_INFO_TXT_2	Individual additional information 2 on the message	☐	String
PRIORITY	Number for individual prioritization of the message [1 - 100]	☐	int
REMARK	Individual comment on the data capture point	☐	String
RESOURCE_NAME	Name of the resource Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	☐	String

Key	Explanation	IMSApi data type
STATION_NUMBER	Station number of the work station in the iTAC system Advice: The newly created data capture point is linked to this station!	☒ String
VARIABLE	Variable in the address range of the control unit	☐ String

Tab. 16-15 "Key Value Pair": *detectionPointKeyValues*

Data capture point of the "measurement data" type (detectionPointType = 1)

Key	Explanation	IMSApi data type
ACTIVE	"Data capture point" activation state [0; 1]: 0 = Data capture point is NOT activated 1 = Data capture point is activated immediately Advice: If this key is not passed along, the [1] value is used by default!	☐ int
CONTROL	Name of the control unit	☐ String
DATA_BLOCK	Address range inside the control unit	☐ String
DETECTION_POINT_DESC	Description of the data capture point	☐ String
ELEMENT_TYPE	Measurement data type: VAL = Measured value NOM = Nominal value TOL = Tolerance MIN = Value (min.) MAX = Value (max.) LAL = Lower intervention limit UAL = Upper intervention limit LSL = Lower scrap limit USL = Upper scrap limit	☒ String
INFO_TXT_1	Individual additional information 1 on the data capture point	☐ String
INFO_TXT_2	Individual additional information 2 on the data capture point	☐ String
INFO_TXT_3	Individual additional information 3 on the data capture point	☐ String
INFO_TXT_4	Individual additional information 4 on the data capture point	☐ String
INFO_TXT_5	Individual additional information 5 on the data capture point	☐ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	☒ String
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_NAME	Name of the resource Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: The newly created data capture point is linked to this station!	☒ String
VARIABLE	Variable in the address range of the control unit	☐ String

Tab. 16-16 "Key Value Pair": *detectionPointKeyValues*

16.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
detectionPoint-Name	Name of the data capture point Advice: <i>The used name must be unique at the site!</i>
detectionPoint-Type	Type of data capture point [0; 1]: 0 = Message 1 = Measurement data Advice: <i>Only the "Message" and "Measurement data" types are supported at present; other types will be added as required!</i>
detectionPoint-KeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": detectionPointKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: <i>Refer to the "Key Value Pair": detectionPointKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!</i>

Tab. 16-17 Parameter (in) »mdataCreateDetectionPoint«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

16.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-706	Failure	Recipe not found
-737	Failure	Recipe -> Measured value not found
-1200	Failure	Machine condition/message not found
-1219	Failure	Capture point -> Type is invalid
-1220	Failure	Capture point already exists
-1221	Failure	Message class not found
-1222	Failure	Message cause not found
-1223	Failure	Message effect not found
-1231	Failure	Data capture point -> Item type not valid
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2136	Failure	Resource not found
-2140	Failure	Resource does not match station

16.6 mdataCreateOrderCluster

16.6.1 Function description

This function enables the creation of a work order cluster. Work order clusters are used to merge different work orders and to manufacture them in a group (in parallel).



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = -1123) in the event of one or more faulty records!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataCreateOrderCluster
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] createOrderClusterValues
    String[] Key
    String[] Value
in String[] createOrderClusterItemUploadKeys
in String[] createOrderClusterItemUploadValues
out String[] createOrderClusterResultValues
out String[] createOrderClusterItemResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `createOrderClusterValues`, `createOrderClusterItemUploadKeys` and `createOrderClusterItemUploadValues` as well as (out) `createOrderClusterResultValues` and `createOrderClusterItemResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for the work order cluster that is to be created (header data)

Key	Explanation	IMSApi data type
CLUSTER_DESC	Description of the work order cluster Advice: This key is only evaluated if the name for the work order cluster is passed (see the CLUSTER_NAME key). If the name is generated there by the system, the name generated there is also used for the description!	☐ String
CLUSTER_NAME	Unique name for the work order cluster Advice: If the key is not passed, the name is generated by the system automatically; it is made up of an identification code for the work order cluster type (OC, WC or PC) followed by an identification number!	☐ String
CLUSTER_QUANTITY	Number of products to be manufactured in the cluster	☒ int
CLUSTER_STATE	State for the work order cluster [E; D]: E = Work order cluster is activated (E nabled) D = Work order cluster is NOT activated (D isabled) Advice: If this key is not passed along, the [E] value is used by default!	☐ String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster Advice: If the key is not passed, the value [O] is used by default; if the value [W] is passed here, it is essential that the desired work step be passed (see below, key WORKSTEP_NUMBER)!	☐ String
COMMENT	Individual comment on work order cluster creation	☐ String
FUNC_MODE	Function mode "work order assignment" [0; 1]: 0 = Work order can only be assigned to one cluster 1 = Work order can be assigned to multiple clusters; Advice: Mode [1] can only be used for the manufacturing of non-serial-numbered products. If the key is not passed, the value [0] is used by default!	☐ int

Tab. 16-18 "Key Value Pair": createOrderClusterValues

Input values for the work order assignment

Key	Explanation	IMSApi data type
CLUSTER_ITEM_STATE	State for the work order in the cluster [E; D]: E = Work order is activated in the cluster (E nabled) D = Work order is NOT activated in the cluster (D isabled) Advice: If this key is not passed along, the [E] value is used by default!	☐ String
CUSTOMER_WORKORDER_NUMBER	Customer-specific number for the production order	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -1123)!	☐ int
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐ boolean
POSITION	Unique position number of the work order in the cluster Advice: If the key is not passed, the current highest position number is increased by [+1] by default!	☐ int
POSITION_MIRRORED	Unique position number of the work order in the cluster for the application type "double-sided panel"; the key must always be used if at least one position in the work order cluster is identified as "mirrored" (IS_MIRRORED = TRUE). Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created! Technical background: This key is required for determining the correct position number if units are placed on the front and rear side using the same placement program.	☐ int
WORKORDER_NUMBER	Number of the production order that is to be assigned to the work order cluster	☒ String
WORKSTEP_NUMBER	Work step number in the iTAC system; this work step determines the work step that is to be manufactured jointly in the work step cluster. Advice: This key is only evaluated if the value [W] was passed as work order cluster type (see key CLUSTER_TYPE)!	☐ int

Tab. 16-19 "Key Value Pair": createOrderClusterItemUploadKeys

16.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
createOrderClusterValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": createOrderClusterValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": createOrderClusterValues table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
createOrderClusterItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>createOrderClusterItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": createOrderClusterItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!
createOrderClusterItemUploadValues	Values for the parameters selected by means of <code>createOrderClusterItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter – if activated – since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>createOrderClusterItemUploadKeys</code> ! Refer to the "Key Value Pair": createOrderClusterItemUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 16-20 Parameter (in) »mdataCreateOrderCluster«

Parameter (out)

Parameter name	Explanation
createOrderClusterResultValues	Depending on parameter (in) <code>createOrderClusterValues</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>createOrderClusterValuesKey</code> !
createOrderClusterItemResultValues	Depending on parameters (in) <code>createOrderClusterItemUploadKeys</code> and <code>createOrderClusterItemUploadValues</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>changeWorkorderBomUploadKeys</code> !

Tab. 16-21 Parameter (out) »mdataCreateOrderCluster«

16.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-132	Failure	Work order is not unique
-138	Failure	Order state is invalid
-1114	Failure	Work order cluster -> Name is already in use
-1116	Failure	Work order cluster -> Phantom work order not unique
-1117	Failure	Work order cluster -> Position is occupied
-1118	Failure	Work order cluster has an invalid state
-1119	Failure	Work order cluster -> Type not found
-1120	Failure	Work order cluster -> Type is invalid

-1121	Failure	Work order cluster -> Work order is already part of another work order cluster
-1123	Failure	Work order cluster -> Processing failed
-1125	Failure	Work order cluster -> APS job ID not unique
-2021	Failure	Work order type not valid
-2030	Failure	Work plan not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2115	Failure	Work step is invalid

16.7

mdataCreateSpecialTime

16.7.1

Function description

This function allows a special time for a data object to be newly created in the shift timetable. Special times are resource-related exceptions in the shift (e.g. in the case of maintenance, a fault, etc.).

Function matrix *ENTITY_TYPE* (shift)

The dependencies between the keys *ENTITY_TYPE* (data object) and *ENTITY_KEY* (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Equipment groups	6	Equipment group number
Production means	7	Number of the production means with index – delimited by the " " (pipe) character; depending on the PM class, this is then equipment or a carrier. Background to "PM class": By default, the value E "Equipment (work step)" is used for the "PM class"; if a different PM class is used (currently A "Equipment (asset)", M "Cavity" or C "Carrier"), this must then be added using a second delimiter. Syntax: Number Index Class
Teams	8	Team number
User	9	User identification
User groups	10	User group number

Tab. 16-22 *Function matrix* *ENTITY_TYPE*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataCreateSpecialTime
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int entityType
in String[] entityKey
in String[] timeEventName
in DateMs[] dateFrom
in DateMs[] dateTo
out long[] specialTimeId
```

16.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
entityType	Determines the data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups
entityKey	Depending on the selected data object (see <code>entityType</code>), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [▶▶▶]!
timeEventName	Name of the time event
dateFrom	Time stamp for the start of the special time (time events); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000
dateTo	Time stamp for the end of the special time (time events); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.

Tab. 16-23 Parameter (in) »mdataCreateSpecialTime«

Parameter (out)

Parameter name	Explanation
specialTimeId	Internal identification number of the special time

Tab. 16-24 Parameter (out) »mdataCreateSpecialTime«

16.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-109	Failure	Line not found
-1130	Failure	Special time already exists
-1139	Failure	Time event not found
-1301	Failure	Production means not found
-1319	Failure	Equipment group not found
-2015	Failure	Date not valid
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2105	Failure	No TR or PM license present for the station
-2131	Failure	Team not found
-2132	Failure	User not found
-2133	Failure	User group not found
-2134	Failure	Entity type is invalid
-2135	Failure	Entity key is invalid

16.8 mdataCreateWorkorder

16.8.1 Function description

With this function, it is possible to create a production order in the iTAC system.



Advice

When creating a work order with an API function, the same checking mechanisms are performed as apply during manual work order creation in the myiTAC workplace (ADM module)!

This API function is influenced by site parameter code nos. 3011 [▶▶▶] and 4420 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataCreateWorkorder
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] workorderUploadKeyValues
    String key
    String value
  in String[] workorderResultKeys
  out String[] workorderResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workorderUploadKeyValues` and `workorderResultKeys` as well as (out) `workorderResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for the work order to be created

Key	Explanation		IMSApi data type
APS_JOB_ID_- COPY_ON_SPLIT	APS service - copying mode for the "job designator" [0; 1]: 0 = JOB designator of the basic order is NOT copied (FALSE) 1 = JOB designator of the basic order is copied (TRUE) Background: This parameter is only relevant if production lots (partial orders) are to be created using this API function; for this purpose, a basic order must be passed using the <code>SPLIT_PARENT_WOR-KORDER</code> key.	☐	boolean
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	☐	int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]; the BOM type is optional; if it is set, a plausibility check with respect to the work plan alternative can be performed (see the bracket term for each BOM type): E = BOM for maintenance order (MAI) F = BOM for production order (01 , WO or CO) I = BOM for receiving (IG) M = BOM for material test (customer-specific) R = BOM for repair order (REP) S = BOM for external products (customer-specific) U = BOM for modification order (U) X = BOM for SCT order (customer-specific) Advice: The work plan alternatives for work order types [M], [S] and [X] are not predetermined; they are defined during the course of commissioning (recommended: MT for [M], EXT for [S] and SCT for [X])!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐	String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐	String
INDEX_1	Index field 1 (alphanumeric) for individual information	☐	String
INDEX_2	Index field 2 (alphanumeric) for individual information	☐	String
INDEX_3	Index field 3 (alphanumeric) for individual information	☐	String
INDEX_4	Index field 4 (alphanumeric) for individual information	☐	String
INDEX_5	Index field 5 (alphanumeric) for individual information	☐	String
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumeric additional information 1	☐	String

Key	Explanation	IMSApi data type
INFO_TXT_2	Alphanumerical additional information 2	☐ String
INFO_TXT_3	Alphanumerical additional information 3	☐ String
INFO_TXT_4	Alphanumerical additional information 4	☐ String
INFO_TXT_5	Alphanumerical additional information 5	☐ String
INFO_TXT_6	Alphanumerical additional information 6	☐ String
PART_GROUP	Part group in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☒ String
POOL_NAME	Name of the serial number pool Advice: This value is only evaluated if the new work order is to be a modification order (see WORKORDER_TYPE = U)!	☐ String
PRIORITY_CODE	APS service - unique designator for the work order priority	☐ String
PROCESS_VERSION	Process version from the iTAC system	☐ int
PRODUCTION_AREA	Production area; the following items are offered by default: 9999 = Flat units 9997 = Assembly 9998 = Rack 9900 = Receiving 9400 = External products The order type, just like the production area, is a way to more exactly specify a work order. The production area can be used individually; this can be used e.g. for product- or production-specific typecasting. Advice: Production areas are created directly in the database; details available upon request!	☐ int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master Advice: If a value is passed here that does not exist in the master data, the function outputs a failure (see error code -2035). If this key is not passed, the default value "Default 0" is used!	☐ String

Key	Explanation		IMSApi data type
RETROGRAD	Material removal mode for the work order. The modes that are available for selection are dependent on site parameter code no. 3011 [▶▶▶]; there, the degrees of freedom for manual processing are defined (see also advice below). A distinction is made between the following modes [0; 1; 2; 3]: 0 = No retrograde booking: the material for the production order is considered to be available for allocation during the entire manufacturing process depending on the actual removal from storage 1 = Complete quantity on first WS: the complete order quantity is booked on the first work step which is obligated to supply confirmation. Thus, the availability for allocation is not dependent on the time of actual removal from storage 2 = RM quantity on the first WS: only the actual quantity resulting from the feedback from the first work step is completely booked. Thus, the availability for allocation is not dependent on the time of actual removal from storage 3 = RM quantity at assigned WS: only the actual quantity resulting from the feedback from the individual work steps is booked. Thus, the availability for allocation is not dependent on the time of actual removal from storage	☐	int
SPLIT_PARENT_WORKORDER	Order number of the basic order in cases where with the API call a production lot (partial order) is to be created on the basis of an existing production order. Advice: <i>To create production lots, the specified basic order must have the "not released" state. The order quantity passed with WORKORDER_QUANTITY must not be greater than the order quantity of the basic order. The part (PART_NUMBER) must match the part of the basic order; if necessary, higher BOM and work plan versions can be used (cf. BOM_VERSION and WORKPLAN_VERS)! </i>	☐	String
WORKORDER_CODE	Description of the control code for work order creation; the following control codes are offered in the system: 8400 = External products 8500 = Maintenance order 8600 = SCT order 8650 = Material test 8700 = Receiving 8805 = Special order (no control function) 8888 = Production order 8889 = First batch order 8892 = Manufacturing sample 8900 = Internal return 8901 = External return 8902 = Modification order 8903 = Business order (no control function) Advice: <i>Currently, only the 8400, 8500, 8600, 8650, 8700, 8888, 8889, 8892, 8900, 8901 and 8902 control codes are supported (see also the key WORKPLAN_ALT); there is no direct function associated with the other entries. Support for additional work order types is planned in the medium term. It will then be possible to use these additional types for customized production control, e.g. as grouping and evaluation criteria!</i>	☐	int
WORKORDER_DELIVER_DATE_PLANNED	Planned delivery date for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs

Key	Explanation		IMSApi data type
WORKORDER_DESC	Order description	☐	String
WORKORDER_EARLIEST_DELIVER_DATE	Earliest delivery date for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes the current date and the current time!	☐	DateMs
WORKORDER_EARLIEST_START_DATE	Earliest start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes the current date and the current time!	☐	DateMs
WORKORDER_MAX_QUANTITY	Maximum order quantity Special case: If the key is NOT passed or is passed with [-1], the default values stored in the part master data are used; detailed information on this topic can be found in the user's guide under ADM module → Master data → Part master for the fields "Order quantity" and "Overbooking quantity". Advice: If a value greater than [-1] or less than the order quantity is passed (see WORKORDER_QUANTITY), the same value as for WORKORDER_QUANTITY is used here!	☐	int
WORKORDER_NUMBER	Customer-specific order number Advice: By default, the order number is automatically created by the system and entered on order release. Depending on the setting for site parameter code no. 4420 [▶▶▶], either an internal sequential number is used (default), or this (customer-specific) order number is used!	☐	String
WORKORDER_QUANTITY	Order quantity	☒	int
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐	long
WORKORDER_START_DATE_PLANNED	Planned start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes the current date and the current time!	☐	DateMs
WORKORDER_STATE	State of the production order [R; F]: R = planned F = opened (released) Advice: If this key is not passed along, the [F] value is used by default! Note that work orders with the "planned" state are stored in a different data structure. They can only be used actively by manually changing the state from "planned" to "opened" (for further information, see the "Order administration" function in the ADM module).	☐	String

Key	Explanation	IMSApi data type
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]; the work order type is indirectly defined by the work plan alternative (see identification code in the WORKPLAN_ALT field). The following assignments are available by default: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific) Advice: The work plan alternatives for work order types [M], [S] and [X] are not predetermined; they are defined during the course of commissioning (recommended: MT for [M], EXT for [S] and SCT for [X])! Depending on the work order type, only certain control codes are valid (see WORKORDER_CODE): - Production order: 8805, 8888, 8889, 8892, 8903, 8904 - Repair order: 8900 and 8901 - Modification order: 8902 - Receiving: 8700 - Maintenance: 8500 - Material test: 8650 - SCT order: 8600 - External products: 8400	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☒ String
WORKPLAN_VERS	Work plan version Advice: If the key is not passed, the highest released and valid work plan version is automatically used provided no ERP change number is passed (see WORKPLAN_VERS_ERP)	☐ String
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: This key is only evaluated if WORKPLAN_VERS is not used!	☐ String

Tab. 16-25 "Key Value Pair": workorderUploadKeyValues

Feedback about the created work order

Key	Explanation	IMSApi data type
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐ String
WORKORDER_ID	Internal identification number of the (new) production order	☐ long
WORKORDER_NUMBER	(New) production order number	☐ String

Tab. 16-26 "Key Value Pair": *workorderResultKeys*

16.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workorderUpload-Keys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": workorderUploadKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workorderUploadKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workorderResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workorderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) workorderResultValues !

Tab. 16-27 Parameter (in) »mdataCreateWorkorder«

Parameter (out)

Parameter name	Explanation
workorderResult-Values	Depending on parameter (in) workorderResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) workorderResultKeys !

Tab. 16-28 Parameter (out) »mdataCreateWorkorder«

16.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-126	Failure	Process version is invalid
-130	Failure	Work order not found
		Advice: The value -130 is only output if production lots (partial orders) are to be created for a basic order and the passed basic order is not found (see the key SPLIT_PARENT_WORKORDER)!
-137	Failure	No work order could be created
-1101	Failure	BOM not found
-1106	Failure	BOM for the work order not found
-2014	Failure	Customer not found

-2015	Failure	Date not valid
-2016	Failure	Work order type does not match work plan type
-2019	Failure	Process state not valid
-2020	Failure	Production area not valid
-2021	Failure	Work order type not valid
-2022	Failure	Control code not valid
-2023	Failure	Retrograde not valid
-2024	Failure	Quantity -> Value outside of the validity range
-2025	Failure	Production order already exists
-2026	Failure	Production order -> PCB assignment invalid
-2027	Failure	Part has an invalid state
-2028	Failure	Part group has an invalid part group type
-2029	Failure	Work plan cannot be used for first batch
-2032	Failure	Work plan contains no work steps with valid confirmation
-2033	Failure	Work plan contains invalid work step
-2034	Failure	Work plan -> No valid order number model found for the WP alternative
-2035	Failure	Production controller not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.9 mdataDeleteDetectionPoint

16.9.1 Function description

This function is used to delete a certain data capture point. If the data capture point has already been used, it is renamed in the background; the existing object references are retained in the system.



Advice

Complete deletion is only possible if the entry has not yet been used (booked)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataDeleteDetectionPoint
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String detectionPointName
```

16.9.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
detectionPoint-Name	Name of the data capture point

Tab. 16-29 Parameter (in) »mdataDeleteDetectionPoint«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

16.9.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-1217	Failure	Capture point not found
-2105	Failure	No TR or PM license present for the station

16.10 mdataDeleteOrderCluster

16.10.1 Function description

This function enables the deletion of a work order cluster; in doing so, the assignment to the work orders contained in the cluster are deleted. Only work order clusters with the **planned** or **opened** state can be deleted.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataDeleteOrderCluster
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] orderClusterDeleteFilter
    String[] key
    String[] value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `orderClusterDeleteFilter`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Input values for the work order cluster that is to be deleted

Key	Explanation		IMSApi data type
CHANGE_COMMENT	Individual comment on work order cluster deletion	☐	String
CLUSTER_NAME	Unique name for the work order cluster	☒	String

Tab. 16-30 "Key Value Pair": `orderClusterDeleteFilter`

16.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
orderCluster-DeleteFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterDeleteFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": orderClusterDeleteFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 16-31 Parameter (in) »mdataDeleteOrderCluster«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

16.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1118	Failure	Work order cluster has an invalid state
-1122	Failure	Work order cluster not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.11

mdataGetAssignedShiftModel

16.11.1

Function description

This function is used to query currently assigned shift models for the shift-capable data objects in the iTAC system; it is possible to apply filter criteria to a data object and data object details here.

Function matrix *ENTITY_TYPE* (shift)

The dependencies between the keys *ENTITY_TYPE* (data object) and *ENTITY_KEY* (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Equipment groups	6	Equipment group number
Production means	7	Number of the production means with index – delimited by the " " (pipe) character; depending on the PM class, this is then equipment or a carrier. Background to "PM class": By default, the value E "Equipment (work step)" is used for the "PM class"; if a different PM class is used (currently A "Equipment (asset)", M "Cavity" or C "Carrier"), this must then be added using a second delimiter. Syntax: Number Index Class
Teams	8	Team number
User	9	User identification
User groups	10	User group number

Tab. 16-32 *Function matrix* *ENTITY_TYPE*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetAssignedShiftModel
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] assignedShiftModelFilter
    String key
    String value
in String[] assignedShiftModelKeys
out String[] assignedShiftModelValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `assignedShiftModelFilter` and `assignedShiftModelKeys` as well as (out) `assignedShiftModelValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [>>>] !	☐	String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☐	int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [1000000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int

Tab. 16-33 "Key Value Pair": `assignedShiftModelFilter`

Definition of the output parameters for the found shift models

Key	Explanation	IMSApi data type
END_DATE	Time stamp for the end of the validity period of the shift model; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
ENTITY_KEY	Depending on the data object (see ENTITY_TYPE), the data object detail specification (e.g. the concrete machine group number) is output here. Advice: For further information, refer to the function matrix above in the section »Function description« [▶▶▶]!	☐ String
ENTITY_TYPE	Data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☐ int
SHIFT_MODEL_DESC	Description of the shift model	☐ String
SHIFT_MODEL_ID	Unique designator of the shift model	☐ long
SHIFT_MODEL_NAME	Name of the shift model	☐ String
START_DATE	Time stamp for the start of the validity period of the shift model; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
START_SHIFT_INDEX	Index (sequential number) of the week plan which the shift model is to initially begin with	☐ int

Tab. 16-34 "Key Value Pair": assignedShiftModelKeys

16.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
assignedShiftModelFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignedShiftModelFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": assignedShiftModelFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
assignedShiftModelKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": assignedShiftModelKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>assignedShiftModelValues</code> !

Tab. 16-35 Parameter (in) »mdataGetAssignedShiftModel«

Parameter (out)

Parameter name	Explanation
assignedShiftModelValues	Depending on parameter (in) <code>assignedShiftModelKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>assignedShiftModelKeys</code> !

Tab. 16-36 Parameter (out) »mdataGetAssignedShiftModel«

16.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-1301	Failure	Production means not found
-1319	Failure	Equipment group not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.12

mdataGetAssignedSpecialTime

16.12.1

Function description

This function is used to query currently assigned special times for the shift-capable data objects in the iTAC system; it is possible to apply filter criteria to a data object and data object details here.

Function matrix *ENTITY_TYPE* (shift)

The dependencies between the keys *ENTITY_TYPE* (data object) and *ENTITY_KEY* (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Equipment groups	6	Equipment group number
Production means	7	Number of the production means with index – delimited by the " " (pipe) character; depending on the PM class, this is then equipment or a carrier. Background to "PM class": By default, the value E "Equipment (work step)" is used for the "PM class"; if a different PM class is used (currently A "Equipment (asset)", M "Cavity" or C "Carrier"), this must then be added using a second delimiter. Syntax: Number Index Class
Teams	8	Team number
User	9	User identification
User groups	10	User group number

Tab. 16-37 *Function matrix* *ENTITY_TYPE*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetAssignedSpecialTime
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] assignedSpecialTimeFilter
    String key
    String value
in String[] assignedSpecialTimeKeys
out String[] assignedSpecialTimeValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `assignedSpecialTimeFilter` and `assignedSpecialTimeKeys` as well as (out) `assignedSpecialTimeValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [>>>] !	☒ String
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☒ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [1000000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int

Tab. 16-38 "Key Value Pair": `assignedShiftModelFilter`

Definition of the output parameters for the found special times

Key	Explanation	IMSApi data type
CONDITION_CODE	Unique designator of the machine condition Advice: At station level (see ENTITY_TYPE = 5), a special time can be stored for a machine condition; this condition automatically becomes active for the station for the validity period of the special time!	☐ String
DAY_OF_INTERVAL	Repetition interval "Selection of day"; the numerical value is always to be interpreted in combination with WEEK_DAY (with/without week-day reference). Examples: a.) Every 3rd day in the month DAY_OF_INTERVAL = 3, WEEK_DAY = -1 b.) Every 2nd Wednesday in the month DAY_OF_INTERVAL = 2, WEEK_DAY = 3 Advice: This key is only relevant for the repetition type MONTHLY (see the key REPETITION_TYPE)!	☐ String
DAYS_IN_WEEK	Selected days of the week within the repetition [0 - 6]: multiple days are separated with ";", e.g. 0;3;6 Only with WEEKLY and MONTHLY	☐ String
END_DATE	Time for the end of the special time for the respective day; time specified in milliseconds starting at 0:00:00.000. Example: 28800000 corresponds to a duration of 8 hours (8x60x60x1000) and is therefore to be interpreted as 8:00.	☐ DateMs
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here.	☐ String
ENTITY_TYPE	Data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☐ int
PRODUCTIVE	Productive identifier [0; 1]: 0 = Unproductive time event 1 = Productive time event	☐ int
REPETITION_- DAY_WEEK_MONTH_END	Time stamp for the end of the validity period of the special time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
REPETITION_- DAY_WEEK_MONTH_FROM	Time stamp for the start of the validity period of the special time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
REPETITION_TYPE	Repetition type for the special time: ONCE DAILY WEEKLY MONTHLY YEARLY	☐ String

Key	Explanation	IMSApi data type
REPETITION_YEAR_END	Specification of the year when repetition ends Advice: This key is only relevant for the repetition type YEARLY (see the key REPETITION_TYPE)! With the other repetition types, the value [3000] is output by default.	☐ int
REPETITION_YEAR_FROM	Specification of the year when repetition starts Advice: This key is only relevant for the repetition type YEARLY (see the key REPETITION_TYPE)! With the other repetition types, the value [3000] is output by default.	☐ int
START_DATE	Time for the start of the special time for the respective day; time specified in milliseconds starting at 0:00:00.000. Example: 28800000 corresponds to a duration of 8 hours (8x60x60x1000) and is therefore to be interpreted as 8:00.	☐ DateMs
TIME_EVENT_DESC	Description of the time event	☐ String
TIME_EVENT_ID	Unique designator of the time event	☐ long
TIME_EVENT_NAME	Name of the time event	☐ String
WEEK_DAY	Day of the week for the repetition interval "Selection of day"; the numerical value is always to be interpreted in combination with DAY_OF_INTERVAL [-1; 0 - 6]: -1 = No specific day of week selected 0 = 1st day of week as defined in system configuration ... 6 = 7th day of week as defined in system configuration Examples: a.) Every 3rd day in the month DAY_OF_INTERVAL = 3, WEEK_DAY = -1 b.) Every 2nd Wednesday in the month DAY_OF_INTERVAL = 2, WEEK_DAY = 3 Advice: This key is only relevant for the repetition type MONTHLY (see the key REPETITION_TYPE)!	☐ int

Tab. 16-39 "Key Value Pair": assignedSpecialTimeKeys

16.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
assignedSpecial-TimeFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": assignedSpecialTimeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": assignedSpecialTimeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
assignedSpecial-TimeKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": assignedSpecialTimeKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>assignedSpecialTimeValues</code> !

Tab. 16-40 Parameter (in) »mdataGetAssignedSpecialTime«

Parameter (out)

Parameter name	Explanation
assignedSpecial-TimeValues	Depending on parameter (in) <code>assignedSpecialTimeKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>assignedSpecialTimeKeys</code> !

Tab. 16-41 Parameter (out) »mdataGetAssignedSpecialTime«

16.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-1301	Failure	Production means not found
-1319	Failure	Equipment group not found
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value
-2131	Failure	Team not found
-2132	Failure	User not found
-2133	Failure	User group not found
-2134	Failure	Entity type is invalid
-2135	Failure	Entity key is invalid

16.13

mdataGetBomData

16.13.1

Function description

This function returns a specific BOM. Used as reference value is the order number, the serial number or the part number in combination with the BOM version and process version; if multiple values are passed, the order number is used. If necessary, the `FUNC_MODE` key = 1 (see parameter `bomDataFilter`) can be used to restrict the output to strictly BOM header information.

For the standard application case (`FUNC_MODE = 0`), the scope of the returned BOM information can be individually adapted via the following three keys (see parameter `bomDataFilter`).

- The input value `BOM_ALTERNATIVE` specifies whether the alternative BOM components are to be output, too.
- The input value `DELTA_MODE` specifies whether the order-based BOM, the original master BOM or the delta BOM (only the differences between order-based and original master BOM) is to be output.
- The input value `PROCESS_BASED` specifies whether the process-related BOM (with station number passed) or the general BOM – valid for all processes – is output.
- The input value `BOM_TYPE` specifies whether the construction BOM or the scheduling BOM is output:
Construction BOM = all components are listed according to their mounting place
Scheduling BOM = identical components are shown in a consolidated manner

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetBomData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] bomDataFilter
    String key
    String value
in String[] bomDataResultKeys
out String[] bomDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `bomDataFilter` and `bomDataResultKeys` as well as (out) `bomDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
BOM_ALTERNATIVE	Controls output for regular and alternative components in the BOM [0; 1]: 0 = Regular component 1 = Regular and alternative components Advice: If this key is not passed along, the [0] value is used by default!	☐	int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Special case: If [-1] is passed, the most current version is used automatically!	☐	String
BOM_TYPE	Controls the output for the type of BOM [0; 1]: 0 = scheduling BOM 1 = construction BOM Advice: If this key is not passed along, the [0] value is used by default!	☐	int
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order	☐	String
BOM_VERSION	BOM version from the iTAC system Special case: If [-1] is passed, the most current version is used automatically. If a WORKORDER_NUMBER or SERIAL_NUMBER is passed, this key is not evaluated!	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
COMP_TYPE	Component type: 10200 = Electrical component 10201 = Mechanical component 10202 = Printed circuit board (component side) 10206 = Printed circuit board (soldering side) 10207 = Documents/drawings Background: Component types are process-related groups that are permanently specified in the system.	☐	long
DELTA_MODE	Determines whether the data for the order-based BOM, the original master BOM or the delta BOM is to be output [0; 1; 2]: 0 = Order-based BOM 1 = Original master BOM 2 = Only delta BOM Advice: If this key is not passed along, the [0] value is used by default. If the value [2] is passed, only the BOM positions that differ are taken into account during output!	☐	int

Key	Explanation	IMSApi data type
FUNC_MODE	Defines the function mode for the output of the bomDataResult-Keys "Key Value Pair" structure [0; 1]: 0 = Output of all selected data fields (default) 1 = Output of the selected "header data fields" (only the BOM_ID and BOM_VERS keys are available for selection here) Advice: If this key is not passed along, the [0] value is used by default!	☐ int
OBSERVE_ZERO_QUANTITY	Zero quantity mode for BOM components [0; 1]: 0 = Do NOT take into account entries with quantity "0" (FALSE) 1 = Take into account entries with quantity "0" (TRUE) Advice: If this key is not passed along, the [0] value is used by default!	☐ boolean
PROCESS_BASED	Controls the output with respect to the process [0; 1]: 0 = General BOM 1 = Process-related BOM Advice: If the key is not passed, the value [0] is used by default; if the value [1] is passed, a valid use type must also be passed (see PROCESS_TYPE)!	☐ int
PROCESS_LAYER	Process layer of the BOM component [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_TYPE	Selection of a use type [10300 - 10304; -1]: 10300 = placement 10301 = visual inspection 10302 = component preparation 10303 = provision by customer 10304 = provision to external -1 = In the case PROCESS_BASED = 0 Advice: The use type is only evaluated if value [1] is passed for PROCESS_BASED; if value [0] is passed for PROCESS_BASED, [-1] must be passed here!	☐ int
PROCESS_VERSION	Process version of the BOM from the iTAC system Special case: If [-1] is passed, the most current version is used automatically. If a WORKORDER_NUMBER or SERIAL_NUMBER is passed, this key is not evaluated!	☐ int
PRODUCT_NUMBER	Part number in the iTAC system Advice: If the key is not passed, an order number or serial number must absolutely be passed (see WORKORDER_NUMBER or SERIAL_NUMBER). If the part number is used, the BOM version and the process version must absolutely be passed!	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: If the key is not passed, an order number or part number must absolutely be passed (see workOrderNumber or material-Number)!	☐ String
WORKORDER_NUMBER	Production order number Advice: If the key is not passed, a serial number or part number must absolutely be passed (see SERIAL_NUMBER or PRODUCT_NUMBER)!	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int

Tab. 16-42 "Key Value Pair": bomDataFilter

Definition of the output parameters for the found BOM

Key	Explanation		IMSApi data type
BOM_ALTERNATIVE	Designator for regular or alternative component in the parts list [0; 1]: 0 = regular component 1 = alternative component	☐	int
BOM_ID	Internal identification number of the BOM in combination with the BOM version; the two pieces of information are separated by a comma ([ID], [VERSION])	☐	String
BOM_PARENT_VERSION	BOM version from the iTAC system (parent integration level) Advice: This information is only ascertained if the entered unit is itself mounted in the parent product and the "material for parent work order" mode is set in the part master (see the "MRP" tab there!)	☐	int
BOM_POSITION_INDEX	Position number of the BOM component	☐	long
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
COMP_NAME	Mounting place conformant with the iTAC BOM	☐	String
COMP_PARENT_NAME	Mounting place conformant with the iTAC BOM (parent integration level) Advice: This information is only ascertained if the entered unit is itself mounted in the parent product and the "material for parent work order" mode is set in the part master (see the "MRP" tab there!)	☐	String
COMP_TYPE	Component type: 10200 = Electrical component 10201 = Mechanical component 10202 = Printed circuit board (component side) 10206 = Printed circuit board (soldering side) 10207 = Documents/drawings Background: Component types are process-related groups that are permanently specified in the system.	☐	long
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.)	☐	String
INFO	Output of the stored BOM information	☐	String
IS_DELTA	Indicates whether the BOM position (component) deviates between order-based and original master BOM [0; 1]: 0 = BOM position is identical (FALSE) 1 = BOM position is NOT identical, i.e., the BOM position was changed for the work order (TRUE)	☐	boolean
MACHINE_GROUP_DESC	Description of the machine group on which the production process was performed according to PROCESS_TYPE	☐	String
MACHINE_GROUP_NUMBER	Number of the machine group on which the production process was performed according to PROCESS_TYPE	☐	String

Key	Explanation	IMSApi data type
MATERIAL_RESERVATION	Indicates whether reservation is required for the material (component) for the passed work order [0; 1]: 0 = Reservation not required 1 = Reservation required Advice: <i>This information is only ascertained if a work order (see workOrderNumber is passed; the reservation requirement can only be activated in the order-based BOM!</i>	☐ int
PART_DESC	Part description in the iTAC system (component)	☐ String
PART_NUMBER	Part number in the iTAC system (component)	☐ String
PART_PARENT_NUMBER	Part number in the iTAC system (parent integration level) Advice: <i>This information is only ascertained if the entered unit is itself mounted in the parent product and the "material for parent work order" mode is set in the part master (see the "MRP" tab there)!</i>	☐ String
PROCESS_INFO_TXT	Output of the stored process information	☐ String
PROCESS_LAYER	Process layer of the BOM component [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_TYPE	Selection of a use type [10300 - 10304]: 10300 = placement 10301 = visual inspection 10302 = component preparation 10303 = provision by customer 10304 = provision to external	☐ int
PRODUCT_FLAG	Designator for raw material / product[0; 1]: 0 = Raw material (component) 1 = Product (unit) Advice: <i>This value corresponds to the product designator from the order BOM (cf. "Product" field for the BOM components in the BOM module). Note here that this value does not necessarily need to match the product designator from the part master!</i>	☐ int
QUANTITY	Quantity of the part (if necessary, also as decimal value)	☐ double
SETUP_FLAG	Designator for the setup requirement [0; 1]: 0 = No setup required 1 = Setup required	☐ int
UNIT	Unit for the value passed	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int

Tab. 16-43 "Key Value Pair": bomDataResultKeys

16.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
bomDataFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": bomDataFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": bomDataFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
bomDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": bomDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) bomDataResultValues !

Tab. 16-44 Parameter (in) »mdataGetBomData«

Parameter (out)

Parameter name	Explanation
bomDataResultValues	Depending on parameter (in) bomDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) bomDataResultKeys !

Tab. 16-45 Parameter (out) »mdataGetBomData«

16.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-119	Failure	Part is not unique
-121	Failure	Part not found
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-161	Failure	Invalid value for "alternative" [0; 1]
-162	Failure	Invalid value for "processBased" [0; 1]
-163	Failure	Invalid value for "bomType"
-242	Failure	BOM version is invalid

-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.14

mdataGetCalendarData

16.14.1

Function description

This function provides information about the current date and time (server). In doing so, the desired "time modules" can be selected.



Advice

*This API function is influenced by station parameter code no. 13200 [▶▶▶]. This must be set up so that the shift type (*SHIFT_TYPE*) can be output. Further information on this topic can be found in the reference section of this documentation!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetCalendarData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] calendarDataResultKeys
  out String[] calendarDataResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) *calendarDataResultKeys* and (out) *calendarDataResultValues*). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
CURRENT_TIME_MILLIS	Current time stamp in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ long
DAY_OF_MONTH	Current day of the month [1 - 31]	☐ int
DAY_OF_WEEK	Current day of the week [1 - 7]	☐ int
DAY_OF_WEEK_IN_MONTH	Display of a continuous integer number of a seven-day period of the month, beginning with the first day of the month [1 - 5] (1st to 7th = 1; 8th to 14th = 2; etc.)	☐ int
DAY_OF_YEAR	Current day of the year [1 - 365]	☐ int
DAY_SECTION	2 x 12 hour counting [0; 1]: 0 = a.m. (ante meridiem -> before noon) 1 = p.m. (post meridiem -> after noon)	☐ int
DISPLAY_DATE	Date string, formatted according to the locale	☐ String
DST_OFFSET	Deviations when switching to daylight savings time (in milliseconds)	☐ int
HOUR	Current hour [1 - 12]	☐ int
HOUR_OF_DAY	Current hour of the day (24 hrs.) [0 - 23]	☐ int
MINUTE	Current minute [0 - 59]	☐ int
MONTH	Current month [1 - 12]	☐ int
SECOND	Current second [0 - 59]	☐ int
SHIFT_TYPE	Shift type Advice: To allow the shift type to be output, station parameter code no. 13200 [▶▶▶] must have been set up!	☐ int
WEEK_OF_MONTH	Current week of the month	☐ int
WEEK_OF_YEAR	Current week of the year [1 - 52]	☐ int
WEEKYEAR	Current calendar year relative to the week to which the current day belongs; the outputs for WEEKYEAR and YEAR are usually identical; if the outputs differ, this will only be the case in the first and last calendar week. Example: 2012-12-31 is a Monday and therefore belongs to the first calendar week of 2013; in this case, the year 2014 would already be output for WEEKYEAR and the year 2013 would still be output for YEAR.	☐ int
YEAR	Current year (e.g. 2016)	☐ int
ZONE_OFFSET	Deviations from the GMT time zone (in milliseconds)	☐ int

Tab. 16-46 "Key Value Pair": calendarDataResultKeys

16.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
calendarDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": calendarDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) calendarDataResultValues !

Tab. 16-47 Parameter (in) »mdataGetCalendarData«

Parameter (out)

Parameter name	Explanation
calendarDataResultValues	Depending on parameter (in) calendarDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) calendarDataResultKeys !

Tab. 16-48 Parameter (out) »mdataGetCalendarData«

16.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1103	Failure	Shift not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.15 mdataGetContactData

16.15.1 Function description

This function enables the querying of business contact data; here, it is possible to filter according to suppliers, manufacturers or customers.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetContactData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] contactFilter
    String key
    String value
in String[] contactResultKeys
out String[] contactResultValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `contactFilter` and `contactResultKeys` as well as (out) `contactResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVE	"Contact data" activation state [0; 1; 2]: 0 = only active contacts 1 = only inactive contacts 2 = active and inactive contacts Advice: If this key is not passed along, the [0] value is used by default!	int
CODE	Business contact designator	String
CONTACT_TYPE	Contact type [C; M; S]: C = only contacts of type "Customer" M = only contacts of type "Manufacturer" S = only contacts of type "Supplier" Advice: If this key is not passed along, the [C] value is used by default! This value is dependent on the selected object type (see OBJECT_TYPE)!	String
COUNTRY	Country code Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
NAME	Name of the company or the contact person Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
OBJECT_NUMBER	Part number, container number or order number Advice: This value is dependent on the selected object type (see OBJECT_TYPE); if the key is not passed, there is no restriction!	String
OBJECT_TYPE	Determines for the OBJECT_NUMBER the object type on the basis of which the function is to be executed [0; 1; 2; 3; 4]: 0 = part number (iTAC) -> the contact data of the provider is taken into account according to the part master 1 = part number (iTAC) -> the contact data of the provider and also of the assigned customers and manufacturers are taken into account (according to the part master) 2 = container number -> the contact data of the suppliers and manufacturers for the component are taken into account 3 = order number -> the contact data of the customers, suppliers and manufacturers for the product is taken into account 4 = external part number (customer/manufacturer) -> the same contact data as in mode 1 is taken into account. Advice: If this key is not passed along, the [0] value is used by default!	int

Key	Explanation	IMSApi data type
SORT_FLAG	Sorting mode for the output [0; 1; 2; 3]: 0 = ascending according to <code>NAME</code> 1 = descending according to <code>NAME</code> 2 = ascending according to <code>CODE</code> 3 = descending according to <code>CODE</code> Advice: If this key is not passed along, the [2] value is used by default!	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for <code>MAX_ROW_COUNT</code> . Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int

Tab. 16-49 "Key Value Pair": `contactFilter`

Definition of the output parameters for the found contacts

Key	Explanation	IMSApi data type
ACTIVE	"Contact data" activation state [0; 1]: 0 = contact is active 1 = contact is NOT active	int
CITY	City	String
CODE	Business contact designator	String
CONTACT_TYPE	Contact type [C; M; S]: C = contact of type "Customer" M = contact of type "Manufacturer" S = contact of type "Supplier"	String
COUNTRY	Country	String
ERP_CODE	Abbreviation from the ERP system	String
GROUP_NAME	Business contact group	String
NAME	Name of the company or the contact person	String
PART_DESC	Part description in the iTAC system	String
PART_DESC_EXTERNAL	External part number of the customer or manufacturer Advice: This parameter is only output if value [1] is passed for the <code>OBJECT_TYPE</code> !	String
PART_NUMBER	Part number in the iTAC system	String
PART_NUMBER_EXTERNAL	External part description of the customer or manufacturer Advice: This parameter is only output if value [1] is passed for the <code>OBJECT_TYPE</code> !	String
SCT_CODE	SCT code for the business contact	String

Key	Explanation	IMSApi data type
SCT_ENABLED	Consideration of SCT service [0; 1]: 0 = contact is NOT "SCT capable" 1 = contact is "SCT capable"	☐ String
STREET	Street	☐ String
ZIP_CODE	ZIP code	☐ String

Tab. 16-50 "Key Value Pair": *contactResultKeys*

16.15.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
contactFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": contactFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": contactFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
contactResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": contactResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>contactResultValues</i> !

Tab. 16-51 Parameter (in) »mdataGetContactData«

Parameter (out)

Parameter name	Explanation
contactResultValues	Depending on parameter (in) <i>contactResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>contactResultKeys</i> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the START POSITION and MAX_ROW_COUNT keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-52 Parameter (out) »mdataGetContactData«

16.15.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.16 mdataGetCostCenter

16.16.1 Function description

This function makes it possible to query cost center information from the iTAC system; a wide range of different filtering criteria can be applied here (e.g. cost center types, invoicing intervals or active/inactive designator). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetCostCenter
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mdataGetCostCenterFilter
    String key
    String value
in String[] mdataGetCostCenterKeys
out String[] mdataGetCostCenterValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mdataGetCostCenterFilter and mdataGetCostCenterKeys as well as (out) mdataGetCostCenterValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVE	"Cost centers" activation state [0; 1]: 0 = Only inactive cost centers 1 = Only active cost centers Special case: If the key is passed with [-1], there is no restriction; active and inactive cost centers are then reviewed! Advice: If this key is not passed along, the [1] value is used by default!	int
COST_CENTER_DESC	Cost center description Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
COST_CENTER_EXTERNAL	Cost center designator from an external system Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
COST_CENTER_NUMBER	Cost center number Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
COST_CENTER_ORGANIZATION	Organizational area for the cost center within the company (customer-specific) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
COST_CENTER_TYPE	Cost center type [A; C; P]: A = Administration C = Controlling P = Production Advice: If the key is not passed or passed with [-1], there is no restriction; all cost center types are taken into consideration!	String
INVOICING_COST_CENTER	Cost center key (customer-specific) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
INVOICING_DEPARTMENT	Invoicing interval (group key) for the cost center (customer-specific) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [100] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int

Key	Explanation	IMSApi data type
WAGE_TYPE_NUMBER	Wage type with correct cost center assignment Advice: The assigned cost centers are ascertained on the basis of the specified wage type!	☐ String

Tab. 16-53 "Key Value Pair": `mdataGetCostCenterFilter`

Definition of the output parameters for the found cost centers

Key	Explanation	IMSApi data type
ACTIVE	"Cost centers" activation state [0; 1]: 0 = cost center is NOT active 1 = cost center is active	☐ int
CALCULATION_COST_FIXED	Fixed amount which is included in the cost calculation in connection with this cost center (customer-specific)	☐ double
CALCULATION_COST_VARIABLE	Variable amount which is included in the cost calculation in connection with this cost center (customer-specific)	☐ double
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
COST_CENTER_DESC	Cost center description	☐ String
COST_CENTER_EXTERNAL	Cost center designator from an external system	☐ String
COST_CENTER_NUMBER	Cost center number	☐ String
COST_CENTER_ORGANIZATION	Organizational area for the cost center within the company (customer-specific)	☐ String
COST_CENTER_TYPE	Cost center type [A; C; P]: A = Administration C = Controlling P = Production	☐ String
CREATION_DATE	Date of creation of the cost center; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
INFO_TEXT	Individual information text for the cost center	☐ String
INVOICING_COST_CENTER	Cost center key (customer-specific)	☐ String
INVOICING_DEPARTMENT	Invoicing interval (group key) for the cost center (customer-specific)	☐ String
PRODUCTIVE	Productive designator of the cost center [0; 1]: 0 = NOT used in production 1 = Used in production	☐ int

Tab. 16-54 "Key Value Pair": `mdataGetCostCenterKeys`

16.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdataGetCostCenterFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetCostCenterFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetCostCenterFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdataGetCostCenterKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetCostCenterKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mdataGetCostCenterValues</code> !

Tab. 16-55 Parameter (in) »mdataGetCostCenter«

Parameter (out)

Parameter name	Explanation
mdataGetCostCenterValues	Depending on parameter (in) <code>mdataGetCostCenterKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mdataGetCostCenterKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-56 Parameter (out) »mdataGetCostCenter«

16.16.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.17 mdataGetDetectionPoint

16.17.1 Function description

This function enables data capture points to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., assigned station, data capture point name, control unit used, etc.). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.



Advice

Only data capture points of the "Message" and "Measurement data" types are supported at present; other types will be added as required!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int mdataGetDetectionPoint
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int detectionPointType
in KeyValue[] detectionPointFilter
    String key
    String value
in String[] detectionPointKeys
out String[] detectionPointValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `detectionPointFilter` and `detectionPointKeys` as well as (out) `detectionPointValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐). The tables are described in the following sequence:

"Key Value Pair" (message): detectionPointFilter

"Key Value Pair" (message): detectionPointKeys

"Key Value Pair" (measurement data): detectionPointFilter

"Key Value Pair" (measurement data): detectionPointKeys

Search for data capture points of "message" type (detectionPointType = 0)

Key	Explanation	IMSApi data type
ACTIVE	"Data capture points" activation state [0; 1]: 0 = Only inactive data capture points 1 = Only active data capture points Advice: If the key is not passed, there is no restriction; active and inactive data capture points are then reviewed!	int
CONDITION_CODE	Unique designator of the machine condition or of the machine message in the iTAC system. Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	String
CONTROL	Name of the control unit Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
DATA_BLOCK	Address range inside the control unit Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
DETECTION_POINT_NAME	Name of the data capture point Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
LOCATION	Position of the capture point in the production plant (message location) Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
MESSAGE_CAUSE_NAME	Name of the message cause Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
MESSAGE_CLASS_NAME	Name of the message class Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	String
PRIORITY	Number for individual prioritization of the message [1 - 100]	int

Key	Explanation	IMSApi data type
RESOURCE_NAME	Name of the resource Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)! Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	☐ String
VARIABLE	Variable in the address range of the control unit Advice: Wildcards can be used when making entries (see the advice notice under the key CONDITION_CODE)!	☐ String

Tab. 16-57 "Key Value Pair" (message): detectionPointFilter

Output of data capture points of "message" type (detectionPointType = 0)

Key	Explanation	IMSApi data type
ACTION_PLAN	Description of the action to be performed	☐ String
ACTIVE	"Data capture point" activation state [0; 1]: 0 = Data capture point is NOT active 1 = Data capture point is active	☐ int
CONDITION_CODE	Unique designator of the machine condition or of the machine message in the iTAC system.	☐ String
CONTROL	Name of the control unit	☐ String
DATA_BLOCK	Address range inside the control unit	☐ String
DETECTION_POINT_DESC	Description of the data capture point	☐ String
DETECTION_POINT_NAME	Name of the data capture point	☐ String
INFO_TXT_1	Individual additional information 1 on the data capture point	☐ String
INFO_TXT_2	Individual additional information 2 on the data capture point	☐ String
INFO_TXT_3	Individual additional information 3 on the data capture point	☐ String
INFO_TXT_4	Individual additional information 4 on the data capture point	☐ String
INFO_TXT_5	Individual additional information 5 on the data capture point	☐ String

Key	Explanation	IMSApi data type
LOCATION	Position of the capture point in the production plant (message location)	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause	☐ String
MESSAGE_CLASS_NAME	Name of the message class	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect	☐ String
MESSAGE_INFO_TXT_1	Individual additional information 1 on the message	☐ String
MESSAGE_INFO_TXT_2	Individual additional information 2 on the message	☐ String
PRIORITY	Number for individual prioritization of the message [1 - 100]	☐ int
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_NAME	Name of the resource	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
VARIABLE	Variable in the address range of the control unit	☐ String

Tab. 16-58 "Key Value Pair" (message): *detectionPointKeys*

Search for data capture points of "measurement data" type (detection-PointType = 1)

Key	Explanation	IMSApi data type
ACTIVE	"Data capture points" activation state [0; 1]: 0 = Only inactive data capture points 1 = Only active data capture points Advice: If the key is not passed, there is no restriction; active and inactive data capture points are then reviewed!	☐ int
CONTROL	Name of the control unit Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
DATA_BLOCK	Address range inside the control unit Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)!	☐ String
DETECTION_POINT_NAME	Name of the data capture point Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)!	☐ String
ELEMENT_TYPE	Measurement data type: VAL = Measured value NOM = Nominal value TOL = Tolerance MIN = Value (min.) MAX = Value (max.) LAL = Lower intervention limit UAL = Upper intervention limit LSL = Lower scrap limit USL = Upper scrap limit	☐ String

Key	Explanation	IMSApi data type
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name) Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)!	String
RESOURCE_NAME	Name of the resource Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)! Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)!	String
VARIABLE	Variable in the address range of the control unit Advice: Wildcards can be used when making entries (see the advice notice under the key CONTROL)!	String

Tab. 16-59 "Key Value Pair" (measurement data): detectionPointFilter

Output of data capture points of "measurement data" type
(detectionPointType = 1)

Key	Explanation	IMSApi data type
ACTIVE	"Data capture point" activation state [0; 1]: 0 = Data capture point is NOT active 1 = Data capture point is active	int
CONTROL	Name of the control unit	String
DATA_BLOCK	Address range inside the control unit	String
DETECTION_POINT_DESC	Description of the data capture point	String
DETECTION_POINT_NAME	Name of the data capture point	String
ELEMENT_TYPE	Measurement data type: VAL = Measured value NOM = Nominal value TOL = Tolerance MIN = Value (min.) MAX = Value (max.) LAL = Lower intervention limit UAL = Upper intervention limit LSL = Lower scrap limit USL = Upper scrap limit	String
INFO_TXT_1	Individual additional information 1 on the data capture point	String
INFO_TXT_2	Individual additional information 2 on the data capture point	String
INFO_TXT_3	Individual additional information 3 on the data capture point	String
INFO_TXT_4	Individual additional information 4 on the data capture point	String
INFO_TXT_5	Individual additional information 5 on the data capture point	String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	String
REMARK	Individual comment on the data capture point	String
RESOURCE_NAME	Name of the resource	String
STATION_NUMBER	Station number of the work station in the iTAC system	String
VARIABLE	Variable in the address range of the control unit	String

Tab. 16-60 "Key Value Pair" (measurement data): *detectionPointKeys*

16.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
detectionPoint-Type	Type of data capture point [0; 1]: 0 = Message 1 = Measurement data Advice: Only the "Message" and "Measurement data" types are supported at present; other types will be added as required!
detectionPoint-Filter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters (depending on the detectionPointType) can be found above in the corresponding "Key Value Pair" (xxx): detectionPointFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": detectionPointFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
detectionPoint-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the detectionPointType) can be found above in the corresponding "Key Value Pair" (xxx): detectionPointKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) detectionPointValues !

Tab. 16-61 Parameter (in) »mdataGetDetectionPoint«

Parameter (out)

Parameter name	Explanation
detectionPoint-Values	Depending on parameter (in) detectionPointKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) detectionPointKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the START_POSITION and MAX_ROW_COUNT keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-62 Parameter (out) »mdataGetDetectionPoint«

16.17.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-1219	Failure	Capture point -> Type is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.18

mdataGetFailureDataForPart

16.18.1

Function description

This function returns the failure information deposited for the queried part(s). The results are output in two parts: the assigned failure types and the part images. Both output structures can be put into context using the key `PART_NUMBER`.



Advice

The part-specific failure information that can be called up here is collected in my-iTAC workplace (in the ADM module → Setup → Failure definitions, see the "Part image assignment" function)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetFailureDataForPart
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mdataGetFailureDataForPartFilter
    String key
    String value
in String[] failureDataKeys
out String[] failureDataValues
in String[] imageDataKeys
out String[] imageDataValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see the parameters (in) `mdataGetFailureDataForPartFilter`, `failureDataKeys` and `imageDataKeys` as well as (out) `failureDataValues` and `imageDataValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i> Special case: Here, the default value MAX_ROWS refers to the number of affected parts and not, as usual, to the number of output structures (records); i.e., for a part with three failure records, the <code>failureDataKeys</code> structure would be output three times even with MAX_ROWS with "1". With this special provision, it is ensured that ALL failure data associated with a part is always output.	☒ int
PART_NUMBER	Part number in the iTAC system Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String

Tab. 16-63 "Key Value Pair": `mdataGetFailureDataForPartFilter`

Definition of the output parameters for the found failure types

Key	Explanation	IMSApi data type
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: <i>The available entries are permanently set in the system!</i>	☐ int
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: <i>The available entries are permanently set in the system!</i>	☐ int
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_NUMBER	Part number in the iTAC system Advice: <i>Based on this entry, the output data can be put into context (see the parameter by the same name in the table "Key Value Pair": <code>imageDataKeys</code>)!</i>	☐ String
PREMIUM_WAGE_RELEVANT	Error booking with bonus-relevant wage type [0; 1]: 0 = NO bonus-relevant wage type was used (FALSE) 1 = A bonus-relevant wage type was used (TRUE)	☐ boolean

Tab. 16-64 "Key Value Pair": `failureDataKeys`

Definition of the output parameters for the found part images

Key	Explanation	IMSApi data type
GRID_X	X value for part image rastering Advice: For visual failure input, a grid can be defined during part image assignment; in combination with the Y value, the position and area of each failure location can then be defined!	☐ int
GRID_Y	Y value for part image rastering Advice: See the advice notice under the key GRID_X!	☐ int
IMAGE_NAME	Name of the part image Advice: The name is assigned in the ADM module during part image assignment (subfunction "Failure definition")!	☐ String
MDA_DOCUMENT_ID	Unique designator (ID) for the attached document (MDA item with the part image) Advice: The content of a document can be called up on the basis of this ID (see API function »mdaGetDocumentContent« [▶▶▶])!	☐ long
PART_NUMBER	Part number in the iTAC system Advice: Based on this entry, the output data can be put into context (see the parameter by the same name in the table "Key Value Pair": failureDataKeys)!	☐ String

Tab. 16-65 "Key Value Pair": imageDataKeys

16.18.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdataGetFailure-DataForPartFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetFailureDataForPartFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetFailureDataForPartFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureDataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureDataValues !
imageDataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": imageDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) imageDataValues !

Tab. 16-66 Parameter (in) »mdataGetFailureDataForPart«

Parameter (out)

Parameter name	Explanation
failureDataValues	Depending on parameter (in) failureDataKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureDataKeys !
imageDataValues	Depending on parameter (in) imageDataKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) imageDataKeys !

Tab. 16-67 Parameter (out) »mdataGetFailureDataForPart«

16.18.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.19 mdataGetFailureDataForStation

16.19.1 Function description

This function returns the failure information (failure symptoms, failure types, failure causes, etc.) stored on the passed station.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int mdataGetFailureDataForStation
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int functionMode
in String[] failureDataResultKeys
out String[] failureDataResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `failureDataResultKeys` and (out) `failureDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Failure types (functionMode = 0)

Key	Explanation		IMSApi data type
FAILURE_TYPE_CATEGORY	Failure type category [C; S] C = Component failure (C omponent) S = Soldering point failure (S oldering) Reason: This setting affects the dpm calculation for the failure type analysis in the QM module.	☐	String
FAILURE_TYPE_CODE	Failure type code	☐	String
FAILURE_TYPE_DESC	Description of the failure type	☐	String
FAILURE_TYPE_GROUP_CODE	Failure type group code	☐	String

Key	Explanation	IMSApi data type
FAILURE_- TYPE_GROUP_DESC	Description of the failure type group	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: The available entries are permanently set in the system!	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = Not a pseudo failure 1 = Pseudo failure	☐ char
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: The available entries are permanently set in the system!	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PREMIUM_WAGE_RELEVANT	Error booking with bonus-relevant wage type [0; 1]: 0 = NO bonus-relevant wage type was used (FALSE) 1 = A bonus-relevant wage type was used (TRUE)	☐ boolean
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 16-68 "Key Value Pair" (failure types): *failureDataResultKeys*

Failure causes (functionMode = 1)

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_- CAUSE_GROUP_CODE	Failure cause group code	☐ String
FAILURE_- CAUSE_GROUP_DESC	Description of the failure cause group	☐ String
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 16-69 "Key Value Pair" (failure causes): *failureDataResultKeys*

Failure symptoms (functionMode = 2)

Key	Explanation	IMSApi data type
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String

Tab. 16-70 "Key Value Pair" (failure symptoms): *failureDataResultKeys***Failure measures** (functionMode = 4)

Key	Explanation	IMSApi data type
FAILURE_MEASURE_CODE	Failure measure code	☐ String
FAILURE_MEASURE_DESC	Description of the failure measure	☐ String

Tab. 16-71 "Key Value Pair" (failure measures): *failureDataResultKeys***Failure actions** (functionMode = 5)

Key	Explanation	IMSApi data type
FAILURE_ACTION_CODE	Failure action code	☐ String
FAILURE_ACTION_DESC	Description of the failure action	☐ String

Tab. 16-72 "Key Value Pair" (failure actions): *failureDataResultKeys***Failure object parts** (functionMode = 6)

Key	Explanation	IMSApi data type
FAILURE_OBJECT_PART_CODE	Failure object part code	☐ String
FAILURE_OBJECT_PART_DESC	Description of the failure object part	☐ String

Tab. 16-73 "Key Value Pair" (failure object parts): *failureDataResultKeys***Failure risks** (functionMode = 7)

Key	Explanation	IMSApi data type
FAILURE_RISK_CODE	Failure risk code	☐ String
FAILURE_RISK_DESC	Description of the failure risk	☐ String

Tab. 16-74 "Key Value Pair" (failure risks): *failureDataResultKeys*

Complete failure structure (functionMode = 9)

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_TYPE_CATEGORY	Failure type category [C; S] C = Component failure (Component) S = Soldering point failure (Soldering) Reason: This setting affects the dpm calculation for the failure type analysis in the QM module.	☐ String
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: The available entries are permanently set in the system!	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = Not a pseudo failure 1 = Pseudo failure	☐ char
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: The available entries are permanently set in the system!	☐ int
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int

Tab. 16-75 "Key Value Pair" (failure type): *failureDataResultKeys*

16.19.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
functionMode	Function mode; defines which failure information is to be output [0; 1; 2; 4; 5; 6; 7; 9]: 0 = Failure types 1 = Failure causes 2 = Failure symptoms 4 = Failure measures 5 = Failure actions 6 = Failure object parts 7 = Failure risks 9 = Complete failure structure (corresponds to 0 + 1 + 2)
failureDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the <code>functionMode</code>) can be found above in the corresponding " Key Value Pair " (xxx): failureDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>failureDataResultValues</code> !

Tab. 16-76 Parameter (in) »mdataGetFailureDataForStation«

Parameter (out)

Parameter name	Explanation
failureDataResultValues	Depending on parameter (in) <code>failureDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureDataResultKeys</code> !

Tab. 16-77 Parameter (out) »mdataGetFailureDataForStation«

16.19.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-168	Failure	Invalid value for the function mode (control flag)
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.20 mdataGetFailureDefinition

16.20.1 Function description

This function returns the deposited failure information for the querying station or for the machine group to be specified. It is possible to select which failure information is to be output. Depending on the input parameter `functionMode`, a distinction is made between the following types:

- Failure type
- Failure cause
- Failure symptom
- Failure measure
- Failure action
- Failure object part
- Failure risk
- Failure structure

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetFailureDefinition
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int stationMode
  in int functionMode
  in KeyValue[] failureDataFilter
    String[] key
    String[] value
  in String[] failureDataResultKeys
  out String[] failureDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `failureDataFilter` and `failureDataResultKeys` as well as (out) `failureDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MACHINE_GROUP_NUMBER	Number of the machine group Advice: This key is only available if the input parameter <i>station-Mode</i> is passed with [1] or [2]!	☐ String

Tab. 16-78 "Key Value Pair": *failureDataFilter*

Definition of the output parameters: failure type (functionMode = 0)

Key	Explanation	IMSApi data type
FAILURE_TYPE_CATEGORY	Failure type category [C; S] C = Component failure (C omponent) S = Soldering point failure (S oldering) Reason: This setting affects the dpm calculation for the failure type analysis in the QM module.	☐ String
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_GROUP_CODE	Failure type group code	☐ String
FAILURE_TYPE_GROUP_DESC	Description of the failure type group	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: The available entries are permanently set in the system!	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = Not a pseudo failure 1 = Pseudo failure	☐ char
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: The available entries are permanently set in the system!	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group	☐ String
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PREMIUM_WAGE_RELEVANT	Error booking with bonus-relevant wage type [0; 1]: 0 = NO bonus-relevant wage type was used (FALSE) 1 = A bonus-relevant wage type was used (TRUE)	☐ boolean
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 16-79 "Key Value Pair": (failure type) *failureDataResultKeys*

Definition of the output parameters: failure cause (functionMode = 1)

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_- CAUSE_GROUP_CODE	Failure cause group code	☐ String
FAILURE_- CAUSE_GROUP_DESC	Description of the failure cause group	☐ String
MACHINE_GROUP_NUM- BER	Number of the machine group	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 16-80 "Key Value Pair": (cause of the failure) *failureDataResultKeys***Definition of the output parameters: failure symptom (functionMode = 2)**

Key	Explanation	IMSApi data type
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String

Tab. 16-81 "Key Value Pair": (failure symptom) *failureDataResultKeys***Definition of the output parameters: failure measure (functionMode = 4)**

Key	Explanation	IMSApi data type
FAILURE_MEA- SURE_CODE	Failure measure code	☐ String
FAILURE_MEASURE_- DESC	Description of the failure measure	☐ String

Tab. 16-82 "Key Value Pair": (failure measure) *failureDataResultKeys***Definition of the output parameters: failure action (functionMode = 5)**

Key	Explanation	IMSApi data type
FAILURE_ACTION_CODE	Failure action code	☐ String
FAILURE_ACTION_DESC	Description of the failure action	☐ String

Tab. 16-83 "Key Value Pair": (failure action) *failureDataResultKeys*

Definition of the output parameters: failure object part (functionMode = 6)

Key	Explanation	IMSApi data type
FAILURE_OBJECT-PART_CODE	Failure object part code	☐ String
FAILURE_OBJECT-PART_DESC	Description of the failure object part	☐ String

Tab. 16-84 "Key Value Pair": (failure object part) *failureDataResultKeys***Definition of the output parameters: failure risk (functionMode = 7)**

Key	Explanation	IMSApi data type
FAILURE_RISK_CODE	Failure risk code	☐ String
FAILURE_RISK_DESC	Description of the failure risk	☐ String

Tab. 16-85 "Key Value Pair": (failure risk) *failureDataResultKeys***Definition of the output parameters: failure structure (functionMode = 9)**

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_TYPE_CATEGORY	Failure type category [C; S] C = Component failure (C omponent) S = Soldering point failure (S oldering) Reason: This setting affects the dpm calculation for the failure type analysis in the QM module.	☐ String
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: The available entries are permanently set in the system!	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = Not a pseudo failure 1 = Pseudo failure	☐ char

Key	Explanation	IMSApi data type
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: <i>The available entries are permanently set in the system!</i>	☐ int
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int

Tab. 16-86 "Key Value Pair": (failure structure) *failureDataResultKeys*

16.20.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationMode	"Station/machine group" query mode [0; 1; 2]: 0 = "Station" failure data (stationNumber) 1 = "Machine group" failure data (MACHINE_GROUP_NUMBER) 2 = "Station" and "Machine group" failure data
functionMode	Function mode; defines which failure information is to be output [0; 1; 2; 4; 5; 6; 7; 9]: 0 = Failure types 1 = Failure causes 2 = Failure symptoms 4 = Failure measures 5 = Failure actions 6 = Failure object parts 7 = Failure risks 9 = Complete failure structure (corresponds to 0 + 1 + 2)
failureDataFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureDataFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": failureDataFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the functionMode) can be found above in the corresponding "Key Value Pair" (xxx): failureDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureDataResultValues!

Tab. 16-87 Parameter (in) »mdataGetFailureDefinition«

Parameter (out)

Parameter name	Explanation
failureDataResultValues	Depending on parameter (in) failureDataResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureDataResultKeys!

Tab. 16-88 Parameter (out) »mdataGetFailureDefinition«

16.20.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-168	Failure	Invalid value for the function mode (control flag)
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.21 mdataGetFormulaCatalog

16.21.1 Function description

This function returns the formulas stored in the formula catalog of the iTAC system. The calculation rule, name, description and also the associated parameters can be output for each formula. These values are used by e.g. the IMSApi function »exec-FormulaCatalog« [▶▶▶] for calculations.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetFormulaCatalog
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] formulaFilter
    String key
    String value
in String[] formulaResultKeys
in String[] formulaParamResultKeys
out String[] formulaResultValues
out String[] formulaParamResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) formulaFilter, formulaResultKeys and formulaParamResultKeys as well as (out) formulaResultValues and formulaParamResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific formulas

Key	Explanation	IMSApi data type
FORMULA	Formula string (e.g. "a + b") Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
FORMULA_DESCRIPTION	Description of the formula Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
FORMULA_NAME	Formula name Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ String

Tab. 16-89 "Key Value Pair": *formulaFilter*

Definition of the output parameters for the found formulas

Key	Explanation	IMSApi data type
FORMULA	Formula string (e.g. "a + b")	☐ String
FORMULA_DESCRIPTION	Description of the formula	☐ String
FORMULA_NAME	Formula name Advice: This key should be used even if it is not identified as a mandatory field. The result data records can only be put into context on the basis of the formula name (see the key of the same name in the "Key Value Pair": <i>formulaParamResultKeys</i> table)!	☐ String

Tab. 16-90 "Key Value Pair": *formulaResultKeys*

Definition of the output parameters for the found parameter

Key	Explanation	IMSApi data type
FORMULA_NAME	Formula name Advice: This key should be used even if it is not identified as a mandatory field. The result data records can only be put into context on the basis of the formula name (see the key of the same name in the "Key Value Pair": <i>formulaResultKeys</i> table)!	☐ String
FORMULA_PARAM_DESC	Description of the parameter	☐ String
FORMULA_PARAM_NAME	Name of the parameter	☐ String

Tab. 16-91 "Key Value Pair": *formulaParamResultKeys*

16.21.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
formulaFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": formulaFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": formulaFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
formulaResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": formulaResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) formulaResultValues !
formulaParamResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": formulaParamResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) formulaParamResultValues !

Tab. 16-92 Parameter (in) »mdataGetFormulaCatalog«

Parameter (out)

Parameter name	Explanation
formulaResultValues	Depending on parameter (in) formulaResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) formulaResultKeys !
formulaParamResultValues	Depending on parameter (in) formulaParamResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) formulaParamResultKeys !

Tab. 16-93 Parameter (out) »mdataGetFormulaCatalog«

16.21.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.22

mdataGetMachineAssetStructure

16.22.1

Function description

This function outputs arbitrary machine asset structures. This also includes information on the resources – these are usually machine components which are subordinate to a station – as well as information on the respective part numbers of the associated asset equipment (key word "system BOM"). The function can be flexibly adapted to the given requirements; as a result, it is possible to, e.g., ascertain and output all stations of a line or all machine groups of a site. Both the entry level as well as the output depth can be freely selected.



Advice

*As a rule, only current machine data is taken into account; i.e., the objects must be valid or active. The filter keys `ONLY_ACTIVE` and `ONLY_VALID` in the "Key Value Pair" structure `machineAssetStructureFilter` only affect the machine asset structures **below** the queried level; e.g. when a machine group is queried with regard to the stations contained there.*

If multiple filter keys are passed for purposes of limitation – e.g. machine group and station, the station must be part of the machine group (keys are "AND"-coupled). If the machine asset structure cannot be resolved, the function returns a technical error code.

Take note here of the various application types that can be activated with the `FUNC_MODE` key (see the `machineAssetStructureFilter` structure).

Application type "A" (machine tree): Query of the machine asset structures below the entry "Site" (there only for "Cells"); the configuration `FUNC_MODE = [0]` applies for this application type. Depending on the entry level, site-specific information can be output for **cells**, **machine groups**, **stations** and **resources**.

Application type "B" (line definition): query of the machine asset structures below the "Line definition" entry; for this application type, the `FUNC_MODE = [1]` configuration applies. Depending on the entry level, site-specific information can be output for **lines** and **stations**.

Application type "C" (ERP group definition): query of the ERP groups with the stations assigned there; for this application type, the `FUNC_MODE = [2]` configuration applies. Depending on the entry level, site-specific information can be output for **ERP groups** and **stations**.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int mDataGetMachineAssetStructure
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue [ ] machineAssetStructureFilter
    key
    value
in String [ ] machineAssetStructureResultKeys
out String [ ] machineAssetStructureValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `machineAssetStructureFilter` and `machineAssetStructureResultKeys` as well as (out) `machineAssetStructureValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CELL_NUMBER	Cell number Advice: This key is only evaluated in <code>FUNC_MODE = 0!</code>	☐ String
DISSOLVING_LEVEL	"Data ascertainment" mode [0; 1]: 0 = only the data of the underlying level 1 = all data down to the lowest level Advice: If this key is not passed along, the [0] value is used by default! Example for application type "A": Mode 0 in combination with entries <code>SITE_NUMBER</code> and <code>CELL_NUMBER</code> -> during data ascertainment, data is resolved down to the "machine group" level! Example for application type "B" or "C": Mode 1 in combination with entry <code>SITE_NUMBER</code> -> during data ascertainment, data is resolved down to the "station" level	☐ int
ERP_GROUP_NUMBER	ERP group number in the iTAC system Advice: This key is only evaluated in <code>FUNC_MODE = 2!</code>	☐ String

Key	Explanation	IMSApi data type
FUNC_MODE	Function mode [0; 1; 2:] 0 = machine tree (application type A) 1 = line definition (application type B) 2 = ERP group definition (application type C) Details on the function modes can be found above in the function description (application types A - C). Advice: If this key is not passed along, the [0] value is used by default!	☐ int
LINE_NUMBER	Line number Advice: This key is only evaluated in FUNC_MODE = 1!	☐ String
MACHINE_GROUP_NUMBER	Machine group number Advice: This key is only evaluated in FUNC_MODE = 0!	☐ String
ONLY_ACTIVE	"Machine asset structures" activation condition [0; 1] 0 = Also deactivated machine asset structures 1 = Only active machine asset structures Advice: This key only affects the machine asset structures below the queried level; e.g. when a machine group is queried with respect to the stations contained there. If the key is not passed, the value [1] is used by default!	☐ int
ONLY_VALID	"Machine asset structures" validity [0; 1]: 0 = including invalid machine asset structures 1 = only valid machine asset structures Advice: This key only affects the machine asset structures below the queried level; e.g. when a machine group is queried with respect to the stations contained there. If the key is not passed, the value [1] is used by default!	☐ int
RESOURCE_NAME	Name of the resource Advice: This key is only evaluated in FUNC_MODE = 0! The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used only if DISSOLVING_LEVEL = 1 is input; pay attention to upper- and lower-case letters! Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	☐ String
SITE_NUMBER	Site number Advice: This key must always be passed since it is evaluated by every FUNC_MODE!	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice regarding "data ascertainment": If this key is used, the data is ascertained from bottom to top, i.e. resolved upward beginning with this station – depending on the FUNC_MODE and DISSOLVING_LEVEL; if the passed station does not exist in the specified site, the function outputs the error code -2046. Advice regarding "dependencies": The station number must always be passed together with STATION_TYPE; the value "T" should be set there by default!	☐ String

Key	Explanation	IMSApi data type
STATION_TYPE	Station type [T; M]: T = Station for tasks in the TR context M = Station for tasks in the PM context Advice: <i>The station type must always be passed together with STATION_NUMBER; the value "T" should be set here by default!</i> Background: With the TR/PM consolidation implemented in release 9.01, a distinction according to station type is only relevant in exceptional cases.	String
USER_NAME	Login name of the user (user identification) Advice: <i>This key is only evaluated in FUNC_MODE = 1!</i> If a user identification is passed, only the lines in the site that are released for the user are taken into account (key word "line authorization" in the setup dialog of the team data of the ADM module).	String

Tab. 16-94 "Key Value Pair": *machineAssetStructureFilter*

Definition of the output parameters for the found machine asset structures

Key	Explanation	IMSApi data type
CELL_ACTIVE	"Cell" activation designator [0; 1]: 0 = Not active 1 = Active	int
CELL_DESC	Cell description	String
CELL_INFO_TXT_1	Individual additional information 1 (cell)	String
CELL_INFO_TXT_10	Individual additional information 10 (cell)	String
CELL_INFO_TXT_2	Individual additional information 2 (cell)	String
CELL_INFO_TXT_3	Individual additional information 3 (cell)	String
CELL_INFO_TXT_4	Individual additional information 4 (cell)	String
CELL_INFO_TXT_5	Individual additional information 5 (cell)	String
CELL_INFO_TXT_6	Individual additional information 6 (cell)	String
CELL_INFO_TXT_7	Individual additional information 7 (cell)	String
CELL_INFO_TXT_8	Individual additional information 8 (cell)	String
CELL_INFO_TXT_9	Individual additional information 9 (cell)	String
CELL_NUMBER	Cell number	String
CELL_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	String
CELL_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	String

Key	Explanation		IMSApi data type
CELL_TYPE	Cell type [T; M]: T = Cell for tasks in the TR context M = Cell for tasks in the PM context Advice: With the TR/PM consolidation implemented in release 9.01, a distinction according to cell type is no longer relevant (default = "T"); this key remains part of the IMSApi interface for downward compatibility reasons only!	☐	String
ERP_GROUP_DESC	Description of the ERP group	☐	String
ERP_GROUP_NUMBER	ERP group number in the iTAC system	☐	String
ERP_GROUP_START_DATE	Start of the validity period for the ERP group; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
ERP_GROUP_TYPE	ERP group type [P; M]: P = Personnel process step M = Machine process step	☐	String
ERP_GROUP_VALID	"ERP group" validity [0; 1]: 0 = Invalid 1 = Valid	☐	int
LINE_ACTIVE	"Line" activation designator [0; 1]: 0 = Not active 1 = Active	☐	int
LINE_DESC	Line description	☐	String
LINE_INFO_TXT_1	Individual additional information 1 (line)	☐	String
LINE_INFO_TXT_10	Individual additional information 10 (line)	☐	String
LINE_INFO_TXT_2	Individual additional information 2 (line)	☐	String
LINE_INFO_TXT_3	Individual additional information 3 (line)	☐	String
LINE_INFO_TXT_4	Individual additional information 4 (line)	☐	String
LINE_INFO_TXT_5	Individual additional information 5 (line)	☐	String
LINE_INFO_TXT_6	Individual additional information 6 (line)	☐	String
LINE_INFO_TXT_7	Individual additional information 7 (line)	☐	String
LINE_INFO_TXT_8	Individual additional information 8 (line)	☐	String
LINE_INFO_TXT_9	Individual additional information 9 (line)	☐	String
LINE_NUMBER	Line number	☐	String
LINE_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐	String
LINE_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐	String
LINE_TYPE	Line type [T; M]: T = TR line M = PM line	☐	String
MACHINE_GROUP_ACTIVE	"Machine group" activation designator [0; 1]: 0 = Not active 1 = Active	☐	int

Key	Explanation	IMSApi data type
MACHINE_GROUP_DESC	Description of the machine group	☐ String
MACHINE_GROUP_END_DATE	End of the validity period for the machine group; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MACHINE_GROUP_NUMBER	Machine group number	☐ String
MACHINE_GROUP_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
MACHINE_GROUP_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
MACHINE_GROUP_START_DATE	Start of the validity period for the machine group; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MACHINE_GROUP_TYPE	Machine group type [T; M]: T = Machine group for tasks in the TR context M = Machine group for tasks in the PM context Advice: With the TR/PM consolidation implemented in release 9.01, a distinction according to machine group type is no longer relevant (default = "T"); this key remains part of the IMSApi interface for downward compatibility reasons only!	☐ String
PARENT_RESOURCE_ACTIVE	"Parent resource" activation designator [0; 1]: 0 = Not active 1 = Active	☐ int
PARENT_RESOURCE_NAME	Name of the parent resource	☐ String
RESOURCE_ACTIVE	"Resource" activation designator [0; 1]: 0 = Not active 1 = Active	☐ int
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String
RESOURCE_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
RESOURCE_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
SITE_DESC	Description of the site	☐ String
SITE_LATITUDE	Value for geographical latitude (site location); the coordinates must be entered beforehand in the "Site setup" view of the ADM module. Advice: To specify the geographical location, the geographical longitude is also required (see the key SITE_LONGITUDE)!	☐ double
SITE_LONGITUDE	Value for geographical longitude (site location); the coordinates must be entered beforehand in the "Site setup" view of the ADM module. Advice: To specify the geographical location, the geographical latitude is also required (see the key SITE_LATITUDE)!	☐ double
SITE_NUMBER	Site number	☐ String
SITE_NUMBER_EXT	Site number from an external system	☐ String

Key	Explanation	IMSApi data type
SITE_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
SITE_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
STATION_ACTIVE	"Station" activation designator [0; 1]: 0 = Not active 1 = Active	☐ boolean
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_END_DATE	End of the validity period for the station; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
STATION_ERP_END_DATE	Point in time by which the station is assigned to the ERP group; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
STATION_ERP_START_DATE	Point in time at which the station was assigned to the ERP group; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
STATION_PART_DESC	Description of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
STATION_PART_NUMBER	Number of the assigned part in the iTAC system and used as asset equipment (key word "system BOM")	☐ String
STATION_START_DATE	Start of the validity period for the station; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
STATION_TYPE	Station type [T; M]: T = Station for tasks in the TR context M = Station for tasks in the PM context Advice: With the TR/PM consolidation implemented in release 9.01, a distinction according to station type is only relevant in exceptional cases (default = "T")!	☐ String

Tab. 16-95 "Key Value Pair": *machineAssetStructureResultKeys*

16.22.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
machineAsset-StructureFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": machineAssetStructureFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the " Key Value Pair ": machineAssetStructureFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
machineAsset-StructureResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the " Key Value Pair ": machineAssetStructureResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>machineAssetStructureValues</code> !

Tab. 16-96 Parameter (in) »mdataGetMachineAssetStructure«

Parameter (out)

Parameter name	Explanation
machineAsset-StructureValues	Depending on parameter (in) <code>machineAssetStructureResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>machineAssetStructureResultKeys</code> !

Tab. 16-97 Parameter (out) »mdataGetMachineAssetStructure«

16.22.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-125	Failure	ERP group not found
-168	Failure	Invalid value for the function mode (control flag)
-2043	Failure	Site not found
-2044	Failure	Station -> Type is invalid
-2045	Failure	Station is not active
-2046	Failure	Station -> Reference not found
-2047	Failure	Cell is not active
-2048	Failure	Cell not found

-2049	Failure	Machine group is not active
-2050	Failure	Machine group not found
-2051	Failure	Line is not active
-2061	Failure	Machine group is not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2139	Failure	Resource is not active
-2144	Failure	Invalid value for "onlyActive" [0; 1]
-2145	Failure	Invalid value for "onlyValid" [0; 1]
-2146	Failure	Station is invalid
-2147	Failure	Machine group is invalid
-2148	Failure	ERP group not active

16.23 mdataGetMessageCause

16.23.1 Function description

This function is used to query message causes.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int mdataGetMessageCause
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] messageCauseFilter
    String key
    String value
in String[] messageCauseKeys
out String[] messageCauseValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `messageCauseFilter` and `messageCauseKeys` as well as (out) `messageCauseValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
MESSAGE_CAUSE_NAME	Name of the message cause Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int

Tab. 16-98 "Key Value Pair": messageCauseFilter

Definition of the output parameters for the found message causes

Key	Explanation	IMSApi data type
MESSAGE_CAUSE_DESC	Description of the message cause	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause	☐ String

Tab. 16-99 "Key Value Pair": messageCauseKeys

16.23.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
messageCauseFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": messageCauseFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": messageCauseFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
messageCauseKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": messageCauseKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>messageCauseValues</code> !

Tab. 16-100 Parameter (in) »mdataGetMessageCause«

Parameter (out)

Parameter name	Explanation
messageCauseValues	Depending on parameter (in) <code>messageCauseKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>messageCauseKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-101 Parameter (out) »mdataGetMessageCause«

16.23.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.24 mdataGetMessageClass

16.24.1 Function description

This function is used to query message classes.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int mdataGetMessageClass
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] messageClassFilter
    String key
    String value
in String[] messageClassKeys
out String[] messageClassValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `messageClassFilter` and `messageClassKeys` as well as (out) `messageClassValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with <code>START_POSITION</code> , it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
MESSAGE_CLASS_NAME	Name of the message class Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for <code>MAX_ROW_COUNT</code> . Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int

Tab. 16-102 "Key Value Pair": *messageClassFilter*

Definition of the output parameters for the found message classes

Key	Explanation	IMSApi data type
MESSAGE_CLASS_DESC	Description of the message class	☐ String
MESSAGE_CLASS_NAME	Name of the message class	☐ String

Tab. 16-103 "Key Value Pair": *messageClassKeys*

16.24.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
messageClassFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": messageClassFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": messageClassFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
messageClassKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": messageClassKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>messageClassValues</code> !

Tab. 16-104 Parameter (in) »mdataGetMessageClass«

Parameter (out)

Parameter name	Explanation
messageClassValues	Depending on parameter (in) <code>messageClassKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>messageClassKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-105 Parameter (out) »mdataGetMessageClass«

16.24.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.25 mdataGetMessageEffect

16.25.1 Function description

This function is used to query message effects.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int mdataGetMessageEffect
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] messageEffectFilter
    String key
    String value
in String[] messageEffectKeys
out String[] messageEffectValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) messageEffectFilter and messageEffectKeys as well as (out) messageEffectValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
MESSAGE_EFFECT_NAME	Name of the message effect Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int

Tab. 16-106 "Key Value Pair": messageEffectFilter

Definition of the output parameters for the found message effects

Key	Explanation	IMSApi data type
MESSAGE_EFFECT_DESC	Description of the message effect	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect	☐ String

Tab. 16-107 "Key Value Pair": messageEffectKeys

16.25.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
messageEffectFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": messageEffectFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": messageEffectFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
messageEffectKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": messageEffectKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>messageEffectValues</code> !

Tab. 16-108 Parameter (in) »mdataGetMessageEffect«

Parameter (out)

Parameter name	Explanation
messageEffectValues	Depending on parameter (in) <code>messageEffectKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>messageEffectKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-109 Parameter (out) »mdataGetMessageEffect«

16.25.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.26 mdataGetOrderCluster

16.26.1 Function description

This function makes it possible to query work order cluster information from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., creation period, cluster name or active/inactive designator). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetOrderCluster
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in KeyValue[] orderClusterFilter
    String Key
    String Value
in String[] orderClusterHeadResultKeys
in String[] orderClusterItemResultKeys
out String[] orderClusterHeadResultValues
out String[] orderClusterItemResultValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `orderClusterFilter`, `orderClusterHeadResultKeys` and `orderClusterItemResultKeys` as well as (out) `orderClusterHeadResultValues` and `orderClusterItemResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CLUSTER_DESC	Description of the work order cluster Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
CLUSTER_NAME	Unique name for the work order cluster Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
CLUSTER_STATE	State for the work order cluster [E; D]: E = Work order cluster is active (E nabled) D = Work order cluster is NOT active (D isabled) Advice: If the key is not passed, only the value [E] is used by default; if the value [-1] is entered, there is no restriction!	☐ String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster Advice: If the key is not passed, only types "O" and "P" are considered by default; if the value [-1] is entered, there is no restriction!	☐ String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
DATE_FROM	Start of the period under review relative to the date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start of the period under review!	☐ DateMs
DATE_TO	End of the period under review relative to the date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 31.12.3000, 23:59:59.999 as the end time of the period under review!	☐ DateMs
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int

Key	Explanation	IMSApi data type
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
WORKORDER_NUMBER	Production order number Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 16-110 "Key Value Pair": *orderClusterFilter*

Definition of the output parameters for the found work order clusters (header data)

Key	Explanation	IMSApi data type
CLUSTER_DESC	Description of the work order cluster	☐ String
CLUSTER_ID	Internal identification number of the work order cluster	☐ long
CLUSTER_NAME	Unique name for the work order cluster	☐ String
CLUSTER_QUANTITY	Number of products to be manufactured in the cluster	☐ int
CLUSTER_STATE	State for the work order cluster [E; D]: E = Work order cluster is active (Enabled) D = Work order cluster is NOT active (Disabled)	☐ String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐ String
COMMENT	Individual comment on work order cluster creation	☐ String
ERP_GROUP_DESC	Description of the ERP group Advice: This key is only relevant for the work order cluster type W (work step cluster); the work step to be produced jointly in the work order cluster is identified on the basis of the ERP group!	☐ String
ERP_GROUP_NUMBER	ERP group number in the iTAC system Advice: See the advice notice under the key ERP_GROUP_DESC!	☐ String
IS_MIRRORED_BOARD	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE) Advice: If the "double-sided panel" entry is selected, the setup information data of the component side and soldering side are combined and considered as a whole for this work order/work step!	☐ boolean

Tab. 16-111 "Key Value Pair": *orderClusterHeadResultKeys*

Definition of the output parameters for the assigned work orders

Key	Explanation	IMSApi data type
CLUSTER_ID	Internal identification number of the work order cluster	☐ long
CLUSTER_ITEM_STATE	State for the work order in the cluster [E; D]: E = Work order is active in the cluster (E nabled) D = Work order is NOT active in the cluster (D isabled)	☐ String
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐ boolean
IS_PHANTOM_MASTER	Designator "Master phantom work order" [0; 1]: 0 = Work order is NOT a master phantom work order (FALSE) 1 = Work order is a master phantom work order (TRUE) Background: The master phantom work order is the initial work order that was adopted from the ERP system. The number of the master phantom work order is used both for a work order position and as the name of the work order cluster (CLUSTER_NAME). All other work order positions are created automatically in the iTAC system; here the order number (WORKORDER_NUMBER) is derived from the number of the master phantom work order.	☐ boolean
POSITION	Unique position number of the work order in the cluster	☐ int
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 16-112 "Key Value Pair": *orderClusterItemResultKeys*

16.26.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
orderClusterFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": orderClusterFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
orderClusterHeadResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>orderClusterHeadResultValues</code> !
orderClusterItemResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterItemResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>orderClusterItemResultValues</code> !

Tab. 16-113 Parameter (in) »mdataGetOrderCluster«

Parameter (out)

Parameter name	Explanation
orderClusterHeadResultValues	Depending on parameter (in) <code>orderClusterHeadResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>orderClusterHeadResultKeys</code> !
orderClusterItemResultValues	Depending on parameter (in) <code>orderClusterItemResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>orderClusterItemResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-114 Parameter (out) »mdataGetOrderCluster«

16.26.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-1118	Failure	Work order cluster has an invalid state
-1120	Failure	Work order cluster -> Type is invalid
-1122	Failure	Work order cluster not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.27 mdataGetPartData

16.27.1 Function description

This function is used for querying part master data.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetPartData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] partFilter
    String key
    String value
in String[] partDataResultKeys
out String[] partDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `partFilter` and `partDataResultKeys` as well as parameter (out) `partDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☐ int
PART_DESC	Part description in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PART_GROUP	Part group in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PART_GROUP_TYPE	Part group type [B; E; F; K; L; M; W; S]: B = Unit E = Equipment F = Tool feeder K = Component L = Printed circuit board M = Material work plan W = Master work plan S = Lot Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 16-115 "Key Value Pair": `partFilter`

Definition of the output parameters for the found parts

Key	Explanation		IMSApi data type
ADDITIONAL_CONSUMPTION_ABSOLUTE	Details about the absolute additional use in units of "pieces" Advice: This is the additional value in the context of the retrograde booking and for the calculation in scheduling (MRP module)!	☐	double
ADDITIONAL_CONSUMPTION_PERCENT	Details about the relative additional use as a % Advice: This is the additional value in the context of the retrograde booking and for the calculation in scheduling (MRP module)!	☐	double
APS_TRANSFER	Indicates whether the part is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0;1]: 0 = Part is NOT taken into consideration (FALSE) 1 = Part is taken into consideration and is therefore "APS-relevant" (TRUE) Background: This specification is evaluated if, during work order detailed planning, the material availability is also to be taken into consideration for the parts required for work order production. If the part is "APS-relevant", all stock in store – i.e. all containers of this part – is always taken into consideration during material availability planning. Advice: Part specification can be overridden for individual containers. Incidentally, this applies for both directions: a.) If the part is "APS-relevant", individual containers can be excluded from the material availability planning; b.) If the the part is not "APS-relevant", it is still possible to activate individual containers for the material availability planning.	☐	boolean
ATTRIBUTE_1	"Attribute 1" free text box from the part master	☐	String
ATTRIBUTE_2	"Attribute 2" free text box from the part master	☐	String
ATTRIBUTE_3	"Attribute 3" free text box from the part master	☐	String
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.)	☐	String
DRAWING_NUMBER	Technical documentation number	☐	String
INFO	Information text from the part master ("MRP" tab, "Info text" field)	☐	String
PART_COST	Average purchase price [€] of the material (see PART_COST_BASE)	☐	double
PART_COST_BASE	Material quantity on which the purchase price is based (see PART_COST)	☐	double
PART_DESC	Part description in the iTAC system	☐	String
PART_GROUP	Part group in the iTAC system	☐	String
PART_GROUP_DESC	Part group description in the iTAC system	☐	String
PART_GROUP_TYPE	Part group type [B; E; F; K; L; M; W; S]: B = Unit E = Equipment F = Tool feeder K = Component L = Printed circuit board M = Material work plan W = Master work plan S = Lot	☐	String

Key	Explanation	IMSApi data type
PART_INDEX	Parts index for customer-side version documentation	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PART_TYPE_DESC	Part type description (see PART_TYPE_NUMBER)	☐ String
PART_TYPE_NUMBER	Part type; the part type enables product-specific typecasting and can be used individually. The part type has no control function; it can, however, be evaluated indirectly for customer-specific queries (e.g. first batch order). Advice: Parts types are created in the myiTAC workplace under the master data!	☐ String
PRODUCT_FLAG	Designator for raw material / product[0; 1]: 0 = Raw material (component) 1 = Product (unit) Advice: This value corresponds to the product designator from the order BOM (cf. "Product" field for the BOM components in the BOM module). Note here that this value does not necessarily need to match the product designator from the part master!	☐ int
QUANTITY_MULTIPLE_BOARD	Number of single panels per unit-array	☐ int
QUANTITY_TRANSPORT	Number of parts per transport unit	☐ double
SETUP_FLAG	"Monitoring of material setup" designation (0; 1): 0 = Monitoring is active 1 = Monitoring is inactive Advice: This parameter corresponds to the "Setup" field from the part master!	☐ int
STORAGE_DESC_DEFAULT	Storage place description of the default storage in the iTAC system	☐ String
STORAGE_NUMBER_DEFAULT	Storage place number of the default storage in the iTAC system Advice: The storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
UNIT	Quantity unit	☐ String
WEIGHT	Weight (value and unit)	☐ double

Tab. 16-116 "Key Value Pair": *partDataResultKeys*

16.27.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": partFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": partFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
partDataResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": partDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>partDataResultValues</code> !

Tab. 16-117 Parameter (in) »mdataGetPartData«

Parameter (out)

Parameter name	Explanation
partDataResult-Values	Depending on parameter (in) <code>partDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>partDataResultKeys</code> !

Tab. 16-118 Parameter (out) »mdataGetPartData«

16.27.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-123	Failure	Part group type not found
-129	Failure	Part description not found
-136	Failure	Customer part not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.28

mdataGetProductInfo

16.28.1

Function description

This function returns product information on a passed part number. This information can only be requested for the passed part number or for the underlying structure levels. In doing so, it is possible to select whether only one additional level with all information is resolved, or if all structures are.



Advice

This API function is influenced by site parameter code no. 18600 [▶▶▶] (maximum value for the number of result records). Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetProductInfo
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String partNumber
  in int bomVersion
  in String bomIndex
  in int processVersion
  in int desolvingFlag
  in int onlyReleased
  in int onlyValid
  in String[] productInfoResultKeys
  out String[] productInfoResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) productInfoResultKeys and (out) productInfoResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐.

Key	Explanation	IMSApi data type
BOM_DATE_CHANGED	Point in time of the last BOM change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_INFO	Individual BOM information (free text) Advice: In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!	☐ String
BOM_STATE	BOM state [0; 1; 2; 3; 4]: 0 = Released 1 = Locked externally 2 = Self-locked by logged-in user 3 = Not released 4 = Partially released]	☐ int
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
BOM_VERSION_VALID_FROM	Start of the validity period for the BOM version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
BOM_VERSION_VALID_TO	End of the validity period for the BOM version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
CAD_PART_DESC	Part description of the fundamental bearer	☐ String
CAD_PART_NUMBER	Part no. of the fundamental bearer, e.g., number of the "bare" printed circuit board	☐ String
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.)	☐ String
LEVEL	Structure level relative to the passed part number [...; -2; -1; 0; 1; 2; ...]; a few examples: -5 = 5 structure levels below the passed part -1 = 1 structure level below the passed part 0 = Structure level of the passed part 2 = 2 structure levels above the passed part	☐ int
LEVEL_VALID	Check of the validity period for the BOM of the corresponding structure level [0; 1]: 0 = Structure level (see LEVEL) is invalid 1 = Structure level (see LEVEL) is valid]	☐ int
PART_DESC	Part description in the iTAC system	☐ String

Key	Explanation	IMSApi data type
PART_GROUP	Part group in the iTAC system	☐ String
PART_GROUP_DESC	Part group description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PROCESS_VERSION	Process version from the iTAC system	☐ int
PROCESS_VERSION_VALID_FROM	Start of the validity period for the process version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PROCESS_VERSION_VALID_TO	End of the validity period for the process version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Tab. 16-119 "Key Value Pair": *productInfoResultKeys*

16.28.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partNumber	Part number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!
bomVersion	BOM version from the iTAC system Special case: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Special case: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!
processVersion	Process version of the BOM from the iTAC system Special case: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!

Parameter name	Explanation
desolvingFlag	Defines the scope of the product information [0; 1; 2; 3; 4; 5]: 0 = Only information on the passed part no. 1 = Information on the passed part no. and an underlying structure level according to the BOM 2 = Information on the passed part no. and all structure levels below the part no. 3 = Information on the passed part no. and an overlying structure level according to the BOM 4 = Information on the passed part no. and all structure levels above the part no. 5 = Information on the passed part no. and all overlying and underlying structure levels Advice: When interpreting the output values, note the LEVEL parameter (see the "Key Value Pair": productInfoResultKeys table). The corresponding structure level is displayed there relative to the passed part!
onlyReleased	Determines that only released BOMs are taken into account [0; 1]: 0 = All BOMs 1 = Only released BOMs
onlyValid	Determines that only BOMs with a valid time range (valid from/to) are taken into account [0; 1]: 0 = All BOMs 1 = Only valid BOMs
productInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": productInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) productInfoResultValues!

Tab. 16-120 Parameter (in) »mdataGetProductInfo«

Parameter (out)

Parameter name	Explanation
productInfoResultValues	Depending on parameter (in) productInfoResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) productInfoResultKeys!

Tab. 16-121 Parameter (out) »mdataGetProductInfo«

16.28.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.29 mdataGetProductsForLineOrStation

16.29.1 Function description

This function ascertains the products that can be manufactured on a specific station or line – i.e., for which it would be possible to create a work order. Both BOM information as well as work plan information can be output for the found products. The function only returns product information if the passed station or a station on the passed line appears in at least one of the ERP groups of the work plans that are to be searched.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetProductsForLineOrStation
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int productForLineOrStationFilter
    String key
    String value
in String[] bomResultKeys
out String[] bomResultValues
in String[] workplanResultKeys
out String[] workplanResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `productForLineOrStationFilter`, `bomResultKeys` and `workplanResultKeys` as well as (out) `bomResultValues` and `workplanResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNC_MODE	"Output" function mode [0; 1; 2]: 0 = BOM data and work plan data 1 = only BOM data (<code>bomResultKeys</code>) 2 = only work plan data (<code>workplanResultKeys</code>) Advice: If the key is not passed, no restriction occurs (as with mode = 0)!	int
LINE_NUMBER	Line numbers (T) Advice: Even though this key is not identified as a mandatory field, either the <code>LINE_NUMBER</code> or the <code>STATION_NUMBER</code> must be passed!	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	int
ONLY_VALID	"Product" validity [0; 1]: 0 = including invalid parts lists/work plans 1 = only valid parts lists/work plans	int
PART_GROUP	Part group in the iTAC system	String
PART_NUMBER	Part number in the iTAC system	String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the <code>LINE_NUMBER</code> or the <code>STATION_NUMBER</code> must be passed!	String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	String

Tab. 16-122 "Key Value Pair": `productForLineOrStationFilter`

Definition of output parameters for BOM data

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_INFO	Individual BOM information (free text) Advice: In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!	☐	String
BOM_VARIANT	BOM type [F; I; M; R; U; S; X]: F = BOM for production order I = BOM for receiving M = BOM for maintenance order R = BOM for repair order U = BOM for modification order S = BOM for external products X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
BOM_VERSION_VALID_FROM	Start of the validity period for the BOM version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
BOM_VERSION_VALID_TO	End of the validity period for the BOM version; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
CUSTOMER_PART_NUMBER	Customer-specific part no. (customer mat. no.)	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PROCESS_VERSION	Process version from the iTAC system	☐	int

Tab. 16-123 "Key Value Pair": *bomResultKeys*

Definition of output parameters for work plan data

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PRODUCTION_CODE	Abbreviation for the process type from the work plan (currently has no use; do not confuse parameter with the process version of the BOM)	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!	☐ String
WORKPLAN_BLOCKED	If the work plan is locked the user who has checked out the work plan is displayed.	☐ String
WORKPLAN_DESC	Work plan description in the iTAC system	☐ String
WORKPLAN_INFO	Individual work plan information (free text)	☐ String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKPLAN_VALID_FROM	Start of the validity period for the work plan; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
WORKPLAN_VALID_TO	End of the validity period for the work plan; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: This key is only evaluated if WORKPLAN_VERS is not used!	☐ String

Tab. 16-124 "Key Value Pair": workplanResultKeys

16.29.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
productForLineOrStationFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": productForLineOrStationFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": productForLineOrStationFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
bomResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": bomResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>bomResultValues</code> !
workplanResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workplanResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workplanResultValues</code> !

Tab. 16-125 Parameter (in) »mdataGetProductsForLineOrStation «

Parameter (out)

Parameter name	Explanation
bomResultValues	Depending on parameter (in) <code>bomResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>bomResultKeys</code> !
workplanResultValues	Depending on parameter (in) <code>workplanResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>workplanResultKeys</code> !

Tab. 16-126 Parameter (out) »mdataGetProductsForLineOrStation «

16.29.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-122	Failure	Part group not found
-168	Failure	Invalid value for the function mode (control flag)
-2038	Failure	Station does not belong to line
-2039	Failure	Part does not belong to the part group
-2040	Failure	Work plan -> Type is invalid
-2041	Failure	Work plan -> Alternative does not match the work plan type
-2042	Failure	Work plan -> Alternative not valid

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.30

mdataGetShiftData

16.30.1

Function description

This function is used to query shift information for the shift-capable data objects in the iTAC system; a wide range of filtering criteria can be applied here (e.g. data object details, productive identifier, validity period, etc.). If required, multiple values (`shiftFilterValues`) can be passed for individual search criteria (`shiftFilterKeys`) (OR link). If different search criteria are used within a "Key Value Pair" structure, the respective values are "AND"-linked so that only the data that meets all used filter conditions is returned.



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Function matrix *ENTITY_TYPE* (shift)

The dependencies between the keys *ENTITY_TYPE* (data object) and *ENTITY_KEY* (data object details) are shown in the following function matrix. Depending in the desired data object, this ALWAYS includes a matching data object detail specification (e.g. name, number).

Data objects /	ENTITY_TYPE	ENTITY_KEY
Sites	1	Site number
Cells	2	Cell number
Lines	3	Line number
Machine groups	4	Machine group number
Stations	5	Station number
Equipment groups	6	Equipment group number
Production means	7	Number of the production means with index – delimited by the " " (pipe) character; depending on the PM class, this is then equipment or a carrier. Background to "PM class": By default, the value E "Equipment (work step)" is used for the "PM class"; if a different PM class is used (currently A "Equipment (asset)", M "Cavity" or C "Carrier"), this must then be added using a second delimiter. Syntax: Number Index Class
Teams	8	Team number
User	9	User identification
User groups	10	User group number

Tab. 16-127 Function matrix *ENTITY_TYPE*

Licensing

In order for this API function to be used, the executing station must be licensed as follows

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetShiftData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] shiftFilterKeys
in String[] shiftFilterValues
in String[] shiftResultKeys
out String[] shiftResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `shiftFilterKeys`, `shiftFilterValues` and `shiftResultKeys` as well as (out) `shiftResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: The value [-1] is passed here and for DATE_TO; only the current shift is considered. If the value [-1] is passed here only, the one-year review period applies: "DATE_TO minus one year".	☒ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: The value [-1] is passed here and for DATE_FROM; only the current shift is considered. If the value [-1] is passed here only, the one-year review period applies: "DATE_FROM plus one year".	☒ DateMs
ENTITY_KEY	Depending on the selected data object (see ENTITY_TYPE), the desired data object detail specification (e.g. the concrete machine group number) is passed here. Advice: For further information, refer to the function matrix above in the section »Function description« [>>>] !	☒ String

Key	Explanation	IMSApi data type
ENTITY_TYPE	Determines the data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☒ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int
ONLY_PRODUCTIVE	"Productive" mode [0; 1]: 0 = Productive and unproductive times 1 = Only productive times Advice: <i>If this key is not passed along, the [0] value is used by default!</i>	☐ boolean
OVERLAPPING	"Overlapping" mode [0; 1]: 0 = No overlapping 1 = With overlapping (consolidated) Advice: <i>If this key is not passed along, the [0] value is used by default!</i> Background: In mode [0] (no overlapping), all time events are output in a precise, chronological sequence. In mode [1], identical time events are combined (consolidated). Example: <i>If the specified break time of 60 minutes for a day is split into two breaks (15 / 45 minutes), in mode [0] both time events are output in separate records. In mode [1], only one record with the break length of 60 minutes is output.</i>	☐ boolean

Tab. 16-128 "Key Value Pair": *shiftFilterKeys*

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
DURATION	Duration of the time event in minutes	☐ int
END_DATE	Time stamp for the end of the time event; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☒ DateMs
ENTITY_KEY	Depending on the data object (see ENTITY_TYPE), the data object detail specification (e.g. the concrete machine group number) is output here. Advice: For further information, refer to the function matrix above in the section »Function description« [▶▶▶]!	☒ String
ENTITY_TYPE	Data object [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Stations 6 = Equipment groups 7 = Production means (equipment / carrier) 8 = Teams 9 = Users 10 = User groups	☒ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
OBJECT_BASE	Data object from which the shift model for the current entry was derived. Possible derivations for station : 1 = Sites 2 = Cells 3 = Lines 4 = Machine groups 5 = Station (NO derivation) Possible derivations for production means : 6 = Equipment groups 7 = Production means (NO derivation) Possible derivations for user : 8 = Teams 9 = Users (NO derivation) 10 = User groups	☒ int
OBJECT_NAME	Data object details for the data object according to the OBJECT_BASE key	☐ String
PRODUCTIVE	Productive identifier [0; 1]: 0 = Unproductive time event 1 = Productive time event	☐ int
SHIFT_MODEL_DESC	Description of the shift model	☐ String
SHIFT_MODEL_NAME	Name of the shift model	☐ String
SPECIAL_TIME	"Special time" designation [0; 1]: 0 = Not a special time 1 = Special time	☐ int

Key	Explanation	IMSApi data type
SPECIAL_TIME_DESC	Description of the special time	☐ String
START_DATE	Time stamp for the start of the time event; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☒ DateMs
TIME_EVENT_DESC	Description of the time event	☐ String
TIME_EVENT_NAME	Name of the time event	☐ String
TIME_EVENT_TYPE	Time event type [0; 1; 2; 3]: 0 = Undefined 1 = Early shift 2 = Late shift 3 = Night shift	☐ int
WEEK_DAY	Day of week [0 - 6] Advice: [0] corresponds to the 1st day of the week as defined in the system configuration!	☐ int
WEEK_SCHEDULE_DESC	Description of the week plan	☐ String
WEEK_SCHEDULE_NAME	Name of the week plan	☐ String

Tab. 16-129 "Key Value Pair": *shiftResultKeys*

16.30.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
shiftFilterKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <i>shiftFilterValues</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": shiftFilterKeys table.
shiftFilterValues	Value list for the parameters passed by means of <i>shiftFilterKeys</i> . Advice: Refer to the "Key Value Pair": shiftFilterKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
shiftResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": shiftResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>shiftResultValues</i> !

Tab. 16-130 Parameter (in) »mdataGetShiftData«

Parameter (out)

Parameter name	Explanation
shiftResultValues	Depending on parameter (in) <code>shiftResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>shiftResultKeys</code> !

Tab. 16-131 Parameter (out) »mdataGetShiftData«

16.30.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key

-174	Failure	Number of records to be output has fallen below the minimum value
-1128	Failure	Shift schedule -> Start date lies after end date
-1301	Failure	Production means not found
-1319	Failure	Equipment group not found
-2015	Failure	Date not valid
-2043	Failure	Site not found
-2048	Failure	Cell not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value
-2131	Failure	Team not found
-2132	Failure	User not found
-2133	Failure	User group not found
-2134	Failure	Entity type is invalid
-2135	Failure	Entity key is invalid

16.31 mdataGetShiftModel

16.31.1 Function description

This function is used to query shift model information from the iTAC system; different filter criteria can be applied (at present possible criteria are the active/inactive designator and the content of the information fields). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetShiftModel
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] shiftModelFilter
    String key
    String value
in String[] shiftModelHeadKeys
in String[] shiftModelItemKeys
out String[] shiftModelHeadValues
out String[] shiftModelItemValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `shiftModelFilter`, `shiftModelHeadKeys` and `shiftModelItemKeys` as well as (out) `shiftModelHeadValues` and `shiftModelItemValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVE	"Shift model" activation state [0; 1]: 0 = Only inactive shift models 1 = Only active shift models Advice: <i>If the key is not passed, there is no restriction; all shift models are taken into consideration!</i>	☐ int
INFO_TXT_1	Individual additional information "Info 1" of the shift model Advice: <i>The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!</i>	☐ String
INFO_TXT_2	Individual additional information "Info 2" of the shift model Advice: <i>See the advice notice under INFO_TXT_1!</i>	☐ String
INFO_TXT_3	Individual additional information "Info 3" of the shift model Advice: <i>See the advice notice under INFO_TXT_1!</i>	☐ String
INFO_TXT_4	Individual additional information "Info 4" of the shift model Advice: <i>See the advice notice under INFO_TXT_1!</i>	☐ String
INFO_TXT_5	Individual additional information "Info 5" of the shift model Advice: <i>See the advice notice under INFO_TXT_1!</i>	☐ String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int
SHIFT_MODEL_NAME	Name of the shift model Advice: <i>The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!</i>	☐ String

Tab. 16-132 "Key Value Pair": *shiftModelFilter*

Definition of the output parameters for the found shift model header data

Key	Explanation	IMSApi data type
ACTIVE	"Shift model" activation state [0; 1]: 0 = Shift model is NOT active 1 = Shift model is active	☐ int
INFO_TXT_1	Individual additional information "Info 1" of the shift model	☐ String
INFO_TXT_2	Individual additional information "Info 2" of the shift model	☐ String
INFO_TXT_3	Individual additional information "Info 3" of the shift model	☐ String
INFO_TXT_4	Individual additional information "Info 4" of the shift model	☐ String
INFO_TXT_5	Individual additional information "Info 5" of the shift model	☐ String
SHIFT_MODEL_DESC	Description of the shift model	☐ String
SHIFT_MODEL_ID	Unique designator of the shift model Advice: The records "shift model header data" and "week plans" are put into context on the basis of this designator (see the key of the same name in the "Key Value Pair": shiftModelItemKeys table)!	☐ long
SHIFT_MODEL_NAME	Name of the shift model	☐ String

Tab. 16-133 "Key Value Pair": *shiftModelHeadKeys*

Definition of output parameters for the associated week plans

Key	Explanation	IMSApi data type
SHIFT_MODEL_ID	Unique designator of the shift model Advice: The records "shift model header data" and "week plans" are put into context on the basis of this designator (see the key of the same name in the "Key Value Pair": shiftModelHeadKeys table)!	☐ int
SHIFT_MODEL_NAME	Name of the shift model	☐ String
WEEK_SCHEDULE_ACTIVE	"Week plan" activation state in the shift model [0; 1]: 0 = Week plan is NOT active 1 = Week plan is active	☐ int
WEEK_SCHEDULE_ID	Unique designator of the week plan	☐ String
WEEK_SCHEDULE_INDEX	Unique number for the week plan within the shift model	☐ int
WEEK_SCHEDULE_NAME	Name of the week plan	☐ String

Tab. 16-134 "Key Value Pair": *shiftModelItemKeys*

16.31.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
shiftModelFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": shiftModelFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": shiftModelFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
shiftModelHeadKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": shiftModelHeadKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>shiftModelHeadValues</code> !
shiftModelItemKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": shiftModelItemKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>shiftModelItemValues</code> !

Tab. 16-135 Parameter (in) »mdataGetShiftModel«

Parameter (out)

Parameter name	Explanation
shiftModelHeadValues	Depending on parameter (in) <code>shiftModelHeadKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>shiftModelHeadKeys</code> !
shiftModelItemValues	Depending on parameter (in) <code>shiftModelItemKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>shiftModelItemKeys</code> !

Tab. 16-136 Parameter (out) »mdataGetShiftModel«

16.31.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.32 mdataGetStationConfiguration

16.32.1 Function description

This function allows the current settings for a station to be read out from the station setup. Settings from the areas **Setup** and **Upload settings**, current **license information** as well as the values for certain **station parameters** (code no.) can be queried.

For the item "station parameters", it is also possible to enter a code number range so that several relevant values can be determined for a station with one function call. The "Key Value Pair" structure `stationConfigFilter` is used for this purpose. The following table shows how the input parameters (Keys) `CODE_NUMBER` and `CODE_NUMBER_TO` that are stored there can be combined:

CODE_NUMBER	CODE_NUMBER_TO	Result
[empty]	[empty]	All
n	[empty]	n
n	m	n - m
[empty]	m	0 - m

Tab. 16-137 Possible combinations of `CODE_NUMBER` and `CODE_NUMBER_TO`

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

TR license (active) OR TR license (passive) OR PM license

Function declaration

```
int mdataGetStationConfiguration
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
KeyValue[] stationConfigFilter
    String key
    String value
in String[] stationConfigCodeResultKeys
out String[] stationConfigCodeResultValues
in String[] stationParamResultKeys
out String[] stationParamResultValues
in String[] stationLicenseResultKeys
out String[] stationLicenseResultValues
```

"Key Value Pair" structures

This function contains several "Key Value Pair" structures (see the parameters (in) `stationConfigFilter`, `stationConfigCodeResultKeys`, `stationParamResultKeys` and `stationLicenseResultKeys` and (out) `stationConfigCodeResultValues`, `stationParamResultValues` and `stationLicenseResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific **station parameters**

Key	Explanation	IMSApi data type
CODE_NUMBER	Code no. of the station parameter Advice: Refer to the possible combinations with the key CODE_NUMBER_TO that are listed in the general function description!	☐ int
CODE_NUMBER_TO	Code no. of the next station parameter (range selection) Advice: Refer to the possible combinations with the key CODE_NUMBER that are listed in the general function description!	☐ int
STATION_TYPE	Station type [T; M]: T = Station for tasks in the TR context M = Station for tasks in the PM context Advice: With the TR/PM consolidation implemented in release 9.01, a distinction according to station type is only relevant in exceptional cases; this key can be ignored in the standard!	☐ String

Tab. 16-138 "Key Value Pair": *stationConfigFilter*

Definition of the output parameters for the found **station parameters**

Key	Explanation	IMSApi data type
CODE_DESC	Textual description of the station parameter	☐ String
CODE_NUMBER	Code no. of the station parameter	☐ int
CODE_VALUE	Value of the station parameter	☐ String
DATE_CHANGED	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
IS_USED	Verification of use [0; 1]: 0 = Parameter is NOT active (FALSE) 1 = Parameter is active (TRUE)	☐ int

Tab. 16-139 "Key Value Pair": *stationConfigCodeResultKeys*

Definition of the output parameters for **Setup** and **Upload settings**:

Key	Explanation	IMSApi data type
COST_CENTER_DESC	Cost center description	☐ String
COST_CENTER_NUMBER	Cost center number	☐ String
STATION_ACTIVE	"Setup" tab -> "Active" field [0; 1]: 0 = Station is NOT active (FALSE) 1 = Station is active (TRUE)	☐ boolean
STATION_DISPLAY_YIELD	"Upload settings" tab -> "Display yield" field [0; 1]: 0 = Yield display (OPM/A module) is NOT active 1 = Yield display (OPM/A module) is active	☐ int
STATION_FAILURE_UNIQUE	"Upload settings" tab -> "Uniqueness of failure" field [0; 1]: 0 = Multiple failure possible 1 = Multiple failure NOT possible Background: Shows whether a failure record for a product and the respective station may occur only once, or if each failure record is accepted. This key is defined for the machine group.	☐ int
STATION_MAX_RECURSION	"Upload settings" tab -> "Max. recursion" field: Maximum number of bookings for a work step on a station; if the value is exceeded, secondary bookings are rejected.	☐ int
STATION_MEASUREMENT_TO_ATTRIBUTIVE	"Upload settings" tab -> "Convert measurement data" field [0; 1]: 0 = Measurement data conversion is NOT active 1 = Measurement data conversion is active Background: Determines whether the measurement data imported into the iTAC system are stored without evaluation (i.e. in protocol form), or if the contents of the measurement data are to be interpreted and, if necessary, attributive failures are to be generated from the measurement data information.	☐ int
STATION_ORDER_GENERATION_MODE	"Setup" tab -> "Order generation" field [0; 1; 2]: 0 = Order generation is NOT active 1 = Order generation is active (production orders) 2 = Order generation is active (repair orders) Background: Specifies whether the station is permitted to independently generate production or repair work orders.	☐ int
STATION_UPLOAD_FAILURE_CHECK	"Upload settings" tab -> "Failure upload check" field [0; 1; 2]: 0 = Load failure without check 1 = Check BOM 2 = Check BOM and placement Background: Determines the program behavior which dictates how intensively reported failures (ICT) are to be checked.	☐ int
STATION_UPLOAD_PART_NO	"Upload settings" tab -> "Part no. for upload" field [0; 1]: 0 = Upload with iTAC part number 1 = Upload with customer-specific part number Background: Determines whether the iTAC part number or the customer-specific part number (Customer mat. no.) is used in the output data.	☐ int
STATION_UPLOAD_STATE_MODE	"Upload settings" tab -> "Upload state mode" field [0; 1; 2]: 0 = Determine the state using the failure data 1 = Use passed state from upload client 2 = Accept state in the failure data Background: Determines the mode for the interpretation and use of the passed failure state information.	☐ int

Tab. 16-140 "Key Value Pair": *stationParamResultKeys*

Definition of the output parameters for **license information**:

Key	Explanation	IMSApi data type
HAS_APS_LICENSE	Checking of APS licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_CAO_LICENSE	Checking of QM licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_EMI_LICENSE	Checking of EMI licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_MESDEVICE_LICENSE	Checking of MESDevice licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_ML_LICENSE	Checking of ML licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_RECEIVINGDEVICE_LICENSE	Checking of ReceivingDevice licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_RM_LICENSE	Checking of PM licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean
HAS_TR_LICENSE	Checking of TR licensing [0; 1]: 0 = NO valid license (FALSE) 1 = Valid license (TRUE)	☐ boolean

Tab. 16-141 "Key Value Pair": *stationLicenseResultKeys*

16.32.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationConfigFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": stationConfigFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": stationConfigFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
stationConfigCodeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationConfigCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationConfigCodeResultValues !
stationParamResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationParamResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationParamResultValues !
stationLicenseResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationLicenseResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationLicenseResultValues !

Tab. 16-142 Parameter (in) »mdataGetStationConfiguration«

Parameter (out)

Parameter name	Explanation
stationConfigCodeResultValues	Depending on parameter (in) stationConfigCodeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationConfigCodeResultKeys !
stationParamResultValues	Depending on parameter (in) stationParamResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationParamResultKeys !
stationLicenseResultValues	Depending on parameter (in) stationLicenseResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationLicenseResultKeys !

Tab. 16-143 Parameter (out) »mdataGetStationConfiguration«

16.32.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.33 mdataGetTimeEvent

16.33.1 Function description

This function is used to query time event information from the iTAC system; the information can be filtered for productive and unproductive time events.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetTimeEvent
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] timeEventFilter
    String key
    String value
  in String[] timeEventResultKeys
  out String[] timeEventResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) timeEventFilter and timeEventResultKeys as well as (out) timeEventResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ String
PRODUCTIVE	Productive identifier [0; 1]: 0 = Unproductive time event 1 = Productive time event Advice: <i>If the key is not passed, there is no restriction; all time events are taken into consideration!</i>	☐ String

Tab. 16-144 "Key Value Pair": *timeEventFilter*

Definition of the output parameters for the found time events

Key	Explanation	IMSApi data type
PRODUCTIVE	Productive identifier [0; 1]: 0 = Unproductive time event 1 = Productive time event	☐ String
TIME_EVENT_CODE	Designator of the time event	☐ String
TIME_EVENT_DESC	Description of the time event	☐ String
TIME_EVENT_ID	Unique designator of the time event	☐ String
TIME_EVENT_NAME	Name of the time event	☐ String
TIME_EVENT_TYPE	Time event type [0; 1; 2; 3] 0 = Undefined 1 = Early shift 2 = Late shift 3 = Night shift	☐ String

Tab. 16-145 "Key Value Pair": *timeEventResultKeys*

16.33.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
timeEventFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": timeEventFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": timeEventFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
timeEventResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": timeEventResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>timeEventResultValues</code> !

Tab. 16-146 Parameter (in) »mdataGetTimeEvent«

Parameter (out)

Parameter name	Explanation
timeEventResultValues	Depending on parameter (in) <code>timeEventResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>timeEventResultKeys</code> !

Tab. 16-147 Parameter (out) »mdataGetTimeEvent«

16.33.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.34 mdataGetUserData

16.34.1 Function description

This function makes it possible to query staff master data from the iTAC System; a wide range of different filtering criteria can be applied here (e.g. name, team membership, cost center). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetUserData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mdataGetUserDataFilter
    String key
    String value
in String[] mdataGetUserDataKeys
out String[] mdataGetUserDataValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mdataGetUserDataFilter and mdataGetUserDataKeys as well as (out) mdataGetUserDataValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
COST_CENTER_NUMBER	Cost center number Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
EXTERNAL_USER_NUMBER	Personnel number in the external system	☐	String
GENDER	Gender of the person [D; F; M; N]: D = Diverse F = Female M = Male N = Not specified	☐	String
LOCKED	Lock state "User account" [-1; 0; 1]: -1 = Locked and not locked 0 = Not locked 1 = Locked Advice: If this key is not passed along, the [0] value is used by default!	☐	int
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
TEAM_NUMBER	Team number Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
TESTER_NUMBER	Tester number Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_FIRST_NAME	First name of the person Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_LAST_NAME	Name of the person Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_NAME	Login name of the user (user identification) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String

Key	Explanation	IMSApi data type
USER_PASSWORD	Password of the user Advice: This key can only be used for the application type "Querying own personal data"; for this purpose, it is essential that the associated user identification (USER_NAME) is also passed!	☐ String
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: This key can only be used for the application type "Querying data of a person"; the query can only be executed by the person in question or by an authorized administrator – the latter must have access to the specific user tokens!	☐ String

Tab. 16-148 "Key Value Pair": *mdataGetUserDataFilter*

Definition of the output parameters for the found staff data

Key	Explanation	IMSApi data type
COST_CENTER_ACTIVE	"Cost center" activation designator [0; 1]: 0 = Not active 1 = Active	☐ int
COST_CENTER_DESC	Cost center description	☐ String
COST_CENTER_NUMBER	Cost center number	☐ String
EXTERNAL_USER_NUMBER	Personnel number in the external system	☐ String
GENDER	Gender of the person [D; F; M; N]: D = Diverse F = Female M = Male N = Not specified	☐ String
LOCKED	Lock state "User account" [0; 1]: 0 = Not locked 1 = Locked	☐ int
TEAM_ACTIVE	"Team" activation designator [0; 1]: 0 = Not active 1 = Active	☐ int
TEAM_CODE	Team designation [A; C; P]: A = Administration C = Controlling P = Production	☐ String
TEAM_DESC	Team description	☐ String
TEAM_NUMBER	Team number	☐ String
TESTER_NUMBER	Tester number	☐ String
USER_FIRST_NAME	First name of the person	☐ String
USER_LAST_NAME	Name of the person	☐ String
USER_NAME	Login name of the user (user identification)	☐ String

Tab. 16-149 "Key Value Pair": *mdataGetUserDataKeys*

16.34.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdataGetUserData-Filter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetUserDataFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetUserDataFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdataGetUserDataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetUserDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mdataGetUserDataValues</code> !

Tab. 16-150 Parameter (in) »mdataGetUserData«

Parameter (out)

Parameter name	Explanation
mdataGetUserDataValues	Depending on parameter (in) <code>mdataGetUserDataKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mdataGetUserDataKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-151 Parameter (out) »mdataGetUserData«

16.34.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.35 mdataGetUserGroupData

16.35.1 Function description

This function enables user group information to be queried from the iTAC system; ONLY the user groups to which persons from the master data have already been assigned are considered here. The query can be filtered according to a wide range of different criteria (e.g. person name, user group name, user group type, etc.). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetUserGroupData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mdataGetUserGroupDataFilter
    String key
    String value
in String[] mdataGetUserGroupDataKeys
out String[] mdataGetUserGroupDataValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mdataGetUserGroupDataFilter and mdataGetUserGroupDataKeys as well as (out) mdataGetUserGroupDataValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
USER_FIRST_NAME	First name of the person Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
USER_GROUP_NAME	Name of the user group Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
USER_GROUP_STATE	User group state [0; 1; 2]: 0 = active 1 = inactive 2 = predefined Special case: If the key is not passed or passed with [-1], the parameter is not evaluated (no restriction)!	☐ String
USER_GROUP_TYPE	User group type [0; 1; 2]: 0 = Configuration group 1 = AMC authorization group 2 = iTAC.BI authorization group Special case: If the key is not passed or passed with [-1], the parameter is not evaluated (no restriction)!	☐ String
USER_LAST_NAME	Name of the person Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
USER_NAME	Login name of the user (user identification) Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
USER_NUMBER	Personnel number in the iTAC system Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String

Tab. 16-152 "Key Value Pair": `mdataGetUserGroupDataFilter`

Definition of the output parameters for the found user groups

Key	Explanation	IMSApi data type
DATE_CHANGED	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_CREATED	Date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
USER_FIRST_NAME	First name of the person	☐ String
USER_GROUP_DESCRIPTION	Description of the user group	☐ String
USER_GROUP_NAME	Name of the user group	☐ String
USER_GROUP_STATE	User group state [0; 1; 2]: 0 = active 1 = inactive 2 = predefined	☐ String
USER_GROUP_TYPE	User group type [0; 1; 2]: 0 = Configuration group 1 = AMC authorization group 2 = iTAC.BI authorization group	☐ String
USER_LAST_NAME	Name of the person	☐ String
USER_NAME	Login name of the user (user identification)	☐ String
USER_NUMBER	Personnel number in the iTAC system	☐ String

Tab. 16-153 "Key Value Pair": *mdataGetUserGroupDataKeys*

16.35.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
mdataGetUser-GroupDataFilter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetUserGroupDataFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetUserGroupDataFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdataGetUser-GroupDataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetUserGroupDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdataGetUserGroupDataValues !

Tab. 16-154 Parameter (in) »mdataGetUserGroupData«

Parameter (out)

Parameter name	Explanation
mdataGetUser-GroupDataValues	Depending on parameter (in) mdataGetUserGroupDataKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdataGetUserGroupDataKeys !

Tab. 16-155 Parameter (out) »mdataGetUserGroupData«

16.35.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.36 mdataGetUserSkill

16.36.1 Function description

This function makes it possible to query staff master data with the stored user skills from the iTAC system; a wide range of different filtering criteria can be applied here (e.g. name, token, user skill). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetUserSkill
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] userSkillFilter
    String key
    String value
in String[] userSkillResultKeys
out String[] userSkillResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `userSkillFilter` and `userSkillResultKeys` as well as (out) `userSkillResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
EXTERNAL_USER_NUMBER	Personnel number in the external system Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
FUNC_MODE	Function mode for the output [0; 1; 2]: 0 = user 1 = user group 2 = user and user group	☐	int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [1000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int
ONLY_VALID	"User" validity [0; 1]: 0 = including invalid users 1 = only valid users	☐	int
SKILL_DESC	Description of the user skill	☐	String
SKILL_LEVEL	Level of the user skill	☐	String
USER_FIRST_NAME	First name of the person Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_LAST_NAME	Name of the person Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_NAME	Login name of the user (user identification) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐	String
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: This key can be used as an alternative for identifying the user using USER_NAME!	☐	String

Tab. 16-156 "Key Value Pair": userSkillFilter

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
EXTERNAL_USER_NUM-BER	Personnel number in the external system	☐ String
LOCKED	Lock state "User account" [0; 1]: 0 = Not locked 1 = Locked	☐ int
SKILL_DESC	Description of the user skill	☐ String
SKILL_LEVEL	Level of the user skill	☐ String
USER_FIRST_NAME	First name of the person	☐ String
USER_GROUP_NAME	Name of the user group	☐ String
USER_LAST_NAME	Name of the person	☐ String
USER_NAME	Login name of the user (user identification)	☐ String
USER_SKILL_COMMENT	Comment on user skill	☐ String
VALID_FROM	Start of the validity period for the user; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
VALID_TO	End of the validity period for the user; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Tab. 16-157 "Key Value Pair": *userSkillResultKeys*

16.36.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
userSkillFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": userSkillFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": userSkillFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
userSkillResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": userSkillResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>userSkillResultValues</code> !

Tab. 16-158 Parameter (in) »mdataGetUserSkill«

Parameter (out)

Parameter name	Explanation
userSkillResultValues	Depending on parameter (in) <code>userSkillResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>userSkillResultKeys</code> !

Tab. 16-159 Parameter (out) »mdataGetUserSkill«

16.36.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.37 mdataGetWageType

16.37.1 Function description

This function makes it possible to query wage type information from the iTAC system; a wide range of different filtering criteria can be applied here (e.g. wage type groups, wage type categories or active/inactive designator). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int mdataGetWageType
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] mdataGetWageTypeFilter
    String key
    String value
  in String[] mdataGetWageTypeKeys
  out String[] mdataGetWageTypeValues
  out boolean hasMore

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mdataGetWageTypeFilter` and `mdataGetWageTypeKeys` as well as (out) `mdataGetWageTypeValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVE	"Wage types" activation state [0; 1]: 0 = Only inactive wage types 1 = Only active wage types Special case: If the key is passed with [-1], there is no restriction; active and inactive wage types are then reviewed! Advice: If this key is not passed along, the [1] value is used by default!	int
COST_TYPE_ERP	Cost type designator from a connected external system (this is usually an ERP system) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String
LOCKED_FOR_USER	"Employee booking" lock [0; 1]: 0 = Wage type NOT locked for bookings 1 = Wage type locked for bookings Advice: If the key is not passed or passed with [-1], the [0] value is used by default!	int
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [100] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
VALID_DATE	Validity time for the data to be ascertained; the wage type information is output that was valid at the time named here (history function); specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and time; this is used to query the current settings!	DateMs
WAGE_TYPE_CATEGORY	Wage type category [d; F; G; M]: d = Not defined (internal dummy entry) F = Manufacturing costs G = Overhead costs M = Cost overruns Advice: If the key is not passed or passed with [-1], there is no restriction; all cost center types are taken into consideration!	String
WAGE_TYPE_DESC	Description of the wage type Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	String

Key	Explanation	IMSApi data type
WAGE_TYPE_ERP	Wage type designator from a connected external system (this is usually an ERP system) Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String
WAGE_TYPE_GROUP	Name of the wage type group Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String

Tab. 16-160 "Key Value Pair": `mdataGetWageTypeFilter`

Definition of the output parameters for the found wage type

Key	Explanation	IMSApi data type
ACTIVE	"Wage types" activation state [0; 1]: 0 = wage type is NOT active 1 = wage type is active	☐ int
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
COST_TYPE_ERP	Cost type designator from a connected external system (this is usually an ERP system)	☐ String
COST_TYPE_EXTERNAL_SYSTEM	Cost type designator from an alternative external system	☐ String
CREATION_DATE	Time of creation for the wage type (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
INFO_TEXT	Individual information text	☐ String
LOCKED_FOR_USER	"Employee booking" lock [0; 1]: 0 = Wage type NOT locked for bookings 1 = Wage type locked for bookings	☐ int
OVERHEAD_COST_TYPE	Overhead cost type [T; O; E]: T = Technology O = Organization E = External Advice: This field has no control function; it is used only for grouping overhead-cost-based wage types!	☐ String
PERMIT_BY_TEAM_LEADER	"Team leader approval requirement" designator [0; 1]: 0 = Approval by team leader NOT required 1 = Approval by team leader required Advice: If the value [1] is returned here, overhead cost and cost overrun bookings performed on the basis of this wage type must be approved by the team leader of the booking station!	☐ int

Key	Explanation	IMSApi data type
USE_MASTER_COST_CENTER	"Master cost center" designator [0; 1]: 0 = Assignment of master cost center is NOT active 1 = Assignment of master cost center is active Advice: If the value [1] is returned here, the bookings for the wage type are assigned to the master cost center of the employee (according to the personnel master data); provided that the employee has not been indirectly assigned any other cost center via the wage type assignment function!	int
WAGE_COST_TYPE_ERP	Wage/cost type designator from a connected ERP system	String
WAGE_COST_TYPE_INTERNAL	Wage/cost type designator from the internal system	String
WAGE_TYPE_CATEGORY	Wage type category [d; F; G; M]: d = Not defined (internal dummy entry) F = Manufacturing costs G = Overhead costs M = Cost overruns	String
WAGE_TYPE_CATEGORY_DESC	Description of the wage type categories	String
WAGE_TYPE_DESC	Description of the wage type	String
WAGE_TYPE_ERP	Wage type designator from a connected external system (this is usually an ERP system)	String
WAGE_TYPE_EXTERNAL_SYSTEM	Wage type identification from an alternative external system	String
WAGE_TYPE_GROUP	Name of the wage type group	String
WAGE_TYPE_GROUP_DESC	Description of the wage type group	String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system	String

Tab. 16-161 "Key Value Pair": *mdataGetWageTypeKeys*

16.37.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdataGetWageType-Filter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetWageTypeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetWageTypeFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdataGetWageType-peKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetWageTypeKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdataGetWageTypeValues !

Tab. 16-162 Parameter (in) »mdataGetWageType«

Parameter (out)

Parameter name	Explanation
mdataGetWageType-Values	Depending on parameter (in) mdataGetWageTypeKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdataGetWageTypeKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-163 Parameter (out) »mdataGetWageType«

16.37.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.38 mdataGetWeekSchedule

16.38.1 Function description

This function is used to query week plan information; the information can be filtered for the active/inactive designator.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int mdataGetWeekSchedule
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] weekScheduleFilter
    String key
    String value
  in String[] weekScheduleHeadKeys
  in String[] weekScheduleItemKeys
  out String[] weekScheduleHeadValues
  out String[] weekScheduleItemValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `weekScheduleFilter`, `weekScheduleHeadKeys` and `weekScheduleItemKeys` as well as (out) `weekScheduleHeadValues` and `weekScheduleItemValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVE	"Week plan" activation state [0; 1]: 0 = Only inactive week plans 1 = Only active week plans Advice: <i>If the key is not passed, there is no restriction; all week plans are taken into consideration!</i>	☐ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int

Tab. 16-164 "Key Value Pair": *weekScheduleFilter*

Definition of the output parameters for the found week plan header data

Key	Explanation	IMSApi data type
ACTIVE	"Week plan" activation state [0; 1]: 0 = Week plan is NOT active 1 = Week plan is active	☐ int
COLOR_BACKGROUND	Background color	☐ int
COLOR_FONT	Font color	☐ int
WEEK_SCHEDULE_DESC	Description of the week plan	☐ String
WEEK_SCHEDULE_ID	Unique designator of the week plan	☐ long
WEEK_SCHEDULE_NAME	Name of the week plan	☐ String

Tab. 16-165 "Key Value Pair": *weekScheduleHeadKeys*

Definition of output parameters for the associated time events

Key	Explanation	IMSApi data type
DURATION	Duration of the time event in minutes	☐ int
END_DATE	Time stamp for the end of the time event; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: <i>1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000</i>	☐ DateMs
START_DATE	Time stamp for the start of the time event; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
TIME_EVENT_ID	Unique designator of the time event	☐ long
TIME_EVENT_NAME	Name of the time event	☐ String
WEEK_DAY	Day of week [0 - 6] Advice: <i>[0] corresponds to the 1st day of the week as defined in the system configuration!</i>	☐ int

Key	Explanation	IMSApi data type
WEEK_SCHEDULE_ID	Unique designator of the week plan	☐ long
WEEK_SCHEDULE_NAME	Name of the week plan	☐ String

Tab. 16-166 "Key Value Pair": *weekScheduleItemKeys*

16.38.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
weekScheduleFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <i>value</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": weekScheduleFilter table.
value	Value list for the parameters passed by means of <i>key</i> . Advice: Refer to the "Key Value Pair": weekScheduleFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
weekScheduleHeadKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": weekScheduleHeadKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>weekScheduleHeadValues</i> !
weekScheduleItemKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": weekScheduleItemKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>weekScheduleItemValues</i> !

Tab. 16-167 Parameter (in) »*mdataGetWeekSchedule*«

Parameter (out)

Parameter name	Explanation
weekScheduleHeadValues	Depending on parameter (in) <i>weekScheduleHeadKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>weekScheduleHeadKeys</i> !
weekScheduleItemValues	Depending on parameter (in) <i>weekScheduleItemKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>weekScheduleItemKeys</i> !

Tab. 16-168 Parameter (out) »*mdataGetWeekSchedule*«

16.38.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.39

mdataGetWorkorders

16.39.1

Function description

This function uses special search criteria to ascertain the production orders in the system. A wide range of search criteria are available; e.g. work order type, order state, part number, part group, date of creation or the order number itself. All search criteria are "AND" coupled such that only the work orders that meet all used filter conditions are returned.

**Advice**

*Work orders in the **R** (planned) state cannot be taken into consideration by this function; only with release of the order are the required order structures created in the system!*

If **order networks** are used in the iTAC system, the work orders that are at a subordinate level as per the order network can be ascertained directly using this function on the basis of a so-called PARENT work order number (see the key `PARENT_WORKORDER_NUMBER`). Similarly, the parent order number can also be ascertained using the subordinate work orders. This allows a multilevel order network to be resolved from the top as well as from the bottom (level by level). As with order networks, this function can also be used to ascertain **production lots** (partial orders) belonging to a basic order (see the key `SPLIT_PARENT_WORKORDER`).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetWorkorders
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] mdataGetWorkordersFilters
    String key
    String value
  in String[] mdataGetWorkordersResultKeys
  out String[] mdataGetWorkordersResultValues
  out boolean hasMore
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `mdataGetWorkordersFilters` and `mdataGetWorkordersResultKeys` as well as (out) `mdataGetWorkordersResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
CUSTOMER_WORKORDER_NUMBER	Customer-specific number for the production order Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: If this key is not passed along, the [1000] value is used by default. In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int
PARENT_WORKORDER_NUMBER	Number of a parent production order in the order network Advice: This key is only relevant if order networks are used in the system; only the work orders that exist directly below this order structure level are taken into consideration!	☐ String
PART_GROUP	Part group in the iTAC system Advice: If the key is not passed or passed with [-1], there is no restriction; all part groups are taken into consideration!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐ String
SORT_FLAG	Sorting mode for the output [0; 1]: 0 = ascending according to the planned production start time (APS) 1 = ascending according to WORKORDER_NUMBER Advice: If this key is not passed along, the [0] value is used by default!	☐ int

Key	Explanation	IMSApi data type
SPLIT_PARENT_WORKORDER	Order number of the basic order Advice: <i>This key is only relevant if production lots (partial orders) are used in the system; only the subordinate partial orders are taken into consideration!</i>	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: <i>If this key is not passed along, the [1] value is used by default!</i> <i>In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!</i>	☐ int
WORKORDER_NUMBER	Production order number Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: <i>Multiple state values may be passed in the input (separated by ";")!</i> <i>If this key is not passed along, the [S,F,N] value is used by default!</i>	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific) Special case: <i>If [-1] is passed, the parameter is not evaluated (no restriction)!</i>	☐ String

Tab. 16-169 "Key Value Pair": `mdataGetWorkordersFilters`

Definition of the output parameters for the found work orders

Key	Explanation		IMSApi data type
APS_ORDER_FIXATION	APS service - "work order fixation" mode [0; 1]: 0 = NOT_FIXED 1 = FIXED	☐	int
APS_RESULT	APS planning result [1; 4; 8; 16]: 1 = not planned 4 = late 8 = critical 16 = OK	☐	int
APS_STATE	APS planning state [1; 2; 4; 8; 32]: 1 = unplanned 2 = partially planned 4 = planned 8 = started 32 = end	☐	int
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	int
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐	String
HAS_ORDER_NET	"TOP production order in order network" designation [0; 1]: 0 = Work order is not a TOP production order (FALSE); no subordinate work orders exist for the work order 1 = Work order is a TOP production order (TRUE); at least one subordinate work order exists for the work order	☐	boolean
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumerical additional information 1	☐	String
INFO_TXT_2	Alphanumerical additional information 2	☐	String
INFO_TXT_3	Alphanumerical additional information 3	☐	String
INFO_TXT_4	Alphanumerical additional information 4	☐	String
INFO_TXT_5	Alphanumerical additional information 5	☐	String

Key	Explanation		IMSApi data type
INFO_TXT_6	Alphanumerical additional information 6	☐	String
PARENT_WORKORDER_DESC	Order description of the TOP production order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a subordinate production order (order network)!	☐	String
PARENT_WORKORDER_NUMBER	Order number of the TOP production order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a subordinate production order (order network)!	☐	String
PARENT_WORKORDER_PART_NUMBER	Part number of the finished product for the TOP production order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a subordinate production order (order network)!	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PLANNED_SHIP_DATE	Delivery date specified by the APS system for the work order	☐	DateMs
PLANNED_START_DATE	Start date specified by the APS system for the work order (start of production)	☐	DateMs
PRIORITY_CODE	APS service - unique designator for the work order priority	☐	String
PRIORITY_NAME	APS service - description for the work order priority	☐	String
PRIORITY_VALUE	Value for the work order priority; the greater the value, the higher the prioritization	☐	int
PROCESS_LAYER	- key not used - Advice: This key remains part of the IMSApi interface for downward compatibility reasons only!	☐	int
PROCESS_VERSION	Process version from the iTAC system	☐	int
PRODUCTION_CONTROLLER	Production controller; this is a customer-specific grouping that is set up in the part master	☐	String
PRODUCTION_CONTROLLER_DESC	Description of the production controller	☐	String
QUANTITY_WORKORDER_PASS	Number of serial numbers with the "PASS" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WORKORDER_SCRAP	Number of serial numbers with the "SCRAP" state booked on the last work step with obligatory confirmation	☐	int
QUANTITY_WORKORDER_TOTAL	Order quantity (total)	☐	int
SPLIT_PARENT_WORKORDER	Order number of the basic order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a production lot (partial order)!	☐	String
SPLIT_PARENT_WORKORDER_DESC	Order description of the basic order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a production lot (partial order)!	☐	String

Key	Explanation	IMSApi data type
SPLIT_PARENT_WORKORDER_PART_NUMBER	Part number of the finished product for the basic order Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a production lot (partial order)!	☐ String
SPLIT_QTY	Order quantity for the production lot Advice: This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a production lot (partial order)!	☐ int
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐ long
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐ String

Tab. 16-170 "Key Value Pair": mdataGetWorkordersResultKeys

16.39.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdataGetWor- kordersFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetWorkordersFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdataGetWorkordersFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdataGetWor- kordersResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdataGetWorkordersResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdataGetWorkordersResultValues !

Tab. 16-171 Parameter (in) »mdataGetWorkorders«

Parameter (out)

Parameter name	Explanation
mdataGetWor- kordersResultVal- ues	Depending on parameter (in) mdataGetWorkordersResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdataGetWorkordersResultKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 16-172 Parameter (out) »mdataGetWorkorders«

16.39.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.40 mdataGetWorkplanActivities

16.40.1 Function description

This function returns detailed work step information, including the stored work step activities of the ascertained work plan. In doing so, it is possible to select whether the information of all work steps is to be output or only for the work step with the passed station. If all work steps are output, they are sorted by work step number. If necessary, the `FUNC_MODE` key (see parameter `workplanFilter`) can be used to restrict the output to strictly work plan header information.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetWorkplanActivities
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue [] workplanFilter
    String key
    String value
in String [] workplanDataResultKeys
in String [] activitiesDataResultKeys
in String [] activityParamResultKeys
in String [] deviceDataResultKeys
out String [] workplanDataResultValues
out String [] activitiesDataResultValues
out String [] activityParamResultValues
out String [] deviceDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workplanFilter`, `workplanDataResultKeys`, `activitiesDataResultKeys`, `activityParamResultKeys` and `deviceDataResultKeys` as well as (out) `workplanDataResultValues`, `activitiesDataResultValues`, `activityParamResultValues` and `deviceDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNC_MODE	Defines the function mode for the output of the <code>workplanDataResultKeys</code> "Key Value Pair" structure [0; 1]: 0 = Output of all selected data fields (default) 1 = Output of the selected "header data fields" (available for selection here are only the <code>WORKPLAN_ID</code> , <code>PART_NUMBER</code> , <code>WORKPLAN_ALT</code> and <code>WORKPLAN_VERS</code> keys) Advice: If this key is not passed along, the [0] value is used by default!	☐ int
OBLIGATORY_CONFIRM_FLAG	Defines whether all work steps or only work steps obligated to supply confirmation are taken into account [0; 1]: 0 = All work steps 1 = Only work steps obligated to supply confirmation Advice: The "Feedb. reqd." field is used on the work plan to define whether or not a work step is obligated to supply confirmation (ADM module -> Master data -> Work plan)!	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: If the key is not passed, there is no restriction!	☐ int
PRODUCT_NUMBER	Part number in the iTAC system; if the work plan is to be ascertained on the basis of the part number, the work order type (<code>WORKPLAN_ALT</code>) must absolutely be passed as well; the <code>WORKORDER_NUMBER</code> and <code>SERIAL_NUMBER</code> parameters must NOT be passed. Advice: If the key is not passed, a valid work order (<code>WORKORDER_NUMBER</code>) or a serial number (<code>SERIAL_NUMBER</code>) must be passed!	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: If the key is not passed, a valid work order (<code>WORKORDER_NUMBER</code>) or a part number (<code>PRODUCT_NUMBER</code>) must be passed in combination with a specific work plan alternative (<code>WORKPLAN_ALT</code>)!	☐ String

Key	Explanation	IMSApi data type
STATION_BASED_WORK- STEPS	Defines whether the stations stored in the ERP group are ascertained and output in separate records [0; 1]: 0 = Stations are not output separately 1 = Stations are output separately Advice: <i>If the value [1] is passed, as many records are output for the work step as there are stations defined in the ERP group (cf. parameters STATION_DESC and STATION_NUMBER in the "Key Value Pair": workplanDataResultKeys table)!</i>	☐ int
WORKORDER_NUMBER	Number of a production order Advice: <i>If the key is not passed, a valid serial number (SERIAL_NUMBER) or a part number (PRODUCT_NUMBER) must be passed in combination with a specific work plan alternative (WORKPLAN_ALT)!</i>	☐ int
WORKPLAN_ALT	Work order type as per work plan alternative [01; FA; U; REP; CO; IG; MAI] Advice: <i>This field is only evaluated if the part number (PRODUCT_NUMBER) is also passed!</i> 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: <i>By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!</i>	☐ String
WORKPLAN_VERS	Work plan version Advice: <i>If the key is not passed, the highest released and valid work plan version is automatically used provided no ERP change number is passed (see WORKPLAN_VERS_ERP)!</i>	☐ int
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: <i>This key is only evaluated if WORKPLAN_VERS is not used!</i>	☐ String
WORKSTEP_FLAG	Controls the scope of the data output [0; 1] 0 = All work steps of the work plan are output 1 = Only the work steps acc. to the passed station (stationNumber) are output	☐ int

Tab. 16-173 "Key Value Pair": workplanFilter

Definition of the output parameters for the found work steps

Key	Explanation	IMSApi data type
ACTIVITY_COUNT	Number of stored activities of the work step	☒ int
COST_CENTER_DESC	Cost center description	☐ String
COST_CENTER_NUMBER	Cost center number	☐ String
CYCLE_TIME_MACHINE	Processing time at the station (TE_M) for the work step in milliseconds (per piece)]	☐ long
CYCLE_TIME_USER	Processing time at the personnel workplace (TE_P) for the work step in milliseconds (per piece)	☐ long
EQUIPMENT_AVAILABLE	Equipment at the work step [0; 1]: 0 = No equipment present 1 = Equipment present	☐ int
ERP_CHANGE_NUMBER	ERP change number for the work plan Advice: This number has no control function in the system; it can, however, be used for customer-specific functions (e.g. reference number of an ERP system)!	☐ String
ERP_GROUP_DESC	ERP group description in the iTAC system	☐ String
ERP_GROUP_NUMBER	ERP group number in the iTAC system	☐ String
MAX_TEST_COUNT	Maximum value for the permitted number of tests (test-retry counter)	☐ int
MDA_DOCUMENT_AVAILABLE	MDA items at the work step [0; 1]: 0 = No MDA items present 1 = MDA items present	☐ int
MSL_OFFSET	Work-step-related offset (in minutes) which is included in the MSL check. This value is subtracted from the remaining mounting time during the calculation to determine whether an MSL object can still be used; thus, it serves as an additional safety time.	☐ int
MSL_RELEVANT	Relevance for "MSL processing" [0; 1]: 0 = work step is NOT "MSL relevant" 1 = work step is "MSL relevant"	☐ int
NUMBER_OF_WORKER_PRODUCTION	Number of employees who have been planned for execution of the work step (production).	☐ int
NUMBER_OF_WORKER_SETUP	Number of employees who have been planned for the necessary work step preparations (e.g. setup).	☐ int
MULTIPLE_COUNT	Multiple panel factor (MNF); it defines the number of single panels which can be processed simultaneously in one work step. Thus, it is used for the calculation of "TE_M" and "TE_P" for the single panels. Example: TE_M = 60s – duration of the work step of the multiple panel, MNF = 10 – that implies a calculated production duration for the single panels of 6s	☐ int
OBLIGATORY_CONFIRM_FLAG	Confirmation obligation [0; 1]: 0 = Work step is not obligated to supply confirmation 1 = Work step is obligated to supply confirmation Advice: The "Feedb. rqd." field is used on the work plan to define whether or not a work step is obligated to supply confirmation (ADM module -> Master data -> Work plan)!	☐ int

Key	Explanation		IMSApi data type
PART_NUMBER	Part number in the iTAC system Advice: Here, this is the part no. on the basis of which the work plan was created!	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
RESTING_TIME	Required resting time between this and the next work step (in seconds)	☐	long
SEPARATION_FLAG	Multiple panel separation [0; 1]: 0 = Work step may not separate any multiple panels 1 = Work step may separate multiple panels Advice: Whether or not a work step may separate multiple panels is defined in the work plan with the "ERP sep. flag" field (ADM module -> Master data -> Work plan)!	☐	int
SETUP_FLAG	Setup requirement [0; 1]: 0 = Work step does not require setup 1 = Work step requires setup Advice: The "Setup flag" field is used on the work plan to define whether or not a work step requires setup (ADM module -> Master data -> Work plan)!	☐	int
SETUP_TIME_MACHINE	Setup time on the station (TR_M) for the work step in milliseconds	☐	long
SETUP_TIME_USER	Setup time at personnel workplace (TR_P) for the work step in milliseconds	☐	long
SKILL_DESC	Description of the user skill	☐	String
SKILL_LEVEL	Level of the user skill	☐	String
STATION_DESC	Station description of the work station in the iTAC system Advice: This information can only be ascertained if the value [1] is passed for the STATION_BASED_WORKSTEPS key!	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This information can only be ascertained if the value [1] is passed for the STATION_BASED_WORKSTEPS key!	☐	String
TRANSPORTING_TIME	Required transport time between this and the next work step (in seconds)	☐	long
VOUCHER_NUMBER	Voucher number in the iTAC system	☐	String
WAGE_GROUP_DESC	Description of the wage group	☐	String
WAGE_GROUP_NUMBER	Wage group designator in the iTAC system	☐	String
WAGE_TYPE_DESC	Description of the wage type	☐	String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system	☐	String

Key	Explanation	IMSApi data type
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	☐ String
WORKPLAN_ID	Internal identification number of the work plan	☐ long
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int
WORKSTEP_AG	Number of the processing thread	☐ int
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_INFO	Individual additional information on the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☒ int
WORKSTEP_NUMBER_ALT	Consecutive number for the alternatives of a work step Advice: <i>Alternative work steps always lie on the same thread!</i>	☐ int
WORKSTEP_TEXT	Work step text that can be defined independently of a WS catalog reference (corresponds to field "WS text" in the myiTAC workplace).	☐ String
WORKSTEP_TYPE	Work step type [M; P]: M = Machine process step P = Personnel process step	☐ String

Tab. 16-174 "Key Value Pair": *workplanDataResultKeys*

Definition of the output parameters for the found work step activities

Key	Explanation		IMSApi data type
ACTION_AFTER_ERROR	Supplementary information on the "Action in case of error" (next activity or next work step) Advice: <i>This parameter is only output if the value [3] or [4] has been output for the "Action in case of error" (see ACTION_AFTER_ERROR_TYPE)!</i>	☐	String
ACTION_AFTER_ERROR_TYPE	Action to be executed in case of error [0; 1; 2; 3; 4]: 0 = continuation of the activity (CONTINUE_ACTIVITY) 1 = redo the activity (REPEAT_ACTIVITY) 2 = redo the work step (REPEAT_WORKSTEP) 3 = go to the activity (GOTO_ACTIVITY) 4 = go to the work step (GOTO_WORKSTEP) Advice: <i>Supplementary to the values [3] and [4], the key ACTION_AFTER_ERROR is output (next activity to be executed or next work step)!</i>	☐	String
ACTIVITY_DESC	Description of the work step activity	☐	String
ACTIVITY_ID	Internal identification number of the work step activity Advice: <i>Based on this number, the work step activity can be put into context with the assigned activity parameters and devices (see the key ACTIVITY_ID in the tables "Key Value Pair": activityParam-ResultKeys and "Key Value Pair": deviceDataResultKeys)!</i>	☒	int
ACTIVITY_NUMBER	Name of the work step activity	☐	String
ACTIVITY_SORT_SEQUENCE	Sorting number for the work step activity Advice: <i>This number determines the processing sequence in the event that multiple activities are stored in a work step!</i>	☐	int
ACTIVITY_STATE	Object state of the work step activity [0; 1; 2]: 0 = Initial (INIT) 1 = RELEASED 2 = LOCKED	☐	int
ACTIVITY_TYPE	Name of the activity type	☐	String
ACTIVITY_TYPE_DESC	Description of the activity type	☐	String
ACTIVITY_TYPE_INFO	Individual additional information on the activity type	☐	String
ACTIVITY_TYPE_STATE	Object state of the activity type [0; 1; 2]: 0 = Initial (INIT) 1 = RELEASED 2 = LOCKED	☐	int
GOTO_ORDER_NUMBER	Number of the next activity (processing sequence) Advice: <i>This key is particularly relevant if an activity occurs multiple times in a process!</i>	☐	int
WORKSTEP_ID	Internal identification number of the work step	☐	long
WORKSTEP_NUMBER	Number of the work step	☒	int

Tab. 16-175 "Key Value Pair": activitiesDataResultKeys

Definition of the output parameters for the found activity parameters

Key	Explanation	IMSApi data type
ACTIVITY_ID	Internal identification number of the work step activity Advice: On the basis of this number, the work step activity can be put into context with the assigned activity parameters (see the ACTIVITY_ID key in the table "Key Value Pair": activitiesDataResultKeys)!	☒ int
ACTIVITY_PARAM_DATA_TYPE	Data type of the activity parameter: - BOOLEAN = TRUE or FALSE - DOUBLE = floating point number (64-bit) - INTEGER = 32-bit signed integer value - LONG = 64-bit signed integer value - STRING = UNICODE string of arbitrary length - MES_PARAM = work plan activity; this is an individual parameter type, which can be set up as "dynamic parameter" in the myiTAC workplace.	☐ String
ACTIVITY_PARAM_DESC	Description of the activity parameter	☐ String
ACTIVITY_PARAM_NAME	Name of the activity parameter	☐ String
ACTIVITY_PARAM_VALUE	Value of the activity parameter	☐ String
SORT_SEQUENCE	Sorting number for the activity parameter Advice: This number determines the sequence in the event that multiple activity parameters are stored!	☐ int

Tab. 16-176 "Key Value Pair": activityParamResultKeys

Definition of the output parameters for the found devices

Key	Explanation	IMSApi data type
ACTIVITY_ID	Internal identification number of the work step activity Advice: On the basis of this number, the work step activity can be put into context with the assigned devices (see the ACTIVITY_ID key in the table "Key Value Pair": activitiesDataResultKeys)!	☒ int
DEVICE_ADDRESS	Device address	☐ String
DEVICE_INIT	Information on device initialization	☐ String
DEVICE_ITEM_DESC	Description of the device	☐ String
DEVICE_ITEM_NAME	Name of the device	☐ String
DEVICE_LOGIC_LEVEL	Logic-Level of the device	☐ int
DEVICE_LOGIC_OPERATOR	Logic operator [0; 1; 2; 3; 4; 5]: 0 = NONE 1 = AND 2 = OR 3 = NAND 4 = NOR 5 = XOR	☐ String
DEVICE_LOGIC_RANK	Logic rank of the device	☐ int
DEVICE_PART_DESC	Part description of the device type	☐ String
DEVICE_PART_NUMBER	Part number of the device type	☐ String
DEVICE_SIGNAL_DURATION	Signal duration	☐ int
DEVICE_SIGNAL_TYPE	Signal type	☐ String
SORT_SEQUENCE	Sorting number for the device Advice: This number determines the sequence in the event that multiple devices are stored!	☐ int

Tab. 16-177 "Key Value Pair": deviceDataResultKeys

16.40.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workplanFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": workplanFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workplanFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workplanDataResultKeys	List of the parameters (work plan) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workplanDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workplanDataResultValues</code> !
activitiesDataResultKeys	List of the parameters (work step activity) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": activitiesDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>activitiesDataResultValues</code> !
activityParamResultKeys	List of the parameters (activity parameters) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": activityParamResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>activityParamResultValues</code> !
deviceDataResultKeys	List of the parameters (device) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": deviceDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>deviceDataResultValues</code> !

Tab. 16-178 Parameter (in) »mdataGetWorkplanActivities«

Parameter (out)

Parameter name	Explanation
workplanDataResultValues	Depending on parameter (in) <code>workplanDataResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>workplanDataResultKeys</code>!</i>
activitiesDataResultValues	Depending on parameter (in) <code>activitiesDataResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>activitiesDataResultKeys</code>!</i>
activityParamResultValues	Depending on parameter (in) <code>activityParamResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>activityParamResultKeys</code>!</i>
deviceDataResultValues	Depending on parameter (in) <code>deviceDataResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>deviceDataResultKeys</code>!</i>

Tab. 16-179 *Parameter (out) »mdataGetWorkplanActivities«*

16.40.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-131	Failure	No work order found for serial number
-150	Failure	Invalid value for "Flag"
-152	Failure	Not enough comparison values exist for the calculation
-338	Failure	User -> Skill not assigned
-340	Failure	Activity for the work step not found
-366	Failure	More than one user skill assigned
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.41

mdataGetWorkplanData

16.41.1

Function description

This function returns detailed work step information about the ascertained work plan. In doing so, it is possible to select whether the information of all work steps is to be output or only for the work step with the passed station. If all work steps are output, they are sorted by work step number. If necessary, the `FUNC_MODE` key (see parameter `workplanFilter`) can be used to restrict the output to strictly work plan header information.

**Advice**

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

The desired work plan can be ascertained on the basis of an order number (`WORKORDER_NUMBER`), a serial number (`SERIAL_NUMBER`) or a part number (`PRODUCT_NUMBER`) – here, in combination with the work plan alternatives. If a serial number and a work order are passed, the serial number must belong to the work order. If the work plan is to be ascertained on the basis of a part number, neither the serial number nor the work order may be passed.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetWorkplanData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
KeyValue[] workplanFilter
    String key
    String value
in String[] workplanDataResultKeys
out String[] workplanDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workplanFilter` and `workplanDataResultKeys` as well as (out) `workplanDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNC_MODE	Defines the function mode for the output of the <code>workplanDataResultKeys</code> "Key Value Pair" structure [0; 1]: 0 = Output of all selected data fields (default) 1 = Output of the selected "header data fields" (available for selection here are only the <code>WORKPLAN_ID</code> , <code>PART_NUMBER</code> , <code>WORKPLAN_ALT</code> and <code>WORKPLAN_VERS</code> keys) Advice: If this key is not passed along, the [0] value is used by default!	☐ int
OBLIGATORY_CONFIRM_FLAG	Defines whether all work steps or only work steps obligated to supply confirmation are taken into account [0; 1]: 0 = All work steps 1 = Only work steps obligated to supply confirmation Advice: The "Feedb. reqd." field is used on the work plan to define whether or not a work step is obligated to supply confirmation (ADM module -> Master data -> Work plan)!	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: If the key is not passed, there is no restriction!	☐ int
PRODUCT_NUMBER	Part number in the iTAC system; if the work plan is to be ascertained on the basis of the part number, the work order type (<code>WORKPLAN_ALT</code>) must absolutely be passed as well; the <code>WORKORDER_NUMBER</code> and <code>SERIAL_NUMBER</code> parameters must NOT be passed. Advice: If the key is not passed, a valid work order (<code>WORKORDER_NUMBER</code>) or a serial number (<code>SERIAL_NUMBER</code>) must be passed!	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: If the key is not passed, a valid work order (<code>WORKORDER_NUMBER</code>) or a part number (<code>PRODUCT_NUMBER</code>) must be passed in combination with a specific work plan alternative (<code>WORKPLAN_ALT</code>)!	☐ String
STATION_BASED_WORKSTEPS	Defines whether the stations stored in the ERP group are ascertained and output in separate records [0; 1]: 0 = Stations are not output separately 1 = Stations are output separately Advice: If the value [1] is passed, as many records are output for the work step as there are stations defined in the ERP group (cf. parameters <code>STATION_DESC</code> and <code>STATION_NUMBER</code> in the "Key Value Pair": <code>workplanDataResultKeys</code> table)!	☐ int

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Number of a production order Advice: <i>If the key is not passed, a valid serial number (SERIAL_NUMBER) or a part number (PRODUCT_NUMBER) must be passed in combination with a specific work plan alternative (WORKPLAN_ALT)!</i>	String
WORKPLAN_ALT	Work order type as per work plan alternative [01; FA; U; REP; CO; IG; MAI] Advice: <i>This field is only evaluated if the part number (PRODUCT_NUMBER) is also passed!</i> 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: <i>By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!</i>	String
WORKPLAN_VERS	Work plan version Advice: <i>This field is only evaluated if the part number (PRODUCT_NUMBER) is passed in combination with the work plan alternative (WORKPLAN_ALT). If the key is not passed, the highest released and valid work plan version is automatically used provided no ERP change number is passed (see WORKPLAN_VERS_ERP)! </i>	int
WORKPLAN_VERS_ERP	ERP change number for the work plan Advice: <i>This field is only evaluated if the part number (PRODUCT_NUMBER) is passed in combination with the work plan alternative (WORKPLAN_ALT); moreover, a work plan version (WORKPLAN_VERS) is not allowed to be passed!</i>	String
WORKSTEP_FLAG	Controls the scope of the data output [0; 1] 0 = All work steps of the work plan are output 1 = Only the work steps acc. to the passed station (stationNumber) are output	int

Tab. 16-180 "Key Value Pair": workplanFilter

Definition of the output parameters for the found work plan

Key	Explanation	IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	DateMs
CONFIRMATION_NUMBER	Unique identification number of the "order-based" work step ("Confirmation ID" in the myiTAC workplace) Advice: This number is usually adopted from a connected ERP system and reported back to the ERP system during order confirmation!	String
COST_CENTER_DESC	Cost center description	String
COST_CENTER_NUMBER	Cost center number	String
CYCLE_TIME_MACHINE	Processing time at the station (TE_M) for the work step in milliseconds (per piece)]	long
CYCLE_TIME_USER	Processing time at the personnel workplace (TE_P) for the work step in milliseconds (per piece)	long
EARLIEST_START_LAY	Earliest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Background: Time-related process specifications can be stored for each work step of an order-based work plan – this data generally comes from a connected ERP system and is used as the basis for planning. By default, it is possible to store the earliest and latest start times for the processes "Production", "Setup", "Stripdown", "Lay", "Transport" and "Wait".	DateMs
EARLIEST_START_PRODUCTION	Earliest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	DateMs
EARLIEST_START_SETUP	Earliest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	DateMs
EARLIEST_START_STRIPDOWN	Earliest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	DateMs
EARLIEST_START_TRANSPORT	Earliest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	DateMs
EARLIEST_START_WAIT	Earliest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	DateMs
EQUIPMENT_AVAILABLE	Equipment at the work step [0; 1]: 0 = No equipment present 1 = Equipment present	int

Key	Explanation		IMSApi data type
ERP_CHANGE_NUMBER	ERP change number for the work plan Advice: <i>This number has no control function in the system; it can, however, be used for customer-specific functions (e.g. reference number of an ERP system)!</i>	☐	String
ERP_GROUP_DESC	ERP group description in the iTAC system	☐	String
ERP_GROUP_NUMBER	ERP group number in the iTAC system	☐	String
LATEST_START_LAY	Latest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
LATEST_START_PRODUCTION	Latest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
LATEST_START_SETUP	Latest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
LATEST_START_STRIPDOWN	Latest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
LATEST_START_TRANSPORT	Latest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
LATEST_START_WAIT	Latest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>See Background for key EARLIEST_START_LAY!</i>	☐	DateMs
MAX_TEST_COUNT	Maximum value for the permitted number of tests (test-retry counter)	☐	int
MDA_DOCUMENT_AVAILABLE	MDA items at the work step [0; 1]: 0 = No MDA items present 1 = MDA items present	☐	int
MSL_OFFSET	Work-step-related offset (in minutes) which is included in the MSL check. This value is subtracted from the remaining mounting time during the calculation to determine whether an MSL object can still be used; thus, it serves as an additional safety time.	☐	int
MSL_RELEVANT	Relevance for "MSL processing" [0; 1]: 0 = work step is NOT "MSL relevant" 1 = work step is "MSL relevant"	☐	int
NUMBER_OF_WORKER_PRODUCTION	Number of employees who have been planned for execution of the work step (production).	☐	int
NUMBER_OF_WORKER_SETUP	Number of employees who have been planned for the necessary work step preparations (e.g. setup).	☐	int
MULTIPLE_COUNT	Multiple panel factor (MNF); it defines the number of single panels which can be processed simultaneously in one work step. Thus, it is used for the calculation of "TE_M" and "TE_P" for the single panels. Example: <i>TE_M = 60s – duration of the work step of the multiple panel, MNF = 10 – that implies a calculated production duration for the single panels of 6s</i>	☐	int

Key	Explanation	IMSApi data type
OBLIGATORY_CONFIRM_FLAG	Confirmation obligation [0; 1]: 0 = Work step is not obligated to supply confirmation 1 = Work step is obligated to supply confirmation Advice: The "Feedb. reqd." field is used on the work plan to define whether or not a work step is obligated to supply confirmation (ADM module -> Master data -> Work plan)!	☐ int
PARALLEL_FLAG	"Parallel work step" designator [0; 1]: 0 = Work step can NOT be processed in parallel (FALSE) 1 = Work step can be processed in parallel (TRUE)	☐ boolean
PART_NUMBER	Part number in the iTAC system Advice: Here, this is the part no. on the basis of which the work plan was created!	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
RESTING_TIME	Required resting time between this and the next work step (in seconds)	☐ long
SEPARATION_FLAG	Multiple panel separation [0; 1]: 0 = Work step may not separate any multiple panels 1 = Work step may separate multiple panels Advice: Whether or not a work step may separate multiple panels is defined in the work plan with the "ERP sep. flag" field (ADM module -> Master data -> Work plan)!	☐ int
SETUP_FLAG	Setup requirement [0; 1]: 0 = Work step does not require setup 1 = Work step requires setup Advice: The "Setup flag" field is used on the work plan to define whether or not a work step requires setup (ADM module -> Master data -> Work plan)!	☐ int
SETUP_TIME_MACHINE	Setup time on the station (TR_M) for the work step in milliseconds	☐ long
SETUP_TIME_USER	Setup time at personnel workplace (TR_P) for the work step in milliseconds	☐ long
SKILL_DESC	Description of the user skill	☐ String
SKILL_LEVEL	Level of the user skill	☐ String
STATION_DESC	Station description of the work station in the iTAC system Advice: This information can only be ascertained if the value [1] is passed for the STATION_BASED_WORKSTEPS key!	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This information can only be ascertained if the value [1] is passed for the STATION_BASED_WORKSTEPS key!	☐ String
TRANSPORTING_TIME	Required transport time between this and the next work step (in seconds)	☐ long
VOUCHER_NUMBER	Voucher number in the iTAC system	☐ String
WAGE_GROUP_DESC	Description of the wage group	☐ String
WAGE_GROUP_NUMBER	Wage group designator in the iTAC system	☐ String

Key	Explanation	IMSApi data type
WAGE_TYPE_DESC	Description of the wage type	☐ String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system	☐ String
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	☐ String
WORKPLAN_ID	Internal identification number of the work plan	☐ long
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int
WORKSTEP_AG	Number of the processing thread	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_INFO	Individual additional information on the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_NUMBER_ALT	Consecutive number for the alternatives of a work step Advice: Alternative work steps always lie on the same thread!	☐ int
WORKSTEP_OVER-ALL_STATE	Work step state (corresponds to the WORKSTEP_STATE key) Advice: A functional extension in Release 9.01.00 made it unnecessary to differentiate between the two keys WORKSTEP_OVER-ALL_STATE and WORKSTEP_STATE!	☐ String
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group! Advice: This state can only be ascertained if a work order (WORKORDER_NUMBER) or a serial number (SERIAL_NUMBER) was passed in the input parameters!	☐ String
WORKSTEP_TEXT	Work step text that can be defined independently of a WS catalog reference (corresponds to field "WS text" in the myiTAC workplace).	☐ String
WORKSTEP_TYPE	Work step type [M; P]: M = Machine process step P = Personnel process step	☐ String

Tab. 16-181 "Key Value Pair": workplanDataResultKeys

16.41.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workplanFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": workplanFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workplanFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workplanDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workplanDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workplanDataResultValues</code> !

Tab. 16-182 Parameter (in) »mdataGetWorkplanData«

Parameter (out)

Parameter name	Explanation
workplanDataResultValues	Depending on parameter (in) <code>workplanDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>workplanDataResultKeys</code> !

Tab. 16-183 Parameter (out) »mdataGetWorkplanData«

16.41.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-130	Failure	Work order not found
-131	Failure	No work order found for serial number
-150	Failure	Invalid value for "Flag"
-338	Failure	User -> Skill not assigned
-366	Failure	More than one user skill assigned
-368	Failure	Skill not found

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.42 mdataGetWorkstepSkill

16.42.1 Function description

This function makes it possible to query user skills required for a work step from the iTAC system; here, the work step can be ascertained using a wide range of different filtering criteria (e.g. work order number, serial numbers).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataGetWorkstepSkill
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] workstepSkillFilter
    String key
    String value
in String[] workstepSkillResultKeys
out String[] workstepSkillResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workstepSkillFilter` and `workstepSkillResultKeys` as well as (out) `workstepSkillResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [1000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int
PRODUCT_NUMBER	Part number in the iTAC system; if the work plan is to be ascertained on the basis of the part number, the work order type (WORKPLAN_ALT) must absolutely be passed as well; the WORKORDER_NUMBER and SERIAL_NUMBER parameters must NOT be passed. Advice: <i>If the key is not passed, a valid work order (WORKORDER_NUMBER) or a serial number (SERIAL_NUMBER) must be passed!</i>	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: <i>If the key is not passed, a valid work order (WORKORDER_NUMBER) or a part number (PRODUCT_NUMBER) must be passed in combination with a specific work plan alternative (WORKPLAN_ALT)!</i>	☐ String
WORKORDER_NUMBER	Number of a production order Advice: <i>If the key is not passed, a valid serial number (SERIAL_NUMBER) or a part number (PRODUCT_NUMBER) must be passed in combination with a specific work plan alternative (WORKPLAN_ALT)!</i>	☐ String
WORKPLAN_ALT	Work order type as per work plan alternative [01; FA; U; REP; CO; IG; MAI] Advice: <i>This field is only evaluated if the part number (PRODUCT_NUMBER) is also passed!</i> 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means Advice: <i>By default, only the work plan alternatives specified above are supported; new work plan alternatives should only be created in close cooperation with the iTAC employee responsible for the project!</i>	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 16-184 "Key Value Pair": workstepSkillFilter

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
OPERATION_MODE	"Process" skill type [0; 1; 2]: 0 = Setup and production 1 = Production 2 = Setup This information is imported from a connected ERP system and forwarded to the APS service; adaptation in the iTAC system is NOT possible. Advice: Depending on this information, the values in the keys <code>REQUIRED_WORKER_CNT</code> and <code>WORKER_UTILIZATION</code> must be interpreted!	int
PART_NUMBER	Part number in the iTAC system	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	int
REQUIRED_WORKER_CNT	Number of employees who are to be planned for the necessary work step preparations ("Setup" skill type) or for work step execution ("Production" skill type). This information is imported from a connected ERP system and forwarded to the APS service; adaptation in the iTAC system is NOT possible. Advice: This key must be interpreted according to the specified skill type (see <code>OPERATION_MODE</code> above)!	int
SKILL_DESC	Description of the user skill	String
SKILL_LEVEL	Level of the user skill	String
WORKER_UTILIZATION	Utilization rate (in percent) of the employees required for the work step preparations ("set up" qualification type) and the execution of the work step ("production" qualification type). This information is imported from a connected ERP system and forwarded to the APS service; adaptation in the iTAC system is NOT possible. Advice: This key must be interpreted according to the specified skill type (setup, production) (see <code>OPERATION_MODE</code> above)!	double
WORKPLAN_ALT	Work plan alternative [01; FA; U; REP; CO; IG; MAI]: 01 = Production order (with automatic work order creation) FA = Production order (without automatic work order creation) U = Modification order (for already-manufactured serial numbers) REP = Repair orders for external returns CO = Production order for work-order-neutral manufacturing IG = Work order for work plan for receiving test MAI = Maintenance order for production means	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_NUMBER	Number of the work step	int

Tab. 16-185 "Key Value Pair": `workstepSkillResultKeys`

16.42.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workstepSkillFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": workstepSkillFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workstepSkillFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workstepSkillResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workstepSkillResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workstepSkillResultValues</code> !

Tab. 16-186 Parameter (in) »mdataGetWorkstepSkill«

Parameter (out)

Parameter name	Explanation
workstepSkillResultValues	Depending on parameter (in) <code>workstepSkillResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>workstepSkillResultKeys</code> !

Tab. 16-187 Parameter (out) »mdataGetWorkstepSkill«

16.42.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-121	Failure	Part not found
-130	Failure	Work order not found
-174	Failure	Number of records to be output has fallen below the minimum value
-2042	Failure	Work plan -> Alternative not valid

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2119	Failure	Number of records to be output has exceeded the maximum value

16.43 mdataRequestWorkorder

16.43.1 Function description

This function enables the transfer of work order requests ("ad hoc" queries) to an external system; unique here is that the query is triggered with the API function and then transferred in combination with the »**TRAN.tran_req_workorder**« transfer table. A new entry is created in this table upon calling the IMSApi. The subsequent processing process, e.g., the retrieval by the external system, the provision of new order data, etc., is to be implemented by the customer.



Advice

*The array returned can be used to return records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataRequestWorkorder
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] requestWorkorderKeys
  in String requestWorkorderValues
  in String[] requestWorkorderResultKeys
  out String[] requestWorkorderResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) requestWorkorderKeys and requestWorkorderResultKeys sowie (out) requestWorkorderResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Definition of the input parameter (work order request)

Key	Explanation	IMSApi data type
DESTINATION	Target of the "ad hoc" query [0; 1; 2]: 0 = ERP system 1 = (Other) external system 2 = Internal processing	int
WORKORDER_NUMBER	Number of the requested work order	String
WORKPLAN_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) R = Repair order (corresponds to "Alt. WP": REP) U = Modification order (corresponds to "Alt. WP": U) Advice: The specified value is forwarded directly from the querying client to the ERP process without a validity check against the MES specifications being performed. Provided that the querying client process is matched to the downstream ERP process, the corresponding work plan alternatives (Alt. WP) can also be directly passed here ("01", REP", "IG", etc.)!	String

Tab. 16-188 "Key Value Pair": `requestWorkorderKeys`

Definition of output parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	int
WORKORDER_NUMBER	Production order number	String

Tab. 16-189 "Key Value Pair": `requestWorkorderResultValues`

16.43.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
requestWorkorder-Keys	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>requestWorkorderValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": requestWorkorderKeys table.
requestWorkorder-Values	Value list for the parameters passed by means of <code>requestWorkorderKeys</code> . Advice: Refer to the "Key Value Pair": requestWorkorderKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
requestWorkorder-ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": requestWorkorderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>requestWorkorderResultValues</code> !

Tab. 16-190 Parameter (in) »mdataRequestWorkorder«

Parameter (out)

Parameter name	Explanation
requestWorkorder-ResultValues	Depending on parameter (in) <code>requestWorkorderResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>requestWorkorderResultKeys</code> !

Tab. 16-191 Parameter (out) »mdataRequestWorkorder«

16.43.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1140	Failure	Invalid value for "destination"
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

16.44 mdataSetProductionCycleTime

16.44.1 Function description

This function enables an adjustment of the following default values from the work plan:

- **TE_M** = Time requirement of the work step for the station (processing time)
- **TR_M** = Setup time of the machine
- **TE_P** = Time requirement of the personnel work step (processing time)
- **TR_P** = Setup time of the personnel workplace

The values are then only changed if they have the value "0" beforehand; changes affect both the master work plan and the order-based work plan.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	--

Function declaration

```

int mdataSetProductionCycleTime
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String serialNumber
    in String serialNumberPos
    in String workOrderNumber
    in int processLayer
    in double setupCycleTime
    in double prodCycleTime
    in String processType
    in int cycleTimeForMultipleBoard

```

16.44.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: The affected work step is ascertained on the basis of the passed station!

Parameter name	Explanation
serialNumber	Serial number of the panel Advice: If [-1] is passed here, the field is not evaluated; in this case, the work order is ascertained on the basis of the <code>wor-kOrderNumber</code> parameter. If order and serial number are being passed, the serial number must be part of the WO!
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
workOrderNumber	Number of a production order Advice: If an empty string or [-1] is passed, the work order active on the station is used or ascertained on the basis of the serial number (<code>serialNumber</code>). If order and serial number are being passed, the serial number must be part of the WO!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
setupCycleTime	Setup time of the machine (TR_M) or setup time of the personnel workplace (TR_P); how the passed value is interpreted depends on the <code>processType</code> parameter Advice: The value only applies to the selected work order (see parameters <code>worOrderNumber</code>). The time value must be specified in seconds (the input of decimal values is supported)! Special case: If the value [-1] is passed, the parameter is not evaluated (time is not reset)!
prodCycleTime	Processing time for the machine (TE_M) or the person (TE_P); how the passed value is interpreted depends on the <code>processType</code> parameter Advice: The value only applies to the selected work order (see parameters <code>worOrderNumber</code>). The time value must be specified in seconds (the input of decimal values is supported)! Special case: If the value [-1] is passed, the parameter is not evaluated (time is not reset)!
processType	Process type on the work step [M; P]: M = Machine process step P = Personnel process step Advice: Depending on this setting, the specified times are booked (see <code>setupCycleTime</code> and <code>prodCycleTime</code>) for machine times (TR_M, TE_M) or for staff times (TR_P, TE_P)!
cycleTimeForMultipleBoard	Single panel times/multiple panel times [0; 1]: 0 = Passed time refers to single panels 1 = Passed time refers to multiple panels Advice: If the value [1] is passed, the multiple panel factor defined on the work step (see the "MNF" field in the work plan) is included; in this case, the time value is divided by the number of panels!

Tab. 16-192 Parameter (in) »mdataSetProductionCycleTime«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

16.44.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-128	Failure	Work step not found
-130	Failure	Work order not found
-189	Failure	Invalid value for "processType" [M; P]
-2105	Failure	No TR or PM license present for the station

16.45 mdataUpdateDetectionPoint

16.45.1 Function description

This function can be used to update certain characteristics of existing data capture points. As a result, it is possible to e.g. change the activation state for a selected data capture point.



Advice

Only data capture points of the "Message" and "Measurement data" types are supported at present; other types will be added as required!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataUpdateDetectionPoint
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String detectionPointName
  in KeyValue[] detectionPointKeyValues
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `detectionPointKeyValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Input values "Data capture point"

Key	Explanation	IMSApi data type
ACTION_PLAN	Description of the action to be performed Advice: This key is evaluated only for data capture points of the "Messages" type!	☐ String
ACTIVE	"Data capture point" activation state [0; 1]: 0 = Data capture point is NOT activated 1 = Data capture point is activated immediately	☐ int
CONDITION_CODE	Unique designator of the machine condition/machine message in the iTAC system; this entry must have been assigned to the station passed using STATION_NUMBER. Advice: This key is evaluated only for data capture points of the "Messages" type! The existing data capture point is linked to this entry!	☐ String
CONTROL	Name of the control unit	☐ String
DATA_BLOCK	Address range inside the control unit	☐ String
DETECTION_POINT_DESC	Description of the data capture point	☐ String
ELEMENT_TYPE	Measurement data type: VAL = Measured value NOM = Nominal value TOL = Tolerance MIN = Value (min.) MAX = Value (max.) LAL = Lower intervention limit UAL = Upper intervention limit LSL = Lower scrap limit USL = Upper scrap limit Advice: This key is evaluated only for data capture points of the "Measurement data" type!	☐ String
INFO_TXT_1	Individual additional information 1 on the data capture point	☐ String
INFO_TXT_2	Individual additional information 2 on the data capture point	☐ String
INFO_TXT_3	Individual additional information 3 on the data capture point	☐ String
INFO_TXT_4	Individual additional information 4 on the data capture point	☐ String
INFO_TXT_5	Individual additional information 5 on the data capture point	☐ String
LOCATION	Position of the capture point in the production plant (message location) Advice: This key is evaluated only for data capture points of the "Messages" type!	☐ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name) Advice: This key is evaluated only for data capture points of the "Measurement data" type!	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: This key is evaluated only for data capture points of the "Messages" type! If the data capture point is to be assigned a message cause, the message cause must already exist in the iTAC system!	☐ String

Key	Explanation	IMSApi data type
MESSAGE_CLASS_NAME	Name of the message class Advice: This key is evaluated only for data capture points of the "Messages" type! If the data capture point is to be assigned a message class, the message class must already exist in the iTAC system!	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: This key is evaluated only for data capture points of the "Messages" type! If the data capture point is to be assigned a message effect, the message effect must already exist in the iTAC system!	☐ String
MESSAGE_INFO_TXT_1	Individual additional information 1 on the message Advice: This key is evaluated only for data capture points of the "Messages" type!	☐ String
MESSAGE_INFO_TXT_2	Individual additional information 2 on the message Advice: This key is evaluated only for data capture points of the "Messages" type!	☐ String
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: This key is evaluated only for data capture points of the "Messages" type!	☐ int
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_NAME	Name of the resource Background: Resources are used to map sub-structures of a station (machine); this enables data to be captured for individual machine components, e.g. motor, conveyor belt or control unit.	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: The created data capture point is linked to this station!	☐ String
VARIABLE	Variable in the address range of the control unit	☐ String

Tab. 16-193 "Key Value Pair": *detectionPointKeyValues*

16.45.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
detectionPoint-Name	Name of the data capture point
detectionPoint-KeyValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": detectionPointKeyValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the " Key Value Pair ": detectionPointKeyValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 16-194 Parameter (in) »mdataUpdateDetectionPoint«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

16.45.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1131	Failure	Message capture point -> Key not valid
-1132	Failure	Message capture point -> State not valid / no station available
-1133	Failure	Message capture point -> Resource not valid / no station available
-1134	Failure	Measurement data capture point -> Key not valid
-1135	Failure	Measurement data capture point -> Resource not valid / no station available.
-1137	Failure	Measured value not found

-1138	Failure	Item type not valid
-1200	Failure	Machine condition/message not found
-1217	Failure	Capture point not found
-1221	Failure	Message class not found
-1222	Failure	Message cause not found
-1223	Failure	Message effect not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2136	Failure	Resource not found
-2140	Failure	Resource does not match station

16.46 mdataUpdateOrderCluster

16.46.1 Function description

This function enables the editing of a work order cluster. Work order clusters are used to merge different work orders and to manufacture them in a group (in parallel). The function can be used to assign other work orders or to change or delete existing work order assignments.



Advice

The respective array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = -1123) in the event of one or more faulty records!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdataUpdateOrderCluster
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String clusterName
in KeyValue[] updateOrderClusterValues
    String key
    String value
in String[] createOrderClusterItemUploadKeys
in String[] createOrderClusterItemUploadValues
in String[] updateOrderClusterItemUploadKeys
in String[] updateOrderClusterItemUploadValues
in String[] deleteOrderClusterItemUploadKeys
in String[] deleteOrderClusterItemUploadValues
out String[] createOrderClusterItemResultValues
out String[] updateOrderClusterItemResultValues
out String[] deleteOrderClusterItemResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `updateOrderClusterValues`, `createOrderClusterItemUploadKeys`, `updateOrderClusterItemUploadKeys` and `deleteOrderClusterItemUploadKeys` as well as (out) `createOrderClusterItemResultValues`, `updateOrderClusterItemResultValues` and `deleteOrderClusterItemResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for the work order cluster that is to be changed (header data)

Key	Explanation	IMSApi data type
CHANGE_COMMENT	Individual change comment	☐ String
CLUSTER_DESC	Description of the work order cluster	☐ String
CLUSTER_NAME	Unique name for the work order cluster Advice: If the key is not passed, the name is generated by the system automatically; it is made up of an identification code for the work order cluster type (OC, WC or PC) followed by an identification number!	☐ String
CLUSTER_QUANTITY	Number of products to be manufactured in the cluster	☐ int
CLUSTER_STATE	State for the work order cluster [E; D]: E = Work order cluster is activated (E nabled) D = Work order cluster is NOT activated (D isabled) Advice: If this key is not passed along, the [E] value is used by default!	☐ String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster Advice: If the key is not passed, the value [O] is used by default; if the value [W] is passed here, it is essential that the desired work step be passed (see below, key WORKSTEP_NUMBER)!	☐ String
COMMENT	Individual comment on work order cluster creation	☐ String
FUNC_MODE	Function mode "work order assignment" [0; 1]: 0 = Work order can only be assigned to one cluster 1 = Work order can be assigned to multiple clusters Advice: If this key is not passed along, the [0] value is used by default!	☐ int

Tab. 16-195 "Key Value Pair": updateOrderClusterValues

Input values for work order assignments that are to be created new

Key	Explanation	IMSApi data type
CLUSTER_ITEM_STATE	State for the work order in the cluster [E; D]: E = Work order is activated in the cluster (E nabled) D = Work order is NOT activated in the cluster (D isabled) Advice: If this key is not passed along, the [E] value is used by default!	☐ String
CUSTOMER_WOR-KORDER_NUMBER	Customer-specific number for the production order	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -1123)!	☐ int
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐ boolean
POSITION	Unique position number of the work order in the cluster Advice: If the key is not passed, the current highest position number is increased by [+1] by default!	☐ int
POSITION_MIRRORED	Unique position number of the work order in the cluster for the application type "double-sided panel"; the key must always be used if at least one position in the work order cluster is identified as "mirrored" (IS_MIRRORED = TRUE). Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created! Technical background: This key is required for determining the correct position number if units are placed on the front and rear side using the same placement program.	☐ int
WORKORDER_NUMBER	Number of the production order that is to be assigned to the work order cluster	☒ String
WORKSTEP_NUMBER	Work step number in the iTAC system; this work step determines the work step that is to be manufactured jointly in the work step cluster. Advice: This key is only evaluated if the value [W] was passed as work order cluster type (see key CLUSTER_TYPE)!	☐ int

Tab. 16-196 "Key Value Pair": createOrderClusterItemUploadKeys

Input values for work order assignments that are to be updated

Key	Explanation	IMSApi data type
CHANGE_COMMENT	Individual change comment	☐ String
CLUSTER_ITEM_STATE	State for the work order in the cluster [E; D]: E = Work order is activated in the cluster (Enabled) D = Work order is NOT activated in the cluster (Disabled)	☒ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -1123)!	☐ int
POSITION	Unique position number of the work order in the cluster	☒ int

Tab. 16-197 "Key Value Pair": `updateOrderClusterItemUploadKeys`

Input values for work order assignments that are to be deleted

Key	Explanation	IMSApi data type
CHANGE_COMMENT	Individual change comment	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -1123)!	☐ int
POSITION	Unique position number of the work order in the cluster	☒ int

Tab. 16-198 "Key Value Pair": `deleteOrderClusterItemUploadKeys`

16.46.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
clusterName	Unique name for the work order cluster that is to be adapted
updateOrderClusterValues	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": updateOrderClusterValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": updateOrderClusterValues table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
createOrderClusterItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>createOrderClusterItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": createOrderClusterItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!
createOrderClusterItemUploadValues	Values for the parameters selected by means of <code>createOrderClusterItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>createOrderClusterItemUploadKeys</code> ! Refer to the "Key Value Pair": createOrderClusterItemUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
updateOrderClusterItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>updateOrderClusterItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": updateOrderClusterItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!
updateOrderClusterItemUploadValues	Values for the parameters selected by means of <code>updateOrderClusterItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>updateOrderClusterItemUploadKeys</code> ! Refer to the "Key Value Pair": updateOrderClusterItemUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Parameter name	Explanation
deleteOrderClusterItemUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>deleteOrderClusterItemUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": deleteOrderClusterItemUploadKeys table. Advice: Only the parameters passed here are processed by the function!
deleteOrderClusterItemUploadValues	Values for the parameters selected by means of <code>deleteOrderClusterItemUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>deleteOrderClusterItemUploadKeys</code> ! Refer to the "Key Value Pair": deleteOrderClusterItemUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 16-199 Parameter (in) »mdataUpdateOrderCluster«

Parameter (out)

Parameter name	Explanation
createOrderClusterItemResultValues	Depending on parameters (in) <code>createOrderClusterItemUploadKeys</code> and <code>createOrderClusterItemUploadValues</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>createOrderClusterItemUploadKeys</code> !
updateOrderClusterItemResultValues	Depending on parameters (in) <code>updateOrderClusterItemUploadKeys</code> and <code>updateOrderClusterItemUploadValues</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>updateOrderClusterItemUploadKeys</code> !
deleteOrderClusterItemResultValues	Depending on parameters (in) <code>deleteOrderClusterItemUploadKeys</code> and <code>deleteOrderClusterItemUploadValues</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>deleteOrderClusterItemUploadKeys</code> !

Tab. 16-200 Parameter (out) »mdataUpdateOrderCluster«

16.46.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-132	Failure	Work order is not unique
-138	Failure	Order state is invalid
-1114	Failure	Work order cluster -> Name is already in use
-1115	Failure	Work order cluster could not be updated
-1116	Failure	Work order cluster -> Phantom work order not unique
-1117	Failure	Work order cluster -> Position is occupied
-1118	Failure	Work order cluster has an invalid state
-1119	Failure	Work order cluster -> Type not found
-1120	Failure	Work order cluster -> Type is invalid

-1121	Failure	Work order cluster -> Work order is already part of another work order cluster
-1122	Failure	Work order cluster not found
-1123	Failure	Work order cluster -> Processing failed
-2021	Failure	Work order type not valid
-2030	Failure	Work plan not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2115	Failure	Work step is invalid

17

Group "MDC" (machine data collection)

17.1

mdcAcknowledgeStationConditions

17.1.1

Function description

This function is used to confirm machine messages and machine conditions already reported in the system. Confirmation is initially performed on the basis of the function parameters `userAcknowledge` (confirming person) and `dateAcknowledge` (time of confirmation). This confirms all as yet UNCONFIRMED machine messages and machine conditions ascertained using the filter.

If confirmations with deviating persons/time stamps are to be passed, the keys `ACKNOWLEDGEMENT_DATE` and `ACKNOWLEDGEMENT_USER_ID` of the respective structures can be used; the following applies in this case: key overrides function parameter.



Advice

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

Application types

The data basis on which the data to be confirmed is selected is controlled using the `selectionType` function parameter; a distinction is made between the following four application types:

- Selection based on the internal **identification number** of the machine message/machine condition (`selectionType = 1`); in this case, the entry to be confirmed is ascertained using the object type (`OBJECT_TYPE = message/condition`) and the identification number (`CONDITION_ID`).
- Selection based on a **data capture point** (`selectionType = 2`); in this case, the entries to be confirmed are ascertained using the data capture point. If multiple unconfirmed messages/conditions exist for one data capture point, all of these messages/conditions are confirmed.
- Selection based on a **machine condition/machine message** (`selectionType = 3`); in this case, the entries to be confirmed are ascertained by combining the data objects "Station" (`STATION_NUMBER`) and "Message/condition code" (`CONDITION_CODE`). If further resources to which messages/conditions are assigned are configured under the station, these messages are also confirmed.
- Selection based on a **resource** (`selectionType = 4`); in this case, the entries to be confirmed are ascertained by combining the data objects "Station" (`STATION_NUMBER`), "Resource" (`RESOURCE_NAME`) and "Message/condition code" (`CONDITION_CODE`); if multiple unconfirmed messages/conditions are found, all are confirmed.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```

int mdcAcknowledgeStationConditions
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String userAcknowledge
in DateMs dateAcknowledge
in String txtAcknowledge
in int selectionType
in String[] stationConditionAcknowledgeKeys
in String[] stationConditionAcknowledgeValues
out String[] stationConditionResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) stationConditionAcknowledgeKeys and stationConditionAcknowledgeValues as well as (out) stationConditionResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Specifications for "identification number" selection type (selectionType = 1)

Key	Explanation	IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, the time of confirmation defined in the dateAcknowledge function parameter is used!	☐ DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text Advice: If the key is not passed, the confirmation text defined in the txtAcknowledge function parameter is used!	☐ String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification) Advice: If the key is not passed, the user defined in the userAcknowledge function parameter is used!	☐ String
CONDITION_ID	Internal identification number of the machine condition or of the machine message Advice: This key is interpreted in combination with OBJECT_TYPE; detailed information on this topic can be found above under the point "Application types"!	☒ long

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
OBJECT_TYPE	Machine information type [1; 2]: 1 = Machine condition 2 = Machine message Advice: This key is interpreted in combination with <code>CONDITION_ID</code> ; detailed information on this topic can be found above under the point "Application types"!	☒ int

Tab. 17-1 "Key Value Pair" (identification number): `stationConditionAcknowledgeKeys`

Specifications for "data capture point" selection type (selectionType = 2)

Key	Explanation	IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, the time of confirmation defined in the <code>dateAcknowledge</code> function parameter is used!	☐ DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text Advice: If the key is not passed, the confirmation text defined in the <code>txtAcknowledge</code> function parameter is used!	☐ String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification) Advice: If the key is not passed, the user defined in the <code>userAcknowledge</code> function parameter is used!	☐ String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DETECTION_POINT_NAME	Name of the data capture point	☒ long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int

Tab. 17-2 "Key Value Pair" (data capture point): `stationConditionAcknowledgeKeys`

Specifications for "machine condition/message" selection type
(selectionType = 3)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_- DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, the time of confirmation defined in the dateAcknowledge function parameter is used!	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text Advice: If the key is not passed, the confirmation text defined in the txtAcknowledge function parameter is used!	☐	String
ACKNOWLEDGEMENT_US- ER_ID	Login name of the "confirming" user (user identification) Advice: If the key is not passed, the user defined in the userAcknowledge function parameter is used!	☐	String
CONDITION_CODE	Unique designator of the machine condition or of the machine message Advice: This key is interpreted in combination with STATION_NUMBER; detailed information on this topic can be found above under the point "Application types"!	☒	long
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This key is interpreted in combination with CONDITION_CODE; detailed information on this topic can be found above under the point "Application types"!	☒	String

Tab. 17-3 "Key Value Pair" (machine condition/message): stationConditionAcknowledgeKeys**Specifications for "resource" selection type** (selectionType = 4)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_- DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed, the time of confirmation defined in the dateAcknowledge function parameter is used!	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text Advice: If the key is not passed, the confirmation text defined in the txtAcknowledge function parameter is used!	☐	String

Key	Explanation	IMSApi data type
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification) Advice: If the key is not passed, the user defined in the <code>userAcknowledge</code> function parameter is used!	☐ String
CONDITION_CODE	Unique designator of the machine condition or of the machine message Advice: This key is interpreted in combination with <code>STATION_NUMBER</code> and <code>RESOURCE_NAME</code> ; detailed information on this topic can be found above under the point "Application types"!	☒ long
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
RESOURCE_NAME	Name of the resource Advice: This key is interpreted in combination with <code>CONDITION_CODE</code> and <code>STATION_NUMBER</code> ; detailed information on this topic can be found above under the point "Application types"!	☒ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This key is interpreted in combination with <code>CONDITION_CODE</code> and <code>RESOURCE_NAME</code> ; detailed information on this topic can be found above under the point "Application types"!	☒ String

Tab. 17-4 "Key Value Pair" (resource): `stationConditionAcknowledgeKeys`

17.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
userAcknowledge	Login name of the "confirming" user (user identification) Advice: By default, the user passed here is used for all confirmations; if required, the <code>ACKNOWLEDGEMENT_USER_ID</code> key can be used to override the users for individual confirmations!
dateAcknowledge	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time! Advice: By default, the time stamp passed here is used for all confirmations; if required, the <code>ACKNOWLEDGEMENT_DATE</code> key can be used to override the time stamps for individual confirmations!
txtAcknowledge	Individual confirmation text Advice: By default, the confirmation text passed here can be used for all confirmations; if required, the <code>ACKNOWLEDGEMENT_TXT</code> key can be used to override the texts for individual confirmations!
selectionType	Data basis for selection [1; 2; 3; 4]: 1 = Identification number 2 = Data capture point 3 = Machine condition/message 4 = Resource Advice: Detailed information on these four variants can be found above in the function description under the point "Application types"!
stationCondition-AcknowledgeKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>stationConditionAcknowledgeValues</code> parameter. A description of the possible parameters (depending on the <code>selectionType</code>) can be found above in the corresponding "Key Value Pair" (xxx): stationConditionAcknowledgeKeys table. Advice: Only the parameters passed here are processed by the function!
stationCondition-AcknowledgeValues	Values for the parameters selected by means of <code>stationConditionAcknowledgeKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>stationConditionAcknowledgeKeys</code> ! Refer to the corresponding "Key Value Pair" (xxx): stationConditionAcknowledgeKeys table above to see whether special rules apply during the input of values (e.g., use of wild-cards)!

Tab. 17-5 Parameter (in) »mdcAcknowledgeStationConditions«

Parameter (out)

Parameter name	Explanation
stationCondition-ResultValues	Depending on parameter (in) <code>stationConditionAcknowledgeKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>stationConditionAcknowledgeKeys</code> !

Tab. 17-6 Parameter (out) »mdcAcknowledgeStationConditions«

17.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid

-198	Failure	Invalid value for "objectType"
-1200	Failure	Machine condition/message not found
-1201	Failure	Machine condition/message is not assigned to the passed station
-1204	Failure	Changing of the machine message not allowed
-1212	Failure	Message capture point invalid -> No condition/message assigned!
-1213	Failure	Message capture point invalid -> No station assigned!
-1214	Failure	Message capture point invalid
-1217	Failure	Capture point not found
-1218	Failure	Message capture point not activated
-1225	Failure	Confirmation already exists
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2136	Failure	Resource not found
-2137	Failure	Invalid value for "SELECTION_TYPE"
-2140	Failure	Resource does not match station

17.2 mdcCloseStationMessage

17.2.1 Function description

This function allows multiple machine messages to be ended at the same time; i.e., for the selected machine messages, the end of the validity period is set to the specified time value (DATE_TO). When searching for machine messages, it is possible to filter according to certain control units and the address ranges and variable names used there.



Advice

This function is ONLY intended for machine messages that were captured via data capture points (for further information, see the IMSApi function »mdcUploadDetectionPoint« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcCloseStationMessage
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] closeStationMessageUploadKeys
    String key
    String value
in KeyValue[] closeStationMessageFilter
    String key
    String value
in String[] dataBlockValues
in String[] controlValues
in String[] variableValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) closeStationMessageUploadKeys and closeStationMessageFilter. This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for the function execution

Key	Explanation	IMSApi data type
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☒ DateMs

Tab. 17-7 "Key Value Pair": *closeStationMessageUploadKeys***Filter according to certain parameter values (not currently used)**

Key	Explanation	IMSApi data type
-	- Structure currently not used - Advice: <i>The filter function is intended for future extensions!</i>	☐ String

Tab. 17-8 "Key Value Pair": *closeStationMessageFilter*

17.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
closeStationMessageUploadKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": closeStationMessageUploadKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": closeStationMessageUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
closeStationMessageFilter	- Structure currently not used - Advice: The filter function is intended for future extensions!
key	
value	
dataBlockValues	Address ranges within the control unit (as array) for ascertaining the desired machine messages. Advice: At least one of the <code>dataBlockValues</code> , <code>controlValues</code> or <code>variableValues</code> parameters must be passed. All three parameters are "AND" linked. As a result, only messages that meet all used filter conditions are ascertained. In the array itself, the values are "OR" linked!
controlValues	Name of the control units (as array) for ascertaining the desired machine messages. Advice: See the advice notice under <code>dataBlockValues</code> !
variableValues	Name of the address range of the control unit (as array) for ascertaining the desired machine messages. Advice: See the advice notice under <code>dataBlockValues</code> !

Tab. 17-9 Parameter (in) »mdcCloseStationMessage«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

17.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1236	Failure	At least one filter value must be specified
-2070	Failure	IMSApi key was passed multiple times to the function

17.3 mdcCreateConditionCode

17.3.1 Function description

This function creates machine conditions (on the basis of SEMI E10) and machine messages on the system and assigns them directly to the desired stations.



Advice

The corresponding machine condition classes/groups and the machine message classes/groups cannot be created with the function; they must already exist in the system (for further information, see the "Machine information" setup dialog in the myiTAC workplace). Further information on this topic can be found in the documentation for the ADM module!

Whether or not a machine condition or machine message is created depends on whether the passed class/group is an item from the machine conditions or machine messages area.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	-------------------

Function declaration

```
int mdcCreateConditionCode
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String conditionClass
    in String conditionGroup
    in String conditionCode
    in String conditionName
    in String conditionDesc
    in String conditionCause
    in String conditionType
    in String colorRGB
    in String[] stationNumbers
    in String[] lineNumbers
```

17.3.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system

Parameter name	Explanation
<code>conditionClass</code>	Name of the machine condition class or of the machine message class Advice: Whether a machine condition or a machine message is created depends on whether the passed class/group is an item from the machine conditions or machine messages "tree"!
<code>conditionGroup</code>	Name of the machine condition group or of the machine message group Advice: Whether a machine condition or a machine message is created depends on whether the passed class/group is an item from the machine conditions or machine messages "tree"!
<code>conditionCode</code>	Unique designator of the (new) machine condition or of the machine message
<code>conditionName</code>	Name of the new machine condition or of the machine message
<code>conditionDesc</code>	Description of the new machine condition or of the machine message
<code>conditionCause</code>	Cause of the machine condition [INTERNAL; EXTERNAL]: INTERNAL = Identifies internal causes EXTERNAL = Identifies external causes Advice: This parameter is only relevant for the creation of machine conditions!
<code>conditionType</code>	Machine condition type [O; Q, S; T]: O = Organization Q = Quality S = Setup T = Technology Advice: This parameter is only relevant for the creation of machine conditions! If an empty string or [-1] is passed, the parameter is not evaluated (machine messages).
<code>colorRGB</code>	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!
<code>stationNumbers</code>	Station numbers of the work stations (as array); the stations passed here are assigned the machine condition or the machine message. Advice: If an empty string or [-1] is passed, the parameter is not evaluated. In addition or alternatively, the <code>lineNumbers</code> parameter can also be used. If neither the <code>stationNumbers</code> nor the <code>lineNumbers</code> parameter is used, the machine condition or the machine message is still created!
<code>lineNumbers</code>	Line numbers of the "PM" type (as array); all stations of the specified line(s) are assigned the machine condition or the machine message. Advice: If an empty string or [-1] is passed, the parameter is not evaluated. In addition or alternatively, the <code>stationNumbers</code> parameter can also be used. If neither the <code>stationNumbers</code> nor the <code>lineNumbers</code> parameter is used, the machine condition or the machine message is still created!

Tab. 17-10 Parameter (in) »mdcCreateConditionCode«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

17.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-196	Failure	A parameter is invalid
-1207	Failure	Machine condition type not found [O; Q; S; T]
-1208	Failure	Machine condition/message group not found
-1209	Failure	Machine condition/message already exists
-1210	Failure	Machine condition/message class not found

17.4 mdcCreateLog

17.4.1 Function description

This function creates a log book entry (type "Log") for the passed station. This can be used to store individual pieces of information in the system in the context of machine data collection (MDC). Log book entries can also be viewed in the myiTAC workplace with the "Log book" MDC function. Further information on this topic can be found in the documentation for the PM module.



Advice

This function should NO LONGER be used for new installations because the edit options have been extended with release 8.52.00 (for further information, see the new function »mdcCreateLogData« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcCreateLog
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in DateMs startDate
  in DateMs endDate
  in String subject
  in String text
  in int forLine
```

17.4.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
startDate	Start of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!

Parameter name	Explanation
endDate	End of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!
subject	Subject line for the log book entry (max. 30 characters)
text	Text of the log book entry (max. 250 characters)
forLine	Entry for station or line [0; 1]: 0 = Entry only applies for the passed station 1 = Entry applies for all stations of the line

Tab. 17-11 Parameter (in) »mdcCreateLog«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

17.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-2011	Failure	End date precedes start date

17.5

mdcCreateLogData

17.5.1

Function description

This function creates a log book entry (type "Log") for the passed stations. This can be used to store individual pieces of information in the system in the context of machine data collection (MDC). Log book entries can also be viewed in the myiTAC workplace with the "Log book" MDC function. Further information on this topic can be found in the documentation for the PM module!

**Advice**

The array returned contains records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

*For new installations, it is **essential** to use this API function because the functionality has been extended compared to the »mdcCreateLog« [▶▶▶] function!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	-------------------

Function declaration

```
int mdcCreateLogData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String subject
  in int forLine
  in String[] stationLogKeys
  in String[] stationLogValues
  out String[] stationLogResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) stationLogKeys, stationLogValues and (out) stationLogResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Input values for the log book entry to be created

Key	Explanation		IMSApi data type
DATE_FROM	Start of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☒	DateMs
DATE_TO	End of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Advice: If the key is not passed or passed with [-1], the current date and time are used by default!	☐	DateMs
DETECTION_POINT_NAME	Name of the data capture point	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐	int
INFO_TXT_1	Individual additional information 1 on the log book entry	☐	String
INFO_TXT_10	Individual additional information 10 on the log book entry	☐	String
INFO_TXT_11	Individual additional information 11 on the log book entry	☐	String
INFO_TXT_12	Individual additional information 12 on the log book entry	☐	String
INFO_TXT_13	Individual additional information 13 on the log book entry	☐	String
INFO_TXT_14	Individual additional information 14 on the log book entry	☐	String
INFO_TXT_15	Individual additional information 15 on the log book entry	☐	String
INFO_TXT_2	Individual additional information 2 on the log book entry	☐	String
INFO_TXT_3	Individual additional information 3 on the log book entry	☐	String
INFO_TXT_4	Individual additional information 4 on the log book entry	☐	String
INFO_TXT_5	Individual additional information 5 on the log book entry	☐	String
INFO_TXT_6	Individual additional information 6 on the log book entry	☐	String
INFO_TXT_7	Individual additional information 7 on the log book entry	☐	String
INFO_TXT_8	Individual additional information 8 on the log book entry	☐	String
INFO_TXT_9	Individual additional information 9 on the log book entry	☐	String
RESOURCE_NAME	Name of the resource	☐	String
TEXT	Text of the log book entry (max. 250 characters)	☐	String

Tab. 17-12 "Key Value Pair": stationLogKeys

17.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
subject	Subject line for the log book entry (max. 30 characters)
forLine	Entry for station or line [0; 1]: 0 = Entry only applies for the passed station 1 = Entry applies for all stations of the line
stationLogKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>stationLogValues</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": stationLogKeys table. Advice: Only the parameters passed here are processed by the function!
stationLogValues	Values for the parameters selected by means of <code>stationLogKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>stationLogKeys</code> ! Refer to the " Key Value Pair ": stationLogKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 17-13 Parameter (in) »mdcCreateLogData«

Parameter (out)

Parameter name	Explanation
stationLogResult-Values	Depending on parameter (in) <code>stationLogKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>stationLogKeys</code> !

Tab. 17-14 Parameter (out) »mdcCreateLogData«

17.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-156	Failure	Date value is invalid
-1217	Failure	Capture point not found
-2011	Failure	End date precedes start date
-2013	Failure	Value is outside of the validity range
-2070	Failure	IMSApi key was passed multiple times to the function
-2118	Failure	Value has fallen below the minimum size
		Advice: The value -2118 is output if the value for the DATE - FROM key is below the limit value (01-01-1970, 0:00:00.000)!

-2136	Failure	Resource not found
-2140	Failure	Resource does not match station
-2141	Failure	Resource and data capture point do not match

17.6

mdcGetConditionCodes

17.6.1

Function description

This function returns all active machine conditions and machine messages. It is possible to specify whether only the entries assigned to the passed station or all entries in the system are output. Furthermore, it is possible to filter for a certain machine condition group or machine message group.



Advice

The assignment of machine conditions/messages to a station is performed in the myiTAC workplace with the "Machine setup" function. Further information on this topic can be found in the documentation for the ADM module!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetConditionCodes
    IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in KeyValue[] conditionCodeFilter
        String key
        String value
    in String[] conditionCodeResultKeys
    out String[] conditionCodeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `conditionCodeFilter` and `conditionCodeResultKeys` as well as (out) `conditionCodeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
ALL_CONDITIONS	Determines whether only the entries assigned to the passed station or all entries in the system are output [0; 1]: 0 = Station 1 = All	☐ int
CONDITION_GROUP	Name of the machine condition group or machine message group (no use of wildcard) Advice: <i>How the value passed here is interpreted depends on the setting for TYPE!</i>	☐ String
DYNAMIC_MESSAGE_ENABLED	"Dynamic messages" designator [0; 1]: 0 = only normal messages (FALSE) 1 = only dynamic messages (TRUE) Advice: <i>If the parameter is not passed, there is no restriction!</i>	☐ boolean
TYPE	Machine information type [0; 1; 2]: 0 = machine conditions and machine messages 1 = machine condition 2 = machine message	☐ String
USAGE_MODE	Usage designator of the parent machine condition group" [A; H; M]: A = Without specific usage specification (All) H = "Human"-specific states (Human) M = "Machine"-specific states (Machine)	☐ String

Tab. 17-15 "Key Value Pair": `conditionCodeFilter`

Definition of the output parameters for the found machine conditions or machine messages

Key	Explanation	IMSApi data type
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: <i>In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!</i>	☐ String
CONDITION_CAUSE	Machine condition cause [EXT; INT; TAD]: EXT = Marks the entry with "external" INT = Marks the entry with "internal" TAD = Marks the entry with "time adjustment" Advice: <i>This parameter is output for machine conditions only; the machine condition cause is defined by the parent machine condition class and influences e.g. calculation of the key performance indicator "Downtime"!</i>	☐ String
CONDITION_CLASS	Name of the machine condition class or of the machine message class	☐ String
CONDITION_CLASS_DESC	Description of the machine condition group or of the machine message group	☐ String
CONDITION_CODE	Unique designator of the machine condition or of the machine message	☐ String
CONDITION_DESC	Description of the machine condition or of the machine message	☐ String
CONDITION_GROUP	Name of the machine condition group or of the machine message group	☐ String

Key	Explanation		IMSApi data type
CONDITION_GROUP_DESC	Description of the machine condition group or of the machine message group	☐	String
CONDITION_NAME	Name of the machine condition or of the machine message	☐	String
CONDITION_TYPE	Machine condition type [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology Advice: This parameter is only output for machine conditions!	☐	String
DESCRIPTION_LONG	Message text with placeholders for the dynamic values. Advice: This text component together with dynamic values forms the dynamic message. The placeholders (%1\$s) represent the position of the dynamic values. The "number" in the placeholder determines which dynamic value is at which position in the message text! The keys DYNAMIC_CONTENT and DYNAMIC_MESSAGE mentioned in the example below are output e.g. in the case of the IMSApi function »mdcGetStationConditions« [▶▶▶]. Example: DESCRIPTION_LONG = The %1\$s %3\$s fox jumps over the %2\$s dog. DYNAMIC_CONTENT = quick lazy brown DYNAMIC_MESSAGE = The quick brown fox jumps over the lazy dog. Placeholder 1 "%1\$s" represents the value "quick" Placeholder 2 "%3\$s" represents the value "brown" Placeholder 3 "%2\$s" represents the value "lazy"	☐	String
DYNAMIC_MESSAGE_ENABLED	"Dynamic messages" designator [0; 1]: 0 = only normal messages (FALSE) 1 = only dynamic messages (TRUE)	☐	boolean
LEVEL_2	Short form of the Operating condition description (level 2) Advice: This parameter is only output for machine conditions!	☐	String
LEVEL_2_DESC	Operating condition description (Level 2) acc. to SEMI E10 Advice: This parameter is only output for machine conditions!	☐	String
LEVEL_3	Short form of the Operating condition description (level 3) Advice: This parameter is only output for machine conditions!	☐	String
LEVEL_3_DESC	Operating condition description (Level 3) acc. to SEMI E10 Advice: This parameter is only output for machine conditions!	☐	String
LEVEL_4	Short form of the Operating condition description (level 4) Advice: This parameter is only output for machine conditions!	☐	String
LEVEL_4_DESC	Operating condition description (Level 4) acc. to SEMI E10 Advice: This parameter is only output for machine conditions!	☐	String
MESSAGE_TYPE_NAME	Name of the message type	☐	String

Key	Explanation	IMSApi data type
TYPE	Machine information type [1; 2]: 1 = Machine condition 2 = Machine message	☐ String
USAGE_MODE	Usage designator of the parent machine condition group" [A; H; M]: A = Without specific usage specification (All) H = "Human"-specific states (Human) M = "Machine"-specific states (Machine)	☐ String

Tab. 17-16 "Key Value Pair": *conditionCodeResultKeys*

17.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
conditionCodeFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <i>value</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": conditionCodeFilter table.
value	Value list for the parameters passed by means of <i>key</i> . Advice: Refer to the "Key Value Pair": conditionCodeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
conditionCodeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": conditionCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>conditionCodeResultValues</i> !

Tab. 17-17 Parameter (in) »mdcGetConditionCodes«

Parameter (out)

Parameter name	Explanation
conditionCodeResultValues	Depending on parameter (in) <i>conditionCodeResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>conditionCodeResultKeys</i> !

Tab. 17-18 Parameter (out) »mdcGetConditionCodes«

17.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function

17.7

mdcGetKeyPerformanceIndicators

17.7.1

Function description

This function returns the KPIs (KPI = **Key Performance Indicator**) available for the passed station within a certain period under review. Other values used to calculate the KPIs can also be output (quantities, order-based cycle times, time events, machine conditions, order-based processing performance).

**Advice**

A description of the KPIs listed here can be found in the reference section of the user's guide under »Characteristic values in the iTAC system«.

This API function is influenced by station parameter code nos. 20000 [▶▶▶], 20001 [▶▶▶], 20002 [▶▶▶], 20003 [▶▶▶], 20004 [▶▶▶], 20013 [▶▶▶] and 20014 [▶▶▶] as well as site parameter code nos. 20000 [▶▶▶], 20001 [▶▶▶], 20002 [▶▶▶], 20003 [▶▶▶], 20004 [▶▶▶], 20013 [▶▶▶] and 20014 [▶▶▶]. Further information on this topic can be found in the reference section of the documentation.

Furthermore, the API function is influenced by the server parameter *maximumDays*; the maximum period under review is specified there in days. If the period is exceeded, the function returns -1230 as the output value (error code). Further information on this topic can be found in the user's guide under ADM module → Server configuration → View "ConfigFramework" (path: *ApplicationServer/imsapi/mdc/mdcGetKeyPerformanceIndicators - maximumDays*).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetKeyPerformanceIndicators
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in DateMs fromDate
in DateMs toDate
in String[] performanceIndicatorResultKeys
out String[] performanceIndicatorResultValues
in String[] mdeAmountResultKeys
out String[] mdeAmountResultValues
in String[] mdePlannedCycleTimeResultKeys
out String[] mdePlannedCycleTimeResultValues
in String[] mdeShiftTimeResultKeys
out String[] mdeShiftTimeResultValues
in String[] mdeConditionTimeResultKeys
out String[] mdeConditionTimeResultValues
in String[] mdePerformanceResultKeys
out String[] mdePerformanceResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) *performanceIndicatorResultKeys*, *mdeAmountResultKeys*, *mdePlannedCycleTimeResultKeys*, *mdeShiftTimeResultKeys*, *mdeConditionTimeResultKeys* and *mdePerformanceResultKeys* as well as (out) *performanceIndicatorResultValues*, *mdeAmountResultValues*, *mdePlannedCycleTimeResultValues*, *mdeShiftTimeResultValues*, *mdeConditionTimeResultValues*

and `mdePerformanceResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Definition of the output parameters for the found KPIs

Key	Explanation	IMSApi data type
AVAILABILITY	Availability: The availability describes the ratio between the manufacturing time reported/calculated as "productive" (production time) and the production time plus the reported/calculated downtime. Advice: When interpreting the value, note the associated availability type (see the key <code>AVAILABILITY_TYPE</code>)!	☐ double
AVAILABILITY_TYPE	Availability type [0; 1; 2; 3]: 0 = total availability 1 = organizational availability 2 = technical availability 3 = system-related availability	☐ int
CYCLE_TIME	Cycle time describes the time required to manufacture a unit at a station (work step) in relation to the selected period under review.	☐ double
DATE_FROM	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DATE_TO	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DOWNTIME	In the iTAC system, downtimes are divided into "internal" or "external" downtimes depending on the cause stored for the machine condition (output in seconds): - Downtime (ext.): also referred to as "external downtime" - Downtime (int.): also referred to as "internal downtime"	☐ long
DOWNTIME_EXT	Downtime (ext.): Also referred to as "external downtime" (output in seconds)	☐ long
DURATION	Time range between <code>DATE_FROM</code> and <code>DATE_TO</code> (output in seconds)	☐ int
EFFICIENCY	Efficiency describes the productivity of the machine; here, the specified production time per part (Planned cycle time) is multiplied by the units actually manufactured in the period under review and then expressed as a ratio to the manufacturing time reported/calculated as "productive" (Production time).	☐ double
FIRST_PASS_YIELD	FPY describes the first pass yield at a machine; here, the ratio of unit quantities with the "Pass" state to unit quantities with the "Fail" or "Scrap" state is given as a percentage value.	☐ double

Key	Explanation		IMSApi data type
MTBF	MTBF (Mean Time Between Failures) describes the average duration between two production stoppages which were unplanned and occurred due to technical reasons (in minutes).	☐	double
MTTR	MTTR (Mean Time To Repair) displays the average repair times for all production standstills occurring in the period under review which were unplanned and due to technical reasons.	☐	double
NEE	NEE (Net Equipment Effectiveness) describes the net overall equipment effectiveness and is based on the OEE (gross overall equipment effectiveness).	☐	double
OEE	OEE (Overall Equipment Effectiveness) is a measure indicating the added value of a system (station).	☐	double
OPERATIVE_TIME	Operative time describes the time available for production on the basis of the shift model (output in seconds).	☐	long
PLACING_PERFORMANCE	Placing performance (generally: processing performance) indicates how many units/components were processed on average at a machine; the value is given as "number per hour".	☐	double
PRODUCTION_TIME	Production time (often also referred to as "productive time") is derived from the times which the production process reports as "producing" to the iTAC system (output in seconds).	☐	long
PRODUCTIVITY_1	Productivity I (according to the SEMI standard) describes the ratio between the manufacturing time actually reported/calculated as "productive" in the period under review (Production time) and the maximum time available for production according to the shift model (Operative time). Unlike Productivity II, here the external downtimes also have a negative effect on the result.	☐	double
PRODUCTIVITY_2	Productivity II (according to the SEMI standard) describes the ratio between the manufacturing time actually reported/calculated as "productive" in the period under review (Production time) and the maximum time available for production according to the shift model (Operative time) minus the external downtime (Downtime (ext.)).	☐	double
QUANTITY_FAULT_MESSAGES	Number of malfunction messages	☐	long
QUANTITY_PASS	Number of "pass" bookings	☐	int
QUANTITY_PASS_FPY	Number of "pass" bookings (FPY)	☐	double
QUANTITY_PROCESSED_UNITS	Total number of processed units	☐	long
QUANTITY_TOTAL	Total number of units	☐	double
REPAIRTIME	Repair time (output in seconds)	☐	long
UTILIZATION	Utilization describes the ratio between the maximum time available for production according to the shift model (Operative time) and the manufacturing time actually reported/calculated as "productive" during this period (Production time).	☐	double
YIELD	Yield describes the actual pass yield at a machine; here, the ratio of unit quantities with the "Pass" state to unit quantities with the "Fail" or "Scrap" state is given as a percentage value.	☐	double

Tab. 17-19 "Key Value Pair": *performanceIndicatorResultKeys*

Definition of the output parameters for the found quantities

Key	Explanation	IMSApi data type
DATE_FROM	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DATE_TO	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
QUANTITY_FAIL	Number of "fail" bookings	☐ int
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run)	☐ int
QUANTITY_PASS	Number of "pass" bookings	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings	☐ int

Tab. 17-20 "Key Value Pair": *mdeAmountResultKeys*

Definition of the output parameters for the found order-based cycle times

Key	Explanation	IMSApi data type
CYCLE_TIME	Planned cycle time defines the specified time requirement for manufacturing a unit at a station (work step).	☐ double
WORKORDER_NUMBER	Production order number	☐ String

Tab. 17-21 "Key Value Pair": *mdePlannedCycleTimeResultKeys*

Definition of the output parameters for the found time events

Key	Explanation	IMSApi data type
DATE_FROM	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
DATE_TO	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
PRODUCTIVE	Productive identifier [0; 1]: 0 = Unproductive time event 1 = Productive time event	☐ int
SHIFT_DURATION	Shift duration	☐ double
TIME_EVENT_NAME	Name of the time event	☐ String

Tab. 17-22 "Key Value Pair": *mdeShiftTimeResultKeys*

Definition of the output parameters for the found machine conditions

Key	Explanation	IMSApi data type
CONDITION_CAUSE	Machine condition cause [EXT; INT; TAD]: EXT = Marks the entry with "external" INT = Marks the entry with "internal" TAD = Marks the entry with "time adjustment" Advice: The machine condition cause is defined by the parent machine condition class and influences e.g. calculation of the key performance indicator "Downtime"!	String
CONDITION_DURATION	Duration of the machine condition	long
CONDITION_LEVEL	Operating condition description (according to SEMI E10)	String
CONDITION_NAME	Name of the machine condition	String
DATE_FROM	Start of the machine condition; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
DATE_TO	End of the machine condition; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs

Tab. 17-23 "Key Value Pair": *mdeConditionTimeResultKeys*

Definition of the output parameters for the found order-based processing performance

Key	Explanation	IMSApi data type
CYCLE_TIME	Cycle time describes the time required to manufacture a unit at a station (work step) in relation to the selected period under review.	double
DATE_FROM	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
DATE_TO	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
DURATION	Time range between DATE_FROM and DATE_TO (output in seconds)	int
PERFORMANCE	Processing performance; indicates the average number of units processed at a machine; the value is given as "number per hour".	double
WORKORDER_NUMBER	Production order number	String

Tab. 17-24 "Key Value Pair": *mdePerformanceResultKeys*

17.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
fromDate	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current day with the time 00:00:00.000!
toDate	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes the current day with the time 23:59:59.999!
performanceIndicatorResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": performanceIndicatorResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) performanceIndicatorResultValues!
mdeAmountResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdeAmountResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdeAmountResultValues!
mdePlannedCycleTimeResultKeys	List of the parameters that are to be ascertained by the function. Description of the possible parameters can be found above in the "Key Value Pair": mdePlannedCycleTimeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdePlannedCycleTimeResultValues!
mdeShiftTimeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdeShiftTimeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdeShiftTimeResultValues!
mdeConditionTimeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdeConditionTimeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdeConditionTimeResultValues!

Parameter name	Explanation
mdePerformanceResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdePerformanceResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdePerformanceResultValues !

Tab. 17-25 Parameter (in) »mdcGetKeyPerformanceIndicators«

Parameter (out)

Parameter name	Explanation
performanceIndicatorResultValues	Depending on parameter (in) performanceIndicatorResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) performanceIndicatorResultKeys !
mdeAmountResultValues	Depending on parameter (in) mdeAmountResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdeAmountResultKeys !
mdePlannedCycleTimeResultValues	Depending on parameter (in) mdePlannedCycleTimeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdePlannedCycleTimeResultKeys !
mdeShiftTimeResultValues	Depending on parameter (in) mdeShiftTimeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdeShiftTimeResultKeys !
mdeConditionTimeResultValues	Depending on parameter (in) mdeConditionTimeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdeConditionTimeResultKeys !
mdePerformanceResultValues	Depending on parameter (in) mdePerformanceResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdePerformanceResultKeys !

Tab. 17-26 Parameter (out) »mdcGetKeyPerformanceIndicators«

17.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1230	Failure	Date -> maximum [days] exceeded
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function

17.8

mdcGetLog

17.8.1

Function description

This function returns all booked log book entries (type "Log") for the passed station within a specific review period. Log book entries can also be viewed in the myiTAC workplace with the "Log book" MDC function. Further information on this topic can be found in the documentation for the PM module.

**Advice**

This function should NO LONGER be used for new installations because the edit options have been extended with release 8.52.00 (for further information, see the new function »mdcGetLogData« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetLog
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in DateMs fromDate
in DateMs toDate
out MdcLog[] logArray
    long startDate
    long endDate
    String subject
    String txt
```

17.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
fromDate	Start of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start time!

Parameter name	Explanation
toDate	End of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!

Tab. 17-27 Parameter (in) »mdcGetLog«

Parameter (out)

Parameter name	Explanation
logArray	Structure (array) with the following variables:
startDate	Start of the validity period for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000
endDate	End of the validity period for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000
subject	Subject line for the log book entry
txt	Text of the log book entry

Tab. 17-28 Parameter (out) »mdcGetLog«

17.8.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station

17.9

mdcGetLogData

17.9.1

Function description

This function returns all booked log book entries (type "Log") for the passed stations within a specific review period. Log book entries can also be viewed in the myiTAC workplace with the "Log book" MDC function. Further information on this topic can be found in the documentation for the PM module.

**Advice**

For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »mdcGetLog« [▶▶▶] function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetLogData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] stationNumbers
  in String[] subjects
  in String[] detectionPoints
  in KeyValue[] stationLogFilter
    String key
    String value
  in String[] stationLogResultKeys
  out String[] stationLogResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) stationLogFilter and stationLogResultKeys as well as (out) stationLogResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation		IMSApi data type
DATE_FROM	Start of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the key is not passed or is passed with [-1], 01-01-1970, 0:00:00.000 is used as the start time!	☐	DateMs
DATE_TO	End of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Advice: If the key is not passed or passed with [-1], 12-31-3000, 23:59:59.999 is used as the end time!	☐	DateMs
INFO_TXT_1	Individual additional information 1 on the log book entry	☐	String
INFO_TXT_10	Individual additional information 10 on the log book entry	☐	String
INFO_TXT_11	Individual additional information 11 on the log book entry	☐	String
INFO_TXT_12	Individual additional information 12 on the log book entry	☐	String
INFO_TXT_13	Individual additional information 13 on the log book entry	☐	String
INFO_TXT_14	Individual additional information 14 on the log book entry	☐	String
INFO_TXT_15	Individual additional information 15 on the log book entry	☐	String
INFO_TXT_2	Individual additional information 2 on the log book entry	☐	String
INFO_TXT_3	Individual additional information 3 on the log book entry	☐	String
INFO_TXT_4	Individual additional information 4 on the log book entry	☐	String
INFO_TXT_5	Individual additional information 5 on the log book entry	☐	String
INFO_TXT_6	Individual additional information 6 on the log book entry	☐	String
INFO_TXT_7	Individual additional information 7 on the log book entry	☐	String
INFO_TXT_8	Individual additional information 8 on the log book entry	☐	String
INFO_TXT_9	Individual additional information 9 on the log book entry	☐	String
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int
SORT_FLAG	Sorting mode for output [0; 1]: 0 = Oldest entries first 1 = Most recent entries first Advice: If the key is not passed, the log book entries are output unsorted!	☐	int
TEXT	Text of the log book entry (max. 250 characters)	☐	String

Tab. 17-29 "Key Value Pair": stationLogFilter

Definition of the output parameters for the found data

Key	Explanation		IMSApi data type
DATE_FROM	Start of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_TO	End of the period under review for the log book entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DETEC-TION_POINT_NAME	Name of the data capture point	☐	String
INFO_TXT_1	Individual additional information 1 on the log book entry	☐	String
INFO_TXT_10	Individual additional information 10 on the log book entry	☐	String
INFO_TXT_11	Individual additional information 11 on the log book entry	☐	String
INFO_TXT_12	Individual additional information 12 on the log book entry	☐	String
INFO_TXT_13	Individual additional information 13 on the log book entry	☐	String
INFO_TXT_14	Individual additional information 14 on the log book entry	☐	String
INFO_TXT_15	Individual additional information 15 on the log book entry	☐	String
INFO_TXT_2	Individual additional information 2 on the log book entry	☐	String
INFO_TXT_3	Individual additional information 3 on the log book entry	☐	String
INFO_TXT_4	Individual additional information 4 on the log book entry	☐	String
INFO_TXT_5	Individual additional information 5 on the log book entry	☐	String
INFO_TXT_6	Individual additional information 6 on the log book entry	☐	String
INFO_TXT_7	Individual additional information 7 on the log book entry	☐	String
INFO_TXT_8	Individual additional information 8 on the log book entry	☐	String
INFO_TXT_9	Individual additional information 9 on the log book entry	☐	String
LOG_SUBJECT	Subject line for the log book entry	☐	String
STATION_NUMBER	Station numbers of the work station	☐	String
TEXT	Text of the log book entry (max. 250 characters)	☐	String

Tab. 17-30 "Key Value Pair": stationLogResultKeys

17.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationNumbers	Station numbers of the work stations (as array) which are to be used for searching for log book entries. Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!
subjects	Subject lines (as array) which are to be used for searching for log book entries. Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!
detectionPoints	Data capture points (as array) which are to be used for searching for log book entries. Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!
stationLogFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": stationLogFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": stationLogFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
stationLogResult-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationLogResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationLogResultValues!

Tab. 17-31 Parameter (in) »mdcGetLogData«

Parameter (out)

Parameter name	Explanation
stationLogResult-Values	Depending on parameter (in) stationLogResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationLogResultKeys!

Tab. 17-32 Parameter (out) »mdcGetLogData«

17.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-156	Failure	Date value is invalid
-174	Failure	Number of records to be output has fallen below the minimum value
-2070	Failure	IMSApi key was passed multiple times to the function
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

17.10 mdcGetMessageType

17.10.1 Function description

This function is used to query the message types existing in the iTAC system. Message types are used in order to specify machine conditions/machine messages more precisely.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetMessageType
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] messageTypeFilter
    String key
    String value
in String[] messageTypeResultKeys
out String[] messageTypeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) messageTypeFilter and messageTypeResultKeys as well as (out) messageTypeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
MESSAGE_TYPE_NAME	Name of the message type Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters! Special case: If the value [-1] is passed, this filter function is deactivated; there is no restriction!	☒ String

Tab. 17-33 "Key Value Pair": messageTypeFilter

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
MESSAGE_TYPE_DESC	Description of the message type	☐ String
MESSAGE_TYPE_NAME	Name of the message type	☐ String

Tab. 17-34 "Key Value Pair": *messageTypeResultKeys*

17.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
messageTypeFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": messageTypeFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": messageTypeFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
messageTypeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": messageTypeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>messageTypeResultValues</code> !

Tab. 17-35 Parameter (in) »mdcGetMessageType«

Parameter (out)

Parameter name	Explanation
messageTypeResultValues	Depending on parameter (in) <code>messageTypeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>messageTypeResultKeys</code> !

Tab. 17-36 Parameter (out) »mdcGetMessageType«

17.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1226	Failure	Message type not found
-2070	Failure	IMSApi key was passed multiple times to the function

17.11

mdcGetRegisteredUser

17.11.1

Function description

This function returns those users who have been logged in on a line (type "PM") over a certain time period. These staff work times can also be evaluated in the PM module with regard to line utilization. Capturing of the times (i.e. the user login and logoff) is performed by means of the API function »mdcRegisterUserAtLine« [▶▶▶].

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetRegisteredUser
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mdcGetRegisteredUserFilter
    String key
    String value
in String mdcGetRegisteredUserKeys
out String mdcGetRegisteredUserValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mdcGetRegisteredUserFilter and mdcGetRegisteredUserKeys as well as (out) mdcGetRegisteredUserValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes along the current date and time; if the key is not passed along, the date and time of 12-31-3000, 23:59:59.999 is used as end time!	☐	DateMs
LINE_NUMBER	Line number ("PM" type) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
STATION_NUMBER	In the iTAC system, station number of the work station that was used to log in on the line	☐	String
USER_NAME	Login name of the user (user identification) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String

Tab. 17-37 "Key Value Pair": *mdcGetRegisteredUserFilter*

Definition of the output parameters for the found data

Key	Explanation		IMSApi data type
DATE_FROM	Booking time of the login process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_TO	Booking time of the logout process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If the user is still logged in, 12-31-3000, 23:59:59.999 is output as the end time!	☐	DateMs
LINE_NUMBER	Line number ("PM" type)	☐	String
STATION_NUMBER	In the iTAC system, station number of the work station that was used to log in on the line	☐	String
USER_FIRST_NAME	First name of the user	☐	String
USER_LAST_NAME	Last name of the user	☐	String
USER_NAME	Login name of the user (user identification)	☐	String

Tab. 17-38 "Key Value Pair": *mdcGetRegisteredUserKeys*

17.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mdcGetRegisteredUserFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mdcGetRegisteredUserFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mdcGetRegisteredUserFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mdcGetRegisteredUserKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mdcGetRegisteredUserKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) mdcGetRegisteredUserValues !

Tab. 17-39 Parameter (in) »mdcGetRegisteredUser«

Parameter (out)

Parameter name	Explanation
mdcGetRegisteredUserValues	Depending on parameter (in) mdcGetRegisteredUserKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mdcGetRegisteredUserKeys !

Tab. 17-40 Parameter (out) »mdcGetRegisteredUser«

17.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-156	Failure	Date value is invalid
-2070	Failure	IMSApi key was passed multiple times to the function

17.12

mdcGetStationConditions

17.12.1

Function description

This function returns all booked machine conditions and machine messages for the passed station within a specific review period.

**Advice**

Booked machine conditions/messages can also be viewed in the myiTAC workplace with the "Analysis/capture" MDC function. Further information on this topic can be found in the documentation for the PM module!

This function should NO LONGER be used for new installations because the edit options have been extended with release 8.52.00 (for further information, see the new function »mdcGetStationConditionsByFilter« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	-------------------

Function declaration

```
int mdcGetStationConditions
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in DateMs fromDate
in DateMs toDate
in String conditionType
in int objectType
in String[] conditionResultKeys
out String[] conditionResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) conditionResultKeys and (out) conditionResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Definition of the output parameters for machine conditions
(objectType = 1)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text	☐	String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification)	☐	String
ACTION_PLAN	Description of the action to be performed Advice: This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!	☐	String
ADDITIONAL_DATA	Detailed information on the machine-condition booking	☐	String
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!	☐	String
CONDITION_CODE	Unique designator of the machine condition	☐	String
CONDITION_DESC	Description of the machine condition	☐	String
CONDITION_ID	Internal identification number of the machine condition	☐	long
CONDITION_NAME	Name of the machine condition	☐	String
CONDITION_TYPE	Machine condition type [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology	☐	String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DESCRIPTION_LONG	The text with the placeholders for the dynamic values is returned here.	☐	String
DETECTION_POINT_NAME	Name of the data capture point	☐	String
INFO_TXT_1	Individual additional information 1 on the machine condition	☐	String
INFO_TXT_10	Individual additional information 10 on the machine condition	☐	String
INFO_TXT_11	Individual additional information 11 on the machine condition	☐	String
INFO_TXT_12	Individual additional information 12 on the machine condition	☐	String
INFO_TXT_13	Individual additional information 13 on the machine condition	☐	String
INFO_TXT_14	Individual additional information 14 on the machine condition	☐	String

Key	Explanation	IMSApi data type
INFO_TXT_15	Individual additional information 15 on the machine condition	☐ String
INFO_TXT_2	Individual additional information 2 on the machine condition	☐ String
INFO_TXT_3	Individual additional information 3 on the machine condition	☐ String
INFO_TXT_4	Individual additional information 4 on the machine condition	☐ String
INFO_TXT_5	Individual additional information 5 on the machine condition	☐ String
INFO_TXT_6	Individual additional information 6 on the machine condition	☐ String
INFO_TXT_7	Individual additional information 7 on the machine condition	☐ String
INFO_TXT_8	Individual additional information 8 on the machine condition	☐ String
INFO_TXT_9	Individual additional information 9 on the machine condition	☐ String
LEVEL_4	Operating condition description (Level 4) acc. to SEMI E10 (Semi-conductor Equipment and Materials International)	☐ String
MESSAGE_CAUSE_DESC	Description of the message cause Advice: <i>This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!</i>	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_DESC	Description of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_NAME	Name of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_DESC	Description of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_TYPE_DESC	Description of the message type	☐ String
MESSAGE_TYPE_NAME	Name of the message type	☐ String
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ int
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String
STATION_DESC	Station description of the work station	☐ String
STATION_NUMBER	Station number of the work station	☐ String
TEXT	Text for the machine condition	☐ String
USER_NAME	Login name of the user (user identification) who made the last change (see also the CHANGE_DATE key)	☐ String

Tab. 17-41 "Key Value Pair" (machine conditions): *conditionResultKeys*

Definition of the output parameters for machine messages
(objectType = 2)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text	☐	String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification)	☐	String
ACTION_PLAN	Description of the action to be performed Advice: This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!	☐	String
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!	☐	String
CONDITION_CODE	Unique designator of the machine message	☐	String
CONDITION_DESC	Description of the machine message	☐	String
CONDITION_ID	Internal identification number of the machine message	☐	long
CONDITION_NAME	Name of the machine message	☐	String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DESCRIPTION_LONG	Message text with placeholders for the dynamic values. Advice: This text component together with DYNAMIC_CONTENT forms the dynamic message DYNAMIC_MESSAGE. The placeholders (%1\$s) represent the position of the dynamic values (DYNAMIC_CONTENT). The "number" in the placeholder determines which dynamic value is at which position in the message text! Example: DESCRIPTION_LONG = The %1\$s %3\$s fox jumps over the %2\$s dog. DYNAMIC_CONTENT = quick lazy brown DYNAMIC_MESSAGE = The quick brown fox jumps over the lazy dog. Placeholder 1 "%1\$s" represents the value "quick" Placeholder 2 "%3\$s" represents the value "brown" Placeholder 3 "%2\$s" represents the value "lazy"	☐	String
DETECTION_POINT_NAME	Name of the data capture point	☐	String
DYNAMIC_CONTENT	Dynamic values. Advice: The individual values are separated by " " (PIPE); this text component together with DESCRIPTION_LONG forms the dynamic message DYNAMIC_MESSAGE; for details, see the parameter DESCRIPTION_LONG!	☐	String

Key	Explanation	IMSApi data type
DYNAMIC_MESSAGE	Dynamic message Advice: <i>The dynamic message consists of the message text with the DESCRIPTION_LONG placeholders and the DYNAMIC_CONTENT dynamic values; for details, see the parameter DESCRIPTION_LONG!</i>	☐ String
DYNAMIC_MESSAGE_ENABLED	"Dynamic messages" designator [0; 1]: 0 = only normal messages (FALSE) 1 = only dynamic messages (TRUE)	☐ boolean
INFO_TXT_1	Individual additional information 1 on the machine message	☐ String
INFO_TXT_10	Individual additional information 10 on the machine message	☐ String
INFO_TXT_11	Individual additional information 11 on the machine message	☐ String
INFO_TXT_12	Individual additional information 12 on the machine message	☐ String
INFO_TXT_13	Individual additional information 13 on the machine message	☐ String
INFO_TXT_14	Individual additional information 14 on the machine message	☐ String
INFO_TXT_15	Individual additional information 15 on the machine message	☐ String
INFO_TXT_2	Individual additional information 2 on the machine message	☐ String
INFO_TXT_3	Individual additional information 3 on the machine message	☐ String
INFO_TXT_4	Individual additional information 4 on the machine message	☐ String
INFO_TXT_5	Individual additional information 5 on the machine message	☐ String
INFO_TXT_6	Individual additional information 6 on the machine message	☐ String
INFO_TXT_7	Individual additional information 7 on the machine message	☐ String
INFO_TXT_8	Individual additional information 8 on the machine message	☐ String
INFO_TXT_9	Individual additional information 9 on the machine message	☐ String
MESSAGE_CAUSE_DESC	Description of the message cause Advice: <i>This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!</i>	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_DESC	Description of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_NAME	Name of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_DESC	Description of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_TYPE_DESC	Description of the message type	☐ String
MESSAGE_TYPE_NAME	Name of the message type	☐ String

Key	Explanation		IMSApi data type
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: See the advice notice under MESSAGE_CAUSE_DESC!	☐	int
REMARK	Individual comment on the data capture point	☐	String
RESOURCE_DESC	Description of the resource	☐	String
RESOURCE_NAME	Name of the resource	☐	String
STATION_DESC	Station description of the work station	☐	String
STATION_NUMBER	Station number of the work station	☐	String
TEXT	Text for the machine message	☐	String
USER_NAME	Login name of the user (user identification) who made the last change (see also the CHANGE_DATE key)	☐	String

Tab. 17-42 "Key Value Pair" (machine messages): *conditionResultKeys*

17.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
fromDate	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
toDate	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!
conditionType	Machine condition type [O; Q, S; T]: O = Organization Q = Quality S = Setup T = Technology Advice: If an empty string or [-1] is passed, the field is not evaluated; this also applies if machine messages are to be ascertained (see objectType = 2)!
objectType	Machine information type [1; 2]: 1 = Machine condition 2 = Machine message
conditionResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): conditionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) conditionResultValues!

Tab. 17-43 Parameter (in) »mdcGetStationConditions«

Parameter (out)

Parameter name	Explanation
conditionResultValues	Depending on parameter (in) conditionResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) conditionResultKeys!

Tab. 17-44 Parameter (out) »mdcGetStationConditions«

17.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-156	Failure	Date value is invalid
-198	Failure	Invalid value for "objectType"
-2070	Failure	IMSApi key was passed multiple times to the function

17.13

mdcGetStationConditionsByFilter

17.13.1

Function description

This function returns the booked machine conditions and machine messages within a specific review period. The function enables simultaneous searching using any stations (`stationArray`), any machine conditions/messages (`conditionCodeArray`), any machine condition types (`conditionTypeArray`), any message types (`messageTypeArray`), any data capture points (`detectionPointArray`) as well as any resource (`resourceArray`).

**Advice**

Booked machine conditions/messages can also be viewed in the myiTAC workplace with the "Analysis/capture" MDC function. Further information on this topic can be found in the documentation for the PM module!

*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »mdcGetStationConditions« [▶▶▶] function!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcGetStationConditionsByFilter
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String[] stationArray
in String[] conditionCodeArray
in String[] conditionTypeArray
in String[] messageTypeArray
in String[] detectionPointArray
in String[] resourceArray
in KeyValue[] conditionResultFilter
    String key
    String value
in String[] conditionResultKeys
out String[] conditionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `conditionResultFilter` and `conditionResultKeys` as well as (out) `conditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
CONDITION_ID	Internal identification number of the machine condition or of the machine message	☐ long
DATE_FROM	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☒ DateMs
DATE_TO	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!	☒ DateMs
DYNAMIC_MESSAGE_ENABLED	"Dynamic messages" designator [0; 1]: 0 = only normal messages (FALSE) 1 = only dynamic messages (TRUE) Advice: If the parameter is not passed, there is no restriction!	☐ boolean
IS_ACKNOWLEDGED	"Confirmation designator" mode [0; 1]: 0 = Only unconfirmed machine conditions/messages 1 = Only confirmed machine conditions/messages Advice: If the key is not passed, confirmed and unconfirmed entries are output!	☐ int
IS_OPEN	Filter for open machine messages/conditions 0 = All machine messages/conditions 1 = Only open machine messages/conditions 2 = Only completed machine messages/conditions	☐ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [100000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒ int
SORT_FLAG	Sorting mode for output [0; 1]: 0 = Oldest entries first 1 = Most recent entries first Advice: If the key is not passed, the found entries are output unsorted!	☐ int

Tab. 17-45 "Key Value Pair": *conditionResultFilter*

Definition of the output parameters for machine conditions
(objectType = 1)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text	☐	String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification)	☐	String
ACTION_PLAN	Description of the action to be performed Advice: This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!	☐	String
ADDITIONAL_DATA	Detailed information on the machine-condition booking	☐	String
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!	☐	String
CONDITION_CODE	Unique designator of the machine condition	☐	String
CONDITION_DESC	Description of the machine condition	☐	String
CONDITION_ID	Internal identification number of the machine condition	☐	long
CONDITION_NAME	Name of the machine condition	☐	String
CONDITION_TYPE	Machine condition type [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology	☐	String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DESCRIPTION_LONG	The text with the placeholders for the dynamic values is returned here.	☐	String
DETECTION_POINT_NAME	Name of the data capture point	☐	String
INFO_TXT_1	Individual additional information 1 on the machine condition	☐	String
INFO_TXT_10	Individual additional information 10 on the machine condition	☐	String
INFO_TXT_11	Individual additional information 11 on the machine condition	☐	String
INFO_TXT_12	Individual additional information 12 on the machine condition	☐	String
INFO_TXT_13	Individual additional information 13 on the machine condition	☐	String
INFO_TXT_14	Individual additional information 14 on the machine condition	☐	String

Key	Explanation	IMSApi data type
INFO_TXT_15	Individual additional information 15 on the machine condition	☐ String
INFO_TXT_2	Individual additional information 2 on the machine condition	☐ String
INFO_TXT_3	Individual additional information 3 on the machine condition	☐ String
INFO_TXT_4	Individual additional information 4 on the machine condition	☐ String
INFO_TXT_5	Individual additional information 5 on the machine condition	☐ String
INFO_TXT_6	Individual additional information 6 on the machine condition	☐ String
INFO_TXT_7	Individual additional information 7 on the machine condition	☐ String
INFO_TXT_8	Individual additional information 8 on the machine condition	☐ String
INFO_TXT_9	Individual additional information 9 on the machine condition	☐ String
LEVEL_4	Operating condition description (Level 4) acc. to SEMI E10 (Semi-conductor Equipment and Materials International)	☐ String
MESSAGE_CAUSE_DESC	Description of the message cause Advice: <i>This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!</i>	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_DESC	Description of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_CLASS_NAME	Name of the message class Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_DESC	Description of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ String
MESSAGE_TYPE_DESC	Description of the message type	☐ String
MESSAGE_TYPE_NAME	Name of the message type	☐ String
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: <i>See the advice notice under MESSAGE_CAUSE_DESC!</i>	☐ int
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String
STATION_DESC	Station description of the work station	☐ String
STATION_NUMBER	Station number of the work station	☐ String
TEXT	Text for the machine condition	☐ String
USER_NAME	Login name of the user (user identification) who made the last change (see also the CHANGE_DATE key)	☐ String

Tab. 17-46 "Key Value Pair" (machine conditions): *conditionResultKeys*

Definition of the output parameters for machine messages
(objectType = 2)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text	☐	String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification)	☐	String
ACTION_PLAN	Description of the action to be performed Advice: This parameter is only output if a data capture point (see the DETECTION_POINT_NAME key) has been assigned!	☐	String
CHANGE_DATE	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!	☐	String
CONDITION_CODE	Unique designator of the machine message	☐	String
CONDITION_DESC	Description of the machine message	☐	String
CONDITION_ID	Internal identification number of the machine message	☐	long
CONDITION_NAME	Name of the machine message	☐	String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
DESCRIPTION_LONG	Message text with placeholders for the dynamic values. Advice: This text component together with DYNAMIC_CONTENT forms the dynamic message DYNAMIC_MESSAGE. The placeholders (%1\$s) represent the position of the dynamic values (DYNAMIC_CONTENT). The "number" in the placeholder determines which dynamic value is at which position in the message text! Example: DESCRIPTION_LONG = The %1\$s %3\$s fox jumps over the %2\$s dog. DYNAMIC_CONTENT = quick lazy brown DYNAMIC_MESSAGE = The quick brown fox jumps over the lazy dog. Placeholder 1 "%1\$s" represents the value "quick" Placeholder 2 "%3\$s" represents the value "brown" Placeholder 3 "%2\$s" represents the value "lazy"	☐	String
DETECTION_POINT_NAME	Name of the data capture point	☐	String
DYNAMIC_CONTENT	Dynamic values. Advice: The individual values are separated by " " (PIPE); this text component together with DESCRIPTION_LONG forms the dynamic message DYNAMIC_MESSAGE; for details, see the parameter DESCRIPTION_LONG!	☐	String

Key	Explanation	IMSApi data type
DYNAMIC_MESSAGE	Dynamic message Advice: The dynamic message consists of the message text with the <code>DESCRIPTION_LONG</code> placeholders and the <code>DYNAMIC_CONTENT</code> dynamic values; for details, see the parameter <code>DESCRIPTION_LONG</code> !	☐ String
DYNAMIC_MESSAGE_ENABLED	"Dynamic messages" designator [0; 1]: 0 = only normal messages (FALSE) 1 = only dynamic messages (TRUE)	☐ boolean
INFO_TXT_1	Individual additional information 1 on the machine message	☐ String
INFO_TXT_10	Individual additional information 10 on the machine message	☐ String
INFO_TXT_11	Individual additional information 11 on the machine message	☐ String
INFO_TXT_12	Individual additional information 12 on the machine message	☐ String
INFO_TXT_13	Individual additional information 13 on the machine message	☐ String
INFO_TXT_14	Individual additional information 14 on the machine message	☐ String
INFO_TXT_15	Individual additional information 15 on the machine message	☐ String
INFO_TXT_2	Individual additional information 2 on the machine message	☐ String
INFO_TXT_3	Individual additional information 3 on the machine message	☐ String
INFO_TXT_4	Individual additional information 4 on the machine message	☐ String
INFO_TXT_5	Individual additional information 5 on the machine message	☐ String
INFO_TXT_6	Individual additional information 6 on the machine message	☐ String
INFO_TXT_7	Individual additional information 7 on the machine message	☐ String
INFO_TXT_8	Individual additional information 8 on the machine message	☐ String
INFO_TXT_9	Individual additional information 9 on the machine message	☐ String
MESSAGE_CAUSE_DESC	Description of the message cause Advice: This parameter is only output if a data capture point (see the <code>DETECTION_POINT_NAME</code> key) has been assigned!	☐ String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: See the advice notice under <code>MESSAGE_CAUSE_DESC</code> !	☐ String
MESSAGE_CLASS_DESC	Description of the message class Advice: See the advice notice under <code>MESSAGE_CAUSE_DESC</code> !	☐ String
MESSAGE_CLASS_NAME	Name of the message class Advice: See the advice notice under <code>MESSAGE_CAUSE_DESC</code> !	☐ String
MESSAGE_EFFECT_DESC	Description of the message effect Advice: See the advice notice under <code>MESSAGE_CAUSE_DESC</code> !	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: See the advice notice under <code>MESSAGE_CAUSE_DESC</code> !	☐ String
MESSAGE_TYPE_DESC	Description of the message type	☐ String
MESSAGE_TYPE_NAME	Name of the message type	☐ String

Key	Explanation	IMSApi data type
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: See the advice notice under MESSAGE_CAUSE_DESC!	☐ int
REMARK	Individual comment on the data capture point	☐ String
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String
STATION_DESC	Station description of the work station	☐ String
STATION_NUMBER	Station number of the work station	☐ String
TEXT	Text for the machine message	☐ String
USER_NAME	Login name of the user (user identification) who made the last change (see also the CHANGE_DATE key)	☐ String

Tab. 17-47 "Key Value Pair" (machine messages): *conditionResultKeys*

17.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system – here for licensing only (cf. stationArray)
objectType	Machine information type [1; 2]: 1 = Machine condition 2 = Machine message
stationArray	Station numbers of the work stations (as array) which are to be used for searching for machine conditions/messages. Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!
conditionCodeArray	Unique designators of machine conditions/machine messages (as array) Advice: Wildcards are supported (for further information, see the advice notice under stationArray)!

Parameter name	Explanation
conditionTypeArray	Machine condition types (as array) which are to be used for searching for machine conditions [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology Advice: If a parameter is not passed, the field is not evaluated (no restriction). If machine messages are to be ascertained (object-Type = 2), this parameter is not interpreted because machine messages do not recognize machine condition types.
messageTypeArray	Message types (as array) which are to be used for searching for machine conditions/messages Advice: Wildcards are supported (for further information, see the advice notice under <code>stationArray</code>)!
detectionPointArray	Data capture points (as array) which are to be used for searching for machine conditions/messages. Advice: Wildcards are supported (for further information, see the advice notice under <code>stationArray</code>)!
resourceArray	Resource names (as array) which are to be used for searching for machine conditions/messages. Advice: Wildcards are supported (for further information, see the advice notice under <code>stationArray</code>)!
conditionResultFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": conditionResultFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": conditionResultFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
conditionResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": conditionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>conditionResultValues</code> !

Tab. 17-48 Parameter (in) »mdcGetStationConditionsByFilter«

Parameter (out)

Parameter name	Explanation
conditionResultValues	Depending on parameter (in) <code>conditionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>conditionResultKeys</code> !

Tab. 17-49 Parameter (out) »mdcGetStationConditionsByFilter«

17.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-156	Failure	Date value is invalid
-174	Failure	Number of records to be output has fallen below the minimum value
-198	Failure	Invalid value for "objectType"
-2070	Failure	IMSApi key was passed multiple times to the function
-2119	Failure	Number of records to be output has exceeded the maximum value

17.14

mdcManageConditionCode

17.14.1

Function description

Using this function, machine conditions (based on SEMI E10) and machine messages in the system can be managed; this includes creation as well as direct assignment to the desired stations.

The corresponding machine condition classes/groups and the machine message classes/groups cannot be created with the function; they must already exist in the system (for further information, see the "Machine information" setup dialog in the myiTAC workplace). Further information on this topic can be found in the documentation for the ADM module!



Advice

*In the array returned, only **incorrectly** processed records (these also include warnings) with a corresponding error code (reason for the failure or warning) are returned (see the `ERROR_CODE` key). If one or more records have been processed with **failures**, the corresponding technical error code (< 0) is output at the record; in this case, the function itself returns a general error (error code = **-2120**); a so-called "rollback" is performed!*

*If one or more records have been processed with **warnings**, the corresponding technical error code (> 0) is output at the record; in this case, the function itself returns a general warning (error code = **5**); processing is continued (no rollback)!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcManageConditionCode
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] manageConditionCodeUploadKeys
in String[] manageConditionCodeUploadValues
out String[] manageConditionCodeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `manageConditionCodeUploadKeys` and `manageConditionCodeUploadValues` as well as (out) `manageConditionCodeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
COLOR_RGB	Color definition (RGB) for the entry; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: <i>In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!</i> Application type "Create machine condition": If the key is not passed, the color definition is adopted from the <code>CONDITION_GROUP</code>. If no group is passed either, the color definition is adopted from the following default group: UA-L3 / UA-L4 / Unassigned class / Unassigned group Application type "Create machine message": If the key is not passed, the color definition is adopted from the specified <code>CONDITION_GROUP</code>.	☐	String
CONDITION_CAUSE	Machine condition cause [EXT; INT; TAD]: EXT = Marks the entry with "external" INT = Marks the entry with "internal" TAD = Marks the entry with "time adjustment" Advice: <i>This parameter is only relevant for the creation of machine conditions; the machine condition cause is defined by the parent machine condition class and influences e.g. calculation of the key performance indicator "Downtime"!</i>	☐	String
CONDITION_CLASS	Name of the machine condition class or of the machine message class Advice: <i>Whether a machine condition or a machine message is created depends on whether the passed class/group is an item from the machine conditions or machine messages "tree"! If both <code>CONDITION_GROUP</code> and <code>CONDITION_CLASS</code> are passed, the class must be contained in the group (consistency check).</i> Application type "Create machine condition": If no class is passed, the machine condition is stored under the specified <code>CONDITION_GROUP</code>. If no group is passed either, the machine condition is stored under the following default group: UA-L3 / UA-L4 / Unassigned class / Unassigned group Application type "Create machine message": If a machine message is to be created, specification of the class is only required if a consistency check for the specified <code>CONDITION_GROUP</code> is to be performed.	☐	String
CONDITION_CODE	Unique designator of the (new) machine condition or of the machine message	☒	String
CONDITION_DESC	Description of the new machine condition or of the machine message	☐	String

Key	Explanation		IMSApi data type
CONDITION_GROUP	Name of the machine condition group or of the machine message group Advice: Whether a machine condition or a machine message is created depends on whether the passed group is an item from the machine conditions or machine messages "tree"! If both <code>CONDITION_GROUP</code> and <code>CONDITION_CLASS</code> are passed, the class must be contained in the group (consistency check). Application type "Create machine condition": If a group is passed, the machine condition is stored under the following default group: UA-L3 / UA-L4 / Unassigned class / Unassigned group Application type "Create machine message": If a machine message is to be created, a valid machine message group must be passed using the key.	☐	String
CONDITION_NAME	Name of the new machine condition or of the machine message	☒	String
CONDITION_TYPE	Machine condition type [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology Advice: This parameter is only relevant for the creation of machine conditions!	☐	String
DESCRIPTION_LONG	Message text with placeholders for the dynamic values. Advice: This text component together with dynamic values forms the dynamic message. The placeholders (%1\$s) represent the position of the dynamic values. The "number" in the placeholder determines which dynamic value is at which position in the message text! The keys <code>DYNAMIC_CONTENT</code> and <code>DYNAMIC_MESSAGE</code> mentioned in the example below are output e.g. in the case of the IMSApi function »mdcGetStationConditions« [▶▶▶]. Example: <code>DESCRIPTION_LONG</code> = The %1\$s %3\$s fox jumps over the %2\$s dog. <code>DYNAMIC_CONTENT</code> = quick lazy brown <code>DYNAMIC_MESSAGE</code> = The quick brown fox jumps over the lazy dog. Placeholder 1 "%1\$s" represents the value "quick" Placeholder 2 "%3\$s" represents the value "brown" Placeholder 3 "%2\$s" represents the value "lazy"	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: If a record (array element) cannot be processed, the corresponding technical error code for the failure (< 0) is output here; in this case, the function itself outputs a general error message (error code = -2120)! If a record has been processed with warnings, the corresponding technical error code for the warning (> 0) is output here; in this case, the function itself outputs a general warning (error code = 5)!	☐	int
LINE_NUMBER	Line number ("PM" type); all stations of the specified line are assigned the machine condition or the machine message. Advice: In addition or alternatively, the <code>STATION_NUMBER</code> key can also be used. If neither the <code>LINE_NUMBER</code> key nor the <code>STATION_NUMBER</code> key is used, the machine condition and machine message is still created in the system!	☐	String

Key	Explanation	IMSApi data type
MESSAGE_TYPE_NAME	Name of the message type	☐ String
STATION_NUMBER	Station number of the work stations; the station passed here is assigned the machine condition or the machine message. Advice: In addition or alternatively, the LINE_NUMBER key can also be used. If neither the STATION_NUMBER key nor the LINE_NUMBER key is used, the machine condition or machine message is still created in the system!	☐ String

Tab. 17-50 "Key Value Pair": *manageConditionCodeUploadKeys*

17.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
manageCondition-CodeUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>manageConditionCodeUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": manageConditionCodeUploadKeys table. Advice: Only the parameters passed here are processed by the function!
manageCondition-CodeUploadValues	Values for the parameters selected by means of <code>manageConditionCodeUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>manageConditionCodeUploadKeys</code>! Refer to the "Key Value Pair": manageConditionCodeUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 17-51 Parameter (in) »mdcManageConditionCode«

Parameter (out)

Parameter name	Explanation
manageCondition-CodeResultValues	The corresponding data is output here only in the event of an error (this also includes warnings) – depending on the parameters (in) <code>manageConditionCodeUploadKeys</code> and <code>manageConditionCodeUploadValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>manageConditionCodeUploadKeys</code>!

Tab. 17-52 Parameter (out) »mdcManageConditionCode«

17.14.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
9	Warning	Machine condition/message already exists
8	Warning	Machine condition/message already assigned
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-109	Failure	Line not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-196	Failure	A parameter is invalid
-1207	Failure	Machine condition type not found [O; Q; S; T]
-1208	Failure	Machine condition/message group not found
-1210	Failure	Machine condition/message class not found
-1222	Failure	Message cause not found
-1226	Failure	Message type not found

-1227	Failure	Machine condition class and group not consistent
-2070	Failure	IMSApi key was passed multiple times to the function
-2120	Failure	At least one record in the array could not be processed
-2143	Failure	No stations for the line

17.15 mdcRegisterUserAtLine

17.15.1 Function description

This function logs a user onto or off of a line (type "PM") and thereby enables person-related time acquisition. These staff work times can be evaluated later in the PM module with regard to line utilization or output using the »mdcGetRegisteredUser« [▶▶▶] API function.



Advice

This API function is influenced by station parameter code no. 8300 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int mdcRegisterUserAtLine
  in IMSEApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String userId
  in String password
  in int registrationType
  in DateMs bookDate
```

17.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: The line is ascertained based on the station number!
userId	Login name of the user (user identification)
password	Password of the user Advice: The correct password is only required for user login; for logoff (registrationType = 0), value [-1] can also be passed here!
registrationType	Login / logoff [0; 1]: 0 = User is logged off 1 = User is logged in
bookDate	Booking time of the login/logout process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, users can also be logged in / off with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 17-53 Parameter (in) »mdcRegisterUserAtLine«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

17.15.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
		Advice: The value -4 is also output if the specified password is invalid!
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-193	Failure	Invalid value for "registrationType" [0; 1]
-1202	Failure	Team has no access authorization for the line
-1211	Failure	Start time already used
-2105	Failure	No TR or PM license present for the station

17.16 mdcUpdateStationCondition

17.16.1 Function description

This function enables an already captured machine message/machine condition to be edited by specifying the internal identification number (see the input parameter `conditionId`). For example, it can be used to book confirmations at a later time (cf. the IMSApi function »`mdcAcknowledgeStationConditions`« [▶▶▶]) or to amend the booked values. The `objectType` function parameter is used to determine whether a machine-condition booking or a machine-message booking is to be changed.



Advice

The function has two function modes:

- 1.) (Subsequent) **confirmation** of a booked record
- 2.) (Subsequent) **editing** of a booked record

Which function mode is active depends on whether the `ACKNOWLEDGEMENT_DATE` key is passed or not!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcUpdateStationCondition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in long conditionId
in KeyValue[] stationConditionUpdateKeys
    String key
    String value
out String[] stationConditionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `stationConditionUpdateKeys` as well as (out) `stationConditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Specifications for "Machine condition" object type (objectType = 1)

Key	Explanation		IMSApi data type
ACKNOWLEDGEMENT_ - DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If this key is used, the "Confirmation" function mode is active (only the keys ACKNOWLEDGEMENT_ DATE, ACKNOWLEDGEMENT_ TXT and ACKNOWLEDGEMENT_ USER_ ID are then evaluated). If the key is not passed, the "Editing" function mode is applicable; only then are all other keys interpreted!	☐	DateMs
ACKNOWLEDGEMENT_ TXT	Individual confirmation text Advice: This key is only evaluated if a valid value has been passed for ACKNOWLEDGEMENT_ DATE!	☐	String
ACKNOWLEDGEMENT_ US- ER_ ID	Login name of the "confirming" user (user identification) Advice: This key is only evaluated if a valid value has been passed for ACKNOWLEDGEMENT_ DATE!	☐	String
ACTION_ PLAN	Description of the action to be performed Advice: This key is only relevant if data capture points are used in the iTAC system!	☐	String
HISTORIZE	History function [0; 1]: 0 = data are NOT historicized 1 = data are historicized Advice: If this key is not passed along, the [0] value is used by default!	☐	int
INFO_ TXT_ 1	Individual additional information 1 on the machine condition	☐	String
INFO_ TXT_ 10	Individual additional information 10 on the machine condition	☐	String
INFO_ TXT_ 11	Individual additional information 11 on the machine condition	☐	String
INFO_ TXT_ 12	Individual additional information 12 on the machine condition	☐	String
INFO_ TXT_ 13	Individual additional information 13 on the machine condition	☐	String
INFO_ TXT_ 14	Individual additional information 14 on the machine condition	☐	String
INFO_ TXT_ 15	Individual additional information 15 on the machine condition	☐	String
INFO_ TXT_ 2	Individual additional information 2 on the machine condition	☐	String
INFO_ TXT_ 3	Individual additional information 3 on the machine condition	☐	String
INFO_ TXT_ 4	Individual additional information 4 on the machine condition	☐	String
INFO_ TXT_ 5	Individual additional information 5 on the machine condition	☐	String
INFO_ TXT_ 6	Individual additional information 6 on the machine condition	☐	String
INFO_ TXT_ 7	Individual additional information 7 on the machine condition	☐	String
INFO_ TXT_ 8	Individual additional information 8 on the machine condition	☐	String
INFO_ TXT_ 9	Individual additional information 9 on the machine condition	☐	String

Key	Explanation	IMSApi data type
MESSAGE_CAUSE_NAME	Name of the message cause Advice: This key is only relevant if data capture points are used in the iTAC system!	☐ String
MESSAGE_CLASS_NAME	Name of the message class Advice: This key is only relevant if data capture points are used in the iTAC system!	☐ String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: This key is only relevant if data capture points are used in the iTAC system!	☐ String
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: This key is only relevant if data capture points are used in the iTAC system!	☐ int
REMARK	Individual comment on the data capture point	☐ String
TEXT	Text for the machine condition	☐ String

Tab. 17-54 "Key Value Pair" (machine condition): `stationConditionUpdateKeys`

Specifications for "Machine message" object type (`objectType = 2`)

Key	Explanation	IMSApi data type
ACKNOWLEDGEMENT_DATE	Time stamp for the confirmation point in time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If this key is used, the "Confirmation" function mode is active (only the keys <code>ACKNOWLEDGEMENT_DATE</code> , <code>ACKNOWLEDGEMENT_TXT</code> and <code>ACKNOWLEDGEMENT_USER_ID</code> are then evaluated). If the key is not passed, the "Editing" function mode is applicable; only then are all other keys interpreted!	☐ DateMs
ACKNOWLEDGEMENT_TXT	Individual confirmation text Advice: This key is only evaluated if a valid value has been passed for <code>ACKNOWLEDGEMENT_DATE</code> !	☐ String
ACKNOWLEDGEMENT_USER_ID	Login name of the "confirming" user (user identification) Advice: This key is only evaluated if a valid value has been passed for <code>ACKNOWLEDGEMENT_DATE</code> !	☐ String
ACTION_PLAN	Description of the action to be performed Advice: This key is only relevant if data capture points are used in the iTAC system!	☐ String
DATE_FROM	Start of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs

Key	Explanation		MSApi data type
DATE_TO	End of the validity period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 31.12.3000, 23:59:59.999 as the end of the period under review!	☐	DateMs
HISTORIZE	History function [0; 1]: 0 = data are NOT historicized 1 = data are historicized Advice: If this key is not passed along, the [0] value is used by default!	☐	int
INFO_TXT_1	Individual additional information 1 on the machine message	☐	String
INFO_TXT_10	Individual additional information 10 on the machine message	☐	String
INFO_TXT_11	Individual additional information 11 on the machine message	☐	String
INFO_TXT_12	Individual additional information 12 on the machine message	☐	String
INFO_TXT_13	Individual additional information 13 on the machine message	☐	String
INFO_TXT_14	Individual additional information 14 on the machine message	☐	String
INFO_TXT_15	Individual additional information 15 on the machine message	☐	String
INFO_TXT_2	Individual additional information 2 on the machine message	☐	String
INFO_TXT_3	Individual additional information 3 on the machine message	☐	String
INFO_TXT_4	Individual additional information 4 on the machine message	☐	String
INFO_TXT_5	Individual additional information 5 on the machine message	☐	String
INFO_TXT_6	Individual additional information 6 on the machine message	☐	String
INFO_TXT_7	Individual additional information 7 on the machine message	☐	String
INFO_TXT_8	Individual additional information 8 on the machine message	☐	String
INFO_TXT_9	Individual additional information 9 on the machine message	☐	String
MESSAGE_CAUSE_NAME	Name of the message cause Advice: This key is only relevant if data capture points are used in the iTAC system!	☐	String
MESSAGE_CLASS_NAME	Name of the message class Advice: This key is only relevant if data capture points are used in the iTAC system!	☐	String
MESSAGE_EFFECT_NAME	Name of the message effect Advice: This key is only relevant if data capture points are used in the iTAC system!	☐	String
PRIORITY	Number for individual prioritization of the message [1 - 100] Advice: This key is only relevant if data capture points are used in the iTAC system!	☐	int
REMARK	Individual comment on the data capture point	☐	String
TEXT	Text for the machine message	☐	String

Tab. 17-55 "Key Value Pair" (machine message): stationConditionUpdateKeys

17.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Machine information type [1; 2]: 1 = Machine condition 2 = Machine message
conditionId	Internal identification number of the machine condition or of the machine message Advice: The ID can be ascertained with »mdcGetStationConditionsByFilter« [▶▶▶]!
stationConditionUpdateKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": stationConditionUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": stationConditionUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 17-56 Parameter (in) »mdcUpdateStationCondition«

Parameter (out)

Parameter name	Explanation
stationConditionResultValues	Depending on parameter (in) <code>stationConditionUpdateKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>stationConditionUpdateKeys</code> !

Tab. 17-57 Parameter (out) »mdcUpdateStationCondition«

17.16.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-198	Failure	Invalid value for "objectType"
-1200	Failure	Machine condition/message not found
-1204	Failure	Changing of the machine message not allowed
-1221	Failure	Message class not found
-1222	Failure	Message cause not found
-1223	Failure	Message effect not found
-1225	Failure	Confirmation already exists

-2011	Failure	End date precedes start date
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function

17.17

mdcUploadDetectionPoint

17.17.1

Function description

This function enables the capture of data capture points of type "message" (machine conditions/messages) or of type "measurement data" (measured values). Which data capture points are assigned to which station can be found in the setup dialog for the data capture points in the ADM module. The captured data can then be found in the PM module under the "Machine data collection" function (log book).



Advice

This API function is influenced by station parameters code no. 7223 [▶▶▶] and 7224 [▶▶▶] (consecutive failure check) as well as 7225 [▶▶▶] (automatic offset for DATE_ - TO for machine messages). Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Special characteristics for "Message" type: Because only one machine condition can be active on a station at a given point in time, the machine conditions that lie in the validity period passed here are overwritten. This does not apply for machine messages, as several different machine messages may be active at a station at a given point in time.

Application type "already active machine condition": If the validity ranges of the passed machine condition overlap with an active machine condition, the active entry is overwritten. Depending on the setting for the DATE_FROM and DATE_TO parameters, the validity range is shortened accordingly. Depending on where the validity ranges overlap, the validity range of the existing entry begins later (on the DATE_FROM of the new entry) or the validity range of the existing entry begins later (on the DATE_TO of the new entry).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR PM license
--	-----------------------------------

Function declaration

```
int mdcUploadDetectionPoint
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String[] detectionPointKeys
    in String[] detectionPointValues
    out String[] detectionPointResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) detectionPointKeys and detectionPointValues as well as (out) detectionPointResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed or passed with [-1], the current date and time are used! Advice: This key is only relevant for capturing data capture points of type "measurement data"!	☐	DateMs
CONDITION_CODE	Unique designator of the machine condition or of the machine message in the iTAC system. Advice: This key is only relevant for capturing data capture points of type "message"!	☐	String
DATE_FROM	Start of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and time; for the application type "already active machine condition", see description further above! Advice: This key is only relevant for capturing data capture points of type "message"!	☐	DateMs
DATE_TO	End of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Advice: This key is only relevant for capturing data capture points of type "message"! Special case (machine condition): [-1] passes 12-31-3000, 23:59:59.999 as the end time; for the application type "already active machine condition", see description further above! Special case (machine message): [-1] passes the DATE_FROM plus 5 minutes as the end time by default; if a different offset is to be used, this can be adjusted with station parameter code no. 7225 [▶▶▶]!	☐	DateMs
DETECTION_POINT_NAME	Name of the data capture point	☒	String

Key	Explanation	IMSApi data type
DYNAMIC_CONTENT	Dynamic values. Advice: The individual values are separated by " " (PIPE); this text component together with the message text (with placeholders) forms the dynamic message; the values passed here replace the placeholders in the message text! Advice: This key is only relevant for capturing data capture points of type "message"!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MEASURE_VALUE	Measured value Advice: This key is only relevant for capturing data capture points of type "measurement data"!	☐ String

Tab. 17-58 "Key Value Pair": *detectionPointKeys*

17.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: The station specified here is used for the license check only; the station to which the data is booked is ascertained via the used capture point (see the DETECTION_POINT_NAME key)!
detectionPointKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the detectionPointValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": detectionPointKeys table. Advice: Only the parameters passed here are processed by the function!
detectionPointValues	Value list for the parameters passed by means of detectionPointKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: Refer to the "Key Value Pair": detectionPointKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 17-59 Parameter (in) »mdcUploadDetectionPoint«

Parameter (out)

Parameter name	Explanation
detectionPointResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) detectionPointKeys and detectionPointValues. Advice: The data is output in the order in which it was entered by means of parameter (in) detectionPointKeys!

Tab. 17-60 Parameter (out) »mdcUploadDetectionPoint«

17.17.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
211	Warning	A secondary failure was detected
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-1200	Failure	Machine condition/message not found
-1201	Failure	Machine condition/message is not assigned to the passed station
-1203	Failure	Condition is not a line condition
-1204	Failure	Changing of the machine message not allowed
-1205	Failure	Machine condition / message has an invalid state
-1206	Failure	Station is in the secondary failure state (no updating possible)
-1212	Failure	Message capture point invalid -> No condition/message assigned!
-1213	Failure	Message capture point invalid -> No station assigned!

-1214	Failure	Message capture point invalid
-1215	Failure	Message capture point has already been booked
-1216	Failure	Message capture point has not yet been booked
-1217	Failure	Capture point not found
-1218	Failure	Message capture point not activated
-1232	Failure	Measurement data capture point not valid
-1233	Failure	Measurement data capture point not activated
-1234	Failure	Measurement data capture point -> Station not valid
-1235	Failure	Measurement data capture point -> Recipe not valid
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

17.18

mdcUploadStationCondition

17.18.1

Function description

This function enables the booking of machine conditions or machine messages. As a rule, it can be assumed that the machine conditions to be booked exist in the system (for further information, see also »[mdcCreateConditionCode](#)« [▶▶▶]). If necessary, this function can also be used to create new machine conditions in the system (see parameter `BOOKING_TARGET = 2`).



Advice

*This API function is influenced by station parameters code no. 7223 [▶▶▶] and 7224 [▶▶▶] (consecutive failure check) as well as 7225 [▶▶▶] (automatic offset for `DATE_TO` for machine messages). Further information on this topic can be found in the reference section of this documentation! Booked machine conditions or messages can be viewed in the myiTAC workplace with the "Analysis/capture" MDE function (for further information, see the documentation for the PM module). The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Because only one machine condition can be active on a station at a given point in time, the machine conditions that lie in the validity period passed here are overwritten. This does not apply for machine messages, as several different machine messages may be active at a station at a given point in time.

Application type "already active machine condition": If the validity ranges of the passed machine condition overlap with an active machine condition, the active entry is overwritten. Depending on the setting for the `DATE_FROM` and `DATE_TO` parameters, the validity range is shortened accordingly. Depending on where the validity ranges overlap, the validity range of the existing entry begins later (on the `DATE_FROM` of the new entry) or the validity range of the existing entry begins later (on the `DATE_TO` of the new entry).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int mdcUploadStationCondition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] stationConditionUploadKeys
in String[] stationConditionUploadValues
out String[] stationConditionResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `stationConditionUploadKeys` and `stationConditionUploadValues` as well as (out) `stationConditionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ADDITIONAL_DATA	Detailed information on the machine-condition booking	☐	String
BOOKING_TARGET	Entry for station or line [0; 1; 2]: 0 = Entry only applies for the passed station 1 = Entry applies for all stations of the line 2 = Unknown machine conditions are created Advice: For mode [2], the unknown machine conditions are stored in a group provided specifically for this purpose (level 4, name: UA-L4 , description: Not assigned , and then Unassigned class and Unassigned group)!	☒	int
CONDITION_CODE	Unique designator of the machine condition or of the machine message	☒	String
CONDITION_DESC	Description of the machine condition or of the machine message	☐	String
CONDITION_NAME	Name of the machine condition or of the machine message	☐	String
DATE_FROM	Start of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and current time; for the application type "already active machine condition", see description further above!	☐	DateMs
DATE_TO	End of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case (machine condition): [-1] passes 12-31-3000, 23:59:59.999 as the end time; for the application type "already active machine condition", see description further above! Special case (machine message): [-1] passes the DATE_FROM plus 5 minutes as the end time by default; if a different offset is to be used, this can be adjusted with station parameter code no. 7225 [▶▶▶]!	☐	DateMs

Key	Explanation	IMSApi data type
DESCRIPTION_LONG	Message text with placeholders for the dynamic values. Advice: This text component together with dynamic values forms the dynamic message. The placeholders (%1\$s) represent the position of the dynamic values. The "number" in the placeholder determines which dynamic value is at which position in the message text! The key DYNAMIC_MESSAGE mentioned in the example below is output e.g. in the case of the IMSApi function »mdcGetStationConditions« [▶▶▶]. Example: DESCRIPTION_LONG = The %1\$s %3\$s fox jumps over the %2\$s dog. DYNAMIC_CONTENT = quick lazy brown DYNAMIC_MESSAGE = The quick brown fox jumps over the lazy dog. Placeholder 1 "%1\$s" represents the value "quick" Placeholder 2 "%3\$s" represents the value "brown" Placeholder 3 "%2\$s" represents the value "lazy"	☐ String
DYNAMIC_CONTENT	Advice: the individual values are separated by " " (PIPE); this text component together with DESCRIPTION_LONG forms the dynamic message; for details, see the parameter DESCRIPTION_LONG!	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
HISTORIZE	History function [0; 1]: 0 = data are NOT historicized 1 = data are historicized Background to "mode [1]": If a machine condition is passed which has the same start date as an existing machine condition, the original record is saved in a history table (historicized).	☐ int
RESOURCE_NAME	Name of the resource	☐ String
TEXT	Text for machine condition or for machine message (max. 200 characters)	☐ String

Tab. 17-61 "Key Value Pair": stationConditionUploadKeys

17.18.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationConditionUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>stationConditionUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": stationConditionUploadKeys table. Advice: Only the parameters passed here are processed by the function!
stationConditionUploadValues	Value list for the parameters passed by means of <code>stationConditionUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: Refer to the "Key Value Pair": stationConditionUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 17-62 Parameter (in) »mdcUploadStationCondition«

Parameter (out)

Parameter name	Explanation
stationConditionResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>stationConditionUploadKeys</code> and <code>stationConditionUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>stationConditionUploadKeys</code> !

Tab. 17-63 Parameter (out) »mdcUploadStationCondition«

17.18.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please contact as soon as possible the iTAC employee responsible for the project!*

Error code	Category	Output text
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-1200	Failure	Machine condition/message not found
-1201	Failure	Machine condition/message is not assigned to the passed station

-1203	Failure	Condition is not a line condition
-1204	Failure	Changing of the machine message not allowed
-1205	Failure	Machine condition / message has an invalid state
-1206	Failure	Station is in the secondary failure state (no updating possible)
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2136	Failure	Resource not found
-2140	Failure	Resource does not match station

18 Group "ML" (material & logistics)

18.1 mlChangeMaterialBinData

18.1.1 Function description

This function changes various parameters (e.g. DateCode, state, original quantity, classification) for a material bin. The quantity passed is subtracted from or added to the quantity already specified on the bin.



Advice

This API function is influenced by site parameter code nos. 7210 [▶▶▶] and 18520 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlChangeMaterialBinData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String materialBinNumber
in KeyValue[] materialBinDataUploadValues
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) material-BinDataUploadValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
APS_TRANSFER	Indicates whether the container – more precisely the current container quantity of the part – is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0;1]: 0 = Container is NOT taken into consideration (FALSE) 1 = Container is taken into consideration and is therefore "APS-relevant" (TRUE) Background: This specification is evaluated if, during work order detailed planning, the material availability is also to be taken into consideration for the parts required for work order production. If the container is "APS-relevant", the current container quantity of this part is taken into consideration during material availability planning.	☐ boolean
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs
CLASSIFICATION	Classification of the container	☐ String
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
HU_NUMBER	Serial number for the lot (HU = Handling Unit) Advice: This key is only relevant if the material bin is a lot!	☐ String
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String
MATERIAL_BIN_QTY_TOTAL	(New) original quantity in the material bin (original container quantity) Advice: A correction of the original quantity affects the current container quantity. If, for example, the original quantity is increased from 100 to 200, the current remaining quantity is also increased by the value 100! The same holds for a reduction of the original quantity; if the reduction in the original quantity causes the current remaining quantity to go negative, the function outputs a warning – provided that the site parameter 7210 [▶▶▶] is set to [2] – (see error code 504)!	☐ double
MATERIAL_BIN_STATE	Container state [B; E; F; Q; R; S; V]: B = Locked (material bin locked) E = Completed (material bin completed) F = Initialized (new material bin created) Q = QA lock (material bin with QA lock) R = Ready (material bin ready) S = In process (material bin in use) V = Pre-release (material bin with pre-release) Advice: States "C" (closed), "L" (MRP LOCKED) and "P" (prepared) are NOT supported in this context!	☐ String
RECEIVING_NUMBER	Receiving number	☐ String

Key	Explanation	IMSApi data type
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String

Tab. 18-1 "Key Value Pair": *materialBinDataUploadValues*

18.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinNumber	Number of the material bin (container no.)
materialBinDataUploadValues	"Key Value Pair" structure with the following variables:
key	List of the parameters to be edited. A description of the possible parameters can be found above in the "Key Value Pair": materialBinDataUploadValues table.
value	Value list for the parameters passed by means of <i>key</i> .

Tab. 18-2 *Parameter (in) »mlChangeMaterialBinData«*



Advice

This function has no output parameters; it only outputs one of the output values listed below!

18.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
504	Warning	The material quantity in the system has become negative
		Advice: The value 504 is only output if site parameter 7210 [▶▶▶] is set to [2]!
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-503	Failure	Container not found
-517	Failure	Retrieval/creation of a supplier not possible
-527	Failure	Retrieval/creation of receiving not possible
-528	Failure	Storage place was not found
-531	Failure	Supplier not found
-578	Failure	The maximum expiration date must NOT be before the expiration date
-2070	Failure	IMSApi key was passed multiple times to the function

-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.2

mlCreateNewMaterialBin

18.2.1

Function description

This function creates a new material bin (container) with the passed parameters and, if necessary, stores them at a storage place that is to be specified. It checks whether the container number already exists. If it exists, is empty, and is not stored at a storage location, the "old" container is deactivated and a new one is created.

If desired, the new material bins can be created with reference to an existing order; the function thereby also performs the tasks of the setup dialog: ML module -> Receiving -> Receiving (order).



Advice

This API function is influenced by station parameter code nos. 21016 [▶▶▶] and 25100 [▶▶▶] as well as site parameter code no. 18520 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlCreateNewMaterialBin
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] materialBinUploadKeys
in String[] materialBinUploadValues
out String[] materialBinResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) materialBinUploadKeys and materialBinUploadValues as well as (out) materialBinResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
APS_TRANSFER	Indicates whether the container – more precisely the current container quantity of the part – is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0;1]: 0 = Container is NOT taken into consideration (FALSE) 1 = Container is taken into consideration and is therefore "APS-relevant" (TRUE) Background: This specification is evaluated if, during work order detailed planning, the material availability is also to be taken into consideration for the parts required for work order production. If the container is "APS-relevant", the current container quantity of this part is taken into consideration during material availability planning.	☐	boolean
BOOK_DATE	Time of creation for the material bin (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, material bins can also be created with a time stamp from the past in this way. Advice: [-1] passes the current date and the current time!	☐	DateMs
DELIVERY_DATE	Point in time at which the material bin was delivered; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
DELIVERY_NOTE	Delivery slip number	☐	String
ECCN_CODE	ECCN (Export Control Classification Number)	☐	String
CLASSIFICATION	Classification of the container	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
HU_NUMBER	Serial number for the lot (HU = Handling Unit) Advice: This key is only relevant if the material bin is a lot!	☐	String
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐	String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒	String

Key	Explanation		IMSApi data type
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system; the entered part number can be interpreted in various ways (for further information, see parameter PART_NUMBER_MODE) Advice: The system decides on the basis of the stored part group whether the container is created as a normal material bin or as a lot for the shipping station (Shipping-Client)!	☒	String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☒	double
MATERIAL_BIN_STATE	Container state [B; E; F; Q; R; S; V]: B = Locked (material bin locked) E = Completed (material bin completed) F = Initialized (new material bin created) Q = QA locked (material bin with QA lock) R = Ready (material bin ready) S = In process (material bin in use) V = Pre-release (material bin with pre-release) States C (closed), L (MRP LOCKED) and P (prepared) are NOT supported in this context. Advice: If the value [-1] is passed here, the "R" state is used by default for normal material containers; if the container is a lot (see MATERIAL_BIN_PART_NUMBER), the "F" state is used!	☐	String
MATERIAL_BIN_TRANS-ACTION_TYPE	Code for the movement type from the iTAC system (see ML module movement types) Advice: If a movement type is not specified, storage is not possible!	☐	int
MSL_FLOOR_LIFE-TIME_REMAIN	Still remaining mounting time (in minutes)	☐	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use Advice: The MSL level is normally stored in the master data (ADM module) based on parts and is therefore automatically valid for all newly created containers; in this case, no entry is required here. This key should only be used if a deviating value which is only valid for the current container is desired. The specification from the master data is not overwritten but only overridden for the current container! Application type "Invalid master data specification": If the check routine determines that no valid MSL level is stored in the master data, the warning 3108 is output for the record at the key ERROR_CODE. In this case, the function itself outputs a general warning (see error code 7); the container is then created without MSL configuration.	☐	String
PART_GROUP	Part group in the iTAC system	☐	String

Key	Explanation		IMSApi data type
PART_NUMBER_MODE	Mode for interpreting the part number passed with MATERIAL_BIN_PART_NUMBER [0; 1; 2]: 0 = No mapping (iTAC part number); the MATERIAL_BIN_PART_NUMBER must match the part number in the iTAC system. 1 = Mapping of the customer material number; in this mode, MATERIAL_BIN_PART_NUMBER must be used to pass the customer-specific part number (customer mat. no.). 2 = Mapping of the manufacturer material number; in this mode, MATERIAL_BIN_PART_NUMBER must be used to pass the manufacturer-specific part number. Advice: In modes [1] and [2] as well, the container is then created on the basis of the ascertained iTAC part number!	☐	int
PART_THICKNESS	Housing thickness of the container material (in millimeters) Advice: The housing thickness is normally stored in the master data (ADM module) based on parts and is therefore automatically valid for all newly created containers; in this case, no entry is required here. This key should only be used if a deviating value which is only valid for the current container is desired. The specification from the master data is not overwritten but only overridden for the current container!	☐	double
PARTFORM_NAME	Name of the form type (housing form)	☐	String
PURCHASE_ORDER_NUMBER	Order number Advice: If the RECEIVING_MODE parameter is passed with the value [2], this field must absolutely be filled!	☐	String
PURCHASE_ORDER_POSITION	Order position Advice: If the RECEIVING_MODE parameter is passed with the value [2], this field must absolutely be filled!	☐	String
PURCHASE_ORDER_TYPE	Order type Advice: If the RECEIVING_MODE parameter is passed with the value [2], this field must absolutely be filled!	☐	String
RECEIVING_INFO	Info on the receiving position	☐	String
RECEIVING_MODE	"Receiving function" mode [0; 1; 2]: 0 = Simple creation of material bins -> without information on receiving position and without reference to an order 1 = Creation of material bins with information on receiving position -> without reference to an order 2 = Creation of material bins with information on receiving position and reference to an order Advice: In mode [1], parameter RECEIVING_NUMBER must absolutely be passed; in mode [2], parameters PURCHASE_ORDER_NUMBER, PURCHASE_ORDER_POSITION and PURCHASE_ORDER_TYPE also must absolutely be passed. If the key is not passed, the value [0] is used by default!	☐	int
RECEIVING_NUMBER	Receiving number	☐	String
RECEIVING_POSITION	Receiving position	☐	int
STAT_GOOD_CODE	Statistical goods number; this number is used for reporting the export and import of goods to the authorities	☐	String

Key	Explanation		IMSApi data type
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Advice: If the container is to be stored, either STORAGE_NUMBER or STORAGE_BARCODE and MATERIAL_BIN_TRANSACTION_TYPE must be passed!	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notices under STORAGE_BARCODE!	☐	String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐	String
SUPPLIER_NAME	Manufacturer name Advice: The keys SUPPLIER_NAME and SUPPLIER_NUMBER can be used to create unknown manufacturers in the system!	☐	String
SUPPLIER_NUMBER	Manufacturer number Advice: The keys SUPPLIER_NAME and SUPPLIER_NUMBER can be used to create unknown manufacturers in the system!	☐	String
WORKORDER_RESERVED_NUMBER	Number of the production order for which the material in the container is to be reserved	☐	String

Tab. 18-3 "Key Value Pair": materialBinUploadKeys

18.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>materialBinUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": materialBinUploadKeys table. Advice: Only the parameters passed here are processed by the function!
materialBinUploadValues	Values for the parameters selected by means of <code>materialBinUploadKeys</code>; no entry is necessary for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>materialBinUploadKeys</code>! Refer to the "Key Value Pair": materialBinUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 18-4 Parameter (in) »mlCreateNewMaterialBin«

Parameter (out)

Parameter name	Explanation
materialBinResultValues	The corresponding data is output here only in the event of an error – depending on the <code>materialBinUploadKeys</code> and <code>materialBinUploadValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>materialBinUploadKeys</code>!

Tab. 18-5 Parameter (out) »mlCreateNewMaterialBin«

18.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3108	Warning	Part has no MSL definition
7	Warning	At least one record of the array has been processed with a warning
		Advice: The value 7 is output only if the check routine determines that no valid MSL level is stored in the master data; in this case, the warning 3108 is output for the record at the key ERROR_CODE; the container is created without MSL configuration!
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-122	Failure	Part group not found
-196	Failure	A parameter is invalid
-503	Failure	Container not found
-504	Failure	An existing bin number can only be created if the old bin is empty

-514	Failure	Invalid character in "materialBinNumber" [?; %; ' ; -1]
-517	Failure	Retrieval/creation of a supplier not possible
-518	Failure	Container state is invalid
-527	Failure	Retrieval/creation of receiving not possible
-528	Failure	Storage place was not found
-531	Failure	Supplier not found
-548	Failure	Failure creating the material lot
-550	Failure	Container number not unique
-560	Failure	Order not found
-561	Failure	Different part in order
-565	Failure	Delivery slip number is invalid
-566	Failure	Delivery slip position is invalid
-578	Failure	The maximum expiration date must NOT be before the expiration date
-2052	Failure	Housing form not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3110	Failure	Invalid value for "MSL_LEVEL"
-3112	Failure	MSL definition not found

18.3 mlGetIncomingGoods

18.3.1 Function description

This function makes it possible to query incoming goods information from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., delivery period, receiving number, supplier). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned. Output occurs separated by receiving header data and the corresponding receiving positions.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetIncomingGoods
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] incomingGoodsFilters
    String key
    String value
  in String[] incomingGoodsHeadResultKeys
  in String[] incomingGoodsItemResultKeys
  out String[] incomingGoodsHeadResultValues
  out String[] incomingGoodsItemResultValues
  out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) incomingGoodsFilters, incomingGoodsHeadResultKeys and incomingGoodsItemResultKeys as well as (out) incomingGoodsHeadResultValues and incomingGoodsItemResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
DELIVERY_DATE_FROM	Time stamp for the start of the period under review (with respect to delivery slip date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DELIVERY_DATE_TO	Time stamp for the end of the period under review (with respect to delivery slip date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
DELIVERY_NOTE	Delivery slip number Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. Advice: In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records! Attention: The maximum number is NOT limited by the system; if the key is not passed, all data records are ascertained and output. With large quantities of data, this can reduce the performance of the system!	☐	int
RECEIVING_DATE_FROM	Time stamp for the start of the period under review (relative to the actual date received); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
RECEIVING_DATE_TO	Time stamp for the start of the period under review (with respect to delivered date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
RECEIVING_NUMBER	Receiving number Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☒	String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
SUPPLIER_NAME	Supplier name Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String
SUPPLIER_NUMBER	Supplier designator (code) Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String

Tab. 18-6 "Key Value Pair": incomingGoodsFilters

Definition of the output parameters for the found incoming goods header data

Key	Explanation	IMSApi data type
DELIVERY_DATE	Delivery time according to delivery slip date; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
DELIVERY_NOTE	Delivery slip number	☐ String
RECEIVING_DATE	Point in time of the receiving booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
RECEIVING_ID	Internal identification number for the receiving number	☐ long
RECEIVING_NUMBER	Receiving number	☐ String
SUPPLIER_NAME	Supplier name	☐ String
SUPPLIER_NUMBER	Supplier designator (code)	☐ String
USER_NAME	Login name of the user (user identification) who created the record	☐ String

Tab. 18-7 "Key Value Pair": *incomingGoodsHeadResultKeys*

Definition of the output parameters for the found receiving positions

Key	Explanation	IMSApi data type
DATE_CHANGED	Date of last change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_CREATED	Date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
DELIVERY_QUANTITY	Delivery quantity for the order position	☐ double
DELIVERY_STATE	State of the receiving position [T; K]: T = open K = complete	☐ String
ECCN_CODE	ECCN (Export Control Classification Number)	☐ String
INFO_TEXT	Individual information text; the text is entered during the creation or editing of a position number	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PURCHASE_ORDER_ITEM	Order position	☐ String
PURCHASE_ORDER_NUMBER	Order number	☐ String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usually used in an ERP system	☐ String

Key	Explanation	IMSApi data type
QUALITY_CHECK	State "QA test": QL = QA locked (entry without test plan) QR = Ready for QA test (entry with test plan) QS = QA test started QT = QA test passed QF = QA test failed QA = QA test failed, still accepted QM = QA test failed; adjust quantity NQ = No QA test A = Accepted R = Rejected (final end state)	String
RECEIVING_ID	Internal identification number for the receiving number	long
RECEIVING_NUMBER	Receiving number	String
RECEIVING_POSITION	Receiving position	int
TRANSACTION_TYPE	Code for the movement type from the iTAC system (see description of the movement types in the ML module)	long
UNIT	Quantity unit	String

Tab. 18-8 "Key Value Pair": *incomingGoodsItemResultKeys*

18.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
incomingGoodsFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": incomingGoodsFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": incomingGoodsFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
incomingGoodsHeadResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": incomingGoodsHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>incomingGoodsHeadResultValues</code> !
incomingGoodsItemResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": incomingGoodsHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>incomingGoodsItemResultValues</code> !

Tab. 18-9 Parameter (in) »mlGetIncomingGoods«

Parameter (out)

Parameter name	Explanation
incomingGoodsHeadResultValues	Depending on parameter (in) <code>incomingGoodsHeadResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>incomingGoodsHeadResultKeys</code> !
incomingGoodsItemResultValues	Depending on parameter (in) <code>incomingGoodsItemResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>incomingGoodsItemResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 18-10 Parameter (out) »mlGetIncomingGoods«

18.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-5	Failure	Password is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.4

mlGetMaterialBinData

18.4.1

Function description

This function uses specific search criteria to ascertain corresponding container data. Available search criteria are among other things part number, part group, container state, change date and the container number itself. In addition, it is also possible to search for containers with specific container attributes. All search criteria are "AND" coupled such that only the container data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetMaterialBinData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] materialBinFilters
    String key
    String value
in AttributeInfo[] attributes
    String attributeCode
    String attributeValue
in String[] materialBinResultKeys
out String[] materialBinResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) materialBinFilters and materialBinResultKeys as well as parameter (out) materialBinResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
DATE_CHANGED	Change date: all containers are ascertained that were created new since this point in time or for which the container state or original container quantity was changed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (MATERIAL_BIN_NUMBER)	☐ String
INCLUDE_EMPTY_BIN	Selection criterion for the containers [0; 1]: 0 = Current containers (with stock) 1 = Current and old containers	☐ int
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system	☐ String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin closed) E = completed (material bin finalized) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = used up (material bin used up) V = pre-release (material bin with pre-release) Advice: Multiple values can also be specified by separating the values with "," (e.g. "B,R")!	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call; the entry is NOT limited by the system, but should not be greater than necessary (excessive data quantities reduce system performance). Advice: If more records exist than are defined here, the function outputs a warning (see error code 3). If the key is not passed, the value [10000] is used by default!	☐ int
PART_GROUP	Part group of the container material in the iTAC system	☐ String
PART_NUMBER_RESERVED	Part number of the production order for which the material was reserved in the container	☐ String
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐ String

Key	Explanation	IMSApi data type
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_RESERVED_NUMBER	Number of the production order for which the material was reserved in the container	☐ String

Tab. 18-11 "Key Value Pair": *materialBinFilters*

Definition of the output parameters for the found containers

Key	Explanation	IMSApi data type
APS_TRANSFER	Indicates whether the container – more precisely the current container quantity of the part – is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0;1]: 0 = Container is NOT taken into consideration (FALSE) 1 = Container is taken into consideration and is therefore "APS-relevant" (TRUE) Background: This specification is evaluated if, during work order detailed planning, the material availability is also to be taken into consideration for the parts required for work order production. If the container is "APS-relevant", the current container quantity of this part is taken into consideration during material availability planning.	☐ boolean
CLASSIFICATION	Classification of the container	☐ String
DATE_CHANGED	Change date: all containers are ascertained that were created new since this point in time or for which the container state or original container quantity was changed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
DATE_CREATED	Date of creation for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: For MSL-relevant containers, the site-related offset is taken into account!	☐ DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
LOCK_STATE	Lock state of the material bin [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐ int
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String

Key	Explanation		IMSApi data type
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system	☒	String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐	double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin closed) E = completed (material bin finalized) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = used up (material bin used up) V = pre-release (material bin with pre-release)	☐	String
MATERIAL_BIN_TYPE	Type of material bin [B; C; I; K; L; S]: B = unit C = container I = inventory (stock container) K = component L = board S = lot (shipping)	☐	String
MSL_DRY_END_DATE	End date of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_FLOOR_LIFE_TIME_REMAIN	Still remaining mounting time (in minutes)	☐	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use	☐	String
MSL_OPEN_DATE	Point in time when the container may be removed from an MSL package (dry pack) or a drying oven/re-drying oven; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (B aking) C = protected/closed(C losed) O = unprotected/open (O pen)	☐	String
PART_COST	Average purchase price [€] of the material (see PART_COST_BASE)	☐	String
PART_COST_BASE	Material quantity on which the purchase price is based (see PART_COST)	☐	String
PART_DESC	Part description in the iTAC system	☐	String

Key	Explanation		IMSApi data type
PART_NUMBER_RESERVED	Part number of the production order for which the material was reserved in the container	☐	String
RECEIVING_NUMBER	Receiving number	☐	String
RECEIVING_POSITION	Receiving position	☐	int
STORAGE_BARCODE	Bar code for the storage place	☐	String
STORAGE_CELL	Storage cell in the iTAC system	☐	String
STORAGE_CELL_DESC	Storage cell description in the iTAC system	☐	String
STORAGE_DESC	Storage description in the iTAC system	☐	String
STORAGE_GROUP	Storage group in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group and subgroups); the various levels are separated by " " (PIPE)!	☐	String
STORAGE_GROUP_DESC	Storage group description in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group description and subgroup descriptions); the various levels are separated by " " (PIPE)!	☐	String
STORAGE_NUMBER	Storage place in the iTAC system	☐	String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐	String
SUPPLIER_NAME	Manufacturer name	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String
UNIT	Unit for the value passed	☐	String
WORKORDER_RESERVED_NUMBER	Number of the production order for which the material was reserved in the container	☐	String

Tab. 18-12 "Key Value Pair": materialBinResultKeys

18.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinFilters	"Key Value Pair" structure with the following variables:
key	List of the container parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter. A description of the possible container parameters can be found above in the "Key Value Pair": materialBinFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": materialBinFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attributes	Array of the structure with the following variables:
attributeCode	List of the container attributes (attribute code) on the basis of which the output is to be filtered; the corresponding values are passed with the <code>attributeValue</code> parameter.
attributeValue	Value list for the container attributes passed by means of <code>attributeCode</code> . Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!
materialBinResultKeys	List of the parameters that are to be output by the function. A description of the possible parameters can be found above in the "Key Value Pair": materialBinResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>materialBinResultValues</code> !

Tab. 18-13 Parameter (in) »mlGetMaterialBinData«

Parameter (out)

Parameter name	Explanation
materialBinResultValues	Depending on parameter (in) <code>materialBinResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>materialBinResultKeys</code> !

Tab. 18-14 Parameter (out) »mlGetMaterialBinData«

18.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2117	Failure	Value has exceeded the maximum size

18.5 mlGetMaterialTransactions

18.5.1 Function description

This function is used to obtain detailed information on order-based material movements in the iTAC system. A wide range of different filter criteria can be applied for the query (e.g. container number, work order number, container state, etc.). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetMaterialTransactions
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] materialTransactionsFilters
    String key
    String value
  in String[] materialTransactionsResultKeys
  out String[] materialTransactionsResultValues
  out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `materialTransactionsFilters` and `materialTransactionsResultKeys` as well as (out) `materialTransactionsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin closed) E = completed (material bin finalized) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = used up (material bin used up) V = pre-release (material bin with pre-release) Advice: Multiple values can also be specified by separating the values with "," (e.g. "C,Q")!	☐ String
MATERIAL_BIN_TYPE	Type of material bin [B; C; I; K; L; S]: B = unit C = container I = inventory (stock container) K = component L = board S = lot (shipping)	☐ String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. The maximum number is limited to [1000] records by the system. Advice: In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☒ int
PART_NUMBER	Part number of the material bin in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐ int

Tab. 18-15 "Key Value Pair": materialTransactionsFilters

Definition of the output parameters for the found material movements

Key	Explanation		IMSApi data type
TRANSACTION_CREATED	Point in time at which the entry was created; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
TRANSACTION_STAMP	Point in time at which the entry was last changed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
COST_CENTER_DESC	Cost center description	☐	String
COST_CENTER_NUMBER	Cost center number	☐	String
INFO	Individual text information on the material movement	☐	String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin closed) E = completed (material bin finalized) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = used up (material bin used up) V = pre-release (material bin with pre-release)	☐	String
MATERIAL_BIN_TRANS-ACTION_DESC	Description of the movement type from the iTAC system	☐	String
MATERIAL_BIN_TRANS-ACTION_DIRECTION	Movement direction [A; E; N]: A = Output E = Input N = Neutral (automatic movement)	☐	String
MATERIAL_BIN_TRANS-ACTION_NAME	Code value for the movement type from the iTAC system (e.g. REC_A, see ML module movement types)	☐	String
MATERIAL_BIN_TRANS-ACTION_NAME_TYPE	Code type of the movement type [A; S; U; X]: A = All S = System U = User X = Universal	☐	String
PART_COST	Average purchase price [€] of the material (see PART_COST_BASE)	☐	String
PART_COST_BASE	Material quantity on which the purchase price is based (see PART_COST)	☐	String
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number of the material bin in the iTAC system	☐	String
QUANTITY	Moved material quantity	☐	double
STORAGE_DESC	Storage description in the iTAC system	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system	☐	String
UNIT	Quantity unit for the moved quantity (see QUANTITY)	☐	String

Key	Explanation	IMSApi data type
WORKORDER_DESC	Production order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 18-16 "Key Value Pair": *materialTransactionsResultKeys*

18.5.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialTransactionsFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": materialTransactionsFilters table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": materialTransactionsFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
materialTransactionsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": materialTransactionsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) materialTransactionsResultValues !

Tab. 18-17 Parameter (in) »mlGetMaterialTransactions«

Parameter (out)

Parameter name	Explanation
materialTransactionsResultValues	Depending on parameter (in) materialTransactionsResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) materialTransactionsResultKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the START_POSITION and MAX_ROW_COUNT keys are to be observed; these determine the start position and the maximum structure size!

Tab. 18-18 Parameter (out) »mlGetMaterialTransactions«

18.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value

18.6 mlGetNextMaterialBinNumber

18.6.1 Function description

Depending on the passed part number, this function returns one unique container number assigned by the iTAC system.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetNextMaterialBinNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String partNumber
out String materialBinNumber
```

18.6.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partNumber	Part number in the iTAC system

Tab. 18-19 Parameter (in) »mlGetNextMaterialBinNumber«

Parameter (out)

Parameter name	Explanation
materialBinNumber	Number of the material bin (container no.)

Tab. 18-20 Parameter (out) »mlGetNextMaterialBinNumber«

18.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-121	Failure	Part not found
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.7 mlGetPurchaseOrderData

18.7.1 Function description

This function can be used to ascertain detailed information on material orders in the iTAC system; this information is usually imported from a connected ERP system. A wide range of different criteria can be applied for the query (e.g. delivery date, part, order state, various manufacturer data, etc.). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetPurchaseOrderData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] purchaseDataFilters
    String key
    String value
  in String[] purchaseDataResultKeys
  out String[] purchaseDataResultValues
  out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `purchaseDataFilters` and `purchaseDataResultKeys` as well as (out) `purchaseDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
DELIVERY_DATE_FROM	Time stamp for the start of the period under review (with respect to the planned delivered date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current day with the time 0:00:00.000 (no value -> no restriction)!	☐	DateMs
DELIVERY_DATE_TO	Time stamp for the start of the period under review (with respect to the planned delivered date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes the current day with the time 23:59:59.999 (no value -> no restriction)!	☐	DateMs
ERP_CONFIRMATION_NO	Order number in the ERP system	☐	String
MANUFACTURER_CODE	Manufacturer's designator (code)	☐	String
MANUFACTURER_NAME	Manufacturer's name	☐	String
MANUFACTURER_PART_DESC	Manufacturer part description	☐	String
MANUFACTURER_PART_NUMBER	Manufacturer part number	☐	String
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with START_POSITION, it is possible to successively "page" through large structures. Advice: In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records! Attention: The maximum number is NOT limited by the system; if the key is not passed, all data records are ascertained and output. With large quantities of data, this can reduce the performance of the system!	☐	int
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PURCHASE_DELIVERY_STATE	Delivery state [0; 1; 19]: 0 = Open 1 = Closed 19 = Deleted Advice: If the key is not passed, only the value [0] is used; if the value [-1] is passed, there is no restriction!	☐	int
PURCHASE_ORDER_ITEM	Order position	☐	String
PURCHASE_ORDER_NUMBER	Order number	☐	String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usually used in an ERP system	☐	String

Key	Explanation	IMSApi data type
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for MAX_ROW_COUNT. Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	int
SUPPLIER_CODE	Supplier designator (code)	String
SUPPLIER_NAME	Supplier name	String

Tab. 18-21 "Key Value Pair": purchaseDataFilters

Definition of the output parameters for the found orders

Key	Explanation	IMSApi data type
CONTRACT_ITEM	Contract position	long
CONTRACT_NUMBER	Contract number	String
CONTRACT_QUANTITY	Contract number	double
CONTRACT_TYPE	Contract type	String
DELIVERY_DATE_FROM	Time stamp for the start of the period under review (with respect to the planned delivered date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
DELIVERY_DATE_PLANNED	Point in time of the "planned delivery date"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
DELIVERY_DATE_TO	Time stamp for the start of the period under review (with respect to the planned delivered date); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
ECCN_CODE	ECCN (Export Control Classification Number)	String
ERP_CONFIRMATION_NO	Order number in the ERP system	String
ERP_CONFIRMATION_TEXT	Order confirmation text from the ERP system	String
ERP_ORDER_STATE	Order confirmation state from the ERP system	int
MANUFACTURER_CODE	Manufacturer's designator (code)	String
MANUFACTURER_NAME	Manufacturer's name	String
MANUFACTURER_PART_DESC	Manufacturer part description	String
MANUFACTURER_PART_NUMBER	Manufacturer part number	String
OVERDELIVERY_TOL	Upper limit for the order quantity (as a percentage)	double

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PURCHASE_DELIVERY- _STATE	Delivery state [0; 1; 19]: 0 = Open 1 = Closed 19 = Deleted	☐ int
PURCHASE_ORDER_ITEM	Order position	☐ String
PURCHASE_ORDER_NUM- BER	Order number	☐ String
PUR- CHASE_ORDER_QUANTI- TY	Order quantity	☐ double
PURCHASE_ORDER_TEXT	Order information	☐ String
PURCHASE_ORDER_TYPE	Order type; the order type is indicated by the three code letters usually used in an ERP system	☐ String
QUANTITY_DELIVERED	Quantity already delivered	☐ double
SUPPLIER_CODE	Supplier designator (code)	☐ String
SUPPLIER_NAME	Supplier name	☐ String
UNDERDELIVERY_TOL	Lower limit for the order quantity (as a percentage)	☐ double

Tab. 18-22 "Key Value Pair": *purchaseDataResultKeys*

18.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
purchaseDataFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": purchaseDataFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": purchaseDataFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
purchaseDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": purchaseDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>purchaseDataResultValues</code> !

Tab. 18-23 Parameter (in) »mlGetPurchaseOrderData«

Parameter name	Explanation
purchaseDataResultValues	Depending on parameter (in) <code>purchaseDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>purchaseDataResultKeys</code> !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 18-24 Parameter (out) »mlGetPurchaseOrderData«

18.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-567	Failure	Order state is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.8

mlGetStorage

18.8.1

Function description

For a given storage location (storage place, storage group or storage cell), this function outputs the underlying or overlying storage structures; the respective structure is then resolved from the top level to the bottom level. The `storageCellNumber` (storage cell), `storageGroupNumber` (storage group) and `storageNumber` (storage place) input parameters are implemented as an "AND" coupled filter such that only the storage structures that meet all used filter conditions are output.

**Advice**

The storage structure in the storage management (ADM module) has been changed as of release 8.51.00 of the iTAC system; in this context, a new IMSApi function for querying the storage data has been provided (see the function »mlGetStorageData« [\[▶▶▶\]](#))!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetStorage
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String storageCellNumber
in String storageGroupNumber
in String storageNumber
in String[] storageResultKeys
out String[] storageResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `storageResultKeys` and (out) `storageResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation	IMSApi data type
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_CELL	Storage cell in the iTAC system	☐ String
STORAGE_CELL_DESC	Storage cell description in the iTAC system	☐ String
STORAGE_DESC	Storage description in the iTAC system	☐ String
STORAGE_GROUP	Storage group in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group and sub-groups); the various levels are separated by " " (PIPE)!	☐ String
STORAGE_GROUP_DESC	Storage group description in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group description and subgroup descriptions); the various levels are separated by " " (PIPE)!	☐ String
STORAGE_ID	Internal identification number of the storage place	☐ long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐ String
STORAGE_TYPE	Designator for the storage place type Advice: Storage types are customer-specific and are created during the course of the initial commissioning!	☐ String

Tab. 18-25 "Key Value Pair": *storageResultKeys*

18.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
storageCellNumber	Storage cell in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If the value [-1] is passed, the parameter is not evaluated (no restriction)!
storageGroupNumber	Storage group in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If the value [-1] is passed, the parameter is not evaluated (no restriction)!
storageNumber	Storage place number in the iTAC system Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If the value [-1] is passed, the parameter is not evaluated (no restriction)!
storageResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": storageResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) storageResultValues !

Tab. 18-26 Parameter (in) »mlGetStorage«

Parameter (out)

Parameter name	Explanation
storageResultValues	Depending on parameter (in) storageResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) storageResultKeys !

Tab. 18-27 Parameter (out) »mlGetStorage«

18.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.9

mlGetStorageData

18.9.1

Function description

This function enables storage information to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., storage place numbers, storage groups, storage cells, storage states or stop point numbers). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

The results are output in four parts: storage cells, storage groups, storage places and the parts assigned to a storage place. The output structures – i.e. which storage cell contains which storage groups, storage subgroups and storage places – can be put into context using the keys `STORAGE_CELL_ID`, `STORAGE_GROUP_PARENT_ID`, `STORAGE_GROUP_ID` and `STORAGE_ID`.

**Advice**

With this function, storage information can only be ascertained if a storage place exists on the lowest level. This means that information regarding storage cells without storage groups and/or storage groups without storage places cannot be queried!

Application types

For the output, a distinction must be made between the two following application types:

- **Complete storage group structure:** If filtering takes place at storage cell level or storage group level (keys `STORAGE_CELL_NUMBER` and `STORAGE_GROUP_NUMBER`), apart from the storage cell the complete storage group structure (all levels) is always ascertained and output.
- **Restricted storage group structure:** If filtering takes place at storage place level or stop point level (keys `STORAGE_BARCODE`, `STORAGE_NUMBER` and `STOP_POINT_NUMBER`), apart from the storage cell only the "lowest" storage group is output; this is the storage group in which the storage place is located. Parent groupings are not taken into consideration here; i.e. in this application type, the key `STORAGE_GROUP_PARENT_ID` always remains empty.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlGetStorageData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] storageDataFilter
    String key
    String value
in String[] storageCellResultKeys
in String[] storageGroupResultKeys
in String[] storageResultKeys
in String[] storagePartAssignmentResultKeys
out String[] storageCellResultValues
```

```
out String[] storageGroupResultValues
out String[] storageResultValues
out String[] storagePartAssignmentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see the parameters (in) `storageDataFilter`, `storageCellResultKeys`, `storageGroupResultKeys`, `storageResultKeys` and `storagePartAssignmentResultKeys` as well as (out) `storageCellResultValues`, `storageGroupResultValues`, `storageResultValue` and `storagePartAssignmentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☒ int
STOP_POINT_NUMBER	Stop point number in the iTAC system Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i> Special case: If [-1] is passed, all storage places which have a stop point number are considered!	☐ String
STORAGE_BARCODE	Bar code for the storage place Advice: <i>The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!</i> Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
STORAGE_CELL_NUMBER	Storage cell number in the iTAC system Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String

Key	Explanation	IMSApi data type
STORAGE_CELL_STATE	State of the storage cell [F; L; O; R]: F = Storage cell is free (Free) L = Storage cell is locked (Locked) O = Storage cell is occupied (Occupied) R = Storage cell is reserved (Reserved) Advice: If multiple values are to be passed, they must be separated by a comma (e.g. "L,R")! By default, only the above-named items are available; if required, further (customer-specific) items can be added to the list (for further information, see the function "Storage management" in the ADM module).	String
STORAGE_GROUP_NUMBER	Storage group number in the iTAC system Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	String
STORAGE_GROUP_STATE	State of the storage group [F; L; O; R]: F = Storage group is free (Free) L = Storage group is locked (Locked) O = Storage group is occupied (Occupied) R = Storage group is reserved (Reserved) Advice: If multiple values are to be passed, they must be separated by a comma (e.g. "L,R")! By default, only the above-named items are available; if required, further (customer-specific) items can be added to the list (for further information, see the function "Storage management" in the ADM module).	String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE! Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	String
STORAGE_STATE	State of the storage place [F; L; O; R]: F = Storage place is free (Free) L = Storage place is locked (Locked) O = Storage place is occupied (Occupied) R = Storage place is reserved (Reserved) Advice: If multiple values are to be passed, they must be separated by a comma (e.g. "L,R")! By default, only the above-named items are available; if required, further (customer-specific) items can be added to the list (for further information, see the function "Storage management" in the ADM module).	String

Tab. 18-28 "Key Value Pair": storageDataFilter

Storage cell: Definition of the output parameters

Key	Explanation	IMSApi data type
STORAGE_CELL_DESC	Storage cell description in the iTAC system	☐ String
STORAGE_CELL_ID	Internal identification number of the storage cell Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ long
STORAGE_CELL_NUMBER	Storage cell number in the iTAC system	☐ String
STORAGE_CELL_STATE	State of the storage cell [F; L; O; R]: F = Storage cell is free (Free) L = Storage cell is locked (Locked) O = Storage cell is occupied (Occupied) R = Storage cell is reserved (Reserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	☐ String
STORAGE_TYPE	Storage type of the storage cell [M; Q; S]: M = Storage cell is of type "default storage" Q = Storage cell is of type "quality storage" S = Storage cell is of type "locked storage" Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	☐ String

Tab. 18-29 "Key Value Pair": *storageCellResultKeys*

Storage group: Definition of the output parameters

Key	Explanation	IMSApi data type
STORAGE_CELL_ID	Internal identification number of the storage cell Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ long
STORAGE_GROUP_DESC	Storage group description in the iTAC system	☐ String
STORAGE_GROUP_ID	Internal identification number of the storage group Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)! If there is a multilevel storage group structure, it is also important to note the key <code>STORAGE_GROUP_PARENT_ID</code> (for further information, see the point "Application types" below).	☐ long
STORAGE_GROUP_NUMBER	Storage group number in the iTAC system	☐ String
STORAGE_GROUP_PARENT_ID	Internal identification number of the parent storage group (only if multilevel groupings exist); in this context, note also the key <code>STORAGE_GROUP_ID</code> (for further information, see the point "Application types" below).	☐ long

Key	Explanation	IMSApi data type
STORAGE_GROUP_STATE	State of the storage group [F; L; O; R]: F = Storage group is free (Free) L = Storage group is locked (Locked) O = Storage group is occupied (Occupied) R = Storage group is reserved (Reserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	String
STORAGE_TYPE	Storage type of the storage group [M; Q; S]: M = Storage group is of type "default storage" Q = Storage group is of type "quality storage" S = Storage group is of type "locked storage" Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	String

Tab. 18-30 "Key Value Pair": *storageGroupResultKeys*

Storage place: Definition of the output parameters

Key	Explanation	IMSApi data type
LATITUDE	Value for the geographical latitude (storage place location); the coordinates must first be entered in the ADM module in the storage management. Advice: To specify the geographical location, the geographical longitude is also required (see the key <code>LONGITUDE</code>)!	double
LONGITUDE	Value for the geographical longitude (storage place location); the coordinates must first be entered in the ADM module in the storage management. Advice: To specify the geographical location, the geographical latitude is also required (see the key <code>LATITUDE</code>)!	double
STOP_POINT_LOAD_COMMENT	Individual comment regarding loading of the storage place	String
STOP_POINT_UNLOAD_COMMENT	Individual comment regarding unloading of the storage place	String
STOP_POINT_NUMBER	Stop point number in the iTAC system	String
STORAGE_BARCODE	Bar code for the storage place Advice: The <code>STORAGE_BARCODE</code> and <code>STORAGE_NUMBER</code> keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	String
STORAGE_DESC	Storage description in the iTAC system	String
STORAGE_GROUP_ID	Internal identification number of the storage group Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	long

Key	Explanation	IMSApi data type
STORAGE_ID	Internal identification number of the storage place Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	String
STORAGE_STATE	State of the storage place [F; L; O; R]: F = Storage place is free (Free) L = Storage place is locked (Locked) O = Storage place is occupied (Occupied) R = Storage place is reserved (Reserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	String
STORAGE_TYPE	Storage type of the storage place [M; Q; S]: M = Storage place is of type "default storage" Q = Storage place is of type "quality storage" S = Storage place is of type "locked storage" Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	String

Tab. 18-31 "Key Value Pair": storageResultKeys

Part data for the storage place: Definition of the output parameters

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system	String
PART_GROUP	Part group in the iTAC system	String
PART_GROUP_DESC	Part group description in the iTAC system	String
PART_NUMBER	Part number in the iTAC system	String
QUANTITY	Maximum storage quantity for the part at the storage place (see STORAGE_ID) Advice: If the maximum storage quantity has been assigned at part group level, it applies for all parts of the part group!	double
STORAGE_ID	Internal identification number of the storage place Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	long

Tab. 18-32 "Key Value Pair": storagePartAssignmentResultKeys

18.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
storageDataFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": storageDataFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": storageDataFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
storageCellResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": storageCellResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>storageCellResultValues</code> !
storageGroupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": storageGroupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>storageGroupResultValues</code> !
storageResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": storageResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>storageResultValues</code> !
storagePartAssignmentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": storagePartAssignmentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>storagePartAssignmentResultValues</code> !

Tab. 18-33 Parameter (in) »mlGetStorageData«

Parameter (out)

Parameter name	Explanation
storageCellResultValues	Depending on parameter (in) <code>storageCellResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>storageCellResultKeys</code>!</i>
storageGroupResultValues	Depending on parameter (in) <code>storageGroupResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>storageGroupResultKeys</code>!</i>
storageResultValues	Depending on parameter (in) <code>storageResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>storageResultKeys</code>!</i>
storagePartAssignmentResultValues	Depending on parameter (in) <code>storagePartAssignmentResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>storagePartAssignmentResultKeys</code>!</i>

Tab. 18-34 Parameter (out) »mlGetStorageData«

18.9.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
-3	Failure	Session is invalid
0	OK	[EMPTY]
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2119	Failure	Number of records to be output has exceeded the maximum value

18.10 mlSetMaterialBinLocation

18.10.1 Function description

This function places the passed material bin at a specific storage location; if the material bin is already at a different storage location, it is moved.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlSetMaterialBinLocation
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String materialBinNumber
in DateMs bookDate
in String binLocation
in String binLocationBarcode
in int transactionType
```

18.10.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinNumber	Number of the material bin (container no.)
bookDate	Booking time of the container storage (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!
binLocation	Storage place no. in the iTAC system Advice: If an empty string or [-1] is passed, the field is not evaluated; in this case, binLocationBarcode must be passed!
binLocationBarcode	Bar code for the storage place; must not coincide with the storage place no. in the iTAC system Advice: If an empty string or [-1] is passed, the field is not evaluated; in this case, binLocation must be passed!

Parameter name	Explanation
<code>transactionType</code>	Code for the movement type from the iTAC system (see ML module movement types)

Tab. 18-35 Parameter (in) »mlSetMaterialBinLocation«**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

18.10.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-503	Failure	Container not found
-525	Failure	Invalid value for "transactionType"
-528	Failure	Storage place was not found
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.11

mlSplitMaterialBin

18.11.1

Function description

This function makes it possible to create new (partial) containers from a container that exists in the system. In doing so, various quantities can be defined for the new containers. Each new partial container reduces the original container by the respective quantity specifications. Condition for this is that the total quantity of all new partial containers may not exceed the current quantity of the original container.



Advice

*The array returned contains records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Observe the following points if you would like to work with this function:

- Only material containers can be split; other bin types, such as the so-called batches or lots, cannot be divided.
- The material quantity in the original container is reduced by the quantities of the partial containers.
- Partitioning can only be performed on the basis of the same quantity unit.
- For the new partial containers, a minimum quantity ">0" must be entered; empty partial containers can NOT be created.
- The original container may be emptied completely (quantity "0")
- Multiple partial containers can be created with a function call; in doing so, the respective quantities can be defined differently; likewise, the container state and storage location for each partial container can be defined separately.
- The splitting operation is logged in the system with the material movement types "610" (material receiving for new partial containers) and "611" (material dispatching for the original container).
- All other container characteristics, such as DateCode, manufacturer, etc., are taken over automatically from the original container.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlSplitMaterialBin
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String materialBinNumber
  in String[] splitMaterialBinKeys
  in String[] splitMaterialBinUploadValues
  in String[] splitMaterialBinResultKeys
  out String[] splitMaterialBinResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `splitMaterialBinKeys`, `splitMaterialBinUploadValues` and `splitMaterialBinResultKeys` as well as (out) `splitMaterialBinResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Parameter on the basis of which the partial container is created

Key	Explanation	IMSApi data type
MATERIAL_BIN_NUMBER	Number of the new material bin (partial container)	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the partial container Advice: <i>The total quantity in all new partial containers must not be greater than the current material quantity in the original container (input parameter <code>materialBinNumber</code>)!</i>	☐ double
MATERIAL_BIN_STATE	Container state [B; E; Q; R; S; V]: B = locked (material bin locked) E = completed (material bin finished) Q = QA lock (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) V = pre-release (material bin with pre-release) Advice: <i>If a different value is passed (e.g., -1), "R" state is used by default!</i>	☐ String
STORAGE_BARCODE	Bar code for the storage place Advice: <i>The <code>STORAGE_BARCODE</code> and <code>STORAGE_NUMBER</code> keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Advice: <i>If the container is to be stored, either <code>STORAGE_NUMBER</code> or <code>STORAGE_BARCODE</code> must be passed!</i></i>	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: <i>See the advice notices under <code>STORAGE_BARCODE</code>!</i>	☐ String

Tab. 18-36 "Key Value Pair": `splitMaterialBinKeys`

Definition of the output parameters for the new partial containers

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
HU_NUMBER	Serial number for the material bin (HU = Handling Unit)	☐ String
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String
MATERIAL_BIN_NUMBER	Number of the new material bin (partial container)	☐ String
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐ double
MATERIAL_BIN_STATE	Container state [B; E; Q; R; S; V]: B = locked (material bin locked) E = completed (material bin finished) Q = QA lock (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) V = pre-release (material bin with pre-release)	☐ String
RECEIVING_NUMBER	Receiving number	☐ String
STORAGE_BARCODE	Bar code for the storage place	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system	☐ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_RESERVED_NUMBER	Number of the production order for which the material in the container is to be reserved	☐ String

Tab. 18-37 "Key Value Pair": *splitMaterialBinResultKeys*

18.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinNumber	Number of the material bin (container no.)
splitMaterial-BinKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>splitMaterialBinUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": splitMaterial-BinKeys table. Advice: Only the parameters passed here are processed by the function!
splitMaterialBin-UploadValues	Values for the parameters selected by means of <code>splitMaterialBinKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>splitMaterialBinKeys</code> ! Refer to the "Key Value Pair": splitMaterialBinKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
splitMaterialBin-ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": splitMaterialBinResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>splitMaterialBinResultValues</code> !

Tab. 18-38 Parameter (in) »mlSplitMaterialBin«

Parameter (out)

Parameter name	Explanation
splitMaterialBin-ResultValues	Depending on parameter (in) <code>splitMaterialBinResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>splitMaterialBinResultKeys</code> !

Tab. 18-39 Parameter (out) »mlSplitMaterialBin«

18.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-503	Failure	Container not found
-504	Failure	An existing bin number can only be created if the old bin is empty
-505	Failure	Failure deactivating the material bin
-508	Failure	Material bin is already stored
-526	Failure	Invalid value for "quantity"
-528	Failure	Storage place was not found
-533	Failure	State of the material bin is invalid
-545	Failure	Container is finished
-549	Failure	Container already exists
-550	Failure	Container number not unique
-551	Failure	Container splitting is not allowed

-578	Failure	The maximum expiration date must NOT be before the expiration date
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

18.12

mlUpdateStorage

18.12.1

Function description

This function can be used to update certain characteristics of existing storage places. It is thus possible to change the current storage state for a selected storage place or to adapt current parts/part group assignments together with the associated maximum storage quantity.



Advice

*NO new storage places can be created with this function (for further information, see "Storage management" in the ADM module).
The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlUpdateStorage
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] storageUpdateKeys
in String[] storageUpdateValues
in String[] storagePartAssignmentUpdateKeys
in String[] storagePartAssignmentUpdateValues
out String[] storageUpdateResultValues
out String[] storagePartAssignmentUpdateResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `storageUpdateKeys` and `storagePartAssignmentUpdateKeys` as well as (out) `storageUpdateResultValues` and `storagePartAssignmentUpdateResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values "Storage place"

Key	Explanation		IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒	int
REFERENCE	Reference number for the storage place Advice: On the basis of this number, the parts/part groups can be put into context with this storage place (see parameter of the same name in the "storagePartAssignmentUpdateKeys" table below)!	☐	int
STORAGE_BARCODE	Bar code for the storage place; it does not have to be the same as the storage place number (STORAGE_NUMBER) Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)! Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_BARCODE, STORAGE_ID or STORAGE_NUMBER must be passed in order to ascertain the record to be changed!	☐	String
STORAGE_ID	Internal identification number of the storage place Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_BARCODE, STORAGE_ID or STORAGE_NUMBER must be passed in order to ascertain the record to be changed!	☐	long
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notices under STORAGE_BARCODE!	☐	String
STORAGE_STATE	State of the storage place [F; L; O; R]: F = Storage place is free (F ree) L = Storage place is locked (L ocked) O = Storage place is occupied (O ccupied) R = Storage place is reserved (R eserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	☐	String

Tab. 18-40 "Key Value Pair": storageUpdateKeys

Input values "Storage place/part assignment"

Key	Explanation	IMSApi data type
DELETE	Indicates whether the part/part group assignment for the storage place is to be created/updated or removed [0; 1]: 0 = Assignment is created/updated (FALSE) 1 = Assignment is removed (TRUE) Background: In mode [1], it is attempted to delete the passed part/part group assignment based on the passed storage place. In mode [0], a new entry is created for an assignment that has not been found; existing assignments are updated (currently only the specification "Maximum storage quantity" using the key <code>QUANTITY</code>).	boolean
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	int
PART_GROUP	Part group in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the <code>PART_GROUP</code> or the <code>PART_NUMBER</code> must be passed!	String
PART_NUMBER	Part number in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the <code>PART_GROUP</code> or the <code>PART_NUMBER</code> must be passed!	String
QUANTITY	Maximum storage quantity for the part at the storage place (see <code>STORAGE_ID</code>) Advice: If the maximum storage quantity is assigned at part group level, it applies for all parts of the part group!	double
REFERENCE	Reference number for the storage place Advice: On the basis of this number, the part/part groups can be put into context with the storage place (see parameter of the same name in the "storageUpdateKeys" table above)!	int

Tab. 18-41 "Key Value Pair": `storagePartAssignmentUpdateKeys`

18.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
storageUpdateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>storageUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": storageUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
storageUpdateValues	Values for the parameters selected by means of <code>storageUpdateKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>storageUpdateKeys</code> ! Refer to the "Key Value Pair": storageUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
storagePartAssignmentUpdateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>storagePartAssignmentUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": storagePartAssignmentUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
storagePartAssignmentUpdateValues	Values for the parameters selected by means of <code>storagePartAssignmentUpdateKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>storagePartAssignmentUpdateKeys</code> ! Refer to the "Key Value Pair": storagePartAssignmentUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 18-42 Parameter (in) »mlUpdateStorage«

Parameter (out)

Parameter name	Explanation
storageUpdateResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>storageUpdateKeys</code> and <code>storageUpdateValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>storageUpdateKeys</code>!
storagePartAssignmentUpdateResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>storagePartAssignmentUpdateKeys</code> and <code>storagePartAssignmentUpdateValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>storagePartAssignmentUpdateKeys</code>!

Tab. 18-43 Parameter (out) »mlUpdateStorage«

18.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-122	Failure	Part group not found
-570	Failure	Store state not available
-571	Failure	Store state not found
-574	Failure	Storage location not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2120	Failure	At least one record in the array could not be processed
-2124	Failure	A value was passed multiple times to the function

18.13

mlUpdateStorageCell

18.13.1

Function description

This function can be used to update certain characteristics of existing storage cells. This allows the current storage state (`STORAGE_STATE`) to be changed for a selected storage cell. If the storage state is changed for the storage cell, the key `CASCADE` can be used to determine whether the new state is also transferred to all subordinate storage groups and storage places.



Advice

*NO new storage cells can be created with this function (for further information, see "Storage management" in the ADM module).
The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlUpdateStorageCell
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] storageUpdateKeys
in String[] storageUpdateValues
out String[] storageUpdateResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `storageUpdateKeys` and `storageUpdateValues` as well as (out) `storageUpdateResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Input values "Storage cell"

Key	Explanation	IMSApi data type
CASCADE	Effect of "storage state change" on the subordinate storage groups/locations [0; 1; 2]: 0 = No effect; the storage state is only changed for the selected storage cell; the subordinate storage groups/locations remain unchanged. 1 = Storage state is changed for the selected storage cell and for the subordinate storage groups/locations with the same storage state. 2 = Storage state is changed for the selected storage cell and for ALL subordinate storage groups/locations – irrespective of their current storage state. Advice: If this key is not passed along, the [0] value is used by default!	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒ int
STORAGE_CELL_ID	Internal identification number of the storage cell Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_CELL_ID or STORAGE_CELL_NUMBER must be passed in order to ascertain the record to be changed!	☐ long
STORAGE_CELL_NUMBER	Storage cell number in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_CELL_ID or STORAGE_CELL_NUMBER must be passed in order to ascertain the record to be changed!	☐ String
STORAGE_CELL_STATE	State of the storage cell [F; L; O; R]: F = Storage cell is free (Free) L = Storage cell is locked (Locked) O = Storage cell is occupied (Occupied) R = Storage cell is reserved (Reserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	☒ String

Tab. 18-44 "Key Value Pair": storageUpdateKeys

18.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
storageUpdateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>storageUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": storageUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
storageUpdateValues	Values for the parameters selected by means of <code>storageUpdateKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>storageUpdateKeys</code> ! Refer to the "Key Value Pair": storageUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wild-cards)!

Tab. 18-45 Parameter (in) »mlUpdateStorageCell«

Parameter (out)

Parameter name	Explanation
storageUpdateResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>storageUpdateKeys</code> and <code>storageUpdateValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>storageUpdateKeys</code> !

Tab. 18-46 Parameter (out) »mlUpdateStorageCell«

18.13.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-570	Failure	Store state not available
-571	Failure	Store state not found
-572	Failure	Storage cell not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2120	Failure	At least one record in the array could not be processed
-2124	Failure	A value was passed multiple times to the function

18.14

mlUpdateStorageGroup

18.14.1

Function description

This function can be used to update certain characteristics of existing storage groups. This allows the current storage state (`STORAGE_STATE`) to be changed for a selected storage group. If the storage state is changed for the storage group, the key `CASCADE` can be used to determine whether the new state is also transferred to all subordinate storage groups and storage places.



Advice

*NO new storage groups can be created with this function (for further information, see "Storage management" in the ADM module).
The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a general failure (error code = -2120)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlUpdateStorageGroup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] storageGroupUpdateKeys
in String[] storageGroupUpdateValues
out String[] storageGroupUpdateResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `storageGroupUpdateKeys` and `storageGroupUpdateValues` as well as (out) `storageGroupUpdateResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Input values "Storage group"

Key	Explanation	IMSApi data type
CASCADE	Effect of "storage state change" on the subordinate storage groups/locations [0; 1; 2]: 0 = No effect; the storage state is only changed for the selected storage group; the subordinate storage groups/locations remain unchanged. 1 = Storage state is changed for the selected storage group and for the subordinate storage groups/locations with the same storage state. 2 = Storage state is changed for the selected storage group and for ALL subordinate storage groups/locations – irrespective of their current storage state. Advice: If this key is not passed along, the [0] value is used by default!	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record; Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (error code = -2120)!	☒ int
STORAGE_CELL_NUMBER	Storage cell number in the iTAC system Advice: Even though this key is not identified as a mandatory field, the STORAGE_CELL_NUMBER must also be passed in order to ascertain the record to be changed on the basis of the key STORAGE_GROUP_NUMBER!	☐ String
STORAGE_GROUP_ID	Internal identification number of the storage group Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_GROUP_ID or STORAGE_GROUP_NUMBER in combination with the STORAGE_CELL_NUMBER must be passed in order to ascertain the record to be changed!	☐ long
STORAGE_GROUP_NUMBER	Storage group number in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the key STORAGE_GROUP_ID or STORAGE_GROUP_NUMBER in combination with the STORAGE_CELL_NUMBER must be passed in order to ascertain the record to be changed!	☐ String
STORAGE_GROUP_STATE	State of the storage group [F; L; O; R]: F = Storage group is free (Free) L = Storage group is locked (Locked) O = Storage group is occupied (Occupied) R = Storage group is reserved (Reserved) Advice: By default, only the above-named entries are available; if required, further (customer-specific) entries can be added to the list!	☒ String

Tab. 18-47 "Key Value Pair": storageGroupUpdateKeys

18.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
storageGroupUpdateKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>storageGroupUpdateValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": storageGroupUpdateKeys table. Advice: Only the parameters passed here are processed by the function!
storageGroupUpdateValues	Values for the parameters selected by means of <code>storageGroupUpdateKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>storageGroupUpdateKeys</code> ! Refer to the "Key Value Pair": storageGroupUpdateKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 18-48 Parameter (in) »mlUpdateStorageGroup«

Parameter (out)

Parameter name	Explanation
storageGroupUpdateResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>storageGroupUpdateKeys</code> and <code>storageGroupUpdateValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>storageGroupUpdateKeys</code> !

Tab. 18-49 Parameter (out) »mlUpdateStorageGroup«

18.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-570	Failure	Store state not available
-571	Failure	Store state not found
-573	Failure	Storage group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-2120	Failure	At least one record in the array could not be processed
-2124	Failure	A value was passed multiple times to the function

18.15 mlUploadMaterialBinBooking

18.15.1 Function description

This function is used to pass material movement data (e.g. quantity changes through consumption) from the production line to the parent ERP system via the iTAC system.



Advice

This API function is affected by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 7210 [▶▶▶]. In addition, this function is controlled by the station settings for "Upload Settings". Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

In general, both positive as well as negative quantity bookings can be processed. On a function call, multiple quantity bookings can be passed – each quantity booking is processed as a unique record in an array. In the event of a transmission error, only the faulty record is returned together with a corresponding error message. Depending on the used input parameters, it is possible to distinguish between functions for material movement with or without container reference. If a container number is passed (MATERIAL_BIN_NUMBER), the quantity changes are booked to the container. In connection with the part number (PART_NUMBER), a check can be performed to determine whether the part on the container matches the part number passed. If only the part number is specified, only a material movement is logged - without a container being changed with regard to quantity.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mlUploadMaterialBinBooking
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String[] materialBinBookingUploadKeys
    in String[] materialBinBookingUploadValues
    out String[] materialBinBookingResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) material-BinBookingUploadKeys and materialBinBookingUploadValues and (out) materialBinBookingResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BOOK_DATE	Point in time of the material movement booking (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, material movements can also be created with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!	☐	DateMs
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: This key is used to ascertain the reservation item from the order-specific BOM if necessary!	☐	String
COST_CENTER	Cost center for the material movement type (TRANSACTION_CODE)	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) Advice: Even though this key is not identified as a mandatory field, the PRODUCT_NUMBER and/or the MATERIAL_BIN_NUMBER must be passed; if only the PRODUCT_NUMBER is passed, the material stock adjustment is not performed for a container, but instead on the part level!	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current (new) quantity in the material bin; here this is the default value for the remaining quantity after the function has been executed Advice: This key should only be used if the value [0] is passed for QUANTITY, i.e. no booking quantity can be passed. In this case, the difference between the actual remaining quantity and the "new" value passed here is formed and used as the booking quantity!	☐	double
PART_NUMBER	Part number of the container material in the iTAC system Advice: Even though this key is not identified as a mandatory field, the PRODUCT_NUMBER and/or the MATERIAL_BIN_NUMBER must be passed; if only the PRODUCT_NUMBER is passed, the material stock adjustment is not performed for a container, but instead on the part level!	☐	String

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐ int
QUANTITY	Material quantity for booking; this quantity is booked in connection with a passed MATERIAL_BIN_NUMBER as a container quantity change and without passing the MATERIAL_BIN_NUMBER as a material quantity change. Advice: Both negative as well as positive quantity movements can be processed. Positive values increase the material stock. Negative values reduce it. If the material stock becomes negative as a result of this booking, the function issues – provided that site parameter 7210 [▶▶▶] is set to [2] – a warning (see error code 504)! Special case: If [0] is passed, the booking quantity in combination with MATERIAL_BIN_QTY_ACTUAL is calculated by the system!	☒ double
STATION_NUMBER	Station number of the work station in the iTAC system Advice: If a work order(WORKORDER_NUMBER) is passed, the station no. must absolutely be entered!	☐ String
TRANSACTION_CODE	Unique designator (ID) of the material movement type Advice: If [0] or [-1] is passed, mapping to the ID of the (default) material movement type occurs (customer-specific function)!	☒ int
WORKORDER_NUMBER	Number of a production order Special case: If [0] is passed, booking is performed without association with a work order! If [-1] is passed, the work order which is active on the station is used; in this case, it must be ensured that the active work order also matches the booking (key word: time-delayed collective booking)!	☐ String

Tab. 18-50 "Key Value Pair": materialBinBookingUploadKeys

18.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBin-BookingUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>materialBinUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": materialBinBookingUploadKeys table. Advice: Only the parameters passed here are processed by the function!
materialBin-BookingUploadValues	Values for the parameters selected by means of <code>materialBin-BookingUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>materialBinBookingUploadKeys</code> ! Refer to the "Key Value Pair": materialBinBookingUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 18-51 Parameter (in) »mlUploadMaterialBinBooking«

Parameter (out)

Parameter name	Explanation
materialBinBookingResultValues	The corresponding data is output here only in the event of an error – depending on the <code>materialBinBookingUploadKeys</code> parameter (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>materialBinBookingUploadKeys</code> !

Tab. 18-52 Parameter (out) »mlUploadMaterialBinBooking«

18.15.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
504	Warning	The material quantity in the system has become negative
		Advice: The value 504 is only output if site parameter 7210 [▶▶▶] is set to [2]!
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-240	Failure	No work step found for work order, orientation of the PCB and station
-334	Failure	Location acc. to BOM not found
-503	Failure	Container not found
-519	Failure	Part does not match container material
-523	Failure	Part is configured for retrograde booking
-525	Failure	Invalid value for "transactionType"

-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site

19 Group "MSG" (iTAC.Messaging)

19.1 msgPublish

19.1.1 Function description

This function can be used to send messages for a specific topic (iTAC.Messaging.Service). Further details on using the iTAC.Messaging.Service can be found in the associated service description »Messaging-Service (Kafka)«.



Advice

The API function is a component of the iTAC.Messaging.Service introduced with Release 9.01.00; the service is currently still in the trial phase and is undergoing continuous further development. iTAC Software AG reserves the right to make subsequent changes to this function!

"Work order creation" messages

Valid **topics** (subject to extensions):

workordercreation/changeorder (modification order)
workordercreation/externalorder ("external products" work order)
workordercreation/incominggoodorder ("receiving" order)
workordercreation/maintenanceorder (maintenance order)
workordercreation/materialtestorder ("material test" work order)
workordercreation/productionorder (production order)
workordercreation/repairorder (repair order)
workordercreation/sctorder (SCT order)

Format: **JSON**

Field description (subject to extensions):

number: <String> = Work order number

type: <String> = Work order type: CHANGE_ORDER, EXTERNAL, INCOMING_GOOD, MAINTAINANCE, MATERIAL_TEST, PRODUCTION, REPAIR, SCT_ORDER

state: <String> = Order state: CREATED, ENDED, NOT_RELEASED, PLANNED, STARTED, STOPPED

quantity: <long> = Order quantity

plannedStartDate = Planned start date for the work order

plannedDeliveryDate = Planned delivery date for the work order

Example:

```
topic=workordercreation/productionorder
value={
  "number" : "98765432",
  "type" : "PRODUCTION",
  "state" : "CREATED",
  "quantity" : 400,
  "plannedStartDate" : "2018-07-09T09:04:59.000-0700",
  "plannedDeliveryDate" : "2018-07-10T09:05:00.000-0700"
}
```

Licensing

In order for this API function to be used, the station that was used for session login (»regLogin« [▶▶▶] function) must be licensed as follows:

	MSG license
--	--------------------

Function declaration

```
int msgPublish
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String topic
in String message
out String msgId
```

19.1.2**Input and output parameters**

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
topic	Message topic with which the message is to be sent.
message	Message to be sent

Tab. 19-1 Parameter (in) »msgPublish«**Parameter (out)**

Parameter name	Explanation
msgId	Unique designator (ID) for the sent message Advice: The designator has no meaning for the application logic; it can, however, be used, e.g., in the event of problems for failure search!

Tab. 19-2 Parameter (out) »msgPublish«

19.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3700	Failure	iTAC.Messaging.Service is deactivated (not installed)
-3702	Failure	Topic is invalid
-3703	Failure	Failure when publishing a message
-3708	Failure	General failure in the iTAC.Messaging.Service

19.2 msgSubscribe

19.2.1 Function description

This function is used to subscribe to specific topics to receive the corresponding messages (iTAC.Messaging.Service). Details on the structure of the messages (currently only for "work order creation") can be found under the function »msgPublish« [▶▶▶] and there item »Messages "Work order creation"« [▶▶▶]. Further details on using the iTAC.Messaging.Service can be found in the associated service description »Messaging-Service (Kafka)«.



Advice

The API function is a component of the iTAC.Messaging.Service introduced with Release 9.01.00; the service is currently still in the trial phase and is undergoing continuous further development. iTAC Software AG reserves the right to make subsequent changes to this function!

Licensing

In order for this API function to be used, the station that was used for session login (»regLogin« [▶▶▶] function) must be licensed as follows:

	MSG license
--	--------------------

Function declaration

```
int msgSubscribe
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String[] topics
```

19.2.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
topics[]	List of topics for which messages are to be received.

Tab. 19-3 Parameter (in) »msgSubscribe«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

19.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3700	Failure	iTAC.Messaging.Service is deactivated (not installed)
-3702	Failure	Topic is invalid
-3704	Failure	Failure when subscribing to a message (only for FailOver/Load-Balancing)
-3705	Failure	A simultaneously running msgCallback is active (not allowed for the same session)
-3708	Failure	General failure in the iTAC.Messaging.Service

19.3 msgUnsubscribe

19.3.1 Function description

This function is used to unsubscribe to all currently subscribed topics and, thus, to the corresponding messages (iTAC.Messaging.Service). After calling this function, new topics can again be subscribed to with the »msgSubscribe« [▶▶▶] function. Further details on using the iTAC.Messaging.Service can be found in the associated service description »Messaging-Service (Kafka)«.



Advice

The API function is a component of the iTAC.Messaging.Service introduced with Release 9.01.00; the service is currently still in the trial phase and is undergoing continuous further development. iTAC Software AG reserves the right to make subsequent changes to this function!

Licensing

In order for this API function to be used, the station that was used for session login (»regLogin« [▶▶▶] function) must be licensed as follows:

	MSG license
--	--------------------

Function declaration

```
int msgUnsubscribe
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
```

19.3.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)

Tab. 19-4 Parameter (in) »msgUnsubscribe«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

19.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3700	Failure	iTAC.Messaging.Service is deactivated (not installed)
-3707	Failure	Failure when deregistering a message
-3708	Failure	General failure in the iTAC.Messaging.Service

20

Group "MSL" (Moisture Sensitivity Level)

20.1

mslBeginDryObject

20.1.1

Function description

By means of this function, it is possible to book the start of a re-drying process for serial numbers or material bins (containers) in the system.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! On passing of the re-drying oven temperature (`DRY_TEMPERATURE`), the next-smaller temperature value is always used to ascertain the re-drying time if there is not a direct match in accordance with the MSL definition. Thus, the end time for the re-drying process is determined from the oven temperature and the re-drying time ascertained on the basis of that temperature!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslBeginDryObject
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] mslBeginDryEventParams
    String key
    String value
  in String[] mslObjectKeys
  in String[] mslObjectValues
  out String[] mslResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslBeginDryEventParams`, `mslObjectKeys` and `mslObjectValues` as well as (out) `mslResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Default values for the re-drying process (start)

Key	Explanation		IMSApi data type
BOOK_DATE	Start time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 The start time must not be dated more than 7 days into the future. If a start time of a re-drying process was already booked, it is not overwritten (see error code -3109). Special case: If the key is not passed, the current date and time are used by default!	☐	DateMs
DRY_TEMPERATURE	Temperature in the re-drying oven	☒	int
MSL_USE_DATE	Desired time of use for the MSL object; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If the MSL object is a serial number, all mounted components are analyzed with respect to their MSL information; the most "expired" component is then used for the calculation ("worst case" principle). Advice: Depending on this specification, the function can output a warning (see error code 3107) or a failure (see error code -3106)!	☐	DateMs
STORAGE_BARCODE	Bar code for the storage place in the re-drying oven where the MSL object is to be stored Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐	String
STORAGE_NUMBER	Storage place number (in the re-drying oven) in the iTAC system where the MSL part is to be stored Advice: See the advice notice under STORAGE_BARCODE!	☐	String

Tab. 20-1 "Key Value Pair": mslBeginDryEventParams

Definition of the MSL object parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) that is to be re-dried Advice: If this key is used, no serial number (SERIAL_NUMBER) may be passed!	☐ String
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether only the passed panel (SERIAL_NUMBER_POS) or all panels are to be re-dried [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	☐ boolean
SERIAL_NUMBER	Serial number of the panel that is to be re-dried Advice: If this key is used, no container number (MATERIAL_BIN_NUMBER) may be passed!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String

Tab. 20-2 "Key Value Pair": *mslObjectKeys*

20.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mslBeginDryEvent-Params	"Key Value Pair" structure with the following variables:
key	List of the default values for the beginning of the re-drying process; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslBeginDryEventParams table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mslBeginDryEvent-Params table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslObjectKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>mslObjectValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslObjectKeys table. Advice: Only the parameters passed here are processed by the function!
mslObjectValues	Value list for the parameters passed by means of <code>mslObjectKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> ! Refer to the "Key Value Pair": mslObjectKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 20-3 Parameter (in) »mslBeginDryObject«

Parameter (out)

Parameter name	Explanation
mslResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>mslObjectKeys</code> and <code>mslObjectValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> !

Tab. 20-4 Parameter (out) »mslBeginDryObject«

20.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3107	Warning	MSL -> Re-drying unnecessary
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-503	Failure	Container not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3100	Failure	MSL -> Re-drying time not found
-3101	Failure	MSL -> Not found for container
-3102	Failure	MSL not unique

-3105	Failure	MSL -> Not found for serial number
-3106	Failure	MSL -> Usage date cannot be reached
-3108	Failure	MSL -> Container is already set up
-3109	Failure	MSL -> Invalid booking date

20.2

mslEndDryObject

20.2.1

Function description

By means of this function, it is possible to book the end of a re-drying process for serial numbers or material bins (containers) in the system.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! If the re-drying process is finished before the calculated re-drying date, the MSL object is considered to be NOT re-dried! Depending on the server configuration, the time in the re-drying oven is interpreted as "dried time" or as "opened time" (for further information, see MSL parameter `pauseMslExpirationOnBake`)!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslEndDryObject
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mslEventParams
    String key
    String value
in String[] mslObjectKeys
in String[] mslObjectValues
out String[] mslResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslEventParams`, `mslObjectKeys` and `mslObjectValues` as well as (out) `mslResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Default values for the re-drying process (end)

Key	Explanation	IMSApi data type
BOOK_DATE	End time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 The end time must not be dated more than 7 days into the future. If an end time of a re-drying process was already booked, it is not overwritten (see error code -3109). Special case: If the key is not passed, the current date and time are used by default!	DateMs
STORAGE_BARCODE	Bar code for the storage place where the re-dried MSL object is to be stored Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	String
STORAGE_NUMBER	Storage place number in the iTAC system where the re-dried MSL object is to be stored Advice: See the advice notice under STORAGE_BARCODE!	String

Tab. 20-5 "Key Value Pair": *mslEventParams*:

Definition of the MSL object parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) that was re-dried Advice: If this key is used, no serial number (SERIAL_NUMBER) may be passed!	String
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether only the passed panel (SERIAL_NUMBER_POS) or all panels are to be booked as re-dried [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	boolean
SERIAL_NUMBER	Serial number of the panel that was re-dried Advice: If this key is used, no container number (MATERIAL_BIN_NUMBER) may be passed!	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	String

Tab. 20-6 "Key Value Pair": *mslObjectKeys*

20.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mslEventParams	"Key Value Pair" structure with the following variables:
key	List of the default values for the end of the re-drying process; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslEventParams table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mslEventParams table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslObjectKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>mslObjectValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslObjectKeys table. Advice: Only the parameters passed here are processed by the function!
mslObjectValues	Value list for the parameters passed by means of <code>mslObjectKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> ! Refer to the "Key Value Pair": mslObjectKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 20-7 Parameter (in) »mslEndDryObject«

Parameter (out)

Parameter name	Explanation
mslResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>mslObjectKeys</code> and <code>mslObjectValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> !

Tab. 20-8 Parameter (out) »mslEndDryObject«

20.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-503	Failure	Container not found
-547	Failure	Storage place not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3100	Failure	MSL -> Re-drying time not found
-3101	Failure	MSL -> Not found for container
-3102	Failure	MSL not unique

-3105	Failure	MSL -> Not found for serial number
-3108	Failure	MSL -> Container is already set up
-3109	Failure	MSL -> Invalid booking date

20.3

mslGetMinDryTimeForSerialnumber

20.3.1

Function description

By means of this function, it is possible – if necessary, with a to-be-specified desired date of use for the processing algorithm – to ascertain the minimum duration for the re-drying of a serial number.

**Advice**

The expiration date of a serial number is determined from the MSL information of all mounted parts. To achieve a certain date of use, it is not required that all components be completely re-dried, rather only those that are expired by the desired time of use!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslGetMinDryTimeForSerialnumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mslInfoFilters
    String key
    String value
in String[] mslWorstCaseMinDryTimeResultKeys
out String[] mslWorstCaseMinDryTimeResultValues
in String[] mslMinDryTimeResultKeys
out String[] mslMinDryTimeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslInfoFilters`, `mslWorstCaseMinDryTimeResultKeys` and `mslMinDryTimeResultKeys` as well as (out) `mslWorstCaseMinDryTimeResultValues` and `mslMinDryTimeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Default values for computing the minimum duration

Key	Explanation	IMSApi data type
BOOK_DATE	Planned start time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 The start time must not be dated more than 7 days into the future. If a start time of a re-drying process was already booked, it is not overwritten (see error code -3109). Special case: If the key is not passed, the current date and time are used by default!	☐ DateMs
DRY_TEMPERATURE	Temperature in the re-drying oven	☒ int
FUNC_MODE	Function mode for the output (0; 1): 0 = all MSL components are output 1 = only the expired MSL components are output	☐ int
MSL_USE_DATE	Desired date of use; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: If an impossible date of use is specified, the function returns error code -3106!	☐ DateMs
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether only the passed panel (SERIAL_NUMBER_POS) or all panels are taken into consideration [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	☐ boolean
SERIAL_NUMBER	Serial number of the panel that is to be re-dried	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☒ String

Tab. 20-9 "Key Value Pair": *mslInfoFilters*

Minimum duration for re-drying (worst case)

Key	Explanation	IMSApi data type
EXPIRATION_DATE	MSL expiration date (without offset); point in time from which the serial number must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_CALCULATED_EXPIRATION_DATE	Calculated MSL expiration date on the basis of all mounted components that would be expired by the desired date (MSL_USE_DATE); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: If no desired date is specified, the expiration date is ascertained on the basis of all mounted components!	☐ DateMs
MSL_DRY_END_DATE	End date of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_DRY_TIME	Time period that would be necessary for re-drying (in seconds)	☐ int

Tab. 20-10 "Key Value Pair": *mslWorstCaseMinDryTimeResultKeys*

Minimum duration for re-drying (components)

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	String
EXPIRATION_DATE	MSL expiration date (without offset); point in time from which the serial number must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	DateMs
LEVEL	Display of the integration level relative to the passed serial number: 0 = Passed serial number < 0 = Number of integration levels downward	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	String
MSL_CALCULATED_EXPIRATION_DATE	Calculated MSL expiration date on the basis of all mounted components that would be expired by the desired date (MSL_USE_DATE); if no desired date is specified, the expiration date is ascertained on the basis of all mounted components	DateMs
MSL_DRY_END_DATE	End date of the re-drying process	DateMs
MSL_DRY_TIME	Time period that would be necessary for re-drying	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use	String
PART_DESC	Part description in the iTAC system (for the container material)	String
PART_NUMBER	Part number in the iTAC system (for the container material)	String
SERIAL_MASTER_NUMBER	Serial number of the final product (highest integration level)	String
SERIAL_MASTER_NUMBER_POS	Position of the single panel in the unit-array (highest integration level)	String
SERIAL_NUMBER	Serial number of the panel that is to be re-dried	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	String

Tab. 20-11 "Key Value Pair": *mslMinDryTimeResultKeys*

20.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mslInfoFilters	"Key Value Pair" structure with the following variables:
key	List of the container parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter. A description of the possible container parameters can be found above in the "Key Value Pair": mslInfoFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mslInfoFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslWorstCaseMinDryTimeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mslWorstCaseMinDryTimeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mslWorstCaseMinDryTimeResultValues</code> !
mslMinDryTimeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mslMinDryTimeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mslMinDryTimeResultValues</code> !

Tab. 20-12 Parameter (in) »mslGetMinDryTimeForSerialnumber«

Parameter (out)

Parameter name	Explanation
mslWorstCaseMinDryTimeResultValues	Depending on parameter (in) <code>mslWorstCaseMinDryTimeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslWorstCaseMinDryTimeResultKeys</code> !
mslMinDryTimeResultValues	Depending on parameter (in) <code>mslMinDryTimeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslMinDryTimeResultKeys</code> !

Tab. 20-13 Parameter (out) »mslGetMinDryTimeForSerialnumber«

20.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3107	Warning	MSL -> Re-drying unnecessary
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3100	Failure	MSL -> Re-drying time not found
-3102	Failure	MSL not unique
-3105	Failure	MSL -> Not found for serial number
-3106	Failure	MSL -> Usage date cannot be reached
-3109	Failure	MSL -> Invalid booking date

20.4

mslGetObjectInfo

20.4.1

Function description

This function returns MSL information (e.g., remaining processing time, MSL level, MSL object state) for serial numbers or material bins (container).



Advice

Only active serial numbers and material bins that have not finished (state FINISHED) are taken into account!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslGetObjectInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in KeyValue[] mslInfoFilters
    String key
    String value
in String[] mslInfoResultKeys
out String[] mslInfoResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslInfoFilters` and `mslInfoResultKeys` as well as (out) `mslInfoResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values for serial numbers
(objectType = 0)

Key	Explanation		IMSApi data type
EXPIRATION_DATE	MSL expiration date (without offset); point in time from which the serial number must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐	int
MSL_DRY_END_DATE	End time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>Only those serial numbers that would be dried by this point in time are taken into account!</i>	☐	DateMs
MSL_FLOOR_LIFE-TIME_REMAIN	Still remaining mounting time (in minutes) Advice: <i>Only those serial numbers that have at least the remaining mounting time specified here are taken into account!</i>	☐	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use Advice: <i>If the key is not specified, there is no restriction!</i>	☐	String
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (Baking) C = protected/closed(Closed) O = unprotected/open (Open) Advice: <i>If the key is not specified, there is no restriction!</i>	☐	String
MULTIPLE_BOARD	Selection of single panels/multiple panels [0; 1]: 0 = Only single panels are reviewed (FALSE) 1 = Only multiple panels are reviewed (TRUE) Advice: <i>If the key is not specified, there is no restriction!</i>	☐	boolean
PART_NUMBER	Part number in the iTAC system (for the serial number)	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐	String
STORAGE_BARCODE	Bar code for the storage place (in the re-drying oven) Advice: <i>The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!</i>	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system (in the re-drying oven) Advice: <i>See the advice notice under STORAGE_BARCODE!</i>	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 20-14 "Key Value Pair" (serial number): *mslInfoFilters*

Searching for specific parameter values for containers
(objectType = 1)

Key	Explanation		IMSApi data type
EXPIRATION_DATE	MSL expiration date (with site offset); point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐	int
MSL_DRY_END_DATE	End time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>Only those containers that would be dried by this point in time are taken into account!</i>	☐	DateMs
MSL_EXPIRATION_DATE_WITHOUT_OFFSET	MSL expiration date (with site offset); point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_FLOOR_LIFE-TIME_REMAIN	Still remaining mounting time (in minutes) Advice: <i>Only those containers that have at least the remaining mounting time specified here are taken into account!</i>	☐	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use Advice: <i>If the key is not specified, there is no restriction!</i>	☐	String
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (Baking) C = protected/closed(Closed) O = unprotected/open (Open) Advice: <i>If the key is not specified, there is no restriction!</i>	☐	String
PART_NUMBER	Part number in the iTAC system (for the container material)	☐	String
STORAGE_BARCODE	Bar code for the storage place (in the re-drying oven)	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system (in the re-drying oven)	☐	String

Tab. 20-15 "Key Value Pair" (container): *mslInfoFilters*

Definition of the output parameters for the found serial numbers
(objectType = 0)

Key	Explanation		IMSApi data type
EXPIRATION_DATE	MSL expiration date; point in time from which the serial number must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_DRY_END_DATE	End time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_EXPIRATION_DATE_WITHOUT_OFFSET	MSL expiration date without offset; point in time from which the serial number must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
MSL_FLOOR_LIFE-TIME_REMAIN	Still remaining mounting time (in minutes)	☐	int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use	☐	String
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (Baking) C = protected/closed(Closed) O = unprotected/open (Open)	☐	String
PART_NUMBER	Part number in the iTAC system (for the serial number)	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐	String
STORAGE_DESC	Storage description in the iTAC system	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system (in the re-drying oven)	☐	String
WORKORDER_NUMBER	Production order number	☐	String

Tab. 20-16 "Key Value Pair" (serial number): *mslInfoResultKeys*

Definition of the output parameters for the found containers
(objectType = 1)

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
EXPIRATION_DATE	MSL expiration date; point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
EXPIRATION_DATE_FINAL	End expiration time for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: <i>The container expires at this point in time even if it was never in an unprotected environment!</i>	☐ DateMs
LOCK_STATE	Lock state of the material bin [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
MATERIAL_BIN_NUMBER2	2. Container number (customer-specific)	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐ double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐ double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐ String
MSL_DRY_END_DATE	End time of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_EXPIRATION_DATE_WITHOUT_OFFSET	MSL expiration date without offset; point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_FLOOR_LIFE_TIME_REMAIN	Still remaining mounting time (in minutes); this value is recalculated on every function call	☐ int
MSL_FLOOR_LIFE_TIME_TOTAL	Original remaining mounting time, total; time is specified as the number of seconds since 01-01-1970	☐ int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use	☐ String

Key	Explanation		IMSApi data type
MSL_OPEN_DATE	Point in time when the container was removed from an MSL package (dry pack) or a drying oven/re-drying oven	☐	DateMs
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (B aking) C = protected/closed(C losed) O = unprotected/open (O pen)	☐	String
PART_DESC	Part description in the iTAC system (for the container material)	☐	String
PART_NUMBER	Part number in the iTAC system (for the container material)	☐	String
STORAGE_DESC	Storage description in the iTAC system	☐	String
STORAGE_NUMBER	Storage place number in the iTAC system (in the re-drying oven)	☐	String

Tab. 20-17 "Key Value Pair" (container): *mslInfoResultKeys*

20.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the MSL object type [0; 1]: 0 = serial number 1 = container
mslInfoFilters	"Key Value Pair" structure with the following variables:
key	List of the MSL object parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters (depending on the <code>objectType</code>) can be found above in the corresponding "Key Value Pair" (xxx): mslInfoFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the corresponding "Key Value Pair" (xxx): mslInfoFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the <code>objectType</code>) can be found above in the corresponding "Key Value Pair" (xxx): mslInfoResultKeys table for the MSL object. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mslInfoResultValues</code> !

Tab. 20-18 Parameter (in) »mslGetObjectInfo«

Parameter (out)

Parameter name	Explanation
mslInfoResultValues	Depending on parameter (in) <code>mslInfoResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslInfoResultKeys</code> !

Tab. 20-19 Parameter (out) »mslGetObjectInfo«

20.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-503	Failure	Container not found
-547	Failure	Storage place not found
-905	Failure	Invalid value for "objectType"
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3101	Failure	MSL -> Not found for container
-3110	Failure	Invalid value for "MSL_LEVEL"

20.5

mslStartObjectExpiration

20.5.1

Function description

By means of this function, it is possible to book the start of the expiry time for serial numbers or material bins (containers) in the system. As a rule, the expiry time starts if an MSL package (so-called "dry pack") is opened or an MSL part is removed unprotected from a re-drying oven.



Advice

This API function is influenced by station parameter code no. 13125 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslStartObjectExpiration
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mslEventParams
    String key
    String value
in String[] mslObjectKeys
in String[] mslObjectValues
out String[] mslResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslEventParams`, `mslObjectKeys` and `mslObjectValues` as well as (out) `mslResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Default values for the expiry time start

Key	Explanation		IMSApi data type
BOOK_DATE	Start time of the expiry time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 The start time must not be dated more than 7 days into the future. If a start time was already booked, it is not overwritten (see error code -3109). Special case: If the key is not passed, the current date and time are used by default!	☐	DateMs
STORAGE_BARCODE	Bar code for the storage place (in the re-drying oven) where the unprotected MSL object is to be stored Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐	String
STORAGE_NUMBER	Storage place number (in the re-drying oven) in the iTAC system where the unprotected MSL object is to be stored Advice: See the advice notice under STORAGE_BARCODE!	☐	String

Tab. 20-20 "Key Value Pair": *mslEventParams*

Definition of the MSL object parameters

Key	Explanation		IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) on which the expiry time is to start Advice: If this key is used, no serial number (SERIAL_NUMBER) may be passed!	☐	String
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether the expiry time is to be started for only the passed panel (SERIAL_NUMBER_POS) or for all panels [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	☐	boolean
SERIAL_NUMBER	Serial number of the panel on which the expiry time is to start Advice: If this key is used, no container number (MATERIAL_BIN_NUMBER) may be passed!	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String

Tab. 20-21 "Key Value Pair": *mslObjectKeys*

20.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mslEventParams	"Key Value Pair" structure with the following variables:
key	List of the default values for expiry time start; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslEventParams table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mslEventParams table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslObjectKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>mslObjectValues</code> parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": mslObjectKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!
mslObjectValues	Values for the parameters selected by means of <code>mslObjectKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> ! Refer to the corresponding table for attribute category "Key Value Pair": "mslObjectKeys" to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 20-22 Parameter (in) »mslStartObjectExpiration«

Parameter (out)

Parameter name	Explanation
mslResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>mslObjectKeys</code> and <code>mslObjectValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> !

Tab. 20-23 Parameter (out) »mslStartObjectExpiration«

20.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3100	Warning	MSL state not changed
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-503	Failure	Container not found
-547	Failure	Storage place not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3100	Failure	MSL -> Re-drying time not found
-3101	Failure	MSL -> Not found for container

-3102	Failure	MSL not unique
-3105	Failure	MSL -> Not found for serial number
-3108	Failure	MSL -> Container is already set up
-3109	Failure	MSL -> Invalid booking date
-3111	Failure	MSL -> Incorrect configuration (site parameter 7070)

20.6

mslStopObjectExpiration

20.6.1

Function description

By means of this function, it is possible to stop a started (running) expiry time for serial numbers or material bins (containers) and to book this stop time in the system. As a rule, the expiry time stops if an opened/unprotected MSL object is again closed in an MSL package (so-called "dry pack") or placed in a drying cabinet/re-drying oven.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
AND	ML service (site license)

Function declaration

```
int mslStopObjectExpiration
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mslEventParams
    String key
    String value
in String[] mslObjectKeys
in String[] mslObjectValues
out String[] mslResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mslEventParams`, `mslObjectKeys` and `mslObjectValues` as well as (out) `mslResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Default values for the expiry time stop

Key	Explanation	IMSApi data type
BOOK_DATE	Stop time of the expiry time; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 The stop time must not be dated more than 7 days into the future. If a stop time was already booked, it is not overwritten (see error code -3109). Special case: If the key is not passed, the current date and time are used by default!	☐ DateMs
STORAGE_BARCODE	Bar code for the storage place (in the re-drying oven) where the again-protected MSL object is to be stored Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_NUMBER	Storage place number (in the re-drying oven) in the iTAC system where the again-protected MSL object is to be stored Advice: See the advice notice under STORAGE_BARCODE!	☐ String

Tab. 20-24 "Key Value Pair": *mslEventParams*

Definition of the MSL object parameters

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) on which the expiry time is to be stopped Advice: If this key is used, no serial number (SERIAL_NUMBER) may be passed!	☐ String
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether the expiry time is to be started for only the passed panel (SERIAL_NUMBER_POS) or for all panels [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	☐ boolean
SERIAL_NUMBER	Serial number of the panel on which the expiry time is to be stopped Advice: If this key is used, no container number (MATERIAL_BIN_NUMBER) may be passed!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String

Tab. 20-25 "Key Value Pair": *mslObjectKeys*

20.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mslEventParams	"Key Value Pair" structure with the following variables:
key	List of the default values for expiry time stop; the corresponding values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslEventParams table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mslEventParams table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
mslObjectKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>mslObjectValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mslObjectKeys table. Advice: Only the parameters passed here are processed by the function!
mslObjectValues	Values for the parameters selected by means of <code>mslObjectKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> ! Refer to the corresponding table for attribute category "Key Value Pair": "mslObjectKeys" to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 20-26 Parameter (in) »mslStopObjectExpiration«

Parameter (out)

Parameter name	Explanation
mslResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>mslObjectKeys</code> and <code>mslObjectValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mslObjectKeys</code> !

Tab. 20-27 Parameter (out) »mslStopObjectExpiration«

20.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3100	Warning	MSL state not changed
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-503	Failure	Container not found
-547	Failure	Storage place not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-2107	Failure	No ML license present for the site
-3100	Failure	MSL -> Re-drying time not found
-3101	Failure	MSL -> Not found for container

-3102	Failure	MSL not unique
-3103	Failure	MSL -> Invalid state change: BAKE -> CLOSED
-3105	Failure	MSL -> Not found for serial number
-3108	Failure	MSL -> Container is already set up
-3109	Failure	MSL -> Invalid booking date

21

Group "PM" (production management)

21.1

pmGetBooking

21.1.1

Function description

This function makes it possible to ascertain already captured, person-related process data bookings (**pFDC** – **personalized Factory Data Collection**). A wide range of different filtering criteria can be applied here (e.g. validity range, cost center, wage type, booking type). It is also possible to search for individual pFDC records; for this purpose, the internal identification number of the booking is required. This number is generated automatically by the system when a new pFDC record is captured (for further information, see the API function »pmUploadBooking« [▶▶▶] -> output parameter `bookingId` or, in the PM module, the "PDC - Manual data capture" function).

All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.



Advice

By default, the executing person can only query pFDC records that he booked himself; this behavior can be changed if necessary with the `FUNC_MODE` key! This API function is influenced by station parameter code no. 26002 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Application type "OC booking during DOWNTIME": Using the API function, it is also possible to output information on machine conditions of the DOWNTIME type (planned/unplanned downtimes), provided that they are responsible for an automatically generated overhead cost booking. Further detailed information on this topic – especially on setting up this function – can be found in the user's guide under PM module (see there the "PDC - Manual data capture" function).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
AND	pFDC license

Function declaration

```
int pmGetBooking
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] pmBookingFilters
    String key
    String value
in String[] pmBookingResultKeys
out String[] pmBookingResultValues
in String[] pmBookingFailureKeys
out String[] pmBookingFailureValues
out boolean hasMore
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `pmBookingFilters`, `pmBookingResultKeys` and `pmBookingFailureKeys` as well as (out) `pmBookingResultValues` and `pmBookingFailureValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation		IMSApi data type
ACTIVITY_TYPE	Booking type; used to determine whether the booking is to be classed as a productive or preparatory (setting up) activity [P; S]: P = Production S = Setup Advice: <i>If the key is not passed or passed with [-1], there is no restriction!</i>	☐	String
BOOK_CODE_EXTERNAL_SYSTEM	Unique booking designator from an external ERP system Advice: <i>If the key is not passed or passed with [-1], there is no restriction!</i>	☐	String
BOOK_MODE	Booking mode [P; Q; S]: P = Parallel booking Q = Quantity booking S = Serial booking Advice: <i>To filter by multiple entries (OR link), the desired values must be passed separated by a comma (e.g.: "P,S"). If the key is not passed or passed with [-1], there is no restriction!</i>	☐	String
BOOKING_ID_FILTER	Internal identification number of the pFDC booking Advice: <i>To filter by multiple entries (OR link), the desired identification number must be passed separated by a comma. Identification numbers can be ascertained in the PM module with the "PDC - Manual data capture" function!</i>	☐	String
COST_CENTER_NUMBER	Cost center number Advice: <i>The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!</i>	☐	String
DATE_FROM	Start of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs

Key	Explanation		IMSApi data type
DATE_TO	End of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!	☐	DateMs
FUNC_MODE	Restriction of the output with respect to user authentication [0; 1; 2]: 0 = The used user (see below, <code>USER_*</code> keys) can only query pFDC records that he booked himself. 1 = The used user can, as team leader, also query the pFDC records of the team members. 2 = Independent of the used user, all pFDC records can be queried that were booked at the passed station (<code>STATION_NUMBER</code>). Advice: The used user must, however, still be successfully authenticated by the system!	☐	int
MAX_ROW_COUNT	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with <code>START_POSITION</code> , it is possible to successively "page" through large structures. The maximum number is limited to [100] records by the system. Advice: If this key is not passed along, the [100] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
SORT_FLAG	Sorting mode for the output [0; 1]: 0 = Ascending according to <code>DATE_FROM</code> 1 = Descending according to <code>DATE_FROM</code> Advice: If this key is not passed along, the [0] value is used by default!	☐	int
START_POSITION	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for <code>MAX_ROW_COUNT</code> . Advice: If this key is not passed along, the [1] value is used by default! In connection with the paging function, the hasMore output parameter is to be observed; this identifies whether the structure contains yet other records!	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system; this is the station which has been assigned to the pFDC booking, and should not be confused with the station that has executed the API function. Advice: If the key is not passed or passed with [-1], there is no restriction!	☐	String
USER_NAME	Login name of the user (user identification) Advice: By default, this key must be passed with the corresponding password (<code>USER_PASSWORD</code>) (for further information, see station parameter code no. 26002 [▶▶▶]); alternatively, user identification can also be performed with the <code>USER_TOKEN</code> key!	☐	String
USER_PASSWORD	Password of the user Advice: By default, this key must be passed with the associated user identification (<code>USER_NAME</code>); alternatively, user identification can also be performed using the <code>USER_TOKEN</code> key!	☐	String

Key	Explanation	IMSApi data type
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: This key can be used as an alternative to identification using USER_NAME and USER_PASSWORD!	☐ String
WAGE_TYPE_CATEGORY	Wage type category [d; F; G; M]: d = Not defined (internal dummy entry) F = Manufacturing costs G = Overhead costs M = Cost overruns Advice: If the key is not passed or passed with [-1], there is no restriction!	☐ String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_NUMBER	Number of a production order; work orders can only be booked in combination with wage type categories "F" (manufacturing costs) and "M" (cost overruns). Advice: The wildcard character "*" can be used when entering values; pay attention to upper- and lower-case letters. If the key is not passed or passed with [-1], there is no restriction!	☐ String

Tab. 21-1 "Key Value Pair": pmBookingFilters

Definition of the output parameters (pFDC booking data)

Key	Explanation	IMSApi data type
ACTIVITY_TYPE	Booking type; used to determine whether the booking is to be classed as a productive or preparatory (setting up) activity [P; S]: P = Production S = Setup	☐ String
BOOK_CODE_EXTERNAL_SYSTEM	Unique booking designator from an external ERP system	☐ String
BOOK_DATE	Booking time of the pFDC data capture (update in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
BOOK_MODE	Booking mode [P; Q; S]: P = Parallel booking Q = Quantity booking S = Serial booking	☐ String
BOOKING_ID	Internal identification number of the pFDC booking	☐ long
COLOR_RGB	- Only application type "OC booking during DOWNTIME" - Color definition (RGB) for the machine condition; the individual color values are separated by ";": e.g. 000;255;255;000 Advice: In addition to the RGB color value definitions, the alpha value (transparency) is defined with the 4th entry!	☐ String
COMMENT	Individual information text for the booking	☐ String

Key	Explanation		IMSApi data type
CONDITION_CAUSE	- Only application type "OC booking during DOWNTIME" - Machine condition cause [EXT; INT; TAD]: EXT = Marks the entry with "external" INT = Marks the entry with "internal" TAD = Marks the entry with "time adjustment" Advice: <i>The machine condition cause is defined by the parent machine condition class and influences e.g. calculation of the key performance indicator "Downtime"!</i>	☐	String
CONDITION_CODE	- Only application type "OC booking during DOWNTIME" - Unique designator of the machine condition or of the machine message	☐	String
CONDITION_DESC	- Only application type "OC booking during DOWNTIME" - Description of the machine condition or of the machine message	☐	String
CONDITION_NAME	- Only application type "OC booking during DOWNTIME" - Name of the machine condition or of the machine message	☐	String
CONDITION_TYPE	- Only application type "OC booking during DOWNTIME" - Machine condition type [O; Q; S; T]: O = Organization Q = Quality S = Setup T = Technology	☐	String
CONFIRMATION_STATE	Process state of the pFDC record [A; I; R]: A = Approved (APPROVED) I = Initialized (INIT) R = Refused (REFUSED)	☐	String
COST_CENTER_DESC	Cost center description	☐	String
COST_CENTER_DESC - ORIGIN	Description of the originating cost center	☐	String
COST_CENTER_NUMBER	Cost center number	☐	String
COST_CENTER_NUM - BER_ORIGIN	Number of the originating cost center	☐	String
DATE_FROM	Start of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_TO	End of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐	DateMs
PART_NUMBER	Part number in the iTAC system (for the production order)	☐	String
PROGRESS_PERCENT	Progress of the work step in percent Advice: <i>This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!</i>	☐	double
QUANTITY_FAIL	Number of "fail" bookings	☐	int
QUANTITY_FIRST_FAIL	Number of "fail" bookings (first test run)	☐	int

Key	Explanation	IMSApi data type
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run)	☐ int
QUANTITY - FIRST_SCRAP	Number of "scrap" bookings (first test run)	☐ int
QUANTITY_PASS	Number of "pass" bookings	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings	☐ int
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: <i>This is the station which has been assigned to the pFDC booking, and should not be confused with the station that has executed the API function!</i>	☐ String
TERMINAL	Unique designator of the terminal from which the booking was made.	☐ String
TIME_CONSUMPTION	Actual validity period for booking (duration in seconds)	☐ long
USER_FIRST_NAME	First name of the person	☐ String
USER_LAST_NAME	Name of the person	☐ String
USER_NAME	Login name of the user (user identification)	☐ String
WAGE_TYPE_DESC	Description of the wage type	☐ String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system	☐ String
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!</i>	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: <i>If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!</i>	☐ String

Tab. 21-2 "Key Value Pair": *pmBookingResultKeys*

Definition of output parameters (failure information)

Key	Explanation	IMSApi data type
BOOKING_ID	Internal identification number of the pFDC booking Advice: <i>The associated booking record can be assigned on the basis of this identification number!</i>	long
FAILURE_CAUSE_CODE	Failure cause code	String
FAILURE_IMAGE_CODE	Failure symptom code	String
FAILURE_TYPE_CODE	Failure type code	String
QUANTITY_FAILURE	Number of failures	int
QUANTITY_SOLDER- ING_POINT_FAILURE	Number of faulty soldering points (pins)	int

Tab. 21-3 "Key Value Pair": *pmBookingFailureKeys*

21.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
pmBookingFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pmBookingFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": pmBookingFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
pmBookingResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": pmBookingResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) pmBookingResultValues !
pmBookingFailureKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": pmBookingFailureKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) pmBookingFailureValues !

Tab. 21-4 Parameter (in) »pmGetBooking«

Parameter (out)

Parameter name	Explanation
pmBookingResultValues	Depending on parameter (in) pmBookingResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) pmBookingResultKeys !
pmBookingFailureValues	Depending on parameter (in) pmBookingFailureKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) pmBookingFailureKeys !
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: In connection with the paging function, the <code>START_POSITION</code> and <code>MAX_ROW_COUNT</code> keys are to be observed; these determine the start position and the maximum structure size!

Tab. 21-5 Parameter (out) »pmGetBooking«

21.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-105	Failure	A required parameter is missing
-111	Failure	No pFDC license present for the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-2015	Failure	Date not valid
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function
-3403	Failure	pFDC booking -> Booking mode is invalid
-3411	Failure	Activity type is invalid

21.2

pmGetFailureData

21.2.1

Function description

This function ascertains the process-related failure data (failure symptoms, failure types and failure causes) for a part or work order; process-related failure data are work-step-related failure bookings without serial number reference (PM module).



Advice

This API function is influenced by station parameter code nos. 8310 [▶▶▶] and 13200 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

As a rule, the process-related failure data is booked with the »pmUploadQuantityAndFailureData« [▶▶▶] IMSApi function.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	-------------------

Function declaration

```
int pmGetFailureData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] failureDataFilters
    String key
    String value
  in String failureDataResultKeys
  out String[] failureDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) failureDataFilters and failureDataResultKeys as well as (out) failureDataResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start time!	☒	DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!	☒	DateMs
FUNC_MODE	"Work step stations" query mode [0; 1]: 0 = only the passed station 1 = all stations on the work step	☐	int
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☐	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: If a [-1] is passed, the field is not evaluated (no restriction)! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be ascertained directly instead of the orientation of the PCB!	☐	int
PRODUCT_NUMBER	Part number in the iTAC system Advice: Even though this key is not identified as a mandatory field, either the PRODUCT_NUMBER or the WORKORDER_NUMBER must be passed!	☐	String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the PRODUCT_NUMBER or the WORKORDER_NUMBER must be passed!	☐	String

Tab. 21-6 "Key Value Pair": failureDataFilters

Definition of the output parameters for the found failure data

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
FAILURE_CAUSE_CODE	Failure cause code	☐	String
FAILURE_CAUSE_DESC	Description of the failure cause	☐	String

Key	Explanation	IMSApi data type
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_SEQUENCE_NUMBER	Display of the faulty test run	☐ int
FAILURE_TYPE_CATEGORY	Failure type category [C; S] C = Component failure (Component) S = Soldering point failure (Soldering) Reason: This setting affects the dpm calculation for the failure type analysis in the QM module.	☐ String
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part failure 4 = production error Advice: The available entries are permanently set in the system!	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = Not a pseudo failure 1 = Pseudo failure	☐ char
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: The available entries are permanently set in the system!	☐ int
GRID_X	Failure position: X value according to part image raster	☐ int
GRID_Y	Failure position: Y value according to part image raster	☐ int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐ String
INFO	Information text	☐ String
INFO_TXT_REPAIR	Information text for the repair	☐ String
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the faulty component!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the faulty component!	☐ String
PRODUCT_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the manufactured product!	☐ String

Key	Explanation	IMSApi data type
PRODUCT_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the manufactured product!</i>	☐ String
PROGRESS_PERCENT	Progress of the work step in percent Advice: <i>This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORKSTEP_STATE)!</i>	☐ double
QUANTITY_FAILURE	Number of failures	☐ int
QUANTITY_REPAIR	Number of failures repaired	☐ int
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
USER_NAME	Login name of the user who confirmed the booking (user identification)	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!</i>	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: <i>If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!</i>	☐ String

Tab. 21-7 "Key Value Pair": *failureDataResultKeys*

21.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: For technical reasons, station parameter code no. 13200 [▶▶▶] must contain a valid PM identification number to enable data exchange between the TR and PM service; this number is specified in the standard by the system and may be changed only in exceptional cases!
failureDataFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureDataFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": failureDataFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>failureDataResultValues</code> !

Tab. 21-8 Parameter (in) »pmGetFailureData «

Parameter (out)

Parameter name	Explanation
failureDataResultValues	Depending on parameter (in) <code>failureDataResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureDataResultKeys</code> !

Tab. 21-9 Parameter (out) »pmGetFailureData «

21.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-185	Failure	Invalid value for the process layer
-2015	Failure	Date not valid
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function

21.3 pmGetFailureTallyCard

21.3.1 Function description

This function enables failure tally card information to be queried from the iTAC system; a wide range of different filtering criteria can be applied here (e.g., period under review, tester, part number or order number). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned.

The results are output in two parts: the data of the failure tally card and the failure data assigned there. The output structures can be put into context using the key `FAILURE_TALLY_CARD_ID`.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int pmGetFailureTallyCard
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] failureTallyCardFilters
    String key
    String value
  in String[] failureTallyCardKeys
  in String[] failureTallyCardFailureKeys
  out String[] failureTallyCardResultValues
  out String[] failureTallyCardFailureResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `failureTallyCardFilters`, `failureTallyCardKeys` and `failureTallyCardFailureKeys` as well as (out) `failureTallyCardResultValues` and `failureTallyCardFailureResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
DATE_FROM	Time stamp for the start of the period under review; the point in time of the failure data booking applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed or passed with [-1], there is no restriction; 01-01-1970, 0:00:00.000 is then used as the start time!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; the point in time of the failure data booking applies (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: If the key is not passed or passed with [-1], there is no restriction; 12-31-3000, 23:59:59.999 is then used as the end time!	☐	DateMs
FAILURE_TALLY_CARD_SEQUENCE_NUMBER	Unique reference number (current number) for the failure tally card	☐	int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☒	int
PART_NUMBER	Part number in the iTAC system (work order part) Advice: The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system that booked the failure data Advice: See the advice notice under the key PART_NUMBER (wild-card)!	☐	String
TESTER	Login name of the user (user identification) who has provided the data on the failure tally card. Advice: See the advice notice under the key PART_NUMBER (wild-card)!	☐	String
WORKORDER_NUMBER	Production order number Advice: See the advice notice under the key PART_NUMBER (wild-card)!	☐	String

Tab. 21-10 "Key Value Pair": failureTallyCardFilters

Definition of the output parameter "Failure tally card"

Key	Explanation	IMSApi data type
BOOK_DATE	Point in time of the failure data booking (entry in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
FAILURE_TALLY_CARD_ID	Internal identification number of the failure tally card Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the table with "Key Value Pair": failureTallyCardFailureKeys)!	☐ long
FAILURE_TALLY_CARD_SEQUENCE_NUMBER	Unique reference number (current number) for the failure tally card	☐ int
PART_NUMBER	Part number in the iTAC system (work order part)	☐ String
QUANTITY_FAIL	Number of "fail" bookings (failure tally card) Advice: Normally, the sum of the failure tally cards should yield the total booking quantity for the work order; no automatic plausibility check of the quantities is performed, however!	☐ int
QUANTITY_PASS	Number of "pass" bookings (failure tally card) Advice: See the advice notice under the key QUANTITY_FAIL!	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings (failure tally card) Advice: See the advice notice under the key QUANTITY_FAIL!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system that booked the failure data	☐ String
TESTER	Login name of the user (user identification) who has provided the data on the failure tally card.	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 21-11 "Key Value Pair": failureTallyCardKeys

Definition of the output parameter "Failure data for failure tally card"

Key	Explanation	IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_TALLY_CARD_ID	Internal identification number of the failure tally card Advice: Based on this number, the output data can be put into context (see the parameter by the same name in the table with "Key Value Pair": failureTallyCardKeys)!	☐ long
FAILURE_TYPE_CODE	Failure type code	☐ String
GRID_X	Failure position: X value according to part image raster	☐ int
GRID_Y	Failure position: Y value according to part image raster	☐ int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐ String
QUANTITY_FAILURE	Number of failures	☐ int
QUANTITY_REPAIR	Number of failures repaired	☐ int

Tab. 21-12 "Key Value Pair": failureTallyCardFailureKeys

21.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
failureTallyCard-Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureTallyCardFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": failureTallyCardFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureTallyCard-Keys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureTallyCardKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureTallyCardResultValues !
failureTallyCard-FailureKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureTallyCardFailureKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureTallyCardFailureResultValues !

Tab. 21-13 Parameter (in) »pmGetFailureTallyCard«

Parameter (out)

Parameter name	Explanation
failureTallyCard-ResultValues	Depending on parameter (in) failureTallyCardKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureTallyCardKeys !
failureTallyCard-FailureResultValues	Depending on parameter (in) failureTallyCardFailureKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureTallyCardFailureKeys !

Tab. 21-14 Parameter (out) »pmGetFailureTallyCard«

21.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-2015	Failure	Date not valid
-2070	Failure	IMSApi key was passed multiple times to the function
-2119	Failure	Number of records to be output has exceeded the maximum value

21.4

pmGetProductQuantity

21.4.1

Function description

This function enables the querying of production details for a station or a work step. Compared to the product-related »trGetProductQuantity« [▶▶▶] function, this function operates **process-related**; i.e., the ascertained booking quantities are not based on serial number bookings, but rather only on the number of work step bookings.

**Advice**

This API function is influenced by station parameter code nos. 8310 [▶▶▶], 13200 [▶▶▶] and 15270 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

A specific period under review can be selected for the query (see `dateFrom` and `dateTo`). In addition, the function enables filtering according to specific part numbers or order numbers (see `filterDataKeys`).

As a result, production figures, such as the number of manufactured products on the work step level, can be output. Note the various application types that are activated by means of the `functionMode` parameter here.

Application type "A" (grouping of all quantities on the station level):

Applicable for this application type is the configuration `functionMode` with [0], the `stationNumber` and `processLayer` parameters must be passed, a period under review can be entered (`dateFrom` and `dateTo`).

Special case: If the value [-1] is passed for `processLayer`, the quantities are ascertained for all work steps in which `stationNumber` occurs.

Application type "B" (grouping of all quantities on work step level):

For this application type, the `functionMode` configuration with [1] applies, parameters `stationNumber` and `processLayer` must be passed, a period under review can be entered (`dateFrom` and `dateTo`).

Special case: If the value [-1] is passed for `processLayer`, the quantities are ascertained over all work steps in which `stationNumber` occurs.

Application type "C" (grouping of all quantities with work order or part reference):

Applicable for this application type is the configuration `functionMode` with [2], the `filterDataArray` parameter must be passed, the `stationNumber` and `processLayer` parameters are not taken into account, a period under review can be entered (`dateFrom` and `dateTo`).

Application type "D" (grouping of all quantities on the work order level):

No work-step-related quantities are output here; all work-step numbers are consolidated for the respective work order. Applicable for this application type is the configuration `functionMode` with [3], the `filterDataArray` parameter must be passed, the `stationNumber` and `processLayer` parameters as well as the period under review (`dateFrom` and `dateTo`) are not taken into account.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int pmGetProductQuantity
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in DateMs dateFrom
  in DateMs dateTo
  in int functionMode
  in KeyValue[] productQuantityFilters
    String key
    String value
  in String[] filterDataKeys
  in String[] filterDataValues
  in String[] productQuantityResultKeys
  out String[] productQuantityResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `productQuantityFilters`, `filterDataKeys` and `productQuantityResultKeys` as well as (out) `filterDataValues` and `productQuantityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)! Special case: If the key is not passed or is passed with [-1], the maximum value configured using station parameter code no. 15270 [▶▶▶] is used!	☐ int

Tab. 21-15 "Key Value Pair": `productQuantityFilters`

Searching for specific parameter values

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system	☐ String
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
WORKORDER_NUMBER	Production order number Advice: Only active work orders are taken into account!	☐ int
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 21-16 "Key Value Pair": *filterDataKeys*

Definition of the output parameters for the found production data

Key	Explanation	IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐ DateMs
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
CUSTOMER_NAME	Customer name in the iTAC system	☐ String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐ String
PART_DESC	Part description in the iTAC system	☐ String

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number is passed directly instead of the orientation of the PCB!	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double
QUANTITY_FAIL	Number of bookings with the FAIL state	☐ int
QUANTITY_OPEN	Current (open) quantity which can be processed at the work step. Advice: This is the quantity booked with PASS at the previous work step, minus the quantity already processed at the current work step!	☐ int
QUANTITY_PASS	Number of bookings with the PASS state	☐ int
QUANTITY_SCRAP	Number of bookings with the FAIL state	☐ int
QUANTITY_WOR-KORDER_FINISHED	Number of products finished Advice: Only those products that were booked with PASS on the last work step with obligatory confirmation are counted here!	☐ int
QUANTITY_WOR-KORDER_STARTED	Start quantity for the work order	☐ int
QUANTITY_WOR-KORDER_TOTAL	Order quantity (total)	☐ int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: In application type "B" (grouping of all quantities on the work step level), this field cannot be filled, since multiple stations may be affected!	☐ String
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Key	Explanation	IMSApi data type
WORKORDER_STATE	State of the production order [R; F; N; S; P; E; X]: R = planned F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!</i>	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: <i>If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!</i>	☐ String

Tab. 21-17 "Key Value Pair": *productQuantityResultKeys*

21.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: For technical reasons, station parameter code no. 13200 [▶▶▶] must contain a valid PM identification number to enable data exchange between the TR and PM service; this number is specified in the standard by the system and may be changed only in exceptional cases!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: If an invalid value is passed, this parameter is not evaluated (no restriction)!
dateFrom	Time stamp for the start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000!
dateTo	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
functionMode	Function mode [0; 1; 2; 3]: 0 = Grouping of all quantities on the station level (application type "A") 1 = Grouping of all quantities on the work step level (application type "B") 2 = Grouping of all quantities with work order or part reference (application type "C") 3 = Grouping of all quantities on the work order level (application type "D") Advice: Details on the function modes can be found above in the function description (application types "A" - "D")!
productQuantity-Filters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": productQuantityFilters table.

Parameter name	Explanation
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": productQuantityFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
filterDataKeys	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>filterDataValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": filterDataKeys table.
filterDataValues	Value list for the parameters passed by means of <code>filterDataKeys</code> . Advice: Refer to the "Key Value Pair": filterDataKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
productQuantityResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": productQuantityResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>productQuantityResultValues</code> !

Tab. 21-18 Parameter (in) »pmGetProductQuantity«

Parameter (out)

Parameter name	Explanation
productQuantityResultValues	Depending on parameter (in) <code>productQuantityResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>productQuantityResultKeys</code> !

Tab. 21-19 Parameter (out) »pmGetProductQuantity«

21.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-149	Failure	Work order or part number is required
-168	Failure	Invalid value for the function mode (control flag)
-2036	Failure	No matching station exists
-2070	Failure	IMSApi key was passed multiple times to the function

21.5

pmUpdateBooking

21.5.1

Function description

This function allows already captured, person-related process data bookings (**pFDC** – **p**ersonalized **F**actory **D**ata **C**ollection) to be ascertained (correction booking). The state of the pFDC booking can, however, not be changed. In order to unambiguously identify the pFDC record to be updated, the internal identification number `BOOKING_ID` of the initial booking is required. The identification number is generated automatically by the system when a new pFDC record is captured (for further information, see the API function »pmUploadBooking« [▶▶▶] -> output parameter `bookingId` or, in the PM module, the "PDC - Manual data capture" function).

The following conditions must be fulfilled in order to update a pFDC record:

- The internal identification number `BOOKING_ID` of the pFDC record must be known.
- The booking person must be the same person that initially created the pFDC record.
Advice: This restriction only applies to the API function; in the PM module ("PDC - Manual data capture" function), persons with the appropriate authorization can also change the pFDC records of other persons!
- The pFDC record has not yet been approved.

In principle, all information of the original pFDC record can be changed, with the exception of the booking mode (serial, parallel, quantity), the wage type category (**F**, **G** and **M**) and the booking person. Here, of course, the same rules apply as for initial capture; for example, no failure information can be updated when an overhead cost booking is updated. Details on the plausibility mechanisms can be found in the function description of the API function »pmUploadBooking« [▶▶▶]; there please note especially the description of the application types.

**Advice**

This API function is influenced by station parameter code no. 26002 [▶▶▶], site parameter 3050 [▶▶▶] as well as by server parameter 2020 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
AND	pFDC license

Function declaration

```
int pmUpdateBooking
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] pmUpdateBookingData
    String key
    String value
in String[] pmBookingFailureKeys
in String[] pmBookingFailureValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) pmUpdateBookingData, pmBookingFailureKeys and pmBookingFailureValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Data for pFDC booking

Key	Explanation	IMSApi data type
ACTIVITY_TYPE	Booking type; used to determine whether the booking is to be classed as a productive or preparatory (setting up) activity [P; S]: P = Production S = Setup Advice: If the key is not passed, the original value remains valid!	☐ String
APPEND_COMMENT	Individual information text for the booking (addition) Advice: If the original booking already has an information text, the text passed here is appended to the end!	☐ String
BOOK_CODE_EXTERNAL_SYSTEM	Unique booking designator from an external ERP system	☐ String
BOOK_DATE	Booking time of the pFDC data capture (update in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed or passed with [-1], the current date and time are used by default!	☒ DateMs
BOOKING_ID	Internal identification number of the pFDC booking to be updated Advice: The identification number can be ascertained in the PM module, "PDC - Manual data capture" function!	☒ long

Key	Explanation		IMSApi data type
COST_CENTER_NUMBER	Cost center number Advice: <i>If the key is not passed, the original value remains valid!</i>	☐	String
COST_CENTER_NUM-BER_ORIGIN	Number of the originating cost center Advice: <i>If the key is not passed, the original value remains valid!</i>	☐	String
DATE_FROM	Start of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: <i>If the key is not passed, the original value remains valid!</i>	☐	DateMs
DATE_TO	End of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Advice: <i>If the key is not passed, the original value remains valid!</i>	☐	DateMs
FUNC_MODE	Function mode "work step state" [0; 1] Finish PDC booking - Condition: serial booking S , wage type category F and DATE_TO = filled 0 = Interrupt work step 1 = Finish work step Advice: <i>If the key is not passed, the current work step state remains valid!</i>	☐	int
QUANTITY_FAIL	Number of "fail" bookings Advice: <i>The entered value is interpreted as the "delta" to the previous value; i.e. positive values are added to the existing value and negative values are subtracted. The key is only evaluated if the pFDC record to be updated has the wage type category "F" and the booking mode "quantity booking" (Q)!</i>	☐	int
QUANTITY_FIRST_FAIL	Number of "fail" bookings (first test run) Advice: <i>See the advice under QUANTITY_FAIL!</i>	☐	int
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run) Advice: <i>See the advice under QUANTITY_FAIL!</i>	☐	int
QUANTITY - FIRST_SCRAP	Number of "scrap" bookings (first test run) Advice: <i>See the advice under QUANTITY_FAIL!</i>	☐	int
QUANTITY_PASS	Number of "pass" bookings Advice: <i>See the advice under QUANTITY_FAIL!</i>	☐	int
QUANTITY_SCRAP	Number of "scrap" bookings Advice: <i>See the advice under QUANTITY_FAIL!</i>	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int

Key	Explanation		IMSApi data type
STATION_SELECTION	Mode for automatically ending current overhead cost bookings [A; S; T]: A = Ends all current overhead cost bookings of the user on all stations S = Ends all current overhead cost bookings of the user on the active station (<code>stationNumber</code>) T = Ends all current overhead cost bookings of the user on all stations provided they are grouped via an internal process (special case) Advice: This parameter acts only on current overhead cost bookings; if the key is not passed, the value "A" is used by default!	☐	String
TIME_CONSUMPTION	Actual validity period for booking (duration in seconds) Advice: If no value is passed, the validity period is automatically ascertained from keys <code>DATE_FROM</code> and <code>DATE_TO</code>!	☐	long
USER_NAME	Login name of the user (user identification) Advice: By default, this key must be passed with the corresponding password (<code>USER_PASSWORD</code>) (for further information, see station parameter code no. 26002 [▶▶▶]); alternatively, user identification can also be performed with the <code>USER_TOKEN</code> key!	☐	String
USER_PASSWORD	Password of the user Advice: By default, this key must be passed with the associated user identification (<code>USER_NAME</code>); alternatively, user identification can also be performed using the <code>USER_TOKEN</code> key!	☐	String
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: This key can be used as an alternative to identification using <code>USER_NAME</code> and <code>USER_PASSWORD</code>!	☐	String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system; if the wage type is to be updated, it must have the same wage type category. Advice: If the key is not passed, the original value remains valid!	☐	String
WORKORDER_NUMBER	Number of a production order Advice: If the key is not passed, the original value remains valid!	☐	String
WORKSTEP_NUMBER	Number of the work step Advice: If the key is not passed, the original value remains valid!	☐	int

Tab. 21-20 "Key Value Pair": `pmUpdateBookingData`

Failure information for the pFDC booking
(only application type: Wage type category "F")

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
QUANTITY_FAILURE	Number of failures Advice: Failure causes, failure types and failure symptoms can be passed individually or together; the number of failures then applies equally for all!	☐ int
QUANTITY_SOLDER- ING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int

Tab. 21-21 "Key Value Pair": *pmBookingFailureKeys*

21.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
pmUpdateBooking-Data	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pmUpdateBookingData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": pmUpdateBookingData table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
pmBookingFailureKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>pmBookingFailureValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pmBookingFailureKeys table. Advice: Only the parameters passed here are processed by the function!
pmBookingFailureValues	Value list for the parameters selected by means of <code>pmBookingFailureKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>pmBookingFailureKeys</code> ! Refer to the "Key Value Pair": pmBookingFailureKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 21-22 Parameter (in) »pmUpdateBooking«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

21.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-105	Failure	A required parameter is missing
-111	Failure	No pFDC license present for the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-128	Failure	Work step not found
-130	Failure	Work order not found
-169	Failure	Quantity is invalid
-196	Failure	A parameter is invalid
-332	Failure	Failure cause not found
-336	Failure	Failure symptoms not found

-337	Failure	Failure type not found
-2015	Failure	Date not valid
-2036	Failure	No matching station exists
-2064	Failure	Production order -> Parameter missing
-2070	Failure	IMSApi key was passed multiple times to the function
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-3400	Failure	Cost center not found
-3401	Failure	Wage type not found
-3402	Failure	Wage type -> Connection to cost center not present
-3403	Failure	pFDC booking -> Booking mode is invalid
-3405	Failure	Failure information -> Quantity invalid
-3406	Failure	pFDC booking not found
-3407	Failure	pFDC booking -> Change not allowed: user different
-3408	Failure	pFDC booking -> Change not allowed: booking is already approved
-3410	Failure	pFDC booking -> Quantity booking not allowed for wage type category
-3412	Failure	Wage type category must not be changed
-3413	Failure	Quantities may be booked for quantity bookings only
-3414	Failure	Invalid value for "STATION_SELECTION"
-3417	Failure	Booking period is completed

21.6

pmUploadBooking

21.6.1

Function description

This function is used to capture person-related process data (**pFDC** – **personalized Factory Data Collection**). Production order bookings (production quantities or production times), times for cost overrun or overhead cost bookings can be booked in the iTAC system. The booking data is transferred to the PM module ("PDC - Manual data capture" function) where it can be assessed, corrected if necessary and then released.

Only one booking can be made per function call. The most important control parameter of this function is the wage type to be passed (see the `WAGE_TYPE_NUMBER` key); the wage type – or more precisely the wage type category stored there – determines the functionality and the processable data (see the description of the application types **Wage type category "F", "M" and "G"** below). The booking mode (see key `BOOK_MODE`) is used to define whether booking times (divided into parallel or serial) or booking quantities are to be captured.

The function checks the dependencies between the specified cost center and the wage type; the following applies here: If NO cost center assignment is entered for the wage type, all cost centers are permitted. If cost centers are assigned, only these are then allowed to be used. The wage type and the cost center must be active.

The function returns as the result an unambiguous booking designator (`bookingId`) by means of which the booked data is stored in the system. If the initially entered booking data is to be adapted again (correction booking), this can be done using the »pmUpdateBooking« [▶▶▶] API function and by specifying the `bookingId`.

**Advice**

This API function is influenced by station parameter code no. 26002 [▶▶▶], site parameter code nos. 3050 [▶▶▶] and 18521 [▶▶▶] as well as server parameter code no. 19200 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Application type: Wage type category "F" (production order booking)

- The passed wage type (`WAGE_TYPE_NUMBER`) must have wage type category "F".
- A work order (`WORKORDER_NUMBER`) and a work step (`WORKSTEP_NUMBER`) must be passed.
- In addition to production times, production quantities can also be passed (see the advice notice following the list).
- A cost center **MUST** be passed for production times and production quantities – if no cost center is specified, the cost center from the user master data must be used. Similarly, an originating cost center **MUST** additionally be passed in the case of production times; this is not required for production quantities.
- When production quantities are booked, failure assignments can also be booked.

Advice: *Apart from the booking modes "P" (parallel) and "S" (serial) for capturing production times, the booking mode "Q" for quantity bookings can be used **ONLY** here (see the `BOOK_MODE` key)! In the "Q" mode, pass, fail and scrap quantities and, if required, also process-related failure data (failure symptoms, failure types and fail-*

ure causes) are booked for a work order. Process-related failure data are work-step-related failure bookings without serial number reference. The process-related failure data can be queried using the »pmGetFailureData« [▶▶▶] IMSApi function!

Application type: Wage type category "M" (cost overrun booking)

- The passed wage type (WAGE_TYPE_NUMBER) must have wage type category "M".
- A work order (WORKORDER_NUMBER) and a work step (WORKSTEP_NUMBER) must be passed.
- NO production quantities can be passed (see the advice notice following the list).
- A cost center MUST be passed – if no cost center is specified, the cost center from the user master data must be used. Additionally, an originating cost center MUST be passed.
- NO failure assignments can be passed.

Advice: The booking mode (BOOK_MODE) is – irrespective of the actual entry – always executed as "P" (parallel)!

Application type: Wage type category "G" (overhead cost booking)

- The passed wage type (WAGE_TYPE_NUMBER) must have wage type category "G".
- NO work order can be passed.
- NO production quantities can be passed (see the advice notice following the list).
- A cost center MUST be passed – if no cost center is specified, the cost center from the user master data must be used. Additionally, an originating cost center MUST be passed.
- NO failure assignments can be passed.

Advice: The booking mode (BOOK_MODE) can only be executed as "P" (parallel) or "S"!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
AND	pFDC license

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int pmUploadBooking
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] pmUploadBookingData
    String key
    String value
in String[] pmBookingFailureKeys
in String[] pmBookingFailureValues
out long[] bookingId
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) pmUploadBookingData as well as pmBookingFailureKeys and pmBookingFailureValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Data for pFDC booking

Key	Explanation		IMSApi data type
ACTIVITY_TYPE	Booking type; used to determine whether the booking is to be classed as a productive or preparatory (setting up) activity [P; S]: P = Production S = Setup	☐	String
BOOK_CODE_EXTERNAL_SYSTEM	Unique booking designator from the external system (this is usually an ERP system).	☐	String
BOOK_DATE	Booking time of the pFDC data capture (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the key is not passed or passed with [-1], the current date and time are used by default!	☒	DateMs
BOOK_MODE	Booking mode [P; Q; S]: P = Parallel booking Q = Quantity booking S = Serial booking Advice: Flexible usage of this key is only possible for the application type Wage type category "F" (production order booking)! Only modes "P" and "S" apply for application type wage type category "G" ; only mode "P" applies for wage type category "M" . Special case: The following special characteristic applies for mode "Q": the booking time BOOK_DATE is automatically also used for the keys DATE_FROM and DATE_TO (irrespective of the entry)!	☒	String
COMMENT	Individual information text for the booking	☐	String

Key	Explanation	IMSApi data type
CONFIRMATION_MODE	Mode for automatic approvals of the bookings [0; 1; 2]: 0 = NO automatic approval 1 = Automatic approval ONLY if all necessary data exists (user, cost center, wage type) 2 = Automatic approval WITHOUT restriction Advice: If mode [2] is used without a valid user, the internal default user (PersID "0") is used! Advice: By default, this parameter is only evaluated in application type wage type category "F" in combination with BOOK_MODE "Q" (quantity booking). If the key is then not passed, mode [0] is used by default! Special case: In combination with site parameter code no. 18521 [▶▶▶], automatic approval can also be extended to parallel and serial bookings (see BOOK_MODE "P" and "S")!	☐ int
COST_CENTER_NUMBER	Cost center number Advice: The situations under which a cost center must be passed are specified above under the description of the application types!	☐ String
COST_CENTER_NUMBER_ORIGIN	Number of the originating cost center Advice: In addition to the key COST_CENTER_NUMBER, the cost center that is actually responsible for the cost booking can be specified here!	☐ String
DATE_FROM	Start of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time! If the booking is a "serial booking" (see the BOOK_MODE key), the time passed here is automatically the end time of the predecessor booking. If the booking is a "quantity booking", the following special characteristic applies: the booking time BOOK_DATE is automatically also used for the keys DATE_FROM and DATE_TO (irrespective of the entry)!	☐ DateMs
DATE_TO	End of the validity period for the booking; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time! If the booking is a "quantity booking" (see the BOOK_MODE key), the following special characteristic applies: the booking time BOOK_DATE is automatically also used for the keys DATE_FROM and DATE_TO (irrespective of the entry)!	☐ DateMs

Key	Explanation		IMSApi data type
FUNC_MODE	Function mode "work step state" [0; 1]; Here a distinction is made between the following three variants: Capture new serial booking (DATE_TO = empty) 0 = Start work step, interrupt previous work step 1 = Start work step, finish previous work step Finish PDC booking (DATE_TO = filled) 0 = Interrupt work step 1 = Finish work step Capture overhead cost booking 0 = Capture and interrupt previous work step 1 = Capture and finish previous work step. Advice: The BOOK_MODE key is used to pass the booking mode (parallel, quantities, serial). The WAGE_TYPE_NUMBER key controls the wage type category (cost overrun booking, overhead cost booking – see the application types). If the key is not passed, the current work step state remains valid!	☐	int
QUANTITY_FAIL	Number of "fail" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "First Fail" is not changed!	☐	int
QUANTITY_FIRST_FAIL	Number of "fail" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "First Fail" is changed if necessary!	☐	int
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "First Pass" is changed if necessary!	☐	int
QUANTITY - FIRST_SCRAP	Number of "scrap" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "First Scrap" is changed if necessary!	☐	int
QUANTITY_PASS	Number of "pass" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "First Pass" is not changed!	☐	int
QUANTITY_SCRAP	Number of "scrap" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "First Scrap" is not changed!	☐	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int

Key	Explanation		IMSApi data type
STATION_SELECTION	Mode for automatically ending current overhead cost bookings [A; S; T]: A = Ends all current overhead cost bookings of the user on all stations S = Ends all current overhead cost bookings of the user on the active station (<code>stationNumber</code>) T = Ends all current overhead cost bookings of the user on all stations provided they are grouped via an internal process (special case) Advice: <i>This parameter acts only on current overhead cost bookings; if the key is not passed, the value "A" is used by default!</i>	☐	String
TERMINAL	Unique designator of the terminal from which the booking was made.	☒	String
TIME_CONSUMPTION	Actual validity period for booking (duration in seconds) Advice: <i>If no value is passed, the validity period is automatically ascertained from keys DATE_FROM and DATE_TO!</i>	☐	long
USER_NAME	Login name of the user (user identification) Advice: <i>By default, this key must be passed with the corresponding password (USER_PASSWORD) (for further information, see station parameter code no. 26002 [▶▶▶]); alternatively, user identification can also be performed with the USER_TOKEN key!</i>	☐	String
USER_PASSWORD	Password of the user Advice: <i>By default, this key must be passed with the associated user identification (USER_NAME); alternatively, user identification can also be performed using the USER_TOKEN key!</i>	☐	String
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: <i>This key can be used as an alternative to identification using USER_NAME and USER_PASSWORD!</i>	☐	String
WAGE_TYPE_NUMBER	Wage type identification in the iTAC system Advice: <i>The wage type category is determined on the basis of the wage type entered here; this wage type category in turn determines the functionality of this method (see the application types)!</i>	☐	String
WORKORDER_NUMBER	Number of a production order Advice: <i>This key must be passed for the application types Wage type category "F" (production order booking) and Wage type category "M" (cost overrun booking)!</i>	☐	String
WORKSTEP_NUMBER	Number of the work step Advice: <i>This key must be passed for the application type Wage type category "F" (production order booking)!</i>	☐	int

Tab. 21-23 "Key Value Pair": `pmUploadBookingData`

Failure information for the pFDC booking (only application type: Wage type category "F")

Key	Explanation	IMSApi data type
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system (see ADM module); station assignment is not required!	☐ String
QUANTITY_FAILURE	Number of failures Advice: Failure causes, failure types and failure symptoms can be passed individually or together; the number of failures then applies equally for all!	☐ int
QUANTITY_SOLDER- ING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int

Tab. 21-24 "Key Value Pair": *pmBookingFailureKeys*

21.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
pmUploadBooking-Data	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pmUploadBookingData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": pmUploadBookingData table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
pmBookingFailure-Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>pmBookingFailureValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": pmBookingFailureKeys table. Advice: Only the parameters passed here are processed by the function!
pmBookingFailureValues	Value list for the parameters selected by means of <code>pmBookingFailureKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>pmBookingFailureKeys</code> ! Refer to the "Key Value Pair": pmBookingFailureKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 21-25 Parameter (in) »pmUploadBooking«

Parameter (out)

Parameter name	Explanation
bookingId	Internal identification number of the pFDC booking that has been made

Tab. 21-26 Parameter (out) »pmUploadBooking«

21.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-105	Failure	A required parameter is missing
-111	Failure	No pFDC license present for the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-128	Failure	Work step not found
-130	Failure	Work order not found
-169	Failure	Quantity is invalid
-196	Failure	A parameter is invalid
-332	Failure	Failure cause not found
-336	Failure	Failure symptoms not found

-337	Failure	Failure type not found
-2015	Failure	Date not valid
-2036	Failure	No matching station exists
-2064	Failure	Production order -> Parameter missing
-2070	Failure	IMSApi key was passed multiple times to the function
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-3400	Failure	Cost center not found
-3401	Failure	Wage type not found
-3402	Failure	Wage type -> Connection to cost center not present
-3403	Failure	pFDC booking -> Booking mode is invalid
-3405	Failure	Failure information -> Quantity invalid
-3410	Failure	pFDC booking -> Quantity booking not allowed for wage type category
-3412	Failure	Wage type category must not be changed
-3413	Failure	Quantities may be booked for quantity bookings only
-3414	Failure	Invalid value for "STATION_SELECTION"
-3415	Failure	Invalid value for "CONFIRMATION_MODE"
-3416	Failure	External booking code is already being used
-3417	Failure	Booking period is completed

21.7

pmUploadQuantityAndFailureData

21.7.1

Function description

This function transfers quantity bookings (pass, fail and scrap quantities) for a work order and work step; if necessary, it also transfers the process-related failure data (failure symptoms, failure types and failure causes). Process-related failure data are work-step-related failure bookings without serial number reference (PM module). If the process-related failure data is booked in combination with so-called **failure tally cards**, the IMSApi function »pmUploadQuantityFailureTallyCardAndFailureData« [▶▶▶] is to be used.

**Advice**

This API function is influenced by station parameter code nos. 8310 [▶▶▶] and 13200 [▶▶▶] as well as server parameter code no. 19200 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!
*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*
The process-related failure data can be queried with the »pmGetFailureData« [▶▶▶] IMSApi function.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int pmUploadQuantityAndFailureData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] bookingFailureData
    String key
    String value
in String[] failureDataUploadKeys
in String[] failureDataUploadValues
out String[] failureDataUploadResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) bookingFailureData, failureDataUploadKeys and failureDataUploadValues as well as (out) failureDataUploadResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input parameter "Quantity bookings"

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time "last component" (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Since the function is normally used to capture multiple components and not all components can have the same booking time, a period for the passed quantity bookings is defined in combination with the START_DATE key. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☒	DateMs
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case AVO: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system can be passed directly instead of the PCB orientation!	☒	int
QUANTITY_FAIL	Number of "fail" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstFail" is not changed!	☐	int
QUANTITY_FIRST_FAIL	Number of "fail" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstFail" is changed if necessary!	☐	int
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstPass" is changed if necessary!	☐	int
QUANTITY - FIRST_SCRAP	Number of "scrap" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstScrap" is changed if necessary!	☐	int
QUANTITY_PASS	Number of "pass" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstPass" is not changed!	☐	int
QUANTITY_SCRAP	Number of "scrap" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstScrap" is not changed!	☐	int

Key	Explanation		IMSApi data type
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
START_DATE	Booking time "first component"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: In this context, also note the explanation of the BOOK_DATE key (booking time of the last component)! Special case: [-1] passes the current date and the current time!	☐	DateMs
WORKORDER_NUMBER	Number of a production order	☒	String

Tab. 21-27 "Key Value Pair": bookingFailureData

Input parameter "Process-related failure data"

Key	Explanation		IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
FAILURE_CAUSE_CODE	Failure cause code	☐	String
FAILURE_IMAGE_CODE	Failure symptom code	☒	String
FAILURE_SEQUENCE_NUMBER	Sequence number of the faulty test run. For entries greater than "0" , the number is searched for; if it is found, the failure data is updated. If it is not found, a new failure record is created with the specified number. Special case: If [-1] is passed, the next-higher number is automatically ascertained and a new failure record thereby created. If no number is found, a new failure record is likewise created; in this case, however, with the number "1"!	☐	int
FAILURE_TYPE_CODE	Failure type code	☒	String
GRID_X	Failure position: X value according to part image raster	☐	int
GRID_Y	Failure position: Y value according to part image raster	☐	int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐	String
INFO	Information text	☐	String
INFO_TXT_REPAIR	Information text for the repair	☐	String

Key	Explanation	IMSApi data type
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the faulty component!</i>	☐ String
QUANTITY_FAILURE	Number of failures	☐ int
QUANTITY_REPAIR	Number of failures repaired	☐ int
QUANTITY_SOLDER- ING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int

Tab. 21-28 "Key Value Pair": *failureDataUploadKeys*

21.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: <i>For technical reasons, station parameter code no. 13200 [▶▶▶] must contain a valid PM identification number to enable data exchange between the TR and PM service; this number is specified in the standard by the system and may be changed only in exceptional cases!</i>
bookingFailureData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": bookingFailureData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": bookingFailureData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureDataUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>failureDataUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureDataUploadKeys table. Advice: <i>Only the parameters passed here are processed by the function!</i>

Parameter name	Explanation
failureDataUploadValues	Value list for the parameters chosen by means of <code>failureDataUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>failureDataUploadKeys</code> ! Refer to the "Key Value Pair": <code>failureDataUploadKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 21-29 Parameter (in) »`pmUploadQuantityAndFailureData`«

Parameter (out)

Parameter name	Explanation
attributeResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>failureDataUploadKeys</code> and <code>failureDataUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureDataUploadKeys</code> !

Tab. 21-30 Parameter (out) »`pmUploadQuantityAndFailureData`«

21.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-156	Failure	Date value is invalid
-169	Failure	Quantity is invalid
-185	Failure	Invalid value for the process layer
-240	Failure	No work step found for work order, orientation of the PCB and station
-245	Failure	Quantities could not be reported to the PM module
-336	Failure	Failure symptoms not found

-337	Failure	Failure type not found
-339	Failure	Failure is not assigned to the station
-2036	Failure	No matching station exists
-2037	Failure	Work step not found for station
-2070	Failure	IMSApi key was passed multiple times to the function
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid

21.8

pmUploadQuantityFailureTallyCardAndFailureData

21.8.1

Function description

This function transfers quantity bookings (pass, fail and scrap quantities) for a work order and work step; if necessary, it also transfers the process-related failure data (failure symptoms, failure types and failure causes). Process-related failure data are work-step-related failure bookings without serial number reference (PM module). Compared to the IMSApi function »pmUploadQuantityAndFailureData« [▶▶▶], this can be used to perform the booking with reference to so-called **failure tally cards**.



Advice

This API function is influenced by station parameter code nos. 8310 [▶▶▶] and 13200 [▶▶▶] as well as server parameter code no. 19200 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!
*The array returned contains only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*
The process-related failure data can be queried with the »pmGetFailureData« [▶▶▶] IMSApi function.

When using this function, note the following basic principles:

- Any number of failure tally cards may exist for each quantity booking.
- Any number of failure records may exist for each failure tally card.
- As multiple failure tally cards can be created per day for a production order, each failure tally card is given a designator which is unique to the work order.
- For each failure type code, any quantity can be booked at a mounting place; the mounting place is not explicitly checked against the order BOM.
- Multiple failure type codes with multiple mounting places can exist for each failure tally card.
- Failure tally cards can be booked in the name of another employee; this is the case, for example, if a tester fills the failure tally cards (analog), but the data is digitally recorded later at a central location by another employee.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	------------

Function declaration

```
int pmUploadQuantityFailureTallyCardAndFailureData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] bookingFailureData
    String key
    String value
in KeyValue[] failureTallyCardData
    String key
    String value Value
```

```

in String[] failureDataUploadKeys
in String[] failureDataUploadValues
out String[] failureDataUploadResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) booking-FailureData, failureTallyCardData, failureDataUploadKeys and failureDataUploadValues as well as (out) failureDataUploadResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input parameter "Quantity bookings"

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time "last component" (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Since the function is normally used to capture multiple components and not all components can have the same booking time, a period for the passed quantity bookings is defined in combination with the START_DATE key. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☒	DateMs
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case AVO: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system can be passed directly instead of the PCB orientation!	☒	int
QUANTITY_FAIL	Number of "fail" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstFail" is not changed!	☐	int
QUANTITY_FIRST_FAIL	Number of "fail" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstFail" is changed if necessary!	☐	int

Key	Explanation	IMSApi data type
QUANTITY_FIRST_PASS	Number of "pass" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstPass" is changed if necessary!	☐ int
QUANTITY - FIRST_SCRAP	Number of "scrap" bookings (first test run) Advice: The data is booked taking into account the sequence number (sequence number = 1); i.e., evaluation number for "FirstScrap" is changed if necessary!	☐ int
QUANTITY_PASS	Number of "pass" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstPass" is not changed!	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings Advice: The data is booked without taking into account the sequence number (test run); i.e., evaluation number for "FirstScrap" is not changed!	☐ int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
START_DATE	Booking time "first component"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: In this context, also note the explanation of the BOOK_DATE key (booking time of the last component)! Special case: [-1] passes the current date and the current time!	☐ DateMs
WORKORDER_NUMBER	Number of a production order	☒ String

Tab. 21-31 "Key Value Pair": bookingFailureData

Input parameter "Failure tally card"

Key	Explanation	IMSApi data type
FAILURE_TALLY - CARD_SEQUENCE_NUMBER	Unique reference number (current number) for the failure tally card. For entries greater than "0" , the number (failure tally card) is searched for; if it is found, the failure data is updated. If it is not found, a new failure tally card is created with the specified number. Special case: If [-1] is passed, the next-higher number is automatically ascertained and a new failure tally card thereby created. If no number is found, a new failure tally card is likewise created; in this case, however, with the number "1"!	☐ int
QUANTITY_FAIL	Number of "fail" bookings (failure tally card) Advice: Normally, the sum of the failure tally cards should yield the booking quantity according to the key by the same name in the "Key Value Pair": bookingFailureData table; no automatic plausibility check of the quantities is performed, however!	☐ int

Key	Explanation	IMSApi data type
QUANTITY_PASS	Number of "pass" bookings (failure tally card) Advice: See the advice notice under the key QUANTITY_FAIL!	☐ int
QUANTITY_SCRAP	Number of "scrap" bookings (failure tally card) Advice: See the advice notice under the key QUANTITY_FAIL!	☐ int
TESTER	Login name of the user (user identification) who has provided the data on the failure tally card.	☐ String

Tab. 21-32 "Key Value Pair": failureTallyCardData

Input parameter "Process-related failure data"

Key	Explanation	IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FAILURE_CAUSE_CODE	Failure cause code	☐ String
FAILURE_IMAGE_CODE	Failure symptom code	☒ String
FAILURE_SEQUENCE_NUMBER	Sequence number of the faulty test run	☐ int
FAILURE_TYPE_CODE	Failure type code	☒ String
GRID_X	Failure position: X value according to part image raster	☐ int
GRID_Y	Failure position: Y value according to part image raster	☐ int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐ String
INFO	Information text	☐ String
INFO_TXT_REPAIR	Information text for the repair	☐ String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the faulty component!	☐ String
QUANTITY_FAILURE	Number of failures	☐ int
QUANTITY_REPAIR	Number of failures repaired	☐ int
QUANTITY_SOLDERING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int

Tab. 21-33 "Key Value Pair": failureDataUploadKeys

21.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: For technical reasons, station parameter code no. 13200 [▶▶▶] must contain a valid PM identification number to enable data exchange between the TR and PM service; this number is specified in the standard by the system and may be changed only in exceptional cases!
bookingFailureData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": bookingFailureData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": bookingFailureData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureTallyCardData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureTallyCardData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": failureTallyCardData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureDataUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>failureDataUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureDataUploadKeys table. Advice: Only the parameters passed here are processed by the function!
failureDataUploadValues	Value list for the parameters chosen by means of <code>failureDataUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>failureDataUploadKeys</code> ! Refer to the "Key Value Pair": failureDataUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 21-34 Parameter (in) »pmUploadQuantityFailureTallyCardAndFailureData«

Parameter (out)

Parameter name	Explanation
failureDataUploadResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>failureDataUploadKeys</code> and <code>failureDataUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureDataUploadKeys</code> !

Tab. 21-35 Parameter (out) »pmUploadQuantityFailureTallyCardAndFailureData«

21.8.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-156	Failure	Date value is invalid

-169	Failure	Quantity is invalid
-185	Failure	Invalid value for the process layer
-240	Failure	No work step found for work order, orientation of the PCB and station
-245	Failure	Quantities could not be reported to the PM module
-336	Failure	Failure symptoms not found
-337	Failure	Failure type not found
-339	Failure	Failure is not assigned to the station
-2036	Failure	No matching station exists
-2037	Failure	Work step not found for station
-2070	Failure	IMSApi key was passed multiple times to the function
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid

22 Group "REG" (registration)

22.1 regCheckLicense

22.1.1 Function description

This function performs a check for the different licensed functions in order to ascertain whether a certain station or line is licensed for the requested function.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int regCheckLicense
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int licenseType
in String[] licenseUploadKeys
in String[] licenseUploadValues
out String[] licenseResultValues
```

"Key Value Pair" structures

This function contains several "Key Value Pair" structures (see the parameters (in) `licenseUploadKeys` and `licenseUploadValues` as well as (out) `licenseResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

SMT interface (licenseType = 1)

Key	Explanation	IMSApi data type
CONFIG_APPID	Application ID (APPID) of the data interface which communicates with the SMT line.	☒ String
CONFIG_IDENTITY	Configuration ID for the CONFIG_APPID Advice: The CONFIG_IDENTITY distinguishes several technical instances of an APPID!	☒ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
LINE_NAME	Name of the line that is to be checked for valid licensing	☒ String
LINE_TYPE	Typcasting according to supported SMT manufacturers and other special types; in this context, the following types are available: ASM = SMT machines from the manufacturer ASM (SIPLACE) ASSEMBLEON = SMT machines from the manufacturer Assembléon COGISCAN = SMT machines from the manufacturer JUKI COMMONSMT = SMT machines from different manufacturers based on .NET/Java plugins FUJI = SMT machines from the manufacturer Fuji PANASONIC = SMT machines from the manufacturer Panasonic SAMSUNG = SMT machines from the manufacturer Samsung YAMAHA = SMT machines from the manufacturer Yamaha	☒ String

Tab. 22-1 "Key Value Pair": (SMT interface) resultKeys**smart.MESDevice** (licenseType = 2)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-2 "Key Value Pair": (smart.MESDevice) resultKeys**smart.ReceivingDevice** (licenseType = 3)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-3 "Key Value Pair": (smart.ReceivingDevice) resultKeys

PLC Connector (licenseType = 4)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
LINE_NAME	Name of the SMT line that is to be checked for valid licensing	☒ String
LINE_TYPE	PLC = Special type for licensing the PLC connector	☒ String

Tab. 22-4 "Key Value Pair": (PLC-Connector) resultKeys

pFDC (licenseType = 5)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-5 "Key Value Pair": (pFDC) resultKeys

smart.FactoryAppStudio (licenseType = 6)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-6 "Key Value Pair": (smart.FactoryAppStudio) resultKeys

MSL (licenseType = 7)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-7 "Key Value Pair": (MSL) resultKeys

OPC UA.Service (licenseType = 8)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
LINE_NAME	Name of the line that is to be checked for valid licensing	☒ String
LINE_TYPE	OPCUA = Special type for licensing the OPC UA.Service	☒ String

Tab. 22-8 "Key Value Pair": (OPC UA.Service) resultKeys**SmartDataAnalytics.Service** (licenseType = 9)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
LINE_NAME	Name of the SMT line that is to be checked for valid licensing	☒ String
LINE_TYPE	SDA_NODE; SDA_BACK_END = Special type for licensing the SmartDataAnalytics.Service	☒ String

Tab. 22-9 "Key Value Pair": (SmartDataAnalytics.Service) resultKeys**Generic Interface** (licenseType = 10)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
STATION_NUMBER	Station number that is to be checked for valid licensing	☒ String

Tab. 22-10 "Key Value Pair": (MSL) resultKeys

22.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
licenseType	Determines the license type [1; 2; 3; 4; 5; 6; 7; 8; 9; 10]: 1 = SMT-Interface 2 = smart.MESDevice 3 = smart.ReceivingDevice 4 = PLC-Connector 5 = pFDC 6 = smart.FactoryAppStudio 7 = MSL 8 = OPC UA.Service 9 = SmartDataAnalytics.Service 10 = Generic Interface
licenseUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>licenseUploadValues</code> parameter. A description of the possible parameters (dependent on <code>licenseType</code>) can be found above in the corresponding table for license type " Key Value Pair " (xxx): licenseUploadKeys . Advice: Only the parameters passed here are processed by the function!
licenseUploadValues	Values for the parameters selected by means of <code>licenseUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>licenseUploadKeys</code> ! Refer to the corresponding " Key Value Pair " (xxx): licenseUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 22-11 Parameter (in) »regCheckLicense«

Parameter (out)

Parameter name	Explanation
licenseResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>licenseUploadKeys</code> and <code>licenseUploadValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>licenseUploadKeys</code> !

Tab. 22-12 Parameter (out) »regCheckLicense«

22.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-8	Failure	No license
-10	Failure	Server error
-100	Failure	Station not found
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-152	Failure	Not enough comparison values exist for the calculation
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station
-3007	Failure	SMT -> Line type is invalid for the configuration
-3304	Failure	Configuration type not found
-3307	Failure	Parameter value not found

22.2 regGetRegisteredUser

22.2.1 Function description

This function returns the user currently logged in at the station.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int regGetRegisteredUser
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
out String userName
out String name
out String firstname
```

22.2.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system

Tab. 22-13 Parameter (in) »regGetRegisteredUser«

Parameter (out)

Parameter name	Explanation
userName	Login name of the user (user identification)
name	Last name of the user
firstname	First name of the user

Tab. 22-14 Parameter (out) »regGetRegisteredUser«

22.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-2105	Failure	No TR or PM license present for the station

22.3

regLogin

22.3.1

Function description

This function must always be called prior to the actual call of the technical function (see section »IMSApi - functions« [▶▶▶]). User or machine authentication is performed by means of this function. Following successful login, the server assigns a unique designator for the session (sessionID); this designator is retained until the »regLogout« [▶▶▶] IMSApi function is called.



Advice

This API function is influenced by station parameter code no. 15280 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

In addition to the login name (user name or station number), a password must be passed both for user authentication and for machine authentication; passwords are defined in the setup dialogs for persons and stations in the myiTAC workplace (ADM module)!

Note the interaction with the API function »regRegisterUser« [▶▶▶]. If this has already been used for a user login, the user specified here will be assigned to the booking by default – i.e. regLogin (with user data) is superimposed on regRegisterUser. This default behavior can be overridden with site parameter code no. 18100 [▶▶▶]; the user of regRegisterUser is then superimposed on the user from regLogin!

Application type "A" (login as user):

The following configuration applies for the application type: `registrationType = U`, valid values must be passed for `user` and `password`; i.e. user authentication is performed using the user identification/password. A user can be logged in multiple times; no locking mechanisms have been implemented.

Application type "B" (login as station without user data):

The following configuration applies for this application type: `registrationType = S`, no values are passed for `user` and `password` (the internal `USER_ID [-11]` is used). Depending on station parameter code no. 15280, a station may or may not be logged in (registered) multiple times. If the parameter is set, the following locking mechanism is active: The IP address is taken over from the middleware and locked. The IP address is deleted by means of the »regLogout« [▶▶▶] IMSApi function, thereby releasing the station. It is possible to manually reset the IP address in the station setup (ADM module -> Machine assets). The same IP address may login (register) repeatedly (e.g., following a program crash on the client).

Application type "C" (login as station with user data: type "S"):

The following configuration applies for this application type: `registrationType = S`, valid values must be passed for `user` and `password`; i.e. user authentication is performed using the user identification/password. This application type is otherwise identical to application type "B".

Application type "D" (login as station with user data: type "T"):

The following configuration applies for this application type: `registrationType = T`, NO value is allowed to be passed for `user` and the user token must be passed for `password`; i.e. user authentication is performed using an electronic key. This application type is otherwise identical to application type "B".

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	--

Function declaration

```
int regLogin
in IMSApiSessionValidationStruct sessionValidationStruct
    String stationNumber
    String stationPassword
    String user
    String password
    String client
    String registrationType
    String systemIdentifier
out IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
```

22.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionValidationStruct	Structure with the following variables:
stationNumber	Station number of the work station in the iTAC system Advice: This field is only evaluated if station registration (registrationType = S or T) is to occur!
stationPassword	Password of the station Advice: This field is only evaluated if station registration (registrationType = S or T) is to occur; the value is only checked if a password was defined in the station setup!
user	User name in the iTAC system Advice: This field is evaluated in the case of login as a user (application type "A" with registrationType = U) or login as a station with user data (application type "C" with registrationType = S). If a user name is passed, the corresponding user password must also be specified. In application type "D" with registrationType = T, this parameter must NOT be passed!
password	Password of the user Advice: If a user name is passed, the corresponding user password must also be specified! Special case "user token": In application type "D" with registrationType = T, the password is not passed here; the user token (electronic keys) is passed instead.
client	Abbreviation for the client
registrationType	Determines the login type [S; T; U]: S = Station T = User token U = User Advice: This determines which of the parameters are evaluated for the login; for further information, see the description of the application types above!
systemIdentifier	Unique designator (ID) of the client Advice: "Host name/IP address" is generally used as the designator. This designator is output in the log file and thereby serves to simplify assignment of log information!

Tab. 22-15 Parameter (in) »regLogin«

Parameter (out)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN) Advice: This is the default setting set on the server!

Tab. 22-16 Parameter (out) »regLogin«

22.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-7	Failure	Client is unknown
-10	Failure	Server error
-14	Failure	System login failed
-100	Failure	Station not found
-103	Failure	Station is exclusively locked
-2008	Failure	Application ID is invalid
-2009	Failure	Application ID not found
-2010	Failure	Application ID is locked

22.4

regLogout

22.4.1

Function description

This function is used to deactivate the specified session. This function should be called before the client application is exited so that all processes of the session can be deconstructed in a defined manner.



Advice

This API function is influenced by station parameter code no. 15280 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int regLogout
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
```

22.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	- Parameter is not evaluated -
locale	- Parameter is not evaluated -

Tab. 22-17 Parameter (in) »regLogout«

22.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error

22.5

regRegisterUser

22.5.1

Function description

This function registers a user at the station. A user is unregistered by means of the »regUnregisterUser« [▶▶▶] API function. With each state booking, users registered in this way are booked by name in the system (key word "personalized machine booking"). In the »Unit history« in the TR module, the user logged in at the booking time can be ascertained (see the "User name" field there).



Advice

Note the interaction with the API function »regLogin« [▶▶▶]. If this has already been used for a user login, only the user used there is assigned to the booking by default – i.e. regLogin (with user data) is superimposed on regRegisterUser! This default behavior can be overridden with site parameter code no. 18100 [▶▶▶]; the user of regRegisterUser is then superimposed on the user from regLogin!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int regRegisterUser
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String userName
in String password
in String client
```

22.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
userName	Login name of the user (user identification) Special case "user token": If user authentication is to be performed using an electronic key (token) instead of the user identification/password, then this parameter must NOT be passed.

Parameter name	Explanation
password	Password of the user Special case "user token": In the case of user authentication by means of an electronic key, the user token must be passed here instead of the password.
client	Abbreviation for the client

Tab. 22-18 Parameter (in) »regRegisterUser«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

22.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-10	Failure	Server error
-11	Failure	Password for user/station is invalid
-100	Failure	Station not found
-106	Failure	This user has already logged into the station
-107	Failure	Another user is already logged into the station
-2105	Failure	No TR or PM license present for the station

22.6

regUnregisterUser

22.6.1

Function description

This function unregisters the user from the station; as only the registered user may unregister, the password or the user token must also be provided.



Advice

A user is registered by means of the »regRegisterUser« [▶▶▶] API function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int regUnregisterUser
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String userName
in String password
in String client
```

22.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
userName	Login name of the user (user identification) Special case "user token" : If user authentication is to be performed using an electronic key (token) instead of the user identification/password, then this parameter must NOT be passed.
password	Password of the user Special case "user token" : In the case of user authentication by means of an electronic key, the user token must be passed here instead of the password.
client	Abbreviation for the client

Tab. 22-19 Parameter (in) »regUnregisterUser«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

22.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-107	Failure	Another user is already logged into the station
-2105	Failure	No TR or PM license present for the station

23

Group "SETUP" (setup)

23.1

setupAttachPreSetup

23.1.1

Function description

This function is used to set up a concrete pre-setup – completely or only certain parts (pre-setup sections) – on a station (without activation). A pre-setup is a prepared set of setup positions (containers) which are fed to the machine by means of one or more setup carriers – these can be setup carts, tables, carousels or similar. If only specific parts (i.e. certain setup positions) from the pre-setup are to be used, a certain area – the so-called pre-setup section – can be preselected first of all. If, for example, a pre-setup is distributed over multiple setup carts, a pre-setup section could be used to select a specific setup cart from which exactly the required setup positions are then selected and set up on the basis of consumer, work order and/or part information.

Important: At least ONE pre-setup section (i.e. one setup carrier) must exist even if a pre-setup is not to be divided.

With this function, it is not only possible to set up just one pre-setup/pre-setup section, it is also possible to strip down an existing pre-setup/pre-setup section and thus remove it from the machine (see the key `FUNC_MODE`).



Advice

*Note that manual or automatic **activation** of the setup is NOT possible using this function; the »setupStateChange« [▶▶▶] function must be used for this purpose. The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code no. 15002 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupAttachPreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupAttachPositionUploadKeys
in String[] presetupAttachPositionUploadValues
in String[] presetupAttachPositionResultKeys
out String[] presetupAttachPositionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presSetupHeadFilter`, `presSetupAttachPositionUploadKeys` and `presSetupAttachPositionResultKeys` as well as parameter (out) `presSetupAttachPositionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining setup positions from pre-setup

Key	Explanation	IMSApi data type
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) within the pre-setup; this record is used to ascertain the setup positions (container) that are actually to be set up. Background: CONSUMER_IDs are assigned by the system automatically when a pre-setup is created, and can be ascertained using the API function » <code>setupGetPreSetupDetails</code> « [▶▶▶]. A pre-setup can contain multiple consumers, i.e. multiple CONSUMER_IDs (key word "family setup"). Advice: Alternatively, the setup positions to be set up can also be selected from the pre-setup according to work order or part (see the keys <code>PRODUCT_NUMBER</code> and <code>WORKORDER_NUMBER</code>). If none of the three keys is passed, it is attempted to ascertain and set up positions via the work order active at the station. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the part number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ long
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys <code>PRESETUP_NAME</code> and <code>PRESETUP_INDEX</code> are ignored! If no identification number is passed, at least the key <code>PRESETUP_NAME</code> MUST be used!	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key <code>PRESETUP_INDEX</code> is required in combination with <code>PRESETUP_NAME</code> for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if <code>PRESETUP_ID</code> is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key <code>PRESETUP_ID</code> is not used (for further information, see also the key <code>PRESETUP_INDEX</code>)!	☐ String

Key	Explanation	IMSApi data type
PRODUCT_NUMBER	Part number (iTAC system) of the product to be manufactured, on the basis of which the appropriate consumer(s) is/are selected from the pre-setup. The consumers ascertained in this way then serve as the basis for the setup positions (containers) that are actually to be set up. Advice: Alternatively, the setup positions to be set up can also be selected from the pre-setup according to consumer or work order (see the keys CONSUMER_ID and WORKORDER_NUMBER). If none of the three keys is passed, it is attempted to ascertain and set up positions via the work order active at the station. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the part number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ String
WORKORDER_NUMBER	Number of the production order, on the basis of which the appropriate consumer(s) is/are selected from the pre-setup. The consumers ascertained in this way then serve as the basis for the setup positions (containers) that are actually to be set up. Advice: Alternatively, the setup positions to be set up can also be selected from the pre-setup according to consumer or part (see the keys CONSUMER_ID and PRODUCT_NUMBER). If none of the three keys is passed, it is attempted to ascertain and set up positions via the work order active at the station. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the order number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ String

Tab. 23-1 "Key Value Pair": *presetupHeadFilter*

Function-controlling input values for execution

Key	Explanation	IMSApi data type
FUNC_MODE	Function mode "set up / strip down" [0; 1]: 0 = Pre-setup is set up at the station 1 = Pre-setup is stripped down at the station Advice: If mode [0] is used, any pre-setups already on the station (STATION_NUMBER) are stripped down automatically!	☒ int
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_ID)!	☐ String
SECTION_ID	Internal identification number of the pre-setup section Advice: If this key is passed, the key SECTION_NAME is ignored! Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ long

Key	Explanation	IMSApi data type
SECTION_NAME	Name of the pre-setup section Advice: <i>This key is only evaluated if the key SECTION_ID is not used!</i> <i>The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_ID)!</i>	☐ String
STATION_NUMBER	In the iTAC system, this is the station number of the work station at which the ascertained setup positions are set up in accordance with the specified pre-setup.	☐ String

Tab. 23-2 "Key Value Pair": *presetupAttachPositionUploadKeys*

Definition of output values for error feedback

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒ int
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side)	☐ String
SECTION_ID	Internal identification number of the pre-setup section	☐ long
SECTION_NAME	Name of the pre-setup section	☐ String
SECTION_POSITION	Position of the material carrier (e.g. feeder); the key SECTION_SUB_POSITION can be used to specify the position even more precisely.	☐ String
SECTION_SUB_POSITION	Position of the material within a material carrier (e.g. feeder with multiple mounts); this key is used to specify the position entered using the key SECTION_POSITION even more precisely.	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Tab. 23-3 "Key Value Pair": *presetupAttachPositionResultKeys*

23.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
presetupHeadFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupAttachPositionUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presetupAttachPositionUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupAttachPositionUploadKeys table. Advice: Only the parameters passed here are processed by the function!
presetupAttachPositionUploadValues	Values for the parameters selected by means of <code>presetupAttachPositionUploadKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupAttachPositionUploadKeys</code> ! Refer to the "Key Value Pair": presetupAttachPositionUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupAttachPositionResultKeys	List of the parameters that are to be ascertained by the function in the event of an error. A description of the possible parameters can be found above in the "Key Value Pair": presetupAttachPositionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupAttachPositionResultValues</code> !

Tab. 23-4 Parameter (in) »setupAttachPreSetup«

Parameter (out)

Parameter name	Explanation
presetupAttachPositionResultValues	The corresponding data is output here only in the event of an error – depending on the <code>presetupAttachPositionResultKeys</code> parameter (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupAttachPositionResultKeys</code> !

Tab. 23-5 Parameter (out) »setupAttachPreSetup«

23.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-518	Failure	Container state is invalid
-625	Failure	Pre-setup -> Invalid state
-626	Failure	Pre-setup not found
-627	Failure	Pre-setup -> Position not found
-628	Failure	Pre-setup is not completed

-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.2

setupChangeSetupForSerialNumber

23.2.1

Function description

This function is used to update a container for a specific component (mounting place) of a unit (serial number). The following actions are possible:

- Component exchange, e.g. during the course of a repair.
- Container assignment to a component if no container data was previously present.
- Removal of a container assignment from the setup data record (mounting place related only).

**Advice**

This API function is influenced by station parameter code nos. 7027 [▶▶▶], 7028 [▶▶▶], 7134 [▶▶▶] and 15002 [▶▶▶] as well as site parameter code no. 7070 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

*The array returned contains **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

There is a deprecated version of this function that is no longer being developed (for further information, see section »Replaced functions« [▶▶▶] and there the »set-upReplaceSetupForSerialNumber« function)!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupChangeSetupForSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] setupChangeSerialKeys
in String[] setupChangeSerialValues
in String[] setupChangePartKeys
in String[] setupChangePartValues
in String[] setupChangeResultKeys
out String[] setupChangeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) setupChangeSerialKeys, setupChangePartKeys and setupChangeResultKeys as well as (out) setupChangeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Selecting the unit

Key	Explanation	IMSApi data type
BOM_ALTERNATIVE	Mode for the "component exchange" [0; 1]: 0 = Exchange only possible for "regular" BOM components; in this mode, no components that are marked as "alternative" in the BOM can be exchanged. 1 = Exchange possible for all BOM components; in this mode, "regular" and "alternative" components can be exchanged. Advice: If this key is not passed along, the [1] value is used by default!	☐ int
BOOK_DATE	Booking time for the container change (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs
COMP_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the unit data can be put into context with the exchanged component data (setup data) (see the "Key Value Pair": setupChangePartKeys table below)!	☒ long
RESOLVE_DIRECTION	Depending on the current integration level of the passed serial number, defines the direction in which the component to be exchanged in the merge structure is checked [0; 1; -1]: 0 = Only the subordinate integration levels are checked for mounted semifinished products. 1 = Only the parent integration levels are checked for mounted semifinished products. -1 = Both the subordinate as well as the parent integration levels are checked for mounted semifinished products. Advice: How many integration levels are taken into account when resolving is defined with RESOLVE_LEVEL!	☐ int
RESOLVE_LEVEL	Depending on the current integration level of the passed serial number, defines how many integration levels are to be taken into account when resolving > 0; -1]: > 0 = Number of integration levels to be checked -1 = No restriction: All integration levels are taken into account Advice: The direction in which the merge structure is resolved is defined with RESOLVE_DIRECTION!	☐ int

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel; here, this is the serial number for which other container data is to be stored	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: <i>If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!</i>	☐ String
SETUP_STATE	Mode for the setup update [0; 1; 2] 0 = No setup data needs to be stored for the unit; as a result, container information can be stored for a component even if no container information data existed previously. 1 = A setup data record must already exist for the unit; i.e., in this mode, old containers are always exchanged with new containers (see the special case below). 2 = For the mounting place whose specification is obligatory in this mode (COMP_NAME), the container assignment is removed from the setup data record. Advice: <i>If this key is not passed along, the [0] value is used by default!</i> Special case for mode [1]: If container data is not stored in the setup data record for all components, then an exchange is ONLY possible if part number and mounting place are passed (see COMP_NAME and PART_NUMBER in the "Key Value Pair": setupChangePartKeys table!)	☐ int

Tab. 23-6 "Key Value Pair": *setupChangeSerialKeys*

Setup data details

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: <i>If the key is NOT passed or if [-1] is passed, the part (PART_NUMBER) must absolutely be passed; in this case, all components with this part number are exchanged. If both PART_NUMBER as well as COMP_NAME are passed, they are checked against each other for plausibility; i.e., the part must be intended for the mounting place!</i>	☐ String
COMP_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: <i>The records of an array must be uniquely numbered; based on this number, the unit data can be put into context with the exchanged component data (setup data) (see the "Key Value Pair": setupChangeSerialKeys table above)!</i>	☒ long
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
PART_NUMBER	Part number in the iTAC system; this is the part number of the component that is to be exchanged! Advice: <i>If the key is NOT passed or if [-1] is passed, the mounting place (COMP_NAME) must absolutely be passed! If both PART_NUMBER as well as COMP_NAME are passed, they are checked against each other for plausibility; i.e., the part must be intended for the mounting place!</i>	☐ String

Tab. 23-7 "Key Value Pair": *setupChangePartKeys*

Results of the container change (optional)

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: <i>In this case, this is the mounting place of the exchanged component!</i>	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☐ int
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the component to be exchanged!</i>	☐ String
RETURN_CODE	Feedback indicating whether the new container was successfully stored in the setup [0; 1]: 0 = Component (container) was successfully stored 1 = Component (container) could NOT be stored Advice: <i>If the value [1] is output here, the cause of the error can generally be determined from the ERROR_CODE key!</i>	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Tab. 23-8 "Key Value Pair": *setupChangeResultKeys*

23.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
setupChangeSerialKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>setupChangeSerialValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": setupChangeSerialKeys table. Advice: Only the parameters passed here are processed by the function!
setupChangeSerialValues	Values for the parameters selected by means of <code>setupChangeSerialKeys</code> Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>setupChangeSerialKeys</code> ! Refer to the "Key Value Pair": setupChangeSerialKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
setupChangePartKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>setupChangePartValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": setupChangePartKeys table. Advice: Only the parameters passed here are processed by the function!
setupChangePartValues	Values for the parameters selected by means of <code>setupChangePartKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>setupChangePartKeys</code> ! Refer to the "Key Value Pair": setupChangePartKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
setupChangeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupChangeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>setupChangeResultValues</code> !

Tab. 23-9 Parameter (in) »setupChangeSetupForSerialNumber«

Parameter (out)

Parameter name	Explanation
setupChangeResultValues	Depending on parameter (in) <code>setupChangeResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>setupChangeResultKeys</code> !

Tab. 23-10 Parameter (out) »setupChangeSetupForSerialNumber«

23.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-119	Failure	Part is not unique
-121	Failure	Part not found
-196	Failure	A parameter is invalid
-334	Failure	Location acc. to BOM not found
-503	Failure	Container not found
-518	Failure	Container state is invalid
-552	Failure	Container is not permitted for part
-601	Failure	No setup data found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.3

setupCheck

23.3.1

Function description

This function compares the current setup data of a station (actual setup) with the BOM data of the unit that is to be placed (target setup). The setup data of the unit is ascertained on the basis of a serial number or on the basis of a work order. Faulty placement due to incorrect setup is prevented.



Advice

This API function is influenced by station parameter code nos. 7027 [▶▶▶], 7028 [▶▶▶], 7134 [▶▶▶], 8310 [▶▶▶], 13120 [▶▶▶], 13145 [▶▶▶], 13150 [▶▶▶], 13300 [▶▶▶] and 13310 [▶▶▶] as well as den site parameter code nos. 4560 [▶▶▶] and 7070 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active)
AND	ML service (site license)

Function declaration

```

int setupCheck
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberRef
in String serialNumberRefPos
in String workOrderNumber
in int processLayer
in int checkMode
in int resultMode
out String workOrderNumber
out String productNumber
out String placementName
out int setupActive
out CheckSetupData[] checkSetupDataArray
    String partNumber
    String parentPartNumber
    String compName
    String materialBinNumber
    double materialBinQtyActual
    String materialBinState
    long expirationDate
    String setupPosition
    long setupStartDate
    int setupPositionActive
    String machineGroup
    int returnCode

```


23.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Reference serial number of the panel Advice: The (target) setup is ascertained on the basis of the passed serial number or on the basis of the work order; if both parameters are filled, the serial number must belong to the work order! Special case: If [-1] is passed here, the setup is ascertained on the basis of the passed work order (workOrderNumber); if workOrderNumber = [-1], the work order active on the station is used!
serialNumberRef-Pos	Position of the single panel in the unit-array Special case: If [-1] is passed here, the parameter is not evaluated!
workOrderNumber	Number of a production order Advice: The (target) setup is ascertained on the basis of the passed work order or the serial number; if both parameters are filled, the serial number must belong to the work order! Special case: If [-1] is passed here, the setup is ascertained on the basis of the serial number; if serialNumberRef = [-1], the work order active on the station is used!
processLayer	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice regarding [3]: As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function (for further information, see the IS_MIRRORED_BOARD key). The "processLayer= 3" mechanism is therefore outdated; the setup data for the component and soldering side was combined and considered as a whole here! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!

Parameter name	Explanation
checkMode	Comparison of the setup data [0; 1; 2]: 0 = All actively and inactively set up materials are compared with current order data (bill-of-material) 1 = Only the actively set up materials are compared with current order data (bill-of-material) Advice: In order to perform the check in mode [1] to determine that the current setup is also activated for the passed work order (or work order of the serial number), station parameter code no. 13300 [▶▶▶] / 13310 [▶▶▶] must be active! 2 = Checks the setup stored for the passed serial number; the serial number MUST already be booked. Special case: If only one work order is passed in mode [2], the complete setup that was activated in the past for the work order is used as the basis. This special function is intended for manufacturing without serial numbers and can only be used in a meaningful way immediately after the first setup activation!
resultMode	Return mode [0; 1; 2] 0 = No detail information is returned (array remains empty) 1 = Only the faulty setup positions are returned in the array. 2 = All setup positions are returned in the array with corresponding state.

Tab. 23-11 Parameter (in) »setupCheck«

Parameter (out)

Parameter name	Explanation
workOrderNumber	Number of a production order for the setup Advice: If the value [0] is passed for the <code>checkMode</code> parameter, the field remains empty!
productNumber	Part number of the finished product in the iTAC system
placementName	Name of the setup or of the placement program
setupActive	Active setup exists [0; 1]: 0 = No 1 = Yes
checkSetupDataArray	Array of the structure with the following variables:
<code>partNumber</code>	Part number of the component requiring setup in the iTAC system
<code>parentPartNumber</code>	Part number of the parent part according to the BOM structure (key word "apparent unit"; for further information, see the "Material for parent work order" option for the "Material resolved" parameter in the "MRP" tab in the part master) Advice: For this component (<code>partNumber</code>), in the MRP module the secondary demands are directly transferred to the parent part according to the BOM structure (<code>parentPartNumber</code>). As a result, the component does not appear during scheduling, but the material is still considered to be allocated!
<code>compName</code>	Mounting place conformant with the iTAC BOM
<code>materialBinNumber</code>	Number of the material bin (container no.)
<code>materialBinQtyActual</code>	Current quantity in the material bin

Parameter name	Explanation
materialBinState	Container state [B; E; F; R; S]: B = LOCKED (container locked) E = FINISHED (container completed) F = INIT (new container created) R = READY (container ready) S = IN PROCESS (container in process)
expirationDate	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000
setupPosition	Setup location (e.g. track) of the material that has been set up
setupStartDate	Activation of the setup position (slot) in the setup; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000
setupPositionActive	Identifies active setup position [-1; 0; 1]: -1 = Unknown setup position 0 = Inactive setup position 1 = Active setup position
machineGroup	Number of the machine group that performed the setup
returnCode	If a setup position is faulty, a corresponding negative error code is returned. If the container at the setup position has a faulty state (e.g., expiration date expired, quantity = 0 or invalid state), a general warning is returned (see error code 607).

Tab. 23-12 Parameter (out) »setupCheck«

23.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
607	Warning	Container is invalid
605	Warning	No station activated for the setup
604	Warning	No BOM position identified as requiring setup
603	Warning	No work step identified as requiring setup
602	Warning	Setup was not activated for all components
601	Warning	Setup was not activated
538	Warning	At least one container is finished, locked, empty or expired
234	Warning	Serial number locked but usable
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-130	Failure	Work order not found
-202	Failure	Serial number is locked
-205	Failure	Serial number belongs to a different work order
-217	Failure	Serial number and work order have different product versions

-240	Failure	No work step found for work order, orientation of the PCB and station
-355	Failure	Production order is part of a work order cluster
-507	Failure	Material bin is available but empty
-531	Failure	Supplier not found
-605	Failure	Setup not found for the station
-606	Failure	Setup is not assigned to the work order
-607	Failure	Not all parts were set up
-614	Failure	Part has been set up on the wrong station
-615	Failure	Station not found in the processing plan
-616	Failure	Part not found in the setup
-617	Failure	Mounting place not found in the setup
-623	Failure	Additional component set up
-624	Failure	No serial number is assigned to panel label
-1101	Failure	BOM not found
-2005	Failure	No active work order found
-2073	Failure	More than one work order is active on the station
-2107	Failure	No ML license present for the site
-2110	Failure	Serial number position is invalid

23.4

setupCheckPreSetup

23.4.1

Function description

This function allows a provided pre-setup to be checked for completeness. As a rule, the information stored in a pre-setup (actual setup) is checked against the currently required components for which setup is necessary, according to the BOM (target setup). Among other things, it is possible to determine whether all or just certain parts of the pre-setup are to be checked and to select the range of test results that are output (e.g. only faulty records). For control of the function, see in particular the keys `CHECK_MODE`, `FUNC_MODE` and `RESULT_MODE`.



Advice

This API function is influenced by station parameter code nos. 7027 [▶▶▶], 7028 [▶▶▶], and 7134 [▶▶▶] as well as site parameter code nos. 4560 [▶▶▶] and 7070 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupCheckPreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupHeadResultKeys
in String[] presetupPositionResultKeys
in String[] presetupPositionComponentResultKeys
out String[] presetupHeadResultValues
out String[] presetupPositionResultValues
out String[] presetupPositionComponentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadFilter`, `presetupHeadResultKeys`, `presetupPositionResultKeys` and `presetupPositionComponentResultKeys` as well as (out) `presetupHeadResultValues`, `presetupPositionResultValues` und `presetupPositionComponentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for function execution and ascertainment of pre-setup

Key	Explanation	IMSApi data type
CHECK_MODE	Mode for the check [0; 1]: 0 = Check for presence of all required setup positions without container data check; i.e. "empty" setup positions do not cause a failure. 1 = Check for presence of all required setup positions including container check; in this mode, all setup positions must be assigned containers.] Advice: If this key is not passed along, the [1] mode is used by default!	☐ int
CONSUMER_ID	Internal identification number of the concrete usage record (consumers) within the pre-setup; this record is used as the basis for ascertaining the setup positions (containers) to be checked. Background: CONSUMER_IDS are assigned by the system automatically when a pre-setup is created, and can be ascertained using the API function »setupGetPreSetupDetails« [▶▶▶]. A pre-setup can contain multiple consumers, i.e. multiple CONSUMER_IDS (key word "family setup"). Advice: Alternatively, the setup positions to be checked can also be selected from the pre-setup according to work order or part (see the keys PRODUCT_NUMBER and WORKORDER_NUMBER). If none of the three keys is passed, the complete pre-setup (i.e. all setup positions of all contained consumers) is checked. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the part number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ long
FUNC_MODE	Function mode "station-related / complete" [0; 1]: 0 = Pre-setup is checked only on the basis of the querying station (stationNumber); i.e. only the components requiring setup there are taken into consideration during the check. 1 = Complete pre-setup is checked. Advice: If this key is not passed along, the [1] mode is used by default!	☐ int
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys PRESETUP_NAME and PRESETUP_INDEX are ignored! If no identification number is passed, at least the key PRESETUP_NAME MUST be used!	☐ long

Key	Explanation	IMSApi data type
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key PRESETUP_INDEX is required in combination with PRESETUP_NAME for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if PRESETUP_ID is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key PRESETUP_ID is not used (for further information, see also the key PRESETUP_INDEX)!	☐ String
PRODUCT_NUMBER	Part number (iTAC system) of the product to be manufactured, on the basis of which the appropriate consumer(s) is/are selected from the pre-setup. The consumers ascertained in this way then serve as the basis for the setup positions (containers) to be checked. Advice: Alternatively, the setup positions to be checked can also be selected from the pre-setup according to consumer or work order (see the keys CONSUMER_ID and WORKORDER_NUMBER). If none of the three keys is passed, the complete pre-setup (i.e. all setup positions of all contained consumers) is checked. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the part number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ String
RESULT_MODE	Return mode for "setup position data" [0; 1; 2]: 0 = No detail information on the setup positions (i.e. the structures presetupPositionResultValues and presetupPosition-ComponentResultValues remain empty). 1 = Only the faulty setup positions are returned. 2 = All setup positions are returned with corresponding state. Advice: If this key is not passed along, the [1] mode is used by default!	☐ int
WORKORDER_NUMBER	Number of the production order, on the basis of which the appropriate consumer(s) is/are selected from the pre-setup. The consumers ascertained in this way then serve as the basis for the setup positions (containers) to be checked. Advice: Alternatively, the setup positions to be checked can also be selected from the pre-setup according to consumer or part (see the keys CONSUMER_ID and PRODUCT_NUMBER). If none of the three keys is passed, the complete pre-setup (i.e. all setup positions of all contained consumers) is checked. If more than one of the three keys are passed, the entries are checked against each other. If there is a problem – e.g. if the order number is not found in any of the consumers of the pre-setup – the function terminates with a failure!	☐ String

Tab. 23-13 "Key Value Pair": presetupHeadFilter

Definition of the output parameters for the ascertained pre-setup (header data)

Key	Explanation	IMSApi data type
COMPLETION_DATE	Point in time at which the pre-setup was changed to "complete" (state R = READY); specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
CREATION_DATE	Time of creation for the pre-setup (entry into the iTAC system data-base); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
PRESETUP_DESC	Description of the pre-setup	☐ String
PRESETUP_ID	Internal identification number of the pre-setup	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup	☐ int
PRESETUP_NAME	Name of the pre-setup	☐ String
PRESETUP_STATE	Pre-setup state [C; F; I; R; PS; S; SR]: C = CREATED -> pre-setup created F = FINISHED -> pre-setup finished I = IN_PROCESS -> pre-setup partly prepared R = READY -> pre-setup completely prepared PS = PARTLY_SETUP -> pre-setup partly set up S = SETUP -> pre-setup completely set up SR = SETUP_REMOVED -> pre-setup stripped down	☐ String
PRESETUP_TYPE	Pre-setup type [0; 1]: S = Pre-setup of type "Standard" (not portable) P = Pre-setup of type "Portable" Background: Portable pre-setups can be used successively on different stations!	☐ String

Tab. 23-14 "Key Value Pair": *presetupHeadResultKeys*

Setup position

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "presetupPositionComponentResultKeys" table)!	☐ long
CONSUMER_ID	Internal identification number of the concrete usage record (consumer)	☐ long
ERROR_CODE	Error code for the case of a faulty container Advice: If a container is not set up or is faulty (locked, empty or finished), the corresponding technical error code is output here; in this case, the function itself outputs a failure (see error code -607) or a warning (see error code 536)!	☐ int

Key	Explanation		MSApi data type
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
EXPIRATION_DATE_FINAL	End expiration time for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The container expires at this point in time even if it was never in an unprotected environment!	☐	DateMs
MATERIAL_BIN_CREATION_DATE	Date of creation for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐	double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the container material!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the container material!	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
PRODUCT_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the product to be manufactured according to consumer!	☐	String
QUANTITY	Material quantity of the component in the BOM (possibly also as decimal value)	☐	double
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_ID!)	☐	String

Key	Explanation	IMSApi data type
SECTION_ID	Internal identification number of the pre-setup section Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ long
SECTION_NAME	Name of the pre-setup section Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_ID)!	☐ String
SECTION_POSITION	Position of the material carrier (e.g. feeder); the key SECTION_SUB_POSITION can be used to specify the position even more precisely.	☐ String
SECTION_SUB_POSITION	Position of the material within a material carrier (e.g. feeder with multiple mounts); this key is used to specify the position entered using the key SECTION_POSITION even more precisely.	☐ String
STATION_NUMBER	In the iTAC system, this is the station number of the work station at which the ascertained setup positions are set up in accordance with the specified pre-setup.	☐ String
WORKORDER_NUMBER	Production order number Advice: In this case, this is the order number according to consumer!	☐ String

Tab. 23-15 "Key Value Pair": *presetupPositionResultKeys*

Mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the mounting place can be put into context with the setup data record (see the parameter of the same name in the "presetupPositionResultKeys" table)!	☒ long
ERROR_CODE	Error code for the case of a faulty mounting place Advice: If a mounting place is not found, the corresponding technical error code is output here; in this case, the function itself outputs a failure (see error code -616)!	☐ int
PANEL_POSITION	Panel position	☐ int
PRODUCT_NUMBER	Part number of the product according to consumer Background: In the context of "family setup", a part number can exist with various mounting places in different BOMs.	☐ String

Tab. 23-16 "Key Value Pair": *presetupPositionComponentResultKeys*

23.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
presetupHeadFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupHeadResultKeys	List of the parameters (header data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupHeadResultValues</code> !
presetupPositionResultKeys	List of the parameters (setup positions) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupPositionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupPositionResultValues</code> !
presetupPositionComponentResultKeys	List of the parameters (mounting places) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupPositionComponentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupPositionComponentResultValues</code> !

Tab. 23-17 Parameter (in) »setupCheckPreSetup«

**Advice**

Whether, and if so, to what extent setup position data is returned by the function depends on the selected return mode (for further information, see key `RESULT_MODE` in the "Key Value Pair" structure `presetupHeadFilter`)!

Parameter (out)

Parameter name	Explanation
presetupHeadResultValues	Depending on parameter (in) <code>presetupHeadResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>presetupHeadResultKeys</code>!</i>
presetupPositionResultValues	Depending on the parameter (in) <code>presetupPositionResultKeys</code> and the selected return mode (<code>RESULT_MODE</code>), the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>presetupPositionResultKeys</code>!</i>
presetupPositionComponentResultValues	Depending on the parameter (in) <code>presetupPositionComponentResultKeys</code> and the selected return mode (<code>RESULT_MODE</code>), the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>presetupPositionComponentResultKeys</code>!</i>

Tab. 23-18 Parameter (out) »`setupCheckPreSetup`«

23.4.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
604	Warning	No BOM position identified as requiring setup
536	Warning	At least one container is empty, finished or locked
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-542	Failure	Container is locked
-543	Failure	Container is expired
-544	Failure	Container is empty
-601	Failure	No setup data found

-607	Failure	Not all parts were set up
-616	Failure	Part not found in the setup
-625	Failure	Pre-setup -> Invalid state
-626	Failure	Pre-setup not found
-627	Failure	Pre-setup -> Position not found
-628	Failure	Pre-setup is not completed
-629	Failure	Pre-setup -> Consumer not found
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.5

setupCreatePreSetup

23.5.1

Function description

This function is used to create a new pre-setup – more precisely the header structure. Generally speaking, a pre-setup is a prepared set of setup positions to which concrete container data is later added (for further information, see the API function »setupUpdatePreSetupPosition« [▶▶▶]). Pre-setups are fed to the machine by means of one or more setup carriers – these can be setup carts, tables carrouseles or similar – and set up (for further information, see the API function »setupAttachPreSetup« [▶▶▶]).

The required setup carrier(s) is/are also defined using this function – each setup carrier is seen as a so-called pre-setup section. At the same time, the setup positions which then contain the required consumer information are created for each pre-setup section.

Important: At least ONE pre-setup section (setup carrier) must be defined even if a pre-setup is not to be divided. If required, setup carriers made known here can be linked to concrete equipment that exists in the iTAC system. Condition for this is that the equipment in the iTAC system was created with the PM type "Pre-setup".

**Advice**

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = -630) in the event of one or more faulty records!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupCreatePreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadUploadData
    String key
    String value
in String[] presetupConsumerUploadKeys
in String[] presetupConsumerUploadValues
in String[] presetupSectionUploadKeys
in String[] presetupSectionUploadValues
in String[] presetupHeadResultKeys
in String[] presetupConsumerResultKeys
in String[] presetupSectionResultKeys
out String[] presetupHeadResultValues
out String[] presetupConsumerResultValues
out String[] presetupSectionResultValues
```


"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadUploadData`, `presetupConsumerUploadKeys`, `presetupConsumerUploadValues`, `presetupSectionUploadKeys`, `presetupSectionUploadValues`, `presetupHeadResultKeys`, `presetupConsumerResultKeys` and `presetupSectionResultKeys` as well as (out) `presetupHeadResultValues`, `presetupConsumerResultValues` and `presetupSectionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for new pre-setup (header data)

Key	Explanation	IMSApi data type
PRESETUP_DESC	Description of the pre-setup	☐ String
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key <code>PRESETUP_INDEX</code> is required in combination with <code>PRESETUP_NAME</code> for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: The name of the pre-setup must be unique at least in combination with the specified designator (<code>PRESETUP_INDEX</code>)!	☒ String
PRESETUP_TYPE	Pre-setup type [0; 1]: S = Pre-setup of type "Standard" (not portable) P = Pre-setup of type "Portable" Background: Portable pre-setups can be used successively on different stations! Advice: If this key is not passed along, the [S] value is used by default!	☐ String

Tab. 23-19 "Key Value Pair": `presetupHeadUploadData`

Input values for consumers (usage data)

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: BOM typecasting is optional; if it is not used, "neutral" BOMs are used!	String
BOM_VERSION	BOM version from the iTAC system Advice: This key is only evaluated if the WORKORDER_NUMBER is not used. If the key is required but not passed, the highest released version is always used!	int
BOM_VERSION_ERP	BOM version in the ERP system Advice: This key is only evaluated if the BOM_VERSION is not used. Furthermore, if the key is required but not specified, the highest released version is always used!	String
MACHINE_GROUP_NUMBER	Number of the machine group for which the pre-setup is to be used Advice: Each consumer requires a unique machine group association; if the key is not passed, the machine group is ascertained on the basis of the station number (see key STATION_NUMBER)!	String
PART_NUMBER	Part number (iTAC system) of the product to be manufactured for which the pre-setup is to be used Advice: Even though this key is not identified as a mandatory field, either the WORKORDER_NUMBER or the PART_NUMBER must be passed!	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: Each consumer requires a unique work step assignment; this key is evaluated if the key WORKSTEP_NUMBER is not passed. In this case, the work step is ascertained on the basis of the layer and current station!	int
PROCESS_VERSION	Process version from the iTAC system Advice: This key is only evaluated if the WORKORDER_NUMBER is not used. If the key is required but not specified, the highest released version is always used!	int
STATION_NUMBER	Station number of the work station in the iTAC system on the basis of which the machine group for the pre-setup is to be ascertained. Advice: Each consumer requires a unique machine group association; if the key MACHINE_GROUP_NUMBER is not passed, the machine group is ascertained on the basis of the station number specified here. If this key is not specified either, the machine group according to the calling station is used (see stationNumber)!	String

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Number of the production order (iTAC system) for which the pre-setup is to be used Advice: Even though this key is not identified as a mandatory field, either the WORKORDER_NUMBER or the PART_NUMBER must be passed! If the order number is used, the keys PART_NUMBER, BOM_VERSION and PROCESS_VERSION are not interpreted.	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system for which the pre-setup is to be used Advice: Each consumer requires a unique work step assignment; if this key is not passed, the key PROCESS_LAYER must be used!	☐ int

Tab. 23-20 "Key Value Pair": presetupConsumerUploadKeys

Input values for pre-setup section (setup data carrier)

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment (setup carrier) Advice: The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!	☐ String
EQUIPMENT_NUMBER	Number of the equipment (setup carrier) Advice: If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed! Background: When the pre-setup section is created, a setup carrier can be optionally selected – this can be a setup cart, table, carrousel or similar. Condition for this is that the setup carrier specified here already exists in the iTAC system as equipment (PM type = Pre-setup).	☐ String
SECTION_DESC	Description of the pre-setup section	☐ String
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_NAME)!	☐ String
SECTION_NAME	Name of the pre-setup section Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ String
STATION_NUMBER	In the iTAC system, this is the station number of the work station for which this pre-setup section is intended. Background: Pre-setup sections can be permanently assigned to a specific station; this is only the case if the referenced pre-setup is NOT of the "portable" type (see key PRESETUP_TYPE in the "presetupHeadUploadData" table).	☐ String

Key	Explanation	IMSApi data type
STORAGE_NUMBER	Storage place number for the equipment (setup carrier) Advice: This key is only considered if a setup carrier is passed using the key EQUIPMENT_NUMBER. Provided that the storage location is valid, the setup carrier with the new pre-setup is then made available there (stored) for later use! Advice: The storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	String

Tab. 23-21 "Key Value Pair": *presetupSectionUploadKeys*

Output values for newly created pre-setup (header data)

Key	Explanation	IMSApi data type
PRESETUP_ID	Internal identification number of the (new) pre-setup Background: The PRESETUP_ID is assigned automatically by the system when a pre-setup is created, and should be used for unambiguous assignment using the function »setupAttachPreSetup« [▶▶▶].	long
PRESETUP_INDEX	Individual designator for the pre-setup	int
PRESETUP_NAME	Name of the pre-setup	String

Tab. 23-22 "Key Value Pair": *presetupHeadResultKeys*

Output values for used consumers (usage data)

Key	Explanation	IMSApi data type
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) for which the pre-setup was created. Background: The CONSUMER_ID is assigned automatically by the system when a pre-setup is created, and is required for unambiguous assignment using the API function »setupAttachPreSetup« [▶▶▶].	long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -630)!	int
PART_NUMBER	Part number (iTAC system) of the product to be manufactured	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	int
WORKORDER_NUMBER	Number of the production order (iTAC system)	String
WORKSTEP_NUMBER	Work step number in the iTAC system	int

Tab. 23-23 "Key Value Pair": *presetupConsumerResultKeys*

Output values for used pre-setup section (setup data carrier)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -630)!	☐ int
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side)	☐ String
SECTION_ID	Internal identification number of the pre-setup section Background: The SECTION_ID is assigned automatically by the system when a pre-setup is created, and should be used for unambiguous assignment using the API function »setupAttachPreSetup« [▶▶▶].	☐ long
SECTION_NAME	Name of the pre-setup section	☐ String

Tab. 23-24 "Key Value Pair": `presetupSectionResultKeys`

23.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
presSetupHeadUploadData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presSetupHeadUploadData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presSetupHeadUploadData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presSetupConsumerUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presSetupConsumerUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presSetupConsumerUploadKeys table. Advice: Only the parameters passed here are processed by the function!
presSetupConsumerUploadValues	Values for the parameters selected by means of <code>presSetupConsumerUploadKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presSetupConsumerUploadKeys</code> ! Refer to the "Key Value Pair": presSetupConsumerUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presSetupSectionUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presSetupSectionUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presSetupSectionUploadKeys table. Advice: Only the parameters passed here are processed by the function!
presSetupSectionUploadValues	Values for the parameters selected by means of <code>presSetupSectionUploadKeys</code> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presSetupSectionUploadKeys</code> ! Refer to the "Key Value Pair": presSetupSectionUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presSetupHeadResultKeys	List of the parameters (header data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presSetupHeadResultValues</code> !

Parameter name	Explanation
presetupConsumer-ResultKeys	List of the parameters (consumer) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupConsumerResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presetupConsumerResultValues !
presetupSection-ResultKeys	List of the parameters (pre-setup section) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupSectionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presetupSectionResultValues !

Tab. 23-25 Parameter (in) »setupCreatePreSetup«

Parameter (out)

Parameter name	Explanation
presetupHeadResultValues	Depending on parameter (in) presetupHeadResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) presetupHeadResultKeys !
presetupConsumer-ResultValues	Depending on parameter (in) presetupConsumerResultKeys , the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) presetupConsumerResultKeys !
presetupSection-ResultValues	Depending on parameter (in) presetupSectionResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) presetupSectionResultKeys !

Tab. 23-26 Parameter (out) »setupCreatePreSetup«

23.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-124	Failure	Part does not match the work order part
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-163	Failure	Invalid value for "bomType"
-196	Failure	A parameter is invalid
-242	Failure	BOM version is invalid
-528	Failure	Storage place was not found
-535	Failure	Storage place is not unique

-630	Failure	Pre-setup not created
-631	Failure	Pre-setup -> Invalid type
-636	Failure	Pre-setup already exists
-637	Failure	Pre-setup -> Consumer already exists
-638	Failure	Pre-setup -> Pre-setup section already exists
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2071	Failure	Machine group for station not found
-2107	Failure	No ML license present for the site

23.6

setupGetMaterialSetup

23.6.1

Function description

This function checks whether there is an active setup for the date specified at the station passed. Depending on the parameter `setupFlag`, the function can supply various setup data.

**Advice**

This API function is influenced by station parameter code nos. 7027 [▶▶▶], 7028 [▶▶▶], and 8310 [▶▶▶] as well as site parameter code no. 4560 [▶▶▶]. This results among other things in the activation or deactivation of monitoring of the expiration date or of the container state at the setup station. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupGetMaterialSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in int setupFlag
in DateMs setupStartDate
in DateMs setupEndDate
in String[] setupResultKeys
out String[] setupResultValues
in String[] componentResultKeys
out String[] componentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `setupResultKeys` and `componentResultKeys` as well as parameters (out) `setupResultValues` and `componentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Setup data

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: <i>On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "Mounting places" table)!</i>	☐ long
ERROR_CODE	Error code for the case of a faulty container Advice: <i>If a container is faulty (locked, empty or expired), the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 537)!</i>	☐ int
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
MATERIAL_BIN_CREATION_DATE	Date of creation for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐ double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐ double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐ String
PART_DESC	Part description in the iTAC system Advice: <i>In this case, this is the part description of the container material!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part no. of the container material!</i>	☐ String
PART_PARENT_NUMBER	Part number of the parent part according to the BOM structure (key word "apparent unit"; for further information, see the "Material for parent work order" option for the "Material resolved" parameter in the "MRP" tab in the part master)	☐ String

Key	Explanation	IMSApi data type
PRODUCT_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the finished product manufactured on the basis of this setup!	☐ String
PRODUCT_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the finished product manufactured on the basis of this setup!	☐ String
SETUP_END_DATE	End date for the period under review (set up to); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SETUP_NAME	Name of the placement program	☐ String
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☐ String
SETUP_START_DATE	Start date for the period under review (set up from); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SETUP_STATE	Scope of the setup data [0; 1]: 0 = All set up components (active and inactive) 1 = Only set up components that are also active Advice: The value [1] is only output if the value [1] or [2] was passed for the <code>setupFlag</code> input parameter!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 23-27 "Key Value Pair": `setupResultKeys`

Mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the mounting places can be put into context with the setup data record (see the "Setup data" table above)!	☐ long
PANEL_POSITION	Panel position	☐ int

Tab. 23-28 "Key Value Pair": `componentResultKeys`

23.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice regarding [3]: As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function (for further information, see the IS_MIRRORED_BOARD key). The "processLayer= 3" mechanism is therefore outdated; the setup data for the component and soldering side was combined and considered as a whole here! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
setupFlag	Scope of the setup data [0; 1; 2]: 0 = Returns components that are set up and NOT activated (key word "pre-setup") 1 = Returns components that are set up and activated 2 = Returns all set up components (corresponds to [0] + [1])
setupStartDate	Start of the "Active setup" review period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
setupEndDate	End of the "Active setup" review period; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes setupStartDate as time stamp!
setupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) setupResultValues!
componentResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": componentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) componentResultValues!

Tab. 23-29 Parameter (in) »setupGetMaterialSetup«

Parameter (out)

Parameter name	Explanation
setupResultValues	Depending on parameter (in) <code>setupResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>setupResultKeys</code>!</i>
componentResult-Values	Depending on parameter (in) <code>componentResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>componentResultKeys</code>!</i>

Tab. 23-30 *Parameter (out) »setupGetMaterialSetup«*

23.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
537	Warning	At least one container is finished, locked or expired
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-156	Failure	Date value is invalid
-542	Failure	Container is locked
-543	Failure	Container is expired
-544	Failure	Container is empty
-605	Failure	Setup not found for the station
-609	Failure	Invalid value for "setupFlag" [0; 1; 2]
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.7 setupGetPreSetupDetails

23.7.1 Function description

This function allows detailed information about a certain pre-setup to be queried. For this purpose, the desired pre-setup must be passed to the function; this is done either using the internal identification number (`PRESETUP_ID`) or alternatively using the name and index of the pre-setup. Information regarding the stored consumers, the pre-setup sections as well as the individual setup positions can then optionally be output for the pre-setup.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupGetPreSetupDetails
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupHeadResultKeys
in String[] presetupConsumerResultKeys
in String[] presetupSectionResultKeys
in String[] presetupPositionResultKeys
in String[] presetupPositionComponentResultKeys
out String[] presetupHeadResultValues
out String[] presetupConsumerResultValues
out String[] presetupSectionResultValues
out String[] presetupPositionResultValues
out String[] presetupPositionComponentResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadFilter`, `presetupHeadResultKeys`, `presetupConsumerResultKeys`, `presetupSectionResultKeys`, `presetupPositionResultKeys` and `presetupPositionComponentResultKeys` as well as (out) `presetupHeadResultValues`, `presetupConsumerResultValues`, `presetupSectionResultValues`, `presetupPositionResultValues` and `presetupPositionComponentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for ascertaining desired pre-setup

Key	Explanation	IMSApi data type
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys PRESETUP_NAME and PRESETUP_INDEX are ignored! If no identification number is passed, at least the key PRESETUP_NAME MUST be used!	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key PRESETUP_INDEX is required in combination with PRESETUP_NAME for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if PRESETUP_ID is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key PRESETUP_ID is not used (for further information, see also the key PRESETUP_INDEX)!	☐ String

Tab. 23-31 "Key Value Pair": *presetupHeadFilter*

Output parameters for the ascertained pre-setup (header data)

Key	Explanation	IMSApi data type
COMPLETION_DATE	Point in time at which the pre-setup was changed to "complete" (state R = READY); specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
CREATION_DATE	Time of creation for the pre-setup (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
PRESETUP_DESC	Description of the pre-setup	☐ String
PRESETUP_ID	Internal identification number of the pre-setup	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup	☐ int
PRESETUP_NAME	Name of the pre-setup	☐ String
PRESETUP_STATE	Pre-setup state [C; F; I; R; PS; S; SR]: C = CREATED -> pre-setup created F = FINISHED -> pre-setup finished I = IN_PROCESS -> pre-setup partly prepared R = READY -> pre-setup completely prepared PS = PARTLY_SETUP -> pre-setup partly set up S = SETUP -> pre-setup completely set up SR = SETUP_REMOVED -> pre-setup stripped down	☐ String

Key	Explanation	IMSApi data type
PRESETUP_TYPE	Pre-setup type [0; 1]: S = Pre-setup of type "Standard" (not portable) P = Pre-setup of type "Portable" Background: Portable pre-setups can be used successively on different stations!	☐ String

Tab. 23-32 "Key Value Pair": *presetupHeadResultKeys*

Output parameters for ascertained consumers

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [F; I; M; R; U; S; X]: F = BOM for production order I = BOM for receiving M = BOM for maintenance order R = BOM for repair order U = BOM for modification order S = BOM for external products X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) for which the pre-setup was created. Background: The CONSUMER_ID is assigned automatically by the system when a pre-setup is created. A pre-setup can contain multiple consumers, i.e. multiple CONSUMER_IDS (key word "family setup").	☐ long
ERROR_CODE	- key not used -	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group for which the pre-setup is intended.	☐ String
PART_NUMBER	Part number (iTAC system) of the product to be manufactured for which the pre-setup is intended.	☐ String
PRESETUP_ID	Internal identification number of the pre-setup	☐ long
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system for which the pre-setup is intended.	☐ String
WORKORDER_NUMBER	Number of the production order (iTAC system) for which the pre-setup is intended.	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system for which the pre-setup is intended.	☐ int

Tab. 23-33 "Key Value Pair": *presetupConsumerResultKeys*

Output parameters for ascertained pre-setup sections

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment (setup carrier)	☐ String
EQUIPMENT_NUMBER	Number of the equipment (setup carrier)	☐ String
ERROR_CODE	- key not used -	☐ int
PRESETUP_ID	Internal identification number of the pre-setup	☐ long
SECTION_DESC	Description of the pre-setup section	☐ String
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_NAME)!	☐ String
SECTION_ID	Internal identification number of the pre-setup section Background: The SECTION_ID is assigned automatically by the system when a pre-setup is created.	☐ long
SECTION_NAME	Name of the pre-setup section Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ String
SECTION_STATE	Pre-setup section state [C; F; I; R; S; SR]: C = CREATED -> pre-setup section created F = FINISHED -> pre-setup section finished I = IN_PROCESS -> pre-setup section partly prepared R = READY -> pre-setup section completely prepared S = SETUP -> pre-setup section completely set up SR = SETUP_REMOVED -> pre-setup section stripped down	☐ String
SETUP_DATE	Point in time at which the pre-setup section was set up at the station; specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
STATION_NUMBER	In the iTAC system, this is the station number of the work station for which this pre-setup section is intended. Background: Pre-setup sections can be permanently assigned to a specific station; this is only the case if the referenced pre-setup is NOT of the "portable" type (see key PRESETUP_TYPE in the "preset-upHeadResultKeys" table).	☐ String
STORAGE_NUMBER	Storage place number for the equipment (setup carrier) Advice: The storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String

Tab. 23-34 "Key Value Pair": *presetupSectionResultKeys*

Output parameters for ascertained setup positions

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "presetupPositionComponentResultKeys" table)!	☐ long
ERROR_CODE	Error code for the container check Advice: As soon as a failure is detected for a record (array element) during the container check, the corresponding technical error code is output here (see error code -542, -543 or -544); in this case, the function itself outputs a general warning (error code 5)!	☐ int
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
EXPIRATION_DATE_FINAL	End expiration time for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The container expires at this point in time even if it was never in an unprotected environment!	☐ DateMs
MATERIAL_BIN_CREATION_DATE	Date of creation for the container; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐ double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐ double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐ String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the container material!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the container material!	☐ String

Key	Explanation	IMSApi data type
SECTION_ID	Internal identification number of the pre-setup section Advice: Based on this number, the associated pre-setup section can be referenced from the "Key Value Pair" structure <code>presetupSectionResultKeys</code> !	☐ long
SECTION_POSITION	Position of the material carrier (e.g. feeder); the key <code>SECTION_SUB_POSITION</code> can be used to specify the position even more precisely.	☐ String
SECTION_SUB_POSITION	Position of the material within a material carrier (e.g. feeder with multiple mounts); this key is used to specify the position entered using the key <code>SECTION_POSITION</code> even more precisely.	☐ String

Tab. 23-35 "Key Value Pair": `presetupPositionResultKeys`

Output parameters for ascertained mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "presetupPositionResultKeys" table)!	☒ long
ERROR_CODE	- key not used -	☐ int
PANEL_POSITION	Panel position	☐ int
PRODUCT_NUMBER	Part number of the product according to consumer Background: In the context of "family setup", a part number can exist with various mounting places in different BOMs.	☐ String

Tab. 23-36 "Key Value Pair": `presetupPositionComponentResultKeys`

23.7.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
presSetupHeadFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the desired pre-setup is to be ascertained; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": presSetupHeadFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": presSetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presSetupHeadResultKeys	List of the parameters (header data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presSetupHeadResultValues!
presSetupConsumerResultKeys	List of the parameters (consumer) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupConsumerResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presSetupConsumerResultValues!
presSetupSectionResultKeys	List of the parameters (pre-setup sections) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupSectionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presSetupSectionResultValues!
presSetupPositionResultKeys	List of the parameters (setup positions) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupPositionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presSetupPositionResultValues!
presSetupPositionComponentResultKeys	List of the parameters (mounting places) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presSetupPositionComponentResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presSetupPositionComponentResultValues!

Tab. 23-37 Parameter (in) »setupGetPreSetupDetails«

Parameter (out)

Parameter name	Explanation
presetupHeadResultValues	Depending on parameter (in) <code>presetupHeadResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupHeadResultKeys</code> !
presetupConsumerResultValues	Depending on parameter (in) <code>presetupConsumerResultKeys</code> , the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupConsumerResultKeys</code> !
presetupSectionResultValues	Depending on parameter (in) <code>presetupSectionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupSectionResultKeys</code> !
presetupPositionResultValues	Depending on parameter (in) <code>presetupPositionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupPositionResultKeys</code> !
presetupPositionComponentResultValues	Depending on parameter (in) <code>presetupPositionComponentResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupPositionComponentResultKeys</code> !

Tab. 23-38 Parameter (out) »`setupGetPreSetupDetails`«

23.7.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-542	Failure	Container is locked
-543	Failure	Container is expired
-544	Failure	Container is empty

-1301	Failure	Production means not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.8

setupGetRequiredMaterialForPreSetup

23.8.1

Function description

This function allows the material required for a specific pre-setup to be queried; in this case, "material required" refers to the components of the consumers which require setup and are listed in the BOMs stored there.

To ascertain the required material, the desired pre-setup must first be passed to the function; this is done either using the internal identification number (`PRESETUP_ID`) or alternatively using the name and index of the pre-setup. On the basis of the consumers contained in the pre-setup, information regarding the components requiring setup as well as regarding the alternatives (provided that alternatives are stored in the BOM) can be output for the pre-setup.

Application types

The type of information output can be controlled using the key `FUNC_MODE`; a distinction is made between the following three application types:

- **Separate mounting place / consumer-specific** (`FUNC_MODE = 0`)
In this mode, the ascertained material is output separately according to mounting place and according to the different consumers. In other words, identical materials at different mounting places are output in separate records.
Advice: *Alternative components can be ascertained and output only for this function mode!*
- **Consolidated mounting place / consumer-specific** (`FUNC_MODE = 1`)
In this mode, the ascertained materials are grouped according to the mounting place but still output separately according to the different consumers. In other words, identical materials at different mounting places for a consumer are grouped together in a record; in this case, the material quantity is added together (`QUANTITY`) and the mounting places are output separated by commas (`COMP_NAME`).
- **Consolidated mounting place / cross-consumer** (`FUNC_MODE = 2`)
In this mode, the ascertained materials are grouped according to the mounting place and the different consumers. In other words, identical materials at different mounting places and different consumers are grouped together in a record; in this case, the material quantity is added together (`QUANTITY`). The keys `COMP_NAME`, `CONSUMER_ID`, `PRODUCT_NUMBER` and `WORKORDER_NUMBER` always remain empty here.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupGetRequiredMaterialForPreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupMaterialResultKeys
```

```
in String[] presetupMaterialAlternativeResultKeys
out String[] presetupMaterialResultValues
out String[] presetupMaterialAlternativeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadFilter`, `presetupMaterialResultKeys` and `presetupMaterialAlternativeResultKeys` as well as (out) `presetupMaterialResultValues` and `presetupMaterialAlternativeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for function execution and ascertainment of pre-setup

Key	Explanation	IMSApi data type
FUNC_MODE	Function mode "data output" [0; 1; 2]: 0 = Separate mounting place / consumer-specific 1 = Consolidated mounting place / consumer-specific 2 = Consolidated mounting place / cross-consumer Advice: Detailed information on this topic can be found above in the function description under the point "Application types". If the key is not passed, the value [1] is used by default!	☐ int
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys <code>PRESETUP_NAME</code> and <code>PRESETUP_INDEX</code> are ignored! If no identification number is passed, at least the key <code>PRESETUP_NAME</code> MUST be used!	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key <code>PRESETUP_INDEX</code> is required in combination with <code>PRESETUP_NAME</code> for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if <code>PRESETUP_ID</code> is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key <code>PRESETUP_ID</code> is not used (for further information, see also the key <code>PRESETUP_INDEX</code>)!	☐ String

Tab. 23-39 "Key Value Pair": `presetupHeadFilter`

Definition of output parameters for components

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☐ int
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: In the function mode "Consolidated mounting place / cross-consumer" (FUNC_MODE = 2), this key always remains empty! In FUNC_MODE = 1, multiple mounting places may be output (separated by commas). Detailed information on this topic can be found above in the function description under the point "Application types"!	☐ String
COMP_REFERENCE	Reference number of the component Advice: On the basis of this number, alternative components can be put into context with the regular components (see the parameter of the same name in the "presetupMaterialAlternativeResultKeys" table)!	☐ long
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) for which the pre-setup was created. Background: The CONSUMER_ID is assigned automatically by the system when a pre-setup is created. A pre-setup can contain multiple consumers, i.e. multiple CONSUMER_IDS (key word "family setup"). Advice: In the function mode "Consolidated mounting place / cross-consumer" (FUNC_MODE = 2), this key always remains empty!	☐ long
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the component!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the component!	☐ String
PRODUCT_NUMBER	Part number (iTAC system) of the product to be manufactured for which the pre-setup is to be used Advice: In the function mode "Consolidated mounting place / cross-consumer" (FUNC_MODE = 2), this key always remains empty!	☐ String
QUANTITY	Material quantity of the component at the mounting place (if necessary, also as decimal value) Advice: Depending on the function mode, material quantities for multiple mounting places are added if necessary. Detailed information on this topic can be found above in the function description under the point "Application types"!	☐ double
WORKORDER_NUMBER	Number of the production order (iTAC system) for which the pre-setup is to be used Advice: In the function mode "Consolidated mounting place / cross-consumer" (FUNC_MODE = 2), this key always remains empty!	☐ String

Tab. 23-40 "Key Value Pair": presetupMaterialResultKeys

Definition of output parameters for alternative components

This "Key Value Pair" structure can only be filled if the function mode "Separate mounting place / consumer-specific" has been selected (FUNC_MODE = 0).

Key	Explanation	IMSApi data type
BOM_VERSION	BOM version from the iTAC system	☐ int
COMP_REFERENCE	Reference number of the component Advice: <i>On the basis of this number, alternative components can be put into context with the regular components (see the parameter of the same name in the "presetupMaterialResultKeys" table)!</i>	☐ long
PART_DESC	Part description in the iTAC system Advice: <i>In this case, this is the part description of the alternative component!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the alternative component!</i>	☐ String
QUANTITY	Material quantity of the alternative component at the mounting place (if necessary, also as decimal value)	☐ double

Tab. 23-41 "Key Value Pair": *presetupMaterialAlternativeResultKeys*

23.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
presetupHeadFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the desired pre-setup is to be ascertained; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of key. Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupMaterial-ResultKeys	List of the parameters (regular components) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupMaterialResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presetupMaterialResultValues!
presetupMaterialAlternativeResultKeys	List of the parameters (alternative components) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupMaterialAlternativeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) presetupMaterialAlternativeResultValues!

Tab. 23-42 Parameter (in) »setupGetRequiredMaterialForPreSetup«

Parameter (out)

Parameter name	Explanation
presetupMaterial-ResultValues	Depending on parameter (in) presetupMaterialResultKeys, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) presetupMaterialResultKeys!
presetupMaterialAlternativeResultValues	Depending on parameter (in) presetupMaterialAlternativeResultKeys, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) presetupMaterialAlternativeResultKeys!

Tab. 23-43 Parameter (out) »setupGetRequiredMaterialForPreSetup«

23.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-626	Failure	Pre-setup not found
-629	Failure	Pre-setup -> Consumer not found
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.9

setupGetSetupDataBySerialNumber

23.9.1

Function description

This function returns the setup data for a serial number. It is possible to filter the existing setup data for the serial number according to certain components (parts) and according to specific mounting places (see variables `partNumber` and `location`). The two filter functions are "AND"-coupled such that only setup data that meet both filter conditions are returned.

**Advice**

Mounting places are optional components of a setup; filtering of the output by specific mounting places only functions if the mounting places are actually defined in the setup!

This API function is influenced by station parameter code no.: 13140 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

This function should NO LONGER be used for new installations because the edit options have been extended with release 8.52.00 (for further information, see the new function »[setupGetSetupDataBySerialNumberRef](#)« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupGetSetupDataBySerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String partNumber
in String location
in int level
in String[] setupResultKeys
out String[] setupResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `setupResultKeys` and (out) `setupResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time of the serial number that was placed with this setup (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
CLASSIFICATION	Classification of the container	☐	String
COMP_NAME	Mounting place conformant with the iTAC BOM	☐	String
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
INFO	Output of the stored BOM position information	☐	String
LEVEL	Position of a BOM in the product hierarchy	☐	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the container material!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part no. of the container material!	☐	String
PART_PARENT_NUMBER	Part number of the parent part according to the BOM structure (key word "apparent unit"; for further information, see the "Material for parent work order" option for the "Material resolved" parameter in the "MRP" tab in the part master)	☐	String

Key	Explanation		IMSApi data type
READY_FOR_USE_DATE	Point in time from which the container may first be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
SERIAL_MASTER_NUMBER	Serial number of the parent material bin (key word "bin-based traceability")	☐	String
SERIAL_MASTER_TYPE	Display of the serial number type for the parent material bin [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: "Normal" serial numbers do not have a special type; an empty string is output in this case!	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐	String
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: "Normal" serial numbers do not have a special type; an empty string is output in this case!	☐	char
SETUP_COMP_NAME	Mounting place from the setup	☐	String
SETUP_END_DATE	End date for the period under review (set up to); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☐	String
SETUP_START_DATE	Start date for the period under review (set up from); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs

Key	Explanation		IMSApi data type
STATION_DESC	Station description of the work station in the iTAC system on which setup took place	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system on which setup took place	☐	String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐	String
SUPPLIER_DATE_CODE	DateCode for the material lot	☐	String
SUPPLIER_NAME	Manufacturer name	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKSTEP_NUMBER	Number of the work step at which setup took place	☐	int

Tab. 23-44 "Key Value Pair": *setupResultKeys*

23.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
partNumber	Part number in the iTAC system Advice: Setup data is only returned for this part (components of the individual item)! Special case: If the value [-1] is passed, this filter function is deactivated; all setup data is returned! Note that the output may, in some cases, be restricted by the "mounting place" filter function (see the location parameter)!
location	Mounting place conformant with the iTAC BOM Advice: The setup data is only returned for these mounting places; several mounting places must be separated by " " (pipe)! Special case: If the value [-1] is passed, this filter function is deactivated; all setup data is returned! Note that the output may, in some cases, be restricted by the "part number" filter function (see the partNumber parameter)!
level	Defines how many integration levels are to be taken into account when ascertaining the setup data [> 0; 0; -1]: > 0 = Number of integration levels to be checked 0 = Only current integration level -1 = No restriction: all integration levels are taken into account
setupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) setupResultValues !

Tab. 23-45 Parameter (in) »setupGetSetupDataBySerialNumber«

Parameter (out)

Parameter name	Explanation
setupResultValues	Depending on parameter (in) setupResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) setupResultKeys !

Tab. 23-46 Parameter (out) »setupGetSetupDataBySerialNumber«

23.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-130	Failure	Work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site
-2138	Failure	Serial number is deleted

23.10

setupGetSetupDataBySerialNumberRef

23.10.1

Function description

This function returns the setup data for a serial number, optionally also for all serial numbers of a multiple panel (SERIAL_NUMBER_DUPLICATE). It is possible to filter the existing setup data for the serial number according to certain components (parts) and according to specific mounting places (see variables PART_NUMBER and COMP_NAME). The two filter functions are "AND"-coupled such that only setup data that meet both filter conditions are returned.

**Advice**

Mounting places are optional components of a setup; filtering of the output by specific mounting places only functions if the mounting places are actually defined in the setup!

This API function is influenced by station parameter code no.: 13140 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »setupGetSetupDataBySerialNumber« [▶▶▶] function!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

Function declaration

```
int setupGetSetupDataBySerialNumberRef
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] serialSetupFilter
    String key
    String value
in String[] setupResultKeys
out String[] setupResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) serialSetupFilter and setupResultKeys as well as (out) setupResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM Advice: The setup data is only returned for these mounting places; several mounting places must be separated by " " (pipe). If the key is not passed, there is no restriction; all setup data is returned! Special case: Note that the output may, in some cases, be restricted by the "part number" filter function (see the PART_NUMBER key)!	☐ String
DISSOLVING_LEVEL	Defines how many integration levels are to be taken into account when ascertaining the setup data [> 0 ; 0]: > 0 = Number of integration levels to be checked 0 = Only current integration level Advice: If this key is not passed along, the [0] value is used by default!	☐ int
MAX_ROWS	Maximum number of records in the array returned with each function call; the entered value must be in the number range from [1] to [10000]. Advice: If this key is not passed along, the [1000] value is used by default. If more records exist than are defined here, the function outputs a warning (see error code 3)! Special case: The default value MAX_ROWS is only interpreted if setup data is output for a multiple panel, i.e. for multiple serial numbers. The following statement always applies: all setup data is ALWAYS output for a serial number. This means that if e.g. 122 setup data records are ascertained for a serial number, then all of these records will be output even if MAX_ROWS has been set to a lower value (e.g. 100).	☐ int
PART_NUMBER	Part number in the iTAC system Advice: Setup data is only returned for this part (components of the individual item). If the key is not passed, there is no restriction; all setup data is returned! Special case: Note that the output may, in some cases, be restricted by the "mounting place" filter function (see the COMP_NAME key)!	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: For the "Multiple panel" application type (SERIAL_NUMBER_DUPLICATE= [1]), the serial number does not necessarily have to be the reference serial number; the data can also be ascertained based on a "normal" serial number of the multiple panel!	☒ String

Key	Explanation	IMSApi data type
SERIAL_NUMBER_DUPLICATE	Consideration of serial numbers in the multiple panel [0; 1]: 0 = Multiple panel function NOT active; only the setup data of the passed serial number (SERIAL_NUMBER) is output 1 = Multiple panel function active; the setup data is output for all serial numbers of the multiple panel (for further information, see the advice notice for the SERIAL_NUMBER key). Advice: If this key is not passed along, the [0] value is used by default!	int

Tab. 23-47 "Key Value Pair": serialSetupFilter

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time of the serial number that was placed with this setup (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
CLASSIFICATION	Classification of the container	String
COMP_NAME	Mounting place conformant with the iTAC BOM	String
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
INFO	Output of the stored BOM position information	String
LEVEL	Position of a BOM in the product hierarchy	int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	String
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	double
MATERIAL_BIN_STATE	Container state [B; C; E; F; L; P; Q; R; S; U; V]: B = locked (material bin locked) C = closed (material bin finalized) E = completed (material bin completed) F = initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = prepared (material bin prepared) Q = QA locked (material bin with QA lock) R = ready (material bin ready) S = in process (material bin in use) U = Used up (material bin used up) V = pre-release (material bin with pre-release)	String

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system Advice: <i>In this case, this is the part description of the container material!</i>	☐ String
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part no. of the container material!</i>	☐ String
PART_PARENT_NUMBER	Part number of the parent part according to the BOM structure (key word "apparent unit"; for further information, see the "Material for parent work order" option for the "Material resolved" parameter in the "MRP" tab in the part master)	☐ String
READY_FOR_USE_DATE	Point in time from which the container may first be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SERIAL_MASTER_NUMBER	Serial number of the parent material bin (key word "bin-based traceability")	☐ String
SERIAL_MASTER_TYPE	Display of the serial number type for the parent material bin [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: <i>"Normal" serial numbers do not have a special type; an empty string is output in this case!</i>	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: <i>"Normal" serial numbers do not have a special type; an empty string is output in this case!</i>	☐ char
SETUP_COMP_NAME	Mounting place from the setup	☐ String

Key	Explanation		IMSApi data type
SETUP_END_DATE	End date for the period under review (set up to); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☐	String
SETUP_START_DATE	Start date for the period under review (set up from); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
STATION_DESC	Station description of the work station in the iTAC system on which setup took place	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system on which setup took place	☐	String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐	String
SUPPLIER_DATE_CODE	DateCode for the material lot	☐	String
SUPPLIER_NAME	Manufacturer name	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String
WORKORDER_NUMBER	Production order number	☐	String
WORKSTEP_NUMBER	Number of the work step at which setup took place	☐	int

Tab. 23-48 "Key Value Pair": *setupResultKeys*

23.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialSetupFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialSetupFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": serialSetupFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
setupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>setupResultValues</code> !

Tab. 23-49 Parameter (in) »`setupGetSetupDataBySerialNumberRef`«

Parameter (out)

Parameter name	Explanation
setupResultValues	Depending on parameter (in) <code>setupResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>setupResultKeys</code> !

Tab. 23-50 Parameter (out) »`setupGetSetupDataBySerialNumberRef`«

23.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-174	Failure	Number of records to be output has fallen below the minimum value
-196	Failure	A parameter is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

-2119	Failure	Number of records to be output has exceeded the maximum value
-2138	Failure	Serial number is deleted

23.11 **setupGetSetupName**

23.11.1 Function description

This function returns the name of the placement program.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupGetSetupName
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
out String setupName
```

23.11.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system

Tab. 23-51 Parameter (in) »setupGetSetupName«

Parameter (out)

Parameter name	Explanation
setupName	Name of the placement program

Tab. 23-52 Parameter (out) »setupGetSetupName«

23.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-602	Failure	Name of the placement program non-unique
-2107	Failure	No ML license present for the site

23.12 setupGetStationSetupDataForMaterialBin

23.12.1 Function description

This function calculates where and when a container was set up and returns respective setup information.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```

int setupGetStationSetupDataForMaterialBin
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String materialBinNumber
in String[] setupResultKeys
out String[] setupResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `setupResultKeys` and (out) `setupResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
MATERIAL_BIN_DATE_CODE	DateCode for the article	☐	String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☐	String
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system	☐	String
SETUP_END_DATE	Last booking for the setup; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs

Key	Explanation	IMSApi data type
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☐ String
SETUP_START_DATE	Start date for the setup; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	☐ DateMs
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String

Tab. 23-53 "Key Value Pair": *setupResultKeys*

23.12.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinNumber	Number of the material bin (container no.)
setupResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": setupResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>setupResultValues</i> !

Tab. 23-54 Parameter (in) »*setupGetStationSetupDataForMaterialBin*«

Parameter (out)

Parameter name	Explanation
setupResultValues	Depending on parameter (in) <i>setupResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>setupResultKeys</i> !

Tab. 23-55 Parameter (out) »*setupGetStationSetupDataForMaterialBin*«

23.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-503	Failure	Container not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.13

setupSearchPreSetup

23.13.1

Function description

This function allows the iTAC system to be searched for selected pre-setups; a wide range of different filtering criteria can be applied (e.g. used equipment (setup carrier), product or work order to be manufactured, name of pre-setup). All search criteria are "AND" coupled such that only the data that meet all used filter conditions are returned. Furthermore, it is also possible to limit the size of the hit list (records to be output) (see key `MAX_ROWS`).

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupSearchPreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupHeadResultKeys
in String[] presetupConsumerResultKeys
in String[] presetupSectionResultKeys
out String[] presetupHeadResultValues
out String[] presetupConsumerResultValues
out String[] presetupSectionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadFilter`, `presetupHeadResultKeys`, `presetupConsumerResultKeys` and `presetupSectionResultKeys` as well as (out) `presetupHeadResultValues`, `presetupConsumerResultValues` and `presetupSectionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining desired pre-setups

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment (setup carrier) Advice: <i>The index only needs to be passed if the EQUIPMENT_NUMBER itself is NON-unique!</i>	☐ String
EQUIPMENT_NUMBER	Equipment number (setup carrier); if this key is used, only pre-setups are considered which are currently assigned to the specified setup carrier. Advice: <i>If the equipment number is non-unique, the EQUIPMENT_INDEX key must also be passed!</i>	☐ String
MACHINE_GROUP_NUMBER	Machine group number; if this key is used, only pre-setups are considered which are assigned to this machine group.	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If this key is not passed along, the [100] value is used by default! If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐ int
PART_NUMBER	Part number (iTAC system) of the product to be manufactured; if this key is used, only pre-setups are considered which are to be assigned to a consumer with the product specified here.	☐ String
PRESETUP_NAME	Name of the pre-setup Advice: <i>The wildcard character "*" (multi-character wildcard) or "?" (single-character wildcard) can be used during input; pay attention to upper- and lower-case letters!</i>	☐ String
PRESETUP_STATE	Pre-setup state [C; I; R; PS; S; SR]: C = CREATED -> pre-setup created I = IN_PROCESS -> pre-setup partly prepared R = READY -> pre-setup completely prepared PS = PARTLY_SETUP -> pre-setup partly set up S = SETUP -> pre-setup completely set up SR = SETUP_REMOVED -> pre-setup stripped down Advice: <i>The state F (pre-setup finished) is not supported; i.e. finished pre-setups are ignored during the search. To filter by multiple entries (OR link), the desired values must be passed separated by a comma (e.g.: "I,R"). If the key is passed along, there is no restriction!</i>	☐ String
WORKORDER_NUMBER	Production order number (iTAC system); if this key is used, only pre-setups are considered which are assigned to a consumer with the work order specified here.	☐ String

Tab. 23-56 "Key Value Pair": *presetupHeadFilter*

Output parameters for the ascertained pre-setup (header data)

Key	Explanation	IMSApi data type
COMPLETION_DATE	Point in time at which the pre-setup was changed to "complete" (state R = READY); specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
CREATION_DATE	Time of creation for the pre-setup (entry into the iTAC system data-base); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
PRESETUP_DESC	Description of the pre-setup	☐ String
PRESETUP_ID	Internal identification number of the pre-setup	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup	☐ int
PRESETUP_NAME	Name of the pre-setup	☐ String
PRESETUP_STATE	Pre-setup state [C; I; R; PS; S; SR]: C = CREATED -> pre-setup created I = IN_PROCESS -> pre-setup partly prepared R = READY -> pre-setup completely prepared PS = PARTLY_SETUP -> pre-setup partly set up S = SETUP -> pre-setup completely set up SR = SETUP_REMOVED -> pre-setup stripped down Advice: The state F (pre-setup finished) is not supported; i.e. finished pre-setups are ignored during the search!	☐ String
PRESETUP_TYPE	Pre-setup type [0; 1]: S = Pre-setup of type "Standard" (not portable) P = Pre-setup of type "Portable" Background: Portable pre-setups can be used successively on different stations!	☐ String

Tab. 23-57 "Key Value Pair": *presetupHeadResultKeys*

Output parameters for ascertained consumers

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String

Key	Explanation	IMSApi data type
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) for which the pre-setup was created. Background: The CONSUMER_ID is assigned automatically by the system when a pre-setup is created. A pre-setup can contain multiple consumers, i.e. multiple CONSUMER_IDs (key word "family setup").	☐ long
ERROR_CODE	- key not used -	☐ int
MACHINE_GROUP_NUMBER	Number of the machine group for which the pre-setup is intended.	☐ String
PART_NUMBER	Part number (iTAC system) of the product to be manufactured for which the pre-setup is intended.	☐ String
PRESETUP_ID	Internal identification number of the pre-setup Advice: On the basis of this number, consumers and pre-setup can be put into context (see the parameter of the same name in the "pre-setupPositionResultKeys" table)!	☐ long
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system for which the pre-setup is intended.	☐ String
WORKORDER_NUMBER	Number of the production order (iTAC system) for which the pre-setup is intended.	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system for which the pre-setup is intended.	☐ int

Tab. 23-58 "Key Value Pair": *presetupConsumerResultKeys*

Output parameters for ascertained pre-setup sections

Key	Explanation	IMSApi data type
EQUIPMENT_INDEX	Individual designator for the equipment (setup carrier)	☐ String
EQUIPMENT_NUMBER	Number of the equipment (setup carrier)	☐ String
ERROR_CODE	- key not used -	☐ int
PRESETUP_ID	Internal identification number of the pre-setup Advice: On the basis of this number, a pre-setup section and a pre-setup can be put into context (see the parameter of the same name in the "presetupPositionResultKeys" table)!	☐ long
SECTION_DESC	Description of the pre-setup section	☐ String
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_NAME)!	☐ String

Key	Explanation	IMSApi data type
SECTION_ID	Internal identification number of the pre-setup section Background: The SECTION_ID is assigned automatically by the system when a pre-setup is created.	☐ long
SECTION_NAME	Name of the pre-setup section Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ String
SECTION_STATE	Pre-setup state [C; I; R; S; SR]: C = CREATED -> pre-setup section created I = IN_PROCESS -> pre-setup section partly prepared R = READY -> pre-setup section completely prepared S = SETUP -> pre-setup section completely set up SR = SETUP_REMOVED -> pre-setup section stripped down	☐ String
SETUP_DATE	Point in time at which the pre-setup section was set up at the station; specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
STATION_NUMBER	In the iTAC system, this is the station number of the work station for which this pre-setup section is intended. Background: Pre-setup sections can be permanently assigned to a specific station; this is only the case if the referenced pre-setup is NOT of the "portable" type (see key PRESETUP_TYPE in the "preset-upHeadResultKeys" table).	☐ String
STORAGE_NUMBER	Storage place number for the equipment (setup carrier) Advice: The storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String

Tab. 23-59 "Key Value Pair": *presetupSectionResultKeys*

23.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
presetupHeadFileter	"Key Value Pair" structure with the following variables:

Parameter name	Explanation
key	List of the parameters on the basis of which pre-setups are searched for; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupHeadResultKeys	List of the parameters (header data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupHeadResultValues</code> !
presetupConsumerResultKeys	List of the parameters (consumer) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupConsumerResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupConsumerResultValues</code> !
presetupSectionResultKeys	List of the parameters (pre-setup sections) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": presetupSectionResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>presetupSectionResultValues</code> !

Tab. 23-60 Parameter (in) »setupSearchPreSetup«

Parameter (out)

Parameter name	Explanation
presetupHeadResultValues	Depending on parameter (in) <code>presetupHeadResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupHeadResultKeys</code> !
presetupConsumerResultValues	Depending on parameter (in) <code>presetupConsumerResultKeys</code> , the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupConsumerResultKeys</code> !
presetupSectionResultValues	Depending on parameter (in) <code>presetupSectionResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>presetupSectionResultKeys</code> !

Tab. 23-61 Parameter (out) »setupSearchPreSetup«

23.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-10	Failure	Server error
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-196	Failure	A parameter is invalid
-1301	Failure	Production means not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.14

setupStateChange

23.14.1

Function description

Depending on the parameter `activateFlag` – this function permits the deletion, activation, or deactivation of the setup at a station.

**Advice**

This API function is influenced by station parameter code nos. 7132 [▶▶▶], 7134 [▶▶▶], 13100 [▶▶▶] (indirectly), 13120 [▶▶▶], and 15002 [▶▶▶] as well as site parameter code no. 7070 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

The following points must be observed for setup creation and setup activation:

- The setup must be created at the station in advance; i.e. for all components requiring setup which are specified in the work order and the BOM stored there, a valid container must be made known beforehand (e.g. by means of IMSApi `setUpUpdateMaterialSetup`).
- If required, components can be set up which are not needed for the work order currently to be manufactured. This provides the possibility of creating a setup which can then be used for multiple work orders. In this context, such components are set-up components which, however, do not require setup for the work order.
- If, during creation of the setup, a product number (`PRODUCT_NUMBER`) is passed in the header data, the setup can be used, i.e. activated, only for work orders with this product number (= work order part).

Advice: *If a setup is to be created across all work orders, a product number must NOT be passed in the header data of the setup!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupStateChange
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String workOrderNumber
in String serialNumber
in int serialNumberPos
in int activateFlag
```

23.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice: If the value [3] is passed, the possibly differing work steps for component and soldering side are combined for this work order and are considered as a whole (e.g. for the setup check)! As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function.
workOrderNumber	Number of a production order Advice: The production order is only needed for "Activate setup" mode (see activateFlag)! Special case: If an empty string or the value [-1] is passed, the work order can be ascertained on the basis of the serial number (serialNumber)!
serialNumber	Serial number of the panel Advice: The serial number is only needed for "Activate setup" mode if no order number was passed (see activateFlag)! If both a serial number and an order number are passed, the serial number must belong to the work order!
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
activateFlag	"Setup" change mode [0; 1; 2]: 0 = Activate setup 1 = Deactivate setup 2 = Delete setup Advice: To activate a setup (mode = 0), a production order must absolutely be passed; use the workOrderNumber or serialNumber parameter for this purpose!

Tab. 23-62 Parameter (in) »setupStateChange«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

23.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
608	Warning	Not all positions stripped down
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-184	Failure	Invalid value for "activateFlag" [0; 1; 2]
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-355	Failure	Production order is part of a work order cluster
-603	Failure	Work order part does not match the setup part
-604	Failure	At least one component at a mounting place causes a problem
-605	Failure	Setup not found for the station
-1105	Failure	BOM for the work order not found

		Advice: Value -1105 is output, in particular, if no components requiring setup were found while attempting to activate the set-up!
-2037	Failure	Work step not found for station
-2073	Failure	More than one work order is active on the station
-2107	Failure	No ML license present for the site

23.15

setupUpdateMaterialSetup

23.15.1

Function description

With this function, it is possible to update the setup for individual setup locations for a station. In other words, it can be used to add or update individual setup data.

If the parameters `MATERIAL_BIN_NUMBER`, `PART_NUMBER` and `MATERIAL_BIN_QTY_TOTAL` are passed and the container is not found in the system, it is created automatically. Only in this case are the manufacturer-specific parameters also taken into account. No provision is made for updating already existing container data.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code nos.: 7134 [▶▶▶], 8310 [▶▶▶], 13110 [▶▶▶], 13130 [▶▶▶] and 15002 [▶▶▶] as well as station parameter code no. 7070 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!*

The following points must be observed for setup creation and setup activation:

- The setup must be created at the station in advance; i.e. for all components requiring setup which are specified in the work order and the BOM stored there, a valid container must be made known beforehand (e.g. by means of `IMSApi setupUpdateMaterialSetup`).
- If required, components can be set up which are not needed for the work order currently to be manufactured. This provides the possibility of creating a setup which can then be used for multiple work orders. In this context, such components are set-up components which, however, do not require setup for the work order.
- If, during creation of the setup, a product number (`PRODUCT_NUMBER`) is passed in the header data, the setup can be used, i.e. activated, only for work orders with this product number (= work order part).

Advice: *If a setup is to be created across all work orders, a product number must NOT be passed in the header data of the setup!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupUpdateMaterialSetup
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String workOrderNumber
  in String productNumber
  in String setupName
  in String[] materialSetupUploadKeys
  in String[] materialSetupUploadValues
  in String[] compPositionsUploadKeys
  in String[] compPositionsUploadValues
  out String[] materialSetupResultValues
  out String[] compPositionsResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) materialSetupUploadKeys and compPositionsUploadKeys as well as parameters (out) materialSetupResultValues and compPositionsResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Setup data

Key	Explanation	IMSApi data type
CLASSIFICATION	Classification of the container	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the mounting places can be put into context with the setup data record (see the "Mounting places" table below)!	☐ long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
MATERIAL_BIN_- DATE_CODE	DateCode for the material bin	☐ String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒ String
MATERIAL_BIN_QTY_- TOTAL	Original quantity in the material bin (original container quantity)	☐ double

Key	Explanation		IMSApi data type
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part no. of the container material!</i>	☐	String
PART_PARENT_NUMBER	Part number of the parent part according to the BOM structure (key word "apparent unit"; for further information, see the "Material for parent work order" option for the "Material resolved" parameter in the "MRP" tab in the part master)	☐	String
PRODUCT_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part no. of the finished product manufactured on the basis of this setup!</i>	☐	String
SETUP_END_DATE	End date for the period under review (set up to); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, setup data from the past can also be adapted in this way. Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!	☐	DateMs
SETUP_NAME	Name of the placement program	☐	String
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☒	String
SETUP_START_DATE	Start date for the period under review (set up from); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, setup data from the past can also be adapted in this way. Special case: [-1] passes the current date and the current time!	☐	DateMs
SETUP_STATE	Setup mode [0; 1] 0 = Components are pre-setup 1 = Components are stripped down Extension of setup mode [0]: If a setup already exists at the machine for this setup location, the material is setup again. If a setup is active on this machine, the new setup data record is also activated. If no setup is yet activated for this machine, it is pre-setup and – depending on station parameter code no. 13110 [▶▶▶] – immediately activated.	☒	int
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐	String
SUPPLIER_NAME	Manufacturer name	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String

Tab. 23-63 "Key Value Pair": materialSetupUploadKeys

Mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the mounting places can be put into context with the setup data record (see the "Setup data" table above)!	☒ long
PANEL_POSITION	Panel position	☐ int

Tab. 23-64 "Key Value Pair": `compPositionsUploadKeys`

23.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
workOrderNumber	Number of a production order Advice: The passing of the order number is relevant for the updating of setup data that lies in the past or for indirectly passing a part number (provided the <code>productNumber</code> parameter is not known); otherwise, an empty string or the value [-1] can be passed here!
productNumber	Part number of the finished product in the iTAC system Advice: For a pre-setup, the part number can be passed for the finished product that is to be set up. In this case, this part number can only be used for work orders of this finished product! If no part number is passed, a work order must absolutely be specified (see the <code>workOrderNumber</code> parameter) from which the part number is then ascertained!
setupName	Name of the setup or of the placement program Advice: This parameter is optional!

Parameter name	Explanation
materialSetupUploadKeys	List of the parameters (setup data) that are to be passed by the function; the corresponding values are passed with the materialSetupUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": materialSetupUploadKeys table. Advice: Only the parameters passed here are processed by the function!
materialSetupUploadValues	Value list for the parameters passed by means of materialSetupUploadKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) materialSetupUploadKeys! Refer to the "Key Value Pair": materialSetupUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
compPositionsUploadKeys	List of the parameters (mounting places) that are to be passed by the function; the corresponding values are passed with the compPositionsUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": compPositionsUploadKeys table. Advice: Only the parameters passed here are processed by the function!
compPositionsUploadValues	Value list for the parameters passed by means of compPositionsUploadKeys. Advice: The data is assigned in the order in which it was entered by means of parameter (in) compPositionsUploadKeys! Refer to the "Key Value Pair": compPositionsUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 23-65 Parameter (in) »setupUpdateMaterialSetup«

Parameter (out)

Parameter name	Explanation
materialSetupResultValues	The corresponding data is output here only in the event of an error – depending on the materialSetupUploadKeys and materialSetupUploadValues parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) materialSetupUploadKeys!
compPositionsResultValues	The corresponding data is output here only in the event of an error – depending on the compPositionsUploadKeys and compPositionsUploadValues parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) compPositionsUploadKeys!

Tab. 23-66 Parameter (out) »setupUpdateMaterialSetup«

23.15.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-124	Failure	Part does not match the work order part
-127	Failure	Part number is invalid
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-355	Failure	Production order is part of a work order cluster
-501	Failure	At least one material bin not found
-502	Failure	Failure creating the material bin
-503	Failure	Container not found
-509	Failure	Material lot not found

-517	Failure	Retrieval/creation of a supplier not possible
-518	Failure	Container state is invalid
-606	Failure	Setup is not assigned to the work order
-610	Failure	Invalid value for "setupState" [0; 1]
-618	Failure	Invalid setup position
-619	Failure	Time range "Setup" is invalid
-620	Failure	Wrong component, slot was stripped down
-2070	Failure	IMSApi key was passed multiple times to the function
-2073	Failure	More than one work order is active on the station
-2107	Failure	No ML license present for the site

23.16

setupUpdatePreSetup

23.16.1

Function description

This function provides various possibilities of changing a pre-setup existing in the system. Both the header data of the pre-setup and information regarding the consumer and/or pre-setup sections can be changed. It is NOT possible to edit setup positions; the API function »setupUpdatePreSetupPosition« [▶▶▶] must be used for this purpose.



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns an error message (error code = -632) in the event of one or more faulty records!

Application types

A list of the typical application types with information on the procedure is given here in order to provide you with a clearer overview of the function range. The application types are distinguished according to the data objects **Pre-setup** (header data), **Consumer** and **Pre-setup section**. If necessary, non-mutually exclusive application types can be combined; e.g. changing of the header data of a pre-setup and deletion of a consumer.

Application types "Editing of pre-setup" (Head)

- **Change** -> Selection of the desired pre-setup using the "Key Value Pair" `pre-setupHeadFilter`; the values to be changed are passed using `presetupHeadUploadKeys`. The state of the pre-setup cannot be changed manually; it is calculated automatically depending on the state of the pre-setup sections.
Condition: Key `DELETE_MODE` = 0
- **Delete** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`.
Condition: Key `DELETE_MODE` = 1 and the pre-setup must be "empty", i.e. it must NOT contain a pre-setup section.

Application types "Editing of consumers" (Consumer)

- **Create** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the new consumer data is passed using `presetupConsumerUploadKeys`.
Condition: Key `DELETE_MODE` = 0 and `CHANGE_MODE` = 0
- **Change** -> Selection of the desired pre-setup using the "Key Value Pair" `pre-setupHeadFilter`; the values to be changed are passed using `presetupConsumerUploadKeys`. The record to be changed is determined using the key `CONSUMER_ID`.
Condition: Key `DELETE_MODE` = 0 and `CHANGE_MODE` = 1
- **Delete** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the entry to be deleted is passed using `CONSUMER_ID`.
Condition: Key `DELETE_MODE` = 0 and `CHANGE_MODE` = 2

Application types "Editing of pre-setup section" (Section)

- **Create** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the data of the new pre-setup section is passed using `presetupSectionUploadKeys`.
Condition: Key `DELETE_MODE = 0`, `CHANGE_MODE = 0` and the parent pre-setup must have the state **C** (`CREATED`).
- **Change** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the values to be changed are passed using `presetupSectionUploadKeys`. The record to be changed is determined using the key `SECTION_ID`.
Condition: Key `DELETE_MODE = 0` and `CHANGE_MODE = 1`; if the state is to be changed (key `SECTION_STATE`), only the following state changes are possible:
 - from **I** (`IN_PROCESS`) to **R** (`READY`) or **F** (`FINISHED`)
 - from **R** (`READY`) to **I** (`IN_PROCESS`) or **F** (`FINISHED`)
 - from **SR** (`SETUP_REMOVED`) to **I** (`IN_PROCESS`) or **F** (`FINISHED`)
 - from **F** (`FINISHED`) to `IN_PROCESS`
- **Delete** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the entry to be deleted is passed using `SECTION_ID`.
Condition: Key `DELETE_MODE = 0`, `CHANGE_MODE = 2`, the parent pre-setup must NOT have the state **S** (`SETUP`) and **PS** (`PARTLY_SETUP`) and the pre-setup section must be "empty", i.e. it must NOT contain any setup positions.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int setupUpdatePreSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in KeyValue[] presetupHeadUploadKeys
    String key
    String value
in String[] presetupConsumerUploadKeys
in String[] presetupConsumerUploadValues
in String[] presetupSectionUploadKeys
in String[] presetupSectionUploadValues
out String[] presetupConsumerResultValues
out String[] presetupSectionResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presetupHeadFilter`, `presetupHeadUploadKeys`, `presetupConsumerUploadKeys`, `presetupConsumerUploadValues`, `presetupSectionUploadKeys` and `presetupSectionUploadValues` as well as (out) `presetupConsumerResultValues` and `presetupSectionResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Input values for ascertaining desired pre-setup

Key	Explanation	IMSApi data type
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys PRESETUP_NAME and PRESETUP_INDEX are ignored! If no identification number is passed, at least the key PRESETUP_NAME MUST be used!	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key PRESETUP_INDEX is required in combination with PRESETUP_NAME for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if PRESETUP_ID is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key PRESETUP_ID is not used (for further information, see also the key PRESETUP_INDEX)!	☐ String

Tab. 23-67 "Key Value Pair": *presetupHeadFilter*

Input values for editing pre-setup (header data)

Key	Explanation	IMSApi data type
DELETE_MODE	Change mode [0; 1]: 0 = Change a new entry 1 = Delete an existing entry Advice: Detailed information on this topic can be found above in the function description under the point "Application types". If the key is not passed, the value [0] is used by default!	☐ int
PRESETUP_DESC	Description of the pre-setup	☐ String
PRESETUP_TYPE	Pre-setup type [0; 1]: S = Pre-setup of type "Standard" (not portable) P = Pre-setup of type "Portable" Background: Portable pre-setups can be used successively on different stations!	☐ String

Tab. 23-68 "Key Value Pair": *presetupHeadUploadKeys*

Input values for editing consumers

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	String
BOM_VERSION	BOM version from the iTAC system Advice: This key is only evaluated if the WORKORDER_NUMBER is not used. If the key is required but not passed, the highest released version is always used!	int
BOM_VERSION_ERP	BOM version in the ERP system	String
CHANGE_MODE	Change mode [0; 1; 2]: 0 = Create a new entry 1 = Change an existing entry 2 = Delete an existing entry Advice: Detailed information on this topic can be found above in the function description under the point "Application types". If the key is not passed, the value [1] is used by default!	int
CONSUMER_ID	Internal identification number of the concrete usage record (consumer) for which the pre-setup was created. Advice: For the application types Delete and Change, this entry is obligatory; detailed information on this topic can be found above in the function description under the point "Application types"!	long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -632)!	int
MACHINE_GROUP_NUMBER	Number of the machine group for which the pre-setup is intended. Advice: Each consumer requires a unique machine group association; if the key is not passed, the machine group is ascertained on the basis of the station number (see key STATION_NUMBER)!	String
PART_NUMBER	Part number (iTAC system) of the product to be manufactured for which the pre-setup is intended. Advice: Even though this key is not identified as a mandatory field, the application type Create requires either WORKORDER_NUMBER or PART_NUMBER to be passed!	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: Each consumer requires a unique work step assignment; this key is evaluated if the key WORKSTEP_NUMBER is not passed. In this case, the work step is ascertained on the basis of the layer and current station!	int

Key	Explanation	IMSApi data type
PROCESS_VERSION	Process version from the iTAC system Advice: This key is only evaluated if the WORKORDER_NUMBER is not used. If the key is required but not specified, the highest released version is always used!	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system for which the pre-setup is intended. Advice: Each consumer requires a unique machine group association; if the key MACHINE_GROUP_NUMBER is not passed, the machine group is ascertained on the basis of the station number specified here. If this key is not specified either, the machine group according to the calling station is used (see stationNumber)!	☐ String
WORKORDER_NUMBER	Number of the production order (iTAC system) for which the pre-setup is intended. Advice: Even though this key is not identified as a mandatory field, the application type Create requires either WORKORDER_NUMBER or PART_NUMBER to be passed! If the order number is used, the keys PART_NUMBER, BOM_VERSION and PROCESS_VERSION are not interpreted.	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system for which the pre-setup is intended. Advice: Each consumer requires a unique work step assignment; if this key is not passed, the key PROCESS_LAYER must be used!	☐ int

Tab. 23-69 "Key Value Pair": presetupConsumerUploadKeys

Input values for editing pre-setup sections

Key	Explanation	IMSApi data type
CHANGE_MODE	Change mode [0; 1; 2]: 0 = Create a new entry 1 = Change an existing entry 2 = Delete an existing entry Advice: Detailed information on this topic can be found above in the function description under the point "Application types". If the key is not passed, the value [1] is used by default!	☐ int
EQUIPMENT_INDEX	Individual designator for the equipment (setup carrier)	☐ String
EQUIPMENT_NUMBER	Number of the equipment (setup carrier)	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general error message (see error code -632)!	☐ int
SECTION_DESC	Description of the pre-setup section	☐ String
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_NAME)!	☐ String

Key	Explanation	IMSApi data type
SECTION_ID	Internal identification number of the pre-setup section Advice: For the application types Delete and Change , this entry is obligatory; detailed information on this topic can be found above in the function description under the point "Application types"!	☐ long
SECTION_NAME	Name of the pre-setup section Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ String
SECTION_STATE	Pre-setup section state [C; F; I; R; S; SR]: C = CREATED -> pre-setup section created F = FINISHED -> pre-setup section finished I = IN_PROCESS -> pre-setup section partly prepared R = READY -> pre-setup section completely prepared S = SETUP -> pre-setup section completely set up SR = SETUP_REMOVED -> pre-setup section stripped down	☐ String
STATION_NUMBER	In the iTAC system, this is the station number of the work station for which this pre-setup section is intended. Background: Pre-setup sections can be permanently assigned to a specific station; this is only the case if the referenced pre-setup is NOT of the "portable" type (see key PRESETUP_TYPE in the "preset-upHeadResultKeys" table).	☐ String
STORAGE_NUMBER	Storage place number for the equipment (setup carrier) Advice: The storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String

Tab. 23-70 "Key Value Pair": *presetupSectionUploadKeys*

23.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
station number	Station number of the work station in the iTAC system
presetupHeadFilter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the desired pre-setup is selected; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupHeadUploadKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadUploadKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupConsumerUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presetupConsumerUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupConsumerUploadKeys table. Advice: Only the parameters passed here are processed by the function!
presetupConsumerUploadValues	Values for the parameters selected by means of <code>presetupConsumerUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupConsumerUploadKeys</code> ! Refer to the "Key Value Pair": presetupConsumerUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupSectionUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presetupSectionUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupSectionUploadKeys table. Advice: Only the parameters passed here are processed by the function!

Parameter name	Explanation
presetupSectionUploadValues	Values for the parameters selected by means of <code>presetupSectionUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupSectionUploadKeys</code>! Refer to the "Key Value Pair": <code>presetupSectionUploadKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 23-71 Parameter (in) »setupUpdatePreSetup«

Parameter (out)

Parameter name	Explanation
presetupConsumerResultValues	Depending on parameters (in) <code>presetupConsumerUploadKeys</code> and <code>presetupConsumerUploadValues</code>, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupConsumerUploadKeys</code>!
presetupSectionResultValues	Depending on parameters (in) <code>presetupSectionUploadKeys</code> and <code>presetupSectionUploadValues</code>, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupSectionUploadKeys</code>!

Tab. 23-72 Parameter (out) »setupUpdatePreSetup«

23.16.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-124	Failure	Part does not match the work order part
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-163	Failure	Invalid value for "bomType"
-196	Failure	A parameter is invalid
-242	Failure	BOM version is invalid
-528	Failure	Storage place was not found
-535	Failure	Storage place is not unique

-626	Failure	Pre-setup not found
-630	Failure	Pre-setup not created
-631	Failure	Pre-setup -> Invalid type
-632	Failure	Pre-setup -> Update failed
-639	Failure	Pre-setup -> Change not allowed
-1101	Failure	BOM not found
-1301	Failure	Production means not found
-2050	Failure	Machine group not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

23.17

setupUpdatePreSetupPosition

23.17.1

Function description

This function allows individual setup positions to be edited/updated for a certain pre-setup section of a pre-setup. Here, new positions can be created and also existing positions can be deleted. Apart from the structural adaptations, it is also possible to assign concrete container information to the existing setup positions. This process is also referred to as "setting up" and involves the assignment of a specific quantity of a specific material to a specific setup position on the setup carrier. Set-up setup positions can also be stripped down, i.e. the assigned containers can be removed.

If the header data of the pre-setup or information regarding consumer and/or pre-setup sections are to be changed, the API function »setupUpdatePreSetup« [▶▶▶] must be used.



Advice

The array returned contains all processed records; faulty records are returned with a corresponding error code (reason for error message) (see key `ERROR_CODE`). The function itself returns a warning (error code = 5) in the event of one or more faulty records!

Application types

A list of the typical application types with information on the procedure is given here in order to provide you with a clearer overview of the function range.

Application types "Editing of setup positions"

- **Create** -> Selection of the desired pre-setup using the "Key Value Pair" `preset-upHeadFilter`; the data of the new setup position is passed using `presetup-HeadUploadKeys`.

Condition: Key `CHANGE_MODE` = 0 and the specified `SECTION_POSITION` must be unique.

Note also the following basic principles:

- The selected pre-setup must NOT have the state **S** (SETUP) and **PS** (PARTLY_SETUP)
- When a new setup position is created, no container information has to be passed; to prepare the setup position, only the part number (`PART_NUMBER`) needs to be passed.
- If container data is passed for a new setup position, this data is set up directly.
- If a container number (`MATERIAL_BIN_NUMBER`) is specified without specification of a part number (`PART_NUMBER`), the material number of the container is stored for the setup position. If the position already exists, the record is updated.
- When a new setup position is created, the specified mounting places are assigned to the position; in this case, all mounting places for the position must also ALWAYS be specified.

- **Set up** -> Selection of the desired pre-setup using the "Key Value Pair" `preset-upHeadFilter`; the entry to be set up is passed using `SECTION_ID` or `SECTION_NAME` (pre-setup section) and `SECTION_POSITION` (position in the pre-setup section).

Condition: Key `CHANGE_MODE` = 1

Note also the following basic principles:

- The selected pre-setup must NOT have the state **S** (SETUP) and **PS** (PARTLY_SETUP)
- During setup, the container information is assigned to the specified setup position.
- If an unknown setup position is passed, the record is rejected.
- **Strip down** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the entry to be stripped down is passed using `SECTION_ID` or `SECTION_NAME` (pre-setup section) and `SECTION_POSITION` (position in the pre-setup section).
Condition: Key `CHANGE_MODE` = 2
Note also the following basic principles:
 - The selected pre-setup must NOT have the state **S** (SETUP) and **PS** (PARTLY_SETUP)
 - During strip-down, the container information for the specified setup position is removed.
 - The setup position with the corresponding part number is maintained after strip-down and therefore continues to indicate the target specification for this position.
 - **Special case:** If neither a pre-setup section nor a setup position is specified in this mode, all container information is removed from all setup positions.
- **Delete** -> Selection of the desired pre-setup using the "Key Value Pair" `presetupHeadFilter`; the entry to be deleted is passed using `SECTION_ID` or `SECTION_NAME` (pre-setup section) and `SECTION_POSITION` (position in the pre-setup section).
Condition: Key `CHANGE_MODE` = 3
Note also the following basic principles:
 - The selected pre-setup must NOT have the state **S** (SETUP) or **PS** (PARTLY_SETUP).
 - During the deletion process, the complete position with all mounting place information is removed from the pre-setup.

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```

int setupUpdatePreSetupPosition
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] presetupHeadFilter
    String key
    String value
in String[] presetupPositionUploadKeys
in String[] presetupPositionUploadValues
in String[] presetupPositionComponentUploadKeys
in String[] presetupPositionComponentUploadValues
out String[] presetupPositionResultValues
out String[] presetupPositionComponentResultValues

```


"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `presSetupHeadFilter`, `presSetupPositionUploadKeys`, `presSetupPositionUploadValues`, `presSetupPositionComponentUploadKeys` and `presSetupPositionComponentUploadValues` as well as (out) `presSetupPositionResultValues` and `presSetupPositionComponentResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
 If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for ascertaining desired pre-setup

Key	Explanation	IMSApi data type
PRESETUP_ID	Internal identification number of the pre-setup Advice: If this key is passed, the keys <code>PRESETUP_NAME</code> and <code>PRESETUP_INDEX</code> are ignored! If no identification number is passed, at least the key <code>PRESETUP_NAME</code> MUST be used!	☐ long
PRESETUP_INDEX	Individual designator for the pre-setup Background: The key <code>PRESETUP_INDEX</code> is required in combination with <code>PRESETUP_NAME</code> for unique identification of a pre-setup; i.e. the name of the pre-setup itself does not necessarily need to be unique! Advice: This key is only evaluated if <code>PRESETUP_ID</code> is not used! Special case: If this key is not passed along, the [1] value is used by default!	☐ int
PRESETUP_NAME	Name of the pre-setup Advice: This key is only evaluated if the key <code>PRESETUP_ID</code> is not used (for further information, see also the key <code>PRESETUP_INDEX</code>)!	☐ String

Tab. 23-73 "Key Value Pair": `presSetupHeadFilter`

Input values for editing setup positions

Key	Explanation	IMSApi data type
CHANGE_MODE	Change mode [0; 1; 2; 3]: 0 = Create a setup position 1 = Set up a setup position 2 = Strip down a setup position 3 = Delete a setup position Advice: Detailed information on this topic can be found above in the function description under the point "Application types"!	☒ int
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "presetupPositionComponentUploadKeys" table)!	☐ long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
MATERIAL_BIN_NUMBER	Number of the material bin (container no.) Advice: If the key PART_NUMBER is not passed, the part number is ascertained on the basis of the container number!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the container material!	☐ String
SECTION_DESTINATION	Target destination of the pre-setup section at the machine (e.g. identification code for right and left side) Advice: The use of "pre-setup sections" may be customer-specific (see "Background" for the key SECTION_NAME)!	☐ String
SECTION_ID	Internal identification number of the pre-setup section Advice: If this key is NOT passed, the key SECTION_NAME MUST be used! The specification of a specific pre-setup section is required for all application types; detailed information on this topic can be found above in the function description under the point "Applications types"!	☐ long
SECTION_NAME	Name of the pre-setup section Advice: This key is only evaluated if the key SECTION_ID is not used! Background: A pre-setup section (setup carrier) contains either the complete pre-setup or a certain part section; if the pre-setup is provided on multiple setup carriers (e.g. distributed over three setup carts), each setup cart can be seen as a separate pre-setup section. The key SECTION_DESTINATION then only becomes relevant if pre-setup sections of a pre-setup are to be placed at different positions of the machine.	☐ String
SECTION_POSITION	Position of the material carrier (e.g. feeder); the key SECTION_SUB_POSITION can be used to specify the position even more precisely.	☒ String
SECTION_SUB_POSITION	Position of the material within a material carrier (e.g. feeder with multiple mounts); this key is used to specify the position entered using the key SECTION_POSITION even more precisely.	☐ String

Tab. 23-74 "Key Value Pair": presetupPositionUploadKeys

Input values for editing mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects for the setup data record can be put into context (see parameter of the same name in the "presetupPositionUploadKeys" table)!	☒ long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
PANEL_POSITION	Panel position	☐ int
PRODUCT_NUMBER	Part number of the product according to consumer Background: In the context of "family setup", a part number can exist with various mounting places in different BOMs.	☐ String

Tab. 23-75 "Key Value Pair": *presetupPositionComponentUploadKeys*

23.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
presetupHeadFilter	"Key Value Pair" structure with the following variables
key	List of the parameters on the basis of which the desired pre-setup is selected; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupHeadFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": presetupHeadFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupPositionUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presetupPositionUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupPositionUploadKeys table. Advice: Only the parameters passed here are processed by the function!

Parameter name	Explanation
presetupPositionUploadValues	Values for the parameters selected by means of <code>presetupPositionUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupPositionUploadKeys</code>! Refer to the "Key Value Pair": presetupPositionUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
presetupPositionComponentUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>presetupPositionComponentUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": presetupPositionComponentUploadKeys table. Advice: Only the parameters passed here are processed by the function!
presetupPositionComponentUploadValues	Values for the parameters selected by means of <code>presetupPositionComponentUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupPositionComponentUploadKeys</code>! Refer to the "Key Value Pair": presetupPositionComponentUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 23-76 Parameter (in) »setupUpdatePreSetupPosition«

Parameter (out)

Parameter name	Explanation
presetupPositionResultValues	Depending on parameters (in) <code>presetupPositionUploadKeys</code> and <code>presetupPositionUploadValues</code>, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupPositionUploadKeys</code>!
presetupPositionComponentResultValues	Depending on parameters (in) <code>presetupPositionComponentUploadKeys</code> and <code>presetupPositionComponentUploadValues</code>, the corresponding data is output here. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>presetupPositionComponentUploadKeys</code>!

Tab. 23-77 Parameter (out) »setupUpdatePreSetupPosition«

23.17.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-196	Failure	A parameter is invalid
-501	Failure	At least one material bin not found
-550	Failure	Container number not unique
-552	Failure	Container is not permitted for part
-625	Failure	Pre-setup -> Invalid state
-626	Failure	Pre-setup not found
-627	Failure	Pre-setup -> Position not found

-633	Failure	Pre-setup already attached
-634	Failure	Pre-setup -> Position not unique
-635	Failure	Pre-setup -> Position already exists
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

24 Group "SHIP" (packing)

24.1 shipActivateShippingLotAtKap

24.1.1 Function description

This function is used to assign lots to a station (shipping station) (for further information, see also »shipDeactivateShippingLotAtKap« [▶▶▶]). A lot can only be assigned to one shipping station at any one time; a plausibility check is carried out before the assignment.



Advice

This API function is influenced by station parameter code no. 21020 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipActivateShippingLotAtKap
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
```

24.1.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot

Tab. 24-1 Parameter (in) »shipActivateShippingLotAtKap«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

24.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
407	Warning	Lot (parentLot) locked but usable
234	Warning	Serial number locked but usable
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-400	Failure	Lot not found
-432	Failure	Lot was not assigned
-453	Failure	No locked lots may be registered at this station.
-454	Failure	Only locked lots may be registered at this station.
-461	Failure	No locked lots or serial numbers may be processed at this station. However, the lot contains locked lots or serial numbers.
-462	Failure	Only locked lots or serial numbers may be processed at this station. However, the lot contains non-locked lots or serial numbers.
-2105	Failure	No TR or PM license present for the station

24.2 shipAddChildLotToParentLot

24.2.1 Function description

This function packs a lot (lotNumberChild) into another lot (lotNumberParent).



Advice

This API function is influenced by station parameter code nos. 21012 [▶▶▶] and 21020 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipAddChildLotToParentLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumberChild
in String lotNumberParent
in DateMs bookDate
```

24.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumberChild	Identification number of the (subordinate) lot Advice: A <code>lotNumberChild</code> is the lot that is or has been packed into a <code>lotNumberParent</code> !
lotNumberParent	Identification number of the (parent) lot Advice: A <code>lotNumberParent</code> is the container in which the <code>lotNumberChild</code> (ren) are or were packed!
bookDate	Booking time of the packing process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-2 Parameter (in) »shipAddChildLotToParentLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.2.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
408	Warning	Lot (childLot) locked but usable
407	Warning	Lot (parentLot) locked but usable
234	Warning	Serial number locked but usable
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-401	Failure	Lot (childLot) not found
-402	Failure	Lot (childLot) is not closed
-403	Failure	Serial numbers in the lot have a different part no.
-404	Failure	Serial numbers in the lot have a different BOM version
-405	Failure	Lot (childLot) is already assigned to the bin
-406	Failure	Lot (childLot) is already assigned to another bin
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-413	Failure	Lot (parentLot) not found
-419	Failure	Lot (childLot) is not unique
-422	Failure	Part no. is not configured for the bin

-457	Failure	No locked lots or serial numbers may be processed. The new lot is locked or contains locked items.
-459	Failure	Only locked lots or serial numbers may be processed. The new lot is not locked or contains non-locked items.
-463	Failure	No locked lots or serial numbers may be processed at this station. However, the new subordinate lot is locked or contains locked items.
-464	Failure	Only locked lots or serial numbers may be processed at this station. However, the new subordinate lot is not locked or contains non-locked items.
-2105	Failure	No TR or PM license present for the station

24.3

shipAddSerialNumberToShippingLot

24.3.1

Function description

This function is used to assign the serial number of a product to a lot at a shipping station; before the assignment is carried out, the validity of the product and of the serial number are checked.

The stored test routine does not cover any special application types – e.g. if an EOL tester and shipping station are mapped on a station. To ensure that the respective serial number really has the "PASS" state, the »trCheckSerialNumberState« [▶▶▶] IMSApi function should be executed before the packing process.



Advice

*This API function is influenced by station parameter code nos. 21010 [▶▶▶] and 21020 [▶▶▶] as well as site parameter code no. 6010 [▶▶▶]. By default, the system is configured such that only **one** product in one BOM version (pure with identical product version) may be packed into a lot. However, it is possible to reconfigure the system to permit different BOM versions of a product to be packed into one lot. The packing of different products into one lot (unpure) is not currently supported!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipAddSerialNumberToShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in String serialNumber
in int serialNumberPos
in DateMs bookDate
```

24.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
bookDate	Booking time of the packing process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-3 Parameter (in) »shipAddSerialNumberToShippingLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.3.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
406	Warning	First serial number
405	Warning	Last serial number
234	Warning	Serial number locked but usable
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-219	Failure	Serial number is not finalized
-222	Failure	The serial no. state is SCRAP on last work step
-229	Failure	Serial number not yet booked by the previous station
-400	Failure	Lot not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-421	Failure	Lot is not registered at the station

-422	Failure	Part no. is not configured for the bin
-423	Failure	Serial number is already assigned to another lot
-424	Failure	Serial number is already assigned to the lot
-425	Failure	Part no. is different from the serial numbers already included
-426	Failure	BOM version is different from the version of the serial numbers that is already included
-437	Failure	Package quantity already reached
-440	Failure	Work order is different from the work order of the serial numbers already included
-450	Failure	The lot attribute of the lot has a different part no. than the serial number
-451	Failure	Invalid value for the lot attribute "Part quantity". Value must be greater than 0.
-452	Failure	Invalid value for the lot attribute "Part name". Part must be present in the system.
-455	Failure	Serial number is locked. No locked serial numbers may be packed at this station.
-456	Failure	Serial number is not locked. Only locked serial numbers may be packed at this station.
-469	Failure	The serial number is already archived and cannot be added to the lot.
-2105	Failure	No TR or PM license present for the station

24.4

shipCheckSerialNumberAddToShippingLot

24.4.1

Function description

This function is used to check whether the assignment of a serial number to a lot is possible and whether the serial number is valid (PASS).

The stored test routine does not cover any special application types – e.g. if an EOL tester and shipping station are mapped on a station. To ensure that the respective serial number really has the "PASS" state, the »trCheckSerialNumberState« [▶▶▶] IMSApi function should be executed before the packing process.



Advice

*This API function is affected by station parameter code no. 21020 [▶▶▶] as well as by site parameter code no. 6010 [▶▶▶] and 6040 [▶▶▶]. By default, the system is configured such that only **one** product in one BOM version (pure with identical product version) may be packed into a lot. However, it is possible to reconfigure the system to permit different BOM versions of a product to be packed into one lot. The packing of different products into one lot (unpure) is not currently supported!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR PM license
--	-----------------------------------

Function declaration

```
int shipCheckSerialNumberAddToShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in String serialNumber
in String serialNumberPos
```

24.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!

Tab. 24-4 Parameter (in) »shipCheckSerialNumberAddToShippingLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
234	Warning	Serial number locked but usable
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-219	Failure	Serial number is not finalized
-222	Failure	The serial no. state is SCRAP on last work step
-229	Failure	Serial number not yet booked by the previous station
-400	Failure	Lot not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-421	Failure	Lot is not registered at the station
-422	Failure	Part no. is not configured for the bin

-423	Failure	Serial number is already assigned to another lot
-424	Failure	Serial number is already assigned to the lot
-425	Failure	Part no. is different from the serial numbers already included
-426	Failure	BOM version is different from the version of the serial numbers that is already included
-437	Failure	Package quantity already reached
-440	Failure	Work order is different from the work order of the serial numbers already included
-2100	Failure	Serial number is sent to SCT
-2105	Failure	No TR or PM license present for the station
-2138	Failure	Serial number is deleted

24.5 shipCheckSerialNumberFromShippingLot

24.5.1 Function description

This function is used to check whether a certain serial number is assigned to a certain lot and whether the assignment is still valid. A previously valid assignment could, for example, subsequently become invalid if the process state of the serial number changes from "Pass" to "Scrap" as the result of a test.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR PM license
--	-----------------------------------

Function declaration

```
int shipCheckSerialNumberFromShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in String serialNumber
in String serialNumberPos
```

24.5.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!

Tab. 24-5 Parameter (in) »shipCheckSerialNumberFromShippingLot«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

24.5.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-400	Failure	Lot not found
-418	Failure	Serial number was not found in the lot
-421	Failure	Lot is not registered at the station
-422	Failure	Part no. is not configured for the bin
-423	Failure	Serial number is already assigned to another lot
-427	Failure	Serial number was not assigned to a lot
-2105	Failure	No TR or PM license present for the station

24.6

shipCompleteLot

24.6.1

Function description

This function specifies the system behavior in the case of a lot whose maximum fill level has not been reached when it is closed.



Advice

This API function is influenced by station parameter code no. 21016 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	--

Function declaration

```
int shipCompleteLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in int allowLessQuantity
in DateMs bookDate
```

24.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
allowLessQuantity	"Max. fill level" mode for lots [0; 1]: 0 = If the maximum fill level is not reached, an error message is output if an attempt is made to close the lot. 1 = Even if the maximum fill level was not reached, lots can be closed (state: Complete). The system stores the information that this lot contains less than its maximum fill level despite having the "complete" state.
bookDate	Booking time of the closing operation (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-6 Parameter (in) »shipCompleteLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.6.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-400	Failure	Lot not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-421	Failure	Lot is not registered at the station
-429	Failure	Bin is not of type "lot"
-430	Failure	Lot has insufficient fill quantity
-436	Failure	Lot is not closed
-467	Failure	Closing failed -> The lot contains locked items; only lots with non-locked items can/may be closed at this station.
-468	Failure	Closing failed -> The lot contains non-locked items; only lots with locked items can/may be closed at this station.
-2105	Failure	No TR or PM license present for the station

24.7 shipDeactivateShippingLotAtKap

24.7.1 Function description

This function cancels the association between lot and the station (shipping station) (for further information, see also »shipActivateShippingLotAtKap« [▶▶▶]).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipDeactivateShippingLotAtKap
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
```

24.7.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot
	Special case: If the value [-1] is passed here, the currently assigned lot is removed from the station!

Tab. 24-7 Parameter (in) »shipDeactivateShippingLotAtKap«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

24.7.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-400	Failure	Lot not found
-2105	Failure	No TR or PM license present for the station

24.8 shipGetChildLotsForParentLot

24.8.1 Function description

This function returns all lots (ChildLots) that are packed into a parent lot (lotNumberParent).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int shipGetChildLotsForParentLot
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String lotNumberParent
  in String[] childLotResultKeys
  out String[] childLotResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) childLotResultKeys and (out) childLotResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐	String
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐	String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐	int
MATERIAL_BIN_NUMBER	Identification number of the lot (Lot no.)	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current fill quantity for the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_QTY_TOTAL	Maximum fill quantity in the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_STATE	Lot state [B; C; E; F; L; P; Q; R; S; V]: B = Locked (lot locked) C = Closed (lot finalized) E = Completed (lot completed) F = Initialized (new lot created) L = MRP locked (lot locked against allocation) P = Prepared (lot prepared) Q = QA locked (lot with QA lock) R = Ready (lot ready) S = In process (lot in use) V = Pre-release (lot with pre-release)	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the subordinate (packed) lot!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the subordinate (packed) lot!	☐	String

Key	Explanation	IMSApi data type
QUANTITY_UNIT	Designator for the quantity unit in the lot: 11000 = (mm) millimeter 11001 = (") inch 11002 = (stk) piece 11003 = (cm) centimeter 11004 = (m) meter 11005 = (g) gram 11006 = (kg) kilogram 11007 = (ml) milliliter 11008 = (l) liter Advice: <i>If necessary, quantity units may be adapted and amended in the system (details upon request)!</i>	☐ long
SHIPPING_LOT_NUMBER	Identification number of the lot (Lot no.)	☐ String
SHIPPING_LOT_NUMBER2	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐ String
SHIP-PING_LOT_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the parent lot!</i>	☐ String
SHIP-PING_LOT_SNO_PART_DESCRIPTION	Part description in the iTAC system Advice: <i>In this case, this is the part description of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_QTY_ACTUAL	Current fill quantity for the lot; here, serial number in the lot	☐ double
SHIP-PING_LOT_SNO_QTY_TOTAL	Maximum fill quantity in the lot; here, serial number in the lot	☐ double
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 24-8 "Key Value Pair": *childLotResultKeys*

24.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumberParent	Identification number of the parent lot; all containers packed therein are output Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
childLotResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": childLotResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) childLotResultValues !

Tab. 24-9 Parameter (in) »shipGetChildLotsForParentLot«

Parameter (out)

Parameter name	Explanation
childLotResultValues	Depending on parameter (in) childLotResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) childLotResultKeys !

Tab. 24-10 Parameter (out) »shipGetChildLotsForParentLot«

24.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-413	Failure	Lot (parentLot) not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.9

shipGetLotFromSerialNumber

24.9.1

Function description

This function returns information about the lot containing the serial number passed.



Advice

Use the [shipGetSerialNumberDataForShippingLot \[▶▶▶\]](#) function to output all serial numbers that are packaged in the lot returned here!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipGetLotFromSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in String[] lotResultKeys
out String[] lotResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) lotResultKeys and (out) lotResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		MSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐	String
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐	String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐	int
MATERIAL_BIN_NUMBER	Identification number of the lot (Lot no.)	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current fill quantity for the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_QTY_TOTAL	Maximum fill quantity in the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_STATE	Lot state [B; C; E; F; L; P; Q; R; S; V]: B = Locked (lot locked) C = Closed (lot finalized) E = Completed (lot completed) F = Initialized (new lot created) L = MRP locked (lot locked against allocation) P = Prepared (lot prepared) Q = QA locked (lot with QA lock) R = Ready (lot ready) S = In process (lot in use) V = Pre-release (lot with pre-release)	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the subordinate (packed) lot!	☐	String
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the subordinate (packed) lot!	☐	String

Key	Explanation	IMSApi data type
QUANTITY_UNIT	Designator for the quantity unit in the lot: 11000 = (mm) millimeter 11001 = (") inch 11002 = (stk) piece 11003 = (cm) centimeter 11004 = (m) meter 11005 = (g) gram 11006 = (kg) kilogram 11007 = (ml) milliliter 11008 = (l) liter Advice: <i>If necessary, quantity units may be adapted and amended in the system (details upon request)!</i>	☐ long
SHIPPING_LOT_NUMBER	Identification number of the lot (Lot no.)	☐ String
SHIPPING_LOT_NUMBER2	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐ String
SHIP-PING_LOT_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the parent lot!</i>	☐ String
SHIP-PING_LOT_SNO_PART_DESCRIPTION	Part description in the iTAC system Advice: <i>In this case, this is the part description of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_QTY_ACTUAL	Current fill quantity for the lot; here, serial number in the lot	☐ double
SHIP-PING_LOT_SNO_QTY_TOTAL	Maximum fill quantity in the lot; here, serial number in the bin	☐ double
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 24-11 "Key Value Pair": *lotResultKeys*

24.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
lotResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": lotResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) lotResultValues !

Tab. 24-12 Parameter (in) »shipGetLotFromSerialNumber«

Parameter (out)

Parameter name	Explanation
lotResultValues	Depending on parameter (in) lotResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) lotResultKeys !

Tab. 24-13 Parameter (out) »shipGetLotFromSerialNumber«

24.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-400	Failure	Lot not found
-418	Failure	Serial number was not found in the lot
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.10 shipGetSerialNumberDataForShippingLot

24.10.1 Function description

This function retrieves the serial numbers of products/containers that have been packed into a certain lot.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```

int shipGetSerialNumberDataForShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in String[] serialNumberResultKeys
out String[] serialNumberResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberResultKeys and (out) serialNumberResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
BATCH_QUANTITY_CURRENT	Current quantity in the material bin Advice: A value is only output here if the key SERIAL_NUMBER is a container (cf. SERIAL_NUMBER_TYPE "B")!	☐ double
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
LOCK_CLASS	Name of the lock class	☐ String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐ int
LOCKGROUP_NAME	Name of the lock group	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: "Normal" serial numbers do not have a special type; an empty string is output in this case!	☐ char

Key	Explanation	IMSApi data type
SHIPPING_DATE	Booking time of the packing process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SHIPPING_STATION_DESC	Station description of the shipping station in the iTAC system	☐ String
SHIPPING_STATION_NUMBER	Station no. of the shipping station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 24-14 "Key Value Pair": *serialNumberResultKeys*

24.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
serialNumberResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>serialNumberResultValues</i> !

Tab. 24-15 Parameter (in) »shipGetSerialNumberDataForShippingLot«

Parameter (out)

Parameter name	Explanation
serialNumberResultValues	Depending on parameter (in) <i>serialNumberResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>serialNumberResultKeys</i> !

Tab. 24-16 Parameter (out) »shipGetSerialNumberDataForShippingLot«

24.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-400	Failure	Lot not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.11 shipGetShippingLotInfo

24.11.1 Function description

This function ascertains and outputs lots on the basis of various filter criteria. This can be used, for example, to search for lots in which specific parts are packed or that were used on the basis of a specific work order. The data is output in a record separately for each found lot (i.e. only **one** OUT array for each lot).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipGetShippingLotInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] shippingLotFilters
    String key
    String value
in String[] shippingLotResultKeys
out String[] shippingLotResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `shippingLotFilters` and `shippingLotResultKeys` as well as parameter (out) `shippingLotResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CUSTOMER_CODE	Customer identification (code) in the iTAC system; this is the customer to whom the lot was shipped	☐	String
DELIVERY_NOTE	Delivery slip number	☐	String
FUNC_MODE	Function mode [0; 1]: 0 = Default mode (or empty string or [-1]) 1 = Active lot at the station	☐	int
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐	String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐	int
MATERIAL_BIN_STATE	Lot state [B; C; E; F; L; P; Q; R; S; V]: B = Locked (lot locked) C = Closed (lot finalized) E = Completed (lot completed) F = Initialized (new lot created) L = MRP locked (lot locked against allocation) P = Prepared (lot prepared) Q = QA locked (lot with QA lock) R = Ready (lot ready) S = In process (lot in use) V = Pre-release (lot with pre-release) Advice: To filter by multiple entries (OR link), the desired values must be passed separated by a comma (e.g.: "E,R,S")!	☐	String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: If more records exist than are defined here, the function outputs a warning (see error code 3)!	☐	int
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the packed serial numbers! When entering the part number to search for, a part schema (part number pattern) may be used instead. By using the wildcard character "*" (e.g., 123*), it is also possible to retrieve the lots of several parts in one step; pay attention to upper- and lower-case letters!	☐	String

Key	Explanation	IMSApi data type
SHIPPING_LOT_NUMBER	Identification number of the lot (Lot no.) Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
SHIP-PING_LOT_PART_NUMBER	Part number of the lot according to the part master Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 24-17 "Key Value Pair": *shippingLotFilters*

Definition of the output parameters for the found lots

Key	Explanation	IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐ String
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐ String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐ int
DELIVERY_NOTE	Delivery slip number	☐ String
MATERIAL_BIN_NUMBER	Identification number of the lot (Lot no.)	☐ String
MATERIAL_BIN_QTY_ACTUAL	Current fill quantity for the lot; here lot in lot (e.g. boxes in a crate)	☐ double
MATERIAL_BIN_QTY_TOTAL	Maximum fill quantity in the lot; here lot in lot (e.g. boxes in a crate)	☐ double

Key	Explanation	IMSApi data type
MATERIAL_BIN_STATE	Lot state [B; C; E; F; L; P; Q; R; S; V]: B = Locked (lot locked) C = Closed (lot finalized) E = Completed (lot completed) F = Initialized (new lot created) L = MRP locked (lot locked against allocation) P = Prepared (lot prepared) Q = QA locked (lot with QA lock) R = Ready (lot ready) S = In process (lot in use) V = Pre-release (lot with pre-release)	String
PART_DESC	Part description in the iTAC system Advice: <i>In this case, this is the part description of the lot!</i>	String
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the packed serial numbers!</i>	String
QUANTITY_UNIT	Designator for the quantity unit in the lot: 11000 = (mm) millimeter 11001 = (") inch 11002 = (stk) piece 11003 = (cm) centimeter 11004 = (m) meter 11005 = (g) gram 11006 = (kg) kilogram 11007 = (ml) milliliter 11008 = (l) liter Advice: <i>If necessary, quantity units may be adapted and amended in the system (details upon request)!</i>	long
SHIPPING_COMPLETE_DATE	Point in time at which the lot was "closed"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
SHIP-PING_LOT_LOT_QTY_ACTUAL	Current fill quantity for the lot; here, container in the lot Advice: <i>Here, the term "container" does not refer to a "normal" lot, but instead to a container associated with "bin-based traceability" (BATCH)!</i>	double
SHIPPING_LOT_NUMBER	Identification number of the lot (Lot no.)	String
SHIPPING_LOT_NUMBER2	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	String
SHIP-PING_LOT_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the lot!</i>	String
SHIP-PING_LOT_SNO_PART_DESCRIPTION	Part description in the iTAC system Advice: <i>In this case, this is the part description of the packed serial numbers!</i>	String
SHIP-PING_LOT_SNO_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the packed serial numbers!</i>	String

Key	Explanation		IMSApi data type
SHIP-PING_LOT_SNO_QTY_ACTUAL	Current fill quantity for the lot; here, serial number in the lot	☐	double
SHIP-PING_LOT_SNO_QTY_TOTAL	Maximum fill quantity in the lot; here, serial number in the lot	☐	double
SHIPPING_SEND_DATE	Point in time at which the lot was "shipped"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐	DateMs
SUPPLIER_NAME	Manufacturer name	☐	String
SUPPLIER_NUMBER	Manufacturer number	☐	String
WORKORDER_NUMBER	Production order number Advice: Output of the order number is only possible if the lot is pure, i.e. if only serial numbers of one work order were packed in the lot!	☐	String

Tab. 24-18 "Key Value Pair": *shippingLotResultKeys*

24.11.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
shippingLotFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": shippingLotFilters table.
value	Value list for the parameters passed by means of <code>key</code> Advice: Refer to the "Key Value Pair": shippingLotFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
shippingLotResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": shippingLotResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>shippingLotResultValues</code> !

Tab. 24-19 Parameter (in) »shipGetShippingLotInfo«

Parameter (out)

Parameter name	Explanation
shippingLotResultValues	Depending on parameter (in) <code>shippingLotResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>shippingLotResultKeys</code> !

Tab. 24-20 Parameter (out) »shipGetShippingLotInfo«

24.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-421	Failure	Lot is not registered at the station
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.12

shipGetShippingPrefs

24.12.1

Function description

This function provides information on which parts (serial numbers) or lots may be packed in which quantity, in adherence to which lot specification, in which (parent) lot. These specifications are stored in the master data under the term "part assignment".

The current assignment stored in the system can be ascertained based on a serial number, an order number, a part number or a lot number (here, that must be a subordinate bin).



Advice

This API function is influenced by server parameter 14600 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipGetShippingPrefs
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int objectType
in String objectNumber
in String objectDetail
in int withPackagingRule
in String[] shippingPrefsResultKeys
out String[] shippingPrefsResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) shippingPrefsResultKeys and (out) shippingPrefsResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐.

Serial number (objectType = 0)

Key	Explanation	IMSApi data type
CHECK_MODE	Mode of the lot check [0; 1; 2]: 0 = Check part number 1 = Check part number and BOM version 2 = Check part number, BOM version and order For the packaging of "bin in bin" (here, the contents of the bins are analyzed), only the check modes [0] and [1] are available; there are no restrictions on the packaging of "serial numbers in bins". Advice: Exactly which of the stated degrees of freedom is to apply to the packing is specified in the part master data of the respective lot!	int
PACK_NO	Lot specification	String
PACK_SPEC	Lot specification description; this must be stored in the system in the form of work order attributes Advice: Only one value is output here if the "Check lot specification" function is active (see withPackagingRule parameter)!	String
PART_MASTER_DESC	Part description of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	String
PART_MASTER_NUMBER	Part number of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	String
PART_SLAVE_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the serial numbers which are to be packed!	String
PART_SLAVE_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the serial numbers which are to be packed!	String
QUANTITY	Maximum package quantity	double

Tab. 24-21 "Key Value Pair" (serial number): shippingPrefsResultKeys**Lot number** (objectType = 1)

Key	Explanation	IMSApi data type
CHECK_MODE	Mode of the lot check [0; 1; 2]: 0 = Check part number 1 = Check part number and BOM version 2 = Check part number, BOM version and order For the packaging of "bin in bin" (here, the contents of the bins are analyzed), only the check modes [0] and [1] are available; there are no restrictions on the packaging of "serial numbers in bins". Advice: Exactly which of the stated degrees of freedom is to apply to the packing is specified in the part master data of the respective lot!	int
PACK_NO	Lot specification	String

Key	Explanation	IMSApi data type
PACK_SPEC	Lot specification description; this must be stored in the system in the form of work order attributes Advice: Only one value is output here if the "Check lot specification" function is active (see withPackagingRule parameter)!	☐ String
PART_MASTER_DESC	Part description of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	☐ String
PART_MASTER_NUMBER	Part number of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	☐ String
PART_SLAVE_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the subordinate lot (container in container)!	☐ String
PART_SLAVE_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the subordinate lot (container in container)!	☐ String
QUANTITY	Maximum package quantity	☐ double

Tab. 24-22 "Key Value Pair" (lot number): *shippingPrefsResultKeys*

Part number (objectType = 2)

Key	Explanation	IMSApi data type
CHECK_MODE	Mode of the lot check [0; 1; 2]: 0 = Check part number 1 = Check part number and BOM version 2 = Check part number, BOM version and order For the packaging of "bin in bin" (here, the contents of the bins are analyzed), only the check modes [0] and [1] are available; there are no restrictions on the packaging of "serial numbers in bins". Advice: Exactly which of the stated degrees of freedom is to apply to the packing is specified in the part master data of the respective lot!	☐ int
PACK_NO	Lot specification	☐ String
PACK_SPEC	Lot specification description; this must be stored in the system in the form of work order attributes Advice: Only one value is output here if the "Check lot specification" function is active (see withPackagingRule parameter)!	☐ String
PART_MASTER_DESC	Part description of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	☐ String
PART_MASTER_NUMBER	Part number of the lot according to the part master Advice: Here, this is the parent lot in which packing is performed!	☐ String
PART_SLAVE_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the serial numbers which are to be packed!	☐ String

Key	Explanation	IMSApi data type
PART_SLAVE_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the serial numbers which are to be packed!</i>	☐ String
QUANTITY	Maximum package quantity	☐ double

Tab. 24-23 "Key Value Pair" (part number): *shippingPrefsResultKeys*

Order number (objectType = 3)

Key	Explanation	IMSApi data type
CHECK_MODE	Mode of the lot check [0; 1; 2]: 0 = Check part number 1 = Check part number and BOM version 2 = Check part number, BOM version and order For the packaging of "bin in bin" (here, the contents of the bins are analyzed), only the check modes [0] and [1] are available; there are no restrictions on the packaging of "serial numbers in bins". Advice: <i>Exactly which of the stated degrees of freedom is to apply to the packing is specified in the part master data of the respective lot!</i>	☐ int
PACK_NO	Lot specification	☐ String
PACK_SPEC	Lot specification description; this must be stored in the system in the form of work order attributes Advice: <i>Only one value is output here if the "Check lot specification" function is active (see withPackagingRule parameter)!</i>	☐ String
PART_MASTER_DESC	Part description of the lot according to the part master Advice: <i>Here, this is the parent lot in which packing is performed!</i>	☐ String
PART_MASTER_NUMBER	Part number of the lot according to the part master Advice: <i>Here, this is the parent lot in which packing is performed!</i>	☐ String
PART_SLAVE_DESC	Part description in the iTAC system Advice: <i>This is the part description as per the production order, and with it the serial numbers to be packed!</i>	☐ String
PART_SLAVE_NUMBER	Part number in the iTAC system Advice: <i>This is the part number as per the production order, and with it the serial numbers to be packed!</i>	☐ String
QUANTITY	Maximum package quantity	☐ double

Tab. 24-24 "Key Value Pair" (order): *shippingPrefsResultKeys*

24.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
objectType	Determines the object type on the basis of which the function is to be executed [0; 1; 2; 3]: 0 = serial number 1 = lot number 2 = part number (of the serial number) 3 = order number Advice: If the object type "lot number" is passed, it must have a parent bin structure (bin in bin)!
objectNumber	Serial number, lot number, part number or order number Advice: This value is dependent on the selected object type (cf. parameter objectType)!
objectDetail	Serial number position Advice: Field is only evaluated if serial numbers are to be processed (objectType = 0)!
withPackagingRule	Check lot specification [0; 1]: 0 = without checking 1 = with checking If the check is active, only those bins are output for which a valid lot specification has been stored. Advice: This field is only evaluated if serial numbers or order numbers are to be processed (objectType = 0 or 3)! Background: The lot specification stored in the part master is checked against the lot specification description; the lot specification description itself is stored in the system as a work order attribute and can be assigned to the desired orders (for further information, see also the server parameter code no. 14600 [▶▶▶]).
shippingPrefsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters (depending on the objectType) can be found above in the corresponding "Key Value Pair" (xxx): shippingPrefsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) shippingPrefsResultValues!

Tab. 24-25 Parameter (in) »shipGetShippingPrefs«

Parameter (out)

Parameter name	Explanation
shippingPrefsResultValues	Depending on parameter (in) shippingPrefsResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) shippingPrefsResultKeys!

Tab. 24-26 Parameter (out) »shipGetShippingPrefs«

24.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-400	Failure	Lot not found
-718	Failure	Object type invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.13

shipMoveChildLots

24.13.1

Function description

This function moves the lots passed (`childLotArray`) from one lot (`actual-ParentLot`) to another lot (`newParentLot`).



Advice

This API function is influenced by station parameter code no. 21012 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipMoveChildLots
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in StringStruct[] childLotArray
    String value
  in String actualParentLot
  in String newParentLot
  in DateMs bookDate
  out ResultData[] resultDataArray
    String name
    String value
  int failCode
```

24.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
childLotArray	Array of the structure with the following variables:
value	Identification number of the (subordinate) lot Special case: If an empty array or the value [-1] is passed, all bins from the current lot are moved into the new lot!
actualParentLot	Identification number of the current lot from which packed bins are to be moved Special case: If an empty array or the value [-1] is passed, the lot currently registered at the station is accepted!
newParentLot	Identification number of the new lot into which the selected bins are to be moved
bookDate	Booking time of the bin repacking (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-27 Parameter (in) »shipMoveChildLots«

Parameter (out)

Parameter name	Explanation
resultDataArray	Array of the structure with the following variables:
value	Identification number of the (subordinate) lot
name	Unique designator for the array element: "childLots[x]"; x represents a sequential number beginning with 0
failCode	Error code for the case of incorrect processing of the respective record; for identification, see name parameter. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 404)!

Tab. 24-28 Parameter (out) »shipMoveChildLots«

24.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
408	Warning	Lot (childLot) locked but usable
407	Warning	Lot (parentLot) locked but usable
404	Warning	One or more subordinate lots were not found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-401	Failure	Lot (childLot) not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-413	Failure	Lot (parentLot) not found
-416	Failure	New lot (parentLot) not found
-457	Failure	No locked lots or serial numbers may be processed. The new lot is locked or contains locked items.
-458	Failure	No locked lots or serial numbers may be processed. The lot is locked or contains locked items.
-459	Failure	Only locked lots or serial numbers may be processed. The new lot is not locked or contains non-locked items.
-464	Failure	Only locked lots or serial numbers may be processed at this station. However, the new subordinate lot is not locked or contains non-locked items.
-2105	Failure	No TR or PM license present for the station

24.14**shipRemoveChildLots**

24.14.1**Function description**

This function removes one or more lots (`lotNumbersChild`) from a parent bin (`lotNumberParent`).

**Advice**

This API function is influenced by station parameter code no. 21012 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	--

Function declaration

```
int shipRemoveChildLots
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumberParent
in String[] lotNumbersChild
in DateMs bookDate
out ResultData[] resultArray
    String name
    String value
    int failCode
```

24.14.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumberParent	Identification number of the (parent) lot from which the specified bins are to be removed
lotNumbersChild	Identification number of the (subordinate) lot Special case: If an empty array or the value [-1] is passed, all bins are removed from the current lot (unpacked)!
bookDate	Booking time of the bin removal (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-29 Parameter (in) »shipRemoveChildLots«

Parameter (out)

Parameter name	Explanation
resultArray	Array of the structure with the following variables:
value	Identification number of the (subordinate) lot
name	Unique designator for the array element: "childLots[x]"; x represents a sequential number beginning with 0
failCode	Error code for the case of incorrect processing of the respective record; for identification, see name parameter. Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 404)!

Tab. 24-30 Parameter (out) »shipRemoveChildLots«

24.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
404	Warning	One or more subordinate lots were not found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-401	Failure	Lot (childLot) not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-413	Failure	Lot (parentLot) not found
-438	Failure	Lot is not contained in the parent bin (parentLot)
-439	Failure	Lot is already empty
-2105	Failure	No TR or PM license present for the station

24.15

shipRemoveSerialNumberFromShippingLot

24.15.1

Function description

This function is used to remove a serial no. from a lot; the fill level (number of serial numbers in the lot) is reduced accordingly. If the lot has a current fill level of "0", the state of the container is reset to "init" and is thus freely available again.



Advice

This API function is influenced by station parameter code no. 21010 [▶▶▶] as well as by site parameter code no. 18401 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipRemoveSerialNumberFromShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in String serialNumber
in String serialNumberPos
in DateMs bookDate
```

24.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
bookDate	Booking time of the unpacking process (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 24-31 Parameter (in) »shipRemoveSerialNumberFromShippingLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.15.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-400	Failure	Lot not found
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-418	Failure	Serial number was not found in the lot
-2105	Failure	No TR or PM license present for the station

24.16

shipReuseCompletedShippingLot

24.16.1

Function description

This function is used to "reactivate" closed lots for editing; the following two function modes are available here:

- In **reinitialization** mode, a completely packed lot (state **C** = closed) is completely finalized (state **E** = completed) and a new, empty lot is created in the system with the same identification number (state **F** = initialized). The requirement for finalizing the lot is that filling is finished and the lot is not registered at any other station. However, the lot does not need to be registered with the current station.
- In **reopen** mode, a completely packed lot (state **C** = closed) can be opened again for processing (state **R** = ready); for example, the »[shipRemoveSerialNumber-FromShippingLot](#)« [▶▶▶] function could then be used to remove a serial number from the lot.

Independent of mode, it is possible to select which detail information of the "reactivated" lot is output.

**Advice**

This API function is influenced by station parameter code no. 21016 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipReuseCompletedShippingLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int functionMode
in String lotNumber
in String[] shippingLotResultKeys
out String[] shippingLotResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `shippingLotResultKeys` and (out) `shippingLotResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CUSTOMER_CODE	Customer identification (code) in the iTAC system	☐	String
HU_NUMBER	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐	String
LOCK_STATE	Lock state of the lot [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meanwhile	☐	int
MATERIAL_BIN_NUMBER	Identification number of the lot (Lot no.) Advice: In this case, this is the lot no. of the parent lot!	☐	String
MATERIAL_BIN_NUMBER2	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (MATERIAL_BIN_NUMBER)	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current fill quantity for the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_QTY_TOTAL	Maximum fill quantity in the lot; here lot in lot (e.g. boxes in a crate)	☐	double
MATERIAL_BIN_STATE	Lot state [B; C; E; F; L; P; Q; R; S; V]: B = Locked (lot locked) C = Closed (lot finalized) E = Completed (lot completed) F = Initialized (new lot created) L = MRP locked (lot locked against allocation) P = Prepared (lot prepared) Q = QA locked (lot with QA lock) R = Ready (lot ready) S = In process (lot in use) V = Pre-release (lot with pre-release)	☐	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the subordinate (packed) lot!	☐	String

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the subordinate (packed) lot!</i>	☐ String
QUANTITY_UNIT	Designator for the quantity unit in the lot: 11000 = (mm) millimeter 11001 = (") inch 11002 = (stk) piece 11003 = (cm) centimeter 11004 = (m) meter 11005 = (g) gram 11006 = (kg) kilogram 11007 = (ml) milliliter 11008 = (l) liter Advice: <i>If necessary, quantity units may be adapted and amended in the system (details upon request)!</i>	☐ long
SHIPPING_LOT_NUMBER	Identification number of the lot (Lot no.) Advice: <i>In this case, this is the lot no. of the parent lot!</i>	☐ String
SHIPPING_LOT_NUMBER2	Serial number for the lot (HU = Handling Unit), which can be assigned to the lot in addition to the lot no. (SHIPPING_LOT_NUMBER)	☐ String
SHIP-PING_LOT_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the parent lot!</i>	☐ String
SHIP-PING_LOT_SNO_PART_DESCRIPTION	Part description in the iTAC system Advice: <i>In this case, this is the part description of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the packed serial numbers!</i>	☐ String
SHIP-PING_LOT_SNO_QTY_ACTUAL	Current fill quantity for the lot; here, serial number in the lot	☐ double
SHIP-PING_LOT_SNO_QTY_TOTAL	Maximum fill quantity in the lot; here, serial number in the lot	☐ double
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 24-32 "Key Value Pair": *shippingLotResultKeys*

24.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
functionMode	Function mode [0; 1]: 0 = Reinitialization 1 = Reopen Advice: Details on the two functions can be found above in the function description!
lotNumber	Identification number of the lot Advice: If the value [-1] is passed, the lot (identification number) to which the station is currently assigned is automatically accepted!
shippingLotResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": shippingLotResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) shippingLotResultValues !

Tab. 24-33 Parameter (in) »shipReuseCompletedShippingLot«

Parameter (out)

Parameter name	Explanation
shippingLotResultValues	Depending on parameter (in) shippingLotResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) shippingLotResultKeys !

Tab. 24-34 Parameter (out) »shipReuseCompletedShippingLot«

24.16.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-400	Failure	Lot not found
-412	Failure	Lot is activated at another station
-436	Failure	Lot is not closed
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

24.17

shipSendLot

24.17.1

Function description

This function identifies a lot with all containers and serial numbers contained therein as shipped by assigning it a shipping date. Lots can only be shipped if they have already been closed. In addition to the shipping date, delivery slip number and recipient can optionally be passed.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive) OR PM license
--	---

Function declaration

```
int shipSendLot
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String lotNumber
in DateMs bookDate
in KeyValue[] shippingLotSendValues
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `shippingLotSendValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
CUSTOMER_CODE	Customer identification (code) in the iTAC system; this is the customer to whom the lot is to be shipped	☐	String
DELIVERY_NOTE	Delivery slip number	☐	String

Tab. 24-35 "Key Value Pair": *resultKeys*

24.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lotNumber	Identification number of the lot Advice: A lot that is to be shipped must have the "closed" state!
bookDate	Time of shipping; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
shippingLotSend-Values	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the " Key Value Pair ": shippingLotSendValues table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the " Key Value Pair ": shippingLotSendValues table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 24-36 Parameter (in) »shipSendLot«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

24.17.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-400	Failure	Lot not found
-441	Failure	Lot is already sent, processing is not permitted
-465	Failure	Shipping failed -> The lot contains locked items; only non-locked items can/may be shipped at this station.
-466	Failure	Shipping failed -> The lot contains non-locked items; only locked items can/may be shipped at this station.
-2014	Failure	Customer not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2105	Failure	No TR or PM license present for the station

25

Group "SMT" (placement data)

25.1

smtCheckMaterialBin

25.1.1

Function description

This API function is used by the API-based "SMT connector" service. The function is normally used to check containers or to prepare them for production. The MSL information is of particular importance here; depending on the configuration, it is possible to start the MSL expiry time for the container concerned automatically. If required, the function can also be used to create unknown containers or to make quantity changes.



Advice

*This API function is influenced by station parameter code no. 13125 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! If the `CREATE_MATERIAL_BIN` key is passed with [0], the values of the selected keys are queried and – enhanced with the information from the iTAC system – are output (**GET**)!*

*If the `CREATE_MATERIAL_BIN` key is passed with [1], the values of the selected keys are transferred to the system (**UPLOAD**); in this case, incorrectly processed records are returned with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active)
AND	ML service (site license)

Function declaration

```
int smtCheckMaterialBin
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] materialBinUploadKeys
in String[] materialBinUploadValues
out String[] materialBinResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `materialBinUploadKeys` and `materialBinUploadValues` as well as (out) `materialBinResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)



Advice

Always take the selected function mode into consideration when interpreting the following table. If the key `CREATE_MATERIAL_BIN` is passed with [1], the values for the selected keys are passed to the system (UPLOAD). If the key `CREATE_MATERIAL_BIN` is passed with [0], the values for the selected keys are queried and output (GET)!

Key	Explanation	IMSApi data type
APS_TRANSFER	Indicates whether the container – more precisely the current container quantity of the part – is to be taken into consideration by the data exchange process of the APS service (work order detailed planning) [0;1]: 0 = Container is NOT taken into consideration (FALSE) 1 = Container is taken into consideration and is therefore "APS-relevant" (TRUE) Background: This specification is evaluated if, during work order detailed planning, the material availability is also to be taken into consideration for the parts required for work order production. If the container is "APS-relevant", the current container quantity of this part is taken into consideration during material availability planning.	☐ boolean
CLASSIFICATION	Classification of the container	☐ String
CREATE_MATERIAL_BIN	This parameter specifies whether an unknown container can be created [0;1]: 1 = Unknown containers are created in the iTAC system provided that the machine provides all data necessary for this -> this includes container and material numbers as well as container quantity (FALSE). 0 = Unknown containers are NOT created (TRUE) If containers are to be created, a default value can also be specified for the container quantity; i.e. even if only container and material numbers are provided, containers can be created. The default value is defined via the SMT interface using the <code>default container quantity</code> parameter. Advice: Note that in the returned function results (see <code>material-BinResultValues</code>) the key <code>CREATE_MATERIAL_BIN</code> is only returned with [1] if a container has been successfully created! If, for example, an attempt is made to create a container that already exists in the system, the function returns the value [0]!	☐ boolean

Key	Explanation		IMSApi data type
DATE_CHANGED	Change date: all containers are ascertained that were created new since this point in time or for which the container state or original container quantity was changed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
DATE_CREATED	Date of creation; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
EXPIRATION_DATE	Point in time from which the container must no longer be used; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
EXPIRATION_DATE_FINAL	Point in time from which the container is definitively no longer allowed to be used (MSL processing); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
HU_NUMBER	Serial number for the lot (HU = Handling Unit) Advice: This key is only relevant if the material bin is a lot!	☐	String
LOCK_STATE	Lock state of the material bin [-1; 0; 1]: -1 = Currently locked 0 = No lock 1 = Was locked in the meantime	☐	int
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐	String
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒	String
MATERIAL_BIN_PART_NUMBER	Part number of the container material in the iTAC system	☐	String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐	double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐	double

Key	Explanation	IMSApi data type
MATERIAL_BIN_STATE	Container state [B; E; F; L; P; Q; R; S; U; V]: B = Locked (material bin locked) E = Finished (material bin finished) F = Initialized (new material bin created) L = MRP locked (material bin locked against allocation) P = Prepared (material bin prepared) Q = QA lock (material bin with QA lock) R = Ready (material bin ready) S = In process (material bin in use) U = Used up (material bin used up) V = Pre-release (material bin with pre-release)	☐ String
MATERIAL_BIN_TYPE	Type of material bin [B; C; I; K; L; S]: B = unit C = container I = inventory (stock container) K = component L = board S = lot (shipping)	☐ String
MSL_DRY_END_DATE	End date of the re-drying process; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_FLOOR_LIFE-TIME_REMAIN	Still remaining mounting time (in minutes)	☐ int
MSL_LEVEL	MSL level (1; 2; 2a; 3; 4; 5; 5a; 6): 1 = unlimited 2 = 1 year 2a = 4 weeks 3 = 168 hours 4 = 72 hours 5 = 48 hours 5a = 24 hours 6 = baking required before use	☐ String
MSL_OPEN_DATE	Point in time when the container may be removed from an MSL package (dry pack) or a drying oven/re-drying oven; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
MSL_STATE	State of the MSL object (B; C; O): B = in re-drying (B aking) C = protected/closed(C losed) O = unprotected/open (O pen)	☐ String
PART_COST	Average purchase price [€] of the material (see PART_COST_BASE)	☐ String
PART_COST_BASE	Material quantity on which the purchase price is based (see PART_COST)	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER_RESERVED	Part number of the production order for which the material was reserved in the container	☐ String
QUANTITY_UPDATE	This parameter determines whether the current quantity can be adapted for the container (MATERIAL_BIN_NUMBER) [0;1]: 0 = Quantity changes are NOT performed (FALSE) 1 = Quantity changes are performed (TRUE) Advice: The new quantity is passed using the key MATERIAL_BIN_QTY_ACTUAL; booking takes place based on the material movement type to be passed (see the key TRANSACTION_CODE)!	☐ boolean

Key	Explanation	IMSApi data type
RECEIVING_NUMBER	Receiving number	☐ String
SETUP_FLAG	Function mode "MSL expiry time" [0; 1]: 0 = MSL expiry time is started depending on the station parameter 13125 [▶▶▶] 1 = No MSL function	☐ int
STORAGE_BARCODE	Bar code for the storage place Advice: The STORAGE_BARCODE and STORAGE_NUMBER keys identify the same physical storage place; the bar code must not necessarily match the storage place number! Unlike the used bar code, the storage place number does NOT have to be unique within a site; here, the uniqueness results from the complete file structure "storage cell/storage group/storage place" (in short: storage location)!	☐ String
STORAGE_CELL	Storage cell in the iTAC system	☐ String
STORAGE_CELL_DESC	Storage cell description in the iTAC system	☐ String
STORAGE_DESC	Storage description in the iTAC system	☐ String
STORAGE_GROUP	Storage group in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group and subgroups); the various levels are separated by " " (PIPE)!	☐ String
STORAGE_GROUP_DESC	Storage group description in the iTAC system Advice: If complex group structures are used in the storage, the complete storage group structure is output here (group description and subgroup descriptions); the various levels are separated by " " (PIPE)!	☐ String
STORAGE_NUMBER	Storage place number in the iTAC system Advice: See the advice notice under STORAGE_BARCODE!	☐ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NAME	Manufacturer name	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String
TRANSACTION_CODE	Unique designator (ID) of the material movement type Advice: If [0] or [-1] is passed, mapping to the ID of the (default) material movement type occurs (customer-specific function)!	☐ int
UNIT	Unit for the value passed	☐ String
WORKORDER_RESERVED_NUMBER	Number of the production order for which the material was reserved in the container	☐ String

Tab. 25-1 "Key Value Pair": materialBinUploadKeys

25.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
materialBinUploadKeys	a.) function mode: CREATE_MATERIAL_BIN = 1 List of parameters which are to be passed by the function; the corresponding values are passed with the materialBinUploadValues parameter. b.) function mode: CREATE_MATERIAL_BIN = 0 List of parameters which are to be output by the function; with the parameter materialBinUploadValues, only the value for the key MATERIAL_BIN_NUMBER has to be passed. A description of the possible parameters can be found above in the "Key Value Pair": materialBinUploadKeys table. Advice: Only the parameters passed here are processed by the function!
materialBinUploadValues	Values for the parameters selected by means of materialBinUploadKeys; no entry is necessary for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) materialBinUploadKeys! Refer to the "Key Value Pair": materialBinUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 25-2 Parameter (in) »smtCheckMaterialBin«

Parameter (out)

Parameter name	Explanation
materialBinResultValues	a.) function mode: CREATE_MATERIAL_BIN = 1; the corresponding data is output here only in the event of an error – depending on the materialBinUploadKeys and materialBinUploadValues parameters (in). b.) function mode: CREATE_MATERIAL_BIN = 0; here the corresponding data is output depending on the materialBinUploadKeys parameter (in). Advice: The data is output in the order in which it was entered by means of parameter (in) materialBinUploadKeys!

Tab. 25-3 Parameter (out) »smtCheckMaterialBin«

25.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
504	Warning	The material quantity in the system has become negative
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-503	Failure	Container not found
-514	Failure	Invalid character in "materialBinNumber" [?; %; ' ; -1]
-517	Failure	Retrieval/creation of a supplier not possible
-548	Failure	Failure creating the material lot
-550	Failure	Container number not unique
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site
-3004	Failure	Container number contains invalid characters

25.2

smtSerialNumberCheckIn

25.2.1

Function description

This API function is used by the API-based "SMT connector" service. The function is normally used to check a printed circuit board (serial number) before being fed into an SMT machine (placement device). Production data is ascertained for the serial number that is then evaluated by a successive process (e.g., client). In this way, it is possible to check the placement program name and ascertain information about a placement program.



Advice

*The array returned contains records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active)
AND	ML service (site license)

Function declaration

```
int smtSerialNumberCheckIn
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] serialNoCheckUploadKeys
in String[] serialNoCheckUploadValues
out String[] serialNoCheckResultValues
in String[] logInfoUploadKeys
in String[] logInfoUploadValues
out String[] logInfoResultValues
in String[] serialNumberStateResultKeys
out String[] serialNumberStateResultValues
in String[] recipeResultKeys
out String[] recipeResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) serialNoCheckUploadKeys, serialNoCheckUploadValues, logInfoUploadKeys, logInfoUploadValues, serialNumberStateResultKeys and recipeResultKeys as well as (out) serialNoCheckResultValues, logInfoResultValues, serialNumberStateResultValues and recipeResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Serial number data

Key	Explanation	IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
CUSTOMER_PART_NUMBER	Customer-specific part number (customer mat. no.)	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the input data can be put into context with the various pieces of output data (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☒ long
OBJECT_TYPE	Reference object for the recipe-based placement-program check [0; 1; 2; 3; 4]: 0 = Part 1 = BOM version 2 = Process version 3 = Change index (BOM) 4 = Work step Advice: This parameter is only evaluated if the PLACEMENT_RECIPE_CHECK_MODE key is passed with the value [2]!	☐ int
PART_NUMBER	Part number in the iTAC system	☐ String
PLACEMENT_RECIPE	Placement program name	☐ String
PLACEMENT_RECIPE_CHECK_MODE	"Recipe-related placement-program check" mode [0; 1; 2]: 0 = Check is deactivated 1 = Information on the product and work order are loaded 2 = According to the OBJECT_TYPE parameter, the recipe is loaded and all information on the product and work order ascertained from it	☐ int

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: If the value [-1] and the <code>PLACEMENT_RECIPE_CHECK_MODE</code> key are passed with [1] or [2], the next work step (<code>trGetNextProductionStep</code>) is ascertained!	☐ int
PROCESS_VERSION	Process version from the iTAC system Advice: This parameter is only evaluated if the <code>PLACEMENT_RECIPE_CHECK_MODE</code> key is passed with the value [1] or [2]!	☐ int
RESOLVE_BADMARKS	Scrap ascertainment [0; 1]: 0 = Function deactivated (FALSE) 1 = The serial number state is ascertained over all single panels (TRUE)	☐ boolean
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_STATE_CHECK_MODE	"Serial number check" mode [0; 1; 2]: 0 = No check of the serial number 1 = Checks whether the serial number is present 2 = Checks whether the serial number is permissible on the work step	☐ int
SETUP_PRE_CHECK	"Setup pre-check" mode [0; 1; 2]: 0 = Deactivated 1 = If the key is passed with [1], the result of the last pre-setup check (<code>PreBomCheck</code>) is ascertained in the machine log book. If the information is not needed, the log book entry for the placement program (<code>PLACEMENT_RECIPE</code>) is checked. 2 = Value is ascertained from the machine log book	☐ int
STATION_PM_NUMBER	Station no. of the work station (PM site)	☐ String
WORKORDER_NUMBER	Production order number	☐ String

Tab. 25-4 "Key Value Pair": `serialNoCheckUploadKeys`

Additional information (machine log book)

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the input data can be put into context with the various pieces of output data (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☒ long
LOG_SUBJECT	Subject line of the log book entry (machine log book) Advice: This parameter is only interpreted if the SETUP_PRE_CHECK key is passed with the value [2]!	☐ String
LOG_TEXT	Content of the log book entry (machine log book) Advice: This parameter is only interpreted if the SETUP_PRE_CHECK key is passed with the value [2]!	☐ String

Tab. 25-5 "Key Value Pair": *logInfoUploadKeys*

"Serial number state" output

Advice: This structure is only filled-in if, in the "Key Value Pair": *serialNoCheckUploadKeys* table, the *RESOLVE_BADMARKS* key is passed with the value [1] or the *SERIAL_NUMBER_STATE_CHECK_MODE* key is passed with the value [2]!

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the input data can be put into context with the various pieces of output data (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ long
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_STATE	Process state of the serial number [0; 1; 2]: 0 = pass 1 = fail 2 = scrap	☐ int

Tab. 25-6 "Key Value Pair": *serialNumberStateResultKeys*

"Placement program data from recipe" output

Advice: This structure is only filled-in if, in the **"Key Value Pair": serialNoCheckUploadKeys** table, the `PLACEMENT_RECIPE_CHECK_MODE` key is passed with the value [2]!

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; based on this number, the input data can be put into context with the various pieces of output data (see the parameter by the same name in the other tables with "Key Value Pair" structures)!	☐ long
MEASURE_NAME	Placement program name (as measurement step name in the recipe)	☒ String
NOMINAL	Process layer (as nominal value in the recipe)	☒ String

Tab. 25-7 "Key Value Pair": `recipeResultValues`

25.2.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNoCheckUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>serialNoCheckUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialNoCheckUploadKeys table. Advice: Only the parameters passed here are processed by the function!
serialNoCheckUploadValues	Values for the parameters selected by means of <code>serialNoCheckUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>serialNoCheckUploadKeys</code> ! Refer to the "Key Value Pair": serialNoCheckUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
logInfoUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the logInfoUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": logInfoUploadKeys table. Advice: Only the parameters passed here are processed by the function!
logInfoUploadValues	Values for the parameters selected by means of logInfoUploadKeys ; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) logInfoUploadKeys ! Refer to the "Key Value Pair": logInfoUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wild-cards)!
serialNumberStateResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberStateResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) serialNumberStateResultValues !
recipeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": recipeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) recipeResultValues !

Tab. 25-8 Parameter (in) »smtSerialNumberCheckIn«

Parameter (out)

Parameter name	Explanation
serialNoCheckResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) serialNoCheckUploadKeys and serialNoCheckUploadValues . Advice: The data is output in the order in which it was entered by means of parameter (in) serialNoCheckUploadKeys !
logInfoResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) logInfoUploadKeys and logInfoUploadValues . Advice: The data is output in the order in which it was entered by means of parameter (in) logInfoUploadKeys !
serialNumberStateResultValues	Depending on parameter (in) serialNumberStateResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) serialNumberStateResultKeys !
recipeResultValues	Depending on parameter (in) recipeResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) recipeResultKeys !

Tab. 25-9 Parameter (out) »smtSerialNumberCheckIn«

25.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
204	Warning	Serial number state at the previous work step: SCRAP
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found
-121	Failure	Part not found
-213	Failure	Maximum number of bookings reached
-216	Failure	Station is invalid for the serial number
-706	Failure	Recipe not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site

-3000	Failure	SMT data is not processed
-3005	Failure	Setup does not match the parts list

25.3

smtSetupData

25.3.1

Function description

This function processes serial-number related setup data, consumption data or setup events from placement machines; the `functionMode` input parameter can be used to select which of these setup data is to be processed. This API function is used by the API-based "SMT connector" service.

**Advice**

The SMT Connector groups together various setup functionalities based on API functions.

There is an obsolete version of this function consisting of three sub-functions which are no longer being developed (for further information, see section »Replaced functions« [▶▶▶]) and there the functions »smtConsumption«, »smtEventSetup« and »smtSerialNumberSetup«!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int smtSetupData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int functionMode
in String[] smtSetupHeaderKeys
in String[] smtSetupHeaderValues
out String[] smtSetupHeaderResultValues
in String[] smtSerialNumberKeys
in String[] smtSerialNumberValues
out String[] smtSerialNumberResultValues
in String[] smtEventKeys
in String[] smtEventValues
out String[] smtEventResultValues
in String[] smtPlacementKeys
in String[] smtPlacementValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `smtSetupHeaderKeys`, `smtSerialNumberKeys`, `smtEventKeys` and `smtPlacementKeys` as well as (out) `smtSetupHeaderResultValues`, `smtSerialNumberResultValues` and `smtEventResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

General setup information

Key	Explanation		IMSApi data type
ACTIVATE_SETUP	This parameter specifies whether an automatic setup activation is to occur [0;1]: 1 = The setup is activated automatically (TRUE). 0 = The setup is NOT activated automatically (FALSE).	☐	boolean
ACTIVATE_WORKORDER	This parameter specifies whether an automatic work order activation is to occur [0;1]: 1 = The work order is activated automatically provided it is not already active on the station (TRUE). 0 = The work order is NOT activated automatically (FALSE). Advice: The [1] setting only makes sense if serial numbers are already known and the work order is activated manually via the my-iTAC workplace!	☐	boolean
ASSIGN_SERIAL_NUMBERS	This parameter specifies whether or not the interface can create/assign missing serial numbers for single panels [0;1]: 1 = The serial numbers can be created/assigned (TRUE) 0 = The serial numbers can NOT be created/assigned (FALSE)	☐	boolean
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOOK_SERIAL_NUMBERS	This parameter specifies if the machine can perform a serial number booking (uploadState) [0;1]: 1 = The serial number booking is allowed (TRUE) 0 = The serial number booking is NOT allowed (FALSE)	☐	boolean
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CONSUMPTION_INTERVAL	Determines in which time interval the consumption data is sent from the interface to the server.	☐	int

Key	Explanation		IMSApi data type
CREATE_MATERIAL_BIN	This parameter specifies whether an unknown container can be created [0;1]: 1 = Unknown containers are created in the iTAC system provided that the machine provides all data necessary for this -> this includes container and material numbers as well as container quantity (TRUE). 0 = Unknown containers are NOT created (FALSE) If containers are to be created, a default value can also be specified for the container quantity; i.e. even if only container and material numbers are provided, containers can be created. The default value is defined via the SMT interface using the <code>default container quantity</code> parameter.	☐	boolean
CREATE_WORKORDER	This parameter specifies whether missing work orders can be created [0;1]: 1 = The missing work orders are created (TRUE). 0 = The missing work orders are NOT created (FALSE). Advice: <i>If this parameter is set to 0, work order activation must also be activated (ACTIVATE_WORKORDER)!</i>	☐	boolean
CYCLE_TIME	Processing time at the station in seconds	☐	double
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒	int
ERROR_TEXT	Output text for output value (ERROR_CODE)	☐	String
IGNORE_MATERIAL_BIN_PROBLEMS	This parameter specifies whether container errors can be ignored [0;1]: 1 = Container errors are ignored (TRUE). 0 = Container errors are NOT ignored (FALSE).	☐	boolean
IS_MIRRORED_BOARD	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE) Advice: <i>If the "double-sided panel" entry is selected, the setup information data of the component side and soldering side are combined and considered as a whole for this work order/work step!</i>	☐	boolean
LINE_NUMBER	Line numbers	☐	String
PLACEMENT_RECIPE	Placement program name	☒	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	String
PROCESSING_FAILURE_TYPE_CODE	This parameter specifies whether undefined process failures with a general failure type are booked in the system; the desired failure type code must be entered as the parameter value here.	☐	String
PRODUCT_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the finished product manufactured on the basis of this setup!</i>	☒	String

Key	Explanation		IMSApi data type
SETUP_CHECK	This parameter specifies whether the interface should perform a serial-number based setup check (BOMCheck) [0;1]: 1 = The setup check is performed (TRUE) 0 = The setup check is NOT performed (FALSE)	☐	boolean
SETUP_CHECK_FAILURE_TYPE_CODE	This parameter specifies whether serial numbers with unsuccessful setup checks are booked in the system, and, if so, with which failure type; the desired failure type code must be entered here as the parameter value.	☐	String
SETUP_CHECK_FAILURESLIP	This parameter specifies if, in the event of an unsuccessful setup check, the result of the setup check is booked as a failure note of the affected serial number [0;1]: 1 = The setup result is booked as a failure note (TRUE) 0 = The setup result is NOT booked as a failure note (FALSE)	☐	boolean
SETUP_CHECK_STATE	This parameter specifies whether the serial number with the unsuccessful check is to be converted and, if so, to which serial number state [1; 2]: 1 = Fail 2 = Scrap	☐	int
SETUP_NAME	Name of the setup data record	☐	String
TRANSACTION_CODE_PLACED	The code for the material movement type to be used in the case of "successfully placed components" can be specified here (e.g. "201" for movement type "Order-based material consumption"); the passed value must be a whole number.	☐	int
TRANSACTION_CODE_WASTED	The code for the material movement type to be used in the case of "lost components" can be specified here (e.g. "202" for movement type "Order-based additional use of material"); the passed value must be a whole number.	☐	int
WORKORDER_NUMBER	Production order number	☐	String

Tab. 25-10 "Key Value Pair": *smtSetupHeaderKeys*

Serial numbers

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FAILURE_TYPE_CODE	Failure type code	☐ String
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐ boolean
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
REPAIRED	Repair state [0; 1]: 0 = Not repaired 1 = Repaired	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_STATE	Process state of the serial number [0; 1; 2]: 0 = Pass 1 = Fail 2 = Scrap	☒ int
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 25-11 "Key Value Pair": smtSerialNumberKeys

Event-based setup data

Key	Explanation	IMSApi data type
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the data objects can be put into context for the setup data record!	☐ long
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FEEDER_ID	Internal identification number of the feeder	☐ String
MATERIAL_BIN_DATE_CODE	DateCode for the material bin	☐ String

Key	Explanation	IMSApi data type
MATERIAL_BIN_NUMBER	Number of the material bin (container no.)	☒ String
MATERIAL_BIN_QTY_ACTUAL	Current quantity in the material bin	☐ double
MATERIAL_BIN_QTY_TOTAL	Original quantity in the material bin (original container quantity)	☐ double
PART_NUMBER	Part number in the iTAC system Advice: In this case, this is the part number of the container material!	☐ String
PART_PARENT_NUMBER	Part number of the parent part according to the BOM structure	☐ String
QUANTITY_PLACED	Number of "successfully placed components"	☐ double
QUANTITY_WASTED	Number of "lost components"	☐ double
SETUP_EVENT_TYPE	Event type [0; 1; 2; 3; 4]: 0 = Standard for every new event (UNUSED_EVENT) 1 = Complete setup change (CHANGE_SETUP) 2 = Setting up a slot that has not yet been set up (COMPLETE_SETUP) 3 = Updating the setup for a slot (CHANGE_SLOT) 4 = Deletion of a slot/setup (REMOVE_SLOT)	☒ int
SETUP_POSITION	Setup location (e.g. track) of the material that has been set up	☒ String
SETUP_START_DATE	Start date for the period under review (set up from); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☒ String
SUPPLIER_CHARGE_NUMBER	Storage unit or supplier charge	☐ String
SUPPLIER_NUMBER	Manufacturer number	☐ String

Tab. 25-12 "Key Value Pair": *smtEventKeys*

Mounting places

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☒ String
COMP_REFERENCE	Reference number of the setup data record Advice: On the basis of this number, the mounting places can be put into context with the setup data record!	☒ long
PANEL_POSITION	Panel position	☐ int

Tab. 25-13 "Key Value Pair": *smtPlacementKeys*

25.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
functionMode	Function mode [0; 1; 2]: 0 = SERIAL_SETUP Depending on the process, this SMT function calls the »mlCreate-NewMaterialBin«, »mlGetMaterialBinData«, »setupGetMaterial-Setup«, »setupGetSetupName«, »setupStateChange«, »setupUpdateMaterialSetup«, »trAssignSerialNumberForProductOrWorkOrder«, »trGetSerialNumberInfo« and »trUploadFailure-AndResultData« API functions in the background. 1 = EVENT_SETUP Depending on the process, this (SMT) function calls the »setupGet-SetupName«, »setupStateChange« and »setupUpdateMaterial-Setup« API functions in the background 2 = CONSUMPTION_SETUP This (SMT) function calls the »mlUploadMaterialBinBooking« API function in the background.
smtSetupHeader-Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>smtSetupHeaderValues</code> parameter. A description of the possible parameters (depending on the <code>functionMode</code>) can be found above in the "Key Value Pair": smtSetupHeaderKeys table. Advice: Only the parameters passed here are processed by the function!
smtSetupHeader-Values	Values for the parameters selected by means of <code>smtSetupHeaderKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>smtSetupHeaderKeys</code>! Refer to the "Key Value Pair": smtSetupHeaderKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
smtSerialNumber-Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>smtSerialNumberValues</code> parameter. A description of the possible parameters (depending on the <code>functionMode</code>) can be found above in the "Key Value Pair": smtSerialNumberKeys table. Advice: Only the parameters passed here are processed by the function!
smtSerialNumber-Values	Values for the parameters selected by means of <code>smtSerialNumberKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>smtSerialNumberKeys</code>! Refer to the "Key Value Pair": smtSerialNumberKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
smtEventKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the smtEventValues parameter. A description of the possible parameters (depending on the functionMode) can be found above in the "Key Value Pair": smtEventKeys table. Advice: Only the parameters passed here are processed by the function!
smtEventValues	Values for the parameters selected by means of smtEventKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) smtEventKeys! Refer to the "Key Value Pair": smtEventKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
smtPlacementKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the smtPlacementValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": smtPlacementKeys table. Advice: Only the parameters passed here are processed by the function!
smtPlacementValues	Values for the parameters selected by means of smtPlacementKeys; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) smtPlacementKeys! Refer to the "Key Value Pair": smtPlacementKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 25-14 Parameter (in) »smtSetupData«

Parameter (out)

Parameter name	Explanation
smtSetupHeaderResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) smtSetupHeaderKeys and smtSetupHeaderValues. Advice: The data is output in the order in which it was entered by means of parameter (in) smtSetupHeaderKeys!
smtSerialNumberResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) smtSerialNumberKeys and smtSerialNumberValues. Advice: The data is output in the order in which it was entered by means of parameter (in) smtSerialNumberKeys!
smtEventResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) smtEventKeys and smtEventValues. Advice: The data is output in the order in which it was entered by means of parameter (in) smtEventKeys!

Tab. 25-15 Parameter (out) »smtSetupData«

25.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-502	Failure	Failure creating the material bin
-503	Failure	Container not found
-622	Failure	Setup belongs to a different work order
-2070	Failure	IMSApi key was passed multiple times to the function
-2107	Failure	No ML license present for the site
-2110	Failure	Serial number position is invalid
-3000	Failure	SMT data is not processed
-3001	Failure	No validator present
-3002	Failure	Validation failed
-3003	Failure	Unknown failure

-3004	Failure	Container number contains invalid characters
-3005	Failure	Setup does not match the parts list
-3006	Failure	SMT -> Event type is invalid

26 Group "TR" (traceability)

26.1 trActivateOrderCluster

26.1.1 Function description

This functions allows all work orders contained in the work order cluster to be activated/deactivated simultaneously on a station. The cluster name or, alternatively, a serial number from a work order of the work order cluster can be used for identification of the work order cluster.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trActivateOrderCluster
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] orderClusterActivateData
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `orderClusterActivateData`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ACTIVATE_FLAG	"Work order cluster activation" mode [1; 2]: 1 = All work orders of the work order cluster are activated on the station 2 = All work orders of the work order cluster are deactivated on the station Advice: If mode [1] is selected, the key <code>CLUSTER_NAME</code> or <code>SERIAL_NUMBER</code> MUST be passed. This data is not required for mode [2] because the current work order cluster is ascertained automatically!	☐ int
CLUSTER_NAME	Unique name for the work order cluster Special case: If the work order cluster name is unknown, it is also possible to use a serial number (<code>SERIAL_NUMBER</code>) to find a work order in the work order cluster and in doing so to identify the work order cluster. If the keys <code>CLUSTER_NAME</code> and <code>SERIAL_NUMBER</code> are passed, the serial number must be a component of a work order in the work order cluster. Advice: No entry is necessary for deactivation (see <code>ACTIVATE_FLAG = [2]</code>) because the current work order cluster is ascertained automatically!	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel	☒ int
SERIAL_NUMBER	Serial number of the panel Special case: If the work order cluster name is unknown, it is also possible to use a serial number (<code>SERIAL_NUMBER</code>) to find a work order in the work order cluster and in doing so to identify the work order cluster. If the keys <code>CLUSTER_NAME</code> and <code>SERIAL_NUMBER</code> are passed, the serial number must be a component of a work order in the work order cluster. Advice: No entry is necessary for deactivation (see <code>ACTIVATE_FLAG = [2]</code>) because the current work order cluster is ascertained automatically!	☐ String
WORKSTEP_STATE_FLAG	Change in "work step state" [0; 1]: 0 = No change 1 = Interrupt the work steps of the previous work order. 2 = Finish the work steps of the previous work order.	☐ int

Tab. 26-1 "Key Value Pair": `orderClusterActivateData`

26.1.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
orderClusterActivateData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterActivateData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": orderClusterActivateData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-2 Parameter (in) »trActivateOrderCluster«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.1.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-135	Failure	Work order activation failed
-150	Failure	Invalid value for "Flag"
-165	Failure	Setting in combination with AVO no. invalid as "processLayer"
-185	Failure	Invalid value for the process layer
-240	Failure	No work step found for work order, orientation of the PCB and station
-356	Failure	Serial number does not belong to a work order cluster
-357	Failure	Serial number belongs to different work order cluster

-359	Failure	Layer not found in work step cluster
-1122	Failure	Work order cluster not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.2 trActivateWorkorder

26.2.1 Function description

This function activates or deactivates the passed work order on the station or line.



Advice

If a new work order is activated, the work order currently on the station/line is deactivated. Deactivation of a work order also causes the setup to be deactivated because the setup is managed based on the work order. Even if the setup matches the new work order, the setup cannot be automatically activated as well. In this case, the setup must be activated using the IMSApi function »setupStateChange« [▶▶▶]!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trActivateWorkOrder
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String workOrderNumber
    in String serialNumber
    in String serialNumberPos
    in int processLayer
    in int flag

```

26.2.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order
	Special case: If [-1] is passed here, the work order to be activated is ascertained on the basis of the serial number!
	Advice: No entry is necessary for deactivation (see flag = [3] or [4] below) since the currently active work order is automatically ascertained!

Parameter name	Explanation
<code>serialNumber</code>	Serial number of the panel Advice: <i>This field is only evaluated if no order number was passed for activation (<code>workOrderNumber = -1</code>)! No entry is necessary for deactivation (see <code>flag = [3]</code> or <code>[4]</code> below) since the currently active work order is automatically ascertained!</i>
<code>serialNumberPos</code>	Position of the single panel in the unit-array Advice: <i>This field is only evaluated if no order number was passed (<code>workOrderNumber = -1</code>)!</i> Special case: If an empty string or the value <code>[-1]</code> is passed, the position is not taken into account; in this case the serial number must be unique!
<code>processLayer</code>	Orientation of the PCB during the work step <code>[0; 1; 2; 3]</code> : 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice regarding [3]: <i>As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function. The "<code>processLayer = 3</code>" mechanism is therefore outdated; the setup data for the component and soldering side was combined and considered as a whole here!</i> Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
<code>flag</code>	Mode "Work order activation" <code>[1; 2; 3; 4]</code> : 1 = Activate the work order only for the station 11 = Activate the work order only for the station and interrupt the preceding work step 21 = Activate the work order only for the station and finish the preceding work step 2 = Activate the work order on the entire line 12 = Activate the work order on the entire line and interrupt the preceding work step 22 = Activate the work order on the entire line and finish the preceding work step 3 = Deactivate the work order on the station and line 13 = Deactivate the work order on the station and line and interrupt the preceding work step 23 = Deactivate the work order on the station and line and finish the preceding work step 4 = Deactivate and finish the work order on the station and line 24 = Deactivate and finish the work order on the station and line and finish all work steps Advice: <i>If mode <code>[1]</code> or <code>[2]</code> is selected, a <code>workOrderNumber</code> or <code>serialNumber</code> must absolutely be passed; for mode <code>[3]</code> or <code>[4]</code>, this data is not necessary!</i>

Tab. 26-3 Parameter (in) »trActivateWorkorder«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.2.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-130	Failure	Work order not found
-135	Failure	Work order activation failed
-150	Failure	Invalid value for "Flag"
-165	Failure	Setting in combination with AVO no. invalid as "processLayer"
-185	Failure	Invalid value for the process layer
-240	Failure	No work step found for work order, orientation of the PCB and station
-2073	Failure	More than one work order is active on the station

26.3

trActivateWorkstep

26.3.1

Function description

This function activates or deactivates a certain work step of a work order on the passed station. The function is thereby specifically intended for split work steps which, in turn, enable optimized work order detailed planning. The required identification number of the split work step (`SPLIT_ID`) is provided by the APS service.



Advice

Compared to the IMSApi function »trActivateWorkorder« [▶▶▶], this function can neither finish work orders nor perform activation/deactivation on the line level!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trActivateWorkstep
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] activateWorkstepKeys
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `activateWorkstepKeys`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ACTIVATE_FLAG	Mode for work order activation [1; 2]: 1 = Activate work order/work step (SPLIT_ID) on station 2 = Deactivate work order/work step (SPLIT_ID) on station	☐	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel	☒	int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
WORKORDER_NUMBER	Number of a production order	☒	String
WORKSTEP_STATE_FLAG	Change in "work step state" [0; 1]: 0 = No change 1 = Interrupt the work steps of the previous work order. 2 = Finish the work steps of the previous work order.	☐	int

Tab. 26-4 "Key Value Pair": *activateWorkstepKeys*

26.3.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
activateWorkstep-Keys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterActivateData table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": orderClusterActivateData table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-5 Parameter (in) »trActivateWorkstep«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.3.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-135	Failure	Work order activation failed
-150	Failure	Invalid value for "Flag"
-185	Failure	Invalid value for the process layer
-240	Failure	No work step found for work order, orientation of the PCB and station
-2070	Failure	IMSApi key was passed multiple times to the function
-2073	Failure	More than one work order is active on the station
-2126	Failure	Split ID not valid

-2128	Failure	Split ID is not unique
-2129	Failure	Station is not part of the split work step

26.4

trAddExternalSerialNumber

26.4.1

Function description

This function is used if external products (serial-numbered units as supplier parts) are to automatically be taken over in the internal production process. This enables "traceable" mounting of external products in the final products produced in-house. All data objects required for this, such as work order, work plan and parts list, can - provided a corresponding configuration exists - be generated automatically. Also observe the following conditions:

- The passed part number (see key `PART_NUMBER`) MUST exist in the iTAC master data; the part must be identified as "product".
- The executing station (see function parameter `stationNumber`) must be authorized for the creation of work orders (station setup, "Setup" tab).

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! When a work order is created using an API function, the same checking mechanisms are performed as applied during manual work order creation in the myiTAC workplace (ADM module)!*

This API function is influenced by station parameter code no. 7320 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 13302 [▶▶▶], 13312 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

The function performs the following individual tasks:

1. Creation of a work order for external products
2. Assignment of external products (serial numbers) to the work order
3. State booking for the external serial numbers

Background to "automatic work order creation": For automatic work order creation, the following entries from the "Key Value Pair" structure **workOrderUploadData** must be passed:

- `PART_NUMBER` (part number)
- `WORKORDER_TYPE` (work order type)
- `WORKPLAN_ALT` (work plan alternative)
- `BOM_VERSION_ERP` (BOM version in the ERP system)
- `WORKORDER_NUMBER` (order number)

Advice "without order number": *If an existing open work order is found for the passed work order type and work plan alternative, this work order is used. If no existing open work order is found, the order number is formed from the part number and current date.*

For automatically created work orders, the current date (time stamp "API call") is used both for the planned start date and for the delivery date.

Background to "work plan ascertainment": If an existing released work plan is found for the passed part, this work plan is used for work order creation. If multiple released work plans are available, the most recent work plan is used by default. By

passing the `WORKPLAN_VERS_ERP` key, it is also possible to determine a specific work plan version for work order creation. If no work plan exists for the part, the master work plan is used for the passed work plan type.

Background to "BOM ascertainment": If an existing, released BOM is found for the passed part, this BOM is used for work order creation. If an ERP BOM version (`BOM_VERSION_ERP`) – also referred to as change index – is passed, this BOM version must be taken into consideration during the search. All BOMs with other ERP BOM versions cannot be used. If no suitable BOM is found, a corresponding BOM header is created (BOM type "external products" is used here).

Background to "serial number assignment": This function can be used to assign both single panels and multiple panels to a work order. Before the serial numbers are assigned, it is ensured that the external serial numbers do not already exist in the system. In this context, there are following different application types:

- **Application type "A"** (single panel -> one serial number):
The data of the serial number to be booked is passed with the function parameters (`serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime`). The "Key Value Pair" structure `serialNumberUploadKeys` is not filled.
- **Application type "B"** (single panels -> multiple serial numbers):
Function parameter `serialNumberRef` = `-1` and `serialNumberRefPos` = `-1` -> the data of the serial numbers to be booked is passed the "Key Value Pair" structure `serialNumberUploadKeys`. The function parameter `serialNumberState` is not evaluated; `bookDate` and `cycleTime` can be passed as default values and then overridden if necessary for individual serial numbers by means of keys `BOOK_DATE` and `CYCLE_TIME`.
- **Application type "C"** (multiple panels):
function parameter `serialNumberRef` = ["reference serial number"] and `serialNumberRefPos` = ["position"] -> the data of the serial numbers to be booked (as well as the reference serial number) is passed with "Key Value Pair" structure `serialNumberUploadKeys`. Function parameters `bookDate` and `cycleTime` can be passed as default values and then be overridden if necessary for individual serial numbers by means of keys `BOOK_DATE` and `CYCLE_TIME`.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```

int trAddExternalSerialNumber
    in IMSApiSessionContextStruct sessionContext
        long sessionId
        long persId
        String locale
    in String stationNumber
    in String serialNumberRef
    in String serialNumberRefPos
    in int serialNumberState
    in DateMs bookDate
    in float cycleTime
    in KeyValue[] workOrderUploadData
    in String[] workOrderResultKeys
    out String[] workOrderResultValues
    in String[] serialNumberUploadKeys
    in String[] serialNumberUploadValues
    out String[] serialNumberResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `resultKeys` and (out) `resultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for the work order to be used or created

Key	Explanation	IMSApi data type
BOM_VERSION_ERP	BOM version in the ERP system Advice: This parameter is required for automatic work order creation!	☐ String
PART_NUMBER	Part number in the iTAC system Advice: This parameter is required for automatic work order creation!	☐ String
WORKORDER_CODE	Description of the control code for work order creation; the following control codes are offered in the system: 8400 = External products 8600 = SCT order	☐ int
WORKORDER_DESC	Order description	☐ String

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Customer-specific order number Advice "without order number": If an existing open work order is found for the passed work order type and work plan alternative, this work order is used. If no existing open work order is found, the order number is formed from the part number and current date ([PART_NUMBER]-[DATUM]).	☐ String
WORKORDER_QUANTITY	Order quantity Advice: If the key is not passed, the value 999,999,999 is used by default for new work orders!	☐ int
WORKORDER_STATE	State of the production order [F]: F = opened (released) Advice: If this key is not passed along, the [F] value is used by default!	☐ String
WORKORDER_TYPE	Work order type [S; X]; the work order type is indirectly defined by the work plan alternative (see identification code in the WORKPLAN_ALT field). The following assignments are available by default: S = Work order for external products (customer-specific) X = SCT order (customer-specific); the work plan alternative for this work order type is created during the course of SCT system configuration. Depending on the work order type, only certain control codes are valid (see WORKORDER_CODE): - SCT work order: 8600 - External products: 8400 Advice: This parameter is essential for automatic work order creation; if the key is not passed, the value [S] is used by default!	☐ String
WORKPLAN_ALT	Work plan alternative [customer-specific]: ... (SCT) = Production order for transferred products ... (EXT) = Production order for external products Advice: This parameter is required for automatic work order creation!	☐ String
WORKPLAN_VERS_ERP	ERP change number for the work plan	☐ String

Tab. 26-6 "Key Value Pair": workOrderUploadData

Feedback about the used or created work order

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
WORKORDER_ID	Internal identification number of the production order	☐ long
WORKORDER_NUMBER	Production order number	☐ String

Tab. 26-7 "Key Value Pair": workOrderResultKeys

Input values for serial number booking

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and time or uses the default according to the <code>bookDate</code> parameter!	☐	DateMs
CYCLE_TIME	Processing time at the station in seconds Special case: [-1] uses the default according to the <code>cycleTime</code> function parameter!	☐	double
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☒	String
SERIAL_NUMBER_STATE	Process state of the serial number [0; 1; 2]: 0 = pass 1 = fail 2 = scrap	☒	int

Tab. 26-8 "Key Value Pair": `serialNumberUploadKeys`

26.4.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Special case: If the value [-1] is passed here, all serial numbers passed in the structure <code>serialNumberUploadKeys</code> are booked as single panels!
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: If the value [-1] is passed, the field is not evaluated; this is, however, only possible for the "single panel" application types!
serialNumberState	Process state of the serial number [0; 1; 2]: 0 = pass 1 = fail 2 = scrap Advice: If the parameter is not filled, the value [0] is passed by default!
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
cycleTime	Processing time at the station in seconds
workOrderUploadData	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>workOrderUploadData</code> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <code>workOrderUploadData</code> table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
workOrderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>workOrderResultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>workOrderResultValues</code> !

Parameter name	Explanation
serialNumberUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>serialNumberUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberUploadKeys table. Advice: <i>Only the parameters passed here are processed by the function!</i>
serialNumberUploadValues	Values for the parameters selected by means of <code>serialNumberUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: <i>The data is assigned in the order in which it was entered by means of parameter (in) <code>serialNumberUploadKeys</code>! Refer to the "Key Value Pair": serialNumberUploadKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!</i>

Tab. 26-9 Parameter (in) »trAddExternalSerialNumber«

Parameter (out)

Parameter name	Explanation
workOrderResultValues	Depending on parameter (in) <code>workOrderResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>workOrderResultKeys</code>!</i>
serialNumberResultValues	The corresponding data is output here only in the event of an error – depending on the <code>serialNumberUploadKeys</code> and <code>serialNumberUploadValues</code> parameters (in). Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>serialNumberUploadKeys</code>!</i>

Tab. 26-10 Parameter (out) »trAddExternalSerialNumber«

26.4.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-121	Failure	Part not found
-126	Failure	Process version is invalid
-132	Failure	Work order is not unique
-138	Failure	Order state is invalid
-151	Failure	The "serialNumberRef" parameter must be specified
-196	Failure	A parameter is invalid
-205	Failure	Serial number belongs to a different work order
-1101	Failure	BOM not found

-1106	Failure	BOM for the work order not found
-2016	Failure	Work order type does not match work plan type
-2021	Failure	Work order type not valid
-2032	Failure	Work plan contains no work steps with valid confirmation
-2064	Failure	Production order -> Parameter missing
-2070	Failure	IMSApi key was passed multiple times to the function
-2111	Failure	Master work plan not found
-2112	Failure	Part is not a product

26.5

trAddSerialNumberToPool

26.5.1

Function description

This function is used to insert one or more serial numbers in a serial number pool. Both the serial number(s) as well as the serial number pool must exist in the system. This IMSApi function is a further development of the »trAddSerialNumberToSerialNumberPool« [▶▶▶] function and should always be preferred – especially with new installations. Thanks to the "Key Value Pair" structure used here, it can be expanded much more easily later on.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Serial number pools are required for repair and modification orders; in this context, also note the »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

This API function is influenced by station parameter code nos. 13301 [▶▶▶], 13302 [▶▶▶], 13311 [▶▶▶] and 13312 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trAddSerialNumberToPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in String[] serialNumberPoolKeys
in String[] serialNumberPoolValues
out String[] serialNumberPoolResults
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberPoolKeys and serialNumberPoolValues as well as (out) serialNumberPoolResults). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
AS-SIGN_TO_CHANGE_WORKORDER	Serial number assignment to the modification order [0; 1]: 0 = Serial number will NOT be assigned (FALSE) 1 = Serial number will be assigned to the work order that has been assigned to the serial number pool; this can be a repair or modification order (TRUE) Advice: If the value [1] is passed, then a failure could occur if a.) No order has been assigned to the serial number pool yet, or b.) the serial number has not yet been identified (can be prevented by means of the IDENTIFY_SERIAL_NUMBER key)!	☐	boolean
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
IDENTIFY_SERIAL_NUMBER	Identification mode [0; 1]: 0 = serial number will NOT be identified (FALSE) 1 = serial number will be identified (TRUE) Advice: The identification process corresponds to the API function »trIdentifyPoolSerialNumber« [▶▶▶]!	☐	boolean
INFO	Individual additional information on the serial number	☐	String
SERIAL_NUMBER	Serial number of the panel that is to be added to the pool	☒	String
SERIAL_NUMBER_DUPLICATE	Addition of all serial numbers of a multiple panel [0; 1]: 0 = multiple panel function NOT active (default setting); this value is used in the case of all entries not equal to [1]. 1 = multiple panel function active Advice: If the value [1] is passed, all serial numbers of the multiple panel are automatically added to the pool; for this purpose, the passed serial number (SERIAL_NUMBER) must be a valid reference serial number!	☐	int
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String

Tab. 26-11 "Key Value Pair": serialNumberPoolKeys

26.5.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool- Name	Name of the sno. pool (pool name)
serialNumber- PoolKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>serialNumberPoolValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberPoolKeys table. Advice: Only the parameters passed here are processed by the function!
serialNumberPool- Values	Values for the parameters selected by means of <code>serialNumberPoolKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>serialNumberPoolKeys</code> ! Refer to the "Key Value Pair": serialNumberPoolKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-12 Parameter (in) »trAddSerialNumberToPool«

Parameter (out)

Parameter name	Explanation
serialNumberPool- Results	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>serialNumberPoolKeys</code> and <code>serialNumberPoolValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>serialNumberPoolKeys</code> !

Tab. 26-13 Parameter (out) »trAddSerialNumberToPool«

26.5.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-202	Failure	Serial number is locked
-287	Failure	Serial number pool does not exist
-289	Failure	Serial number is already in the pool
-290	Failure	Serial number is on the SCRAP work step
-291	Failure	Serial number is not yet booked
-293	Failure	Serial number is not assigned to any pool
-294	Failure	Serial number is assigned to another pool
-296	Failure	Serial number is on the FAIL work step

-299	Failure	Serial number is already assigned to a repair order
-312	Failure	Serial number pool is already finished
-318	Failure	Sno. pool has not assigned any modification order
-363	Failure	Serial number pool -> At least one serial number must have the PASS state
-2138	Failure	Serial number is deleted

26.6

trAddSerialNumberToSerialNumberPool

26.6.1

Function description

With this function, a serial number can be inserted into a serial number pool. Both the serial number as well as the serial number pool must exist in the system. This IMSApi function was further developed with the »trAddSerialNumberToPool« [▶▶▶] function and should now only be used in existing workflows. For new application types, the further developed version should always be used.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trAddSerialNumberToSerialNumberPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in String serialNumberPoolName
out int state
out String partNumber
out String workOrderNumber
```

26.6.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
serialNumberPool-Name	Name of the sno. pool (pool name)

Tab. 26-14 Parameter (in) »trAddSerialNumberToSerialNumberPool«

Parameter (out)

Parameter name	Explanation
state	State of the product for the serial number [0; 1; 2; -1]: 0 = pass 1 = fail 2 = scrap -1 = unknown
partNumber	Part number of the serial number in the iTAC system
workOrderNumber	Production order number for the serial number

Tab. 26-15 Parameter (out) »trAddSerialNumberToSerialNumberPool«

26.6.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-201	Failure	Serial number is not active
-202	Failure	Serial number is locked
-287	Failure	Serial number pool does not exist
-289	Failure	Serial number is already in the pool
-290	Failure	Serial number is on the SCRAP work step
-291	Failure	Serial number is not yet booked
-294	Failure	Serial number is assigned to another pool
-296	Failure	Serial number is on the FAIL work step
-299	Failure	Serial number is already assigned to a repair order
-312	Failure	Serial number pool is already finished
-2138	Failure	Serial number is deleted

26.7

trAssignSerialNumberForProductOrWorkOrder

26.7.1

Function description

This function is used to assign one or more serial numbers to a work order. Here, the work order does not need to be active on the station. To determine the work order the serial number should be assigned to, and to determine which tests are to be carried out for this, the input values `partNumber` (part), `bomVersion` (BOM version), and `workOrderNumber` (work order) may be combined.



Advice

If only one serial number is to be assigned, no entry is required in the `serialNumberArray` structure.

If no entry is passed by means of `workOrderNumber`, a search for a suitable work order is carried out based on the passed product data (`partNumber` and `bomVersion`). If no work order is found, a new work order is created. It is a condition for automatic work order creation that the work plan uses work order type "01". This API function is influenced by station parameter code nos. 7208 [▶▶▶], 13301 [▶▶▶], 13302 [▶▶▶], 13311 [▶▶▶], and 13312 [▶▶▶] as well as site parameter 4610 [▶▶▶]. In addition, the "Work order generation" station setting ("Setup" tab) must be activated. Further information on this topic can be found in the reference section of this documentation!

Combinations if the order number is known:

- `partNumber`, `bomVersion` and `workOrderNumber`
all dependencies are checked for plausibility, the work order passed must match the part and BOM version
- `partNumber` and `workOrderNumber`
the work order passed must match the part, it is not checked whether the BOM version is correct
- `workOrderNumber`
the serial number is assigned to the work order passed without further checking

Combinations if the order number is unknown:

- `partNumber` and `bomVersion`
searches for a suitable WO for the serial number based on the passed part and on the BOM version; if several WOs are found, the serial number is assigned to the most recent WO; if no WO is found, a new WO is created
- `partNumber`
searches for a suitable WO for the serial number based on the passed part; if several WOs are found, the serial number is assigned to the most recent WO; if no WO is found, a new WO is created

Application type "Addition in multiple panel": If a multiple panel with serial numbers has already been assigned to a work order, further serial numbers can if necessary be added in the multiple panel and thus subsequently assigned to the work order. For this application type, the function must be called up using the corresponding reference or label serial number (panel label) of the multiple panel. The serial numbers passed in the array are then added to the multiple panel.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trAssignSerialNumberForProductOrWorkOrder
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String partNumber
in String bomVersion
in String serialNumberRef
in String serialNumberRefPos
in int processLayer
in SerialNumberData[] serialNumberArray
    String serialNumber
    String serialNumberPos
    String serialNumberOld
in int activateWorkOrder
```

26.7.2**Input and output parameters****Parameter (in)**

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order Advice: If the value [-1] is passed here, the work order active on the station is used or a suitable work order is searched for on the basis of the product data (partNumber and bomVersion)!
partNumber	Part number in the iTAC system Advice: If this field is not to be evaluated (workOrderNumber is used), an empty string or the value [-1] must be passed!
bomVersion	BOM version from the iTAC system Advice: If this field is not to be evaluated (workOrderNumber and/or partNumber are/is used), an empty string or the value [-1] must be passed!
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Special case: If the value [-1] is passed here, all serial numbers passed in the array are booked as single panels! Advice for panel labels: If a serial number is specified that is NOT contained in the serialNumberArray structure, this serial number is stored as label serial number (abb.: panel label); it is then recommended that the value [0] be used for serialNumberRefPos!

Parameter name	Explanation
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: If the value [-1] is passed for serialNumberRef, this parameter is not evaluated!
processLayer	Orientation of the PCB during the work step [-1; 0; 1; 2]: -1 = if activateWorkOrder = 0 0 = component side 1 = soldering side 2 = position independent Advice: The orientation of the PCB only needs to be passed if work order activation is used (activateWorkOrder = 1)!
serialNumberArray	Array of the structure with the following variables:
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array
serialNumberOld	- Parameter not used -
activateWorkOrder	Mode "Work order activation" [0; 1]: 1 = Activate the work order 11 = Activate the work order and interrupt the preceding work steps 21 = Activate the work order and finish the preceding work steps 0 = Do not activate the work order Advice: If work order activation is used (activateWorkOrder = 1), the orientation of the PCB of the current work step must be passed (see processLayer)!

Tab. 26-16 Parameter (in) »trAssignSerialNumberForProductOrWorkOrder«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.7.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-108	Failure	Station may not add serial numbers
-114	Failure	Serial number is archived
-120	Failure	Work plan not found
-121	Failure	Part not found
-124	Failure	Part does not match the work order part
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-134	Failure	Default value for maximum order quantity has been reached
-137	Failure	No work order could be created
-138	Failure	Order state is invalid
-139	Failure	BOM version does not match the version from the work order
-196	Failure	A parameter is invalid
-201	Failure	Serial number is not active

-202	Failure	Serial number is locked
-205	Failure	Serial number belongs to a different work order
-206	Failure	Serial number was already assigned to this work order
-221	Failure	The slave serial no. is not released from previous process
-235	Failure	New serial number could not be created
-237	Failure	Serial number belongs to a different station
-239	Failure	Serial number array is empty
-244	Failure	Serial number is in the array more than once
-290	Failure	Serial number is on the SCRAP work step
-291	Failure	Serial number is not yet booked
-296	Failure	Serial number is on the FAIL work step
-299	Failure	Serial number is already assigned to a repair order
-301	Failure	Serial number is mounted
-355	Failure	Production order is part of a work order cluster
-1101	Failure	BOM not found
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.8

trAssignSerialNumberMergeAndUploadState

26.8.1

Function description

This function combines several API functions which are also available as individual functions. The functions that are combined are »trAssignSerialNumberForProductOrWorkOrder« [▶▶▶], »trMergeParts« [▶▶▶] and »trUploadState« [▶▶▶]. The direct association between the assignment of a serial no. to a WO (assign) and the product merging with another serial no. (merge) prevents a serial no. from being assigned to several WOs at the same time. Thus, this function can ensure higher process reliability compared to the separate function calls.

**Advice**

*This API function is influenced by station parameter code nos. 7203 [▶▶▶], 7208 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 13301 [▶▶▶], 13302 [▶▶▶], 13311 [▶▶▶], 13312 [▶▶▶] and 13400 [▶▶▶] as well as site parameter 4610 [▶▶▶]. In addition, the "Work order generation" station setting ("Setup" tab) must be activated. Further information on this topic can be found in the reference section of this documentation! Depending on the configuration of the station parameter 7208 [▶▶▶], this API function – unlike nearly all other API functions – can be used **cross-site**. Thus, products (serial numbers) from different sites can be mounted in one another!*

The functional sequence looks like this:

1. During a transaction, serial numbers that are returned are assigned to a product version and work order that have been configured on the line; independent of whether this is a multiple or single panel, the serial numbers are passed via the `serial-NumberArray` array. Subsequently, a product merging is carried out (merge slave product into master product), during which both the serial no. passed via `serial-NumberSlave` and the referenced serial nos. (in case of multiple printed panels) are discarded.

A plausibility check is performed before assignment to determine whether the serial numbers can be used. In the case of faulty product merging, the assignment to the work order is also undone.

2. After successful assignment and merging, an optional »trUploadState« [▶▶▶] may be carried out. This causes the serial nos. and their states to be booked in a work step conformant manner. If an unexpected abort takes place, only the »trUpload-State« [▶▶▶] must be repeated. Point 1. is no longer affected.

Application type "Addition in multiple panel": If a multiple panel with serial numbers has already been assigned to a work order, further serial numbers can if necessary be added in the multiple panel and thus subsequently assigned to the work order. For this application type, the function must be called up using the corresponding reference or label serial number (panel label) of the multiple panel. The serial numbers passed in the array are then added to the multiple panel.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trAssignSerialNumberMergeAndUploadState
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumberRef
in String serialNumberRefPos
in SerialNumberData[] serialNumberArray
    String serialNumber
    String serialNumberPos
    String serialNumberOld
in String serialNumberSlave
in int doUploadState
in int serialNumberState
in int multiplePanel
  
```

26.8.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
serialNumberRef	Serial no. of the first single panel or of the unit-array (cf. parameter multiplePanel); this number is used as reference serial number. Advice: Serial no. which becomes active during a product merging (see serialNumberSlave)! Advice for panel labels: If a serial number is specified that is NOT contained in the serialNumberArray structure, this serial number is stored as label serial number (abb.: panel label); it is then recommended that the value [0] be used for serialNumberRefPos!
serialNumberRefPos	Position of the serial number of the first single panel or of the unit-array (cf. parameter multiplePanel)
serialNumberArray	Structure (array) with the following variables:
serialNumber	Serial no. of the panel Advice: This is the serial number which becomes active during a product merging (see serialNumberSlave)!
serialNumberPos	Position of the single panel in the unit-array
serialNumberOld	- Parameter not used -

Parameter name	Explanation
serialNumberSlave	Serial no. which becomes inactive during product merging (see serialNumberRef or serialNumber) Advice: The serial number can still be used as a new serial-NumberRef (multiple panel) or serialNumber (single panel)!
doUploadState	Activation/deactivation [0; 1]: 0 = Function »trUploadState« [▶▶▶] is NOT executed 1 = Function »trUploadState« [▶▶▶] is executed
serialNumberState	State of the product for the serial no. [0; 1; 2; 3]: 0 = Pass 1 = Fail 2 = Scrap 3 = In process
multiplePanel	If the serialNumberSlave is part of a multiple panel, this parameter determines whether only the passed or all panels are mounted [0; 1; 2; 3]: 0 = Single panel; only the passed slave serial number is mounted (only work steps with obligatory confirmation) 1 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and mounted (only work steps with obligatory confirmation) 2 = Single panel; the passed slave serial number is mounted only (all work steps) 3 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and mounted (all work steps) Advice regarding "mode differentiation": The difference between mode 0 and 2 and between 1 and 3 is that in mode 2 and 3 the state of work steps without obligatory confirmation is also taken into consideration during the mounting check! For all modes, either all or only the current work step is checked; this is controlled using station parameter code no. 7203 [▶▶▶]! Advice: If the value [1] or [3] is passed (multiple panel), slave serial numbers are assigned to master serial numbers on the basis of the same panel position; i.e., slave panel position 1 is checked against master panel position 1; the number of panels must be identical for slave and master!

Tab. 26-17 Parameter (in) »trAssignSerialNumberMergeAndUploadState«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.8.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
 If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station
-108	Failure	Station may not add serial numbers
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-121	Failure	Part not found
-130	Failure	Work order not found

-133	Failure	No active work order found for the station
-134	Failure	Default value for maximum order quantity has been reached
-137	Failure	No work order could be created
-138	Failure	Order state is invalid
-139	Failure	BOM version does not match the version from the work order
-151	Failure	The "serialNumberRef" parameter must be specified
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-205	Failure	Serial number belongs to a different work order
-219	Failure	Serial number is not finalized
-220	Failure	Mounting (merge) is invalid
-221	Failure	The slave serial no. is not released from previous process
-223	Failure	There are multiple valid serial numbers for the merge
-227	Failure	The slave serial no. has been booked from a previous process as SCRAP
-228	Failure	Slave serial number is locked
-231	Failure	Booking of the serial number failed
-232	Failure	Failure during product merging (merge)
-234	Failure	Number of panels (master) and slave serial nos. are not equal
-239	Failure	Serial number array is empty
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-244	Failure	Serial number is in the array more than once
-355	Failure	Production order is part of a work order cluster
-364	Failure	Serial number must be ejected
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.9

trAssignSerialnumberToOrderCluster

26.9.1

Function description

This function is used to assign multiple serial numbers to a multiple panel manufactured in the work order cluster. Each panel position is assigned a certain work order; the assignment – i.e. which work orders are manufactured at which position – takes place when the work order cluster is created. Serial numbers **for all** panel positions of the work order cluster must always be passed together in one operation; step-by-step assignment is not possible. This also applies if the same work order is to be manufactured at multiple panel positions.

Example: If a work order cluster consists of a multiple panel with four positions (i.e. a four-way panel), four serial numbers must be passed using this function.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

When using this function, note the following basic principles:

- The serial numbers are assigned on the basis of the position number in accordance with the work order cluster definition; the position number is therefore conceptually identical to the serial number position and, for this reason, is passed using the key `SERIAL_NUMBER_POS`.
- If the passed position number is not known, the function terminates with a failure.
- Serial numbers for all ACTIVE panel positions of the work order cluster must always be passed together in one operation.
- Serial numbers must be assigned to phantom work orders on the basis of the IM-SApi function »trAssignSerialnumberForProductOrWorkOrder« [▶▶▶].
- If necessary, the work orders of the work order cluster can be activated directly (key `ACTIVATE_WORKORDER`); the function »trActivateOrderCluster« [▶▶▶] can be used for subsequent activation.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trAssignSerialnumberToOrderCluster
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] orderClusterSerialnumberHeader
    String key
    String value
  in String[] orderClusterSerialnumberAssignKeys
  in String[] orderClusterSerialnumberAssignValues
  out String[] orderClusterSerialnumberAssignResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `orderClusterSerialnumberHeader`, `orderClusterSerialnumberAssignKeys` and `orderClusterSerialnumberAssignValues` as well as (out) `orderClusterSerialnumberAssignResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input values for work order cluster to be used

Key	Explanation		IMSApi data type
ACTIVATE_WORKORDER	"Work order activation" mode for work order clusters [0; 1]: 1 = All work orders in the work order cluster are activated automatically when the serial numbers are assigned (FALSE). 0 = Work orders are NOT activated automatically (TRUE). Advice: If the key is not passed, the function is executed without work order activation (mode [0]). If work order activation is active (mode [1]), the PCB orientation of the current work step must be passed (see key <code>PROCESS_LAYER</code>)!	☐	boolean
CLUSTER_NAME	Unique name for the work order cluster	☒	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: The PCB orientation only needs to be passed if work order activation is used (see key <code>ACTIVATE_WORKORDER</code>)!	☐	int
SERIAL_NUMBER_REF	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Advice for panel labels: If a serial number is specified that is NOT contained in the "Key Value Pair" structure <code>orderClusterSerialnumberAssignKeys</code>, this serial number is stored as a label serial number (i.e. a panel label). Label serial numbers are always assigned to the work order of the panel position that was passed first!	☒	String
SERIAL_NUMBER_REF_POS	Position of the serial number of the first single panel or unit-array Advice for panel labels: If <code>SERIAL_NUMBER_REF</code> is a label serial number, the value [0] must be used here!	☒	String

Tab. 26-18 "Key Value Pair": `orderClusterSerialnumberHeader`

Input values for serial numbers to be assigned

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: In the context of "work order clusters", this is the position number of the work order on the multiple panel. This entry is used to ascertain the production order to which the serial number passed using SERIAL_NUMBER is to be assigned!	☒ String
WORKORDER_NUMBER	Production order number Advice: Specification of the order number is optional and is used for checking purposes only. If the work order does not match the work order ascertained on the basis of the position number (SERIAL_NUMBER_POS), the function terminates with a failure!	☐ String

Tab. 26-19 "Key Value Pair": orderClusterSerialnumberAssignKeys

26.9.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
orderClusterSerialnumberHeader	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterSerialnumberHeader table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": orderClusterSerialnumberHeader table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
orderClusterSerialnumberAssignKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>orderClusterSerialnumberAssignValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": orderClusterSerialnumberAssignKeys table. Advice: Only the parameters passed here are processed by the function!
orderClusterSerialnumberAssignValues	Values for the parameters selected by means of <code>orderClusterSerialnumberAssignKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>orderClusterSerialnumberAssignKeys</code> ! Refer to the "Key Value Pair": orderClusterSerialnumberAssignKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-20 Parameter (in) »trAssignSerialnumberToOrderCluster«

Parameter (out)

Parameter name	Explanation
orderClusterSerialnumberAssignResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>orderClusterSerialnumberAssignKeys</code> and <code>orderClusterSerialnumberAssignValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>orderClusterSerialnumberAssignKeys</code> !

Tab. 26-21 Parameter (out) »trAssignSerialnumberToOrderCluster«

26.9.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-108	Failure	Station may not add serial numbers
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-201	Failure	Serial number is not active
-202	Failure	Serial number is locked
-205	Failure	Serial number belongs to a different work order
-206	Failure	Serial number was already assigned to this work order

-221	Failure	The slave serial no. is not released from previous process
-235	Failure	New serial number could not be created
-237	Failure	Serial number belongs to a different station
-239	Failure	Serial number array is empty
-244	Failure	Serial number is in the array more than once
-290	Failure	Serial number is on the SCRAP work step
-291	Failure	Serial number is not yet booked
-296	Failure	Serial number is on the FAIL work step
-299	Failure	Serial number is already assigned to a repair order
-301	Failure	Serial number is mounted
-358	Failure	Serial number position does not match work order cluster
-360	Failure	Serial number position -> Number does not match cluster size
-361	Failure	Work order cluster -> Position is invalid for work order
-362	Failure	Reference serial no. must belong to phantom work order
-1122	Failure	Work order cluster not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.10

trChangePoolSerialNumber

26.10.1

Function description

This function makes it possible to subsequently adapt the individual information text to serial numbers that are already in a serial number pool.



Advice

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trChangePoolSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in String[] poolSerialNumberKeys
in String[] poolSerialNumberValues
out String[] poolSerialNumberResults
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `poolSerialNumberKeys` and `poolSerialNumberValues` as well as (out) `poolSerialNumberResults`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
INFO	Information text	☐	String
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String

Tab. 26-22 "Key Value Pair": `poolSerialNumberKeys`

26.10.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool- Name	Name of the sno. pool (pool name)
poolSerialNumber- Keys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>poolSerialNumberValues</code> parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": poolSerialNumberKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!
poolSerialNumber- Values	Values for the parameters selected by means of <code>poolSerialNumberKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>poolSerialNumberKeys</code> ! Refer to the corresponding table for attribute category "Key Value Pair": "poolSerialNumberKeys" to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 26-23 Parameter (in) »trChangePoolSerialNumber«

Parameter (out)

Parameter name	Explanation
poolSerialNumber- Results	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>poolSerialNumberKeys</code> and <code>poolSerialNumberValues</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>poolSerialNumberKeys</code> !

Tab. 26-24 Parameter (out) »trChangePoolSerialNumber«

26.10.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-287	Failure	Serial number pool does not exist
-2070	Failure	IMSApi key was passed multiple times to the function

26.11 trChangeSerialNumberPool

26.11.1 Function description

This function makes it possible to subsequently edit the header data of a serial number pool; this way the pool description can be modified, for example.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trChangeSerialNumberPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in KeyValue[] serialNumberPoolData
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) serialNumberPoolData). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐.

Key	Explanation	IMSApi data type
Info	Individual additional information	☐ String
POOL_DESC	Description of the serial number pool	☐ String

Tab. 26-25 "Key Value Pair": *serialNumberPoolData*

26.11.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool-Name	Name of the sno. pool (pool name)
serialNumberPool-Data	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <i>serialNumberPoolData</i> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <i>serialNumberPoolData</i> table above to see whether special rules apply during the input of values (e.g. use of wildcards)!

Tab. 26-26 Parameter (in) »trChangeSerialNumberPool«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.11.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-287	Failure	Serial number pool does not exist
-2070	Failure	IMSApi key was passed multiple times to the function

26.12

trChangeWorkOrder

26.12.1

Function description

This function activates a new (unknown) work order on a complete line or on a station (work step). By default, the oldest work order (according to "Planned start day") is used; further details on the functionality can be found in the description of the input parameter `lineFlag`.

**Advice**

Only unbooked, released work orders (state = opened) can be activated using this function!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trChangeWorkOrder
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int lineFlag
  in String[] workorderResultKeys
  out String[] workorderResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `workorderResultKeys` and (out) `workorderResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Definition of the output parameters for the found order data

Key	Explanation	IMSApi data type
ATTRIBUTE_1	"Attribute 1" free text box from the part master; e.g., H for high, L for low	☐ String
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	☐ String
CUSTOMER_NAME	Customer name in the iTAC system	☐ String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
QUANTITY	Quantity of production order	☐ double
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: State "F" is always output here, since this function only takes into account released work orders!	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-27 "Key Value Pair": *workorderResultKeys*

26.12.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
lineFlag	"Work order activation" modes [0; 1; 2; 3]: 0 = The work order is activated on the entire line. 10 = The work order is activated on the entire line and the preceding work step is interrupted. 20 = The work order is activated on the entire line and the preceding work step is finished. 1 = The work order is activated only on the work step of this station and therefore also on the parallel stations according to the ERP group. 11 = The work order is activated only on the work step of this station and the preceding work step is interrupted. 21 = The work order is activated only on the work step of this station and the preceding work step is finished. Advice: Additionally, with modes [0] and [1] only work orders without bookings are used on all work steps! 2 = The work order is activated only on the work step of this station and therefore also on the parallel stations according to the ERP group (see restriction in advice below). 12 = The work order is activated only on the work step of this station and the preceding work step is interrupted. 22 = The work order is activated only on the work step of this station and the preceding work step is finished. 3 = The work order is activated only on this station (see restriction in advice below). 13 = Activate the work order on this station and interrupt the preceding work step. 23 = Activate the work order on this station and finish the preceding work step. Advice: Additionally, with modes [2] and [3] only work steps without bookings are used. If the station is present in multiple work steps of a work order, the first unbooked work step is used!
workorderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workorderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) workorderResultValues!

Tab. 26-28 Parameter (in) »trChangeWorkOrder«

Parameter (out)

Parameter name	Explanation
workorderResultValues	Depending on parameter (in) workorderResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) workorderResultKeys!

Tab. 26-29 Parameter (out) »trChangeWorkOrder«

26.12.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-108	Failure	Station may not add serial numbers
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-130	Failure	Work order not found
-355	Failure	Production order is part of a work order cluster
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2073	Failure	More than one work order is active on the station

26.13

trCheckMergedPartsForSerialNumber

26.13.1

Function description

This function checks whether all necessary components (semifinished products) were mounted in the finished product. In the array returned, either all or only the missing components can be output (for further information, see key `ALL_MERGED_PARTS_VISIBLE`). Components are returned according to their mounting place, i.e. components that are present multiple times are also output separately.

**Advice**

This API function is influenced by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 24010 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trCheckMergedPartsForSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mergedPartsFilters
    String key
    String value
in String[] mergePartsResultKeys
out String[] mergePartsResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `mergedPartsFilters` and `mergePartsResultKeys` as well as (out) `mergePartsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Definition of the function parameters

Key	Explanation		IMSApi data type
ALL_MERGED_PARTS_VISIBLE	Determines which components are output [0; 1]: 0 = All components 1 = Only the components that are not yet mounted	☐	int
MULTIPLE_BOARD	If the finished product (SERIAL_NUMBER) is part of a multiple panel, this specifies whether only the passed panel (acc. to SERIAL_NUMBER_POS) or all panels are taken into consideration [0; 1]: 0 = only the passed panel (FALSE) 1 = all panels (TRUE)	☐	boolean
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String

Tab. 26-30 "Key Value Pair": *mergedPartsFilters*

Definition of the output parameters for the found components

Key	Explanation		IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐	String
PART_ID	Internal identification number of the part (semifinished product)	☐	long
PART_NUMBER	Part number of the semifinished product in the iTAC system	☐	String
QUANTITY_BOM	Number of semifinished products in the parts list	☐	double
QUANTITY_MERGED	Number of semifinished products already mounted in the finished product	☐	double
SERIAL_NUMBER	Serial number of the respective panel (queried finished product) Advice: If the value [0] is passed for the key MULTIPLE_BOARD, the serial number passed using SERIAL_NUMBER is always output here (one result record). If [1] is passed for MULTIPLE_BOARD, the multiple panel is resolved and the respective single panel serial numbers are output here (in separate records)!	☐	String
SERIAL_NUMBER_POS	Position of the respective single panel in the unit-array (see Advice for the key SERIAL_NUMBER)	☐	String

Tab. 26-31 "Key Value Pair": *mergePartsResultKeys*

26.13.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: The station number must be present in the work plan of the queried serial number; the function will otherwise return a failure (error code = -216)!
mergedPartsFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": mergedPartsFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": mergedPartsFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
mergePartsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mergePartsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mergePartsResultValues</code> !

Tab. 26-32 Parameter (in) »trCheckMergedPartsForSerialNumber«

Parameter (out)

Parameter name	Explanation
mergePartsResultValues	Depending on parameter (in) <code>mergePartsResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mergePartsResultKeys</code> !

Tab. 26-33 Parameter (out) »trCheckMergedPartsForSerialNumber«

26.13.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-216	Failure	Station is invalid for the serial number
-224	Failure	Serial number is not finalized
-2070	Failure	IMSApi key was passed multiple times to the function

26.14

trCheckMergedPartsForSerialNumberComplete

26.14.1

Function description

This function checks whether all necessary components (semifinished products) were mounted in the finished product. In the array returned, either all or only the missing components can be output (see also parameter `allMergedPartsVisible`). Components are returned according to their mounting place, i.e. components that are present multiple times are also output separately.

**Advice**

This API function is influenced by site parameter code no. 24010 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation! The function »trCheckMergedPartsForSerialNumber« [▶▶▶] should be used for new applications; there the input parameters are also passed as a "Key Value Pair" structure!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trCheckMergedPartsForSerialNumberComplete
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in int serialNumberPos
in int allMergedPartsVisible
in int checkMultipleBoard
in String[] mergePartsResultKeys
out String[] mergePartsResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `mergePartsResultKeys` and (out) `mergePartsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped! If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
COMP_NAME	Mounting place conformant with the iTAC BOM	☐ String
PART_ID	Internal identification number of the part (semifinished product)	☐ long
PART_NUMBER	Part number of the semifinished product in the iTAC system	☐ String
QUANTITY_BOM	Number of semifinished products in the parts list	☐ double
QUANTITY_MERGED	Number of semifinished products already mounted in the finished product	☐ double
SERIAL_NUMBER	Serial no. of the respective panel (queried finished product) Advice: If the value [0] is passed for the input parameter <code>checkMultipleBoard</code> , the serial number passed using <code>serialNumber</code> is always output here (one result record). If [1] is passed for <code>checkMultipleBoard</code> , the multiple panel is resolved and the respective single panel serial numbers are output here (in separate records)!	☐ String
SERIAL_NUMBER_POS	Position of the respective single panel in the unit-array (see Advice for the key <code>SERIAL_NUMBER</code>)	☐ String

Tab. 26-34 "Key Value Pair": `mergePartsResultKeys`

26.14.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
<code>sessionContext</code>	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
<code>stationNumber</code>	Station number of the work station in the iTAC system
<code>serialNumber</code>	Serial no. of the panel
<code>serialNumberPos</code>	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
<code>allMergedParts-Visible</code>	Determines which components are output [0; 1]: 0 = Only the components that are not yet mounted 1 = All components
<code>checkMultiple-Board</code>	If the finished product (<code>serialNumber</code>) is part of a multiple panel, this specifies whether only the passed panels (<code>serialNumber-Pos</code>) or all panels are checked [0; 1]: 0 = only the passed panel 1 = all panels
<code>mergePartsResult-Keys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>mergePartsResultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>mergePartsResultValues</code> !

Tab. 26-35 Parameter (in) »trCheckMergedPartsForSerialNumberComplete«

Parameter (out)

Parameter name	Explanation
mergePartsResult-Values	Depending on parameter (in) <code>mergePartsResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>mergePartsResultKeys</code> !

Tab. 26-36 Parameter (out) »trCheckMergedPartsForSerialNumberComplete«

26.14.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-120	Failure	Work plan not found
-216	Failure	Station is invalid for the serial number
-224	Failure	Serial number is not finalized
-2070	Failure	IMSApi key was passed multiple times to the function

26.15

trCheckSerialNumberState

26.15.1

Function description

This function checks the processing state of a serial number or a complete multiple printed panel separately for all single panels it contains. This ensures that the passed serial number is permissible for subsequent processing; a check is performed here in particular to determine whether the state of progress is OK.

Depending on station parameter code no. 7203 [▶▶▶], the system checks whether only the immediately preceding work step or all previous work steps have been booked and whether the current serial number state is "PASS" on the previous work step(s). In addition to this general specification, additional settings are to be performed for the individual application types – e.g., for working with scrapped serial numbers – that can only be defined in close consultation with the iTAC employee responsible for the project.



Advice

In the array returned, the results of the check are output for each record. If one or more serial numbers are faulty, a corresponding reason for error is returned with the `ERROR_CODE` key. In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

With activation of station parameter code no. 13300 [▶▶▶] or 13310 [▶▶▶], a check is also performed to determine whether the work order to which the serial number is assigned matches the work order currently active on the station.



Advice

This API function is influenced by station parameter code nos. 7203 [▶▶▶], 7204 [▶▶▶], 7209 [▶▶▶], 8310 [▶▶▶], 13010 [▶▶▶], 13011 [▶▶▶], 13300 [▶▶▶], 13310 [▶▶▶], 16001 [▶▶▶], site parameter code no. 7070 [▶▶▶] as well as server parameter code no. 2032 [▶▶▶]. In addition, this function is controlled by the station settings for "Upload Settings". Further information on this topic can be found in the reference section of this documentation!

This API function is intended for checking the state of one or all single panels of a printed circuit board (for further information, see parameter `checkMultiBoard`). In general, only the last work step – if applicable, the last work step obligated to supply confirmation – is checked. If other work steps are to be included in the check (code no. 7203 [▶▶▶] must be active for this), serial number data is then only returned if the state bookings of all work steps or of all work steps obligated to supply confirmation are complete and have the value "PASS". Otherwise, only the corresponding error code is returned.

When checking multiple panels with scrap parts, special note is to be taken of server parameter code no. 2032 [▶▶▶]; depending on the settings, the function returns different values for the `ERROR_CODE` and `SERIAL_NUMBER_STATE` keys as well as the functional output value, although it has the same booking history.



Example

Testing a multiple panel in the 3rd work step (here a double panel for simplicity's sake): panel 2 was booked with SCRAP in the 2nd work step; panel 1 had the PASS state.

"Simplified" output if **server parameter 2032 = "N"**:

functional output value = **6** (At least one panel has SCRAP state)

```
.serialNumberStateResultValues[0].ERROR_CODE=0
.serialNumberStateResultValues[0].SERIAL_NUMBER=SNO_01
.serialNumberStateResultValues[0].SERIAL_NUMBER_POS=1
.serialNumberStateResultValues[0].SERIAL_NUMBER_STATE=0
.serialNumberStateResultValues[1].ERROR_CODE=207
.serialNumberStateResultValues[1].SERIAL_NUMBER=SNO_02
.serialNumberStateResultValues[1].SERIAL_NUMBER_POS=2
.serialNumberStateResultValues[1].SERIAL_NUMBER_STATE=2
```

"Simplified" output if **server parameters 2032 = "J"**:

functional output value = **5** (not all records were successfully processed)

```
.serialNumberStateResultValues[0].ERROR_CODE=0
.serialNumberStateResultValues[0].SERIAL_NUMBER=SNO_01
.serialNumberStateResultValues[0].SERIAL_NUMBER_POS=1
.serialNumberStateResultValues[0].SERIAL_NUMBER_STATE=0
.serialNumberStateResultValues[1].ERROR_CODE=-201
.serialNumberStateResultValues[1].SERIAL_NUMBER=SNO_02
.serialNumberStateResultValues[1].SERIAL_NUMBER_POS=2
.serialNumberStateResultValues[1].SERIAL_NUMBER_STATE=-1
```

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trCheckSerialNumberState
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in int checkMultiBoard
in String serialNumber
in String serialNumberPos
in String[] serialNumberStateResultKeys
out String[] serialNumberStateResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberStateResultKeys and (out) serialNumberStateResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
BOOKING_VALID	"Invalid" designation [0; 1]: 0 = Serial number is in the "Invalid" state (FALSE) 1 = Serial number is NOT in the "Invalid" state (TRUE) Advice: <i>If the serial number has the "Invalid" state, the value [1] (rework) is output by default for key SERIAL_NUMBER_STATE to ensure the regular process interlock!</i>	☐	boolean
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
ERROR_CODE	Error code that describes the result of the check (for further information, see section »Output value« [▶▶▶]) Advice: <i>If one or more serial numbers are faulty, this key returns a corresponding error code (reason for error); in this case, the function itself outputs a general warning (see error code 5)!</i>	☐	int
IN_PROCESS	"In process" designation [0; 1]: 0 = Serial number is NOT in the "In process" state (FALSE) 1 = Serial number is in the "In process" state (TRUE) Advice: <i>If the serial number has the "In process" state, the value [1] (rework) is output by default for key SERIAL_NUMBER_STATE to ensure the regular process interlock!</i>	☐	boolean
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: <i>This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!</i>	☐	boolean
IS_MIRRORED_BOARD	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE) Advice: <i>If the "double-sided panel" entry is selected, the setup information data of the component side and soldering side are combined and considered as a whole for this work order/work step!</i>	☐	boolean

Key	Explanation	IMSApi data type
IS_MULTI_PANEL	"Panel type" designation [0; 1]: 0 = Single panel 1 = Multiple panel Advice: <i>If the serial number is locked or archived, the value [-1] is returned for this key; the panel type is not checked!</i>	☐ int
LOCK_CLASS	Name of the lock class	☐ String
LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	☐ int
LOCKGROUP_NAME	Name of the lock group	☐ String
LOOP_COUNTER	Number of bookings against this serial number and this work step (stations of the associated ERP group) Advice: <i>Only bookings of stations of the type "tester" are counted; bookings of repair and diagnosis stations are not counted! Depending on the setting for station parameter code no. 13010 [▶▶▶], either the "loopCounter" of the previous (Y) or of the current work step (N) is output here.</i>	☐ int
PHANTOM_WOR-KORDER_NUMBER	Number of the (parent) production order for the phantom work order cluster	☐ String
RETRY_COUNTER	Display of the tests that have currently been performed at the work step (test retry counter) Background: All bookings of a work step after a diagnosis station are counted (e.g. multiple bookings at the ICT); a booking at the parallel diagnosis station resets the "counter" to zero. The maximum value is specified for the respective work step in the work plan (see the field "No. of tests" there).	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_STATE	Process state of the serial number [-1; 0; 1; 2]: -1 = no state booking present 0 = pass 1 = fail 2 = scrap Advice: <i>The process states [3] "In process" and [4] "Invalid" are intermediate conditions of a work step and are taken into consideration only indirectly. Indirect means that the state [1] is output in both cases here to ensure the regular process interlock. In this context, observe the BOOKING_VALID and IN_PROCESS keys!</i>	☐ int
WORKORDER_NUMBER	Production order number	☐ String

Tab. 26-37 "Key Value Pair": serialNumberStateResultKeys

26.15.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
checkMultiBoard	If the serialNumber is part of a multiple panel, this parameter determines whether only the passed or all panels are checked [0; 1; 2; 3]: 0 = Single panel; only the passed serial number is checked (only work steps with obligatory confirmation) 1 = Multiple panel; all other serial numbers of the multiple panel are ascertained and checked on the basis of the passed serial number (only work steps with obligatory confirmation) 2 = Single panel; only the passed serial number is checked (all work steps) 3 = Multiple panel; all other serial numbers are also ascertained and checked on the basis of the passed serial number (all work steps) Advice regarding "mode differentiation": The difference between mode 0 and 2 and between 1 and 3 is that in mode 2 and 3 the state of work steps without obligatory confirmation is also taken into consideration during the check! For all modes, either all or only the current work step is checked; this is controlled using station parameter code no. 7203 [▶▶▶]! Advice: For mode 2 and 3, PASS bookings on work steps without obligatory confirmation are not taken into consideration; in this case, the key SERIAL_NUMBER_STATE returns the value -1!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
serialNumberStateResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberStateResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) serialNumberStateResultValues!

Tab. 26-38 Parameter (in) »trCheckSerialNumberState«

Parameter (out)

Parameter name	Explanation
serialNumberSta- teResultValues	Depending on parameter (in) <code>serialNumberStateResult- Keys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>serialNumberStateResultKeys</code> !

Tab. 26-39 Parameter (out) »trCheckSerialNumberState«

26.15.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
212	Warning	Serial number is valid for this station, but it has not yet been booked
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
		Advice: The value 208 is only output if station parameter code no. 7203 [▶▶▶] is active (cf. error code 201)!
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
		Advice: The value 207 is only output if station parameter code no. 7203 [▶▶▶] is active (cf. error code 204)!
206	Warning	PASS state but number of permitted tests reached
205	Warning	FAIL state but number of permitted tests reached
204	Warning	Serial number state at the previous work step: SCRAP
		Advice: The value 204 is only output if station parameter code no. 7203 [▶▶▶] is inactive (cf. error code 207)! The value 204 is also output if the serial number is inactive!
203	Warning	Serial number is invalid for this station -> no booking at at least one of the previous work steps

		Advice: The value 203 is only output if station parameter code no. 7203 [▶▶▶] is active (cf. error code 202)! If the value 203 is output, the parameter SERIAL_NUMBER_STATE (see the table "Key Value Pair": serialNumberStateResultKeys) can be used to determine which serial number(s) of the multiple panel is/are invalid!
202	Warning	Serial number is invalid for this station -> no booking at the previous work step
		Advice: The value 202 is only output if station parameter code no. 7203 [▶▶▶] is inactive (cf. error code 203)!
201	Warning	Serial number state at the previous work step: FAIL
		Advice: The value 201 is only output if station parameter code no. 7203 [▶▶▶] is inactive (cf. error code 208)!
6	Warning	At least one panel position has SCRAP state
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-124	Failure	Part does not match the work order part
-128	Failure	Work step not found
-188	Failure	Invalid value for "checkMultiboard" [0; 1]
-201	Failure	Serial number is not active
-202	Failure	Serial number is locked

-213	Failure	Maximum number of bookings reached
-301	Failure	Serial number is mounted
-543	Failure	Container is expired
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.16

trCheckUserSkill

26.16.1

Function description

This function is used to check whether the user is sufficiently qualified for a given work step. Here, the qualification level stored on the work step ("User skill" field) is checked against the qualification profile of the user.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 4571 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

This function should NO LONGER be used for new installations because the edit options have been extended with release 9.00.00 (for further information, see the new function »trCheckUserSkills« [▶▶▶])!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trCheckUserSkill
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String userId
in KeyValue[] checkUserSkillFilter
    String key
    String value
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameter (in) `checkUserSkillFilter`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Parameters for **work-order** and **work-plan** ascertainment:

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER or the WORKORDER_NUMBER must be passed! If order and serial number are being passed, the serial number must be part of the WO!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the SERIAL_NUMBER must be unique!	☐ String
WORKORDER_NUMBER	Production order number Advice: Even though this key is not identified as a mandatory field, either the SERIAL_NUMBER or the WORKORDER_NUMBER must be passed! If order and serial number are being passed, the serial number must be part of the WO!	☐ String

Tab. 26-40 "Key Value Pair": *checkUserSkillFilter*

26.16.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
userId	Login name of the user (user identification) Advice: Pay attention to upper- and lower-case letters!
checkUserSkillFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": checkUserSkillFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": checkUserSkillFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-41 Parameter (in) »trCheckUserSkill«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.16.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-128	Failure	Work step not found
-130	Failure	Work order not found
-185	Failure	Invalid value for the process layer
-205	Failure	Serial number belongs to a different work order
-240	Failure	No work step found for work order, orientation of the PCB and station
-338	Failure	User -> Skill not assigned
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.17

trCheckUserSkills

26.17.1

Function description

This function is used to check whether the user is sufficiently qualified for a given work step. Here, the qualification level stored on the work step ("User skill" field) is checked against the qualification profile of the user. If the check shows that one or more skills are missing, these skills are output in the array returned.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶] as well as by site parameter code no. 4571 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!
*For new installations, it is **essential** to use this API function because its functionality has been extended compared to the »trCheckUserSkill« [▶▶▶] function!*

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trCheckUserSkills
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] checkUserSkillsFilter
    String key
    String value
  in String[] checkUserSkillsResultKeys
  out String[] checkUserSkillsResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `checkUserSkillsFilter` and `checkUserSkillsResultKeys` as well as (out) `checkUserSkillsResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐.

Searching for specific parameter values

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☒ int
SERIAL_NUMBER	Serial number of the panel Advice: If the key is not passed, a valid work order (WORKORDER_NUMBER) must be passed!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!	☐ String
USER_NAME	Login name of the user (user identification) Advice: Pay attention to upper- and lower-case letters; alternatively, the USER_TOKEN key can also be used to identify the user!	☐ String
USER_TOKEN	Electronic key for identification of a certain user (user token) Advice: This key can be used as an alternative for identifying the user using USER_NAME!	☐ String
WORKORDER_NUMBER	Number of a production order Advice: If the key is not passed, a valid serial number (SERIAL_NUMBER) must be passed!	☐ String

Tab. 26-42 "Key Value Pair": checkUserSkillsFilter

Definition of the output parameters for the found data

Key	Explanation	IMSApi data type
SKILL_DESC	Description of the user skill	☐ String
SKILL_LEVEL	Level of the user skill	☐ String
WORKORDER_NUMBER	Number of a production order	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-43 "Key Value Pair": checkUserSkillsResultKeys

26.17.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
checkUserSkills-Filter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": checkUserSkillsFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": checkUserSkillsFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
checkUserSkillsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": checkUserSkillsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>attributeResultValues</code> !

Tab. 26-44 Parameter (in) »trCheckUserSkills«

Parameter (out)

Parameter name	Explanation
attributeResultValues	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>checkUserSkillsFilter</code> and <code>checkUserSkillsResultKeys</code> . Advice: The data is output in the order in which it was entered by means of parameter (in) <code>checkUserSkillsResultKeys</code> !

Tab. 26-45 Parameter (out) »trCheckUserSkills«

26.17.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-120	Failure	Work plan not found
-130	Failure	Work order not found
-185	Failure	Invalid value for the process layer
-205	Failure	Serial number belongs to a different work order
-240	Failure	No work step found for work order, orientation of the PCB and station

-338	Failure	User -> Skill not assigned
-2070	Failure	IMSApi key was passed multiple times to the function

26.18 trCreateSerialNumberPool

26.18.1 Function description

This function creates a new serial number pool in the system; if necessary, a modification order can be directly assigned.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trCreateSerialNumberPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in String serialNumberPoolDesc
in String changeOrderNumber
in String infoTxt
in int serialNumberPoolType

```

26.18.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool-Name	Name of the serial number pool
serialNumberPool-Desc	Description of the serial number pool
changeOrderNumber	Number of a modification order Advice: The input is optional; if [-1] or [empty] is passed, the field is not evaluated!
infoTxt	Individual additional information
serialNumberPool-Type	Type of the serial number pool [0; 1; 2; 3]: 0 = no typecasting (unknown) 1 = serial number pool for modification orders 2 = serial number pool for delivery slips 3 = serial number pool for repair orders Advice: If the value [-1] is passed, the default type is used (currently "1")!

Tab. 26-46 Parameter (in) »trCreateSerialNumberPool«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.18.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-130	Failure	Work order not found
-196	Failure	A parameter is invalid
-283	Failure	Serial number pool already exists
-284	Failure	Creation of the serial number pool failed
-302	Failure	Work order type is not valid
-303	Failure	Modification order is already assigned to a sno. pool

26.19 trDeleteSerialNumberPool

26.19.1 Function description

This function removes a serial number pool from the system; this only applies to serial number pools with the "Initial" or "Sno. pool prepared" state.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trDeleteSerialNumberPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
```

26.19.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool-Name	Name of the serial number pool

Tab. 26-47 Parameter (in) »trDeleteSerialNumberPool«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.19.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-286	Failure	Deletion of the serial number pool failed
-287	Failure	Serial number pool does not exist
-288	Failure	Deletion of the serial number pool not possible

26.20

trGetCapturedRepairHistoryData

26.20.1

Function description

This function can be used to output failure information stored in the system for a certain unit over a certain period under review. In particular, the intent here is to relate the failure notes (ICT failures) present in the system to the failure types/sources booked on a diagnosis station for these.



Advice

Failure data without assigned failure notes are not taken into account by the function. This API function is influenced by station parameter code nos. [8310 \[▶▶▶\]](#) and [13000 \[▶▶▶\]](#). Further information on this topic can be found in the documentation for the ADM module!

Based on the station number (`stationNumber`) and layer specification (`processLayer`), the function ascertains the work step to be considered. With the work step, all affected testers and thereby the corresponding failure information can be found. The output of the function can be further restricted by ascertaining the unit using the serial number/work order. In addition to the output of all failure information on the part number, the output can be restricted with respect to the current BOM version or parts of type "printed circuit board" (see station parameter code no. [13000 \[▶▶▶\]](#)).

The function outputs two lists that are referenced together – the failure note list and the failure data list – as the result. The connection must be established in advance via the repair station (PR-Client). If failure notes are booked in the system without a failure assignment having taken place on the repair station, the `failureDataResultKey` "Key Value Pair" structure remains empty.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetCapturedRepairHistoryData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in KeyValue[] repairHistoryFilter
    String key
    String value
in String[] failureDataResultKey
in String[] failureSlipDataResultKey
out String[] failureDataResultValues
out String[] failureSlipDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `repairHistoryFilter`, `failureDataResultKey` and `failureSlipDataResultKey` as well as (out) `failureDataResultValues` and `failureSlipDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
FUNC_MODE	"Data ascertainment" function mode [0; 1; 2]: 0 = All work steps 1 = Works steps matching stationNumber 2 = Work steps matching stationNumber and processLayer Advice: If this key is not passed along, the [2] function mode is used by default!	☐ int
SERIAL_NUMBER	Serial number of the panel Advice: Even though this key is not identified as a mandatory field, the SERIAL_NUMBER and/or the WORKORDER_NUMBER must be passed (ascertainment of unit); if both keys are passed, the serial number must belong to the work order!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: This key is only evaluated if a serial number (SERIAL_NUMBER) is passed and it is non-unique!	☐ String
TEST_STEP_ID	Internal identification number of the test step	☐ long
WORKORDER_NUMBER	Number of a production order Advice: Even though this key is not identified as a mandatory field, the SERIAL_NUMBER and/or the WORKORDER_NUMBER must be passed (ascertainment of unit); if both keys are passed, the serial number must belong to the work order!	☐ String

Tab. 26-48 "Key Value Pair": repairHistoryFilter

Failure data

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_CAUSE_INFO	Individual information about the failure cause Advice: This information can be passed in combination with the failure booking!	☐ String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_INFO	Individual information about the failure type Advice: This information can be passed in combination with the failure booking!	☐ String
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure data can be put into context with the failure note data (see parameter of the same name in the "Key Value Pair": failureSlipDataResultKeys failure note table)!	☐ long
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
STATION_REPAIR_NUMBER	Station no. of the work station (repair/diagnosis station) that performed the repair/failure booking	☐ String

Tab. 26-49 "Key Value Pair": failureDataResultKey

Failure note

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure note data can be put into context with the failure data (see parameter of the same name in the "Key Value Pair": failureDataResultKeys failure table)!	☐	long
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
TEST_STEP_NAME	Name of the test step Advice: Do not confuse this name with the measurement step name!	☐	String

Tab. 26-50 "Key Value Pair": failureSlipDataResultKey

26.20.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
repairHistoryFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": repairHistoryFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": repairHistoryFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
failureDataResultKey	List of the parameters (failure data) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureDataResultKey table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureDataResultValues !
failureSlipDataResultKey	List of the parameters (failure note) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": failureSlipDataResultKey table. Advice: Only the parameters passed here are output by the function; see parameter (out) failureSlipDataResultValues !

Tab. 26-51 Parameter (in) »trGetCapturedRepairHistoryData«

Parameter (out)

Parameter name	Explanation
failureDataResultValues	Depending on parameter (in) failureDataResultKey , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureDataResultKey !
failureSlipDataResultValues	Depending on parameter (in) failureSlipDataResultKey , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureSlipDataResultKey !

Tab. 26-52 Parameter (out) »trGetCapturedRepairHistoryData«

26.20.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-168	Failure	Invalid value for the function mode (control flag)
-205	Failure	Serial number belongs to a different work order
-240	Failure	No work step found for work order, orientation of the PCB and station
-2070	Failure	IMSApi key was passed multiple times to the function
-2110	Failure	Serial number position is invalid

26.21

trGetMergeHistory

26.21.1

Function description

With this function, dependent on the current integration level of the passed serial number, the mounted semifinished products are output in an array. In conjunction with this, both the direction and the integration level number can be specified for the resolution of the merge structure (identical to the function »trGetMergeParts« [▶▶▶]). In addition to this, semifinished products that have been removed again in the meantime can also be ascertained (history function).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetMergeHistory
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] mergedHistoryFilters
    String key
    String value
in String[] mergedHistoryResultKeys
out String[] mergedHistoryResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) mergedHistoryFilters and mergedHistoryResultKeys as well as (out) mergedHistoryResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
RESOLVE_DIRECTION	Depending on the current integration level of the passed serial number, defines the direction in which the merge structure is checked [0; 1; -1]: 0 = Only the subordinate integration levels are checked for mounted semifinished products. 1 = Only the parent integration levels are checked for mounted semifinished products. -1 = Both the subordinate as well as the parent integration levels are checked for mounted semifinished products. Advice: <i>How many integration levels are taken into account when resolving is defined with RESOLVE_LEVEL!</i>	☐	int
RESOLVE_LEVEL	Depending on the current integration level of the passed serial number, defines how many integration levels are to be taken into account when resolving [> 0; -1]: > 0 = Number of integration levels to be checked -1 = No restriction: All integration levels are taken into account Advice: <i>The direction in which the merge structure is resolved is defined with RESOLVE_DIRECTION!</i>	☐	int
RESOLVE_TYPE	Resolution type (history function); this determines which semifinished products are to be ascertained [0; 1; 2]: 0 = only currently mounted semifinished products 1 = only formerly mounted semifinished products 2 = currently and formerly mounted semifinished products	☐	int
SERIAL_NUMBER	Serial number of the panel Advice: <i>The merge structure is ascertained based on this serial number!</i>	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: <i>If this key is not used, the serial number must be unique!</i>	☐	String

Tab. 26-53 "Key Value Pair": mergedHistoryFilters

Definition of the output parameters for the found products

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_INFO	Individual BOM information (free text) Advice: <i>In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!</i>	☐	String

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	String
BOM_VERSION	BOM version from the iTAC system Advice: Here, this is the BOM version of the mounted semifinished product!	int
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	String
DATE_MERGED	Point in time mounting was performed (product merging); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
DATE_UNMERGED	Point in time removal was performed (product separation); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: This parameter is only output if the mode [1] or [2] has been selected for the RESOLVE_TYPE key (see table "Key Value Pair": mergedHistoryFilters)!	DateMs
LEVEL	Display of the integration level relative to the passed serial number [> 0; < 0]: > 0 = Number of integration levels downward < 0 = Number of integration levels upward Thus, subordinate integration levels are returned with values > 0; i.e., the value increases the deeper the merge structure is resolved. For parent integration levels, a level < 0 is returned; i.e., the highest integration level has the lowest value. Example: The integration levels should be resolved in all directions and over all integration levels; query with SN3: <pre> PROD_A (SN1) LEVEL = -2 -> PROD_B (SN2) LEVEL = -1 -> PROD_C (SN3) -> PROD_D (SN4) LEVEL = 1 -> PROD_E (SN5) LEVEL = 2 </pre> Advice: This key is influenced particularly by the settings for the keys RESOLVE_DIRECTION and RESOLVE_LEVEL (see table "Key Value Pair": mergedHistoryFilters)!	int
LOCK_CLASS	Name of the lock class	String
LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	int

Key	Explanation	IMSApi data type
LOCKGROUP_NAME	Name of the lock group	☐ String
PART_DESC	Part description in the iTAC system Advice: <i>In this case, this is the part description of the mounted semi-finished product!</i>	☐ String
PART_ID	Internal identification number of the part	☐ long
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the mounted semifinished product!</i>	☐ String
SCT_SOURCE_MANUFACTURER_CODE	Designator (code) of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_MANUFACTURER_NAME	Name of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_CODE	Designator (code) of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_NAME	Name of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the mounted semi-finished product!</i>	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_SCT_STATE	SCT serial number state [R; S; U]: R = Ready to ship S = Shipped U = Ready to ship change Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ char
SERIAL_PARENT_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the parent product – according to BOM structure!</i>	☐ String
SERIAL_PARENT_NUMBER_ID	Internal identification number of the serial number (SERIAL_PARENT_NUMBER)	☐ long
SERIAL_PARENT_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_SLAVE_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the subordinate product – according to BOM structure!</i>	☐ String
SERIAL_SLAVE_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Key	Explanation	IMSApi data type
SITE_NUMBER	Site number Advice: <i>In this case, this is the number of the site in which the semi-finished product was mounted!</i>	☐ String
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
UNMERGED	"Mounting history" designation [0; 1]: 0 = Semifinished product has no mounting history 1 = Semifinished product was removed in the past	☐ int
WORKORDER_NUMBER	Production order number Advice: <i>In this case, this is the production order with which the mounted semifinished product was manufactured!</i>	☐ String

Tab. 26-54 "Key Value Pair": *mergedHistoryResultKeys*

26.21.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
mergedHistoryFilters	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": mergedHistoryFilters table.
value	Value list for the parameters passed by means of key . Advice: <i>Refer to the "Key Value Pair": mergedHistoryFilters table above to see whether special rules apply during the input of values (e.g. use of wildcards)!</i>
mergedHistoryResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": mergedHistoryResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) mergedHistoryResultValues!</i>

Tab. 26-55 Parameter (in) »trGetMergeHistory«

Parameter (out)

Parameter name	Explanation
mergedHistoryResultValues	Depending on parameter (in) <i>mergedHistoryResultKeys</i> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) mergedHistoryResultKeys!</i>

Tab. 26-56 Parameter (out) »trGetMergeHistory«

26.21.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-214	Failure	Invalid value for "resolveDirection"
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.22

trGetMergeParts

26.22.1

Function description

With this function, dependent on the current integration level of the passed serial number, the mounted semifinished products are output in an array. In doing so, the direction as well as the integration level number can be specified for the resolution of the merge structure.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetMergeParts
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in int resolveDirection
in int resolveLevel
in String[] mergePartsResultKeys
out String[] mergePartsResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) mergePartsResultKeys and (out) mergePartsResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_INFO	Individual BOM information (free text) Advice: In the BOM module this parameter is displayed in the BOM header table in the "BOM revision" field!	☐ String

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	String
BOM_VERSION	BOM version from the iTAC system Advice: Here, this is the BOM version of the mounted semifinished product!	int
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	String
DATE_MERGED	Point in time mounting was performed (product merging); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
LEVEL	Display of the integration level relative to the passed serial number [> 0; < 0]: > 0 = Number of integration levels downward < 0 = Number of integration levels upward Thus, subordinate integration levels are returned with values > 0; i.e., the value increases the deeper the merge structure is resolved. For parent integration levels, a level < 0 is returned; i.e., the highest integration level has the lowest value. Example: The integration levels should be resolved in all directions and over all integration levels; query with SN3: <pre> PROD_A (SN1) LEVEL = -2 -> PROD_B (SN2) LEVEL = -1 -> PROD_C (SN3) -> PROD_D (SN4) LEVEL = 1 -> PROD_E (SN5) LEVEL = 2 </pre> Advice: This key is influenced in particular by the settings for the <code>resolveDirection</code> and <code>resolveLevel</code> input parameters!	int
LOCK_CLASS	Name of the lock class	String
LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	int
LOCKGROUP_NAME	Name of the lock group	String
PART_DESC	Part description in the iTAC system Advice: In this case, this is the part description of the mounted semifinished product!	String
PART_ID	Internal identification number of the part	long

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system Advice: <i>In this case, this is the part number of the mounted semifinished product!</i>	☐ String
SCT_SOURCE_MANUFACTURER_CODE	Designator (code) of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_MANUFACTURER_NAME	Name of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_CODE	Designator (code) of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_NAME	Name of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SERIAL_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the mounted semi-finished product!</i>	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_SCT_STATE	SCT serial number state [R; S; U]: R = Ready to ship S = Shipped U = Ready to ship change Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ char
SERIAL_PARENT_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the parent product – according to BOM structure!</i>	☐ String
SERIAL_PARENT_NUMBER_ID	Internal identification number of the serial number (SERIAL_PARENT_NUMBER)	☐ long
SERIAL_PARENT_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_SLAVE_NUMBER	Serial number of the panel Advice: <i>In this case, this is the serial number of the subordinate product – according to BOM structure!</i>	☐ String
SERIAL_SLAVE_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SITE_NUMBER	Site number Advice: <i>In this case, this is the number of the site in which the semi-finished product was mounted!</i>	☐ String
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number Advice: In this case, this is the production order with which the mounted semifinished product was manufactured!	☐ String

Tab. 26-57 "Key Value Pair": *mergePartsResultKeys*

26.22.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
resolveDirection	Depending on the current integration level of the passed serial number, defines the direction in which the merge structure is checked [0; 1; -1]: 0 = Only the subordinate integration levels are checked for mounted semifinished products. 1 = Only the parent integration levels are checked for mounted semifinished products. -1 = Both the subordinate as well as the parent integration levels are checked for mounted semifinished products. Advice: How many integration levels are taken into account when resolving is defined with <i>resolveLevel</i> !
resolveLevel	Depending on the current integration level of the passed serial number, defines how many integration levels are to be taken into account when resolving [> 0; -1]: > 0 = Number of integration levels to be checked -1 = No restriction: All integration levels are taken into account Advice: The direction in which the merge structure is resolved is defined with <i>resolveDirection</i> !
mergePartsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <i>mergePartsResultKeys</i> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>mergePartsResultValues</i> !

Tab. 26-58 Parameter (in) »trGetMergeParts«

Parameter (out)

Parameter name	Explanation
mergePartsResult-Values	Depending on parameter (in) mergePartsResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) mergePartsResultKeys!

Tab. 26-59 Parameter (out) »trGetMergeParts«

26.22.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-214	Failure	Invalid value for "resolveDirection"
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.23

trGetMultiStationSettings

26.23.1

Function description

This function ascertains the work orders active on a station together with the associated product versions. As a result, the function is especially suited for use in the "Work order cluster" context because here more than one work order is usually active on a station at the same time. If production does not take place in the work order cluster – i.e. only one work order is active on a station at any one time – the function »trGetStationSetting« [▶▶▶] should be used.

**Advice**

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetMultiStationSettings
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String[] stationMultiSettingsResultKeys
  out String[] stationMultiSettingsResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) stationMultiSettingsResultKeys as well as (out) stationMultiSettingsResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ATTRIBUTE_1	"Attribute 1" free text box from the part master; e.g., H for high, L for low	☐ String
ATTRIBUTE_2	"Attribute 2" free text box from the part master	☐ String

Key	Explanation		IMSApi data type
ATTRIBUTE_3	"Attribute 3" free text box from the part master	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	☐	String
CLUSTER_DESC	Description of the work order cluster	☐	String
CLUSTER_ITEM_STATE	State for the work order in the cluster [E; D]: E = Work order is active in the cluster (Enabled) D = Work order is NOT active in the cluster (Disabled)	☐	String
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CLUSTER_QUANTITY	Number of products to be manufactured in the cluster	☐	int
CLUSTER_STATE	State for the work order cluster [E; D]: E = Work order cluster is active (Enabled) D = Work order cluster is NOT active (Disabled)	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
CUSTOMER_NAME	Customer name in the iTAC system	☐	String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐	String
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumerical additional information 1	☐	String
INFO_TXT_2	Alphanumerical additional information 2	☐	String
INFO_TXT_3	Alphanumerical additional information 3	☐	String
INFO_TXT_4	Alphanumerical additional information 4	☐	String
INFO_TXT_5	Alphanumerical additional information 5	☐	String

Key	Explanation		IMSApi data type
INFO_TXT_6	Alphanumerical additional information 6	☐	String
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐	boolean
IS_MIRRORED_BOARD	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE) Advice: If the "double-sided panel" entry is selected, the setup information data of the component side and soldering side are combined and considered as a whole for this work order/work step!	☐	boolean
IS_PHANTOM_MASTER	Designator "Master phantom work order" [0; 1]: 0 = Work order is NOT a master phantom work order (FALSE) 1 = Work order is a master phantom work order (TRUE) Background: The master phantom work order is the initial work order that was adopted from the ERP system. The number of the master phantom work order is used both for a work order position and as the name of the work order cluster (CLUSTER_NAME). All other work order positions are created automatically in the iTAC system; here the order number (WORKORDER_NUMBER) is derived from the number of the master phantom work order.	☐	boolean
MAX_RECURSION	Maximum number of bookings permitted for a work step on the station	☐	long
PART_DESC	Part description in the iTAC system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String
PLANNED_SHIP_DATE	Planned delivery date for the work order	☐	DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production)	☐	DateMs
POSITION	Unique position number of the work order in the cluster	☒	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice regarding [3]: As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function. The "PROCESS_LAYER= 3" mechanism is therefore outdated; the setup data for the component and soldering side was combined and considered as a whole here! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐	int
PROCESS_VERSION	Process version from the iTAC system	☐	int

Key	Explanation		IMSApi data type
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐	double
QUANTITY	Quantity of production order	☐	double
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐	int
STATION_DESC	Station description of the work station in the iTAC system	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String
TYPE_DEPENDENCY	Function type for the station [D; R; B; L]: D = Diagnosis station: workplace for diagnosis R = Repair station: workplace for repair B = Type-independent: product-independent workplace L = Soldering station (old systems only): product-dependent workplace Advice: The value is initially defined upon creation of the station and cannot be changed afterwards!	☐	char
WORKORDER_DESC	Order description	☐	String
WORKORDER_NUMBER	Production order number	☒	String
WORKORDER_START_DATE	Start date for the work order (start of production)	☐	DateMs
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐	String
WORKSTEP_DESC	Description of the work step	☐	String
WORKSTEP_NUMBER	Number of the work step	☐	int

Key	Explanation	IMSApi data type
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 26-60 "Key Value Pair": stationMultiSettingsResultKeys

26.23.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationMultiSettingsResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationMultiSettingsResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationMultiSettingsResultValues!

Tab. 26-61 Parameter (in) »trGetMultiStationSettings«

Parameter (out)

Parameter name	Explanation
stationMultiSettingsResultValues	Depending on parameter (in) stationMultiSettingsResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationMultiSettingsResultKeys!

Tab. 26-62 Parameter (out) »trGetMultiStationSettings«

26.23.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.24 trGetNextProductionStep

26.24.1 Function description

This function determines for a serial number the next work step that is to be booked (including layer information). The highest already booked work step is considered to be the predecessor to the next work step to be ascertained here; this is independent of whether or not it requires confirmation.

For the "next work step" to be ascertained, it is possible to define whether a check of the serial number state is performed (`stateCheck`), whether only the work steps with obligatory confirmation are output (`confirmFlag`) and whether only the work steps that apply for the current station are output (`functionMode`).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetNextProductionStep
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in int functionMode
in int stateCheck
in int confirmFlag
in String[] productionStepResultKeys
out String[] productionStepResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `productionStepResultKeys` and (out) `productionStepResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
ERP_GROUP_DESC	ERP group description in the iTAC system	☐	String
ERP_GROUP_NUMBER	ERP group number in the iTAC system	☐	String
OBLIGATORY_CONFIRM_FLAG	Confirmation obligation [0; 1]: 0 = Work step is not obligated to supply confirmation 1 = Work step is obligated to supply confirmation Advice: The "Feedb. reqd." field is used on the work plan to define whether or not a work step is obligated to supply confirmation (ADM module -> Master data -> Work plan)!	☐	int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORKSTEP_STATE)!	☐	double
SEPARATION_FLAG	Multiple panel separation [0; 1]: 0 = Work step may not separate any multiple panels 1 = Work step may separate multiple panels Advice: Whether or not a work step may separate multiple panels is defined in the work plan with the "ERP sep. flag" field (ADM module -> Master data -> Work plan)!	☐	int
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐	String
SETUP_FLAG	Setup requirement [0; 1]: 0 = Work step does not require setup 1 = Work step requires setup Advice: The "Setup flag" field is used on the work plan to define whether or not a work step requires setup (ADM module -> Master data -> Work plan)!	☐	int
STATION_DESC	Station description of the work station in the iTAC system Advice: This entry is ascertained from the stored ERP group; if multiple stations are defined in an ERP group, the "Key Value Pair" structure is output a corresponding number of times!	☐	String

Key	Explanation	IMSApi data type
STATION_NUMBER	Station number of the work station in the iTAC system Advice: This entry is ascertained from the stored ERP group; if multiple stations are defined in an ERP group, the "Key Value Pair" structure is output a corresponding number of times!	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 26-63 "Key Value Pair": *productionStepResultKeys*

26.24.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
functionMode	Determines whether all work step are taken into account or whether only the work steps for which the calling station (<code>stationNumber</code>) is stored via the ERP group [0; 1]: 0 = All work steps 1 = Only work steps with the station Advice: For both modes, only the work steps after the highest booked work step are taken into consideration!
stateCheck	Serial number state check [0; 1]: 0 = without checking 1 = with checking Advice: If the check is active, the last state booking (highest sequence) of the current work step is evaluated. If the state of the serial number is not equal to PASS, then not the next, but rather the most recently booked work step is returned!
confirmFlag	Defines whether all work steps or only work steps obligated to supply confirmation are taken into account [0; 1]: 0 = All work steps 1 = Only work steps obligated to supply confirmation Advice: For both modes, only the work steps after the highest booked work step are taken into consideration!
productionStepResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": productionStepResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>productionStepResultValues</code> !

Tab. 26-64 Parameter (in) »trGetNextProductionStep«

Parameter (out)

Parameter name	Explanation
productionStepResultValues	Depending on parameter (in) <code>productionStepResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>productionStepResultKeys</code> !

Tab. 26-65 Parameter (out) »trGetNextProductionStep«

26.24.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-168	Failure	Invalid value for the function mode (control flag)
-2006	Failure	Invalid value for "stateCheck" [0; 1]
-2007	Failure	Invalid value for "confirmFlag" [0; 1]
-2070	Failure	IMSApi key was passed multiple times to the function

26.25

trGetNextSerialNumber

26.25.1

Function description

This function returns the serial numbers provided by the iTAC system for a part that is to be produced. Depending on the setting stored for the passed work order (work order part) or for the passed part, there are two different behaviors:

Application type "A" (output only): In combination with an order number, all already assigned serial numbers are output here.

Background: If the function for automatic creation of serial numbers is activated for the part/part group (see the "Create serial numbers on work order creation" check box in the "Assign ID generator head" dialog), all serial numbers required for the work order are generated automatically during work order creation according to the order quantity and assigned to the work order. In this case, the function does not return any newly generated serial numbers, but instead outputs the already assigned serial numbers.

Application type "B" (generate and output): Here, a certain number of new serial numbers are generated and output.

Background: If the automatic creation of serial numbers is deactivated for the part/part group, a certain number of "new" serial numbers can be generated. The ID generator to be used is ascertained on the basis of the passed order number or part number. The number of the required serial numbers is passed by means of the `numberOfRecords` variable. These serial numbers are not initially associated with a work order; the assignment to a work order must be performed separately by means of the »trAssignSerialNumberForProductOrWorkOrder« [▶▶▶] API function.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetNextSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workOrderNumber
in String partNumber
in int numberOfRecords
out SerialNumberData[] serialNumberArray
    String serialNumber
    String serialNumberPos
    String serialNumberOld
```

26.25.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workOrderNumber	Number of a production order; if necessary, with the ID alternative that is to be used for the ID generator (see paragraph below "Specification of the ID alternative") Advice: The required part settings or part group settings are ascertained on the basis of this order number! Special case: If [-1] is passed here, the part settings or part group settings are ascertained on the basis of the part number (<code>partNumber</code>)! If a [-1] is passed both here as well as for <code>partNumber</code>, the work order active on the station is used! Specification of the ID alternative: There may be different ID alternatives for each ID generator (customer = C, internal = I, external = E, etc.); if a specific alternative is to be used, this can be specified here separated by a " " (PIPE) (e.g. ORDER12345 E for external)! If nothing is specified, the ID alternative identified as "default" is used (for further information, see the "ID models" setup dialog in the ADM module).
partNumber	Part number in the iTAC system Advice: Is only evaluated for application type "B"! On the basis of this part number, the required part settings or part group settings are ascertained! Special case: The field is only evaluated if [-1] is passed for the <code>workOrderNumber</code> parameter! If a [-1] is passed both here as well as for <code>workOrderNumber</code>, the work order active on the station is used!
numberOfRecords	Default value for the number of entries in the array Advice: For data-technical reasons, an integer greater than [0] must be passed for all application types; the entered value is, however, only interpreted in application type "B"!

Tab. 26-66 Parameter (in) »trGetNextSerialNumber«

Parameter (out)

Parameter name	Explanation
serialNumberArray	Array of the structure with the following variables:
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array
serialNumberOld	Old serial number prior to a change of product serial number Advice: If the serial number was changed in the past, the old serial number is output here (cf. function »trSwitchSerialNumber« [▶▶▶])!

Tab. 26-67 Parameter (out) »trGetNextSerialNumber«

26.25.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-121	Failure	Part not found
-130	Failure	Work order not found
-173	Failure	Negative value for "numberOfRecords"
-320	Failure	No serial number generation configured for this part type and order type
-321	Failure	Serial number generation failed
-344	Failure	Serial number model -> range exceeded
-345	Failure	Serial number model is invalid
-355	Failure	Production order is part of a work order cluster

26.26

trGetPools

26.26.1

Function description

This function returns the serial number pools present in the system; the `serial-NumberPoolName`, `changeOrder` and `snrPoolState` input parameters can be used here as search criteria. All search criteria are "AND" coupled such that only the serial number pools that meet all used filter conditions are returned.

**Advice**

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGet-PoolSerialNumber« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetPools
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumberPoolName
  in String changeOrder
  in int snrPoolState
  in String[] serialNumberPoolResultKeys
  out String[] serialNumberPoolResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `serial-NumberPoolResultKeys` and (out) `serialNumberPoolResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
CHANGE_WOR-KORDER_NUMBER	Number of a modification order Advice: Value is only returned if a modification order was assigned to the pool!	☐ String
INFO	Individual additional information	☐ String
POOL_DESC	Description of the serial number pool	☐ String
POOL_NAME	Name of the serial number pool	☐ String
POOL_STATE	State of the serial number pool [0; 1; 2; 3]: 0 = initial 1 = started 2 = sno. pool prepared 3 = finished	☐ int
QUANTITY_ASSIGNED_SERIAL_NUMBERS	Number of assigned serial numbers in the pool	☐ int
QUANTITY_CO_ASSIGNED_SERIAL_NUMBERS	Number of serial numbers in the pool that were assigned to the modification order Advice: Value is only returned if a modification order was assigned to the pool!	☐ int
QUANTITY_IDENTIFIED_SERIAL_NUMBERS	Number of identified serial numbers in the pool	☐ int
REPAIR_WOR-KORDER_NUMBER	Number of a repair order Advice: Value is only returned if a repair order was assigned to the pool!	☐ String

Tab. 26-68 "Key Value Pair": *serialNumberPoolResultKeys*

26.26.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool-Name	Name of the serial number pool; the wildcard "*" may be used in the input. Advice: If an empty string or the value [-1] is passed, the field is not evaluated!
changeOrder	Number of a repair or modification order; the wildcard character "*" may be used in the input Advice: If an empty string or the value [-1] is passed, the field is not evaluated!
snrPoolState	State of the serial number pool [0; 1; 2; 3]: 0 = initial 1 = started 2 = sno. pool prepared 3 = finished Advice: If an empty string or the value [-1] is passed, the field is not evaluated!
serialNumberPool-ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberPoolResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) serialNumberPoolResultValues !

Tab. 26-69 Parameter (in) »trGetPools«

Parameter (out)

Parameter name	Explanation
serialNumberPool-ResultValues	Depending on parameter (in) serialNumberPoolResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) serialNumberPoolResultKeys !

Tab. 26-70 Parameter (out) »trGetPools«

26.26.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-302	Failure	Work order type is not valid
-317	Failure	Invalid value for "snrPoolState" [0; 1; 2; 3]
-2070	Failure	IMSApi key was passed multiple times to the function

26.27

trGetPoolSerialNumber

26.27.1

Function description

This function returns additional information for serial numbers and serial number pools; the `serialNumberPoolName`, `serialNumber`, `processingStateFlag` and `snrStateFlag` input parameters can be used here as search criteria. All search criteria are "AND" coupled such that only the serial number pools that meet all used filter conditions are returned. The three following query variants may be used:

1. Input of a serial number pool:

all assigned serial numbers are output with corresponding detail information (entries in "Key Value Pair" structure ≥ 1).

2. Input of a serial number:

the assigned serial number pool is ascertained and output with detail information on the serial number (entries in the "Key Value Pair" structure = 1).

3. Input of a serial number pool and a serial number:

Check for assignment; if successful, the detail information on the serial number is output (entries in the "Key Value Pair" structure = 1).

**Advice**

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetPools« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetPoolSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in String serialNumber
in String serialNumberPos
in int startPosition
in int maxRowCount
in int processingStateFlag
in int snrStateFlag
in String[] poolSerialNumberResultKeys
out String[] poolSerialNumberResultValues
out boolean hasMore
out String outSerialNumberPoolName
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `poolSerialNumberResultKeys` and (out) `poolSerialNumberResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
CHANGE_WOR-KORDER_NUMBER	Number of a modification order Advice: Value is only returned if a modification order was assigned to the pool!	☐	String
INFO	Individual additional information	☐	String
POOL_NAME	Name of the serial number pool	☐	String
POOL_TYPE	Serial number pool type [1; 2; 3]: 1 = serial number pool for modification orders 2 = serial number pool for delivery slips 3 = serial number pool for repair orders	☐	int
PROCESSING_STATE	Process state in the serial number pool [1; 2]: 1 = initial (serial numbers that have not been checked) 2 = identified (serial numbers that have been checked)	☐	int
REPAIR_WOR-KORDER_NUMBER	Number of a repair order Advice: Value is only returned if a repair order was assigned to the pool!	☐	String
SERIAL_NUMBER	Serial number of the panel	☐	String
SERIAL_NUMBER_ACTIVE	Identifies whether the serial number in the original production order is active or inactive [0; 1]: 1 = active; serial number is (still) active in the production order 0 = inactive; serial number is deactivated in the production order, i.e. it is active in the repair or modification order	☐	int
SERIAL_NUMBER_POOL_STATE	Modification state in the serial number pool [P; R; U]: P = pool assigned R = repair order assigned U = modification order assigned	☐	String

Tab. 26-71 "Key Value Pair": `poolSerialNumberResultKeys`

26.27.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPool- Name	Name of the serial number pool; if necessary, this parameter can be combined with <code>serialNumber</code> (see query variant 3 above) Advice: <i>If an empty string or the value [-1] is passed, the field is not evaluated!</i>
serialNumber	Serial number of the panel; if necessary, this parameter can be combined with <code>serialNumberPoolName</code> (see query variant 3 above) Advice: <i>If an empty string or the value [-1] is passed, the field is not evaluated! The passed serial number must not yet be assigned to a modification order.</i>
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
startPosition	Position in the structure from which the data is to be output (paging function); for the first function call, this value should be [1] and then increased with each following call by the value for <code>maxRowCount</code> . Advice: <i>In connection with the paging function, the <code>hasMore</code> output parameter is to be observed; this identifies whether the structure contains yet other records!</i>
maxRowCount	Maximum number of entries in the "Key Value Pair" structure per function call (paging function); in connection with <code>startPosition</code> , it is possible to successively "page" through large structures. Advice: <i>In connection with the paging function, the <code>hasMore</code> output parameter is to be observed; this identifies whether the structure contains yet other records!</i>
processingStateF- lag	Process state in the serial number pool [1; 2]: 1 = initial (serial numbers that have not been checked) 2 = identified (serial numbers that have been checked) Advice: <i>If an empty string or the value [-1] is passed, the field is not evaluated!</i>
snrStateFlag	Modification state in the serial number pool [1; 2; 3]: 1 = pool assigned 2 = modification order assigned 3 = repair order assigned Advice: <i>If an empty string or the value [-1] is passed, the field is not evaluated!</i>
poolSerialNumber- ResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": poolSerialNumberResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) <code>poolSerialNumberResultValues</code>!</i>

Tab. 26-72 Parameter (in) »trGetPoolSerialNumber«

Parameter (out)

Parameter name	Explanation
poolSerialNumber-ResultValues	Depending on parameter (in) <code>poolSerialNumberResultKeys</code> , the corresponding data is output here. Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>poolSerialNumberResultKeys</code>!</i>
hasMore	Trigger for the paging function [TRUE; FALSE]: TRUE = Other records present in the structure FALSE = No other records present in the structure Advice: <i>In connection with the paging function, the <code>startPosition</code> and <code>maxRowCount</code> input parameters are to be observed; these determine the start position and the maximum structure size!</i>
outSerialNumber-PoolName	Name of the serial number pool Advice: <i>This field is only filled if the function finds a single serial number pool!</i>

Tab. 26-73 Parameter (out) »trGetPoolSerialNumber«

26.27.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-287	Failure	Serial number pool does not exist
-293	Failure	Serial number is not assigned to any pool
-294	Failure	Serial number is assigned to another pool
-2070	Failure	IMSApi key was passed multiple times to the function

26.28

trGetProductQuantity

26.28.1

Function description

This function enables the querying of production details for a station or a work step. Compared to the »pmGetProductQuantity« [▶▶▶] function, this function uses ascertained booking quantities based on serial number bookings.



Advice

Order quantities based on TR bookings can be queried using this function; order quantities based on PM bookings must be queried using »pmGetProductQuantity« [▶▶▶].

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

A specific period under review can be selected for the query if necessary (see `workorderFilters`). In addition, the function enables filtering according to specific part numbers or order numbers (see `filterDataArray`).

As a result, production figures, such as the number of manufactured products on the work step level, can be output. Note the various application types that are activated by means of the `functionMode` parameter here.

Application type "A" (grouping of all quantities on station level):

For this application type, the `functionMode` configuration with [0] applies, parameters `stationNumber` and `processLayer` must be passed, a period under review can be entered (`DATE_FROM` and `DATE_TO`).

Special case: If the value [-1] is passed for `processLayer`, the quantities are ascertained over all work steps in which `stationNumber` occurs.

Application type "B" (grouping of all quantities on work step level):

For this application type, the `functionMode` configuration with [1] applies, parameters `stationNumber` and `processLayer` must be passed, a period under review can be entered (`DATE_FROM` and `DATE_TO`).

Special case: If the value [-1] is passed for `processLayer`, the quantities are ascertained over all work steps in which `stationNumber` occurs.

Application type "C" (grouping of all quantities with work order or part reference):

Applicable for this application type is the configuration `functionMode` with [2], the `filterDataArray` parameter must be passed, the `stationNumber` and `processLayer` parameters are not taken into account, a period under review can be entered (`DATE_FROM` and `DATE_TO`).

Application type "D" (grouping of all quantities on the work order level):

No work-step-related quantities are output here; all work-step numbers are consolidated for the respective work order (serial numbers). Applicable for this application type is the configuration `functionMode` with [3], the `filterDataArray` parameter must be passed, the `stationNumber` and `processLayer` parameters as well as the period under review (`DATE_FROM` and `DATE_TO`) are not taken into account.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trGetProductQuantity
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    string locale
in string stationNumber
in int processLayer
in int functionMode
in KeyValue[] workorderFilters
    string key
    string value
in KeyValue[] filterDataArray
    string key
    string value
in String[] productQuantityResultKeys
out String[] productQuantityResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workorderFilters`, `filterDataArray` and `productQuantityResultKeys` as well as (out) `productQuantityResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
DATE_FROM	Time stamp for the start of the period under review (here: booking period); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If the <code>DATE_FROM</code> key and the <code>DATE_TO</code> key are not passed, process-related booking quantities (PM) can also be output in <code>functionMode 0</code> or <code>1</code>!	☐ DateMs
DATE_TO	Time stamp for the end of the period under review (here: booking period); specification of the total time in milliseconds since 01.01.1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs

Key	Explanation	IMSApi data type
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 26-74 "Key Value Pair": *workorderFilters*

Definition of the output parameters for the found production figures

Key	Explanation	IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐ DateMs
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
CUSTOMER_NAME	Customer name in the iTAC system	☐ String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐ String

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
PROGRESS_PERCENT	Progress of the work step in percent Advice: <i>This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)! </i>	☐ double
QUANTITY_FAIL	Number of products with the FAIL state	☐ int
QUANTITY_OPEN	Current (open) quantity which can be processed at the work step. Advice: <i>This is the quantity booked with PASS at the previous work step, minus the quantity already processed at the current work step!</i>	☐ int
QUANTITY_PASS	Number of products with the PASS state	☐ int
QUANTITY_SCRAP	Number of products with the SCRAP state	☐ int
QUANTITY_WOR-KORDER_FINISHED	Number of products finished Advice: <i>Only those products that were booked with PASS on the last work step with obligatory confirmation are counted here!</i>	☐ int
QUANTITY_WOR-KORDER_STARTED	Start quantity for the work order	☐ int
QUANTITY_WOR-KORDER_TOTAL	Order quantity (total)	☐ int
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: <i>In application type "B" (grouping of all quantities on the work step level), this field cannot be filled, since multiple stations may be affected!</i>	☐ String
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String

Key	Explanation	IMSApi data type
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_DESC	Description of the work step	String
WORKSTEP_NUMBER	Number of the work step	int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!</i>	String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: <i>If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!</i>	String

Tab. 26-75 "Key Value Pair": *productQuantityResultKeys*

26.28.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
functionMode	Function mode [0; 1; 2; 3]: 0 = Grouping of all quantities on the station level (application type "A") 1 = Grouping of all quantities on the work step level (application type "B") 2 = Grouping of all quantities with work order or part reference (application type "C") 3 = Grouping of all quantities on the work order level (application type "D") Advice: Details on the function modes can be found above in the function description (application types "A" - "D")!
workorderFilters	"Key Value Pair" structure with the following variables:
key	List of the order parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter. A description of the possible order parameters can be found above in the "Key Value Pair": workorderFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": workorderFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
filterDataArray	"Key Value Pair" structure with the following variables:
key	List of the filter parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter [0; 1]: 0 = Order number 1 = Part number Advice: Because the filter parameters are passed as a structure, multiple entries are possible as well. The use of wildcards is only supported if an entry is passed in the array. If both filter parameters are filled, they must match one another (part = work order part)! Example: If the production figures for three parts are to be searched for (e.g., 4711, 4712 and 4713), the following applies: key = 1 and value = 4711 key = 1 and value = 4712 key = 1 and value = 4713

Parameter name	Explanation
value	Value list for the filter parameters passed by means of <code>key</code> . Advice: Refer to the explanation of <code>key</code> above to see whether special rules apply during the input of values (e.g., use of wildcards)!
productQuantityResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": productQuantityResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>productQuantityResultValues</code> !

Tab. 26-76 Parameter (in) »trGetProductQuantity«

Parameter (out)

Parameter name	Explanation
productQuantityResultValues	Depending on parameter (in) <code>productQuantityResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>productQuantityResultKeys</code> !

Tab. 26-77 Parameter (out) »trGetProductQuantity«

26.28.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-149	Failure	Work order or part number is required
-168	Failure	Invalid value for the function mode (control flag)
-2070	Failure	IMSApi key was passed multiple times to the function

26.29

trGetResultDataForSerialNumber

26.29.1

Function description

This function outputs the booked measurement data for the passed serial number. It is possible to select for the serial number whether the data for a specific work step, for all work steps of the current integration level or for all work steps of the current and the parent integration levels are to be output (`allProductEntries`). In addition to the filtering of the output by measurement step name (`name`) and parameter type (`type`), it is also possible to output the measurement data for all test runs of a work step or only for the last test run (`onlyLastEntry`).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetResultDataForSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumber
in int serialNumberPos
in String type
in String name
in int allProductEntries
in int onlyLastEntry
in KeyValue[] resultDataFilters
    string key
    string value
in String[] resultDataKeys
out String[] resultDataValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `resultDataFilters` and `resultDataKeys` as well as (out) `resultDataValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNCTION	Name of the function group in the recipe	☐ String
FUNCTION_STEP	Name of the function group step in the recipe	☐ String

Tab. 26-78 "Key Value Pair": *resultDataFilters*

Definition of the output parameters for the found measurement data

Key	Explanation	IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐ DateMs
DATE_CREATED	Booking time of the measurement data; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
LOWER_LIMIT	Lower limit for the measurement step	☐ String
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail"! Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	☐ String
MEASURE_NAME	Name of measurement step	☐ String
MEASURE_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐ int

Key	Explanation	IMSApi data type
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN "." D = Decimal DE "," Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐ String
MEASURE_VALUE	Measured value	☐ String
NOMINAL	Nominal value	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
TOLERANCE	Tolerance value for measurement step	☐ String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐ String
UNIT	Unit for the value passed	☐ String
UPPER_LIMIT	Upper limit for the measurement step	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group!	☐ String

Tab. 26-79 "Key Value Pair": resultDataKeys

26.29.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: The layer is only evaluated, if the variable <code>allProductEntries</code> has the value 0.
serialNumber	Serial no. of the panel Advice: All relevant order data is ascertained on the basis of the serial number!
serialNumberPos	Position of the single panel in the unit-array Advice: If the value [-1] is passed, the position is ascertained automatically!
type	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic Special case: If the value [-1] is passed, the measurement data is returned independent of the parameter type!
name	Name of measurement step Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! Special case: If the value [-1] is passed, the measurement data is returned independent of the name!
allProductEntries	Selection of the measurement data to be output [0; 1; 2]: 0 = Measurement data of a work step; the work step is ascertained on the basis of <code>stationNumber</code> and <code>processLayer</code> . 1 = Measurement data of the current integration level (work order) 2 = Measurement data of the current and of all parent integration levels in which the <code>serialNumber</code> was installed

Parameter name	Explanation				
onlyLastEntry	Selection of the test runs to be output [0; 1; 2; 3]: 0 = Measurement data for all test runs 1 = Measurement data of the last test run with multiple bookings; i.e. measurement steps not present in the last test run are NOT output (multiple values may possibly be output for one measurement step). 2 = All measurement data from all test runs; here only the most recent measured value for each measurement step (consolidation mode) 3 = Measurement data of the last test run without multiple bookings; as in mode [1], measurement steps not present in the last test run are NOT output (only the most recent value is output for a measurement step).				
resultDataFilters	"Key Value Pair" structure with the following variables: <table> <tr> <td>key</td><td>List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": resultDataFilters table.</td></tr> <tr> <td>value</td><td> Value list for the parameters passed by means of <code>key</code>. Advice: Refer to the "Key Value Pair": resultDataFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)! </td></tr> </table>	key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": resultDataFilters table.	value	Value list for the parameters passed by means of <code>key</code>. Advice: Refer to the "Key Value Pair": resultDataFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": resultDataFilters table.				
value	Value list for the parameters passed by means of <code>key</code>. Advice: Refer to the "Key Value Pair": resultDataFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!				
resultDataKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": resultDataKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>resultDataValues</code>!				

Tab. 26-80 Parameter (in) »trGetResultDataForSerialNumber«

Parameter (out)

Parameter name	Explanation
resultDataValues	Depending on parameter (in) <code>resultDataKeys</code>, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>resultDataKeys</code>!

Tab. 26-81 Parameter (out) »trGetResultDataForSerialNumber«

26.29.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-153	Failure	Invalid value for "type" [V, R, B, D, -1]
-171	Failure	Invalid value for "allProductEntries"
-172	Failure	Invalid value for "onlyLastEntry"
-185	Failure	Invalid value for the process layer
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.30

trGetSerialNumberActivityUploadInfo

26.30.1

Function description

This function returns detailed information on the booked work step activities of a serial number.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberActivityUploadInfo
in IMSApiSessionContextStruct sessionContext
  long sessionId
  long persId
  String locale
in String stationNumber
in String serialNumberRef
in String serialNumberRefPos
in KeyValue[] activityFilter
  key
  value
in String[] activityDataResultKeys
out String[] activityDataResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) activityFilter and activityDataResultKeys as well as (out) activityDataResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
ACTIVITY_SORT_SEQUENCE	Sorting number for the work step activity Advice: This number determines the processing sequence in the event that multiple activities are stored in a work step!	☐ int
FUNC_MODE	Function mode [0; 1]: 0 = Only the record that was booked last for the respective work step activity is output. 1 = All record booked for the respective work step activity are output (history function). Advice: If the key is not passed along, the function is executed in mode [0]!	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [-1; 0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: If the value [-1] is passed along, there is no restriction!	☒ int
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int

Tab. 26-82 "Key Value Pair": activityFilter

Definition of the output parameters for the booked work step activities

Key	Explanation	IMSApi data type
ACTIVITY_BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
ACTIVITY_NUMBER	Number of the work step activity	☐ String
ACTIVITY_SORT_SEQUENCE	Sorting number for the work step activity Advice: This number determines the processing sequence in the event that multiple activities are stored in a work step!	☐ int
ACTIVITY_STATE	Booking state of the work step activity [0; 1; 2; 3]: 0 = PASS 1 = FAIL 2 = OPEN 3 = SKIP	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double

Key	Explanation	IMSApi data type
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 26-83 "Key Value Pair": *activityDataResultKeys*

26.30.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array
activityFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible parameters can be found above in the "Key Value Pair": activityFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": activityFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
activityDataResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": activityDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) activityDataResultValues !

Tab. 26-84 Parameter (in) »trGetSerialNumberActivityUploadInfo«

Parameter (out)

Parameter name	Explanation
activityDataResultValues	Depending on parameter (in) activityDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) activityDataResultKeys !

Tab. 26-85 Parameter (out) »trGetSerialNumberActivityUploadInfo«

26.30.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.31 trGetSerialNumberBySerialNumberRef

26.31.1 Function description

On the basis of a reference serial number of a multiple panel, this function returns an array with the single panel serial numbers produced using this array – including the respective panel position specification. If the reference serial number is not known, the individual serial numbers can also be determined on the basis of a single panel serial number (if the serial number is unique, the panel position does not need to be specified).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberBySerialNumberRef
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberRef
in String serialNumberRefPos
out SerialNumberData[] serialNumberArray
    string serialNumber
    string serialNumberPos
    string serialNumberOld
```

26.31.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Reference serial number or single panel serial number
serialNumberRef-Pos	Panel position of the specified serial number; as soon as you specify a panel position, it is checked against the value stored in the system. If the number is different, the function is terminated with the error code -110. Advice: If the value [-1] is passed here, the panel position is not taken into consideration and therefore not checked; in this case, the specified serial numbers must be unique!

Tab. 26-86 Parameter (in) »trGetSerialNumberBySerialNumberRef«

Parameter (out)

Parameter name	Explanation
serialNumberArray	Array of the structure with the following variables:
serialNumber	Serial number of the single panel
serialNumberPos	Position of the single panel in the multiple panel
serialNumberOld	Old serial number prior to a change of product serial number Advice: If the serial number was changed in the past, the old serial number is output here (cf. function »trSwitchSerialNumber« [▶▶▶])!

Tab. 26-87 Parameter (out) »trGetSerialNumberBySerialNumberRef«

26.31.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
209	Warning	Serial number is not assigned to a panel
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-203	Failure	Serial number is not a reference serial number
-2138	Failure	Serial number is deleted

26.32

trGetSerialNumberForWorkorderAndWorkstep

26.32.1

Function description

This function returns information on the serial numbers that were booked for a work order on a certain work step; only **active** serial numbers are taken into account.

**Advice**

This API function is influenced by station parameter code nos. 8310 [▶▶▶] and 15270 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberForWorkOrderAndWorkstep
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String workOrderNumber
in int sequenceType
in int workstepNumber
in int multiplePanel
in int state
in int maxRows
in int confirmFlag
in int sortFlag
in String[] serialNumberResultKeys
out String[] serialNumberResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberResultKeys and (out) serialNumberResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
BOOKING_VALID	Validity of the serial number booking [0; 1]: 0 = Serial number booking is invalid (FALSE) 1 = Serial number booking is valid (TRUE) Background: This designator should not be confused with the serial number state "invalid"; there is, however, a direct link. If a serial number with the "invalid" state is booked, all subsequent serial number bookings automatically become invalid (reset) to allow them to be performed again if required. Example: A serial number is booked through to the 5th work step with the "pass" state; at the 6th work step (EOL test), a failure is detected and the serial number is booked with "fail". The failure can be traced back to a problem at the 3rd work step. The serial number is sent back to the 3rd work step where it is initially booked with the "invalid" state. As a result, the bookings already stored in the system for work steps 4 - 6 are given the designator "BOOKING_VALID = 0" (in other words, are reset). The serial number can then be booked again by means of the work process.	☐	boolean
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
IS_MIRRORED	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE)	☐	boolean
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐	String
SERIAL_NUMBER_REF	Reference serial number: serial number of the 1st single panel or of the unit-array	☐	String
SERIAL_NUMBER_REF_POS	Position of the reference serial number	☐	String

Key	Explanation	IMSApi data type
SERIAL_NUM- BER_SCT_STATE	SCT serial number state [R; S; U]: R = Ready to ship S = Shipped U = Ready to ship change Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ char
SERIAL_NUMBER_STATE	Process state of the serial number [0; 1; 2; 3; 4]: 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid	☒ int
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: <i>"Normal" serial numbers do not have a special type; an empty string is output in this case!</i>	☐ char
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group!</i>	☐ String

Tab. 26-88 "Key Value Pair": serialNumberResultKeys

26.32.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: If a [-1] is passed, the field is not evaluated! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
workOrderNumber	Number of a production order Special case: If [-1] is passed here, the work order which is active on the station is used!
sequenceType	"Sequence number" output control [0; 1]: 0 = All bookings (sequence number [1-n]) of the corresponding work step are output 1 = Only the last booking (highest sequence number [n]) of the respective work step is output
workstepNumber	Controls the scope of serial number output relative to the work step [> 0; 0; -1; -2; 3]: > 0 = Number of a certain work step (iTAC system) or of the corresponding AVO no. from a parent ERP system, for which the serial numbers are to be output. Serial numbers already booked at subsequent work steps are NOT taken into consideration/output. 0 = No restriction: all serial numbers of the work order are output; this mode enables an overview of the progress of all serial numbers in the work order -1 = All serial numbers which are pending for processing at the current work step and which do not have any further bookings to subsequent work steps are output; the work step is ascertained on the basis of the active work order, stationNumber and processLayer -2 = All serial numbers which are booked at the current work step and which do not have any further bookings to subsequent work steps are output; the work step is ascertained on the basis of the active work order, stationNumber and processLayer -3 = Takes into account all serial numbers that were booked on the current or a higher work step; the bookings of the highest work step in the highest sequence are then output. The current work step is ascertained on the basis of the active work order, stationNumber and processLayer. Advice: This result set can be further restricted by means of the multiplePanel and state parameters!

Parameter name	Explanation
<code>multiplePanel</code>	Controls the scope of serial number output with respect to multiple panels [0; > 0]: 0 = No restriction on multiple panels > 0 = Only multiple panels with a number of panels $\geq$ this value are taken into account for the output Advice: <i>This result set can be further restricted by means of the <code>workstepNumber</code> and state parameters!</i>
<code>state</code>	Controls the scope of serial number output with respect to the state of the serial numbers [-1; 0; 1; 2; 3; 4]: -1 = No restriction; the serial numbers are output independent of state 0 = Only serial numbers with the "PASS" state 1 = Only serial numbers with the "FAIL" state 2 = Only serial numbers with the "SCRAP" state 3 = Only serial numbers with the "In process" state 4 = Only serial numbers with the "Invalid" state Advice: <i>This result set can be further restricted by means of the <code>workstepNumber</code> and <code>multiplePanel</code> parameters!</i> In the case of "multiple panels", the total state is decisive here; the total state, in turn, is determined by the worst single panel state ("worst case" principle). Example: <i>For a four-way panel with three PASS serial numbers and one FAIL serial number, no serial number would be output in the event of <code>state = 0</code>.</i>
<code>maxRows</code>	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i> Advice: In the case of a multiple panel, this number can be exceeded by the value "number of panels minus 1"; this ensures that all serial numbers of the multiple panel are output! Special case: If the value [-1] is passed, the maximum value configured using station parameter code no. 15270 [▶▶▶] is used!
<code>confirmFlag</code>	Review mode [0; 1]: 0 = Only work steps obligated to supply confirmation are reviewed 1 = All work steps are reviewed
<code>sortFlag</code>	Sorting mode [0; 1]: 0 = FIFO principle (first in, first out): The first booked serial numbers are also the first to be output 1 = LIFO principle (last in, first out): The most recently booked serial numbers are the first to be output
<code>serialNumberResultKeys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>serialNumberResultKeys</code> table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) <code>serialNumberResultValues</code>!</i>

Tab. 26-89 Parameter (in) »trGetSerialNumberForWorkorderAndWorkstep«

Parameter (out)

Parameter name	Explanation
serialNumberResultValues	Depending on parameter (in) <code>serialNumberResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>serialNumberResultKeys</code> !

Tab. 26-90 Parameter (out) »trGetSerialNumberForWorkorderAndWorkstep«

26.32.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-174	Failure	Number of records to be output has fallen below the minimum value
-183	Failure	Invalid value for "sequenceType" [0; 1]

-190	Failure	Invalid value for "state" [-1; 0; 1; 2; 3; 4]
-191	Failure	Invalid value for "workstepNumber" [-2; -1; 0 or valid AVO no.]
-192	Failure	Invalid value for "multiplePanel" [>= 0]
-194	Failure	Invalid value for "sortFlag" [0; 1]
-240	Failure	No work step found for work order, orientation of the PCB and station
-243	Failure	Work step for the work order not found
-355	Failure	Production order is part of a work order cluster
-2070	Failure	IMSApi key was passed multiple times to the function

26.33

trGetSerialNumberHistoryData

26.33.1

Function description

This function returns all history data for a serial number. On the basis of the entered serial number, the highest integration level is first ascertained. The `desolvingSerialNumber` parameter can be used to select whether only the history data of the top integration level (final product) or whether all subordinate integration levels are to be output.



Advice

This API function is influenced by station parameter code nos. 8310 [▶▶▶] and 15300 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberHistoryData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in int processLayer
in int desolvingSerialNumber
in int desolvingLevel
in String[] bookingResultKeys
out String[] bookingResultValues
in String[] failureDataResultKeys
out String[] failureDataResultValues
in String[] failureSlipDataResultKeys
out String[] failureSlipDataResultValues
in String[] measureDataResultKeys
out String[] measureDataResultValues
out String workOrderNumber
out String partNumber
out String customerPartNumber
out String partDesc
out String quantity
out long lastReportDate
out String lotNumber
out int isLocked
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `bookingResultKeys`, `failureDataResultKeys`, `failureSlipDataResultKeys` and `measureDataResultKeys` and parameters (out) `bookingResultValues`, `failureDataResultValues`, `failureSlipDataResultValues` and `measureDataResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Booking data

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
BOOK_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context for the booking record (see parameter of the same name in the following "Failure data", "Failure notes" and "Measurement data" tables)!	☐	int
BOOK_STATE	Process state of the serial number [0; 1; 2; 3; 4]: 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid	☐	int

Key	Explanation	IMSApi data type
BOOKING_VALID	Validity of the serial number booking [0; 1]: 0 = Serial number booking is invalid (FALSE) 1 = Serial number booking is valid (TRUE) Background: This designator should not be confused with the serial number state "invalid"; there is, however, a direct link. If a serial number with the "invalid" state is booked, all subsequent serial number bookings automatically become invalid (reset) to allow them to be performed again if required. Example: A serial number is booked through to the 5th work step with the "pass" state; at the 6th work step (EOL test), a failure is detected and the serial number is booked with "fail". The failure can be traced back to a problem at the 3rd work step. The serial number is sent back to the 3rd work step where it is initially booked with the "invalid" state. As a result, the bookings already stored in the system for work steps 4 - 6 are given the designator "BOOKING_VALID = 0" (in other words, are reset). The serial number can then be booked again by means of the work process.	☐ boolean
CYCLE_TIME	Processing time at the station in seconds	☐ double
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
SCT_SOURCE_MANUFACTURER_CODE	Designator (code) of the manufacturer who manufactured the serial number Advice: This field is only relevant in the context of Supply Chain Traceability!	☐ String
SCT_SOURCE_MANUFACTURER_NAME	Name of the manufacturer who manufactured the serial number Advice: This field is only relevant in the context of Supply Chain Traceability!	☐ String
SCT_SOURCE_SUPPLIER_CODE	Designator (code) of the supplier who shipped the serial number Advice: This field is only relevant in the context of Supply Chain Traceability!	☐ String
SCT_SOURCE_SUPPLIER_NAME	Name of the supplier who shipped the serial number Advice: This field is only relevant in the context of Supply Chain Traceability!	☐ String
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_ARCHIV_STAMP	Date of archiving; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SERIAL_NUMBER_LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	☐ int
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Key	Explanation	IMSApi data type
SERIAL_NUM- BER_STATE_VALID	"Work-step-concluding booking" designator [0; 1]: 0 = No work-step-concluding booking 1 = Work-step-concluding booking Advice: <i>If the value [0] is output here, the work step has not yet been fully completed because the booking station does not have the authorization to do so. In this case, the station is usually a repair or diagnosis station (see the STATION_DIAG_NUMBER key) whose result still needs to be confirmed by a downstream tester (key word "repair loop")!</i>	int
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: <i>"Normal" serial numbers do not have a special type; an empty string is output in this case!</i>	char
STATION_DESC	Station description of the work station in the iTAC system	String
STATION_DIAG_DESC	Station description of the work station (diagnosis station) that performed the diagnosis (key word "same ERP group")	String
STATION_DIAG_NUMBER	Station number of the work station (diagnosis station) that performed the diagnosis (key word "same ERP group")	String
STATION_NUMBER	Station number of the work station in the iTAC system	String
WORKORDER_ID	Internal identification number of the production order	long
WORKORDER_NUMBER	Production order number	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String
WORKSTEP_DESC	Description of the work step	String
WORKSTEP_NUMBER	Number of the work step	int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group!</i>	String

Tab. 26-91 "Key Value Pair": bookingResultKeys

Failure data

Key	Explanation	IMSApi data type
BOOK_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure data can be put into context with the booking record (see the "Booking data" table above)!	☐ int
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
FAILURE_ACTION_CODE	Failure action code Advice: The failure actions must be created in advance in the iTAC system (see ADM module)!	☐ String
FAILURE_ACTION_INFO	Description of the failure action	☐ String
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_CAUSE_INFO	Individual information about the failure cause Advice: This information can be passed in combination with the failure booking!	☐ String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_MEASURE_CODE	Failure measure code Advice: The failure measures must be created in advance in the iTAC system (see ADM module)!	☐ String
FAILURE_MEASURE_INFO	Description of the failure measure	☐ String
FAILURE_OBJECT-PART_CODE	Failure object part code Advice: The failure object parts must be created in advance in the iTAC system (see ADM module)!	☐ String
FAILURE_RISK_CODE	Failure risk code Advice: The failure risks must be created in advance in the iTAC system (see ADM module)!	☐ String
FAILURE_RISK_INFO	Description of the failure risk	☐ String
FAILURE_SEQUENCE_NUMBER	Display of the faulty test run	☐ int
FAILURE_SLIP_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure data can be put into context with the failure note data (see parameter of the same name in the "Key Value Pair": failureSlipDataResultKeys failure note table)!	☐ long
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String

Key	Explanation	IMSApi data type
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_INFO	Individual information about the failure type Advice: This information can be passed in combination with the failure booking!	☐ String
GRID_X	Failure position: X value according to part image raster	☐ int
GRID_Y	Failure position: Y value according to part image raster	☐ int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐ String
INFO	Information text	☐ String
INFO_TXT_REPAIR	Information text for the repair	☐ String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number of the faulty component conformant with the iTAC BOM	☐ String
QUANTITY_SOLDERING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int
REPAIRED	Repair state [0; 1]: 0 = not repaired 1 = repaired	☐ int
REPAIRED_DATE	Point in time of the repair; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SERIAL_ERROR	Serial failure [0; 1]: 0 = No serial failure 1 = Serial failure	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
STATION_DIAG_NUMBER	Station number of the work station (diagnosis station) that performed the diagnosis (may be identical to STATION_REPAIR_NUMBER)	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
STATION_REPAIR_DESC	Station description of the work station that performed the repair/diagnosis	☐ String
STATION_REPAIR_NUMBER	Station number of the work station (repair station) that performed the repair (may be identical to STATION_DIAG_NUMBER)	☐ String
WORKORDER_NUMBER	Number of a production order	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-92 "Key Value Pair": *failureDataResultKeys*

Failure note

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
BOOK_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure note data can be put into context with the booking record (see the "Booking data" table above)!	☐ int
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
FAILURE_SLIP_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the failure note data can be put into context with the failure data (see parameter of the same name in the "Key Value Pair": failureDataResultKeys failure table)!	☐ long
INFO	Failure note text	☐ String
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
TEST_STEP_ID	Internal identification number of the test step	☐ long
TEST_STEP_NAME	Name of the test step Advice: Do not confuse this name with the measurement step name!	☐ String
WORKORDER_NUMBER	Number of a production order	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-93 "Key Value Pair": failureSlipDataResultKeys

Measurement data

Key	Explanation	IMSApi data type
BOOK_REFERENCE	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the measurement data can be put into context with the booking record (see the "Booking data" table above)!	☐ int
FUNCTION	Function name from the recipe	☐ String
FUNCTION_DESC	Explanation of the function	☐ String
FUNCTION_STEP	Function step from the recipe	☐ String
FUNCTION_STEP_DESC	Explanation of the function step	☐ String
LOWER_LIMIT	Lower limit for the measurement step	☐ String

Key	Explanation	IMSApi data type
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail"! Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	String
MEASURE_NAME	Name of measurement step	String
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ", " D = Decimal DE ", " Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	String
MEASURE_VALUE	Measured value	String
NOMINAL	Nominal value	String
REMARK	Comment text	String
SEQUENCE_NUMBER	Display of the sequence number (test run)	int
SERIAL_NUMBER	Serial number of the panel	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	String
STATION_DESC	Station description of the work station in the iTAC system	String
STATION_NUMBER	Station number of the work station in the iTAC system	String
TOLERANCE	Tolerance value for measurement step	String
UNIT	Unit for the value passed	String
UPPER_LIMIT	Upper limit for the measurement step	String
WORKORDER_NUMBER	Number of a production order	String
WORKSTEP_NUMBER	Number of the work step	int

Tab. 26-94 "Key Value Pair": *measureDataResultKeys*

26.33.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel Advice: <i>The serial number must be active!</i>
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
desolvingSerial-Number	Resolving of the product structure [0; 1; 2]: 0 = Product structure is NOT resolved 1 = Product structure is resolved UPWARD/DOWNWARD 2 = Product structure is resolved DOWNWARD only
desolvingLevel	Specification of "data scope" [0; 1; 2]: 0 = Data of all work steps 1 = Data of the work steps with station 2 = Data of the work steps with the station and orientation
bookingResultKeys	List of the parameters (booking data) that are to be ascertained by the function. A description of the possible parameters can be found above in the " Key Value Pair ": bookingResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) bookingResultValues!</i>
failureDataResultKeys	List of the parameters (failure data) that are to be ascertained by the function. A description of the possible parameters can be found above in the " Key Value Pair ": failureDataResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) failureDataResultValues!</i>
failureSlip-DataResultKeys	List of the parameters (failure note) that are to be ascertained by the function. A description of the possible parameters can be found above in the " Key Value Pair ": failureSlipDataResultKeys table. Advice: <i>Only the parameters passed here are output by the function; see parameter (out) failureSlipDataResultValues!</i>

Parameter name	Explanation
measureDataResultKeys	List of the parameters (measurement data) that are to be ascertained by the function. A description of the possible parameters can be found above in the " Key Value Pair ": measureDataResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) measureDataResultValues !

Tab. 26-95 Parameter (in) »trGetSerialNumberHistoryData«

Parameter (out)

Parameter name	Explanation
bookingResultValues	Depending on parameter (in) bookingResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) bookingResultKeys !
failureDataResultValues	Depending on parameter (in) failureDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureDataResultKeys !
failureSlipDataResultValues	Depending on parameter (in) failureSlipDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) failureSlipDataResultKeys !
measureDataResultValues	Depending on parameter (in) measureDataResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) measureDataResultKeys !
workOrderNumber	Number of a production order
partNumber	Part number in the iTAC system
customerPartNumber	Customer-specific part number (customer mat. no.)
partDesc	Part description in the iTAC system
quantity	Quantity of production order
lastReportDate	Date of the last work order change; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000
lotNumber	Identification number of the lot Advice: Value is only output if the serial number was packed!
isLocked	Lock state of the serial number [0; 1]: 0 = Not locked 1 = Locked

Tab. 26-96 Parameter (out) »trGetSerialNumberHistoryData«

26.33.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-170	Failure	Invalid value for "desolvingSerialNumber"
-185	Failure	Invalid value for the process layer
-2070	Failure	IMSApi key was passed multiple times to the function
-2072	Failure	Invalid value for "desolvingLevel"
-2138	Failure	Serial number is deleted

26.34

trGetSerialNumberInfo

26.34.1

Function description

This function returns information about the serial number passed and the order it is assigned to.

**Advice**

A unique serial number must be passed (wildcards are not permitted); uniqueness is defined by serial number and panel position!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberInfo
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumber
  in String serialNumberPos
  in String[] serialNumberResultKeys
  out String[] serialNumberResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberResultKeys and (out) serialNumberResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
ATTRIBUTE_1	"Attribute 1" free text box from the part master; e.g., H for high, L for low	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	☐	String
CLUSTER_NAME	Unique name for the work order cluster	☐	String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐	String
CUSTOMER_NAME	Customer name in the iTAC system	☐	String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐	String
IS_MIRRORED	"Mirroring" label for double-sided panels [0; 1]: 0 = Position number is NOT mirrored (FALSE) 1 = Position number is mirrored (TRUE) Advice: This key is only relevant for products of the "double-sided panel" type; definition of the "double-sided panel" type takes place when the work order cluster is created!	☐	boolean
IS_MIRRORED_BOARD	"Double-sided panel" designation [0; 1]: 0 = Product is NOT a double-sided panel (FALSE) 1 = Product is a double-sided panel (TRUE) Advice: If the "double-sided panel" entry is selected, the setup information data of the component side and soldering side are combined and considered as a whole for this work order/work step!	☐	boolean

Key	Explanation	IMSApi data type
IS_MULTI_PANEL	"Panel type" designation [0; 1; 2]: 0 = Single panel 1 = Multiple panel 2 = Separated (was part of a multiple panel)	☐ int
LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	☐ int
PACK_STATE	Packing state [0; 1]: 0 = Serial number is not packed 1 = Serial number is packed	☐ int
PARENT_WOR-KORDER_NUMBER	Order number of the TOP production order Advice: <i>This information is only determined if the ascertained work order (see WORKORDER_NUMBER) is a subordinate production order (order network)!</i>	☐ String
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PHANTOM_WOR-KORDER_NUMBER	Number of the (parent) production order for the phantom work order cluster	☐ String
PROCESS_VERSION	Process version from the iTAC system	☐ int
QUANTITY	Quantity of production order	☐ double
SCT_DESTINATION_-CUSTOMER_CODE	Designator (code) of the customer to whom the serial number was shipped Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_DESTINATION_-CUSTOMER_NAME	Name of the customer to whom the serial number was shipped Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_MANUFAC-TURER_CODE	Designator (code) of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_MANUFAC-TURER_NAME	Name of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLI-ER_CODE	Designator (code) of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLI-ER_NAME	Name of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SERIAL_NUMBER	Serial number of the panel	☐ String

Key	Explanation	IMSApi data type
SERIAL_NUMBER_ASSEMBLY_STATE	Processing state [C; L; M; P; R; U]: C = Forwarded L = Replaced M = Mounted P = Pool assigned R = Repair order assigned U = Modification order assigned	☐ String
SERIAL_NUMBER_ID	Internal identification number of the serial number	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_POS_DESC	Description of the panel position	☐ String
SERIAL_NUMBER_SCRAP_STAMP	Point in time of the SCRAP booking (entry in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
SERIAL_NUMBER_SCT_STATE	SCT serial number state [R; S; U]: R = Ready to ship S = Shipped U = Ready to ship change Advice: This field is only relevant in the context of Supply Chain Traceability !	☐ char
SERIAL_NUMBER_TYPE	Display of the serial number type [B; E; G; L; V; X]: B = Serial number of a material bin with unserialized units (key word "bin-based traceability") E = Serial number of a unit which was passed from an external system G = Serial number of a "Golden Unit" (unit for the purpose of machine capability determination) L = Label serial number (used solely for identifying serial numbers on a multiple panel) V = SCT serial number of a material bin with unserialized units (key word "bin-based traceability" in the context of Supply Chain Traceability) X = SCT serial number of a unit in the context of Supply Chain Traceability Advice: "Normal" serial numbers do not have a special type; an empty string is output in this case!	☐ char
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String

Key	Explanation	IMSApi data type
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group!	String

Tab. 26-97 "Key Value Pair": *serialNumberResultKeys*

26.34.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel Advice: The serial number must be active!
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
serialNumberResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>serialNumberResultValues</i> !

Tab. 26-98 Parameter (in) »trGetSerialNumberInfo«

Parameter (out)

Parameter name	Explanation
serialNumberResultValues	Depending on parameter (in) <i>serialNumberResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>serialNumberResultKeys</i> !

Tab. 26-99 Parameter (out) »trGetSerialNumberInfo«

26.34.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.35

trGetSerialNumberPoolInfo

26.35.1

Function description

This function returns detailed information on a specific serial number pool.

**Advice**

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶], »trIdentifyPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberPoolInfo
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberPoolName
in int workOrderInfo
in String[] poolInfoResultKeys
out String[] poolInfoResultValues
out String changeOrder
out String partNumber
out String erpBomIndex
out String infoTxtChangeOrder
out String infoTxtSnrPool
out int qtyAssignedSerialNumbers
out int qtyIdentifiedSerialNumbers
out int qtyCOAssignedSerialNumbers
out int snrPoolState
out String snrPoolDescription
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) poolInfoResultKeys and (out) poolInfoResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
PART_NUMBER	Part number in the iTAC system (acc. to BOM of the production order) Advice: Value is only returned if the <code>workOrderInfo</code> parameter was passed with [1]!	☐ String
SERIAL_NUMBER_AMOUNT	Number of serial numbers in the pool from the original production order Advice: Value is only returned if the <code>workOrderInfo</code> parameter was passed with [1]!	☐ int
WORKORDER_NUMBER	Number of a production order (original work order from which the serial numbers were assigned to the modification order) Advice: Value is only returned if the <code>workOrderInfo</code> parameter was passed with [1]!	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 26-100 "Key Value Pair": `poolInfoResultKeys`

26.35.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
<code>sessionContext</code>	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
<code>stationNumber</code>	Station number of the work station in the iTAC system
<code>serialNumberPool-Name</code>	Name of the serial number pool
<code>workOrderInfo</code>	Determines whether the <code>poolInfoResultKeys</code> and <code>poolInfoResultValues</code> parameters are evaluated [0; 1]: 0 = The "Key Value Pair" structure is NOT used 1 = The "Key Value Pair" structure is used
<code>poolInfoResult-Keys</code>	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": <code>poolInfoResultKeys</code> table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>poolInfoResultValues</code> ! This parameter is only used if the value [1] was passed for <code>workOrderInfo</code> !

Tab. 26-101 Parameter (in) »trGetSerialNumberPoolInfo«

Parameter (out)

Parameter name	Explanation
poolInfoResult-Values	Depending on parameter (in) <code>poolInfoResultKeys</code> , the corresponding data is output here. Advice: The data is output in the sequence in which they were entered by means of parameter (in) <code>poolInfoResultKeys</code> ! This parameter is only used if the value [1] was passed for <code>workOrderInfo</code> !
changeOrder	Number of a modification order Advice: Value is only returned if a modification order was assigned to the pool!
partNumber	Part number in the iTAC system (acc. to BOM of the modification order) Advice: Value is only returned if a modification order was assigned to the pool!
erpBomIndex	Change index of the BOM from an external ERP system (SAP) Advice: Value is only returned if a modification order was assigned to the pool!
infoTxtChangeOrder	Individual additional information on the modification order Advice: Value is only returned if a modification order was assigned to the pool!
infoTxtSnrPool	Individual additional information on the serial number pool
qtyAssignedSerialNumbers	Number of assigned serial numbers in the pool
qtyIdentifiedSerialNumbers	Number of identified serial numbers in the pool
qtyCOAssignedSerialNumbers	Number of serial numbers in the pool that were assigned to the modification order. Advice: Value is only returned if a modification order was assigned to the pool!
snrPoolState	State of the serial number pool [0; 1; 2; 3]: 0 = initial 1 = started 2 = sno. pool prepared 3 = finished
snrPoolDescription	Description of the serial number pool

Tab. 26-102 Parameter (out) »trGetSerialNumberPoolInfo«

26.35.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-287	Failure	Serial number pool does not exist
-316	Failure	The serial number pool is not unique (pool name and site must be unique)
-2070	Failure	IMSApi key was passed multiple times to the function

26.36

trGetSerialNumberUploadInfo

26.36.1

Function description

This function returns information about the last state of the serial number passed. This information either relates to the state in the current work step or to the most recently booked state (cf. `checkProcessStep`).

If a passed serial number cannot be found, it is always checked whether it has been converted into a new serial number (for further information, see API function `trSwitchSerialNumber` [▶▶▶]). There are two possibilities:

- The serial number searched for never existed. The error code `-110` is returned; all other output values remain empty.
- The serial number searched for existed. Error code `-110` is returned here as well; in addition, the now "new" serial number is returned as the output value for `SERIAL_NUMBER_SWITCHED`. With this new serial number, the function can then be called again.

**Advice**

This API function is influenced by station parameter code nos. 7209 [▶▶▶] and 7226 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetSerialNumberUploadInfo
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String serialNumber
  in String serialNumberPos
  in int checkProcessStep
  in String[] uploadInfoResultKeys
  out String[] uploadInfoResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `uploadInfoResultKeys` and (out) `uploadInfoResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
ACTIVATE_DATE	Activation time for the work step; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Background: With work order activation (work order changes to the "started" state), the activation time is also set by the system. In this context, the work step state (key WORKSTEP_STATE) is likewise converted by the system to "started". Advice: Work step state and activation time of the work step are currently only evaluated by the "APS service"!	☐	DateMs
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐	DateMs
BOOKING_VALID	Validity of the serial number booking [0; 1]: 0 = Serial number booking is invalid (FALSE) 1 = Serial number booking is valid (TRUE) Background: This designator should not be confused with the serial number state "invalid"; there is, however, a direct link. If a serial number with the "invalid" state is booked, all subsequent serial number bookings automatically become invalid (reset) to allow them to be performed again if required. Example: A serial number is booked through to the 5th work step with the "pass" state; at the 6th work step (EOL test), a failure is detected and the serial number is booked with "fail". The failure can be traced back to a problem at the 3rd work step. The serial number is sent back to the 3rd work step where it is initially booked with the "invalid" state. As a result, the bookings already stored in the system for work steps 4 - 6 are given the designator "BOOKING_VALID = 0" (in other words, are reset). The serial number can then be booked again by means of the work process.	☐	boolean
LOOP_COUNTER	Number of bookings against this serial number and this work step (stations of the associated ERP group) Advice: Only bookings of stations of the type "tester" are counted; bookings of repair and diagnosis stations are not counted!	☐	int
PART_NUMBER	Part number in the iTAC system	☐	String

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
RETRY_COUNTER	Display of the tests that have currently been performed at the work step (test retry counter) Background: All bookings of a work step after a diagnosis station are counted (e.g. multiple bookings at the ICT); a booking at the parallel diagnosis station resets the "counter" to zero. The maximum value is specified for the respective work step in the work plan (see the field "No. of tests" there).	☐ int
SCT_SOURCE_MANUFACTURER_CODE	Designator (code) of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_MANUFACTURER_NAME	Name of the manufacturer who manufactured the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_CODE	Designator (code) of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SCT_SOURCE_SUPPLIER_NAME	Name of the supplier who shipped the serial number Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ String
SERIAL_NUMBER_ACTUAL_STATE	Current process state of the serial number in the repair and diagnosis process [0; 1; 2]: 0 = Pass 1 = Fail 2 = Scrap Advice: <i>This key is only output if the work step has not yet been fully completed, i.e. the booking station does not have the authorization to do so. In this case, the station is usually a repair or diagnosis station (see the STATION_DIAG_NUMBER key) whose result still needs to be confirmed by a downstream tester (key word "repair loop")! The state of the last "work-step-concluding booking" is always indicated in the SERIAL_NUMBER_STATE key; here, this is the last "official" serial number state according to the tester.</i>	☐ int
SERIAL_NUMBER_LOCK_STATE	Serial number lock state [-2; -1; 0; 1]: -2 = Serial number is archived -1 = Serial number is locked 0 = Serial number is not (and never was) locked 1 = Serial number was locked in the meanwhile	☐ int
SERIAL_NUMBER_SCT_STATE	SCT serial number state [R; S; U]: R = Ready to ship S = Shipped U = Ready to ship change Advice: <i>This field is only relevant in the context of Supply Chain Traceability!</i>	☐ char

Key	Explanation	IMSApi data type
SERIAL_NUMBER_STATE	Process state of the serial number [-1; 0; 1; 2; 3; 4]: -1 = no state booking present 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid	int
SERIAL_NUM- BER_SWITCHED	"Old" serial number that has been replaced by a change Special case: If the serial number passed was not found because it had already been replaced by another one, the "new" serial number is output here!	String
STATION_DIAG_NUMBER	Station number of the work station (diagnosis station) that performed the diagnosis (key word "same ERP group")	String
STATION_NUMBER	Station number of the work station in the iTAC system	String
WORKORDER_NUMBER	Production order number	String
WORKSTEP_NUMBER	Number of the work step	int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group!	String

Tab. 26-103 "Key Value Pair": *uploadInfoResultKeys*

26.36.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Advice: This field is only evaluated if the value [1] is passed for the checkProcessStep parameter!
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
checkProcessStep	Determines the content of the state information about the serial number [0; 1]: 0 = Returns the most recently booked state 1 = Returns the state from the current work step Advice: If the value [1] is passed and no booking has yet occurred at the current work step, the function does return output value "0", but the SERIAL_NUMBER_STATE key is output with [-1] in this case!
uploadInfoResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": uploadInfoResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) uploadInfoResultValues!

Tab. 26-104 Parameter (in) »trGetSerialNumberUploadInfo«

Parameter (out)

Parameter name	Explanation
uploadInfoResultValues	Depending on parameter (in) uploadInfoResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) uploadInfoResultKeys!

Tab. 26-105 Parameter (out) »trGetSerialNumberUploadInfo«

26.36.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-167	Failure	Invalid value for "checkProcessStep" [0; 1]
-237	Failure	Serial number belongs to a different station
-240	Failure	No work step found for work order, orientation of the PCB and station
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.37

trGetStationResult

26.37.1

Function description

This function is used to query process data (measurement data) for a station. The data can be queried with/without serial number reference or with/without work order reference. Process data can thereby be ascertained on each station.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetStationResult
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String workorderNumber
in String partNumber
in int bomVersion
in String bomIndex
in String bomVersionErp
in DateMs fromDate
in DateMs toDate
in String[] measurementNames
in int functionMode
in String[] stationResultKeys
out String[] stationResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `stationResultKeys` and (out) `stationResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	String
BOM_VERSION	BOM version from the iTAC system	int
BOOK_DATE	Booking time for the process data (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green).	double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red).	double
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green).	double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red).	double
DATE_FROM	Start of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
DATE_TO	End of the validity period for the entry; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000	DateMs
DETECTION_POINT_NAME	Name of the data capture point	String
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	int

Key	Explanation	IMSApi data type
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!	☐ int
FUNCTION	Function name from the recipe	☐ String
FUNCTION_DESC	Explanation of the function	☐ String
FUNCTION_STEP	Function step from the recipe	☐ String
FUNCTION_STEP_DESC	Explanation of the function step	☐ String
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_LIMIT	Lower limit for the measurement step	☐ String
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail"! Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	☐ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	☐ String
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ", " D = Decimal DE ", " Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐ String
MEASURE_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*.[0-9]* Text (example: Hello world).*	☐ String

Key	Explanation	IMSApi data type
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis)	☐ double
NOMINAL	Nominal value	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
REMARK	Supplementary comment	☐ String
RESOURCE_DESC	Description of the resource	☐ String
RESOURCE_NAME	Name of the resource	☐ String
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
TOLERANCE	Tolerance value for measurement step	☐ String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐ String
UNIT	Unit for the value passed	☐ String
UPPER_ACTION_LIMIT	Upper intervention limit	☐ String
UPPER_LIMIT	Upper limit for the measurement step	☐ String
UPPER_SCRAP_LIMIT	Upper scrap limit	☐ String
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: <i>This parameter is an internal designator and does not currently have an official use!</i>	☐ String
WEIGHTING	Weighting for the PAA [0 - 9]	☐ int
WORKORDER_NUMBER	Production order number	☐ String

Tab. 26-106 "Key Value Pair": *stationResultKeys*

26.37.2

Input and output parameters

Parameter (in)

**Advice**

No product version needs to be passed for this function; all corresponding parameters can be passed with [-1]. However, note that passed parameters are always checked against one another; if the values do not match, the function aborts with a failure!

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
workorderNumber	Production order number Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
partNumber	Part number in the iTAC system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomVersion	BOM version from the iTAC system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomVersionErp	BOM version in the ERP system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
fromDate	Start of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
toDate	End of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Special case: [-1] passes the current date and current time; in this case, however, only the process data of the last booking are output independent of the selected function mode (see advice for <code>functionMode</code>)!

Parameter name	Explanation
measurementNames	Measurement step names (as array); this can be used to restrict the output to certain measurement step names Advice: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!
functionMode	"Output" function mode [0; 1]: 0 = all booked process data (multiple bookings) 1 = only process data of the last booking Advice: Mode [0] only functions if parameter toDate is NOT passed with [-1]!
stationResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationResultValues !

Tab. 26-107 Parameter (in) »trGetStationResult«

Parameter (out)

Parameter name	Explanation
stationResultValues	Depending on parameter (in) stationResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) stationResultKeys !

Tab. 26-108 Parameter (out) »trGetStationResult«

26.37.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-130	Failure	Work order not found
-168	Failure	Invalid value for the function mode (control flag)
-186	Failure	Product version is not valid
-187	Failure	Product version not found
-242	Failure	BOM version is invalid
-1105	Failure	BOM for the work order not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.38

trGetStationSetting

26.38.1

Function description

This function queries the product version and the corresponding work order that are currently set at a station. If no work order is active, the function is finished with a corresponding error message.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*If production takes place in the **work order cluster** – i.e. there are usually multiple work orders active on one station – the »trGetMultiStationSettings« [▶▶▶] function must be used!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetStationSetting
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] stationSettingResultKeys
out String[] stationSettingResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) stationSettingResultKeys and (out) stationSettingResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
ATTRIBUTE_1	"Attribute 1" free text box from the part master; e.g., H for high, L for low	☐	String
ATTRIBUTE_2	"Attribute 2" free text box from the part master	☐	String

Key	Explanation		IMSApi data type
ATTRIBUTE_3	"Attribute 3" free text box from the part master	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
CAD_PART_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board	☐	String
CUSTOMER_NAME	Customer name in the iTAC system	☐	String
CUSTOMER_PART_NUMBER	Customer mat. no. in the iTAC system	☐	String
EARLIEST_START_LAY	Earliest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Background: Time-related process specifications can be stored for each work step of an order-based work plan – this data generally comes from a connected ERP system and is used as the basis for planning. By default, it is possible to store the earliest and latest start times for the processes "Production", "Setup", "Stripdown", "Lay", "Transport" and "Wait".	☐	DateMs
EARLIEST_START_PRODUCTION	Earliest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_SETUP	Earliest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_STRIPDOWN	Earliest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_TRANSPORT	Earliest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_WAIT	Earliest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double

Key	Explanation	IMSApi data type
INFO_NUMBER_4	Numerical additional information 4	☐ double
INFO_TXT_1	Alphanumerical additional information 1	☐ String
INFO_TXT_2	Alphanumerical additional information 2	☐ String
INFO_TXT_3	Alphanumerical additional information 3	☐ String
INFO_TXT_4	Alphanumerical additional information 4	☐ String
INFO_TXT_5	Alphanumerical additional information 5	☐ String
INFO_TXT_6	Alphanumerical additional information 6	☐ String
LATEST_START_LAY	Latest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
LATEST_START_PRODUCTION	Latest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
LATEST_START_SETUP	Latest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
LATEST_START_STRIPDOWN	Latest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
LATEST_START_TRANSPORT	Latest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
LATEST_START_WAIT	Latest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐ DateMs
MAX_RECURSION	Maximum number of bookings permitted for a work step on the station	☐ long
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PLANNED_SHIP_DATE	Planned delivery date for the work order	☐ DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production)	☐ DateMs

Key	Explanation	IMSApi data type
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2; 3]: 0 = component side 1 = soldering side 2 = position independent 3 = double-sided panel ("old" mechanism) Advice regarding [3]: As of release 8.51.00, the "Double-sided panel" application type should be implemented using the "work order cluster" function. The "PROCESS_LAYER= 3" mechanism is therefore outdated; the setup data for the component and soldering side was combined and considered as a whole here! Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double
QUANTITY	Quantity of production order	☐ double
SPLIT_ID	Identification number within the APS service for split work steps. The ID identifies a work step for which the quantity to be manufactured can be a.) distributed between multiple workplaces (stations) and b.) produced at different times on one station. Background: If a work step is split in the detailed planning tool (Ganttplan), an ID is generated automatically for each sub-work step and passed to the iTAC system.	☐ int
STATION_DESC	Station description of the work station in the iTAC system	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
TYPE_DEPENDENCY	Function type for the station [D; R; B; L]: D = Diagnosis station: workplace for diagnosis R = Repair station: workplace for repair B = Type-independent: product-independent workplace L = Soldering station (old systems only): product-dependent workplace Advice: The value is initially defined upon creation of the station and cannot be changed afterwards!	☐ char
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_START_DATE	Start date for the work order (start of production)	☐ DateMs
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String

Key	Explanation	IMSApi data type
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 26-109 "Key Value Pair": stationSettingResultKeys

26.38.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
stationSettingResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": stationSettingResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) stationSettingResultValues!

Tab. 26-110 Parameter (in) »trGetStationSetting«

Parameter (out)

Parameter name	Explanation
stationSettingResultValues	Depending on parameter (in) <code>stationSettingResultKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>stationSettingResultKeys</code> !

Tab. 26-111 Parameter (out) »trGetStationSetting«

26.38.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-126	Failure	Process version is invalid
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-355	Failure	Production order is part of a work order cluster
-1101	Failure	BOM not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.39

trGetTestplanData

26.39.1

Function description

This function enables the querying of the tests to be performed on the basis of a test plan; in doing so, it is possible to individually select which tests of which work steps and which test characteristics are to be taken into account (output); for further information, see the `FUNC_MODE` and `WORKSTEP_MODE` keys in the **"Key Value Pair": testplanFilters** table.

Only the data of test plans that are already active in a work order can be ascertained.



Advice

This API function is influenced by station parameter code no. 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

To book test results, the »trUploadTestplanResult« [▶▶▶] function must be used!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetTestplanData
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in int processLayer
in String workorderNumber
in KeyValue[] testplanFilters
    String key
    String value
in String[] varParamResultKeys
out String[] varParamResultValues
in String[] varFailCodeResultKeys
out String[] varFailCodeResultValues
in String[] attribParamResultKeys
out String[] attribParamResultValues
in String[] attribParamValueResultKeys
out String[] attribParamValueResultValues
in String[] attribFailCodeResultKeys
out String[] attribFailCodeResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `testplanFilters`, `varParamResultKeys`, `varFailCodeResultKeys`, `attribParamResultKeys`, `attribParamValueResultKeys` and (out) `varParamResultValues`, `varFailCodeResultValues`, `attribParamResultValues`, `attribParamValueResultValues`, `attribFailCodeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Searching for specific parameter values

Key	Explanation	IMSApi data type
FUNC_MODE	Determines which test characteristics of the respective test are to be taken into account (output). A distinction is made here between test characteristics with and without random sample definitions, between test characteristics that have already been checked for serial number and those that are unchecked, etc. [0; 1; 2; 3; 4; 5]: 0 = All test characteristics of the test (random sample definitions are not taken into account/output) 1 = All untested test characteristics (with and without random sample definition) 2 = All test characteristics with random sample definitions (active); only those test characteristics are output for which a sampling test is currently pending (cf. mode [3]). 3 = All test characteristics with random sample definitions (active + inactive); all test characteristics are output independent of whether or not the sampling test is currently necessary (cf. mode [2]). 4 = All checked test characteristics for which a failure was ascertained (with and without random sample definition). 5 = All checked test characteristics for which a failure was ascertained (with and without random sample definition) as well as the formula parameters that were used for calculating a formula with faulty result (even if the formula parameters themselves were "good") Advice: In modes [1], [4] and [5], a serial number must absolutely be passed (see input parameter <code>serialNumber</code>)! If this key is not passed (used), the function is executed in default mode [0] (no restriction)!	☐ int
FUNCTION	Name of the (test-)group in the test plan Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
FUNCTION_STEP	Name of the (test-)group step in the test plan Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String
TEST_STEP_NAME	Name of the test Advice: The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters!	☐ String

Key	Explanation	IMSApi data type
TEST_STEP_TYPE	Type of test characteristic [A; V]: A = Only attributive test characteristics V = Only variable test characteristics Advice: <i>If the key is not passed, no restriction takes place; both the attributive as well as the variable test characteristics are taken into account!</i>	☐ String
WORKSTEP_MODE	Determines which tests of which work steps are to be taken into account (output) [0; 1; 2; 3]: 0 = Only from the current work step 1 = From the current and all previous work steps 2 = Only from the previous work steps 3 = All work steps Advice: <i>The current work step is ascertained on the basis of the passed order number (<code>workorderNumber</code>), the station (<code>stationNumber</code>) and the layer (<code>processLayer</code>)! If the key is not passed, the function is executed in default mode [0] (only the current work step)!</i>	☐ int

Tab. 26-112 "Key Value Pair": `testplanFilters`

Definition of the output parameters for the variable test characteristics

Key	Explanation	IMSApi data type
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	☐ int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green).	☐ double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐ double
CP_VIOLATION_FAIL-CODE_REFERENCE	Reference number for the failure record for the case of "Cp violation" Advice: <i>On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the <code>FAIL-CODE_REFERENCE</code> parameter in the "Key Value Pair": <code>varFailCodeResultKeys</code> table)!</i>	☐ int
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green).	☐ double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐ double

Key	Explanation	IMSApi data type
CPK_VIOLATION_FAIL-CODE_REFERENCE	Reference number for the failure record for the case of "Cpk violation" Advice: <i>On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the FAIL-CODE_REFERENCE parameter in the "Key Value Pair": varFailCodeResultKeys table)!</i>	☐ int
DATA_STATE	Assessment of the last test; the following values are possible: GOOD = measured value within specifications MAX_OK_VIOLATION = upper intervention limit violated MAX_SCRAP_VIOLATION = upper scrap limit violated MAX_VIOLATION = upper limit value violated MIN_OK_VIOLATION = lower intervention limit violated MIN_SCRAP_VIOLATION = lower scrap limit violated MIN_VIOLATION = lower limit value violated Advice: <i>An assessment of the last measurement result can only occur if the LAST_MEASUREMENT_VALUE parameter is also output!</i>	☐ String
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	☐ int
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: <i>Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!</i>	☐ int
FORMULA_ID	Internal identification number of the formula used for the calculation of the measured value (if defined) Advice: <i>The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »execFormula« [▶▶▶])!</i>	☐ long
FUNCTION	Name of the (test-)group in the test plan	☐ String
FUNCTION_DESC	Description of the (test-)group in the test plan	☐ String
FUNCTION_STEP	Name of the (test-)group step in the test plan	☐ String
FUNCTION_STEP_DESC	Description of the (test-)group step in the test plan	☐ String
HAS_MDA	"Linked items" designation [0; 1]: 0 = Test characteristic has NO MDA items (FALSE) 1 = Test characteristic has MDA items (TRUE)	☐ boolean
LAST_MEASUREMENT_DATE	Point in time of the last test (entry in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
LAST_MEASUREMENT_STATION	Station number that performed the last test for the serial number Advice: <i>A station is only returned if this test characteristic has already been checked for the serial number!</i>	☐ String

Key	Explanation	IMSApi data type
LAST_MEASURE- MENT_VALUE	Value of the last test Advice: A value is only returned if the test characteristic has already been checked for the serial number!	☐ String
LOWER_ACTION_FORMU- LA_ID	Internal identification number of the formula used for the calculation of the lower intervention limit (if defined) Advice: The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »execFormula« [▶▶▶])!	☐ long
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_ACTION_LIM- IT_FAILCODE_REFER- ENCE	Reference number for the failure record for the case of "violation of lower intervention limit" Advice: On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the FAIL-CODE_REFERENCE parameter in the "Key Value Pair": varFailCo-deResultKeys table)!	☐ int
LOWER_FORMULA_ID	Internal identification number of the formula used for the calculation of the lower limit value (if defined) Advice: The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »execFormula« [▶▶▶])!	☐ long
LOWER_LIMIT	Lower limit for the measurement step Advice: Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!	☐ String
LOWER_LIMIT_FAIL- CODE_REFERENCE	Reference number for the failure record for the case of "violation of the lower limit value" Advice: On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the FAIL-CODE_REFERENCE parameter in the "Key Value Pair": varFailCo-deResultKeys table)!	☐ int
LOWER_SCRAP_FORMU- LA_ID	Internal identification number of the formula used for the calculation of the lower scrap limit (if defined) Advice: The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »execFormula« [▶▶▶])!	☐ long
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
LOWER_SCRAP_LIMIT_- FAILCODE_REFERENCE	Reference number for the failure record for the case of "violation of the lower scrap limit" Advice: On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the FAIL-CODE_REFERENCE parameter in the "Key Value Pair": varFailCo-deResultKeys table)!	☐ int
MEASURE_NAME	Unique identifier of the test characteristic (measurement step name)	☐ String
MEASURE_SERIE	Number of measured values; if the value is > 1, it is a measurement series	☐ int

Key	Explanation		IMSApi data type
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ". " D = Decimal DE ", " Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐	String
MEASURE_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*.[0-9]* Text (example: Hello world).*	☐	String
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis)	☐	double
NOMINAL	Nominal value Advice: Alternatively, the limit value can also be defined with the LOWER_LIMIT and UPPER_LIMIT parameters; the LOWER_LIMIT and UPPER_LIMIT parameters override this setting!	☐	String
RECIPE_VERSION_ID	Identification number of the test plan version	☐	long
REMARK	Supplementary comment	☐	String
SAMPLE_MODE	Urgency for a random-sample-based test [-1; 0; 1; 2; 3; 4]: Advice: A test may be required due to time-, item- or event-based reasons; the urgency of a test is determined by the position in the tolerance range (start, middle, end)! -1 = No test necessary; there is currently no reason for a test (the start of the tolerance range has not yet been reached). 0 = Test is optional, there is no compelling reason for testing but the start of the tolerance range has been exceeded. 1 = Test will soon be required because the end of the tolerance range will be reached shortly. 2 = Test is imperative -> the pending serial number must be tested because the end of the tolerance range has been reached. 3 = Test is absolutely required -> the pending serial number must be tested because there is a dependency on another test/test characteristic. 4 = Test is absolutely required -> each serial number must always be tested in this case because no random sample definition is stored.	☐	int
SAMPLE_NAME	Name of the random sample definition	☐	String
SAMPLE_NUMBER_OF_TESTS_TO_BE_DONE	Number of open tests according to definition Advice: This field is filled if the test in question has an item-based random sample definition!	☐	int

Key	Explanation	IMSApi data type
SAMPLE_REASON	Trigger for the test [E; Q; T; D; U]: E = Event-based: In this case, the next test must be performed after a certain event occurs (e.g. work order change). Q = Item-based: In this case, the next test must be performed after a certain number of manufactured products is reached (e.g. every fifth product). T = Time-based: In this case, the next test must be performed after a certain length of time has elapsed (e.g. after 15 min.). D = Test-based: The test is performed depending on another test. U = Unknown: Output if a function mode has been selected for which no random sample definitions are considered (e.g. FUNC_MODE = 0).	☐ String
SAMPLE_TO_BE_DONE_UNTIL	Point in time by which this test must be performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: This field is filled if the test in question has a time-based random sample definition!	☐ DateMs
SAMPLE_TYPE	Severity level for the test [H; L; S]: H = Increased test severity L = Reduced test severity S = Normal test severity	☐ String
SEQUENCE_NUMBER	Test characteristic number; this number can be assigned during the creation of a test characteristic – it has no control function.	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☐ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »trUploadTestplanResult«)!	☐ long
TOLERANCE	Tolerance value for measurement step	☐ String
TP_CRITERIA	Test plan criteria according to the assigned test plan criteria group; the contained test plan criteria are output according to the syntax: [internal ID]#[name]#[description] Example: A test plan criteria group that contains two criteria (A and B) has been assigned to the test characteristic: A: internal ID = 12, name = TP1, desc. = Temperature1 B: internal ID = 55, name = L1, desc. = Light1 The function then returns the following string as the result: 12#TP1#Temperature#55#L1#Light1]	☐ String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐ String
UNIT	Unit for the value passed	☐ String
UPPER_ACTION_FORMULA_ID	Internal identification number of the formula used for the calculation of the upper intervention limit (if defined) Advice: The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., »execFormula« [▶▶▶])!	☐ long
UPPER_ACTION_LIMIT	Upper intervention limit	☐ String

Key	Explanation	IMSApi data type
UPPER_ACTION_LIMIT_FAILCODE_REFERENCE	Reference number for the failure record for the case of "violation of the upper intervention limit" Advice: <i>On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the <code>FAILCODE_REFERENCE</code> parameter in the "Key Value Pair": <code>varFailCodeResultKeys</code> table)!</i>	☐ int
UPPER_FORMULA_ID	Internal identification number of the formula used for the calculation of the upper limit value (if defined) Advice: <i>The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., <code>»execFormula«</code> [▶▶▶])!</i>	☐ long
UPPER_LIMIT	Upper limit for the measurement step	☐ String
UPPER_LIMIT_FAILCODE_REFERENCE	Reference number for the failure record for the case of "violation of the upper limit value" Advice: <i>On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the <code>FAILCODE_REFERENCE</code> parameter in the "Key Value Pair": <code>varFailCodeResultKeys</code> table)!</i>	☐ int
UPPER_SCRAP_FORMULA_ID	Internal identification number of the formula used for the calculation of the upper scrap limit (if defined) Advice: <i>The formula is uniquely defined on the basis of this identification number, which simplifies the call in other API functions (e.g., <code>»execFormula«</code> [▶▶▶])!</i>	☐ long
UPPER_SCRAP_LIMIT	Upper scrap limit	☐ String
UPPER_SCRAP_LIMIT_FAILCODE_REFERENCE	Reference number for the failure record for the case of "violation of the upper scrap limit" Advice: <i>On the basis of this number, the test characteristic can be put into context with the corresponding failure (see the <code>FAILCODE_REFERENCE</code> parameter in the "Key Value Pair": <code>varFailCodeResultKeys</code> table)!</i>	☐ int
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: <i>This parameter is an internal designator and does not currently have an official use!</i>	☐ String
WEIGHTING	Weighting for the PAA [0 - 9]	☐ int
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-113 "Key Value Pair": `varParamResultKeys`

Definition of the output parameters for the failures found for the variable test characteristics

Key	Explanation	IMSApi data type
FAILCODE_REFERENCE	Reference number for the test characteristic for the failure cases (Cp or Cpk violation or a violation of the various intervention limits). Advice: <i>On the basis of this number, the failure can be put into context with the corresponding test characteristic (see the CP_VIOLATION_FAILCODE_REFERENCE, CPK_VIOLATION_FAILCODE_REFERENCE, LOWER_ACTION_LIMIT_FAILCODE_REFERENCE, LOWER_LIMIT_FAILCODE_REFERENCE, LOWER_SCRAP_LIMIT_FAILCODE_REFERENCE, UPPER_ACTION_LIMIT_FAILCODE_REFERENCE, UPPER_LIMIT_FAILCODE_REFERENCE and UPPER_SCRAP_LIMIT_FAILCODE_REFERENCE parameters in the "Key Value Pair": varParamResultKeys table)!</i>	☐ int
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_CODE_ID	Internal identification number for the failure type code	☐ int
FAILURE_TYPE_DESC	Description of the failure type	☐ String
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
SEQUENCE_NUMBER	Display of the test characteristic number; this number can be assigned during the creation of a test characteristic – it has no control function.	☐ int

Tab. 26-114 "Key Value Pair": varFailCodeResultKeys

Definition of the output parameters for the attributive test characteristics

Key	Explanation	IMSApi data type
FUNCTION	Name of the (test-)group in the test plan	☐ String
FUNCTION_DESC	Description of the (test-)group in the test plan	☐ String
FUNCTION_STEP	Name of the (test-)group step in the test plan	☐ String
FUNCTION_STEP_DESC	Description of the (test-)group step in the test plan	☐ String
HAS_MDA	"Linked items" designation [0; 1]: 0 = Test characteristic has NO MDA items (FALSE) 1 = Test characteristic has MDA items (TRUE)	☐ boolean
LAST_MEASUREMENT_DATE	Point in time of the last test (entry in the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	☐ DateMs
LAST_MEASUREMENT_PASS	Result value of the last test [0; 1]: 0 = Not passed 1 = Passed	☐ String

Key	Explanation		IMSApi data type
LAST_MEASUREMENT_STATION	Station number that performed the last test for the serial number Advice: A station is only returned if this test characteristic has already been checked for the serial number!	☐	String
LAST_MEASUREMENT_VALUE	Value of the last test Advice: A value is only returned if the test characteristic has already been checked for the serial number!	☐	String
LAST_MEASUREMENT_VALUE_ID	Internal identification number for the value of the last test	☐	long
SAMPLE_MODE	Urgency for a random-sample-based test [-1; 0; 1; 2; 3; 4]: Advice: A test may be required due to time-, item- or event-based reasons; the urgency of a test is determined by the position in the tolerance range (start, middle, end)! -1 = No test necessary; there is currently no reason for a test (the start of the tolerance range has not yet been reached). 0 = Test is optional, there is no compelling reason for testing but the start of the tolerance range has been exceeded. 1 = Test will soon be required because the end of the tolerance range will be reached shortly. 2 = Test is imperative -> the pending serial number must be tested because the end of the tolerance range has been reached. 3 = Test is absolutely required -> the pending serial number must be tested because there is a dependency on another test/test characteristic. 4 = Test is absolutely required -> each serial number must always be tested in this case because no random sample definition is stored.	☐	int
SAMPLE_NAME	Name of the random sample definition	☐	String
SAMPLE_NUMBER_OF_TESTS_TO_BE_DONE	Number of open tests according to definition Advice: This field is filled if the test in question has an item-based random sample definition!	☐	int
SAMPLE_REASON	Trigger for the test [E; Q; T; D]: E = Event-based: In this case, the next test must be performed after a certain event occurs (e.g. work order change). Q = Item-based: In this case, the next test must be performed after a certain number of manufactured products is reached (e.g. every fifth product). T = Time-based: In this case, the next test must be performed after a certain length of time has elapsed (e.g. after 15 min.). D = The test is performed depending on another test.	☐	String
SAMPLE_TO_BE_DONE_UNTIL	Point in time by which this test must be performed; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: This field is filled if the test in question has a time-based random sample definition!	☐	DateMs
SAMPLE_TYPE	Severity level for the test [H; L; S; X]: H = Increased test severity L = Reduced test severity S = Normal test severity X = Test skipped; is only used in the context of receiving tests that are based on AQL reference tables according to DIN2859.	☐	String
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String

Key	Explanation	IMSApi data type
TEST_STEP_DESC	Description of the test characteristic	☐ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects of the test characteristic can be put into context with the property values (see parameter of the same name in the "Key Value Pair": attribParamValueResultKeys table!	☐ long
TEST_STEP_INFO_TEXT	Individual information text for the test characteristic	☐ String
TEST_STEP_NAME	Name of the test characteristic	☐ String
TEST_STEP_NUMBER	Test step no. in the test plan (measurement step number)	☐ double
TP_CRITERIA	Test plan criteria according to the assigned test plan criteria group; the contained test plan criteria are output according to the syntax: [internal ID]#[name]#[description] Example: A test plan criteria group that contains two criteria (A and B) has been assigned to the test characteristic: A: internal ID = 12, name = TP1, desc. = Temperature1 B: internal ID = 55, name = L1, desc. = Light1 The function then returns the following string as the result: 12#TP1#Temperature#55#L1#Light1]	☐ String
WORKSTEP_NUMBER	Number of the work step	☐ int

Tab. 26-115 "Key Value Pair": attribParamResultKeys

Definition of the output parameters for the property values for the attributive test characteristics

Key	Explanation	IMSApi data type
FAILCODE_REFERENCE	Reference number for the failure record Advice: On the basis of this number, the property value can be put into context with the corresponding failure (see the FAILCODE_REFERENCE parameter in the "Key Value Pair": attribFailCodeResultKeys table)!	☐ int
RESULT_DESC	Description of the results instance	☐ String
RESULT_NAME	Name of the results instance	☐ String
RESULT_PASS	Assessment of the results instance [0; 1]: 0 = Not passed (FALSE) 1 = Passed (TRUE)	☐ boolean
SEQUENCE_NUMBER	Display of the test characteristic number; this number can be assigned during the creation of a test characteristic – it has no control function.	☐ int
TEST_STEP_ID	Internal identification number of the corresponding test characteristic Advice: The records of an array must be uniquely numbered; on the basis of this number, the property values can be put into context with test characteristic (see parameter of the same name in the "Key Value Pair": attribParamResultKeys table!	☐ long
TEST_STEP_VALUE	Display of the (individual) weight for the property value	☐ String

Key	Explanation	IMSApi data type
TEST_STEP_VALUE_-DESC	Description of the property value	☐ String
TEST_STEP_VALUE_ID	Internal identification number of the property value	☐ long
TEST_STEP_VALUE_INFO_TEXT	Individual information text for the property value	☐ String
TEST_STEP_VALUE_NAME	Name of the property value	☐ String

Tab. 26-116 "Key Value Pair": *attribParamValueResultKeys*

Definition of the output parameters for the failure types for the property values of the attributive test characteristics

Key	Explanation	IMSApi data type
FAILCODE_REFERENCE	Reference number for the property value in the event of an error Advice: <i>On the basis of this number, the failure can be put into context with the corresponding property value (see the FAILCODE_REFERENCE parameter in the "Key Value Pair": attribParamValueResultKeys table)!</i>	☐ int
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_CODE_ID	Internal identification number for the failure type code	☐ int
FAILURE_TYPE_DESC	Description of the failure type	☐ String
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
SEQUENCE_NUMBER	Display of the test characteristic number; this number can be assigned during the creation of a test characteristic – it has no control function.	☐ int

Tab. 26-117 "Key Value Pair": *attribFailCodeResultKeys*

26.39.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel Advice: For function modes [1], [4] and [5] (see the <code>FUNC_MODE</code> parameter in the "Key Value Pair": testplanFilters table), it is absolutely necessary that a serial number be passed; otherwise an empty string or the value [-1] can be passed here!
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
workorderNumber	Number of a production order Special case: If an empty string is passed, the work order is ascertained on the basis of the serial number (<code>serialNumber</code>); if order number and serial number are passed, the serial number must be part of the work order!
testplanFilters	"Key Value Pair" structure with the following variables:
key	List of the test plan parameters on the basis of which the output is to be filtered; the corresponding values are passed with the <code>value</code> parameter. A description of the possible order parameters can be found above in the "Key Value Pair": testplanFilters table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": testplanFilters table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
varParamResultKeys	List of the parameters (variable test characteristics) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": varParamResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <code>varParamResultValues</code> !

Parameter name	Explanation
varFailCodeResultKeys	List of the parameters (test failures for variable test characteristics) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": varFailCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) varFailCodeResultValues!
attribParamResultKeys	List of the parameters (attributive test characteristics) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": attribParamResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) attribParamResultValues!
attribParamValueResultKeys	List of the parameters (property values for attributive test characteristics) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": attribParamValueResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) attribParamValueResultValues!
attribFailCodeResultKeys	List of the parameters (test failures for attributive test characteristics) that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": attribFailCodeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) attribFailCodeResultValues!

Tab. 26-118 Parameter (in) »trGetTestplanData«

Parameter (out)

Parameter name	Explanation
varParamResultValues	Depending on parameter (in) varParamResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) varParamResultKeys!
varFailCodeResultValues	Depending on parameter (in) varFailCodeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) varFailCodeResultKeys!
attribParamResultValues	Depending on parameter (in) attribParamResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) attribParamResultKeys!
attribParamValueResultValues	Depending on parameter (in) attribParamValueResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) attribParamValueResultKeys!
attribFailCodeResultValues	Depending on parameter (in) attribFailCodeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) attribFailCodeResultKeys!

Tab. 26-119 Parameter (out) »trGetTestplanData«

26.39.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-128	Failure	Work step not found
-130	Failure	Work order not found
-205	Failure	Serial number belongs to a different work order
-1103	Failure	Shift not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.40

trGetTopFailures

26.40.1

Function description

This function outputs the booked failures, sorted by frequency, for a freely selectable period under review. In doing so, it is possible to select whether failures should be ascertained relating to the part, order or station. In addition, it is possible to limit the maximum number of failures to be considered (`MAX_ROWS`); this makes, for example, a "TopTen" or even "TopThree" failure type analysis possible.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetTopFailures
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] trGetTopFailuresFilter
    String key
    String value
  in String[] topFailuresResultKeys
  out String[] topFailuresResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `trGetTopFailuresFilter` and `topFailuresResultKeys` as well as (out) `topFailuresResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation		IMSApi data type
BARE_BORD_NUMBER	Part number of the fundamental bearer, e.g., number of the "bare" printed circuit board Advice: <i>Even though this key is not identified as a mandatory field, either the BARE_BORD_NUMBER, the PRODUCT_NUMBER or the WORKORDER_NUMBER must be passed!</i>	☐	String
BOM_VERSION	BOM version from the iTAC system Advice: <i>If this key is to be used, the PRODUCT_NUMBER must also be passed without fail!</i>	☐	int
DATE_FROM	Time stamp for the start of the period under review (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes 01-01-1970, 0:00:00.000 as start time!	☐	DateMs
DATE_TO	Time stamp for the end of the period under review; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Special case: [-1] passes 12-31-3000, 23:59:59.999 as the end time!	☐	DateMs
FAILURE_CAUSE_CODE	Failure cause code	☐	String
FAILURE_IMAGE_CODE	Failure symptom code	☐	String
FAILURE_TYPE_CODE	Failure type code	☐	String
FUNC_MODE	"Secondary booking" function mode [0; 1]: 0 = No restriction 1 = Only those failures are considered which were booked as PASS again with the secondary booking (next sequence) Advice: <i>If this key is not passed along, the [0] function mode is used by default!</i>	☐	int
MAX_ROWS	Maximum number of records in the array returned per function call; the maximum is limited to [1000] by the system. Advice: <i>If this key is not passed along, the [10] value is used by default! If more records exist than are defined here, the function outputs a warning (see error code 3)!</i>	☐	int
PRODUCT_NUMBER	Part number in the iTAC system Advice: <i>Even though this key is not identified as a mandatory field, either the BARE_BORD_NUMBER, the PRODUCT_NUMBER or the WORKORDER_NUMBER must be passed!</i>	☐	String
REPAIRED	Repair state [0; 1]: 0 = Not repaired 1 = Repaired Advice: <i>If the key is not passed, the repair state is not evaluated; there is no restriction!</i>	☐	int
STATION_NUMBER	Station number of the work station in the iTAC system	☐	String

Key	Explanation	IMSApi data type
WORKORDER_NUMBER	Production order number Advice: <i>Even though this key is not identified as a mandatory field, either the BARE_BOARD_NUMBER, the PRODUCT_NUMBER or the WORKORDER_NUMBER must be passed!</i>	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String

Tab. 26-120 "Key Value Pair": `trGetTopFailuresFilter`

Definition of the output parameters for the found failures

Key	Explanation	IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
FAILURE_CAUSE_CODE	Failure cause code Advice: <i>The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!</i>	☐ String
FAILURE_CAUSE_DESC	Description of the failure cause	☐ String
FAILURE_COUNT	Failure frequency	☒ int
FAILURE_IMAGE_CODE	Failure symptom code	☐ String
FAILURE_IMAGE_DESC	Description of the failure symptoms	☐ String
FAILURE_TYPE_CATEGORY	Failure type category [C; S]: C = Component failure (Component) S = Soldering point failure (Soldering)	☐ String
FAILURE_TYPE_CODE	Failure type code	☐ String
FAILURE_TYPE_DESC	Description of the failure type	☐ String
FAILURE_TYPE_ORIGIN	Classification of the failure origin [1; 2; 3; 4]: 1 = design error (hardware) 2 = design error (software) 3 = purchased part error 4 = production error Advice: <i>The available entries are permanently set in the system!</i>	☐ int
FAILURE_TYPE_PSEUDO	Identifies the failure type as "pseudo failure" [0; 1]: 0 = not a pseudo failure 1 = pseudo failure	☐ char

Key	Explanation	IMSApi data type
FAILURE_TYPE_WEIGHT	Classification of the failure severity [1; 2; 3; 4]: 1 = A-failure 2 = B-failure 3 = C-failure 4 = safe launch Advice: <i>The available entries are permanently set in the system!</i>	int
MAJOR_PRESET	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	int
MAX_BOOK_DATE	Booking time of the last failure booking in the period under review (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
MIN_BOOK_DATE	Booking time of the first failure booking in the period under review (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000	DateMs
PART_NUMBER	Part number in the iTAC system	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	int
PRODUCT_NUMBER	Part number in the iTAC system	String
REPAIRED	Repair state [0; 1]: 0 = Not repaired 1 = Repaired	int
STATION_NUMBER	Station number of the work station in the iTAC system	String
WORKORDER_NUMBER	Production order number	String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	String
WORKSTEP_AVO	Work step number in the ERP system (AVO)	String

Tab. 26-121 "Key Value Pair": *topFailuresResultKeys*

26.40.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
trGetTopFailures-Filter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": trGetTopFailuresFilter table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": trGetTopFailuresFilter table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
topFailuresResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": topFailuresResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) topFailuresResultValues !

Tab. 26-122 Parameter (in) »trGetTopFailures«

Parameter (out)

Parameter name	Explanation
topFailuresResultValues	Depending on parameter (in) topFailuresResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) topFailuresResultKeys !

Tab. 26-123 Parameter (out) »trGetTopFailures«

26.40.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-168	Failure	Invalid value for the function mode (control flag)
-174	Failure	Number of records to be output has fallen below the minimum value
-2040	Failure	Work plan -> Type is invalid
-2070	Failure	IMSApi key was passed multiple times to the function
-2119	Failure	Number of records to be output has exceeded the maximum value

26.41 trGetWorkOrderForStation

26.41.1 Function description

This function uses special search criteria to ascertain the production orders relevant for a specific line. Available search criteria are order state, part number, part group, date of creation, and the order number itself. All search criteria are "AND" coupled such that only the work orders that meet all used filter conditions are returned.



Advice

This API function is influenced by station parameter code no. 15270 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trGetWorkOrderForStation
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] workorderFilter
    String key
    String value
  in String[] workorderResultKeys
  out String[] workorderResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workorderFilter` and `workorderResultKeys` as well as parameter (out) `workorderResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Searching for specific parameter values

Key	Explanation	IMSApi data type
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐ String
MAX_ROWS	Maximum number of records in the array returned per function call Advice: <i>If more records exist than are defined here, the function outputs a warning (see error code 3)!</i> Special case: If the key is not passed or is passed with [-1], the maximum value configured using station parameter code no. 15270 [▶▶▶] is used!	☐ int
PART_GROUP	Part group of the finished product in the iTAC system Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!</i>	☐ String
PART_NUMBER	Part number of the finished product in the iTAC system Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!</i>	☐ String
PRODUCTION_CONTROLLER	Production controller Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!</i>	☐ String
SORT_FLAG	Sorting mode for the output [0; 1; 2]: 0 = ascending according to APS_START_DATE 1 = ascending according to WORKORDER_START_DATE 2 = ascending according to WORKORDER_SEQUENCE Advice: <i>If the key is not passed, the work orders are output unsorted!</i>	☐ int
WORKORDER_NUMBER	Number of a production order Advice: <i>The wildcard character "*" can be used during input; pay attention to upper- and lower-case letters! If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!</i>	☐ String
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!	☐ DateMs
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted) Advice: <i>Multiple state values may be passed in the input (separated by ";")!</i> Special case: If an empty string or [-1] is passed, the parameter is not evaluated (no restriction)!	☐ String

Key	Explanation	IMSApi data type
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": <i>This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!</i> Advice: <i>Multiple state values may be passed in the input (separated by ";", " or ";")!</i>	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice "station-specific": <i>If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!</i> Advice: <i>Multiple state values may be passed in the input (separated by ";", " or ";")!</i>	☐ String

Tab. 26-124 "Key Value Pair": *workorderFilter*

Definition of the output parameters for the found work orders

Key	Explanation	IMSApi data type
APS_END_DATE	The initial production end time (delivery date) planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Advice: The point in time is set when the work order is started (cf. the <code>PLANNED_SHIP_DATE</code> key)!	☐ DateMs
APS_START_DATE	The initial production start time planned by the APS service for the work order; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: The point in time is set when the work order is started (cf. the <code>PLANNED_START_DATE</code> key)!	☐ DateMs
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐ String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]: E = BOM for maintenance order F = BOM for production order I = BOM for receiving M = BOM for material test R = BOM for repair order S = BOM for external products U = BOM for modification order X = BOM for SCT order Advice: The BOM type is optional; if the BOM is not assigned a type, the key returns an empty string!	☐ String
BOM_VERSION	BOM version from the iTAC system	☐ int
BOM_VERSION_ERP	BOM version in the ERP system	☐ String
CLUSTER_DESC	Description of the work order cluster	☐ String
CLUSTER_ID	Internal identification number of the work order cluster	☐ long
CLUSTER_NAME	Unique name for the work order cluster	☐ String
CLUSTER_TYPE	Work order cluster type [O; W; P]: O = Work order cluster (default) W = Work step cluster P = Phantom work order cluster	☐ String
EARLIEST_START_LAY	Earliest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Background: Time-related process specifications can be stored for each work step of an order-based work plan – this data generally comes from a connected ERP system and is used as the basis for planning. By default, it is possible to store the earliest and latest start times for the processes "Production", "Setup", "Stripdown", "Lay", "Transport" and "Wait".	☐ DateMs
EARLIEST_START_PRODUCTION	Earliest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See <i>Background</i> for key <code>EARLIEST_START_LAY</code> !	☐ DateMs

Key	Explanation		IMSApi data type
EARLIEST_START_SETUP	Earliest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_STRIPDOWN	Earliest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_TRANSPORT	Earliest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
EARLIEST_START_WAIT	Earliest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
INFO_NUMBER_1	Numerical additional information 1	☐	double
INFO_NUMBER_2	Numerical additional information 2	☐	double
INFO_NUMBER_3	Numerical additional information 3	☐	double
INFO_NUMBER_4	Numerical additional information 4	☐	double
INFO_TXT_1	Alphanumerical additional information 1	☐	String
INFO_TXT_2	Alphanumerical additional information 2	☐	String
INFO_TXT_3	Alphanumerical additional information 3	☐	String
INFO_TXT_4	Alphanumerical additional information 4	☐	String
INFO_TXT_5	Alphanumerical additional information 5	☐	String
INFO_TXT_6	Alphanumerical additional information 6	☐	String
LATEST_START_LAY	Latest start time for "Lay"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_PRODUCTION	Latest start time for "Production"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_SETUP	Latest start time for "Setup"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_STRIPDOWN	Latest start time for "Stripdown"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_TRANSPORT	Latest start time for "Transport"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs
LATEST_START_WAIT	Latest start time for "Wait"; specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Advice: See Background for key EARLIEST_START_LAY!	☐	DateMs

Key	Explanation	IMSApi data type
PART_DESC	Part description in the iTAC system	☐ String
PART_NUMBER	Part number in the iTAC system	☐ String
PLANNED_SHIP_DATE	Planned delivery date for the work order (cf. the APS_END_DATE key); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PLANNED_START_DATE	Planned start date for the work order (start of production) (cf. the APS_START_DATE key); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs
PRIORITY_CODE	APS service - unique designator for the work order priority	☐ String
PRIORITY_NAME	APS service - description for the work order priority	☐ String
PRIORITY_VALUE	Value for the work order priority; the greater the value, the higher the prioritization	☐ int
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
PROCESS_VERSION	Process version from the iTAC system	☐ int
PRODUCTION_CONTROLLER	Production controller	☐ String
PRODUCTION_CONTROLLER_DESC	Description of the production controller	☐ String
QUANTITY	Quantity of production order	☐ double
PROGRESS_PERCENT	Progress of the work step in percent Advice: This value is passed manually; automatic calculation does not take place. The work step progress is only relevant in combination with the "started" work step state (see the key WORK-STEP_STATE)!	☐ double
SPLIT_PARENT_WORKORDER	Order number of the basic order Advice: This information is only determined if the requested work order is a production lot (partial order)!	☐ String
WORKORDER_DESC	Order description	☐ String
WORKORDER_NUMBER	Production order number	☐ String
WORKORDER_SEQUENCE	Sequence number of the production order Background: Sequence numbers are used for processing production orders in a specified sequence. This key could then be applied as a sorting criterion e.g. in the case of JIS production (Just In Sequence). The sequence number is usually adopted from an external ERP system, if necessary optimized by a connected APS system, and then passed to a customer-specific client application. In other words, the sequence number is only interpreted by customer-specific client logic.	☐ long
WORKORDER_START_DATE	Start date for the work order (start of production); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000.	☐ DateMs

Key	Explanation	IMSApi data type
WORKORDER_STATE	State of the production order [F; N; S; P; E; X]: F = opened (released) N = not released S = started P = paused E = finished X = canceled (deleted)	☐ String
WORKORDER_TYPE	Work order type [E; F; I; M; R; S; U; X]: E = Maintenance order (corresponds to "Alt. WP": MAI) F = Production order (corresponds to "Alt. WP": 01, WO or CO) I = Receiving (corresponds to "Alt. WP": IG) M = Material test ("Alt. WP" is customer-specific) R = Repair order (corresponds to "Alt. WP": REP) S = External products ("Alt. WP" is customer-specific) U = Modification order (corresponds to "Alt. WP": U) X = SCT order ("Alt. WP" is customer-specific)	☐ String
WORKPLAN_VERS	Work plan version from the iTAC system	☐ int
WORKSTEP_AVO	Work step number in the ERP system (AVO)	☐ String
WORKSTEP_DESC	Description of the work step	☐ String
WORKSTEP_NUMBER	Work step number in the iTAC system	☐ int
WORKSTEP_STATE	Work step state "cross-station" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice regarding "cross-station": This work step state is the total state of all stations assigned to the work step via the ERP group (cf. the key WORKSTEP_STATION_STATE)!	☐ String
WORKSTEP_STATION_STATE	Work step state "station-specific" [P; R; S; L; B; F]: P = planned R = released S = started L = locked B = interrupted F = finished Advice: If multiple stations are assigned to the work step (or, more precisely, to the assigned ERP group), the key WORKSTEP_STATE (cross-station total state) must also be taken into consideration!	☐ String

Tab. 26-125 "Key Value Pair": *workorderResultKeys*

26.41.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: On basis of the station number, the line is ascertained for which the corresponding production orders are output!
workorderFilter	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the output is to be filtered; the respective values are passed with the value parameter. A description of the possible filter parameters can be found above in the "Key Value Pair": workorderFilter table.
value	Value list for the parameters passed by means of key . Advice: Refer to the "Key Value Pair": workorderFilter table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
workorderResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": workorderResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) workorderResultValues !

Tab. 26-126 Parameter (in) »trGetWorkOrderForStation«

Parameter (out)

Parameter name	Explanation
workorderResultValues	Depending on parameter (in) workorderResultKeys , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) workorderResultKeys !

Tab. 26-127 Parameter (out) »trGetWorkOrderForStation«

26.41.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-2070	Failure	IMSApi key was passed multiple times to the function

26.42 trIdentifyPoolSerialNumber

26.42.1 Function description

This function is used to identify one or all serial numbers of a serial number pool.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶] and »trRemoveSerialNumberFromSerialNumberPool« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trIdentifyPoolSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in String serialNumberPoolName
```

26.42.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial no. of the panel Special case: If value [-1] is passed, all serial numbers of the pool are identified; in this case, the pool name (serialNumberPoolName) must absolutely be passed!
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!

Parameter name	Explanation
<code>serialNumberPool-Name</code>	Name of the serial number pool Special case: If value [-1] is passed, no assignment check takes place; in this case, a serial number (<code>serialNumber</code>) must absolutely be passed!

Tab. 26-128 Parameter (in) »trIdentifyPoolSerialNumber«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.42.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-287	Failure	Serial number pool does not exist
-293	Failure	Serial number is not assigned to any pool

26.43

trMergeParts

26.43.1

Function description

The function merges two products (merge slave product into master product). Prior to installation, a check is performed to determine whether the state of progress of the slave serial number is OK, i.e., that all work steps with obligatory confirmation were booked with "PASS" and that the serial numbers may be mounted in one another according to the BOM; the product structure must be mapped in the BOM module. Furthermore, the executing station must be deposited at the first work step with obligatory confirmation of the master work plan via the ERP group used there.

If the value of the parameter `duplicateSerialNumber` is [0] or [2], only the reference serial numbers of two products are merged. If the value of the `duplicateSerialNumber` parameter is [1] or [3], every sub-number (single panel) is merged with a master according to a specified order. Take special notice here that scrapped serial numbers of a multiple panel can only be merged with another product if server parameter code no. 2032 [▶▶▶] is deactivated. Reason: If the parameter is active, all scrapped serial numbers are automatically deactivated, making product merging impossible.



Advice

This API function is influenced by station parameter code nos. 7203 [▶▶▶], 7206 [▶▶▶], 7208 [▶▶▶], 7223 [▶▶▶], 7260 [▶▶▶], 8310 [▶▶▶] and 18200 [▶▶▶], the site parameters 7070 [▶▶▶], 7260 [▶▶▶] and 18200 [▶▶▶] as well as server parameter 2032 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*Depending on the configuration of the station parameter 7208 [▶▶▶], this API function – unlike nearly all other API functions – can be used **cross-site**. Thus, products (serial numbers) from different sites can be mounted in one another!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trMergeParts
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in int duplicateSerialNumber
in String serialNumberMaster
in String serialNumberMasterPos
in String serialNumberSlave
in String serialNumberSlavePos
```

26.43.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: The orientation is needed in connection with the passed station in order to uniquely determine the "installing" work step!
duplicateSerial-Number	If the serialNumberSlave is part of a multiple panel, this parameter determines whether only the passed or all panels are mounted [0; 1; 2; 3]: 0 = Single panel; only the passed slave serial number is mounted (only work steps with obligatory confirmation) 1 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and mounted (only work steps with obligatory confirmation) 2 = Single panel; the passed slave serial number is mounted only (all work steps) 3 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and mounted (all work steps) Advice regarding "mode differentiation": The difference between mode 0 and 2 and between 1 and 3 is that in mode 2 and 3 the state of work steps without obligatory confirmation is also taken into consideration during the mounting check! For all modes, either all or only the current work step is checked; this is controlled using station parameter code no. 7203 [▶▶▶]! Advice: If the value [1] or [3] is passed (multiple panel), slave serial numbers are assigned to master serial numbers on the basis of the same panel position; i.e., slave panel position 1 is checked against master panel position 1; the number of panels must be identical for slave and master!
serialNumberMaster	Serial number which is retained during a product merging (see serialNumberSlave)
serialNumberMasterPos	Position of the single panel (here serialNumberMaster) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the master serial number must be unique!
serialNumberSlave	Serial number which is deleted during a product merging (see serialNumberMaster)

Parameter name	Explanation
serialNumber-SlavePos	Position of the single panel (here <code>serialNumberSlave</code>) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the slave serial number must be unique!

Tab. 26-129 *Parameter (in) »trMergeParts«*



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.43.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
221	Warning	The slave serial no. state is FAIL on last work step
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
		Advice: The value 207 is only output if station parameter code no. 18200 [▶▶▶] is active!
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-221	Failure	The slave serial no. is not released from previous process
-227	Failure	The slave serial no. has been booked from a previous process as SCRAP
-228	Failure	Slave serial number is locked

-232	Failure	Failure during product merging (merge)
-234	Failure	Number of panels (master) and slave serial nos. are not equal
-353	Failure	Mounting without order network not possible
		Advice: The value -353 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 2!
-354	Failure	Work orders of serial numbers to be mounted do not match order network
		Advice: The value -354 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 1!
-355	Failure	Production order is part of a work order cluster
-2100	Failure	Serial number is sent to SCT
-2101	Failure	Slave serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.44

trRemoveMergeParts

26.44.1

Function description

This function takes the serial number passed to it and removes it from the master product. This requires that the serial number has been built into the product as a slave, i.e., that a product merger has taken place in the past. In the default mode (for further information, see station parameter code no. 7207 [▶▶▶]), the original slave serial number is reactivated after a successful product separation ("unmerge").

**Advice**

This API function is influenced by station parameter code nos. 7207 [▶▶▶] and 8310 [▶▶▶] as well as site parameter code no. 22106 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*The API function can – unlike nearly all other API functions – be used **cross-site**. Thus, products (serial numbers) from other sites can be disassembled!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trRemoveMergeParts
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumberSlave
in String serialNumberSlavePos
in String textInfo
```

26.44.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: The orientation is needed in connection with the passed station in order to uniquely determine the "uninstalling" work step!
serialNumberSlave	Serial number which is deleted during a product merging (see serialNumberMaster)
serialNumber-SlavePos	Position of the single panel (here serialNumberSlave) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the slave serial number must be unique!
textInfo	Information regarding the removal from the assembly (e.g. name, reason for error) Advice: This field must absolutely be filled!

Tab. 26-130 Parameter (in) »trRemoveMergeParts«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.44.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-238	Failure	Failure during product separation (unmerge)

26.45

trRemoveSerialNumberFromOrder

26.45.1

Function description

This function is used to remove a non-processed serial number from the assigned work order. It can be used to e.g. undo an inadvertently performed work order assignment (see e.g. the IMSApi function »trAssignSerialNumberForProductOrWorkOrder« [▶▶▶]).



Advice

A serial number can only be removed from the work order if it has not yet been processed (booked)! All work order types are supported (e.g. production, repair or modification order)!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trRemoveSerialNumberFromOrder
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in KeyValue[] removeSerialNumberKeys
    String key
    String value
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameter (in) `removeSerialNumberKeys`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Definition of the serial numbers to be removed

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: This parameter is only relevant for the "Multiple panel" application type; it MUST be passed if the uniqueness of the serial number (SERIAL_NUMBER) can only be guaranteed by specifying the position!	☐ String
WORKORDER_NUMBER	Number of the work order; all work order types are supported (e.g. production, repair or modification order). Advice: A work order number must always be passed if the uniqueness of the serial numbers can only be determined in combination with the associated work order (special case)!	☐ String

Tab. 26-131 "Key Value Pair": `removeSerialNumberKeys`

26.45.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
removeSerialNumberKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>removeSerialNumberKeys</code> table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": <code>removeSerialNumberKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-132 Parameter (in) »`trRemoveSerialNumberFromOrder`«**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.45.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-131	Failure	No work order found for serial number
-202	Failure	Serial number is locked
		Advice: Setting a lock is also possible for unbooked serial numbers (special case); in this case, the function outputs the value -202. If the lock is removed, the serial number can also be removed from the work order!
-301	Failure	Serial number is mounted
-315	Failure	Serial number has already been booked

-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.46

trRemoveSerialNumberFromSerialNumberPool

26.46.1

Function description

With this function, a specific serial number can be removed from a serial number pool.



Advice

Serial number pools are required for modification orders; in this context, also note the »trAddSerialNumberToPool« [▶▶▶], »trAddSerialNumberToSerialNumberPool« [▶▶▶], »trChangePoolSerialNumber« [▶▶▶], »trChangeSerialNumberPool« [▶▶▶], »trCreateSerialNumberPool« [▶▶▶], »trDeleteSerialNumberPool« [▶▶▶], »trGetSerialNumberPoolInfo« [▶▶▶], »trGetPools« [▶▶▶], »trGetPoolSerialNumber« [▶▶▶] and »trIdentifyPoolSerialNumber« [▶▶▶] functions!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trRemoveSerialNumberFromSerialNumberPool
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in String serialNumberPoolName
```

26.46.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
serialNumberPos	Position of the single panel in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case the serial number must be unique!
serialNumberPool-Name	Name of the serial number pool Special case: If value [-1] is passed, no assignment check takes place; in this case, it is irrelevant in which pool the serial number is located!

Tab. 26-133 Parameter (in) »trRemoveSerialNumberFromSerialNumberPool«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

26.46.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-287	Failure	Serial number pool does not exist
-293	Failure	Serial number is not assigned to any pool
-294	Failure	Serial number is assigned to another pool
-295	Failure	Removing the serial number is not allowed (invalid state)

26.47

trSerialNumberPassing

26.47.1

Function description

This function can be used to move serial numbers – if required also complete multiple panels – to a different production order.



Advice

This API function is influenced by station parameter code nos. 7223 [▶▶▶], 7224 [▶▶▶], 7258 [▶▶▶] and 13420 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!

The array returned contains only records that were incorrectly processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!

When using this function, note the following basic principles:

- One or more serial numbers can be moved to a different production order simultaneously.
- Serial numbers which have been successfully moved to a different work order are identified by the "Forwarded" (C) processing state in the original work order.
- As soon as one of the passed serial numbers cannot be adopted, only the serial number concerned is rejected; all successfully processed serial numbers of the array are adopted.
- Serial numbers can be moved ONLY when they are active; furthermore, they are not allowed to be in the locked or archived state.
- Serial numbers CANNOT be moved to a work order cluster.
- **Multiple panels:**
If SERIAL_NUMBER_DUPLICATE is set, all panels are adopted in the new production order. If a panel in a multiple panel cannot be processed, the entire panel will not be adopted. In this case, it must be determined whether or not the serial numbers have already been booked. In the case of a multiple panel, the "booked: yes/no" test must also be performed for all individual serial numbers.
 - If the serial numbers have not yet been booked, they can be transferred to the target work order without further testing. The serial numbers are deactivated in the source work order.
 - If the serial numbers have already been booked, all booked work steps must also be present in the target work order. The work steps up to the last booking are important here. After the last booking, the work steps of the source and target work order can differ. However, the ERP groups in the new work order must also contain the stations of the existing bookings. Furthermore, the position of the work steps must be identical in both the source and target work order. The target work order can contain additional work steps; however, these work steps must NOT be obligated to supply confirmation.
 - When the serial numbers are moved, the serial numbers in the source work order are deactivated.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trSerialNumberPassing
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String[] serialNumberPassingKeys
in String[] serialNumberPassingValues
in String[] serialNumberPassingResultKeys
out String[] serialNumberPassingResultValues
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) serialNumberPassingKeys, serialNumberPassingValues and serialNumberPassingResultKeys as well as (out) serialNumberPassingResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Input parameters for moving serial numbers

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel which is to be moved to the production order (WORKORDER_NUMBER) Advice: If the multiple panel function is active (see SERIAL_NUMBER_DUPLICATE), a label serial number (panel label) can be entered here instead!	☒ String
SERIAL_NUMBER_DUPLICATE	Moving all serial numbers of a multiple panel [0; 1]: 0 = Multiple panel function NOT active (default setting) 1 = Multiple panel function active Advice: If this key is not passed along, the [0] value is used by default! Background [1]: If the value [1] is passed, all serial numbers of the multiple panel are moved to the specified work order; for this purpose, the passed serial number (SERIAL_NUMBER) must not necessarily be the reference serial number!	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String
WORKORDER_NUMBER	Production order number	☒ String

Tab. 26-134 "Key Value Pair": serialNumberPassingKeys

Output parameters for result feedback

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☐ int
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String

Tab. 26-135 "Key Value Pair": *serialNumberPassingResultKeys*

26.47.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberPassingKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <i>serialNumberPassingValues</i> parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberPassingValues table. Advice: Only the parameters passed here are processed by the function!
serialNumberPassingValues	Values for the parameters selected by means of <i>serialNumberPassingKeys</i> . Advice: The data is assigned in the order in which it was entered by means of parameter (in) <i>serialNumberPassingKeys</i> ! Refer to the "Key Value Pair": serialNumberPassingKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
serialNumberPassingResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberPassingResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) <i>serialNumberPassingResultValues</i> !

Tab. 26-136 Parameter (in) »trSerialNumberPassing«

Parameter (out)

Parameter name	Explanation
serialNumberPassingResultValues	Depending on parameter (in) <i>serialNumberPassingResultKeys</i> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <i>serialNumberPassingResultKeys</i> !

Tab. 26-137 Parameter (out) »trSerialNumberPassing«

26.47.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
		Advice: The value 233 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
		Advice: The value 230 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
222	Warning	The slave serial no. state is SCRAP on last work step
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
210	Warning	At least one serial number has the SCRAP state
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station

-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-128	Failure	Work step not found
-130	Failure	Work order not found
		Advice: The value -168 is then output if a value other than [0] or [2] is entered in the "Key Value Pair" structure failureSlipKeys for the UPDATE_FLAG key!
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked
-206	Failure	Serial number was already assigned to this work order
-213	Failure	Maximum number of bookings reached
-222	Failure	The serial no. state is SCRAP on last work step
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-269	Failure	FAIL state due to anomaly
-322	Failure	No previous booking found for this serial number and this work step
-355	Failure	Production order is part of a work order cluster

-364	Failure	Serial number must be ejected
-365	Failure	Serial number has already been copied to the work order
-702	Failure	Recipe header data not found
-709	Failure	Measurement step names are not unique
-710	Failure	Measurement data does not match the active recipe
-712	Failure	No measurement data passed
-721	Failure	Recipe required for measurement data booking
-1105	Failure	BOM for the work order not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2110	Failure	Serial number position is invalid
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-2127	Failure	Split ID not activated
-2138	Failure	Serial number is deleted

26.48

trSimulateMerge

26.48.1

Function description

This function checks whether it would be possible to mount a semifinished product that has already been mounted (slave serial number) in another semifinished product (master serial number). This function thereby simulates that a mounted slave serial number is "theoretically" not mounted and then executes a (simplified) version of the »trVerifyMerge« [▶▶▶] function. When using this function, note the following points:

- The (simulated) check is only possible for **single panels**.
- The (simulated) check takes into account not only active but also **inactive** slave or master serial numbers.
- The (simulated) check is also performed if the permissible number of mounted slave serial numbers – as specified in parts list – is reached.
- No provision is made for the automatic determination of the **active work order** on the station and use for the (simulated) check.
- The (simulated) check is only possible if slave and master serial numbers can be uniquely assigned. If unique assignment is not possible (see below for example), the possible integration levels of the corresponding serial numbers are returned in the "Key Value Pair" structure. Based on the order numbers specified there, the function can be called again.

Background: "Non-uniqueness" occurs if a serial number is ascertained (independent of whether slave or master) that is used in multiple integration levels and no order number was passed.

Example: With work order 1, semifinished product A (slave) was mounted; in connection with work order 2, the semifinished product was mounted in finished product B (master). During this process, the serial number of A (SNO12) was also used for B. In this case, serial number SNO12 exists in two different integration levels: once inactive (lower integration level, work order 1) and once active (upper integration level, work order 2). A unique determination of serial number SNO12 is thus only possible in connection with the order number.



Advice

This API function is influenced by station parameter code nos. 7208 [▶▶▶] and 7260 [▶▶▶] as well as site parameter code no. 7260 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Depending on the configuration of the station parameter 7208 [▶▶▶], this API function – unlike nearly all other API functions – can be used **cross-site**. Thus, products (serial numbers) from different sites can be mounted in one another!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```

int trSimulateMerge
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberMaster
in String serialNumberMasterPos
in String workorderNumberMaster
in String serialNumberSlave
in String serialNumberSlavePos
in String workorderNumberSlave
in String[] simulateMergeResultKeys
out String[] simulateMergeResultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `simulateMergeResultKeys` and (out) `simulateMergeResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BOM_INDEX	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems	☐	String
BOM_VARIANT	BOM type [E; F; I; M; R; S; U; X]; the BOM type is optional; if it is set, a plausibility check with respect to the work plan alternative can be performed (see the bracket term for each BOM type): E = BOM for maintenance order (MAI) F = BOM for production order (01 , WO or CO) I = BOM for receiving (IG) M = BOM for material test (customer-specific) R = BOM for repair order (REP) S = BOM for external products (customer-specific) U = BOM for modification order (U) X = BOM for SCT order (customer-specific) Advice: The work plan alternatives for work order types [M], [S] and [X] are not predetermined; they are defined during the course of commissioning (recommended: MT for [M], EXT for [S] and SCT for [X])!	☐	String
BOM_VERSION	BOM version from the iTAC system	☐	int
BOM_VERSION_ERP	BOM version in the ERP system	☐	String
PART_NUMBER	Part number in the iTAC system	☐	String

Key	Explanation	IMSApi data type
SERIAL_NUMBER	Serial number of the panel	☐ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_TYPE	Identifies whether this is a master or a slave serial number [M; S]: M = Master serial number (upper integration level) S = Slave serial number (lower integration level)	☐ char
WORKORDER_NUMBER	Production order number	☐ String

Tab. 26-138 "Key Value Pair": *simulateMergeResultKeys*

26.48.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberMaster	Serial number which is retained during a product merging (see serialNumberSlave)
serialNumberMasterPos	Position of the single panel (here serialNumberMaster) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the master serial number must be unique!
workorderNumberMaster	Number of the production order (master serial number) Advice: If the value [-1] is passed here, the position is not taken into account; in this case the master serial number must be unique! If a unique assignment is not possible, the corresponding integration levels of the serial number are returned in the "Key Value Pair" structure!
serialNumberSlave	Serial number which is deleted during a product merging (see serialNumberMaster)
serialNumberSlavePos	Position of the single panel (here serialNumberSlave) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the slave serial number must be unique!
workorderNumberSlave	Number of the production order (slave serial number) Advice: If the value [-1] is passed here, the position is not taken into account; in this case the slave serial number must be unique! If a unique assignment is not possible, the corresponding integration levels of the serial number are returned in the "Key Value Pair" structure!
simulateMergeResultKeys	List of the parameters that are to be ascertained by the function. A description of the possible parameters can be found above in the "Key Value Pair": simulateMergeResultKeys table. Advice: Only the parameters passed here are output by the function; see parameter (out) simulateMergeResultValues!

Tab. 26-139 Parameter (in) »trSimulateMerge«

Parameter (out)

Parameter name	Explanation
simulateMergeResultValues	Depending on parameter (in) simulateMergeResultKeys, the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) simulateMergeResultKeys!

Tab. 26-140 Parameter (out) »trSimulateMerge«

26.48.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-220	Failure	Mounting (merge) is invalid
-240	Failure	No work step found for work order, orientation of the PCB and station
-301	Failure	Serial number is mounted
		Advice: Output value -301 is not to be interpreted as a failure, but rather as a warning; it is possible to mount the serial number if it is not already installed elsewhere!
-353	Failure	Mounting without order network not possible
		Advice: The value -354 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 2!

-354	Failure	Work orders of serial numbers to be mounted do not match order network
		Advice: The value -354 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 1!
-2070	Failure	IMSApi key was passed multiple times to the function

26.49

trSplitPanel

26.49.1

Function description

This function enables the splitting of a multiple panel. It can be called based on a reference serial number or a label serial number (panel label). It is important to note that not only complete splitting is possible but that "partial multiple panels" can also be generated. In this way, it would be possible, for example, to split an 8x panel into four single panels and two 2x panels.



Advice

*The array returned contains records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Observe the following points if you would like to work with this function:

- The serial number state of the respective panel remains unchanged when using this function
- Multiple panels can be identified via a reference serial number, a single panel serial number or a label serial number.
- When breaking down into "partial multiple panels", new label serial numbers can also be created if necessary.
- If a failure occurs during processing, the entire separation process – i.e., including separations that have already been successfully completed – is reversed (rollback).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trSplitPanel
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumberRef
  in String serialNumberRefPos
  in int functionMode
  in String[] splitPanelKeys
  in String[] splitPanelValues
  out String[] splitPanelResults
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `splitPanelKeys` and `splitPanelValues` as well as (out) `splitPanelResults`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
SERIAL_NUMBER	Serial number of the panel that is to be assigned to the new "partial" multiple panel or single panel	☐ String
SERIAL_NUMBER_POS	New position of the panel on the new "partial" multiple panel or single panel Advice: This key is only evaluated in connection with a passed SERIAL_NUMBER!	☐ String
SERIAL_NUMBER_REF	Serial number for the identification of the "new" partial multiple panel; possible here are the entries: - reference serial number - label serial number (abbreviated: panel label) - single panel serial number (see advice)	☒ String
SERIAL_NUMBER_REF_POS	Position of the reference serial number or label serial number for the new "partial" multiple panel (see SERIAL_NUMBER_REF). Advice: In the case of "label serial number", the value [0] must be passed here as position!	☒ String

Tab. 26-141 "Key Value Pair": *splitPanelKeys*

26.49.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Serial number of the multiple panel; possible here are the entries: - single panel serial number - reference serial number - label serial number (abbreviated: panel label) Advice: On the basis of the number passed here – if necessary, in connection with the <code>serialNumberRefPos</code> parameter – the multiple panel that is to be split is ascertained in the system!
serialNumberRefPos	Position of the serial number on the multiple panel; possible here are the entries: - single panel position - reference panel position - panel label position (always "0") Advice: If the value [-1] is passed, the position is ascertained automatically!
functionMode	Mode for the separation process [-1; 0] -1 = Completely break down into individual panels; the new serial number positions are always "1"; no data needs to be passed with the "Key Value Pair" structure 0 = The multiple panel is split on the basis of the passed structures – for further information, see the "Key Value Pair": splitPanelKeys table.
splitPanelKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>splitPanelValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": splitPanelKeys table. Advice: Only the parameters passed here are processed by the function!
splitPanelValues	Values for the parameters selected by means of <code>splitPanelKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>splitPanelKeys</code> ! Refer to the "Key Value Pair": splitPanelKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-142 Parameter (in) »trSplitPanel«

Parameter (out)

Parameter name	Explanation
splitPanelResults	Depending on parameter (in) <code>splitPanelKeys</code> , the corresponding data is output here. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>splitPanelKeys</code> !

Tab. 26-143 Parameter (out) »trSplitPanel«

26.49.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
209	Warning	Serial number is not assigned to a panel
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-150	Failure	Invalid value for "Flag"
-196	Failure	A parameter is invalid
-234	Failure	Number of panels (master) and slave serial nos. are not equal
-272	Failure	Serial number already exists

-348	Failure	Splitting of multiple panel failed
-2063	Failure	Invalid value for "functionMode"
-2070	Failure	IMSApi key was passed multiple times to the function
-2138	Failure	Serial number is deleted

26.50

trSwitchSerialNumber

26.50.1

Function description

This function switches (names) serial numbers of single panels or serial numbers within a multiple panel. The serial numbers of the multiple panel for which the "switch" (renaming) is to be performed are specified with the aid of the reference serial number and the panel position.

**Advice**

*Serial numbers which have already been registered in the system can no longer be used for renaming. The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Application type "multiple panel"

Condition: If the `serialNumberRef` parameter is set – i.e., no empty string and a value other than [-1] is passed, all serial numbers of the multiple panel are loaded – the function is in the "multiple panel" mode. In this case, the `serialNumberRefPos` parameter is also considered; it can take the value [-1].

Sequence: For each serial number of the multiple panel, the corresponding `serialNumberPos` is searched for in the passed `serialNumberArray`. If it is found, the serial number of the multiple panel is switched at this position to the value for `serialNumberNew`. In addition, observe the following:

- The `serialNumberPos` parameter in the `serialNumberArray` must be specified and **MUST** not have the value [-1].
- The `serialNumberAct` parameter in the `serialNumberArray` is not evaluated for this case.

Application type "single panel"

Condition: If the `serialNumberRef` parameter is NOT set – i.e., an empty string or the value [-1] is passed – the serial numbers of the multiple panel are not loaded – the function is in the "single panel" mode.

Sequence: Here, the system iterates over all entries in the `serialNumberArray` and searches for the serial number for each entry according to `serialNumberAct` and `serialNumberPos`. As long as the serial number is unique, the value [-1] can be passed for `serialNumberPos`. If a serial number is found, it is switched to the value for `serialNumberNew`. If no serial number is found or if it is non-unique, a failure is output. In addition, observe the following:

- The `serialNumberPos` parameter in the `serialNumberArray` does not need to be specified; it may have the value [-1].
- The `serialNumberAct` parameter in the `serialNumberArray` is **ALWAYS** evaluated.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trSwitchSerialNumber
in IMSSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumberRef
in String serialNumberRefPos
inout SwitchSerialNumberData[] serialNumberArray
    int index
    String serialNumberNew
    String serialNumberPos
    String serialNumberAct
    int returnCode
```

26.50.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Special case: If value [-1] is passed here and for serialNumberRefPos, the serialNumberAct parameter must be passed in the array!
serialNumberRefPos	Position of the serial number of the first single panel or unit-array Special case: If value [-1] is passed here and for serialNumberRefPos, the serialNumberAct parameter must be passed in the array!

Tab. 26-144 Parameter (in) »trSwitchSerialNumber«

Parameter (inout)

Parameter name	Explanation
serialNumberArray	Array of the structure with the following variables:
index	in: Unique reference number (sequential number) for each record in the array out: Reference number of the faulty record Advice: The records of an array must be numbered uniquely; in the event of an error (see the returnCode parameter), the affected record can be assigned on the basis of this reference number!
serialNumberNew	in: "New" serial number is to be used out: "New" serial number that should have been used Advice: The "out" value is only returned in the event of an error! Serial numbers which have already been registered in the system can no longer be used for renaming!

Parameter name	Explanation
serialNumberPos	in: Position of the single panel in the unit-array out: Position of the single panel in the unit-array Advice: The "out" value is only returned in the event of an error! Special case: Only in the "single panel" application type can the value [-1] be passed here; in this case, the serial number must be unique!
serialNumberAct	in: "Current" serial number that is to be renamed out: "Current" serial number that should have been renamed Advice: The "out" value is only returned in the event of an error! This parameter is only evaluated in application type "Single panel"!
returnCode	in: [empty] - value is not evaluated out: Error code for the case of a faulty processing of the respective record; to identify the faulty record, see the <code>index</code> parameter Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!

Tab. 26-145 Parameter (inout) »trSwitchSerialNumber«

26.50.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-196	Failure	A parameter is invalid
-239	Failure	Serial number array is empty
-246	Failure	Rename of the serial number failed
-272	Failure	Serial number already exists

26.51

trTestPaa

26.51.1

Function description

This function carries out a PAA (Part Average Analysis) for the serial number passed, which permits an early detection of (sporadic) failures in the field. Here, the measurement data of the individual item are compared with those of other individual items, inspected for unusual features (anomalies), and a characteristic risk figure is calculated. As a result, this function shows whether or not the individual item is an outlier and carries a higher risk of failure.



Advice

This function is designed to post-test individual items already manufactured; if you wish to carry out the PAA directly in the production process, the »trUploadResultDataAndRecipe« [▶▶▶] or »trUploadFailureAndResultData« [▶▶▶] API functions may be used for this purpose!

This API function is influenced by station parameter code nos. 7210 [▶▶▶], 7211 [▶▶▶], 7212 [▶▶▶], 7213 [▶▶▶], 7214 [▶▶▶], 7215 [▶▶▶], 7216 [▶▶▶] and 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trTestPaa
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumber
  in int processLayer
  in int model
  out int resultPaa
  out String infoText
```


26.51.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
model	Determines with which other serial numbers the passed serial number is to be compared [0; 1; 2]: 0 = Band end (default); the measurement data of the passed serial number are compared with the data of previously produced individual items 1 = Band center; the measurement data of passed serial number are compared with the data of previously produced individual items and of subsequently produced individual items in equal amounts 2 = Band start; the measurement data of the passed serial number are only compared with the data of subsequently produced individual items Advice: The number of individual items to be compared (PAA interval) is specified in the work plan!

Tab. 26-146 Parameter (in) »trTestPaa«

Parameter (out)

Parameter name	Explanation
resultPaa	Result of the PAA [0; 1]: 0 = OK >0 = Anomaly; the output value corresponds to the highest risk ratio of the observed measured values
infoText	Output of an information text

Tab. 26-147 Parameter (out) »trTestPaa«

26.51.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
233	Warning	Anomaly detected (PAA)
230	Warning	Not enough values for the PAA
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-175	Failure	Invalid value for "model" [0; 1; 2]
-263	Failure	Station is not configured for the PAA test
-264	Failure	Work plan is not configured for the PAA test
-265	Failure	Recipe is not configured for the PAA test
-266	Failure	Serial number is not configured for the PAA test
-725	Failure	Measurement data could not be processed

26.52

trTestSpa

26.52.1

Function description

This function carries out an SPA (Statistical Process Analysis) for the serial number passed (individual item), which permits an early detection of systematic failures in the field. The function calculates the process capability indices Cp and Cpk for the individual item and compares them to the values of other individual items. As a result, the function returns pre-interpreted Cp and Cpk values, hence permitting a classification as to whether or not the individual item is an outlier. The results of this calculation affect the overall PAA result.



Advice

This function is designed to post-test individual items already manufactured; if you wish to carry out the SPA in combination with the PAA directly in the production process, the »trUploadResultDataAndRecipe« [▶▶▶] or »trUploadFailureAndResultData« [▶▶▶] API functions may be used for this purpose!

This API function is influenced by station parameter code nos. 7210 [▶▶▶], 7211 [▶▶▶], 7212 [▶▶▶], 7213 [▶▶▶], 7214 [▶▶▶], 7215 [▶▶▶], 7216 [▶▶▶], 7250 [▶▶▶], 7251 [▶▶▶], 7252 [▶▶▶], 7253 [▶▶▶], 7254 [▶▶▶], 7255 [▶▶▶], 7256 [▶▶▶] and 8310 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trTestSpa
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumber
  in int processLayer
  in int model
  out int resultCp
  out int resultCpk
  out String infoText
```

26.52.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumber	Serial number of the panel
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
model	Determines with which other serial numbers the passed serial number is to be compared [0; 1; 2]: 0 = Band end (default); the measurement data of the passed serial number are compared with the data of previously produced individual items 1 = Band center; the measurement data of passed serial number are compared with the data of previously produced individual items and of subsequently produced individual items in equal amounts 2 = Band start; the measurement data of the passed serial number are only compared with the data of subsequently produced individual items Advice: The number of individual items to be compared (PAA interval) is specified in the work plan!

Tab. 26-148 Parameter (in) »trTestSpa«

Parameter (out)

Parameter name	Explanation
resultCp	Result of the Cp calculation [0; 1; 2]: 0 = OK (green) 1 = Conditionally OK (yellow) 2 = Not OK (red)
resultCpk	Result of the Cpk calculation [0; 1; 2]: 0 = OK (green) 1 = Conditionally OK (yellow) 2 = Not OK (red)
infoText	Output of an information text

Tab. 26-149 Parameter (out) »trTestSpa«

26.52.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
232	Warning	Cpk violation
231	Warning	Cp violation
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-175	Failure	Invalid value for "model" [0; 1; 2]
-260	Failure	Station is not configured for the SPA test
-261	Failure	Work plan is not configured for the SPA test
-262	Failure	Recipe is not configured for the SPA test

26.53

trUploadActivity

26.53.1

Function description

This function books the process state in a work-step-conformant manner - including the executed work step activities - of one or more individual products (single panel) or of a complete multiple panel.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code nos. 8310 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 13420 [▶▶▶] and 22000 [▶▶▶] as well as server parameter code no. 19200 [▶▶▶]. Further information on this topic can be found in the documentation for the ADM module!*

Application type "A"

Single panel -> one serial number (`duplicateSerialNumber = 0`): The data of the serial number to be booked is passed with the function parameters (`processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate`, `cycleTime`); only the work step activity data has to be passed in the "Key Value Pair" structure.

Application type "B"

Single panel -> multiple serial numbers (`duplicateSerialNumber = 0` and `serialNumberRef = -1`): The data of the serial numbers to be booked is passed with the "Key Value Pair" structure. The `processLayer`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime` function parameters are not evaluated.

Application type "C"

Multiple panel -> all serial numbers of the multiple panel with identical data (`duplicateSerialNumber = 1`): The data of the multiple panel to be booked is passed with the function parameters `processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime`; all serial numbers (single panels) of the multiple panel are booked with the same values; only the work step activity data has to be passed in the "Key Value Pair" structure.

Application case "D"

Multiple panel -> all serial numbers of the multiple panel with different data (`duplicateSerialNumber = 1`): the (default) data of the multiple panel to be booked are passed with the `processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime` function parameters. In the "Key Value Pair" structure, only those serial numbers are passed which deviate from the default data, here the `serialNumberState`, `bookDate` and `cycleTime` function parameters; the `processLayer`, `serialNumberRef` and `serialNumberRefPos` function parameters are not evaluated in this case.

**Advice**

*If no work step activity data is entered – "unofficially" this would then be **application type "E"** – the function executes a simple state booking; here, only the value 0 (PASS), 1 (FAIL) or 2 (SCRAP) is allowed to be used as the serial number state!*

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trUploadActivity
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumberRef
in String serialNumberRefPos
in int serialNumberState
in int duplicateSerialNumber
in DateMs bookDate
in float cycleTime
in String[] activityDataKeys
in String[] activityDataValues
out String[] activityDataResults
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `activityDataKeys` and `activityDataValues` as well as (out) `activityDataResults`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐)

Key	Explanation		IMSApi data type
ACTIVITY_NUMBER	Number of the work step activity	☒	String
ACTIVITY_PROCESSING	Processing control of the activities [-1; 0; 1]: -1 = books all activities with the specified state 0 = books only the specified activities with the specified state 1 = books all activities up to the specified activity number with the specified state	☐	int
ACTIVITY_SORT_SEQUENCE	Sorting number for the work step activity Advice: <i>This number determines the processing sequence in the event that multiple activities are stored in a work step!</i>	☒	int
ACTIVITY_STATE	Booking state of the work step activity [0; 1; 2; 3]: 0 = PASS 1 = FAIL 2 = OPEN 3 = SKIP	☒	int
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case (application type "B"): [-1] passes the current date and the current time! Special case (application types "A", "C" and "D"): [-1] uses the value from the <code>bookDate</code> function parameter!	☐	DateMs
CYCLE_TIME	Processing time at the station in seconds Special case: [-1] uses the value from the <code>cycleTime</code> function parameter!	☐	double
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: <i>As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!</i>	☒	int
SERIAL_NUMBER	Serial number of the panel	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: <i>If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!</i>	☐	String
SERIAL_NUMBER_STATE	Process state of the serial number [-2; -1; 0; 1; 2; 3; 4]: -2 = "Successor booking" state -1 = "Predecessor booking" state 0 = Pass 1 = Fail 2 = Scrap 3 = In process 4 = Invalid Advice: <i>In mode [-1], the activities are assigned to the state booking of the predecessor booking of the same work step; if the value [-2] is passed, the activities are stored temporarily and then assigned to the next »trUploadActivity« booking (successor booking)!</i>	☒	int

Tab. 26-150 "Key Value Pair": `activityDataKeys`

26.53.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: This field is not evaluated for application type B!
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Advice: In application type B, the value [-1] must be passed here. Also note the special case "Carrier" described above!
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: If the value [-1] is passed, the position is ascertained automatically! This field is not evaluated in application type B!
serialNumberState	Process state of the serial number [-2; -1; 0; 1; 2; 3; 4]: -2 = "Successor booking" state -1 = "Predecessor booking" state 0 = Pass 1 = Fail 2 = Scrap 3 = In process 4 = Invalid Background to [-1] and [-2]: In mode [-1], the activities are assigned to the state booking of the predecessor booking of the same work step; if value [-2] is passed, the activities are stored temporarily and then assigned to the next state booking (successor booking). Advice: This field is not evaluated for application type B!
duplicateSerial-Number	Single panel/multiple panel [0; 1]: 0 = Single panel (see application types A and B) 1 = Multiple panel (see application types C and D)
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time! Advice: This field is not evaluated for application type B!
cycleTime	Processing time at the station in seconds Advice: This field is not evaluated for application type B!

Parameter name	Explanation
activityDataKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>activityDataValues</code> parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": activityDataKeys table for attribute category. Advice: Only the parameters passed here are processed by the function! In application type A, nothing may be passed here!
activityDataValues	Values for the parameters selected by means of <code>activityDataKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: Refer to the corresponding table for attribute category "Key Value Pair": activityDataKeys to see whether special rules apply during the input of values (e.g., use of wildcards)! In application type A, nothing may be passed here!

Tab. 26-151 Parameter (in) »trUploadActivity«

Parameter (out)

Parameter name	Explanation
activityDataResults	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>activityDataKeys</code> and <code>activityDataValues</code>. Advice: The data is output in the order in which it was entered by means of parameter (in) <code>activityDataKeys</code>!

Tab. 26-152 Parameter (out) »trUploadActivity«

26.53.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
211	Warning	A secondary failure was detected
210	Warning	At least one serial number has the SCRAP state
209	Warning	Serial number is not assigned to a panel
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-155	Failure	Invalid value for "duplicateSerialNumber" [0; 1]

-181	Failure	Station is locked
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked
-205	Failure	Serial number belongs to a different work order
-222	Failure	The serial no. state is SCRAP on last work step
-240	Failure	No work step found for work order, orientation of the PCB and station
-340	Failure	Activity for the work step not found
-364	Failure	Serial number must be ejected
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2110	Failure	Serial number position is invalid
-2138	Failure	Serial number is deleted

26.54

trUploadFailureAndResultData

26.54.1

Function description

This function books work-step-conformant attributive failure data, failure notes or measurement data; in doing so, it is possible to select whether a state booking is to occur directly. Both single panels as well as complete multiple panels can be processed (`duplicateSerialNumber`). This function can be used not only to book new failure and measurement data, but also allows specific failure records for specific test runs (`SEQUENCE_NUMBER`) to be changed or deleted (for further information, see key `UPDATE_FLAG`).



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code nos. 4010 [▶▶▶], 4011 [▶▶▶], 7200 [▶▶▶], 7202 [▶▶▶], 7209 [▶▶▶], 7210 [▶▶▶], 7211 [▶▶▶], 7212 [▶▶▶], 7213 [▶▶▶], 7214 [▶▶▶], 7215 [▶▶▶], 7216 [▶▶▶], 7223 [▶▶▶], 7224 [▶▶▶], 7250 [▶▶▶], 7251 [▶▶▶], 7252 [▶▶▶], 7253 [▶▶▶], 7254 [▶▶▶], 7255 [▶▶▶], 7256 [▶▶▶], 7257 [▶▶▶], 7258 [▶▶▶], 7320 [▶▶▶], 7400 [▶▶▶], 7401 [▶▶▶], 7402 [▶▶▶], 8200 [▶▶▶], 8310 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 8510 [▶▶▶], 13410 [▶▶▶], 13420 [▶▶▶], 15250 [▶▶▶], 15260 [▶▶▶], 16000 [▶▶▶], 18200 [▶▶▶] and 22000 [▶▶▶] as well as site parameter code nos. 7070 [▶▶▶], 18200 [▶▶▶], 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]. In addition, the station settings for "Upload Settings" and "Setup" control this function. Further information on this topic can be found in the reference section of this documentation!*

If a **-1** is passed for `serialNumberState`, measurement data can be passed without a new state booking. This makes sense if failure data have already been booked via another "Upload" function and a measurement record is to be created retroactively for this booking without its own state booking. The requirement for this, however, is that a previous state booking of the respective station must be found. Otherwise, the process aborts with a failure (see error code = -322).

To ensure the assignment of state booking to measurement data, the measurement data should be booked with the state booking in quasi real-time (there is otherwise a risk of faulty DWH consolidation).

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trUploadFailureAndResultData
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String serialNumberRef
  in String serialNumberRefPos
  in int serialNumberState
  in int duplicateSerialNumber
  in float cycleTime
  in DateMs bookDate
  in String[] measureKeys
  in String[] measureValues
  out String[] measureResultValues
  in String[] failureKeys
  in String[] failureValues
  out String[] failureResultValues
  in String[] failureSlipKeys
  in String[] failureSlipValues
  out String[] failureSlipResultValues

```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) measureKeys, measureValues, failureKeys, failureValues, failureSlipKeys and failureSlipValues as well as (out) measureResultValues, failureResultValues and failureSlipResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Measurement data

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; the data objects can be put into context on the basis of this number (see parameter of the same name in the following "Failure data" and "failure note" tables)!	☐ long
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail"! Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	☒ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name) Advice: The measurement step name must occur only once within a recipe (the program differentiates between upper- and lower-case letters) and must not be longer than 80 characters; an empty string may not be passed!	☒ String
MEASURE_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*,[0-9]* Text (example: Hello world) .* Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☒ String
TEST_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐ double

Tab. 26-153 "Key Value Pair": *measureKeys*

Failure data

Key	Explanation		IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
FAILURE_ACTION_CODE	Failure action code Advice: The failure actions must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_ACTION_INFO	Description of the failure action	☐	String
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐	String
FAILURE_CAUSE_INFO	Individual information about the failure cause	☐	String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐	String
FAILURE_MEASURE_CODE	Failure measure code Advice: The failure measures must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_MEASURE_INFO	Description of the failure measure	☐	String
FAILURE_OBJECT-PART_CODE	Failure object part code Advice: The failure object parts must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_RISK_CODE	Failure risk code Advice: The failure risks must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_RISK_INFO	Description of the failure risk	☐	String
FAILURE_SEQUENCE_NUMBER	Sequence number of the faulty test run	☐	int
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☒	String
FAILURE_TYPE_INFO	Individual information about the failure type	☐	String
GRID_X	Failure position: X value according to part image raster	☐	int
GRID_Y	Failure position: Y value according to part image raster	☐	int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐	String

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; the data objects can be put into context on the basis of this number (see parameter of the same name in the following "Measurement data" and "Failure note" tables)! Special case: If value [1] (failure updating) is passed for the UP-DATE_FLAG key, the following special rules apply here: -1 = existing references are retained 0 = all previous references are deleted >0 = new references are created	long
INFO	Information text on failures	String
INFO_TXT_REPAIR	Information text for the repair	String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	int
PART_NUMBER	Part number of the faulty component conformant with the iTAC BOM	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	int
QUANTITY_SOLDER-ING_POINT_FAILURE	Number of faulty soldering points (pins)	int
REPAIRED	Repair state [0; 1; 2; 3]: 0 = New failure - not repaired 1 = New failure - repaired 2 = Existing failure - from repaired to not repaired 3 = Existing failure - from not repaired to repaired Advice: Failures are interpreted as "existing" if the passed failure location (COMP_NAME) and the failure type code (FAILURE_TYPE_CODE) already exist!	int
SEQUENCE_NUMBER	Display of the sequence number (test run)	int
UPDATE_FLAG	"Error handling" mode [0; 1; 2; 3]: 0 = new failure is created 1 = failure data is updated (e.g. repair state) 2 = reference number (INDEX) is updated 3 = error is deleted ("diagnosis stations" only) Advice: If this parameter is not passed along, the [0] mode is used by default! If the value [1] is passed here, the special cases for the INDEX key are to be observed! In mode [1], the following keys can be adapted at a later time: - INFO - INFO_TXT_REPAIR - FAILURE_CAUSE_CODE - FAILURE_IMAGE_CODE - FAILURE_TYPE_CODE - MAJOR_FAILURE - REPAIRED	int

Tab. 26-154 "Key Value Pair": failureKeys

Failure note

Key	Explanation	IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; the data objects can be put into context on the basis of this number (see parameter of the same name in the following "Measurement data" and "Failure data" tables)!	☐ long
TEST_STEP_NAME	Name of the test step Advice: Do not confuse this name with the measurement step name!	☒ String
TEST_STEP_VALUE	Failure note text Advice: The entry is limited to 1000 characters; if too many are entered, they are cut off; break characters are currently not supported!	☐ String
UPDATE_FLAG	"Failure handling" mode [0; 2]: 0 = new failure note is created 2 = Reference number (INDEX) is updated Advice: If this parameter is not passed along, the [0] mode is used by default!	☐ int

Tab. 26-155 "Key Value Pair": *failureSlipKeys*

26.54.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Advice: Also note the special case "Carrier" described above!
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: If the value [-1] is passed, the position is ascertained automatically!
serialNumberState	Process state of the serial number [-1; 0; 1; 2; 3; 4]: -1 = no state booking 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid Special case: If the value [-1] is passed, no state booking is carried out; the "loopCounter" remains unchanged and the most recently booked state remains valid! Advice for failure notes: For the case [-1], the failure note is appended to the highest sequence. If the function is again executed with serialNumberState = [-1] but a different failure note is supplied, the previous failure note is overwritten. Overwriting can only be avoided if the function is used to book a new serial number state ([0; 1; 2]) and thereby create a new sequence number!
duplicateSerial-Number	Single panel/multiple panel [0; 1]: 0 = single panel 1 = multiple panel; all serial numbers (single panels) of the multiple panel are booked with the same values
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
cycleTime	Processing time of the station in seconds

Parameter name	Explanation
measureKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>measureValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": measureKeys table. Advice: Only the parameters passed here are processed by the function!
measureValues	Values for the parameters selected by means of <code>measureKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>measureKeys</code>! Refer to the "Key Value Pair": measureKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>failureValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureKeys table. Advice: Only the parameters passed here are processed by the function!
failureValues	Values for the parameters selected by means of <code>failureKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>failureKeys</code>! Refer to the "Key Value Pair": failureKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
failureSlipKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>failureSlipValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": failureSlipKeys table. Advice: Only the parameters passed here are processed by the function!
failureSlipValues	Values for the parameters selected by means of <code>failureSlipKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>failureSlipKeys</code>! Refer to the "Key Value Pair": failureSlipKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-156 Parameter (in) »trUploadFailureAndResultData«

Parameter (out)

Parameter name	Explanation
measureResultValues	The corresponding data is output here only in the event of an error – depending on the <code>measureKeys</code> and <code>measureValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>measureKeys</code>!
failureResultValues	The corresponding data is output here only in the event of an error – depending on the <code>failureKeys</code> and <code>failureValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureKeys</code>!
failureSlipResultValues	The corresponding data is output here only in the event of an error – depending on the <code>failureSlipKeys</code> and <code>failureSlipValues</code> parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) <code>failureSlipKeys</code>!

Tab. 26-157 Parameter (out) »trUploadFailureAndResultData«

26.54.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
233	Warning	Anomaly detected (PAA)
		Advice: The value 233 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
		Advice: The value 230 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
222	Warning	The slave serial no. state is SCRAP on last work step
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
210	Warning	At least one serial number has the SCRAP state
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found

-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-128	Failure	Work step not found
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-168	Failure	Invalid value for the function mode (control flag)
		Advice: The value -168 is then output if a value other than [0] or [2] is entered in the "Key Value Pair" structure failureSlipKeys for the UPDATE_FLAG key!
-176	Failure	Ambiguous value for "REPAIRED" -> the data were booked as "not repaired"
-177	Failure	Mounting place is invalid
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked

-213	Failure	Maximum number of bookings reached
-222	Failure	The serial no. state is SCRAP on last work step
-231	Failure	Booking of the serial number failed
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-322	Failure	No previous booking found for this serial number and this work step
-330	Failure	Invalid value for "MAJOR_FAILURE"
-331	Failure	Failure type code is invalid
-332	Failure	Failure cause not found
-333	Failure	No value passed for "COMP_NAME"
-334	Failure	Location acc. to BOM not found
-335	Failure	Invalid process
		Advice: Value -335 is output if the combination of mounting place and orientation of the PCB is invalid!
-336	Failure	Failure symptoms not found
-342	Failure	Deleting failures not permitted
-343	Failure	Deleting failures not possible
-346	Failure	Failure note not found
-349	Failure	Failure measure not found
-350	Failure	Failure action not found
-351	Failure	Failure object part not found
-364	Failure	Serial number must be ejected
-369	Failure	Failure risk not found
-709	Failure	Measurement step names are not unique
-710	Failure	Measurement data does not match the active recipe

-721	Failure	Recipe required for measurement data booking
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2110	Failure	Serial number position is invalid
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-2127	Failure	Split ID not activated
-2138	Failure	Serial number is deleted

26.55

trUploadFailuresForMergedParts

26.55.1

Function description

This function enables the booking of failure data for previously mounted semifinished products; in doing so, you can select whether a state booking should be performed for the slave and optionally for the master serial number too.

**Advice**

The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code nos. 4010 [▶▶▶], 4011 [▶▶▶], 7202 [▶▶▶], 7209 [▶▶▶], 7223 [▶▶▶], 7224 [▶▶▶], 7320 [▶▶▶], 8310 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 13410 [▶▶▶], 13420 [▶▶▶], 16000 [▶▶▶], 18200 [▶▶▶] and 22000 [▶▶▶] as well as site parameter code nos. 18200 [▶▶▶], 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]. In addition, the station settings for "Upload Settings" and "Setup" control this function. Further information on this topic can be found in the reference section of this documentation!

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trUploadFailuresForMergedParts
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in KeyValue[] uploadMasterSerialNumberKeys
    String key
    String value
in String[] uploadSlaveSerialNumberKeys
in String[] uploadSlaveSerialNumberValues
out String[] uploadSlaveSerialNumberResults
in String[] uploadFailuresForSlaveSerialNumberKeys
in String[] uploadFailuresForSlaveSerialNumberValues
out String[] uploadFailuresForSlaveSerialNumberResults
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `uploadMasterSerialNumberKeys`, `uploadSlaveSerialNumberKeys` and `uploadFailuresForSlaveSerialNumberKeys` as well as (out) `uploadSlaveSerialNumberResults` and `uploadFailuresForSlaveSerialNumberResults`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

Data for master serial number

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!	☒	int
SERIAL_NUMBER	Serial number of the panel (master serial number) Advice: Also note the special case "Carrier" described above!	☒	String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐	String
SERIAL_NUMBER_STATE	Process state of the serial number [-1; 0; 1; 2; 3; 4]: -1 = no state booking 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid Special case: If the value [-1] is passed, no state booking is carried out; the "loopCounter" remains unchanged and the most recently booked state remains valid!	☐	int

Tab. 26-158 "Key Value Pair": `uploadMasterSerialNumberKeys`

Data for slave serial number

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☒ DateMs
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
INDEX	Unique reference number (sequential number) for each record in the array Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see parameter of the same name in the "Key Value Pair": uploadFailuresForSlaveSerialNumberKeys failure table)!	☒ long
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☒ int
SERIAL_NUMBER	Serial number of the panel (slave serial number)	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array	☐ String
SERIAL_NUMBER_STATE	Process state of the serial number [-1; 0; 1]: -1 = no state booking 0 = pass 1 = fail	☐ int
STATION_NUMBER	Station number of the work station in the iTAC system	☒ String
WORKORDER_NUMBER	Production order number	☒ String

Tab. 26-159 "Key Value Pair": uploadSlaveSerialNumberKeys

Error data for slave serial number

Key	Explanation		IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
FAILURE_ACTION_CODE	Failure action code Advice: The failure actions must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_ACTION_INFO	Description of the failure action	☐	String
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐	String
FAILURE_CAUSE_INFO	Individual information about the failure cause	☐	String
FAILURE_IMAGE_CODE	Failure symptom code Advice: The failure symptoms must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐	String
FAILURE_MEASURE_CODE	Failure measure code Advice: The failure measures must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_MEASURE_INFO	Description of the failure measure	☐	String
FAILURE_OBJECT-PART_CODE	Failure object part code Advice: The failure object parts must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_RISK_CODE	Failure risk code Advice: The failure risks must be created in advance in the iTAC system (see ADM module)!	☐	String
FAILURE_RISK_INFO	Description of the failure risk	☐	String
FAILURE_SEQUENCE_NUMBER	Sequence number of the faulty test run	☐	int
FAILURE_TYPE_CODE	Failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☒	String
FAILURE_TYPE_INFO	Individual information about the failure type	☐	String
GRID_X	Failure position: X value according to part image raster	☐	int
GRID_Y	Failure position: Y value according to part image raster	☐	int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐	String

Key	Explanation	IMSApi data type
INDEX	Unique reference number (sequential number) for each record in the array Advice: <i>The records of an array must be uniquely numbered; on the basis of this number, the data objects can be put into context with one another (see parameter of the same name in the "Key Value Pair": uploadSlaveSerialNumberKeys serial number table)!</i>	☐ long
INFO	Information text	☐ String
INFO_TXT_REPAIR	Information text for the repair	☐ String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_NUMBER	Part number of the faulty component conformant with the iTAC BOM	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
QUANTITY_SOLDERING_POINT_FAILURE	Number of faulty soldering points (pins)	☐ int
REPAIRED	Repair state [0; 1; 2; 3]: 0 = New failure - not repaired 1 = New failure - repaired 2 = Existing failure - from repaired to not repaired 3 = Existing failure - from not repaired to repaired Advice: <i>Failures are interpreted as "existing" if the passed failure location (COMP_NAME) and the failure type code (FAILURE_TYPE_CODE) already exist!</i>	☐ int
SEQUENCE_NUMBER	Display of the sequence number (test run)	☐ int
UPDATE_FLAG	"Error handling" mode [0; 1; 2; 3]: 0 = new failure is created 1 = failure data is updated (e.g. repair state) 2 = reference number (INDEX) is updated 3 = failure is deleted ("diagnosis stations" only) Advice: <i>If this parameter is not passed along, the [0] mode is used by default!</i>	☐ int

Tab. 26-160 "Key Value Pair": uploadFailuresForSlaveSerialNumberKeys

26.55.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
uploadMasterSerialNumberKeys	"Key Value Pair" structure with the following variables:
key	List of the parameters on the basis of which the function is to be executed; the respective values are passed with the <code>value</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": uploadMasterSerialNumberKeys table.
value	Value list for the parameters passed by means of <code>key</code> . Advice: Refer to the "Key Value Pair": uploadMasterSerialNumberKeys table above to see whether special rules apply during the input of values (e.g. use of wildcards)!
uploadSlaveSerialNumberKeys	List of the parameters (slave serial number) that are to be passed by the function; the corresponding values are passed with the <code>uploadSlaveSerialNumberValues</code> parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": uploadSlaveSerialNumberKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!
uploadSlaveSerialNumberValues	Values for the parameters selected by means of <code>uploadSlaveSerialNumberKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>uploadSlaveSerialNumberKeys</code> ! Refer to the corresponding table for attribute category "Key Value Pair": "uploadSlaveSerialNumberKeys" to see whether special rules apply during the input of values (e.g. use of wildcards)!
uploadFailuresForSlaveSerialNumberKeys	List of the parameters (error data) that are to be passed by the function; the corresponding values are passed with the <code>uploadFailuresForSlaveSerialNumberValues</code> parameter. A description of the possible parameters can be found above in the corresponding "Key Value Pair": uploadFailuresForSlaveSerialNumberKeys table for attribute category. Advice: Only the parameters passed here are processed by the function!

Parameter name	Explanation
uploadFailures- ForSlaveSerial- NumberValues	Values for the parameters selected by means of <code>uploadFailuresForSlaveSerialNumberKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: <i>The data is assigned in the order in which it was entered by means of parameter (in) <code>uploadFailuresForSlaveSerialNumberKeys</code>! Refer to the corresponding table for attribute category "Key Value Pair": "<code>uploadFailuresForSlaveSerialNumberKeys</code>" to see whether special rules apply during the input of values (e.g. use of wild-cards)!</i>

Tab. 26-161 Parameter (in) »trUploadFailuresForMergedParts«

Parameter (out)

Parameter name	Explanation
uploadSlaveSerialNumberResults	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>uploadSlaveSerialNumberKeys</code> and <code>uploadSlaveSerialNumberValues</code> . Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>uploadSlaveSerialNumberKeys</code>!</i>
uploadFailures- ForSlaveSerial- NumberResults	The corresponding data is output here only in the event of an error – depending on parameters (in) <code>uploadFailuresForSlaveSerialNumberKeys</code> and <code>uploadFailuresForSlaveSerialNumberValues</code> . Advice: <i>The data is output in the order in which it was entered by means of parameter (in) <code>uploadFailuresForSlaveSerialNumberKeys</code>!</i>

Tab. 26-162 Parameter (out) »trUploadFailuresForMergedParts«

26.55.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
222	Warning	The slave serial no. state is SCRAP on last work step
211	Warning	A secondary failure was detected
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing

-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-128	Failure	Work step not found
-130	Failure	Work order not found
-176	Failure	Ambiguous value for "REPAIRED" -> the data were booked as "not repaired"
-181	Failure	Station is locked
-185	Failure	Invalid value for the process layer
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked
-213	Failure	Maximum number of bookings reached
-231	Failure	Booking of the serial number failed
-240	Failure	No work step found for work order, orientation of the PCB and station
-322	Failure	No previous booking found for this serial number and this work step
-330	Failure	Invalid value for "MAJOR_FAILURE"
-331	Failure	Failure type code is invalid
-332	Failure	Failure cause not found
-335	Failure	Invalid process
-336	Failure	Failure symptoms not found
-349	Failure	Failure measure not found
-350	Failure	Failure action not found

-351	Failure	Failure object part not found
-364	Failure	Serial number must be ejected
-369	Failure	Failure risk not found
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2102	Failure	Index is not unique
-2103	Failure	Index is invalid
-2110	Failure	Serial number position is invalid
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2138	Failure	Serial number is deleted

26.56

trUploadPmQuantity

26.56.1

Function description

This functions books the quantities passed into the PM module. The specification of the station number, layer, and order number determines the work step for which the acknowledgment is to take place.

**Advice**

This API function is influenced by station parameter code nos. 4010 [▶▶▶] and 13200 [▶▶▶] as well as server parameter code nos. 2020 [▶▶▶] and 19200 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	PM license
--	-------------------

Function declaration

```
int trUploadPmQuantity
  in IMSSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String workOrderNumber
  in double passQty
  in double failQty
  in double scrapQty
```

26.56.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: For technical reasons, station parameter code no. 13200 [▶▶▶] must contain a valid PM identification number to enable data exchange between the TR and PM service; this number is specified in the standard by the system and may be changed only in exceptional cases!
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
workOrderNumber	Number of a production order
passQty	Number of "pass" bookings Advice: The entered value is interpreted as the "delta" to the previous value; i.e. positive values are added to the existing value and negative values are subtracted (key word "correction booking")!
failQty	Number of "fail" bookings Advice: See the advice under passQty!
scrapQty	Number of "scrap" bookings Advice: See the advice under passQty!

Tab. 26-163 Parameter (in) »trUploadPmQuantity«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.56.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-102	Failure	No PM license present for the station
-104	Failure	No user has logged into the station
-169	Failure	Quantity is invalid
-240	Failure	No work step found for work order, orientation of the PCB and station
-245	Failure	Quantities could not be reported to the PM module
-2036	Failure	No matching station exists

26.57

trUploadResultDataAndRecipe

26.57.1

Function description

This function books work-step-conformant serial numbers, including state and measurement data; both single panels as well as complete multiple panels can be processed (`duplicateSerialNumber`). The passed measurement data is checked by comparing with the recipe that is active on the station. Depending on the configuration (for further information, see in particular station parameters 8100, 8110 and 8200), if there are deviations, the function is finished with a failure, assigned a different recipe or a new recipe is created. In this context, the `recipeVersionMode` parameter, which intervenes in the "recipe creation" process on the basis of the passed data (key word "mandatory fields" and "optional fields"), is also to be taken into account.



Advice

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! Only measurement data that is assigned to a recipe can be evaluated in the QM module! This API function is influenced by station parameter code nos. 4010 [▶▶▶], 4011 [▶▶▶], 7202 [▶▶▶], 7210 [▶▶▶], 7211 [▶▶▶], 7212 [▶▶▶], 7213 [▶▶▶], 7214 [▶▶▶], 7215 [▶▶▶], 7216 [▶▶▶], 7223 [▶▶▶], 7224 [▶▶▶], 7250 [▶▶▶], 7251 [▶▶▶], 7252 [▶▶▶], 7253 [▶▶▶], 7254 [▶▶▶], 7255 [▶▶▶], 7256 [▶▶▶], 7257 [▶▶▶], 7258 [▶▶▶], 7320 [▶▶▶], 7400 [▶▶▶], 7401 [▶▶▶], 7402 [▶▶▶], 8100 [▶▶▶], 8110 [▶▶▶], 8200 [▶▶▶], 8310 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 8510 [▶▶▶], 13410 [▶▶▶], 13420 [▶▶▶], 15200 [▶▶▶], 15201 [▶▶▶], 15210 [▶▶▶], 15220 [▶▶▶], 15250 [▶▶▶], 15260 [▶▶▶], 16000 [▶▶▶] and 22000 [▶▶▶] as well as site parameter code nos. 7070 [▶▶▶], 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]. In addition, this function is controlled by the station settings for "Upload Settings", "Setup", and "QA Settings". Further information on this topic can be found in the reference section of this documentation!*

If a **-1** is passed for `serialNumberState`, measurement data can be passed without a new state booking. This makes sense if failure data have already been booked via another "Upload" function and a measurement record is to be created retroactively for this booking without its own state booking. The requirement for this, however, is that a previous state booking of the respective station must be found. Otherwise, the process aborts with a failure (see error code = -322).

To ensure the assignment of state booking to measurement data, the measurement data should be booked with the state booking in quasi real-time (there is otherwise a risk of faulty DWH consolidation).

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```

int trUploadResultDataAndRecipe
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in int recipeVersionId
  in String serialNumberRef
  in String serialNumberRefPos
  in int serialNumberState
  in int duplicateSerialNumber
  in DateMs bookDate
  in float cycleTime
  in int recipeVersionMode
  in String[] resultUploadKeys
  in String[] resultUploadValues
  out String[] resultValues

```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `resultUploadKeys` and `resultUploadValues` as well as (out) `resultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	☐	int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green).	☐	double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green).	☐	double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double

Key	Explanation	IMSApi data type
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	☐ int
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!	☐ int
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FUNCTION	Function name from the recipe	☐ String
FUNCTION_DESC	Explanation of the function	☐ String
FUNCTION_STEP	Function step from the recipe	☐ String
FUNCTION_STEP_DESC	Explanation of the function step	☐ String
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_LIMIT	Lower limit for the measurement step Advice: If a value is passed here, the UPPER_LIMIT parameter must also be passed. Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!	☐ String
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail". Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	☒ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name) Advice: The measurement step name must occur only once within a recipe (the program differentiates between upper- and lower-case letters) and must not be longer than 80 characters; an empty string may not be passed!	☒ String

Key	Explanation		IMSApi data type
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ". " D = Decimal DE ", " Special case: If no value is passed, measurement value type B is used as default! Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☐	String
MEASURE_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*.[0-9]* Text (example: Hello world) . *	☒	String
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis)	☐	double
NOMINAL	Nominal value Advice: If a value is passed here, the TOLERANCE parameter must also be passed. Alternatively, the limit value can also be defined with the LOWER_LIMIT and UPPER_LIMIT parameters; the LOWER_LIMIT and UPPER_LIMIT parameters override this setting!	☐	String
REMARK	Supplementary comment	☐	String
SEQUENCE_NUMBER	Sequence number of the test run	☐	int
TEST_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐	double
TOLERANCE	Tolerance value for measurement step Advice: If a value is passed here, the NOMINAL parameter must also be passed. Alternatively, the limit value can also be defined with the LOWER_LIMIT and UPPER_LIMIT parameters; the LOWER_LIMIT and UPPER_LIMIT parameters override this setting!	☐	String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic Advice: If no or a different format is entered, the measurement step type "Config-Result" is used!	☐	String
UNIT	Unit for the value passed	☒	String
UPPER_ACTION_LIMIT	Upper intervention limit	☐	String
UPPER_LIMIT	Upper limit for the measurement step Advice: If a value is passed here, the LOWER_LIMIT parameter must also be passed. Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!	☐	String
UPPER_SCRAP_LIMIT	Upper scrap limit	☐	String

Key	Explanation	IMSApi data type
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: <i>This parameter is an internal designator and does not currently have an official use!</i>	String
WEIGHTING	Weighting for the PAA [0 - 9]	int

Tab. 26-164 "Key Value Pair": *resultUploadKeys*

26.57.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB!
recipeVersionId	Identification number of a specific recipe version Advice: <i>This field is intended for internal testing purposes and enables the checking of an already existing recipe. If an identification number is passed, no recipe can be created; the recipeVersionMode parameter is ignored in this case!</i> Default case: If an empty string or the value [-1] is passed, this parameter is not taken into account; no check is performed against a specific recipe!
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Advice: <i>Also note the special case "Carrier" described above!</i>
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: <i>If the value [-1] is passed, the position is ascertained automatically!</i>

Parameter name	Explanation
serialNumberState	Process state of the serial number [-1; 0; 1; 2; 3; 4]: -1 = no state booking 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid Special case: If the value [-1] is passed, no state booking is carried out; the "loopCounter" remains unchanged and the most recently booked state remains valid!
duplicateSerial-Number	Single panel/multiple panel [0; 1]: 0 = single panel 1 = multiple panel; all serial numbers (single panels) of the multiple panel are booked with the same values Advice: The upload of a multiple panel with <code>duplicateSerial-Number = 1</code> is only possible if the multiple panel is not part of a work order cluster. Otherwise the function outputs the error code - 355!
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!
cycleTime	Processing time of the station in seconds
recipeVersionMode	Control "recipe creation" [0; 1; 2]: Advice: In connection with this parameter, note the mandatory field designation in the "Key Value Pair": resultUploadKeys table! 0 = Check that mandatory fields match; if the mandatory fields differ, a new recipe is created on the basis of the passed data. 1 = Check that mandatory and optional fields match; if only the optional fields differ, no new recipe is created; the optional fields are, however, updated accordingly. 2 = Check that mandatory and optional fields match; any deviations result in the creation of a new recipe, even if the mandatory fields contain identical content. Advice: The following applies for the case of the new creation of a recipe: If optional fields are not passed, they are taken over from the predecessor version or the default!
resultUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>resultUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": resultUploadKeys table. Advice: Only the parameters passed here are processed by the function!
resultUploadValues	Values for the parameters selected by means of <code>resultUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>resultUploadKeys</code>! Refer to the "Key Value Pair": resultUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wild-cards)!

Tab. 26-165 Parameter (in) »trUploadResultDataAndRecipe«

Parameter (out)

Parameter name	Explanation
resultValues	The corresponding data is output here only in the event of an error – depending on the resultUploadKeys and resultUploadValues parameters (in). Advice: The data is output in the order in which it was entered by means of parameter (in) attributeUploadKeys!

Tab. 26-166 Parameter (out) »trUploadResultDataAndRecipe«

26.57.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
233	Warning	Anomaly detected (PAA)
		Advice: The value 233 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
		Advice: The value 230 is only output if station parameter code no. 7258 [▶▶▶] is set to TRUE!
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
210	Warning	At least one serial number has the SCRAP state
5	Warning	Not all records in array were successfully processed

0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-15	Failure	At least one passed value is too large for the respective data-base field
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-130	Failure	Work order not found
-133	Failure	No active work order found for the station
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked

-213	Failure	Maximum number of bookings reached
-222	Failure	The serial no. state is SCRAP on last work step
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-269	Failure	FAIL state due to anomaly
-322	Failure	No previous booking found for this serial number and this work step
-355	Failure	Production order is part of a work order cluster
-364	Failure	Serial number must be ejected
-367	Failure	Sequence for measurement data does not exist
-702	Failure	Recipe header data not found
-709	Failure	Measurement step names are not unique
-710	Failure	Measurement data does not match the active recipe
-712	Failure	No measurement data passed
-721	Failure	Recipe required for measurement data booking
-738	Failure	Booking references do not match
-1105	Failure	BOM for the work order not found
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2110	Failure	Serial number position is invalid
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-2127	Failure	Split ID not activated
-2138	Failure	Serial number is deleted

26.58

trUploadState

26.58.1

Function description

This function books, in a work-step-conformant manner, the process state of one or more individual products (single panel) or of a complete multiple panel. In doing so, a time stamp is set for the component lot capture. This function is the universal function for the booking of a product.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code nos. 7202 [▶▶▶], 7223 [▶▶▶], 7224 [▶▶▶], 7320 [▶▶▶], 7400 [▶▶▶], 7401 [▶▶▶], 7402 [▶▶▶], 8310 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 8510 [▶▶▶], 13302 [▶▶▶], 13312 [▶▶▶], 13410 [▶▶▶], 13420 [▶▶▶], 16000 [▶▶▶], 18200 [▶▶▶] and 22000 [▶▶▶] as well as site parameter code nos. 7070 [▶▶▶], 18200 [▶▶▶], 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]. In addition, this function is controlled by the station settings for "Upload Settings". Further information on this topic can be found in the reference section of this documentation!*

Application type "A"

Single panel -> one serial number (`duplicateSerialNumber = 0`): The data of the serial number to be booked is passed with the function parameter (`processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate`, `cycleTime`); the "Key Value Pair" structure is not filled.

Application type "B"

Single panel -> multiple serial numbers (`duplicateSerialNumber = 0` and `serialNumberRef = -1`): The data of the serial numbers to be booked is passed with the "Key Value Pair" structure. The `processLayer`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime` function parameters are not evaluated.

Application type "C"

Multiple panel -> all serial numbers of the multiple panel with identical data (`duplicateSerialNumber = 1`): The data of the multiple panel to be booked is passed with the `processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime` function parameters; all serial numbers (single panels) of the multiple panel are booked with the same values; the "Key Value Pair" structure is not filled.

Application case "D"

Multiple panel -> all serial numbers of the multiple panel with different data (`duplicateSerialNumber = 1`): the (default) data of the multiple panel to be booked are passed with the `processLayer`, `serialNumberRef`, `serialNumberRefPos`, `serialNumberState`, `bookDate` and `cycleTime` function parameters. In the "Key Value Pair" structure, only those serial numbers are passed which deviate from the default data, here the `serialNumberState`, `bookDate` and `cycleTime` function parameters; the `processLayer`, `serialNumberRef` and `serialNumberRefPos` function parameters are not evaluated in this case.

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified

process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trUploadState
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumberRef
in String serialNumberRefPos
in int serialNumberState
in int duplicateSerialNumber
in DateMs bookDate
in float cycleTime
in String[] serialNumberUploadKeys
in String[] serialNumberUploadValues
out String[] serialNumberResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) serialNumberUploadKeys, serialNumberUploadValues and (out) serialNumberResultValues). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

*Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!
If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.*

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case (application type "B"): [-1] passes the current date and the current time! Special case (application types "A", "C" and "D"): [-1] uses the value from the bookDate function parameter!	☐ DateMs

Key	Explanation	IMSApi data type
CYCLE_TIME	Processing time at the station in seconds Special case: [-1] uses the value from the <code>cycleTime</code> function parameter!	☐ double
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
SERIAL_NUMBER	Serial number of the panel	☒ String
SERIAL_NUMBER_POS	Position of the single panel in the unit-array Advice: If this parameter is not passed, the serial number (SERIAL_NUMBER) must be unique!	☐ String
SERIAL_NUMBER_STATE	Process state of the serial number [0; 1; 2; 3; 4]: 0 = Pass 1 = Fail 2 = Scrap 3 = In process 4 = Invalid	☒ int

Tab. 26-167 "Key Value Pair": `serialNumberUploadKeys`

26.58.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
<code>sessionId</code>	Unique designator (ID) of the active session
<code>persId</code>	Unique designator (ID) of the logged in user
<code>locale</code>	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent Special case: Depending on the setting of station parameter code no. 8310 [▶▶▶], the work step number (AVO) used in an ERP system or the iTAC work step number can be passed directly instead of the orientation of the PCB! Advice: This field is not evaluated for application type B!
serialNumberRef	Serial number of the 1st single panel or of the unit-array; this number is used as a reference serial number Advice: In application type B, the value [-1] must be passed here. Also note the special case "Carrier" described above!
serialNumberRef-Pos	Position of the serial number of the first single panel or unit-array Advice: If the value [-1] is passed, the position is ascertained automatically! This field is not evaluated in application type B!

Parameter name	Explanation
serialNumberState	Process state of the serial number [0; 1; 2; 3; 4]: 0 = pass 1 = fail 2 = scrap 3 = in process 4 = invalid Advice: <i>This field is not evaluated for application type B!</i>
duplicateSerial-Number	Single panel/multiple panel [0; 1]: 0 = Single panel (see application types A and B) 1 = Multiple panel (see application types C and D)
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time! Advice: <i>This field is not evaluated for application type B!</i>
cycleTime	Processing time at the station in seconds Advice: <i>This field is not evaluated for application type B!</i>
serialNumberUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the serialNumberUploadValues parameter. A description of the possible parameters can be found above in the "Key Value Pair": serialNumberUploadKeys table. Advice: <i>Only the parameters passed here are processed by the function!</i> <i>In application type A, nothing may be passed here!</i>
serialNumberUploadValues	Value list for the parameters passed by means of serialNumberUploadKeys ; no input is required for the ERROR_CODE parameter since it is automatically filled by the system in the event of an error. Advice: <i>Refer to the "Key Value Pair" : serialNumberUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!</i> <i>In application type A, nothing may be passed here!</i>

Tab. 26-168 Parameter (in) »trUploadState«

Parameter (out)

Parameter name	Explanation
serialNumberResultValues	The corresponding data is output here only in the event of an error – depending on the serialNumberUploadKeys and serialNumberUploadValues parameters (in). Advice: <i>The data is output in the order in which it was entered by means of parameter (in) serialNumberUploadKeys!</i>

Tab. 26-169 Parameter (out) »trUploadState«

26.58.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
504	Warning	The material quantity in the system has become negative
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
210	Warning	At least one serial number has the SCRAP state
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived

-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-155	Failure	Invalid value for "duplicateSerialNumber" [0; 1]
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked
-203	Failure	Serial number is not a reference serial number
-205	Failure	Serial number belongs to a different work order
-213	Failure	Maximum number of bookings reached
-222	Failure	The serial no. state is SCRAP on last work step
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-364	Failure	Serial number must be ejected
-1303	Failure	Production means number found more than once
-2070	Failure	IMSApi key was passed multiple times to the function
-2100	Failure	Serial number is sent to SCT
-2110	Failure	Serial number position is invalid
-2125	Failure	Split ID -> Quantity exceeded
-2126	Failure	Split ID not valid
-2127	Failure	Split ID not activated
-2138	Failure	Serial number is deleted

26.59

trUploadStationResult

26.59.1

Function description

This function logs the process data for a station. The data is stored in a recipe for the passed period. The passed measurement data is checked by comparing with the recipe that is active on the station. Depending on the configuration (for further information, see in particular station parameters 8100 [▶▶▶], 8110 [▶▶▶] and 8200 [▶▶▶]), if there are deviations, the function is finished with a failure, assigned a different recipe or a new recipe is created.

If necessary, the serial numbers booked in the same period can be associated with these process data (see the `serialUnitTrace` parameter), which is the basis for evaluation in the QM module.

**Advice**

*The array returned contains only records that were **incorrectly** processed, along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)! This API function is influenced by station parameter code no.: 8100 [▶▶▶], 8110 [▶▶▶], 8200 [▶▶▶], 15220 [▶▶▶], 15250 [▶▶▶], 15260 [▶▶▶] and 18200 [▶▶▶] as well as station parameter code no. 18200 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!*

If no serial numbers are to be assigned to measured values during the booking (`serialUnitTrace = 0`), the assignment can be performed subsequently by means of a separate process (for further information, see process "assignment of serial numbers to measured values").

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trUploadStationResult
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String partNumber
  in int bomVersion
  in String bomIndex
  in String bomVersionErp
  in String workOrderNumber
  in DateMs bookDate
  in int serialUnitTrace
  in String[] stationResultUploadKeys
  in String[] stationResultUploadValues
  out String[] stationResultValues
```

"Key Value Pair" structures

This function contains a "Key Value Pair" structure (see parameters (in) `stationResultUploadKeys`, `stationResultUploadValues` and (out) `stationResultValues`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.



Advice

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following table, all parameters of the "Key Value Pair" structure are listed and described (mandatory field = ☒, optional = ☐).

Key	Explanation		IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐	DateMs
CONFIDENCE_INTERVAL	Confidence region [0; 1; 2; 3]: 0 = 3 sigma 1 = 4 sigma 2 = 6 sigma 3 = 8 sigma	☐	int
CP_GREEN	Default value for the process capability index (Cp) which must at least be reached before the full process capability is valid (green).	☐	double
CP_RED	Default value for the process capability index (Cp) which must at least be reached in order to ensure at least a conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double
CPK_GREEN	Default value for the critical process capability index (Cpk) which must at least be reached before the full process capability is valid (green).	☐	double
CPK_RED	Default value for the critical process capability index (Cpk) which must at least be reached in order to ensure at least conditional process capability (yellow); if this value is not met, no process capability exists (red).	☐	double
DISTRIBUTION_TYPE	Distribution function [0; 1]: 0 = normal distribution 1 = log. normal distribution	☐	int
DUTY_DOCUMENTATION	Include in documentation [0; 1]: 0 = measurement record is to be included in documentation (default) 1 = measurement record does NOT need to be included in documentation Advice: Only the measurement data labeled as "to be included in documentation" are taken into account by the archiving process; measurement data that do not need to be included in documentation can be deleted from the system if necessary!	☐	int
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int

Key	Explanation	IMSApi data type
FUNCTION	Function name from the recipe	☐ String
FUNCTION_DESC	Explanation of the function	☐ String
FUNCTION_STEP	Function step from the recipe	☐ String
FUNCTION_STEP_DESC	Explanation of the function step	☐ String
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_LIMIT	Lower limit for the measurement step Advice: <i>If a value is passed here, the UPPER_LIMIT parameter must also be passed. Alternatively, the limit value can also be defined with the NOMINAL and TOLERANCE parameters!</i>	☐ String
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
MEASURE_FAIL_CODE	State information for the measurement step: 0 = Passed measurement step 1 = Failed measurement step ... = By default, other integer values are interpreted as canceled tests and are then not included in the statistics as either "Pass" or "Fail". Advice: <i>Within the iTAC system, a measurement step with failure code "0" is always interpreted as "pass", and one with a failure code "1" is always interpreted as "fail"!</i> Furthermore, customer-specific values can be permanently declared as pseudo failure (code no. 15250 [▶▶▶]) or as "individual item with canceled test" (code no. 15260 [▶▶▶]) depending on the station configuration. If the "anomaly analysis" function (PAA) is activated for a station (code no. 7210 [▶▶▶]), the passed failure code of an anomalous measurement step can be converted to one of the failure codes defined for this purpose (code no. 7212 [▶▶▶]).	☒ String
MEASURE_NAME	Unique identifier of the recipe parameter (measurement step name)	☒ String
MEASURE_TYPE	Specification of the measurement value type [U; T; B; O; H; N; D]: U = Unknown T = Text B = Binary O = Octal H = Hexadecimal N = Decimal EN ". " D = Decimal DE ", " Advice: <i>The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!</i>	☐ String
MEASURE_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*.[0-9]* Text (example: Hello world) . *	☒ String
NOISE_FIGURE	Default value for the noise figure (Part Average Analysis)	☐ double

Key	Explanation	IMSApi data type
NOMINAL	Nominal value Advice: If a value is passed here, the <code>TOLERANCE</code> parameter must also be passed. Alternatively, the limit value can also be defined with the <code>LOWER_LIMIT</code> and <code>UPPER_LIMIT</code> parameters; the <code>LOWER_LIMIT</code> and <code>UPPER_LIMIT</code> parameters override this setting!	☐ String
REMARK	Supplementary comment	☐ String
SEQUENCE_NUMBER	Sequence number of the test run	☐ int
TEST_STEP_NUMBER	Test step no. in the recipe (measurement step number)	☐ double
TOLERANCE	Tolerance value for measurement step Advice: If a value is passed here, the <code>NOMINAL</code> parameter must also be passed. Alternatively, the limit value can also be defined with the <code>LOWER_LIMIT</code> and <code>UPPER_LIMIT</code> parameters; the <code>LOWER_LIMIT</code> and <code>UPPER_LIMIT</code> parameters override this setting!	☐ String
TYPE	Specification of the parameter type [V; R; B; D]: V = Config R = Result B = Config-Result D = Dynamic	☐ String
UNIT	Unit for the value passed	☒ String
UPPER_ACTION_LIMIT	Upper intervention limit	☐ String
UPPER_LIMIT	Upper limit for the measurement step Advice: If a value is passed here, the <code>LOWER_LIMIT</code> parameter must also be passed. Alternatively, the limit value can also be defined with the <code>NOMINAL</code> and <code>TOLERANCE</code> parameters!	☐ String
UPPER_SCRAP_LIMIT	Upper scrap limit	☐ String
VISIBLE	Display of the parameter [Y; N]: Y = Yes (visible) N = No (invisible) Advice: This parameter is an internal designator and does not currently have an official use!	☐ String
WEIGHTING	Weighting for the PAA [0 - 9]	☐ int

Tab. 26-170 "Key Value Pair": `stationResultUploadKeys`

26.59.2

Input and output parameters

Parameter (in)

**Advice**

No product version needs to be passed for this function; all corresponding parameters can be passed with [-1]. However, note that passed parameters are always checked against one another; if the values do not match, the function aborts with a failure!

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partNumber	Part number in the iTAC system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomVersion	BOM version from the iTAC system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bomVersionErp	BOM version in the ERP system Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
workOrderNumber	Number of a production order Advice: The parameter is not evaluated if the value [-1] is passed; for this purpose, observe the advice above the table!
bookDate	Booking time for the process data (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special cases: a.) If the value [-1] is passed, the current date and the current time are used as the start date! b.) If the value passed is between [1] and [3153600000], i.e., within the period "one year", it is used as offset; in this case, the start date corresponds to the current date/time plus the period passed here c.) The entry [0] is NOT allowed here.

Parameter name	Explanation
<code>serialUnitTrace</code>	System behavior "serial number link" [0; 1]: 0 = Serial numbers are not linked with the previous process data. 1 = All serial numbers that have a state booking for the passed station in the previous time interval are linked to the previous – i.e. most recently booked – process data. Advice for the mode "0": <i>If necessary, the linking process between serial numbers and process data can be time-controlled; in this way, it is possible to influence to which process record the serial numbers are assigned (for further information, see "Assignment of serial numbers to measured values" process)!</i>
<code>stationResultUploadKeys</code>	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>stationResultUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": stationResultUploadKeys table. Advice: <i>Only the parameters passed here are processed by the function!</i>
<code>stationResultUploadValues</code>	Value list for the parameters passed by means of <code>stationResultUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: <i>Refer to the "Key Value Pair": stationResultUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!</i>

Tab. 26-171 Parameter (in) »trUploadStationResult«

Parameter (out)

Parameter name	Explanation
<code>stationResultValues</code>	The corresponding data is output here only in the event of an error – depending on the <code>stationResultUploadKeys</code> and <code>stationResultUploadValues</code> parameters (in). Advice: <i>The data is output in the order in which it was entered by means of parameter (in) stationResultUploadKeys!</i>

Tab. 26-172 Parameter (out) »trUploadStationResult«

26.59.3

Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.

**Advice**

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-128	Failure	Work step not found
-130	Failure	Work order not found
-186	Failure	Product version is not valid
-242	Failure	BOM version is invalid
-187	Failure	Product version not found
-328	Failure	Station state not created
-329	Failure	Measurement data not found for the station
-352	Failure	Measurement data for period already exists

-706	Failure	Recipe not found
-710	Failure	Measurement data does not match the active recipe
-712	Failure	No measurement data passed
-714	Failure	Invalid value for "serialUnitTrace" [0; 1]
-1105	Failure	BOM for the work order not found
-2070	Failure	IMSApi key was passed multiple times to the function

26.60

trUploadTestplanResult

26.60.1

Function description

This function books the test results for a test plan. In doing so, the results (including the corresponding failures) can be passed for both attributive as well as variable test characteristics. It is not mandatory that the station passed by means of `station-Number` be the booking station (for further information, see the `STATION_NUMBER` key).

**Advice**

*This API function is influenced by station parameter code nos. 7223 [▶▶▶], 7224 [▶▶▶], 8400 [▶▶▶], 8401 [▶▶▶], 8402 [▶▶▶], 13420 [▶▶▶] and 16100 [▶▶▶] as well as site parameter code no. 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]. Test results can be booked with or without a separate state booking (for further information, see the `SERIAL_NUMBER_STATE` key in the "Key Value Pair": **work-StepKeys** table!*

*To query test plan data, the »trGetTestplanData« [▶▶▶] function must be used! The arrays returned contain only **incorrectly** processed records along with a corresponding error code (reason for error). In the case of one or more incorrectly processed records, the function itself returns a warning (error code = 5)!*

Special case "Carrier"

If a carrier number is passed instead of a serial number, all serial numbers assigned to the carrier are ascertained in the background and then booked with the specified process state. As different products can theoretically be on a carrier at the same time, the **collective state booking** only functions if the station and the process layer match.

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active) OR TR license (passive)
--	---

Function declaration

```
int trUploadTestplanResult
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String serialNumber
in String serialNumberPos
in long bookDate
in String[] workStepKeys
in String[] workStepValues
out String[] workStepResults
in String[] varUploadKeys
in String[] varUploadValues
out String[] varUploadResults
in String[] varFailUploadKeys
in String[] varFailUploadValues
out String[] varFailUploadResults
in String[] attribUploadKeys
in String[] attribUploadValues
out String[] attribUploadResult
in String[] attribFailUploadKeys
in String[] attribFailUploadValues
out String[] attribFailUploadResults
```

"Key Value Pair" structures

This function contains "Key Value Pair" structures (see parameters (in) `workStepKeys`, `workStepValues`, `varUploadKeys`, `varUploadValues`, `varFailUploadKeys` and `varFailUploadValues` as well as parameters (out) `workStepResults`, `varUploadResults`, `varFailUploadResults`, `attribUploadResult` and `attribFailUploadResults`). This allows accommodations to be made for individual requirements, as exactly those data objects that are needed for the current task can be selected.

**Advice**

Note that all parameters from the "Key Value Pair" structures are of the "string" data type and are, thus, untyped!

If parameters from these structures are used as input parameters for other API functions (key word "API workflows"), a data type conversion may be necessary. Which regular data type is expected during further processing can be found in the "IMSApi data type" column of the "Key Value Pair" table.

In the following tables, all parameters of the "Key Value Pair" structures are listed and described (mandatory field = ☒, optional = ☐).

General booking data

Key	Explanation	IMSApi data type
BOOK_DATE	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 Special case: [-1] passes the current date and the current time!	☐ DateMs
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
SERIAL_NUMBER_STATE	Process state of the serial number [-2; -1; 0; 1; 2; 3; 4]: -2 = "Successor booking" state -1 = "Predecessor booking" state 0 = Pass 1 = Fail 2 = Scrap 3 = In process 4 = Invalid Advice: In mode [-1], the test results are assigned to the state booking of the predecessor booking of the same work step; if the value [-2] is passed, the results are stored temporarily and then assigned to the next »trUploadTestplanResult« booking (successor booking)!	☒ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: If this parameter is not passed, the data is booked on the basis of the station according to the <code>stationNumber</code> function parameter!	☐ String
WORKSTEP_NUMBER	Number of the work step	☒ int

Tab. 26-173 "Key Value Pair": `workStepKeys`

Variable test characteristics

Key	Explanation	IMSApi data type
DATA_VALUE	Measured value; the following format definitions apply: Binary (example: 0b110010) 0b[0-1]* Octal (example: 0o173210) 0o[0-7]* Hexadecimal (example: 0x05FB7A) 0x[0-9A-Fa-f]* Decimal EN (example: 1366.665) [0-9]*.[0-9]* Decimal DE (example: 1366,665) [0-9]*,[0-9]* Text (example: Hello world).* Advice: The measurement value types "Hexadecimal", "Octal" and "Binary" support whole numbers only; NO decimal values can be processed!	☒ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
LOWER_ACTION_LIMIT	Lower intervention limit	☐ String
LOWER_LIMIT	Lower limit for the measurement step	☐ String
LOWER_SCRAP_LIMIT	Lower scrap limit	☐ String
MEASURE_FAIL_CODE	State information for the measurement step: 0 = passed measurement step [...] (other integer values) = failed measurement step Special case [without value]: If the key is not passed or is passed [empty], the lower/upper limit values or nominal values/tolerance values are used in combination with the measured value (DATA_VALUE) to automatically ascertain the state. This state is then used ONLY for restricting/reducing the sample.	☐ String
STATION_NUMBER	Station number of the work station in the iTAC system Advice: If this parameter is not passed, the station number is ascertained on the basis of the work step (WORKSTEP_NUMBER)!	☐ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long
TP_CRITERIA	Test plan criterion for the test characteristic; to ensure uniqueness, the internal ID of the desired test plan criterion must be passed. If multiple criteria are to be passed in one process, the following syntax must be observed: [internal ID of criterion A]#[internal ID of criterion B]#[...] Advice: The internal ID of the respective test plan criterion can be queried using the API function »trGetTestplanData« [▶▶▶]!	☐ String
UPPER_ACTION_LIMIT	Upper intervention limit	☐ String
UPPER_LIMIT	Upper limit for the measurement step	☐ String
UPPER_SCRAP_LIMIT	Upper scrap limit	☐ String

Key	Explanation	IMSApi data type
WORKSTEP_NUMBER	Number of the work step Advice: The test characteristic is put into context with the booking work step on the basis of the work step number (see the "Key Value Pair": workStepKeys table above!	☒ int

Tab. 26-174 "Key Value Pair": varUploadKeys

Failure data for the variable test characteristics

Key	Explanation	IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐ String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐ String
FAILURE_TYPE_CODE_ID	Internal identification number for the failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☒ int
GRID_X	Failure position: X value according to part image raster	☐ int
GRID_Y	Failure position: Y value according to part image raster	☐ int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐ String
INFO	Information text on failures	☐ String
INFO_TXT_REPAIR	Information text for the repair	☐ String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐ int
PART_NUMBER	Part number of the faulty component conformant with the iTAC BOM	☐ String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐ int
REPAIRED	Repair state [0; 1; 2; 3]: 0 = New failure - not repaired 1 = New failure - repaired 2 = Existing failure - from repaired to not repaired 3 = Existing failure - from not repaired to repaired Advice: Failures are interpreted as "existing" if the passed failure location (COMP_NAME) and the failure type code (FAILURE_TYPE_CODE) already exist!	☐ int

Key	Explanation	IMSApi data type
SEQUENCE_NUMBER	Sequential number for each failure record in the array	☒ int
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects of the test characteristic can be put into context with the failure data (see parameter of the same name in the "Key Value Pair": varUploadKeys table!	☒ long

Tab. 26-175 "Key Value Pair": varFailUploadKeys

Attributive test characteristics

Key	Explanation	IMSApi data type
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒ int
STATION_NUMBER	Station number of the work station in the iTAC system Advice: If this parameter is not passed, the station number is ascertained on the basis of the work step (WORKSTEP_NUMBER)!	☐ String
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The test characteristic is uniquely defined on the basis of this identification number. The identification number can be queried by means of the »trGetTestplanData« [▶▶▶] API function!	☒ long
TEST_STEP_PARAM_ID	Internal identification number of the catalog number for the attributive test characteristic	☐ long
TEST_STEP_VALUE	Display of the (individual) weight for the property value	☐ String
TEST_STEP_VALUE_ID	Internal identification number of the property value	☒ long
TP_CRITERIA	Test plan criterion for the test characteristic; to ensure uniqueness, the internal ID of the desired test plan criterion must be passed. If multiple criteria are to be passed in one process, the following syntax must be observed: [internal ID of criterion A]#[internal ID of criterion B]#[...] Advice: The internal ID of the respective test plan criterion can be queried using the API function »trGetTestplanData« [▶▶▶]!	☐ String
WORKSTEP_NUMBER	Number of the work step Advice: The test characteristic is put into context with the booking work step on the basis of the work step number (see the "Key Value Pair": workStepKeys table above!	☒ int

Tab. 26-176 "Key Value Pair": attribUploadKeys

Failure data for the attributive test characteristics

Key	Explanation		IMSApi data type
COMP_NAME	Failure location (mounting place) conformant with the iTAC BOM	☐	String
ERROR_CODE	Error code for the case of incorrect processing of the respective record Advice: As soon as a record (array element) could not be processed, the corresponding technical error code is output here; in this case, the function itself outputs a general warning (see error code 5)!	☒	int
FAILURE_CAUSE_CODE	Failure cause code Advice: The failure causes must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☐	String
FAILURE_- TYPE_CODE_ID	Internal identification number for the failure type code Advice: The failure types must be created in advance in the iTAC system and must be assigned to the station (see ADM module)!	☒	int
GRID_X	Failure position: X value according to part image raster	☐	int
GRID_Y	Failure position: Y value according to part image raster	☐	int
IMAGE_NAME	Name of the part image which was used during failure input (see also GRID_X and GRID_Y)	☐	String
INFO	Information text on failures	☐	String
INFO_TXT_REPAIR	Information text for the repair	☐	String
MAJOR_FAILURE	Weighting factor for the failure [0; 1]: 0 = Not a severe failure 1 = Severe failure	☐	int
PART_NUMBER	Part number of the faulty component conformant with the iTAC BOM	☐	String
PROCESS_LAYER	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent	☐	int
REPAIRED	Repair state [0; 1; 2; 3]: 0 = New failure - not repaired 1 = New failure - repaired 2 = Existing failure - from repaired to not repaired 3 = Existing failure - from not repaired to repaired Advice: Failures are interpreted as "existing" if the passed failure location (COMP_NAME) and the failure type code (FAILURE_-TYPE_CODE) already exist!	☐	int
SEQUENCE_NUMBER	Sequential number for each failure record in the array	☒	int
TEST_STEP_ID	Internal identification number of the test characteristic Advice: The records of an array must be uniquely numbered; on the basis of this number, the data objects of the test characteristic can be put into context with the failure data (see parameter of the same name in the "Key Value Pair": attribUploadKeys table!	☒	long

Tab. 26-177 "Key Value Pair": attribFailUploadKeys

26.60.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system Advice: It is not mandatory that this station be the booking station (for further information, see the <code>STATION_NUMBER</code> key in the "Key Value Pair": <code>workStepKeys</code> table)!
serialNumber	Serial number of the panel Advice: Also note the special case "Carrier" described above!
serialNumberPos	Position of the single panel in the unit-array
bookDate	Booking time (entry into the iTAC system database); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000. Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000. If necessary, bookings can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!
workStepKeys	List of the general booking data that is to be passed by the function; the corresponding values are passed with the <code>workStepValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>workStepKeys</code> table. Advice: Only the parameters passed here are processed by the function!
workStepValues	Values for the parameters selected by means of <code>workStepKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>workStepKeys</code> ! Refer to the "Key Value Pair": <code>workStepKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
varUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>varUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": <code>varUploadKeys</code> table. Advice: Only the parameters passed here are processed by the function!
varUploadValues	Values for the parameters selected by means of <code>varUploadKeys</code> ; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>varUploadKeys</code> ! Refer to the "Key Value Pair": <code>varUploadKeys</code> table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Parameter name	Explanation
varFailUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>varFailUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": varFailUploadKeys table. Advice: Only the parameters passed here are processed by the function!
varFailUploadValues	Values for the parameters selected by means of <code>varFailUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>varFailUploadKeys</code>! Refer to the "Key Value Pair": varFailUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attribUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>attribUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attribUploadKeys table. Advice: Only the parameters passed here are processed by the function!
attribUploadValues	Values for the parameters selected by means of <code>attribUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>attribUploadKeys</code>! Refer to the "Key Value Pair": attribUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!
attribFailUploadKeys	List of the parameters that are to be passed by the function; the corresponding values are passed with the <code>attribFailUploadValues</code> parameter. A description of the possible parameters can be found above in the "Key Value Pair": attribUploadKeys table. Advice: Only the parameters passed here are processed by the function!
attribFailUploadValues	Values for the parameters selected by means of <code>attribFailUploadKeys</code>; no input is required for the <code>ERROR_CODE</code> parameter since it is automatically filled by the system in the event of an error. Advice: The data is assigned in the order in which it was entered by means of parameter (in) <code>attribFailUploadKeys</code>! Refer to the "Key Value Pair": attribFailUploadKeys table above to see whether special rules apply during the input of values (e.g., use of wildcards)!

Tab. 26-178 Parameter (in) »trUploadTestplanResult«

Parameter (out)

Parameter name	Explanation
workStepResults	The corresponding data is output here only in the event of an error – depending on parameters (in) workStepKeys and workStepValues. Advice: The data is output in the order in which it was entered by means of parameter (in) varUploadKeys!
varUploadResults	The corresponding data is output here only in the event of an error – depending on parameters (in) varUploadKeys and varUploadValues. Advice: The data is output in the order in which it was entered by means of parameter (in) varUploadKeys!
varFailUploadResults	The corresponding data is output here only in the event of an error – depending on parameters (in) varFailUploadKeys and varFailUploadValues. Advice: The data is output in the order in which it was entered by means of parameter (in) varFailUploadKeys!
attribUploadResult	The corresponding data is output here only in the event of an error – depending on parameters (in) attribUploadKeys and attribUploadValues. Advice: The data is output in the order in which it was entered by means of parameter (in) attribUploadKeys!
attribFailUploadResults	The corresponding data is output here only in the event of an error – depending on parameters (in) attribFailUploadKeys and attribFailUploadValues. Advice: The data is output in the order in which it was entered by means of parameter (in) attribFailUploadKeys!

Tab. 26-179 Parameter (out) »trUploadTestplanResult«

26.60.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
224	Warning	Serial number -> State was changed
		Advice: The value 224 is only output if station parameter code no. 13420 [▶▶▶] is set to TRUE!
211	Warning	A secondary failure was detected
		Advice: The value 211 is only output if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to FALSE; if code no. 7224 is set to TRUE, the upload function aborts with error -181!
5	Warning	Not all records in array were successfully processed
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
		Advice: The value -102 is output only if transfer of the TR bookings into the PM module has been activated (see station parameter code no. 4010 [▶▶▶] and 13200 [▶▶▶]), but the PM license required for this is not available on the station!
-104	Failure	No user has logged into the station

-110	Failure	Serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-130	Failure	Work order not found
-181	Failure	Station is locked
		Advice: The value -181 is only output if the station has been manually locked or if station parameter code no. 7223 [▶▶▶] is active (key word "consecutive failure check") and code no. 7224 [▶▶▶] is set to TRUE; if code no. 7224 is set to FALSE, the upload function is executed with warning 211!
-185	Failure	Invalid value for the process layer
-196	Failure	A parameter is invalid
-200	Failure	Invalid value for "serialNumberState"
-202	Failure	Serial number is locked
-213	Failure	Maximum number of bookings reached
-222	Failure	The serial no. state is SCRAP on last work step
-240	Failure	No work step found for work order, orientation of the PCB and station
-322	Failure	No previous booking found for this serial number and this work step
-364	Failure	Serial number must be ejected
-701	Failure	Recipe data not found
-702	Failure	Recipe header data not found
-707	Failure	No active recipe found
-710	Failure	Measurement data does not match the active recipe
-721	Failure	Recipe required for measurement data booking
-1303	Failure	Production means number found more than once

-2000	Failure	Formula parameter missing
-2001	Failure	Formula parameter not convertible
-2002	Failure	Formula is invalid
-2003	Failure	Formula calculation failed
-2004	Failure	No booking exists for this serial number
-2070	Failure	IMSApi key was passed multiple times to the function
-2110	Failure	Serial number position is invalid
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2138	Failure	Serial number is deleted

26.61

trVerifyMerge

26.61.1

Function description

This function checks whether it is possible to mount serial numbers (slave serial number) in another product version (master serial number or active work order). Both single panels as well as complete multiple panels can be processed. In addition to the mounting check, the state and the state of progress are also checked.

**Advice**

This API function is influenced by station parameter code nos. 7203 [▶▶▶], 7206 [▶▶▶], 7208 [▶▶▶], 7209 [▶▶▶], 7218 [▶▶▶], 7219 [▶▶▶], 7223 [▶▶▶], 7260 [▶▶▶], 13010 [▶▶▶] and 18200 [▶▶▶] as well as site parameters 7260 [▶▶▶] and 18200 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

*Depending on the configuration of the station parameter 7208 [▶▶▶], this API function – unlike nearly all other API functions – can be used **cross-site**. It is therefore possible to check whether products (serial numbers) from different sites are allowed to be merged!*

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trVerifyMerge
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in String serialNumberSlave
  in String serialNumberSlavePos
  in String serialNumberMaster
  in String serialNumberMasterPos
  in int checkMultiBoard
  out SerialNumberStateData[] serialNumberStateDataArray
    String serialNumberSlave
    String serialNumberSlavePos
    int state
```

26.61.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
serialNumberSlave	Serial number which is deleted during a product merging (see serialNumberMaster)
serialNumber-SlavePos	Position of the single panel (here serialNumberSlave) in the unit-array Special case : If an empty string or value [-1] is passed, the position is not taken into account; in this case the slave serial number must be unique!
serialNumberMaster	Serial number which is retained during a product merging (see serialNumberSlave) Special case: If an empty string or the value [-1] is passed, the product version is ascertained on the basis of the active work order! Advice: If the product version is ascertained on the basis of the active work order, only the "general" mounting is checked. If, on the other hand, the product version is specified on the basis of a master serial number, the specific mounting is checked – if activated by means of station parameters – i.e., the current quantity is compared with the specified BOM quantities!
serialNumberMasterPos	Position of the single panel (here serialNumberMaster) in the unit-array Special case: If an empty string or the value [-1] is passed, the position is not taken into account; in this case, the master serial number must either be unique or passed with an empty string or [-1] (the active work order is then used)!

Parameter name	Explanation
checkMultiBoard	If <code>serialNumberSlave</code> is part of a multiple panel, this parameter determines whether only the passed or all panels are checked [0; 1; 2; 3]: 0 = Single panel; only the mounting of the passed slave serial number is checked (only work steps with obligatory confirmation) 1 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and checked (only work steps with obligatory confirmation) 2 = Single panel; only the mounting of the passed slave serial number is checked (all work steps) 3 = Multiple panel; on the basis of the slave serial number, all other serial numbers of the multiple panel are also ascertained and checked (all work steps) Advice regarding "mode differentiation": <i>The difference between mode 0 and 2 and between 1 and 3 is that in mode 2 and 3 the state of work steps without obligatory confirmation is also taken into consideration during the mounting check! For all modes, either all or only the current work step is checked; this is controlled using station parameter code no. 7203 [▶▶▶]!</i> Advice: <i>If the value [1] or [3] is passed (multiple panel), slave serial numbers are assigned to master serial numbers on the basis of the same panel position; i.e., slave panel position 1 is checked against master panel position 1; the number of panels must be identical for slave and master!</i>

Tab. 26-180 Parameter (in) »trVerifyMerge«

Parameter (out)

Parameter name	Explanation
serialNumberStateDataArray	Array of the structure with the following variables:
<code>serialNumberSlave</code>	Serial number which is deleted during a product merging (see <code>serialNumberMaster</code>)
<code>serialNumberSlavePos</code>	Position of the single panel (here <code>serialNumberSlave</code>) in the unit-array
<code>state</code>	Process state of the serial number [0; 1; 2]: 0 = pass 1 = fail 2 = scrap

Tab. 26-181 Parameter (out) »trVerifyMerge«

26.61.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
225	Warning	Serial number -> Mounting permitted at local site
221	Warning	The slave serial no. state is FAIL on last work step
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
		Advice: The value 207 is only output if station parameter code no. 18200 [▶▶▶] is active!
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-114	Failure	Serial number is archived
-120	Failure	Work plan not found
-133	Failure	No active work order found for the station
-181	Failure	Station is locked
-202	Failure	Serial number is locked

-216	Failure	Station is invalid for the serial number
-220	Failure	Mounting (merge) is invalid
-221	Failure	The slave serial no. is not released from previous process
-223	Failure	There are multiple valid serial numbers for the merge
-224	Failure	Serial number is not finalized
-226	Failure	The slave serial no. has been booked from a previous process as FAIL
-227	Failure	The slave serial no. has been booked from a previous process as SCRAP
-234	Failure	Number of panels (master) and slave serial nos. are not equal
-301	Failure	Serial number is mounted
-353	Failure	Mounting without order network not possible
		Advice: The value -353 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 2!
-354	Failure	Work orders of serial numbers to be mounted do not match order network
		Advice: The value -354 is only output if station parameter code no. 7260 [▶▶▶] or site parameter 7260 [▶▶▶] is operated in mode 1!
-355	Failure	Production order is part of a work order cluster
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

26.62

trVerifyMergeProduct

26.62.1

Function description

This function checks whether it is permitted to install the unit with the serial number passed (`serialNumberSlave`) into the product (part number of the product passed and product version). During this process, a check is also performed to determine whether the slave serial number has been completely assembled and was booked at the last work step with obligatory confirmation with the "PASS" state.



Advice

This API function is influenced by station parameter code nos. 7203 [▶▶▶], 7217 [▶▶▶], 7218 [▶▶▶], 7219 [▶▶▶] and 7223 [▶▶▶] beeinflusst. Further information on this topic can be found in the reference section of this documentation!

Unlike with the »trVerifyMerge« [▶▶▶] function, which only performs a check for the next integration level, the number of integration levels to be searched can be specified here (parameter `level`); it is also possible to ascertain whether the slave serial number may be mounted anywhere within the product structure tree (beginning with the passed product).

Licensing

In order for this API function to be used, the executing station must be licensed as follows:

	TR license (active)
--	---------------------

Function declaration

```
int trVerifyMergeProduct
in IMApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in String partNumber
in int bomVersion
in String bomIndex
in String serialNumberSlave
in int level
```

26.62.2

Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
partNumber	Part number of the finished product in the iTAC system
bomVersion	BOM version from the iTAC system Special case: If [-1] is passed, the most current version is used automatically!
bomIndex	Bill-of-material version (index) that is passed to the system by means of changes in BOMs; these are manual changes or changes that have been initiated by higher-level ERP systems Special case: If [-1] is passed, the most current version is used automatically!
serialNumberSlave	Serial number which is deleted during a product merging
level	Number of integration levels to be searched, beginning with the passed product (see <code>partNumber</code>) > 0 = Maximum resolution depth (integration level number) Advice: If the value [-1] is passed here, all integration levels of the product structure tree can be searched; in some cases, this process may take a relatively long time!

Tab. 26-182 Parameter (in) »trVerifyMergeProduct«

**Advice**

This function has no output parameters; it only outputs one of the output values listed below!

26.62.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-110	Failure	Serial no. not found
-112	Failure	Slave serial no. not found
-114	Failure	Serial number is archived
-120	Failure	Work plan not found
-121	Failure	Part not found
-181	Failure	Station is locked
-202	Failure	Serial number is locked
-220	Failure	Mounting (merge) is invalid
-221	Failure	The slave serial no. is not released from previous process
-355	Failure	Production order is part of a work order cluster

-1101	Failure	BOM not found
-2100	Failure	Serial number is sent to SCT
-2138	Failure	Serial number is deleted

27

References

27.1

IMSApi - error codes

The IMSApi distinguishes between technical and technological error codes. Only for the technical error codes can a corresponding output text can be queried if necessary. Technological error codes are dependent on the used IMSApi library and do not contain any output texts.

Customer-specific error codes

In addition to the technical and technological error codes described in the following, there is a third block of numbers for the development of so-called "CustomFunctions". The customer-specific error messages are thereby separated from the standardized IMSApi error messages.

-30,000 to -32,000 = Reserved range for customer-specific failures

30,000 to 32,000 = Reserved range for customer-specific warnings

27.1.1

Technological error codes

Technological error codes

The following blocks of numbers were – depending on the used IMSAPI library – reserved for the technological error codes:

-10000 -10099	Error code block for IMSAPI Shared Lib -10001 = JNI_JVM_UNKNOWN_ERROR -10002 = JNI_JVM_THREAD_ERROR -10003 = JNI_JVM_VERSION_ERROR -10004 = JNI_JVM_OUT_OF_MEMORY -10005 = JNI_JVM_ALREADY_CREATED -10006 = JNI_JVM_INVALID_ARGUMENTS -10009 = JNI_CALL_IMSAPI_METHOD -10010 = JNI_STATIC_METHOD_NOT_FOUND -10011 = JNI_CANT_CREATE_UTF_STRING -10012 = JNI_CANT_RELEASE_UTF_STRING -10013 = JNI_CALL_STATIC_METHOD -10014 = JNI_CALL_OBJECT_METHOD -10015 = JNI_CLASS_NOT_FOUND_FOR_OBJECT -10016 = JNI_CLASS_NOT_FOUND -10017 = JNI_METHOD_NOT_FOUND -10018 = JNI_FIELD_NOT_FOUND -10019 = JNI_CANT_CREATE_OBJECT -10020 = JNI_CANT_RELEASE_OBJECT -10021 = JNI_CANT_ACCESS_ARRAY_ELEMENTS -10022 = JNI_CANT_QUERY_ARRAY_SIZE -10023 = JNI_RETURNED_REFERENCE_IS_NULL -10024 = JNI_POINTER_IS_NULL -10025 = JNI_CANT_DELETE_REF -10030 = IMSAPI_NOT_INITIALIZED -10031 = IMSAPI_ALREADY_INITIALIZED -10032 = IMSAPI_NO_VALID_LOGIN -10033 = IMSAPI_MAX_SESSIONS_REACHED -10035 = IMSAPI_NO_ENTRIES_IN_OPTIONS_FILE -10036 = IMSAPI_NOT_ENOUGH_MEMORY -10037 = IMSAPI_CANT_WRITE_LOGFILE -10038 = IMSAPI_NULL_POINTER -10039 = IMSAPI_INVALID_ARRAY_SIZE -10040 = IMSAPI_UNICODE_CONVERSION
-10100 -10199	Error code block for IMSAPI Java Library -10101 = IMSAPI_NOT_INITIALIZED -10102 = IMSAPI_ALREADY_INITIALIZED -10103 = IMSAPI_NULL_POINTER -10104 = IMSAPI_SERVICE_LOOKUP_FAILED -10105 = IMSAPI_REMOTE_CALL_FAILED

-10200 -10299	Error code block for IMSApi Socket Adapter -10200 = INPUSTREAM_CANT_READ -10201 = INPUTSTREAM_EOF -10202 = OUTPUTSTREAM_CANT_WRITE -10205 = CHARSET_UNKNOWN -10210 = FKT_UNKNOWN_NAME -10211 = TOO_LESS_ARGUMENTS -10212 = TOO_MUCH_ARGUMENTS -10213 = FKT_CANT_CALL -10214 = FKT_RESULT_OBJECT_CANT_ACCESS -10220 = DATATYPE_INVALID -10225 = PARSE_INTEGER_VALUE -10226 = PARSE_DOUBLE_VALUE -10227 = PARSE_LONG_VALUE -10228 = PARSE_BOOLEAN_VALUE -10229 = PARSE_SHORT_VALUE -10230 = PARSE_FLOAT_VALUE -10231 = PARSE_BYTE_VALUE -10240 = ARRAY_PARSE_SIZE -10241 = ARRAY_SIZE_NEGATIVE -10242 = ARRAY_CANT_CREATE -10250 = STRUCT_CANT_FIND_CLASS -10251 = STRUCT_UNKNOWN_FIELD_NAME -10252 = STRUCT_CANT_CREATE_OBJECT -10253 = CANT_SET_ELEMENT -10292 = RS232_CANT_SET_LISTENER -10293 = RS232_WRONG_PORT_TYPE -10294 = RS232_PORT_PARAM_CANNOT_SET -10295 = RS232_PORT_IN_USE -10296 = WRONG_RS232_PORTNAME -10297 = WRONG_RS232_PROPERTY -10298 = REMOTE_SERVICE_LOOKUP -10299 = SERVER_SOCKET_CREATE
-10300 -10399	Error code block for IMSApi DotNet -10301 = imsapi is not initialized -10302 = imsapi is initialized already -10303 = locator init failed -10304 = lookup for imsapi failed -10305 = remote server call failed
-10400 -10499	Error code block for IMSApi PLC Connector -10401 = PLCIF_E_ARRAY_TOO_SMALL -10402 = PLCIF_E_ILLEGAL_ARGUMENT
-10500 -10599	Error code block for IMSApi COM -10501 = imsapi is not initialized -10502 = imsapi is initialized already -10503 = logging configuration error -10504 = locator init failed -10505 = lookup for imsapi failed -10506 = remote server call failed -10507 = invalid 64-bit integer value



Advice

Details on the actually used error codes can be found in the detailed description of the respective IMSApi library!

27.1.2

Group "COMMON" (error codes)

*Technical error codes
(COMMON)*

Listed below are all "technical" output values (error codes) from the "COMMON" failure group with the associated output texts. The following blocks of numbers apply for this failure group:

0 - 199
2000 - 2199

Error code	Category	Output text
9	Warning	Machine condition/message already exists
8	Warning	Machine condition/message already assigned
7	Warning	At least one record of the array has been processed with a warning
6	Warning	At least one panel position has SCRAP state
5	Warning	Not all records in array were successfully processed
4	Warning	Passed "locality" not found - default (server) will be used
3	Warning	More data found
1	Warning	No data found
0	OK	[EMPTY]
-3	Failure	Session is invalid
-4	Failure	User is unknown
-5	Failure	Password is invalid
-6	Failure	User/station is locked
-7	Failure	Client is unknown
-8	Failure	No license
-9	Failure	Session login failed
-10	Failure	Server error
-11	Failure	Password for user/station is invalid
-13	Failure	Error code not found
-14	Failure	System login failed

-15	Failure	At least one passed value is too large for the respective data-base field
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-102	Failure	No PM license present for the station
-103	Failure	Station is exclusively locked
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-106	Failure	This user has already logged into the station
-107	Failure	Another user is already logged into the station
-108	Failure	Station may not add serial numbers
-109	Failure	Line not found
-110	Failure	Serial no. not found
-111	Failure	No pFDC license present for the station
-112	Failure	Slave serial no. not found
-113	Failure	Serial number not unique
-114	Failure	Serial number is archived
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-118	Failure	Invalid number format in IMSApi key
-119	Failure	Part is not unique
-120	Failure	Work plan not found
-121	Failure	Part not found
-122	Failure	Part group not found
-123	Failure	Part group type not found
-124	Failure	Part does not match the work order part

-125	Failure	ERP group not found
-127	Failure	Part number is invalid
-126	Failure	Process version is invalid
-128	Failure	Work step not found
-129	Failure	Part description not found
-130	Failure	Work order not found
-131	Failure	No work order found for serial number
-132	Failure	Work order is not unique
-133	Failure	No active work order found for the station
-134	Failure	Default value for maximum order quantity has been reached
-135	Failure	Work order activation failed
-136	Failure	Customer part not found
-137	Failure	No work order could be created
-138	Failure	Order state is invalid
-139	Failure	BOM version does not match the version from the work order
-141	Failure	Failure while reading the quantities for the work order
-144	Failure	No decision table found for this rule
-145	Failure	Failure while accessing the rule
-146	Failure	Failure in the rule
-147	Failure	Rule type is not correct
-148	Failure	Required rule parameters are missing
-149	Failure	Work order or part number is required
-150	Failure	Invalid value for "Flag"
-151	Failure	The "serialNumberRef" parameter must be specified
-152	Failure	Not enough comparison values exist for the calculation
-153	Failure	Invalid value for "type" [V, R, B, D, -1]

-154	Failure	Invalid value for "checkActiveWorkOrder" [0; 1]
-155	Failure	Invalid value for "duplicateSerialNumber" [0; 1]
-156	Failure	Date value is invalid
-157	Failure	Mode 2 ("workstepFlag") requires a serial number
-158	Failure	Mode 3 ("workstepFlag") requires a valid orientation of the PCB
-159	Failure	Invalid value for "verifyCompNameBased" [0; 1]
-160	Failure	Invalid value for "createNewBom" [0; 1]
-161	Failure	Invalid value for "alternative" [0; 1]
-162	Failure	Invalid value for "processBased" [0; 1]
-163	Failure	Invalid value for "bomType"
-164	Failure	Wrong parameter
-165	Failure	Setting in combination with AVO no. invalid as "processLayer"
-167	Failure	Invalid value for "checkProcessStep" [0; 1]
-168	Failure	Invalid value for the function mode (control flag)
-169	Failure	Quantity is invalid
-170	Failure	Invalid value for "desolvingSerialNumber"
-171	Failure	Invalid value for "allProductEntries"
-172	Failure	Invalid value for "onlyLastEntry"
-173	Failure	Negative value for "numberOfRecords"
-174	Failure	Number of records to be output has fallen below the minimum value
-175	Failure	Invalid value for "model" [0; 1; 2]
-176	Failure	Ambiguous value for "REPAIRED" -> the data were booked as "not repaired"
-177	Failure	Mounting place is invalid
-181	Failure	Station is locked
-183	Failure	Invalid value for "sequenceType" [0; 1]

-184	Failure	Invalid value for "activateFlag" [0; 1; 2]
-185	Failure	Invalid value for the process layer
-186	Failure	Product version is not valid
-187	Failure	Product version not found
-188	Failure	Invalid value for "checkMultiboard" [0; 1]
-189	Failure	Invalid value for "processType" [M; P]
-190	Failure	Invalid value for "state" [-1; 0; 1; 2; 3; 4]
-191	Failure	Invalid value for "workstepNumber" [-2; -1; 0 or valid AVO no.]
-192	Failure	Invalid value for "multiplePanel" [>= 0]
-193	Failure	Invalid value for "registrationType" [0; 1]
-194	Failure	Invalid value for "sortFlag" [0; 1]
-195	Failure	Mount point not found
-196	Failure	A parameter is invalid
-198	Failure	Invalid value for "objectType"
-199	Failure	Formula not found
-2000	Failure	Formula parameter missing
-2001	Failure	Formula parameter not convertible
-2002	Failure	Formula is invalid
-2003	Failure	Formula calculation failed
-2004	Failure	No booking exists for this serial number
-2005	Failure	No active work order found
-2006	Failure	Invalid value for "stateCheck" [0; 1]
-2007	Failure	Invalid value for "confirmFlag" [0; 1]
-2008	Failure	Application ID is invalid
-2009	Failure	Application ID not found
-2010	Failure	Application ID is locked

-2011	Failure	End date precedes start date
-2012	Failure	Invalid value for "batchComplete" [0; 1]
-2013	Failure	Value is outside of the validity range
-2014	Failure	Customer not found
-2015	Failure	Date not valid
-2016	Failure	Work order type does not match work plan type
-2017	Failure	Part not available
-2018	Failure	Part type is invalid
-2019	Failure	Process state not valid
-2020	Failure	Production area not valid
-2021	Failure	Work order type not valid
-2022	Failure	Control code not valid
-2023	Failure	Retrograde not valid
-2024	Failure	Quantity -> Value outside of the validity range
-2025	Failure	Production order already exists
-2026	Failure	Production order -> PCB assignment invalid
-2027	Failure	Part has an invalid state
-2028	Failure	Part group has an invalid part group type
-2029	Failure	Work plan cannot be used for first batch
-2030	Failure	Work plan not valid
-2031	Failure	Work plan is not released
-2032	Failure	Work plan contains no work steps with valid confirmation
-2033	Failure	Work plan contains invalid work step
-2034	Failure	Work plan -> No valid order number model found for the WP alternative
-2035	Failure	Production controller not found

-2036	Failure	No matching station exists
-2037	Failure	Work step not found for station
-2038	Failure	Station does not belong to line
-2039	Failure	Part does not belong to the part group
-2040	Failure	Work plan -> Type is invalid
-2041	Failure	Work plan -> Alternative does not match the work plan type
-2042	Failure	Work plan -> Alternative not valid
-2043	Failure	Site not found
-2044	Failure	Station -> Type is invalid
-2045	Failure	Station is not active
-2046	Failure	Station -> Reference not found
-2047	Failure	Cell is not active
-2048	Failure	Cell not found
-2049	Failure	Machine group is not active
-2050	Failure	Machine group not found
-2051	Failure	Line is not active
-2052	Failure	Housing form not found
-2060	Failure	Changing the order quantity is not allowed
-2061	Failure	Machine group is not unique
-2062	Failure	Bin is empty
-2063	Failure	Invalid value for "functionMode"
-2064	Failure	Production order -> Parameter missing
-2070	Failure	IMSApi key was passed multiple times to the function
-2071	Failure	Machine group for station not found
-2072	Failure	Invalid value for "desolvingLevel"
-2073	Failure	More than one work order is active on the station

-2100	Failure	Serial number is sent to SCT
-2101	Failure	Slave serial number is sent to SCT
-2102	Failure	Index is not unique
-2103	Failure	Index is invalid
-2104	Failure	Serial number has already been assigned to the work order
-2105	Failure	No TR or PM license present for the station
-2106	Failure	No iRule license present for the site
-2107	Failure	No ML license present for the site
-2108	Failure	No APS license present for the station
-2109	Failure	Production order has "not released" state
-2110	Failure	Serial number position is invalid
-2111	Failure	Master work plan not found
-2112	Failure	Part is not a product
-2113	Failure	ERP group is invalid
-2115	Failure	Work step is invalid
-2116	Failure	BOM -> Type is invalid
-2117	Failure	Value has exceeded the maximum size
-2118	Failure	Value has fallen below the minimum size
-2119	Failure	Number of records to be output has exceeded the maximum value
-2120	Failure	At least one record in the array could not be processed
-2121	Failure	Part image -> X-coordinate is invalid
-2122	Failure	Part image -> Y-coordinate is invalid
-2123	Failure	Part image not found
-2124	Failure	A value was passed multiple times to the function
-2125	Failure	Split ID -> Quantity exceeded

-2126	Failure	Split ID not valid
-2127	Failure	Split ID not activated
-2128	Failure	Split ID is not unique
-2129	Failure	Station is not part of the split work step
-2130	Failure	Test plan data not found
-2131	Failure	Team not found
-2132	Failure	User not found
-2133	Failure	User group not found
-2134	Failure	Entity type is invalid
-2135	Failure	Entity key is invalid
-2136	Failure	Resource not found
-2137	Failure	Invalid value for "SELECTION_TYPE"
-2138	Failure	Serial number is deleted
-2139	Failure	Resource is not active
-2140	Failure	Resource does not match station
-2141	Failure	Resource and data capture point do not match
-2142	Failure	Invalid value for "isAcknowledged" [0; 1; null]
-2143	Failure	No stations for the line
-2144	Failure	Invalid value for "onlyActive" [0; 1]
-2145	Failure	Invalid value for "onlyValid" [0; 1]
-2146	Failure	Station is invalid
-2147	Failure	Machine group is invalid
-2148	Failure	ERP group not active
-2149	Failure	Timer ID not found
-2150	Failure	Value is empty
-2151	Failure	Value is NULL

-2152	Failure	Value is not in the future
-2153	Failure	Value is not in the past
-2154	Failure	Value has invalid size
-2155	Failure	Value is an invalid expression
-2156	Failure	Value must be NULL
-2157	Failure	Invalid value for "isOpen" [0; 1; 2; null]

27.1.3

Group "TR" (error codes)

Technical error codes (TR)

Listed below are all "technical" output values (error codes) from the "TR" failure group with the associated output texts. The following block of numbers applies for this failure group:

200 - 399

Error code	Category	Output text
234	Warning	Serial number locked but usable
233	Warning	Anomaly detected (PAA)
232	Warning	Cpk violation
231	Warning	Cp violation
230	Warning	Not enough values for the PAA
225	Warning	Serial number -> Mounting permitted at local site
224	Warning	Serial number -> State was changed
222	Warning	The slave serial no. state is SCRAP on last work step
221	Warning	The slave serial no. state is FAIL on last work step
212	Warning	Serial number is valid for this station, but it has not yet been booked
211	Warning	A secondary failure was detected
210	Warning	At least one serial number has the SCRAP state
209	Warning	Serial number is not assigned to a panel
208	Warning	Serial number is invalid for this station -> FAIL booking at at least one of the previous work steps
207	Warning	Serial number is invalid for this station -> SCRAP booking at at least one of the previous work steps
206	Warning	PASS state but number of permitted tests reached
205	Warning	FAIL state but number of permitted tests reached
204	Warning	Serial number state at the previous work step: SCRAP
203	Warning	Serial number is invalid for this station -> no booking at at least one of the previous work steps

202	Warning	Serial number is invalid for this station -> no booking at the previous work step
201	Warning	Serial number state at the previous work step: FAIL
-200	Failure	Invalid value for "serialNumberState"
-201	Failure	Serial number is not active
-202	Failure	Serial number is locked
-203	Failure	Serial number is not a reference serial number
-205	Failure	Serial number belongs to a different work order
-206	Failure	Serial number was already assigned to this work order
-213	Failure	Maximum number of bookings reached
-214	Failure	Invalid value for "resolveDirection"
-215	Failure	Serial number is invalid for the state booking
-216	Failure	Station is invalid for the serial number
-217	Failure	Serial number and work order have different product versions
-219	Failure	Serial number is not finalized
-220	Failure	Mounting (merge) is invalid
-221	Failure	The slave serial no. is not released from previous process
-222	Failure	The serial no. state is SCRAP on last work step
-223	Failure	There are multiple valid serial numbers for the merge
-224	Failure	Serial number is not finalized
-226	Failure	The slave serial no. has been booked from a previous process as FAIL
-227	Failure	The slave serial no. has been booked from a previous process as SCRAP
-228	Failure	Slave serial number is locked
-229	Failure	Serial number not yet booked by the previous station
-230	Failure	Station is invalid for this work order
-231	Failure	Booking of the serial number failed

-232	Failure	Failure during product merging (merge)
-234	Failure	Number of panels (master) and slave serial nos. are not equal
-235	Failure	New serial number could not be created
-237	Failure	Serial number belongs to a different station
-238	Failure	Failure during product separation (unmerge)
-239	Failure	Serial number array is empty
-240	Failure	No work step found for work order, orientation of the PCB and station
-241	Failure	The work step is not unique for the orientation of the PCB and the station
-242	Failure	BOM version is invalid
-243	Failure	Work step for the work order not found
-244	Failure	Serial number is in the array more than once
-245	Failure	Quantities could not be reported to the PM module
-246	Failure	Rename of the serial number failed
-260	Failure	Station is not configured for the SPA test
-261	Failure	Work plan is not configured for the SPA test
-262	Failure	Recipe is not configured for the SPA test
-263	Failure	Station is not configured for the PAA test
-264	Failure	Work plan is not configured for the PAA test
-265	Failure	Recipe is not configured for the PAA test
-266	Failure	Serial number is not configured for the PAA test
-269	Failure	FAIL state due to anomaly
-271	Failure	Serial number is packed
-272	Failure	Serial number already exists
-280	Failure	Serial number has already been locked with this lock group
-281	Failure	Lock group not found

-282	Failure	Serial number is not locked
-283	Failure	Serial number pool already exists
-284	Failure	Creation of the serial number pool failed
-286	Failure	Deletion of the serial number pool failed
-287	Failure	Serial number pool does not exist
-288	Failure	Deletion of the serial number pool not possible
-289	Failure	Serial number is already in the pool
-290	Failure	Serial number is on the SCRAP work step
-291	Failure	Serial number is not yet booked
-293	Failure	Serial number is not assigned to any pool
-294	Failure	Serial number is assigned to another pool
-295	Failure	Removing the serial number is not allowed (invalid state)
-296	Failure	Serial number is on the FAIL work step
-299	Failure	Serial number is already assigned to a repair order
-301	Failure	Serial number is mounted
-302	Failure	Work order type is not valid
-303	Failure	Modification order is already assigned to a sno. pool
-304	Failure	Modification order not found
-305	Failure	The modification order has already been assigned another sno. pool
-306	Failure	The serial number pool has an invalid state
-307	Failure	The serial number pool is not assigned to the modification order
-308	Failure	The modification order was already assigned this sno. pool
-309	Failure	Serial no. is not identified
-310	Failure	Serial no. is not assigned to the modification order
-311	Failure	Serial no. was not transferred

-312	Failure	Serial number pool is already finished
-313	Failure	Sno. pool is already assigned to a modification order
-314	Failure	Sno. pool has not finished
-315	Failure	Serial number has already been booked
-316	Failure	The serial number pool is not unique (pool name and site must be unique)
-317	Failure	Invalid value for "snrPoolState" [0; 1; 2; 3]
-318	Failure	Sno. pool has not assigned any modification order
-320	Failure	No serial number generation configured for this part type and order type
-321	Failure	Serial number generation failed
-322	Failure	No previous booking found for this serial number and this work step
-327	Failure	Station has an invalid state
-328	Failure	Station state not created
-329	Failure	Measurement data not found for the station
-330	Failure	Invalid value for "MAJOR_FAILURE"
-331	Failure	Failure type code is invalid
-332	Failure	Failure cause not found
-333	Failure	No value passed for "COMP_NAME"
-334	Failure	Location acc. to BOM not found
-335	Failure	Invalid process
-336	Failure	Failure symptoms not found
-337	Failure	Failure type not found
-338	Failure	User -> Skill not assigned
-339	Failure	Failure is not assigned to the station
-340	Failure	Activity for the work step not found
-342	Failure	Deleting failures not permitted

-343	Failure	Deleting failures not possible
-344	Failure	Serial number model -> range exceeded
-345	Failure	Serial number model is invalid
-346	Failure	Failure note not found
-348	Failure	Splitting of multiple panel failed
-349	Failure	Failure measure not found
-350	Failure	Failure action not found
-351	Failure	Failure object part not found
-352	Failure	Measurement data for period already exists
-353	Failure	Mounting without order network not possible
-354	Failure	Work orders of serial numbers to be mounted do not match order network
-355	Failure	Production order is part of a work order cluster
-356	Failure	Serial number does not belong to a work order cluster
-357	Failure	Serial number belongs to different work order cluster
-358	Failure	Serial number position does not match work order cluster
-359	Failure	Layer not found in work step cluster
-360	Failure	Serial number position -> Number does not match cluster size
-361	Failure	Work order cluster -> Position is invalid for work order
-362	Failure	Reference serial no. must belong to phantom work order
-363	Failure	Serial number pool -> At least one serial number must have the PASS state
-364	Failure	Serial number must be ejected
-365	Failure	Serial number has already been copied to the work order
-366	Failure	More than one user skill assigned
-367	Failure	Sequence for measurement data does not exist
-368	Failure	Skill not found

-369	Failure	Failure risk not found
-370	Failure	Work order cluster -> Position not unique for work order
-371	Failure	Work order cluster -> Position not found for work order

27.1.4

Group "SHIP" (error codes)

Technical error codes (SHIP)

Listed below are all "technical" output values (error codes) from the "SHIP" failure group with the associated output texts. The following block of numbers applies for this failure group:

400 - 499

Error code	Category	Output text
408	Warning	Lot (childLot) locked but usable
407	Warning	Lot (parentLot) locked but usable
406	Warning	First serial number
405	Warning	Last serial number
404	Warning	One or more subordinate lots were not found
-400	Failure	Lot not found
-401	Failure	Lot (childLot) not found
-402	Failure	Lot (childLot) is not closed
-403	Failure	Serial numbers in the lot have a different part no.
-404	Failure	Serial numbers in the lot have a different BOM version
-405	Failure	Lot (childLot) is already assigned to the bin
-406	Failure	Lot (childLot) is already assigned to another bin
-409	Failure	Lot is already completed, processing is not permitted
-410	Failure	Lot is already closed, processing is not permitted
-412	Failure	Lot is activated at another station
-413	Failure	Lot (parentLot) not found
-416	Failure	New lot (parentLot) not found
-418	Failure	Serial number was not found in the lot
-419	Failure	Lot (childLot) is not unique
-421	Failure	Lot is not registered at the station
-422	Failure	Part no. is not configured for the bin

-423	Failure	Serial number is already assigned to another lot
-424	Failure	Serial number is already assigned to the lot
-425	Failure	Part no. is different from the serial numbers already included
-426	Failure	BOM version is different from the version of the serial numbers that is already included
-427	Failure	Serial number was not assigned to a lot
-429	Failure	Bin is not of type "lot"
-430	Failure	Lot has insufficient fill quantity
-432	Failure	Lot was not assigned
-436	Failure	Lot is not closed
-437	Failure	Package quantity already reached
-438	Failure	Lot is not contained in the parent bin (parentLot)
-439	Failure	Lot is already empty
-440	Failure	Work order is different from the work order of the serial numbers already included
-441	Failure	Lot is already sent, processing is not permitted
-450	Failure	The lot attribute of the lot has a different part no. than the serial number
-451	Failure	Invalid value for the lot attribute "Part quantity". Value must be greater than 0.
-452	Failure	Invalid value for the lot attribute "Part name". Part must be present in the system.
-453	Failure	No locked lots may be registered at this station.
-454	Failure	Only locked lots may be registered at this station.
-455	Failure	Serial number is locked. No locked serial numbers may be packed at this station.
-456	Failure	Serial number is not locked. Only locked serial numbers may be packed at this station.
-457	Failure	No locked lots or serial numbers may be processed. The new lot is locked or contains locked items.

-458	Failure	No locked lots or serial numbers may be processed. The lot is locked or contains locked items.
-459	Failure	Only locked lots or serial numbers may be processed. The new lot is not locked or contains non-locked items.
-461	Failure	No locked lots or serial numbers may be processed at this station. However, the lot contains locked lots or serial numbers.
-462	Failure	Only locked lots or serial numbers may be processed at this station. However, the lot contains non-locked lots or serial numbers.
-463	Failure	No locked lots or serial numbers may be processed at this station. However, the new subordinate lot is locked or contains locked items.
-464	Failure	Only locked lots or serial numbers may be processed at this station. However, the new subordinate lot is not locked or contains non-locked items.
-465	Failure	Shipping failed -> The lot contains locked items; only non-locked items can/may be shipped at this station.
-466	Failure	Shipping failed -> The lot contains non-locked items; only locked items can/may be shipped at this station.
-467	Failure	Closing failed -> The lot contains locked items; only lots with non-locked items can/may be closed at this station.
-468	Failure	Closing failed -> The lot contains non-locked items; only lots with locked items can/may be closed at this station.
-469	Failure	The serial number is already archived and cannot be added to the lot.

27.1.5

Group "ML" (error codes)

*Technical error codes
(ML)*

Listed below are all "technical" output values (error codes) from the "ML" failure group with the associated output texts. The following block of numbers applies for this failure group:

500 - 599

Error code	Category	Output text
538	Warning	At least one container is finished, locked, empty or expired
537	Warning	At least one container is finished, locked or expired
536	Warning	At least one container is empty, finished or locked
535	Warning	One or more containers are empty and expired
504	Warning	The material quantity in the system has become negative
503	Warning	One or more containers are empty and expired
502	Warning	One or more containers are expired
501	Warning	One or more containers are empty
500	Warning	At least one container is completed or locked
-501	Failure	At least one material bin not found
-502	Failure	Failure creating the material bin
-503	Failure	Container not found
-504	Failure	An existing bin number can only be created if the old bin is empty
-505	Failure	Failure deactivating the material bin
-506	Failure	Material bin is not yet processed at another station
-507	Failure	Material bin is available but empty
-508	Failure	Material bin is already stored
-509	Failure	Material lot not found
-511	Failure	Material bin is not registered at the station
-512	Failure	Material bin is already registered at another station
-513	Failure	A material bin is already registered at this station

-514	Failure	Invalid character in "materialBinNumber" [?; %; ' ; -1]
-517	Failure	Retrieval/creation of a supplier not possible
-518	Failure	Container state is invalid
-519	Failure	Part does not match container material
-520	Failure	Material bin and serial number belong to another work order
-521	Failure	Material bin and serial number belong to the same work order
-523	Failure	Part is configured for retrograde booking
-525	Failure	Invalid value for "transactionType"
-526	Failure	Invalid value for "quantity"
-527	Failure	Retrieval/creation of receiving not possible
-528	Failure	Storage place was not found
-531	Failure	Supplier not found
-532	Failure	Material lot is locked
-533	Failure	State of the material bin is invalid
-534	Failure	Material bin is already registered
-535	Failure	Storage place is not unique
-537	Failure	Container is already locked with the lock group
-538	Failure	Material lot is already locked with the lock group
-539	Failure	Not all containers are unlocked
-542	Failure	Container is locked
-543	Failure	Container is expired
-544	Failure	Container is empty
-545	Failure	Container is finished
-546	Failure	The material lot no. is not unique
-547	Failure	Storage place not found
-548	Failure	Failure creating the material lot

-549	Failure	Container already exists
-550	Failure	Container number not unique
-551	Failure	Container splitting is not allowed
-552	Failure	Container is not permitted for part
-560	Failure	Order not found
-561	Failure	Different part in order
-565	Failure	Delivery slip number is invalid
-566	Failure	Delivery slip position is invalid
-567	Failure	Order state is invalid
-568	Failure	Receiving number not found
-569	Failure	Delivery slip number not found
-570	Failure	Store state not available
-571	Failure	Store state not found
-572	Failure	Storage cell not found
-573	Failure	Storage group not found
-574	Failure	Storage location not found
-575	Failure	Storage location data is inconsistent
-576	Failure	Receiving not found
-577	Failure	Receiving position not found
-578	Failure	The maximum expiration date must NOT be before the expiration date
-579	Failure	Max. number of registered containers exceeded
-580	Failure	Container belongs to another work order or work step
-581	Failure	Manufacturer number is not unique

27.1.6

Group "SETUP" (error codes)

Technical error codes (SETUP)

Listed below are all "technical" output values (error codes) from the "SETUP" failure group with the associated output texts. The following block of numbers applies for this failure group:

600 - 699

Error code	Category	Output text
608	Warning	Not all positions stripped down
607	Warning	Container is invalid
605	Warning	No station activated for the setup
604	Warning	No BOM position identified as requiring setup
603	Warning	No work step identified as requiring setup
602	Warning	Setup was not activated for all components
601	Warning	Setup was not activated
600	Warning	Setup activated
-601	Failure	No setup data found
-602	Failure	Name of the placement program non-unique
-603	Failure	Work order part does not match the setup part
-604	Failure	At least one component at a mounting place causes a problem
-605	Failure	Setup not found for the station
-606	Failure	Setup is not assigned to the work order
-607	Failure	Not all parts were set up
-609	Failure	Invalid value for "setupFlag" [0; 1; 2]
-610	Failure	Invalid value for "setupState" [0; 1]
-614	Failure	Part has been set up on the wrong station
-615	Failure	Station not found in the processing plan
-616	Failure	Part not found in the setup
-617	Failure	Mounting place not found in the setup

-618	Failure	Invalid setup position
-619	Failure	Time range "Setup" is invalid
-620	Failure	Wrong component, slot was stripped down
-622	Failure	Setup belongs to a different work order
-623	Failure	Additional component set up
-624	Failure	No serial number is assigned to panel label
-625	Failure	Pre-setup -> Invalid state
-626	Failure	Pre-setup not found
-627	Failure	Pre-setup -> Position not found
-628	Failure	Pre-setup is not completed
-629	Failure	Pre-setup -> Consumer not found
-630	Failure	Pre-setup not created
-631	Failure	Pre-setup -> Invalid type
-632	Failure	Pre-setup -> Update failed
-633	Failure	Pre-setup already attached
-634	Failure	Pre-setup -> Position not unique
-635	Failure	Pre-setup -> Position already exists
-636	Failure	Pre-setup already exists
-637	Failure	Pre-setup -> Consumer already exists
-638	Failure	Pre-setup -> Pre-setup section already exists
-639	Failure	Pre-setup -> Change not allowed

27.1.7

Group "MDA" (error codes)

Technical error codes (MDA)

Listed below are all "technical" output values (error codes) from the "MDA" failure group with the associated output texts. The following block of numbers applies for this failure group:

700 - 799

Error code	Category	Output text
-701	Failure	Recipe data not found
-702	Failure	Recipe header data not found
-704	Failure	Recipe not activated
-705	Failure	Recipe not deactivated
-706	Failure	Recipe not found
-707	Failure	No active recipe found
-709	Failure	Measurement step names are not unique
-710	Failure	Measurement data does not match the active recipe
-712	Failure	No measurement data passed
-714	Failure	Invalid value for "serialUnitTrace" [0; 1]
-716	Failure	Recipe not created
-717	Failure	Invalid value for "functionMode" [0; 1]
-718	Failure	Object type invalid
-719	Failure	Station or machine group not set
-721	Failure	Recipe required for measurement data booking
-722	Failure	Invalid value for "targetType"
-723	Failure	MDA header not found
-724	Failure	MDA header non-unique
-725	Failure	Measurement data could not be processed
-726	Failure	File name not found for the MDA object
-727	Failure	URL not found for the MDA object

-728	Failure	Description not found for the MDA object
-730	Failure	Creation of the file failed
-731	Failure	MDA item -> Version is already in use
-732	Failure	Document not found
-733	Failure	Document is already activated
-734	Failure	Document is already deactivated
-735	Failure	Document is not unique
-736	Failure	Invalid value for MDA_ACTIVE [0;1]
-737	Failure	Recipe -> Measured value not found
-738	Failure	Booking references do not match

27.1.8

Group "ATTRIB" (error codes)

Technical error codes (ATTRIB)

Listed below are all "technical" output values (error codes) from the "ATTRIB" failure group with the associated output texts. The following block of numbers applies for this failure group:

900 - 999

Error code	Category	Output text
903	Warning	Attribute validation -> Attribute class condition not fulfilled
902	Warning	Attribute validation -> Attribute value condition not fulfilled
901	Warning	Attribute validation -> Type is invalid
-901	Failure	Attribute code not found
-904	Failure	Attribute code must contain at least one character
-905	Failure	Invalid value for "objectType"
-906	Failure	The attribute value in combination with "attributeValueKey" is not unique
-907	Failure	Attribute already exists
-910	Failure	Attribute code is already assigned to the serial number
-911	Failure	Attribute not found for the serial number
-912	Failure	No serial numbers found for the attribute
-913	Failure	No attributes found for the serial number
-920	Failure	Attribute code is already assigned to the work order
-921	Failure	Attribute not found for the work order
-922	Failure	No work orders found for the attribute
-923	Failure	No attributes found for the work order
-930	Failure	Attribute code is already assigned to the container
-931	Failure	No attributes found for the container
-932	Failure	No containers found for the attribute
-933	Failure	No attribute found for the container
-935	Failure	Attribute code is already assigned

-936	Failure	Attribute not found
-937	Failure	Attribute class -> State flag is not valid
-938	Failure	Attribute value condition -> State flag is not valid
-939	Failure	Attribute -> Removal failed
-940	Failure	Attribute value condition -> Type is invalid
-941	Failure	Attribute value condition not found
-942	Failure	Attribute class not found
-943	Failure	Attribute class not released
-944	Failure	Attribute class -> Invalid type
-945	Failure	Attribute value condition not released
-946	Failure	Attribute code catalog not found
-947	Failure	Attribute class has an invalid state
-948	Failure	Attribute -> Unknown validation error
-949	Failure	Attribute code -> Change not allowed
-950	Failure	Attribute validation -> Attribute class not found
-951	Failure	Attribute validation -> Attribute class not active
-952	Failure	Attribute validation -> Attribute not part of attribute class
-953	Failure	Attribute -> "Edited by" - Designator is not valid
-954	Failure	Attribute value cannot be modified

27.1.9

Group "ADVICE" (error codes)

Technical error codes (ADVICE)

Listed below are all "technical" output values (error codes) from the "ADVICE" failure group with the associated output texts. The following block of numbers applies for this failure group:

1000 - 1099

Error code	Category	Output text
-1001	Failure	Advice is unknown
-1002	Failure	Advice confirmation has failed

27.1.10

Group "MDATA" (error codes)

*Technical error codes
(MDATA)*

Listed below are all "technical" output values (error codes) from the "MDATA" failure group with the associated output texts. The following block of numbers applies for this failure group:

1100 - 1199

Error code	Category	Output text
-1101	Failure	BOM not found
-1102	Failure	BOM comparison failed
-1103	Failure	Shift not found
-1104	Failure	Invalid value for "desolvingFlag" [0; 1; 2; 3; 4; 5]
-1105	Failure	BOM for the work order not found
-1106	Failure	BOM for the work order not found
-1107	Failure	BOM position not processed
-1108	Failure	Not all BOM positions processed
-1109	Failure	BOM position -> Component name (mounting place) invalid
-1110	Failure	BOM position -> Component type invalid
-1111	Failure	BOM position not found
-1112	Failure	BOM position not created
-1113	Failure	BOM position could not be updated
-1114	Failure	Work order cluster -> Name is already in use
-1115	Failure	Work order cluster could not be updated
-1116	Failure	Work order cluster -> Phantom work order not unique
-1117	Failure	Work order cluster -> Position is occupied
-1118	Failure	Work order cluster has an invalid state
-1119	Failure	Work order cluster -> Type not found
-1120	Failure	Work order cluster -> Type is invalid
-1121	Failure	Work order cluster -> Work order is already part of another work order cluster

-1122	Failure	Work order cluster not found
-1123	Failure	Work order cluster -> Processing failed
-1124	Failure	Work order cluster -> Position not found
-1125	Failure	Work order cluster -> APS job ID not unique
-1126	Failure	Lock group has an invalid state
-1127	Failure	Alternative BOM position already exists
-1128	Failure	Shift schedule -> Start date lies after end date
-1129	Failure	State change for work step not allowed
-1130	Failure	Special time already exists
-1131	Failure	Message capture point -> Key not valid
-1132	Failure	Message capture point -> State not valid / no station available
-1133	Failure	Message capture point -> Resource not valid / no station available
-1134	Failure	Measurement data capture point -> Key not valid
-1135	Failure	Measurement data capture point -> Resource not valid / no station available.
-1136	Failure	Measurement data capture point -> Measurement step name not valid / no station available
-1137	Failure	Measured value not found
-1138	Failure	Item type not valid
-1139	Failure	Time event not found
-1140	Failure	Invalid value for "destination"

27.1.11

Group "MDC" (error codes)

*Technical error codes
(MDC)*

Listed below are all "technical" output values (error codes) from the "MDC" failure group with the associated output texts. The following block of numbers applies for this failure group:

1200 - 1299

Error code	Category	Output text
-1200	Failure	Machine condition/message not found
-1201	Failure	Machine condition/message is not assigned to the passed station
-1202	Failure	Team has no access authorization for the line
-1203	Failure	Condition is not a line condition
-1204	Failure	Changing of the machine message not allowed
-1205	Failure	Machine condition / message has an invalid state
-1206	Failure	Station is in the secondary failure state (no updating possible)
-1207	Failure	Machine condition type not found [O; Q; S; T]
-1208	Failure	Machine condition/message group not found
-1209	Failure	Machine condition/message already exists
-1210	Failure	Machine condition/message class not found
-1211	Failure	Start time already used
-1212	Failure	Message capture point invalid -> No condition/message assigned!
-1213	Failure	Message capture point invalid -> No station assigned!
-1214	Failure	Message capture point invalid
-1215	Failure	Message capture point has already been booked
-1216	Failure	Message capture point has not yet been booked
-1217	Failure	Capture point not found
-1218	Failure	Message capture point not activated
-1219	Failure	Capture point -> Type is invalid

-1220	Failure	Capture point already exists
-1221	Failure	Message class not found
-1222	Failure	Message cause not found
-1223	Failure	Message effect not found
-1224	Failure	Machine message for the line could not be created
-1225	Failure	Confirmation already exists
-1226	Failure	Message type not found
-1227	Failure	Machine condition class and group not consistent
-1228	Failure	Machine condition cause and group not consistent
-1229	Failure	Machine condition cause and class not consistent
-1230	Failure	Date -> maximum [days] exceeded
-1231	Failure	Data capture point -> Item type not valid
-1232	Failure	Measurement data capture point not valid
-1233	Failure	Measurement data capture point not activated
-1234	Failure	Measurement data capture point -> Station not valid
-1235	Failure	Measurement data capture point -> Recipe not valid
-1236	Failure	At least one filter value must be specified

27.1.12

Group "EQU" (error codes)

*Technical error codes
(EQU)*

Listed below are all "technical" output values (error codes) from the "EQU" failure group with the associated output texts. The following block of numbers applies for this failure group:

1300 - 1399

Error code	Category	Output text
1308	Warning	Cavity is invalid
1307	Warning	Cavity test returned warnings
1306	Warning	Entity already stripped down
1305	Warning	Entity already set up
1304	Warning	Equipment was stripped down
1303	Warning	The equipment has expired
1302	Warning	Equipment is already assigned to the storage place
1301	Warning	The equipment is already set up
1300	Warning	Equipment test returned warnings
-1301	Failure	Production means not found
-1302	Failure	Production means are invalid
-1303	Failure	Production means number found more than once
-1304	Failure	Production means have an invalid state
-1305	Failure	Production means are expired
-1306	Failure	Production means are unavailable
-1307	Failure	Production means data is not consistent
-1308	Failure	Station is not assigned to the work order
-1309	Failure	Equipment test returned failure
-1310	Failure	Invalid value for "setupFlag"
-1311	Failure	The equipment is set up
-1312	Failure	Production means type is invalid

-1313	Failure	Number of equipment items is invalid
-1314	Failure	Equipment -> Default part for feeder/nozzle/head not found
-1315	Failure	Equipment -> Default part for feeder/nozzle/head not configured
-1316	Failure	Production means part not found
-1317	Failure	Equipment -> Default part for feeder/nozzle/head type not found
-1318	Failure	Production means -> Change not allowed
-1319	Failure	Equipment group not found
-1320	Failure	Production means already exist
-1321	Failure	Production means class is invalid
-1322	Failure	Carrier is already assigned to a carrier
-1323	Failure	Carrier must not be assigned to the carrier
-1324	Failure	Carrier is not assigned to a carrier
-1325	Failure	Serial number is not assigned to a carrier
-1326	Failure	Serial number must not be assigned to the carrier
-1327	Failure	Serial number is already assigned to a carrier
-1328	Failure	Asset equipment must not be set up
-1329	Failure	Production means not stored
-1330	Failure	Unit part has already been removed from cavity part
-1331	Failure	Unit part is already assigned to a cavity part
-1332	Failure	Cavity not found
-1333	Failure	Cavity is not unique
-1334	Failure	Cavity part is not allowed for assignment
-1335	Failure	Cavity has already been removed from equipment
-1336	Failure	Production means type not found
-1337	Failure	Unit part must not be managed

-1338	Failure	Part number of cavity is required
-1339	Failure	Cavity part cannot be managed
-1340	Failure	Cavity part has already been removed from equipment part
-1341	Failure	Cavity part is already assigned to an equipment part
-1342	Failure	More than one assigned cavity found
-1343	Failure	Not a unique cavity setup
-1344	Failure	Cavity position is already occupied for equipment
-1345	Failure	Equipment cavity has an invalid state
-1346	Failure	Equipment cavity has expired
-1347	Failure	Cavity is invalid
-1348	Failure	Equipment cavity not available
-1349	Failure	Setup check for cavity and equipment has failed

27.1.13

Group "IMSAPI" (error codes)

Technical error codes (IMSAPI)

Listed below are all "technical" output values (error codes) from the "IMSAPI" failure group with the associated output texts. The following block of numbers applies for this failure group:

1400 - 1499

Error code	Category	Output text
-1400	Failure	Function group not found
-1401	Failure	Function names not found
-1402	Failure	Function parameter not found

27.1.14

Group "CUSTOM" (error codes)

*Technical error codes
(CUSTOM)*

Listed below are all "technical" output values (error codes) from the "CUSTOM" failure group with the associated output texts. The following block of numbers applies for this failure group:

1500 - 1599

Error code	Category	Output text
1509	Warning	Function metadata in script is incorrect
1508	Warning	No valid function metadata found in script
-1500	Failure	Script not found
-1501	Failure	Script is not active
-1502	Failure	Script was not executed
-1503	Failure	DbObject invalid
-1504	Failure	DbObject not found
-1505	Failure	DbObject -> Change failed
-1506	Failure	DbObject not licensed
-1507	Failure	DbObject -> Maximum number of columns exceeded
-1510	Failure	Function type is invalid

27.1.15

Group "SMT" (error codes)

Technical error codes (SMT)

Listed below are all "technical" output values (error codes) from the "SMT" failure group with the associated output texts. The following block of numbers applies for this failure group:

3000 - 3099

Error code	Category	Output text
-3000	Failure	SMT data is not processed
-3001	Failure	No validator present
-3002	Failure	Validation failed
-3003	Failure	Unknown failure
-3004	Failure	Container number contains invalid characters
-3005	Failure	Setup does not match the parts list
-3006	Failure	SMT -> Event type is invalid
-3007	Failure	SMT -> Line type is invalid for the configuration

27.1.16

Group "MSL" (error codes)

*Technical error codes
(MSL)*

Listed below are all "technical" output values (error codes) from the "MSL" failure group with the associated output texts. The following block of numbers applies for this failure group:

3100 - 3199

Error code	Category	Output text
3108	Warning	Part has no MSL definition
3107	Warning	MSL -> Re-drying unnecessary
3100	Warning	MSL state not changed
-3100	Failure	MSL -> Re-drying time not found
-3101	Failure	MSL -> Not found for container
-3102	Failure	MSL not unique
-3103	Failure	MSL -> Invalid state change: BAKE -> CLOSED
-3104	Failure	MSL -> Part does not have the BAKE state
-3105	Failure	MSL -> Not found for serial number
-3106	Failure	MSL -> Usage date cannot be reached
-3108	Failure	MSL -> Container is already set up
-3109	Failure	MSL -> Invalid booking date
-3110	Failure	Invalid value for "MSL_LEVEL"
-3111	Failure	MSL -> Incorrect configuration (site parameter 7070)
-3112	Failure	MSL definition not found

27.1.17

Group "EDA" (error codes)

Technical error codes (EDA)

Listed below are all "technical" output values (error codes) from the "EDA" failure group with the associated output texts. The following block of numbers applies for this failure group:

3200 - 3299

Error code	Category	Output text
-3200	Failure	External data not found
-3201	Failure	External data not found under this name
-3202	Failure	External data not assigned
-3203	Failure	External data not active
-3204	Failure	External data -> Too few data
-3205	Failure	External data has already been assigned
-3206	Failure	It is not possible to delete the external data
-3207	Failure	Invalid value for the control parameter
-3208	Failure	Invalid state
-3209	Failure	Maximum size exceeded
-3210	Failure	Type not found
-3211	Failure	Type is invalid
-3212	Failure	External data already exist
-3213	Failure	External data - > No name specified

27.1.18

Group "CONFIG" (error codes)

*Technical error codes
(CONFIG)*

Listed below are all "technical" output values (error codes) from the "CONFIG" failure group with the associated output texts. The following block of numbers applies for this failure group:

3300 - 3399

Error code	Category	Output text
-3300	Failure	Dimension is unknown
-3301	Failure	Parameter already exists
-3302	Failure	Parameter not found
-3303	Failure	Validity range not found
-3304	Failure	Configuration type not found
-3305	Failure	Parameter filter not unique
-3307	Failure	Parameter value not found
-3308	Failure	Parameter value not unique
-3309	Failure	Parameter key is missing
-3311	Failure	Dimension filter is missing
-3312	Failure	Only one dimension allowed as filter criterion
-3313	Failure	Parameter could not be deleted
-3314	Failure	Authorization for parameter is missing
-3315	Failure	Parameter filter is missing
-3316	Failure	Parameter value has invalid format

27.1.19

Group "PM" (error codes)

Technical error codes (PM)

Listed below are all "technical" output values (error codes) from the "PM" failure group with the associated output texts. The following block of numbers applies for this failure group:

3400 - 3499

Error code	Category	Output text
-3400	Failure	Cost center not found
-3401	Failure	Wage type not found
-3402	Failure	Wage type -> Connection to cost center not present
-3403	Failure	pFDC booking -> Booking mode is invalid
-3405	Failure	Failure information -> Quantity invalid
-3406	Failure	pFDC booking not found
-3407	Failure	pFDC booking -> Change not allowed: user different
-3408	Failure	pFDC booking -> Change not allowed: booking is already approved
-3409	Failure	pFDC booking invalid -> Order parameters missing
-3410	Failure	pFDC booking -> Quantity booking not allowed for wage type category
-3411	Failure	Activity type is invalid
-3412	Failure	Wage type category must not be changed
-3413	Failure	Quantities may be booked for quantity bookings only
-3414	Failure	Invalid value for "STATION_SELECTION"
-3415	Failure	Invalid value for "CONFIRMATION_MODE"
-3416	Failure	External booking code is already being used
-3417	Failure	Booking period is completed

27.1.20

Group "CRP" (error codes)

*Technical error codes
(CRP)*

Listed below are all "technical" output values (error codes) from the "CRP" failure group with the associated output texts. The following block of numbers applies for this failure group:

3500 - 3599

Error code	Category	Output text
-3500	Failure	CRP license does not exist for station (stop point)
-3501	Failure	Route not found
-3502	Failure	Route already exists
-3503	Failure	Route segment not found
-3504	Failure	Route segment already exists
-3505	Failure	Route segment -> Sequence already exists
-3506	Failure	Invalid value for UPDATE_FLAG (route)
-3507	Failure	Invalid value for UPDATE_FLAG (route segment)
-3508	Failure	ROUTE_SEGMENT_SEQUENCE missing in route segment
-3509	Failure	Invalid value for UPDATE_FLAG (transport order)
-3510	Failure	Transport order not found
-3511	Failure	Transport order not assigned
-3512	Failure	Transport order already assigned
-3513	Failure	Invalid value for PICKING_POSITION
-3514	Failure	Invalid value for OBJECT_TYPE
-3515	Failure	Stop point not found
-3516	Failure	Object number is empty; it is required if an object type is passed
-3517	Failure	Object number not found
-3518	Failure	Part number is required
-3519	Failure	Part no. not found
-3520	Failure	Production order is required

-3521	Failure	Invalid value for UPDATE_FLAG (transport order - production order)
-3522	Failure	Invalid value for UPDATE_FLAG (transport order - picking list)
-3523	Failure	Invalid value for UPDATE_FLAG (transport order - commissioning list)
-3524	Failure	Production order for this transport order not found
-3525	Failure	Production order for this transport order already exists
-3526	Failure	Picking list for this transport order not found
-3527	Failure	Picking list for this transport order already exists
-3528	Failure	Commissioning list for this transport order not found
-3529	Failure	Commissioning list for this transport order already exists
-3530	Failure	State of the transport order is invalid
-3531	Failure	State of the commissioning list for this transport order is invalid
-3532	Failure	Transport order already exists
-3533	Failure	Production order for picking list is not a valid production order for this transport order
-3534	Failure	Production order for this transport order is still referenced in the picking list
-3535	Failure	Commissioning list number for this transport order is too long

27.1.21

Group "APS" (error codes)

*Technical error codes
(APS)*

Listed below are all "technical" output values (error codes) from the "APS" failure group with the associated output texts. The following block of numbers applies for this failure group:

3600 - 3699

Error code	Category	Output text
-3600	Failure	The result gave multiple optimization groups -> Use the IMSApi "apsGetWorkorderDetailsOptGroup"

27.1.22

Group "MSG" (error codes)

Technical error codes (MSG)

Listed below are all "technical" output values (error codes) from the "MSG" failure group with the associated output texts. The following block of numbers applies for this failure group:

3700 - 3799

Error code	Category	Output text
3700	Warning	Timeout: no new messages - call msgCallback again
-3700	Failure	iTAC.Messaging.Service is deactivated (not installed)
-3702	Failure	Topic is invalid
-3703	Failure	Failure when publishing a message
-3704	Failure	Failure when subscribing to a message (only for FailOver/Load-Balancing)
-3705	Failure	A simultaneously running msgCallback is active (not allowed for the same session)
-3706	Failure	msgCallback already started
-3707	Failure	Failure when deregistering a message
-3708	Failure	General failure in the iTAC.Messaging.Service

27.2 Replaced functions

27.2.1 setupReplaceSetupForSerialNumber



Advice

This function is **NOT** being developed further; it was replaced by the »[setupChangeSetupForSerialNumber](#)« [▶▶▶] function and only remains a part of the IMSApi interface for reasons of downward compatibility!

27.2.1.1 Function description

This function is used to exchange the container for a specific component of a unit (individual item); e.g. component exchange during the course of a repair.



Advice

This API function is influenced by site parameter code no. 7070 [▶▶▶]. Further information on this topic can be found in the reference section of this documentation!

Function declaration

```
int setupReplaceSetupForSerialNumber
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String serialNumber
in int serialNumberPos
in String materialNumber
in String compName
in String materialBinNumberOld
in String materialBinNumberNew
in DateMs bookDate
```

27.2.1.2 Input and output parameters

Parameter (in)

Parameter name	Explanation
sessionContext	Structure with the following variables:
sessionId	Unique designator (ID) of the active session
persId	Unique designator (ID) of the logged in user
locale	Language settings (e.g. en_EN)
stationNumber	Station number of the work station in the iTAC system
processLayer	Orientation of the PCB during the work step [0; 1; 2]: 0 = component side 1 = soldering side 2 = position independent
serialNumber	Serial no. of the panel

Parameter name	Explanation
<code>serialNumberPos</code>	Position of the single panel in the unit-array Advice: If the value [-1] is passed, the position is ascertained automatically!
<code>materialNumber</code>	Part number in the iTAC system for the component
<code>compName</code>	Mounting place conformant with the iTAC BOM
<code>materialBinNumberOld</code>	Number of the old material bin (container no.)
<code>materialBinNumberNew</code>	Number of the new material bin (container no.) Advice: If the value [-1] is passed, the old material bin (<code>materialBinNumberOld</code>) is only removed and not exchanged. In this case, the serial number is immediately booked to FAIL!
<code>bookDate</code>	Booking time for the container change (change in the setup); specification of the total time in milliseconds since 01-01-1970, 0:00:00.000 Example: 1262443605000 milliseconds correspond to 01-02-2010, 14:46:45.000 If necessary, container change can also be captured with a time stamp from the past in this way. Special case: [-1] passes the current date and the current time!

Tab. 27-1 Parameter (in) »`setupReplaceSetupForSerialNumber`«



Advice

This function has no output parameters; it only outputs one of the output values listed below!

27.2.1.3

Output values

Error codes



This function returns one of the "technical" error codes listed below as the result.

Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station
-110	Failure	Serial no. not found
-121	Failure	Part not found
-130	Failure	Work order not found
-503	Failure	Container not found

27.2.2

smtConsumption



Advice

This function is **NOT** being developed further; it was replaced by the »smtSetupData« [▶▶▶] function and only remains a part of the IMSApi interface for reasons of downward compatibility!

27.2.2.1

Function description

This function processes the consumption data from placement machines and is used exclusively by the API-based "SMT Connector" service.



Advice

The SMT Connector includes various setup functions which are based on API functions. This (SMT) function calls the »mlUploadMaterialBinBooking« [▶▶▶] API function in the background. In the context of the SMT Connector, the »smtEventSetup« [▶▶▶], »smtSerialNumberSetup« [▶▶▶] and »smtSetupData« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int smtConsumption
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String lineName
in String setupName
in String productName
in String bomVersion
in String placementRecipe
in boolean assignSerialNumbers
in boolean bookSerialNumbers
in boolean activateSetup
inout SmtSerialNo[] serialNumbers
    String serialNumber
    int serialNumberPosition
    int serialNumberState
    int validationFlag
    int processFlag
inout SmtEvent[] events
    int eventId
    long eventDate
    int validationFlag
    int processFlag
    String position
    String feederBank
    String feederId
```

```

    int placementId
    int qtyRemain
    int qtyPlaced
    int qtyWasted
    String materialBinNumber
    boolean createMaterialBin
    String partNumber
    double quantity
    String supplier
    String lotNumber
    String dateCode
    String parentPartNumber
    inout SmtPlacement[] placements
        int placementId
        String compName
        String xPosition
        String yPosition
        int panelPosition
    in boolean ignoreContainerProblems
    in boolean createWorkOrder
    in boolean activateWorkOrder
    in String workOrderNumber
    in double cycleTime

```

27.2.2.2 Input and output parameters

Parameter (in, inout)

For the SMT functions »smtConsumption« [▶▶▶], »smtEventSetup« [▶▶▶] and »smtSerialNumberSetup« [▶▶▶], the same structures are used. Each individual function only needs a partial quantity of the input parameters specified in the declaration for processing. A detailed description of the parameters is not provided here.

27.2.2.3 Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!

If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error
-100	Failure	Station not found
-101	Failure	No TR license present for the station

-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-501	Failure	At least one material bin not found
-2107	Failure	No ML license present for the site
-3000	Failure	SMT data is not processed
-3001	Failure	No validator present
-3002	Failure	Validation failed
-3003	Failure	Unknown failure
-3004	Failure	Container number contains invalid characters

27.2.3

smtEventSetup

**Advice**

This function is **NOT** being developed further; it was replaced by the »smtSetupData« [▶▶▶] function and only remains a part of the IMSApi interface for reasons of downward compatibility!

27.2.3.1

Function description

This function processes the consumption data from placement machines and is used exclusively by the API-based "SMT Connector" service.

**Advice**

The SMT Connector includes various setup functions which are based on API functions. Depending on the process, this (SMT) function calls the »setupGetSetupName« [▶▶▶], »setupStateChange« [▶▶▶] and »setupUpdateMaterialSetup« [▶▶▶] API functions in the background. In the context of the SMT Connector, the »smtConsumption« [▶▶▶], »smtSerialNumberSetup« [▶▶▶] and »smtSetupData« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int smtEventSetup
  in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
  in String stationNumber
  in int processLayer
  in String lineName
  in String setupName
  in String productName
  in String bomVersion
  in String placementRecipe
  in boolean assignSerialNumbers
  in boolean bookSerialNumbers
  in boolean activateSetup
  inout SmtSerialNo[] serialNumbers
    String serialNumber
    int serialNumberPosition
    int serialNumberState
    int validationFlag
    int processFlag
  inout SmtEvent[] events
    int eventId
    long eventDate
    int validationFlag
    int processFlag
    String position
    String feederBank
```

```

String feederId
int placementId
int qtyRemain
int qtyPlaced
int qtyWasted
String materialBinNumber
boolean createMaterialBin
String partNumber
double quantity
String supplier
String lotNumber
String dateCode
String parentPartNumber
inout SmtPlacement[] placements
    int placementId
    String compName
    String xPosition
    String yPosition
    int panelPosition
in boolean ignoreContainerProblems
in boolean createWorkOrder
in boolean activateWorkOrder
in String workOrderNumber
in double cycleTime

```

27.2.3.2 Input and output parameters

Parameter (in, inout)

For the SMT functions »smtConsumption« [▶▶▶], »smtEventSetup« [▶▶▶] and »smtSerialNumberSetup« [▶▶▶], the same structures are used. Each individual function only needs a partial quantity of the input parameters specified in the declaration for processing. A detailed description of the parameters is not provided here.

27.2.3.3 Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
601	Warning	Setup was not activated
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error

-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-121	Failure	Part not found
-240	Failure	No work step found for work order, orientation of the PCB and station
503	Warning	One or more containers are empty and expired
-601	Failure	No setup data found
-605	Failure	Setup not found for the station
-610	Failure	Invalid value for "setupState" [0; 1]
-2107	Failure	No ML license present for the site
-3002	Failure	Validation failed

27.2.4

smtSerialNumberSetup



Advice

This function is **NOT** being developed further; it was replaced by the »smtSetupData« [▶▶▶] function and only remains a part of the IMSApi interface for reasons of downward compatibility!

27.2.4.1

Function description

This function processes serial number-related setup data and is used exclusively by the API-based "SMT Connector" service.



Advice

The SMT Connector includes various setup functions which are based on API functions. Depending on the process, this SMT function calls the »mlCreateNewMaterialBin« [▶▶▶], »mlGetMaterialBinData« [▶▶▶], »setupGetMaterialSetup« [▶▶▶], »setupGetSetupName« [▶▶▶], »setupStateChange« [▶▶▶], »setupUpdateMaterialSetup« [▶▶▶], »trAssignSerialNumberForProductOrWorkOrder« [▶▶▶], »trGetSerialNumberInfo« [▶▶▶] and »trUploadFailureAndResultData« [▶▶▶] API functions in the background. In the context of the SMT Connector, the »smtConsumption« [▶▶▶], »smtEventSetup« [▶▶▶] and »smtSetupData« [▶▶▶] functions must also be taken into account!

Licensing

In order for this API function to be used, the executing station and the associated site must be licensed as follows:

	TR license (active) OR TR license (passive)
AND	ML service (site license)

Function declaration

```
int smtSerialNumberSetup
in IMSApiSessionContextStruct sessionContext
    long sessionId
    long persId
    String locale
in String stationNumber
in int processLayer
in String lineName
in String setupName
in String productName
in String bomVersion
in String placementRecipe
in boolean assignSerialNumbers
in boolean bookSerialNumbers
in boolean activateSetup
inout SmtSerialNo[] serialNumbers
    String serialNumber
    int serialNumberPosition
    int serialNumberState
    int validationFlag
    int processFlag
inout SmtEvent[] events
    int eventId
    long eventDate
```

```

    int validationFlag
    int processFlag
    String position
    String feederBank
    String feederId
    int placementId
    int qtyRemain
    int qtyPlaced
    int qtyWasted
    String materialBinNumber
    boolean createMaterialBin
    String partNumber
    double quantity
    String supplier
    String lotNumber
    String dateCode
    String parentPartNumber
    inout SmtPlacement[] placements
    int placementId
    String compName
    String xPosition
    String yPosition
    int panelPosition
    in boolean ignoreContainerProblems
    in boolean createWorkOrder
    in boolean activateWorkOrder
    in String workOrderNumber
    in double cycleTime

```

27.2.4.2 Input and output parameters

Parameter (in, inout)

For the SMT functions »smtConsumption« [▶▶▶], »smtEventSetup« [▶▶▶] and »smtSerialNumberSetup« [▶▶▶], the same structures are used. Each individual function only needs a partial quantity of the input parameters specified in the declaration for processing. A detailed description of the parameters is not provided here.

27.2.4.3 Output values

Error codes

This function returns one of the "technical" error codes listed below as the result.



Advice

*The output text is not output automatically; it can, however, be queried by means of the »imsapiGetErrorText« [▶▶▶] function if necessary!
If the function returns an error code that is not documented here, please inform the iTAC employee responsible for the project!*

Error code	Category	Output text
0	OK	[EMPTY]
-3	Failure	Session is invalid
-10	Failure	Server error

-100	Failure	Station not found
-101	Failure	No TR license present for the station
-104	Failure	No user has logged into the station
-105	Failure	A required parameter is missing
-116	Failure	IMSApi key not found
-117	Failure	IMSApi key is invalid for this function
-502	Failure	Failure creating the material bin
-503	Failure	Container not found
-622	Failure	Setup belongs to a different work order
-2107	Failure	No ML license present for the site
-2110	Failure	Serial number position is invalid
-3000	Failure	SMT data is not processed
-3001	Failure	No validator present
-3002	Failure	Validation failed
-3003	Failure	Unknown failure
-3004	Failure	Container number contains invalid characters
-3005	Failure	Setup does not match the parts list
-3006	Failure	SMT -> Event type is invalid

27.3 Configuration parameter

27.3.1 Station parameter (IMSApi)

4010

4010	Threshold for the number of TR bookings that are passed as collective booking to the PM module (PDC threshold). 0 = TR bookings not transferred to the PM module 1 = Direct transfer of the TR bookings; each booking is passed individually (directly) to the PM module >1 = Cumulative transfer of the TR bookings; if e.g. the value 20 is specified, transfer to the PM module is only initiated when 20 TR bookings are available. In general, the higher the specified threshold is, the better the performance of the total system will be. Though note here, that there may be differences in the evaluations in the TR module and PM module for newer data. Advice: In addition to the threshold, a PM identification number must for technical reasons also be specified to enable data exchange between TR and PM service; this number is set in the standard by the system (for further information, see station parameter code no. 13200 [▶▶▶])! Also observe station parameter code nos. 4011 [▶▶▶] and 4012; they are used to control whether the shift timetable is to be taken into account when performing the collective booking and whether the quantities to be booked are checked for plausibility against the previous work step! The setting for the threshold influences, among other items, the behavior of the ERP-ServiceAdapter, as this only detects bookings passed to the PM module. This applies, in particular, for application types in which feedback is returned to the ERP system (e.g. order state, order lot or consumption quantities). In addition to the selected threshold, a special process can be configured in the system that transfers all open TR bookings in a certain time interval to the PM module across all stations independent of the number of bookings (see "TR/PM bookings" process). On each process call, this booking counter is automatically set to "0". Advice: This parameter is evaluated by the following API functions: ewAPI-Service: - reportRmQuantity IMSApi-Service: - trAssignSerialNumberMergeAndUploadState - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadPmQuantity - trUploadState - trUploadTestplanResult
------	--

4011

4011	Determines whether a collective booking (see station parameter code no. 4010 [▶▶▶]) is to be split on a shift change (Y) or not (N). Example: - code no. 4010 = 20 - 15 bookings on the change from early to late shift With shift-conformant booking (Y), the 15 bookings are assigned to the early shift (thus, the threshold is ignored in this case); the counter is then reset to 0 for the late shift. If the shift-conformant booking is inactive (N), the 15 bookings are transferred from the early shift to the late shift; there, counting continues up to 20, and all bookings are booked to the late shift. Advice: This parameter is evaluated by the following API functions (IMSApi service): - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe
-------------	--

4016

4016	Activates/deactivates (TRUE/FALSE) the acceptance of PDC bookings (PM service) for evaluation in the QM module. If the function is activated (TRUE), quantity-based bookings without serial number reference can also be displayed in the QM module. Advice: In the QM module, PDC bookings can be displayed only if a PM identification number has been specified for the data import; this number is set in the standard by the system (for further information, see station parameter code no. 13200 [▶▶▶]). This parameter is evaluated by the following API functions: - unregisterBatch (ewAPI service) - batchUnregisterBatch (IMSApi service)
-------------	--

7027

7027	Activates/deactivates the monitoring of the container state upon setup creation at the (manual) setup station (active: Y, J or TRUE, inactive: N or FALSE). If the option is active, only valid containers are accepted for setup creation. In connection with code no. 7028 [▶▶▶] and 7134 [▶▶▶], the monitoring of the container quantity and expiration date can also be activated. Advice: This parameter is evaluated by the following API functions and returns additional output values: ewAPI service: - getCurrentMaterialSetup - getMaterialSetupData IMSApi service: - setupChangeSetupForSerialNumber - setupCheck - setupCheckPreSetup - setupGetMaterialSetup
-------------	--

7028

7028	Activates/deactivates the monitoring of the container quantity upon setup creation at the (manual) setup station (active: Y, J or TRUE, inactive: N or FALSE). If the option is active, only containers for creating a setup are accepted whose container quantity is > 0. In connection with station parameters code no. 7027 [▶▶▶] and 7134 [▶▶▶], the monitoring of the container state and expiration date can also be activated. Advice: This parameter is evaluated by the following API functions and returns additional output values: ewAPI service: - getCurrentMaterialSetup - getMaterialSetupData IMSApi service: - setupChangeSetupForSerialNumber - setupCheck - setupCheckPreSetup - setupGetMaterialSetup
-------------	---

7029

7029	Activates/deactivates whether previously locked containers are stripped down automatically. If the parameter is set, a check is performed when a container is locked to establish whether this container is stored in a setup; the container is then removed from the setup automatically. TRUE = Locked containers are stripped down automatically FALSE = Locked containers are NOT stripped down Advice: This parameter is evaluated by the following API function (IMSApi service): - mlChangeMaterialBinData
-------------	--

7132

7132	Activates/deactivates checking of the setup (active: Y, J or TRUE; inactive: N or FALSE). If the option is active, the setup is only activated for the current station when the result of the setup check (bomCheck API function) does not cause any objections, i.e. all parts are properly setup.
-------------	---

7134

7134	Activates/deactivates the monitoring of the expiration date (MSL) upon setup creation at the (manual) setup station (active: Y, J or TRUE, inactive: N or FALSE). If the option is active, only valid containers are accepted for setup creation. In connection with station parameters code no. 7027 [▶▶▶] and 7028 [▶▶▶], the monitoring of the container state and quantity can also be activated. Advice: This parameter is evaluated by the following API functions and returns additional output values: ewAPI service: - bomCheck - bomCheckForWorkorder - updateMaterialSetup - setupActivation IMSApi service: - setupChangeSetupForSerialNumber - setupCheck - setupCheckPreSetup - setupStateChange - setupUpdateMaterialSetup
-------------	--

7200

7200	Determines the tester type for the ICT-Interface (failure note upload). Depending on the tester type used, an appropriate designator must be entered here. Testers that comply with the standard format (MKS testers) must lie within the designator range of 1000 to 1005; designator numbers for other tester types (e.g., Spea) are available on request. Advice: <i>The key number entered here must match the value in the parser configuration (see parameter ICT tester file descriptions in section »Parameter "Application server"«; further information can be found in the separate documentation »ICT-Interface«!</i> Advice: <i>This parameter is evaluated by the following API function:</i> - trUploadFailureAndResultData (IMSApi service)
-------------	--

7202

7202	Activates/deactivates the automatic generation of feedback for previous work steps that have not yet been booked and which are obligated to supply confirmation (active: Y, J or TRUE; inactive: N or FALSE). If the option is active, it is no longer possible to track down gaps in the message chain. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState
-------------	---

7203

7203	Activates/deactivates the option to check the chain of all work steps with obligatory confirmation or only the immediately preceding work step (active: Y, J or TRUE, inactive: N or FALSE). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSerialNumberState - checkSnrStateBySnrRef - verifyMerge - verifyMergeBySnrRef - verifyMergeProduct IMSApi service: - trCheckSerialNumberState - trMergeParts - trVerifyMerge - trVerifyMergeProduct <i>Only those work steps which are obligated to supply confirmation and which are required to be labeled as such are taken into account here.</i>
-------------	---

7204

7204	Activates/deactivates the possibility of using e.g. API functions to monitor or ignore the maximum values which are stored for individual work steps and which define the permitted number of tests (test retry counter). The maximum value is specified for the respective work step in the work plan (see the "No. of tests" column in the section »Parameter "Work steps"«). Y = Monitoring of the "test retry counter" is active; here the parameter value must be set at the tester. All bookings of a work step after a diagnosis station are counted (e.g. multiple bookings at the ICT); a booking at the parallel diagnosis station resets the "counter" to zero. D = Monitoring of the "test retry counter" is also active for the diagnosis station; this parameter value must be set at the diagnosis station. In addition to the bookings counted in mode Y, the bookings of diagnosis stations are also counted here; the counter is not reset. N = Monitoring of the "test retry counter" is NOT active. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSerialNumberState - checkSnrStateBySnrRef IMSApi service: - trCheckSerialNumberState
-------------	--

7206

7206	Activates/deactivates the possibility of reconciling both the quantity and the BOM version of the semi-finished good to be mounted against the current BOM before the actual product merging takes place (key word: "merge slave product into master product"). - Check active: Y, J or TRUE, - Check inactive: N or FALSE). If the check is active, the "merge" is rejected if all necessary semi-finished goods are already mounted and/or the BOM version of the semi-finished good to be mounted does not match the specification from the BOM position. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - mergeParts - mergeBatch - verifyMerge IMSApi service: - trMergeParts - batchMergeBatch - trVerifyMerge
-------------	--

7207

7207	This parameter controls the program behavior for the activation/deactivation of individual serial nos. in the event of product separation (unmerge). N = If a mounted serial no. is removed from a master serial no. (parent product), the removed serial no. is reactivated; the master serial no. also remains active. S = A serial number deactivated once by a product merge remains deactivated even in the event of an "unmerge"; the master serial no. remains active. M = A serial number deactivated by a product merge is reactivated in the event of an "Unmerge"; in this case the master serial no. is deactivated. Advice: <i>This parameter is evaluated by the <code>removeMergeParts</code> (ewAPI service) and <code>trRemoveMergeParts</code> (IMSApi service) API functions, which remove passed serial nos. from a parent product!</i>
-------------	--

7208

7208	Activates/deactivates the option of merging a serial number across multiple integration levels if system-wide uniqueness of the serial number is required (active: Y, J or TRUE, inactive: N or FALSE); for further information, see site parameter code no. 4610 [▶▶▶]. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - assignSerialNumberForProductOrWorkorder - assignSerialNumberMergeAndUploadState - assignSerialNumberToWorkOrder - mergeBatch - mergeParts IMSApi service: - batchMergeBatch - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState - trMergeParts - trSimulateMerge - trVerifyMerge
-------------	---

7209

7209	Activates/deactivates the option of incrementing the "loopCounter" (booking counter) only in the event of failure bookings (active: Y, J or TRUE; inactive: N or FALSE). It is possible to specify here whether all or only certain failure types are to be taken into account. The failure types to be taken into account must be explicitly selected (for further information, see section »Failure type setup«); pay special attention there to the setting for the "loopCounter active" parameter. If this parameter is inactive, all state bookings – with or without failure and independent of the failure type – increase the counter. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSnrStateBySnrRef - checkSerialNumberState - getSnrUploadInfo - queryTestData - verifyMergeBySnrRef - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trCheckSerialNumberState - trGetSerialNumberUploadInfo - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trVerifyMerge
-------------	--

7210

7210	Activates/deactivates the anomaly analysis (active: Y, J or TRUE; inactive: N or FALSE). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - mlUploadMaterialBinBooking - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7211

7211	Failure type code of the attributive failure which is to be used in the event of an anomaly. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7212

7212	Failure code with which the measurement data is stored in the system in the event of an anomaly. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7213

7213	Time (in seconds) after which the calculation is reinitialized. This enables simple restarting of production without parts being labeled as anomalous which are, in fact, not anomalous. On the other hand, this means that calculations of the PAA (Part Average Analysis) are to be considered outdated after this time and are no longer taken into account. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	--

7214

7214	Defines the comparison values (measurement data) on which another PAA is performed for an individual item already labeled as anomalous. 0 = For purposes of comparison, measurement data are used for the PAA which were current at the point in time at which the anomaly reoccurred for the items preceding the individual item to be analyzed. The PAA model used here is always "Band end"; station parameter code no. 7215 [▶▶▶] is not evaluated in this mode. 1 = The measurement data used for comparison in the PAA were current when the anomaly was first detected – depending on the PAA model – for the items before and possibly after the individual item to be analyzed. In this mode, station parameter code no. 7215 is also used for ascertaining the measurement data which are to be compared; parameter code no. 7215 defines whether the measurement data before, around or following the individual item to be analyzed are to be used (PAA model). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
------	--

7215

7215	Defines whether a different model should be used for ascertaining the comparison values (measurement data) during the renewed PAA for an individual item already labeled as anomalous. 0 = No other model is used; also applicable to the renewed PAA is the "Band end" default model. With this model, the measurement data of the individual item to be analyzed are only compared with the data of the previously produced individual items. 1 = If the parameter is activated, a different model can be used for the renewed PAA. Which model is used can be specified in the appropriate work step of the work plan (see "PAA model" field there). The "band center" and "band start" models are available: Band center -> The measurement data of the individual item to be analyzed is compared in equal parts with the previous data and the data of subsequently produced individual items. Band start -> The measurement data of the individual item to be analyzed is only compared with the data of subsequently produced individual items. Advice: <i>This parameter is only evaluated if station parameter code no. 7214 [▶▶▶] is set to 1. The reason for this is that the PAA models "Band start" and "Band center" can only function if the measurement data from the time of the first anomaly detection are used!</i> <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
------	--

7216

7216	Failure type code of the pseudo failure which is used during a renewed anomaly analysis (PAA), provided the previously detected anomaly is not confirmed. The booked "PAA" failure (see station parameter code no. 7211 [▶▶▶]) is then converted to a pseudo failure and disappears from the failure statistics. In addition, this failure type is used if the unit is in what is called an anomaly transition range, i.e., has a risk ratio (Rki) between > 0 and < 9 according to PAA 2.0. Advice: If a 0 is passed in the "Value" column, no failure conversion occurs. This parameter is evaluated by the following API functions: ewAPI service: - testPaa - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	--

7217

7217	Activates/deactivates the checking of the material quantity in the bin (TRUE = Active, FALSE = Inactive). Advice: This parameter is evaluated by the following API functions: - verifyMerge - verifyMergeProduct
-------------	--

7218

7218	Activates/deactivates the check to determine whether all components are mounted in the slave serial number (TRUE = Active, FALSE = Inactive). Advice: This parameter is evaluated by the following API functions: ewAPI service: - verifyMerge - verifyMergeBySnrRef - verifyMergeProduct IMSApi service: - trVerifyMerge - trVerifyMergeProduct
-------------	--

7219

7219	Activates/deactivates checking of the BOM versions of the installed units in the final product (TRUE = active, FALSE = inactive) Advice: This parameter is evaluated by the following API functions: ewAPI service: - verifyMerge - verifyMergeBySnrRef - verifyMergeProduct IMSApi service: - trVerifyMerge - trVerifyMergeProduct
-------------	---

7223

7223	Activates/deactivates (TRUE/FALSE) the secondary failure monitoring function; if corresponding secondary failure events are configured for the station (see section »Tab "Secondary failure setup"«), the affected station is automatically locked and a corresponding machine malfunction booked (see also station parameter code no. 7224 [▶▶▶]). Advice: This parameter is evaluated by the following API functions: ewAPI service: - mdcUploadStationCondition - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData - verifyMerge - verifyMergeBySnrRef - verifyMergeProduct IMSApi service: - mdcUploadStationCondition - trMergeParts - trSerialNumberPassing - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState - trVerifyMerge - trVerifyMergeProduct
------	--

7224

7224	Activates/deactivates (TRUE/FALSE) whether or not locked stations may continue to book upload functions on the basis of the secondary failure monitoring function (see station parameter code no. 7223 [▶▶▶]). If the value is set to TRUE, the functions listed below are interrupted with a failure; if the value is set to FALSE, only a warning is output. Advice: This parameter is evaluated by the following API functions: ewAPI service: - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trSerialNumberPassing - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState - trUploadTestplanResult
------	--

7225

7225	Defines an offset (in minutes) that can be used during the booking of machine messages (NOT machine conditions) to automatically pass the time at which validity ended. Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> The default value is interpreted if [-1] is passed for parameter <code>endDate</code> in the API function (see below). In this case, the value specified here is added to the time at which validity begins (parameter <code>startDate</code>) and is used as the end time. Advice: <i>If the parameter is not used ("not used" state) and [-1] is still passed for <code>endDate</code>, an offset of 5 minutes is used.</i> <i>This parameter is evaluated by the following API functions (IMSApi service) (but only if machine messages are booked):</i> - <code>mdcUploadStationCondition</code> - <code>mdcUploadDetectionPoint</code>
-------------	---

7226

7226	This is used to influence the output serial number state of the API functions listed below; the parameter is used for determining whether the last booking of a parallel diagnosis station (station parameter code no. 16000 [▶▶▶] = active) has been performed and if the work step must still be completed (active: Y, J or TRUE, inactive: N or FALSE). If the parameter is active, the booking of the diagnosis station causes only the counter for the "maximum value for the permitted number of tests (test-retry counter)" to be reset. Advice: <i>This parameter is evaluated by the following API function:</i> - <code>getSnrUploadInfo</code> (ewAPI service) - <code>trGetSerialNumberUploadInfo</code> (IMSApi service)
-------------	---

7228

7228	If more than one container is to be registered at a station at a time (key word "bin-based traceability"), the maximum number of containers can be defined here. If the parameter is not active, only one container may be registered at a time. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - <code>batchRegisterBatch</code>
-------------	--

7250

7250	Activates/deactivates the process capability analysis (1 = active, 0 = inactive). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>testSpa</code> - <code>uploadResultDataAndRecipe</code> - <code>uploadStateAndResultData</code> IMSApi service: - <code>trTestSpa</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadResultDataAndRecipe</code>
-------------	--

7251

7251	Time after which the calculation of the SPA (Statistical Process Analysis) is reinitialized. This enables simple restarting of production without parts being labeled as unfit for processing which are, in fact, not. On the other hand, this means that calculations are to be considered outdated after this time and are no longer taken into account. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	--

7252

7252	Defines whether a rework booking is generated for the individual item upon violation of the Cp default value during the course of the SPA (1 = Yes, 0 = No). To interpret the setting, the parameter 7250 [▶▶▶] must be active as well. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	--

7253

7253	Defines whether a rework booking is generated for the individual item upon violation of the Cpk default value during the course of the SPA (1 = Yes, 0 = No). To interpret the setting, the parameter 7250 [▶▶▶] must be active as well. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7254

7254	Failure type code of the attributive failure which is used upon a violation of the Cp default value. To interpret the setting, the parameter 7250 [▶▶▶] must be active as well. Advice: The Cp default value is stored directly in the recipe; detailed information on this topic can be found in section »Recipe editor«. This parameter is evaluated by the following API functions: ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe Background of "faulty measurement record": The violation of the Cp default value is ultimately triggered by the booking of a measurement record not in the specification. The Cp failure-triggering measurement record is identified in the measurement data display of the unit history (TR module). To do so, the internal identification number (ObjectID) of the failure type code specified above is automatically entered in the "Failure code" table column. In the event of a simultaneous "anomaly detection" (PAA), the entry in the "Failure code" column is overridden by the default value in accordance with parameter 7212 [▶▶▶].
-------------	---

7255

7255	Failure type code of the pseudo failure which is used during a renewed process capability analysis (SPA), provided the previously detected Cp violation is not confirmed. The booked "Cp" failure (see station parameter code no. 7254 [▶▶▶]) is then converted to a pseudo failure and disappears from the failure statistics. Advice: If a 0 is passed in the "Value" column, no failure conversion occurs. This parameter is evaluated by the following API functions: ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7256

7256	Failure type code of the attributive failure which is used upon a violation of the Cpk default value. Advice: <i>The Cpk default value is stored directly in the recipe; detailed information on this topic can be found in section »Recipe editor«. This parameter is evaluated by the following API functions:</i> ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe Background of "faulty measurement record": Violation of the Cpk default value is ultimately triggered by the booking of a measurement record not in the specification. The Cpk failure-triggering measurement record is identified in the measurement data display of the unit history (TR module). To do so, the internal identification number (ObjectID) of the failure type code specified above is automatically entered in the "Failure code" table column. In the event of a simultaneous Cp violation or anomaly detection (PAA), the entry in the "Failure code" column is overridden by the respective default value in accordance with parameter 7254 [▶▶▶] (Cp) or 7212 [▶▶▶] (PAA).
-------------	---

7257

7257	Failure type code of the pseudo failure which is used during a renewed process capability analysis (SPA), provided the previously detected Cpk violation is not confirmed. The booked "Cpk" failure (see station parameter code no. 7256 [▶▶▶]) is then converted to a pseudo failure and disappears from the failure statistics. Advice: <i>If a 0 is passed in the "Value" column, no failure conversion occurs. This parameter is evaluated by the following API functions:</i> ewAPI service: - testSpa - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7258

7258	Activates/deactivates (TRUE/FALSE) whether, in the context of the PAA (Part Average Analysis), special warnings are output by the API functions which book measurement data. If the value is set to TRUE, the PAA test results are output directly when the measurement data is booked (e.g. error code 230 "Not enough values for the PAA" or 233 "Anomaly detected (PAA)"). Advice: <i>This parameter is only evaluated when station parameter code no. 7210 [▶▶▶] is active; the parameter is evaluated by the following API functions:</i> ewAPI service: - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trSerialNumberPassing - trUploadFailureAndResultData - trUploadResultDataAndRecipe
-------------	---

7260

7260	This parameter determines whether and to what extent the order networks created in the iTAC system are to be taken into consideration when products/units are mounted. The following function modes are available here: 0 = Order networks are NOT checked during the mounting process 1 = Order networks are ONLY checked if order networks have been found for the product structure to be mounted 2 = Order networks are ALWAYS expected and checked accordingly; if no order networks are defined for the product structure to be mounted, the check will have a negative result. Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trMergeParts - trSimulateMerge - trVerifyMerge <i>This program behavior is defined centrally as the site default (for further information, see site parameter code no. 7260 [▶▶▶]); ACTIVATE this parameter ONLY for the stations whose behavior should deviate from the site default!</i>
-------------	--

7320

7320	This parameter controls the program behavior with regard to state bookings and failure bookings for the units of a multiple printed panel: P (position based) = The serial number for the booking is determined from the passed position; if no serial number is known for the passed position, the correct serial number is determined using the reference serial number and the position. S (serial-number related) = The correct position for the booking is determined from the passed serial number; if the position is not correct for the serial number, this is corrected. - (both) = For a successful booking, both the position as well as the serial number must be passed correctly. Advice: <i>If the state and failure bookings for the multiple panels are to be performed with reference to position (P), it must also be ensured that the panel flag for the part is set at the point in time of order generation (for further information, see the "Panel" field in the part master)!</i> <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState
-------------	--

7400

7400	Activates/deactivates (TRUE/FALSE/KONSI) the option of automatically booking the processed components for the processing of an individual item by quantity from the set up containers. Booking then occurs with the internal movement type "WA_TR" (Code "89"). In KONS mode, the material consumptions are collected and cyclically passed to the material history and, if applicable, to the ERP interface. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadState
-------------	---

7401

7401	In the event of multiple bookings of a station to a serial number, this activates/deactivates whether the consumed materials are booked on all bookings (TRUE) or whether this only occurs for the first booking (FALSE). In this context, also take into account station parameter code no. 7402 [▶▶▶]. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadState
-------------	--

7402

7402	Used to configure under which conditions material consumptions are reported back to a connected ERP system using the »TRAN.traninvmove« transfer table. This previously applied to feedback either with the first booking or with every booking; finer, state-dependent configuration is now possible. The following options are available: 0 = Material consumption booking according to station parameter code no. 7401 [▶▶▶] 1 = Material consumption booking with the first PASS booking; the material consumption is booked with the first PASS booking for the serial number. 2 = Material consumption with every PASS booking; the material consumption for the serial number is booked with every PASS booking. 3 = Material consumption with the first SCRAP or PASS booking; the material consumption for the serial number is booked with the first PASS or SCRAP booking. 4 = Material consumption with every SCRAP or PASS booking; the material consumptions for the serial number are booked with every PASS or SCRAP booking. Advice: <i>This parameter must always be seen in the context of station parameter code no. "7401". If the value 0 is set here, the specifications selected there continue to apply. If a value not equal to 0 is selected, the parameter "7401" is overridden!</i> <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadState
------	--

8100

8100	Determines the program behavior for whether and under what conditions the automatic creation of recipes is enabled with the "Upload" functions (API or interface); this can be used, for example, to prevent the repeated creation of recipes, provided an older, still suitable recipe can be used. The following options are available: 0 = The automatic creation of a recipe is not permitted in this case! The new recipe data, which are passed by the station, are first compared with the active recipe. If differences are found, the "Upload" function is interrupted with an error message; an error message is also displayed if no recipe is active. 1 = The automatic creation of a recipe is permitted in this case! Only the active recipe is compared with the new recipe data, which are passed by the station. If a match is found, no new recipe is created; instead, the active recipe is used. In the event of differences, a new recipe is automatically created and activated. 10 = The automatic creation of a recipe is permitted in this case! The last ten recipes are compared with the new recipe data. If a match is found, no new recipe is created; instead the first match is used and activated. If all ten recipes are determined to be different, a new recipe is automatically created and activated. Advice: <i>In combination with station parameter code no. 8110 [▶▶▶], 0 must be passed here as the value! This parameter is evaluated by the following API functions:</i> <i>ewAPI service:</i> - uploadResultDataAndRecipe <i>IMSApi service:</i> - trUploadResultDataAndRecipe - trUploadStationResult
------	---

8110

8110	When passing recipes, defines whether the parameters (measurement steps) of the recipe currently active at the station must be passed in their entirety, or if it is sufficient that the passed parameters are defined in the active recipe. 0 or FALSE = The passed parameters only need to be defined in the active recipe; the check for completeness is not performed in this case. 1 or TRUE = The passed parameters must be defined in the active recipe and must be complete; i.e. all parameters of the active recipe must be passed. For both variants it holds that when passing undefined parameters to the active recipe, the "upload" function is interrupted with an error message; if no recipe is active, an error message also occurs. Advice: <i>This parameter is only evaluated if station parameter code no. 8100 [▶▶▶] is passed with the value 0, i.e. the function for the automatic creation of recipes is deactivated. As only the check is controlled here, this parameter has no influence on the uploadStationResult API function (ewAPI service)!</i> <i>This parameter is evaluated by the following API functions:</i> IMSApi service: - trUploadResultDataAndRecipe - trUploadStationResult
-------------	---

8200

8200	Activates/deactivates (Y/N) the check to determine whether an active recipe must exist (Y) in the system. In combination with station parameter code no. 8100 [▶▶▶]: when the check is activated, automatic creation of recipes is only possible when a valid recipe already exists. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - uploadResultDataAndRecipe - uploadStateAndResultData IMSApi service: - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadStationResult
-------------	--

8300

8300	Activates/deactivates whether a user check should be performed prior (TRUE) or not (FALSE) to the execution of the API function. If the check is active, the API functions can only be executed if a valid user was logged into the system with API function mdcRegisterUserAtLine (IMSApi service) or registerUserAtLine (ewAPI service).
-------------	---

8310

8310	Determines for the API functions listed in the advice below whether the <code>processLayer</code> variable is used to pass the layer L (= Layer), the work step number passed by the ERP system A (= AVO number) or the internal work step number W (= Workstep). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - createRecipe - getCompleteMaterialSetup - uploadFailureslip - uploadMaterialBinBookingsExt - uploadResultDataAndRecipe - uploadState - uploadStateAndFailData - uploadStateAndFailureData - uploadStateAndResultData IMSApi service: - apsGetWorkorderDetails - apsGetWorkorderDetailsOptGroup - equGetRequiredEquipmentData - equGetRequiredEquipmentDetail - mdaCreateRecipe - mdataChangeWorkOrderWorkStepState - mdataSetProductionCycleTime - mlUploadMaterialBinBooking - pmGetProductQuantity - setupCheck - setupGetMaterialSetup - setupUpdateMaterialSetup - trCheckSerialNumberState - trCheckUserSkill - trCheckUserSkills - trGetMultiStationSettings - trGetSerialNumberForWorkorderAndWorkstep - trGetSerialNumberHistoryData - trGetStationSetting - trGetTestplanData - trMergeParts - trRemoveMergeParts - trTestPaa - trTestSpa - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadState
------	---

8400

8400	Activates/deactivates the usage counter for work-step-related equipment; if the parameter is set (TRUE), the number of usages of the equipment set up at the station is increased by "1" with each state booking (all <code>upload</code> functions for serial numbers); detailed information on this topic can be found in the master data maintenance in the section »Equipment«. If equipment is assigned cavities, the usage counts can be incremented for them as well; this requires station parameter 8402 [▶▶▶] to be activated. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>uploadResultDataAndRecipe</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> IMSApi service: - <code>trAddExternalSerialNumber</code> - <code>trAssignSerialNumberMergeAndUploadState</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadResultDataAndRecipe</code> - <code>trUploadState</code> - <code>trUploadTestplanResult</code> - <code>trUploadActivity</code> <i>The usage counter for asset-related equipment is controlled using station parameter code no. 8401 [▶▶▶]!</i> If the usage count is exceeded, maintenance orders can be created automatically (for further information, see site parameter code no. 13060 [▶▶▶]!)
-------------	--

8401

8401	Activates/deactivates the usage counter for asset-related equipment; if the parameter is set (TRUE), the number of usages of the equipment set up at the station as well as that of the equipment set up at the subordinate resources are increased by "1" with each state booking (all <code>upload</code> functions for serial numbers); detailed information on this topic can be found in the master data maintenance in the section »Equipment«. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>uploadResultDataAndRecipe</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> IMSApi service: - <code>trAddExternalSerialNumber</code> - <code>trAssignSerialNumberMergeAndUploadState</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadResultDataAndRecipe</code> - <code>trUploadState</code> - <code>trUploadTestplanResult</code> - <code>trUploadActivity</code> <i>The usage counter for work-step-related equipment is controlled using station parameter code no. 8400 [▶▶▶].</i> If the usage count is exceeded, maintenance orders can be created automatically (for further information, see site parameter code no. 13060 [▶▶▶]!)
-------------	---

8402

8402	Activates/deactivates the usage counter for cavities; if the parameter is set (TRUE), the number of usages for cavities is increased by "1" with each state booking (all <code>upload</code> functions for serial numbers). This only applies in cases where the cavity is actively assigned to equipment AND the parent equipment is set up at the station. This parameter also needs to be active for manual capture of usage counts for cavities (see IMSApi function <code>equReportQuantity</code>). Further details on this topic can be found in the master data maintenance in section »Cavities«. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>uploadResultDataAndRecipe</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> IMSApi service: - <code>equReportQuantity</code> - <code>trAddExternalSerialNumber</code> - <code>trAssignSerialNumberMergeAndUploadState</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadResultDataAndRecipe</code> - <code>trUploadState</code> - <code>trUploadTestplanResult</code> - <code>trUploadActivity</code> <i>The usage counter for work-step-related equipment is controlled using station parameter code no. 8400 [▶▶▶].</i> If the usage count is exceeded, maintenance orders can be created automatically (for further information, see site parameter code no. 13060 [▶▶▶])!
-------------	--

8410

8410	Used to determine whether one particular item of equipment is allowed to be set up on a station multiple times. If the parameter is set (TRUE), the equipment required for a work step can be set up multiple times, provided that a unique setup position is specified (key <code>SETUP_POSITION</code>). If the parameter is not set (FALSE), the IMSApi function given below terminates the setup process with a warning. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - <code>equUpdateEquipmentData</code> The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 13050 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
-------------	--

8500

8500	Determines whether a completed and closed bin (key word "bin-based traceability") is automatically passed to the transfer process (1 = yes, 0 = no). Advice: <i>This parameter influences the ERP transfer process for transferring so-called batches (for further information, see section »Process "Bins with unserialized units"«)!</i> <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>unregisterBatch</code> IMSApi service: - <code>batchUnregisterBatch</code>
-------------	---

8510

8510	Used to configure under which conditions serial numbers are reported back to a connected ERP system using the »TRAN.transnrinit« transfer table. This previously applied to feedback either with the first booking or with every booking; finer, state-dependent configuration is now possible. The following options are available: 0 = Feedback with every serial number booking 1 = Feedback with the first PASS booking 2 = Feedback with every PASS booking 3 = Feedback with the first SCRAP or PASS booking 4 = Feedback with every SCRAP or PASS booking Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trUploadFailureAndResultData - trUploadResultDataAndRecipe - trUploadState
-------------	---

13000

13000	Determines the function mode for querying failure information and related failure notes (ICT error) stored in the system; the following values are possible: OBJECT_ID = related to the part number OBJECT_ID&STK_VERS = related to the BOM version LP_ID = related to the printed circuit board number Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - trGetCapturedRepairHistoryData
--------------	---

13010

13010	Activates/deactivates whether the "loopCounter" of the previous work step (Y) or of the current, i.e. own, work step (N) of the calling station is output via the API. Independent of this setting, the state of the previous work is always the state that is output for checking. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSnrStateBySnrRef - verifyMergeBySnrRef IMSApi service: - trCheckSerialNumberState - trVerifyMerge
--------------	---

13011

13011	Activates/deactivates whether only the state of the previous work step (N) or only the state of the current work step (Y), i.e. that of the calling station, is checked. This parameter acts only on the first work step of a work plan. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSerialNumberState - checkSnrStateBySnrRef IMSApi service: - trCheckSerialNumberState
--------------	---

13100

13100	Determines (TRUE/FALSE) whether the current station returns an "OK" as the result for the automatic "Setup check" (see station parameter code no.7132 [▶▶▶]) without the check actually being performed. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - setupActivation IMSApi service: - setupStateChange
--------------	--

13110

13110	Determines (TRUE/FALSE) whether the changes in the setup become active immediately upon upload of the setup data (automatic activation of the changed setup). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - updateMaterialSetup - updateSetupData IMSApi service: - setupUpdateMaterialSetup
--------------	--

13120

13120	Determines whether the setup check and setup activation based on a station number also automatically address all other stations of the work step (TRUE/FALSE); e.g., if an SMD line is setting up on several different stations. Advice: <i>On TRUE, all stations of the work step on the respective line are addressed with one station number!</i> <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - bomCheckForWorkorder - setupActivation IMSApi service: - setupCheck - setupStateChange
--------------	---

13125

13125	Determines whether the expiry time for the affected container is started with the setup process/setup activation by means of this station. The following settings are possible: false = expiry time does NOT automatically start setup = expiry time starts with the setup process activation = expiry time starts with setup activation Background: As a rule, the expiry time starts if an MSL package (so-called "dry pack") is opened or an MSL part is removed unprotected from a re-drying oven. Advice: <i>This parameter is evaluated by the following API functions:</i> IMSApi service: - mslStartObjectExpiration - smtCheckMaterialBin
--------------	--

13130

13130	Determines whether a container change can only occur for a set up position (slot) if the new container contains the same components (identical part number) as the components on the currently set up container (TRUE/FALSE). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - updateMaterialSetup - updateSetupData - updateSetupDataBySnr IMSApi service: - setupUpdateMaterialSetup
--------------	--

13140

13140	Determines the booking (sequence number) for which the set up components of a serial number are to be displayed for individual API functions as well as evaluation dialogs in the TR module (see sections »Material history for units« and »Material trace«). Distinctions are made between the following three modes: F = Display of the setup data only for the first booking of the station at the work step. A = Display of the setup data for all bookings of the station at the work step. L = Display of the setup data only for the last booking of the station at the work step. Advice: <i>This parameter is evaluated by the following API functions:</i> - getSetupDataBySerialNumber (ewAPI service) - setupGetSetupDataBySerialNumber (IMSApi service) - setupGetSetupDataBySerialNumberRef (IMSApi service)
--------------	--

13145

13145	In the context of "serial-number related setup", specifies whether, in addition to the missing components (FALSE), the setup check is to also evaluate too many set-up components as a failure (TRUE). Advice: <i>This parameter is used by the bomCheck and bomCheckForWor-korder API functions (ewAPI service) and is evaluated if "2" is passed there for the checkActivPreparation parameter. In the IMSApi service, this affects the setupCheck function and there the checkMode parameter in mode "2"!</i>
--------------	---

13150

13150	In the context of "serial-number related setup", specifies whether the setup check must be performed individually for all serial numbers of a multiple panel (TRUE), or whether the check of the entire multiple panel is performed on the basis of the passed serial number (FALSE). Advice: <i>This parameter is used by the bomCheck and bomCheckForWor-korder API functions (ewAPI service) and is evaluated if "2" is passed there for the checkActivPreparation parameter! In the IMSApi service, this affects the setupCheck function and there the checkMode parameter in mode "2".</i> <i>If all serial numbers are to be checked individually, longer runtimes may result for the API function for large numbers of panels in combination with complex bills of material!</i>
--------------	---

13200

13200	Entry of the station reference – here this is the internal PM identification number – which is to be used for accepting the TR booking numbers and displaying them in the PM module. The PM identification number is displayed in the "KapID (PM)" field of the station overview. Advice: <i>This parameter is managed by default by the system and may be changed ONLY in consultation with the iTAC employee responsible for the project!</i> Background: For technical reasons, the PM identification number of the station must be specified for data exchange between TR and PM service so that both product-related and process-related bookings can be performed. Advice: <i>In this context, station parameter code nos. 4010 [▶▶▶] (PDC threshold) and 4016 [▶▶▶] (display in QM module) are also to be taken into consideration!</i> <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - pmGetFailureData - pmGetProductQuantity - pmUploadQuantityAndFailureData - trUploadPmQuantity - mdataGetCalendarData
--------------	---

13300

13300	Activates/deactivates the general check of the set work orders against the current work order of the serial number. The parameter applies only to production orders; configuration parameter code no. 13310 [▶▶▶] must be used for all other work order types. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - checkSerialNumberState - checkSnrStateBySnrRef IMSApi service: - setupCheck - trCheckSerialNumberState
--------------	---

13301

13301	Activates/deactivates checking of the order state (stopped or finished) prior to the assignment of serial numbers (active: Y, J or TRUE; inactive: N or FALSE). This can be used to ensure that serial numbers are only assigned to a work order if the work order is valid and is not yet completely filled with respect to quantity. The parameter applies only to production orders; configuration parameter code no. 13311 [▶▶▶] must be used for all other work order types. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - assignBatchNoToWorkorder - assignSerialNumberForProductOrWorkorder - assignSerialNumberMergeAndUploadState - assignSerialNumberToWorkOrder IMSApi service: - batchAssignBatchNumberToWorkOrder - trAddSerialNumberToPool - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState
--------------	---

13302

13302	Activates/deactivates the automatic work order termination once the order quantity is reached. In addition to the value that is set here, automated work order termination also requires that the used work plan alternative be marked with "Close automatically". The parameter applies only to production orders; configuration parameter code no. 13312 [▶▶▶] must be used for all other work order types. 0 or FALSE = The system does not set the order state to "finished". 1 or TRUE = The system sets the order state to "finished" when the order quantity has been reached at all work steps with obligatory confirmation. 2 = The system sets the order state to "finished" when the order quantity at the last work step with obligatory confirmation has been reached. 3 = The system sets the order state to "finished" when the order quantity has been reached at the first work step with obligatory confirmation. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - assignBatchNoToWorkorder - assignSerialNumberForProductOrWorkorder - assignSerialNumberMergeAndUploadState - assignSerialNumberToWorkOrder IMSApi service: - batchAssignBatchNumberToWorkOrder - trAddExternalSerialNumber - trAddSerialNumberToPool - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState - trUploadState
--------------	---

13310

13310	Activates/deactivates the general check of the set work orders against the current work order of the serial number. The parameter does NOT apply to production orders; configuration parameter code no. 13300 [▶▶▶] must be used here. Advice: This parameter is evaluated by the following API functions (IMSApi service): - setupCheck - trCheckSerialNumberState
--------------	--

13311

13311	Activates/deactivates checking of the order state (stopped or finished) prior to the assignment of serial numbers (active: Y, J or TRUE; inactive: N or FALSE). This can be used to ensure that serial numbers are only assigned to a work order if the work order is valid and is not yet completely filled with respect to quantity. The parameter does NOT apply to production orders; configuration parameter code no. 13301 [▶▶▶] must be used here. Advice: This parameter is evaluated by the following API functions (IMSApi service): - batchAssignBatchNumberToWorkOrder - trAddSerialNumberToPool - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState
--------------	--

13312

13312	Activates/deactivates the automatic work order termination once the order quantity is reached. In addition to the value that is set here, automated work order termination also requires that the used work plan alternative be marked with "Close automatically". The parameter does NOT apply to production orders; configuration parameter code no. 13302 [▶▶▶] must be used here. 0 or FALSE = The system does not set the order state to "finished". 1 or TRUE = The system sets the order state to "finished" when the order quantity has been reached at all work steps with obligatory confirmation. 2 = The system sets the order state to "finished" when the order quantity at the last work step with obligatory confirmation has been reached. 3 = The system sets the order state to "finished" when the order quantity has been reached at the first work step with obligatory confirmation. Advice: This parameter is evaluated by the following API functions (IMSApi service): - batchAssignBatchNumberToWorkOrder - trAddExternalSerialNumber - trAddSerialNumberToPool - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState - trUploadState
--------------	--

13400

13400	Activates/deactivates whether scrapped serial numbers of a multiple panel can be mounted in the next integration level (active: Y, J or TRUE, inactive: N or FALSE). Advice: This parameter is only evaluated if site parameter code no. 18200 [▶▶▶] or station parameter 18200 [▶▶▶] has been set to the value checkAll; in this context, station parameter code no. 25002 [▶▶▶] as well as server parameter code no. 2032 [▶▶▶] are also to be observed! Advice: This parameter is evaluated by the following API function (IMSApi service): - trAssignSerialNumberMergeAndUploadState
--------------	---

13410

13410	Activates/deactivates whether a state booking can be triggered with this station although the last work step booking was SCRAP (active: Y, J or TRUE, inactive: N or FALSE). This can be used to subsequently rebook a scrapped serial number with PASS. Advice: <i>This parameter is only evaluated if site parameter code no. 18200 [▶▶▶] or station parameter 18200 [▶▶▶] has been set to the value checkAll!</i> Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState
--------------	---

13420

13420	In connection with the "Upload settings" station settings (the "Ascertain state using the failure data" mode there), this parameter defines whether the state can be changed from FAIL to GOOD (TRUE) or not (FALSE). If the function is activated, if the state changes from FAIL to PASS for the API functions listed below, a corresponding warning is output (error code = 224) Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trSerialNumberPassing - trUploadActivity - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState - trUploadTestplanResult
--------------	--

13163

13163	In combination with the SMT interface, this parameter is used to coordinate execution of the setup check (BOMCheck) with the production process. It is therefore possible to determine whether the setup check is executed after a delay (see below under "Background"). 0 = Setup check takes place without a delay; here the parameters 13161 and 13162 are not evaluated. 1 = Setup check takes place with a delay; here it is assumed that the machines to be checked are deposited in one work step (parallel stations) and that only the current work step therefore has to be checked. 2 = The setup check takes place with a delay; here it is assumed that the machines to be checked are deposited in different work steps and that the previous work step therefore has to be checked. Advice: <i>The station parameter code nos. 13161 (maximum waiting period) and 13162 (test interval) must be noted in conjunction with this parameter! This parameter is evaluated by the following API functions:</i> ewAPI service: - smtSerialNumberSetup IMSApi service: - smtSerialNumberSetup Background: Sometimes, the first machines in a line place a large number of smaller components and the last machine places just a few large components. This can lead to very different API runtimes; this can have a noticeable negative effect especially in the case of "work order change". If at the end of processing the last machine performs a setup check (BomCheck) as part of the work order change but the penultimate machine is not yet ready, the complete setup is missing there, which leads to a failure.
--------------	---

13600

13600	Determines whether a presence check is performed during the IMSApi-based query of attributes (1) or not (0). If the check is enabled, all of the queried attributes must be assigned to the specified data object. The API interface returns a failure for attributes that are not found. If the check is disabled, only the attribute data that is actually assigned is output. Example: The values for the attributes "Color" and "Weight" are to be determined for certain containers; the attribute "Weight" has never been used. Both attributes are now queried by means of the IMSApi function. If the check is disabled (0), the function only returns an array with the determined color values. If the check is active (1), an array for the weight information that has not been found is output in addition to the array with the color values; this is, however, output with a technical failure. Advice: This parameter is evaluated by the following API function: IMSApi service: - attribGetAttributeValues
--------------	---

15001

15001	Assignment of a storage location to a station by means of a bar code (the corresponding bar code must be entered as a value; 0 = no assignment). Background of "automatic storage rebooking during setup": It is possible to automatically restore containers for the setup process. Configured accordingly, during setup on this station the container is restored automatically to the storage place named here. During stripping down, the default storage place of the container material according to the part master is used. Both restoring variants can be assigned a desired material movement type. Advice: In this context, also note station parameter code no. 15002 [▶▶▶], 15003 [▶▶▶] and 15004 [▶▶▶]!
--------------	--

15002

15002	Used to activate the "automatic storage rebooking during setup" function; for background information, see station parameter code no. 15001 [▶▶▶]. The following settings are possible: 0 = Automatic storage rebooking OFF 1 = Automatic storage rebooking ON: for setup only 2 = Automatic storage rebooking ON: for setup and stripping down Advice: In this context, also note station parameter code no. 15003 [▶▶▶] and 15004 [▶▶▶]! This parameter is evaluated by the following API functions (IMSApi service): - setupAttachPreSetup - setupChangeSetupForSerialNumber - setupStateChange - setupUpdateMaterialSetup - smtSetupData
--------------	---

15003

15003	Used for the "automatic storage rebooking during setup" function to determine the material movement type for the setup process at the station; for more background information, see station parameter code no. 15001 [▶▶▶]. The material movement type to be used must exist in the system; the internal designator of the movement type must be entered here as the parameter value (corresponds to the "Code" column in the movement type setup). Advice: In this context, also note station parameter code no. 15002 [▶▶▶] and 15004 [▶▶▶]; this parameter is only evaluated if the value 1 or 2 is set for "15002"!
--------------	--

15004

15004	Used for the "Automatic storage rebooking during setup" function to determine the material movement type for the stripping down process at the station; for background information, see station parameter code no. 15001 [▶▶▶]. The material movement type to be used must exist in the system; the internal designator of the movement type must be entered here as the parameter value (corresponds to the "Code" column in the movement type setup). Advice: <i>In this context, also note station parameter code no. 15002 [▶▶▶] and 15003 [▶▶▶]; this parameter is only evaluated if the value 2 is set for "15002"!</i>
--------------	--

15200

15200	If, for the station, the "Convert measurement data" option in the "Upload settings" tab is set to enable the generation of attributive failures, this option can be used to specify whether new failure types are also created for these attributive failures or not (active: Y, J or TRUE, inactive: N or FALSE). Advice: <i>This function is of central significance where measurement data is passed to the iTAC system by means of an interface: FKT, ICT and API. Only when this parameter is activated are the settings for parameter "15201" [▶▶▶] evaluated! This parameter is evaluated by e.g. the following API functions:</i> ewAPI service: - uploadStateAndResultData - uploadResultDataAndRecipe IMSApi service: - trUploadResultDataAndRecipe
--------------	--

15201

15201	This parameter is used to specify which level – i.e. violation of which limit value – is to be applied in the creation of failure types: 0 = Failure types are only created in the event of deviations for min. and max. 1 = Failure types are created in the event of deviation for min. and max. as well as LIL and UIL (lower and upper intervention limits) 2 = Failure types are created in the event of deviation for min., max. and LIL, UIL, as well as LSL and USL (lower and upper scrap limits) Advice: <i>This parameter is only evaluated if parameter "15200" [▶▶▶] is set! In addition, note that this parameter has an effect on the output of the queryRecipeData (ewAPI service) and trUploadResultDataAndRecipe (IMSApi service) API function. Depending on the level selected, the values for Min. and Max., and if necessary also for LIL and UIL as well as LSL and USL are output there!</i>
--------------	--

15210

15210	If a new recipe is created, this can be used to specify which values are used there for the intervention and scrap limits – and in mode R for other configurable recipe parameters as well: R = Values are taken over from the previous recipe; this applies not only for the intervention and scrap limits, but also for all other configurable recipe parameters (e.g. PAA specifications such as "weighting", "noise figure" etc.). D = The default values are used (intervention limit = 75%; scrap limit = 125%) K = The current values of the station are used "15211" and "15212") Advice: <i>This parameter is evaluated by the uploadResultDataAndRecipe (ewAPI service) and trUploadResultDataAndRecipe (IMSApi service) API function!</i>
--------------	--

15220

15220	This parameter specifies when and how, in the context of API-based measurement data booking, the recipe data field "Type" (Config, Result, Config-Result, Dynamic) affects an "automatic" recipe update [0; 1; 2]: 0 = Type-based recipe update is NOT active; the content of the data field "Type" is not taken into consideration. 1 = Type-based recipe update is active (only for type <code>Dynamic</code>); a conformity check takes place, whereby an update is only performed if the new measurement record has the type <code>Dynamic</code>. 2 = Type-based recipe update is active (no restriction); a conformity check takes place and the recipe is updated whenever a deviation is found. Advice: In this context, also note station parameter code no. 8100 [▶▶▶]; this is used to determine whether the automatic recipe update is actually permitted; a value unequal to [0] must be set here, otherwise the recipe data field "Type" will not be taken into consideration during the check! This parameter is evaluated by the following API functions: ewAPI service: - uploadResultDataAndRecipe IMSApi service: - trUploadResultDataAndRecipe - trUploadStationResult
-------	---

15250

15250	For measured data import (FKT, ICT) via API, state information – the so-called failure code – is passed for each measurement step. At this point, one or several individual failure codes can be specified which are automatically interpreted as pseudo failures in the system. If several failure codes are to be interpreted as pseudo failures, these must be separated by semicolons: e.g.: "2;4;8". Attention: Failure codes "0" and "1" must not be used because they are preset to "Pass" and "Fail" in the system! This parameter has a direct influence on the "Conversion of measurement data to attributive failures" function (see section »Parameter "Upload settings"«, "Convert measurement data" option), because the failure types created by the system on this basis contain the "pseudo failure" designator. For the assessment of the measurement data booked with these failure codes, this means that they are booked as a "PASS" measurement step. Advice: In the API service, the respective failure code of the measurement step is passed using the <code>failcode</code> variable; cf. the <code>uploadResultDataAndRecipe</code>, <code>uploadStateAndResultData</code> and <code>uploadStationResult</code> API functions. In the IMSApi service, the corresponding failure code of the measurement step is passed using the <code>MEASURE_FAIL_CODE</code> variable; cf. the <code>trUploadFailureAndResultData</code>, <code>trUploadResultDataAndRecipe</code> and <code>trUploadStationResult</code> API functions. Further details on this topic can be found in the concept description "Failure codes for measurement data"!
-------	--

15260

15260	For measured data import (FKT, ICT) via API, state information – the so-called failure code – is passed for each measurement step. At this point, one or more individual failure codes that are to remain unevaluated by the system can be specified. Attention: Failure codes "0" and "1" must not be used because they are pre-set to "Pass" and "Fail" in the system! The failure codes listed here are not taken into account by the "Conversion of measurement data to attributive failures" function (see section »Parameter "Upload settings"«, "Convert measurement data" option). In the context of measurement data assessment, they are evaluated as "Canceled test" and are, thus, considered neither a passed nor failed measurement step. If multiple failure codes are to be excluded from the evaluation, these must be separated by semicolons: e.g. "3;5;9". Advice: In the API service, the respective failure code of the measurement step is passed using the <code>failcode</code> variable; cf. the <code>uploadResultDataAndRecipe</code>, <code>uploadStateAndResultData</code> and <code>uploadStationResult</code> API functions. In the IMSApi service, the corresponding failure code of the measurement step is passed using the <code>MEASURE_FAIL_CODE</code> variable; cf. the <code>trUploadFailureAndResultData</code>, <code>trUploadResultDataAndRecipe</code> and <code>trUploadStationResult</code> API functions. Further details on this topic can be found in the concept description "Failure codes for measurement data"!
-------	---

15270

15270	Determines for the API functions the maximum number of data objects in the array returned; if the parameter is not used ("not used" state), the default value is "1000". Advice: This parameter is evaluated by the following API functions: ewAPI service: - <code>getSerialNumbersForAttribute</code> IMSApi service: - <code>pmGetProductQuantity</code> - <code>trGetSerialNumberForWorkorderAndWorkstep</code> - <code>trGetWorkOrderForStation</code>
-------	--

15280

15280	Determines whether (TRUE) or not (FALSE) a station may be registered multiple times by means of the <code>regLogin</code> IMSApi function. If the parameter is active, the following locking mechanism is active: The IP address is taken over from the middleware and locked. The IP address is deleted by means of the <code>regLogout</code> IMSApi function, thereby releasing the station. It is possible to manually reset the IP address in the station setup. The same IP address may login (register) repeatedly (e.g., following a program crash on the client). Advice: This parameter is evaluated by the following API functions: IMSApi service: - <code>regLogout</code> - <code>regLogin</code>
-------	--

15300

15300	Determines whether all or only the most recent measurement records for a serial number booking – i.e. here the same sequence number – can be ascertained. If the parameter is set to FALSE (default), only the most recent (last) measurement data is ascertained. If the parameter is set to TRUE, the following condition must be fulfilled. Condition: Multiple measurement records can be stored for a serial number booking only if measurement records are repeatedly booked for a serial number in one work step using the IMSApi function <code>trUploadResultDataAndRecipe</code> without a serial number state being booked (the input parameter <code>serialNumberState</code> must be passed with -1 starting with the 2nd measurement record). Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - <code>trGetSerialNumberHistoryData</code>
--------------	---

16000

16000	Determines whether or not stations of type D (diagnosis) and R (repair) are capable of supplying feedback (active: Y, J or TRUE; inactive: N or FALSE). If the parameter is active, the station can perform process-locking bookings and is therefore able to complete work steps. In the case of a scrap booking, the work step is by default completed even if the parameter is inactive. If necessary, this behavior can be changed using station parameter code no. 16001 [▶▶▶] (see there the value "A"). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>uploadFailureslip</code> - <code>uploadResultDataAndRecipe</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> IMSApi service: - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadResultDataAndRecipe</code> - <code>trUploadState</code> <i>This function can have serious consequences on the process reliability. This function should, therefore, only be activated in conjunction with close consultation with the iTAC-employee responsible for the project!</i>
--------------	--

16001

16001	This parameter makes it possible to ascertain repair loops not completely passed through on the previous work step during the course of the serial number check [Y; N; A]. If the parameter is set to "Y", the bookings of stations without confirmation capability of type D (diagnosis) or R (repair) are ignored. I.e. also for the case that the repair station executed the last booking on the work step (highest sequence), only the last booking of the assigned tester is interpreted. If the parameter is set to "N", the system detects that the last booking was executed by a repair station and interprets the work step as NOT completed (independent of the state); the tester must rebook the affected serial number in any case. If the parameter is set to "A", SCRAP bookings are not taken into account by stations of type D (diagnosis) or R (repair) (i.e. also if station parameter 16000 = N). As a result, the default behavior "Scrap booking always concludes a work step" is overridden. Advice: <i>The check is only possible when the diagnosis or repair stations of the previous work step of station parameter code no. 16000 [▶▶▶] is not active!</i> This parameter is evaluated by the following API functions: ewAPI service: - checkSerialNumberState - checkSnrStateBySnrRef IMSApi service: - trCheckSerialNumberState
--------------	--

16100

16100	Determines whether the booking of test results using the API function increases the random sample counter (Y) or not (N). Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - trUploadTestplanResult
--------------	--

18200

18200	Parameter for controlling the program behavior for whether serial numbers may be processed further even if they were booked once with SCRAP at a work step. With this parameter, a generally valid process interlock can be activated for scrapped serial numbers without needing to explicitly implement it via special API processes. For all modes listed below, a station at the same work step can overwrite a previously performed scrap booking. Advice: <i>This program behavior is defined centrally as the site default (for further information, see site parameter code no. 18200 [▶▶▶]); ACTIVATE this parameter ONLY for the stations whose behavior should deviate from the site default!</i> off = No check of the previous work steps is performed; serial numbers can still be booked into the system by default even if they were booked into the system with the SCRAP state. check = The previous work steps are checked only to determine whether a scrap booking was performed there in the highest sequence. If this is the case, no further bookings can occur on the affected serial number. When booking with one of the <code>upload</code> functions listed below, a warning or error message is output depending on whether the scrapped serial number is in a multiple panel. checkAll = All work steps are checked (if necessary, successive steps as well) to determine whether a scrap booking was performed in the highest sequence. If this is the case, no further bookings can occur on the affected serial number. When booking with one of the <code>upload</code> functions listed below, a warning or error message is output depending on whether the scrapped serial number is in a multiple panel (for further information, see station parameter code no. 13400 [▶▶▶]). Only in this mode is it possible to authorize individual stations to rebook scrapped serial numbers with PASS (for further information, see station parameter code no. 13410 [▶▶▶]). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>assignSerialNumberToWorkOrder</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> - <code>verifyMerge</code> - <code>verifyMergeBySnrRef</code> IMSApi service: - <code>trMergeParts</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadState</code> - <code>trUploadStationResult</code> - <code>trVerifyMerge</code>
-------	--

21020

21020	This parameter determines which mode is to apply for the "packing of locked serial numbers" application type [I; F; L]: I (IGNORE) = The lock state of the serial number is not taken into account in the context of "packing"; i.e. locked serial numbers are allowed to be packed. F (FORBIDDEN) = Locked serial numbers/lots are not allowed to be packed. L (LOCKED) = ONLY locked serial numbers are allowed to be packed (locked storage function). Advice: <i>In this context, also note site parameter code no. 6035 and 6036. This parameter is evaluated by the following API functions:</i> IMSApi service: - <code>shipActivateShippingLotAtKap</code> - <code>shipAddChildLotToParentLot</code> - <code>shipAddSerialNumberToShippingLot</code> - <code>shipCheckSerialNumberAddToShippingLot</code>
-------	---

18500

18500	Determines whether the characters "%" and "_" are interpreted as placeholders (Oracle wildcards) or as fixed characters when data objects are queried on the basis of attribute values. 0 = fixed character; only returns data objects if they precisely match the queried attribute value 1 = placeholder; may return multiple data objects as a result of "fuzzy" matches (LIKE) with the attribute value Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - getMaterialBinsForAttribute - getSerialNumbersForAttribute IMSApi service: - attribGetObjectsForAttributeValues - attribGetAttributeHistory <i>This program behavior is defined centrally as the site default (for further information, see site parameter code no. 18500 [▶▶▶]); ACTIVATE this parameter ONLY for the stations whose behavior should deviate from the site default!</i>
--------------	--

18510

18510	Determines how the entry of an empty string is interpreted when data objects are ascertained on the basis of attribute values (see e.g. the API functions) [0; 1]: 0 = Empty string as the attribute value returns ALL data objects 1 = Empty string as the attribute value does NOT return any data objects Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - getSerialNumbersForAttribute IMSApi service: - attribGetObjectsForAttributeValues
--------------	---

20000

20000	Displays whether the reported machine times are used for computation of the production time (1) or whether the production time is calculated on the basis of the manufactured serial numbers in combination with the work plan default times "TE_M" and "TE_P" (0); the setup times "TR_M" and "TR_P" are not taken into consideration here. Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20000 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	---

20001

20001	Displays whether the First Pass Yield (1) or the Yield (0) is used for the "quality" factor for the calculation of the OEE (Overall Equipment Effectiveness). Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20001 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	--

20002

20002	Displays whether the First Pass Yield (1) or the Yield (0) is used for the "quality" factor for the calculation of the NEE (Net Equipment Effectiveness). Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20002 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	---

20003

20003	Displays whether the calculation of the placing performance is performed on the basis of units (1) or components (0). Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20003 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	---

20004

20004	Displays whether the reported machine times are used for the calculation of the MTBF (Mean Time Between Failure) (1) or whether the production time is used according to the shift timetable (0). Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20004 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	---

20013

20013	Displays which availability is used as the basis for calculating the OEE (Overall Equipment Effectiveness) [0; 1; 2; 3]: 0 = Total availability 1 = Organizational availability 2 = Technical availability 3 = System-related availability Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20013 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	--

20014

20014	Displays whether the cycle time based on a selected time range or the operative time according to the shift timetable is used for calculating the OEE (Overall Equipment Effectiveness) [0; 1]: 0 = Selected time range 1 = Operative time according to shift timetable Advice: <i>This parameter is only relevant in the context of PM (PM = Production Management)!</i> <i>The current mode should only be changed in the setup dialog for the station; for further information, see section »Tab "Characteristic values"«!</i> <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior is also defined centrally as the site default (for further information, see site parameter code no. 20014 [▶▶▶]); change this parameter ONLY for the stations whose behavior should deviate from the site default.
--------------	--

21010

21010	This parameter determines whether the packing process for serial numbers is excluded from the feedback to a connected ERP system. This applies to the movement types 810 (individual item to lot) and 811 (individual item from lot). If the parameter is active (TRUE), the movement types named above are ignored by the »Process "Material movement data"« for the corresponding bookings (by means of API or directly in the Shipping-Client). At the Shipping-Client, the activated exception rule only applies if this station was used for login. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - shipAddSnrToShippingLot - shipRemoveSnrFromShippingLot IMSApi service: - shipAddSerialNumberToShippingLot - shipRemoveSerialNumberFromShippingLot
--------------	---

21012

21012	This parameter determines whether the packing process for lots (key word "bins in bin") is excluded from the feedback to a connected ERP system. This applies to the movement types 820 (lot to parent lot), 821 (lot from parent lot), 822 (child lot to lot) and 823 (child lot from lot). If the parameter is active (TRUE), the movement types named above are ignored by the »Process "Material movement data"« for the corresponding bookings (by means of API or directly in the Shipping-Client). At the Shipping-Client, the activated exception rule only applies if this station was used for login. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - shipAddChildLotToParentLot - shipMoveAllChildLotsToNewParentLot - shipMoveChildLotsFromLotToLot - shipMoveChildLotToNewParentLot - shipRemoveAllChildLotFromParentLot - shipRemoveChildLotFromLot IMSApi service: - shipAddChildLotToParentLot - shipMoveChildLots - shipRemoveChildLots
--------------	--

21016

21016	This parameter determines whether the state change for lots is excluded from the feedback to a connected ERP system. This applies to the movement types 830 (lot created), 831 (lot closed), 832 (lot reopened) and 833 (lot finished). If the parameter is active (TRUE), the movement types named above are ignored by the »Process "Material movement data"« for the corresponding bookings (by means of API or directly in the Shipping-Client). At the Shipping-Client, the activated exception rule only applies if this station was used for login. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - mlCreateNewMaterialBin - shipCompleteLot - shipReopenShippingLot IMSApi service: - mlCreateNewMaterialBin - shipCompleteLot - shipReuseCompletedShippingLot
--------------	---

22000

22000	This parameter determines whether splitting into single panels is to take place automatically (TRUE) or not (FALSE) when a multiple panel is booked. Condition for this is that the "ERP sep. flag" field is set at the corresponding work step. If automatic splitting is not to be performed, this process can also be performed separately by means of the <code>trSplitPanel</code> IMSApi function. Advice: <i>This parameter is evaluated by the following API functions:</i> IMSApi service: - trUploadActivity - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState Background: If a work step configured in this way books a multiple panel, this multiple panel is split into single panels in the system. It must therefore be ensured that all panels of the multiple panel are uniquely identified (labeled with own serial number) before the booking takes place!
--------------	---

25002

25002	Activates/deactivates whether, when a multiple panel is mounted, the serial numbers of the upper integration level automatically receive a SCRAP booking if the mounted serial number is SCRAP (active: TRUE, inactive: FALSE). If the parameter is inactive, the serial number of the upper integration level does not receive a state booking; it is then initially in the "No state booking present" state (<code>SERIAL_NUMBER_STATE = -1</code>). Condition: In combination with station parameter code no. 13400 [▶▶▶], it is possible to determine whether scrapped serial numbers of a multiple panel can actually be mounted in the next integration level. This parameter is only evaluated if mounting is permitted. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - trAssignSerialNumberMergeAndUploadState
--------------	---

25100

25100	Used to determine that containers newly created in the iTAC system using the current station are passed directly to the ERP transfer process, i.e. written to the transfer table »TRAN.tranpuinit« (TRUE). This avoids the time delay which would otherwise occur as a result of the processing interval (timer). With FALSE, the time-controlled default process based on the timer "timer.erp.packUnitSend" is used. Advice: This parameter is evaluated by the following API function (IMSApi service): - mlCreateNewMaterialBin <i>Note in particular the key MATERIAL_BIN_TRANSACTION_TYPE; here a movement type must be used which is configured for the ERP data transfer (see the master data field "To ERP" in the ADM module for movement types)!</i>
--------------	---

26002

26002	Determines whether, in the context of personalized Production Data Collection (pPDC), the password associated with a user must be passed (TRUE) during user authentication based on the user name, or not (FALSE). By default, the password must be passed with the user name; the parameter should only be set to FALSE if customer-specific API workflows make this necessary. Advice: The parameter influences neither the pFDC functions in the HMI-Client nor the transfer of pFDC booking records by means of the ERP transfer interface; furthermore, it is only interpreted if NO electronic key (user token) is used for user authentication. This parameter is evaluated by the following API functions (IMSApi service): - pmGetBooking - pmUpdateBooking - pmUploadBooking Advice: This parameter is only relevant in the context of PM (PM = Production Management)!
--------------	---

27.3.2

Server parameter (IMSApi)

2020

2020 / 1	This parameter is used to influence the feedback from quantity bookings to an ERP system (TRUE/FALSE). Because all state bookings are always counted, negative quantities may occur temporarily (prior to repairs). This parameter must be activated if the connected ERP system cannot process any negative quantities. In this case, only first PASS or SCRAP bookings of a serial number are passed to the ERP system. Default: FALSE Advice regarding value entry: The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted! Advice: In this context, the "Quantity booking" ERP transfer process is to be taken into account (see timer description "timer.erp.orderconfirmsend"). This process makes quantity bookings for finalized products available for retrieval by an external ERP system (transfer table »TRAN.tran_order_confirmation«). This parameter influences, among other items, the behavior of the pmUpdateBooking and trUploadPmQuantity API functions (IMSApi service)!
----------	---

2032

2032 / 0	Determines whether serial numbers are automatically deactivated by a scrap booking (Y) or not (N). Default: N Advice regarding value entry: The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted! Advice: Among other things this parameter influences the behavior of the mergeParts, mergeBatch (ewAPI service), trCheckSerialNumberState, trMergeParts and batchMergeBatch (IMSApi service) API functions; scrapped serial numbers of a multiple panel/bin can thus only be merged with other products if the configuration parameter is set to "N"! Special case "AUTOSWITCH": If this parameter is set to "Y" and site parameter code no. 4610 [▶▶▶] is set to TRUE, scrapped serial numbers are automatically renamed by the system, whereby a sequential counter (-XX) is appended to the original serial number. The original serial number is thus available again for another individual item – this mechanism should not be used unless an uninterrupted sequence of serial numbers is absolutely necessary.
----------	--

2080

Advice: These parameters are only used for migrating old installations to the new release 7.x. For new installations and migrated old installations, this parameter has no function. The settings are then made in the central ConfigFramework; for further information, see the description of the documentLevel, documentShare and documentURL configuration parameters in the section »Parameter "Application server"«! The parameters listed below are used to determine the specifications for data storage and data access of MDA items from the iTAC system. The configuration distinguishes two variants: The files are stored locally on the executing application server or centrally on an external file server.	
2080 / 0	MDA Document Root local: relative path under \workplace (e.g., MDADocs) central: complete path (e.g., z:\MES_Suite\MDADocs\)
2080 / 1	MDA Document SHARE local/central: Windows share directory (e.g., MDADocs)
2080 / 2	MDA Document URL local: URL is ascertained by the application server (-) central: URL (e.g., http://fileserver/MDADocs)

Advice: *These parameters are mandatory when using the `getMdaDocuments` API function!*

14600

14600 / 0	Determines the work order attribute designator that is to be used for the automatic lot specification determination. All attribute codes with this designator are taken into account during the lot check. When doing so, a comparison is made to determine whether a work order attribute with the same lot specification exists for the respective work order, as is specified in the master data of the lot (for further information, see the "Part assignment" field in section »Tab "Material"«). Default: <code>PACK_SPEC</code> Advice regarding value entry: <i>The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted! The input values "" and "-1" are interpreted in the same way as no entry.</i> Advice:: <i>This parameter influences, among other items, the behavior of the <code>shipGetShippingPrefs</code> API functions (IMSApi service)!</i>
------------------	--

19000

19000 / 0	This parameter is used to determine whether the engine uses the JavaScript Compile function when the JavaScript-based <code>customerFunctions</code> are executed (TRUE). If the function is not active (FALSE), the scripts must be interpreted with each function call. Default: <code>FALSE</code> Advice regarding value entry: <i>The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted!</i>
------------------	---

19010

19010 / 0	In the context of IMSApi communication, this parameter is used to enforce strict observance of the used IMSApi keys. If the check is active (TRUE), the following is checked at each function call: - a.) All keys with mandatory field designation must be passed - b.) All keys that are passed must be defined in the function If a failure is detected during the check, the IMSApi function is terminated with an error message. If the parameter is deactivated (FALSE), no check takes place. Default: <code>TRUE</code> Advice regarding value entry: <i>The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted!</i> Advice: <i>This parameter should only be deactivated in exceptional cases because failure-free implementation of the IMSApi functions is only guaranteed if the check is active!</i>
------------------	--

19100

19100 / 0	In the context of IMSApi communication, this parameter allows the data type for the parameter <code>sessionId</code> (unique designator of the active session) to be changed from the default value long to int (TRUE). Background: Owing to a restriction in the 7.11.00 C-DLL under Windows and Linux 32 Bit, the <code>sessionId</code> had to be of the data type "integer". As of release 7.11.01, this restriction has been eliminated and "long" <code>sessionIds</code> are now supported on all IMSApi adapters. Default: <code>FALSE</code> Advice regarding value entry: <i>The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted!</i> Advice: <i>This parameter should only be set from FALSE to TRUE (for test purposes) if, when upgrading from release 7.11.00 and using IMSApi-C-DLL, the failure [-3] "Session is invalid" occurs when IMSApi functions are executed!</i>
------------------	--

19200

19200	This parameter specifies whether overbooking is permitted to take place for split work steps (key word "work step splitting"). FALSE = Overbooking is NOT allowed TRUE = Overbooking is allowed Default: TRUE Advice regarding value entry: <i>The entry must be made in the "Value" column; the "Num. value" column is NOT interpreted!</i> Advice: <i>This parameter is particularly relevant for the APS service and is evaluated by the following API functions (IMSApi service):</i> - batchUnregisterBatch - pmUploadBooking - pmUploadQuantityAndFailureData - pmUploadQuantityFailureTallyCardAndFailureData - trUploadActivity - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadPmQuantity - trUploadResultDataAndRecipe - trUploadState - trUploadTestplanResult
-------	--

27.3.3

Site parameter (IMSApi)

3011

3011	Determines the maximum degree of freedom which applies for the retrograde booking; this limits the options for manual adjustment in the ADM module or MRP module (see description of modes under parameter 3010): 0 = Manual adjustment is not possible; retrograde booking is always deactivated 1 = Manual change between the modes 0 and 1 is possible 2 = Manual change between the modes 0, 1 and 2 is possible 3 = Manual change between the modes 0, 1, 2 and 3 is possible
-------------	--

3050

3050	In the context of personalized production data collection (pPDC), this parameter determines the maximum booking period for a booking (in minutes); if the value [-1] is entered, NO check takes place. Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - pmUpdateBooking - pmUploadBooking
-------------	--

4420

4420	Determines whether (☒) or not an order number specified by the customer (e.g., from an ERP system) is also used as an internal order number. Advice: <i>If this parameter is set, the "Customer order no." field in the system becomes a mandatory field; for further information, see section »View "Order administration"«!</i> <i>This parameter is evaluated by the following API function:</i> IMSApi service: - mdataCreateWorkorder
-------------	---

4560

4560	Activates/deactivates whether the apparent units are resolved (for further information, see the "MRP" tab in the part master and there the "Material for parent work order" option for the "Material resolved" parameter). Advice: <i>This parameter is evaluated by the following API functions:</i> IMSApi service: - setupCheck - setupCheckPreSetup - setupGetMaterialSetup - trAssignSerialNumberMergeAndUploadState - trMergeParts
-------------	--

4570

4570	Used to determine which point in time is used for ascertaining the equipment relevant for a work step: 0 = Creation date of the work order (default): i.e. in this mode, changes made to the equipment assignment after work order creation are not considered/output. 1 = Current point in time: here, the current settings including the changes made to the master work plan after work order creation are considered/output. Advice: <i>This parameter is evaluated by the following API functions; it also influences the display in the "Work order information" dialog box and the "Setup equipment" display in the HMI-Client:</i> IMSApi service: - equCheckEquipmentData - equGetRequiredEquipmentData - equGetRequiredEquipmentDetail
-------------	---

4571

4571	Used to determine which period under review is used for ascertaining the user skills relevant for a work step: 0 = Creation date of the work order (default): i.e. in this mode, changes made to the user skill assignment after work order creation are not considered/output. 1 = Current point in time: here, the current settings including the changes made to the master work plan after work order creation are considered/output. Advice: <i>This parameter is evaluated by the following API functions; it also influences the display in the "Work order information" dialog box:</i> IMSApi service: - trCheckUserSkill - trCheckUserSkills
-------------	--

4572

4572	Used to determine which point in time is used for ascertaining the optimization groups required for a work step: 0 = Creation date of the work order (default): i.e. in this mode, changes made to the optimization group assignment after work order creation are not considered/output. 1 = Current point in time: here, the current settings including the changes made to the master work plan after work order creation are considered/output. Advice: <i>This parameter is evaluated by the following API functions; it also influences the display in the "Work order information" dialog box:</i> IMSApi service: - apsGetWorkorderDetails - apsGetWorkorderDetailsOptGroup
-------------	---

4610

4610	Determines whether a serial no. must be unique in the entire system (☒) or if a serial no. must be unique only within a work order. Advice: <i>Deactivation (☐) can have serious consequences on the process reliability. This parameter should, therefore, only be changed in conjunction with close consultation with the iTAC employee responsible for the project! Also note the interaction with station parameter code nos. 7208 [▶▶▶] and 7222!</i> <i>This parameter is evaluated by the following API functions::</i> ewAPI service: - assignSerialNumberForProductOrWorkorder - assignSerialNumberMergeAndUploadState IMSApi service: - trAssignSerialNumberForProductOrWorkOrder - trAssignSerialNumberMergeAndUploadState Special case "AUTOSWITCH": If this parameter is set to "TRUE" and server parameter code no. 2032 [▶▶▶] is set to "Y", scrapped serial numbers are automatically renamed by the system, whereby a sequential counter (-xx) is appended to the original serial number. The original serial number is thus available again for another individual item – this mechanism should not be used unless an uninterrupted sequence of serial numbers is absolutely necessary.
-------------	--

6010

6010	Determines which test mode is to be in effect when adding parts (serial numbers) to lots; here, in particular, the list of usable parts for a lot is checked (refer to the "Part assignment" option in the section »Tab "Material"«). Advice: <i>The first packed serial number determines the nominal values for the test (part number, BOM version, order). A change in the check mode does not affect already started lots; both the check mode and the nominal values remain valid; the specifications are only reset via a reinitialization or complete emptying of the lot!</i> 1 = No test 2 = A check is performed to ensure that only those serial numbers which match a product from the part assignment may be packed; it is ensured here that only serial numbers of one part may be packed in a lot (pure); if multiple parts are valid according to the "part assignment", the first serial number "packed" in the lot determines the valid part. 3 = As with check mode 2 whereby a check is also performed to ensure that only the serial numbers of a part with identical BOM version may be packed in a lot. 4 = As with check mode 3 whereby a check is also performed to ensure that only the serial numbers of the same work order may be packed in a lot. Note that check mode [3] is used by default if no entry is made or the entry is invalid. Advice: <i>For individual lots, the central site setting can be overridden. In this case, it is only possible to "restrict" the check for individual parts. The restriction of the check mode is performed in the part master of the lot (in section »Tab "Material"«, see the "Lot check" option)!</i> <i>This parameter is evaluated by the following API functions::</i> ewAPI service: - shipAddSnrToShippingLot - shipCheckSnrAddToShippingLot IMSApi service: - shipAddSerialNumberToShippingLot - shipCheckSerialNumberAddToShippingLot
------	---

6040

6040	Used to determine whether the state of the lot is automatically changed from initialized to in process and the start date to the current date/time (☒) or not (☐) as soon as the packaging check is performed on the first serial number (key word "CheckAdd"). If the function is not active, the lot remains in the state initialized and the default value 31.12.3000 00:00:00 continues to be used as the start date; the lot is then only changed when a serial number is actually packed. Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - shipCheckSnrAddToShippingLot IMSApi service: - shipCheckSerialNumberAddToShippingLot
------	---

7070

7070	Determines for the current site whether the MSL logic (Moisture Sensitive Level) is to be activated for the expiry time calculation; the following values are possible: 0 = no MSL logic 1 = obsolete MSL logic (only for old installations) 2 = complete MSL logic (with serial number reference) The "MSL" preset in the part master is only checked if the MSL logic is active. Advice: <i>In connection with this parameter, station parameter code no. 7134 [▶▶▶] is also to be observed; this activates the (additional) check of the MSL time for the station!</i>
-------------	--

7210

7210	Determines how the system reacts to a material movement which would result in a negative stock quantity. 0 = The quantity booking is only performed with the quantity that sets the stock quantity to "0". For later traceability, the remaining quantity is booked with the special movement type "CS_NEG" (code 299) without reducing the stock quantity further. 1 = Quantity bookings that result in a negative stock quantity are generally permitted and are passed with the selected movement type without inspection. The stock quantity is also reduced by the overbooking quantity and takes on a corresponding negative value, which is also apparent in the "QUANTITY_TOTAL" field in the "TRAN.traninvmove" transfer table. 2 = Quantity bookings that result in a negative stock quantity are not permitted and are rejected by the system without booking (API functions, among others, output a warning in this case). Advice: <i>This parameter is evaluated by the following API function:</i> - uploadMaterialBinBookingsExt (ewAPI service) - mlUploadMaterialBinBooking (IMSApi service) - mlChangeMaterialBinData (IMSApi service)
-------------	--

7260

7260	This parameter determines whether and to what extent the order networks created in the iTAC system are to be taken into consideration when products/units are mounted. The following function modes are available here: 0 = Order networks are NOT checked during the mounting process 1 = Order networks are ONLY checked if order networks have been found for the product structure to be mounted 2 = Order networks are ALWAYS expected and checked accordingly; if no order networks are defined for the product structure to be mounted, the check will have a negative result. Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trMergeParts - trSimulateMerge - trVerifyMerge <i>The program behavior selected here is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 7260 [▶▶▶])!</i>
-------------	--

13030

13030	Determines whether production means (equipment, cavities or carriers) which have become invalid may continue to be used (TRUE). If the value is FALSE, production means with an invalid state (e.g. "locked", "deleted", "defective" and "in repair") and production means in which the maintenance interval has expired cannot generally be booked in the system with the API functions; detailed information on this topic can be found in section »Production means«. Advice: <i>This parameter also affects the behavior of the <code>checkEquipmentData</code>, <code>updateEquipmentData</code> (ewAPI-Service), <code>equCheckEquipmentData</code>, <code>equGetRequiredEquipmentData</code>, <code>equGetRequiredEquipmentDetail</code> and <code>equUpdateEquipmentData</code> (IMSApi service) API functions!</i> The following applies to the application type "Equipment" (with/without cavity) only: The various <code>upload</code> functions that increment the usage counter for the maintenance interval do NOT check the equipment/cavity for validity; to check, the API function <code>checkEquipmentData</code> or <code>equCheckEquipmentData</code> must be used.
--------------	---

13040

13040	Determines whether ONLY the data of the equipment type "Feeder", "Nozzle" and "Head" are booked in the system (TRUE). If the value is FALSE, all equipment types are considered. Advice: <i>This parameter affects the behavior of the API function <code>equReportQuantity</code> (ewAPI service and IMSApi service)!</i>
--------------	---

13050

13050	Used to determine whether one particular item of equipment is allowed to be set up on a station multiple times. If the parameter is set (TRUE), the equipment required for a work step can be set up multiple times, provided that a unique setup position is specified (key <code>SETUP_POSITION</code>). If the parameter is not set (FALSE), the IMSApi function given below terminates the setup process with a warning. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - <code>equUpdateEquipmentData</code> <i>The program behavior selected here is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 8410 [▶▶▶])!</i>
--------------	---

13060

13060	Parameter for controlling the automatic creation of maintenance orders for production means. Work order creation functions only with correspondingly designated production means. The designation takes place in the production means master data, "Maintenance" tab and there in the "Create maintenance order" field – the field only becomes visible if the parameter was activated here (> 0). Advice: <i>The automatic creation of maintenance orders only functions if the maximum number of usages is exceeded – exceedance of the time-based usage duration is NOT taken into consideration!</i> The following modes are available for controlling the function: 0 = deactivated: no automatic creation of maintenance orders (default behavior) 1 = activated: the system creates a maintenance order for each device/holder class, mold and carrier class (part number of the equipment, cavities and carriers) 2 = activated: the system creates a maintenance order for each production means (concrete equipment, cavity, or carrier number) 3 = activated: same as mode [2], only the system creates a separate maintenance order for each maintenance operation Advice: <i>The structure of the work order number generated by the system is defined by means of site parameter code no. 13061 [▶▶▶]!</i> Background to "usage counter": The usage counter for equipment/cavities can be incremented automatically via upload bookings; to activate, see the following station parameters: - code no. 8400 [▶▶▶] for work-step-related equipment - code no. 8402 [▶▶▶] for active cavities on equipment - code no. 8401 [▶▶▶] for asset-related equipment Also applicable for all production means classes is that the usage figures can be "manually" transferred to the iTAC system by means of the IMSApi function <code>equReportQuantity</code>.
--------------	--

13061

13061	This is used to define the structure of the work order number for automatically created maintenance orders. In addition to arbitrary free texts, the following variables, among others, are available for the structure of the number; the term "production means" stands for equipment and carriers: <EQUNO> = Number of the production means <EQUNOEXT> = External number of the production means <EQUINDEX> = Index of the production means <PARTNO> = Part number of the production means <DD> = Day <MM> = Month <YY> = Year, 2-digit <YYYY> = Year 4-digit <HH> = Hour (24 hours) <MI> = Minute Example: MAI-<EQUNO>_<MM><DD>: MAI-343_0815 When selecting the variables, note the selected mode for site parameter code no. 13060 [▶▶▶]; if, e.g., mode [1] is selected there (maintenance order per device class/carrier class), the variables <EQUNO>, <EQUNOEXT> and <EQUINDEX> may not be used. Advice: <i>This parameter is only evaluated if site parameter code no. 13060 [▶▶▶] has been activated!</i>
--------------	---

18100

18100	Determines whether, in the context of "personalized machine booking", the user logged into the station is always used (☑). If the parameter is inactive, the standard behavior applies: the user used for the session – e.g., through the user login on the client – is superimposed on the user logged into the station. Advice: <i>This parameter thereby indirectly influences the system behavior of IMSApi functions <code>regRegisterUser</code> and <code>regLogin</code>!</i>
--------------	--

18200

18200	Parameter for controlling the program behavior for whether serial numbers may be processed further even if they were booked once with SCRAP at a work step. With this parameter, a generally valid process interlock can be activated for scrapped serial numbers without needing to explicitly implement it via special API processes. For all modes listed below, a station at the same work step can overwrite a previously performed scrap booking. Advice: <i>The program behavior selected here is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 18200 [▶▶▶])!</i> off = No check of the previous work steps is performed; serial numbers can still be booked into the system by default even if they were booked into the system with the SCRAP state. check = The previous work steps are checked only to determine whether a scrap booking was performed there in the highest sequence. If this is the case, no further bookings can occur on the affected serial number. When booking with one of the <code>upload</code> functions listed below, a warning or error message is output depending on whether the scrapped serial number is in a multiple panel. checkAll = All work steps are checked (if necessary, successive steps as well) to determine whether a scrap booking was performed in the highest sequence. If this is the case, no further bookings can occur on the affected serial number. When booking with one of the <code>upload</code> functions listed below, a warning or error message is output depending on whether the scrapped serial number is in a multiple panel (for further information, see station parameter code no. 13400 [▶▶▶]). Only in this mode is it possible to authorize individual stations to rebook scrapped serial numbers with PASS (for further information, see station parameter code no. 13410 [▶▶▶]). Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - <code>assignSerialNumberToWorkOrder</code> - <code>uploadState</code> - <code>uploadStateAndFailData</code> - <code>uploadStateAndFailureData</code> - <code>uploadStateAndResultData</code> - <code>verifyMerge</code> - <code>verifyMergeBySnrRef</code> IMSApi service: - <code>trMergeParts</code> - <code>trUploadFailureAndResultData</code> - <code>trUploadFailuresForMergedParts</code> - <code>trUploadState</code> - <code>trUploadStationResult</code> - <code>trVerifyMerge</code>
--------------	---

18210

18210	Activates/deactivates (TRUE/FALSE) whether the serial number/work order number is renamed automatically when a serial number is scrapped (TRUE). The condition for this is that only ONE serial number is manufactured with the work order (order quantity "1"). During renaming, the following extension is appended to both the serial number and the work order number: *_SCRAP[YYYYMMddhhmm] In this context, the renamed work order is changed automatically to the finished state. Advice: <i>The requirement is associated with JIT/JIS where there is only one serial number per work order and the delivery MUST take place using this serial number. This parameter is evaluated by the following API functions (IMSApi service):</i> - trUploadFailureAndResultData - trUploadFailuresForMergedParts - trUploadResultDataAndRecipe - trUploadState - trUploadTestplanResult
--------------	---

18300

18300	This parameter determines the part number for the "Feeder" equipment type which is used by default in the event of automatic new creation of equipment. Advice: <i>This specification is required when the performance data passed by the SMT Interface is passed with an unknown equipment number; in this case, the equipment is created based on the part number specified here. The data are visualized either in the "MIS machine data → equipment" function of the TR module (see the "SMT overview" view there) or in real-time in the Dashboard-Client!</i>
--------------	--

18301

18301	This parameter determines the part number for the "Nozzle" equipment type which is used by default in the event of automatic new creation of equipment. Advice: <i>This specification is required when the performance data passed by the SMT Interface is passed with an unknown equipment number; in this case, the equipment is created based on the part number specified here. The data are visualized either in the "MIS machine data → equipment" function of the TR module (see the "SMT overview" view there) or in real-time in the Dashboard-Client!</i>
--------------	--

18306

18306	This parameter determines the part number for the "Head" equipment type which is used by default in the event of automatic new creation of equipment. Advice: <i>This specification is required when the performance data passed by the SMT Interface is passed with an unknown equipment number; in this case, the equipment is created based on the part number specified here. The data are visualized either in the "MIS machine data → equipment" function of the TR module (see the "SMT overview" view there) or in real-time in the Dashboard-Client!</i>
--------------	--

18309

18309	Activates/deactivates (TRUE/FALSE) whether already set-up equipment is still allowed to be edited. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - equUpdateEquipment
--------------	---

18401

18401	Determines whether, during IMSApi-based removal of serial numbers from a lot, the quantity of serial numbers still in the lot is output instead of the default output value "0" (☒) – by default, the parameter is deactivated (☐) . Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - shipRemoveSerialNumberFromShippingLot
--------------	---

18500

18500	Determines whether the characters "%" and "_" are interpreted as placeholders (Oracle wildcards) or as fixed characters when data objects are queried on the basis of attribute values. 0 = fixed character; only returns data objects if they precisely match the queried attribute value 1 = placeholder; may return multiple data objects as a result of "fuzzy" matches (LIKE) with the attribute value Advice: <i>This parameter is evaluated by the following API functions:</i> ewAPI service: - getMaterialBinsForAttribute - getSerialNumbersForAttribute IMSApi service: - attribGetObjectsForAttributeValues <i>The program behavior selected here is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 18500 [▶▶▶])!</i>
--------------	---

18501

18501	Determines whether, during IMSApi-based querying of attribute values for serial numbers in repair and modification orders, the units installed there are also considered. If the parameter is active (☒) , the complete merge structure is resolved; as a result, the attributes of installed serial numbers are also ascertained and output – by default, the parameter is not active (☐) . Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - attribGetAttributeValues - attribGetAttributeValuesForObjectype
--------------	---

18520

18520	Determines whether, in the case of IMSApi-based locking of a material bin (container), the lock group specified here is to be assigned automatically. Application type "create container": If a new container is created using the IMSApi mlCreateNewMaterialBin with the Q state (QA locked) or B (locked), the lock group entered here is assigned. If no lock group is specified (value = [EMPTY]), the default behavior applies: The container number is used as the lock group. Application type "change container": If for an existing container the state is changed to Q using the IMSApi mlChangeMaterialBinData, the same behavior as for "create container" applies. If the container is changed to the B state, it is first checked whether serial numbers were manufactured and locked as a result. If locks already exist, the previous lock group of the assigned container is not changed. Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - mlChangeMaterialBinData - mlCreateNewMaterialBin <i>A similar function also exists for the ERP service; for further information, see site parameter code no. 5150!</i>
--------------	--

18521

18521	Determines whether for the IMSApi-based capture of person-related production data, automatic approval is possible not only for quantity bookings but also for parallel and serial bookings. If the parameter is active (☒) , all three booking modes are supported. The parameter is deactivated by default (☐) , which means that automatic approval is allowed for quantity bookings only. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - pmUploadBooking Pay particular attention there to the keys BOOK_MODE and CONFIRMATION_MODE!
--------------	--

18600

18600	Default value for the maximum value of permissible result records when using the IMSApi function mdataGetProductInfo.
--------------	--

20000

20000	Displays whether the reported machine times are used for computation of the production time (☒) or whether the production time is calculated on the basis of the manufactured serial numbers in combination with the work plan default times "TE_M" and "TE_P" (☐) ; the setup times "TR_M" and "TR_P" are not taken into consideration here. Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20000 [▶▶▶]). Advice: <i>The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!</i>
--------------	--

20001

20001	Displays whether the First Pass Yield (☒) or the Yield (☐) is used for the "quality" factor for the calculation of the OEE (Overall Equipment Effectiveness). Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20001 [▶▶▶]). Advice: <i>The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!</i>
--------------	--

20002

20002	Displays whether the First Pass Yield (☒) or the Yield (☐) is used for the "quality" factor for the calculation of the NEE (Net Equipment Effectiveness). Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20002 [▶▶▶]). Advice: <i>The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!</i>
--------------	--

20003

20003	Displays whether the calculation of the placing performance is performed on the basis of units (☒) or components (☐) Advice: This parameter is evaluated by the following API function (IMSApi service): <pre>- mdcGetKeyPerformanceIndicators</pre> The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20003 [▶▶▶]). Advice: The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!
--------------	--

20004

20004	Displays whether the reported machine times are used for the calculation of the MTBF (Mean Time Between Failure) (☒) or whether the production time is used according to the shift timetable (☐) Advice: This parameter is evaluated by the following API function (IMSApi service): <pre>- mdcGetKeyPerformanceIndicators</pre> The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20004 [▶▶▶]). Advice: The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!
--------------	--

20013

20013	Displays which availability is used as the basis for calculating the OEE (Overall Equipment Effectiveness) [0; 1; 2; 3]: 0 = Total availability 1 = Organizational availability 2 = Technical availability 3 = System-related availability Advice: This parameter is evaluated by the following API function (IMSApi service): <pre>- mdcGetKeyPerformanceIndicators</pre> The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20013 [▶▶▶]). Advice: The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!
--------------	---

20014

20014	Displays whether the cycle time based on a selected time range or the operative time according to the shift timetable is used for calculating the OEE (Overall Equipment Effectiveness) [0; 1]: 0 = Selected time range 1 = Operative time according to shift timetable Advice: <i>This parameter is evaluated by the following API function (IMSApi service):</i> - mdcGetKeyPerformanceIndicators The program behavior selected is initially valid for all stations in the site; if necessary, the behavior for individual stations can be overridden (for further information, see station parameter code no. 20014 [▶▶▶]). Advice: <i>The current mode should not be changed here, rather only in the set-up dialog for the machine information; for further information, see section »View "Characteristic values"«!</i>
--------------	--

22106

22106	Determines whether or not semifinished products that are removed from a locked unit are to automatically be locked (☒). In this way, locked semifinished products can again be unlocked manually. Advice: <i>This parameter is evaluated by the <code>trRemoveMergeParts</code> (IMSApi service) and <code>removeMergeParts</code> API functions (ewAPI service); in this context, station parameter 7207 [▶▶▶] is also to be taken into account!</i>
--------------	--

24010

24010	Determines whether units which have not set the product designator in the order-based BOM are also taken into consideration during the mounting check. FALSE = Only with product designator TRUE = Also without product designator Advice: <i>This parameter is evaluated by the following API functions (IMSApi service):</i> - trCheckMergedPartsForSerialNumberComplete - trCheckMergedPartsForSerialNumber
--------------	---

27.4

Change history

Original document:**Name:** IMSApi-Description-8.51-V1.0**Document:** IMSAPI_R8_51_V1_0_DE**Current as of:** 11/2016**Current documentation "IMSApi Description" (this document):****Name:** IMSApi-Description-9.10-V1.0**Document:** IMSAPI_R9_10_V1_0_DE**Current as of:** 01/2019***Inclusion of new IMSApi functions***

The respective IMSApi functions were introduced with the release specified at the right and mentioned for the first time in the document specified at the left. Changes to existing functions are listed in the next table (see section »Adaptation of existing function descriptions« [▶▶▶]).

Document	Newly added IMSApi functions	since release
IMSAPI_R9_10_V1_0_EN	New API functions associated with "Production means" (cavities):	
	»equAssignCavityToEquipment« [▶▶▶]	Rel. 9.10.00
	»equGetAssignedCavity« [▶▶▶]	Rel. 9.10.00
	»equGetCavityEquipmentAssignRule« [▶▶▶]	Rel. 9.10.00
	»equGetPartCavityAssignRule« [▶▶▶]	Rel. 9.10.00
	»equManageCavityEquipmentRule« [▶▶▶]	Rel. 9.10.00
	»equManagePartCavityRule« [▶▶▶]	Rel. 9.10.00
	»equRemoveCavityFromEquipment« [▶▶▶]	Rel. 9.10.00
	»equUpdateAssignedCavity« [▶▶▶]	Rel. 9.10.00
	New API functions from various subject areas:	
	»mlGetMaterialTransactions« [▶▶▶]	Rel. 9.10.00
	»trRemoveSerialNumberFromOrder« [▶▶▶]	Rel. 9.10.00
IMSAPI_R9_01_V1_1_EN	- none -	
IMSAPI_R9_01_V1_0_EN	New API functions associated with "APS":	
	»apsGetEquipments« [▶▶▶]	Rel. 9.01.00
	»apsGetStations« [▶▶▶]	Rel. 9.01.00
	»apsGetWorkerGroups« [▶▶▶]	Rel. 9.01.00
	»apsGetWorkers« [▶▶▶]	Rel. 9.01.00

Document	Newly added IMSApi functions	since release
	New API functions associated with "CustomFunction":	
	»customGetFunctionInfo« [▶▶▶]	Rel. 9.01.00
	»customGetFunctions« [▶▶▶]	Rel. 9.01.00
	»mdaUploadCustomFunction« [▶▶▶]	Rel. 9.01.00
	New API functions associated with "Production means":	
	»equAssignCarrierToCarrier« [▶▶▶]	Rel. 9.01.00
	»equAssignSerialNumberToCarrier« [▶▶▶]	Rel. 9.01.00
	»equCreateEquipment« [▶▶▶]	Rel. 9.01.00
	»equGetAssetEquipmentSetup« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierAssigned« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierAssignmentHistoryData« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierAssignRule« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierLocation« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierToCarrier« [▶▶▶]	Rel. 9.01.00
	»equGetCarrierToSerialNumber« [▶▶▶]	Rel. 9.01.00
	»equGetEquipmentLocationHist« [▶▶▶]	Rel. 9.01.00
	»equRemoveCarrierFromCarrier« [▶▶▶]	Rel. 9.01.00
	»equRemoveSerialNumberFromCarrier« [▶▶▶]	Rel. 9.01.00
	»equSetEquipmentLocation« [▶▶▶]	Rel. 9.01.00
	»equSetupAssetEquipment« [▶▶▶]	Rel. 9.01.00
	New API functions associated with "Data capture points":	
	»mdaDeleteDetectionPoint« [▶▶▶]	Rel. 9.01.00
	»mdaUpdateDetectionPoint« [▶▶▶]	Rel. 9.01.00
	New API functions associated with "Shift model":	
	»mdaGetAssignedShiftModel« [▶▶▶]	Rel. 9.01.00
	»mdaGetAssignedSpecialTime« [▶▶▶]	Rel. 9.01.00
	»mdaGetShiftModel« [▶▶▶]	Rel. 9.01.00
	»mdaGetTimeEvent« [▶▶▶]	Rel. 9.01.00
	»mdaGetWeekSchedule« [▶▶▶]	Rel. 9.01.00
	New API functions associated with "iTAC.Messaging":	
	»msgPublish« [▶▶▶]	Rel. 9.01.00
	»msgSubscribe« [▶▶▶]	Rel. 9.01.00
	»msgUnsubscribe« [▶▶▶]	Rel. 9.01.00

Document	Newly added IMSApi functions	since release
	New API functions from various subject areas:	
	»attribGetAttributeHistory« [▶▶▶]	Rel. 9.01.00
	»execFormulaCatalog« [▶▶▶]	Rel. 9.01.00
	»mdataGetFormulaCatalog« [▶▶▶]	Rel. 9.01.00
	»mdataRequestWorkorder« [▶▶▶]	Rel. 9.01.00
	»mdcCloseStationMessage« [▶▶▶]	Rel. 9.01.00
IMSAPI_R9_00_V1_0_EN	New API functions associated with "user skill":	
	»mdataGetUserSkill« [▶▶▶]	Rel. 9.00.00
	»mdataGetWorkstepSkill« [▶▶▶]	Rel. 9.00.00
	»trCheckUserSkills« [▶▶▶]	Rel. 9.00.00
	New API functions from various subject areas:	
	»apsGetWorkorderDetailsOptGroup« [▶▶▶]	Rel. 9.00.00
	»attribCheckAttributeValues« [▶▶▶]	Rel. 9.00.00
	»equGetRequiredEquipmentDetail« [▶▶▶]	Rel. 9.00.00
	»execTimerProcess« [▶▶▶]	Rel. 9.00.00
	»mdataChangeWorkOrderWorkStepState« [▶▶▶]	Rel. 9.00.00
	»mdataCreateSpecialTime« [▶▶▶]	Rel. 9.00.00
	»mdcGetKeyPerformanceIndicators« [▶▶▶]	Rel. 9.00.00
IMSAPI_R8_52_V1_0_EN	New API functions associated with "Data capture points":	
	»mdataCreateDetectionPoint« [▶▶▶]	Rel. 8.52.00
	»mdataGetDetectionPoint« [▶▶▶]	Rel. 8.52.00
	»mdataGetMessageCause« [▶▶▶]	Rel. 8.52.00
	»mdataGetMessageClass« [▶▶▶]	Rel. 8.52.00
	»mdataGetMessageEffect« [▶▶▶]	Rel. 8.52.00
	»mdcAcknowledgeStationConditions« [▶▶▶]	Rel. 8.52.00
	»mdcCreateLogData« [▶▶▶]	Rel. 8.52.00
	»mdcGetLogData« [▶▶▶]	Rel. 8.52.00
	»mdcGetMessageType« [▶▶▶]	Rel. 8.52.00
	»mdcGetStationConditionsByFilter« [▶▶▶]	Rel. 8.52.00
	»mdcUpdateStationCondition« [▶▶▶]	Rel. 8.52.00
	»mdcUploadDetectionPoint« [▶▶▶]	Rel. 8.52.00
	New API functions from various subject areas:	

Document	Newly added IMSApi functions	since release
	»attribGetAttributeValuesForObjectType« [▶▶▶]	Rel. 8.52.00
	»batchUnregBatch« [▶▶▶]	Rel. 8.52.00
	»mdaActivateDocument« [▶▶▶]	Rel. 8.52.00
	»mdaGetShiftData« [▶▶▶]	Rel. 8.52.00
	»mdcManageConditionCode« [▶▶▶]	Rel. 8.52.00
	»setupGetSetupDataBySerialNumberRef« [▶▶▶]	Rel. 8.52.00
	»trSerialNumberPassing« [▶▶▶]	Rel. 8.52.00

Tab. 27-2 Inclusion of new IMSApi functions

Adaptation of existing function descriptions

The descriptions of the respective IMSApi functions were adapted in the document specified at the left according to the adjacent short description. Functions newly added with a release are listed in the preceding table (see section »Inclusion of new IMSApi functions« [▶▶▶]).

Document	Adapted function descriptions
IMSAPI_R9_10_V1_0_EN	General With release 9.10.00, the data object Cavity was introduced and assigned to the "Production means" grouping. Formulations have been adapted at the relevant points in order to ensure that a distinction is made between the terms "equipment", "carrier" and "cavity". Use of the Messaging-Service now requires a license; those stations used for session login need an MSG license (concerns all API functions under section »Group "MSG" (iTAC.Messaging)« [▶▶▶]). »customModifyDBObjectData« [▶▶▶] Function description adapted: the maximum number of columns per table was increased from 25 to 45.
	»equCheckEquipmentData« [▶▶▶] Output values (error codes) 1307 and -1349 added; in the "Key Value Pair" structure equipmentResultKeys, keys CAVITY_ASSIGNED, CAVITY_CHECKSTATE, CAVITY_INDEX, CAVITY_NUMBER, CAVITY_POSITION, CAVITY_REQUIRED and CAVITY_TYPE added
	»equGetEquipment« [▶▶▶] In the "Key Value Pair" structure equipmentGetResultKeys, keys CAVITY_ASSIGNED and CAVITY_REQUIRED added
	»equGetRequiredEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentResultKeys, keys CAVITY_ASSIGNED and CAVITY_REQUIRED added
	»equGetRequiredEquipmentDetail« [▶▶▶] In the "Key Value Pair" structure requiredEquipmentResultKeys, keys CAVITY_ASSIGNED and CAVITY_REQUIRED added
	»equGetSetupEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentSetupResultKeys, keys CAVITY_ASSIGNED and CAVITY_REQUIRED added

Document	Adapted function descriptions
	»equReportQuantity« [▶▶▶] Amended reference for station parameter code no. 8402 [▶▶▶] and site parameter code no. 13060 [▶▶▶]
	»equUpdateEquipmentData« [▶▶▶] Output values (error codes) 1308, 1307, -1345, -1346, -1347, -1348 and -1349 added
	»mdataGetBomData« [▶▶▶] In the "Key Value Pair" structure bomDataFilter, key OBSERVE_ZERO_QUANTITY added
	»mdataGetFailureDataForStation« [▶▶▶] »mdataGetFailureDefinition« [▶▶▶] Mode 7 (failure risk) added to the description of the parameter functionMode; in the "Key Value Pair" structure failureDataResultKeys, keys FAILURE_RISK_CODE and FAILURE_RISK_DESC added
	»mdataGetUserData« [▶▶▶] In the "Key Value Pair" structures mdataGetUserDataKeys and mdataGetUserDataKeys, key GENDER added
	»mdcGetKeyPerformanceIndicators« [▶▶▶] In the "Key Value Pair" structure performanceIndicatorResultKeys, key AVAILABILITY_TYPE added
	»mdcGetRegisteredUser« [▶▶▶] In the "Key Value Pair" structures mdcGetRegisteredUserFilter and mdcGetRegisteredUserKeys, key STATION_NUMBER added
	»mdcUploadStationCondition« [▶▶▶] Output value (error code) -2015 added
	»pmUploadBooking« [▶▶▶] Amended reference for site parameter code no. 18521 [▶▶▶]
	»trAddSerialNumberToPool« [▶▶▶] Amended reference for station parameter code nos. 13301 [▶▶▶], 13302 [▶▶▶], 13311 [▶▶▶] and 13312 [▶▶▶]
	»trAssignSerialNumberMergeAndUploadState« [▶▶▶] Output value (error code) -102 added
	»trGetCapturedRepairHistoryData« [▶▶▶] In the "Key Value Pair" structure failureDataResultKey: keys FAILURE_CAUSE_INFO and FAILURE_TYPE_INFO added
	»trGetSerialNumberHistoryData« [▶▶▶] In the "Key Value Pair" structure failureDataResultKeys: keys FAILURE_CAUSE_INFO, FAILURE_TYPE_INFO, FAILURE_RISK_CODE, and FAILURE_RISK_INFO added
	»trUploadFailureAndResultData« [▶▶▶] Output values (error code) -102 and -369 added; in the "Key Value Pair" structure failureKeys: keys FAILURE_CAUSE_INFO, FAILURE_TYPE_INFO, FAILURE_RISK_CODE, and FAILURE_RISK_INFO added

Document	Adapted function descriptions
	»trUploadFailuresForMergedParts« [▶▶▶] Output values (error code) -102 and -369 added; in the "Key Value Pair" structure uploadFailuresForSlaveSerialNumberKeys: keys FAILURE_CAUSE_INFO, FAILURE_TYPE_INFO, FAILURE_RISK_CODE, and FAILURE_RISK_INFO added
	»trUploadResultDataAndRecipe« [▶▶▶] Output values (error codes) -15 and -102 added
	»trUploadState« [▶▶▶] Output value (error code) -102 added
	»trUploadTestplanResult« [▶▶▶] Output value (error code) -102 added
IMSAPI_R9_01_V1_1_EN	General With the release 9.01.00, the site TR and site PM structure trees were merged together in the machine assets; as a result, it is no longer necessary to configure TR and PM stations separately. This has also affected this documentation because in most cases the two no longer need to be dealt with separately. »customGetDBObjectData« [▶▶▶] »customGetDBObjectMetaData« [▶▶▶] »customModifyDBObjectData« [▶▶▶] Name of the function parameter stationName changed to stationNumber
	»equAssignSerialNumberToCarrier« [▶▶▶] Parameter setState added in the function declaration
	»equGetCarrierAssigned« [▶▶▶] In the "Key Value Pair" structure carrierGetFilters: mandatory field designation removed for the key EQUIPMENT_NUMBER
	»mdataGetWorkplanData« [▶▶▶] In the "Key Value Pair" structure workplanDataResultKeys key CONFIRMATION_NUMBER added
	»mdataGetWorkstepSkill« [▶▶▶] In the "Key Value Pair" structure workstepSkillResultKeys: keys OPERATION_MODE, REQUIRED_WORKER_CNT and WORKER_UTILIZATION added
	»setupStateChange« [▶▶▶] Output value (error code) -240 added
	»trUploadFailureAndResultData« [▶▶▶] »trUploadFailuresForMergedParts« [▶▶▶] Special case "Carrier" added to the function description
	»trUploadActivity« [▶▶▶] »trUploadResultDataAndRecipe« [▶▶▶] »trUploadState« [▶▶▶] »trUploadTestplanResult« [▶▶▶] Output code (error code) -185 added; special case "Carrier" added to the function description

Document	Adapted function descriptions
IMSAPI_R9_01_V1_0_EN	General With release 9.01.00, the data objects "carrier" and "production means" were introduced; production means is used as a generic term referring to the new carriers and the existing equipment. At the relevant points, the term "equipment" has been changed to the generic term "production means" (see e.g. the attribute category (objectType) 15 and the MDA data objects (targetType) 116 and 117). »apsGetWorkorderDetails« [▶▶▶] Amended reference to site parameter code no. 4572 [▶▶▶]; output value (error code) – 3600 added; in the "Key Value Pair" structure apsGetWorkorderDetailResultKeys: keys EARLIEST_START_LAY, EARLIEST_START_PRODUCTION, EARLIEST_START_SETUP, EARLIEST_START_STRIPDOWN, EARLIEST_START_TRANSPORT, EARLIEST_START_WAIT, LATEST_START_LAY, LATEST_START_PRODUCTION, LATEST_START_SETUP, LATEST_START_STRIPDOWN, LATEST_START_TRANSPORT and LATEST_START_WAIT added »apsGetWorkorderDetailsOptGroup« [▶▶▶] Amended reference to site parameter code no. 4572 [▶▶▶]; in the "Key Value Pair" structure apsGetWorkorderDetailsOptGroupResultKeys: keys EARLIEST_START_LAY, EARLIEST_START_PRODUCTION, EARLIEST_START_SETUP, EARLIEST_START_STRIPDOWN, EARLIEST_START_TRANSPORT, EARLIEST_START_WAIT, LATEST_START_LAY, LATEST_START_PRODUCTION, LATEST_START_SETUP, LATEST_START_STRIPDOWN, LATEST_START_TRANSPORT and LATEST_START_WAIT added »attribAppendAttributeValues« [▶▶▶] Output value (error code) –954 added »attribCheckAttributeValues« [▶▶▶] Output value (error code) –954 added »attribCreateAttributeCode« [▶▶▶] Output value (error code) –953 added; in the "Key Value Pair" structure codeKeys, key EDITABLE_BY added »attribGetAllAttributeValue« [▶▶▶] Output value (error code) –953 added; in the "Key Value Pair" structure attributeValueSearchKeys and attributeValueResultKeys: key EDITABLE_BY added »attribGetAttributeClass« [▶▶▶] In the "Key Value Pair" structure classItemResultKeys, key EDITABLE_BY added »attribGetAttributeCode« [▶▶▶] In the "Key Value Pair" structure codeResultKeys, key EDITABLE_BY added »attribGetAttributeValues« [▶▶▶] In the "Key Value Pair" structure attributeResultKeys, key EDITABLE_BY added »attribGetAttributeValuesForObjectType« [▶▶▶] In the "Key Value Pair" structure attributeResultKeys, key EDITABLE_BY added »attribGetObjectsForAttributeValues« [▶▶▶] In the "Key Value Pair" structure objectResultKeys: keys EQUIPMENT_CLASS and EQUIPMENT_TYPE added for objectType 15 »attribRemoveAttributeClassValue« [▶▶▶] Output value (error code) –954 added

Document	Adapted function descriptions
	»attribRemoveAttributeValue« [▶▶▶] Output value (error code) -954 added
	»attribUpdateAttributeCode« [▶▶▶] Output value (error code) -953 added; in the "Key Value Pair" structure codeKeys, key EDITABLE_BY added
	»batchRegisterBatch« [▶▶▶] Advice notice added under output value (error code) -133 (key word "unequal work orders"); output values (error code) -579 and -580 added
	»equGetEquipment« [▶▶▶] In the "Key Value Pair" structure equipmentGetFilters: key EQUIPMENT_CLASS added; in the "Key Value Pair" structure equipmentGetResultKeys: keys EQUIPMENT_CHECKSTATE, EQUIPMENT_CLASS, STORAGE_BARCODE and STORAGE_NUMBER added
	»equReportQuantity« [▶▶▶] In the "Key Value Pair" structure equReportQuantityKeys key EQUIPMENT_CLASS added
	»equUpdateEquipment« [▶▶▶] In the "Key Value Pair" structure equipmentUpdateKeys key EQUIPMENT_CLASS added
	»equUpdateEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentUploadKeys key EQUIPMENT_CLASS added
	»lockObjects« [▶▶▶] »lockUnlockObjects« [▶▶▶] In the "Key Value Pair" structure objectUploadKeys key WORKORDER_NUMBER added
	»mdaGetDocuments« [▶▶▶] Output values (error code) -1312 and -1321 added; in the "Key Value Pair" structure attributeFilters: keys EQUIPMENT_CLASS and EQUIPMENT_TYPE added; in the "Key Value Pair" structure mdaResultKeys: key PROTECTED added
	»mdaUploadDocument« [▶▶▶] Output values (error code) -729, -1312 and -1321 added; keys EQUIPMENT_CLASS and EQUIPMENT_TYPE added in the "Key Value Pair" structure attributeValues for targetTypes 116 and 117
	»mdaCreateWorkorder« [▶▶▶] In the "Key Value Pair" structure workorderResultKeys: key CUSTOMER_WORKORDER_NUMBER added
	»mdaCreateDetectionPoint« [▶▶▶] Output values (error codes) -706, -737 and -1231 added
	»mdaChangeWorkOrderWorkStepState« [▶▶▶] In the "Key Value Pair" structure workstepChangeKeys: key CUSTOMER_WORKORDER_NUMBER added and, in this context, mandatory field designation removed for WORKORDER_NUMBER
	»mdaCreateOrderCluster« [▶▶▶] Output value (error code) -1125 added; in the "Key Value Pair" structure createOrderClusterItemUploadKeys, key CUSTOMER_WORKORDER_NUMBER added

Document	Adapted function descriptions
	»mdataCreateWorkorder« [▶▶▶] In the "Key Value Pair" structure workorderUploadKeyValues: key CUSTOMER_WORKORDER_NUMBER added
	»mdataCreateSpecialTime« [▶▶▶] Output value (error code) -1139 added
	»mdataGetDetectionPoint« [▶▶▶] Data capture point type "measurement data" added to the function (detectionPointType = 1))
	»mdataGetMachineAssetStructure« [▶▶▶] Output value (error code) -168 added
	»mdataGetOrderCluster« [▶▶▶] In the "Key Value Pair" structure orderClusterFilter, key CUSTOMER_WORKORDER_NUMBER added
	»mdataGetShiftData« [▶▶▶] In the "Key Value Pair" structure shiftFilterKeys: key OVERLAPPING added and ONLY_PRODUCTIVE adapted
	»mdataGetWorkorders« [▶▶▶] In the "Key Value Pair" structures mdataGetWorkordersFilters and mdataGetWorkordersResultKeys: key CUSTOMER_WORKORDER_NUMBER added
	»mdataGetWorkplanData« [▶▶▶] In the "Key Value Pair" structure workplanDataResultKeys: keys EARLIEST_START_LAY, EARLIEST_START_PRODUCTION, EARLIEST_START_SETUP, EARLIEST_START_STRIPDOWN, EARLIEST_START_TRANSPORT, EARLIEST_START_WAIT, LATEST_START_LAY, LATEST_START_PRODUCTION, LATEST_START_SETUP, LATEST_START_STRIPDOWN, LATEST_START_TRANSPORT and LATEST_START_WAIT added and key WORKSTEP_OVERALL_STATE adapted
	»mdataUpdateOrderCluster« [▶▶▶] In the "Key Value Pair" structure createOrderClusterItemUploadKeys, key CUSTOMER_WORKORDER_NUMBER added
	»mdcGetConditionCodes« [▶▶▶] In the "Key Value Pair" structure conditionCodeResultKeys, key CONDITION_CAUSE adapted
	»mdcGetKeyPerformanceIndicators« [▶▶▶] In the "Key Value Pair" structure mdcConditionTimeResultKeys: key CONDITION_CAUSE adapted; amended reference to server parameter maximumDays
	»mdcGetStationConditionsByFilter« [▶▶▶] In the "Key Value Pair" structure conditionResultFilter: key IS_OPEN added
	»mdcManageConditionCode« [▶▶▶] In the "Key Value Pair" structure manageConditionCodeUploadKeys, key CONDITION_CAUSE adapted
	»mdcUploadDetectionPoint« [▶▶▶] Licensing changed from "PM license" to "TR license (active) OR PM license"; output value (error code) -102 removed and -1232, -1233, -1234, -1235, -2105 added; in the "Key Value Pair" structure detectionPointKeys: keys BOOK_DATE and MEASURE_VALUE added

Document	Adapted function descriptions
	»mlChangeMaterialBinData« [▶▶▶] Amended reference for site parameter code no. 18520 [▶▶▶]
	»mlCreateNewMaterialBin« [▶▶▶] Output values (error code) 7, -122 and -2052 added; in the "Key Value Pair" structure materialBinUploadKeys: keys PART_GROUP and PARTFORM_NAME added; reference to site parameter code no. 18520 [▶▶▶] amended
	»pmGetBooking« [▶▶▶] In the "Key Value Pair" structure pmBookingResultKeys, key CONDITION_CAUSE adapted
	»regLogin« [▶▶▶] Amended reference for site parameter code no. 18100 [▶▶▶]
	»regRegisterUser« [▶▶▶] Amended reference for site parameter code no. 18100 [▶▶▶]
	»setupCheck« [▶▶▶] Output value (error code) -2073 added
	»setupSearchPreSetup« [▶▶▶] Output values (error code) -121 and -130 removed
	»setupStateChange« [▶▶▶] Function description adapted; amended reference to station parameter code nos. 7132 [▶▶▶] and 13100 [▶▶▶]; output values (error code) -241, -603, -604, -1105, -2037 and -2073 added
	»setupUpdateMaterialSetup« [▶▶▶] Output value (error code) -2073 added
	»trActivateWorkorder« [▶▶▶] »trActivateWorkstep« [▶▶▶] Output value (error code) -2073 added
	»trAssignSerialNumberMergeAndUploadState« [▶▶▶] Description of the input parameter multiplePanel adapted; amended reference to station parameter code no. 13400 [▶▶▶]
	»trChangeWorkOrder« [▶▶▶] Output value (error code) -2073 added
	»trCheckMergedPartsForSerialNumberComplete« [▶▶▶] Description of the input parameter allMergedPartsVisible adapted
	»trGetStationResult« [▶▶▶] In the "Key Value Pair" structure detectionPointKeys: keys DETECTION_POINT_NAME, RESOURCE_DESC and RESOURCE_NAME added
	»trGetStationSetting« [▶▶▶] In the "Key Value Pair" structure stationSettingResultKeys: keys EARLIEST_START_LAY, EARLIEST_START_PRODUCTION, EARLIEST_START_SETUP, EARLIEST_START_STRIPDOWN, EARLIEST_START_TRANSPORT, EARLIEST_START_WAIT, LATEST_START_LAY, LATEST_START_PRODUCTION, LATEST_START_SETUP, LATEST_START_STRIPDOWN, LATEST_START_TRANSPORT and LATEST_START_WAIT added

Document	Adapted function descriptions
	»trGetWorkOrderForStation« [▶▶▶] In the "Key Value Pair" structure <code>workorderResultKeys</code>: keys <code>EARLIEST_START_LAY</code>, <code>EARLIEST_START_PRODUCTION</code>, <code>EARLIEST_START_SETUP</code>, <code>EARLIEST_START_STRIPDOWN</code>, <code>EARLIEST_START_TRANSPORT</code>, <code>EARLIEST_START_WAIT</code>, <code>LATEST_START_LAY</code>, <code>LATEST_START_PRODUCTION</code>, <code>LATEST_START_SETUP</code>, <code>LATEST_START_STRIPDOWN</code>, <code>LATEST_START_TRANSPORT</code> and <code>LATEST_START_WAIT</code> added »trMergeParts« [▶▶▶] Description of the <code>duplicateSerialNumber</code> input parameter adapted
	»trUploadActivity« [▶▶▶] Output value (error code) -1303 added »trUploadFailureAndResultData« [▶▶▶] Output values (error code) -222 and -1303 added; amended reference for station parameter code no. 8401 [▶▶▶]
	»trUploadFailuresForMergedParts« [▶▶▶] Amended reference for station parameter code no. 8401 [▶▶▶] »trUploadResultDataAndRecipe« [▶▶▶] Output values (error code) -367, -738 and -1303 added; in the "Key Value Pair" structure <code>resultUploadKeys</code>: key <code>SEQUENCE_NUMBER</code> added; amended reference for station parameter code no. 8401 [▶▶▶]
	»trUploadState« [▶▶▶] Output value (error code) -1303 added; amended reference for station parameter code no. 8401 [▶▶▶] »trUploadStationResult« [▶▶▶] In the "Key Value Pair" structure <code>stationResultUploadKeys</code>, key <code>SEQUENCE_NUMBER</code> added
	»trUploadTestplanResult« [▶▶▶] Output value (error code) -1303 added »trVerifyMerge« [▶▶▶] Description of the <code>checkMultiBoard</code> input parameter adapted
IMSAPI_R9_00_V1_0_EN	»apsGetWorkorderDetails« [▶▶▶] Output value (error code) -3600 added; in the "Key Value Pair" structure <code>apsGetWorkorderDetailsFilters</code>, keys <code>WORKSTEP_STATE</code> and <code>WORKSTEP_STATION_STATE</code> added; in the "Key Value Pair" structure <code>apsGetWorkorderDetailResultKeys</code>, keys <code>OPTIMIZATION_GROUP_ID</code>, <code>PROGRESS_PERCENT</code> and <code>WORKSTEP_STATION_STATE</code> added as well as <code>WORKSTEP_STATE</code> adapted
	»attribGetAttributeValues« [▶▶▶] »attribGetAttributeValuesForObjectType« [▶▶▶] Amended reference for site parameter code no. 18501 [▶▶▶] »attribGetObjectsForAttributeValues« [▶▶▶] In the "Key Value Pair" structure <code>objectResultKeys</code>: key <code>EXPIRATION_DATE_FINAL</code> added for objectType 2
	»batchUnregisterBatch« [▶▶▶] Amended reference for server parameter code no. 19200 [▶▶▶]

Document	Adapted function descriptions
	»equCheckEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentResultKeys: keys WORKPLAN_ALT, WORKPLAN_NUMBER, WORKPLAN_TYPE, WORKPLAN_VERS, WORKPLAN_VERS_ERP, WORKSTEP_AVO and WORKSTEP_NUMBER added
	»equGetRequiredEquipmentData« [▶▶▶] Output value (error code) -128 added; in the "Key Value Pair" structure equipmentResultKeys: keys WORKPLAN_ALT, WORKPLAN_NUMBER, WORKPLAN_TYPE, WORKPLAN_VERS, WORKPLAN_VERS_ERP, WORKSTEP_AVO and WORKSTEP_NUMBER added; description of the function parameters partNumber and processLayer adapted
	»equUpdateEquipment« [▶▶▶] Amended reference for site parameter code no. 18309 [▶▶▶]
	»equUpdateEquipmentData« [▶▶▶] Amended reference for station parameter code no. 8410 [▶▶▶] and site parameter code no. 13050 [▶▶▶]
	»mdaActivateDocument« [▶▶▶] Output value (error code) -736 added
	»mdaGetRecipeHeaderAndVersion« [▶▶▶] In the "Key Value Pair" structure objectResultKeys: description of the keys BOM_VERSION, PROCESS_VERSION, PART_GROUP, WORKORDER_NUMBER adapted for ObjectType 3 and mandatory field designation set for PART_NUMBER
	»mdaCreateWorkorder« [▶▶▶] Output value (error code) -137 added
	»mdaGetBomData« [▶▶▶] In the "Key Value Pair" structure bomDataFilter: keys PROCESS_LAYER and WORKSTEP_NUMBER added
	»mdaGetCalendarData« [▶▶▶] Amended reference for station parameter code no. 13200 [▶▶▶]
	»mdaGetMachineAssetStructure« [▶▶▶] Output value (error code) -2013, -2139, -2144, -2145, -2146, -2147 and -2148 added; output value (error code) -1209 and -1224 removed; in the "Key Value Pair" structure machineAssetStructureFilter: keys ONLY_ACTIVE and ONLY_VALID added as well as RESOURCE_NAME, STATION_NUMBER and STATION_TYPE adapted; in the "Key Value Pair" structure machineAssetStructureResultKeys: keys CELL_ACTIVE, ERP_GROUP_VALID, MACHINE_GROUP_ACTIVE, MACHINE_GROUP_END_DATE, PARENT_RESOURCE_ACTIVE, RESOURCE_ACTIVE, STATION_END_DATE, STATION_ERP_END_DATE and STATION_ACTIVE added
	»mdaGetProductInfo« [▶▶▶] Amended reference for site parameter code no. 18600 [▶▶▶]
	»mdaGetWorkplanActivities« [▶▶▶] »mdaGetWorkplanData« [▶▶▶] Output values (error codes) -338 and -366 added
	»mdcAcknowledgeStationConditions« [▶▶▶] In the "Key Value Pair" structure stationConditionAcknowledgeKeys: keys DATE_FROM and DATE_TO added for the selection types 2, 3, 4

Document	Adapted function descriptions
	»mdcGetConditionCodes« [▶▶▶] In the "Key Value Pair" structure conditionCodeFilter: key DYNAMIC_MESSAGE_ENABLED added; in the "Key Value Pair" structure conditionCodeResultKeys: keys DESCRIPTION_LONG and DYNAMIC_MESSAGE_ENABLED added
	»mdcGetStationConditions« [▶▶▶] In the "Key Value Pair" structure conditionResultKeys: key DESCRIPTION_LONG added for objectType 1; in the "Key Value Pair" structure conditionResultKeys: keys DESCRIPTION_LONG, DYNAMIC_CONTENT, DYNAMIC_MESSAGE and DYNAMIC_MESSAGE_ENABLED added for objectType 2
	»mdcGetStationConditionsByFilter« [▶▶▶] In the "Key Value Pair" structure conditionResultFilter: key DYNAMIC_MESSAGE_ENABLED added; in the "Key Value Pair" structure conditionResultKeys: key DESCRIPTION_LONG added for objectType 1; in the "Key Value Pair" structure conditionResultKeys: keys DESCRIPTION_LONG, DYNAMIC_CONTENT, DYNAMIC_MESSAGE and DYNAMIC_MESSAGE_ENABLED added for objectType 2
	»mdcManageConditionCode« [▶▶▶] In the "Key Value Pair" structure manageConditionCodeUploadKeys, key DESCRIPTION_LONG added
	»mdcUploadDetectionPoint« [▶▶▶] In the "Key Value Pair" structure detectionPointKeys: keys CONDITION_CODE and DYNAMIC_CONTENT added
	»mdcUploadStationCondition« [▶▶▶] In the "Key Value Pair" structure stationConditionUploadKeys: keys DESCRIPTION_LONG and DYNAMIC_CONTENT added
	»mlChangeMaterialBinData« [▶▶▶] Output value (error code) -578 added; in the "Key Value Pair" structure materialBinDataUploadValues, key EXPIRATION_DATE_FINAL added
	»mlCreateNewMaterialBin« [▶▶▶] Output values (error code) 3108 and -578 added; in the "Key Value Pair" structure materialBinUploadKeys: key EXPIRATION_DATE_FINAL added
	»mlGetMaterialBinData« [▶▶▶] In the "Key Value Pair" structure materialBinResultKeys: key EXPIRATION_DATE_FINAL adapted
	»mlSplitMaterialBin« [▶▶▶] Output value (error code) -578 added; in the "Key Value Pair" structure splitMaterialBinResultKeys, key EXPIRATION_DATE_FINAL added
	»pmGetBooking« [▶▶▶] In the "Key Value Pair" structure pmBookingResultKeys, keys PROGRESS_PERCENT, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»pmGetFailureData« [▶▶▶] In the "Key Value Pair" structure failureDataResultKeys, keys PROGRESS_PERCENT, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»pmGetProductQuantity« [▶▶▶] In the "Key Value Pair" structure productQuantityResultKeys, keys PROGRESS_PERCENT, QUANTITY_OPEN and WORKSTEP_STATION_STATE added
	»pmUpdateBooking« [▶▶▶] In the "Key Value Pair" structure pmUpdateBookingData: key FUNC_MODE added

Document	Adapted function descriptions
	»pmUploadBooking« [▶▶▶] In the "Key Value Pair" structure pmUpdateBookingData: key FUNC_MODE added; amended reference for server parameter code no. 19200 [▶▶▶]
	»pmUploadQuantityAndFailureData« [▶▶▶] »pmUploadQuantityFailureTallyCardAndFailureData« [▶▶▶] Amended reference for server parameter code no. 19200 [▶▶▶]
	»regLogin« [▶▶▶] Application type "D" (login as station using user token) added to the function description; in this context, the input parameters registrationType and password have been adapted
	»regRegisterUser« [▶▶▶] »regUnregisterUser« [▶▶▶] Description of the input parameters userName and password adapted
	»setupAttachPreSetup« [▶▶▶] In the "Key Value Pair" structure presetupHeadFilter: keys CONSUMER_ID, PRODUCT_NUMBER and WORKORDER_NUMBER adapted
	»setupChangeSetupForSerialNumber« [▶▶▶] In the "Key Value Pair" structure setupChangePartKeys: mandatory field designation removed for the key MATERIAL_BIN_NUMBER
	»setupGetSetupDataBySerialNumber« [▶▶▶] »setupGetSetupDataBySerialNumberRef« [▶▶▶] In the "Key Value Pair" structure setupResultKeys, key EXPIRATION_DATE_FINAL added
	»setupStateChange« [▶▶▶] Output value (error code) 608 added
	»shipRemoveSerialNumberFromShippingLot« [▶▶▶] Amended reference for site parameter code no. 18401 [▶▶▶]
	»smtCheckMaterialBin« [▶▶▶] In the "Key Value Pair" structure materialBinUploadKeys: key EXPIRATION_DATE_FINAL added
	»trActivateOrderCluster« [▶▶▶] In the "Key Value Pair" structure orderClusterActivateData, key WORKSTEP_FLAG added
	»trActivateWorkorder« [▶▶▶] Description of the flag input parameter adapted
	»trActivateWorkstep« [▶▶▶] In the "Key Value Pair" structure activateWorkstepKeys, key WORKSTEP_STATE_FLAG added
	»trAssignSerialNumberForProduktOrWorkOrder« [▶▶▶] Description of the activateWorkOrder input parameter adapted
	»trChangeWorkOrder« [▶▶▶] Description of the lineFlag input parameter adapted
	»trCheckMergedPartsForSerialNumber« [▶▶▶] »trCheckMergedPartsForSerialNumberComplete« [▶▶▶] Amended reference for site parameter code no. 24010 [▶▶▶]

Document	Adapted function descriptions
	»trGetMultiStationSettings« [▶▶▶] In the "Key Value Pair" structure stationMultiSettingsResultKeys, keys PROGRESS_PERCENT, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»trGetNextProductionStep« [▶▶▶] In the "Key Value Pair" structure productionStepResultKeys, keys PROGRESS_PERCENT and WORKSTEP_STATION_STATE added
	»trGetProductQuantity« [▶▶▶] Function description adapted; in the "Key Value Pair" structure productQuantityResultKeys: keys PROGRESS_PERCENT, QUANTITY_OPEN and WORKSTEP_STATION_STATE added
	»trGetSerialNumberActivityUploadInfo« [▶▶▶] In the "Key Value Pair" structure activityDataResultKeys, keys PROGRESS_PERCENT, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»trGetStationSetting« [▶▶▶] In the "Key Value Pair" structure stationSettingsResultKeys, keys PROGRESS_PERCENT, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»trGetWorkOrderForStation« [▶▶▶] In the "Key Value Pair" structure workorderFilter: keys WORKSTEP_STATE and WORKSTEP_STATION_STATE added; in the "Key Value Pair" structure workorderResultKeys: keys PROCESS_LAYER, PROGRESS_PERCENT, WORKSTEP_AVO, WORKSTEP_DESC, WORKSTEP_NUMBER, WORKSTEP_STATE and WORKSTEP_STATION_STATE added
	»trUploadActivity« [▶▶▶] Amended reference for server parameter code no. 19200 [▶▶▶]
	»trUploadFailureAndResultData« [▶▶▶] »trUploadFailuresForMergedParts« [▶▶▶] Amended reference for site parameter code no. 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]
	»trUploadPmQuantity« [▶▶▶] Amended reference for server parameter code no. 19200 [▶▶▶]
	»trUploadResultDataAndRecipe« [▶▶▶] Description of the parameter duplicateSerialNumber adapted; amended reference for site parameter code no. 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]
	»trUploadState« [▶▶▶] Amended reference for site parameter code no. 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]
	»trUploadStationResult« [▶▶▶] In the "Key Value Pair" structure stationResultUploadKeys, key BOOK_DATE added
	»trUploadTestplanResult« [▶▶▶] Amended reference for site parameter code no. 18210 [▶▶▶] and server parameter code no. 19200 [▶▶▶]
	»trVerifyMerge« [▶▶▶] »trVerifyMergeProduct« [▶▶▶] Amended reference for station parameter code no. 7219 [▶▶▶]

Document	Adapted function descriptions
IMSAPI_R8_52_V1_0_EN	General a.) Release 8.52.00 includes a new attribute-capable data object – the resource: cf. attribute category (objectType) 21. b.) In combination with the BigData-Service (replication), serial numbers can be deleted in the productive system; for this purpose, the output value (error code) –2138 has been added to the "serial-number-processing" IMSApi functions.
	»apsGetWorkorderDetails« [▶▶▶] In the "Key Value Pair" structure apsGetWorkorderDetailResultKeys, keys APS_END_DATE, APS_START_DATE, SPLIT_MAX, SPLIT_MIN and WORKORDER_SEQUENCE added
	»apsGetWorkorders« [▶▶▶] In the "Key Value Pair" structure apsGetWorkordersResultKeys, keys APS_END_DATE, APS_START_DATE and WORKORDER_SEQUENCE added
	»attribAppendAttributeValues« [▶▶▶] Output values (error codes) –950, –951, –952 and –2136 added and 904, 905 and 906 deleted
	»attribGetAttributeClass« [▶▶▶] Output value (error code) –168 added; in the "Key Value Pair" structures attributeClassFilter and classResultKeys, key ACTIVE added
	»attribGetAttributeValues« [▶▶▶] Output value (error code) –2136 added
	»batchGetBatchInfo« [▶▶▶] In the "Key Value Pair" structure batchInfoResultKeys for the keys BATCH_QUANTITY_CURRENT and BATCH_QUANTITY_TOTAL, the data type changed from int to double
	»batchGetRegisteredBatch« [▶▶▶] In the "Key Value Pair" structure batchResultKeys for the keys BATCH_QUANTITY_CURRENT and BATCH_QUANTITY_TOTAL, the data type changed from int to double
	»crpCreateTransportOrder« [▶▶▶] Output value (error code) –3535 added
	»crpManageRoute« [▶▶▶] Output values (error codes) 5 added and –2120 removed
	»edaAssignExternalData« [▶▶▶] Output value (error code) –2138 added
	»edaGetExternalDataForObject« [▶▶▶] Output value (error code) –2138 added
	»equCheckEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentResultKeys key WORKORDER_NUMBER added; amended reference for site parameter code no. 4570 [▶▶▶]
	»equGetRequiredEquipmentData« [▶▶▶] In the "Key Value Pair" structure equipmentResultKeys key WORKORDER_NUMBER added; amended reference for site parameter code no. 4570 [▶▶▶]
	»equUpdateEquipment« [▶▶▶] Output values (error codes) 7 and 1304 added

Document	Adapted function descriptions
	»execRuleSet« [▶▶▶] Description of the input parameters <code>ruleType</code> and <code>ruleProperties</code> adapted »lockObjects« [▶▶▶] Output value (error code) -2138 added
	»lockUnlockObjects« [▶▶▶] Output value (error code) -2138 added »mdaCreateRecipe« [▶▶▶] In the "Key Value Pair" structure <code>recipeFilters</code> for the object types 0, 2, 3 and 5, key <code>WORKORDER_NUMBER</code> added; as a result, mandatory field designations have been removed there
	»mdaGetDocuments« [▶▶▶] In the "Key Value Pair" structure <code>attributeFilters</code>, key <code>RESOURCE_NAME</code> and in the "Key Value Pair" structure <code>mdaResultKeys</code>, keys <code>DATE_CHANGED</code> and <code>DATE_CREATED</code> added »mdaGetRecipeData« [▶▶▶] Output values (error codes) -130 and -164 added; in the "Key Value Pair" structure <code>recipeFilters</code> for the object types 0, 2, 3 and 5, key <code>WORKORDER_NUMBER</code> added; as a result, mandatory field designations have been removed there
	»mdaGetRecipeHeaderAndVersion« [▶▶▶] In the "Key Value Pair" structure <code>recipeHeaderFilter</code> for the object types 0, 2, 3 and 5, key <code>WORKORDER_NUMBER</code> added; in the "Key Value Pair" structure <code>recipeHeader-ResultKeys</code>, key <code>BOM_VARIANT</code> added »mdaUploadDocument« [▶▶▶] Output value (error code) -113 added; addition of the data object "Resource" (<code>targetType = 126</code>); in the "Key Value Pair" structure <code>attributeValues</code> for the data object "Serial number" (<code>targetType = 122</code>), key <code>WORKORDER_NUMBER</code> added
	»mdaChangeWorkorderBomData« [▶▶▶] Output value (error code) -1127 added »mdaChangeWorkorderData« [▶▶▶] In the "Key Value Pair" structure <code>workorderChangeData</code>, key <code>WORKORDER_SEQUENCE</code> added
	»mdaCreateOrderCluster« [▶▶▶] In the "Key Value Pair" structure <code>createOrderClusterValues</code> key <code>CLUSTER_DESC</code> adapted »mdaCreateWorkorder« [▶▶▶] In the "Key Value Pair" structure <code>workorderUploadKeyValues</code>, key <code>WORKORDER_SEQUENCE</code> added
	»mdaGetBomData« [▶▶▶] In the "Key Value Pair" structure <code>bomDataFilter</code>, key <code>BOM_ALTERNATIVE</code> adapted

Document	Adapted function descriptions
	»mdataGetMachineAssetStructure« [▶▶▶] In the "Key Value Pair" structure machineAssetStructureFilter, keys DISSOLVING_LEVEL and FUNC_MODE adapted as well as RESOURCE_NAME added; in the "Key Value Pair" structure machineAssetStructureResultKeys, keys CELL_INFO_TXT_1 - CELL_INFO_TXT_10, CELL_PART_DESC, CELL_PART_NUMBER, LINE_INFO_TXT_1 - LINE_INFO_TXT_10, LINE_PART_DESC, LINE_PART_NUMBER, MACHINE_GROUP_PART_DESC, MACHINE_GROUP_PART_NUMBER, PARENT_RESOURCE_NAME, RESOURCE_DESC, RESOURCE_NAME, RESOURCE_PART_DESC, RESOURCE_PART_NUMBER, SITE_PART_DESC, SITE_PART_NUMBER, STATION_PART_DESC and STATION_PART_NUMBER added
	»mdataGetWorkorders« [▶▶▶] In the "Key Value Pair" structure mdataGetWorkordersResultKeys, key WORKORDER_SEQUENCE added
	»mdataGetWorkplanData« [▶▶▶] In the "Key Value Pair" structure workplanDataResultKeys key WORKSTEP_OVERALL_STATE added
	»mdcGetConditionCodes« [▶▶▶] In the "Key Value Pair" structure conditionCodeResultKeys key MESSAGE_TYPE_NAME added
	»mdcGetStationConditions« [▶▶▶] In the "Key Value Pair" structure conditionResultKeys, keys ACKNOWLEDGEMENT_DATE, ACKNOWLEDGEMENT_TXT, ACKNOWLEDGEMENT_USER_ID, ACTION_PLAN, CHANGE_DATE, CONDITION_ID, DETECTION_POINT_NAME, INFO_TXT_1 - INFO_TXT_15, MESSAGE_CAUSE_DESC, MESSAGE_CAUSE_NAME, MESSAGE_CLASS_DESC, MESSAGE_CLASS_NAME, MESSAGE_EFFECT_DESC, MESSAGE_EFFECT_NAME, MESSAGE_TYPE_DESC, MESSAGE_TYPE_NAME, PRIORITY, REMARK, RESOURCE_DESC, RESOURCE_NAME, STATION_DESC, STATION_NUMBER and USER_NAME added
	»mdcUploadStationCondition« [▶▶▶] Output value (error code) -2136 and -2140 added; in the "Key Value Pair" structure stationConditionUploadKeys, key RESOURCE_NAME added
	»mlUploadMaterialBinBooking« [▶▶▶] Output value (error code) -334 added; in the "Key Value Pair" structure materialBinBookingUploadKeys, key COMP_NAME added
	»pmGetProductQuantity« [▶▶▶] Amended reference for station parameter code no. 15270 [▶▶▶]
	»pmUpdateBooking« [▶▶▶] Amended reference for server parameter code no. 2020 [▶▶▶]
	»regLogin« [▶▶▶] Description of the input parameters stationNumber and user from the structure sessionValidationStruct adapted
	»shipGetSerialNumberDataForShippingLot« [▶▶▶] In the "Key Value Pair" structure serialNumberResultKeys, data type for the key BATCH_QUANTITY_CURRENT changed from int to double
	»setupGetSetupDataBySerialNumber« [▶▶▶] Output value (error code) -2138 added; in the "Key Value Pair" structure setupResultKeys, key SERIAL_NUMBER_POS added
	»shipCheckSerialNumberAddToShippingLot« [▶▶▶] Output value (error code) -2138 added

Document	Adapted function descriptions
	»shipGetShippingLotInfo« [▶▶▶] In the "Key Value Pair" structure shippingLotResultKeys: key SHIP-PING_LOT_LOT_QTY_ACTUAL added
	»smtSerialNumberCheckIn« [▶▶▶] Output value (error code) -213 added
	»trAddSerialNumberToPool« [▶▶▶] Output value (error code) -2138 added
	»trAddSerialNumberToSerialNumberPool« [▶▶▶] Output value (error code) -2138 added
	»trAssignSerialNumberForProductOrWorkOrder« [▶▶▶] Output value (error code) -2138 added
	»trAssignSerialNumberMergeAndUploadState« [▶▶▶] Output value (error code) -2138 added
	»trAssignSerialnumberToOrderCluster« [▶▶▶] Output value (error code) -2138 added
	»trCheckUserSkill« [▶▶▶] Output value (error code) -2138 added; amended reference for site parameter code no. 4571 [▶▶▶]
	»trCheckSerialNumberState« [▶▶▶] Output values (error code) -301 and -2138 added; in the "Key Value Pair" structure serialNumberStateResultKeys, the description of key IN_PROCESS adapted
	»trGetMergeHistory« [▶▶▶] Output value (error code) -2138 added
	»trGetMergeParts« [▶▶▶] Output value (error code) -2138 added
	»trGetResultDataForSerialNumber« [▶▶▶] Output value (error code) -2138 added
	»trGetSerialNumberActivityUploadInfo« [▶▶▶] Output value (error code) -2138 added
	»trGetSerialNumberBySerialNumberRef« [▶▶▶] Output values (error codes) 209, -203 and -2138 added
	»trGetSerialNumberForWorkorderAndWorkstep« [▶▶▶] Amended reference for station parameter code no. 15270 [▶▶▶]
	»trGetSerialNumberHistoryData« [▶▶▶] Output value (error code) -2138 added; in the "Key Value Pair" structure bookingResultKeys, key SERIAL_NUMBER_STATE_VALID added
	»trGetSerialNumberInfo« [▶▶▶] Output value (error code) -2138 added; in the "Key Value Pair" structure serialNumberResultKeys, key PROCESS_VERSION added

Document	Adapted function descriptions
	»trGetSerialNumberUploadInfo« [▶▶▶] Output value (error code) -2138 added; in the "Key Value Pair" structure uploadInfoResultKeys, keys BOM_VARIANT, BOM_VERSION, BOM_VERSION_ERP, PROCESS_VERSION, SERIAL_NUMBER_ACTUAL_STATE and STATION_DIAG_NUMBER added
	»trGetTestplanData« [▶▶▶] In the "Key Value Pair" structure varParamResultKeys, key SAMPLE_REASON adapted
	»trGetWorkOrderForStation« [▶▶▶] In the "Key Value Pair" structure workorderFilter, key SORT_FLAG added; in the "Key Value Pair" structure workorderResultKeys, keys APS_END_DATE, APS_START_DATE and WORKORDER_SEQUENCE added; amended reference for station parameter code no. 15270 [▶▶▶]
	»trMergeParts« [▶▶▶] Output value (error code) -2138 added
	»trSplitPanel« [▶▶▶] Output value (error code) -2138 added
	»trUploadActivity« [▶▶▶] Output value (error code) -2138 added
	»trUploadFailureAndResultData« [▶▶▶] Output value (error code) -2138 added
	»trUploadFailuresForMergedParts« [▶▶▶] Output value (error code) -2138 added
	»trUploadPmQuantity« [▶▶▶] Amended reference for server parameter code no. 2020 [▶▶▶]
	»trUploadResultDataAndRecipe« [▶▶▶] Output value (error code) -2138 added
	»trUploadState« [▶▶▶] Output value (error code) -2138 added
	»trUploadTestplanResult« [▶▶▶] Output value (error code) -2138 added
	»trVerifyMerge« [▶▶▶] Output values (error codes) -301 and -2138 added
	»trVerifyMergeProduct« [▶▶▶] Output value (error code) -2138 added

Tab. 27-3 Adaptation of existing function descriptions

Index

A

adviceConfirmAdvice		Parameter (in)	98
Error codes	60	Parameter (out)	98
Function declaration	59	apsGetWorkorderDetailsOptGroup	
Licensing	59	"Key Value Pair" structures	100
Parameter (in)	59	Error codes	113
adviceGetAdvice		Function declaration	100
"Key Value Pair" structures	61	Licensing	100
Error codes	64	Parameter (in)	111
Function declaration	61	Parameter (out)	112
Licensing	61	apsGetWorkorders	
Parameter (in)	62	"Key Value Pair" structures	114
Parameter (out)	63	Error codes	121
apsGetEquipments		Function declaration	114
"Key Value Pair" structures	65	Licensing	114
Error codes	71	Parameter (in)	120
Function declaration	65	Parameter (out)	120
Licensing	65	attribAppendAttributeValues	
Parameter (in)	70	"Key Value Pair" structures	126
Parameter (out)	70	Error codes	151
apsGetStations		Function declaration	126
"Key Value Pair" structures	72	Licensing	125
Error codes	76	Parameter (in)	149
Function declaration	72	Parameter (out)	150
Licensing	72	attribCheckAttributeValues	
Parameter (in)	75	"Key Value Pair" structures	156
Parameter (out)	75	Error codes	182
apsGetWorkergroups		Function declaration	156
"Key Value Pair" structures	77	Licensing	156
Error codes	81	Parameter (in)	180
Function declaration	77	Parameter (out)	181
Licensing	77	attribCreateAttribute	
Parameter (in)	80	Error codes	186
Parameter (out)	80	Function declaration	185
apsGetWorkers		Licensing	184
"Key Value Pair" structures	82	Parameter (in)	185
Error codes	86	attribCreateAttributeClass	
Function declaration	82	"Key Value Pair" structures	188
Licensing	82	Error codes	191
Parameter (in)	85	Function declaration	188
Parameter (out)	85	Licensing	187
apsGetWorkorderDetails		Parameter (in)	190
"Key Value Pair" structures	87	Parameter (out)	190
Error codes	99	attribCreateAttributeCode	
Function declaration	87	"Key Value Pair" structures	193
Licensing	87	Error codes	196
		Function declaration	193
		Licensing	193

Parameter (in)	195	Parameter (in)	255
Parameter (out)	196	Parameter (out)	256
attribCreateAttributeCodeByCatalog		attribGetAttributeValues	
"Key Value Pair" structures	198	"Key Value Pair" structures	261
Error codes	200	Error codes	267
Function declaration	197	Function declaration	261
Licensing	197	Licensing	261
Parameter (in)	199	Parameter (in)	285
attribCreateAttributeCondition		Parameter (out)	286
"Key Value Pair" structures	201	attribGetAttributeValuesForObjectType	
Error codes	204	"Key Value Pair" structures	291
Function declaration	201	Error codes	296
Licensing	201	Function declaration	291
Parameter (in)	203	Licensing	291
Parameter (out)	203	Parameter (in)	295
attribEditAttributeClassState		Parameter (out)	295
"Key Value Pair" structures	205	attribGetObjectsForAttributeValues	
Error codes	207	"Key Value Pair" structures	299
Function declaration	205	Error codes	337
Licensing	205	Function declaration	299
Parameter (in)	206	Licensing	298
attribGetAllAttributeValue		Parameter (in)	335
"Key Value Pair" structures	210	Parameter (out)	336
Error codes	217	attribManageAttributeClassAttribute	
Function declaration	210	"Key Value Pair" structures	339
Licensing	210	Error codes	342
Parameter (in)	216	Function declaration	339
Parameter (out)	216	Licensing	338
attribGetAttributeClass		Parameter (in)	341
"Key Value Pair" structures	219	Parameter (out)	341
Error codes	227	attribManageAttributeClassCondition	
Function declaration	219	"Key Value Pair" structures	344
Licensing	219	Error codes	348
Parameter (in)	225	Function declaration	344
Parameter (out)	226	Licensing	343
attribGetAttributeCode		Parameter (in)	347
"Key Value Pair" structures	229	Parameter (out)	347
Error codes	233	attribManageAttributeCondition	
Function declaration	229	"Key Value Pair" structures	350
Licensing	229	Error codes	354
Parameter (in)	232	Function declaration	350
Parameter (out)	232	Licensing	349
attribGetAttributeCondition		Parameter (in)	353
"Key Value Pair" structures	234	Parameter (out)	353
Error codes	238	attribRemoveAttributeClassValue	
Function declaration	234	"Key Value Pair" structures	357
Licensing	234	Error codes	360
Parameter (in)	237	Function declaration	357
Parameter (out)	237	Licensing	357
attribGetAttributeHistory		Parameter (in)	359
"Key Value Pair" structures	241	Parameter (out)	359
Error codes	257	attribRemoveAttributeValue	
Function declaration	241	Error codes	366
Licensing	241	Function declaration	364

Licensing	364
Parameter (in)	364
attribUpdateAttributeCode	
"Key Value Pair" structures	369
Error codes	372
Function declaration	369
Licensing	369
Parameter (in)	371
Parameter (out)	371

B

batchAssignBatchNumberToWorkOrder	
Error codes	376
Function declaration	374
Licensing	374
Parameter (in)	374
batchGetBatchInfo	
"Key Value Pair" structures	378
Error codes	382
Function declaration	378
Licensing	378
Parameter (in)	381
Parameter (out)	381
batchGetRegisteredBatch	
"Key Value Pair" structures	383
Error codes	385
Function declaration	383
Licensing	383
Parameter (in)	384
Parameter (out)	384
batchMergeBatch	
Error codes	388
Function declaration	386
Licensing	386
Parameter (in)	387
batchRegisterBatch	
Error codes	391
Function declaration	389
Licensing	389
Parameter (in)	390
batchSplitBatchNumberToSerialNumber	
Error codes	394
Function declaration	392
Licensing	392
Parameter (in)	392
Parameter (inout)	393
batchUnregBatch	
"Key Value Pair" structures	395
Error codes	398
Function declaration	395
Licensing	395
Parameter (in)	397
batchUnregisterBatch	

Error codes	401
Function declaration	399
Licensing	399
Parameter (in)	400

C

configCreateParameters	
"Key Value Pair" structures	403
Error codes	411
Function declaration	403
Parameter (in)	410
Parameter (out)	410
configDeleteParameters	
"Key Value Pair" structures	412
Error codes	417
Function declaration	412
Parameter (in)	416
Parameter (out)	416
configDeleteValues	
"Key Value Pair" structures	418
Error codes	422
Function declaration	418
Parameter (in)	421
Parameter (out)	421
configGetDimensionValues	
"Key Value Pair" structures	423
Error codes	425
Function declaration	423
Parameter (in)	424
Parameter (out)	424
configGetParameters	
"Key Value Pair" structures	426
Error codes	433
Function declaration	426
Parameter (in)	432
Parameter (out)	432
configGetValues	
"Key Value Pair" structures	434
Error codes	442
Function declaration	434
Parameter (in)	440
Parameter (out)	441
configUpdateParameters	
"Key Value Pair" structures	448
Error codes	456
Function declaration	448
Parameter (in)	455
Parameter (out)	455
configUpdateValues	
"Key Value Pair" structures	443
Error codes	447
Function declaration	443
Parameter (in)	446

Parameter (out)	446	Parameter (out)	510
crpChangeTransportOrder		customGetFunctionInfo	
"Key Value Pair" structures	458	"Key Value Pair" structures	515
Error codes	465	Definition of "metadata"	512
Function declaration	457	Error codes	519
Licensing	457	Function declaration	515
Parameter (in)	463	Licensing	514
Parameter (out)	464	Parameter (in)	517
crpCreateTransportOrder		Parameter (out)	518
"Key Value Pair" structures	467	customGetFunctions	
Error codes	473	"Key Value Pair" structures	520
Function declaration	467	Error codes	523
Licensing	467	Function declaration	520
Parameter (in)	471	Licensing	520
crpGetRoute		Parameter (in)	522
"Key Value Pair" structures	475	Parameter (out)	522
Error codes	481	customModifyDbObjectData	
Function declaration	475	"Key Value Pair" structures	525
Licensing	475	Error codes	531
Parameter (in)	479	Example "DELETE"	528
Parameter (out)	479	Example "INSERT"	528
crpGetTransportOrder		Example "UPDATE"	528
"Key Value Pair" structures	482	Function declaration	525
Error codes	490	Licensing	525
Function declaration	482	Parameter (in)	529
Licensing	482	Parameter (out)	530
Parameter (in)	488		
Parameter (out)	489	E	
crpManageRoute			
"Key Value Pair" structures	491	edaAssignExternalData	
Error codes	496	"Key Value Pair" structures	533
Function declaration	491	Error codes	535
Licensing	491	Function declaration	533
Parameter (in)	494	Licensing	533
Parameter (out)	496	Parameter (in)	534
customFunction		edaGetExternalDataForObject	
Error codes	501	"Key Value Pair" structures	537
Function declaration	499	Error codes	541
Parameter (in)	500	Function declaration	537
Parameter (out)	500	Licensing	537
customGetDbObjectData		Parameter (in)	540
"Key Value Pair" structures	503	Parameter (out)	540
Error codes	506	edaGetNextFreeExternalData	
Example "SELECT"	502	"Key Value Pair" structures	542
Function declaration	503	Error codes	545
Licensing	502	Function declaration	542
Parameter (in)	505	Licensing	542
Parameter (out)	506	Parameter (in)	544
customGetDbObjectMetaData		Parameter (out)	545
"Key Value Pair" structures	508	edaGetObjectsForExternalData	
Error codes	511	"Key Value Pair" structures	547
Function declaration	508	Error codes	550
Licensing	508	Function declaration	547
Parameter (in)	510	Licensing	547

Parameter (in)	549	Licensing	590
Parameter (out)	549	Parameter (in)	594
edaManageExternalData		Parameter (out)	594
"Key Value Pair" structures	551	equGetCarrierAssigned	
Error codes	554	"Key Value Pair" structures	596
Function declaration	551	Error codes	601
Licensing	551	Function declaration	596
Parameter (in)	553	Licensing	596
equAssignCarrierToCarrier		Parameter (in)	600
"Key Value Pair" structures	557	Parameter (out)	601
Error codes	560	equGetCarrierAssignmentHistoryData	
Function declaration	557	"Key Value Pair" structures	603
Licensing	557	Error codes	608
Parameter (in)	559	Function declaration	603
Parameter (out)	559	Licensing	603
equAssignCavityToEquipment		Parameter (in)	607
"Key Value Pair" structures	562	Parameter (out)	607
Error codes	565	equGetCarrierAssignRule	
Function declaration	562	"Key Value Pair" structures	611
Licensing	562	Error codes	616
Parameter (in)	564	Function declaration	610
Parameter (out)	564	Licensing	610
equAssignSerialNumberToCarrier		Parameter (in)	614
"Key Value Pair" structures	567	Parameter (out)	615
Error codes	570	equGetCarrierLocation	
Function declaration	567	"Key Value Pair" structures	618
Licensing	567	Error codes	621
Parameter (in)	569	Function declaration	618
Parameter (out)	569	Licensing	618
equCheckEquipmentData		Parameter (in)	620
"Key Value Pair" structures	572	Parameter (out)	621
Error codes	577	equGetCarrierToCarrier	
Function declaration	571	"Key Value Pair" structures	623
Licensing	571	Error codes	626
Parameter (in)	576	Function declaration	623
Parameter (out)	576	Licensing	623
equCreateEquipment		Parameter (in)	625
"Key Value Pair" structures	578	Parameter (out)	625
Error codes	583	equGetCarrierToSerialNumber	
Function declaration	578	"Key Value Pair" structures	628
Licensing	578	Error codes	631
Parameter (in)	582	Function declaration	628
Parameter (out)	582	Licensing	628
equGetAssetEquipmentSetup		Parameter (in)	630
"Key Value Pair" structures	585	Parameter (out)	630
Error codes	588	equGetCavityEquipmentAssignRule	
Function declaration	585	"Key Value Pair" structures	633
Licensing	584	Error codes	636
Parameter (in)	587	Function declaration	632
Parameter (out)	587	Licensing	632
equGetAssignedCavity		Parameter (in)	635
"Key Value Pair" structures	590	Parameter (out)	635
Error codes	595	equGetEquipment	
Function declaration	590	"Key Value Pair" structures	637

Error codes	644	equRemoveCarrierFromCarrier	
Function declaration	637	"Key Value Pair" structures	682
Licensing	637	Error codes	685
Parameter (in)	643	Function declaration	682
Parameter (out)	643	Licensing	682
equGetEquipmentLocationHist		Parameter (in)	684
"Key Value Pair" structures	645	Parameter (out)	684
Error codes	649	equRemoveCavityFromEquipment	
Function declaration	645	"Key Value Pair" structures	686
Licensing	645	Error codes	689
Parameter (in)	648	Function declaration	686
Parameter (out)	648	Licensing	686
equGetPartCavityAssignRule		Parameter (in)	688
"Key Value Pair" structures	650	Parameter (out)	688
Error codes	654	equRemoveEquipment	
Function declaration	650	Error codes	691
Licensing	650	Function declaration	690
Parameter (in)	653	Licensing	690
Parameter (out)	653	Parameter (in)	690
equGetRequiredEquipmentData		equRemoveSerialNumberFromCarrier	
"Key Value Pair" structures	656	"Key Value Pair" structures	692
Error codes	660	Error codes	695
Function declaration	655	Function declaration	692
Licensing	655	Licensing	692
Parameter (in)	658	Parameter (in)	694
Parameter (out)	659	Parameter (out)	694
equGetRequiredEquipmentDetail		equReportQuantity	
"Key Value Pair" structures	662	"Key Value Pair" structures	697
Error codes	667	Error codes	701
Function declaration	661	Function declaration	697
Licensing	661	Licensing	697
Parameter (in)	666	Parameter (in)	700
Parameter (out)	666	Parameter (out)	700
equGetSetupEquipmentData		equSetEquipmentLocation	
"Key Value Pair" structures	668	"Key Value Pair" structures	704
Error codes	672	Error codes	707
Function declaration	668	Function declaration	704
Licensing	668	Licensing	703
Parameter (in)	671	Parameter (in)	706
Parameter (out)	671	Parameter (out)	706
equManageCavityEquipmentRule		equSetLocation	
"Key Value Pair" structures	674	Error codes	710
Error codes	676	Function declaration	708
Function declaration	673	Licensing	708
Licensing	673	Parameter (in)	709
Parameter (in)	675	equSetupAssetEquipment	
Parameter (out)	675	"Key Value Pair" structures	712
equManagePartCavityRule		Error codes	716
"Key Value Pair" structures	677	Function declaration	712
Error codes	680	Licensing	712
Function declaration	677	Parameter (in)	715
Licensing	677	Parameter (out)	715
Parameter (in)	679	equUpdateAssignedCavity	
Parameter (out)	679	"Key Value Pair" structures	719

Error codes	722	IMSApi mandatory fields	41
Function declaration	718	Logging "IMSApi service"	41
Licensing	718	Technical error codes (ADVCE)	2015
Parameter (in)	721	Technical error codes (APS)	2032
Parameter (out)	721	Technical error codes (ATTRIB)	2013
equUpdateEquipment		Technical error codes (COMMON)	1986
"Key Value Pair" structures	724	Technical error codes (CONFIG)	2028
Error codes	728	Technical error codes (CRP)	2030
Function declaration	724	Technical error codes (CUSTOM)	2024
Licensing	724	Technical error codes (EDA)	2027
Parameter (in)	727	Technical error codes (EQU)	2020
Parameter (out)	727	Technical error codes (IMSAPI)	2023
equUpdateEquipmentData		Technical error codes (MDA)	2011
"Key Value Pair" structures	729	Technical error codes (MDATA)	2016
Error codes	732	Technical error codes (MDC)	2018
Function declaration	729	Technical error codes (ML)	2006
Licensing	729	Technical error codes (MSG)	2033
execFormula		Technical error codes (MSL)	2026
"Key Value Pair" structures	735	Technical error codes (PM)	2029
Error codes	740	Technical error codes (SETUP)	2009
Function declaration	735	Technical error codes (SHIP)	2003
Licensing	735	Technical error codes (SMT)	2025
Parameter (in)	738	Technical error codes (TR)	1996
Parameter (out)	739	Technological error codes	1984
execFormulaCatalog		Test option	38
"Key Value Pair" structures	741	imsapiGetErrorList	
Error codes	744	"Key Value Pair" structures	753
Function declaration	741	Error codes	756
Licensing	741	Function declaration	753
Parameter (in)	743	Parameter (in)	754
Parameter (out)	743	Parameter (out)	756
execRuleSet		imsapiGetErrorText	
Error codes	747	Error codes	758
Function declaration	745	Function declaration	757
Licensing	745	Parameter (in)	757
Parameter (in)	745	Parameter (out)	757
Parameter (out)	746	imsapiGetFunctionInfo	
execTimerProcess		Error codes	760
"Key Value Pair" structures	748	Function declaration	759
Error codes	751	Parameter (in)	759
Function declaration	748	Parameter (out)	759
Licensing	748	imsapiGetFunctions	
Parameter (in)	750	Error codes	762
Parameter (out)	750	Function declaration	761
		Parameter (in)	761
		Parameter (out)	761
		imsapiGetGroups	
		Error codes	764
		Function declaration	763
		Parameter (in)	763
		Parameter (out)	763
		imsapiGetParameterConditions	
		Error codes	766
		Function declaration	765

I

IMSApi	
"Key Value Pair" structures	43
Connection possibilities	45
Customer-specific error codes	1983
Example of a function declaration	40
IMSApi communication model	45
IMSApi data types	41

Parameter (in)	765
Parameter (out)	765
imsapiGetParameterInfo	
Error codes	769
Function declaration	767
Parameter (in)	767
Parameter (out)	768
imsapiGetSystemInfos	
"Key Value Pair" structures	770
Error codes	772
Function declaration	770
Parameter (in)	771
Parameter (out)	771

L

lockGetLockedObjects	
"Key Value Pair" structures	773
Error codes	779
Function declaration	773
Licensing	773
Parameter (in)	777
Parameter (out)	778
lockGetLockGroups	
"Key Value Pair" structures	780
Error codes	785
Function declaration	780
Licensing	780
Parameter (in)	783
Parameter (out)	784
lockObjects	
"Key Value Pair" structures	786
Error codes	790
Function declaration	786
Licensing	786
Parameter (in)	789
Parameter (out)	790
lockUnlockObjects	
"Key Value Pair" structures	792
Error codes	797
Function declaration	792
Licensing	792
Parameter (in)	795
Parameter (out)	796

M

mdaActivateDocument	
"Key Value Pair" structures	799
Error codes	802
Function declaration	799
Licensing	799
Parameter (in)	801
Parameter (out)	801

mdaActivateRecipe	
Error codes	804
Function declaration	803
Licensing	803
Parameter (in)	803
mdaCreateRecipe	
"Key Value Pair" structures	806
Error codes	814
Function declaration	806
Licensing	806
Parameter (in)	812
Parameter (out)	813
mdaGetDocumentContent	
Error codes	817
Function declaration	816
Licensing	816
Parameter (in)	816
Parameter (out)	816
mdaGetDocuments	
"Key Value Pair" structures	818
Error codes	825
Function declaration	818
Licensing	818
Parameter (in)	824
Parameter (out)	824
mdaGetRecipeData	
"Key Value Pair" structures	827
Error codes	835
Function declaration	827
Licensing	827
Parameter (in)	833
Parameter (out)	834
mdaGetRecipeHeaderAndVersion	
"Key Value Pair" structures	836
Error codes	847
Function declaration	836
Licensing	836
Parameter (in)	846
Parameter (out)	846
mdataBomVerify	
"Key Value Pair" structures	866
Error codes	871
Function declaration	866
Licensing	865
Parameter (in)	869
Parameter (out)	870
mdataChangeWorkorderBomData	
"Key Value Pair" structures	874
Application types	872
Error codes	877
Function declaration	874
Licensing	874
Parameter (in)	876
Parameter (out)	877

mdataChangeWorkorderData		Licensing	920
"Key Value Pair" structures	880	Parameter (in)	923
Error codes	884	Parameter (out)	923
Function declaration	880	mdataGetAssignedSpecialTime	
Licensing	880	"Key Value Pair" structures	927
Parameter (in)	883	Error codes	931
mdataChangeWorkOrderWorkStepState		Function declaration	926
"Key Value Pair" structures	886	Licensing	926
Error codes	890	Parameter (in)	930
Function declaration	886	Parameter (out)	930
Licensing	886	mdataGetBomData	
Parameter (in)	889	"Key Value Pair" structures	933
Parameter (out)	889	Error codes	939
mdataCreateDetectionPoint		Function declaration	933
"Key Value Pair" structures	891	Licensing	933
Error codes	895	Parameter (in)	938
Function declaration	891	Parameter (out)	938
Licensing	891	mdataGetCalendarData	
Parameter (in)	894	"Key Value Pair" structures	941
mdataCreateOrderCluster		Error codes	944
"Key Value Pair" structures	897	Function declaration	941
Error codes	901	Licensing	941
Function declaration	897	Parameter (in)	943
Licensing	897	Parameter (out)	943
Parameter (in)	900	mdataGetContactData	
Parameter (out)	900	"Key Value Pair" structures	945
mdataCreateSpecialTime		Error codes	949
Error codes	905	Function declaration	945
Function declaration	903	Licensing	945
Licensing	903	Parameter (in)	948
Parameter (in)	904	Parameter (out)	948
Parameter (out)	904	mdataGetCostCenter	
mdataCreateWorkorder		"Key Value Pair" structures	950
"Key Value Pair" structures	906	Error codes	954
Error codes	913	Function declaration	950
Function declaration	906	Licensing	950
Licensing	906	Parameter (in)	953
Parameter (in)	912	Parameter (out)	953
Parameter (out)	912	mdataGetDetectionPoint	
mdataDeleteDetectionPoint		"Key Value Pair" structures	955
Error codes	916	Error codes	962
Function declaration	915	Function declaration	955
Licensing	915	Licensing	955, 955
Parameter (in)	915	Parameter (in)	961
mdataDeleteOrderCluster		Parameter (out)	961
"Key Value Pair" structures	917	mdataGetFailureDataForPart	
Error codes	919	"Key Value Pair" structures	963
Function declaration	917	Error codes	967
Licensing	917	Function declaration	963
Parameter (in)	918	Licensing	963
mdataGetAssignedShiftModel		Parameter (in)	966
"Key Value Pair" structures	921	Parameter (out)	966
Error codes	924	mdataGetFailureDataForStation	
Function declaration	920	"Key Value Pair" structures	968

Error codes	973	mdataGetPartData	
Function declaration	968	"Key Value Pair" structures	1013
Licensing	968	Error codes	1018
Parameter (in)	972	Function declaration	1013
Parameter (out)	972	Licensing	1013
mdataGetFailureDefinition		Parameter (in)	1017
"Key Value Pair" structures	974	Parameter (out)	1017
Error codes	980	mdataGetProductInfo	
Function declaration	974	"Key Value Pair" structures	1019
Licensing	974	Error codes	1023
Parameter (in)	979	Function declaration	1019
Parameter (out)	979	Licensing	1019
mdataGetFormulaCatalog		Parameter (in)	1021
"Key Value Pair" structures	981	Parameter (out)	1022
Error codes	984	mdataGetProductsForLineOrStation	
Function declaration	981	"Key Value Pair" structures	1024
Licensing	981	Error codes	1029
Parameter (in)	983	Function declaration	1024
Parameter (out)	983	Licensing	1024
mdataGetMachineAssetStructure		Parameter (in)	1028
"Key Value Pair" structures	986	Parameter (out)	1028
Error codes	993	mdataGetShiftData	
Function declaration	986	"Key Value Pair" structures	1032
Licensing	985	Error codes	1036
Parameter (in)	992	Function declaration	1032
Parameter (out)	992	Licensing	1032
mdataGetMessageCause		Parameter (in)	1035
"Key Value Pair" structures	995	Parameter (out)	1036
Error codes	998	mdataGetShiftModel	
Function declaration	995	"Key Value Pair" structures	1038
Licensing	995, 995	Error codes	1042
Parameter (in)	997	Function declaration	1038
Parameter (out)	997	Licensing	1038
mdataGetMessageClass		Parameter (in)	1041
"Key Value Pair" structures	999	Parameter (out)	1041
Error codes	1002	mdataGetStationConfiguration	
Function declaration	999	"Key Value Pair" structures	1043
Licensing	999, 999	Error codes	1048
Parameter (in)	1001	Function declaration	1043
Parameter (out)	1001	Licensing	1043
mdataGetMessageEffect		Parameter (in)	1047
"Key Value Pair" structures	1003	Parameter (out)	1047
Error codes	1006	mdataGetTimeEvent	
Function declaration	1003	"Key Value Pair" structures	1049
Licensing	1003, 1003	Error codes	1052
Parameter (in)	1005	Function declaration	1049
Parameter (out)	1005	Licensing	1049
mdataGetOrderCluster		Parameter (in)	1051
"Key Value Pair" structures	1007	Parameter (out)	1051
Error codes	1012	mdataGetUserData	
Function declaration	1007	"Key Value Pair" structures	1053
Licensing	1007	Error codes	1057
Parameter (in)	1011	Function declaration	1053
Parameter (out)	1011	Licensing	1053

Parameter (in)	1056	Function declaration	1108
Parameter (out)	1056	Licensing	1108
mdataGetUserGroupData		Parameter (in)	1111
"Key Value Pair" structures	1058	Parameter (out)	1111
Error codes	1062	mdataRequestWorkorder	
Function declaration	1058	"Key Value Pair" structures	1114
Licensing	1058	Error codes	1117
Parameter (in)	1061	Function declaration	1114
Parameter (out)	1061	Licensing	1114
mdataGetUserSkill		Parameter (in)	1116
"Key Value Pair" structures	1063	Parameter (out)	1116
Error codes	1067	mdataSetProductionCycleTime	
Function declaration	1063	Error codes	1120
Licensing	1063	Function declaration	1118
Parameter (in)	1066	Licensing	1118
Parameter (out)	1066	Parameter (in)	1118
mdataGetWageType		mdataUpdateDetectionPoint	
"Key Value Pair" structures	1068	"Key Value Pair" structures	1121
Error codes	1073	Error codes	1125
Function declaration	1068	Function declaration	1121
Licensing	1068	Licensing	1121
Parameter (in)	1072	Parameter (in)	1124
Parameter (out)	1072	mdataUpdateOrderCluster	
mdataGetWeekSchedule		"Key Value Pair" structures	1127
"Key Value Pair" structures	1074	Error codes	1133
Error codes	1077	Function declaration	1127
Function declaration	1074	Licensing	1127
Licensing	1074	Parameter (in)	1131
Parameter (in)	1076	Parameter (out)	1132
Parameter (out)	1076	mdaUploadCustomFunction	
mdataGetWorkorders		"Key Value Pair" structures	848
"Key Value Pair" structures	1078	Error codes	851
Error codes	1085	Function declaration	848
Function declaration	1078	Licensing	848
Licensing	1078	Parameter (in)	850
Parameter (in)	1084	mdaUploadDocument	
Parameter (out)	1084	"Key Value Pair" structures	852
mdataGetWorkplanActivities		Error codes	863
"Key Value Pair" structures	1086	Function declaration	852
Error codes	1097	Licensing	852
Function declaration	1086	Parameter (in)	861
Licensing	1086	mdcAcknowledgeStationConditions	
Parameter (in)	1095	"Key Value Pair" structures	1136
Parameter (out)	1096	Error codes	1141
mdataGetWorkplanData		Function declaration	1136
"Key Value Pair" structures	1098	Licensing	1135
Error codes	1106	Parameter (in)	1140
Function declaration	1098	Parameter (out)	1141
Licensing	1098	mdcCloseStationMessage	
Parameter (in)	1105	"Key Value Pair" structures	1143
Parameter (out)	1105	Error codes	1146
mdataGetWorkstepSkill		Function declaration	1143
"Key Value Pair" structures	1108	Licensing	1143
Error codes	1112	Parameter (in)	1145

mdcCreateConditionCode		Function declaration	1180
Error codes	1149	Licensing	1180
Function declaration	1147	Parameter (in)	1182
Licensing	1147	Parameter (out)	1182
Parameter (in)	1147	mdcGetStationConditions	
mdcCreateLog		"Key Value Pair" structures	1184
Error codes	1151	Error codes	1191
Function declaration	1150	Function declaration	1184
Licensing	1150	Licensing	1184
Parameter (in)	1150	Parameter (in)	1190
mdcCreateLogData		Parameter (out)	1190
"Key Value Pair" structures	1152	mdcGetStationConditionsByFilter	
Error codes	1155	"Key Value Pair" structures	1192
Function declaration	1152	Error codes	1200
Licensing	1152	Function declaration	1192
Parameter (in)	1154	Licensing	1192
Parameter (out)	1154	Parameter (in)	1198
mdcGetConditionCodes		Parameter (out)	1199
"Key Value Pair" structures	1157	mdcManageConditionCode	
Error codes	1161	"Key Value Pair" structures	1201
Function declaration	1157	Error codes	1205
Licensing	1157	Function declaration	1201
Parameter (in)	1160	Licensing	1201
Parameter (out)	1160	Parameter (in)	1204
mdcGetKeyPerformanceIndicators		Parameter (out)	1204
"Key Value Pair" structures	1162	mdcRegisterUserAtLine	
Error codes	1169	Error codes	1209
Function declaration	1162	Function declaration	1207
Licensing	1162	Licensing	1207
Parameter (in)	1167	Parameter (in)	1208
Parameter (out)	1168	mdcUpdateStationCondition	
mdcGetLog		"Key Value Pair" structures	1210
Error codes	1171	Error codes	1215
Function declaration	1170	Function declaration	1210
Licensing	1170	Licensing	1210
Parameter (in)	1170	Parameter (in)	1214
Parameter (out)	1171	Parameter (out)	1214
mdcGetLogData		mdcUploadDetectionPoint	
"Key Value Pair" structures	1172	"Key Value Pair" structures	1217
Error codes	1176	Error codes	1221
Function declaration	1172	Function declaration	1217
Licensing	1172	Licensing	1217
Parameter (in)	1175	Parameter (in)	1220
Parameter (out)	1175	Parameter (out)	1220
mdcGetMessageType		mdcUploadStationCondition	
"Key Value Pair" structures	1177	"Key Value Pair" structures	1223
Error codes	1179	Error codes	1227
Function declaration	1177	Function declaration	1223
Licensing	1177	Licensing	1223
Parameter (in)	1178	Parameter (in)	1226
Parameter (out)	1178	Parameter (out)	1226
mdcGetRegisteredUser		mlChangeMaterialBinData	
"Key Value Pair" structures	1180	"Key Value Pair" structures	1229
Error codes	1183	Error codes	1232

Function declaration	1229	Function declaration	1272
Licensing	1229	Licensing	1272
Parameter (in)	1231	Parameter (in)	1278
Parameter (out)		Parameter (out)	1279
mlCreateNewMaterialBin		mlSetMaterialBinLocation	
"Key Value Pair" structures	1234	Error codes	1282
Error codes	1240	Function declaration	1281
Function declaration	1234	Licensing	1281
Licensing	1234	Parameter (in)	1281
Parameter (in)	1239	mlSplitMaterialBin	
Parameter (out)	1239	"Key Value Pair" structures	1284
mlGetIncomingGoods		Error codes	1287
"Key Value Pair" structures	1242	Function declaration	1283
Error codes	1247	Licensing	1283
Function declaration	1242	Parameter (in)	1286
Licensing	1242	Parameter (out)	1286
Parameter (in)	1246	mlUpdateStorage	
Parameter (out)	1246	"Key Value Pair" structures	1289
mlGetMaterialBinData		Error codes	1294
"Key Value Pair" structures	1248	Function declaration	1289
Error codes	1254	Licensing	1289
Function declaration	1248	Parameter (in)	1292
Licensing	1248	Parameter (out)	1293
Parameter (in)	1253	mlUpdateStorageCell	
Parameter (out)	1253	"Key Value Pair" structures	1295
mlGetMaterialTransactions		Error codes	1298
"Key Value Pair" structures	1255	Function declaration	1295
Error codes	1259	Licensing	1295
Function declaration	1255	Parameter (in)	1297
Licensing	1255	Parameter (out)	1297
Parameter (in)	1258	mlUpdateStorageGroup	
Parameter (out)	1258	"Key Value Pair" structures	1299
mlGetNextMaterialBinNumber		Error codes	1302
Error codes	1261	Function declaration	1299
Function declaration	1260	Licensing	1299
Licensing	1260	Parameter (in)	1301
Parameter (in)	1260	Parameter (out)	1301
Parameter (out)	1260	mlUploadMaterialBinBooking	
mlGetPurchaseOrderData		"Key Value Pair" structures	1303
"Key Value Pair" structures	1262	Error codes	1307
Error codes	1267	Function declaration	1303
Function declaration	1262	Licensing	1303
Licensing	1262	Parameter (in)	1306
Parameter (in)	1266	Parameter (out)	1306
mlGetStorage		msgPublish	
"Key Value Pair" structures	1268	"Work order creation" messages	1309
Error codes	1271	Error codes	1311
Function declaration	1268	Function declaration	1310
Licensing	1268	Licensing	1310
Parameter (in)	1270	Parameter (in)	1310
Parameter (out)	1270	Parameter (out)	1310
mlGetStorageData		msgSubscribe	
"Key Value Pair" structures	1273	Error codes	1313
Application types	1272	Function declaration	1312
Error codes	1280		

Licensing	1312	Error codes	1359
Parameter (in)	1312	Function declaration	1351
msgUnsubscribe		Licensing	1351
Error codes	1315	Parameter (in)	1358
Function declaration	1314	Parameter (out)	1358
Licensing	1314	pmGetFailureData	
Parameter (in)	1314	"Key Value Pair" structures	1360
mslBeginDryObject		Error codes	1365
"Key Value Pair" structures	1317	Function declaration	1360
Error codes	1321	Licensing	1360
Function declaration	1317	Parameter (in)	1364
Licensing	1317	Parameter (out)	1364
Parameter (in)	1320	pmGetFailureTallyCard	
Parameter (out)	1320	"Key Value Pair" structures	1366
mslEndDryObject		Error codes	1371
"Key Value Pair" structures	1323	Function declaration	1366
Error codes	1326	Licensing	1366
Function declaration	1323	Parameter (in)	1370
Licensing	1323	Parameter (out)	1370
Parameter (in)	1325	pmGetProductQuantity	
Parameter (out)	1325	"Key Value Pair" structures	1373
mslGetMinDryTimeForSerialNumber		Error codes	1379
"Key Value Pair" structures	1328	Function declaration	1373
Error codes	1332	Licensing	1372
Function declaration	1328	Parameter (in)	1377
Licensing	1328	Parameter (out)	1378
Parameter (in)	1331	pmUpdateBooking	
Parameter (out)	1331	"Key Value Pair" structures	1381
mslGetObjectInfo		Error codes	1386
"Key Value Pair" structures	1333	Function declaration	1381
Error codes	1340	Licensing	1380
Function declaration	1333	Parameter (in)	1385
Licensing	1333	pmUploadBooking	
Parameter (in)	1339	"Key Value Pair" structures	1390
Parameter (out)	1339	Error codes	1396
mslStartObjectExpiration		Function declaration	1389
"Key Value Pair" structures	1341	Licensing	1389, 1389
Error codes	1344	Parameter (in)	1395
Function declaration	1341	Parameter (out)	1395
Licensing	1341	pmUploadQuantityAndFailureData	
Parameter (in)	1343	"Key Value Pair" structures	1398
Parameter (out)	1343	Error codes	1403
mslStopObjectExpiration		Function declaration	1398
"Key Value Pair" structures	1346	Licensing	1398
Error codes	1349	Parameter (in)	1401
Function declaration	1346	Parameter (out)	1402
Licensing	1346	pmUploadQuantityFailureTallyCardAndFailureData	
Parameter (in)	1348	"Key Value Pair" structures	1406
Parameter (out)	1348	Error codes	1410
		Function declaration	1405
		Licensing	1405
		Parameter (in)	1409
		Parameter (out)	1410
P			
pmGetBooking			
"Key Value Pair" structures	1352		

R

regCheckLicense		Parameter (out)	1435
"Key Value Pair" structures	1413	setupChangeSetupForSerialNumber	
Error codes	1418	"Key Value Pair" structures	1438
Function declaration	1413	Error codes	1443
Licensing	1413	Function declaration	1438
Parameter (in)	1417	Licensing	1438
Parameter (out)	1417	Parameter (in)	1442
regGetRegisteredUser		Parameter (out)	1442
Error codes	1420	setupCheck	
Function declaration	1419	Error codes	1448
Licensing	1419	Function declaration	1444
Parameter (in)	1419	Licensing	1444
Parameter (out)	1419	Parameter (in)	1445
regLogin		Parameter (out)	1446
Error codes	1424	setupCheckPreSetup	
Function declaration	1422	"Key Value Pair" structures	1450
Licensing	1422	Error codes	1458
Parameter (in)	1423	Function declaration	1450
Parameter (out)	1423	Licensing	1450
regLogout		Parameter (in)	1456
Error codes	1426	Parameter (out)	1457
Function declaration	1425	setupCreatePreSetup	
Licensing	1425	"Key Value Pair" structures	1461
Parameter (in)	1425	Error codes	1468
regRegisterUser		Function declaration	1460
Error codes	1428	Licensing	1460
Function declaration	1427	Parameter (in)	1466
Licensing	1427	Parameter (out)	1467
Parameter (in)	1427	setupGetMaterialSetup	
regUnregisterUser		"Key Value Pair" structures	1470
Error codes	1430	Error codes	1475
Function declaration	1429	Function declaration	1470
Licensing	1429	Licensing	1470
Parameter (in)	1429	Parameter (in)	1473
		Parameter (out)	1474
		setupGetPreSetupDetails	
		"Key Value Pair" structures	1476
		Error codes	1484
		Function declaration	1476
		Licensing	1476
		Parameter (in)	1482
		Parameter (out)	1483
		setupGetRequiredMaterialForPreSetup	
		"Key Value Pair" structures	1487
		Application types	1486
		Error codes	1491
		Function declaration	1486
		Licensing	1486
		Parameter (in)	1490
		Parameter (out)	1490
		setupGetSetupDataBySerialNumber	
		"Key Value Pair" structures	1492
		Error codes	1497
		Function declaration	1492

S

Server parameters	
14600	2086
19000	2086
19010	2086
19100	2086
19200	2087
2020	2085
2032	2085
2080	2085
setupAttachPreSetup	
"Key Value Pair" structures	1432
Error codes	1436
Function declaration	1431
Licensing	1431
Parameter (in)	1435

Licensing	1492	setupUpdatePreSetupPosition	
Parameter (in)	1496	"Key Value Pair" structures	1541
Parameter (out)	1496	Application types	1539
setupGetSetupDataBySerialNumberRef		Error codes	1545
"Key Value Pair" structures	1498	Function declaration	1540
Error codes	1504	Licensing	1540
Function declaration	1498	Parameter (in)	1543
Licensing	1498, 1498	Parameter (out)	1544
Parameter (in)	1503	shipActivateShippingLotAtKap	
Parameter (out)	1503	Error codes	1548
setupGetSetupName		Function declaration	1547
Error codes	1507	Licensing	1547
Function declaration	1506	Parameter (in)	1547
Licensing	1506	shipAddChildLotToParentLot	
Parameter (in)	1506	Error codes	1551
Parameter (out)	1506	Function declaration	1549
setupGetStationSetupDataForMaterialBin		Licensing	1549
"Key Value Pair" structures	1508	Parameter (in)	1550
Error codes	1510	shipAddSerialNumberToShippingLot	
Function declaration	1508	Error codes	1555
Licensing	1508	Function declaration	1553
Parameter (in)	1509	Licensing	1553
Parameter (out)	1509	Parameter (in)	1554
setupReplaceSetupForSerialNumber		shipCheckSerialNumberAddToShippingLot	
Error codes	2036	Error codes	1559
Function declaration	2034	Function declaration	1557
Parameter (in)	2034	Licensing	1557
setupSearchPreSetup		Parameter (in)	1558
"Key Value Pair" structures	1511	shipCheckSerialNumberFromShippingLot	
Error codes	1517	Error codes	1562
Function declaration	1511	Function declaration	1561
Licensing	1511	Licensing	1561
Parameter (in)	1515	Parameter (in)	1561
Parameter (out)	1516	shipCompleteLot	
setupStateChange		Error codes	1565
Error codes	1520	Function declaration	1563
Function declaration	1518	Licensing	1563
Licensing	1518	Parameter (in)	1564
Parameter (in)	1519	shipDeactivateShippingLotAtKap	
setupUpdateMaterialSetup		Error codes	1567
"Key Value Pair" structures	1523	Function declaration	1566
Error codes	1527	Licensing	1566
Function declaration	1523	Parameter (in)	1566
Licensing	1522	shipGetChildLotsForParentLot	
Parameter (in)	1525	"Key Value Pair" structures	1568
Parameter (out)	1526	Error codes	1572
setupUpdatePreSetup		Function declaration	1568
"Key Value Pair" structures	1530	Licensing	1568
Application types	1529	Parameter (in)	1571
Error codes	1537	Parameter (out)	1571
Function declaration	1530	shipGetLotFromSerialNumber	
Licensing	1530	"Key Value Pair" structures	1573
Parameter (in)	1535	Error codes	1577
Parameter (out)	1536	Function declaration	1573

Licensing	1573	Site parameter	
Parameter (in)	1576	13030	2092
Parameter (out)	1576	13040	2092
shipGetSerialNumberDataForShippingLot		13050	2092
"Key Value Pair" structures	1578	13060	2093
Error codes	1581	13061	2093
Function declaration	1578	18100	2094
Licensing	1578	18200	2094
Parameter (in)	1580	18210	2095
Parameter (out)	1580	18300	2095
shipGetShippingLotInfo		18301	2095
"Key Value Pair" structures	1582	18306	2095
Error codes	1588	18309	2095
Function declaration	1582	18401	2096
Licensing	1582	18500	2096
Parameter (in)	1587	18501	2096
Parameter (out)	1587	18520	2096
shipGetShippingPrefs		18521	2097
"Key Value Pair" structures	1589	18600	2097
Error codes	1594	20000	2097
Function declaration	1589	20002	2097
Licensing	1589	20003	2098
Parameter (in)	1593	20004	2098
Parameter (out)	1593	20013	2098
shipMoveChildLots		20014	2099
Error codes	1597	22106	2099
Function declaration	1595	24010	2099
Licensing	1595	3011	2088
Parameter (in)	1596	3050	2088
Parameter (out)	1596	4420	2088
shipRemoveChildLots		4560	2088
Error codes	1600	4570	2088
Function declaration	1598	4571	2089
Licensing	1598	4572	2089
Parameter (in)	1599	4610	2089
Parameter (out)	1599	6010	2090
shipRemoveSerialNumberFromShippingLot		6040	2090
Error codes	1603	7070	2091
Function declaration	1601	7210	2091
Licensing	1601	7260	2091
Parameter (in)	1602	smtCheckMaterialBin	
shipReuseCompletedShippingLot		"Key Value Pair" structures	1613
"Key Value Pair" structures	1604	Error codes	1619
Error codes	1608	Function declaration	1613
Function declaration	1604	Licensing	1613
Licensing	1604	Parameter (in)	1618
Parameter (in)	1607	Parameter (out)	1618
Parameter (out)	1607	smtConsumption	
shipSendLot		Error codes	2038
"Key Value Pair" structures	1609	Function declaration	2037
Error codes	1611	Licensing	2037
Function declaration	1609	Parameter (in, inout)	2038
Licensing	1609	smtEventSetup	
Parameter (in)	1610	Error codes	2041

Function declaration	2040	15250	2075
Licensing	2040	15260	2076
Parameter (in, inout)	2041	15270	2076
smtSerialNumberCheckIn		15280	2076
"Key Value Pair" structures	1620	15300	2077
Error codes	1626	16000	2077
Function declaration	1620	16001	2078
Licensing	1620	16100	2078
Parameter (in)	1624	18200	2079
Parameter (out)	1625	18500	2080
smtSerialNumberSetup		18510	2080
Error codes	2044	20000	2080
Function declaration	2043	20001	2080, 2097
Licensing	2043	20002	2081
Parameter (in, inout)	2044	20003	2081
smtSetupData		20004	2081
"Key Value Pair" structures	1628	20013	2081
Error codes	1636	20014	2082
Function declaration	1628	21010	2082
Licensing	1628	21012	2082
Parameter (in)	1634	21016	2083
Parameter (out)	1635	21020	2079
Station parameter		22000	2083
13000	2067	25002	2083
13010	2067	25100	2084
13011	2067	26002	2084
13100	2068	4010	2046
13110	2068	4011	2047
13120	2068	4016	2047
13125	2068	7027	2047
13130	2068	7028	2048
13140	2069	7029	2048
13145	2069	7132	2048
13150	2069	7134	2048
13163	2072	7200	2049
13200	2069	7202	2049
13300	2070	7203	2049
13301	2070	7204	2050
13302	2070	7206	2050
13310	2071	7207	2050
13311	2071	7208	2051
13312	2071	7209	2051
13400	2071	7210	2051
13410	2072	7211	2052
13420	2072	7212	2052
13600	2073	7213	2052
15001	2073	7214	2053
15002	2073	7215	2053
15003	2073	7216	2054
15004	2074	7217	2054
15200	2074	7218	2054
15201	2074	7219	2054
15210	2074	7223	2055
15220	2075	7224	2055

7225	2056	Parameter (out)	1658
7226	2056	trAddSerialNumberToPool	
7228	2056	"Key Value Pair" structures	1661
7250	2056	Error codes	1664
7251	2057	Function declaration	1661
7252	2057	Licensing	1661
7253	2057	Parameter (in)	1663
7254	2058	Parameter (out)	1663
7255	2058	trAddSerialNumberToSerialNumberPool	
7256	2059	Error codes	1668
7257	2059	Function declaration	1666
7258	2059	Licensing	1666
7260	2060	Parameter (in)	1667
7320	2060	Parameter (out)	1667
7400	2061	trAssignSerialNumberForProductOrWorkOrder	
7401	2061	Error codes	1672
7402	2062	Function declaration	1670
8100	2062	Licensing	1670
8110	2063	Parameter (in)	1670
8200	2063	trAssignSerialNumberMergeAndUploadState	
8300	2063	Error codes	1677
8310	2064	Function declaration	1675
8400	2065	Licensing	1674
8401	2065	Parameter (in)	1675
8402	2066	trAssignSerialnumberToOrderCluster	
8410	2066	"Key Value Pair" structures	1680
8500	2066	Error codes	1683
8510	2067	Function declaration	1679
		Licensing	1679
		Parameter (in)	1682
		Parameter (out)	1682
		trChangePoolSerialNumber	
		"Key Value Pair" structures	1685
		Error codes	1688
		Function declaration	1685
		Licensing	1685
		Parameter (in)	1687
		Parameter (out)	1687
		trChangeSerialNumberPool	
		"Key Value Pair" structures	1689
		Error codes	1691
		Function declaration	1689
		Licensing	1689
		Parameter (in)	1690
		trChangeWorkOrder	
		"Key Value Pair" structures	1692
		Error codes	1695
		Function declaration	1692
		Licensing	1692
		Parameter (in)	1694
		Parameter (out)	1694
		trCheckMergedPartsForSerialNumber	
		"Key Value Pair" structures	1696
		Error codes	1699

T

trActivateOrderCluster			
"Key Value Pair" structures	1639		
Error codes	1642		
Function declaration	1639		
Licensing	1639		
Parameter (in)	1641		
trActivateWorkorder			
Error codes	1646		
Function declaration	1644		
Licensing	1644		
Parameter (in)	1644		
trActivateWorkstep			
"Key Value Pair" structures	1647		
Error codes	1650		
Function declaration	1647		
Licensing	1647		
Parameter (in)	1649		
trAddExternalSerialNumber			
"Key Value Pair" structures	1654		
Error codes	1659		
Function declaration	1654		
Licensing	1653		
Parameter (in)	1657		

Function declaration	1696	Parameter (out)	1735
Licensing	1696	trGetMergeParts	
Parameter (in)	1698	"Key Value Pair" structures	1737
Parameter (out)	1698	Error codes	1741
trCheckMergedPartsForSerialNumberComplete		Function declaration	1737
"Key Value Pair" structures	1700	Licensing	1737
Error codes	1702	Parameter (in)	1740
Function declaration	1700	Parameter (out)	1741
Licensing	1700	trGetMultiStationSettings	
Parameter (in)	1701	"Key Value Pair" structures	1742
Parameter (out)	1702	Error codes	1747
trCheckSerialNumberState		Function declaration	1742
"Key Value Pair" structures	1704	Licensing	1742
Error codes	1708	Parameter (in)	1746
Function declaration	1704	Parameter (out)	1746
Licensing	1704	trGetNextProductionStep	
Parameter (in)	1707	"Key Value Pair" structures	1748
Parameter (out)	1708	Error codes	1752
trCheckUserSkill		Function declaration	1748
"Key Value Pair" structures	1711	Licensing	1748
Error codes	1714	Parameter (in)	1751
Function declaration	1711	Parameter (out)	1751
Licensing	1711	trGetNextSerialNumber	
Parameter (in)	1713	Error codes	1755
trCheckUserSkills		Function declaration	1753
"Key Value Pair" structures	1715	Licensing	1753
Error codes	1718	Parameter (in)	1754
Function declaration	1715	Parameter (out)	1754
Licensing	1715	trGetPools	
Parameter (in)	1717	"Key Value Pair" structures	1756
Parameter (out)	1717	Error codes	1759
trCreateSerialNumberPool		Function declaration	1756
Error codes	1722	Licensing	1756
Function declaration	1720	Parameter (in)	1758
Licensing	1720	Parameter (out)	1758
Parameter (in)	1721	trGetPoolSerialNumber	
trDeleteSerialNumberPool		"Key Value Pair" structures	1761
Error codes	1724	Error codes	1764
Function declaration	1723	Function declaration	1760
Licensing	1723	Licensing	1760
Parameter (in)	1723	Parameter (in)	1762
trGetCapturedRepairHistoryData		Parameter (out)	1763
"Key Value Pair" structures	1725	trGetProductQuantity	
Error codes	1730	"Key Value Pair" structures	1766
Function declaration	1725	Error codes	1772
Licensing	1725	Function declaration	1766
Parameter (in)	1729	Licensing	1765
Parameter (out)	1729	Parameter (in)	1770
trGetMergeHistory		Parameter (out)	1771
"Key Value Pair" structures	1731	trGetResultDataForSerialNumber	
Error codes	1736	"Key Value Pair" structures	1773
Function declaration	1731	Error codes	1778
Licensing	1731	Function declaration	1773
Parameter (in)	1735	Licensing	1773

Parameter (in)	1776	Licensing	1820
Parameter (out)	1777	Parameter (in)	1824
trGetSerialNumberActivityUploadInfo		Parameter (out)	1825
"Key Value Pair" structures	1779	trGetStationSetting	
Error codes	1783	"Key Value Pair" structures	1827
Function declaration	1779	Error codes	1832
Licensing	1779	Function declaration	1827
Parameter (in)	1782	Licensing	1827
Parameter (out)	1782	Parameter (in)	1831
trGetSerialNumberBySerialNumberRef		Parameter (out)	1832
Error codes	1785	trGetTestplanData	
Function declaration	1784	"Key Value Pair" structures	1833
Licensing	1784	Error codes	1847
Parameter (in)	1784	Function declaration	1833
Parameter (out)	1785	Licensing	1833
trGetSerialNumberForWorkorderAndWorkstep		Parameter (in)	1845
"Key Value Pair" structures	1786	Parameter (out)	1846
Error codes	1791	trGetTopFailures	
Function declaration	1786	"Key Value Pair" structures	1848
Licensing	1786	Error codes	1853
Parameter (in)	1789	Function declaration	1848
Parameter (out)	1791	Licensing	1848
trGetSerialNumberHistoryData		Parameter (in)	1852
"Key Value Pair" structures	1793	Parameter (out)	1852
Error codes	1803	trGetWorkOrderForStation	
Function declaration	1793	"Key Value Pair" structures	1854
Licensing	1793	Error codes	1862
Parameter (in)	1801	Function declaration	1854
Parameter (out)	1802	Licensing	1854
trGetSerialNumberInfo		Parameter (in)	1861
"Key Value Pair" structures	1804	Parameter (out)	1861
Error codes	1809	trIdentifyPoolSerialNumber	
Function declaration	1804	Error codes	1864
Licensing	1804	Function declaration	1863
Parameter (in)	1808	Licensing	1863
Parameter (out)	1808	Parameter (in)	1863
trGetSerialNumberPoolInfo		trMergeParts	
"Key Value Pair" structures	1810	Error codes	1868
Error codes	1813	Function declaration	1865
Function declaration	1810	Licensing	1865
Licensing	1810	Parameter (in)	1866
Parameter (in)	1811	trRemoveMergeParts	
Parameter (out)	1812	Error codes	1872
trGetSerialNumberUploadInfo		Function declaration	1870
"Key Value Pair" structures	1814	Licensing	1870
Error codes	1819	Parameter (in)	1871
Function declaration	1814	trRemoveSerialNumberFromOrder	
Licensing	1814	"Key Value Pair" structures	1873
Parameter (in)	1818	Error codes	1875
Parameter (out)	1818	Function declaration	1873
trGetStationResult		Licensing	1873
"Key Value Pair" structures	1820	Parameter (in)	1874
Error codes	1826	trRemoveSerialNumberFromSerialNumberPool	
Function declaration	1820	Error codes	1878

Function declaration	1877	Parameter (in)	1919
Licensing	1877	Parameter (out)	1921
Parameter (in)	1877	trUploadFailuresForMergedParts	
trSerialNumberPassing		"Key Value Pair" structures	1926
"Key Value Pair" structures	1880	Error codes	1933
Error codes	1882	Function declaration	1926
Function declaration	1880	Licensing	1926
Licensing	1879	Parameter (in)	1931
Parameter (in)	1881	Parameter (out)	1932
Parameter (out)	1881	trUploadPmQuantity	
trSimulateMerge		Error codes	1938
"Key Value Pair" structures	1886	Function declaration	1936
Error codes	1889	Licensing	1936
Function declaration	1886	Parameter (in)	1937
Licensing	1885	trUploadResultDataAndRecipe	
Parameter (in)	1888	"Key Value Pair" structures	1940
Parameter (out)	1888	Error codes	1945
trSplitPanel		Function declaration	1940
"Key Value Pair" structures	1891	Licensing	1939
Error codes	1894	Parameter (in)	1943
Function declaration	1891	Parameter (out)	1945
Licensing	1891	trUploadState	
Parameter (in)	1893	"Key Value Pair" structures	1949
Parameter (out)	1893	Error codes	1952
trSwitchSerialNumber		Function declaration	1949
Error codes	1899	Licensing	1949
Function declaration	1897	Parameter (in)	1950
Licensing	1896	Parameter (out)	1951
Parameter (in)	1897	trUploadStationResult	
Parameter (inout)	1897	"Key Value Pair" structures	1954
trTestPaa		Error codes	1960
Error codes	1902	Function declaration	1954
Function declaration	1900	Licensing	1954
Licensing	1900	Parameter (in)	1958
Parameter (in)	1901	Parameter (out)	1959
Parameter (out)	1901	trUploadTestplanResult	
trTestSpa		"Key Value Pair" structures	1963
Error codes	1905	Error codes	1971
Function declaration	1903	Function declaration	1962
Licensing	1903	Licensing	1962
Parameter (in)	1904	Parameter (in)	1968
Parameter (out)	1904	Parameter (out)	1970
trUploadActivity		trVerifyMerge	
"Key Value Pair" structures	1907	Error codes	1977
Error codes	1911	Function declaration	1974
Function declaration	1907	Licensing	1974
Licensing	1907	Parameter (in)	1975
Parameter (in)	1909	Parameter (out)	1976
Parameter (out)	1910	trVerifyMergeProduct	
trUploadFailureAndResultData		Error codes	1981
"Key Value Pair" structures	1914	Function declaration	1979
Error codes	1922	Licensing	1979
Function declaration	1914	Parameter (in)	1980
Licensing	1913		

U

updateEquipmentData	
Parameter (in)	731
Parameter (out)	731

